AIClassDocuments

Giáo trình Python, Toán, Thống kê, Machine Learning và AI

46 chương · Song ngữ Anh - Việt

Bản xem trước để học tập và rà soát. Chưa phải chứng nhận học thuật độc lập.

Mục lục / Contents

PY-01 - PY-01 — Biến, đầu vào và đầu ra trong Python

PY-02 - PY-02 — Kiểu số, Boolean và chuỗi

PY-03 - PY-03 — Toán tử số học, so sánh và logic

PY-04 - PY-04 — Câu lệnh điều kiện if, elif, else

PY-05 - PY-05 — Vòng lặp for, while, break và continue

PY-06 - PY-06 — List, chỉ số, cắt danh sách và sắp xếp

PY-07 - PY-07 — Tuple, Set và các phép toán tập hợp

PY-08 - PY-08 — Dictionary: khóa, giá trị và thêm sửa xóa

PY-09 - PY-09 — Hàm, return, lambda và phạm vi biến

PY-10 - PY-10 — Class, Object, constructor và đóng gói

PY-11 - PY-11 — Kế thừa, đa hình và lớp trừu tượng ABC

PY-12 - PY-12 — File I/O, CSV và xử lý ngoại lệ

PY-13 - PY-13 — NumPy ndarray, shape, dtype và axis

PY-14 - PY-14 — NumPy indexing, slicing, boolean và fancy indexing

PY-15 - PY-15 — NumPy tổng hợp, axis và broadcasting

PY-16 - PY-16 — NumPy reshape, transpose, ma trận và lưu file

PY-17 - PY-17 — Pandas Series, DataFrame và đọc CSV

PY-18 - PY-18 — Pandas: missing values, duplicates và kiểu dữ liệu

PY-19 - PY-19 — Pandas: lọc, groupby, merge và sắp xếp

PY-20 - PY-20 — Logic mệnh đề, bảng chân trị và quy tắc đếm

PY-21 - PY-21 — Tập hợp, bao hàm–loại trừ và phân hoạch

PY-22 - PY-22 — Hoán vị, tổ hợp và tam giác Pascal

PY-23 - PY-23 — Đồ thị, danh sách kề và Jaccard similarity

PY-24 - PY-24 — Scalar, vector, norm, đơn vị và dot product

PY-25 - PY-25 — Ma trận, chuyển vị, nhân ma trận và định thức

PY-26 - PY-26 — Hệ phương trình tuyến tính, rank và nghiệm

PY-27 - PY-27 — Eigenvalue, Eigenvector và hướng biến thiên

PY-28 - PY-28 — Hàm tuyến tính, bậc hai, đa thức và hàm mũ

PY-29 - PY-29 — Activation: ReLU, sigmoid, tanh, softmax

PY-30 - PY-30 — Loss: MSE, MAE, Huber, cross-entropy và KL

PY-31 - PY-31 — Giới hạn, tính liên tục và đạo hàm

PY-32 - PY-32 — Đạo hàm riêng, gradient descent và learning rate

PY-33 - PY-33 — Jacobian, Hessian và điểm cực trị

PY-34 - PY-34 — Xác suất, không gian mẫu và biến cố

PY-35 - PY-35 — Xác suất có điều kiện, Bayes và xác suất toàn phần

PY-36 - PY-36 — Phân phối nhị thức và phân phối chuẩn

PY-37 - PY-37 — Thống kê mô tả: mean, median, mode, variance và std

PY-38 - PY-38 — Quantile, IQR, outlier, skewness và kurtosis

PY-39 - PY-39 — Lấy mẫu, CLT và sai số chuẩn

PY-40 - PY-40 — Kiểm định giả thuyết, p-value và khoảng tin cậy

PY-41 - PY-41 — Phân hoạch dữ liệu, classification và clustering

PY-42 - PY-42 — Tiền xử lý văn bản, token và Jaccard trong NLP

PY-43 - PY-43 — Git: quản lý phiên bản và hợp tác trên mã nguồn

PY-44 - PY-44 — Ứng dụng CLI: CRUD, kiểm tra dữ liệu và thống kê hồ sơ

PY-45 - PY-45 — Matplotlib: biểu đồ line, scatter và histogram

PY-46 - PY-46 — Mini project: dữ liệu → làm sạch → mô hình → đánh giá

PY-01 — Biến, đầu vào và đầu ra trong Python

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Variable — biến; **Input** — đầu vào.

`Chương trình → Biến → Kiểu dữ liệu → Điều kiện`

Học xong dùng để làm gì? Nhận tên, tuổi hoặc số lượng từ người dùng, lưu vào biến và in ra kết quả có thể kiểm tra.

Kết nối kiến thức tiếp theo: Sang PY-02, dùng kiểu số và chuỗi để dữ liệu trong biến được tính toán đúng thay vì chỉ hiển thị.

2. English vocabulary — Từ vựng

English	Nghĩa tiếng Việt	Giải thích
Variable	biến	Tên dùng để tham chiếu một giá trị
Input	đầu vào	Thông tin đưa vào chương trình
Output	đầu ra	Thông tin chương trình tạo ra
Type	kiểu dữ liệu	Nhóm giá trị như str, int, float

3. Các hàm và tùy chọn Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và ý nghĩa lựa chọn
`print()`	Hiển thị kết quả, nhận nhiều giá trị phân cách bằng dấu phẩy
`input()`	Nhận văn bản từ bàn phím; kết quả luôn là chuỗi
`int()/float()`	Đổi chuỗi hợp lệ sang số trước khi tính toán
`type()`	Kiểm tra kiểu dữ liệu khi chưa rõ đầu vào

4. Khái niệm liên quan

Một chương trình thường đi theo mạch **nhận thông tin → lưu vào biến → xử lý → đưa ra kết quả**. Bài này bắt đầu từ biến và cho thấy đầu vào, đầu ra và kiểu dữ liệu phối hợp ra sao trong mạch đó.

4.1. Variable — Biến

Từ vựng trong mục: Variable — biến; **Assignment** — phép gán.

Biến là tên để chương trình ghi nhớ một giá trị. Khi viết `total = 12`, Python gắn tên `total` với số 12; dòng tiếp theo đọc lại giá trị qua tên đó. Biến giúp ta dùng lại dữ liệu và thay đổi dữ liệu ở một chỗ thay vì viết số trực tiếp khắp chương trình.

No Note — Không chú thích

```
total = 12
print(total)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đặt số lượng 12 vào biến total rồi hiển thị giá trị đã lưu.
# Biến total giữ một số nguyên; dấu = là phép gán, không phải so sánh bằng.
total = 12
# print() xuất giá trị đang giữ trong biến; kết quả là 12.
print(total)
```

Kết quả khi chạy:

```
12
```

`total` là tên biến, còn 12 là giá trị được in ra.

4.2. Input — Đầu vào

Từ vựng trong mục: Input — đầu vào; **String** — chuỗi.

Đầu vào là dữ liệu chương trình nhận để xử lý: có thể từ bàn phím, biểu mẫu hoặc tệp. Ví dụ, biểu mẫu nhận tên khách `An`, ứng dụng lưu tên đó trong biến `name`. Để ví dụ tự chạy không cần gõ phím, ta mô phỏng dữ liệu đã nhận bằng một phép gán; khi nhận trực tiếp từ bàn phím, có thể dùng `input()`.

No Note — Không chú thích

```
name = 'An'
print(name)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lưu tên người dùng có sẵn trong biến name và in ra màn hình; ví dụ không gọi input() để nhập từ bàn phím.
# Chuỗi trong dấu nháy là dữ liệu tên có sẵn; không phải dữ liệu đang được nhập từ bàn phím.
name = 'An'
# print(name) in nội dung của biến, khác với print('name') in nguyên chữ name.
print(name)
```

Kết quả khi chạy:

```
An
```

Chương trình in đúng tên đã nhận. Biến có thể đến từ nhiều nguồn, không chỉ bàn phím.

4.3. Output — Đầu ra

Từ vựng trong mục: Output — đầu ra; **Print** — hiển thị.

Đầu ra là thông tin chương trình gửi ra ngoài sau khi xử lý. Đầu ra có thể xuất hiện trên màn hình, được lưu vào file hoặc gửi đến hệ thống khác. Trong bài căn bản, `print()` giúp nhìn thấy giá trị đang được tạo ra; chuỗi trong dấu nháy là nội dung cần hiển thị.

No Note — Không chú thích

```
print('Xin chào')
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng print() để đưa lời chào ra màn hình.
# print() gửi chuỗi trong dấu nháy ra vùng kết quả, kèm ký tự xuống dòng.
print('Xin chào')
```

Kết quả khi chạy:

```
Xin chào
```

`print()` hiển thị dòng chữ; dấu nháy chỉ là cú pháp Python, không xuất hiện trong kết quả.

4.4. Type — Kiểu dữ liệu

Từ vựng trong mục: Type — kiểu dữ liệu; **Float** — số thực.

Cùng một biến có thể chứa chữ hoặc số, nhưng **kiểu dữ liệu** quyết định phép xử lý nào phù hợp. Chẳng hạn mã đơn `"0007"` là chuỗi (`str`) cần giữ số 0 đầu; giá trị `2.5` là số thực (`float`) có thể dùng để tính toán. Hàm `type()` cho biết kiểu của một giá trị.

No Note — Không chú thích

```
print(type(2.5))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xem Python nhận diện số 2.5 thuộc kiểu dữ liệu nào.
# type(2.5) trả về kiểu của số thập phân 2.5 (float); print() hiển thị đối tượng kiểu đó.
print(type(2.5))
```

Kết quả khi chạy:

```
<class 'float'>
```

`float` báo đây là số thực, không phải chuỗi ký tự `"2.5"`.

5. Năm tình huống ứng dụng — từ cơ bản đến nâng cao

Tình huống 1 — Cấp 1: Hiển thị danh sách đăng ký

Từ vựng trong mục: Variable — biến; **Output** — đầu ra.

Tình huống thực tế: Quầy ghi danh đang kiểm tra danh sách của một buổi học. Nhân viên đã nhập **10 tên**, theo đúng thứ tự từng người đến quầy; mỗi tên ứng với một học viên. Họ cần biết đã ghi được bao nhiêu người, ai đứng đầu danh sách và ai đứng cuối để đối chiếu với sổ giấy.

Vì sao chọn cách này? Một **biến** là tên đặt cho thông tin cần dùng lại, chẳng hạn `names` giữ danh sách tên và `student_count` giữ số học viên. `len(names)` đếm số phần tử trong danh sách nên không phải tự đếm bằng mắt; `print()` gắn nhãn cho kết quả để người đọc biết con số nói về điều gì. Lấy `names[0]` đọc tên đầu tiên; `names[-1]` đọc tên cuối cùng. Bài này chỉ giới thiệu cách lưu và hiển thị, danh sách sẽ được học kỹ hơn ở PY-06.

Quy trình:

1. Ghi 10 tên theo thứ tự tiếp nhận.
2. Lưu số người vào biến riêng.
3. In ba thông tin có nhãn để dễ đọc.

No Note — Không chú thích

```
names = ["An","Bình","Chi","Dung","Hà","Khánh","Lan","Minh","Nam","Trang"]
student_count = len(names)
if student_count != 10:
    raise ValueError("Danh sách đăng ký phải có đúng 10 học viên.")
print("Sô´học viên:", student_count)
print("Học viên đâù:", names[0])
print("Học viên cuôí:", names[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đêḿ 10 học viên đăng ký và đọc tên đâù/cuôí của danh sách.
# Một danh sách chứa 10 chuôĩ tên theo thứ tự đăng ký; môĩ phâǹ tư ̉đại diện một học viên.
names = ["An","Bình","Chi","Dung","Hà","Khánh","Lan","Minh","Nam","Trang"]
# len(names) trả vêǹsô´phâǹ tư ̉trong danh sách, ở đây là 10.
student_count = len(names)
# != có nghĩa 'khác'; điêù kiện đảm bảo sô´học viên đúng với dữ liệu bài toán.
if student_count != 10:
    # raise ValueError(...) chủ động báo lôĩ đâù vào, dừng xư ̉lý khi sô´lượng không đúng.
    raise ValueError("Danh sách đăng ký phải có đúng 10 học viên.")
print("Sô´học viên:", student_count)
# Chỉ sô´[0] là phâǹ tư ̉đâù tiên vì Python đánh sô´từ 0; print ghép nhãn và tên.
print("Học viên đâù:", names[0])
# Chỉ sô´[-1] lâý phâǹ tư ̉cuôí mà không cân̂ biêt́ độ dài danh sách.
print("Học viên cuôí:", names[-1])
```

Kết quả khi chạy:

```
Sô´học viên: 10
Học viên đâù: An
Học viên cuôí: Trang
```

Mỗi phần tử là một học viên, không phải một ký tự. Chỉ số `0` là người đầu, `-1` là người cuối; chưa có phân loại hay tính toán nâng cao.

Tình huống 2 — Cấp 2: Đổi chữ số thành tiền của 12 đơn

Từ vựng trong mục: Input — đầu vào; **Integer** — số nguyên; **Conversion** — chuyển kiểu.

Tình huống thực tế: Cửa hàng nhận **12 đơn mua hàng** từ một biểu mẫu. Mỗi đơn có mã riêng, giá một món tính bằng **đồng** và số món khách mua; ví dụ giá `12000` đồng và số lượng `2` nghĩa là mua hai món cùng giá. Biểu mẫu gửi các số dưới dạng chữ. Thu ngân cần tính tổng tiền của cả 12 đơn mà không ghép nhầm chữ với số.

Vì sao chọn cách này? Ở biểu mẫu, chuỗi `"12000"` trông giống số nhưng là **chữ**, nên chưa thể dùng như giá tiền trong phép tính. `int()` chuyển những chữ số hợp lệ thành số nguyên: chuyển cả đơn giá và số lượng rồi nhân để ra tiền từng đơn, sau đó cộng dần 12 đơn. Vòng `for` cho phép lặp lại cùng bước xử lý mà không viết 12 đoạn code giống nhau. Việc chuyển kiểu cần làm trước phép tính; nếu ô nhập có chữ không phải số thì phải xử lý lỗi thay vì tự tính thành 0.

Quy trình:

1. Tạo 12 bản ghi chuỗi và in thử mẫu đầu.
2. Chuyển giá và số lượng bằng `int()` trước khi nhân.
3. Cộng tổng các giá trị đã đổi và xác nhận đủ 12 đơn.

No Note — Không chú thích

```
orders = [
    ("O01", "11000", "2"),
    ("O02", "12000", "3"),
    ("O03", "13000", "1"),
    ("O04", "14000", "2"),
    ("O05", "15000", "3"),
    ("O06", "16000", "1"),
    ("O07", "17000", "2"),
    ("O08", "18000", "3"),
    ("O09", "19000", "1"),
    ("O10", "20000", "2"),
    ("O11", "21000", "3"),
    ("O12", "22000", "1"),
]
total_vnd = 0
for order_id, raw_price, raw_quantity in orders:
    price_vnd = int(raw_price)
    quantity = int(raw_quantity)
    total_vnd += price_vnd * quantity
if len(orders) != 12:
    raise ValueError("Cần có đủ 12 đơn hàng để tổng hợp.")
print("Số đơn:", len(orders))
print("Tổng tiền (đồng):", total_vnd)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuyển giá, số lượng từ chuỗi sang số để cộng doanh thu của 12 đơn.
# Mỗi bộ ba trong 12 đơn có (mã đơn, đơn giá dạng chuỗi VND, số lượng dạng chuỗi); các dấu nháy cho
biết chưa phải số.
orders = [
    ("O01", "11000", "2"),
    ("O02", "12000", "3"),
```

```
    ("O03", "13000", "1"),
    ("O04", "14000", "2"),
    ("O05", "15000", "3"),
    ("O06", "16000", "1"),
    ("O07", "17000", "2"),
    ("O08", "18000", "3"),
    ("O09", "19000", "1"),
    ("O10", "20000", "2"),
    ("O11", "21000", "3"),
    ("O12", "22000", "1"),
]
# Biến cộng dồn khởi đầu từ 0 đồng để lưu tổng tiền của mọi đơn.
total_vnd = 0
# for đi lần lượt từng bộ ba; cách tách ba biến cho phép dùng trực tiếp mã, giá và số lượng.
for order_id, raw_price, raw_quantity in orders:
    # int() đổi chữ số dạng chuỗi như '11000' thành số nguyên 11000 để tính toán bằng đồng.
    price_vnd = int(raw_price)
    # int() đổi số lượng dạng chuỗi thành số nguyên, tránh nối chuỗi hoặc lỗi khi nhân.
    quantity = int(raw_quantity)
    # *= không xuất hiện ở đây: * nhân đơn giá với số lượng; += cộng tiền của đơn hiện tại vào
tổng.
    total_vnd += price_vnd * quantity
# len() đếm số đơn; != kiểm tra khác 12 để phát hiện thiếu/thừa bản ghi.
if len(orders) != 12:
    # ValueError thông báo dữ liệu không hợp lệ và dừng chương trình khi kiểm tra thất bại.
    raise ValueError("Cần có đủ 12 đơn hàng để tổng hợp.")
print("Số đơn:", len(orders))
# Giá trị cuối là tổng doanh thu của 12 đơn, đơn vị đồng Việt Nam.
print("Tổng tiền (đồng):", total_vnd)
```

Kết quả khi chạy:

```
Số đơn: 12
Tổng tiền (đồng): 392000
```

Mỗi hàng là **một đơn**; phép nhân tính tiền từng đơn, phép cộng gom 12 đơn. Các đơn có số lượng khác nhau nên không thể lấy một đơn mẫu rồi nhân 12.

Tình huống 3 — Cấp 3: Định dạng bản tin từ 15 cảm biến

Từ vựng trong mục: Float — số thực; **F-string** — chuỗi định dạng; **Precision** — số chữ số thập phân.

Tình huống thực tế: Phòng giám sát có **15 cảm biến nhiệt độ**, đánh mã T01 đến T15. Mỗi cảm biến gửi một giá trị tính bằng **°C**; có giá trị là số lẻ như 26,2 °C. Nhân viên muốn tạo bản tin gồm mã cảm biến và nhiệt độ hiển thị thống nhất với **một chữ số sau dấu thập phân**, đồng thời báo đã nhận bao nhiêu cảm biến.

Vì sao chọn cách này? Nhiệt độ có thể có phần lẻ nên chương trình giữ ở dạng số thực `float`. **f-string** là cách đưa giá trị biến vào một dòng chữ; phần `:.1f` yêu cầu hiển thị đúng một chữ số thập phân, ví dụ 27 thành `27.0`. Như vậy 15 dòng có cùng kiểu trình bày, không làm người trực trạm nhầm cách ghi số. Dùng `enumerate(..., 1)` đánh số cảm biến từ 1 để ghép với mã T01, T02… mà không phải tạo từng câu riêng.

Quy trình:

1. Lưu đủ 15 giá trị °C theo thứ tự cảm biến.
2. Tạo tên mã theo vị trí và đưa nhiệt độ vào chuỗi định dạng.
3. In số lượng và hai dòng kiểm mẫu; các dòng còn lại được tạo cùng quy tắc.

No Note — Không chú thích

```
temperatures = [26.2,27,28.5,29.1,30,27.5,28.2,29.9,31.1,30.5,29,28.4,27.7,30.2,29.6]
reports = []
for index, temperature in enumerate(temperatures, 1):
    reports.append(f"T{index:02d}: {temperature:.1f} °C")
if len(reports) != 15:
    raise ValueError("Bản tin phải có đúng 15 cảm biến.")
print("Sô´cảm biến:", len(reports))
print("Dòng đâu:", reports[0])
print("Dòng cuôí:", reports[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo bản tin từ 15 phép đo nhiệt độ với mã cảm biến dê˜đọc.
# 15 giá trị là nhiệt độ °C từ 15 cảm biến theo cùng thứ tự.
temperatures = [26.2,27,28.5,29.1,30,27.5,28.2,29.9,31.1,30.5,29,28.4,27.7,30.2,29.6]
# Danh sách rông sẽ chứa các dòng báo cáo sau khi định dạng.
reports = []
# enumerate(..., 1) duyệt nhiệt độ kèm sô´thứ tự băt đâu từ 1, không phải 0.
for index, temperature in enumerate(temperatures, 1):
    # append() thêm một dòng vào cuôí danh sách; f-string thay index và temperature bằng giá trị
thực; :02d viêt mã 2 chữ sô´có thêm 0; :.1f hiển thị 1 chữ sô´thập phân.
    reports.append(f"T{index:02d}: {temperature:.1f} °C")
# len() xác nhận đã tạo đúng 15 bản tin, không bỏ mât cảm biến nào.
if len(reports) != 15:
    # Báo ValueError nêú sô´dòng báo cáo không đúng kỳ vọng.
    raise ValueError("Bản tin phải có đúng 15 cảm biến.")
print("Sô´cảm biến:", len(reports))
# [0] lây bản tin đâu tiên (T01) trong danh sách có thứ tự.
print("Dòng đâu:", reports[0])
# [-1] lây bản tin cuôí (T15) mà không cân tự đêm chỉ sô´.
print("Dòng cuôí:", reports[-1])
```

Kết quả khi chạy:

```
Sô´cảm biến: 15
Dòng đâu: T01: 26.2 °C
Dòng cuôí: T15: 29.6 °C
```

Tất cả 15 bản tin đã được tạo; hiển thị hai dòng biên để kiểm tra định dạng. Chữ số `.1f` điều khiển **cách in**, không làm thay đổi nguồn dữ liệu ban đầu.

Tình huống 4 — Cấp 4: Báo số đo chưa gửi từ 18 lượt nhập

Từ vựng trong mục: Missing value — giá trị thiếu; **Type** — kiểu dữ liệu; **Validation** — kiểm tra hợp lệ.

Tình huống thực tế: Một trạm khí tượng nhận **18 lượt đo nhiệt độ** tính bằng °C. Mỗi dòng ứng với một lần thiết bị gửi số liệu, nhưng một dòng trống và một dòng ghi `N/A` vì chưa nhận được kết quả. Người trực trạm cần biết có bao nhiêu lượt đo được, bao nhiêu lượt thiếu và nhiệt độ trung bình của những lượt thật sự có số; thông tin thiếu không được hiểu là 0 °C.

Vì sao chọn cách này? `None`, chuỗi rỗng và `N/A` đều báo **chưa có dữ liệu**, không phải nhiệt độ bằng 0: 0 °C là một phép đo hợp lệ. Vì thế chương trình đọc từng dòng, kiểm tra dấu hiệu thiếu trước, chỉ chuyển những dòng có số sang `float`, rồi cộng và chia theo **số lượt hợp lệ**. Nếu chia cho cả 18 lượt hoặc biến `N/A` thành 0, kết quả trung bình sẽ bị kéo sai. `continue` chỉ bỏ qua dòng thiếu và vẫn xử lý những dòng còn lại.

Quy trình:

1. Duyệt lần lượt 18 chuỗi, bỏ khoảng trắng ở hai đầu.
2. Chỉ chuyển `float` cho chuỗi thực sự mang số; ghi lại lượt thiếu.
3. Tính trung bình các giá trị nhận được và in bảng tóm tắt.

No Note — Không chú thích

```
raw_readings = [" 26.0 ","27.5","28.0","","29.0"," 30.5","31.0","28.7","27.3","26.9","30.0","N/
A","29.8","28.5","27.0","31.2","30.6","29.4"]
valid_temperatures = []
missing_count = 0
for raw in raw_readings:
    value = raw.strip()
    if value in ("", "N/A"):
        missing_count += 1
        continue
    valid_temperatures.append(float(value))
average_celsius = sum(valid_temperatures) / len(valid_temperatures)
if len(raw_readings) != 18:
    raise ValueError("Dữ liệu đầu vào phải có đúng 18 lượt đo.")
print("Hợp lệ:", len(valid_temperatures))
print("Thiếu:", missing_count)
print(f"Trung bình: {average_celsius:.2f} °C")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc 18 chuỗi đo nhiệt độ, bỏ giá trị thiếu rồi tính trung bình các số đo hợp lệ.
# 18 chuỗi nhiệt độ °C; khoảng trắng thừa, chuỗi rỗng và 'N/A' thể hiện dữ liệu chưa sạch hoặc
thiếu.
raw_readings = [" 26.0 ","27.5","28.0","","29.0"," 30.5","31.0","28.7","27.3","26.9","30.0","N/
A","29.8","28.5","27.0","31.2","30.6","29.4"]
# Chỉ lưu số đo hợp lệ dưới dạng số thực vào danh sách này.
valid_temperatures = []
missing_count = 0
# Xử lý từng chuỗi đo, không cộng trực tiếp chuỗi vào số.
for raw in raw_readings:
    # strip() bỏ khoảng trắng ở hai đầu, giúp nhận diện chuỗi trống và đọc số chính xác.
    value = raw.strip()
    # in kiểm tra giá trị có thuộc hai dấu hiệu thiếu ('', 'N/A') hay không.
    if value in ("", "N/A"):
        # += 1 tăng bộ đếm dữ liệu thiếu.
        missing_count += 1
        # continue bỏ phần xử lý còn lại của lượt đo này để tránh float() trên dữ liệu thiếu.
        continue
    # float() đổi chuỗi như '26.0' thành số thập phân; append() thêm số đã đổi.
    valid_temperatures.append(float(value))
# sum() cộng các nhiệt độ hợp lệ, len() đếm chúng; chia để lấy trung bình °C, không tính bản ghi
thiếu.
average_celsius = sum(valid_temperatures) / len(valid_temperatures)
# Xác nhận tổng số lượt nhận, bao gồm cả lượt thiếu, bằng 18.
if len(raw_readings) != 18:
    # Báo lỗi nếu thiếu hoặc thừa bản ghi đầu vào.
    raise ValueError("Dữ liệu đầu vào phải có đúng 18 lượt đo.")
print("Hợp lệ:", len(valid_temperatures))
print("Thiếu:", missing_count)
# f-string chèn nhiệt độ trung bình; :.2f làm tròn hiển thị đến hai chữ số sau dấu thập phân.
print(f"Trung bình: {average_celsius:.2f} °C")
```

Kết quả khi chạy:

```
Hợp lệ: 16
Thiếu: 2
Trung bình: 28.84 °C
```

Đọc đúng số lượt hợp lệ trước khi tính trung bình; nếu đưa hai mã mất tín hiệu vào như 0 °C thì kết quả sẽ bị kéo thấp giả tạo.

Tình huống 5 — Cấp 5: Tạo báo cáo 20 phiếu thanh toán và đối chiếu đầu vào

Từ vựng trong mục: Input — đầu vào; **Conversion** — chuyển kiểu; **Report** — báo cáo.

Tình huống thực tế: Cuối ngày, cửa hàng nhận **20 phiếu thanh toán** O01 đến O20. Mỗi phiếu ghi giá của một món (**đồng/món**) và số món mua; cả hai ban đầu là chữ do phần mềm nhập liệu gửi đến. Phiếu O07 và O18 có dấu ? thay cho giá vì chưa nhập. Chủ cửa hàng muốn biết tổng số phiếu, số phiếu có thể tính tiền, tổng tiền từ những phiếu đó và hai mã phải bổ sung giá.

Vì sao chọn cách này? Báo cáo cần tách **phiếu hợp lệ** khỏi **phiếu chưa có giá**. `isdigit()` kiểm tra chuỗi giá có toàn chữ số hay chưa trước khi dùng `int()`; dấu ? sẽ không qua bước này. Với phiếu hợp lệ, nhân giá một món với số lượng và cộng vào tổng. Với O07, O18, chương trình ghi mã cần sửa rồi dùng `continue` để chuyển sang phiếu kế tiếp; không được coi giá thiếu là 0 vì như vậy sẽ báo doanh thu thấp hơn thực tế mà không ai biết đang thiếu dữ liệu. Cuối cùng kiểm tra số phiếu tính được cộng số phiếu thiếu có đúng 20.

Quy trình:

1. Ghi đầy đủ 20 phiếu và giữ mã gồm hai chữ số cuối.
2. Kiểm tra giá có phải chữ số trước khi chuyển `int`; ghi lại mã phiếu lỗi.
3. Tính tổng chỉ trên phiếu hợp lệ, xác nhận số lượng phiếu rồi in báo cáo.

No Note — Không chú thích

```
raw_prices =
["12000","14000","16000","18000","20000","22000","?","26000","28000","30000","32000","34000","36000
","38000","40000","42000","44000","?","48000","50000"]
raw_quantities = ["1","2","3","4","1","2","3","4","1","2","3","4","1","2","3","4","1","2","3","4"]
orders = []
for i in range(1, 21):
    order_id = f"O{i:02d}"
    price = raw_prices[i - 1]
    quantity = raw_quantities[i - 1]
    orders.append((order_id, price, quantity))
total_vnd = 0
missing_ids = []
paid_count = 0
for order_id, raw_price, raw_qty in orders:
    if not raw_price.isdigit():
        missing_ids.append(order_id)
        continue
    total_vnd += int(raw_price) * int(raw_qty)
    paid_count += 1
if len(orders) != 20 or paid_count + len(missing_ids) != 20:
    raise ValueError("Phải đối chiếu đủ 20 phiếu, gồm cả phiếu thiếu giá.")
print("Số phiếu:", len(orders))
print("Phiếu hợp lệ:", paid_count)
print("Tổng tiền (đồng):", total_vnd)
print("Cần bổ sung:", ", ".join(missing_ids))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tra 20 phiếu bán hàng, chỉ tính tiền phiếu có đơn giá là chữ số;
# 20 giá tiền ở dạng chuỗi đồng; dấu '?' là phiếu thiếu giá cần tách riêng.
raw_prices =
["12000","14000","16000","18000","20000","22000","?","26000","28000","30000","32000","34000","36000
","38000","40000","42000","44000","?","48000","50000"]
# 20 số lượng dạng chuỗi; mỗi vị trí i tương ứng đúng vị trí i trong raw_prices.
raw_quantities = ["1","2","3","4","1","2","3","4","1","2","3","4","1","2","3","4","1","2","3","4"]
# Danh sách tích lũy bộ ba (mã phiếu, chuỗi giá, chuỗi số lượng).
orders = []
# range(1, 21) sinh các số 1 đến 20, không gồm 21.
for i in range(1, 21):
    # f-string O{i:02d} ghép chữ O với số thứ tự có hai chữ số; ví dụ O01.
    order_id = f"O{i:02d}"
    # i bắt đầu 1 nhưng chỉ số list bắt đầu 0, nên dùng [i - 1] để lấy đúng giá.
    price = raw_prices[i - 1]
    # Cùng chỉ số [i - 1] bảo đảm số lượng khớp đúng giá của cùng phiếu.
    quantity = raw_quantities[i - 1]
    # append() thêm bộ ba vào cuối danh sách theo thứ tự mã phiếu.
    orders.append((order_id, price, quantity))
total_vnd = 0
missing_ids = []
paid_count = 0
# Tách từng bộ ba; giá và số lượng lúc này vẫn là chuỗi.
for order_id, raw_price, raw_qty in orders:
    # isdigit() kiểm tra toàn chuỗi có phải chữ số; not đảo kết quả để tìm '?' hoặc giá thiếu.
    if not raw_price.isdigit():
        # Lưu mã phiếu lỗi để báo người dùng bổ sung giá.
        missing_ids.append(order_id)
        # Không cộng doanh thu của phiếu có giá không hợp lệ.
        continue
    # int() chuyển hai chuỗi thành số rồi nhân giá (đồng) với số lượng; += cộng vào tổng.
    total_vnd += int(raw_price) * int(raw_qty)
    paid_count += 1
# len() đếm phiếu; or báo lỗi khi số phiếu khác 20 hoặc tổng hợp lệ + thiếu không bằng 20.
if len(orders) != 20 or paid_count + len(missing_ids) != 20:
    # raise ValueError() dừng chương trình nếu kiểm tra toàn vẹn dữ liệu không đạt.
    raise ValueError("Phải đủ chiếu đủ 20 phiếu, gồm cả phiếu thiếu giá.")
print("Số phiếu:", len(orders))
print("Phiếu hợp lệ:", paid_count)
print("Tổng tiền (đồng):", total_vnd)
# ', '.join(...) nối mã phiếu thiếu bằng dấu phẩy và khoảng trắng để dễ đọc.
print("Cần bổ sung:", ", ".join(missing_ids))
```

Kết quả khi chạy:

```
Số phiếu: 20
Phiếu hợp lệ: 18
Tổng tiền (đồng): 1436000
Cần bổ sung: O07, O18
```

Đây là mức tổng hợp: hệ thống vừa tính tiền vừa trả lời **vì sao** có hai phiếu không thể đưa vào kết quả. Mã đơn được giữ dạng chuỗi để số 0 ở đầu không biến mất.

6. Những nhầm lẫn thường gặp

- `input()` nhận chữ; cần đổi sang `int()` hoặc `float()` khi muốn tính toán với số. Chuỗi không hợp lệ có thể gây `ValueError`.
- Mã đơn `0007` phải lưu dạng chuỗi nếu cần giữ hai số `0` ở đầu.

- `print()` in kết quả, còn `type()` xem kiểu dữ liệu; hai hàm có nhiệm vụ khác nhau.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a variable in Python? | Biến trong Python là gì?

A. A name referring to a value | Tên tham chiếu đến một giá trị

B. Information produced by a program | Thông tin do chương trình tạo ra

C. Data received for processing | Dữ liệu được nhận để xử lý

D. How a value is represented and used | Cách biểu diễn và sử dụng một giá trị

Đáp án: A

Giải thích: Biến Python là tên được liên kết với đối tượng, chẳng hạn `age = 20`. Biến khác dữ liệu đầu vào hay kết quả in ra; dùng `age` sau đó sẽ truy cập giá trị đang được gán.

Q2. What is program input? | Đầu vào chương trình là gì?

A. Information produced by a program | Thông tin do chương trình tạo ra

B. How a value is represented and used | Cách biểu diễn và sử dụng một giá trị

C. A name referring to a value | Tên tham chiếu đến một giá trị

D. Data received for processing | Dữ liệu được nhận để xử lý

Đáp án: D

Giải thích: Đầu vào là dữ liệu chương trình nhận, ví dụ văn bản người dùng nhập cho `input()`. Chương trình có thể xử lý văn bản rồi tạo đầu ra; đầu vào không phải kết quả chương trình.

Q3. What is program output? | Đầu ra chương trình là gì?

A. How a value is represented and used | Cách biểu diễn và sử dụng một giá trị

B. Information produced by a program | Thông tin do chương trình tạo ra

C. Data received for processing | Dữ liệu được nhận để xử lý

D. A name referring to a value | Tên tham chiếu đến một giá trị

Đáp án: B

Giải thích: Đầu ra là thông tin chương trình đưa ra ngoài, ví dụ dòng chữ do `print(5)` hiển thị. Nó khác đầu vào do người dùng cung cấp và khác biến dùng để tham chiếu giá trị trong chương trình.

Q4. What does a data type describe? | Kiểu dữ liệu mô tả điều gì?

A. Information produced by a program | Thông tin do chương trình tạo ra

B. How a value is represented and used | Cách biểu diễn và sử dụng một giá trị

C. A name referring to a value | Tên tham chiếu đến một giá trị

D. Data received for processing | Dữ liệu được nhận để xử lý

Đáp án: B

Giải thích: Kiểu dữ liệu quyết định ý nghĩa giá trị và các phép toán phù hợp: `3` thuộc kiểu `int`, còn `'3'` thuộc `str`. Vì vậy `3 + 2` là phép cộng số, còn `'3' + '2'` nối văn bản.

Q5. Which function prints a value? | Hàm nào in một giá trị?

A. `int()` | `int()`

B. `type()` | `type()`

C. `input()` | `input()`

D. `print()` | `print()`

Đáp án: D

Giải thích: `print()` ghi biểu diễn của giá trị ra đầu ra chuẩn; chẳng hạn `print(5)` hiển thị `5`. Hàm này không nhận câu trả lời người dùng như `input()`.

Q6. Which function reads keyboard input? | Hàm nào đọc dữ liệu bàn phím?

A. `input()` | `input()`

B. `int()` | `int()`

C. `type()` | `type()`

D. `print()` | `print()`

Đáp án: A

Giải thích: `input()` nhận một dòng người dùng gõ và trả về văn bản không kèm ký tự xuống dòng cuối. Dù người dùng gõ `12`, kết quả vẫn là chuỗi `'12'`, không tự đổi thành số nguyên `12`.

Q7. Which function finds a value's type? | Hàm nào kiểm tra kiểu giá trị?

A. `input()` | `input()`

B. `print()` | `print()`

C. `type()` | `type()`

D. `int()` | `int()`

Đáp án: C

Giải thích: `type(value)` trả về lớp/kiểu của đối tượng: `type(3)` là `int` còn `type('3')` là `str`. Hàm kiểm tra kiểu chứ không chuyển đổi giá trị.

Q8. Which function converts valid decimal digits to an integer? | Hàm nào đổi chuỗi chữ số hợp lệ thành số nguyên?

A. `int()` | `int()`

B. `print()` | `print()`

C. `type()` | `type()`

D. `input()` | `input()`

Đáp án: A

Giải thích: `int()` phân tích chuỗi biểu diễn số nguyên hợp lệ nên `int('012')` cho số nguyên `12`; số 0 đầu không được giữ lại. Chuỗi `'12.5'` không phải chuỗi số nguyên thập phân hợp lệ và gây `ValueError`.

Q9. Which type represents a count of whole items? | Kiểu nào biểu diễn số lượng món nguyên?

A. `str` | `str`

B. `float` | `float`

C. `int` | `int`

D. `bool` | `bool`

Đáp án: C

Giải thích: `int` biểu diễn số lượng nguyên như 4 vé hoặc 0 lỗi, không có phần lẻ. `float` phù hợp số đo có thể có phần thập phân, không cần thiết cho số đếm các đơn vị nguyên vẹn.

Q10. Which type suits a measurement with fractions? | Kiểu nào phù hợp với số đo có phần lẻ?

A. `int` | `int`

B. `float` | `float`

C. `bool` | `bool`

D. `str` | `str`

Đáp án: B

Giải thích: `float` biểu diễn giá trị số có thể chứa phần lẻ, ví dụ 2,5 mét. Khác số đếm nguyên, số đo có thể nằm giữa hai số nguyên; số thập phân dạng dấu phẩy động đôi khi chỉ là xấp xỉ.

Q11. Which type preserves a leading zero in an identifier? | Kiểu nào giữ số 0 đầu mã định danh?

A. `str` | `str`

B. `bool` | `bool`

C. `float` | `float`

D. `int` | `int`

Đáp án: A

Giải thích: Mã định danh như `'007'` nên lưu bằng `str` vì mọi ký tự, kể cả hai số 0 đầu, đều có ý nghĩa. Đổi sang `int` được `7` và làm mất định dạng mã.

Q12. Which type stores a true/false state? | Kiểu nào lưu trạng thái đúng/sai?

A. `bool` | `bool`

B. `int` | `int`

C. `str` | `str`

D. `float` | `float`

Đáp án: A

Giải thích: `bool` có hai giá trị logic `True` và `False`, thích hợp biểu diễn trạng thái như có được phép truy cập hay không. Đây không phải kiểu đo lường số hay nhãn văn bản.

Q13. What does `x = 5` do? | `x = 5` thực hiện điều gì?

A. Tests whether `x` equals `5` | Kiểm tra `x` có bằng `5` không

B. Adds `5` to the old value of `x` | Cộng `5` vào giá trị cũ của `x`

C. Displays `5` without storing it | Hiển thị `5` mà không gán

D. Assigns the integer `5` to `x` | Gán số nguyên `5` cho `x`

Đáp án: D

Giải thích: Dấu bằng đơn gán tên `x` cho số nguyên `5`, không kiểm tra bằng nhau và cũng không cộng thêm số. Muốn so sánh `x` có bằng `5` thì dùng `x == 5`.

Q14. What is the type of `input()`'s return value? | Giá trị trả về của `input()` có kiểu nào?

A. An integer when only digits are typed | Số nguyên nếu chỉ nhập chữ số

B. A floating-point number by default | Số thực theo mặc định

C. A Boolean indicating whether input exists | Boolean cho biết có đầu vào hay không

D. A string, even if it contains digits | Chuỗi, kể cả khi chỉ chứa chữ số

Đáp án: D

Giải thích: `input()` luôn trả về `str` chứa nội dung được nhập, ví dụ `'42'`. Nếu cần tính toán số, hãy dùng `int(input())` hoặc phép đổi kiểu phù hợp khi đầu vào hợp lệ.

Q15. What does `float('2.5')` produce? | `float('2.5')` tạo ra gì?

A. The string `'2.5'` | Chuỗi `'2.5'`

B. The integer `2` | Số nguyên `2`

C. The floating-point number `2.5` | Số thực `2.5`

D. `ValueError` for any decimal point | `ValueError` nếu có dấu thập phân

Đáp án: C

Giải thích: `float('2.5')` chuyển văn bản `'2.5'` thành giá trị số thực `2.5`. Kết quả không còn là chuỗi có dấu nháy và không cắt phần lẻ thành số nguyên `2`.

Q16. What does `int('abc')` raise? | `int('abc')` phát sinh gì?

A. `TypeError` | `TypeError`

B. `ValueError` | `ValueError`

C. The integer `0` | Số nguyên `0`

D. `NameError` | `NameError`

Đáp án: B

Giải thích: `int('abc')` phát sinh `ValueError` vì các chữ cái không tạo thành số nguyên hệ mười hợp lệ. Hàm vẫn tồn tại và nhận được chuỗi, nên đây là lỗi giá trị chuyển đổi chứ không phải `NameError`.

Q17. What does `print(2, 3)` display by default? | `print(2, 3)` mặc định hiển thị gì?

A. `2,3` | `2,3`

B. `2 3` | `2 3`

C. `(2, 3)` | `(2, 3)`

D. `23` | `23`

Đáp án: B

Giải thích: Mặc định `print` ngăn cách các đối số bằng một dấu cách: `print(2, 3)` hiển thị `2 3`. Nó không dính hai số thành `23` và không in dạng tuple `(2, 3)`.

Q18. If `x = 2` is followed by `x = 4` on the next line, what is the final value of `x`? | Nếu `x = 2` rồi `x = 4` ở dòng tiếp theo, giá trị cuối của `x` là gì?

A. `ValueError` | `ValueError`

B. `2` | `2`

C. `4` | `4`

D. `6` | `6`

Đáp án: C

Giải thích: Lệnh đầu gán `x` bằng `2`, lệnh tiếp theo gán lại chính tên đó bằng `4`. Vì vậy giá trị cuối là `4`, không phải tổng `6` hay giá trị cũ `2`.

Q19. Which Python type does `type(2.0)` identify? | `type(2.0)` xác định kiểu Python nào?

A. `int` | `int`

B. `bool` | `bool`

C. `float` | `float`

D. `str` | `str`

Đáp án: C

Giải thích: Literal `2.0` là đối tượng số thực, nên `type(2.0)` xác định kiểu `float` (hiển thị `<class 'float'>`). Trong khi đó `type(2)` là `int`; dấu thập phân làm thay đổi kiểu literal.

Q20. What does `print('Hi')` display? | `print('Hi')` hiển thị gì?

A. `hi` | `hi`

B. `None` | `None`

C. `'Hi'` including the quotes | `'Hi'` gồm cả dấu nháy

D. `Hi` without quotes | `Hi` không có dấu nháy

Đáp án: D

Giải thích: Dấu nháy dùng để bao chuỗi trong mã nguồn chứ không phải ký tự thuộc nội dung chuỗi. Vì vậy `print('Hi')` hiển thị `Hi` rồi xuống dòng, không in dấu nháy hoặc chữ thường `hi`.

8. Nối sang bài kế tiếp

`Chương trình → Biến → Kiểu dữ liệu → Điều kiện`.

PY-02 — Kiểu số, Boolean và chuỗi

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Integer — số nguyên; **Float** — số thực.

`Biến → int/float/bool/str → Slicing → Xử lý văn bản`

Học xong dùng để làm gì? Phân biệt số và chữ khi đọc dữ liệu, đổi kiểu để tính tiền, số lượng hoặc điểm trung bình mà không ghép nhầm chuỗi.

Kết nối kiến thức tiếp theo: Sang PY-03, dùng toán tử tính toán và so sánh các giá trị đã có kiểu phù hợp.

2. English vocabulary — Từ vựng

English	Nghĩa tiếng Việt	Giải thích
Integer	số nguyên	Giá trị không có phần thập phân
Float	số thực	Số có thể có phần lẻ
Boolean	giá trị logic	Giá trị True hoặc False
String	chuỗi	Dãy ký tự có thể cắt lát

3. Các hàm và tùy chọn Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và ý nghĩa lựa chọn
`str()/int()/float()`	Chuyển kiểu phù hợp trước khi xử lý
`strip()/lower()/upper()`	Dọn khoảng trắng và chuẩn hóa chữ
`split()/replace()`	Tách chuỗi hoặc thay thế đoạn văn bản
`s[start:stop]`	Cắt ký tự theo vị trí, không lấy chỉ số stop

4. Khái niệm liên quan

Kiểu dữ liệu (data type) cho Python biết một giá trị biểu diễn điều gì và có thể thực hiện những phép toán nào. Số lượng món hàng thường là số nguyên, nhiệt độ có thể là số thực, điều kiện đạt/chưa đạt là Boolean, còn mã sản phẩm phải được giữ như chuỗi.

4.1. Integer — Số nguyên

Từ vựng trong mục: Integer — số nguyên; **Type** — kiểu dữ liệu.

`int` biểu diễn những số không có phần thập phân, ví dụ 5 sản phẩm hoặc 12 học viên. Dùng `int` khi cần đếm hoặc tính toán số lượng nguyên; khác với chuỗi `"5"`, số `5` thực hiện được phép cộng số học. Ở đây biến `n` giữ số 5 và `type(n)` kiểm tra kiểu.

No Note — Không chú thích

```
n = 5
print(type(n))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng type() để xác nhận biến n lưu số nguyên int.
# 5 không có dấu thập phân nên biến n thuộc kiểu int (số nguyên).
n = 5
# type(n) xác định kiểu dữ liệu của n; print() hiển thị kết quả <class 'int'>.
print(type(n))
```

Kết quả khi chạy:

```
<class 'int'>
```

Kết quả cho biết `n` là số nguyên, không phải chuỗi.

4.2. Float — Số thực

Từ vựng trong mục: Float — số thực; **Decimal** — phần thập phân.

`float` lưu các số có phần thập phân, như 3,5 kg hàng hoặc nhiệt độ 36,5 °C. Nếu cần tính trên những số đo này, dùng số thực thay vì chữ. Trong ví dụ, `x` nhận `3.5` và `print()` hiển thị giá trị của nó.

No Note — Không chú thích

```
x = 3.5
print(x)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lưu và in giá trị x dạng số thực float.
# Số có phần thập phân được lưu dưới kiểu float.
x = 3.5
# print() hiển thị giá trị số thực, không thêm dấu nháy như khi in chuỗi.
print(x)
```

Kết quả khi chạy:

```
3.5
```

Dấu chấm trong code biểu diễn phần thập phân; đơn vị đo phải được xác định theo bài toán.

4.3. Boolean — Giá trị đúng/sai

Từ vựng trong mục: Boolean — giá trị logic; **True/False** — đúng/sai.

`bool` chỉ có hai giá trị `True` và `False`, phù hợp với câu hỏi dạng "đã thanh toán chưa?" hoặc "đủ điều kiện chưa?". Biến `ok = True` ở đây tượng trưng cho trạng thái kiểm tra đã đạt. Đây không phải chữ `"True"`: Python hiểu là giá trị logic.

No Note — Không chú thích

```
ok = True
print(ok)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thử kiểu Boolean: True biểu thị điều kiện đúng.
# True là giá trị Boolean đúng; phải viết hoa chữ T trong Python.
ok = True
# print() xuất True, không phải chuỗi ký tự 'True'.
print(ok)
```

Kết quả khi chạy:

```
True
```

Đầu ra là giá trị logic `True`, không phải số 1 hoặc một thông báo khác.

4.4. String — Chuỗi ký tự

Từ vựng trong mục: String — chuỗi; **Slice** — cắt chuỗi.

`str` lưu dãy ký tự theo đúng thứ tự, kể cả chữ số nằm trong mã như `AB12` hoặc số điện thoại bắt đầu bằng 0. Với `code = 'AB12'`, các vị trí lần lượt là 0:A, 1:B, 2:1, 3:2. Cú pháp `[:2]` lấy ký tự ở vị trí 0 và 1, không lấy vị trí 2.

No Note — Không chú thích

```
code = 'AB12'
print(code[:2])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy hai ký tự đầu của mã AB12 bằng phép cắt chuỗi.
# Mã chứa 4 ký tự dạng str (chuỗi), chưa phải số.
code = 'AB12'
# Cú pháp [bắt_đầu:kết_thúc] cắt chuỗi; [:2] lấy vị trí 0, 1, loại trừ vị trí 2 nên trả 'AB'.
print(code[:2])
```

Kết quả khi chạy:

```
AB
```

Hai ký tự đầu được tách ra mà chuỗi ban đầu vẫn giữ nguyên.

5. Năm tình huống ứng dụng — tăng dần mức độ

Tình huống 1 — Cấp 1: Chuẩn hóa mười địa chỉ liên hệ

Từ vựng trong mục: String — chuỗi; **Strip** — bỏ khoảng trắng; **Lower** — chữ thường.

Tình huống thực tế: Một nhóm hỗ trợ muốn gửi thông báo cho **10 tài khoản thử nghiệm**. Danh sách email được nhập với khoảng trắng ở đầu hoặc cuối và chữ hoa không thống nhất, như `Member01@Example.INVALID` . Mỗi dòng là một địa chỉ; nhóm cần nhận lại danh sách có cách viết thống nhất, số địa chỉ và địa chỉ đầu/ cuối sau khi xử lý.

Vì sao chọn cách này? `strip()` bỏ khoảng trắng **chỉ ở hai đầu** chuỗi, còn `lower()` đổi chữ hoa thành chữ thường. Ví dụ `" Member01@Example.INVALID "` trở thành `member01@example.invalid`, nên cùng một địa chỉ sẽ có cách viết nhất quán khi đối chiếu. Vòng `for` tạo danh sách đã xử lý từng dòng để dễ nhìn bước làm. Lưu ý đây mới là **chuẩn hóa cách viết**, không phải kiểm tra một email có tồn tại hoặc nhận được thư hay không; đuôi `.invalid` dùng cho dữ liệu minh họa.

Quy trình:

1. Lưu mười chuỗi email thử nghiệm.
2. Bỏ khoảng trắng biên rồi chuyển thành chữ thường.
3. In số email và hai email mẫu đã chuẩn hóa.

No Note — Không chú thích

```
raw_emails = ["  Member01@Example.INVALID  ","  Member02@Example.INVALID  ","
Member03@Example.INVALID  ","  Member04@Example.INVALID  ","  Member05@Example.INVALID  ","
Member06@Example.INVALID  ","  Member07@Example.INVALID  ","  Member08@Example.INVALID  ","
Member09@Example.INVALID  ","  Member10@Example.INVALID  "]
cleaned = []
for email in raw_emails:
    normalized_email = email.strip().lower()
    cleaned.append(normalized_email)
if len(cleaned) != 10:
    raise ValueError("Phải chuẩn hóa đủ 10 địa chỉ email.")
print("Sô´email:", len(cleaned))
print("Đâù:", cleaned[0])
print("Cuôí:", cleaned[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuẩn hóa 10 email: xóa khoảng trăńg thừa và đôỉ vê`chữ thường.
# 10 chuôĩ email ví dụ có khoảng trăńg thừa và chữ hoa; môĩ phâǹ tư ̛là một địa chỉ.
raw_emails = ["  Member01@Example.INVALID  ","  Member02@Example.INVALID  ","
Member03@Example.INVALID  ","  Member04@Example.INVALID  ","  Member05@Example.INVALID  ","
Member06@Example.INVALID  ","  Member07@Example.INVALID  ","  Member08@Example.INVALID  ","
Member09@Example.INVALID  ","  Member10@Example.INVALID  "]
# Danh sách rôńg lưu email sau khi chuẩn hóa theo thứ tự đâù vào.
cleaned = []
# Vòng for lâý từng email làm đâù vào cho chuôĩ thao tác làm sạch.
for email in raw_emails:
    # strip() bỏ khoảng trăńg hai đâù; lower() đôỉ tât̂ cả chữ cái thành chữ thường; gọi nôí tiêṕ đê ̂
làm cả hai bước.
    normalized_email = email.strip().lower()
    # append() đưa email đã làm sạch vào cuôí danh sách.
    cleaned.append(normalized_email)
# len() đêḿ sô´email sau khi xư ̉lý; != kiêm̉ tra có thiêú/thừa so với 10 đâù vào.
if len(cleaned) != 10:
```

```
    # ValueError báo dữ liệu đầu vào sai số´lượng và ngừng chương trình.
    raise ValueError("Phải chuẩn hóa đủ 10 địa chỉ email.")
print("Số´email:", len(cleaned))
# [0] truy cập địa chỉ đầu tiên sau chuẩn hóa.
print("Đầu:", cleaned[0])
# [-1] truy cập địa chỉ cuối cùng, không cần tự tính chỉ số´.
print("Cuối:", cleaned[-1])
```

Kết quả khi chạy:

```
Số´email: 10
Đầu: member01@example.invalid
Cuối: member10@example.invalid
```

Toàn bộ 10 email đã được chuyển đổi. Miền `.invalid` được dùng có chủ đích cho dữ liệu minh họa, không phải tài khoản thật.

Tình huống 2 — Cấp 2: Tách nhãn sản phẩm trong 12 dòng nhập liệu

Từ vựng trong mục: Split — tách chuỗi; **Delimiter** — dấu phân cách; **Normalization** — chuẩn hóa.

Tình huống thực tế: Thư viện có **12 dòng ghi tên môn học** trên các thẻ đăng ký. Mỗi dòng chứa **hai nhãn** ngăn bằng dấu phẩy, ví dụ `python, AI`, nên 12 thẻ tạo ra nhiều hơn 12 lượt ghi nhãn. Nhân viên muốn biết từng môn như Python, AI, SQL xuất hiện bao nhiêu lần, dù người nhập có dùng chữ hoa hoặc khoảng trắng khác nhau.

Vì sao chọn cách này? Dấu phẩy đang có nhiệm vụ ngăn hai nhãn trong một dòng, vì vậy `split(',')` tách dòng thành từng phần. Sau đó `strip()` xóa khoảng trắng thừa và `lower()` giúp `Python` và `python` được đếm là **cùng một nhãn**. Dictionary `counts` lưu số lần xuất hiện của từng nhãn; mỗi khi gặp nhãn, tăng số đếm lên một. Không thể đếm số dòng thay cho số nhãn, vì một dòng chứa tới hai nhãn.

Quy trình:

1. Duyệt đủ 12 dòng, tách mỗi dòng thành hai nhãn.
2. Chuẩn hóa nhãn trước khi đưa vào bảng đếm.
3. In tổng dòng và số lần xuất hiện của từng nhãn theo thứ tự chữ cái.

No Note — Không chú thích

```
raw_labels = ["python, AI","SQL, python","math, data","Python, SQL","AI, data","math,
python","data, SQL","AI, Math","python, data","SQL, AI","Math, SQL","Python, AI"]
counts = {}
for text in raw_labels:
    for part in text.split(","):
        label = part.strip().lower()
        counts[label] = counts.get(label, 0) + 1
if len(raw_labels) != 12:
    raise ValueError("Phải có đủ 12 dòng nhãn sản phẩm.")
print("Số´dòng:", len(raw_labels))
for label in sorted(counts):
    print(label, counts[label])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách 12 dòng nhãn sản phẩm, đếm số´lần xuất hiện của từng nhãn.
# Mỗi chuỗi là một dòng gắn hai nhãn phân cách bởi dấu phẩy; 12 dòng có thể viết hoa/thường lẫn
```

```
nhau.
raw_labels = ["python, AI","SQL, python","math, data","Python, SQL","AI, data","math,
python","data, SQL","AI, Math","python, data","SQL, AI","Math, SQL","Python, AI"]
# Dictionary (từ điển) ánh xạ tên nhãn sang sô´ lần xuất hiện, ban đầu chưa có nhãn nào.
counts = {}
# Duyệt từng dòng; vòng for bên trong xử lý từng nhãn của dòng.
for text in raw_labels:
    # split(',') tách chuỗi tại mỗi dấu phẩy và trả danh sách các phần nhãn.
    for part in text.split(","):
        # strip() loại khoảng trắng cạnh nhãn; lower() gộp 'Python' và 'python' thành một nhãn.
        label = part.strip().lower()
        # get(label, 0) đọc sô´ đếm hiện có hoặc lấy 0 nếu nhãn chưa gặp; cộng 1 và ghi lại vào
dictionary.
        counts[label] = counts.get(label, 0) + 1
# Xác nhận đã nhận đủ 12 dòng nguồn, không phải 12 nhãn.
if len(raw_labels) != 12:
    # raise ValueError báo lỗi nếu sô´ dòng nguồn bị sai.
    raise ValueError("Phải có đủ 12 dòng nhãn sản phẩm.")
print("Sô´ dòng:", len(raw_labels))
# sorted(counts) lấy các khóa nhãn theo thứ tự chữ cái để in báo cáo ổn định.
for label in sorted(counts):
    # counts[label] tra sô´ lần xuất hiện của nhãn; print in tên nhãn và sô´ lượng.
    print(label, counts[label])
```

Kết quả khi chạy:

```
Sô´ dòng: 12
ai 5
data 4
math 4
python 6
sql 5
```

Mỗi dòng đóng góp hai nhãn, tổng tất cả số đếm phải bằng 24. Đây là sự khác biệt giữa dữ liệu gốc có nhiều cách viết và từ vựng đã chuẩn hóa.

Tình huống 3 — Cấp 3: Giữ số 0 ở đầu của 15 mã đơn hàng

Từ vựng trong mục: Identifier — mã định danh; **Slice** — cắt chuỗi; **Integer** — số nguyên.

Tình huống thực tế: Kho nhận **15 mã đơn hàng** gồm đúng **5 chữ số**, ví dụ `00001`. Đây là **mã nhận diện** dùng để tìm đơn, không phải số lượng, và lúc nhập có thể có khoảng trắng ở hai đầu. Người quản lý muốn kiểm tra từng mã còn đủ 5 chữ số, đồng thời hiển thị mã đầu và cuối mà vẫn giữ số 0 ở đầu.

Vì sao chọn cách này? Chữ số 0 đầu tiên trong `00001` là **một phần của mã**. Nếu đổi sang số nguyên `int`, chương trình sẽ chỉ còn `1`, làm mất dạng mã khi đối chiếu chứng từ. Giữ mã là **chuỗi**, dùng `strip()` bỏ khoảng trắng, `isdigit()` kiểm tra có đúng toàn chữ số, rồi `len()` kiểm tra đúng năm ký tự. Slicing như `code[:3]` chỉ lấy một phần ký tự để xem, không đổi mã gốc.

Quy trình:

1. Lưu 15 mã chuỗi.
2. Bỏ khoảng trắng và kiểm tra mã chỉ có năm chữ số.
3. In các mẫu đầu/cuối cùng ba ký tự đầu của mã đầu.

No Note — Không chú thích

```
raw_codes = ["  00001  ","  00002  ","  00003  ","  00004  ","  00005  ","  00006  ","  00007  ","
00008  ","  00009  ","  00010  ","  00011  ","  00012  ","  00013  ","  00014  ","  00015  "]
codes = []
for code in raw_codes:
    cleaned_code = code.strip()
    codes.append(cleaned_code)
valid = True
for code in codes:
    if not code.isdigit() or len(code) != 5:
        valid = False
        break
if len(codes) != 15:
    raise ValueError("Phải kiểm tra đủ 15 mã đơn hàng.")
print("Sô´mã:", len(codes))
print("Đêù hợp lệ:", valid)
print("Mã đâù:", codes[0], "| đâù mã:", codes[0][:3])
print("Mã cuôí:", codes[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Làm sạch và kiểm tra 15 mã sô´vâñ giữ đủ năm ký tự, kể cả sô´0 đâù.
# 15 mã sô´dạng chuôĩ có các sô´0 đâù và khoảng trăńg thừa; giữ kiểu chuôĩ để không mất sô´0.
raw_codes = ["  00001  ","  00002  ","  00003  ","  00004  ","  00005  ","  00006  ","  00007  ","
00008  ","  00009  ","  00010  ","  00011  ","  00012  ","  00013  ","  00014  ","  00015  "]
# Danh sách lưu mã đã làm sạch, vâñ là chuôĩ.
codes = []
for code in raw_codes:
    # strip() bỏ khoảng trăńg đâù/cuôí nhưng bảo toàn '00001'.
    cleaned_code = code.strip()
    # append() thêm mã sạch vào danh sách.
    codes.append(cleaned_code)
# Giả định các mã hợp lệ; gặp mã lôĩ thì chuyển cờ valid thành False.
valid = True
for code in codes:
    # isdigit() kiểm tra tất cả ký tự là chữ sô´; len() phải bằng 5; not và or bắt mọi trường hợp
sai.
    if not code.isdigit() or len(code) != 5:
        valid = False
        # break dừng vòng kiểm tra ngay khi tìm thấy mã không hợp lệ.
        break
# Đảm bảo đã xử lý đủ 15 mã từ nguôǹ.
if len(codes) != 15:
    # Báo ValueError nêú thiếú hoặc thừa mã nguôǹ.
    raise ValueError("Phải kiểm tra đủ 15 mã đơn hàng.")
print("Sô´mã:", len(codes))
print("Đêù hợp lệ:", valid)
# [0] lấy mã đâù, [:3] cắt ba ký tự đâù ở vị trí 0, 1, 2; không biến mã thành sô´.
print("Mã đâù:", codes[0], "| đâù mã:", codes[0][:3])
# [-1] là mã cuôí danh sách.
print("Mã cuôí:", codes[-1])
```

Kết quả khi chạy:

```
Sô´mã: 15
Đêù hợp lệ: True
Mã đâù: 00001 | đâù mã: 000
Mã cuôí: 00015
```

Đủ 15 mã được giữ nguyên 5 ký tự. Nếu dùng phép đổi sang số nguyên, thông tin định danh `00001` sẽ thành `1` và không còn đúng định dạng.

Tình huống 4 — Cấp 4: Làm sạch 18 giá đọc từ CSV

Từ vựng trong mục: Float — số thực; **Conversion** — đổi kiểu; **Invalid input** — đầu vào sai.

Tình huống thực tế: Nhà cung cấp gửi bảng **18 đơn giá** tính bằng **nghìn đồng/món**, tương ứng lần lượt các mã P01 đến P18. Mỗi giá đang được lưu ở dạng chữ để mô phỏng nội dung đọc từ CSV; dòng P03 ghi `bad`, dòng P14 để trống. Nhân viên cần cộng tổng các giá có thể đọc thành số, đếm các giá hợp lệ và báo đúng mã phải sửa; không được cộng các dòng lỗi như số 0.

Vì sao chọn cách này? Không phải chuỗi nào cũng đổi được thành số: `"12.5"` là giá có thể đọc, còn `bad` hoặc chuỗi rỗng là thông tin cần sửa. Chương trình làm sạch khoảng trắng rồi kiểm tra xem nội dung có chữ số và nhiều nhất một dấu chấm trước khi dùng `float()`. Phải **tách mã lỗi** và chỉ cộng các giá hợp lệ; nếu cố đổi `bad`, chương trình có thể dừng vì lỗi. Giá ở đây có đơn vị nghìn đồng, vì vậy tổng cũng là nghìn đồng, không tự hiểu thành đồng.

Quy trình:

1. Gán 18 mã giá theo thứ tự.
2. Bỏ khoảng trắng, kiểm tra định dạng số thập phân không âm.
3. Đổi số hợp lệ thành `float`, ghi lại ID không hợp lệ rồi cộng các giá trị hợp lệ.

No Note — Không chú thích

```
raw_prices =
["12.5","8.0","bad","7.5","13.0","10.0","19.5","6.5","11.0","15.5","9.0","16.0","20.0","","10.5","1
4.5","8.5","12.0"]
valid_prices = []
invalid_ids = []
for index, raw in enumerate(raw_prices, 1):
    clean = raw.strip()
    if not clean.replace(".", "", 1).isdigit():
        invalid_ids.append(f"P{index:02d}")
        continue
    valid_prices.append(float(clean))
if len(raw_prices) != 18:
    raise ValueError("Phải có đúng 18 chuỗi đơn giá.")
print("Giá hợp lệ:", len(valid_prices))
print("Giá lỗi:", ", ".join(invalid_ids))
print(f"Tổng giá (nghìn đồng): {sum(valid_prices):.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phát hiện các chuỗi đơn giá không hợp lệ trong 18 phiếu và cộng giá hợp lệ.
# 18 đơn giá ở dạng chuỗi, đơn vị nghìn đồng; 'bad' và chuỗi rỗng là dữ liệu lỗi.
raw_prices =
["12.5","8.0","bad","7.5","13.0","10.0","19.5","6.5","11.0","15.5","9.0","16.0","20.0","","10.5","1
4.5","8.5","12.0"]
# Lưu các số thực đã được xác minh để cộng, tách khỏi chuỗi không hợp lệ.
valid_prices = []
# Lưu mã dòng lỗi theo thứ tự xuất hiện.
invalid_ids = []
# enumerate(..., 1) đánh số dòng từ 1, giúp mã P01 tương ứng giá đầu tiên.
for index, raw in enumerate(raw_prices, 1):
    # strip() xóa khoảng trắng hai đầu trước khi kiểm tra chuỗi.
    clean = raw.strip()
    # replace('.', '', 1) xóa tối đa một dấu chấm thập phân; isdigit() kiểm tra phần còn lại là chữ
số; not tìm chuỗi sai. Đây là quy tắc minh họa cho giá không âm, chưa xử lý dấu âm hay định dạng số
tổng quát.
    if not clean.replace(".", "", 1).isdigit():
```

```
        # append() lưu mã giá lỗi; f-string :02d tạo số thứ tự hai chữ số như P03.
        invalid_ids.append(f"P{index:02d}")
        # continue bỏ phiếu giá lỗi, không gọi float() với chuỗi không chuyển được.
        continue
    # float() chuyển chuỗi thập phân sang số thực; append() thêm vào danh sách hợp lệ.
    valid_prices.append(float(clean))
# Kiểm tra đúng 18 dòng đầu vào, gồm cả giá lỗi.
if len(raw_prices) != 18:
    # ValueError thông báo thiếu/thừa dòng trong bộ dữ liệu.
    raise ValueError("Phải có đúng 18 chuỗi đơn giá.")
print("Giá hợp lệ:", len(valid_prices))
# join() ghép danh sách mã lỗi bằng dấu phẩy và khoảng trắng.
print("Giá lỗi:", ", ".join(invalid_ids))
# sum() cộng toàn bộ giá hợp lệ; :.1f hiển thị kết quả với một chữ số sau dấu thập phân, đơn vị
nghìn đồng.
print(f"Tổng giá (nghìn đồng): {sum(valid_prices):.1f}")
```

Kết quả khi chạy:

```
Giá hợp lệ: 16
Giá lỗi: P03, P14
Tổng giá (nghìn đồng): 194.0
```

Đây là tổng **16 đơn giá mẫu**, không phải doanh thu; để tính doanh thu còn cần số lượng bán tương ứng. Quy tắc kiểm tra ký tự chỉ phù hợp với định dạng số không âm đơn giản của bài.

Tình huống 5 — Cấp 5: Chuẩn hóa, kiểm tra và che 20 mã hồ sơ

Từ vựng trong mục: Boolean — đúng/sai; **String validation** — kiểm tra chuỗi; **Masking** — che thông tin.

Tình huống thực tế: Hệ thống nhận **20 mã hồ sơ thử nghiệm**. Mã đúng có dạng `ID-` rồi tới **bốn chữ số**, chẳng hạn `ID-0001`; khoảng trắng đầu/cuối không mang ý nghĩa của mã. Trong dữ liệu có mã sai tiền tố, mã thiếu chữ số và mã xuất hiện hai lần. Nhân viên cần đếm số mã hợp lệ duy nhất, tách danh sách lỗi và trùng, sau đó hiển thị một mã dưới dạng che bớt phần đầu khi báo cáo.

Vì sao chọn cách này? Đầu tiên `strip().upper()` đưa mọi mã về cách viết thống nhất, rồi chương trình kiểm tra phần tiền tố `ID-` và bốn chữ số cuối. Một mã xuất hiện lần thứ hai là **trùng**, khác với mã nhập sai định dạng; nên chia chúng thành các nhóm riêng. Set `seen` ghi nhớ mã đã gặp: trước khi thêm mã mới, kiểm tra nó đã có trong set chưa. Việc chỉ hiển thị hai chữ số cuối giúp báo cáo không đưa nguyên mã ra màn hình; đây là ví dụ **che bớt ký tự**, chưa phải giải pháp bảo mật dữ liệu đầy đủ.

Quy trình:

1. Chuẩn hóa 20 chuỗi, tách tiền tố và hậu tố.
2. Xác nhận độ dài và chữ số, kiểm tra mã đã xuất hiện chưa.
3. Chỉ giữ mã hợp lệ, báo mã sai/trùng và in một mã đã che.

No Note — Không chú thích

```
raw_ids = [" ID-0001 "," ID-0002 "," ID-0003 "," ID-0004 "," ID-0005 "," ID-0006 ","X-0007","
ID-0008 "," ID-0009 "," ID-0010 "," ID-0011 "," ID-0004 "," ID-0013 "," ID-0014 "," ID-0015 ","
ID-0016 "," ID-0017 "," ID- "," ID-0019 "," ID-0020 "]
accepted = []
invalid = []
duplicates = []
seen = set()
for raw in raw_ids:
    code = raw.strip().upper()
```

```
    suffix = code[3:]
    if not (code.startswith("ID-") and len(suffix) == 4 and suffix.isdigit()):
        invalid.append(code)
    elif code in seen:
        duplicates.append(code)
    else:
        seen.add(code)
        accepted.append(code)
if len(raw_ids) != 20:
    raise ValueError("Phải có đúng 20 mã hồ sơ để kiểm tra.")
print("Đầu vào:", len(raw_ids))
print("Hợp lệ duy nhất:", len(accepted))
print("Sai định dạng:", len(invalid))
print("Trùng:", duplicates)
print("Mã đầu đã che:", "ID-**" + accepted[0][-2:])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tra 20 mã hồ sơ: đúng định dạng, không trùng và chỉ hiển thị một phần mã.
# 20 mã hồ sơ dạng chuỗi; có mã sai tiền tố, mã rút ngắn và mã lặp cần phát hiện.
raw_ids = [" ID-0001 "," ID-0002 "," ID-0003 "," ID-0004 "," ID-0005 "," ID-0006 ","X-0007","
ID-0008 "," ID-0009 "," ID-0010 "," ID-0011 "," ID-0004 "," ID-0013 "," ID-0014 "," ID-0015 ","
ID-0016 "," ID-0017 "," ID- "," ID-0019 "," ID-0020 "]
accepted = []
invalid = []
duplicates = []
# set() tạo tập hợp lưu mã đã gặp, giúp kiểm tra trùng nhanh bằng toán tử in.
seen = set()
# Kiểm tra lần lượt từng mã gốc, giữ thứ tự báo cáo.
for raw in raw_ids:
    # strip() bỏ khoảng trắng; upper() đổi chữ cái thành viết hoa để chuẩn hóa tiền tố.
    code = raw.strip().upper()
    # [3:] lấy mọi ký tự từ vị trí 3 trở đi sau tiền tố ba ký tự 'ID-'.
    suffix = code[3:]
    # startswith('ID-') kiểm tra tiền tố; len()==4 kiểm tra 4 ký tự hậu tố; isdigit() đảm bảo toàn
chữ số; and đòi đủ điều kiện, not nhận diện mã sai.
    if not (code.startswith("ID-") and len(suffix) == 4 and suffix.isdigit()):
        # append() ghi riêng mã không đúng định dạng, chưa đưa vào danh sách hợp lệ.
        invalid.append(code)
    # in kiểm tra mã đã xuất hiện trong tập seen; elif chỉ chạy khi định dạng đã hợp lệ.
    elif code in seen:
        # Ghi mã trùng vào danh sách để rà soát.
        duplicates.append(code)
    else:
        # add() thêm mã hợp lệ mới vào tập để phát hiện lần xuất hiện sau.
        seen.add(code)
        # append() giữ danh sách mã hợp lệ duy nhất theo thứ tự ban đầu.
        accepted.append(code)
# Xác nhận đầu vào đủ 20 hồ sơ, kể cả mã sai.
if len(raw_ids) != 20:
    # Dừng và báo ValueError nếu số hồ sơ không đủ.
    raise ValueError("Phải có đúng 20 mã hồ sơ để kiểm tra.")
print("Đầu vào:", len(raw_ids))
print("Hợp lệ duy nhất:", len(accepted))
print("Sai định dạng:", len(invalid))
print("Trùng:", duplicates)
# accepted[0] là mã hợp lệ đầu, [-2:] lấy hai ký tự cuối; nối với 'ID-**' để che phần giữa khi hiển
thị.
print("Mã đầu đã che:", "ID-**" + accepted[0][-2:])
```

Kết quả khi chạy:

```
Đầu vào: 20
Hợp lệ duy nhất: 17
Sai định dạng: 2
Trùng: ['ID-0004']
Mã đầu đã che: ID-**01
```

20 đầu vào được chia thành 17 mã hợp lệ duy nhất, hai mã sai và một bản sao; phép che chỉ giảm khả năng lộ mã trong bản tin, **không phải mã hóa hay biện pháp bảo mật hoàn chỉnh**.

6. Những nhầm lẫn thường gặp

- `int("12.5")` gây lỗi vì chuỗi này không viết theo dạng số nguyên; dùng `float("12.5")` nếu cần số có phần thập phân.
- Không chuyển mã đơn hoặc số điện thoại sang `int` nếu cần giữ số `0` ở đầu.
- `strip()` chỉ bỏ khoảng trắng ở hai đầu chuỗi; `lower()` không tự kiểm tra email hợp lệ.
- `s[:3]` lấy ba ký tự đầu (vị trí 0, 1, 2), không lấy ký tự ở vị trí 3.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. Which description best defines an integer in Python? | Mô tả nào đúng nhất về số nguyên trong Python?

A. An ordered sequence of characters | Dãy ký tự có thứ tự

B. A negative, zero, or positive integer without a fractional part | Số nguyên âm, số 0 hoặc số nguyên dương không có phần lẻ

C. A true/false value | Một giá trị đúng/sai

D. A floating-point number | Số thực dấu phẩy động

Đáp án: B

Giải thích: Kiểu `int` của Python lưu chính xác các số nguyên âm, số 0 và số nguyên dương như `-3`, `0`, `12`. Không loại số âm khỏi kiểu này; khác với `int`, `float` biểu diễn số dấu phẩy động, kể cả `2.0`.

Q2. What is a float? | Float là kiểu dữ liệu nào?

A. A whole number without a fractional part | Số không có phần thập phân

B. A floating-point number | Số thực dấu phẩy động

C. A true/false value | Một giá trị đúng/sai

D. An ordered sequence of characters | Dãy ký tự có thứ tự

Đáp án: B

Giải thích: `float` là kiểu số dấu phẩy động trong Python, ví dụ `1.5` và `2.0`. Ngay cả `2.0` dù có giá trị toán học nguyên vẫn là `float` chứ không phải `int`.

Q3. What is a Boolean? | Boolean là gì?

A. A true/false value | Một giá trị đúng/sai

B. A floating-point number | Số thực dấu phẩy động

C. An ordered sequence of characters | Dãy ký tự có thứ tự

D. A whole number without a fractional part | Số không có phần thập phân

Đáp án: A

Giải thích: Boolean biểu diễn một trong hai trạng thái `True` hoặc `False`, thường dùng trong điều kiện như `3 > 2`. Vai trò của nó khác việc lưu số hoặc dãy ký tự có thứ tự.

Q4. What is a string? | Chuỗi là gì?

A. An ordered sequence of characters | Dãy ký tự có thứ tự

B. A whole number without a fractional part | Số không có phần thập phân

C. A floating-point number | Số thực dấu phẩy động

D. A true/false value | Một giá trị đúng/sai

Đáp án: A

Giải thích: Chuỗi (`str`) là dãy ký tự Unicode có thứ tự, chẳng hạn `'Python'`. Chuỗi chữ số như `'12'` vẫn là văn bản nên cộng với chuỗi khác sẽ nối chữ, không phải cộng số.

Q5. What does `strip()` normally remove? | `strip()` thường loại bỏ gì?

A. Whitespace at both ends | Khoảng trắng ở hai đầu

B. Text converted to lowercase | Văn bản chuyển sang chữ thường

C. Text converted to uppercase | Văn bản chuyển sang chữ hoa

D. Substitutes matching text | Thay các đoạn chữ khớp mẫu

Đáp án: A

Giải thích: Khi không truyền tham số, `strip()` bỏ khoảng trắng ở đầu và cuối, không xóa khoảng trắng giữa chuỗi. Ví dụ `' a b '.strip()` thành `'a b'` và giữ dấu cách giữa `a` và `b`.

Q6. What does `lower()` return? | `lower()` trả về gì?

A. Substitutes matching text | Thay các đoạn chữ khớp mẫu

B. Text converted to uppercase | Văn bản chuyển sang chữ hoa

C. Text converted to lowercase | Văn bản chuyển sang chữ thường

D. Whitespace at both ends | Khoảng trắng ở hai đầu

Đáp án: C

Giải thích: `lower()` trả về bản sao chữ thường, nên `'PyThOn'.lower()` được `'python'`. Hàm không sửa chuỗi gốc tại chỗ vì chuỗi là bất biến.

Q7. What does `upper()` return? | `upper()` trả về gì?

A. Whitespace at both ends | Khoảng trắng ở hai đầu

B. Substitutes matching text | Thay các đoạn chữ khớp mẫu

C. Text converted to lowercase | Văn bản chuyển sang chữ thường

D. Text converted to uppercase | Văn bản chuyển sang chữ hoa

Đáp án: D

Giải thích: `upper()` trả về chữ hoa, ví dụ `'ab'.upper()` là `'AB'`. Khác `lower()`, nó đổi chữ cái về dạng hoa; ký tự như chữ số vẫn giữ nguyên.

Q8. What does `replace(old, new)` do? | `replace(old, new)` có tác dụng gì?

A. Whitespace at both ends | Khoảng trắng ở hai đầu

B. Text converted to uppercase | Văn bản chuyển sang chữ hoa

C. Text converted to lowercase | Văn bản chuyển sang chữ thường

D. Substitutes matching text | Thay các đoạn chữ khớp mẫu

Đáp án: D

Giải thích: `replace(old, new)` trả về chuỗi mới sau khi thay các đoạn khớp; `'aba'.replace('a', 'x')` được `'xbx'`. Hàm không sửa chuỗi gốc và không chỉ đơn thuần đổi hoa/thường.

Q9. What does `'a,b'.split(',')` return? | `'a,b'.split(',')` trả về gì?

A. `['a', 'b']` | `['a', 'b']`

B. `'ab'` | `'ab'`

C. `('a', 'b')` | `('a', 'b')`

D. `['a,b']` | `['a,b']`

Đáp án: A

Giải thích: `split(',')` tách văn bản tại mỗi dấu phẩy và trả về danh sách các chuỗi con: `'a,b'` thành `['a', 'b']`. Dấu phẩy là dấu phân cách nên bị bỏ đi; kết quả là list chứ không phải một chuỗi.

Q10. What does `'ABCD'[1:3]` yield? | `'ABCD'[1:3]` cho kết quả gì?

A. `'CD'` | `'CD'`

B. `'AB'` | `'AB'`

C. `'BC'` | `'BC'`

D. `'BCD'` | `'BCD'`

Đáp án: C

Giải thích: Chỉ số chuỗi bắt đầu từ 0, nên trong `'ABCD'`, vị trí 1 và 2 là `B` và `C`. Lát cắt `[1:3]` lấy vị trí 1 nhưng không lấy mốc dừng 3, cho `'BC'`.

Q11. What does `len('Python')` yield? | `len('Python')` trả về gì?

A. `5` | `5`

B. `7` | `7`

C. `'6'` (string) | `'6'` (chuỗi)

D. `6` (integer) | `6` (số nguyên)

Đáp án: D

Giải thích: `len()` đếm số ký tự trong chuỗi: `P`, `y`, `t`, `h`, `o`, `n` là sáu ký tự. Hàm trả về số nguyên `6`, không phải chuỗi `'6'`; chữ hoa/thường không đổi số lượng.

Q12. What does `'A'.join(['x','y'])` yield? | `'A'.join(['x','y'])` tạo gì?

A. `['x', 'A', 'y']` | `['x', 'A', 'y']`

B. `'xyA'` | `'xyA'`

C. `'xAy'` | `'xAy'`

D. `'Axy'` | `'Axy'`

Đáp án: C

Giải thích: `join()` chèn chuỗi đứng trước dấu chấm vào giữa các phần tử: `'A'.join(['x','y'])` cho `'xAy'`. Nó không thêm `A` ở đầu toàn danh sách hoặc sau phần tử cuối.

Q13. What does `int('12')` create? | `int('12')` tạo ra gì?

A. `ValueError` | `ValueError`

B. `12` (integer) | `12` (số nguyên)

C. `12.0` (float) | `12.0` (số thực)

D. `'12'` (string) | `'12'` (chuỗi)

Đáp án: B

Giải thích: `int('12')` đổi chuỗi số nguyên hợp lệ thành số nguyên `12` để có thể tính toán. Kết quả không còn là văn bản `'12'` và cũng không phải số thực `12.0`.

Q14. What does `float('1.5')` create? | `float('1.5')` tạo ra gì?

A. `ValueError` | `ValueError`

B. `1` (integer) | `1` (số nguyên)

C. `'1.5'` (string) | `'1.5'` (chuỗi)

D. `1.5` (float) | `1.5` (số thực)

Đáp án: D

Giải thích: `float('1.5')` chuyển văn bản số hợp lệ thành số thực `1.5`. Nó giữ phần lẻ thay vì cắt thành số nguyên `1`, và chuỗi hợp lệ này không gây lỗi.

Q15. What does `str(12)` create? | `str(12)` tạo gì?

A. `['1', '2']` | `['1', '2']`

B. `'12'` (string) | `'12'` (chuỗi)

C. `12.0` (float) | `12.0` (số thực)

D. `12` (integer) | `12` (số nguyên)

Đáp án: B

Giải thích: `str(12)` tạo biểu diễn văn bản `'12'`, phù hợp làm nhãn hoặc nối chuỗi. Kết quả không còn là số nguyên 12: `'12' + '3'` tạo `'123'`, không phải tổng số 15.

Q16. What does `bool('False')` return? | `bool('False')` trả về gì?

A. `True` | `True`

B. `False` | `False`

C. `ValueError` | `ValueError`

D. `'False'` (string) | `'False'` (chuỗi)

Đáp án: A

Giải thích: `bool()` kiểm tra tính đúng/sai của giá trị, không đọc nghĩa chữ `True` hay `False` trong văn bản. Do chuỗi `'False'` không rỗng nên `bool('False')` bằng `True`; với chuỗi, chỉ chuỗi rỗng được xem là sai.

Q17. What is true of Python strings? | Điều nào đúng về chuỗi Python?

A. Strings are unordered collections | Chuỗi là tập hợp không có thứ tự

B. Strings store only numbers | Chuỗi chỉ lưu số

C. Strings are immutable | Chuỗi không thể sửa trực tiếp từng ký tự

D. Strings can be changed in place with `s[0] = 'x'` | Có thể đổi chuỗi tại chỗ bằng `s[0] = 'x'`

Đáp án: C

Giải thích: Chuỗi là bất biến: gán `s[0] = 'x'` gây `TypeError`, kể cả khi `s` không rỗng. Các phép như `replace()` tạo chuỗi mới thay vì sửa từng ký tự của chuỗi cũ.

Q18. What does `s[-1]` select for a nonempty string? | `s[-1]` lấy gì khi chuỗi không rỗng?

A. The first character | Ký tự đầu

B. Always raises `IndexError` | Luôn gây `IndexError`

C. The final two characters | Hai ký tự cuối

D. The last character | Ký tự cuối

Đáp án: D

Giải thích: Chỉ số `-1` đếm từ cuối và lấy ký tự cuối của chuỗi không rỗng. Với `s = 'cat'`, `s[-1]` là `'t'`, còn `s[0]` lấy `'c'`.

Q19. What does `'Ab' == 'ab'` evaluate to? | `'Ab' == 'ab'` có giá trị gì?

A. `'False'` (string) | `'False'` (chuỗi)

B. `True` (Boolean) | `True` (Boolean)

C. `False` (Boolean) | `False` (Boolean)

D. `0` (integer) | `0` (số nguyên)

Đáp án: C

Giải thích: So sánh chuỗi xét từng ký tự và phân biệt chữ hoa/thường: `'A'` khác `'a'` nên `'Ab' == 'ab'` cho Boolean `False`. Nếu đổi cả hai chuỗi về chữ thường trước khi so sánh thì kết quả sẽ khác.

Q20. What is the result of `'4' + '2'`? | `'4' + '2'` cho kết quả gì?

A. `TypeError` | `TypeError`

B. `'42'` (string) | `'42'` (chuỗi)

C. `'6'` (string) | `'6'` (chuỗi)

D. 6 (integer) | 6 (số nguyên)

Đáp án: B

Giải thích: Cả hai toán hạng đều là chuỗi nên + nối theo thứ tự: `'4' + '2'` được `'42'`. Muốn cộng số, cần đổi kiểu trước, ví dụ `int('4') + int('2')` cho số nguyên 6.

8. Nối sang bài kế tiếp

`Biến → int/float/bool/str → Slicing → Xử lý văn bản.`

PY-03 — Toán tử số học, so sánh và logic

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Arithmetic — số học; **Comparison** — so sánh.

`Kiểu dữ liệu → Toán tử → Điều kiện → Quy tắc nghiệp vụ`

Học xong dùng để làm gì? Tính tổng tiền, so sánh ngưỡng điểm và kết hợp nhiều điều kiện thành kết quả đúng/sai.

Kết nối kiến thức tiếp theo: Sang PY-04, dùng chính các phép so sánh và logic đó để quyết định chương trình đi nhánh nào.

2. English vocabulary — Từ vựng

English	Nghĩa tiếng Việt	Giải thích
Arithmetic	số học	Phép cộng trừ nhân chia
Comparison	so sánh	So sánh hai giá trị
Logical	logic	Kết hợp các điều kiện
Bitwise	thao tác bit	Phép tính trên các bit của số nguyên

3. Các hàm và tùy chọn Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và ý nghĩa lựa chọn
`+, -, *, /`	Tính số học và tỷ lệ
`//, %, **`	Chia nguyên, lấy dư, lũy thừa
`==, !=, <=, >=, and, or`	So sánh và kết hợp điều kiện
`&, |, ^`	Chỉ dùng cho cờ bit hoặc toán bit được thiết kế rõ

4. Khái niệm liên quan

Toán tử (operator) là dấu hoặc từ khóa thực hiện một thao tác trên dữ liệu. Với một đơn hàng, ta dùng toán tử số học để tính tiền, so sánh để kiểm tra ngưỡng, rồi dùng logic để ghép điều kiện. Toán tử bit cũng dùng ký hiệu nhưng xử lý các chữ số nhị phân thay vì điều kiện thông thường.

4.1. Arithmetic — Toán tử số học

Từ vựng trong mục: Arithmetic — số học; **Multiplication** — phép nhân.

Số học giúp tính từ các dữ liệu có sẵn: `+` cộng, `-` trừ, `*` nhân, `/` chia. Ví dụ cửa hàng bán 5 món, mỗi món 3 nghìn đồng: lấy số lượng nhân đơn giá để được tổng 15 nghìn đồng. Python còn có `//` chia lấy phần nguyên và `%` lấy phần dư khi chia thùng hoặc chia nhóm.

No Note — Không chú thích

```
print(5 * 3)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhân 5 với 3 để tính tổng số món.
# * là phép nhân; print() in kết quả 15.
print(5 * 3)
```

Kết quả khi chạy:

```
15
```

Giá trị `15` là tổng tiền theo đơn vị nghìn đồng của ví dụ.

4.2. Comparison — Toán tử so sánh

Từ vựng trong mục: Comparison — so sánh; **Boolean** — đúng/sai.

So sánh trả lời một câu hỏi bằng `True` hoặc `False`. Nếu kho có 8 món và đơn hàng cần 5 món, điều kiện `8 >= 5` kiểm tra đủ hàng để giao không. Các dấu `>`, `<`, `>=`, `<=`, `==`, `!=` biểu diễn lớn hơn, nhỏ hơn, bằng hoặc khác nhau. Phân biệt `==` để kiểm tra với `=` để gán.

No Note — Không chú thích

```
print(8 >= 5)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So sánh 8 với ngưỡng 5.
# >= là lớn hơn hoặc bằng; kết quả so sánh kiểu bool là True.
print(8 >= 5)
```

Kết quả khi chạy:

```
True
```

Kho có ít nhất 5 món nên điều kiện đủ hàng là đúng.

4.3. Logical — Toán tử logic

Từ vựng trong mục: Logical — logic; **AND** — và; **OR** — hoặc.

Sau khi có các điều kiện đúng/sai, `and` yêu cầu mọi điều kiện cùng đúng; `or` chỉ cần một điều kiện đúng; `not` đảo trạng thái. Ví dụ người dùng đã đăng nhập (`True`) nhưng chưa xác minh (`False`): nếu hệ thống bắt buộc cả hai thì `and` trả về `False`.

No Note — Không chú thích

```
print(True and False)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng and logic để ghép hai điều kiện.
# and chỉ đúng khi cả hai điều kiện đúng; True and False trả False.
print(True and False)
```

Kết quả khi chạy:

```
False
```

Một trong hai điều kiện chưa đạt, nên không thỏa yêu cầu `and`.

4.4. Bitwise — Toán tử theo bit

Từ vựng trong mục: Bitwise — theo bit; **Binary** — nhị phân.

Máy tính biểu diễn số nguyên bằng các bit `0` và `1`. Toán tử `&` so từng vị trí bit và chỉ giữ `1` khi hai phía cùng có `1`. Số 5 là `0101`, số 3 là `0011`; phần chung chỉ còn `0001`, tương ứng số 1. Đây là phép xử lý số nguyên theo bit, **khác** `and` dùng để nối điều kiện logic.

No Note — Không chú thích

```
print(5 & 3)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Làm phép AND từng bit nhị phân thay vì logic and.
# & là AND từng bit: 5=0101, 3=0011 nên được 0001 (1), khác toán tử logic and.
print(5 & 3)
```

Kết quả khi chạy:

```
1
```

Số `1` là kết quả phép AND theo bit giữa 5 và 3.

5. Năm tình huống ứng dụng — từ phép toán đến quy tắc tổng hợp

Tình huống 1 — Cấp 1: Tính giảm giá cho 10 mặt hàng

Từ vựng trong mục: Arithmetic — toán tử số học; **Discount** — giảm giá.

Tình huống thực tế: Một cửa hàng bán đồ dùng học tập đang áp dụng giảm giá **10% cho mọi mặt hàng**. Bảng có **10 giá bán**, lần lượt 100, 150, 200, 250, 300, 120, 180, 220, 400 và 160, đơn vị **nghìn đồng/món**. Một khách mua đúng một món của mỗi loại. Thu ngân cần báo tổng tiền nếu không giảm và tổng tiền khách thực trả sau chương trình giảm giá.

Vì sao chọn cách này? Giảm 10% nghĩa là mỗi món bớt đi một phần mười giá gốc, nên khách trả 90% giá ban đầu. Chương trình dùng `sum()` cộng 10 giá gốc, sau đó dùng `for` lấy từng giá nhân với phần còn phải trả và cộng dồn vào tổng mới. Ví dụ món giá 100 nghìn đồng sẽ còn 90 nghìn đồng; làm giống vậy cho đủ 10 món thì không bị bỏ sót. Không trừ thẳng 10 nghìn đồng ở mỗi món, vì đó là mức giảm cố định, không phải 10%. Cũng không giảm hai lần vì cả bảng chỉ có một ưu đãi.

Quy trình:

1. Lưu 10 đơn giá và tỷ lệ giảm 10%.
2. Lấy 90% của mỗi giá rồi cộng tổng.
3. So sánh tổng trước/sau giảm.

No Note — Không chú thích

```
prices = [100,150,200,250,300,120,180,220,400,160]
discount = 0.10
original = sum(prices)
after_discount = 0
for price in prices:
    discounted_price = price * (1 - discount)
    after_discount += discounted_price
if len(prices) != 10:
    raise ValueError("Phải có đúng 10 giá mặt hàng.")
print("Mặt hàng:", len(prices))
print("Gôć (nghìn đôìng):", original)
print("Sau giảm (nghìn đôìng):", round(after_discount))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính tổng đơn giá sau giảm 10% cho 10 món.
# Dữ liệu: 10 đơn giá tính bằng nghìn đồng, môĩ phần tư ̉cho một mặt hàng.
prices = [100,150,200,250,300,120,180,220,400,160]
# 0.10 tương ứng giảm 10%; 1-discount là tỉ lệ 90% cần thanh toán.
discount = 0.10
# sum() cộng mọi giá gôć trong danh sách.
original = sum(prices)
# 0.10 tương ứng giảm 10%; 1-discount là tỉ lệ 90% cần thanh toán.
after_discount = 0
# for đọc môĩ đơn giá theo thứ tự.
for price in prices:
    # * nhân đơn giá với 90% đê ̉tính tiền còn phải trả.
    discounted_price = price * (1 - discount)
    # += cộng giá từng món vào tổng sau giảm.
    after_discount += discounted_price
# len() đêḿ phần tư;̀ != phát hiện sô ́lượng khác mức yêu câù.
if len(prices) != 10:
    # raise ValueError(...) báo lôĩ rôi dừng xư ̉lý khi dữ liệu không nhất quán.
    raise ValueError("Phải có đúng 10 giá mặt hàng.")
print("Mặt hàng:", len(prices))
print("Gôć (nghìn đôìng):", original)
# round() làm tròn tổng khi hiển thị, không làm tròn từng món.
print("Sau giảm (nghìn đôìng):", round(after_discount))
```

Kết quả khi chạy:

```
Mặt hàng: 10
Gôć (nghìn đôìng): 2080
Sau giảm (nghìn đôìng): 1872
```

Đầu vào là **giá từng mặt hàng**, không phải số lượng bán. Mức giảm 10% áp dụng như nhau nên có thể đối chiếu với 90% tổng tiền gốc.

Tình huống 2 — Cấp 2: So hai điều kiện VIP của 12 khách

Từ vựng trong mục: Comparison — so sánh; **Logical AND** — phép và.

Tình huống thực tế: Cửa hàng muốn chọn khách được ưu đãi VIP trong danh sách **12 khách C01 đến C12**. Mỗi người có số năm mua hàng tại cửa hàng (đơn vị **năm**) và tổng tiền đã chi (đơn vị **đồng**); chẳng hạn một người có 5 năm và đã chi 900.000 đồng. Theo quy định, khách VIP phải mua hàng **ít nhất 5 năm** và đã chi **ít nhất 1.000.000 đồng**. Nhân viên cần biết có bao nhiêu VIP và mã của từng người.

Vì sao chọn cách này? Điều kiện là một yêu cầu cần được kiểm tra đúng hoặc sai. >= có nghĩa **lớn hơn hoặc bằng**: khách đúng 5 năm vẫn đủ điều kiện thời gian; đúng 1.000.000 đồng vẫn đạt mức chi. `and` yêu cầu **cả hai điều kiện cùng đúng**, nên khách có 8 năm nhưng chỉ chi 850.000 đồng không được xếp VIP. Nếu dùng `or` (chỉ cần một điều kiện), hệ thống sẽ ưu đãi nhầm. Vòng `for` kiểm từng khách, ghi mã đạt yêu cầu vào danh sách và đếm số mã.

Quy trình:

1. Ghép năm thành viên với số tiền của đúng khách ở cùng hàng.
2. So hai ngưỡng bằng >= và nối bằng `and`.
3. Liệt kê đủ khách đạt và đếm kết quả.

No Note — Không chú thích

```
years = [2,5,6,4,8,5,1,9,3,7,5,2]
spending_vnd =
[1200000,900000,1400000,2200000,1800000,1000000,700000,850000,1500000,3000000,990000,1600000]
customers = []
for i in range(1, 13):
    customer_id = f"C{i:02d}"
    customers.append((customer_id, years[i - 1], spending_vnd[i - 1]))
vip_ids = []
for code, member_years, spending in customers:
    if member_years >= 5 and spending >= 1000000:
        vip_ids.append(code)
if len(customers) != 12:
    raise ValueError("Phải có đúng 12 khách hàng.")
print("Sô´khách:", len(customers))
print("VIP:", len(vip_ids))
print("Mã VIP:", ", ".join(vip_ids))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân loại 12 khách VIP theo năm và sô´tiền.
# Dữ liệu: 12 năm thành viên khớp 12 khoản chi VND theo cùng vị trí.
years = [2,5,6,4,8,5,1,9,3,7,5,2]
spending_vnd =
[1200000,900000,1400000,2200000,1800000,1000000,700000,850000,1500000,3000000,990000,1600000]
customers = []
# range(1,13) sinh sô´thứ tự 1..12, không gôm 13.
for i in range(1, 13):
    # f-string :02d viêt mã C01..C12 với hai chữ sô´
    customer_id = f"C{i:02d}"
    # append() thêm bộ (mã, năm, sô´tiền); [i-1] chuyển thứ tự 1..12 sang chỉ sô´0..11.
    customers.append((customer_id, years[i - 1], spending_vnd[i - 1]))
vip_ids = []
```

```
# Tách từng bộ ba của một khách để đánh giá.
for code, member_years, spending in customers:
    # and yêu cầu đồng thời ít nhất 5 năm và chi tiêu từ 1 triệu đồng.
    if member_years >= 5 and spending >= 1000000:
        # append() lưu mã khách đủ tiêu chí.
        vip_ids.append(code)
# len() đếm phần tử; != phát hiện số lượng khác mức yêu cầu.
if len(customers) != 12:
    # raise ValueError(...) báo lỗi rồi dừng xử lý khi dữ liệu không nhất quán.
    raise ValueError("Phải có đúng 12 khách hàng.")
print("Số khách:", len(customers))
print("VIP:", len(vip_ids))
# join() nối các mã VIP bằng dấu phẩy và khoảng trắng.
print("Mã VIP:", ", ".join(vip_ids))
```

Kết quả khi chạy:

```
Số khách: 12
VIP: 4
Mã VIP: C03, C05, C06, C10
```

Mẫu C06 nằm **đúng cả hai ngưỡng** nên vẫn là VIP; C02 chỉ đạt năm thành viên nhưng chưa đủ tiền nên không thuộc danh sách.

Tình huống 3 — Cấp 3: Đóng thùng 15 mã hàng, tính cả số dư

Từ vựng trong mục: Integer division — chia nguyên; **Modulo** — chia lấy dư; **Check** — đối chiếu.

Tình huống thực tế: Kho kiểm kê **15 loại hàng**; mỗi con số là số món hiện còn của một loại, ví dụ loại đầu có 16 món. Kho chỉ đóng **6 món cùng loại trong một thùng**, không trộn các loại khác nhau. Nhân viên cần tính có thể đóng được tổng cộng bao nhiêu thùng đầy và còn bao nhiêu món lẻ chưa đủ thùng, để đối chiếu tổng số món trong kho.

Vì sao chọn cách này? SKU ở đây chỉ là một mã đại diện cho **một loại hàng**. Với loại có 16 món, cứ đủ 6 món thì được một thùng: đóng hai thùng và còn bốn món. Phép chia nguyên `//` lấy số thùng đủ, phép `%` lấy số món dư; lặp qua 15 loại rồi cộng số thùng và số dư riêng. Không cộng tất cả tồn kho trước rồi mới chia, vì như vậy có thể lấy món dư của loại này ghép với loại khác, trái quy định. Đối chiếu bằng cách nhân tổng thùng với 6 rồi cộng món dư: phải ra đúng tổng món ban đầu.

Quy trình:

1. Tính `stock // 6` và `stock % 6` cho từng SKU.
2. Cộng hai kết quả theo 15 SKU.
3. Xác minh `6 × tổng thùng + món rời` bằng tổng hàng gốc.

No Note — Không chú thích

```
stocks = [16,23,34,7,12,5,31,42,18,27,15,9,35,14,20]
box_size = 6
full_boxes = 0
loose_items = 0
for stock in stocks:
    full_boxes += stock // box_size
    loose_items += stock % box_size
if len(stocks) != 15:
    raise ValueError("Phải có đúng 15 mặt hàng tồn kho.")
if full_boxes * box_size + loose_items != sum(stocks):
    raise ValueError("Số thùng đầy và món lẻ phải khớp tổng tồn kho.")
```

```
print("Sô´SKU:", len(stocks))
print("Thùng đầy:", full_boxes)
print("Món rời:", loose_items)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính thùng đầy sáu món và món lẻ của 15 SKU.
# Dữ liệu: 15 sô´món tồn theo từng SKU, môĩ thùng chứa 6 món cùng SKU.
stocks = [16,23,34,7,12,5,31,42,18,27,15,9,35,14,20]
# Môĩ thùng có 6 món cùng SKU; phần lẻ SKU này không gộp với SKU khác.
box_size = 6
full_boxes = 0
loose_items = 0
for stock in stocks:
    # // chia lấy phần nguyên tức sô´thùng đầy; += cộng vào tổng.
    full_boxes += stock // box_size
    # % chia lấy phần dư tức món rời; += cộng vào tổng.
    loose_items += stock % box_size
# len() đếm phần tư; != phát hiện sô´lượng khác mức yêu câù.
if len(stocks) != 15:
    # raise ValueError(...) báo lôĩ rồi dừng xư lý khi dữ liệu không nhất quán.
    raise ValueError("Phải có đúng 15 mặt hàng tồn kho.")
# Đôí chiếú (sô´thùng × 6 + món lẻ) với sum(stocks), tổng sô´món.
if full_boxes * box_size + loose_items != sum(stocks):
    # raise ValueError(...) báo lôĩ rồi dừng xư lý khi dữ liệu không nhất quán.
    raise ValueError("Sô´thùng đầy và món lẻ phải khớp tổng tồn kho.")
print("Sô´SKU:", len(stocks))
print("Thùng đầy:", full_boxes)
print("Món rời:", loose_items)
```

Kết quả khi chạy:

```
Sô´SKU: 15
Thùng đầy: 45
Món rời: 38
```

Số dư được giữ **riêng cho từng mã hàng**. Điều kiện đối chiếu bằng `assert` giúp phát hiện nếu một bước tính đã làm mất hàng.

Tình huống 4 — Cấp 4: Đọc cờ bit của 18 thiết bị, phân biệt mất tín hiệu

Từ vựng trong mục: Bitwise AND — phép và theo bit; **Mask** — mặt nạ bit; **Missing** — thiếu dữ liệu.

Tình huống thực tế: Một phòng điều hành nhận **18 mã trạng thái** từ các thiết bị. Mỗi mã là một số mà thiết bị dùng để lưu nhiều tín hiệu bật/tắt trong cùng một giá trị: tín hiệu cảnh báo nằm tại vị trí `0b0010`, còn tín hiệu đang gửi dữ liệu tại `0b1000`. Thiết bị thứ 17 không gửi mã nên được ghi `None`. Người trực cần biết có bao nhiêu mã hợp lệ, bao nhiêu thiết bị phát cảnh báo, bao nhiêu thiết bị báo đang gửi dữ liệu và bao nhiêu lượt thiếu thông tin.

Vì sao chọn cách này? Bit có thể hình dung là một công tắc chỉ có hai trạng thái 0 (tắt) và 1 (bật). Một mã như `10` có thể chứa nhiều công tắc cùng lúc; `0b0010` và `0b1000` là cách viết **nhị phân** để chỉ đúng hai vị trí cần xem. Toán tử `&` giữ lại các bit đang bật ở **cả mã trạng thái và mặt nạ cần kiểm tra**; kết quả khác 0 tức là tín hiệu đó đang bật. Cần kiểm tra `None` trước, vì đây là **chưa nhận mã** chứ không phải thiết bị báo tất cả công tắc đều tắt. `and` không dùng để đọc từng bit, nó chỉ kết hợp các điều kiện đúng/sai.

Quy trình:

1. Đọc 18 giá trị cờ, tách những dòng chưa gửi.
2. Dùng `status & mask` để kiểm tra từng bit trên dữ liệu hợp lệ.
3. Đếm riêng cảnh báo, gửi tín hiệu và trạng thái thiếu.

No Note — Không chú thích

```
flags = [10,2,0,8,3,11,1,10,14,2,0,9,10,12,3,11,None,2]
valid = []
warning_count = 0
sending_count = 0
for status in flags:
    if status is None:
        continue
    valid.append(status)
    if (status & 0b0010) != 0:
        warning_count += 1
    if (status & 0b1000) != 0:
        sending_count += 1
if len(flags) != 18:
    raise ValueError("Phải nhận đủ 18 trạng thái thiết bị.")
print("Trạng thái hợp lệ:", len(valid))
print("Cảnh báo:", warning_count)
print("Đang gửi:", sending_count)
print("Thiếu:", len(flags) - len(valid))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Giải mã cờ nhị phân của 18 trạng thái thiết bị.
# Dữ liệu: 18 trạng thái bitmask, None là một bản ghi thiếu.
flags = [10,2,0,8,3,11,1,10,14,2,0,9,10,12,3,11,None,2]
valid = []
warning_count = 0
sending_count = 0
for status in flags:
    # is None nhận diện bản ghi thiếu; số 0 là cờ hợp lệ.
    if status is None:
        # continue bỏ lượt thiếu để không xử lý bitmask None.
        continue
    # append() lưu trạng thái đã nhận.
    valid.append(status)
    # 0b0010 là mặt nạ bit cảnh báo (2); & giữ bit chung, khác 0 nghĩa bật.
    if (status & 0b0010) != 0:
        warning_count += 1
    # 0b1000 là mặt nạ bit đang gửi (8); có thể bật đồng thời hai cờ.
    if (status & 0b1000) != 0:
        sending_count += 1
# len() đếm phần tử; != phát hiện số lượng khác mức yêu cầu.
if len(flags) != 18:
    # raise ValueError(...) báo lỗi rồi dừng xử lý khi dữ liệu không nhất quán.
    raise ValueError("Phải nhận đủ 18 trạng thái thiết bị.")
print("Trạng thái hợp lệ:", len(valid))
print("Cảnh báo:", warning_count)
print("Đang gửi:", sending_count)
# Tổng 18 bản ghi trừ số hợp lệ = số lượt thiếu.
print("Thiếu:", len(flags) - len(valid))
```

Kết quả khi chạy:

```
Trạng thái hợp lệ: 17
Cảnh báo: 11
Đang gửi: 9
Thiếu: 1
```

Một thiết bị có thể vừa cảnh báo vừa đang gửi, nên hai tập không loại trừ nhau. Trường hợp thiết bị mất tín hiệu cần được xử lý riêng, không gán trạng thái mặc định không có cảnh báo.

Tình huống 5 — Cấp 5: Xếp thứ tự xử lý 20 đơn hàng bằng nhiều quy tắc

Từ vựng trong mục: Logical OR — phép hoặc; **Logical AND** — phép và; **Boolean** — đúng/sai.

Tình huống thực tế: Đội giao hàng nhận **20 đơn O01 đến O20**. Mỗi đơn ghi số tiền khách mua (**đồng**), số ngày đã giao trễ (**ngày**) và tình trạng hàng còn trong kho (`True` nghĩa là còn, `False` là hết). Cửa hàng quy định chỉ ưu tiên đơn **còn hàng** và thỏa **ít nhất một** trong hai điều kiện: giá trị từ 2.000.000 đồng hoặc đã trễ từ 3 ngày. Điều phối viên cần thống kê số đơn được ưu tiên, liệt kê mã của chúng và cộng tổng giá trị các đơn đó.

Vì sao chọn cách này? Trước tiên kiểm **còn hàng** vì đơn hết hàng chưa thể được ưu tiên xử lý, dù giá trị cao hay đã trễ. Sau đó xét hai lý do ưu tiên: đơn có số tiền đạt ngưỡng **hoặc** số ngày trễ đạt ngưỡng; `or` chỉ cần một lý do đúng. Cuối cùng dùng `and` kết hợp kết quả ấy với trạng thái còn hàng, nên đơn phải vừa **đủ lý do ưu tiên vừa còn hàng**. Dấu ngoặc trong `(amount >= 2000000 or delay >= 3) and available` giúp nhìn rõ quy định, không bị nhầm với việc ưu tiên mọi đơn giá cao dù hết hàng. Vòng `for` lưu mã và số tiền những đơn đạt yêu cầu để vừa đếm vừa cộng tổng, thay vì cộng cả 20 đơn.

Quy trình:

1. Ghép dữ liệu theo đúng mã đơn và xác nhận ba cột đồng hàng.
2. Tính `(giá trị >= ngưỡng or ngày trễ >= ngưỡng) and có hàng`.
3. Lọc 20 đơn, tính tổng tiền thuộc nhóm ưu tiên và kiểm tra mọi đơn chọn được đều còn hàng.

No Note — Không chú thích

```
amount_vnd =
[500000,900000,1300000,1700000,2100000,2500000,500000,900000,1300000,1700000,2100000,2500000,500000
,900000,1300000,1700000,2100000,2500000,500000,900000]
late_days = [0,1,2,3,4,0,1,2,3,4,0,1,2,3,4,0,1,2,3,4]
in_stock =
[True,True,True,False,True,True,True,True,True,True,True,False,True,True,True,True,True,False,True,
True]
orders = []
for i in range(1, 21):
    order_id = f"O{i:02d}"
    orders.append((order_id, amount_vnd[i - 1], late_days[i - 1], in_stock[i - 1]))
priority = []
priority_ids = []
for code, amount, delay, available in orders:
    if (amount >= 2000000 or delay >= 3) and available:
        priority.append((code, amount))
        priority_ids.append(code)
if len(orders) != 20:
    raise ValueError("Phải có đủ 20 đơn để chọn ưu tiên.")
for code in priority_ids:
    found_available = False
    for order_code, amount, delay, available in orders:
        if order_code == code and available:
            found_available = True
```

```
            break
    if not found_available:
        raise ValueError("Đơn ưu tiên phải tồn tại và còn hàng.")
print("Số đơn:", len(orders))
print("Ưu tiên:", len(priority))
print("Mã ưu tiên:", ", ".join(priority_ids))
priority_value = 0
for code, amount in priority:
    priority_value += amount
print("Giá trị ưu tiên (đồng):", priority_value)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn, kiểm chứng và cộng giá trị đơn ưu tiên trong 20 đơn.
# Dữ liệu: 20 đơn: số tiền VND, số ngày trễ và cờ tồn kho nằm ở 3 list cùng thứ tự.
amount_vnd =
[500000,900000,1300000,1700000,2100000,2500000,500000,900000,1300000,1700000,2100000,2500000,500000
,900000,1300000,1700000,2100000,2500000,500000,900000]
late_days = [0,1,2,3,4,0,1,2,3,4,0,1,2,3,4,0,1,2,3,4]
in_stock =
[True,True,True,False,True,True,True,True,True,True,True,False,True,True,True,True,True,False,True,
True]
orders = []
# range(1,21) tạo 1..20, cận cuối 21 không lấy.
for i in range(1, 21):
    # f-string :02d định dạng O01..O20.
    order_id = f"O{i:02d}"
    # append() thêm (mã, tiền đồng, ngày trễ, còn hàng); [i-1] chuyển thứ tự sang chỉ số list.
    orders.append((order_id, amount_vnd[i - 1], late_days[i - 1], in_stock[i - 1]))
priority = []
priority_ids = []
# Tách bốn thuộc tính thuộc cùng đơn.
for code, amount, delay, available in orders:
    # or là giá >= 2 triệu HOẶC trễ >= 3 ngày; ngoặc nhóm hai điều kiện, and bắt buộc còn hàng.
    if (amount >= 2000000 or delay >= 3) and available:
        # append() lưu (mã, tiền) được chọn.
        priority.append((code, amount))
        # append() lưu mã để in và kiểm tra lại.
        priority_ids.append(code)
# len() đếm phần tử; != phát hiện số lượng khác mức yêu cầu.
if len(orders) != 20:
    # raise ValueError(...) báo lỗi rồi dừng xử lý khi dữ liệu không nhất quán.
    raise ValueError("Phải có đủ 20 đơn để chọn ưu tiên.")
# Duyệt mã ưu tiên để đối chiếu với đơn nguồn.
for code in priority_ids:
    # Cờ False cho biết chưa tìm thấy đơn còn hàng.
    found_available = False
    # Vòng lặp lồng nhau kiểm tra từng đơn nguồn.
    for order_code, amount, delay, available in orders:
        # == so sánh mã; and kiểm tra đơn còn hàng.
        if order_code == code and available:
            # Cờ False cho biết chưa tìm thấy đơn còn hàng.
            found_available = True
            # break dừng tìm ngay khi gặp đơn hợp lệ.
            break
    # not đảo cờ để phát hiện không tìm thấy đơn phù hợp.
    if not found_available:
        # raise ValueError(...) báo lỗi rồi dừng xử lý khi dữ liệu không nhất quán.
        raise ValueError("Đơn ưu tiên phải tồn tại và còn hàng.")
print("Số đơn:", len(orders))
print("Ưu tiên:", len(priority))
print("Mã ưu tiên:", ", ".join(priority_ids))
priority_value = 0
for code, amount in priority:
    # += cộng tiền (đồng) các đơn ưu tiên.
```

```
    priority_value += amount
print("Giá trị ưu tiên (đồng):", priority_value)
```

Kết quả khi chạy:

```
Số đơn: 20
Ưu tiên: 10
Mã ưu tiên: O05, O06, O09, O10, O11, O14, O15, O17, O19, O20
Giá trị ưu tiên (đồng): 15400000
```

Ba cột là thuộc tính của **một đơn**, không phải ba danh sách độc lập để so riêng. Với kho hết hàng, điều kiện `and available` ngăn hệ thống ưu tiên những đơn chưa thể xử lý.

6. Những nhầm lẫn thường gặp

- `=` dùng để gán giá trị, còn `==` dùng để so sánh bằng nhau.
- `and` và `or` kiểm tra điều kiện; `&` và `|` thao tác trên các bit, không thay thế tùy tiện cho nhau.
- `/` cho kết quả chia thông thường, `//` lấy phần nguyên theo hướng làm tròn xuống, `%` lấy số dư.
- Với giá tiền cần tính chính xác đến từng đồng, nên cân nhắc dùng số nguyên đồng hoặc `Decimal` thay vì dựa vào số thực cho phép tính tiền phức tạp.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. Which operator performs multiplication? | Toán tử nào thực hiện phép nhân?

A. `*` | *

B. `&` | &

C. `==` | ==

D. `and` | and

Đáp án: A

Giải thích: `*` nhân các số, ví dụ `3 * 4` cho 12. `==` so sánh giá trị, `and` kết hợp điều kiện logic, còn `&` thực hiện AND theo bit trên số nguyên.

Q2. Which operator tests equality? | Toán tử nào kiểm tra bằng nhau?

A. `*` | *

B. `==` | ==

C. `&` | &

D. `and` | and

Đáp án: B

Giải thích: `==` kiểm tra hai giá trị có bằng nhau và tạo Boolean, ví dụ `2 == 2` là `True`. Một dấu `=` dùng để gán giá trị cho tên, chứ không so sánh.

Q3. Which Python keyword performs short-circuit logical AND? | Từ khóa Python nào thực hiện phép VÀ logic có đánh giá ngắn mạch?

A. == | ==

B. & | &

C. and | and

D. * | *

Đáp án: C

Giải thích: Từ khóa `and` thực hiện phép VÀ logic ngắn mạch: nếu vế trái sai, Python không đánh giá vế phải. Ký hiệu `&` thực hiện AND theo bit trên số nguyên và kể cả dùng với Boolean cũng không đánh giá ngắn mạch.

Q4. Which operator performs bitwise AND on integers? | Toán tử nào thực hiện AND theo bit trên số nguyên?

A. and | and

B. == | ==

C. * | *

D. & | &

Đáp án: D

Giải thích: Với số nguyên, `&` thực hiện AND trên từng bit: `5 & 3` đối chiếu `0101` và `0011`, được `0001` (1). Đây không phải phép nhân thông thường hay từ khóa logic `and`.

Q5. What does `7 // 2` return? | `7 // 2` trả về gì?

A. `1` | `1`

B. `3.5` | `3.5`

C. `3` | `3`

D. `8` | `8`

Đáp án: C

Giải thích: `//` thực hiện chia lấy sàn: `7 / 2` bằng `3.5`, làm tròn xuống được `3`. Với thương âm, làm tròn xuống nghĩa là về âm vô cực nên `-7 // 2` bằng `-4`.

Q6. What does `7 % 2` return? | `7 % 2` trả về gì?

A. `1` | `1`

B. `3.5` | `3.5`

C. `3` | `3`

D. `8` | `8`

Đáp án: A

Giải thích: `%` trả phần dư theo phép chia lấy sàn: `7 = 2 * 3 + 1`, nên `7 % 2` bằng `1`. Nó không phải thương `3` hay kết quả chia thực `3.5`.

Q7. What does `2 ** 3` return? | `2 ** 3` trả về gì?

A. `3` | `3`

B. `1` | `1`

C. `3.5` | `3.5`

D. `8` | `8`

Đáp án: D

Giải thích: `**` là phép lũy thừa: `2 ** 3` nhân 2 với chính nó ba lần, cho `8`. Một dấu `*` là phép nhân, còn `^` trong Python là XOR theo bit, không phải lũy thừa.

Q8. What does `7 / 2` return? | `7 / 2` trả về gì?

A. `3.5` | `3.5`

B. `3` | `3`

C. `1` | `1`

D. `8` | `8`

Đáp án: A

Giải thích: Trong Python 3, `/` chia thực nên `7 / 2` trả về số thực `3.5`. Toán tử `//` lấy sàn thương được `3`, còn `%` trả phần dư `1`.

Q9. What is `True and False`? | `True and False` bằng gì?

A. `True` (Boolean) | `True` (Boolean)

B. `None` | `None`

C. `0` (integer) | `0` (số nguyên)

D. `False` (Boolean) | `False` (Boolean)

Đáp án: D

Giải thích: `and` Boolean yêu cầu cả hai vế đều đúng, nhưng vế thứ hai là `False`. Vì thế `True and False` cho Boolean `False`, không phải số nguyên `0` hay `None`.

Q10. What is `False or True`? | `False or True` bằng gì?

A. `1` (integer) | `1` (số nguyên)

B. `True` (Boolean) | `True` (Boolean)

C. `False` (Boolean) | `False` (Boolean)

D. `None` | `None`

Đáp án: B

Giải thích: `or` Boolean đúng nếu có ít nhất một vế đúng; vế phải của `False or True` là `True`. Kết quả ở đây là Boolean `True`, không phải số nguyên `1`, dù `True` có thể hoạt động như 1 trong một số phép số học.

Q11. What does `not True` produce? | `not True` cho kết quả gì?

A. `0` (integer) | `0` (số nguyên)

B. `None` | `None`

C. `False` (Boolean) | `False` (Boolean)

D. `True` (Boolean) | `True` (Boolean)

Đáp án: C

Giải thích: `not` đảo giá trị logic nên `not True` thành Boolean `False`. Ngược lại `not False` là `True`; số nguyên không là giá trị thuộc kiểu khác.

Q12. What does `5 & 3` produce? | `5 & 3` tạo ra gì?

A. `False` (Boolean) | `False` (Boolean)

B. `5` (integer) | `5` (số nguyên)

C. `True` (Boolean) | `True` (Boolean)

D. `1` (integer) | `1` (số nguyên)

Đáp án: D

Giải thích: Hai số nguyên là `5 = 0101` và `3 = 0011` ở hệ nhị phân bốn bit. Toán tử `&` chỉ giữ vị trí có cả hai bit bằng 1, cho `0001`, tức số nguyên `1`, không phải Boolean `True`.

Q13. Which operator tests that two values differ? | Toán tử nào kiểm tra hai giá trị khác nhau?

A. `!=` | `!=`

B. `>=` | `>=`

C. `=` | `=`

D. `in` | `in`

Đáp án: A

Giải thích: `!=` kiểm tra hai giá trị có khác nhau: `3 != 4` là `True`, còn `3 != 3` là `False`. Nó đối lập với `==` dùng để kiểm tra bằng nhau.

Q14. Which operator means 'at least'? | Toán tử nào biểu diễn 'lớn hơn hoặc bằng'?

A. `!=` | `!=`

B. `in` | `in`

C. `>=` | `>=`

D. `=` | `=`

Đáp án: C

Giải thích: >= nghĩa là lớn hơn hoặc bằng, nên `5 >= 5` và `6 >= 5` đều là `True`. Trường hợp bằng ngưỡng rất quan trọng: dùng > sẽ loại đúng giá trị 5.

Q15. Which operator checks membership in a collection? | Toán tử nào kiểm tra phần tử thuộc một tập dữ liệu?

A. `!=` | `!=`

B. `in` | `in`

C. >= | >=

D. = | =

Đáp án: B

Giải thích: Toán tử `in` kiểm tra phần tử có thuộc tập hợp: `2 in [1, 2, 3]` là `True`, còn `4 in [1, 2, 3]` là `False`. Nó không gán giá trị hay so sánh độ lớn.

Q16. Which operator assigns a new value to a variable? | Toán tử nào gán giá trị mới cho biến?

A. >= | >=

B. `!=` | `!=`

C. = | =

D. `in` | `in`

Đáp án: C

Giải thích: Toán tử gán = liên kết tên với giá trị, như `total = 10`. Muốn so sánh hai giá trị thì dùng ==; dùng = trong điều kiện so sánh thông thường sẽ sai cú pháp.

Q17. Which result comes from `3 > 5`? | `3 > 5` trả về gì?

A. `0` (integer) | `0` (số nguyên)

B. `False` (Boolean) | `False` (Boolean)

C. `True` (Boolean) | `True` (Boolean)

D. `None` | `None`

Đáp án: B

Giải thích: > kiểm tra lớn hơn nghiêm ngặt, mà 3 nhỏ hơn 5 nên `3 > 5` cho Boolean `False`. Kết quả là giá trị logic, không phải số nguyên `0`.

Q18. What result comes from `2 + 3 * 4`? | `2 + 3 * 4` bằng bao nhiêu?

A. `9` | `9`

B. `14` | `14`

C. `False` | `False`

D. `20` | `20`

Đáp án: B

Giải thích: Phép nhân có độ ưu tiên cao hơn cộng: tính `3 * 4 = 12` trước, rồi `2 + 12 = 14`. Thêm ngoặc như `(2 + 3) * 4` sẽ đổi kết quả thành `20`.

Q19. What does `True or False` return? | `True or False` trả về gì?

A. `True` (Boolean) | `True` (Boolean)

B. `1` (integer) | `1` (số nguyên)

C. `False` (Boolean) | `False` (Boolean)

D. `None` | `None`

Đáp án: A

Giải thích: `or` đúng khi ít nhất một toán hạng Boolean đúng; vì vế trái là `True` nên `True or False` trả `True`. Với đầu vào Boolean, đây là Boolean `True`, không phải số nguyên `1`.

Q20. What does `5 ^ 3` return for integers? | `5 ^ 3` trả về gì với số nguyên?

A. `5` | `5`

B. `2` | `2`

C. `15` | `15`

D. `6` | `6`

Đáp án: D

Giải thích: ^ là XOR theo bit, không phải lũy thừa: `5 = 0101` và `3 = 0011` khác nhau tại các vị trí bit `0100` và `0010`, cho `0110` = 6. Lũy thừa phải dùng **, không dùng ^.

8. Nối sang bài kế tiếp

`Kiểu dữ liệu → Toán tử → Điều kiện → Quy tắc nghiệp vụ.`

PY-04 — Câu lệnh điều kiện if, elif, else

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Condition — điều kiện; **Branch** — nhánh.

`Toán tử → Điều kiện → Phân loại theo quy tắc`

Học xong dùng để làm gì? Tạo quy tắc quyết định như phân loại điểm, xác định mức ưu đãi và xử lý trường hợp không đạt điều kiện.

Kết nối kiến thức tiếp theo: Sang PY-05, lặp lại quy tắc này cho nhiều bản ghi thay vì viết từng câu điều kiện thủ công.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Condition	điều kiện	Phép kiểm tra đúng hoặc sai
Branch	nhánh	Phần chương trình được chọn để chạy
Threshold	ngưỡng	Ranh giới để chuyển trạng thái
Nested if	điều kiện lồng	Kiểm tra bổ sung bên trong một nhánh

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`if expression:`	Chỉ chạy nhánh nếu điều kiện đúng
`elif expression:`	Thử điều kiện tiếp theo khi nhánh trước không đúng
`else:`	Trường hợp còn lại không cần điều kiện mới
`in, and, or`	Kiểm tra thành viên hoặc kết hợp nhiều điều kiện

4. Khái niệm liên quan

Rẽ nhánh giúp chương trình chọn hành động theo dữ liệu thay vì lúc nào cũng chạy mọi lệnh. Python xét điều kiện `if` trước; nếu sai mới xét các `elif` theo thứ tự, và dùng `else` khi không nhánh nào đúng. Nhờ vậy một giá trị đầu vào được xếp đúng một trạng thái trong cùng chuỗi nhánh.

4.1. If / elif / else — Cấu trúc rẽ nhánh

Từ vựng trong mục: Condition — điều kiện; **If / elif / else** — nếu / nếu khác / còn lại.

Giả sử một cảm biến có nhiệt độ 30 °C: từ 35 °C trở lên gọi là `Hot`, từ 20 đến dưới 35 °C gọi là `Warm`, còn thấp hơn gọi là `Cold`. Chương trình kiểm tra điều kiện từ trên xuống: `if` xét ngưỡng cao, `elif` xét ngưỡng tiếp theo khi nhánh trước sai, còn `else` nhận các trường hợp còn lại. Chỉ một nhánh trong chuỗi này được chạy.

No Note — Không chú thích

```
temperature = 30

if temperature >= 35:
    state = 'Hot'
elif temperature >= 20:
    state = 'Warm'
else:
    state = 'Cold'

print(state)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân loại nhiệt độ thành Hot, Warm hoặc Cold qua if/elif/else.
temperature = 30

# if được thử trước: từ 35 °C trở lên là Hot.
if temperature >= 35:
    state = 'Hot'
# elif chỉ thử khi if trước sai: 20..34 °C thuộc Warm.
elif temperature >= 20:
    state = 'Warm'
# else nhận mọi giá trị còn lại dưới 20 °C (Cold).
else:
    state = 'Cold'

# print() in đúng nhãn của nhánh đã được chọn.
print(state)
```

Kết quả khi chạy:

```
Warm
```

Nhiệt độ 30 °C không đạt mức `Hot` nhưng đạt mức `Warm`, nên nhánh `elif` được chọn.

4.2. Branch — Nhánh lệnh

Từ vựng trong mục: Branch — nhánh; **Boolean** — đúng/sai.

Một nhánh là nhóm câu lệnh được thụt vào dưới `if` hoặc `else`; nó chỉ chạy khi chương trình chọn nhánh ấy. Ví dụ kho có `0` sản phẩm: nếu số lượng dương thì in `Còn hàng`, ngược lại in `Hết hàng`. Dấu `:` mở đầu nhánh; bốn dấu cách cho biết câu lệnh nào thuộc nhánh.

No Note — Không chú thích

```
stock = 0

if stock > 0:
    print('Còn hàng')
```

```
else:
    print('Hết hàng')
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thông báo còn hàng hay hết hàng dựa vào số tồn.
stock = 0

# if kiểm tra tồn kho dương để xác nhận còn hàng.
if stock > 0:
    print('Còn hàng')
# else thực hiện khi stock không lớn hơn 0, ở đây stock=0.
else:
    print('Hết hàng')
```

Kết quả khi chạy:

```
Hết hàng
```

Vì `stock > 0` sai, chương trình bỏ nhánh `if` và chạy nhánh `else`.

4.3. Threshold — Ngưỡng

Từ vựng trong mục: Threshold — ngưỡng; **Comparison** — phép so sánh.

Ngưỡng là ranh giới dùng để chia dữ liệu thành các nhóm. Chẳng hạn trung tâm yêu cầu từ 80 điểm trở lên để đạt mức B. Với điểm 85, `85 >= 80` là đúng. Dấu >= có nghĩa **lớn hơn hoặc bằng**, nên đúng cả khi điểm bằng chính mốc 80; điều này quan trọng khi phân loại các giá trị ở ranh giới.

No Note — Không chú thích

```
print(85 >= 80)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tra điểm số có chạm ngưỡng đạt 80 hay không.
# >= trả True nếu 85 lớn hơn hoặc bằng 80; print in kết quả so sánh.
print(85 >= 80)
```

Kết quả khi chạy:

```
True
```

Điểm 85 đạt ngưỡng 80. Nếu dùng > thì người đạt đúng 80 sẽ bị loại ngoài ý muốn.

4.4. Nested if — Điều kiện lồng nhau

Từ vựng trong mục: Nested if — điều kiện lồng; **Modulo** — chia lấy dư.

Đôi khi chỉ nên kiểm tra điều kiện thứ hai khi điều kiện đầu tiên đúng. Ví dụ chỉ phân loại số chẵn sau khi xác định số đó dương. `x > 0` mở nhánh ngoài; bên trong, `x % 2 == 0` hỏi phép chia cho 2 có dư 0 không. Với `x = 2`, cả hai điều kiện đúng nên in `even`.

No Note — Không chú thích

```
x = 2

if x > 0:
    if x % 2 == 0:
        print('even')
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tra sô´vừa dương vừa chãñ bằng if lôňg nhau.
x = 2

# Nhánh ngoài yêu câù x là sô´dương, nêú sai sẽ không kiểm tra chãñ.
if x > 0:
    # % lấý phần dư chia 2; == 0 nghĩa sô´chãñ. Nhánh trong chỉ chạy khi sô´dương.
    if x % 2 == 0:
        print('even')
```

Kết quả khi chạy:

```
even
```

Số 2 vừa dương vừa chia hết cho 2, nên chương trình đi vào cả hai nhánh `if`.

5. Năm tình huống — quy tắc rẽ nhánh tăng dần

Tình huống 1 — Cấp 1: Xác định đạt/chưa đạt của 10 học viên

Từ vựng trong mục: If — nếu; **Else** — nếu không; **Threshold** — ngưỡng.

Tình huống thực tế: Giáo vụ đang chấm **10 bài kiểm tra**, mỗi bài có một điểm trên thang **0 đến 100**; điểm theo thứ tự gồm cả một bài đúng 50 điểm. Nhà trường quy định **từ 50 điểm là đạt**. Người phụ trách cần đếm bao nhiêu học viên đạt, bao nhiêu người chưa đạt, đồng thời xác nhận kết quả của học viên H02, người được 50 điểm.

Vì sao chọn cách này? `if` nghĩa là **nếu** điều kiện đúng thì thực hiện một nhóm lệnh; `else` nghĩa là **nếu không** thì đi sang nhóm lệnh còn lại. Vì trường chỉ có hai kết quả đạt/chưa đạt ở một ngưỡng, không cần `elif` (nhánh bổ sung). Chương trình đọc từng điểm bằng `for`, thử `score >= 50`, rồi tăng một trong hai bộ đếm; `>=` tính cả người đúng 50. Sau cùng, kiểm số đạt cộng số chưa đạt phải đúng 10 để bảo đảm không bỏ quên bài.

Quy trình:

1. Lưu mười điểm theo thứ tự học viên H01…H10.
2. Với từng điểm, chọn `Đạt` nếu từ 50 trở lên, còn lại `Chưa đạt`.
3. Cộng số lượng từng nhánh và in trạng thái học viên H02 ở đúng biên.

No Note — Không chú thích

```
points = [48,50,72,30,85,90,49,61,44,100]
passed = 0
failed = 0
for score in points:
```

```
    if score >= 50:
        passed += 1
    else:
        failed += 1
if len(points) != 10:
    raise ValueError("Phải có đúng 10 điểm học viên.")
if passed + failed != 10:
    raise ValueError("Tổng học viên đạt và chưa đạt phải bằng 10.")
print("Sô´học viên:", len(points))
print("Đạt:", passed)
print("Chưa đạt:", failed)
if points[1] >= 50:
    h02_result = "Đạt"
else:
    h02_result = "Chưa đạt"
print("H02:", h02_result)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chia điểm của 10 học viên thành đạt/chưa đạt, kiểm riêng H02.
# Dữ liệu: 10 điểm trên thang 100, môĩ phần tư ̉là một học viên theo thứ tự.
points = [48,50,72,30,85,90,49,61,44,100]
passed = 0
failed = 0
# Môĩ vòng for duyệt điểm của một học viên.
for score in points:
    # Điểm >=50 được xem là đạt, kể ̉cả đúng bằng 50.
    if score >= 50:
        # += 1 tăng bộ đếm học viên đạt.
        passed += 1
    else:
        # += 1 tăng bộ đếm chưa đạt.
        failed += 1
# len() đếm sô´bản ghi; != và or kiểm tra tổng quan sát đúng kỳ vọng.
if len(points) != 10:
    # raise ValueError(...) báo lôĩ và dừng nêú kiểm tra dữ liệu không đạt.
    raise ValueError("Phải có đúng 10 điểm học viên.")
# Đôí chiếú sô´đạt và chưa đạt với tổng 10 học viên.
if passed + failed != 10:
    # raise ValueError(...) báo lôĩ và dừng nêú kiểm tra dữ liệu không đạt.
    raise ValueError("Tổng học viên đạt và chưa đạt phải bằng 10.")
print("Sô´học viên:", len(points))
print("Đạt:", passed)
print("Chưa đạt:", failed)
# [1] là học viên thứ hai H02, vì list đánh sô´từ 0.
if points[1] >= 50:
    h02_result = "Đạt"
else:
    h02_result = "Chưa đạt"
# In nhãn đạt/chưa đạt được đặt bởi nhánh if/else.
print("H02:", h02_result)
```

Kết quả khi chạy:

```
Sô´học viên: 10
Đạt: 6
Chưa đạt: 4
H02: Đạt
```

Quy tắc dùng >= nên giá trị 50 nằm trong nhóm đạt. Không có trường hợp vừa đạt vừa chưa đạt nếu hai nhánh được đặt chung trong một `if/else`.

Tình huống 2 — Cấp 2: Phân ba mức tồn kho cho 12 SKU

Từ vựng trong mục: Elif — điều kiện khác; **Branch** — nhánh; **Stock** — tồn kho.

Tình huống thực tế: Nhân viên kho cần kiểm tra **12 loại sản phẩm**. Mỗi loại có một số lượng còn lại trong kho, tính bằng **món**; có loại còn 0 món, loại còn 5 món và loại còn nhiều hơn. Kho dùng ba tên trạng thái: `Out` = hết hàng, `Low` = còn từ 1 đến 5 món, `OK` = còn từ 6 món trở lên. Nhân viên muốn biết có bao nhiêu loại nằm trong từng trạng thái.

Vì sao chọn cách này? Trước hết cần hiểu **trạng thái** là tên tóm tắt tình hình tồn kho của một loại hàng. Chương trình xét `quantity == 0` trước để đưa vào `Out`. Nếu không phải 0 thì xét `quantity <= 5` và đưa vào `Low`; phần còn lại tự động là `OK`. `elif` chỉ được thử khi nhánh phía trước không đúng, còn `else` nhận các trường hợp chưa rơi vào nhánh nào, vì vậy mỗi sản phẩm chỉ được đếm **một lần**. Nếu đặt điều kiện `<= 5` trước, lượng 0 cũng bị gộp nhầm vào nhóm gần hết.

Quy trình:

1. Duyệt danh sách 12 giá trị tồn.
2. Dùng chuỗi `if` → `elif` → `else` theo ngưỡng.
3. Kiểm tra số lượng ba trạng thái cộng lại bằng 12.

No Note — Không chú thích

```
stocks = [0,1,4,7,12,5,3,0,8,2,16,5]
count = {"Out": 0, "Low": 0, "OK": 0}
for quantity in stocks:
    if quantity == 0:
        state = "Out"
    elif quantity <= 5:
        state = "Low"
    else:
        state = "OK"
    count[state] += 1
if len(stocks) != 12:
    raise ValueError("Cần có đủ 12 mã sản phẩm.")
if sum(count.values()) != 12:
    raise ValueError("Phải phân loại đủ 12 mã vào ba mức tồn kho.")
print("Out:", count["Out"])
print("Low:", count["Low"])
print("OK:", count["OK"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân loại 12 tồn kho thành Out, Low hoặc OK theo mức số lượng.
# Dữ liệu: 12 số tồn theo mã sản phẩm, đơn vị món.
stocks = [0,1,4,7,12,5,3,0,8,2,16,5]
# dict lưu bộ đếm theo từng mức Out (hết), Low (ít), OK (đủ).
count = {"Out": 0, "Low": 0, "OK": 0}
for quantity in stocks:
    # == kiểm tra không còn món nào nên Out.
    if quantity == 0:
        state = "Out"
    # elif chỉ xét sau khi loại 0: tồn từ 1 đến 5 là Low.
    elif quantity <= 5:
        state = "Low"
    # Các mức còn lại lớn hơn 5 là OK.
    else:
        state = "OK"
    # Dùng state làm khóa dictionary rồi tăng bộ đếm của mức đó.
```

```
    count[state] += 1
# len() đếm số bản ghi; != và or kiểm tra tổng quan sát đúng kỳ vọng.
if len(stocks) != 12:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Cần có đủ 12 mã sản phẩm.")
# values() lấy các số đếm từ dictionary; sum() cộng lại để đối chiếu 12 sản phẩm.
if sum(count.values()) != 12:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Phải phân loại đủ 12 mã vào ba mức tồn kho.")
print("Out:", count["Out"])
print("Low:", count["Low"])
print("OK:", count["OK"])
```

Kết quả khi chạy:

```
Out: 2
Low: 6
OK: 4
```

Cấp này khó hơn vì có ba nhánh không chồng lấn. Học viên có thể đối chiếu `0`, `5` và `6` để thấy ngưỡng được dùng nhất quán.

Tình huống 3 — Cấp 3: Phân hạng 15 khách theo hai điều kiện

Từ vựng trong mục: Nested condition — điều kiện phối hợp; **Comparison** — so sánh; **Tier** — hạng.

Tình huống thực tế: Bộ phận chăm sóc có dữ liệu của **15 khách**. Mỗi người có số năm gắn bó (**năm**) và tổng tiền đã mua (**đồng**) để xét ba hạng ưu đãi: **Gold** khi từ 5 năm và từ 2.000.000 đồng; **Silver** khi từ 5 năm và từ 1.000.000 đến dưới 2.000.000 đồng; còn lại là **Standard**. Nhân viên cần thống kê số khách ở từng hạng, mỗi người chỉ được nằm trong một hạng.

Vì sao chọn cách này? Ba hạng không phải ba câu hỏi độc lập: mỗi khách phải nhận **một hạng cao nhất phù hợp**. Vì vậy chương trình thử Gold trước bằng hai điều kiện nối `and` (đều phải đúng). Nếu không đạt Gold mới thử Silver; người không đạt cả hai được gán Standard. Với khách 6 năm và 2.500.000 đồng, cả ngưỡng của Gold lẫn mức tối thiểu của Silver đều có thể đúng, nhưng chỉ Gold được chọn. Nếu dùng ba lệnh `if` riêng, một người có thể bị cộng vào nhiều hạng. Sau mỗi khách tăng đúng một bộ đếm và đối chiếu tổng 15.

Quy trình:

1. Ghép 15 cặp năm và số tiền đúng khách.
2. Xét Gold trước, sau đó Silver rồi Standard.
3. Thống kê đúng một hạng mỗi khách và kiểm tra tổng 15.

No Note — Không chú thích

```
years = [1,5,6,8,2,5,10,3,7,5,9,6,2,4,8]
spending =
[800000,1000000,900000,2000000,1800000,2500000,1100000,900000,500000,700000,3500000,1200000,1700000
,4000000,1500000]
counts = {"Gold": 0, "Silver": 0, "Standard": 0}
for year, amount in zip(years, spending):
    if year >= 5 and amount >= 2000000:
        category = "Gold"
    elif year >= 5 and amount >= 1000000:
        category = "Silver"
    else:
```

```
        category = "Standard"
    counts[category] += 1
if len(years) != 15 or len(spending) != 15:
    raise ValueError("Cần có đủ 15 cặp năm thành viên và chi tiêu.")
if sum(counts.values()) != 15:
    raise ValueError("Phải phân hạng đủ 15 khách hàng.")
print("Gold:", counts["Gold"])
print("Silver:", counts["Silver"])
print("Standard:", counts["Standard"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chia hạng 15 khách thành Gold, Silver và Standard bằng điều kiện ưu tiên.
# Dữ liệu: 15 năm thành viên và 15 khoản chi VND khớp theo vị trí.
years = [1,5,6,8,2,5,10,3,7,5,9,6,2,4,8]
spending =
[800000,1000000,900000,2000000,1800000,2500000,1100000,900000,500000,700000,3500000,1200000,1700000
,4000000,1500000]
counts = {"Gold": 0, "Silver": 0, "Standard": 0}
# zip() ghép từng năm và số´tiền ở cùng vị trí thành cặp của một khách.
for year, amount in zip(years, spending):
    # and yêu cầu cả hai: thành viên ít nhất 5 năm, chi >=2 triệu đồng, xếp Gold.
    if year >= 5 and amount >= 2000000:
        category = "Gold"
    # elif chỉ đến nếu không Gold; đủ 5 năm và chi >=1 triệu thì Silver.
    elif year >= 5 and amount >= 1000000:
        category = "Silver"
    # Khách không đạt hai mức ưu tiên trên thuộc Standard.
    else:
        category = "Standard"
    # Tra khóa hạng khách trong dict rồi cộng 1.
    counts[category] += 1
# len() đếm số´bản ghi; != và or kiểm tra tổng quan sát đúng kỳ vọng.
if len(years) != 15 or len(spending) != 15:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Cần có đủ 15 cặp năm thành viên và chi tiêu.")
# values() lấy số´khách từng hạng, sum() kiểm tổng bằng 15.
if sum(counts.values()) != 15:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Phải phân hạng đủ 15 khách hàng.")
print("Gold:", counts["Gold"])
print("Silver:", counts["Silver"])
print("Standard:", counts["Standard"])
```

Kết quả khi chạy:

```
Gold: 3
Silver: 4
Standard: 8
```

Không suy ra khách Gold từ tiền chi tiêu đơn lẻ: khách cũng phải đáp ứng số năm. Đây là bài phối hợp hai cột với thứ tự ưu tiên nhánh.

Tình huống 4 — Cấp 4: Cảnh báo nhiệt độ và mất kết nối của 18 cảm biến

Từ vựng trong mục: Validation — kiểm tra đầu vào; **Online** — kết nối; **Missing** — thiếu số đo.

Tình huống thực tế: Trạm giám sát theo dõi **18 cảm biến nhiệt độ**, mỗi cảm biến có hai thông tin: đã kết nối hay chưa và số nhiệt độ tính bằng **°C**. Một số máy có thể mất kết nối, hoặc vẫn kết nối nhưng chưa gửi được số (`None`). Quy định phân loại là: mất kết nối `Offline`; vẫn kết nối nhưng thiếu nhiệt độ `Missing`; vẫn

kết nối và nhiệt độ từ 80 °C `Hot`; các trường hợp còn lại `Normal`. Trực ca muốn biết số cảm biến của mỗi loại.

Vì sao chọn cách này? `None` là dấu hiệu **chưa có số đo**, không có nghĩa là 0 °C. Đồng thời một máy mất kết nối có thể vẫn lưu nhiệt độ của lần trước, nên phải xét `Offline` **trước tiên**. Nếu máy đang kết nối, xét tiếp có số đo hay không; chỉ khi nhiệt độ tồn tại mới được so với 80 °C. `if/elif/else` đảm bảo lần lượt một trong bốn trạng thái được chọn. So ngay `None >= 80` sẽ gây lỗi; coi `None` là số 0 sẽ giấu đi sự cố thiếu thông tin.

Quy trình:

1. Ghép đủ 18 cặp kết nối/nhiệt độ.
2. Phân nhánh theo độ ưu tiên: offline → thiếu → nóng → bình thường.
3. Đếm từng trạng thái, kiểm tra tổng bằng 18.

No Note — Không chú thích

```
temperatures = [26,35,80,81,79,20,None,85,95,32,80,70,90,None,18,40,79,88]
online =
[True,True,True,True,False,True,True,True,False,True,True,True,True,True,True,True,False,True]
counts = {"Offline": 0, "Missing": 0, "Hot": 0, "Normal": 0}
for active, temp in zip(online, temperatures):
    if not active:
        state = "Offline"
    elif temp is None:
        state = "Missing"
    elif temp >= 80:
        state = "Hot"
    else:
        state = "Normal"
    counts[state] += 1
if len(temperatures) != 18 or len(online) != 18:
    raise ValueError("Phải có đủ 18 nhiệt độ và trạng thái kết nôí.")
if sum(counts.values()) != 18:
    raise ValueError("Phải phân loại đủ 18 cảm biến.")
for state in ("Offline", "Missing", "Hot", "Normal"):
    print(state + ":", counts[state])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân loại 18 cảm biến theo trạng thái kết nôí, thiếú dữ liệu và nhiệt độ.
# Dữ liệu: 18 nhiệt độ °C (None là thiếú) và 18 cờ online khớp theo vị trí.
temperatures = [26,35,80,81,79,20,None,85,95,32,80,70,90,None,18,40,79,88]
online =
[True,True,True,True,False,True,True,True,False,True,True,True,True,True,True,True,False,True]
counts = {"Offline": 0, "Missing": 0, "Hot": 0, "Normal": 0}
# zip() ghép trạng thái online và nhiệt độ cùng vị trí của một cảm biến.
for active, temp in zip(online, temperatures):
    # not đảo cờ: nêú mất kết nôí thì Offline, ưu tiên trước kiểm tra dữ liệu thiếú.
    if not active:
        state = "Offline"
    # is None phát hiện nhiệt độ thiếú, chỉ xét khi cảm biến online.
    elif temp is None:
        state = "Missing"
    # Nhiệt độ >=80 °C là Hot, chỉ khi có dữ liệu và cảm biến online.
    elif temp >= 80:
        state = "Hot"
    # Nhánh còn lại có trạng thái Normal.
    else:
        state = "Normal"
    # Đếm thêm một cảm biến ở hạng đã chọn trong dictionary.
```

```
    counts[state] += 1
# len() đếm số bản ghi; != và or kiểm tra tổng quan sát đúng kỳ vọng.
if len(temperatures) != 18 or len(online) != 18:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Phải có đủ 18 nhiệt độ và trạng thái kết nối.")
# values() lấy các bộ đếm, sum() phải đủ 18 cảm biến.
if sum(counts.values()) != 18:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Phải phân loại đủ 18 cảm biến.")
# Duyệt các hạng theo thứ tự cố định để in bảng tóm tắt.
for state in ("Offline", "Missing", "Hot", "Normal"):
    print(state + ":", counts[state])
```

Kết quả khi chạy:

```
Offline: 3
Missing: 2
Hot: 6
Normal: 7
```

Những trạng thái này loại trừ nhau theo thứ tự: số đo 80 °C được báo Hot nếu thiết bị online; một thiết bị offline với số đo 95 °C **không bị đọc nhầm là hiện đang Hot**.

Tình huống 5 — Cấp 5: Ra quyết định giao 20 đơn: kiểm dữ liệu, tồn kho và phí

Từ vựng trong mục: Decision tree — cây quyết định; **Validation** — kiểm tra hợp lệ; **Priority** — thứ tự xét.

Tình huống thực tế: Cửa hàng có **20 đơn giao hàng O01 đến O20**. Mỗi đơn có số tiền (**đồng**), quãng đường giao (**km**) và thông tin còn hàng (`True`/`False`). Một số đơn bị thiếu tiền, thiếu km hoặc có tiền âm. Quy định: đơn thiếu/sai dữ liệu là `INVALID` (không hợp lệ); đơn hợp lệ nhưng hết hàng là `HOLD` (tạm giữ); đơn còn hàng trị giá từ 1.000.000 đồng là `FREE` (miễn phí giao); những đơn còn lại giao đến 5 km thu 15.000 đồng (`LOCAL`), xa hơn thu 30.000 đồng (`FAR`). Quản lý cần số đơn mỗi nhóm, tổng tiền phí giao thực thu và mã những đơn chưa thể giao.

Vì sao chọn cách này? Đây là **quy trình ưu tiên kiểm tra**, không phải năm nhóm tùy chọn ngang nhau. Trước hết phát hiện thiếu dữ liệu (`None`), tiền âm hoặc km âm để đưa vào `INVALID`; không thể quyết định phí khi chưa biết giá trị đúng. Với đơn hợp lệ, kiểm tra có hàng không; hết hàng thì `HOLD`. Chỉ đơn còn hàng mới xét miễn phí nếu đủ 1.000.000 đồng; nếu không, dùng mốc 5 km để chọn `LOCAL` hay `FAR`. Cách đi từ dữ liệu hợp lệ → có hàng → được miễn phí → khoảng cách giúp tránh tính phí sai hoặc giao đơn không thể thực hiện. Chỉ cộng 15.000 hay 30.000 đồng khi đơn thực sự thuộc nhóm thu phí; `INVALID`, `HOLD`, `FREE` có phí 0 trong báo cáo. Cuối cùng kiểm tổng các nhóm đúng 20.

Quy trình:

1. Đọc 20 đơn và kiểm tra giá trị/ khoảng cách trước khi so ngưỡng.
2. Dùng chuỗi `if`/`elif`/`else` theo đúng ưu tiên; đồng thời lưu phí cho từng đơn.
3. Đối chiếu tổng năm trạng thái bằng 20 và báo mã `HOLD`/`INVALID` cần xử lý.

No Note — Không chú thích

```
amount_vnd =
[350000,530000,710000,890000,None,1250000,350000,530000,710000,890000,1070000,1250000,-5000,530000,
710000,890000,1070000,1250000,350000,530000]
distance_km = [1,2,3,4,5,6,7,None,1,2,3,4,5,6,7,8,1,2,3,4]
in_stock =
```

```
[True,True,False,True,True,True,True,True,True,False,True,True,True,True,True,True,True,False,True,
True]
counts = {"INVALID": 0, "HOLD": 0, "FREE": 0, "LOCAL": 0, "FAR": 0}
blocked_ids = []
total_fee_vnd = 0
for idx, (amount, km, available) in enumerate(zip(amount_vnd, distance_km, in_stock), 1):
    if amount is None or km is None or amount < 0 or km < 0:
        state = "INVALID"
        fee = 0
    elif not available:
        state = "HOLD"
        fee = 0
    elif amount >= 1000000:
        state = "FREE"
        fee = 0
    elif km <= 5:
        state = "LOCAL"
        fee = 15000
    else:
        state = "FAR"
        fee = 30000
    counts[state] += 1
    total_fee_vnd += fee
    if state in ("INVALID", "HOLD"):
        blocked_ids.append(f"O{idx:02d}")
if len(amount_vnd) != 20 or len(distance_km) != 20 or len(in_stock) != 20:
    raise ValueError("Phải có đúng 20 đơn, khoảng cách và cờ tồn kho.")
if sum(counts.values()) != 20:
    raise ValueError("Phải phân loại đủ 20 đơn giao.")
for name in ("INVALID", "HOLD", "FREE", "LOCAL", "FAR"):
    print(name + ":", counts[name])
print("Phí thu (đồng):", total_fee_vnd)
print("Cần xử lý:", ", ".join(blocked_ids))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân 20 đơn giao theo dữ liệu lỗi, giữ hàng, miễn phí hoặc phí địa phương/xa.
# Dữ liệu: 20 số tiền đơn vị đồng, 20 khoảng cách km (None là thiếu), 20 cờ tồn kho; ba list ứng
với cùng đơn.
amount_vnd =
[350000,530000,710000,890000,None,1250000,350000,530000,710000,890000,1070000,1250000,-5000,530000,
710000,890000,1070000,1250000,350000,530000]
distance_km = [1,2,3,4,5,6,7,None,1,2,3,4,5,6,7,8,1,2,3,4]
in_stock =
[True,True,False,True,True,True,True,True,True,False,True,True,True,True,True,True,True,False,True,
True]
counts = {"INVALID": 0, "HOLD": 0, "FREE": 0, "LOCAL": 0, "FAR": 0}
blocked_ids = []
total_fee_vnd = 0
# zip() ghép ba thuộc tính một đơn; enumerate(...,1) đánh số đơn từ 1 để tạo mã O01..O20.
for idx, (amount, km, available) in enumerate(zip(amount_vnd, distance_km, in_stock), 1):
    # or báo dữ liệu lỗi nếu thiếu tiền/khoảng cách hoặc có số âm; ưu tiên xử lý trước phí.
    if amount is None or km is None or amount < 0 or km < 0:
        state = "INVALID"
        fee = 0
    # not available là hết hàng nên HOLD, chưa thu phí.
    elif not available:
        state = "HOLD"
        fee = 0
    # Đơn từ 1 triệu đồng trở lên được miễn phí vận chuyển (FREE).
    elif amount >= 1000000:
        state = "FREE"
        fee = 0
    # Khoảng cách tối đa 5 km nhận phí LOCAL 15.000 đồng.
    elif km <= 5:
```

```
        state = "LOCAL"
        fee = 15000
    # Đơn còn lại thuộc FAR và thu phí 30.000 đồng.
    else:
        state = "FAR"
        fee = 30000
    # Dictionary counts đếm số đơn của loại trạng thái đã chọn.
    counts[state] += 1
    # += cộng phí của từng đơn, kể cả phí 0 đồng cho đơn bị chặn.
    total_fee_vnd += fee
    # in kiểm tra đơn nằm trong hai hạng không được xử lý giao hàng.
    if state in ("INVALID", "HOLD"):
        # append() lưu mã đơn chặn; f-string :02d hiển thị số thứ tự hai chữ số.
        blocked_ids.append(f"O{idx:02d}")
# len() đếm số bản ghi; != và or kiểm tra tổng quan sát đúng kỳ vọng.
if len(amount_vnd) != 20 or len(distance_km) != 20 or len(in_stock) != 20:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Phải có đúng 20 đơn, khoảng cách và cờ tồn kho.")
# values() lấy bộ đếm từng loại và sum() xác nhận đủ 20 đơn.
if sum(counts.values()) != 20:
    # raise ValueError(...) báo lỗi và dừng nếu kiểm tra dữ liệu không đạt.
    raise ValueError("Phải phân loại đủ 20 đơn giao.")
for name in ("INVALID", "HOLD", "FREE", "LOCAL", "FAR"):
    print(name + ":", counts[name])
print("Phí thu (đồng):", total_fee_vnd)
# join() nối các mã cần xử lý bằng dấu phẩy và khoảng trắng.
print("Cần xử lý:", ", ".join(blocked_ids))
```

Kết quả khi chạy:

```
INVALID: 3
HOLD: 3
FREE: 4
LOCAL: 6
FAR: 4
Phí thu (đồng): 210000
Cần xử lý: O03, O05, O08, O10, O13, O18
```

Mỗi đơn chỉ rơi vào **một trạng thái** và phí giao chỉ thu khi đã xác định có thể giao. Điều kiện biên 1.000.000 đồng và 5 km được xử lý bằng >=/<=, trong khi thiếu dữ liệu được tách trước khi so sánh.

6. Nhầm lẫn dễ gặp

- Phải đặt dấu `:` sau `if`, `elif`, `else` và thụt các lệnh trong nhánh.
- Nếu một `if`/`elif` đã đúng, các nhánh sau trong cùng chuỗi sẽ không chạy.
- `else` không có điều kiện riêng. Không nhầm = (gán giá trị) với == (so sánh bằng nhau).
- Khi chia khoảng điểm hay nhiệt độ, phải chọn đúng >=, >, <= để không xếp nhầm giá trị tại ngưỡng.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does `if` do? | `if` dùng để làm gì?

A. Always runs every conditional branch | Luôn chạy mọi nhánh điều kiện

B. Runs its body only when the condition is false | Chỉ chạy thân khối khi điều kiện sai

C. Runs its body once regardless of the condition | Luôn chạy thân khối một lần bất kể điều kiện

D. Runs its indented body when the condition is true | Chạy thân khối thụt lề khi điều kiện đúng

Đáp án: D

Giải thích: `if` đánh giá điều kiện rồi chỉ chạy thân khối được thụt lề nếu điều kiện có giá trị đúng. Ví dụ với `x = 5`, `if x > 3:` sẽ chạy thân khối; không bắt buộc phải có `else`.

Q2. When is an `elif` condition checked? | Khi nào Python xét điều kiện `elif`?

A. Even after a preceding branch has matched | Kể cả khi nhánh trước đã đúng

B. Before the preceding `if` is checked | Trước khi kiểm tra `if` đứng trước

C. After `else` has executed | Sau khi `else` chạy

D. Only if earlier branches in the same chain did not match | Chỉ khi các nhánh trước trong cùng chuỗi không khớp

Đáp án: D

Giải thích: Python chỉ xét `elif` khi những điều kiện `if` hoặc `elif` đứng trước trong cùng chuỗi đều không khớp. Khi một nhánh trước đã đúng, các `elif` sau không được kiểm tra, nên không có nhiều nhánh cùng chạy.

Q3. When does an `else` block run? | Khi nào khối `else` chạy?

A. Every time the initial `if` is true | Mỗi khi `if` đầu tiên đúng

B. After any successful `elif` branch | Sau bất kỳ nhánh `elif` nào đúng

C. When no earlier condition in its chain was true | Khi mọi điều kiện trước trong cùng chuỗi đều sai

D. Only when a program raises an error | Chỉ khi chương trình phát sinh lỗi

Đáp án: C

Giải thích: `else` là nhánh dự phòng không có điều kiện riêng. Nó chỉ chạy khi mọi điều kiện `if` hoặc `elif` đứng trước trong cùng chuỗi đều sai, không chạy sau nhánh đã khớp.

Q4. What is a nested `if`? | `if` lồng nhau là gì?

A. An `if` statement inside another conditional block | Một lệnh `if` bên trong khối điều kiện khác

B. A condition joined by `and` | Điều kiện kết hợp bởi `and`

C. An `else` branch following `if` | Nhánh `else` theo sau `if`

D. Two `if` statements at the same indentation level | Hai lệnh `if` có cùng mức thụt lề

Đáp án: A

Giải thích: `if` lồng nhau là câu lệnh điều kiện nằm trong thân một câu lệnh điều kiện khác nên điều kiện trong chỉ được xét khi chương trình đi vào khối đó. Nó khác hai câu `if` độc lập, vốn có thể cùng chạy.

Q5. Which keyword introduces another conditional test? | Từ khóa nào bổ sung một điều kiện tiếp theo?

A. `if` | `if`

B. `else` | `else`

C. `elif` | `elif`

D. `in` | `in`

Đáp án: C

Giải thích: `elif` mở thêm một nhánh có điều kiện nối với `if` trước đó. Khác `else`, nhánh này có điều kiện riêng, ví dụ `elif score >= 80:`.

Q6. Which keyword gives an unconditional fallback branch? | Từ khóa nào tạo nhánh còn lại không cần điều kiện?

A. `in` | `in`

B. `elif` | `elif`

C. `if` | `if`

D. `else` | `else`

Đáp án: D

Giải thích: `else` là nhánh dự phòng không ghi điều kiện trong chuỗi `if/elif/else`; Python chỉ vào đó khi các điều kiện trước sai. Không thể viết `else condition:` vì `else` không nhận điều kiện.

Q7. Which keyword starts a conditional branch? | Từ khóa nào bắt đầu một nhánh điều kiện?

A. `else` | `else`

B. `if` | `if`

C. `elif` | `elif`

D. `in` | `in`

Đáp án: B

Giải thích: Nhánh điều kiện đầu tiên dùng `if`, tiếp theo là biểu thức và dấu hai chấm. `elif` chỉ có thể nối sau một `if` trước đó, không tự mở đầu chuỗi mới.

Q8. Which operator asks whether an item belongs to a list? | Toán tử nào kiểm tra phần tử có trong List?

A. `if` | `if`

B. `elif` | `elif`

C. `in` | `in`

D. `else` | `else`

Đáp án: C

Giải thích: `in` kiểm tra phần tử thuộc tập hợp: `3 in [1, 3]` là `True`. Trong khi đó `if`, `elif`, `else` là từ khóa điều khiển luồng, không phải toán tử kiểm tra phần tử.

Q9. What does `score >= 80` include? | Điều kiện `score >= 80` bao gồm gì?

A. Only the score exactly equal to 80 | Chỉ điểm đúng bằng 80

B. Every score of at least 80, including 80 | Mọi điểm từ 80 trở lên, kể cả 80

C. Only scores strictly above 80 | Chỉ điểm lớn hơn 80

D. Scores below 80 | Điểm thấp hơn 80

Đáp án: B

Giải thích: >= bao gồm chính giá trị ngưỡng, nên `score >= 80` đúng với 80, 81 và các giá trị lớn hơn. Dùng so sánh nghiêm ngặt `score > 80` sẽ loại đúng điểm 80.

Q10. What is the difference between = and ==? | Dấu = khác == ra sao?

A. = assigns; == compares equality | = để gán; == so sánh bằng nhau

B. = compares; == assigns | = để so sánh; == để gán

C. Both operators assign variables | Cả hai toán tử đều dùng gán biến

D. Both operators compare equality | Cả hai toán tử so sánh bằng nhau

Đáp án: A

Giải thích: = gán giá trị như `score = 80`, còn == so sánh và trả Boolean như `score == 80`. Không thể thay hai toán tử cho nhau trong điều kiện `if`.

Q11. What defines the body of an `if` in Python? | Điều gì xác định thân khối `if` trong Python?

A. The indentation of following statements | Mức thụt lề của các câu lệnh theo sau

B. The number of printed statements | Số câu lệnh in ra

C. The order of imported libraries | Thứ tự thư viện được nhập

D. The variable name in the condition | Tên biến trong điều kiện

Đáp án: A

Giải thích: Thụt lề xác định câu lệnh nào thuộc thân `if`, thông thường bốn dấu cách cho mỗi cấp. Dấu hai chấm kết thúc dòng điều kiện, còn các dòng thụt lề đúng sau đó mới nằm trong nhánh.

Q12. How are cases checked in an `if/elif` chain? | Các trường hợp được xét theo thứ tự nào trong chuỗi `if/elif`?

A. All branches run concurrently | Mọi nhánh chạy đồng thời

B. Top to bottom, stopping after the first match | Từ trên xuống, dừng tại nhánh đúng đầu tiên

C. From the last `elif` upward | Từ `elif` cuối lên trên

D. By whichever condition is cheapest | Theo điều kiện tốn ít tính toán nhất

Đáp án: B

Giải thích: Python xét nhánh từ trên xuống và chạy nhánh đầu tiên có điều kiện đúng. Nếu điểm 95 cũng thỏa `>= 80`, đặt `>= 80` trước `>= 90` sẽ chọn mức điểm thấp hơn quá sớm.

Q13. What does `True and False` evaluate to? | `True and False` bằng gì?

A. `0` (integer) | `0` (số nguyên)

B. `True` (Boolean) | `True` (Boolean)

C. `None` | `None`

D. `False` (Boolean) | `False` (Boolean)

Đáp án: D

Giải thích: `and` với hai toán hạng Boolean chỉ trả `True` khi cả hai đều `True`. Vì vế thứ hai là `False`, `True and False` cho Boolean `False`, không phải số nguyên `0`.

Q14. What does `True or False` evaluate to? | `True or False` bằng gì?

A. `None` | `None`

B. `1` (integer) | `1` (số nguyên)

C. `True` (Boolean) | `True` (Boolean)

D. `False` (Boolean) | `False` (Boolean)

Đáp án: C

Giải thích: `or` với toán hạng Boolean chỉ cần một vế đúng; vế trái là `True` nên `True or False` cho Boolean `True`. Kết quả không cùng kiểu với số nguyên `1`.

Q15. What does `not True` evaluate to? | `not True` bằng gì?

A. `0` (integer) | `0` (số nguyên)

B. `True` (Boolean) | `True` (Boolean)

C. `None` | `None`

D. `False` (Boolean) | `False` (Boolean)

Đáp án: D

Giải thích: `not` đảo giá trị logic của toán hạng, nên `not True` cho Boolean `False`. Số nguyên `0` cũng được xem là sai trong điều kiện nhưng không phải giá trị trả về ở đây.

Q16. What does `5 % 2 == 0` evaluate to? | `5 % 2 == 0` có kết quả gì?

A. `False` (Boolean) | `False` (Boolean)

B. `None` | `None`

C. `0` (integer) | `0` (số nguyên)

D. `True` (Boolean) | `True` (Boolean)

Đáp án: A

Giải thích: Phần dư `5 % 2` là `1`; vì vậy so sánh `1 == 0` là sai. Toàn bộ biểu thức trả về Boolean `False`, nghĩa là 5 không phải số chẵn.

Q17. How many branches run in one `if/elif/else` chain? | Có bao nhiêu nhánh chạy trong một chuỗi `if/elif/else`?

A. Every branch whose condition is true | Mọi nhánh có điều kiện đúng

B. Every `elif` plus `else` | Mọi nhánh `elif` và `else`

C. At most one branch, possibly none without `else` | Tối đa một nhánh, có thể không nhánh nào nếu thiếu `else`

D. Exactly two branches always | Luôn đúng hai nhánh

Đáp án: C

Giải thích: Trong một chuỗi `if/elif/else`, chương trình chạy tối đa một nhánh: nhánh đúng đầu tiên hoặc `else` khi không nhánh nào đúng. Nếu không có `else` và mọi điều kiện đều sai thì không nhánh nào chạy.

Q18. What happens if an `if` condition is false and there is no `else`? | Nếu `if` sai và không có `else` thì sao?

A. The body runs once regardless | Thân nhánh vẫn chạy một lần

B. The body is skipped and execution continues | Bỏ qua thân nhánh và tiếp tục chương trình

C. A `SyntaxError` is raised | Phát sinh `SyntaxError`

D. Python inserts and runs an implicit `else` | Python tự chèn và chạy `else`

Đáp án: B

Giải thích: Khi điều kiện `if` sai, thân nhánh đó được bỏ qua. Nếu không có `else`, chương trình tiếp tục sau khối điều kiện; đây là Python hợp lệ, không phải lỗi cú pháp.

Q19. How can two conditions both be required for access? | Cách nào yêu cầu đồng thời hai điều kiện để cho phép truy cập?

A. Use == alone | Chỉ dùng ==

B. Use `and` | Dùng `and`

C. Use `or` | Dùng `or`

D. Use `not` | Dùng `not`

Đáp án: B

Giải thích: Phép `and` bắt buộc đồng thời hai điều kiện như `has_ticket and age >= 18`. Dùng `or` sẽ chấp nhận người chỉ thỏa một điều kiện, nên không đảm bảo đủ cả hai yêu cầu.

Q20. In one `if/elif` grade chain, which order correctly tests `score >= 90`, `score >= 80`, and `score >= 70` to select the highest applicable grade? | Trong một chuỗi `if/elif` phân loại điểm, cần xét `score >= 90`, `score >= 80`, `score >= 70` theo thứ tự nào để chọn mức cao nhất phù hợp?

A. Test `>= 90`, then `>= 80`, then `>= 70` | Xét `>= 90`, rồi `>= 80`, rồi `>= 70`

B. Write independent `if` statements to select a single grade | Viết các `if` độc lập để chọn duy nhất một nhóm

C. Test `>= 70` before `>= 90` | Xét `>= 70` trước `>= 90`

D. Use any order because all matches run | Thứ tự tùy ý vì mọi nhánh đúng đều chạy

Đáp án: A

Giải thích: Chuỗi điều kiện chọn nhánh đúng đầu tiên; hãy xét ngưỡng điểm cao nhất trước, ví dụ `>= 90`, rồi `>= 80`, `>= 70`. Nếu bắt đầu từ `>= 70`, điểm 95 sẽ bị xếp nhóm quá sớm.

8. Nối sang bài tiếp theo

`Toán tử → Điều kiện → Phân loại theo quy tắc.`

PY-05 — Vòng lặp for, while, break và continue

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Iteration — lần lặp; **For loop** — vòng for.

`Điều kiện → Vòng lặp → Xử lý hàng loạt`

Học xong dùng để làm gì? Tự động tính toán hoặc kiểm tra nhiều dòng dữ liệu bằng for/while, tránh sao chép cùng thao tác.

Kết nối kiến thức tiếp theo: Sang PY-06, lưu nhiều giá trị trong List để vòng lặp có dữ liệu thực tế mà xử lý.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Iteration	lần lặp	Một lượt thực hiện trong quá trình lặp
For loop	vòng for	Lặp qua từng phần tử trong một nhóm
While loop	vòng while	Lặp khi một điều kiện còn đúng
Break/continue	dừng/bỏ qua	Dừng hẳn vòng lặp hoặc bỏ qua lượt hiện tại

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`range(start,stop)`	Tạo chỉ số nguyên, không bao gồm stop
`enumerate(iterable)`	Lấy đồng thời vị trí và giá trị
`break`	Dừng sớm khi đã tìm thấy
`continue`	Bỏ qua dữ liệu của một lượt lặp

4. Khái niệm liên quan

Vòng lặp cho phép chương trình thực hiện một công việc nhiều lần mà không phải sao chép cùng câu lệnh. Một lần thực hiện được gọi là **lần lặp**. Dùng `for` khi duyệt các phần tử đã có; dùng `while` khi còn phải lặp theo một điều kiện. Hai lệnh `break` và `continue` kiểm soát cách vòng lặp kết thúc hoặc bỏ qua một lượt.

4.1. Loop / Iteration — Vòng lặp và lần lặp

Từ vựng trong mục: Loop — vòng lặp; **Iteration** — lần lặp.

Ví dụ cần đánh số hai lượt kiểm tra liên tiếp. Thay vì viết hai lần `print()`, ta dùng `for` và `range(2)` để Python lần lượt cấp giá trị 0 và 1 cho biến `i`. Mỗi lần thân vòng lặp chạy là một **lần lặp**; ở ví dụ có hai lần lặp và hai dòng được in ra.

No Note — Không chú thích

```
for i in range(2):
    print(i)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: In hai số thứ tự đầu bằng vòng for và range().
# range(2) tạo 0,1: bắt đầu 0 và không lấy cận cuối 2.
for i in range(2):
    # print() in mỗi giá trị trong từng lượt for, mỗi giá trị trên một dòng.
    print(i)
```

Kết quả khi chạy:

```
0
1
```

`range(2)` bắt đầu từ 0 và dừng trước 2, nên in 0 rồi 1.

4.2. For loop — Duyệt từng phần tử

Từ vựng trong mục: For loop — vòng `for`; **Iterable** — dữ liệu duyệt được.

Khi có sẵn danh sách `[1, 2]` đại diện hai mã số, `for` lấy từng phần tử theo thứ tự rồi chạy thân vòng lặp. Biến `x` lần lượt nhận 1 rồi 2; nếu danh sách có thêm phần tử, code không cần thêm một câu `print()` riêng cho phần tử mới.

No Note — Không chú thích

```
for x in [1, 2]:
    print(x)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Duyệt lần lượt phần tử của list bằng for.
# for lần lượt gán x bằng 1 rồi 2, không cần tự quản lý chỉ số.
for x in [1, 2]:
    # print() hiển thị phần tử hiện tại trong vòng lặp.
    print(x)
```

Kết quả khi chạy:

```
1
2
```

Mỗi phần tử tạo ra một dòng đầu ra, theo thứ tự đã lưu trong danh sách.

4.3. While loop — Lặp khi điều kiện còn đúng

Từ vựng trong mục: While loop — vòng `while`; **Condition** — điều kiện.

`while` kiểm tra một điều kiện **trước mỗi lượt**. Ví dụ số việc còn lại là `n = 2`; mỗi lượt hoàn thành một việc nên giảm `n` đi 1. Khi `n` bằng 0 thì `n > 0` sai, vòng lặp dừng. Điều kiện phải có cơ hội trở thành sai để tránh lặp vô hạn.

No Note — Không chú thích

```
n = 2

while n > 0:
    n -= 1

print(n)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa while lặp đến khi biến đếm bằng 0.
n = 2

# while kiểm tra điều kiện trước mỗi lượt: còn n > 0 thì vẫn tiếp tục.
while n > 0:
    # -= 1 giảm biến đếm, giúp điều kiện while cuối cùng thành False để không lặp vô tận.
    n -= 1

# In n sau khi thoát vòng lặp, bằng 0.
print(n)
```

Kết quả khi chạy:

```
0
```

Hai lượt giảm từ 2 xuống 1 rồi 0; câu `print` bên ngoài vòng lặp cho thấy giá trị cuối cùng.

4.4. Break / continue — Dừng hoặc bỏ qua

Từ vựng trong mục: Break — dừng vòng lặp; **Continue** — bỏ qua lượt hiện tại.

`continue` bỏ phần còn lại của **lượt đang chạy** nhưng vẫn xét lượt tiếp theo. `break` thoát khỏi **toàn bộ vòng lặp hiện tại**. Ví dụ duyệt `[0, 1, 2, 3]`: bỏ qua 0, in 1 và 2, rồi dừng khi gặp 3. Hai lệnh có tác dụng khác nhau dù đều làm một số dòng không được thực hiện.

No Note — Không chú thích

```
for x in [0, 1, 2, 3]:
    if x == 0:
        continue
    if x == 3:
        break
    print(x)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt continue bỏ qua lượt hiện tại và break dừng vòng lặp.
# for duyệt bốn số 0,1,2,3.
for x in [0, 1, 2, 3]:
    # == kiểm tra x có bằng 0 hay không.
    if x == 0:
        # continue bỏ phần còn lại của lượt x=0, chuyển sang lượt sau.
        continue
    # Nếu x đạt 3 thì cần ngừng cả vòng for.
    if x == 3:
        # break rời khỏi vòng lặp ngay lập tức, khác continue chỉ bỏ một lượt.
        break
    # Chỉ in số đi qua được cả hai điều kiện, ở đây là 1 và 2.
    print(x)
```

Kết quả khi chạy:

```
1
2
```

Số 0 bị bỏ qua bởi `continue`; khi đến 3, `break` dừng hẳn nên không in 3.

5. Năm tình huống vòng lặp — từ đơn giản đến kết hợp

Tình huống 1 — Cấp 1: Cộng doanh số của 10 đơn bằng for

Từ vựng trong mục: For loop — vòng lặp for; **Accumulator** — biến cộng dồn.

Tình huống thực tế: Cuối ca, thu ngân cần tổng kết **10 đơn bán hàng**. Danh sách lưu số tiền của từng đơn, theo đơn vị **nghìn đồng**: 120, 80, 200, 150, 90, 70, 180, 210, 130 và 160. Mỗi số là tiền của đúng một đơn. Người quản lý chỉ cần biết có bao nhiêu đơn và tổng số tiền cả ca để so với sổ thu ngân.

Vì sao chọn cách này? `for` có nghĩa là **làm lần lượt cho từng phần tử**: chương trình lấy tiền đơn đầu, đơn thứ hai... đến đơn thứ mười. Biến `total` ban đầu bằng 0; mỗi lần đọc một đơn thì cộng số tiền ấy vào tổng đang có. Cách này phù hợp khi đã có danh sách 10 đơn và giúp không phải viết 10 phép cộng riêng. `len(orders)` đếm số đơn, còn `total` là tổng doanh số **nghìn đồng**, nên không được nhầm số đơn với số tiền.

Quy trình:

1. Khởi tạo `total = 0` trước khi lặp.
2. Với từng giá trị, cộng vào `total`.
3. Kiểm tra đủ 10 dòng và in tổng tiền.

No Note — Không chú thích

```
orders = [120,80,200,150,90,70,180,210,130,160]
total = 0
for amount in orders:
    total += amount
if len(orders) != 10:
    raise ValueError("Cần đủ 10 đơn bán hàng.")
print("Số đơn:", len(orders))
print("Doanh số (nghìn đồng):", total)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cộng doanh số´mười đơn hàng bằng vòng lặp for.
# Dữ liệu: 10 số´là doanh thu môĩ đơn (nghìn đồ`ng), một phâ`n tư’cho một đơn.
orders = [120,80,200,150,90,70,180,210,130,160]
# Khởi tạo biế´n tổ’ng (nghìn đồ`ng) bằng 0 trước khi cộng các đơn.
total = 0
# Môĩ vòng lâ´y doanh số´nghìn đồ`ng của một đơn.
for amount in orders:
    # += cộng doanh số´đơn hiện tại vào tổ’ng.
    total += amount
# len() đế´m số´đơn; != so sánh số´lượng khác 10.
if len(orders) != 10:
    # raise ValueError() dừng và thông báo nê´u số´đơn sai.
    raise ValueError("Cà`n đủ 10 đơn bán hàng.")
# len() đế´m số´phâ`n tư’thực tê´trong danh sách.
print("Sô´đơn:", len(orders))
print("Doanh sô´(nghìn đồ`ng):", total)
```

Kết quả khi chạy:

```
Sô´đơn: 10
Doanh sô´(nghìn đồ`ng): 1390
```

Mỗi lượt lặp đưa vào tổng **một đơn**; tổng chỉ đúng nếu biến cộng dồn được đặt ngoài vòng lặp và không bị đặt về 0 sau mỗi dòng.

Tình huống 2 — Cấp 2: Dùng continue bỏ hai lượt đo trống trong 12 lượt

Từ vựng trong mục: Continue — bỏ qua lượt lặp; **Missing** — số liệu thiếu; **Average** — trung bình.

Tình huống thực tế: Trạm thời tiết ghi **12 lượt nhiệt độ** tính bằng **°C**, mỗi lượt ứng với một phút. Ở lượt thứ 3 và thứ 8, thiết bị không gửi được kết quả nên bảng ghi `None`; đây là thông tin thiếu chứ không phải 0 °C. Người trực muốn đếm số lượt có kết quả, số lượt thiếu và nhiệt độ trung bình của các lượt có số.

Vì sao chọn cách này? `continue` nghĩa là **bỏ phần còn lại của lượt lặp hiện tại rồi chuyển sang lượt kế tiếp**. Khi gặp `None`, chương trình tăng số lượt thiếu và chuyển qua phút sau, không cộng hay đếm lượt này là nhiệt độ hợp lệ. Các giá trị có số được cộng vào `total` và tăng `valid_count`; cuối cùng lấy tổng nhiệt độ chia cho **số lượt có giá trị**, không chia cho cả 12. Nếu thay `None` bằng 0 thì sẽ làm thấp nhiệt độ trung bình và che giấu hai lần thiết bị mất tín hiệu.

Quy trình:

1. Khởi tạo tổng, số lượt hợp lệ và số lượt thiếu.
2. Duyệt 12 giá trị; nếu `None` thì tăng số thiếu rồi `continue`.
3. Chia tổng cho **số lượt hợp lệ**, không phải 12.

No Note — Không chú thích

```
readings = [27,28,None,29,30,31,28,None,27,32,30,29]
total = 0
valid_count = 0
missing_count = 0
for temperature in readings:
    if temperature is None:
        missing_count += 1
        continue
    total += temperature
    valid_count += 1
```

```
if len(readings) != 12 or valid_count + missing_count != 12:
    raise ValueError("Phải đối chiếu đủ 12 lượt nhiệt độ, kể cả dữ liệu thiếu.")
print("Hợp lệ:", valid_count)
print("Thiếu:", missing_count)
print(f"Trung bình: {total / valid_count:.1f} °C")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Bỏ nhiệt độ thiếu rồi tính trung bình từ 12 lượt đo.
# Dữ liệu: 12 số đo nhiệt độ °C, None biểu diễn hai lượt thiếu dữ liệu.
readings = [27,28,None,29,30,31,28,None,27,32,30,29]
total = 0
valid_count = 0
missing_count = 0
for temperature in readings:
    # is None nhận diện lượt không ghi được nhiệt độ, khác giá trị số 0.
    if temperature is None:
        # Tăng số đo thiếu để đối chiếu với tổng 12 lượt.
        missing_count += 1
        # continue bỏ qua việc cộng nhiệt độ thiếu và chuyển lượt khác.
        continue
    # += cộng số đo °C hợp lệ vào tổng.
    total += temperature
    # Tăng số đo thực sự tham gia trung bình.
    valid_count += 1
# len() đếm cả lượt thiếu; or yêu cầu tổng hợp lệ + thiếu khớp 12.
if len(readings) != 12 or valid_count + missing_count != 12:
    # Dừng chương trình bằng ValueError nếu số liệu không khớp.
    raise ValueError("Phải đối chiếu đủ 12 lượt nhiệt độ, kể cả dữ liệu thiếu.")
print("Hợp lệ:", valid_count)
print("Thiếu:", missing_count)
# Chia tổng nhiệt độ hợp lệ cho số lượt hợp lệ; :.1f hiển thị một chữ số thập phân °C.
print(f"Trung bình: {total / valid_count:.1f} °C")
```

Kết quả khi chạy:

```
Hợp lệ: 10
Thiếu: 2
Trung bình: 29.1 °C
```

Vòng lặp đã đọc 12 lượt nhưng chỉ 10 lượt được tính trung bình. Nếu chia cho 12, thiết bị sẽ cho kết quả thấp hơn thực tế.

Tình huống 3 — Cấp 3: Tìm điểm hết hàng đầu tiên trong 15 kệ

Từ vựng trong mục: Enumerate — duyệt kèm vị trí; **Break** — dừng vòng lặp.

Tình huống thực tế: Người quản lý đi kiểm kê **15 kệ K01 đến K15** theo thứ tự từ đầu kho tới cuối kho. Mỗi kệ có một số món đang còn; một kệ có **0 món** nghĩa là đã hết hàng. Họ cần tìm **mã kệ đầu tiên** hết hàng để giao nhân viên đi bổ sung ngay, không cần liệt kê mọi kệ trống.

Vì sao chọn cách này? `enumerate(stocks, 1)` giúp đọc cùng lúc **vị trí kệ** và **số món trên kệ**, bắt đầu đếm từ 1 để khớp K01, K02... Khi gặp tồn kho bằng 0, chương trình tạo mã kệ đó rồi dùng `break`. `break` là **thoát hẳn vòng lặp**, khác `continue` chỉ bỏ một lượt và vẫn tiếp tục. Vì yêu cầu chỉ tìm kệ trống đầu tiên, dừng ngay sẽ không vô tình chọn kệ trống thứ hai ở phía sau.

Quy trình:

1. Ghi 15 số lượng theo thứ tự kệ.
2. Duyệt cùng số thứ tự; khi gặp `0`, lưu mã kệ và thoát vòng.
3. In vị trí đầu tiên rồi kiểm tra đúng là kệ trống.

No Note — Không chú thích

```
stocks = [12,8,7,5,3,4,9,1,0,6,0,2,8,10,4]
first_empty = None
for shelf_number, amount in enumerate(stocks, 1):
    if amount == 0:
        first_empty = f"K{shelf_number:02d}"
        break
if len(stocks) != 15:
    raise ValueError("Cần kiểm kê đủ 15 kệ.")
if first_empty is None:
    raise ValueError("Không tìm thấy kệ trống cần báo cáo.")
if stocks[int(first_empty[1:]) - 1] != 0:
    raise ValueError("Kệ được chọn phải có tồn kho bằng 0.")
print("Số kệ kiểm kê:", len(stocks))
print("Kệ hết hàng đầu:", first_empty)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng break tìm kệ hết hàng đầu tiên trong 15 kệ.
# Dữ liệu: 15 số tồn của 15 kệ theo thứ tự; 0 là kệ trống.
stocks = [12,8,7,5,3,4,9,1,0,6,0,2,8,10,4]
# None nghĩa chưa tìm thấy kệ hết hàng.
first_empty = None
# enumerate(...,1) gán số kệ bắt đầu 1 thay vì chỉ số list bắt đầu 0.
for shelf_number, amount in enumerate(stocks, 1):
    # == 0 xác định kệ trống.
    if amount == 0:
        # f-string :02d định dạng mã kệ K01..K15.
        first_empty = f"K{shelf_number:02d}"
        # break rời vòng for ở kệ trống đầu tiên, không kiểm những kệ sau.
        break
if len(stocks) != 15:
    # Báo lỗi nếu dữ liệu hoặc kết quả tìm kệ không đạt kiểm tra.
    raise ValueError("Cần kiểm kê đủ 15 kệ.")
# Nếu vẫn None thì toàn bộ kệ đều không trống; báo lỗi nghiệp vụ.
if first_empty is None:
    # Báo lỗi nếu dữ liệu hoặc kết quả tìm kệ không đạt kiểm tra.
    raise ValueError("Không tìm thấy kệ trống cần báo cáo.")
# [1:] bỏ ký tự K, int() đổi số kệ thành int; trừ 1 để lấy chỉ số list đúng.
if stocks[int(first_empty[1:]) - 1] != 0:
    # Báo lỗi nếu dữ liệu hoặc kết quả tìm kệ không đạt kiểm tra.
    raise ValueError("Kệ được chọn phải có tồn kho bằng 0.")
# len() đếm số phần tử thực tế trong danh sách.
print("Số kệ kiểm kê:", len(stocks))
print("Kệ hết hàng đầu:", first_empty)
```

Kết quả khi chạy:

```
Số kệ kiểm kê: 15
Kệ hết hàng đầu: K09
```

K09 trống sớm hơn 1 kệ trống còn lại, nhưng chương trình chủ động dừng trước; đây là lý do dùng `break` thay vì lặp hết toàn bộ danh sách.

Tình huống 4 — Cấp 4: Dùng while giới hạn số lần kết nối của 18 thiết bị

Từ vựng trong mục: While — lặp theo điều kiện; **Retry** — thử lại; **Termination** — điều kiện dừng.

Tình huống thực tế: Đội kỹ thuật thử kết nối **18 thiết bị D01 đến D18**. Đây là dữ liệu mô phỏng: với mỗi thiết bị, một số từ 1 đến 3 chỉ lần thử mà kết nối sẽ thành công; số 0 hoặc 4 nghĩa là không thành công trong ba lần đầu. Mỗi thiết bị được phép thử **tối đa ba lần**. Nhóm cần biết có bao nhiêu thiết bị kết nối được, bao nhiêu thất bại và tổng số lần đã thử.

Vì sao chọn cách này? `while` nghĩa là **còn đúng điều kiện thì còn thử tiếp**; nó hợp với việc chưa biết mỗi thiết bị sẽ thành công ở lần thứ mấy. Biến `attempts` bắt đầu từ 0 và tăng sau mỗi lần thử. Vòng lặp chỉ tiếp tục khi **chưa thử đủ 3 lần** `and` **chưa kết nối được**; nếu một điều kiện không còn đúng thì dừng. Các mã 0 và 4 là giả định mô phỏng rằng ba lần đầu đều thất bại, không phải kết quả kết nối thực tế. Đặt giới hạn ba lần giúp tránh vòng chạy mãi và cho phép cộng tổng số lượt thử của mọi thiết bị.

Quy trình:

1. Duyệt 18 thiết bị, mỗi thiết bị khởi tạo lại số lần thử và cờ kết nối.
2. Trong `while`, tăng số lần thử; mô phỏng thành công đúng lượt được cấu hình.
3. Cộng kết quả thành công/thất bại và tổng lượt thử; dùng `if` kiểm tra không thiết bị nào vượt 3 lần.

No Note — Không chú thích

```
success_on_try = [0,2,3,1,4,0,1,2,4,1,0,3,4,2,3,0,4,3]
success_count = 0
failed_count = 0
total_attempts = 0
for planned in success_on_try:
    attempts = 0
    connected = False
    while attempts < 3 and not connected:
        attempts += 1
        connected = (planned == attempts)
    total_attempts += attempts
    if connected:
        success_count += 1
    else:
        failed_count += 1
    if attempts > 3:
        raise RuntimeError("Mỗi thiết bị chỉ được thử kết nối tối đa 3 lần.")
if len(success_on_try) != 18:
    raise ValueError("Cần đúng 18 thiết bị để thống kê.")
if success_count + failed_count != 18:
    raise ValueError("Tổng thiết bị thành công và thất bại phải bằng 18.")
print("Thiết bị:", len(success_on_try))
print("Kết nối được:", success_count)
print("Thất bại:", failed_count)
print("Tổng lượt thử:", total_attempts)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Giới hạn tối đa ba lần thử kết nối cho từng thiết bị trong 18 thiết bị.
# Dữ liệu: 18 chỉ số lần thử thành công; 0 hoặc >3 nghĩa không thành công trong tối đa ba lần.
# 0 hoặc 4 là không thể thành công trong giới hạn ba lượt thử; 1,2,3 chỉ lần thành công.
success_on_try = [0,2,3,1,4,0,1,2,4,1,0,3,4,2,3,0,4,3]
success_count = 0
failed_count = 0
# Bắt đầu đếm lượt thử lại từ 0 cho mỗi thiết bị.
total_attempts = 0
```

```
# Lặp qua kết quả dự kiến của từng thiết bị.
for planned in success_on_try:
    # Bắt đầu đếm lượt thử lại từ 0 cho mỗi thiết bị.
    attempts = 0
    # Cờ bool False cho biết thiết bị chưa kết nối được.
    connected = False
    # and yêu cầu vừa chưa đủ ba lần, vừa chưa kết nối; while kiểm tra lại sau mỗi lần thử.
    while attempts < 3 and not connected:
        # += tăng số lượt đã thử.
        attempts += 1
        # == kiểm tra lần thử hiện tại trùng với lần dự kiến thành công hay không.
        connected = (planned == attempts)
    # Cộng tổng số lần thử thực tế từ mỗi thiết bị.
    total_attempts += attempts
    # Khi cờ True, tăng số thiết bị thành công; ngược lại tăng thất bại.
    if connected:
        success_count += 1
    else:
        failed_count += 1
    # Kiểm tra an toàn tránh vô tình chạy quá ba lượt mỗi thiết bị.
    if attempts > 3:
        # RuntimeError báo vi phạm quy tắc xử lý lượt thử.
        raise RuntimeError("Mỗi thiết bị chỉ được thử kết nối tối đa 3 lần.")
# len() đếm số thiết bị, != phát hiện không đủ 18.
if len(success_on_try) != 18:
    # ValueError báo dữ liệu đầu vào hoặc kết quả thống kê không khớp.
    raise ValueError("Cần đúng 18 thiết bị để thống kê.")
# Đối chiếu số thành công + thất bại phải bằng 18.
if success_count + failed_count != 18:
    # ValueError báo dữ liệu đầu vào hoặc kết quả thống kê không khớp.
    raise ValueError("Tổng thiết bị thành công và thất bại phải bằng 18.")
# len() đếm số phần tử thực tế trong danh sách.
print("Thiết bị:", len(success_on_try))
print("Kết nối được:", success_count)
print("Thất bại:", failed_count)
print("Tổng lượt thử:", total_attempts)
```

Kết quả khi chạy:

```
Thiết bị: 18
Kết nối được: 10
Thất bại: 8
Tổng lượt thử: 45
```

Độ khó nằm ở điều kiện dừng của **vòng lặp lồng trong quá trình xử lý 18 thiết bị**. Khi một thiết bị thất bại, lần sau bắt đầu lại với số lượt thử bằng 0; không được dùng chung cờ `connected` giữa hai thiết bị.

Tình huống 5 — Cấp 5: Lập lịch hai ca cho 20 học viên và tổng hợp điểm danh

Từ vựng trong mục: Nested loop — vòng lặp lồng; **Continue** — bỏ lượt; **Validation** — kiểm tra dữ liệu.

Tình huống thực tế: Trung tâm có **20 học viên H01 đến H20**, mỗi người đăng ký hai ca thực hành là **Sáng** và **Chiều**. Như vậy có 40 lượt học theo lịch trước khi điểm danh. Ba học viên H04, H11 và H19 vắng đúng ca Chiều, nhưng vẫn đi học ca Sáng. Giáo vụ muốn lập danh sách lượt có mặt, biết có bao nhiêu lượt vắng và đếm số lượt có mặt của từng ca, bảo đảm không ghi trùng một học viên trong cùng một ca.

Vì sao chọn cách này? Vòng `for` **bên ngoài** chọn lần lượt 20 học viên; vòng `for` **bên trong** chọn hai ca của đúng học viên ấy, nên mỗi người tạo hai lượt dự kiến. Chương trình chỉ dùng `continue` khi gặp ba cặp cụ thể (H04, Chiều), (H11, Chiều), (H19, Chiều); vì thế ca Sáng của họ không bị xóa. Một **cặp (mã học viên, ca)** xác định một lượt duy nhất: ví dụ H04-Sáng và H04-Chiều là hai lượt khác nhau. Chuyển danh sách cặp

thành `set` giúp kiểm tra có trùng lượt không; số lượt có mặt cộng số lượt vắng phải bằng 40, nếu không thì lịch đang bị ghi thiếu hoặc ghi thừa.

Quy trình:

1. Tạo 20 mã học viên và hai ca mỗi ngày.
2. Duyệt vòng lặp lồng; khi trùng ca vắng, tăng số vắng rồi `continue`.
3. Kiểm tra số lượt dự kiến 40 = lượt hợp lệ + lượt vắng, các cặp ca không trùng.

No Note — Không chú thích

```
students =
["H01","H02","H03","H04","H05","H06","H07","H08","H09","H10","H11","H12","H13","H14","H15","H16","H
17","H18","H19","H20"]
sessions = ["Sáng", "Chiều"]
absent_afternoon = ["H04","H11","H19"]
schedule = []
absent_count = 0
for student in students:
    for session in sessions:
        if student in absent_afternoon and session == "Chiều":
            absent_count += 1
            continue
        schedule.append((student, session))
if len(students) != 20:
    raise ValueError("Cần đúng 20 học viên.")
if len(schedule) != len(set(schedule)):
    raise ValueError("Không được trùng cặp học viên và ca học.")
if len(students) * len(sessions) != len(schedule) + absent_count:
    raise ValueError("Lượt có mặt và lượt vắng phải khớp 40 lượt dự kiến.")
print("Học viên:", len(students))
print("Lượt dự kiến:", len(students) * len(sessions))
print("Lượt có mặt:", len(schedule))
print("Lượt vắng:", absent_count)
morning_count = 0
afternoon_count = 0
for student, session in schedule:
    if session == "Sáng":
        morning_count += 1
    elif session == "Chiều":
        afternoon_count += 1
print("Ca Sáng:", morning_count)
print("Ca Chiều:", afternoon_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghép 20 học viên vào hai ca, bỏ ca vắng và kiểm tra lịch không trùng.
# Dữ liệu: 20 mã học viên và hai ca sáng/chiều; danh sách absent_afternoon nêu học viên vắng ca
chiều.
students =
["H01","H02","H03","H04","H05","H06","H07","H08","H09","H10","H11","H12","H13","H14","H15","H16","H
17","H18","H19","H20"]
sessions = ["Sáng", "Chiều"]
absent_afternoon = ["H04","H11","H19"]
schedule = []
absent_count = 0
# Vòng ngoài lấy từng mã học viên trong 20 học viên.
for student in students:
    # Vòng trong ghép mỗi học viên với hai ca (Sáng và Chiều), dự kiến 20 × 2 = 40 lượt.
    for session in sessions:
        # in kiểm tra danh sách ba học viên vắng; and chỉ bỏ ca Chiều của các học viên đó.
        if student in absent_afternoon and session == "Chiều":
```

```
            # Đếm lượt không tham gia ca chiều.
            absent_count += 1
            # continue bỏ việc thêm lượt học đang vắng, nhưng vẫn duyệt học viên/ca tiếp.
            continue
        # append() thêm tuple (mã học viên, ca) cho lượt có mặt.
        schedule.append((student, session))
if len(students) != 20:
    # Dừng chương trình nếu danh sách hoặc lịch có mâu thuẫn.
    raise ValueError("Cần đúng 20 học viên.")
# set() chuyển các cặp (học viên, ca) thành tập không trùng; so độ dài phát hiện cặp bị lặp.
if len(schedule) != len(set(schedule)):
    # Dừng chương trình nếu danh sách hoặc lịch có mâu thuẫn.
    raise ValueError("Không được trùng cặp học viên và ca học.")
# Số lượt dự kiến = số học viên nhân số ca; phải bằng lượt có mặt cộng lượt vắng.
if len(students) * len(sessions) != len(schedule) + absent_count:
    # Dừng chương trình nếu danh sách hoặc lịch có mâu thuẫn.
    raise ValueError("Lượt có mặt và lượt vắng phải khớp 40 lượt dự kiến.")
# len() đếm số phần tử thực tế trong danh sách.
print("Học viên:", len(students))
# Số lượt dự kiến = số học viên nhân số ca; phải bằng lượt có mặt cộng lượt vắng.
print("Lượt dự kiến:", len(students) * len(sessions))
# len() đếm số phần tử thực tế trong danh sách.
print("Lượt có mặt:", len(schedule))
print("Lượt vắng:", absent_count)
morning_count = 0
afternoon_count = 0
# Tách mã học viên và ca từ từng tuple trong lịch đã lọc.
for student, session in schedule:
    # Nhánh if đếm ca sáng.
    if session == "Sáng":
        # += 1 cộng một lượt ca sáng.
        morning_count += 1
    # elif chỉ xét khi không phải ca sáng, đếm ca chiều.
    elif session == "Chiều":
        # += 1 cộng một lượt ca chiều.
        afternoon_count += 1
print("Ca Sáng:", morning_count)
print("Ca Chiều:", afternoon_count)
```

Kết quả khi chạy:

```
Học viên: 20
Lượt dự kiến: 40
Lượt có mặt: 37
Lượt vắng: 3
Ca Sáng: 20
Ca Chiều: 17
```

Đây là bài tổng hợp: học viên được ghép với từng ca, ngoại lệ chỉ áp dụng cho đúng ca, rồi kiểm chứng 40 lượt dự kiến. Một học viên vẫn có thể xuất hiện trong **hai dòng khác nhau** mà không phải trùng lịch.

6. Nhầm lẫn dễ gặp

- `range(1, 4)` tạo `1, 2, 3`, không chứa `4`.
- `break` dừng toàn bộ vòng lặp hiện tại, còn `continue` chỉ bỏ qua phần còn lại của một lượt.
- Với `while`, phải thay đổi biến hoặc trạng thái để vòng lặp có thể kết thúc.
- `enumerate()` mặc định đếm vị trí từ 0; dùng `start=1` nếu muốn đếm từ 1.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does a `for` loop normally iterate over? | Vòng `for` thường duyệt qua gì?

A. Only dictionary keys | Chỉ khóa của dictionary

B. Successive items from an iterable | Các phần tử liên tiếp của đối tượng duyệt được

C. Only a Boolean condition | Chỉ một điều kiện Boolean

D. Always index-value pairs | Luôn là cặp chỉ số-giá trị

Đáp án: B

Giải thích: Vòng `for` lần lượt lấy phần tử từ đối tượng duyệt được, ví dụ các giá trị `0`, `1`, `2` của `range(3)`. Cách hoạt động này khác `while` kiểm tra lại điều kiện ở mỗi lượt.

Q2. When does a `while` loop repeat? | Khi nào vòng `while` tiếp tục lặp?

A. Always for a predetermined count | Luôn theo số lượt đã xác định

B. Until exactly one iteration has completed | Chỉ đến khi chạy xong đúng một lượt

C. As long as its condition remains true | Chừng nào điều kiện vẫn đúng

D. Only when iterating over a list | Chỉ khi duyệt một list

Đáp án: C

Giải thích: Vòng `while` kiểm tra điều kiện trước mỗi lượt và chỉ lặp khi điều kiện còn đúng. Ví dụ `while n < 3:` dừng khi `n` đạt 3 nếu thân vòng lặp tăng `n` sau mỗi lượt.

Q3. What is an iteration? | Một lần lặp là gì?

A. A Boolean test before the loop | Phép kiểm tra Boolean trước vòng lặp

B. Two loops inside each other | Hai vòng lặp lồng nhau

C. One pass through the loop body | Một lượt thực hiện thân vòng lặp

D. A collection that can be iterated | Một tập hợp có thể duyệt

Đáp án: C

Giải thích: Một lần lặp là một lượt chạy thân vòng lặp. Với `for x in range(3):`, thân vòng lặp chạy ba lần, lần lượt `x` bằng 0, 1, 2.

Q4. What are nested loops? | Vòng lặp lồng nhau là gì?

A. A loop that runs exactly once | Một vòng chỉ chạy đúng một lần

B. Two loops written one after another | Hai vòng lặp viết nối tiếp nhau

C. A `for` and `while` with the same variable | `for` và `while` dùng cùng một biến

D. A loop placed inside another loop's body | Một vòng đặt trong thân vòng khác

Đáp án: D

Giải thích: Vòng lặp lồng nhau đặt một vòng trong thân vòng khác. Nếu vòng ngoài có 2 phần tử và vòng trong có 3 phần tử, thân vòng trong có thể chạy 2 nhân 3 bằng 6 lần, ứng với từng cặp kết hợp.

Q5. What does `break` do inside a loop? | `break` làm gì trong vòng lặp?

A. Skips only the current iteration's remaining statements | Chỉ bỏ qua các lệnh còn lại của lượt hiện tại

B. Does nothing and continues with the next statement | Không làm gì rồi chạy lệnh tiếp

C. Immediately exits the entire Python process | Thoát toàn bộ tiến trình Python

D. Immediately exits the innermost enclosing loop | Thoát ngay vòng lặp bao gần nhất

Đáp án: D

Giải thích: `break` kết thúc ngay vòng lặp bao gần nhất đang chạy. Với vòng lặp lồng nhau, nó không tự kết thúc mọi vòng ngoài; `continue` thì vẫn để vòng lặp tiếp tục.

Q6. What does `continue` do? | `continue` có tác dụng gì?

A. Skips the remaining body and moves toward the next iteration | Bỏ phần thân còn lại và chuyển tới lượt kế tiếp

B. Restarts the complete program | Khởi động lại toàn bộ chương trình

C. Exits the current loop completely | Kết thúc hẳn vòng lặp hiện tại

D. Acts as a no-op placeholder | Là câu lệnh giữ chỗ không thao tác

Đáp án: A

Giải thích: `continue` bỏ qua những câu lệnh còn lại trong lượt hiện tại rồi tiếp tục theo loại vòng lặp. Với `for`, chương trình lấy phần tử kế tiếp; với `while`, nó kiểm tra lại điều kiện.

Q7. What does `pass` do? | `pass` làm gì?

A. Skips the remainder of the iteration | Bỏ phần còn lại của lượt lặp

B. Rechecks the condition without running later statements | Kiểm tra lại điều kiện mà bỏ qua các lệnh sau

C. Performs no operation and then continues normally | Không thao tác gì rồi tiếp tục bình thường

D. Leaves the loop | Rời khỏi vòng lặp

Đáp án: C

Giải thích: `pass` là câu lệnh hợp lệ nhưng không làm gì, hữu ích để giữ chỗ cho thân khối bắt buộc. Khác `continue`, nó không bỏ qua những câu lệnh phía sau trong cùng lượt lặp.

Q8. When does a loop's `else` execute? | `else` của vòng lặp chạy khi nào?

A. Every time a loop body executes | Mỗi khi thân vòng lặp chạy

B. When the loop finishes normally without `break` | Khi vòng lặp kết thúc bình thường mà không gặp `break`

C. Only when `break` was executed | Chỉ khi đã chạy `break`

D. At the start of the first iteration | Khi bắt đầu lượt lặp đầu

Đáp án: B

Giải thích: `else` của vòng lặp chạy sau khi vòng lặp kết thúc bình thường: `for` duyệt hết phần tử hoặc điều kiện `while` thành sai. Nó bị bỏ qua nếu `break` kết thúc vòng lặp, khác `else` dự phòng của `if`.

Q9. What does `range(1, 4)` represent? | `range(1, 4)` biểu diễn gì?

A. Values 1, 2, 3 | Các giá trị 1, 2, 3

B. Values 1, 2, 3, 4 | Các giá trị 1, 2, 3, 4

C. Values 1, 3 | Các giá trị 1, 3

D. Values 0, 1, 2, 3 | Các giá trị 0, 1, 2, 3

Đáp án: A

Giải thích: `range(1, 4)` biểu diễn các số 1, 2, 3 vì mốc dừng 4 không được lấy. Nếu muốn bao gồm 4, cần đặt mốc dừng là 5.

Q10. What does `enumerate(items)` provide? | `enumerate(items)` cung cấp gì?

A. Pairs sorted by item value | Các cặp được sắp theo giá trị

B. Pairs of index and item | Các cặp chỉ số và phần tử

C. Only the items, without their indexes | Chỉ các phần tử, không kèm chỉ số

D. Only the indexes, without items | Chỉ chỉ số, không kèm phần tử

Đáp án: B

Giải thích: `enumerate(items)` tạo các cặp chỉ số và phần tử, mặc định bắt đầu từ chỉ số 0. Với `['a', 'b']`, khi duyệt sẽ nhận `(0, 'a')` rồi `(1, 'b')`; hàm không sắp xếp danh sách.

Q11. What does `zip(a, b)` do? | `zip(a, b)` làm gì?

A. Pairs elements at corresponding positions | Ghép phần tử ở cùng vị trí

B. Concatenates both inputs into one sequence | Nối hai đầu vào thành một dãy

C. Sorts the inputs together | Sắp xếp hai đầu vào cùng nhau

D. Generates every possible pair | Tạo mọi cặp kết hợp

Đáp án: A

Giải thích: `zip(a, b)` ghép phần tử ở cùng vị trí, ví dụ `list(zip([1, 2], ['x', 'y']))` cho `[(1, 'x'), (2, 'y')]`. Mặc định hàm dừng khi dãy ngắn nhất hết phần tử, chứ không tạo mọi cặp kết hợp.

Q12. What does `reversed(items)` provide for a sequence? | `reversed(items)` cung cấp gì cho một dãy?

A. Always returns a fully materialized list | Luôn trả về list đã tạo sẵn

B. Provides an iterator in reverse positional order | Cung cấp iterator theo thứ tự vị trí đảo ngược

C. Sorts elements from largest to smallest | Sắp phần tử từ lớn đến nhỏ

D. Preserves the original order | Giữ nguyên thứ tự ban đầu

Đáp án: B

Giải thích: `reversed()` cung cấp bộ duyệt ngược thứ tự vị trí của dãy phù hợp. Với `[3, 1, 2]`, nó cho `2, 1, 3`, không phải sắp xếp số giảm dần `3, 2, 1`.

Q13. How many iterations does a complete `for` loop over `range(3)` execute? | Vòng `for` duyệt hết `range(3)` thực hiện bao nhiêu lượt lặp?

A. 3 iterations | 3 lượt lặp

B. 4 iterations | 4 lượt lặp

C. 0 iterations | 0 lượt lặp

D. 2 iterations | 2 lượt lặp

Đáp án: A

Giải thích: `range(3)` tạo các giá trị `0`, `1`, `2`: bắt đầu mặc định từ 0 và không lấy mốc dừng 3. Vòng `for` duyệt hết sẽ chạy thân đúng ba lượt; đây là số lượt lặp, không có nghĩa dãy chứa giá trị 3.

Q14. What does `range(2, 7, 2)` yield? | `range(2, 7, 2)` cho những số nào?

A. 3, 5 | 3, 5

B. 2, 4, 6 | 2, 4, 6

C. 2, 3, 4, 5, 6 | 2, 3, 4, 5, 6

D. 2, 4, 6, 8 | 2, 4, 6, 8

Đáp án: B

Giải thích: `range(start, stop, step)` bắt đầu tại 2, mỗi lượt cộng thêm 2 và không lấy giá trị đạt hoặc vượt mốc dừng 7. Vì vậy dãy là 2, 4, 6, không phải 2, 4, 6, 8.

Q15. What does `enumerate(['a','b'])` start indexing from? | `enumerate(['a','b'])` bắt đầu chỉ số từ đâu?

A. The number of items | Số phần tử

B. -1 | -1

C. 0 | 0

D. 1 | 1

Đáp án: C

Giải thích: `enumerate()` mặc định bắt đầu chỉ số bằng `0` nếu không truyền `start`. Với `['a', 'b']`, cặp đầu là `(0, 'a')`; `enumerate(items, start=1)` mới bắt đầu từ 1.

Q16. What does `list(zip([1,2], ['a']))` contain? | `list(zip([1,2], ['a']))` gồm gì?

A. `[(1, 'a'), (2, 'a')]` | `[(1, 'a'), (2, 'a')]`

B. `[(1, 2), ('a',)]` | `[(1, 2), ('a',)]`

C. `[(1, 'a'), (2, None)]` | `[(1, 'a'), (2, None)]`

D. `[(1, 'a')]` | `[(1, 'a')]`

Đáp án: D

Giải thích: `zip()` lấy một phần tử từ mỗi dãy ở một lượt và dừng khi dãy ngắn hơn hết. Vì vậy `list(zip([1,2], ['a']))` chỉ chứa `[(1, 'a')]`, không có cặp chứa 2.

Q17. What is a common cause of an infinite `while` loop? | Nguyên nhân thường gặp khiến `while` lặp vô hạn là gì?

A. A function containing `pass` is never called | Một hàm có `pass` không được gọi

B. A `break` is reached in the first iteration | Gặp `break` ở lượt đầu

C. A `for` loop exhausts a finite iterable | `for` duyệt hết một dãy hữu hạn

D. The `while` condition stays true and no exit is reached | Điều kiện `while` luôn đúng và không gặp lối thoát

Đáp án: D

Giải thích: Vòng `while` có thể lặp vô hạn khi điều kiện luôn đúng, ví dụ `while n < 3:` không bao giờ thay đổi `n` và không gặp `break`. Cần cập nhật biến hướng tới điều kiện sai hoặc chủ động có lối thoát.

Q18. How can a loop ignore invalid data and continue processing? | Làm sao bỏ qua dữ liệu lỗi nhưng vẫn xử lý phần tử sau?

A. Use `continue` for that invalid item | Dùng `continue` với phần tử lỗi

B. Use `return` to exit the whole function | Dùng `return` thoát toàn bộ hàm

C. Use only `pass` in the invalid-data branch | Chỉ dùng `pass` trong nhánh dữ liệu lỗi

D. Use `break` when the invalid item appears | Dùng `break` khi gặp phần tử lỗi

Đáp án: A

Giải thích: Dùng `continue` khi cần bỏ qua một phần tử không hợp lệ nhưng vẫn xử lý các phần tử sau. `break` sẽ kết thúc vòng lặp, còn chỉ dùng `pass` không ngăn những lệnh phía sau xử lý phần tử lỗi.

Q19. How can a search stop immediately after a match? | Làm sao dừng tìm kiếm ngay khi đã thấy phần tử phù hợp?

A. Use `pass` after the match | Dùng `pass` sau khi khớp

B. Use `continue` after the match | Dùng `continue` sau khi khớp

C. Use `else` instead of checking | Dùng `else` thay kiểm tra

D. Use `break` after the match | Dùng `break` sau khi khớp

Đáp án: D

Giải thích: Dùng `break` ngay khi tìm thấy phần tử cần tìm vì không cần kiểm tra những phần tử còn lại. `continue` chỉ chuyển tới phần tử tiếp theo, còn `pass` không dừng việc tìm kiếm.

Q20. How do you traverse every combination of two small collections? | Làm sao duyệt mọi kết hợp của hai nhóm nhỏ?

A. Use `enumerate()` on one collection | Dùng `enumerate()` trên một nhóm

B. Use two separate, nonnested loops | Dùng hai vòng độc lập không lồng nhau

C. Use one loop nested inside the other | Đặt một vòng lồng bên trong vòng còn lại

D. Use `zip()` on the two collections | Dùng `zip()` trên hai nhóm

Đáp án: C

Giải thích: Muốn duyệt mọi cặp kết hợp của hai nhóm, đặt vòng duyệt nhóm thứ hai bên trong vòng duyệt nhóm thứ nhất. Với 2 phần tử ngoài và 3 phần tử trong, ta được 6 cặp; `zip()` chỉ ghép phần tử cùng vị trí.

8. Nối sang bài tiếp theo

`Điều kiện → Vòng lặp → Xử lý hàng loạt.`

PY-06 — List, chỉ số, cắt danh sách và sắp xếp

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: List — danh sách; **Index** — chỉ số.

`Vòng lặp → List → Vector/Array`

Học xong dùng để làm gì? Lưu và xử lý danh sách điểm, đơn hàng hoặc sản phẩm theo thứ tự; thêm, xóa, sửa và duyệt từng phần tử.

Kết nối kiến thức tiếp theo: PY-07 dùng Tuple cho dữ liệu cố định và Set để bỏ giá trị lặp.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
List	danh sách	Tập hợp có thứ tự, có thể thay đổi
Index	chỉ số	Vị trí tính từ 0 để lấy phần tử
Slice	cắt lát	Lấy đoạn phần tử theo start:stop
Mutability	khả năng sửa	Có thể cập nhật phần tử của danh sách

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`append(), extend(), insert()`	Thêm từng phần tử hoặc nhiều phần tử vào danh sách
`pop(), remove(), del`	Xóa theo vị trí hoặc giá trị
`sort(), sorted()`	Sắp xếp tại chỗ hoặc tạo danh sách mới
`items[start:stop]`	Lấy vùng con không bao gồm stop

4. Khái niệm liên quan

Một cửa hàng cần ghi các món trong giỏ theo đúng thứ tự khách thêm vào. **List** là cấu trúc Python lưu nhiều giá trị trong một biến, được viết bằng dấu ngoặc vuông `[]`. Từ giỏ hàng này, ta lần lượt học cách tạo danh sách, tìm theo vị trí, lấy một đoạn và thay đổi một món; đó cũng là nền tảng để duyệt dữ liệu bằng vòng `for`.

4.1. List — Danh sách

Từ vựng trong mục: List — danh sách; **Element** — phần tử.

List giữ các phần tử theo thứ tự, và mỗi phần tử có thể là tên món hàng, số hoặc một kiểu dữ liệu khác. Giỏ hàng ban đầu gồm `milk` (sữa) và `bread` (bánh mì); ta đặt cả hai vào biến `basket` để xử lý cùng nhau. Dùng dấu `[]` để tạo danh sách, dấu phẩy ngăn cách các phần tử.

No Note — Không chú thích

```
basket = ['milk', 'bread']
print(basket)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo list giỏ hàng lưu nhiêù món theo thứ tự.
# Dữ liệu: List giữ hai chuôĩ tên món, có thứ tự và có thể sửả.
basket = ['milk', 'bread']
# print() in toàn bộ list theo thứ tự và dạng ngoặc vuông.
print(basket)
```

Kết quả khi chạy:

```
['milk', 'bread']
```

Danh sách có hai phần tử theo thứ tự sữa trước, bánh mì sau. Khác với một chuỗi dài, từng món có thể được truy cập và thay đổi riêng.

4.2. Index — Vị trí phần tử

Từ vựng trong mục: Index — chỉ số; **Element** — phần tử.

Khi biết món cần lấy nằm ở vị trí nào, ta dùng **chỉ số**. Python bắt đầu đếm từ `0`: với `['milk', 'bread']`, `basket[0]` là `milk`, còn `basket[1]` là `bread`; `basket[-1]` cũng chỉ phần tử cuối. Chỉ số nằm ngoài giới hạn danh sách sẽ gây `IndexError`.

No Note — Không chú thích

```
basket = ['milk', 'bread']
print(basket[0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Truy cập phần tử đầu tiên của list bằng chỉ số.
# Dữ liệu: Hai món đứng ở vị trí 0 và 1.
basket = ['milk', 'bread']
# [0] lấy phần tử đầu tiên vì chỉ số list bắt đầu từ 0.
print(basket[0])
```

Kết quả khi chạy:

```
milk
```

Chỉ số `0` lấy đúng món sữa ở vị trí đầu tiên, không phải phần tử thứ hai.

4.3. Slice — Lấy một đoạn danh sách

Từ vựng trong mục: Slice — cắt lát; **Start/stop** — vị trí bắt đầu/kết thúc.

Nếu giỏ có ba món `milk`, `bread`, `rice` và chỉ muốn lấy hai món đầu, ta dùng `basket[:2]`. Với lát cắt `[start:stop]`, vị trí `start` được lấy, còn `stop` **không được lấy**. Bỏ trống `start` nghĩa là bắt đầu từ đầu danh sách. Lát cắt tạo list mới, không xóa phần tử khỏi giỏ gốc.

No Note — Không chú thích

```
basket = ['milk', 'bread', 'rice']
print(basket[:2])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy một lát cắt gồm hai món đầu danh sách.
# Dữ liệu: Ba món trong list, thao tác cắt không làm thay đổi list.
basket = ['milk', 'bread', 'rice']
# [:2] lấy phần tử ở vị trí 0 và 1, loại trừ cận phải 2; tạo list mới.
print(basket[:2])
```

Kết quả khi chạy:

```
['milk', 'bread']
```

Kết quả gồm sữa và bánh mì; `rice` ở chỉ số `2` nằm ngoài đoạn được lấy.

4.4. Mutability — Thay đổi nội dung danh sách

Từ vựng trong mục: Mutability — khả năng thay đổi; **Assignment** — phép gán.

Khác với tuple ở bài kế tiếp, **List là kiểu có thể sửa** sau khi tạo. Nếu cửa hàng đổi món thứ hai từ `bread` thành `rice`, ta gán trực tiếp `basket[1] = 'rice'`. Danh sách vẫn có hai phần tử, chỉ giá trị tại vị trí 1 được thay. Muốn thêm món mới vào cuối thì dùng `append()`, không phải phép gán này.

No Note — Không chú thích

```
basket = ['milk', 'bread']
basket[1] = 'rice'
print(basket)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thay đổi món ở vị trí thứ hai trong list.
# Dữ liệu: Thay món thứ hai của hai món có sẵn.
basket = ['milk', 'bread']
# [1] là món thứ hai; phép gán thay món này trực tiếp trong list gốc.
basket[1] = 'rice'
print(basket)
```

Kết quả khi chạy:

```
['milk', 'rice']
```

Món thứ hai đổi thành gạo; món sữa đầu tiên vẫn giữ nguyên.

4.5. Sorting — Sắp xếp List

Từ vựng trong mục: Sorting — sắp xếp; **Ascending/descending** — tăng dần/giảm dần.

Khi có điểm ưu tiên `[7, 9, 4]`, ta thường muốn xem điểm cao trước. Hàm `sorted(scores, reverse=True)` tạo **List mới** theo thứ tự giảm dần; `reverse=False` (mặc định) cho thứ tự tăng dần. List ban đầu không bị sửa bởi `sorted()`, khác với phương thức `scores.sort()` vốn sắp xếp trực tiếp List gốc. Với ví dụ này, điểm 9 đứng đầu, rồi 7 và 4.

No Note — Không chú thích

```
scores = [7, 9, 4]
ranked = sorted(scores, reverse=True)
print(ranked)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo danh sách điểm xếp giảm dần mà giữ list gốc.
# Dữ liệu: Ba điểm cần xếp cao xuống thấp.
scores = [7, 9, 4]
# sorted() tạo list mới, không đổi scores; reverse=True xếp từ điểm lớn xuống nhỏ.
ranked = sorted(scores, reverse=True)
print(ranked)
```

Kết quả khi chạy:

```
[9, 7, 4]
```

Danh sách kết quả được sắp theo điểm giảm dần. Các điểm số không bị thay đổi, chỉ thứ tự của chúng khác đi.

5. Năm tình huống List — tăng độ khó từ thao tác đến hàng đợi

Tình huống 1 — Cấp 1: Chỉnh giỏ hàng 10 sản phẩm

Từ vựng trong mục: List — danh sách; **Append** — thêm cuối; **Index** — chỉ số.

Tình huống thực tế: Ở quầy thanh toán, nhân viên đã quét **10 mặt hàng**, mỗi tên nằm trong danh sách theo đúng thứ tự quét, bắt đầu từ `Gạo` và kết thúc bằng `Táo`. Khách muốn mua thêm một món `Bánh` trước khi chốt hóa đơn. Nhân viên cần biết giỏ có bao nhiêu món trước và sau khi thêm, và tên món nằm cuối danh sách mới; không được làm mất mười món ban đầu.

Vì sao chọn cách này? List là một danh sách có thứ tự: vị trí của tên cho biết món nào được quét trước. `append('Bánh')` thêm một phần tử vào **cuối** danh sách, không thay thế các món cũ. `len()` đếm số phần tử, vì thế sau khi thêm phải nhiều hơn ban đầu đúng một món; dùng chỉ số `[-1]` để đọc món cuối. Không cần sắp xếp theo chữ cái, vì khách muốn giữ thứ tự quét để đối chiếu hóa đơn.

Quy trình:

1. Lưu đúng 10 tên món trong list.

2. Ghi độ dài ban đầu và thêm sản phẩm bằng `append()`.
3. Đối chiếu độ dài tăng đúng một và đọc phần tử cuối.

No Note — Không chú thích

```
basket = ["Gạo", "Sữa", "Trứng", "Dâu", "Muối", "Đường", "Mì", "Cam", "Bơ", "Táo"]
before = len(basket)
basket.append("Bánh")
if before != 10 or len(basket) != before + 1:
    raise ValueError("Danh sách phải bắt đầu với 10 món và tăng đúng một món.")
print("Ban đầu:", before)
print("Sau thêm:", len(basket))
print("Món cuối:", basket[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thêm món mới vào giỏ có 10 món và kiểm sổ´lượng.
# Dữ liệu: 10 món hàng, mỗi phần tử là tên một món.
basket = ["Gạo", "Sữa", "Trứng", "Dâu", "Muối", "Đường", "Mì", "Cam", "Bơ", "Táo"]
# len() trả sổ´phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
before = len(basket)
# append() thêm một món vào cuối list gốc.
basket.append("Bánh")
# Kiểm tra sổ´phần tử đã tăng đúng một sau thao tác append().
if before != 10 or len(basket) != before + 1:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Danh sách phải bắt đầu với 10 món và tăng đúng một món.")
print("Ban đầu:", before)
# len() trả sổ´phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Sau thêm:", len(basket))
# [-1] lấy món cuối cùng mới thêm.
print("Món cuối:", basket[-1])
```

Kết quả khi chạy:

```
Ban đầu: 10
Sau thêm: 11
Món cuối: Bánh
```

Đầu vào có 10 bản ghi sản phẩm; danh sách sau thao tác có 11 món vì khách thêm đúng một mặt hàng.

Tình huống 2 — Cấp 2: Lấy ba yêu cầu quan trọng nhất từ 12 phiếu

Từ vựng trong mục: Sort — sắp xếp; **Slice** — cắt lát; **Stable** — ổn định thứ tự.

Tình huống thực tế: Đội sửa chữa có **12 phiếu yêu cầu T01 đến T12** theo thứ tự tiếp nhận. Mỗi phiếu được chấm một mức độ gấp từ **1 đến 10**, số càng lớn thì cần xử lý sớm; nhiều phiếu có thể bằng điểm. Quản lý muốn chọn ba phiếu cần làm trước, nhưng khi đồng điểm phải ưu tiên phiếu gửi tới sớm hơn và vẫn giữ nguyên bảng điểm ban đầu để đối chiếu.

Vì sao chọn cách này? Mức ưu tiên là số do đội kỹ thuật quy ước để biểu thị độ gấp, không phải số lần phiếu được gửi. Trước tiên ghép mỗi điểm với vị trí phiếu để biết phiếu nào đến trước. `sorted()` tạo danh sách **được sắp xếp mới**, không sửa điểm gốc: xếp điểm từ lớn xuống nhỏ, nếu điểm bằng nhau thì dùng thứ tự nhận từ nhỏ đến lớn. Cuối cùng `[:3]` chỉ lấy ba phiếu đầu; lấy ba dòng đầu của danh sách chưa sắp xếp sẽ chọn theo giờ đến chứ không đúng mức gấp.

Quy trình:

1. Đánh số phiếu theo vị trí list (bắt đầu từ 1).
2. Tạo danh sách `(điểm âm, thứ tự, mã phiếu)` để sắp xếp từ ưu tiên cao xuống.
3. Lấy ba phiếu đầu; kiểm tra list điểm nguồn vẫn giữ thứ tự.

No Note — Không chú thích

```
priorities = [4,9,7,9,3,8,6,10,2,10,5,8]
ranked = []
for position, score in enumerate(priorities, 1):
    ranked.append((-score, position, f"T{position:02d}"))
ranked = sorted(ranked)
top_three = []
for row in ranked[:3]:
    top_three.append(row[2])
if len(priorities) != 12:
    raise ValueError("Cần đủ 12 phiếu ưu tiên.")
if priorities[0] != 4:
    raise ValueError("Danh sách gốc đã bị thay đổi ở phiếu T01.")
print("Phiếu:", len(priorities))
print("Ba phiếu trước:", ", ".join(top_three))
print("Điểm gốc T01:", priorities[0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xếp 12 phiếu theo điểm ưu tiên giảm dần và thứ tự đến.
# Dữ liệu: 12 phiếu có điểm ưu tiên; nếu bằng điểm, phiếu đến trước xếp trước.
priorities = [4,9,7,9,3,8,6,10,2,10,5,8]
ranked = []
# enumerate(...,1) ghép điểm với thứ tự phiếu từ 1.
for position, score in enumerate(priorities, 1):
    # Tuple (-score, position, mã): dấu âm làm điểm cao được xếp trước khi sort tăng; position phá
hòa ưu tiên đến trước.
    ranked.append((-score, position, f"T{position:02d}"))
# sorted() so từng phần tử tuple theo thứ tự từ trái sang phải; không chỉnh list nguồn priorities.
ranked = sorted(ranked)
top_three = []
# [:3] lấy ba tuple đứng đầu, không bao gồm vị trí 3.
for row in ranked[:3]:
    # [2] đọc mã phiếu (trường thứ ba) trong tuple đã xếp.
    top_three.append(row[2])
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(priorities) != 12:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 12 phiếu ưu tiên.")
# [0] xác minh điểm đầu danh sách gốc không bị sắp lại.
if priorities[0] != 4:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Danh sách gốc đã bị thay đổi ở phiếu T01.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Phiếu:", len(priorities))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Ba phiếu trước:", ", ".join(top_three))
# [0] xác minh điểm đầu danh sách gốc không bị sắp lại.
print("Điểm gốc T01:", priorities[0])
```

Kết quả khi chạy:

```
Phiếu: 12
Ba phiếu trước: T08, T10, T02
Điểm gốc T01: 4
```

Dùng chỉ số gốc để phân xử hòa điểm giúp kết quả ổn định; không làm mất thông tin phiếu nào có điểm 10.

Tình huống 3 — Cấp 3: Lọc 15 điểm và chọn nhóm cần hỗ trợ

Từ vựng trong mục: Comprehension — tạo list ngắn gọn; **Slicing** — cắt lát; **Threshold** — ngưỡng.

Tình huống thực tế: Giáo viên có điểm cuối kỳ của **15 học viên H01 đến H15**, ghi theo thứ tự lớp và chấm trên thang **10 điểm**. Điểm từ **5 trở lên** là đạt, dưới 5 cần hỗ trợ. Giáo viên muốn biết mã học viên đạt, có bao nhiêu người chưa đạt, và ba học viên chưa đạt xuất hiện sớm nhất trong danh sách để liên hệ trước.

Vì sao chọn cách này? Vòng `for` xét lần lượt từng điểm cùng mã học viên tương ứng. Điều kiện `score >= 5` đưa người đạt vào một danh sách, còn các điểm thấp hơn vào danh sách cần hỗ trợ. `len()` đếm nhóm chưa đạt, còn `[:3]` lấy ba mã đầu của chính nhóm đó, theo thứ tự lớp. Không được sắp điểm từ thấp nhất rồi lấy ba người nếu yêu cầu là **ba người xuất hiện trước**; đây là hai cách chọn khác nhau.

Quy trình:

1. Dùng `enumerate(..., 1)` để liên kết điểm với mã học viên.
2. Lọc hai nhóm bằng điều kiện từ 5 trở lên hoặc dưới 5.
3. In số lượng, ba mã cần hỗ trợ và kiểm tra tổng vẫn bằng 15.

No Note — Không chú thích

```
grades = [4.5,7,8,3,9,5,7.5,4,6.5,10,8,2,7,5.5,9.5]
passed = []
needs_help = []
for i, grade in enumerate(grades, 1):
    student_code = f"H{i:02d}"
    if grade >= 5:
        passed.append(student_code)
    else:
        needs_help.append(student_code)
if len(grades) != 15:
    raise ValueError("Cần đủ 15 điểm học viên.")
if len(passed) + len(needs_help) != 15:
    raise ValueError("Hai nhóm điểm phải bao phủ đủ 15 học viên.")
print("Số học viên:", len(grades))
print("Đạt:", len(passed))
print("Cần hỗ trợ:", len(needs_help))
print("Gọi trước:", ", ".join(needs_help[:3]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chia 15 điểm thành đạt hoặc cần hỗ trợ và lấy ba học viên cần hỗ trợ đầu.
# Dữ liệu: 15 điểm học viên, ngưỡng đạt là 5/10.
grades = [4.5,7,8,3,9,5,7.5,4,6.5,10,8,2,7,5.5,9.5]
passed = []
needs_help = []
# enumerate(...,1) ghép mỗi điểm với mã học viên 1..15.
for i, grade in enumerate(grades, 1):
    # Định dạng f-string :02d cho số thứ tự thành 2 chữ số với số 0 bổ sung.
    student_code = f"H{i:02d}"
```

```
    # Điểm từ 5 trở lên vào nhóm đạt, còn lại cần hỗ trợ.
    if grade >= 5:
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        passed.append(student_code)
    else:
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        needs_help.append(student_code)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(grades) != 15:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 15 điểm học viên.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(passed) + len(needs_help) != 15:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Hai nhóm điểm phải bao phủ đủ 15 học viên.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Số học viên:", len(grades))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Đạt:", len(passed))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Cần hỗ trợ:", len(needs_help))
# [:3] lấy tối đa ba mã đầu trong nhóm cần hỗ trợ.
print("Gọi trước:", ", ".join(needs_help[:3]))
```

Kết quả khi chạy:

```
Số học viên: 15
Đạt: 11
Cần hỗ trợ: 4
Gọi trước: H01, H04, H08
```

Điểm chính xác bằng 5 thuộc nhóm đạt. Ở cấp này yêu cầu kết hợp lọc, đếm và lấy một lát cắt có ý nghĩa.

Tình huống 4 — Cấp 4: Sửa tồn kho 18 SKU nhưng bảo toàn bản kiểm kê

Từ vựng trong mục: Mutable — có thể thay đổi; **Copy** — sao chép; **Missing** — dữ liệu thiếu.

Tình huống thực tế: Kho lưu một bản kiểm kê của **18 mã hàng K01 đến K18**; mỗi vị trí chứa số món còn lại, riêng K18 chưa đo được nên ghi `None` (chưa có dữ liệu). Sau khi kiểm tra lại, nhân viên phát hiện số lượng K02 cần sửa từ 6 thành 4 và K08 từ 7 thành 5. Quản lý muốn tạo bản số liệu đã sửa để dùng tiếp, nhưng vẫn giữ nguyên bản kiểm kê đầu tiên làm chứng cứ, đồng thời báo các thay đổi và mã còn thiếu số liệu.

Vì sao chọn cách này? `original` là **bản ghi gốc**, cần giữ lại để so với số liệu sau sửa. Nếu dùng `edited = original`, hai tên cùng chỉ tới **một list**, sửa bản mới sẽ vô tình sửa cả bản gốc. `copy()` tạo một list khác rồi mới thay số của K02 và K08. Vì chỉ số Python bắt đầu từ **0**, K02 ở vị trí 1, K08 ở vị trí 7. `None` của K18 phải được giữ là **chưa có số lượng**, không tự đổi thành 0 món vì 0 có nghĩa là kiểm đếm thấy hết hàng.

Quy trình:

1. Tạo list 18 giá trị và sao chép bằng `copy()`.
2. Sửa đúng hai vị trí sau khi kiểm kê và nhận diện dữ liệu thiếu.
3. In dữ liệu trước/sau, số lượt thiếu; xác nhận list cũ chưa bị sửa.

No Note — Không chú thích

```
original = [8,6,9,5,11,4,12,7,3,10,6,9,8,5,7,4,0,None]
updated = original.copy()
```

```
updated[1] = 4
updated[7] = 5
missing = []
for i, value in enumerate(updated, 1):
    if value is None:
        missing.append(f"K{i:02d}")
if len(original) != 18 or len(updated) != 18:
    raise ValueError("Cần đủ 18 mục tồn kho trong cả hai danh sách.")
if original[1] != 6 or original[7] != 7:
    raise ValueError("Sổ kiểm kê gốc không được thay đổi.")
print("SKU:", len(updated))
print("K02 cũ/mới:", original[1], updated[1])
print("K08 cũ/mới:", original[7], updated[7])
print("Chưa kiểm kê:", ", ".join(missing))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Sao chép sổ tồn 18 SKU, chỉnh hai kệ mà không đổi bản gốc.
# Dữ liệu: 18 số tồn theo SKU K01..K18, None là chưa có kiểm kê.
original = [8,6,9,5,11,4,12,7,3,10,6,9,8,5,7,4,0,None]
# copy() tạo bản sao list độc lập theo từng phần tử; sửa updated không đổi original (giá trị đơn
giản int/None).
updated = original.copy()
# [1] ứng với SKU K02, không phải K01 vì chỉ số bắt đầu 0.
updated[1] = 4  # K02
# [7] ứng với SKU K08.
updated[7] = 5  # K08
missing = []
# enumerate(...,1) tạo mã kệ K01..K18 theo vị trí.
for i, value in enumerate(updated, 1):
    # is None nhận bản ghi chưa kiểm kê; không nhầm tồn kho 0 là thiếu.
    if value is None:
        # append() ghi lại mã kệ chưa có dữ liệu.
        missing.append(f"K{i:02d}")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(original) != 18 or len(updated) != 18:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 18 mục tồn kho trong cả hai danh sách.")
if original[1] != 6 or original[7] != 7:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Sổ kiểm kê gốc không được thay đổi.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("SKU:", len(updated))
# [1] ứng với SKU K02, không phải K01 vì chỉ số bắt đầu 0.
print("K02 cũ/mới:", original[1], updated[1])
# [7] ứng với SKU K08.
print("K08 cũ/mới:", original[7], updated[7])
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Chưa kiểm kê:", ", ".join(missing))
```

Kết quả khi chạy:

```
SKU: 18
K02 cũ/mới: 6 4
K08 cũ/mới: 7 5
Chưa kiểm kê: K18
```

`None` không có nghĩa tồn kho bằng 0; cấp này đòi hỏi giữ nguyên chứng cứ, sửa đúng chỉ số và không lặng lẽ thay số đo thiếu.

Tình huống 5 — Cấp 5: Xây hàng đợi 20 phiếu theo mức độ ưu tiên và FIFO

Từ vựng trong mục: Queue — hàng đợi; **Stable sort** — sắp xếp ổn định; **Validation** — kiểm tra.

Tình huống thực tế: Bàn hỗ trợ có **20 phiếu R01 đến R20**, xếp theo thời điểm gửi đến. Mỗi phiếu đã được đánh mức gấp từ **1 đến 3**, trong đó 3 là gấp nhất; riêng R12 chưa được chấm nên ghi `None`. Trưởng nhóm muốn biết năm phiếu đủ thông tin cần xử lý đầu tiên và số phiếu phải chờ xác nhận. Khi hai phiếu có cùng mức gấp, phiếu gửi trước phải đứng trước.

Vì sao chọn cách này? FIFO nghĩa là *đến trước, xử lý trước*; ở đây quy tắc ấy chỉ áp dụng khi các phiếu **bằng mức ưu tiên**. Mỗi phiếu đủ dữ liệu được ghép với số thứ tự nhận. Khi sắp, dùng mức ưu tiên âm để đưa mức 3 lên trước mức 2 và 1, rồi dùng thứ tự nhận tăng dần để xử lý hòa điểm. Phiếu R12 có `None` phải tách vào nhóm chờ: tự đổi thành mức 0 sẽ khiến người xem tưởng phiếu đã được đánh giá. Slicing `[:5]` chỉ chọn năm việc đầu, không xóa các việc khác trong hàng đợi.

Quy trình:

1. Tạo 20 bản ghi vị trí–mức ưu tiên và xác nhận hàng R12 thiếu điểm.
2. Lọc phiếu có ưu tiên hợp lệ; tạo khóa ưu tiên giảm và vị trí tới tăng.
3. Sắp xếp thành hàng đợi xử lý, đối chiếu số dòng và in năm phiếu đầu.

No Note — Không chú thích

```
priorities = [3,1,2,3,2,1,3,2,1,3,2,None,1,3,2,1,3,2,1,2]
queue = []
for i, value in enumerate(priorities, 1):
    queue.append((f"R{i:02d}", value))
missing = []
ready = []
for arrival, (code, level) in enumerate(queue):
    if level is None:
        missing.append(code)
    else:
        ready.append((-level, arrival, code))
ready.sort()
next_five = []
for neg_level, arrival, code in ready[:5]:
    next_five.append(code)
if len(queue) != 20:
    raise ValueError("Cần đủ 20 phiếu hỗ trợ.")
if len(ready) + len(missing) != 20:
    raise ValueError("Các phiếu sẵn sàng và thiếu điểm phải đủ 20.")
print("Tổng phiếu:", len(queue))
print("Sẵn sàng:", len(ready))
print("Thiếu ưu tiên:", ", ".join(missing))
print("Xử lý trước:", ", ".join(next_five))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Sắp xếp 20 yêu cầu hỗ trợ theo mức ưu tiên, bỏ phiếu thiếu.
# Dữ liệu: 20 mức ưu tiên 1..3; None là chưa có mức; thứ tự tiếp nhận dùng khi đồng hạng.
priorities = [3,1,2,3,2,1,3,2,1,3,2,None,1,3,2,1,3,2,1,2]
queue = []
# Đánh mã 20 yêu cầu R01..R20 từ vị trí 1.
for i, value in enumerate(priorities, 1):
    # append() ghép mã và mức ưu tiên vào một tuple.
    queue.append((f"R{i:02d}", value))
missing = []
ready = []
```

```
# enumerate() không truyền start mặc định từ 0 để lưu thứ tự đến.
for arrival, (code, level) in enumerate(queue):
    # is None tìm phiếu thiếu mức ưu tiên, không đưa vào hàng xử lý.
    if level is None:
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        missing.append(code)
    else:
        # Lưu tuple (-mức, thứ tự đến, mã); mức lớn có số âm nhỏ hơn nên đứng trước.
        ready.append((-level, arrival, code))
# sort() sắp xếp list tại chỗ; tuple so từ trường thứ nhất đến thứ hai giúp giải quyết đồng hạng.
ready.sort()
next_five = []
# [:5] lấy đúng năm phiếu đầu sau khi xếp (bỏ cận phải 5).
for neg_level, arrival, code in ready[:5]:
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    next_five.append(code)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(queue) != 20:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 20 phiếu hỗ trợ.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(ready) + len(missing) != 20:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Các phiếu sẵn sàng và thiếu điểm phải đủ 20.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Tổng phiếu:", len(queue))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Sẵn sàng:", len(ready))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Thiếu ưu tiên:", ", ".join(missing))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Xử lý trước:", ", ".join(next_five))
```

Kết quả khi chạy:

```
Tổng phiếu: 20
Sẵn sàng: 19
Thiếu ưu tiên: R12
Xử lý trước: R01, R04, R07, R10, R14
```

Mỗi vị trí trong list là thời điểm nhận phiếu; dùng thêm vị trí như khóa phụ giúp minh bạch vì sao một phiếu cùng ưu tiên phải được xử lý trước phiếu khác.

6. Nhầm lẫn dễ gặp

- Chỉ số list bắt đầu từ `0`; truy cập vượt ngoài list gây `IndexError`.
- `append([3,4])` thêm cả list như một phần tử, còn `extend([3,4])` thêm lần lượt `3` và `4`.
- `list.sort()` sửa list gốc và trả về `None`; `sorted(list)` tạo list mới.
- `pop(0)` sẽ lỗi nếu list rỗng. Với hàng đợi lớn, nên dùng `collections.deque`.
- Cắt list bằng `[:2]` không lấy phần tử có chỉ số `2`.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a Python List? | List trong Python là gì?

A. A mapping of keys to values | Ánh xạ từ khóa sang giá trị

B. An ordered, mutable collection | Tập hợp có thứ tự và có thể sửa

C. An unordered collection of unique items | Tập hợp không thứ tự gồm phần tử duy nhất

D. An immutable sequence of items | Dãy phần tử bất biến

Đáp án: B

Giải thích: List trong Python là tập hợp có thứ tự và có thể thay đổi, nên mỗi phần tử có vị trí và có thể được sửa sau khi tạo. Ví dụ `[10, 20]` chứa hai phần tử theo thứ tự xác định; set không hỗ trợ chỉ số vị trí.

Q2. What is the index of the first list element? | Chỉ số của phần tử List đầu tiên là gì?

A. `0` | `0`

B. `-1` | `-1`

C. `len(a)` | `len(a)`

D. `1` | `1`

Đáp án: A

Giải thích: Chỉ số List bắt đầu từ `0`, vì vậy `a[0]` lấy phần tử đầu và `a[1]` lấy phần tử thứ hai. Chỉ số âm `-1` mang nghĩa khác: phần tử cuối.

Q3. For a List `a`, what does `a[:2]` return? | Với `a` là một List, `a[:2]` trả về gì?

A. Items from index 2 to the end | Các phần tử từ chỉ số 2 đến cuối

B. A new list of the first two items | List mới gồm hai phần tử đầu

C. A new list of the last two items | List mới gồm hai phần tử cuối

D. Only item at index 2 | Chỉ phần tử tại chỉ số 2

Đáp án: B

Giải thích: Lát cắt `a[:2]` mặc định bắt đầu tại chỉ số 0 và dừng trước chỉ số 2, tạo List mới gồm hai phần tử đầu. Với `[4, 5, 6]`, kết quả là `[4, 5]`, không phải phần tử tại chỉ số 2.

Q4. What does 'mutable' mean for a list? | List có tính mutable nghĩa là gì?

A. Its contents can be changed after creation | Có thể sửa nội dung sau khi tạo

B. Its elements cannot be changed | Không thể sửa các phần tử

C. Every element must have the same type | Mọi phần tử phải cùng kiểu

D. Its elements sort automatically | Các phần tử được tự động sắp xếp

Đáp án: A

Giải thích: Mutable nghĩa là có thể thay, thêm hoặc xóa phần tử trên List hiện có. Ví dụ gán `a = [1, 2]` rồi thực hiện `a[0] = 9` làm `a` thành `[9, 2]`; tuple không cho phép gán phần tử như vậy.

Q5. What does `append(x)` do? | `append(x)` làm gì?

A. Inserts `x` before the first item | Chèn `x` trước phần tử đầu

B. Removes the final element and returns it | Xóa phần tử cuối và trả về

C. Adds `x` as a single final item | Thêm `x` thành một phần tử cuối

D. Adds every element of `x` separately | Thêm riêng từng phần tử của `x`

Đáp án: C

Giải thích: `append(x)` thêm toàn bộ `x` thành đúng một phần tử cuối List và sửa List tại chỗ. Vì vậy thêm `[2, 3]` bằng `append` vào `[1]` tạo phần tử List lồng nhau, không thêm từng số; muốn thêm từng số thì dùng `extend()`.

Q6. What does `extend(items)` do? | `extend(items)` làm gì?

A. Deletes items matching `items` | Xóa phần tử khớp `items`

B. Appends the iterable as one nested item | Thêm cả iterable thành một phần tử lồng

C. Adds the individual elements from `items` | Thêm từng phần tử từ `items`

D. Replaces the first element with `items` | Thay phần tử đầu bằng `items`

Đáp án: C

Giải thích: `extend(items)` duyệt đối số và thêm từng phần tử riêng vào List hiện có. Với `a = [1]`, `a.extend([2, 3])` tạo `[1, 2, 3]`, còn `append([2, 3])` sẽ tạo `[1, [2, 3]]`.

Q7. What does `insert(0, x)` do? | `insert(0, x)` làm gì?

A. Replaces the first element with `x` | Thay phần tử đầu bằng `x`

B. Inserts `x` at the front, shifting the others | Chèn `x` vào đầu, đẩy các phần tử khác sang phải

C. Removes the first element | Xóa phần tử đầu

D. Adds `x` at the end | Thêm `x` vào cuối

Đáp án: B

Giải thích: `insert(0, x)` chèn `x` trước phần tử đầu tiên và đẩy các phần tử cũ sang phải. Ví dụ chèn `9` vào đầu `[1, 2]` cho `[9, 1, 2]`, không ghi đè số `1`.

Q8. What does `pop(0)` do on a nonempty list? | `pop(0)` làm gì với List không rỗng?

A. Removes and returns the last item | Xóa và trả phần tử cuối

B. Returns the first item without removing it | Trả phần tử đầu nhưng không xóa

C. Deletes the first occurrence of the value zero | Xóa lần xuất hiện đầu của giá trị không

D. Removes and returns the first item | Xóa và trả phần tử đầu

Đáp án: D

Giải thích: `pop(0)` xóa phần tử đầu của List không rỗng và trả chính phần tử đã xóa. Với `[7, 8]`, kết quả trả về là `7` và List còn `[8]`; chỉ đọc `a[0]` thì không xóa phần tử nào.

Q9. What does `remove(x)` do? | `remove(x)` dùng để làm gì?

A. Deletes every matching value | Xóa tất cả giá trị khớp

B. Returns a sorted copy | Trả bản sao đã sắp xếp

C. Deletes the first matching value | Xóa giá trị khớp đầu tiên

D. Deletes the item at index `x` | Xóa phần tử tại chỉ số `x`

Đáp án: C

Giải thích: `remove(x)` xóa lần xuất hiện đầu tiên có giá trị bằng `x`; nó tìm theo giá trị chứ không xem `x` là chỉ số. Với `[2, 3, 2]`, `remove(2)` còn `[3, 2]`, và giá trị không tồn tại sẽ gây `ValueError`.

Q10. What does `del a[1]` do? | `del a[1]` có tác dụng gì?

A. Returns the removed item | Trả về phần tử đã xóa

B. Deletes the item whose value is `1` | Xóa phần tử có giá trị bằng `1`

C. Deletes the item at index one | Xóa phần tử có chỉ số 1

D. Replaces the item at index one with `None` | Thay phần tử chỉ số 1 bằng `None`

Đáp án: C

Giải thích: `del a[1]` xóa phần tử tại chỉ số 1, tức phần tử thứ hai vì chỉ số bắt đầu từ 0. Khác `remove(1)`, lệnh này dùng vị trí; khác `pop(1)`, câu lệnh `del` không trả về giá trị bị xóa.

Q11. What does `a.sort()` return? | `a.sort()` trả về gì?

A. The sorted indexes | Các chỉ số đã sắp xếp

B. A new sorted list | List mới đã sắp xếp

C. The same list as a return value | Trả về chính List đó

D. `None` | `None`

Đáp án: D

Giải thích: `a.sort()` sắp List tại chỗ và trả về `None`, nên chính List cũ nhận thứ tự mới. Gán `result = a.sort()` sẽ cho `None`, không phải List mới đã sắp; muốn List riêng thì dùng `sorted(a)`.

Q12. What does `sorted(a)` do? | `sorted(a)` làm gì?

A. Reverses the original list in place | Đảo List gốc tại chỗ

B. Produces a set of unique items | Tạo Set các phần tử duy nhất

C. Creates a new sorted list | Tạo List mới đã sắp xếp

D. Sorts `a` in place and returns `None` | Sắp `a` tại chỗ và trả `None`

Đáp án: C

Giải thích: `sorted(a)` trả về List mới đã sắp xếp và không đổi thứ tự List gốc. Nếu `a = [3, 1]`, `sorted(a)` là `[1, 3]` còn `a` vẫn `[3, 1]`, khác `a.sort()`.

Q13. What does `a[-1]` read from a nonempty list? | `a[-1]` lấy gì từ List không rỗng?

A. The last element | Phần tử cuối

B. The first element | Phần tử đầu

C. The number of elements | Số phần tử

D. An `IndexError` for every nonempty list | Luôn có `IndexError` với List không rỗng

Đáp án: A

Giải thích: Chỉ số `-1` chỉ phần tử cuối của List không rỗng, bất kể List dài bao nhiêu. Với `[4, 7, 9]`, `a[-1]` là `9`; còn `len(a)` đếm đủ ba phần tử.

Q14. What does `len(a)` report? | `len(a)` cho biết gì?

A. The number of distinct values | Số giá trị khác nhau

B. The list's memory size in bytes | Kích thước List tính theo byte

C. The final valid index | Chỉ số hợp lệ cuối

D. The number of elements | Số phần tử

Đáp án: D

Giải thích: `len(a)` trả về số phần tử ở cấp ngoài của List; một List lồng trong được tính là một phần tử. Với `[1, [2, 3]]`, độ dài là `2`, không phải `3` và không phải chỉ số cuối `1`.

Q15. What happens for `a[10]` on a two-item list? | `a[10]` sẽ thế nào nếu List có hai phần tử?

A. The last list element | Phần tử cuối List

B. `None` | `None`

C. The integer `0` | Số nguyên `0`

D. `IndexError` | `IndexError`

Đáp án: D

Giải thích: List gồm hai phần tử chỉ có chỉ số dương hợp lệ `0` và `1`; đọc `a[10]` vượt giới hạn nên gây `IndexError`. Python không tự trả `None` hoặc mở rộng List khi đọc chỉ số không có.

Q16. What does `a[1] = 5` do? | `a[1] = 5` làm gì?

A. Changes the first item to `5` | Thay phần tử đầu thành `5`

B. Replaces the second element with `5` | Thay phần tử thứ hai bằng `5`

C. Inserts a new element at index `1` | Chèn phần tử mới tại chỉ số `1`

D. Adds `5` to the end | Thêm `5` vào cuối

Đáp án: B

Giải thích: `a[1] = 5` thay phần tử hiện có tại chỉ số `1` thành `5`, miễn chỉ số đó tồn tại. Với `[8, 9]`, kết quả là `[8, 5]`; lệnh không chèn thêm phần tử hay sửa chỉ số 0.

Q17. Which operation sorts a list in place? | Thao tác nào sắp xếp ngay List gốc?

A. `a.sort()` | `a.sort()`

B. `sorted(a)` | `sorted(a)`

C. `a.reverse()` | `a.reverse()`

D. `reversed(a)` | `reversed(a)`

Đáp án: A

Giải thích: `a.sort()` sắp xếp ngay trên List gốc nên đọc `a` sau đó thấy thứ tự mới. `sorted(a)` trả List khác, còn `reverse()` chỉ đảo vị trí hiện có, không tính thứ tự sắp xếp.

Q18. Which expression sorts without mutating the original? | Biểu thức nào sắp xếp mà không sửa List gốc?

A. `sorted(a)` | `sorted(a)`

B. `a.reverse()` | `a.reverse()`

C. `a.extend(sorted(a))` | `a.extend(sorted(a))`

D. `a.sort()` | `a.sort()`

Đáp án: A

Giải thích: `sorted(a)` tạo List mới theo thứ tự sắp xếp và giữ nguyên `a`. Ngược lại `a.sort()` sửa trực tiếp `a` và trả `None`, nên không thể dùng thay nhau khi cần giữ dữ liệu gốc.

Q19. What does `reverse=True` mean in a numeric sort? | `reverse=True` có ý nghĩa gì khi sắp xếp số?

A. Reverse each number's digits | Đảo chữ số trong từng số

B. Ascending order | Thứ tự tăng dần

C. Random element order | Thứ tự phần tử ngẫu nhiên

D. Descending numeric order | Thứ tự số giảm dần

Đáp án: D

Giải thích: Trong phép sắp số mặc định tăng dần, `reverse=True` đảo thứ tự thành giảm dần. Với `[3, 1, 2]`, `sorted(a, reverse=True)` cho `[3, 2, 1]`, không đơn thuần đảo thứ tự vị trí gốc.

Q20. How do `append([3,4])` and `extend([3,4])` differ? | `append([3,4])` và `extend([3,4])` khác gì?

A. Both operations sort the existing list | Cả hai đều sắp xếp List

B. `append` adds one nested list; `extend` adds two elements | `append` thêm một List con; `extend` thêm hai phần tử

C. Both operations create the same flat list | Cả hai tạo cùng List phẳng

D. `append` adds both separately and `extend` nests them | `append` thêm riêng còn `extend` lồng List

Đáp án: B

Giải thích: `append([3, 4])` thêm nguyên List con thành một phần tử, còn `extend([3, 4])` thêm riêng hai số nguyên. Bắt đầu từ `[1, 2]`, hai kết quả lần lượt là `[1, 2, [3, 4]]` và `[1, 2, 3, 4]`.

8. Nối sang bài tiếp theo

`Vòng lặp → List → Vector/Array`.

PY-07 — Tuple, Set và các phép toán tập hợp

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Tuple — bộ giá trị; **Set** — tập hợp.

`List → Tuple và Set → Tập hợp/Jaccard`

Học xong dùng để làm gì? Giữ cặp tọa độ cố định và lấy các nhãn không trùng nhau, so sánh phần tử chung/riêng của hai nhóm.

Kết nối kiến thức tiếp theo: PY-08 dùng Dictionary để liên kết mã hoặc tên với thông tin chi tiết.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Tuple	bộ giá trị	Dãy có thứ tự không sửa phần tử được
Set	tập hợp	Nhóm phần tử không trùng lặp
Union	phép hợp	Mọi phần tử có trong ít nhất một tập
Intersection	phép giao	Phần tử chung của hai tập

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`tuple(items)`	Lưu một bộ dữ liệu có thứ tự và cố định
`set(items)`	Khử giá trị trùng, không giữ thứ tự đáng tin
`a.union(b)`, `a.intersection(b)`	Lấy toàn bộ phần tử của hai tập (hợp), hoặc chỉ phần tử chung (giao); tương đương `a | b` và `a & b`
`a - b`, `a ^ b`	Tìm phần tử còn thiếu hoặc khác nhau

4. Khái niệm liên quan

Sau List, có hai nhu cầu khác nhau: **giữ một bộ dữ liệu có thứ tự nhưng không sửa trực tiếp** (Tuple), hoặc **chỉ lưu các giá trị không trùng và so sánh hai nhóm** (Set). Hai cấu trúc không thay thế lẫn nhau: tọa độ `(3, 4)` phù hợp với Tuple, còn danh sách kỹ năng không trùng phù hợp với Set. Từ Set, bài học mở rộng sang phép hợp và phép giao.

4.1. Tuple — Bộ giá trị cố định

Từ vựng trong mục: Tuple — bộ giá trị; **Coordinate** — tọa độ.

Tuple chứa các phần tử **có thứ tự**, thường dùng cho một cặp thông tin gắn với nhau như tọa độ `(x, y)`. Với điểm ở vị trí `x = 3`, `y = 4` mét, ta viết `point = (3, 4)`; `point[0]` là x và `point[1]` là y. Không thể gán `point[0] = 5` như List; nếu cần vị trí mới, tạo Tuple mới.

No Note — Không chú thích

```
point = (3, 4)
print(point)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo tuple tọa độ (x,y) của một điểm.
# Dữ liệu: Tuple gồm 2 tọa độ x=3, y=4, không đổi phần tử sau khi tạo.
# Dấu ngoặc tròn tạo tuple cố định gồm hai phần tử 3 và 4.
point = (3, 4)
print(point)
```

Kết quả khi chạy:

```
(3, 4)
```

Dấu ngoặc tròn biểu thị Tuple gồm hai tọa độ, khác dấu ngoặc vuông của List.

4.2. Set — Tập hợp không trùng

Từ vựng trong mục: Set — tập hợp; **Duplicate** — giá trị lặp.

Set giúp lưu **những giá trị khác nhau** mà không cần ghi cùng một giá trị nhiều lần. Một nhóm học ghi ba nhãn `Python`, `AI`, `Python`; khi chuyển sang Set, nhãn `Python` bị trùng chỉ còn một lần. Set không có chỉ số vị trí như `set[0]` và không đảm bảo thứ tự in. `set()` tạo tập rỗng, còn `{}` tạo Dictionary rỗng.

No Note — Không chú thích

```
skills = set(['Python', 'AI', 'Python'])
print(len(skills))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng set bỏ kỹ năng bị lặp và đếm số loại.
# Dữ liệu: Ba chuỗi kỹ năng có 'Python' lặp.
# set() loại giá trị 'Python' trùng; set không lưu nhiều bản sao cùng kỹ năng.
skills = set(['Python', 'AI', 'Python'])
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print(len(skills))
```

Kết quả khi chạy:

```
2
```

Có hai kỹ năng khác nhau: Python và AI. Ta in **số lượng** để không phụ thuộc thứ tự hiển thị của Set.

4.3. Union — Gộp hai nhóm

Từ vựng trong mục: Union — phép hợp; **Set** — tập hợp.

Giả sử nhóm A học `{Python, AI}`, nhóm B học `{AI, SQL}`. Cửa hàng khóa học muốn biết **tất cả kỹ năng có trong ít nhất một nhóm**. Phép hợp `a | b` giữ Python, AI và SQL nhưng chỉ giữ AI một lần. Với Set, ký hiệu `|` là phép hợp tập hợp, không phải toán tử `or` nối điều kiện Boolean.

No Note — Không chú thích

```
a = {'Python', 'AI'}
b = {'AI', 'SQL'}
print(len(a | b))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Hợp hai tập kỹ năng và đếm kết quả không trùng.
# Dữ liệu: Hai tập không trùng, có một phần tử chung 'AI'.
a = {'Python', 'AI'}
b = {'AI', 'SQL'}
# | hợp hai set: các kỹ năng ở ít nhất một nhóm, không đếm trùng; len() đếm số phần tử.
print(len(a | b))
```

Kết quả khi chạy:

```
3
```

Phép hợp gồm ba kỹ năng khác nhau; ký hiệu `|` gộp hai Set và tự loại phần trùng.

4.4. Intersection — Tìm phần chung

Từ vựng trong mục: Intersection — phép giao; **Set** — tập hợp.

Vẫn với hai nhóm A có `{Python, AI}` và B có `{AI, SQL}`, ta muốn tìm **kỹ năng cả hai nhóm đều học**. Phép giao `a & b` chỉ giữ AI, vì Python không có ở B và SQL không có ở A. Phép giao khác phép hợp: hợp trả ba kỹ năng, giao chỉ một kỹ năng.

No Note — Không chú thích

```
a = {'Python', 'AI'}
b = {'AI', 'SQL'}
print(a & b)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy phần giao hai tập: kỹ năng xuất hiện ở cả hai.
# Dữ liệu: Hai tập kỹ năng, thứ tự khi in set không cố định.
a = {'Python', 'AI'}
b = {'AI', 'SQL'}
# & lấy giao: chỉ các kỹ năng thuộc đồng thời hai set; thứ tự in set không đảm bảo.
print(a & b)
```

Kết quả khi chạy:

```
{'AI'}
```

Chỉ có AI xuất hiện trong cả hai nhóm; tập chỉ có một phần tử nên thứ tự hiển thị không gây nhầm lẫn.

4.5. Difference — Phần tử còn thiếu trong Set

Từ vựng trong mục: Difference — hiệu tập hợp; **Set** — tập hợp.

Ngoài phép hợp và phép giao, ta còn cần biết **phần tử chỉ có ở nhóm A nhưng chưa có ở nhóm B**. Ví dụ khóa học yêu cầu `Python`, `SQL`, `AI`, còn học viên đã hoàn thành `Python`; hiệu `required - done` giữ `SQL` và `AI`. Dùng `sorted()` nếu muốn in các kỹ năng theo một thứ tự rõ ràng. Hiệu `a - b` khác **hiệu đối xứng** `a ^ b`, vốn lấy các phần tử chỉ thuộc một trong hai nhóm.

No Note — Không chú thích

```
required = {'Python', 'SQL', 'AI'}
done = {'Python'}
print(sorted(required - done))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy kỹ năng còn thiếu bằng phép trừ tập hợp.
# Dữ liệu: required là bộ kỹ năng phải biết, done là bộ đã học.
required = {'Python', 'SQL', 'AI'}
done = {'Python'}
# - lấy hiệu set: phần tử required chưa có trong done; sorted() tạo danh sách xếp chữ cái để hiển
thị ổn định.
print(sorted(required - done))
```

Kết quả khi chạy:

```
['AI', 'SQL']
```

Kết quả là một **List đã sắp xếp**, gồm hai kỹ năng cần học tiếp. Set gốc không bị thay đổi.

5. Năm tình huống Tuple/Set — từ bản ghi đến đối chiếu kỹ năng

Tình huống 1 — Cấp 1: Lưu 10 tọa độ bằng Tuple

Từ vựng trong mục: Tuple — bộ giá trị; **Unpacking** — tách bộ.

Tình huống thực tế: Một robot di chuyển trên sàn và ghi lại **10 vị trí** theo đơn vị **mét**. Mỗi vị trí gồm hai số `(x, y)`: `x` là khoảng dịch sang ngang, `y` là khoảng dịch theo chiều dọc so với điểm xuất phát `(0, 0)`. Nhóm vận hành muốn lưu nguyên từng cặp tọa độ, đọc cặp đầu tiên và đếm bao nhiêu điểm cách điểm xuất phát ít nhất 5 mét.

Vì sao chọn cách này? Tuple là cách giữ hai giá trị đi cùng nhau như một cặp tọa độ, ví dụ `(3, 4)` là vị trí sang ngang 3 mét và đi dọc 4 mét. Có thể viết `x, y = point` để lấy hai số ra; bản thân tuple không cho sửa trực tiếp từng phần tử nên phù hợp với việc giữ nguyên tọa độ ghi nhận. Muốn biết một điểm cách gốc ít nhất 5 mét, so tổng **bình phương hai độ dịch chuyển** với 25; ví dụ `(3, 4)` có các phần 9 và 16,

cộng thành 25 nên vừa chạm ngưỡng. Cách so này tránh phải tính căn bậc hai nhưng vẫn cho đúng kết quả.

Quy trình:

1. Lưu mười tuple tọa độ, mỗi hàng là một vị trí.
2. Tách `x, y` của từng tuple và đếm điểm đạt ngưỡng.
3. In điểm đầu, số quan sát và số điểm đủ xa.

No Note — Không chú thích

```
positions = [(3,4),(0,2),(5,0),(1,1),(2,3),(4,1),(6,0),(1,4),(2,2),(5,5)]
first_x, first_y = positions[0]
far_count = 0
for x, y in positions:
    if x*x + y*y >= 25:
        far_count += 1
if len(positions) != 10:
    raise ValueError("Cần đủ 10 vị trí robot.")
print("Số vị trí:", len(positions))
print("Điểm đầu:", first_x, first_y)
print("Điểm xa:", far_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm trong 10 tọa độ robot có bao nhiêu điểm xa gốc ít nhất 5 đơn vị.
# Dữ liệu: 10 tuple (x,y) vị trí robot, đơn vị tọa độ trong ví dụ.
positions = [(3,4),(0,2),(5,0),(1,1),(2,3),(4,1),(6,0),(1,4),(2,2),(5,5)]
# [0] lấy tọa độ đầu, tách hai phần tuple vào first_x và first_y.
first_x, first_y = positions[0]
far_count = 0
# Tách từng tuple thành x, y để so ngưỡng.
for x, y in positions:
    # Bình phương x,y rồi cộng; >=25 tương đương khoảng cách đến (0,0) ít nhất 5 đơn vị, không cần
lấy căn.
    if x*x + y*y >= 25:
        far_count += 1
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(positions) != 10:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 10 vị trí robot.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Số vị trí:", len(positions))
print("Điểm đầu:", first_x, first_y)
print("Điểm xa:", far_count)
```

Kết quả khi chạy:

```
Số vị trí: 10
Điểm đầu: 3 4
Điểm xa: 4
```

Mỗi tuple là một điểm đo có hai thành phần. Số 25 mang đơn vị m^2 do chương trình so bình phương khoảng cách, không phải khoảng cách 25 mét.

Tình huống 2 — Cấp 2: Khử email trùng sau chuẩn hóa 12 bản ghi

Từ vựng trong mục: Set — tập hợp; **Duplicate** — trùng; **Normalization** — chuẩn hóa.

Tình huống thực tế: Một tổ chức chuẩn bị gửi thông báo cho **12 dòng email thử nghiệm**. Mỗi dòng là email đã được một người điền, nhưng cùng một địa chỉ có thể xuất hiện vài lần hoặc khác chữ hoa/thường và khoảng trắng, chẳng hạn `Test@Example.INVALID` với `test@example.invalid`. Trước khi gửi, nhân viên muốn biết có bao nhiêu địa chỉ khác nhau và xem danh sách không lặp, theo thứ tự dễ kiểm tra.

Vì sao chọn cách này? `set` là **tập hợp** chỉ giữ mỗi giá trị khác nhau một lần. Tuy nhiên `set` không tự hiểu `Test@Example.INVALID` và `test@example.invalid` là cùng một chuỗi, nên phải làm sạch khoảng trắng bằng `strip()` và thống nhất chữ thường bằng `lower()` trước. Sau đó thêm từng email vào set để xóa bản ghi lặp. Vì set không giữ thứ tự ổn định cho việc in báo cáo, dùng `sorted()` để sắp danh sách. Đây là phép lọc **địa chỉ trùng theo cách viết**, không chứng minh địa chỉ thật sự tồn tại.

Quy trình:

1. Chuẩn hóa mười hai email sang chữ thường, bỏ khoảng trắng biên.
2. Đưa dữ liệu vào Set để loại các bản ghi giống nhau.
3. Báo số email gốc, số email duy nhất và số lượt thừa.

No Note — Không chú thích

```
raw = ["Lan@EXAMPLE.invalid","an@example.invalid"," LAN@example.invalid
","binh@example.invalid","chi@example.invalid","An@Example.INVALID","dung@example.invalid","ha@exam
ple.invalid","Minh@example.invalid","lan@example.invalid","trang@example.invalid","ha@example.inval
id"]
normalized = []
for email in raw:
    normalized.append(email.strip().lower())
unique = set(normalized)
if len(raw) != 12:
    raise ValueError("Cần đủ 12 dòng email đầu vào.")
print("Dòng nhận:", len(raw))
print("Email duy nhất:", len(unique))
print("Bản ghi trùng:", len(raw) - len(unique))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuẩn hóa 12 email rồi đếm số địa chỉ khác nhau.
# Dữ liệu: 12 địa chỉ email với chữ hoa và khoảng trắng thừa.
raw = ["Lan@EXAMPLE.invalid","an@example.invalid"," LAN@example.invalid
","binh@example.invalid","chi@example.invalid","An@Example.INVALID","dung@example.invalid","ha@exam
ple.invalid","Minh@example.invalid","lan@example.invalid","trang@example.invalid","ha@example.inval
id"]
normalized = []
for email in raw:
    # strip() xóa khoảng trắng hai đầu; lower() hạ chữ hoa để cùng email có dạng thống nhất.
    normalized.append(email.strip().lower())
# set() bỏ email trùng sau khi làm sạch, len() của set cho số địa chỉ khác nhau.
unique = set(normalized)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(raw) != 12:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 12 dòng email đầu vào.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Dòng nhận:", len(raw))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Email duy nhất:", len(unique))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Bản ghi trùng:", len(raw) - len(unique))
```

Kết quả khi chạy:

```
Dòng nhận: 12
Email duy nhất: 8
Bản ghi trùng: 4
```

Tên miền `.invalid` chỉ dành cho dữ liệu minh họa. Không dùng số email thô làm số tài khoản duy nhất khi dữ liệu chưa được chuẩn hóa.

Tình huống 3 — Cấp 3: Tìm học viên học cả hai môn từ 15 lượt đăng ký

Từ vựng trong mục: Intersection — phép giao; **Union** — phép hợp; **Membership** — thành viên.

Tình huống thực tế: Trung tâm có **15 lượt đăng ký môn**, mỗi dòng chứa mã học viên cùng tên môn **Python** hoặc **AI**. Một học viên đăng ký cả hai môn sẽ có hai dòng, vì thế số dòng không bằng số người. Giáo vụ cần danh sách những người học **cả Python và AI**, cùng tổng số học viên khác nhau đã đăng ký ít nhất một môn.

Vì sao chọn cách này? Mỗi môn cần một tập mã học viên: tập Python và tập AI. Với một người H03 có mặt trong cả hai tập, phép **giao** `&` sẽ giữ H03 vì người ấy xuất hiện ở cả hai; phép **hợp** `|` giữ những ai đăng ký ít nhất một môn. Số người khác nhau được đếm từ hợp, chứ không phải đếm 15 lượt vì một người có thể ghi hai lần. Set phù hợp ở đây vì mục tiêu là kiểm tra **sự có mặt của một mã**, không phải đếm số lần học lại cùng môn.

Quy trình:

1. Lưu 15 cặp `(mã học viên, môn)` có nghĩa rõ.
2. Tạo Set mã cho từng môn, lấy giao và hợp.
3. Sắp mã trong giao để in ổn định và đếm số người riêng biệt.

No Note — Không chú thích

```
registrations = [("SV01","Python"),("SV02","Python"),("SV03","AI"),("SV01","AI"),("SV04","Python"),
("SV05","AI"),("SV06","Python"),("SV07","AI"),("SV02","AI"),("SV08","Python"),("SV09","Python"),
("SV10","AI"),("SV03","Python"),("SV11","Python"),("SV12","AI")]
python_students = set()
ai_students = set()
for student, subject in registrations:
    if subject == "Python":
        python_students.add(student)
    if subject == "AI":
        ai_students.add(student)
both = sorted(python_students & ai_students)
all_students = python_students | ai_students
if len(registrations) != 15:
    raise ValueError("Cần đúng 15 lượt đăng ký hai môn.")
print("Lượt đăng ký:", len(registrations))
print("Học cả hai:", ", ".join(both))
print("Học viên khác nhau:", len(all_students))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm học viên đăng ký cả Python và AI trong 15 lượt.
# Dữ liệu: 15 bộ (mã học viên, tên môn); một học viên có thể đăng ký hai môn.
registrations = [("SV01","Python"),("SV02","Python"),("SV03","AI"),("SV01","AI"),("SV04","Python"),
("SV05","AI"),("SV06","Python"),("SV07","AI"),("SV02","AI"),("SV08","Python"),("SV09","Python"),
("SV10","AI"),("SV03","Python"),("SV11","Python"),("SV12","AI")]
# set() tạo tập rỗng, mỗi mã học viên xuất hiện tối đa một lần.
python_students = set()
```

```
# set() tạo tập rỗng, mỗi mã học viên xuất hiện tối đa một lần.
ai_students = set()
for student, subject in registrations:
    if subject == "Python":
        # add() đưa mã vào tập học Python, tự bỏ trùng.
        python_students.add(student)
    if subject == "AI":
        # add() đưa mã vào tập học AI, tự bỏ trùng.
        ai_students.add(student)
# & lấy học viên có ở cả hai tập; sorted() để in mã theo thứ tự.
both = sorted(python_students & ai_students)
# | lấy tất cả học viên thuộc ít nhất một môn, bỏ trùng.
all_students = python_students | ai_students
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(registrations) != 15:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đúng 15 lượt đăng ký hai môn.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Lượt đăng ký:", len(registrations))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Học cả hai:", ", ".join(both))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Học viên khác nhau:", len(all_students))
```

Kết quả khi chạy:

```
Lượt đăng ký: 15
Học cả hai: SV01, SV02, SV03
Học viên khác nhau: 12
```

Cùng một mã xuất hiện hai lần là hai lượt đăng ký hợp lệ, không phải nhập trùng nếu môn khác nhau. Phép hợp khử trùng mã để đếm số người.

Tình huống 4 — Cấp 4: Lọc mã thành viên lỗi và so sánh 18 lượt vào nhóm

Từ vựng trong mục: Difference — hiệu tập hợp; **Intersection** — phép giao; **Invalid ID** — mã lỗi.

Tình huống thực tế: Hai câu lạc bộ A và B nhận **18 lượt đăng ký**, mỗi lượt có mã thành viên và tên câu lạc bộ. Một người có thể tham gia cả hai, nhưng lượt thứ 15 không có mã người (`None`) nên chưa thể xác định ai đăng ký. Người quản lý muốn tách lượt thiếu mã, liệt kê người **chỉ tham gia A**, và người **tham gia cả A lẫn B**.

Vì sao chọn cách này? Trước tiên kiểm tra mã thành viên có bị thiếu không. `None` nghĩa là **chưa biết người đăng ký**, nên không thêm vào tập; nếu thêm, hệ thống sẽ xem `None` như một thành viên giả và đếm sai. Từ các mã hợp lệ, tạo tập A và tập B. **Hiệu** `A - B` nghĩa là người có ở A nhưng không có ở B, còn **giao** `A & B` là người có ở cả hai. Hai phép trả lời hai câu hỏi khác nhau; số lượt đăng ký không dùng thay số người vì người có thể đăng ký hai nhóm.

Quy trình:

1. Kiểm tra từng cặp, bỏ lượt có ID None và đếm lỗi.
2. Lập tập thành viên A và B từ 17 lượt hợp lệ.
3. Lấy hiệu/giao rồi in danh sách mã đã sắp xếp.

No Note — Không chú thích

```
registrations = [("U01","A"),("U02","A"),("U03","B"),("U04","A"),("U05","B"),("U06","A"),
("U07","B"),("U08","A"),("U09","A"),("U10","B"),("U02","B"),("U04","B"),("U11","A"),("U12","B"),
```

```
(None,"B"),("U13","A"),("U14","B"),("U10","A")]
valid = []
a = set()
b = set()
for code, group in registrations:
    if code is None:
        continue
    valid.append((code, group))
    if group == "A":
        a.add(code)
    if group == "B":
        b.add(code)
if len(registrations) != 18:
    raise ValueError("Cần đúng 18 lượt tham gia nhóm.")
print("Lượt ghi:", len(registrations))
print("Thiếu ID:", len(registrations) - len(valid))
print("Chỉ nhóm A:", ", ".join(sorted(a - b)))
print("Cả hai:", ", ".join(sorted(a & b)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc mã thiếu trong 18 lượt tham gia, tìm nhóm A-only và tham gia cả hai.
# Dữ liệu: 18 bộ (mã người, nhóm A/B), None là lượt thiếu ID.
registrations = [("U01","A"),("U02","A"),("U03","B"),("U04","A"),("U05","B"),("U06","A"),
("U07","B"),("U08","A"),("U09","A"),("U10","B"),("U02","B"),("U04","B"),("U11","A"),("U12","B"),
(None,"B"),("U13","A"),("U14","B"),("U10","A")]
valid = []
# set() tạo tập hợp các phần tử không lặp; tập hợp không bảo đảm thứ tự in.
a = set()
# set() tạo tập hợp các phần tử không lặp; tập hợp không bảo đảm thứ tự in.
b = set()
for code, group in registrations:
    # is None nhận lượt không có ID; không đưa vào tập người.
    if code is None:
        # continue bỏ lượt ID thiếu.
        continue
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    valid.append((code, group))
    if group == "A":
        # add() thêm mã duy nhất vào nhóm A.
        a.add(code)
    if group == "B":
        # add() thêm mã duy nhất vào nhóm B.
        b.add(code)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(registrations) != 18:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đúng 18 lượt tham gia nhóm.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Lượt ghi:", len(registrations))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Thiếu ID:", len(registrations) - len(valid))
# a - b là người chỉ thuộc nhóm A; sorted() xếp mã để in ổn định.
print("Chỉ nhóm A:", ", ".join(sorted(a - b)))
# a & b là người thuộc cả hai nhóm.
print("Cả hai:", ", ".join(sorted(a & b)))
```

Kết quả khi chạy:

```
Lượt ghi: 18
Thiếu ID: 1
```

```
Chỉ nhóm A: U01, U06, U08, U09, U11, U13
Cả hai: U02, U04, U10
```

Không gộp hai lượt của cùng người thành một lỗi: cùng ID ở A/B có nghĩa là người đó tham gia cả hai nhóm.

Tình huống 5 — Cấp 5: Kiểm kê kỹ năng còn thiếu của 8 học viên qua 20 lượt học

Từ vựng trong mục: Set difference — hiệu tập hợp; **Data validation** — kiểm dữ liệu; **Tuple** — cặp giá trị.

Tình huống thực tế: Một khóa học theo dõi **8 học viên S01 đến S08** qua **20 lượt ghi nhận kỹ năng**. Mỗi lượt là cặp `(mã học viên, tên kỹ năng)`; khóa học yêu cầu đủ ba kỹ năng **Python, SQL và AI**. Trong các lượt ghi có một lần S06–Python bị lặp và một lần S06 chưa có tên kỹ năng (`None`). Giáo viên muốn biết ai đã học đủ ba kỹ năng, riêng S06 còn thiếu kỹ năng nào, và số lượt ghi trùng hoặc thiếu thông tin.

Vì sao chọn cách này? Tuple `(người, kỹ năng)` giúp ghi rõ **ai đã học gì**; hai dòng chỉ là trùng khi **cả người lẫn kỹ năng đều giống nhau**. Set các cặp loại bỏ lần ghi S06–Python bị lặp nhưng vẫn giữ S06–SQL nếu có. `None` là **kỹ năng chưa xác định**, không thể tự coi là một môn đã hoàn thành, nên tách sang nhóm cần bổ sung thông tin. Với từng học viên, so tập kỹ năng đã ghi với tập yêu cầu `{Python, SQL, AI}`: **hiệu** `required - done` chính là các môn còn thiếu. Chỉ học viên có đủ cả ba mới được tính hoàn thành.

Quy trình:

1. Đọc đủ 20 lượt và tách những lượt chưa biết tên kỹ năng.
2. Dùng Set các tuple `(student, skill)` để loại lượt giống hệt nhau.
3. Với mỗi người, so `required` với tập đã học, tổng hợp người đủ khóa và báo chi tiết S06.

No Note — Không chú thích

```
records = [("S01","Python"),("S01","SQL"),("S01","AI"),("S02","Python"),("S02","SQL"),("S03","AI"),
("S03","SQL"),("S04","Python"),("S04","AI"),("S05","Python"),("S05","SQL"),("S05","AI"),
("S06","Python"),("S06","Python"),("S06",None),("S07","SQL"),("S07","AI"),("S08","Python"),
("S08","SQL"),("S08","AI")]
required = {"Python", "SQL", "AI"}
valid = []
all_student_codes = set()
for student, skill in records:
    all_student_codes.add(student)
    if skill is not None:
        valid.append((student, skill))
unique_pairs = set(valid)
all_students = sorted(all_student_codes)
completed = []
missing_s06 = set()
for student in all_students:
    learned = set()
    for user, skill in unique_pairs:
        if user == student:
            learned.add(skill)
    remaining = required - learned
    if not remaining:
        completed.append(student)
    if student == "S06":
        missing_s06 = remaining
if len(records) != 20:
    raise ValueError("Cần kiểm tra đủ 20 lượt kỹ năng.")
print("Lượt ghi:", len(records))
print("Thiếu kỹ năng:", len(records) - len(valid))
```

```
print("Lượt trùng:", len(valid) - len(unique_pairs))
print("Hoàn tất:", ", ".join(completed))
print("S06 cần học:", ", ".join(sorted(missing_s06)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đối chiếu 20 lượt học kỹ năng, gộp trùng và tìm học viên đủ ba môn.
# Dữ liệu: 20 bộ (mã học viên, kỹ năng); None thiếu kỹ năng và có lượt trùng.
records = [("S01","Python"),("S01","SQL"),("S01","AI"),("S02","Python"),("S02","SQL"),("S03","AI"),
("S03","SQL"),("S04","Python"),("S04","AI"),("S05","Python"),("S05","SQL"),("S05","AI"),
("S06","Python"),("S06","Python"),("S06",None),("S07","SQL"),("S07","AI"),("S08","Python"),
("S08","SQL"),("S08","AI")]
required = {"Python", "SQL", "AI"}
valid = []
# set() tạo tập hợp các phần tử không lặp; tập hợp không bảo đảm thứ tự in.
all_student_codes = set()
for student, skill in records:
    # add() lưu toàn bộ mã đã thấy, kể cả khi lượt có kỹ năng None.
    all_student_codes.add(student)
    # is not None chỉ giữ bản ghi có tên kỹ năng, khác bản ghi thiếu.
    if skill is not None:
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        valid.append((student, skill))
# set() loại các cặp (học viên, kỹ năng) trùng nhau.
unique_pairs = set(valid)
# sorted() xếp mã học viên để duyệt ổn định.
all_students = sorted(all_student_codes)
completed = []
# set() tạo tập hợp các phần tử không lặp; tập hợp không bảo đảm thứ tự in.
missing_s06 = set()
for student in all_students:
    # set() tạo tập hợp các phần tử không lặp; tập hợp không bảo đảm thứ tự in.
    learned = set()
    for user, skill in unique_pairs:
        if user == student:
            # add() gom kỹ năng khác nhau của cùng học viên.
            learned.add(skill)
    # Hiệu set tìm bộ kỹ năng bắt buộc chưa học.
    remaining = required - learned
    # not kiểm tra set còn thiếu đã rỗng, tức học viên đủ ba kỹ năng.
    if not remaining:
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        completed.append(student)
    if student == "S06":
        missing_s06 = remaining
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(records) != 20:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần kiểm tra đủ 20 lượt kỹ năng.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Lượt ghi:", len(records))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Thiếu kỹ năng:", len(records) - len(valid))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Lượt trùng:", len(valid) - len(unique_pairs))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Hoàn tất:", ", ".join(completed))
# sorted() làm thứ tự tên môn thiếu ổn định khi in.
print("S06 cần học:", ", ".join(sorted(missing_s06)))
```

Kết quả khi chạy:

```
Lượt ghi: 20
Thiếu kỹ năng: 1
Lượt trùng: 1
Hoàn tất: S01, S05, S08
S06 cần học: AI, SQL
```

Khóa hoàn tất đòi **đủ cả ba kỹ năng**; việc có ba lượt ghi chưa chắc đã là ba kỹ năng khác nhau. Báo riêng lượt thiếu trước khi xét tiến độ học viên.

6. Nhầm lẫn dễ gặp

- Tuple không cho gán lại từng phần tử như `point[0] = 5`; nếu cần sửa tọa độ, tạo tuple mới.
- Set không có thứ tự cố định, không dùng `my_set[0]` và không dựa vào thứ tự khi in.
- Set chỉ giữ phần tử có thể băm được (hashable); không thể đặt list trực tiếp làm phần tử của set.
- Phép hợp `|` lấy tất cả phần tử; phép giao `&` chỉ lấy phần tử chung; phép trừ `-` tìm phần tử còn thiếu.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a tuple? | Tuple là gì?

A. An ordered mutable sequence | Dãy có thứ tự và có thể sửa

B. An unordered collection of unique items | Tập hợp không thứ tự của phần tử duy nhất

C. A key-to-value mapping | Ánh xạ khóa–giá trị

D. An ordered immutable collection | Tập hợp có thứ tự và bất biến

Đáp án: D

Giải thích: Tuple là dãy có thứ tự và bất biến: `(2, 3)` giữ tọa độ đầu trước tọa độ sau, nhưng không cho phép gán lại từng phần tử. List cũng có thứ tự nhưng có thể sửa các phần tử.

Q2. What is a set? | Set là gì?

A. An ordered collection of key-value entries | Tập hợp có thứ tự các cặp khóa–giá trị

B. A collection that always keeps duplicates | Tập hợp luôn giữ giá trị trùng

C. A collection of unique hashable members | Tập hợp gồm phần tử có thể băm và không trùng

D. A sequence accessed by positional indexes | Dãy truy cập bằng chỉ số vị trí

Đáp án: C

Giải thích: Set chứa các phần tử có thể băm và không trùng nhau, đồng thời không hỗ trợ truy cập theo chỉ số vị trí. Tạo `set([1, 1, 2])` chỉ còn hai phần tử `1` và `2`, khác List vốn giữ phần tử lặp.

Q3. What does set union collect? | Phép hợp Set lấy gì?

A. Only members of the first set not in the second | Chỉ phần tử tập đầu mà không thuộc tập sau

B. Distinct members appearing in either set | Phần tử khác nhau thuộc ít nhất một tập

C. Only members appearing in exactly one set | Chỉ phần tử thuộc đúng một tập

D. Only members common to both sets | Chỉ phần tử chung của hai tập

Đáp án: B

Giải thích: Phép hợp lấy mỗi phần tử khác nhau xuất hiện trong ít nhất một tập. Với `{1, 2}` và `{2, 3}`, hợp chứa `{1, 2, 3}`; phép giao chỉ giữ phần tử chung `2`.

Q4. What does set intersection collect? | Phép giao Set lấy gì?

A. Only members of the first set | Chỉ phần tử tập đầu

B. Members occurring in exactly one set | Phần tử xuất hiện đúng một tập

C. Only members shared by both sets | Chỉ phần tử chung của cả hai tập

D. All distinct members of either set | Mọi phần tử khác nhau thuộc ít nhất một tập

Đáp án: C

Giải thích: Phép giao chỉ giữ phần tử có mặt ở cả hai tập. Với `{1, 2}` và `{2, 3}`, giao là `{2}`; tập này nhỏ hơn phép hợp `{1, 2, 3}`.

Q5. Which expression constructs a two-element tuple? | Biểu thức nào tạo Tuple có hai phần tử?

A. `(2, 3)` | `(2, 3)`

B. `[2, 3]` | `[2, 3]`

C. `{2, 3}` | `{2, 3}`

D. `{2: 3}` | `{2: 3}`

Đáp án: A

Giải thích: `(2, 3)` là Tuple gồm hai phần tử có thứ tự với các giá trị phân cách bằng dấu phẩy. Ngược lại, `{2, 3}` là Set và `[2, 3]` là List có thể sửa.

Q6. Which expression constructs a nonempty set? | Biểu thức nào tạo Set không rỗng?

A. `[2, 3]` | `[2, 3]`

B. `{2: 3}` | `{2: 3}`

C. `(2, 3)` | `(2, 3)`

D. `{2, 3}` | `{2, 3}`

Đáp án: D

Giải thích: `{2, 3}` tạo Set không rỗng gồm hai số nguyên khác nhau. Dấu ngoặc tròn tạo Tuple và ngoặc vuông tạo List, còn ngoặc nhọn rỗng `{}` tạo Dictionary.

Q7. Which expression constructs an empty set? | Biểu thức nào tạo Set rỗng?

A. `[]` | `[]`

B. `set()` | `set()`

C. `()` | `()`

D. `{} | {}`

Đáp án: B

Giải thích: `set()` tạo Set rỗng. Ký hiệu gần giống `{}` lại là Dictionary rỗng, vì vậy không thể dùng cặp ngoặc nhọn trống để tạo Set không có phần tử.

Q8. Which expression constructs an empty dictionary? | Biểu thức nào tạo Dictionary rỗng?

A. `set() | set()`

B. `[] | []`

C. `() | ()`

D. `{} | {}`

Đáp án: D

Giải thích: `{}` biểu diễn Dictionary rỗng, tức ánh xạ khóa–giá trị chưa có mục nào. Muốn tạo Set rỗng phải dùng `set()`; List và Tuple rỗng lần lượt là `[]` và `()`.

Q9. What does `a | b` do for sets? | `a | b` làm gì với hai Set?

A. Set difference from `a` to `b` | Hiệu Set từ `a` trừ `b`

B. Symmetric difference | Hiệu đối xứng

C. Set union | Phép hợp Set

D. Set intersection | Phép giao Set

Đáp án: C

Giải thích: Với Set, `a | b` thực hiện phép hợp, giữ phần tử khác nhau thuộc `a`, `b` hoặc cả hai. Nếu `a={1,2}` và `b={2,3}`, kết quả chứa 1, 2, 3, khác giao vốn chỉ giữ 2.

Q10. What does `a & b` do for sets? | `a & b` làm gì với hai Set?

A. Set intersection | Phép giao Set

B. Set difference | Phép hiệu Set

C. Symmetric difference | Hiệu đối xứng

D. Set union | Phép hợp Set

Đáp án: A

Giải thích: `a & b` là phép giao Set, chỉ giữ giá trị xuất hiện ở cả hai tập. Với `{1,2}` và `{2,3}`, kết quả là `{2}`; nó không lấy mọi phần tử khác nhau như phép hợp.

Q11. What does `a - b` do for sets? | `a - b` làm gì với hai Set?

A. Members in `a` but not in `b` | Phần tử thuộc `a` nhưng không thuộc `b`

B. Members in exactly one input | Phần tử thuộc đúng một tập

C. Set union | Phép hợp Set

D. Set intersection | Phép giao Set

Đáp án: A

Giải thích: `a - b` chỉ giữ phần tử thuộc `a` mà không thuộc `b`, nên thứ tự hai vế quan trọng: `{1,2} - {2,3}` cho `{1}`. Đảo hai vế lại được `{3}`, khác hiệu đối xứng giữ cả 1 và 3.

Q12. What does `a ^ b` do for sets? | `a ^ b` làm gì với hai Set?

A. Set union | Phép hợp Set

B. Set intersection | Phép giao Set

C. Members of `a` only | Chỉ phần tử của `a`

D. Members in exactly one of the sets | Phần tử thuộc đúng một trong hai tập

Đáp án: D

Giải thích: `a ^ b` tính hiệu đối xứng: lấy phần tử có mặt đúng ở một trong hai tập. Với `{1,2}` và `{2,3}`, kết quả chứa 1 và 3 nhưng loại phần tử chung 2.

Q13. What is the length of `set([1,1,2])`? | `set([1,1,2])` có bao nhiêu phần tử?

A. `2` | `2`

B. `3` | `3`

C. `1` | `1`

D. `ValueError` | `ValueError`

Đáp án: A

Giải thích: Tạo Set sẽ loại giá trị trùng, vì vậy `set([1,1,2])` chỉ còn phần tử 1 và 2. Độ dài do đó bằng số nguyên `2`, không phải ba vị trí của List gốc.

Q14. What happens when assigning `t[0] = 9` on a tuple? | Gán `t[0] = 9` trên Tuple sẽ thế nào?

A. A `TypeError` is raised | Phát sinh `TypeError`

B. A new tuple is created automatically | Tự động tạo Tuple mới

C. An `IndexError` is always raised | Luôn phát sinh `IndexError`

D. The tuple's first item changes to 9 | Phần tử đầu Tuple đổi thành 9

Đáp án: A

Giải thích: Tuple không cho phép gán lại từng phần tử nên `t[0] = 9` trên Tuple gây `TypeError`. Khác với List, việc gán vào một chỉ số hợp lệ được cho phép.

Q15. How do you test membership in a set? | Làm sao kiểm tra một giá trị thuộc Set?

A. `s[x]` | `s[x]`

B. `x in s` | `x in s`

C. `x == s` | `x == s`

D. `s.index(x)` | `s.index(x)`

Đáp án: B

Giải thích: Dùng `x in s` để kiểm tra phần tử, kết quả là Boolean. Với `s={1,3}`, `3 in s` là `True` và `2 in s` là `False`; Set không có phương thức `.index()` theo vị trí.

Q16. Is order of printed set elements guaranteed? | Thứ tự phần tử in từ Set có được bảo đảm không?

A. Always sorted in ascending order | Luôn theo thứ tự tăng dần

B. Always insertion order | Luôn theo thứ tự thêm vào

C. Always a new random permutation | Luôn tạo hoán vị ngẫu nhiên mới

D. No fixed display order is guaranteed | Không bảo đảm thứ tự hiển thị cố định

Đáp án: D

Giải thích: Set không bảo đảm thứ tự khi hiển thị hay duyệt các phần tử. Dù một lần chạy in `{1, 2, 3}`, không nên phụ thuộc thứ tự đó ở các lần sau; sắp xếp Set là thao tác riêng.

Q17. Which built-in immutable ordered sequence best represents a fixed coordinate pair `(x, y)`? | Kiểu dãy có thứ tự và bất biến nào phù hợp nhất biểu diễn cặp tọa độ cố định `(x, y)`?

A. Dictionary | Dictionary

B. Tuple | Tuple

C. List | List

D. Set | Set

Đáp án: B

Giải thích: Một cặp có thứ tự và bất biến như `(x, y)` phù hợp tọa độ cố định vì vị trí thứ nhất và thứ hai có ý nghĩa riêng. Set không truy cập theo vị trí, còn List cho phép gán lại phần tử.

Q18. Which built-in collection directly keeps only distinct tags? | Kiểu tập hợp nào trực tiếp chỉ giữ những nhãn khác nhau?

A. String, which holds one sequence of characters | String lưu dãy ký tự

B. Tuple, which retains duplicates | Tuple giữ nguyên các phần tử trùng

C. Set, which retains distinct tags | Set chỉ giữ nhãn khác nhau

D. List, which retains duplicates | List giữ nguyên các phần tử trùng

Đáp án: C

Giải thích: Set trực tiếp loại nhãn trùng vì mỗi giá trị chỉ xuất hiện một lần. Với nhãn `['red', 'red', 'blue']`, tạo Set giữ hai nhãn khác nhau `red` và `blue`, nhưng không bảo đảm thứ tự.

Q19. Which method checks whether sets share no elements? | Phương thức nào kiểm tra hai Set không có phần tử chung?

A. `union()` | `union()`

B. `isdisjoint()` | `isdisjoint()`

C. `intersection()` | `intersection()`

D. `issubset()` | `issubset()`

Đáp án: B

Giải thích: `a.isdisjoint(b)` trả `True` khi và chỉ khi hai Set không có phần tử chung. Với `{1,2}` và `{3}`, kết quả là `True`, còn `{1,2}` và `{2,4}` cho `False`.

Q20. Which function creates a sorted list from a set of comparable numbers? | Hàm nào tạo List đã sắp từ một Set chứa các số có thể so sánh?

A. `list(s)` without ordering | `list(s)` không sắp xếp

B. `reversed(s)` | `reversed(s)`

C. `sorted(s)` | `sorted(s)`

D. `s.sort()` | `s.sort()`

Đáp án: C

Giải thích: `sorted(s)` trả về List mới đã sắp xếp, ví dụ `sorted({3,1,2})` cho `[1,2,3]`. Nó không tạo thứ tự cố định cho Set gốc, vốn vẫn là tập không có thứ tự vị trí.

8. Nối sang bài tiếp theo

`List → Tuple và Set → Tập hợp/Jaccard`.

PY-08 — Dictionary: khóa, giá trị và thêm sửa xóa

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Dictionary — từ điển; **Key** — khóa.

`List/Set → Dictionary → JSON/CSV → DataFrame`

Học xong dùng để làm gì? Tra cứu hồ sơ học viên theo mã và biểu diễn bản ghi có nhiều trường bằng khóa–giá trị.

Kết nối kiến thức tiếp theo: PY-09 viết hàm tái sử dụng các bước xử lý những bản ghi đó.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Dictionary	từ điển	Ánh xạ khóa sang giá trị
Key	khóa	Tên hoặc mã để truy tìm giá trị
Value	giá trị	Thông tin được lưu theo một khóa
Nested dict	từ điển lồng	Một từ điển có từ điển con

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`obj[key], obj.get(key)`	Tra cứu khóa hoặc nhận giá trị mặc định
`obj.items(), obj.values()`	Duyệt cặp khóa/giá trị hay riêng giá trị
`obj[key] = value`	Thêm hoặc cập nhật dữ liệu
`del, pop(key)`	Xóa thông tin bằng khóa

4. Khái niệm liên quan

List lấy dữ liệu theo **vị trí**, còn **Dictionary** lấy theo **khóa có ý nghĩa** như mã đơn hàng hoặc tên trường. Mỗi khóa gắn với một giá trị để tra cứu, thêm, sửa hoặc xóa thông tin. Bài dùng hồ sơ đơn hàng làm ví dụ xuyên suốt, từ một mã đơn đơn giản tới hồ sơ có nhiều thuộc tính.

4.1. Dictionary — Dữ liệu dạng khóa và giá trị

Từ vựng trong mục: Dictionary — từ điển; **Key–value pair** — cặp khóa–giá trị.

Một cửa hàng lưu đơn hàng mã `O1` có trạng thái `paid` (đã thanh toán). Trong Dictionary `order = {'id': 'O1', 'status': 'paid'}`, **`id` và `status` là hai khóa**; `O1` và `paid` là các giá trị tương ứng. Dấu `{}` bao các cặp `khóa: giá trị`. Dùng Dictionary khi cần đọc thông tin theo tên trường thay vì phải nhớ chỉ số trong List.

No Note — Không chú thích

```
order = {'id': 'O1', 'status': 'paid'}
print(order)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo dictionary lưu trạng thái đơn theo khóa và giá trị.
# Dữ liệu: Khóa id là mã đơn và status là chuỗi trạng thái.
# Dictionary dùng khóa dạng chuỗi để truy cập giá trị tương ứng.
order = {'id': 'O1', 'status': 'paid'}
print(order)
```

Kết quả khi chạy:

```
{'id': 'O1', 'status': 'paid'}
```

Một đơn hàng có hai trường. Mỗi cặp khóa–giá trị mô tả một thông tin riêng.

4.2. Key — Khóa để tra cứu

Từ vựng trong mục: Key — khóa; **Lookup** — tra cứu.

Khóa là tên hoặc mã duy nhất **trong cùng một Dictionary**, được dùng để lấy giá trị đúng trường. Với hồ sơ `order`, biểu thức `order['id']` lấy mã O1; còn `order['status']` lấy trạng thái. Nếu dùng dấu `[]` với khóa không tồn tại, Python báo `KeyError`; có thể dùng `get()` khi cần giá trị mặc định.

No Note — Không chú thích

```
order = {'id': 'O1', 'status': 'paid'}
print(order['id'])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tra cứu mã đơn trực tiếp theo khóa id.
# Dữ liệu: Dictionary có hai khóa id/status; truy cập qua tên khóa, không phải chỉ số.
order = {'id': 'O1', 'status': 'paid'}
# ['id'] tra đúng khóa id và lấy giá trị O1; khóa không có sẽ gây KeyError.
print(order['id'])
```

Kết quả khi chạy:

```
O1
```

Python tra theo khóa `id`; không cần duyệt qua các trường bằng vòng lặp.

4.3. Value và thao tác thêm, sửa, xóa

Từ vựng trong mục: Value — giá trị; **Update** — cập nhật.

Giá trị là dữ liệu gắn với từng khóa; có thể là chuỗi, số, List hoặc Dictionary khác. Ví dụ đơn `01` đang có trạng thái `pending` (đang chờ). Lệnh `order['status'] = 'paid'` **sửa** giá trị của khóa đã có. Lệnh `order['amount'] = 100` **thêm** trường số tiền mới; `order.pop('amount')` **xóa** trường ấy. Trong bài này, ta in Dictionary trước và sau khi xóa để nhìn thấy thay đổi; nếu xóa khóa không tồn tại mà không truyền giá trị mặc định cho `pop`, Python báo `KeyError`.

No Note — Không chú thích

```
order = {'id': 'O1', 'status': 'pending'}
order['status'] = 'paid'
order['amount'] = 100
print(order)
order.pop('amount')
print(order)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thêm, sửa và xóa một mục dữ liệu trong dictionary.
# Dữ liệu: Cùng một khóa status bị ghi đè; khóa amount mới được thêm, rồi xóa.
order = {'id': 'O1', 'status': 'pending'}
# Gán lại giá trị cho khóa tồn tại status từ pending thành paid.
order['status'] = 'paid'
# Gán cho khóa mới amount để thêm một trường vào dictionary.
order['amount'] = 100
print(order)
# pop('amount') lấy ra và xóa khóa amount, làm dictionary chỉ còn các khóa khác.
order.pop('amount')
print(order)
```

Kết quả khi chạy:

```
{'id': 'O1', 'status': 'paid', 'amount': 100}
{'id': 'O1', 'status': 'paid'}
```

Dòng đầu có thêm trường `amount` giá trị 100; dòng thứ hai không còn trường này sau `pop()`. Mã đơn O1 và trạng thái `paid` vẫn giữ nguyên.

4.4. Nested dictionary — Hồ sơ có thông tin con

Từ vựng trong mục: Nested dictionary — từ điển lồng; **Field** — trường dữ liệu.

Nếu quản lý nhiều đơn hàng, ta có thể dùng **Dictionary ngoài** với khóa là mã đơn (`01`, `02`), và **Dictionary trong** lưu thông tin riêng như `status`, `amount`. Khi cần lấy số tiền đơn `02`, tra trước `orders['02']` để lấy hồ sơ con, sau đó tra `['amount']`. Cách này giữ các thuộc tính của từng đơn bên nhau.

No Note — Không chú thích

```
orders = {
    'O1': {'status': 'paid', 'amount': 100},
    'O2': {'status': 'pending', 'amount': 200},
```

```
}
print(orders['O2']['amount'])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc số tiền ở dictionary con của đơn O2.
# Dữ liệu: Mỗi mã đơn ánh xạ sang một dictionary con có status và amount.
orders = {
    'O1': {'status': 'paid', 'amount': 100},
    'O2': {'status': 'pending', 'amount': 200},
}
# Truy cập hai tầng: lấy dict của mã O2 rồi lấy khóa amount, giá trị 200.
print(orders['O2']['amount'])
```

Kết quả khi chạy:

```
200
```

Kết quả `200` là số tiền của đơn O2 (đơn vị tiền theo quy định của cửa hàng), không phải mã đơn hay chỉ số dòng.

5. Năm tình huống Dictionary — từ tra cứu đến nhập/xuất có kiểm chứng

Tình huống 1 — Cấp 1: Tra cứu 10 đơn không phát sinh KeyError

Từ vựng trong mục: Dictionary — từ điển; **Key** — khóa; **Get** — tra cứu.

Tình huống thực tế: Một cửa hàng quản lý **10 đơn hàng O01 đến O10**. Mỗi mã gắn với một trạng thái: `paid` nghĩa là đã trả tiền, `pending` đang chờ và `cancelled` đã hủy. Nhân viên cần xem trạng thái O04, thử tra O11 là mã chưa có trong bảng, rồi đếm số đơn đã thanh toán; việc tra mã lạ không được làm cả báo cáo dừng lại.

Vì sao chọn cách này? Dictionary là bảng ghép **khóa** với **giá trị**: ở đây khóa là mã đơn O01, O02..., giá trị là trạng thái thanh toán. Từ mã có thể tra đúng trạng thái mà không cần dò từng hàng. `orders.get('O11', 'unknown')` trả về giá trị dự phòng khi mã chưa có, nên chương trình không bị lỗi `KeyError`. Truy cập thẳng `orders['O11']` chỉ phù hợp khi chắc chắn khóa tồn tại; trường hợp này không chắc. Muốn đếm đơn `paid`, duyệt các trạng thái và tăng bộ đếm chỉ khi trạng thái bằng `paid`.

Quy trình:

1. Tạo Dictionary 10 mã và trạng thái.
2. Tra cứu O04 và O11 bằng `get`.
3. Đếm trạng thái paid và kiểm tra số khóa.

No Note — Không chú thích

```
orders =
{"O01":"paid","O02":"pending","O03":"paid","O04":"cancelled","O05":"pending","O06":"paid","O07":"pe
nding","O08":"paid","O09":"cancelled","O10":"paid"}
if len(orders) != 10:
    raise ValueError("Cần đủ 10 đơn hàng.")
print("Số đơn:", len(orders))
```

```
print("O04:", orders.get("O04", "Not found"))
print("O11:", orders.get("O11", "Not found"))
paid_count = 0
for value in orders.values():
    if value == "paid":
        paid_count += 1
print("Đã trả:", paid_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tra trạng thái 10 đơn, kể cả mã không có, và đếm đã thanh toán.
# Dữ liệu: 10 mã đơn có các trạng thái paid/pending/cancelled.
orders =
{"O01":"paid","O02":"pending","O03":"paid","O04":"cancelled","O05":"pending","O06":"paid","O07":"pe
nding","O08":"paid","O09":"cancelled","O10":"paid"}
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(orders) != 10:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 10 đơn hàng.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Số đơn:", len(orders))
# dict.get(khóa, mặc_định) đọc giá trị; nếu thiếu khóa trả 'Not found' thay vì KeyError.
print("O04:", orders.get("O04", "Not found"))
# dict.get(khóa, mặc_định) đọc giá trị; nếu thiếu khóa trả 'Not found' thay vì KeyError.
print("O11:", orders.get("O11", "Not found"))
paid_count = 0
# values() lấy toàn bộ trạng thái đơn để duyệt; không lấy mã khóa.
for value in orders.values():
    # == so với chuỗi paid để đếm đơn đã trả.
    if value == "paid":
        paid_count += 1
print("Đã trả:", paid_count)
```

Kết quả khi chạy:

```
Số đơn: 10
O04: cancelled
O11: Not found
Đã trả: 5
```

O11 không có khóa trong bảng, nên `Not found` là thông báo tra cứu, không phải một trạng thái đơn hàng thật.

Tình huống 2 — Cấp 2: Cập nhật tồn của 12 SKU theo số đã bán

Từ vựng trong mục: Key-value — cặp khóa–giá trị; **Update** — cập nhật.

Tình huống thực tế: Kho có **12 loại hàng P01 đến P12**. Với mỗi mã, bảng ghi số món đang còn và số món vừa bán; các con số đều là **số lượng món**, không phải tiền. Nhân viên muốn trừ đúng lượng đã bán ở từng mã, báo tồn cuối của P05 và tổng số món còn lại trên cả 12 mã. Nếu ai đó bán nhiều hơn tồn thì không được tạo số tồn âm.

Vì sao chọn cách này? SKU là mã nhận biết một loại hàng, chẳng hạn P05. Dictionary lưu tồn kho theo khóa là SKU, nên `stock[sku]` luôn tham chiếu đúng loại hàng đó. Với mỗi lượt bán, trước tiên kiểm tra số món có đủ không; nếu đủ mới trừ và lưu tồn còn lại. Dấu `-=` nghĩa là lấy giá trị đang có trừ số đã bán rồi cập nhật lại cùng khóa. Nếu trừ mà không kiểm tra, một loại có thể còn -2 món, vốn không phải tồn kho thực tế. Sau khi xử lý, cộng tồn của 12 khóa và đọc riêng khóa P05.

Quy trình:

1. Khởi tạo 12 mã và số tồn đầu ngày.
2. Duyệt đúng cặp mã–số bán; kiểm tra rồi trừ vào từng khóa.
3. In số SKU, tồn tổng cộng, số tồn P05.

No Note — Không chú thích

```
starting = [10,8,12,5,20,9,13,6,7,11,14,4]
sold = [2,1,3,0,4,1,2,2,1,3,4,0]
stock = {}
for i, amount in enumerate(starting, 1):
    sku = f"P{i:02d}"
    stock[sku] = amount
for i, quantity in enumerate(sold, 1):
    sku = f"P{i:02d}"
    if quantity > stock[sku]:
        raise ValueError(f"SKU {sku}: số´bán vượt quá tồn kho.")
    stock[sku] -= quantity
if len(stock) != 12:
    raise ValueError("Cần đủ 12 mã sản phẩm trong kho.")
print("SKU:", len(stock))
print("Tồn tổng:", sum(stock.values()))
print("P05 còn:", stock["P05"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Theo dõi tồn kho 12 SKU từ số´đầu kỳ và số´đã bán.
# Dữ liệu: 12 tồn đầu kỳ và 12 lượng bán (món), các phần tử cùng vị trí thuộc một SKU.
starting = [10,8,12,5,20,9,13,6,7,11,14,4]
sold = [2,1,3,0,4,1,2,2,1,3,4,0]
stock = {}
# enumerate(...,1) đặt mã SKU P01..P12 tương ứng từng số´tồn.
for i, amount in enumerate(starting, 1):
    # Định dạng f-string :02d cho số´thứ tự thành 2 chữ số´với số´0 bổ sung.
    sku = f"P{i:02d}"
    # Dictionary stock ánh xạ mã SKU thành số´món đầu kỳ.
    stock[sku] = amount
# Duyệt lượng bán cùng thứ tự mã P01..P12.
for i, quantity in enumerate(sold, 1):
    # Định dạng f-string :02d cho số´thứ tự thành 2 chữ số´với số´0 bổ sung.
    sku = f"P{i:02d}"
    # Không trừ lượng bán vượt số´tồn hiện tại, tránh âm kho.
    if quantity > stock[sku]:
        # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
        raise ValueError(f"SKU {sku}: số´bán vượt quá tồn kho.")
    # -= trừ số´bán khỏi lượng hàng của SKU tương ứng.
    stock[sku] -= quantity
# len() trả số´phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(stock) != 12:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 12 mã sản phẩm trong kho.")
# len() trả số´phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("SKU:", len(stock))
# values() trả các lượng tồn sau bán; sum() cộng thành tổng món còn.
print("Tồn tổng:", sum(stock.values()))
print("P05 còn:", stock["P05"])
```

Kết quả khi chạy:

```
SKU: 12
Tồn tổng: 96
P05 còn: 16
```

Dictionary được cập nhật theo mã thay vì dựa vào vị trí số khi xử lý các lần nhập dữ liệu sau.

Tình huống 3 — Cấp 3: Đếm 15 nhãn và xếp thứ hạng bằng Dictionary

Từ vựng trong mục: Frequency — tần suất; **Items** — cặp khóa và giá trị; **Ranking** — xếp hạng.

Tình huống thực tế: Trung tâm nhận **15 yêu cầu học tập**, mỗi lượt được gắn một nhãn môn trong các môn **AI, Python, SQL và Math**. Người điều phối muốn biết mỗi môn được yêu cầu bao nhiêu lượt và hai môn phổ biến nhất. Nếu hai môn bằng số lượt thì xếp theo tên môn để báo cáo luôn có thứ tự giống nhau.

Vì sao chọn cách này? Đếm tần suất nghĩa là tính một nhãn xuất hiện bao nhiêu lần. Dictionary `counts` ghi số lượt theo tên nhãn; `get(label, 0)` lấy số đếm cũ hoặc 0 nếu chưa thấy môn đó, rồi cộng 1 cho lượt mới. Nhờ vậy, 15 lượt được đếm đủ mà không phải chuẩn bị riêng biến cho từng môn. Để chọn hai môn phổ biến nhất, sắp các cặp `(tên môn, số lượt)` theo số lượt **giảm dần**, nếu bằng nhau theo tên môn **tăng dần**; chỉ lấy hai cặp đầu. Không lấy nhãn xuất hiện đầu tiên vì thứ tự tiếp nhận không cho biết môn phổ biến.

Quy trình:

1. Tạo Dictionary đếm ban đầu rỗng.
2. Duyệt 15 nhãn, tăng bộ đếm theo khóa.
3. Sắp danh sách cặp số đếm và in hai nhãn đứng đầu.

No Note — Không chú thích

```
labels =
["AI","Python","SQL","AI","Math","Python","AI","SQL","Python","Math","SQL","AI","Python","Math","AI
"]
counts = {}
for label in labels:
    counts[label] = counts.get(label, 0) + 1
def ranking_key(item):
    return (-item[1], item[0])
ranking = sorted(counts.items(), key=ranking_key)
if len(labels) != 15 or sum(counts.values()) != 15:
    raise ValueError("Cần đối chiếu đủ 15 nhãn được đếm.")
print("Lượt:", len(labels))
count_labels = []
for name in sorted(counts):
    count_labels.append(f"{name}={counts[name]}")
top_names = []
for name, count in ranking[:2]:
    top_names.append(name)
print("Đếm:", ", ".join(count_labels))
print("Top 2:", ", ".join(top_names))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm 15 nhãn trong dictionary và xếp loại theo tần suất.
# Dữ liệu: 15 tên nhãn với các loại AI/Python/SQL/Math.
labels =
["AI","Python","SQL","AI","Math","Python","AI","SQL","Python","Math","SQL","AI","Python","Math","AI
```

```
"]
counts = {}
for label in labels:
    # get(label,0) đọc bộ đếm hiện tại hoặc 0 nếu nhãn mới; +1 ghi nhận thêm một lượt.
    counts[label] = counts.get(label, 0) + 1
# Hàm key quyết định thứ tự xếp một cặp (tên nhãn, số lần).
def ranking_key(item):
    # item[1] là số lần: đổi âm để xếp nhiều trước; item[0] là tên để phá hòa theo ABC.
    return (-item[1], item[0])
# items() lấy các cặp (nhãn, số); sorted(...,key=...) dùng khóa tùy chỉnh, tạo list mới.
ranking = sorted(counts.items(), key=ranking_key)
# values() lấy các giá trị dictionary, không lấy khóa.
# sum() cộng tất cả số trong đối tượng được đưa vào.
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(labels) != 15 or sum(counts.values()) != 15:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đối chiếu đủ 15 nhãn được đếm.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Lượt:", len(labels))
count_labels = []
# sorted(counts) sắp xếp tên khóa để bảng số đếm nhất quán.
for name in sorted(counts):
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    count_labels.append(f"{name}={counts[name]}")
top_names = []
# [:2] lấy hai nhãn có thứ hạng cao nhất.
for name, count in ranking[:2]:
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    top_names.append(name)
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Đếm:", ", ".join(count_labels))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Top 2:", ", ".join(top_names))
```

Kết quả khi chạy:

```
Lượt: 15
Đếm: AI=5, Math=3, Python=4, SQL=3
Top 2: AI, Python
```

Mỗi nhãn là một khóa. Việc có hai nhãn cùng tần suất chỉ ảnh hưởng thứ tự in; số đếm từng nhãn vẫn phải đúng.

Tình huống 4 — Cấp 4: Kiểm tra 18 hồ sơ học viên lồng nhau

Từ vựng trong mục: Nested dictionary — Dictionary lồng nhau; **Missing** — thiếu điểm; **Validation** — kiểm tra hợp lệ.

Tình huống thực tế: Phòng đào tạo có **18 hồ sơ S01 đến S18**. Mỗi hồ sơ gồm mã, tên giả lập và điểm của học viên trên thang **0 đến 10**. Hai học viên chưa được nhập điểm nên để `None`; một học viên có điểm 11 là sai thang. Giáo vụ cần đếm riêng số hồ sơ có điểm hợp lệ, số chưa có điểm và số nhập sai, mà không dừng khi gặp dữ liệu thiếu.

Vì sao chọn cách này? Một hồ sơ có nhiều thông tin đi với cùng mã học viên nên lưu thành **dictionary bên trong dictionary**: tìm đúng mã trước rồi mới đọc trường điểm. `None` nghĩa là chưa nhập điểm, khác với 0 điểm là bài làm đã có kết quả. Chương trình phải kiểm tra `None` trước khi so với khoảng 0–10, vì không thể so sánh trực tiếp số với `None`. Điểm 11 phải đếm vào nhóm sai chứ không cho vào nhóm hợp lệ hay tự sửa thành 10. Cuối cùng ba nhóm hợp lệ, thiếu và sai phải khớp tổng 18 hồ sơ.

Quy trình:

1. Tạo 18 mã và các thuộc tính có ý nghĩa.
2. Kiểm tra giá trị thiếu trước, sau đó kiểm tra khoảng điểm hợp lệ.
3. Thống kê ba nhóm và xác minh tổng số hồ sơ.

No Note — Không chú thích

```
scores = [8,7,9,None,6.5,10,4,7.5,8.5,9,5,6,8,11,7,None,4.5,9]
students = {}
for i, value in enumerate(scores, 1):
    student_code = f"S{i:02d}"
    students[student_code] = {"name": f"Học viên {i:02d}", "score": value}
valid, missing, invalid = [], [], []
for code, info in students.items():
    score = info["score"]
    if score is None:
        missing.append(code)
    elif not (0 <= score <= 10):
        invalid.append(code)
    else:
        valid.append(code)
if len(students) != 18 or len(valid) + len(missing) + len(invalid) != 18:
    raise ValueError("Cần phân loại đủ 18 hồ sơ học viên.")
print("Hồ sơ:", len(students))
print("Hợp lệ:", len(valid))
print("Thiếu:", ", ".join(missing))
print("Sai thang:", ", ".join(invalid))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân loại 18 hồ sơ học viên thành điểm hợp lệ, thiếu và ngoài thang.
# Dữ liệu: 18 điểm học viên, None là thiếu; điểm hợp lệ thuộc đoạn 0..10.
scores = [8,7,9,None,6.5,10,4,7.5,8.5,9,5,6,8,11,7,None,4.5,9]
students = {}
# enumerate(...,1) gán mã S01..S18 theo thứ tự điểm.
for i, value in enumerate(scores, 1):
    # Định dạng f-string :02d cho số thứ tự thành 2 chữ số với số 0 bổ sung.
    student_code = f"S{i:02d}"
    # Dictionary con lưu hai trường name và score của từng học viên.
    students[student_code] = {"name": f"Học viên {i:02d}", "score": value}
valid, missing, invalid = [], [], []
# items() duyệt từng cặp (mã học viên, hồ sơ dict).
for code, info in students.items():
    score = info["score"]
    # is None nhận diện điểm chưa có, không nhầm với số 0.
    if score is None:
        # append() lưu mã chưa có điểm.
        missing.append(code)
    # Chuỗi so sánh kiểm tra ngoài khoảng 0..10, kể cả điểm 11.
    elif not (0 <= score <= 10):
        # append() lưu mã điểm hợp lệ.
        # append() lưu mã điểm ngoài thang.
        invalid.append(code)
    else:
        # append() lưu mã điểm hợp lệ.
        valid.append(code)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(students) != 18 or len(valid) + len(missing) + len(invalid) != 18:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần phân loại đủ 18 hồ sơ học viên.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Hồ sơ:", len(students))
```

```
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Hợp lệ:", len(valid))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Thiếu:", ", ".join(missing))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Sai thang:", ", ".join(invalid))
```

Kết quả khi chạy:

```
Hồ sơ: 18
Hợp lệ: 15
Thiếu: S04, S16
Sai thang: S14
```

Một giá trị 11 không phải thiếu nhưng vẫn không hợp lệ. Hai nguyên nhân cần được phân biệt để hướng dẫn nhân viên sửa đúng hồ sơ.

Tình huống 5 — Cấp 5: Xử lý 20 giao dịch nhập/xuất kho có kiểm chứng

Từ vựng trong mục: CRUD — thêm/đọc/sửa/xóa; **Dictionary get** — tra cứu mặc định; **Transaction** — giao dịch.

Tình huống thực tế: Một cửa hàng ghi lại **20 giao dịch kho** theo đúng thứ tự xảy ra. Mỗi dòng có mã sản phẩm, thao tác `add` (nhập thêm) hoặc `sell` (bán/xuất), và số món muốn nhập hoặc xuất. Có lượt bán vượt số lượng hiện có, lượt bán một mã chưa từng nhập và lượt thiếu số món. Người quản lý muốn biết giao dịch nào được chấp nhận hay bị từ chối, tổng số lần thành công/thất bại và tồn cuối của từng loại hàng.

Vì sao chọn cách này? Tồn kho là **trạng thái thay đổi theo từng giao dịch**, vì thế thứ tự 20 lượt rất quan trọng: một lượt nhập trước có thể giúp lượt bán sau đủ hàng. Dictionary giữ số món hiện tại của mỗi mã. Với `add`, kiểm tra mã và lượng nhập rồi cộng; với `sell`, chỉ trừ khi mã đã có và còn đủ món. Dòng thiếu số lượng, bán SKU chưa tồn tại hay bán quá số tồn đều bị **từ chối và không làm thay đổi kho**. Sau mỗi lượt tăng bộ đếm thành công hoặc thất bại, rồi đọc dictionary để có tồn cuối. Không thể cộng tất cả lượt nhập rồi trừ tất cả lượt bán trước, vì cách đó bỏ qua những giao dịch bị từ chối ở thời điểm chúng xảy ra.

Quy trình:

1. Khởi tạo kho rỗng, duyệt lần lượt 20 tuple giao dịch.
2. Với nhập, cộng tồn; với xuất, kiểm tra số dương, SKU tồn tại và đủ số món.
3. Ghi lại số thứ tự giao dịch lỗi, đối chiếu số lượt và in bảng tồn đã sắp mã.

No Note — Không chú thích

```
events = [("P01","add",12),("P02","add",9),("P03","add",8),("P01","sell",2),("P02","sell",3),
("P04","add",5),("P05","add",7),("P03","sell",1),("P01","sell",20),("P06","add",15),
("P04","sell",2),("P02","sell",1),("P07","add",4),("P05","sell",2),("P08","add",6),
("P03","sell",3),("P02","add",5),("P09","sell",1),("P06","sell",4),("P07","sell",None)]
stock, rejected = {}, []
for index, (sku, action, quantity) in enumerate(events, 1):
    if quantity is None or quantity <= 0:
        rejected.append(index)
    elif action == "add":
        stock[sku] = stock.get(sku, 0) + quantity
    elif action == "sell" and sku in stock and stock[sku] >= quantity:
        stock[sku] -= quantity
    else:
        rejected.append(index)
if len(events) != 20:
```

```
    raise ValueError("Cần kiểm tra đủ 20 giao dịch kho.")
print("Giao dịch:", len(events))
print("Đã nhận:", len(events) - len(rejected))
rejected_labels = []
for index in rejected:
    rejected_labels.append(str(index))
stock_labels = []
for sku in sorted(stock):
    stock_labels.append(f"{sku}={stock[sku]}")
print("Từ chối lượt:", ", ".join(rejected_labels))
print("Tồn cuối:", ", ".join(stock_labels))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Áp dụng 20 giao dịch kho qua dictionary, từ chối giao dịch sai.
# Dữ liệu: 20 tuple (SKU, hành động add/sell, số lượng), None nghĩa chưa có số lượng.
events = [("P01","add",12),("P02","add",9),("P03","add",8),("P01","sell",2),("P02","sell",3),
("P04","add",5),("P05","add",7),("P03","sell",1),("P01","sell",20),("P06","add",15),
("P04","sell",2),("P02","sell",1),("P07","add",4),("P05","sell",2),("P08","add",6),
("P03","sell",3),("P02","add",5),("P09","sell",1),("P06","sell",4),("P07","sell",None)]
stock, rejected = {}, []
# enumerate(...,1) đánh số giao dịch kho 1..20 để báo dòng lỗi.
for index, (sku, action, quantity) in enumerate(events, 1):
    # or từ chối số lượng thiếu hoặc không dương ngay từ đầu.
    if quantity is None or quantity <= 0:
        # append() lưu số thứ tự giao dịch từ chối.
        rejected.append(index)
    # Nhánh add tăng tồn kho của SKU.
    elif action == "add":
        # get(sku,0) lấy tồn hiện có hoặc 0 khi SKU mới, rồi cộng số nhập.
        stock[sku] = stock.get(sku, 0) + quantity
    # and buộc hành động sell, SKU đã tồn tại và lượng kho đủ xuất.
    elif action == "sell" and sku in stock and stock[sku] >= quantity:
        # -= giảm tồn của chính SKU sau khi kiểm đủ.
        stock[sku] -= quantity
    else:
        # append() lưu số thứ tự giao dịch từ chối.
        rejected.append(index)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(events) != 20:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần kiểm tra đủ 20 giao dịch kho.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Giao dịch:", len(events))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Đã nhận:", len(events) - len(rejected))
rejected_labels = []
for index in rejected:
    # str() chuyển số thứ tự giao dịch thành chuỗi cho join().
    rejected_labels.append(str(index))
stock_labels = []
# sorted(stock) xếp tên các SKU trước khi in.
for sku in sorted(stock):
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    stock_labels.append(f"{sku}={stock[sku]}")
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Từ chối lượt:", ", ".join(rejected_labels))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Tồn cuối:", ", ".join(stock_labels))
```

Kết quả khi chạy:

```
Giao dịch: 20
Đã nhận: 17
Từ chối lượt: 9, 18, 20
Tồn cuối: P01=10, P02=10, P03=4, P04=3, P05=5, P06=11, P07=4, P08=6
```

Phiếu bị từ chối giữ nguyên số tồn trước đó. Không được âm thầm tạo SKU P09 tồn 0 rồi trừ thành -1, vì kho chưa hề nhận mặt hàng này.

6. Nhầm lẫn dễ gặp

- `obj[key]` gây `KeyError` nếu khóa chưa tồn tại; `obj.get(key)` trả về `None` hoặc giá trị mặc định đã truyền vào.
- Gán `obj[key] = value` sẽ thêm khóa mới hoặc ghi đè giá trị đang có.
- `pop(key)` lấy và xóa giá trị; nếu khóa không tồn tại, nên truyền giá trị mặc định khi phù hợp.
- Khóa Dictionary phải là kiểu có thể băm (hashable), như chuỗi hoặc số; không dùng List làm khóa.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a dictionary? | Dictionary là gì?

A. An unordered collection of unique values only | Tập không thứ tự chỉ gồm giá trị duy nhất

B. An immutable pair of numbers | Cặp số bất biến

C. A mapping from hashable keys to values | Ánh xạ từ khóa có thể băm sang giá trị

D. A sequence with numeric positions only | Dãy chỉ truy cập bằng vị trí số

Đáp án: C

Giải thích: Dictionary ánh xạ các khóa khác nhau, có thể băm, tới giá trị tương ứng, ví dụ `{'name':'An', 'score':9}`. Bạn truy cập giá trị qua khóa thay vì chỉ số vị trí như List.

Q2. What does a dictionary key do? | Khóa Dictionary có vai trò gì?

A. Specifies the dictionary's length | Xác định độ dài Dictionary

B. Stores all keys as values | Lưu mọi khóa thành giá trị

C. Sorts the stored values | Sắp xếp các giá trị đã lưu

D. Identifies an entry for lookup | Xác định mục để tra cứu

Đáp án: D

Giải thích: Khóa xác định một mục để tra cứu, cập nhật hoặc xóa giá trị. Trong `{'age': 20}`, khóa `'age'` dẫn đến giá trị `20`; khóa không đồng nghĩa với giá trị đi kèm.

Q3. What is a dictionary value? | Giá trị Dictionary là gì?

A. The key's index position | Vị trí chỉ số của khóa

B. Data associated with a key | Dữ liệu gắn với khóa

C. Always a second dictionary | Luôn là Dictionary thứ hai

D. A name used only in a `for` loop | Tên chỉ dùng trong vòng `for`

Đáp án: B

Giải thích: Giá trị Dictionary là dữ liệu lưu dưới một khóa, chẳng hạn số `20` dưới khóa `'age'`. Giá trị có thể thuộc nhiều kiểu kể cả List hoặc Dictionary, trong khi khóa phải là đối tượng có thể băm.

Q4. What is a nested dictionary? | Dictionary lồng nhau là gì?

A. A dictionary stored inside another dictionary | Dictionary được lưu trong Dictionary khác

B. A list nested inside a set | List lồng trong Set

C. A set of duplicate keys | Set gồm khóa trùng

D. A dictionary with only numeric keys | Dictionary chỉ có khóa số

Đáp án: A

Giải thích: Dictionary lồng nhau là Dictionary được lưu làm giá trị trong Dictionary khác. Ví dụ `{'A': {'score': 9}}` gom trường `score` dưới khóa `'A'`, và hai lần truy cập ngoặc vuông sẽ lấy được con số.

Q5. What does `d['x']` do when x exists? | `d['x']` làm gì khi khóa x tồn tại?

A. Always returns `None` | Luôn trả `None`

B. Retrieves the value stored under `'x'` | Lấy giá trị lưu dưới khóa `'x'`

C. Returns `'x'` as a string | Trả chuỗi `'x'`

D. Deletes the entry for `'x'` | Xóa mục có khóa `'x'`

Đáp án: B

Giải thích: Khi khóa `'x'` tồn tại, `d['x']` lấy giá trị gắn với khóa đó. Khác `d.get('x')`, truy cập ngoặc vuông sẽ gây `KeyError` nếu khóa không tồn tại.

Q6. What does `d.get('x')` return if x is absent? | `d.get('x')` trả về gì nếu thiếu khóa x?

A. `KeyError` | `KeyError`

B. `None` by default | Mặc định là `None`

C. The empty dictionary `{}` | Dictionary rỗng `{}`

D. The string `'x'` | Chuỗi `'x'`

Đáp án: B

Giải thích: Khi không cung cấp mặc định, `d.get('x')` trả `None` nếu thiếu khóa thay vì gây `KeyError`. Cách này giúp xử lý dữ liệu thiếu, nhưng bản thân `None` cũng có thể là giá trị được lưu thật.

Q7. What does `d.get('x', 0)` return if x is absent? | `d.get('x', 0)` trả về gì nếu thiếu x?

A. The integer `0` | Số nguyên `0`

B. A newly inserted entry | Một mục mới được thêm

C. The string `'0'` | Chuỗi `'0'`

D. `None` | `None`

Đáp án: A

Giải thích: Đối số thứ hai của `get` là giá trị dự phòng khi không tìm thấy khóa, nên `d.get('x', 0)` trả số nguyên `0` nếu thiếu `'x'`. Lệnh không tạo khóa đó và không ghi đè giá trị đang có.

Q8. What does `d['missing']` raise? | `d['missing']` phát sinh gì khi thiếu khóa?

A. `0` | `0`

B. An empty list | List rỗng

C. `KeyError` | `KeyError`

D. `None` | `None`

Đáp án: C

Giải thích: Truy cập trực tiếp `d['missing']` gây `KeyError` khi không có khóa. Với dữ liệu tùy chọn, `get` có thể trả giá trị dự phòng; sự khác biệt này quyết định có xem việc thiếu khóa là lỗi hay không.

Q9. What does `d.items()` provide? | `d.items()` cung cấp gì?

A. A sorted list of keys | List khóa đã sắp

B. A view of key-value pairs | View của các cặp khóa–giá trị

C. Only a view of values | Chỉ view các giá trị

D. Only a view of keys | Chỉ view các khóa

Đáp án: B

Giải thích: `d.items()` trả một view động của các cặp khóa–giá trị, thường duyệt bằng `for key, value in d.items()`. Nó khác `keys()` chỉ đưa ra khóa và `values()` chỉ đưa ra giá trị.

Q10. What does `d.keys()` provide? | `d.keys()` cung cấp gì?

A. A view of values | View các giá trị

B. A list of keys sorted automatically | List khóa tự sắp

C. A view of pairs | View các cặp khóa–giá trị

D. A view of dictionary keys | View của các khóa Dictionary

Đáp án: D

Giải thích: `d.keys()` cung cấp view động của các khóa Dictionary. Với `{'a': 1}`, khi duyệt view này nhận `'a'`, không phải giá trị `1` hay cả cặp khóa–giá trị.

Q11. What does `d.values()` provide? | `d.values()` cung cấp gì?

A. A view of dictionary values | View các giá trị Dictionary

B. A dictionary copy | Bản sao Dictionary

C. A view of keys | View các khóa

D. A view of key-value pairs | View các cặp khóa–giá trị

Đáp án: A

Giải thích: `d.values()` trả view động của các giá trị đang lưu. Với `{'a': 1, 'b': 2}`, khi duyệt thu được 1 và 2; muốn kèm khóa tương ứng thì dùng `items()`.

Q12. What does `d.pop('x', None)` do if x is present? | `d.pop('x', None)` làm gì nếu x tồn tại?

A. Returns the value without deletion | Trả giá trị nhưng không xóa

B. Removes the key but always returns `None` | Xóa khóa nhưng luôn trả `None`

C. Deletes every dictionary entry | Xóa mọi mục Dictionary

D. Removes `'x'` and returns its former value | Xóa `'x'` và trả giá trị cũ

Đáp án: D

Giải thích: Khi `'x'` tồn tại, `d.pop('x', None)` xóa mục tương ứng và trả về giá trị cũ. Đối số `None` chỉ là giá trị dự phòng nếu không có khóa; nó không ép kết quả luôn thành `None`.

Q13. What does `d['x'] = 5` do if x already exists? | `d['x'] = 5` làm gì khi x đã tồn tại?

A. Replaces the value associated with `'x'` | Thay giá trị gắn với `'x'`

B. Adds a second duplicate `'x'` key | Thêm khóa `'x'` trùng thứ hai

C. Deletes the key `'x'` | Xóa khóa `'x'`

D. Tests whether `'x'` exists | Kiểm tra `'x'` tồn tại không

Đáp án: A

Giải thích: Gán `d['x'] = 5` khi khóa `'x'` đã tồn tại sẽ thay giá trị của mục đó thành `5`. Lệnh không tạo thêm một khóa `'x'` thứ hai vì mỗi khóa Dictionary là duy nhất.

Q14. What does `d['new'] = 5` do if new is absent? | `d['new'] = 5` làm gì khi khóa new chưa có?

A. Deletes an absent key | Xóa một khóa không tồn tại

B. Returns the total dictionary length | Trả tổng độ dài Dictionary

C. Replaces an unrelated existing key | Thay một khóa cũ không liên quan

D. Inserts a new key-value entry | Thêm một mục khóa–giá trị mới

Đáp án: D

Giải thích: Gán `d['new'] = 5` khi chưa có khóa sẽ thêm cặp `'new'` ánh xạ tới 5. Cùng cú pháp gán này lại cập nhật giá trị nếu khóa đã tồn tại.

Q15. What does `del d['x']` do for an existing key? | `del d['x']` làm gì với khóa đang tồn tại?

A. Deletes that key and its value | Xóa khóa đó cùng giá trị

B. Deletes the entire dictionary | Xóa toàn bộ Dictionary

C. Leaves the entry intact and returns its value | Giữ nguyên mục và trả giá trị

D. Replaces its value with `None` | Thay giá trị bằng `None`

Đáp án: A

Giải thích: `del d['x']` xóa khóa có sẵn cùng giá trị tương ứng khỏi Dictionary. Khác `pop`, câu lệnh `del` không trả về giá trị đã xóa.

Q16. What does `'x' in d` test? | `'x' in d` kiểm tra gì?

A. Whether `'x'` is a numeric index | `'x'` có phải chỉ số số hay không

B. Whether any key maps to `'x'` | Có khóa nào ánh xạ tới `'x'` không

C. Whether `'x'` is a dictionary key | `'x'` có là khóa Dictionary không

D. Whether `'x'` appears in values only | `'x'` có nằm trong các giá trị hay không

Đáp án: C

Giải thích: Với Dictionary, `in` mặc định kiểm tra khóa: `'x' in {'x': 3}` là `True`, còn `3 in {'x': 3}` là `False`. Muốn kiểm tra giá trị thay vì khóa, hãy xét thành viên trong `d.values()`.

Q17. May a List be a normal dictionary key? | Có thể dùng List làm khóa Dictionary thông thường không?

A. Yes, if the list has two elements | Có, nếu List có hai phần tử

B. Yes, if list values are distinct | Có, nếu các giá trị List khác nhau

C. No, lists are unhashable | Không, List không thể băm

D. Yes, every list is hashable | Có, mọi List đều có thể băm

Đáp án: C

Giải thích: List có thể thay đổi nên không thể băm và không dùng làm khóa Dictionary thông thường. Thử `{[1,2]: 'x'}` gây `TypeError`; một Tuple chứa các số có thể băm thì dùng được.

Q18. Can a tuple of integers be a dictionary key? | Tuple chứa số nguyên có thể làm khóa Dictionary không?

A. No, all tuples are unhashable | Không, mọi Tuple đều không thể băm

B. Yes, a tuple of integers is hashable | Có, Tuple chứa số nguyên có thể băm

C. Only if the tuple has one element | Chỉ nếu Tuple có một phần tử

D. Only if the tuple is empty | Chỉ nếu Tuple rỗng

Đáp án: B

Giải thích: Tuple chỉ chứa số nguyên có thể băm nên dùng làm khóa Dictionary, ví dụ `{(1,2): 'point'}`. Không phải mọi Tuple đều dùng được: Tuple chứa List cũng không thể băm vì một phần tử bên trong không thể băm.

Q19. How do you access the score in `d['A']` when it is a nested dictionary? | Lấy điểm trong Dictionary con `d['A']` thế nào?

A. `d['A', 'score']` | `d['A', 'score']`

B. `d['score']['A']` | `d['score']['A']`

C. `d.get('A', 'score')` | `d.get('A', 'score')`

D. `d['A']['score']` | `d['A']['score']`

Đáp án: D

Giải thích: `d['A']['score']` trước tiên lấy Dictionary con dưới khóa `'A'`, rồi tra mục `'score'` trong đó. Với `d={'A':{'score':9}}`, biểu thức trả 9; nếu thiếu một trong các khóa thì cách truy cập trực tiếp gây `KeyError`.

Q20. What must be unique within one dictionary? | Điều gì phải duy nhất trong một Dictionary?

A. Dictionary values | Các giá trị Dictionary

B. Both all keys and all values must be unique | Cả khóa và giá trị đều phải duy nhất

C. Dictionary keys | Các khóa Dictionary

D. Neither keys nor values must be unique | Cả khóa lẫn giá trị đều không cần duy nhất

Đáp án: C

Giải thích: Các khóa là duy nhất trong một Dictionary; gán lại cùng một khóa sẽ cập nhật mục cũ thay vì tạo thêm bản sao. Giá trị có thể lặp ở nhiều khóa nên tính duy nhất áp dụng cho khóa chứ không phải giá trị.

8. Nối sang bài tiếp theo

`List/Set → Dictionary → JSON/CSV → DataFrame`.

PY-09 — Hàm, return, lambda và phạm vi biến

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

`Vòng lặp → Hàm → OOP → Pipeline`

Học xong dùng để làm gì? Đóng gói phép tính thành hàm nhận dữ liệu, trả kết quả; dùng lambda làm khóa sắp xếp.

Kết nối kiến thức tiếp theo: PY-10 tổ chức dữ liệu và thao tác của một đối tượng thành class.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Function	hàm	Khối code tái sử dụng và nhận tham số
Parameter	tham số	Tên đầu vào dùng trong định nghĩa hàm
Return	trả về	Kết quả gửi từ hàm về nơi gọi
Lambda	hàm ẩn danh	Biểu thức hàm ngắn không cần đặt tên

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`def func(...):`	Định nghĩa một hàm, sử dụng return để trả kết quả
`default / keyword arguments`	Dùng tham số mặc định hoặc gọi theo tên cho dễ đọc
`lambda x: expr`	Dùng biểu thức ngắn khi hàm không cần nhiều câu lệnh
`sorted(..., key=...)`	Chọn tiêu chí để sắp xếp bản ghi
Biến cục bộ trong `def`	Tên tạo trong hàm thường chỉ dùng được trong hàm, không truy cập trực tiếp từ bên ngoài

4. Khái niệm liên quan

Một **hàm (function)** gom nhiều bước xử lý thành một công việc có tên, có thể gọi lại bằng dữ liệu khác nhau. Khi gọi hàm, giá trị đi vào qua **tham số**, kết quả có thể quay lại bằng **`return`**. `lambda` phù hợp cho một biểu thức ngắn, còn **phạm vi biến (scope)** giúp phân biệt biến chỉ dùng bên trong hàm với biến dùng bên ngoài. Ta dùng cùng ý tưởng tính tiền để lần lượt hiểu năm phần này.

4.1. Function — Hàm tái sử dụng

Từ vựng trong mục: Function — hàm; **Call** — lời gọi hàm.

Giả sử cửa hàng cần tính giá của món hàng nhân đôi khi mua hai phần giống nhau. Nếu viết phép nhân tại nhiều chỗ, thay đổi quy tắc sẽ khó. Ta định nghĩa `twice(x)` bằng từ khóa `def`: `x` là đầu vào, phép `x * 2` tạo kết quả. Khi viết `twice(3)`, Python chạy thân hàm với `x = 3`, rồi trả số 6. Dòng `print()` bên ngoài mới đưa số này lên màn hình.

No Note — Không chú thích

```
def twice(x):
    return x * 2

print(twice(3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo hàm twice để nhân đôi số theo đối số.
# Dữ liệu: twice nhận số x và trả giá trị gấp đôi, không in trong hàm.
# def tạo hàm có tham số x; chưa thực thi thân hàm cho đến khi gọi.
def twice(x):
    # return trả x nhân 2 về bên gọi; không tự in.
    return x * 2

# Gọi twice với đối số 3, print in kết quả 6.
print(twice(3))
```

Kết quả khi chạy:

```
6
```

Một định nghĩa hàm có thể dùng lại cho nhiều giá trị như 3, 5 hoặc 10 mà không viết lại phép nhân.

4.2. Parameter và Argument — Tham số và đối số

Từ vựng trong mục: Parameter — tham số; **Argument** — đối số.

Tham số là tên xuất hiện trong dòng định nghĩa, ví dụ `name` của `hello(name)`. **Đối số** là giá trị thật được truyền lúc gọi hàm, ví dụ `'Lan'` trong `hello('Lan')`. Trong thân hàm, `name` tạm nhận `'Lan'`, tạo chuỗi lời chào rồi `return` gửi nó về nơi gọi.

No Note — Không chú thích

```
def hello(name):
    return f'Xin chào {name}!'

print(hello('Lan'))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt tham số khi định nghĩa hàm và đối số khi gọi.
# Dữ liệu: hello nhận tên làm tham số, chuỗi kết quả tạo bằng f-string.
# name là tham số nhận chuỗi khi gọi hàm.
def hello(name):
```

```
    # f-string chèn giá trị name vào thông điệp và return trả chuôĩ.
    return f'Xin chào {name}!'

# Truyền 'Lan' làm đôí sô´thực tê´cho tham sô´name.
print(hello('Lan'))
```

Kết quả khi chạy:

```
Xin chào Lan!
```

`name` là tham số, còn `'Lan'` là đối số của lời gọi cụ thể. Truyền `'An'` sẽ tạo lời chào khác.

4.3. Return — Kết quả trả về

Từ vựng trong mục: Return — trả về; **Print** — hiển thị.

`return` khác `print()`: `return` **gửi giá trị về nơi gọi hàm** để chương trình có thể lưu hoặc tính tiếp; `print()` chỉ hiển thị. Ví dụ `add(a, b)` tính tổng hai số. Khi gán `total = add(2, 3)`, biến `total` nhận 5; lệnh `print(total)` mới làm số 5 xuất hiện trên màn hình.

No Note — Không chú thích

```
def add(a, b):
    return a + b

total = add(2, 3)
print(total)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng return trả tổng hai đôí sô´cho biến bên ngoài.
# Dữ liệu: Hàm add có hai tham sô´a, b; return đưa tổng vê`bên gọi.
# Hai tham sô´a,b nhận hai sô´của người gọi.
def add(a, b):
    # return đưa tổng cho bên gọi, kêt thúc hàm.
    return a + b

# Gọi hàm với hai đôí sô´2 và 3, gán kêt quả 5 vào total.
total = add(2, 3)
print(total)
```

Kết quả khi chạy:

```
5
```

Giá trị trả về có thể lưu trong biến `total`; nếu hàm chỉ `print()` mà không `return`, nơi gọi không nhận được tổng để tính tiếp.

4.4. Lambda — Hàm ngắn dùng một biểu thức

Từ vựng trong mục: Lambda — hàm ẩn danh; **Expression** — biểu thức.

Khi chỉ cần phép biến đổi đơn giản, `lambda x: x + 1` là cách viết ngắn của một hàm nhận `x` và trả về `x + 1`. Chẳng hạn tăng số lượng hiện tại 5 lên thêm một món sẽ ra 6. `lambda` thích hợp làm tiêu chí sắp xếp hay lọc; công việc có nhiều bước nên dùng `def` để dễ đọc.

No Note — Không chú thích

```
increase = lambda x: x + 1
print(increase(5))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo hàm lambda một biểu thức cộng 1.
# Dữ liệu: lambda là hàm không đặt tên tạo trực tiếp bằng một biểu thức.
# lambda nhận tham sô´x và trả x+1 ngay, không cần def/return.
increase = lambda x: x + 1
# Gọi hàm vừa tạo, nhận 6.
print(increase(5))
```

Kết quả khi chạy:

```
6
```

Hàm được tạo bằng `lambda` nhận 5 và cho 6. Đây là một biểu thức, không có thân nhiều câu lệnh như `def`.

4.5. Scope — Phạm vi biến

Từ vựng trong mục: Scope — phạm vi biến; **Local variable** — biến cục bộ.

Phạm vi biến xác định nơi một tên biến có thể được dùng. Trong hàm `with_fee(price)`, biến `fee = 5` chỉ được tạo khi hàm chạy và dùng trong hàm; đó là **biến cục bộ (local)**. Bên ngoài hàm, chương trình chỉ dùng giá trị `return`, không truy cập trực tiếp `fee`. Ví dụ đơn hàng giá 100 (nghìn đồng) có phí xử lý 5 (nghìn đồng).

No Note — Không chú thích

```
def with_fee(price):
    fee = 5
    return price + fee

final_price = with_fee(100)
print(final_price)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt biến fee cục bộ trong hàm và giá bên ngoài.
# Dữ liệu: fee là biến chỉ tồn tại khi chạy with_fee, không phải biến toàn cục.
# price là tham sô´của hàm, tách khỏi giá trị bên ngoài.
def with_fee(price):
    # fee là biến cục bộ tạo trong hàm mỗi lần gọi.
    fee = 5
    # return cộng phí 5 vào giá và trả lại kết quả.
    return price + fee

# Truyền giá 100, kết quả là 105.
final_price = with_fee(100)
print(final_price)
```

Kết quả khi chạy:

```
105
```

Số 105 là tổng giá và phí, đơn vị nghìn đồng. Biến `fee` ở trong hàm; nếu gọi `print(fee)` ở ngoài, Python sẽ báo `NameError`.

5. Năm tình huống hàm — từ return cơ bản đến pipeline 20 đơn

Tình huống 1 — Cấp 1: Dùng một hàm tính giá đã cộng thuế của 10 món

Từ vựng trong mục: Function — hàm; **Return** — trả về; **Default argument** — đối số mặc định.

Tình huống thực tế: Một cửa hàng văn phòng phẩm có bảng giá trước thuế của **10 mặt hàng**, mỗi giá tính bằng **đồng/món**. Trong ví dụ giả lập, cửa hàng áp dụng mức thuế **10%** đồng đều và khách mua một món mỗi loại. Thu ngân muốn tính giá sau thuế từng món rồi cộng tổng số tiền phải trả, không phải nhập lại cùng cách tính cho cả mười món.

Vì sao chọn cách này? Hàm là một đoạn xử lý được đặt tên để dùng lại nhiều lần. `def` tạo hàm tính giá một món sau thuế; **tham số** nhận đơn giá của từng mặt hàng. `return` trả số tiền mới về nơi gọi, nên vòng `for` có thể lấy kết quả của 10 món để cộng tổng. Nếu chỉ dùng `print()` trong hàm thì màn hình có thể hiện số nhưng chỗ tính tổng không nhận được giá trị để cộng. Thuế 10% chỉ là giả định cho dữ liệu minh họa, không khẳng định mọi sản phẩm ngoài đời đều chịu mức này.

Quy trình:

1. Định nghĩa hàm `with_tax(price, rate=0.10)` có tham số mặc định.
2. Gọi hàm cho mỗi trong 10 đơn giá.
3. Tổng hợp, kiểm tra số lượng và in tổng.

No Note — Không chú thích

```
prices_vnd = [50000,72000,90000,110000,85000,60000,95000,120000,140000,100000]
def with_tax(price, rate=0.10):
    return round(price * (1 + rate))
prices_after_tax = []
for price in prices_vnd:
    prices_after_tax.append(with_tax(price))
if len(prices_after_tax) != 10:
    raise ValueError("Cần tính thuế đủ 10 đơn giá.")
print("Mặt hàng:", len(prices_vnd))
print("Giá món đầu:", prices_after_tax[0])
print("Tổng tiền (đồng):", sum(prices_after_tax))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Viết hàm tính giá có thuế mặc định 10% cho 10 món.
# Dữ liệu: 10 giá đồng Việt Nam; rate=0.10 là thuế suất mặc định 10%.
prices_vnd = [50000,72000,90000,110000,85000,60000,95000,120000,140000,100000]
# rate có giá trị mặc định 0.10: có thể không truyền để dùng thuế 10%.
def with_tax(price, rate=0.10):
    # Nhân đơn giá với 1.10 để cộng thuế, round() làm tròn tới số nguyên đồng.
    return round(price * (1 + rate))
prices_after_tax = []
for price in prices_vnd:
    # append() tích lũy tiền sau thuế từng món.
    prices_after_tax.append(with_tax(price))
```

```
# len() trả sô´phần tư, dùng đẻđếm hoặc kiểm tra đủ bản ghi.
if len(prices_after_tax) != 10:
    # raise báo lôĩ rôì dừng luôồng hiện tại nêú dữ liệu không đáp ứng yêu câù.
    raise ValueError("Cần tính thuê´đủ 10 đơn giá.")
# len() trả sô´phần tư, dùng đẻđếm hoặc kiểm tra đủ bản ghi.
print("Mặt hàng:", len(prices_vnd))
# [0] chỉ phần tư đầu tiên; Python đánh sô´vị trí từ 0.
print("Giá món đầu:", prices_after_tax[0])
# sum() cộng tổng mười mức giá đã có thuê´
print("Tổng tiền (đồng):", sum(prices_after_tax))
```

Kết quả khi chạy:

```
Mặt hàng: 10
Giá món đầu: 55000
Tổng tiền (đồng): 1014200
```

Hàm trả về số, không tự in ra; có thể dùng tiếp trong các phép tính khác mà không phải sao chép biểu thức.

Tình huống 2 — Cấp 2: Dùng lambda sắp 12 học viên giữ thứ tự hòa điểm

Từ vựng trong mục: Lambda — hàm ngắn; **Key function** — hàm chọn khóa; **Sorted** — sắp xếp.

Tình huống thực tế: Trung tâm nhận **12 hồ sơ học viên** theo thứ tự gửi đơn. Mỗi hồ sơ có tên và điểm trên thang **10**; vài học viên có cùng điểm. Người phụ trách muốn xếp học viên theo điểm từ cao xuống thấp để chọn **bốn người phỏng vấn trước**. Khi hai người đồng điểm, ai gửi hồ sơ trước phải đứng trước, không được đảo ngẫu nhiên thứ tự.

Vì sao chọn cách này? Sắp xếp là thay đổi thứ tự hiển thị, còn **khóa sắp xếp** (key) cho Python biết cần nhìn vào thông tin nào. Ở đây khóa là trường `score`, không phải tên. `lambda item: item['score']` là một hàm nhỏ chỉ dùng tại chỗ để lấy điểm của từng hồ sơ; `reverse=True` cho điểm cao đứng trước. `sorted()` tạo danh sách mới; Python giữ thứ tự ban đầu của các phần tử có **khóa bằng nhau**, nên người nộp trước vẫn trước khi hòa điểm. Sau đó chỉ chọn bốn hồ sơ đầu, không dùng thứ tự tên hay thời gian nộp để thay thế điểm.

Quy trình:

1. Lưu 12 Dictionary theo thứ tự tiếp nhận.
2. Viết lambda đọc cột `score` làm khóa cho `sorted`.
3. Lấy bốn tên đầu và kiểm tra danh sách nguồn không đổi.

No Note — Không chú thích

```
students = [{"name":"An","score":8},{"name":"Bình","score":9},{"name":"Chi","score":7},
{"name":"Dung","score":9},{"name":"Hà","score":6},{"name":"Khánh","score":10},
{"name":"Lan","score":8},{"name":"Minh","score":9},{"name":"Nam","score":5},
{"name":"Trang","score":10},{"name":"Vy","score":7},{"name":"Yến","score":9}]
ranked = sorted(students, key=lambda item: item["score"], reverse=True)
if len(students) != 12:
    raise ValueError("Cần đủ 12 hô`sơ đăng ký.")
if students[0]["name"] != "An":
    raise ValueError("Danh sách gôć không được đôỉ thứ tự nộp.")
print("Học viên:", len(students))
interview_names = []
for student in ranked[:4]:
    interview_names.append(student["name"])
```

```
print("Phỏng vấn trước:", ", ".join(interview_names))
print("Nguồn đầu:", students[0]["name"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng lambda làm khóa xếp 12 học viên theo điểm giảm.
# Dữ liệu: 12 dict hồ sơ gồm name và score, điểm dùng để ưu tiên phỏng vấn.
students = [{"name":"An","score":8},{"name":"Bình","score":9},{"name":"Chi","score":7},
{"name":"Dung","score":9},{"name":"Hà","score":6},{"name":"Khánh","score":10},
{"name":"Lan","score":8},{"name":"Minh","score":9},{"name":"Nam","score":5},
{"name":"Trang","score":10},{"name":"Vy","score":7},{"name":"Yến","score":9}]
# key=lambda item: item['score'] chọn trường điểm làm khóa; reverse=True xếp điểm lớn trước,
sorted() không sửa list gốc.
ranked = sorted(students, key=lambda item: item["score"], reverse=True)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(students) != 12:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 12 hồ sơ đăng ký.")
# [0] chỉ phần tử đầu tiên; Python đánh số vị trí từ 0.
if students[0]["name"] != "An":
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Danh sách gốc không được đổi thứ tự nộp.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Học viên:", len(students))
interview_names = []
# [:4] lấy bốn học viên đứng đầu, cận 4 bị loại.
for student in ranked[:4]:
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    interview_names.append(student["name"])
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Phỏng vấn trước:", ", ".join(interview_names))
# [0] chỉ phần tử đầu tiên; Python đánh số vị trí từ 0.
print("Nguồn đầu:", students[0]["name"])
```

Kết quả khi chạy:

```
Học viên: 12
Phỏng vấn trước: Khánh, Trang, Bình, Dung
Nguồn đầu: An
```

Lambda chỉ định trường điểm để so, không làm thay đổi thứ tự ban đầu của hồ sơ khi các điểm bằng nhau.

Tình huống 3 — Cấp 3: Tách hàm kiểm tra điểm và hàm xét đạt cho 15 lượt

Từ vựng trong mục: Parameter — tham số; **Boolean return** — trả về đúng/sai; **Composition** — phối hợp hàm.

Tình huống thực tế: Giáo vụ có **15 lượt ghi điểm kiểm tra**, mỗi lượt là một giá trị trên thang **0 đến 10** hoặc dữ liệu chưa có. Danh sách có điểm 11 (quá cao), -1 (không hợp lệ) và `None` (chưa nhập). Điểm hợp lệ từ 5 trở lên được tính là đạt. Giáo vụ muốn biết có bao nhiêu điểm hợp lệ, bao nhiêu người đạt và bao nhiêu lượt chưa thể xét.

Vì sao chọn cách này? Để tránh xét sai, tách thành **hai câu hỏi**: giá trị này có phải điểm hợp lệ 0–10 không, và nếu hợp lệ thì có đạt mức 5 không. Hàm `valid_score()` trả kết quả đúng/sai cho câu hỏi đầu; hàm `passed()` dùng lại kiểm tra đó trước khi xét ngưỡng. Như vậy 11, -1 và `None` không bị nhầm thành điểm đạt hoặc trượt bình thường. Hàm nhỏ dễ thử bằng nhiều dữ liệu; viết riêng quy tắc ở từng nơi sẽ dễ có chỗ quên kiểm tra số thiếu.

Quy trình:

1. Định nghĩa `valid_score()` trả `True/False` trên dữ liệu đầu vào.
2. Định nghĩa `passed()` dùng kết quả kiểm tra để xét ngưỡng 5.
3. Duyệt đủ 15 lượt và báo các nhóm không chồng lấn.

No Note — Không chú thích

```
scores = [4,7,9,10,2,5,11,8,None,6,0,3,7.5,9.5,-1]
def valid_score(value):
    return isinstance(value, (int, float)) and 0 <= value <= 10

def passed(value):
    return valid_score(value) and value >= 5
valid = []
passed_scores = []
for score in scores:
    if valid_score(score):
        valid.append(score)
        if passed(score):
            passed_scores.append(score)
if len(scores) != 15:
    raise ValueError("Cần kiểm tra đủ 15 kết quả điểm.")
print("Lượt nhận:", len(scores))
print("Hợp lệ:", len(valid))
print("Đạt:", len(passed_scores))
print("Không hợp lệ:", len(scores) - len(valid))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách hàm kiểm tra điểm hợp lệ và hàm kiểm tra điểm đạt cho 15 kết quả.
# Dữ liệu: 15 số điểm gồm None, số âm và số >10 cần lọc.
scores = [4,7,9,10,2,5,11,8,None,6,0,3,7.5,9.5,-1]
# Hàm kiểm tra đầu vào thuộc số nguyên hoặc số thực và trong thang điểm.
def valid_score(value):
    # isinstance() kiểm tra kiểu là int hoặc float trước khi so 0..10, tránh lỗi với None.
    return isinstance(value, (int, float)) and 0 <= value <= 10

# Hàm tái dùng valid_score rồi xét >=5 để tính đạt.
def passed(value):
    # and bảo đảm điểm hợp lệ và từ 5 trở lên mới đạt.
    return valid_score(value) and value >= 5
valid = []
passed_scores = []
for score in scores:
    # Chỉ đưa điểm hợp lệ vào danh sách; tránh dùng None hoặc 11/-1.
    if valid_score(score):
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        valid.append(score)
        # Lọc tiếp người đạt trong nhóm đã hợp lệ.
        if passed(score):
            # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
            passed_scores.append(score)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(scores) != 15:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần kiểm tra đủ 15 kết quả điểm.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Lượt nhận:", len(scores))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Hợp lệ:", len(valid))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Đạt:", len(passed_scores))
```

```
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Không hợp lệ:", len(scores) - len(valid))
```

Kết quả khi chạy:

```
Lượt nhận: 15
Hợp lệ: 12
Đạt: 8
Không hợp lệ: 3
```

Điểm 0 là điểm thật nhưng chưa đạt, không bị hiểu là dữ liệu thiếu. Phân biệt hai bước kiểm tra là trọng tâm của cấp 3.

Tình huống 4 — Cấp 4: Hàm trả cả thống kê và các dòng lỗi trong 18 giao dịch

Từ vựng trong mục: Return tuple — trả nhiều giá trị; **Validation** — kiểm tra hợp lệ; **Scope** — phạm vi biến.

Tình huống thực tế: Bộ phận kế toán nhận **18 khoản chi** tính bằng **nghìn đồng**. Trong dữ liệu có hai khoản chưa nhập (`None`) và một khoản -20, không hợp lệ vì chi phí của bài này phải là số không âm. Nhân viên muốn một kết quả báo cáo chung: có bao nhiêu dòng đúng, tổng chi phí hợp lệ, chi phí trung bình và mã các khoản cần sửa, trong khi giữ nguyên bảng gốc.

Vì sao chọn cách này? Hàm trả nhiều giá trị có thể gom thông tin báo cáo thành một tuple, ví dụ `(số khoản hợp lệ, tổng chi, trung bình, danh sách mã lỗi)`. Chương trình lặp qua đủ 18 dòng; `None` là chưa biết chi phí, còn -20 vi phạm quy định không âm, nên đều được ghi vào nhóm cần sửa, không đưa vào tổng. Biến tổng và danh sách lỗi được tạo **bên trong hàm** (biến cục bộ), không làm sửa dữ liệu đầu vào. Trung bình phải được tính từ **số dòng hợp lệ**, nếu lấy cả 18 dòng làm mẫu số thì kết quả sẽ sai.

Quy trình:

1. Xác định dữ liệu hợp lệ: là số và không âm.
2. Duyệt trong hàm, tích lũy các số hợp lệ và ghi mã lỗi.
3. Trả tuple chứa thống kê và danh sách lỗi, in kết quả.

No Note — Không chú thích

```
amounts = [120,200,150,300,None,180,220,0,400,350,-20,280,310,130,260,140,None,230]
def summarize(values):
    accepted = []
    bad_ids = []
    for index, amount in enumerate(values, 1):
        if not isinstance(amount, (int, float)) or amount < 0:
            bad_ids.append(f"G{index:02d}")
        else:
            accepted.append(amount)
    total = sum(accepted)
    if accepted:
        average = total / len(accepted)
    else:
        average = None
    return len(accepted), total, average, bad_ids

count, total, average, errors = summarize(amounts)
if len(amounts) != 18:
    raise ValueError("Cần đủ 18 giao dịch để thống kê.")
print("Giao dịch:", len(amounts))
```

```
print("Hợp lệ:", count)
print("Tổng (nghìn đồng):", total)
print(f"Trung bình (nghìn đồng): {average:.1f}")
print("Cần sửa:", ", ".join(errors))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Viết hàm trả nhiều kết quả thống kê trên 18 giao dịch có lỗi.
# Dữ liệu: 18 khoản tiền nghìn đồng, None và số âm là không hợp lệ.
amounts = [120,200,150,300,None,180,220,0,400,350,-20,280,310,130,260,140,None,230]
# Hàm nhận một list giao dịch và trả số lượng, tổng, trung bình và mã lỗi.
def summarize(values):
    accepted = []
    bad_ids = []
    # enumerate(...,1) tạo mã giao dịch G01..G18.
    for index, amount in enumerate(values, 1):
        # not isinstance() nhận khoản không phải số; or nhận số âm.
        if not isinstance(amount, (int, float)) or amount < 0:
            # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
            # Định dạng f-string :02d cho số thứ tự thành 2 chữ số với số 0 bổ sung.
            bad_ids.append(f"G{index:02d}")
        else:
            # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
            accepted.append(amount)
    # sum() cộng tiền hợp lệ, đơn vị nghìn đồng.
    total = sum(accepted)
    # List rỗng là False; kiểm trước chia để tránh chia 0.
    if accepted:
        # len() đếm số giao dịch hợp lệ; chia tổng để lấy trung bình nghìn đồng.
        average = total / len(accepted)
    else:
        # None báo không thể tính trung bình khi không có số hợp lệ.
        average = None
    # return trả một tuple 4 thành phần để tách vào count,total,average,errors.
    return len(accepted), total, average, bad_ids

count, total, average, errors = summarize(amounts)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(amounts) != 18:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 18 giao dịch để thống kê.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Giao dịch:", len(amounts))
print("Hợp lệ:", count)
print("Tổng (nghìn đồng):", total)
# f-string :.1f in trung bình một chữ số thập phân (chỉ chạy khi không phải None).
print(f"Trung bình (nghìn đồng): {average:.1f}")
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Cần sửa:", ", ".join(errors))
```

Kết quả khi chạy:

```
Giao dịch: 18
Hợp lệ: 15
Tổng (nghìn đồng): 3270
Trung bình (nghìn đồng): 218.0
Cần sửa: G05, G11, G17
```

Trường có giá trị 0 vẫn là một chi phí hợp lệ và được tính vào mẫu số. `None` và âm có nguyên nhân khác nhau nhưng đều không được đưa vào tổng của bài toán này.

Tình huống 5 — Cấp 5: Xây pipeline các hàm xử lý 20 hóa đơn theo vùng

Từ vựng trong mục: Pipeline — chuỗi xử lý; **Reusable function** — hàm tái sử dụng; **Lambda** — hàm ngắn.

Tình huống thực tế: Một doanh nghiệp nhận **20 dòng hóa đơn** thuộc ba vùng **A, B, C**. Mỗi dòng ghi mã hóa đơn, vùng, giá một món tính bằng **nghìn đồng/món** và số món mua. Hóa đơn O07 chưa có giá; O15 có giá -1 nên không được tính. Kế toán muốn doanh thu của từng vùng, danh sách mã cần sửa và xác nhận cả 20 dòng đã được xem xét, kể cả những dòng không thể cộng tiền.

Vì sao chọn cách này? Pipeline nghĩa là một chuỗi bước xử lý theo thứ tự, giống kế toán nhận phiếu → kiểm tra → tính tiền → cộng báo cáo. Hàm kiểm tra quyết định một dòng có đủ giá, số lượng và thông tin vùng hay chưa. Chỉ khi hợp lệ mới nhân giá với số món để tính tiền, rồi cộng vào vùng tương ứng. Tách các bước thành hàm nhỏ giúp tìm lỗi và kiểm thử từng phần, thay vì một hàm dài vừa kiểm tra vừa tính. O07 thiếu giá và O15 có giá âm phải được ghi là lỗi, không biến thành hóa đơn 0 đồng; dù không cộng doanh thu, cả hai vẫn thuộc **20 dòng đã xét**. `lambda` trong phần sắp tên vùng chỉ hỗ trợ thứ tự in, không được làm đổi tổng doanh thu.

Quy trình:

1. Định nghĩa `valid_order` kiểm tra dữ liệu có giá/số lượng hợp lệ.
2. Tạo hàm `amount` tính doanh thu dòng và hàm `report` gom theo vùng.
3. Xử lý 20 hóa đơn, in số hợp lệ, mã lỗi và doanh thu đúng đơn vị.

No Note — Không chú thích

```
orders = [{"id":"O01","area":"A","price":50,"qty":1},{"id":"O02","area":"B","price":60,"qty":2},
{"id":"O03","area":"C","price":70,"qty":3},{"id":"O04","area":"A","price":80,"qty":4},
{"id":"O05","area":"B","price":90,"qty":1},{"id":"O06","area":"C","price":50,"qty":2},
{"id":"O07","area":"A","price":None,"qty":3},{"id":"O08","area":"B","price":70,"qty":4},
{"id":"O09","area":"C","price":80,"qty":1},{"id":"O10","area":"A","price":90,"qty":2},
{"id":"O11","area":"B","price":50,"qty":3},{"id":"O12","area":"C","price":60,"qty":4},
{"id":"O13","area":"A","price":70,"qty":1},{"id":"O14","area":"B","price":80,"qty":2},
{"id":"O15","area":"C","price":-1,"qty":3},{"id":"O16","area":"A","price":50,"qty":4},
{"id":"O17","area":"B","price":60,"qty":1},{"id":"O18","area":"C","price":70,"qty":2},
{"id":"O19","area":"A","price":80,"qty":3},{"id":"O20","area":"B","price":90,"qty":4}]
def valid_order(order):
    return isinstance(order["price"], (int, float)) and order["price"] >= 0 and order["qty"] > 0

def amount(order):
    return order["price"] * order["qty"]

def report(items):
    totals = {"A": 0, "B": 0, "C": 0}
    invalid = []
    for order in items:
        if valid_order(order):
            totals[order["area"]] += amount(order)
        else:
            invalid.append(order["id"])
    return totals, invalid

totals, invalid = report(orders)
if len(orders) != 20:
    raise ValueError("Cần đủ 20 hóa đơn.")
if len(orders) - len(invalid) != 18:
    raise ValueError("Số hóa đơn hợp lệ phải bằng 18.")
print("Số hóa đơn:", len(orders))
print("Hợp lệ:", len(orders) - len(invalid))
```

```
print("Cần sửa:", ", ".join(invalid))
for area in sorted(totals, key=lambda name: name):
    print(f"Vùng {area} (nghìn đồng): {totals[area]}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách thành các hàm xác minh đơn, tính tiền và tổng hợp 20 hóa đơn theo vùng.
# Dữ liệu: 20 dict đơn: id, area A/B/C, price (nghìn đồng), qty (món).
orders = [{"id":"O01","area":"A","price":50,"qty":1},{"id":"O02","area":"B","price":60,"qty":2},
{"id":"O03","area":"C","price":70,"qty":3},{"id":"O04","area":"A","price":80,"qty":4},
{"id":"O05","area":"B","price":90,"qty":1},{"id":"O06","area":"C","price":50,"qty":2},
{"id":"O07","area":"A","price":None,"qty":3},{"id":"O08","area":"B","price":70,"qty":4},
{"id":"O09","area":"C","price":80,"qty":1},{"id":"O10","area":"A","price":90,"qty":2},
{"id":"O11","area":"B","price":50,"qty":3},{"id":"O12","area":"C","price":60,"qty":4},
{"id":"O13","area":"A","price":70,"qty":1},{"id":"O14","area":"B","price":80,"qty":2},
{"id":"O15","area":"C","price":-1,"qty":3},{"id":"O16","area":"A","price":50,"qty":4},
{"id":"O17","area":"B","price":60,"qty":1},{"id":"O18","area":"C","price":70,"qty":2},
{"id":"O19","area":"A","price":80,"qty":3},{"id":"O20","area":"B","price":90,"qty":4}]
# Hàm kiểm tra giá là số´không âm và số´lượng món dương.
def valid_order(order):
    # isinstance() loại giá None; and chỉ xét so sánh số´tiền và số´lượng khi kiểu đúng.
    return isinstance(order["price"], (int, float)) and order["price"] >= 0 and order["qty"] > 0

# Hàm trả tiền đơn = giá (nghìn đồng) nhân số´lượng.
def amount(order):
    # Nhân đơn giá nghìn đồng với số´lượng để ra tiền hóa đơn cùng đơn vị.
    return order["price"] * order["qty"]

# Hàm gom tổng tiền từng vùng và danh sách mã lỗi.
def report(items):
    # Dictionary dùng mã vùng A/B/C làm khóa; số´tiền ban đầu là 0.
    totals = {"A": 0, "B": 0, "C": 0}
    invalid = []
    for order in items:
        # Chỉ cộng hóa đơn qua được kiểm tra giá/số´lượng.
        if valid_order(order):
            # += cộng tiền đơn vào đúng mã vùng.
            totals[order["area"]] += amount(order)
        else:
            # append() lưu mã hóa đơn sai để báo sửa.
            invalid.append(order["id"])
    # Trả hai kết quả cùng lúc bằng tuple để bên ngoài tách.
    return totals, invalid

totals, invalid = report(orders)
# len() trả số´phần tư, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(orders) != 20:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 20 hóa đơn.")
# len() trả số´phần tư, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(orders) - len(invalid) != 18:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Số´hóa đơn hợp lệ phải bằng 18.")
# len() trả số´phần tư, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Số´hóa đơn:", len(orders))
# len() trả số´phần tư, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Hợp lệ:", len(orders) - len(invalid))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Cần sửa:", ", ".join(invalid))
# sorted() tạo danh sách mã vùng theo thứ tự chữ cái; lambda name:name dùng chính chuỗi làm khóa.
for area in sorted(totals, key=lambda name: name):
    print(f"Vùng {area} (nghìn đồng): {totals[area]}")
```

Kết quả khi chạy:

```
Sô´hóa đơn: 20
Hợp lệ: 18
Cần sửả: 007, 015
Vùng A (nghìn đồng): 1060
Vùng B (nghìn đồng): 1220
Vùng C (nghìn đồng): 770
```

Đây là ví dụ tích hợp: mỗi hàm làm một việc, phép biến đổi dễ kiểm thử. Giá âm và thiếu được tách riêng trước khi tổng doanh thu theo vùng.

6. Sai lầm thường gặp

- `return` trả giá trị về chỗ gọi hàm; nếu muốn thấy giá trị trên màn hình thì gọi `print()`.
- Tham số mặc định như `rate=0.10` được dùng khi không truyền giá trị khác vào.
- `lambda` phù hợp biểu thức ngắn, còn xử lý nhiều bước nên dùng `def` cho dễ đọc.
- Hàm tính trung bình cần xử lý danh sách rỗng trước khi chia cho `len(values)`.
- Khi sắp xếp bản ghi, phải chọn đúng trường dữ liệu trong `key`.
- Không dùng trực tiếp biến cục bộ của hàm từ bên ngoài; đưa dữ liệu cần dùng ra ngoài bằng `return`.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a Python function? | Hàm Python là gì?

A. Only a variable declared at module level | Chỉ biến khai báo ở cấp module

B. Only code that prints text | Chỉ mã in văn bản

C. Only a loop executed once | Chỉ vòng lặp chạy một lần

D. Reusable callable code that can accept inputs | Phần mã có thể gọi lại, có thể nhận đầu vào

Đáp án: D

Giải thích: Hàm Python là phần mã có thể gọi lại nhiều lần, có thể nhận đầu vào và trả kết quả. Hàm thường có thể định nghĩa bằng `def`, còn `lambda` cũng tạo hàm từ một biểu thức; không phải mọi hàm đều bắt buộc có tên.

Q2. What is a parameter? | Tham số là gì?

A. A value supplied by the caller | Giá trị truyền từ nơi gọi

B. A name declared in a function's definition | Tên khai báo trong định nghĩa hàm

C. The value printed by a function | Giá trị được hàm in ra

D. A variable always declared global | Biến luôn được khai báo toàn cục

Đáp án: B

Giải thích: Tham số là tên khai báo trong định nghĩa hàm để nhận đầu vào. Trong `def double(x):`, `x` là tham số, còn số `3` trong `double(3)` là đối số được truyền khi gọi.

Q3. What is an argument? | Đối số là gì?

A. The name in the function definition | Tên trong định nghĩa hàm

B. A keyword beginning a function body | Từ khóa bắt đầu thân hàm

C. A value supplied in a function call | Giá trị truyền khi gọi hàm

D. The result returned by a function | Kết quả hàm trả về

Đáp án: C

Giải thích: Đối số là giá trị thực tế được truyền vào khi gọi hàm. Với `double(3)`, số nguyên `3` là đối số gắn vào tham số `x`; tên tham số xuất hiện trong phần định nghĩa.

Q4. What is a return value? | Giá trị trả về là gì?

A. A result sent back to the caller | Kết quả gửi về nơi gọi

B. A parameter name | Tên tham số

C. The function's source-code text | Văn bản mã nguồn hàm

D. A displayed message from `print` | Thông báo hiển thị bởi `print`

Đáp án: A

Giải thích: Giá trị trả về là thông tin hàm gửi về nơi gọi qua câu lệnh `return`. Khác `print` dùng để hiển thị, `return` giúp mã gọi hàm tiếp tục sử dụng kết quả tính toán.

Q5. Which keyword defines a normal function? | Từ khóa nào định nghĩa hàm thông thường?

A. `return` | `return`

B. `lambda` | `lambda`

C. `if` | `if`

D. `def` | `def`

Đáp án: D

Giải thích: `def` mở đầu định nghĩa hàm có tên, tiếp theo là tham số, dấu hai chấm và thân hàm thụt lề. `lambda` tạo biểu thức hàm, còn `return` dùng bên trong thân để trả kết quả.

Q6. Which keyword sends a value back to the caller? | Từ khóa nào trả giá trị về nơi gọi?

A. `lambda` | `lambda`

B. `if` | `if`

C. `def` | `def`

D. `return` | `return`

Đáp án: D

Giải thích: `return` kết thúc lần gọi hàm hiện tại và chuyển giá trị đã chỉ ra về nơi gọi. Với `return x * 2`, nơi gọi nhận giá trị gấp đôi; `print(x * 2)` chỉ hiển thị rồi trả `None`.

Q7. Which keyword defines a short anonymous function expression? | Từ khóa nào tạo biểu thức hàm ngắn không tên?

A. `def` | `def`

B. `return` | `return`

C. `lambda` | `lambda`

D. `if` | `if`

Đáp án: C

Giải thích: `lambda` tạo hàm từ một biểu thức, chẳng hạn `lambda x: x * 2`. Hàm có thể gán cho tên hoặc truyền làm đối số, khác `def` mở định nghĩa hàm theo dạng câu lệnh.

Q8. Which keyword supplies a conditional branch inside a function? | Từ khóa nào tạo nhánh điều kiện bên trong hàm?

A. `lambda` | `lambda`

B. `if` | `if`

C. `def` | `def`

D. `return` | `return`

Đáp án: B

Giải thích: `if` mở nhánh điều kiện bên trong hàm, giúp hàm chọn đường thực thi theo đầu vào. Bản thân `if` không tạo hàm và cũng không trả giá trị về nơi gọi.

Q9. What does a default parameter provide? | Tham số mặc định có tác dụng gì?

A. A value calculated only after the function returns | Giá trị chỉ tính sau khi hàm trả về

B. A value that must be supplied for every call | Giá trị bắt buộc phải truyền ở mọi lần gọi

C. A fallback value used when the caller omits an argument | Giá trị dự phòng khi nơi gọi không truyền đối số

D. A value that overrides every explicit argument | Giá trị ghi đè mọi đối số truyền vào

Đáp án: C

Giải thích: Giá trị mặc định của tham số được dùng khi nơi gọi bỏ qua đối số đó; với `def greet(name='An')`, `greet()` dùng `'An'`. Nếu gọi `greet('Binh')` thì giá trị truyền vào thay mặc định, vì vậy đây là dự phòng chứ không phải giá trị bắt buộc.

Q10. What is a keyword argument? | Đối số theo tên là gì?

A. An argument supplied using its parameter's name | Đối số truyền bằng tên tham số

B. A local variable defined outside the function | Biến cục bộ định nghĩa ngoài hàm

C. A function name passed without calling | Tên hàm được truyền mà không gọi

D. An argument always matched only by position | Đối số luôn ghép bằng vị trí

Đáp án: A

Giải thích: Đối số theo tên nêu rõ tên tham số lúc gọi, ví dụ `greet(name='An')`. Nó khác đối số theo vị trí như `greet('An')`, vốn ghép với tham số dựa trên vị trí truyền vào.

Q11. What does `sorted(rows, key=func)` use `func` for? | Trong `sorted(rows, key=func)`, func dùng để làm gì?

A. Filtering out all false-valued items before sorting | Loại mọi phần tử có giá trị sai trước khi sắp

B. Computing a sort key for each item | Tính khóa sắp xếp cho từng phần tử

C. Replacing every row with `None` | Thay mỗi hàng bằng `None`

D. Printing each row during sorting | In từng hàng trong lúc sắp

Đáp án: B

Giải thích: Trong `sorted(rows, key=func)`, `func` tính giá trị khóa so sánh cho từng phần tử; phép sắp xếp dùng các khóa đó. Ví dụ `key=len` sẽ ưu tiên chuỗi ngắn trước theo độ dài thay vì thứ tự chữ cái.

Q12. What does `filter(test, items)` keep? | `filter(test, items)` giữ gì?

A. Only items whose test result is false | Chỉ phần tử có kết quả kiểm tra sai

B. Items in descending order | Phần tử theo thứ tự giảm dần

C. Items for which `test(item)` is truthy | Phần tử mà `test(item)` có giá trị đúng

D. All items, regardless of the test | Mọi phần tử, bất kể phép kiểm tra

Đáp án: C

Giải thích: `filter(test, items)` chọn phần tử mà `test(item)` có giá trị đúng, trả iterator trong Python 3. Với `filter(lambda x: x > 0, [-1,2])`, chuyển sang `list` được `[2]`, không phải mọi phần tử ban đầu.

Q13. What is a local variable? | Biến cục bộ là gì?

A. Any name defined at module level | Bất kỳ tên nào khai báo cấp module

B. A name automatically shared by every function | Tên tự chia sẻ cho mọi hàm

C. A name that can never change value | Tên không bao giờ đổi giá trị

D. A name bound within a function's local scope | Tên được gán trong phạm vi cục bộ hàm

Đáp án: D

Giải thích: Biến cục bộ là tên được gán trong phạm vi cục bộ của hàm, như `total` gán trong `def add():`. Nó không tự tồn tại ngoài hàm, khác tên toàn cục khai báo ở cấp module.

Q14. What is a global variable? | Biến toàn cục là gì?

A. A name bound at module level | Tên được gán ở cấp module

B. A name that exists only inside a class | Tên chỉ tồn tại trong class

C. A name inaccessible to all functions | Tên mọi hàm đều không truy cập được

D. A name created only by `lambda` | Tên chỉ do `lambda` tạo

Đáp án: A

Giải thích: Biến toàn cục là tên được gán ở cấp module và thường có thể đọc trong hàm nếu không bị che khuất. Muốn gán lại chính biến toàn cục từ trong hàm thường phải khai báo `global`, khác việc chỉ đọc giá trị.

Q15. What is a lambda expression limited to? | Nội dung của lambda bị giới hạn như thế nào?

A. A complete `try/except` statement block | Khối câu lệnh `try/except` đầy đủ

B. A single expression | Một biểu thức

C. Only a numeric literal | Chỉ một literal số

D. A series of ordinary statements | Chuỗi các câu lệnh thông thường

Đáp án: B

Giải thích: Thân `lambda` trong Python chỉ chứa một biểu thức và giá trị biểu thức trở thành kết quả hàm. Nó không chứa khối nhiều câu lệnh thông thường như chuỗi lệnh `for` và gán; cần dùng `def` trong trường hợp đó.

Q16. What is a closure? | Closure là gì?

A. A function forced to return `None` | Hàm bị buộc trả `None`

B. A function that prints its enclosing scope | Hàm in phạm vi bao ngoài

C. A function that discards all free variables | Hàm loại bỏ mọi biến tự do

D. A function whose enclosing names remain accessible | Hàm vẫn truy cập được tên của phạm vi bao ngoài

Đáp án: D

Giải thích: Closure là hàm vẫn truy cập được các tên thuộc phạm vi bao ngoài, kể cả sau khi hàm ngoài đã trả về. Nó giữ liên kết tham chiếu tới các tên đó chứ không nhất thiết chụp cố định giá trị tại một thời điểm.

Q17. When a function reaches its end without executing `return`, what value does it return? | Khi hàm chạy đến cuối mà không thực hiện `return`, nó trả giá trị nào?

A. `None` | `None`

B. An empty string `''` | Chuỗi rỗng `''`

C. The last local variable | Biến cục bộ cuối

D. `NameError` | `NameError`

Đáp án: A

Giải thích: Nếu hàm chạy đến cuối mà không có `return` tường minh, Python ngầm trả `None`. Hàm vẫn có thể in thông báo nhưng việc in không biến thông báo đó thành giá trị trả về.

Q18. What does `print()` do instead of returning a useful result? | `print()` làm gì thay vì trả một kết quả để tính tiếp?

A. Displays output while returning `None` | Hiển thị đầu ra và trả `None`

B. Returns a tuple with the printed text | Trả Tuple chứa văn bản đã in

C. Raises `TypeError` whenever called | Mỗi khi gọi đều gây `TypeError`

D. Always returns the printed string | Luôn trả chuỗi đã in

Đáp án: A

Giải thích: `print()` đưa văn bản ra đầu ra chuẩn và giá trị trả về là `None`. Ví dụ `result = print('Hi')` hiển thị `Hi` nhưng lưu `None` vào `result`; dùng `return` trong hàm nếu mã phía sau cần kết quả.

Q19. What does a function's `return a, b` actually return? | `return a, b` thực chất trả gì?

A. The result of adding `a` and `b` | Kết quả cộng `a` với `b`

B. A tuple containing both values | Tuple chứa cả hai giá trị

C. A list containing both values | List chứa cả hai giá trị

D. The two values as separate function return objects | Hai giá trị thành hai đối tượng trả về riêng

Đáp án: B

Giải thích: Biểu thức trả về có các giá trị cách nhau bằng dấu phẩy đóng gói chúng thành Tuple, nên `return a, b` trả `(a, b)`. Nơi gọi có thể giải nén ra hai tên, nhưng hàm thực tế trả một đối tượng Tuple.

Q20. What happens when code outside a function reads an undefined local name? | Đọc biến cục bộ không tồn tại ở ngoài hàm gây gì?

A. The most recent printed value is used | Dùng giá trị vừa được in

B. The local name is exported automatically | Tên cục bộ tự xuất ra ngoài

C. `NameError` for the undefined name | `NameError` vì tên chưa được định nghĩa

D. `None` is silently substituted | Tự thay bằng `None`

Đáp án: C

Giải thích: Tên cục bộ chỉ được gán bên trong hàm không tự tồn tại ở phạm vi module. Đọc tên chưa được định nghĩa ở ngoài gây `NameError`; cần trả về tường minh hoặc lưu vào liên kết có thể truy cập.

8. Nối sang bài tiếp theo

`Vòng lặp → Hàm → OOP → Pipeline.`

PY-10 — Class, Object, constructor và đóng gói

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Class — lớp; **Object** — đối tượng.

`Hàm / Dictionary → OOP → Kế thừa`

Học xong dùng để làm gì? Thiết kế đối tượng học viên hoặc tài khoản với thuộc tính, phương thức và kiểm tra đầu vào.

Kết nối kiến thức tiếp theo: PY-11 mở rộng các loại đối tượng bằng kế thừa và ghi đè phương thức.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Class	lớp	Khuôn mô tả dữ liệu và cách xử lý
Object	đối tượng	Một thể hiện cụ thể của lớp
Constructor	hàm khởi tạo	Thiết lập thuộc tính lúc tạo instance
Encapsulation	đóng gói	Đặt dữ liệu và hành vi bên trong đối tượng

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`class Name:`	Định nghĩa kiểu đối tượng và các phương thức
`__init__(self, ...)`	Khởi tạo trạng thái của từng instance
`self.attribute`	Truy cập thuộc tính thuộc về chính đối tượng
`__str__/set_*/get_*`	Hiển thị và kiểm tra thao tác thay đổi giá trị

4. Khái niệm liên quan

Lập trình hướng đối tượng (OOP) giúp gom **dữ liệu của một đối tượng** và **những thao tác dùng dữ liệu ấy** vào cùng một cấu trúc. Ví dụ một học viên có tên, điểm và cách hiển thị hồ sơ; thay vì đặt các biến rời rạc, ta tạo một lớp `Student`. Từ lớp này, bài học lần lượt giải thích class, object, `__init__`, `self` và cách đóng gói thao tác thay đổi dữ liệu.

4.1. Class — Khuôn mẫu đối tượng

Từ vựng trong mục: Class — lớp; **Method** — phương thức.

Class giống bản thiết kế: định nghĩa những việc các đối tượng cùng loại có thể thực hiện. Ví dụ lớp `Student` có phương thức `greet()` trả một lời chào. Dòng `class Student:` tạo bản thiết kế, `def greet(self)` định nghĩa hành động; `self` đại diện cho đối tượng cụ thể đang gọi phương thức. Chỉ viết class chưa tạo hồ sơ học viên nào; `Student()` ở cuối mới tạo một đối tượng để gọi `greet()`.

No Note — Không chú thích

```
class Student:
    def greet(self):
        return 'Xin chào'

student = Student()
print(student.greet())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Định nghĩa class Student như khuôn tạo đối tượng và gọi phương thức greet.
# Dữ liệu: class Student mô tả kiểu đối tượng; phương thức có self truy cập đối tượng hiện tại.
# class khai báo khuôn mẫu Student; Student() tạo một đối tượng mới.
class Student:
    # self tham chiếu đối tượng hiện tại; greet là phương thức của lớp.
    def greet(self):
        # return trả giá trị cho nơi gọi và kết thúc hàm.
        return 'Xin chào'

student = Student()
# Dấu chấm gọi phương thức greet() của chính instance student.
print(student.greet())
```

Kết quả khi chạy:

```
Xin chào
```

Đối tượng `student` gọi phương thức được định nghĩa bên trong lớp `Student`.

4.2. Object — Mỗi học viên là một đối tượng

Từ vựng trong mục: Object — đối tượng; **Attribute** — thuộc tính.

Object là một thể hiện được tạo từ class. Cùng một mẫu `Student`, ta có thể tạo hai học viên An và Lan, mỗi người có tên riêng. Thuộc tính `self.name` lưu tên **trên từng đối tượng**, vì vậy thay tên của An không tự thay tên của Lan. Ta dùng dấu chấm `student.name` để lấy một thuộc tính.

No Note — Không chú thích

```
class Student:
    def __init__(self, name):
        self.name = name

a = Student('An')
b = Student('Lan')
print(a.name, b.name)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo hai đối tượng Student với thuộc tính name độc lập.
# Dữ liệu: a và b là hai đối tượng riêng biệt, mỗi đối tượng giữ thuộc tính name riêng.
# class định nghĩa mẫu có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
class Student:
    # __init__ chạy khi Student(name) được tạo; self là instance và name là đối số nhập.
    def __init__(self, name):
        # Thuộc tính self.name thuộc từng instance, nên hai học viên lưu tên độc lập.
        self.name = name

a = Student('An')
b = Student('Lan')
# Dấu chấm truy cập thuộc tính name của từng đối tượng.
print(a.name, b.name)
```

Kết quả khi chạy:

```
An Lan
```

Cả `a` và `b` đều thuộc lớp Student, nhưng chứa dữ liệu tên riêng.

4.3. Constructor và self — Khởi tạo trạng thái

Từ vựng trong mục: Constructor — bước khởi tạo; **init** — phương thức khởi tạo; **self** — chính đối tượng.

Khi tạo `Student('An', 8)`, Python tạo đối tượng mới rồi tự gọi `__init__` để thiết lập tên và điểm ban đầu. Tham số `self` là đối tượng vừa được tạo; `name` và `score` nhận đối số truyền khi gọi. Lệnh `self.score = score` lưu điểm bên trong học viên, không chỉ trong biến tạm của hàm. Điểm ở đây tính trên thang 10.

No Note — Không chú thích

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

student = Student('An', 8)
print(student.name, student.score)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Khởi tạo đối tượng Student có name và score qua __init__.
# Dữ liệu: Hàm khởi tạo nhận tên và điểm để lưu trên đúng instance.
# class định nghĩa mẫu có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
class Student:
    # __init__ nhận hai giá trị và khởi tạo trạng thái của đối tượng.
    def __init__(self, name, score):
        # Gắn tên lên thuộc tính instance self.name.
        self.name = name
        # Gắn điểm lên thuộc tính instance self.score.
        self.score = score

student = Student('An', 8)
# Đọc hai thuộc tính từ instance student.
print(student.name, student.score)
```

Kết quả khi chạy:

```
An 8
```

Phương thức `__init__` đưa dữ liệu đầu vào thành hai thuộc tính của cùng hồ sơ học viên.

4.4. Encapsulation — Đóng gói dữ liệu và quy tắc xử lý

Từ vựng trong mục: Encapsulation — đóng gói; **Method** — phương thức; **Validation** — kiểm tra hợp lệ.

Đóng gói nghĩa là quản lý dữ liệu cùng các thao tác tác động lên nó trong lớp. Ví dụ điểm chỉ được phép trong khoảng 0–10: lớp `Score` lưu số hiện tại tại `self._value` và có phương thức `set_value()` kiểm tra trước khi cập nhật. Dấu `_` là **quy ước** nhắc người lập trình không nên sửa trực tiếp, không phải cơ chế cấm truy cập tuyệt đối. Đọc điểm qua `get_value()` giúp giữ cùng một cách sử dụng.

No Note — Không chú thích

```
class Score:
    def __init__(self):
        self._value = 0

    def set_value(self, x):
        if not 0 <= x <= 10:
            raise ValueError('Điểm ngoài khoảng 0–10')
        self._value = x

    def get_value(self):
        return self._value

score = Score()
score.set_value(9)
print(score.get_value())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đóng gói điểm trong Score, chỉ đổi qua setter có kiểm tra.
# Dữ liệu: Giá trị _value quy ước nội bộ, không phải thuộc tính private tuyệt đối.
# class định nghĩa mẫu có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
class Score:
    # __init__ tự chạy khi tạo object; self là instance hiện tại.
    def __init__(self):
        # Dấu _ quy ước thuộc tính nội bộ; khởi tạo điểm bằng 0.
        self._value = 0

    # Setter nhận điểm mới x để kiểm tra trước khi cập nhật trạng thái.
    def set_value(self, x):
        # not phủ định điều kiện 0..10; ngoài thang sẽ báo ValueError.
        if not 0 <= x <= 10:
            # raise ValueError chặn điểm sai thang và dừng setter.
            raise ValueError('Điểm ngoài khoảng 0–10')
        # Chỉ gán giá trị mới khi đã vượt qua kiểm tra.
        self._value = x

    # Getter trả điểm nội bộ, không cho bên gọi tự sửa trong phương thức.
    def get_value(self):
        # return trả giá trị cho nơi gọi và kết thúc hàm.
        return self._value

score = Score()
# Gọi setter trên đối tượng score với điểm 9.
score.set_value(9)
```

```
# Gọi getter để lấy số đã được lưu.
print(score.get_value())
```

Kết quả khi chạy:

```
9
```

Điểm 9 hợp lệ nên được lưu và đọc lại. Với điểm 11, `set_value` sẽ báo `ValueError` thay vì ghi dữ liệu sai.

5. Năm tình huống OOP — từ đối tượng đến quản lý trạng thái

Tình huống 1 — Cấp 1: Tạo 10 đối tượng học viên có trạng thái riêng

Từ vựng trong mục: Class — lớp; **Object** — đối tượng; **Constructor** — hàm khởi tạo.

Tình huống thực tế: Một lớp có **10 học viên S01 đến S10**, mỗi người có một điểm kiểm tra theo thang **10 điểm**. Giáo viên cần lưu mã và điểm gắn với đúng từng người, rồi xem điểm của S01 và S10. Họ không muốn phải tạo 10 cặp biến rời rạc dễ nhập nhầm giữa học viên này với học viên khác.

Vì sao chọn cách này? OOP (lập trình hướng đối tượng) là cách tổ chức thông tin cùng hành động của một sự vật vào một chỗ. **Class** `Student` giống một mẫu để tạo học viên, còn **object** là học viên cụ thể được tạo từ mẫu ấy. Trong `__init__`, `self.id` và `self.score` giữ mã và điểm thuộc **đúng đối tượng đang tạo**. Vì thế sửa điểm một học viên không tự làm đổi điểm học viên khác. Chương trình tạo 10 đối tượng từ các cặp mã–điểm, rồi đọc thuộc tính của S01 và S10; bài này chưa cần những kỹ thuật OOP phức tạp hơn.

Quy trình:

1. Định nghĩa lớp có hàm `__init__` lưu mã và điểm.
2. Tạo 10 đối tượng từ bảng điểm, mỗi hàng là một người.
3. In số đối tượng và điểm đầu/cuối.

No Note — Không chú thích

```
scores = [8,7,9,6,8.5,10,4,7.5,6.5,9.5]
class Student:
    def __init__(self, code, score):
        self.code = code
        self.score = score

students = []
for i, score in enumerate(scores, 1):
    students.append(Student(f"S{i:02d}", score))
if len(students) != 10:
    raise ValueError("Cần tạo đủ 10 đối tượng học viên.")
print("Học viên:", len(students))
print("S01:", students[0].score)
print("S10:", students[-1].score)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo 10 đối tượng Student từ dữ liệu điểm bằng vòng lặp.
# Dữ liệu: 10 điểm từng học viên; class Student chứa code và score trên từng đối tượng.
scores = [8,7,9,6,8.5,10,4,7.5,6.5,9.5]
# class định nghĩa mẫu có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
class Student:
```

```
    # Mỗi object Student nhận mã riêng và điểm riêng.
    def __init__(self, code, score):
        # self.code lưu mã trên instance.
        self.code = code
        # self.score lưu điểm trên instance.
        self.score = score

students = []
# enumerate(...,1) tạo thứ tự học viên từ 1.
for i, score in enumerate(scores, 1):
    # f-string :02d tạo mã S01..S10, sau đó tạo instance Student.
    students.append(Student(f"S{i:02d}", score))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(students) != 10:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần tạo đủ 10 đối tượng học viên.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Học viên:", len(students))
# [0] truy cập đối tượng đầu để đọc thuộc tính score.
print("S01:", students[0].score)
# [-1] truy cập đối tượng cuối.
print("S10:", students[-1].score)
```

Kết quả khi chạy:

```
Học viên: 10
S01: 8
S10: 9.5
```

Thuộc tính `self.score` thuộc về từng đối tượng, không phải biến dùng chung cho toàn bộ lớp.

Tình huống 2 — Cấp 2: Giỏ hàng có phương thức add và total cho 12 món

Từ vựng trong mục: Method — phương thức; **Self** — đối tượng hiện tại; **Encapsulation** — đóng gói.

Tình huống thực tế: Một khách hàng mua **12 mặt hàng**; danh sách có giá một món của từng loại, đơn vị **nghìn đồng** và mỗi loại mua một món. Thu ngân muốn lưu các món đã chọn trong một **giỏ hàng riêng** và nhờ chính giỏ ấy tính tổng số tiền, để lần sau có thể làm thêm giỏ khác mà không bị trộn giá của hai khách.

Vì sao chọn cách này? Phương thức là một hàm thuộc về đối tượng. Với `Cart`, `add()` thêm giá một món vào danh sách của giỏ, còn `total()` cộng các giá đã nằm trong chính giỏ đó. `__init__` tạo danh sách rỗng **mỗi lần tạo giỏ mới**, thay vì để mọi giỏ dùng chung một danh sách. Nếu dùng dữ liệu chung, thêm món của khách B có thể làm tổng tiền khách A tăng sai. Giữ cách tính tổng trong `Cart` giúp người dùng giỏ chỉ cần gọi `cart.total()` mà không viết phép cộng bên ngoài.

Quy trình:

1. Tạo lớp `Cart` với thuộc tính `prices` khởi tạo riêng.
2. Lần lượt gọi `add()` với 12 đơn giá.
3. Gọi `total()` và kiểm tra một giỏ mới không chứa hàng cũ.

No Note — Không chú thích

```
prices = [20,30,10,15,40,25,35,20,30,10,50,45]
class Cart:
    def __init__(self):
        self.prices = []
```

```
    def add(self, price):
        self.prices.append(price)
    def total(self):
        return sum(self.prices)

cart = Cart()
for price in prices:
    cart.add(price)
if len(cart.prices) != 12:
    raise ValueError("Giỏ hàng phải chứa đủ 12 món.")
if Cart().total() != 0:
    raise ValueError("Giỏ hàng mới phải rỗng, không chia sẻ danh sách cũ.")
print("Mặt hàng:", len(cart.prices))
print("Tổng (nghìn đồng):", cart.total())
print("Giỏ mới (nghìn đồng):", Cart().total())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo mỗĩ giỏ Cart có list giá riêng và phương thức cộng tổng.
# Dữ liệu: 12 giá mặt hàng tính nghìn đồng, mỗĩ Cart tự có list prices.
prices = [20,30,10,15,40,25,35,20,30,10,50,45]
# class định nghĩa mâũ có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
class Cart:
    # Mỗĩ lần Cart() gọi __init__, tạo list prices mới riêng cho giỏ.
    def __init__(self):
        # List lưu đơn giá theo nghìn đồng của chính giỏ này.
        self.prices = []
    # Phương thức thêm một giá vào giỏ.
    def add(self, price):
        # append() thêm giá vào cuối list của instance.
        self.prices.append(price)
    # Phương thức cộng toàn bộ giá đang có.
    def total(self):
        # sum() cộng giá mỗĩ món trong giỏ, vẫn tính bằng nghìn đồng.
        return sum(self.prices)

cart = Cart()
for price in prices:
    # Gọi method add của giỏ hiện tại cho từng món.
    cart.add(price)
# len() trả sô´phần tư,̉ dùng đẻđếm hoặc kiểm tra đủ bản ghi.
if len(cart.prices) != 12:
    # raise báo lôĩ rồi dừng luồng hiện tại nếú dữ liệu không đáp ứng yêu câù.
    raise ValueError("Giỏ hàng phải chứa đủ 12 món.")
# Cart() tạo giỏ mới, rồi total() phải bằng 0; kiểm không chia sẻ list giữa các object.
if Cart().total() != 0:
    # raise báo lôĩ rồi dừng luồng hiện tại nếú dữ liệu không đáp ứng yêu câù.
    raise ValueError("Giỏ hàng mới phải rỗng, không chia sẻ danh sách cũ.")
# len() trả sô´phần tư,̉ dùng đẻđếm hoặc kiểm tra đủ bản ghi.
print("Mặt hàng:", len(cart.prices))
print("Tổng (nghìn đồng):", cart.total())
# Cart() tạo giỏ mới, rồi total() phải bằng 0; kiểm không chia sẻ list giữa các object.
print("Giỏ mới (nghìn đồng):", Cart().total())
```

Kết quả khi chạy:

```
Mặt hàng: 12
Tổng (nghìn đồng): 330
Giỏ mới (nghìn đồng): 0
```

Phương thức được gọi trên một đối tượng cụ thể; việc tạo giỏ mới không kéo theo các mặt hàng của giỏ cũ.

Tình huống 3 — Cấp 3: Đóng gói quy tắc lưu điểm cho 15 hồ sơ

Từ vựng trong mục: Setter — phương thức gán; **Validation** — kiểm tra; **Exception** — ngoại lệ.

Tình huống thực tế: Phòng đào tạo tiếp nhận **15 điểm của học viên S01 đến S15**, theo thang **0 đến 10**. Trong danh sách có S06 ghi 11 và S10 ghi -1, đều ngoài thang điểm hợp lệ. Giáo viên muốn lưu điểm riêng cho từng hồ sơ, biết những mã nào không thể ghi điểm và tránh đưa điểm sai vào kết quả học tập.

Vì sao chọn cách này? Đóng gói ở đây có nghĩa là đặt quy tắc kiểm tra ngay nơi dữ liệu được thay đổi. Đối tượng `ScoreRecord` giữ điểm của một học viên; khi `set_score()` được gọi, nó kiểm tra điểm phải là số nằm từ 0 đến 10 rồi mới lưu. `ValueError` là thông báo lỗi khi giá trị nhập không được chấp nhận; chương trình xử lý lỗi theo từng học viên nên điểm sai của S06, S10 không khiến 13 hồ sơ còn lại bị bỏ qua. Không nên gán thẳng điểm sai vào thuộc tính rồi mới kiểm tra sau, vì lúc đó đối tượng đã giữ thông tin không hợp lệ.

Quy trình:

1. Tạo lớp với trạng thái điểm ban đầu `None`.
2. Gọi setter với mỗi trong 15 điểm, bắt lỗi cho đúng mã hồ sơ.
3. Đếm số hồ sơ lưu được và mã cần sửa.

No Note — Không chú thích

```
raw_scores = [4,8,7,10,6,11,9,5,7.5,-1,0,8.5,3,9,10]
class ScoreRecord:
    def __init__(self, code):
        self.code = code
        self._score = None
    def set_score(self, value):
        if not (0 <= value <= 10):
            raise ValueError("score must be within 0..10")
        self._score = value
    def get_score(self):
        return self._score

records, errors = [], []
for index, score in enumerate(raw_scores, 1):
    record = ScoreRecord(f"S{index:02d}")
    try:
        record.set_score(score)
    except ValueError:
        errors.append(record.code)
    records.append(record)
if len(records) != 15:
    raise ValueError("Cần lưu đủ 15 hồ sơ.")
print("Hồ sơ:", len(records))
saved_count = 0
for rec in records:
    if rec.get_score() is not None:
        saved_count += 1
print("Đã lưu điểm:", saved_count)
print("Sai:", ", ".join(errors))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lưu 15 điểm qua object setter, bắt ngoại lệ khi điểm sai thang.
# Dữ liệu: 15 điểm thang 0..10, gồm điểm vượt thang cần báo sai.
raw_scores = [4,8,7,10,6,11,9,5,7.5,-1,0,8.5,3,9,10]
# class định nghĩa mẫu có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
```

```
class ScoreRecord:
    # __init__ tự chạy khi tạo object; self là instance hiện tại.
    def __init__(self, code):
        self.code = code
        # Điểm ban đầu chưa có, dấu _ đánh dấu thuộc tính nội bộ theo quy ước.
        self._score = None
    # Setter kiểm tra thang điểm 0..10 trước khi lưu.
    def set_score(self, value):
        # Ngoài thang 0..10 báo ValueError; cả -1 và 11 đều bị từ chối.
        if not (0 <= value <= 10):
            # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
            raise ValueError("score must be within 0..10")
        # Lưu điểm sau khi xét hợp lệ.
        self._score = value
    # Getter trả lại điểm đã lưu hoặc None nếu chưa lưu.
    def get_score(self):
        # return trả giá trị cho nơi gọi và kết thúc hàm.
        return self._score

records, errors = [], []
# enumerate(...,1) đặt mã S01..S15 theo thứ tự nhận.
for index, score in enumerate(raw_scores, 1):
    # Định dạng f-string :02d cho số thứ tự thành 2 chữ số với số 0 bổ sung.
    record = ScoreRecord(f"S{index:02d}")
    # try bắt đầu khối có thể phát sinh ValueError từ setter.
    try:
        # Gọi setter thay vì gán trực tiếp để sử dụng quy tắc kiểm tra.
        record.set_score(score)
    # except ValueError đón điểm sai và tiếp tục duyệt, không dừng toàn chương trình.
    except ValueError:
        # append() giữ mã hồ sơ sai.
        errors.append(record.code)
    # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
    records.append(record)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(records) != 15:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần lưu đủ 15 hồ sơ.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Hồ sơ:", len(records))
saved_count = 0
for rec in records:
    # is not None xác nhận có giá trị thực, khác giá trị thiếu.
    if rec.get_score() is not None:
        saved_count += 1
print("Đã lưu điểm:", saved_count)
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Sai:", ", ".join(errors))
```

Kết quả khi chạy:

```
Hồ sơ: 15
Đã lưu điểm: 13
Sai: S06, S10
```

Điểm 0 là hợp lệ, khác với điểm chưa nhập (`None`). Cấp này thêm ràng buộc và xử lý lỗi mà không kết thúc toàn bộ lượt nhập.

Tình huống 4 — Cấp 4: Định dạng 18 hóa đơn qua str và xử lý tiền thiếu

Từ vựng trong mục: Dunder method — phương thức đặc biệt; **str** — chuỗi hiển thị; **Missing value** — giá trị thiếu.

Tình huống thực tế: Ứng dụng kế toán có **18 hóa đơn O01 đến O18**, mỗi hóa đơn cần lưu số tiền thanh toán tính bằng **đồng**. Hai hóa đơn O06 và O15 chưa được nhập tiền nên có giá trị `None`. Nhân viên muốn in hóa đơn theo mẫu dễ đọc, đồng thời báo tổng tiền của những hóa đơn đã có số và mã hai hóa đơn còn thiếu; không được hiển thị thiếu tiền như 0 đồng.

Vì sao chọn cách này? `__str__()` là phương thức cho biết **đối tượng được hiển thị thành dòng chữ như thế nào**, chẳng hạn khi in một hóa đơn. Còn `is_ready()` kiểm tra hóa đơn đã có số tiền để tính tổng chưa. Hai việc khác nhau: hóa đơn thiếu tiền vẫn có mã nhưng chưa đủ dữ liệu để tính doanh thu; `None` không đồng nghĩa 0 đồng. Chương trình tạo đối tượng cho 18 hóa đơn, chỉ cộng tiền của các đối tượng sẵn sàng và ghi riêng mã thiếu. Đặt quy tắc hiển thị và kiểm tra trong đối tượng giúp mọi nơi in hóa đơn theo cùng một cách.

Quy trình:

1. Tạo đối tượng `Invoice` cho đủ 18 dòng và lưu mã/số tiền.
2. Định nghĩa `is_ready()` và `__str__()` theo trạng thái có/thiếu tiền.
3. In hóa đơn mẫu, thống kê số hợp lệ, số thiếu và tổng giá trị.

No Note — Không chú thích

```
amounts =
[100000,125000,150000,175000,200000,None,125000,150000,175000,200000,100000,125000,150000,175000,No
ne,100000,125000,150000]
class Invoice:
    def __init__(self, code, amount):
        self.code, self.amount = code, amount
    def is_ready(self):
        return isinstance(self.amount, (int, float)) and self.amount >= 0
    def __str__(self):
        if self.is_ready():
            value = f"{self.amount:,} đồng"
        else:
            value = "Chưa có sô´tiền"
        return f"{self.code}: {value}"

invoices = []
for i, amount in enumerate(amounts, 1):
    invoices.append(Invoice(f"O{i:02d}", amount))
ready = []
missing = []
for order in invoices:
    if order.is_ready():
        ready.append(order)
    else:
        missing.append(order.code)
if len(invoices) != 18:
    raise ValueError("Cần đủ 18 hóa đơn.")
print("Sô´hóa đơn:", len(invoices))
print("Săn sàng:", len(ready))
print("Thiếú:", ", ".join(missing))
total_amount = 0
for order in ready:
    total_amount += order.amount
print("Tổng (đồng):", total_amount)
print("Mẫũ:", str(invoices[0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo 18 Invoice có cách hiển thị riêng qua __str__ và tính tổng hóa đơn hợp lệ.
# Dữ liệu: 18 khoản tiền đồng, None là hóa đơn chưa có số tiền.
amounts =
[100000,125000,150000,175000,200000,None,125000,150000,175000,200000,100000,125000,150000,175000,No
ne,100000,125000,150000]
# class định nghĩa mẫu có thuộc tính và phương thức; các instance được tạo bằng cách gọi tên lớp.
class Invoice:
    # __init__ tự chạy khi tạo object; self là instance hiện tại.
    def __init__(self, code, amount):
        # Gán đồng thời hai thuộc tính instance từ mã và tiền.
        self.code, self.amount = code, amount
    # Phương thức kiểm tra tiền hóa đơn đã là số không âm hay chưa.
    def is_ready(self):
        # isinstance() xác nhận số int/float, tránh so sánh >=0 với None.
        return isinstance(self.amount, (int, float)) and self.amount >= 0
    # __str__ quy định chuỗi khi gọi str(invoice) hoặc in object.
    def __str__(self):
        # Dùng quy tắc kiểm tra đã định nghĩa trong class trước khi định dạng.
        if self.is_ready():
            # f-string :, thêm dấu phân nhóm nghìn khi in số tiền đồng.
            value = f"{self.amount:,} đồng"
        else:
            value = "Chưa có số tiền"
        # return trả giá trị cho nơi gọi và kết thúc hàm.
        return f"{self.code}: {value}"

invoices = []
# enumerate() trả (số thứ tự, phần tử); start=1 khi truyền 1, mặc định bắt đầu 0.
for i, amount in enumerate(amounts, 1):
    # append() thêm object Invoice mới; f-string :02d tạo mã O01..O18.
    invoices.append(Invoice(f"O{i:02d}", amount))
ready = []
missing = []
for order in invoices:
    # Gọi kiểm tra hóa đơn ngay trên từng instance.
    if order.is_ready():
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        ready.append(order)
    else:
        # append() thêm một phần tử vào cuối danh sách và cập nhật danh sách gốc.
        missing.append(order.code)
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
if len(invoices) != 18:
    # raise báo lỗi rồi dừng luồng hiện tại nếu dữ liệu không đáp ứng yêu cầu.
    raise ValueError("Cần đủ 18 hóa đơn.")
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Số hóa đơn:", len(invoices))
# len() trả số phần tử, dùng để đếm hoặc kiểm tra đủ bản ghi.
print("Sẵn sàng:", len(ready))
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Thiếu:", ", ".join(missing))
total_amount = 0
for order in ready:
    # += cộng chỉ các hóa đơn hợp lệ, đơn vị đồng.
    total_amount += order.amount
print("Tổng (đồng):", total_amount)
# str() gọi Invoice.__str__ của phần tử đầu [0] để tạo chuỗi dễ đọc.
print("Mẫu:", str(invoices[0]))
```

Kết quả khi chạy:

```
Số hóa đơn: 18
Sẵn sàng: 16
Thiếu: O06, O15
```

```
Tổng (đồng): 2325000
Mẫu: O01: 100,000 đồng
```

Chỉ tính tổng trên các đối tượng `is_ready() == True`. Giá trị `None` là một trạng thái thiếu cần sửa, không phải số tiền 0.

Tình huống 5 — Cấp 5: Xây đối tượng tồn kho độc lập và xử lý 20 lượt xuất

Từ vựng trong mục: State — trạng thái; **Encapsulation** — đóng gói; **Instance method** — phương thức đối tượng.

Tình huống thực tế: Một kho có **5 loại sản phẩm P01 đến P05**, tồn đầu ngày lần lượt là **10, 9, 8, 7, 5 món**. Trong ngày có **20 lượt yêu cầu xuất hàng**; mỗi lượt chứa mã sản phẩm và số món muốn lấy. Có yêu cầu lấy quá số đang còn, một yêu cầu chưa nhập số lượng (`None`) và một yêu cầu lấy 0 món. Nhân viên cần biết yêu cầu nào bị từ chối và số tồn cuối của từng loại, bảo đảm không loại nào bị trừ xuống số âm.

Vì sao chọn cách này? Trạng thái của đối tượng là dữ liệu hiện tại được giữ bên trong nó; ở đây `StockItem` giữ số món còn của **một mã sản phẩm**. Phương thức `reserve()` kiểm tra số muốn xuất có phải lượng hợp lệ và có vượt tồn không. Chỉ khi yêu cầu được chấp nhận mới trừ vào `self.quantity`; nếu thiếu số lượng, lấy 0 món hoặc lấy nhiều hơn tồn thì từ chối và **giữ nguyên số tồn**. Cần xử lý 20 lượt **đúng thứ tự** vì sau khi xuất một lượt, số còn lại cho lượt tiếp theo đã khác. Mỗi SKU có đối tượng riêng nên xuất P01 không thể vô tình trừ tồn của P02.

Quy trình:

1. Khởi tạo 5 đối tượng độc lập từ tồn đầu ngày.
2. Duyệt 20 yêu cầu theo thứ tự; `reserve()` trả True/False thay vì trừ vô điều kiện.
3. Đếm số yêu cầu thành công, ghi mã lỗi và đối chiếu tồn không âm.

No Note — Không chú thích

```
initial = {"P01":10,"P02":9,"P03":8,"P04":7,"P05":5}
requests = [("P01",2),("P02",1),("P03",3),("P01",4),("P02",5),("P03",2),("P04",1),("P01",3),
("P03",6),("P04",3),("P02",2),("P01",7),("P05",1),("P03",1),("P04",4),("P05",2),("P01",None),
("P02",0),("P04",2),("P05",8)]
class StockItem:
    def __init__(self, code, quantity):
        self.code, self.quantity = code, quantity
    def reserve(self, requested):
        if not isinstance(requested, int) or requested <= 0 or requested > self.quantity:
            return False
        self.quantity -= requested
        return True

items = {}
for code, qty in initial.items():
    items[code] = StockItem(code, qty)
rejected = []
for index, (code, requested) in enumerate(requests, 1):
    if not items[code].reserve(requested):
        rejected.append(f"T{index:02d}")
if len(requests) != 20:
    raise ValueError("Cần xử lý đủ 20 yêu cầu xuất kho.")
for item in items.values():
    if item.quantity < 0:
        raise ValueError("Tồn kho của sản phẩm không được âm.")
print("Yêu cầu:", len(requests))
print("Chấp nhận:", len(requests) - len(rejected))
```

```
print("Từ chôí:", ", ".join(rejected))
stock_labels = []
for code in sorted(items):
    stock_labels.append(f"{code}={items[code].quantity}")
print("Tôn cuôí:", ", ".join(stock_labels))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo đôi tượng StockItem đe̓ từ chôí yêu câù xuât́ kho không hợp lệ.
# Dữ liệu: initial là tôn kho 5 SKU theo món; requests gôm̀ 20 bộ (SKU, lượng xin xuât́), None/0/vượt
tôn bị từ chôí.
initial = {"P01":10,"P02":9,"P03":8,"P04":7,"P05":5}
requests = [("P01",2),("P02",1),("P03",3),("P01",4),("P02",5),("P03",2),("P04",1),("P01",3),
("P03",6),("P04",3),("P02",2),("P01",7),("P05",1),("P03",1),("P04",4),("P05",2),("P01",None),
("P02",0),("P04",2),("P05",8)]
# Môĩ instance StockItem lưu tôn của một SKU và cung cấp phương thức xuât́.
class StockItem:
    # __init__ tự chạy khi tạo object; self là instance hiện tại.
    def __init__(self, code, quantity):
        # Gán mã SKU và số món hiện còn lên object.
        self.code, self.quantity = code, quantity
    # Phương thức cố gắńg dành hàng, trả True nêú đủ điêù kiện.
    def reserve(self, requested):
        # isinstance(...,int) loại None/kiểủ sai; or loại lượng <=0 hoặc lớn hơn tôn.
        if not isinstance(requested, int) or requested <= 0 or requested > self.quantity:
            # False nghĩa yêu câù bị từ chôí, không trừ tôn.
            return False
        # -= trừ món đã cấp, chỉ sau kiêm̉ tra an toàn.
        self.quantity -= requested
        # True báo xuât́ thành công.
        return True

items = {}
# items() duyệt các cặp (SKU, số lượng tôn đâù kỳ) đe̓ tạo object.
for code, qty in initial.items():
    # Dictionary ánh xạ SKU sang object giữ trạng thái tôn riêng.
    items[code] = StockItem(code, qty)
rejected = []
# enumerate(...,1) găń số thứ tự yêu câù 1..20.
for index, (code, requested) in enumerate(requests, 1):
    # Gọi phương thức trên đúng SKU; not tìm các lần từ chôí đe̓ ghi mã.
    if not items[code].reserve(requested):
        # append() thêm một phần tử vào cuôí danh sách và cập nhật danh sách gôć.
        # Định dạng f-string :02d cho số thứ tự thành 2 chữ số với số 0 bổ sung.
        rejected.append(f"T{index:02d}")
# len() trả số phần tử, dùng đe̓ đêḿ hoặc kiêm̉ tra đủ bản ghi.
if len(requests) != 20:
    # raise báo lôĩ rôì dừng luông̀ hiện tại nêú dữ liệu không đáp ứng yêu câù.
    raise ValueError("Cần xử lý đủ 20 yêu câù xuât́ kho.")
# values() lâý các giá trị dictionary, không lâý khóa.
for item in items.values():
    # Kiêm̉ tra cuôí không có số tôn âm.
    if item.quantity < 0:
        # raise báo lôĩ rôì dừng luông̀ hiện tại nêú dữ liệu không đáp ứng yêu câù.
        raise ValueError("Tôn kho của sản phẩm̉ không được âm.")
# len() trả số phần tử, dùng đe̓ đêḿ hoặc kiêm̉ tra đủ bản ghi.
print("Yêu câù:", len(requests))
# len() trả số phần tử, dùng đe̓ đêḿ hoặc kiêm̉ tra đủ bản ghi.
print("Châṕ nhận:", len(requests) - len(rejected))
# join() nôí các chuôĩ bằng ký tự phân cách, ở đây là dâú phâỷ và khoảng trăńg.
print("Từ chôí:", ", ".join(rejected))
stock_labels = []
# sorted() xêṕ các khóa SKU theo ABC trước khi in.
for code in sorted(items):
    # append() thêm một phần tử vào cuôí danh sách và cập nhật danh sách gôć.
```

```
    stock_labels.append(f"{code}={items[code].quantity}")
# join() nối các chuỗi bằng ký tự phân cách, ở đây là dấu phẩy và khoảng trắng.
print("Tồn cuối:", ", ".join(stock_labels))
```

Kết quả khi chạy:

```
Yêu cầu: 20
Chấp nhận: 14
Từ chối: T09, T12, T15, T17, T18, T20
Tồn cuối: P01=1, P02=1, P03=2, P04=1, P05=2
```

Người học cần theo dõi trạng thái thay đổi **qua 20 yêu cầu nối tiếp**, không chỉ tính một đơn hàng. Khi một yêu cầu bị từ chối, tồn kho phải giữ nguyên trước khi xét yêu cầu tiếp theo.

6. Sai lầm thường gặp

- `class` là khuôn mẫu; đối tượng chỉ xuất hiện khi gọi lớp như `Student('An', 8)`.
- `self.name` là thuộc tính của đối tượng, không phải biến dùng chung giữa mọi học viên. Mỗi đối tượng có trạng thái riêng.
- `__init__` chuẩn bị dữ liệu ban đầu, còn `__str__` quy định cách hiển thị; hai phương thức có mục đích khác nhau.
- `self._value` dùng dấu gạch dưới theo quy ước, không tự tạo thuộc tính hoàn toàn riêng tư. Muốn bảo vệ dữ liệu cần kiểm soát cách cập nhật và kiểm tra hợp lệ.
- Điểm vượt 0–10 sẽ phát sinh `ValueError` trong ví dụ `Score`; không bỏ qua xử lý lỗi ở chương trình thực tế.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a class in Python OOP? | Class trong OOP Python là gì?

A. Only a built-in Python function | Chỉ một hàm Python có sẵn

B. A template defining object state and behavior | Khuôn mẫu định nghĩa trạng thái và hành vi đối tượng

C. A concrete instance created from a class | Instance cụ thể được tạo từ class

D. A value stored in one object | Giá trị lưu trong một đối tượng

Đáp án: B

Giải thích: Class định nghĩa cấu trúc dùng chung cho các đối tượng, gồm thuộc tính và phương thức. Ví dụ class `Student` có thể định nghĩa trường tên và phương thức `display()`; từng sinh viên là một instance của class đó.

Q2. What is an object or instance? | Object hay instance là gì?

A. A statement defining a new class | Câu lệnh định nghĩa class mới

B. A concrete instance of a class | Instance cụ thể của class

C. An attribute name only | Chỉ tên thuộc tính

D. A block of import statements | Khối lệnh import

Đáp án: B

Giải thích: Object hay instance là giá trị cụ thể được tạo từ class. Hai instance `Student` có thể chạy cùng các phương thức nhưng có giá trị `name` khác nhau, vì thế class không đồng nghĩa với một đối tượng.

Q3. What is an attribute? | Attribute là gì?

A. A method invocation | Lời gọi phương thức

B. Data or a value associated with an object | Dữ liệu hoặc giá trị gắn với đối tượng

C. A syntax token beginning every class | Token cú pháp bắt đầu mọi class

D. A class declaration | Câu lệnh khai báo class

Đáp án: B

Giải thích: Attribute là dữ liệu hoặc giá trị gắn với đối tượng được truy cập qua tên, như `student.name`. Nó biểu diễn trạng thái hay đặc điểm, còn phương thức được gọi để thực hiện hành vi.

Q4. What is an instance method? | Phương thức đối tượng là gì?

A. An object stored in a list | Đối tượng được lưu trong List

B. A value that cannot be called | Giá trị không thể gọi

C. A class-defined function operating on an instance | Hàm định nghĩa trong class thao tác trên instance

D. A class with no methods | Class không có phương thức

Đáp án: C

Giải thích: Phương thức instance là hàm định nghĩa trong class và nhận đối tượng làm đối số đầu khi gọi qua instance. Với `student.show()`, phương thức có thể đọc `self.name`; thuộc tính như `student.name` là dữ liệu, không phải lời gọi phương thức.

Q5. Which syntax begins a class definition? | Cú pháp nào mở đầu định nghĩa class?

A. `__init__` | `__init__`

B. `self` | `self`

C. `class Student:` | `class Student:`

D. `Student()` | `Student()`

Đáp án: C

Giải thích: `class Student:` mở đầu định nghĩa class; các lệnh thụt lề bên dưới tạo thân class. Gọi `Student()` là thao tác khác: tạo instance sau khi class phù hợp đã được định nghĩa.

Q6. If `Student` requires no constructor arguments, which expression creates a Student instance? | Nếu `Student` không yêu cầu đối số khởi tạo, biểu thức nào tạo một instance Student?

A. `Student()` | `Student()`

B. `def Student:` | `def Student:`

C. `self.Student` | `self.Student`

D. `class Student:` | `class Student:`

Đáp án: A

Giải thích: Gọi `Student()` tạo instance nếu class có thể khởi tạo mà không cần đối số bắt buộc. Nếu `__init__` yêu cầu đầu vào khác thì phải truyền chúng khi gọi, không được cho rằng ngoặc trống luôn hợp lệ.

Q7. Which special method initializes instance attributes? | Phương thức đặc biệt nào khởi tạo thuộc tính đối tượng?

A. `__repr__` | `__repr__`

B. `__newstate__` | `__newstate__`

C. `__str__` | `__str__`

D. `__init__` | `__init__`

Đáp án: D

Giải thích: `__init__` khởi tạo instance vừa được tạo, thường gán thuộc tính như `self.name = name`. Python tạo đối tượng trước rồi mới gọi hàm khởi tạo; bản thân `__init__` không thực hiện cấp phát đối tượng.

Q8. What does `self` refer to in an instance method? | `self` trong phương thức đối tượng chỉ gì?

A. The class object, never an instance | Đối tượng class, không bao giờ là instance

B. The first attribute value | Giá trị thuộc tính đầu tiên

C. A newly created unrelated object | Đối tượng khác vừa được tạo

D. The instance receiving the method call | Instance nhận lời gọi phương thức

Đáp án: D

Giải thích: Trong phương thức instance thông thường, `self` chỉ đối tượng cụ thể nhận lời gọi. Vì vậy `a.show()` và `b.show()` có thể đọc các thuộc tính khác nhau qua `self` dù cùng định nghĩa class.

Q9. What does `self.name = name` do? | `self.name = name` làm gì?

A. Stores `name` on the current instance | Lưu `name` trên instance hiện tại

B. Sets the same name on all instances | Đặt cùng tên trên mọi instance

C. Changes the class's name | Đổi tên class

D. Calls a method named `name` | Gọi phương thức tên `name`

Đáp án: A

Giải thích: `self.name = name` gán giá trị tham số `name` cho thuộc tính của instance hiện tại. Vì vậy mỗi đối tượng có thể mang tên khác nhau; lệnh không tạo một thuộc tính cấp class được dùng chung.

Q10. What does `__str__` normally customize? | `__str__` thường tùy chỉnh điều gì?

A. The readable string representation of an object | Biểu diễn chuỗi dễ đọc của đối tượng

B. The list of all class attributes | Danh sách mọi thuộc tính class

C. The class inheritance hierarchy | Hệ phân cấp kế thừa class

D. The instance's constructor arguments | Các đối số khởi tạo instance

Đáp án: A

Giải thích: `__str__` điều khiển chuỗi dễ đọc trả về từ `str(obj)` và thường được `print(obj)` hiển thị. Phương thức này phải trả một chuỗi thay vì tự in trực tiếp; nó khác `__init__` dùng để khởi tạo trạng thái.

Q11. What does `raise ValueError(...)` do? | `raise ValueError(...)` làm gì?

A. Raises an exception for an invalid value | Phát sinh ngoại lệ cho giá trị không hợp lệ

B. Marks an attribute private | Đánh dấu thuộc tính riêng tư

C. Converts invalid values to `None` | Chuyển giá trị sai thành `None`

D. Silently ignores invalid data | Lặng lẽ bỏ qua dữ liệu sai

Đáp án: A

Giải thích: `raise ValueError(...)` báo giá trị đầu vào không hợp lệ và ngắt luồng bình thường bằng một ngoại lệ. Ví dụ setter có thể từ chối tuổi âm thay vì lặng lẽ lưu trạng thái không hợp lý.

Q12. What does the prefix `_value` conventionally mean? | Tiền tố `_value` theo quy ước có nghĩa gì?

A. A constant that cannot be changed | Hằng không thể thay đổi

B. An attribute invisible to external code | Thuộc tính mã ngoài không thể thấy

C. An internal-use detail by convention | Chi tiết sử dụng nội bộ theo quy ước

D. An automatically encrypted value | Giá trị được tự động mã hóa

Đáp án: C

Giải thích: Một dấu gạch dưới đầu tên như `_value` theo quy ước đánh dấu chi tiết nội bộ. Python không thực thi tính riêng tư chỉ nhờ cách đặt tên này, nên mã bên ngoài vẫn có thể đọc thuộc tính nếu nó tồn tại.

Q13. What is encapsulation? | Đóng gói trong OOP là gì?

A. Copying every attribute into a global variable | Sao chép mọi thuộc tính sang biến toàn cục

B. Converting methods into strings | Chuyển phương thức thành chuỗi

C. Creating a new instance on every access | Tạo instance mới mỗi lần truy cập

D. Grouping related data and methods | Gom dữ liệu và phương thức liên quan

Đáp án: D

Giải thích: Đóng gói gom dữ liệu và các thao tác liên quan vào cùng class hoặc giao diện đối tượng. Ví dụ phương thức của `BankAccount` có thể cập nhật số dư kèm kiểm tra thay vì để mọi nơi gọi tự sửa các biến rời rạc.

Q14. Why validate values in a setter method? | Vì sao kiểm tra dữ liệu trong setter?

A. To make every instance share its private data | Buộc mọi instance chia sẻ dữ liệu riêng

B. To prevent all future method calls | Ngăn mọi lời gọi phương thức sau đó

C. To bypass all input validation | Bỏ qua toàn bộ kiểm tra dữ liệu

D. To validate inputs and preserve valid object state | Kiểm tra đầu vào và giữ trạng thái hợp lệ

Đáp án: D

Giải thích: Setter kiểm tra dữ liệu trước khi thay đổi trạng thái đối tượng, giúp giữ các ràng buộc như tuổi không được âm. Nếu thiếu kiểm tra, nơi gọi có thể lưu giá trị sai khiến phương thức khác hoạt động không đúng.

Q15. How do two instances of the same class relate? | Hai đối tượng cùng class có quan hệ gì?

A. They always share all instance attribute values | Chúng luôn dùng chung mọi giá trị thuộc tính instance

B. They cannot invoke the same methods | Chúng không thể gọi cùng phương thức

C. They share class behavior but can have different states | Chúng chia sẻ hành vi class nhưng có trạng thái khác nhau

D. They are the same object in memory | Chúng là cùng một đối tượng trong bộ nhớ

Đáp án: C

Giải thích: Các instance cùng class dùng chung định nghĩa phương thức nhưng có thể chứa các thuộc tính instance riêng. Nếu `a.name='An'` và `b.name='Binh'`, sửa `a.name` không tự thay đổi `b.name`.

Q16. What is a class attribute? | Thuộc tính cấp class là gì?

A. A method's local variable | Biến cục bộ của phương thức

B. An attribute defined on the class and found by lookup | Thuộc tính định nghĩa trên class và được tìm bằng tra cứu

C. An attribute defined on an object instance only | Thuộc tính chỉ định nghĩa trên instance

D. A required parameter of `__init__` | Tham số bắt buộc của `__init__`

Đáp án: B

Giải thích: Thuộc tính cấp class được khai báo trên class thay vì riêng từng instance và được tìm thấy khi tra cứu qua class hoặc instance. Instance có thể che khuất thuộc tính đó bằng giá trị riêng, nên đây không phải trường luôn đồng bộ vĩnh viễn.

Q17. What does `obj.method()` do? | `obj.method()` làm gì?

A. Invokes a method on `obj` | Gọi phương thức trên `obj`

B. Returns the object's class without invoking anything | Trả class của đối tượng mà không gọi gì

C. Always creates a new instance | Luôn tạo instance mới

D. Always deletes an attribute | Luôn xóa thuộc tính

Đáp án: A

Giải thích: `obj.method()` tìm và gọi phương thức có tên trên `obj`, thông thường truyền instance đó làm `self`. Chỉ đọc `obj.attribute` thì lấy giá trị, không tự gọi phương thức.

Q18. When the attribute exists, what does `obj.attribute` read? | Khi thuộc tính tồn tại, `obj.attribute` đọc gì?

A. Creates a new method automatically | Tự tạo phương thức mới

B. Reads the named attribute on `obj` | Đọc thuộc tính có tên trên `obj`

C. Invokes every method of the object | Gọi mọi phương thức của đối tượng

D. Returns `True` only for class attributes | Chỉ trả `True` với thuộc tính cấp class

Đáp án: B

Giải thích: `obj.attribute` truy cập giá trị của thuộc tính có tên nếu nó tồn tại. Khác `obj.method()`, biểu thức không gọi hàm; nếu thiếu thuộc tính, Python thông thường phát sinh `AttributeError`.

Q19. What does `isinstance(obj, Student)` check? | `isinstance(obj, Student)` kiểm tra gì?

A. The object's exact class only, excluding subclasses | Chỉ class chính xác, loại class con

B. Whether the object has a method named `Student` | Đối tượng có phương thức tên `Student` không

C. Whether the object is a `Student` or its subclass | Đối tượng thuộc `Student` hay class con không

D. The class name as a string | Tên class ở dạng chuỗi

Đáp án: C

Giải thích: `isinstance(obj, Student)` đúng khi `obj` thuộc class `Student` hoặc một class con của nó. Ngược lại, `type(obj) is Student` chỉ kiểm tra class thực tế chính xác và loại các instance của class con.

Q20. What does `type(obj)` return? | `type(obj)` trả về gì?

A. Whether a class membership test is true | Phép kiểm tra thuộc class có đúng không

B. The object itself with unchanged type | Chính đối tượng và kiểu không đổi

C. A Boolean checking only its attributes | Boolean chỉ kiểm tra thuộc tính

D. The object's actual runtime class | Class thực tế của đối tượng khi chạy

Đáp án: D

Giải thích: `type(obj)` trả về class thực tế của đối tượng lúc chạy, chẳng hạn `Student`. `isinstance(obj, Student)` thì trả Boolean về quan hệ thuộc class và có thể chấp nhận instance của class con.

8. Nối sang bài tiếp theo

`Hàm / Dictionary → OOP → Kế thừa`.

PY-11 — Kế thừa, đa hình và lớp trừu tượng ABC

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Inheritance — kế thừa; **Polymorphism** — đa hình.

`Class/Object → Inheritance → Abstract API → Kiến trúc ứng dụng`

Học xong dùng để làm gì? Dùng chung thao tác cho nhiều loại đối tượng, ví dụ các loại phí có công thức khác nhau.

Kết nối kiến thức tiếp theo: PY-12 đọc và lưu dữ liệu từ file/CSV, có xử lý lỗi.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Inheritance	kế thừa	Lớp con dùng lại các đặc điểm lớp cha
Polymorphism	đa hình	Cùng lời gọi nhưng các lớp có cách đáp khác nhau
Abstract class	lớp trừu tượng	Khung lớp ràng buộc phương thức cần cài đặt
Override	ghi đè	Lớp con cài lại phương thức của lớp cha

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`class Child(Parent)`	Kế thừa thông tin chung
`super().__init__(...)`	Gọi khởi tạo lớp cha để không lặp code
`ABC/@abstractmethod`	Bắt lớp con định nghĩa giao diện bắt buộc
`isinstance(obj,Class)`	Xác minh đối tượng thuộc lớp khi cần

4. Khái niệm liên quan

Kế thừa, đa hình và lớp trừu tượng giải quyết ba nhu cầu khác nhau trong OOP: dùng lại đặc điểm chung, xử lý cùng thao tác theo cách riêng và đặt ra chức năng bắt buộc cho lớp con. Từ ví dụ sản phẩm đến dịch vụ thông báo, ta xem từng khái niệm tác động lên lớp, đối tượng và phương thức thế nào.

4.1. Inheritance — Kế thừa từ lớp cha

Từ vựng trong mục: Inheritance — kế thừa; **Parent/child class** — lớp cha/lớp con.

Cửa hàng có nhiều loại sản phẩm, nhưng mỗi món đều có tên. Lớp cha `Product` giữ tên chung, còn lớp con `Electronic(Product)` nhận lại cách lưu tên và có thêm thuộc tính riêng `warranty_months` (tháng bảo hành). `super().__init__(name)` gọi phương thức khởi tạo của lớp cha; nhờ vậy không phải viết lại cách lưu tên ở mọi loại sản phẩm. Đây là **kế thừa**: lớp con dùng lại và mở rộng hành vi, thuộc tính từ lớp cha.

No Note — Không chú thích

```
class Product:
    def __init__(self, name):
        self.name = name

class Electronic(Product):
    def __init__(self, name, warranty_months):
        super().__init__(name)
        self.warranty_months = warranty_months

phone = Electronic('Phone', 12)
print(phone.name, phone.warranty_months)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng Electronic kế´thừa Product để tái sử dụng tên và bổ sung tháng bảo hành.
# Dữ liệu: Object Phone có bảo hành 12 tháng.
# class định nghĩa kiểu object và các phương thức của nó.
class Product:
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số´sau self là dữ
liệu đầu vào.
    def __init__(self, name):
        # self.thuộc_tính lưu trạng thái riêng của object hiện tại.
        self.name = name

# Cú pháp (Product) cho Electronic kế´thừa Product.
class Electronic(Product):
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số´sau self là dữ
liệu đầu vào.
    def __init__(self, name, warranty_months):
        # super() gọi hàm __init__ của cha để lưu name.
        super().__init__(name)
        # Thuộc tính warranty_months giữ số´tháng bảo hành riêng của object.
        self.warranty_months = warranty_months

phone = Electronic('Phone', 12)
# Truy cập thuộc tính name do lớp cha khởi tạo bằng dấu chấm.
print(phone.name, phone.warranty_months)
```

Kết quả khi chạy:

```
Phone 12
```

`Phone` là tên được lớp cha khởi tạo; `12` là số tháng bảo hành được lớp con bổ sung.

4.2. Polymorphism — Đa hình

Từ vựng trong mục: Polymorphism — đa hình; **Method** — phương thức.

Giả sử ứng dụng có thông báo bằng email và tin nhắn. Cả hai đều có phương thức `send(message)`, nhưng kết quả khác nhau tùy loại. **Đa hình** nghĩa là code gọi cùng một tên phương thức trên các đối tượng khác

nhau mà mỗi đối tượng tự quyết định cách xử lý. Vòng `for` phía dưới không cần viết hai nhánh kiểm tra loại thông báo.

No Note — Không chú thích

```
class Email:
    def send(self, message):
        return 'Email: ' + message

class SMS:
    def send(self, message):
        return 'SMS: ' + message

for sender in [Email(), SMS()]:
    print(sender.send('Hello'))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cùng lời gọi send() trên Email và SMS tạo kết quả theo loại đối tượng.
# Dữ liệu: Chuỗi Hello được chuyển qua hai loại object; không gửi ra mạng.
# Class Email có hành vi gửi dạng chuỗi bắt đầu bằng Email:.
class Email:
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def send(self, message):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 'Email: ' + message

# Class SMS có cùng tên method send(), nhưng kết quả có tiền tố SMS:.
class SMS:
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def send(self, message):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 'SMS: ' + message

# Lần lượt tạo Email() và SMS(); đa hình tự chọn send của từng object.
for sender in [Email(), SMS()]:
    # Cùng lệnh .send('Hello') cho kết quả theo loại thật của sender, rồi print().
    print(sender.send('Hello'))
```

Kết quả khi chạy:

```
Email: Hello
SMS: Hello
```

Cùng lời gọi `send('Hello')` nhưng hai loại đối tượng trả về hai kết quả khác nhau.

4.3. Abstract class — Lớp trừu tượng

Từ vựng trong mục: Abstract class — lớp trừu tượng; **Abstract method** — phương thức trừu tượng.

Khi mọi bộ gửi thông báo **bắt buộc** phải có `send()`, ta tạo lớp trừu tượng `Notifier(ABC)` và đánh dấu `@abstractmethod`. Lớp này quy định hợp đồng nhưng chưa đưa ra cách gửi cụ thể. Lớp `Console` phải cài đặt `send()` mới tạo đối tượng được. Chỉ kế thừa `ABC` mà **không có phương thức trừu tượng** thì chưa áp đặt yêu cầu đó.

No Note — Không chú thích

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    @abstractmethod
    def send(self, message):
        pass

class Console(Notifier):
    def send(self, message):
        return 'Console: ' + message

print(Console().send('Hello'))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Quy định phương thức send bắt buộc bằng lớp trừu tượng ABC.
# Dữ liệu: Console hiện thực hóa giao diện Notifier.
# Thư viện abc cung cấp ABC (lớp trừu tượng) và abstractmethod (đánh dấu method bắt buộc).
from abc import ABC, abstractmethod

# Kế thừa ABC quy định giao diện không thể tạo nếu còn phương thức trừu tượng.
class Notifier(ABC):
    # Decorator bắt lớp con cụ thể cài send().
    @abstractmethod
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def send(self, message):
        # pass là chỗ giữ lệnh rỗng, chưa cung cấp cách xử lý.
        pass

# Console hiện thực hóa send() nên tạo object được.
class Console(Notifier):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def send(self, message):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 'Console: ' + message

# Console() tạo object, .send() trả chuỗi, print() hiển thị.
print(Console().send('Hello'))
```

Kết quả khi chạy:

```
Console: Hello
```

`Console` đáp ứng hợp đồng `send()`; nếu bỏ phương thức ấy khỏi `Console`, Python sẽ không cho khởi tạo đối tượng.

4.4. Override — Ghi đè phương thức

Từ vựng trong mục: Override — ghi đè; **Inheritance** — kế thừa.

Lớp cha `Shipping` có phí mặc định 0 đồng, nhưng giao hàng nhanh cần phí 35.000 đồng. Lớp con `Express` **ghi đè (override)** phương thức `fee()` bằng định nghĩa mới cùng tên. Khi gọi `Express().fee()`, Python dùng bản của `Express`, không dùng bản của `Shipping`. Đây là một cách để đa hình xuất hiện thông qua quan hệ kế thừa.

No Note — Không chú thích

```
class Shipping:
    def fee(self):
        return 0

class Express(Shipping):
    def fee(self):
        return 35000

print(Shipping().fee(), Express().fee())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lớp Express ghi đè phương thức fee của lớp Shipping.
# Dữ liệu: Phí thông thường 0 đồng, express 35.000 đồng.
# class định nghĩa kiểu object và các phương thức của nó.
class Shipping:
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 0

# Express kế thừa Shipping, ghi đè fee() để trả mức phí riêng.
class Express(Shipping):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # Phí 35.000 đồng thay cho 0 đồng mặc định của lớp cha.
        return 35000

# Gọi fee trên hai instance khác loại, method của mỗi loại quyết định kết quả.
print(Shipping().fee(), Express().fee())
```

Kết quả khi chạy:

```
0 35000
```

Hai đối tượng gọi cùng `fee()` nhưng phí khác nhau: gói thường 0 đồng, gói nhanh 35.000 đồng.

5. Năm tình huống kế thừa/đa hình — từ đối tượng tới giao diện thanh toán

Tình huống 1 — Cấp 1: Hai loại sản phẩm kế thừa tên chung

Từ vựng trong mục: Inheritance — kế thừa; **Parent class** — lớp cha; **Override** — ghi đè.

Tình huống thực tế: Cửa hàng lập danh sách **10 sản phẩm** để giới thiệu trên bảng hàng. Mỗi dòng có tên sản phẩm, chẳng hạn Laptop hoặc Bàn, và nhóm hàng: `digital` nghĩa là thiết bị điện tử, `physical` nghĩa là đồ dùng thông thường. Nhân viên muốn lưu mỗi sản phẩm thành một đối tượng, xem tên và nhóm của sản phẩm đầu tiên, rồi đếm xem có bao nhiêu thiết bị và đồ dùng.

Vì sao chọn cách này? Kế thừa cho phép lớp con dùng lại phần thông tin chung của lớp cha. Ở đây lớp `Product` giữ tên sản phẩm; lớp `Digital` và `Physical` nhận cách lưu tên ấy mà không viết lại đoạn khởi tạo. **Ghi đè phương thức** nghĩa là cả hai lớp con đều có `category()` nhưng trả nhãn nhóm của chính mình. Chương trình tạo đúng đối tượng theo nhãn của từng dòng, rồi gọi cùng `category()` để đếm; không cần mỗi chỗ xem sản phẩm lại kiểm tra tên Laptop hay Bàn. Cách này có ích khi nhiều nhóm hàng chia sẻ thông tin, nhưng cách phân nhóm vẫn khác nhau.

Quy trình:

1. Định nghĩa lớp `Product` giữ tên và có phương thức `category()`.
2. Tạo lớp `Digital` và `Physical` ghi đè tên nhóm.
3. Tạo 10 đối tượng và đếm mỗi kiểu.

No Note — Không chú thích

```
data = [("Laptop","digital"),("Bàn","physical"),("Chuột","digital"),("Ghế","physical"),("Màn
hình","digital"),("Sổ","physical"),("Bàn phím","digital"),("Bút","physical"),("Tablet","digital"),
("Tủ","physical")]
class Product:
    def __init__(self, name):
        self.name = name
    def category(self):
        return "other"
class Digital(Product):
    def category(self):
        return "digital"
class Physical(Product):
    def category(self):
        return "physical"
items = []
for name, kind in data:
    if kind == "digital":
        product = Digital(name)
    else:
        product = Physical(name)
    items.append(product)
if len(items) != 10:
    raise ValueError("Cần tạo đủ 10 mặt hàng.")
print("Sản phẩm:", len(items))
print("Đầu tiên:", items[0].name, items[0].category())
digital_count = 0
physical_count = 0
for product in items:
    if product.category() == "digital":
        digital_count += 1
    if product.category() == "physical":
        physical_count += 1
print("Thiết bị:", digital_count)
print("Đồ dùng:", physical_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo 10 sản phẩm Digital/Physical và đếm qua cùng phương thức category.
# Dữ liệu: 10 tuple (tên hàng, nhãn digital/physical).
# 10 bản ghi gồm tên hàng và loại sản phẩm.
data = [("Laptop","digital"),("Bàn","physical"),("Chuột","digital"),("Ghế","physical"),("Màn
hình","digital"),("Sổ","physical"),("Bàn phím","digital"),("Bút","physical"),("Tablet","digital"),
("Tủ","physical")]
# class định nghĩa kiểu object và các phương thức của nó.
class Product:
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số sau self là dữ
liệu đầu vào.
    def __init__(self, name):
        # self.thuộc_tính lưu trạng thái riêng của object hiện tại.
        self.name = name
    # Method đa hình: cùng tên nhưng giá trị trả tùy lớp con.
    def category(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return "other"
# Digital kế thừa Product, ghi đè category trả 'digital'.
```

```
class Digital(Product):
    # Method đa hình: cùng tên nhưng giá trị trả tùy lớp con.
    def category(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return "digital"
# Physical cũng kế thừa Product, category trả 'physical'.
class Physical(Product):
    # Method đa hình: cùng tên nhưng giá trị trả tùy lớp con.
    def category(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return "physical"
items = []
# Tách tên và nhãn của từng bản ghi.
for name, kind in data:
    if kind == "digital":
        product = Digital(name)
    else:
        product = Physical(name)
    # append() đưa object vào list.
    items.append(product)
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
if len(items) != 10:
    # raise ValueError báo đầu vào sai rồi dừng nhánh hiện tại.
    raise ValueError("Cần tạo đủ 10 mặt hàng.")
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
print("Sản phẩm:", len(items))
# [0] là object đầu; .name là thuộc tính và category() là method.
print("Đầu tiên:", items[0].name, items[0].category())
digital_count = 0
physical_count = 0
for product in items:
    # Gọi category() trên object thực tế, không cần ép kiểu về class con.
    if product.category() == "digital":
        digital_count += 1
    # Gọi category() trên object thực tế, không cần ép kiểu về class con.
    if product.category() == "physical":
        physical_count += 1
print("Thiết bị:", digital_count)
print("Đồ dùng:", physical_count)
```

Kết quả khi chạy:

```
Sản phẩm: 10
Đầu tiên: Laptop digital
Thiết bị: 5
Đồ dùng: 5
```

Lớp con không cần viết lại `__init__()` vì dùng được hành vi khởi tạo của lớp cha. Cùng lời gọi `category()` trả giá trị khác nhau tùy kiểu đối tượng.

Tình huống 2 — Cấp 2: Áp giao diện gửi thông báo cho 12 yêu cầu

Từ vựng trong mục: ABC — lớp trừu tượng; **Abstract method** — phương thức bắt buộc; **Polymorphism** — đa hình.

Tình huống thực tế: Một ứng dụng quản lý lớp dự định xử lý **12 thông báo N01–N12**. Mỗi thông báo được giao cho một trong hai kênh: `email` (thư điện tử) hoặc `screen` (hiển thị trên màn hình). Đây là chương trình **mô phỏng**, không gửi thư hay hiện thông báo thật. Người quản lý muốn chạy đủ 12 yêu cầu, xem mỗi kênh nhận bao nhiêu thông báo và đọc thử kết quả của thông báo đầu tiên.

Vì sao chọn cách này? **ABC** (lớp cơ sở trừu tượng) là bản quy ước cho các lớp xử lý thông báo: bất kỳ kênh nào cũng phải có phương thức `send(message)`. `@abstractmethod` đánh dấu việc lớp con phải tự cài đặt phương thức ấy. `EmailNotifier` trả chuỗi bắt đầu `email:`, `ScreenNotifier` trả `screen:`; không có kết nối mạng. Nhờ cùng tên `send()`, chương trình dùng một vòng `for` để gọi đúng kênh của mỗi thông báo mà không phải tự viết hai quy trình riêng cho mọi yêu cầu. Đếm các tiền tố trong chuỗi trả về giúp xác nhận chương trình đã định tuyến đúng 12 thông báo.

Quy trình:

1. Tạo `Notifier(ABC)` yêu cầu phương thức `send`.
2. Tạo hai lớp cài đặt `send()` với tiền tố kết quả riêng.
3. Duyệt 12 yêu cầu, chọn đối tượng và tổng hợp theo kênh.

No Note — Không chú thích

```
from abc import ABC, abstractmethod
channels =
["email","screen","email","screen","screen","email","screen","email","email","screen","email","scre
en"]
class Notifier(ABC):
    @abstractmethod
    def send(self, message):
        pass
class EmailNotifier(Notifier):
    def send(self, message):
        return "email:" + message
class ScreenNotifier(Notifier):
    def send(self, message):
        return "screen:" + message
notifiers = {"email": EmailNotifier(), "screen": ScreenNotifier()}
results = []
for i, channel in enumerate(channels, 1):
    message = notifiers[channel].send(f"N{i:02d}")
    results.append(message)
if len(results) != 12:
    raise ValueError("Cần xử lý đủ 12 thông báo.")
print("Yêu câù:", len(results))
email_count = 0
screen_count = 0
for text in results:
    if text.startswith("email:"):
        email_count += 1
    if text.startswith("screen:"):
        screen_count += 1
print("Kênh email:", email_count)
print("Kênh màn hình:", screen_count)
print("Ví dụ:", results[0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng giao diện Notifier chung để tạo 12 thông báo email/screen giả lập.
# Dữ liệu: 12 loại kênh email/screen, mã thông báo N01..N12.
# ABC/abstractmethod xác định giao diện send() bắt buộc.
from abc import ABC, abstractmethod
channels =
["email","screen","email","screen","screen","email","screen","email","email","screen","email","scre
en"]
class Notifier(ABC):
    # Đánh dấu send là method bắt buộc cho lớp con.
    @abstractmethod
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
```

```
    def send(self, message):
        # pass là chỗ giữ lệnh rỗng, chưa cung cấp cách xử lý.
        pass
class EmailNotifier(Notifier):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def send(self, message):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return "email:" + message
# class định nghĩa kiểu object và các phương thức của nó.
class ScreenNotifier(Notifier):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def send(self, message):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return "screen:" + message
# Dictionary ánh xạ chuỗi tên kênh tới object gửi tương ứng.
notifiers = {"email": EmailNotifier(), "screen": ScreenNotifier()}
results = []
# enumerate(...,1) đánh số kênh bắt đầu từ 1.
for i, channel in enumerate(channels, 1):
    # Tra dictionary lấy instance, .send() trả chuỗi tương ứng kênh.
    # f-string :02d tạo mã N01..N12, thêm số 0 ở đầu.
    message = notifiers[channel].send(f"N{i:02d}")
    # append() lưu chuỗi đầu ra mô phỏng.
    results.append(message)
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
if len(results) != 12:
    # raise ValueError báo đầu vào sai rồi dừng nhánh hiện tại.
    raise ValueError("Cần xử lý đủ 12 thông báo.")
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
print("Yêu cầu:", len(results))
email_count = 0
screen_count = 0
for text in results:
    # startswith(prefix) kiểm chuỗi có bắt đầu bằng tiền tố email:/screen: hay không.
    if text.startswith("email:"):
        email_count += 1
    # startswith(prefix) kiểm chuỗi có bắt đầu bằng tiền tố email:/screen: hay không.
    if text.startswith("screen:"):
        screen_count += 1
print("Kênh email:", email_count)
print("Kênh màn hình:", screen_count)
# [0] lấy dòng thông báo đầu.
print("Ví dụ:", results[0])
```

Kết quả khi chạy:

```
Yêu cầu: 12
Kênh email: 6
Kênh màn hình: 6
Ví dụ: email:N01
```

Có hai lớp con nhưng chỉ một lời gọi `send()`. Trả chuỗi xác nhận để kiểm chứng bài học, không khẳng định đã gửi thông báo đến người thật.

Tình huống 3 — Cấp 3: Ghi đè cách tính phí của 15 đơn giao

Từ vựng trong mục: Override — ghi đè; **Method dispatch** — chọn phương thức; **super()** — gọi lớp cha.

Tình huống thực tế: Cửa hàng phải tính phí cho **15 đơn vận chuyển**. Mỗi đơn được chọn một trong ba dịch vụ theo quy tắc giả lập: `standard` giao thường miễn phí; `express` giao nhanh tính **35.000 đồng/đơn**;

`cold` giao lạnh tính **50.000 đồng/đơn**. Giá không phụ thuộc số km trong bài này. Kế toán cần tổng tiền phí của tất cả đơn và số đơn sử dụng dịch vụ giao lạnh.

Vì sao chọn cách này? Lớp cha `Shipping` chứa loại dịch vụ và mức phí mặc định 0; hai **lớp con** `Express`, `Cold` có cùng phương thức `fee()` nhưng trả mức phí khác nhau. `super().__init__(...)` gọi phần khởi tạo của lớp cha để lưu tên loại dịch vụ, nên lớp con không phải lặp lại cách lưu. Chương trình tạo đối tượng theo từng loại rồi chỉ cần gọi `order.fee()` cho cả 15 đơn: đây là **đa hình**, cùng một lời gọi nhưng kết quả do loại đối tượng quyết định. Vì bảng phí trong bài cố định theo loại, không thêm phép tính khoảng cách hay thu thêm cho giao thường.

Quy trình:

1. Định nghĩa lớp cha `Shipping` với `fee()` mặc định 0.
2. Hai lớp con `Express`/`Cold` ghi đè phí và dùng `super()` để lưu loại.
3. Tạo 15 đối tượng, cộng kết quả gọi cùng một phương thức.

No Note — Không chú thích

```
types =
["standard","express","cold","standard","express","standard","cold","express","express","standard",
"cold","standard","express","cold","standard"]
class Shipping:
    def __init__(self, kind):
        self.kind = kind
    def fee(self):
        return 0
class Express(Shipping):
    def __init__(self):
        super().__init__("express")
    def fee(self):
        return 35000
class Cold(Shipping):
    def __init__(self):
        super().__init__("cold")
    def fee(self):
        return 50000
def create_standard_shipping():
    return Shipping("standard")

factory = {"standard": create_standard_shipping,
           "express": Express, "cold": Cold}
orders = []
for kind in types:
    orders.append(factory[kind]())
if len(orders) != 15:
    raise ValueError("Cần xử lý đủ 15 đơn vận chuyển.")
print("Đơn:", len(orders))
cold_count = 0
total_fee = 0
for order in orders:
    if order.kind == "cold":
        cold_count += 1
    total_fee += order.fee()
print("Giao lạnh:", cold_count)
print("Tổng phí (đồng):", total_fee)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo 15 kiểu vận chuyển bằng factory và tính phí theo lớp con.
# Dữ liệu: 15 nhãn standard/express/cold; phí lần lượt 0/35.000/50.000 đồng.
```

```
types =
["standard","express","cold","standard","express","standard","cold","express","express","standard",
"cold","standard","express","cold","standard"]
# Lớp cha lưu kind và cung cấp fee() mặc định.
class Shipping:
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số sau self là dữ
liệu đầu vào.
    def __init__(self, kind):
        # self.thuộc_tính lưu trạng thái riêng của object hiện tại.
        self.kind = kind
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 0
# Express là dịch vụ nhanh, cài phí 35.000 đồng.
class Express(Shipping):
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số sau self là dữ
liệu đầu vào.
    def __init__(self):
        # super() gọi __init__ của lớp cha để lưu đúng nhãn kind.
        super().__init__("express")
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 35000
# Cold là dịch vụ lạnh, cài phí 50.000 đồng.
class Cold(Shipping):
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số sau self là dữ
liệu đầu vào.
    def __init__(self):
        # super() gọi __init__ của lớp cha để lưu đúng nhãn kind.
        super().__init__("cold")
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 50000
# Factory tạo Shipping('standard') khi cần.
def create_standard_shipping():
    # return đưa kết quả trở lại nơi gọi và kết thúc method.
    return Shipping("standard")

# Dictionary chứa callable: hàm factory hoặc class có thể gọi để tạo object.
factory = {"standard": create_standard_shipping,
           "express": Express, "cold": Cold}
orders = []
for kind in types:
    # Lấy callable theo nhãn rồi gọi () tạo instance phù hợp.
    orders.append(factory[kind]())
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
if len(orders) != 15:
    # raise ValueError báo đầu vào sai rồi dừng nhánh hiện tại.
    raise ValueError("Cần xử lý đủ 15 đơn vận chuyển.")
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
print("Đơn:", len(orders))
cold_count = 0
total_fee = 0
for order in orders:
    if order.kind == "cold":
        cold_count += 1
    # Cùng method fee() nhưng tính phí khác nhau theo object; += cộng tổng đồng.
    total_fee += order.fee()
print("Giao lạnh:", cold_count)
print("Tổng phí (đồng):", total_fee)
```

Kết quả khi chạy:

```
Đơn: 15
Giao lạnh: 4
Tổng phí (đồng): 375000
```

Phí giao lạnh và nhanh cao hơn loại thường vì chính sách giả định của cửa hàng; không lấy bảng giá này làm giá thực tế của đơn vị vận chuyển.

Tình huống 4 — Cấp 4: Đọc 18 cảm biến nhiều kiểu và tách dữ liệu thiếu

Từ vựng trong mục: Polymorphism — đa hình; **Instance** — đối tượng; **Validation** — kiểm tra dữ liệu.

Tình huống thực tế: Trạm kỹ thuật ghi **18 kết quả cảm biến**, xen kẽ hai loại phép đo: nhiệt độ tính bằng **°C** và độ ẩm tính bằng **%**. Hai lượt không có giá trị nên để `None`; các lượt khác có số như 24 °C hoặc 56%. Nhân viên muốn một danh sách các kết quả được gắn đúng đơn vị, biết bao nhiêu lượt hợp lệ, bao nhiêu lượt thiếu và xem kết quả đầu tiên.

Vì sao chọn cách này? Đa hình cũng hữu ích khi các đối tượng phải cùng trả kết quả nhưng **đơn vị hiển thị khác nhau**. Lớp `Sensor` giữ giá trị chung; lớp `Temperature` định dạng `°C`, còn `Humidity` định dạng `%` trong phương thức `read()`. Chương trình đọc lần lượt cả 18 đối tượng bằng một lệnh gọi `read()` mà không phải kiểm tra loại sau mỗi lần đọc. Nếu giá trị là `None`, phương thức trả `None` trước khi định dạng, vì không thể in số thập phân từ dữ liệu thiếu. Sau đó đếm riêng kết quả có số và chưa có số; không coi phép đo thiếu là 0.

Quy trình:

1. Tạo lớp cha lưu số đo và cung cấp giao diện `read`.
2. Ghi đè `read()` cho nhiệt độ/độ ẩm theo đơn vị.
3. Duyệt 18 đối tượng, loại dòng thiếu và đếm kết quả.

No Note — Không chú thích

```
records = [("temp",24),("humidity",56),("temp",25),("humidity",58),("temp",None),("humidity",60),
("temp",27),("humidity",62),("temp",28),("humidity",64),("temp",29),("humidity",66),("temp",30),
("humidity",None),("temp",31),("humidity",70),("temp",32),("humidity",72)]
class Sensor:
    def __init__(self, value):
        self.value = value
    def read(self):
        raise NotImplementedError
class Temperature(Sensor):
    def read(self):
        if self.value is None:
            return None
        return f"{self.value:.1f} °C"
class Humidity(Sensor):
    def read(self):
        if self.value is None:
            return None
        return f"{self.value:.1f} %"
sensors = []
for kind, value in records:
    if kind == "temp":
        sensor = Temperature(value)
    else:
        sensor = Humidity(value)
    sensors.append(sensor)
results = []
valid = []
```

```
for sensor in sensors:
    text = sensor.read()
    results.append(text)
    if text is not None:
        valid.append(text)
if len(results) != 18:
    raise ValueError("Cần nhận đủ 18 bản ghi cảm biến.")
print("Cảm biến:", len(results))
print("Hợp lệ:", len(valid))
print("Thiếu:", len(results) - len(valid))
print("Mẫu đầu:", results[0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhận 18 mẫu nhiệt độ/độ ẩm bằng class đa hình, giữ None là dữ liệu thiếu.
# Dữ liệu: 18 bộ (loại temp/humidity, giá trị °C/%), None là mất số đo.
records = [("temp",24),("humidity",56),("temp",25),("humidity",58),("temp",None),("humidity",60),
("temp",27),("humidity",62),("temp",28),("humidity",64),("temp",29),("humidity",66),("temp",30),
("humidity",None),("temp",31),("humidity",70),("temp",32),("humidity",72)]
# Lớp cha chứa giá trị gốc và khai giao diện read().
class Sensor:
    # __init__ chạy khi tạo instance; self trỏ đối tượng hiện tại và các tham số sau self là dữ
liệu đầu vào.
    def __init__(self, value):
        # self.thuộc_tính lưu trạng thái riêng của object hiện tại.
        self.value = value
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def read(self):
        # Thông báo read của lớp cha chưa cài, buộc class con ghi đè.
        raise NotImplementedError
# Class đọc nhiệt độ theo đơn vị °C.
class Temperature(Sensor):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def read(self):
        # is None nhận dữ liệu thiếu, không biến thành 0.
        if self.value is None:
            # return đưa kết quả trở lại nơi gọi và kết thúc method.
            return None
        # f-string :.1f định dạng số với một chữ số thập phân.
        return f"{self.value:.1f} °C"
# Class đọc độ ẩm theo đơn vị %.
class Humidity(Sensor):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def read(self):
        # is None nhận dữ liệu thiếu, không biến thành 0.
        if self.value is None:
            # return đưa kết quả trở lại nơi gọi và kết thúc method.
            return None
        # f-string :.1f định dạng số với một chữ số thập phân.
        return f"{self.value:.1f} %"
sensors = []
for kind, value in records:
    if kind == "temp":
        sensor = Temperature(value)
    else:
        sensor = Humidity(value)
    # append() thêm instance của class cảm biến thích hợp.
    sensors.append(sensor)
results = []
valid = []
for sensor in sensors:
    # Đa hình gọi read() theo loại class thực tế.
    text = sensor.read()
    results.append(text)
    # Giữ những dòng read() cho chuỗi hợp lệ; loại None.
```

```
    if text is not None:
        valid.append(text)
# len() đếm số́ phần tư,̉ dùng kiểm đủ bản ghi hoặc báo số́ lượt.
if len(results) != 18:
    # raise ValueError báo đâù vào sai rôì dừng nhánh hiện tại.
    raise ValueError("Cần nhận đủ 18 bản ghi cảm biến.")
# len() đếm số́ phần tư,̉ dùng kiểm đủ bản ghi hoặc báo số́ lượt.
print("Cảm biến:", len(results))
# len() đếm số́ phần tư,̉ dùng kiểm đủ bản ghi hoặc báo số́ lượt.
print("Hợp lệ:", len(valid))
# Số́ thiếú bằng tổng 18 bản ghi trừ số́ hợp lệ.
print("Thiếú:", len(results) - len(valid))
print("Mẫũ đâù:", results[0])
```

Kết quả khi chạy:

```
Cảm biến: 18
Hợp lệ: 16
Thiếú: 2
Mẫũ đâù: 24.0 °C
```

Cùng lời gọi `read()` nhưng cách tạo chuỗi khác nhau tùy lớp con; dữ liệu mất tín hiệu phải được đếm riêng, không gán bằng 0 °C hoặc 0%.

Tình huống 5 — Cấp 5: Thiết kế hai loại giao dịch theo hợp đồng phí chung

Từ vựng trong mục: Abstract class — lớp trừu tượng; **Override** — ghi đè; **Dispatch** — phân phối phương thức.

Tình huống thực tế: Hệ thống thanh toán thử nghiệm nhận **20 giao dịch P01–P20**. Mỗi dòng gồm số tiền tính bằng **đồng** và loại xử lý `bank` (chuyển khoản) hoặc `card` (thẻ). Theo mức phí minh họa, chuyển khoản thu **2.500 đồng** mỗi giao dịch, còn thẻ thu **2% số tiền**. Giao dịch P07 chưa có số tiền (`None`), P16 có tiền âm nên cần sửa. Bộ phận vận hành cần tổng phí của **18 giao dịch hợp lệ** và danh sách hai mã lỗi; không thu tiền thật.

Vì sao chọn cách này? Hợp đồng chung ở đây là mọi phương thức thanh toán đều phải có `fee()`, được lớp `Payment` khai báo bằng ABC. Lớp `Bank` trả khoản phí cố định 2.500 đồng, lớp `Card` tính 2% trên số tiền giao dịch rồi làm tròn theo code. Phần kiểm tra đầu vào được đặt tại lớp cha `Payment`: chỉ nhận số tiền là số và không âm. Vì thế khi P07 là `None` hoặc P16 âm, chương trình bắt `ValueError` và đưa mã vào danh sách sửa **trước khi tính phí**, không biến lỗi thành 0 đồng. Chỉ đối tượng hợp lệ mới được thêm vào danh sách để cộng phí, nên không cần tự chọn công thức trong từng lần cộng.

Quy trình:

1. Tạo giao diện `Payment(ABC)` giữ số tiền hợp lệ và bắt buộc phương thức `fee()`.
2. Tạo `Bank` và `Card` ghi đè `fee()` theo chính sách giả lập.
3. Duyệt 20 giao dịch, ghi mã sai, cộng phí các đối tượng hợp lệ và đối chiếu số bản ghi.

No Note — Không chú thích

```
from abc import ABC, abstractmethod
amounts =
[100000,125000,150000,175000,200000,225000,None,100000,125000,150000,175000,200000,225000,250000,10
0000,-10000,150000,175000,200000,225000]
methods =
["bank","card","card","bank","card","card","bank","card","card","bank","card","card","bank","card",
```

```
"card","bank","card","card","bank","card"]
class Payment(ABC):
    def __init__(self, amount):
        if not isinstance(amount, (int, float)) or amount < 0:
            raise ValueError("Sô´tiêǹ không hợp lệ")
        self.amount = amount
    @abstractmethod
    def fee(self):
        pass
class Bank(Payment):
    def fee(self):
        return 2500
class Card(Payment):
    def fee(self):
        return round(self.amount * 0.02)
valid, invalid = [], []
for i, (amount, method) in enumerate(zip(amounts, methods), 1):
    try:
        if method == "bank":
            payment = Bank(amount)
        else:
            payment = Card(amount)
        valid.append(payment)
    except ValueError:
        invalid.append(f"P{i:02d}")
if len(amounts) != 20 or len(valid) + len(invalid) != 20:
    raise ValueError("Cân đôí chiếú đủ 20 giao dịch, kể̉ cả giao dịch sai.")
print("Giao dịch:", len(amounts))
print("Hợp lệ:", len(valid))
print("Cân sưả:", ", ".join(invalid))
total_fee = 0
for item in valid:
    total_fee += item.fee()
print("Phí tôủg (đôòng):", total_fee)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xử lý 20 thanh toán, chặn sô´tiêǹ sai và tính phí Bank/Card.
# Dữ liệu: 20 sô´tiêǹ đôòng ghép vị trí với 20 loại thanh toán bank/card; None, -10.000 là lôĩ.
# ABC và abstractmethod thiêt́ lập hợp đôòng cho các phương thức phí.
from abc import ABC, abstractmethod
amounts =
[100000,125000,150000,175000,200000,225000,None,100000,125000,150000,175000,200000,225000,250000,10
0000,-10000,150000,175000,200000,225000]
methods =
["bank","card","card","bank","card","card","bank","card","card","bank","card","card","bank","card",
"card","bank","card","card","bank","card"]
# Class cha trừu tượng kiêm̉ sô´tiêǹ hợp lệ.
class Payment(ABC):
    # __init__ chạy khi tạo instance; self trỏ đôí tượng hiện tại và các tham sô´sau self là dữ
liệu đâù vào.
    def __init__(self, amount):
        # isinstance() xác minh là số; not/or loại None, sô´âm và tránh so None với 0.
        if not isinstance(amount, (int, float)) or amount < 0:
            # raise ValueError báo đâù vào sai rôì dừng nhánh hiện tại.
            raise ValueError("Sô´tiêǹ không hợp lệ")
        # self.thuộc_tính lưu trạng thái riêng của object hiện tại.
        self.amount = amount
    # Decorator yêu câù Bank/Card phải có fee().
    @abstractmethod
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # pass là chỗ giữ lệnh rôỗng, chưa cung cấp cách xử lý.
        pass
# Bank có phí cô´định 2.500 đôòng.
```

```
class Bank(Payment):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # return đưa kết quả trở lại nơi gọi và kết thúc method.
        return 2500
# Card tính phí 2% số tiền giao dịch.
class Card(Payment):
    # Phương thức dùng self để truy cập đúng instance, return trả dữ liệu cho bên gọi.
    def fee(self):
        # 0.02 là 2%, round() làm tròn thành tiền đồng.
        return round(self.amount * 0.02)
valid, invalid = [], []
# zip() ghép tiền và phương thức theo vị trí; enumerate(...,1) đánh thứ tự P01..P20.
for i, (amount, method) in enumerate(zip(amounts, methods), 1):
    # try bao đoạn khởi tạo có thể báo lỗi để except xử lý riêng.
    try:
        if method == "bank":
            payment = Bank(amount)
        else:
            payment = Card(amount)
        valid.append(payment)
    # Bắt ValueError khi tiền sai để tiếp tục xét giao dịch khác.
    except ValueError:
        # append() lưu mã lỗi với f-string :02d.
        invalid.append(f"P{i:02d}")
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
if len(amounts) != 20 or len(valid) + len(invalid) != 20:
    # raise ValueError báo đầu vào sai rồi dừng nhánh hiện tại.
    raise ValueError("Cần đối chiếu đủ 20 giao dịch, kể cả giao dịch sai.")
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
print("Giao dịch:", len(amounts))
# len() đếm số phần tử, dùng kiểm đủ bản ghi hoặc báo số lượt.
print("Hợp lệ:", len(valid))
print("Cần sửa:", ", ".join(invalid))
total_fee = 0
for item in valid:
    # Gọi fee theo class Bank/Card rồi cộng dồn phí đồng.
    total_fee += item.fee()
print("Phí tổng (đồng):", total_fee)
```

Kết quả khi chạy:

```
Giao dịch: 20
Hợp lệ: 18
Cần sửa: P07, P16
Phí tổng (đồng): 56500
```

Vì mỗi giao dịch là một đối tượng thanh toán khác loại, chương trình gọi `fee()` thống nhất nhưng nhận kết quả theo đúng lớp con. Không sử dụng số âm và `None` như khoản thu bằng 0.

6. Sai lầm thường gặp

- **Kế thừa** không tự biến mọi phương thức thành đa hình; lớp con có thể dùng lại hoặc ghi đè tùy nhu cầu.
- Kế thừa `ABC` **chưa đủ** để buộc lớp con triển khai phương thức; cần thêm `@abstractmethod`.
- `super().__init__(...)` dùng logic khởi tạo của lớp cha; nếu bỏ khi cần, thuộc tính chung như `name` có thể chưa được tạo.
- Ghi đè phương thức phải dùng đúng tên và cách nhận tham số phù hợp; khác tên sẽ là phương thức mới.

• Ví dụ cảm biến in hai giá trị có **đơn vị khác nhau**: không được cộng trực tiếp 26,5 °C và 62%.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does class inheritance primarily enable? | Kế thừa lớp chủ yếu giúp gì?

A. Convert a class into a file | Chuyển lớp thành tệp

B. Reuse and extend behavior of a parent class | Dùng lại và mở rộng hành vi của lớp cha

C. Copy every object in memory | Sao chép mọi đối tượng trong bộ nhớ

D. Prevent all method calls | Ngăn mọi lời gọi phương thức

Đáp án: B

Giải thích: Kế thừa giúp lớp con dùng lại phương thức của lớp cha và bổ sung hành vi riêng. Ví dụ `Dog(Animal)` có thể kế thừa `eat()` rồi thêm `bark()`; kế thừa không sao chép mọi đối tượng đã tạo.

Q2. What does `class Dog(Animal):` express? | `class Dog(Animal):` thể hiện điều gì?

A. Dog inherits from Animal | Dog kế thừa Animal

B. Dog is a function inside Animal | Dog là hàm trong Animal

C. Animal inherits from Dog | Animal kế thừa Dog

D. Dog is an instance of Animal | Dog là đối tượng tạo từ Animal

Đáp án: A

Giải thích: Trong `class Dog(Animal):`, `Dog` là lớp con còn `Animal` là lớp cơ sở. Cú pháp khai báo quan hệ kế thừa, không tạo đối tượng `Dog` mới hay một hàm bên trong `Animal`.

Q3. Why call `super().__init__(name)` in a subclass? | Vì sao lớp con gọi `super().__init__(name)`?

A. To delete the parent class | Để xóa lớp cha

B. To replace every attribute with None | Để thay mọi thuộc tính bằng None

C. To make the child an abstract class | Để biến lớp con thành lớp trừu tượng

D. To initialize inherited parent state | Để khởi tạo dữ liệu chung từ lớp cha

Đáp án: D

Giải thích: `super().__init__(name)` chuyển việc khởi tạo cho lớp kế tiếp trong thứ tự phân giải phương thức, thường là lớp cha. Nếu hàm đó gán `self.name`, lớp con dùng được thuộc tính ấy mà không phải viết lặp phần khởi tạo.

Q4. Which statement best describes polymorphism? | Câu nào mô tả đúng đa hình?

A. A child class cannot override methods | Lớp con không thể ghi đè phương thức

B. All methods must have different names | Mọi phương thức phải có tên khác nhau

C. Different objects respond to the same method call | Các đối tượng khác nhau xử lý cùng một lời gọi phương thức

D. All objects must store identical values | Mọi đối tượng phải có giá trị giống nhau

Đáp án: C

Giải thích: Đa hình cho phép mã gọi cùng một thao tác trên nhiều đối tượng tương thích. Chẳng hạn `dog.sound()` và `cat.sound()` có thể cho âm thanh khác nhau dù đều gọi `sound()`.

Q5. What is method overriding? | Ghi đè phương thức là gì?

A. Removing every inherited attribute | Xóa mọi thuộc tính kế thừa

B. Calling the parent method unchanged | Gọi nguyên phương thức lớp cha

C. Defining a same-named method in a subclass | Định nghĩa lại phương thức cùng tên ở lớp con

D. Changing a local variable only | Chỉ thay đổi một biến cục bộ

Đáp án: C

Giải thích: Ghi đè xảy ra khi lớp con định nghĩa phương thức trùng tên với phương thức kế thừa. Gọi trên đối tượng lớp con thường chọn bản của lớp con, nhưng phương thức gốc của lớp cha vẫn tồn tại.

Q6. What does `@abstractmethod` mark? | `@abstractmethod` đánh dấu điều gì?

A. A property unique to integers | Thuộc tính riêng của số nguyên

B. A method that can never be called | Phương thức không bao giờ được gọi

C. A method that prints automatically | Phương thức tự động in

D. A method subclasses must implement to become concrete | Phương thức lớp con phải triển khai để có thể khởi tạo

Đáp án: D

Giải thích: Khi lớp dùng cơ chế ABC của Python, `@abstractmethod` đánh dấu yêu cầu mà lớp con cụ thể phải đáp ứng. Phương thức trừu tượng vẫn có thể có phần thân; hạn chế nằm ở việc tạo đối tượng từ lớp còn yêu cầu trừu tượng chưa hoàn thành.

Q7. Is inheriting from `ABC` alone enough to require an override? | Chỉ kế thừa `ABC` có buộc ghi đè phương thức không?

A. Yes, every method becomes abstract | Có, mọi phương thức đều trừu tượng

B. No, an abstract method must be declared | Không, cần khai báo phương thức trừu tượng

C. Yes, all subclass methods are deleted | Có, mọi phương thức lớp con bị xóa

D. No, ABC forbids inheritance | Không, ABC cấm kế thừa

Đáp án: B

Giải thích: Kế thừa `ABC` bật cơ chế lớp trừu tượng nhưng không tự khiến mọi phương thức thành trừu tượng. Khai báo `@abstractmethod` phía trên `def area(self):` mới đặt ra yêu cầu trừu tượng cụ thể.

Q8. What happens if an abstract method is left unimplemented? | Điều gì xảy ra nếu phương thức trừu tượng chưa được cài đặt?

A. The concrete instance cannot be created | Không thể tạo đối tượng từ lớp còn phương thức trừu tượng

B. The method automatically sends an email | Phương thức tự gửi email

C. The base class is erased | Lớp cha bị xóa

D. Python silently returns zero | Python âm thầm trả về 0

Đáp án: A

Giải thích: Nếu lớp con vẫn còn phương thức trừu tượng, Python từ chối tạo instance và báo `TypeError`. Cài đặt đầy đủ các phương thức bắt buộc mới giúp lớp có thể khởi tạo; chỉ kế thừa tên phương thức là chưa đủ.

Q9. What does `isinstance(obj, Parent)` check? | `isinstance(obj, Parent)` kiểm tra gì?

A. Whether obj is an instance of Parent or its subclasses | obj có thuộc Parent hoặc lớp con hay không

B. Whether obj is a string | obj có phải chuỗi không

C. Whether Parent has no methods | Parent có rỗng phương thức không

D. Whether obj has been printed | obj đã được in hay chưa

Đáp án: A

Giải thích: `isinstance(obj, Parent)` kiểm tra đối tượng thuộc `Parent` hoặc lớp con tương thích và trả Boolean. Với `Child(Parent)` thông thường, `isinstance(Child(), Parent)` là `True`, khác phép kiểm tra chính xác `type(obj) is Parent`.

Q10. If a child overrides `fee()`, which version runs on `Child().fee()`? | Nếu lớp con ghi đè `fee()`, `Child().fee()` chạy bản nào?

A. The parent's version in every case | Luôn bản của lớp cha

B. Neither version | Không chạy bản nào

C. The child's version | Bản của lớp con

D. Both versions automatically | Tự động chạy cả hai bản

Đáp án: C

Giải thích: Tra cứu phương thức trên instance ưu tiên bản ghi đè ở `Child` trước phương thức cùng tên được kế thừa. Vì vậy `Child().fee()` chạy bản lớp con; bản lớp cha chỉ chạy thêm nếu phương thức ghi đè chủ động gọi nó.

Q11. Can a child access an inherited attribute set in the parent initializer? | Lớp con có thể dùng thuộc tính khởi tạo ở lớp cha không?

A. Only if the attribute is named `class` | Chỉ khi tên thuộc tính là class

B. Yes, if the parent initialization is performed | Có, nếu đã thực hiện khởi tạo lớp cha

C. No, inherited fields are always hidden | Không, thuộc tính kế thừa luôn bị ẩn

D. Only for integer attributes | Chỉ với thuộc tính số nguyên

Đáp án: B

Giải thích: Lớp con đọc được thuộc tính instance đã được phần khởi tạo thích hợp tạo ra. Nếu `Parent.__init__` gán `self.name`, lớp con gọi `super().__init__(name)` sẽ dùng được `self.name`; bỏ bước cần thiết có thể khiến thuộc tính chưa tồn tại.

Q12. Must polymorphic objects share exactly the same class? | Đối tượng đa hình có bắt buộc cùng một class không?

A. Yes, they must be identical objects | Có, phải là cùng một đối tượng

B. Yes, inheritance is forbidden | Có, không được kế thừa

C. No, they can provide the same expected method | Không, có thể thuộc các lớp khác nhau nhưng cùng phương thức

D. No, methods must all be absent | Không, không được có phương thức

Đáp án: C

Giải thích: Đa hình dựa vào giao diện thao tác chung, không yêu cầu các đối tượng cùng lớp cụ thể. Hàm gọi `obj.sound()` có thể làm việc với `Dog` và `Cat` nếu cả hai cung cấp phương thức đó, dù cách triển khai khác nhau.

Q13. What does `self` refer to inside an instance method? | `self` trong phương thức đối tượng chỉ gì?

A. Every object in the program | Toàn bộ đối tượng của chương trình

B. The current object receiving the call | Đối tượng hiện tại đang nhận lời gọi

C. Only the parent class definition | Chỉ phần định nghĩa lớp cha

D. The name of the Python module | Tên module Python

Đáp án: B

Giải thích: Trong phương thức instance thông thường, `self` là đối tượng cụ thể đang nhận lời gọi. Vì vậy `self.name` của `dog1` có thể khác `self.name` của `dog2`; `self` không phải lớp cha hay module.

Q14. Does overriding a method erase it from the parent class? | Ghi đè phương thức có xóa bản lớp cha không?

A. Yes, it deletes all parent code | Có, xóa toàn bộ code lớp cha

B. Yes, it deletes the class name | Có, xóa tên lớp

C. Only when `print()` is used | Chỉ khi dùng print

D. No, the parent method still exists | Không, phương thức của lớp cha vẫn tồn tại

Đáp án: D

Giải thích: Ghi đè ở lớp con thay đổi phương thức được dùng bởi đối tượng lớp con chứ không xóa định nghĩa trên `Parent`. `Parent().fee()` vẫn dùng bản của cha, còn `Child().fee()` có thể dùng bản ghi đè.

Q15. How can an override call the parent's implementation? | Phương thức ghi đè gọi bản của lớp cha bằng gì?

A. `super().method()` | `super().method()`

B. `None.method()` | `None.method()`

C. `print(super)` only | Chỉ `print(super)`

D. `del Parent` | `del Parent`

Đáp án: A

Giải thích: Trong phương thức ghi đè, `super().method()` chuyển lời gọi tới bản kế tiếp trong thứ tự phân giải phương thức, thường là lớp cha. Nó không gọi tùy ý một lớp bất kỳ; với đa kế thừa, lớp kế tiếp có thể khác lớp cha trực tiếp.

Q16. Can an abstract class define ordinary implemented methods? | Lớp trừu tượng có thể có phương thức thông thường đã triển khai không?

A. No, it cannot define attributes | Không, không thể có thuộc tính

B. No, it must only contain pass | Không, chỉ được chứa pass

C. Only when it has no subclasses | Chỉ khi không có lớp con

D. Yes, it can provide reusable behavior too | Có, nó vẫn có thể cung cấp hành vi dùng chung

Đáp án: D

Giải thích: Lớp trừu tượng có thể vừa có hàm hỗ trợ đã cài đặt vừa có yêu cầu trừu tượng. Chẳng hạn lớp cung cấp `describe()` nhưng buộc lớp con triển khai `area()`; trừu tượng không có nghĩa mọi phương thức phải rỗng.

Q17. What is the relation between overriding and polymorphism? | Ghi đè liên quan gì đến đa hình?

A. They both mean deleting attributes | Cả hai đều là xóa thuộc tính

B. Overriding can provide distinct behavior for a common call | Ghi đè tạo hành vi khác nhau cho cùng lời gọi

C. They only apply to CSV files | Chúng chỉ áp dụng cho CSV

D. Overriding prevents polymorphism | Ghi đè ngăn đa hình

Đáp án: B

Giải thích: Ghi đè là một cách thực hiện đa hình: các lớp con cung cấp thân hàm khác nhau cho cùng tên phương thức. Mã có thể gọi `shape.area()` mà không cần tự chọn hàm riêng cho từng loại hình.

Q18. Which part of `class Child(Parent):` names the superclass? | Trong `class Child(Parent):`, đâu là tên lớp cha?

A. `class` | `class`

B. `Child` | `Child`

C. `Parent` | `Parent`

D. : | :

Đáp án: C

Giải thích: Trong dòng `class Child(Parent):`, tên bên trong ngoặc là lớp cơ sở `Parent`. Lớp mới đang được khai báo là `Child`; dấu hai chấm mở đầu phần thân lớp được thụt lề.

Q19. What is a good reason to create a shared base class? | Khi nào nên tạo lớp cha chung?

A. To avoid using any methods | Để không cần phương thức

B. Every object is unrelated | Mọi đối tượng hoàn toàn không liên quan

C. To make all values constant | Để mọi giá trị là hằng số

D. Several subclasses share data or behavior | Nhiều lớp con có chung dữ liệu hoặc hành vi

Đáp án: D

Giải thích: Lớp cha chung hữu ích khi nhiều lớp con có trạng thái hoặc hành vi giống nhau. Ví dụ `Car` và `Bike` có thể dùng chung giao diện `Vehicle` thay vì lặp định nghĩa cùng hợp đồng phương thức ở từng lớp.

Q20. Which error may follow if a child skips necessary parent initialization? | Nếu lớp con bỏ khởi tạo lớp cha cần thiết, điều gì có thể xảy ra?

A. An expected inherited instance attribute may be missing | Một thuộc tính đối tượng cần thiết có thể chưa được tạo

B. All integers turn into strings | Mọi số nguyên biến thành chuỗi

C. Python automatically runs CSV conversion | Python tự chuyển CSV

D. Every method prints twice | Mọi phương thức in hai lần

Đáp án: A

Giải thích: Nếu lớp con ghi đè `__init__` mà bỏ phần khởi tạo cần thiết của cha, các trường dự kiến có thể chưa được tạo. Đọc `self.name` sau đó có thể gây `AttributeError`; Python không tự gọi hàm khởi tạo cha đã bị ghi đè.

8. Nối sang bài tiếp theo

`Class/Object → Inheritance → Abstract API → Kiến trúc ứng dụng.`

PY-12 — File I/O, CSV và xử lý ngoại lệ

DRAFT · Tài liệu tự học · Các mã và dữ liệu mẫu cần được kiểm tra trước khi công bố chính thức

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: File I/O — đọc ghi file; **CSV** — dữ liệu phân cách.

`Dictionary → File/CSV → DataFrame → Dataset AI`

Học xong dùng để làm gì? Đọc CSV, ghi kết quả, xử lý lỗi chuyển kiểu và đóng file an toàn.

Kết nối kiến thức tiếp theo: PY-13 đưa bảng số liệu vào mảng NumPy để tính theo hàng/cột.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
File I/O	đọc ghi file	Lấy hoặc lưu dữ liệu trên đĩa
CSV	dữ liệu phân cách	Định dạng bảng dạng văn bản
Exception	ngoại lệ	Lỗi xuất hiện khi chạy chương trình
Context manager	quản lý ngữ cảnh	Tự đóng tài nguyên sau khối with

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`open(...,encoding='utf-8')`	Đọc/ghi Unicode và tự đóng bằng with
`csv.DictReader/DictWriter`	Đọc và ghi CSV theo tên cột thay vì vị trí
`try/except ValueError`	Bắt riêng lỗi dữ liệu nhập không hợp lệ
`Path.read_text()/write_text()`	Đọc/ghi file với đường dẫn kiểu Path

4. Khái niệm liên quan

Một chương trình thực tế cần **lưu và đọc lại dữ liệu**, hiểu cấu trúc bảng CSV và xử lý đúng lỗi đầu vào. Bốn khái niệm File I/O, CSV, Exception và Context manager liên kết thành một quy trình: nhận dữ liệu → đọc/ghi an toàn → xử lý trường hợp bất thường.

4.1. File I/O — Đọc và ghi dữ liệu

Từ vựng trong mục: File I/O — đọc/ghi tệp; **Path** — đường dẫn.

File I/O (input/output) là việc đưa dữ liệu **từ tệp vào chương trình** hoặc **từ chương trình ra tệp**. Ví dụ hệ thống cần lưu trạng thái `Started` để lần chạy sau có thể đọc lại. Trong đoạn minh họa, `TemporaryDirectory()` tạo thư mục tạm, `Path` xác định đường dẫn `log.txt`, `write_text()` ghi và `read_text()` đọc. Dữ liệu chỉ tồn tại trong thư mục tạm trong suốt khối `with`, tránh để lại file rác.

No Note — Không chú thích

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    file_path = Path(folder) / 'log.txt'
    file_path.write_text('Started', encoding='utf-8')
    print(file_path.read_text(encoding='utf-8'))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Viết và đọc lại văn bản UTF-8 trong thư mục tạm, tự dọn sau khi dùng.
# Dữ liệu: Path tạo đường dẫn hệ điều hành; TemporaryDirectory() tạo thư mục tạm và xóa khi ra khỏi
with.
# Path biểu diễn đường dẫn file dễ ghép bằng toán tử /.
from pathlib import Path
# TemporaryDirectory() tạo thư mục tạm, tự xóa sau khối with.
from tempfile import TemporaryDirectory

# with nhận đường dẫn tạm qua as folder và bảo đảm thư mục được dọn khi kết thúc.
with TemporaryDirectory() as folder:
    # Path(folder) / 'log.txt' ghép đường dẫn thư mục với tên file độc lập hệ điều hành.
    file_path = Path(folder) / 'log.txt'
    # write_text() tạo/ghi file; encoding='utf-8' giữ tiếng Việt và ký tự Unicode.
    file_path.write_text('Started', encoding='utf-8')
    # read_text(encoding='utf-8') đọc lại toàn bộ file thành chuỗi.
    # print() xuất nội dung vừa đọc để kiểm chứng ghi/đọc.
    print(file_path.read_text(encoding='utf-8'))
```

Kết quả khi chạy:

```
Started
```

Đầu ra đúng chuỗi đã ghi. `encoding='utf-8'` hỗ trợ văn bản tiếng Việt; tệp tạm được xóa sau khi ra khỏi khối.

4.2. CSV — Bảng dữ liệu phân cách bằng dấu phẩy

Từ vựng trong mục: CSV — dữ liệu phân cách; **Row/column** — hàng/cột.

CSV lưu dữ liệu dạng văn bản: **mỗi dòng là một bản ghi**, các giá trị trên một dòng được tách bằng dấu phẩy. Ví dụ bảng sản phẩm có **hai cột** `name` (tên hàng) và `price` (giá) cùng **hai hàng** Pen/10 và Book/35. `csv.DictReader` coi hàng đầu là tên cột và đọc mỗi hàng sau thành Dictionary, giúp tra `row['name']` thay vì nhớ vị trí cột. Giá đọc từ CSV ban đầu là **chuỗi**, muốn tính toán phải chuyển kiểu sau.

No Note — Không chú thích

```
import csv
from io import StringIO

csv_text = 'name,price\nPen,10\nBook,35\n'
```

```
rows = list(csv.DictReader(StringIO(csv_text)))
print(rows[1]['name'], rows[1]['price'])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc dữ liệu CSV ở bộ nhớ thành các hàng dictionary theo tên cột.
# Dữ liệu: Chuỗi csv_text gồm tiêu đề name,price và hai hàng Pen/Book; CSV là văn bản phân cột bằng dấu phẩy.
# csv là thư viện chuẩn đọc/ghi bảng phân cách dấu phẩy.
import csv
# StringIO tạo file văn bản giả trong RAM, không tạo file thật.
from io import StringIO

# Ký tự \n trong chuỗi là xuống dòng: dòng đầu đặt tên các cột.
csv_text = 'name,price\nPen,10\nBook,35\n'
# DictReader đọc mỗi hàng thành dictionary, khóa lấy từ tiêu đề cột; list() thu toàn bộ hàng.
rows = list(csv.DictReader(StringIO(csv_text)))
# [1] lấy hàng thứ hai (Book), ['name']/['price'] lấy các ô theo tên cột; giá vẫn là chuỗi.
print(rows[1]['name'], rows[1]['price'])
```

Kết quả khi chạy:

```
Book 35
```

Hàng dữ liệu thứ hai là Book, giá 35 đang ở dạng chuỗi; mã không tự đổi nó thành số nguyên.

4.3. Exception — Lỗi có thể xử lý

Từ vựng trong mục: Exception — ngoại lệ; **ValueError** — lỗi giá trị.

Khi chương trình nhận dữ liệu sai kiểu, không phải toàn bộ ứng dụng đều cần dừng. Chẳng hạn biểu mẫu yêu cầu số nguyên nhưng khách nhập `'abc'`; `int('abc')` sinh **`ValueError`**. Khối `try` chứa thao tác có thể lỗi, còn `except ValueError` chỉ bắt loại lỗi dự kiến và in thông báo. Không dùng `except:` quá rộng vì sẽ che mất những lỗi khác cần được sửa.

No Note — Không chú thích

```
try:
    number = int('abc')
    print(number)
except ValueError:
    print('Invalid integer')
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Bắt ValueError khi chuyển một chuỗi không phải số thành int.
# Dữ liệu: Chuỗi abc không thể chuyển sang số nguyên; cần xử lý lỗi có kiểm soát.
# try chứa lệnh có thể thất bại, luồng chuyển sang except nếu gặp lỗi.
try:
    # int() yêu cầu chuỗi có biểu diễn số nguyên; 'abc' gây ValueError.
    number = int('abc')
    print(number)
# Bắt riêng ValueError để hiện thông báo mà không làm chương trình dừng bất ngờ.
except ValueError:
    print('Invalid integer')
```

Kết quả khi chạy:

```
Invalid integer
```

Chỉ nhánh `except` chạy, vì chuỗi `abc` không thể chuyển thành số nguyên.

4.4. Context manager — Tự quản lý tài nguyên

Từ vựng trong mục: Context manager — quản lý ngữ cảnh; **with** — khối sử dụng tài nguyên.

Khi mở file hoặc bộ đệm, ta cần **đóng tài nguyên** sau khi dùng, kể cả khi có lỗi. Cấu trúc `with ... as ...:` nhận một tài nguyên rồi tự đóng khi thoát khối. Ví dụ `StringIO` là bộ đệm văn bản trong bộ nhớ, có thao tác đọc giống file, không cần tạo file thật. Ta có thể kiểm tra `stream.closed` sau khối `with` để thấy nó đã đóng.

No Note — Không chú thích

```
from io import StringIO

with StringIO('abc') as stream:
    print(stream.read())

print(stream.closed)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng with quản lý StringIO và kiểm tra luồng đã đóng.
# Dữ liệu: StringIO là luồng văn bản nằm trong RAM; with sẽ đóng luồng khi thoát.
# StringIO cung cấp luồng văn bản trong bộ nhớ.
from io import StringIO

# with gọi cơ chế đóng luồng khi ra khỏi khối, ngay cả khi có lỗi.
with StringIO('abc') as stream:
    # read() đọc hết nội dung còn lại từ con trỏ của luồng.
    print(stream.read())

# Thuộc tính closed là True sau khi with đóng tài nguyên.
print(stream.closed)
```

Kết quả khi chạy:

```
abc
True
```

Dòng đầu là nội dung đã đọc; `True` xác nhận bộ đệm được đóng sau khi ra khỏi `with`.

5. Năm tình huống File/CSV — từ đọc một bảng đến pipeline kiểm lỗi

Tình huống 1 — Cấp 1: Đọc CSV 10 mặt hàng bằng DictReader

Từ vựng trong mục: CSV — bảng ngăn cách; **DictReader** — đọc theo tên cột.

Tình huống thực tế: Quầy văn phòng phẩm nhận bảng hàng xuất từ phần mềm dưới dạng **CSV**, tức văn bản có các cột ngăn bằng dấu phẩy. Trong ví dụ có **10 sản phẩm P01–P10**, mỗi dòng chứa mã `sku`, tên

`name` và giá một món `price_vnd` tính bằng **đồng**. Nhân viên cần tìm tên và giá của P08 và xác nhận bảng có đúng 10 sản phẩm, không đếm dòng tiêu đề là hàng bán.

Vì sao chọn cách này? `csv.DictReader` đọc mỗi dòng CSV thành một **dictionary** lấy tên cột làm khóa, nên có thể hỏi `row['sku']` hoặc `row['name']` thay vì phải nhớ mã ở cột số mấy. Dòng `sku,name,price_vnd` được hiểu là **tiêu đề cột**, không phải một sản phẩm. `StringIO` tạo đối tượng giống file ngay trong bộ nhớ từ văn bản CSV mẫu, để ví dụ chạy mà không cần chuẩn bị tệp bên ngoài. Chương trình duyệt các dòng cho tới khi tìm thấy `P08`, đọc tên và giá của dòng đó, rồi đối chiếu số sản phẩm là 10.

Quy trình:

1. Tạo văn bản CSV gồm header và 10 hàng.
2. Đọc từng hàng qua `DictReader` thành list Dictionary.
3. Tra P08 và in số bản ghi.

No Note — Không chú thích

```
import csv
from io import StringIO
csv_text = """sku,name,price_vnd
P01,Bút,12000
P02,Vở,18000
P03,Thước,10000
P04,Tẩy,7000
P05,Giấy,25000
P06,Sổ,35000
P07,Bìa,9000
P08,Bút màu,42000
P09,Kẹp,8000
P10,Hộp,30000
"""
rows = list(csv.DictReader(StringIO(csv_text)))
match = None
for row in rows:
    if row["sku"] == "P08":
        match = row
        break
if match is None:
    raise ValueError("Không tìm thấy sản phẩm P08.")
if len(rows) != 10:
    raise ValueError("Cần đủ 10 sản phẩm CSV.")
print("Sản phẩm:", len(rows))
print("P08:", match["name"])
print("Giá P08 (đồng):", match["price_vnd"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc CSV 10 sản phẩm, tìm hàng có mã P08 và in đơn giá.
# Dữ liệu: CSV gồm 10 hàng sku/name/price_vnd, mỗi hàng là một sản phẩm và giá đồng dạng chuỗi.
# csv xử lý bảng text, không tự đổi giá trong cột thành số.
import csv
# StringIO biến chuỗi CSV thành luồng có thể đọc.
from io import StringIO
# Nội dung nhiều dòng gồm tiêu đề sku,name,price_vnd và 10 sản phẩm; giá đơn vị đồng dạng text.
csv_text = """sku,name,price_vnd
P01,Bút,12000
P02,Vở,18000
P03,Thước,10000
P04,Tẩy,7000
P05,Giấy,25000
P06,Sổ,35000
```

```
P07,Bìa,9000
P08,Bút màu,42000
P09,Kẹp,8000
P10,Hộp,30000
"""
# DictReader trả từng hàng dict dùng tên cột làm khóa; list() giữ 10 hàng.
rows = list(csv.DictReader(StringIO(csv_text)))
match = None
# Duyệt từng hàng CSV để tìm theo mã SKU.
for row in rows:
    # Tra khóa sku của hàng; == so chính xác với mã P08.
    if row["sku"] == "P08":
        match = row
        # break dừng tìm ngay khi gặp SKU mong muốn.
        break
# is None cho biết chưa tìm được mã, thay vì giả định tồn tại.
if match is None:
    # ValueError báo dữ liệu không hợp lệ và dừng khi kiểm tra thất bại.
    raise ValueError("Không tìm thấy sản phẩm P08.")
# len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
if len(rows) != 10:
    # ValueError báo dữ liệu không hợp lệ và dừng khi kiểm tra thất bại.
    raise ValueError("Cần đủ 10 sản phẩm CSV.")
# len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
print("Sản phẩm:", len(rows))
print("P08:", match["name"])
# Giá được đọc dưới dạng chuỗi từ CSV; in như số nhưng chưa dùng int().
print("Giá P08 (đồng):", match["price_vnd"])
```

Kết quả khi chạy:

```
Sản phẩm: 10
P08: Bút màu
Giá P08 (đồng): 42000
```

Giá trong CSV được đọc thành chuỗi; nếu cần tính tiền thì phải chuyển kiểu. Hàng `sku,name,price_vnd` là tiêu đề chứ không phải bản ghi thứ 11.

Tình huống 2 — Cấp 2: Ghi 12 bước nhật ký Unicode trong thư mục tạm

Từ vựng trong mục: File I/O — đọc/ghi file; **UTF-8** — mã hóa văn bản; **Context manager** — quản lý tài nguyên.

Tình huống thực tế: Giáo viên muốn kiểm tra việc lưu văn bản tiếng Việt có dấu. Một chương trình tạo **12 dòng nhật ký** mô phỏng các bước như `Khởi động`, `Đọc tập tin` và `Kết thúc`, mỗi dòng là một sự kiện. Bài cần ghi các dòng vào tệp tạm, đọc lại để xem dấu tiếng Việt có nguyên vẹn không, rồi cho biết số sự kiện và nội dung dòng đầu, dòng cuối mà không để tệp mẫu tồn tại lâu trên máy.

Vì sao chọn cách này? Mã hóa **UTF-8** là cách lưu chữ dưới dạng byte mà vẫn hỗ trợ ký tự tiếng Việt như ă, ê, ơ và dấu thanh. Khi `write_text(..., encoding='utf-8')` ghi nhật ký thì `read_text()` cũng phải dùng UTF-8 để đọc lại nhất quán, tránh chữ bị sai. `TemporaryDirectory()` tạo một **thư mục dùng thử**, rồi tự dọn khi thoát khối `with`, nên máy học viên không bị giữ lại tệp `log.txt`. Chương trình so từng dòng đọc lại với 12 sự kiện ban đầu: không chỉ đếm dòng mà còn xác nhận nội dung không bị thay đổi.

Quy trình:

1. Tạo 12 chuỗi sự kiện và ghép bằng ký tự xuống dòng.
2. Ghi nhật ký UTF-8 trong thư mục tạm, sau đó đọc lại.

3. So sánh nội dung và đếm dòng để bảo đảm không mất dữ liệu.

No Note — Không chú thích

```
from pathlib import Path
from tempfile import TemporaryDirectory
events = ["Khởi động","Đọc tập tin","Kết nối nội bộ","Tải dữ liệu","Kiểm kiểủ","Chuẩn hóa","Lưu
nháp","Chạy báo cáo","Xác nhận","Ghi nhận","Sao lưu","Kết thúc"]
with TemporaryDirectory() as temp:
    path = Path(temp) / "log.txt"
    path.write_text("\n".join(events) + "\n", encoding="utf-8")
    readback = path.read_text(encoding="utf-8").splitlines()
    if readback != events:
        raise ValueError("Nội dung ghi và đọc lại phải trùng nhau.")
    print("Sự kiện:", len(readback))
    print("Đâù:", readback[0])
    print("Cuôí:", readback[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghi rôì đọc lại 12 sự kiện log trong thư mục tạm bằng UTF-8.
# Dữ liệu: 12 chuôĩ sự kiện từ khởi động đến kết thúc, môĩ sự kiện ghi thành một dòng log.
# Path ghép đường dâñ tệp theo cách phù hợp hệ điêù hành.
from pathlib import Path
# Thư mục tạm tránh ghi tệp vào vị trí người dùng thật.
from tempfile import TemporaryDirectory
events = ["Khởi động","Đọc tập tin","Kết nối nội bộ","Tải dữ liệu","Kiểm kiểủ","Chuẩn hóa","Lưu
nháp","Chạy báo cáo","Xác nhận","Ghi nhận","Sao lưu","Kết thúc"]
# with tự dọn file và thư mục khi kết thúc khôí.
with TemporaryDirectory() as temp:
    # Ghép đường dâñ tạm với log.txt bằng / của Path.
    path = Path(temp) / "log.txt"
    # join(events) ghép các sự kiện bằng xuống dòng; + '\n' thêm kết thúc dòng cuôí.
    # write_text(encoding='utf-8') ghi chuôĩ Unicode vào file.
    path.write_text("\n".join(events) + "\n", encoding="utf-8")
    # read_text() trả chuôĩ; splitlines() tách thành list dòng, bỏ dâú kết thúc dòng.
    readback = path.read_text(encoding="utf-8").splitlines()
    # So từng dòng đã đọc với list gôć; != báo lôĩ khi dữ liệu bị sai.
    if readback != events:
        # ValueError báo dữ liệu không hợp lệ và dừng khi kiểm tra thất bại.
        raise ValueError("Nội dung ghi và đọc lại phải trùng nhau.")
    # len() đếm bản ghi để đôí chiếú sô´lượng kỳ vọng.
    print("Sự kiện:", len(readback))
    # [0] là sự kiện đâù, [-1] là sự kiện cuôí.
    print("Đâù:", readback[0])
    print("Cuôí:", readback[-1])
```

Kết quả khi chạy:

```
Sự kiện: 12
Đâù: Khởi động
Cuôí: Kết thúc
```

Chính xác 12 dòng đã được đọc lại. Kiểu dữ liệu là chuỗi, không phải thời gian thực từ hệ điều hành; đây là nhật ký giả lập.

Tình huống 3 — Cấp 3: Bắt ValueError khi nhập 15 số lượng dạng chữ

Từ vựng trong mục: try/except — thử/bắt lỗi; **ValueError** — lỗi giá trị; **Conversion** — đổi kiểu.

Tình huống thực tế: Nhân viên kho nhận **15 ô nhập số lượng điều chỉnh** cho các mã R01–R15; dữ liệu từ biểu mẫu đều là **chuỗi** như `"12"`, `"-3"` hoặc `"0"`. Đây là chênh lệch điều chỉnh nên có thể có giá trị âm hoặc bằng 0. Một số ô lại chứa `abc`, `12.5` hoặc để trống. Người quản lý muốn cộng các số nguyên đổi được, đếm lượt hợp lệ và liệt kê mã cần nhập lại mà không để một ô sai làm dừng toàn bộ báo cáo.

Vì sao chọn cách này? `int()` đổi một chuỗi chứa **số nguyên** thành số để tính toán, nên nhận `"12"`, `"0"` và `"-3"`. Nhưng `abc`, `12.5` và chuỗi rỗng không có dạng số nguyên hợp lệ nên `int()` báo `ValueError`. `try` cho phép thử chuyển kiểu trên từng dòng, còn `except ValueError` ghi lại mã lỗi rồi tiếp tục xử lý các dòng sau. Chỉ bắt loại lỗi **được dự đoán từ đầu vào** này, không dùng một lệnh bắt mọi lỗi khiến các vấn đề lập trình khác bị che giấu. Cuối cùng tổng điều chỉnh được tính từ các dòng đổi thành công.

Quy trình:

1. Duyệt 15 chuỗi, mỗi lượt thử chuyển bằng `int`.
2. Nếu lỗi, ghi mã dòng; nếu đúng, cộng vào tổng.
3. Xác nhận số hợp lệ cộng số lỗi bằng 15.

No Note — Không chú thích

```
raw_quantities = ["12","8","0","-3","15","abc","6","7","9","10","12.5","5","4","","18"]
converted, invalid = [], []
for i, raw in enumerate(raw_quantities, 1):
    try:
        converted.append(int(raw))
    except ValueError:
        invalid.append(f"R{i:02d}")
if len(raw_quantities) != 15 or len(converted) + len(invalid) != 15:
    raise ValueError("Cần kiểm tra đủ 15 dòng số́ lượng.")
print("Dòng nhận:", len(raw_quantities))
print("Hợp lệ:", len(converted))
print("Tổng điều chỉnh:", sum(converted))
print("Cần sửa:", ", ".join(invalid))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thử chuyển 15 chuỗi số́ lượng thành int và thu thập mã dòng bị lỗi.
# Dữ liệu: 15 lượng hàng dạng chuỗi; -3 là int hợp lệ nhưng âm, 12.5/abc/rỗng không chuyển int
được.
raw_quantities = ["12","8","0","-3","15","abc","6","7","9","10","12.5","5","4","","18"]
converted, invalid = [], []
# enumerate(...,1) gắn dòng R01..R15 với từng chuỗi đầu vào.
for i, raw in enumerate(raw_quantities, 1):
    # try bắt đầu thử chuyển chuỗi thành số́ nguyên.
    try:
        # int() chấp nhận số́ nguyên có dấu như -3, nhưng không nhận '12.5', 'abc' hay chuỗi rỗng.
        converted.append(int(raw))
    # Bắt đúng lỗi chuyển đổi mà không làm mất các dòng hợp lệ.
    except ValueError:
        # append() lưu mã dòng lỗi; f-string :02d luôn có hai chữ số́.
        invalid.append(f"R{i:02d}")
# len() đếm bản ghi để đối chiếu số́ lượng kỳ vọng.
if len(raw_quantities) != 15 or len(converted) + len(invalid) != 15:
    # ValueError báo dữ liệu không hợp lệ và dừng khi kiểm tra thất bại.
    raise ValueError("Cần kiểm tra đủ 15 dòng số́ lượng.")
# len() đếm bản ghi để đối chiếu số́ lượng kỳ vọng.
print("Dòng nhận:", len(raw_quantities))
# len() đếm bản ghi để đối chiếu số́ lượng kỳ vọng.
print("Hợp lệ:", len(converted))
# sum() cộng các giá trị chuyển được, bao gồm -3 vì tiêu chí chỉ kiểm cú pháp int.
print("Tổng điều chỉnh:", sum(converted))
```

```
# join() ghép các mã bằng dấu phẩy và khoảng trắng.
print("Cần sửa:", ", ".join(invalid))
```

Kết quả khi chạy:

```
Dòng nhận: 15
Hợp lệ: 12
Tổng điều chỉnh: 91
Cần sửa: R06, R11, R14
```

Giá trị -3 được chấp nhận vì bài toán cho phép điều chỉnh âm; với số lượng hàng tồn kho thực tế có thể cần quy tắc chặn âm riêng. Không nhầm `0` với dữ liệu thiếu.

Tình huống 4 — Cấp 4: Đọc 18 đơn giá, bỏ dòng lỗi và xuất CSV sạch

Từ vựng trong mục: DictWriter — ghi CSV theo cột; **Schema** — cấu trúc cột; **Validation** — kiểm tra.

Tình huống thực tế: Kế toán nhận **18 dòng giá sản phẩm P01–P18** trong bảng CSV, mỗi giá là **đồng/món** và đang được ghi dưới dạng chữ. Mã P06 có giá `bad`, mã P12 không có giá. Họ cần xuất một bảng CSV khác chỉ còn các dòng giá hợp lệ, báo số dòng bị loại và tổng các đơn giá được đọc thành công; không sửa bảng nguồn hoặc tự coi giá thiếu là 0 đồng.

Vì sao chọn cách này? `DictReader` đọc CSV theo tên cột để giữ đúng mã sản phẩm và chuỗi giá. `isdigit()` trong ví dụ dùng để kiểm tra giá có toàn **chữ số thập phân** hay không; chuỗi `bad` hoặc rỗng không đạt, nên đưa mã vào danh sách lỗi. Giá hợp lệ được đổi thành `int` rồi ghi sang file CSV mới bằng `DictWriter`, có dòng tiêu đề rõ ràng. `StringIO` giúp mô phỏng cả file đầu vào lẫn đầu ra trong bộ nhớ. Tính tổng từ các dòng đã được xác minh, không thay giá lỗi bằng 0 vì như vậy báo cáo sẽ thiếu doanh thu mà không cảnh báo.

Quy trình:

1. Đọc 18 hàng CSV và kiểm tra chuỗi giá chỉ gồm chữ số.
2. Dòng hợp lệ được chuyển sang `int` rồi đưa vào danh sách mới; dòng lỗi ghi mã.
3. Dùng `DictWriter` xuất dữ liệu sạch và đọc lại để đếm.

No Note — Không chú thích

```
import csv
from io import StringIO
raw_prices =
["10000","11000","12000","13000","14000","bad","16000","17000","18000","19000","20000","","22000","
23000","24000","25000","26000","27000"]
source = StringIO()
writer = csv.DictWriter(source, fieldnames=["sku", "price_vnd"], lineterminator="\n")
writer.writeheader()
for i, price in enumerate(raw_prices, 1):
    writer.writerow({"sku": f"P{i:02d}", "price_vnd": price})
source.seek(0)
cleaned, errors = [], []
for row in csv.DictReader(source):
    if row["price_vnd"].isdigit():
        cleaned.append({"sku": row["sku"], "price_vnd": int(row["price_vnd"])})
    else:
        errors.append(row["sku"])
output = StringIO()
writer = csv.DictWriter(output, fieldnames=["sku", "price_vnd"], lineterminator="\n")
writer.writeheader()
```

```
writer.writerows(cleaned)
output.seek(0)
written = list(csv.DictReader(output))
if len(raw_prices) != 18 or len(cleaned) + len(errors) != 18:
    raise ValueError("Cần đối chiếu đủ 18 dòng CSV.")
print("Dòng đầu vào:", len(raw_prices))
print("Dòng CSV sạch:", len(written))
print("Cần sửa:", ", ".join(errors))
total_price = 0
for row in cleaned:
    total_price += row["price_vnd"]
print("Tổng đơn giá (đồng):", total_price)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Làm sạch 18 giá từ CSV ở bộ nhớ, chỉ xuất dòng giá nguyên hợp lệ.
# Dữ liệu: 18 giá hàng chuỗi VND; bad và chuỗi rỗng là sai, các dòng còn lại thuộc một SKU.
# csv cung cấp DictWriter/DictReader để xuất và đọc bảng có tên cột.
import csv
# StringIO lưu CSV trong RAM, không tạo file trên ổ đĩa.
from io import StringIO
raw_prices =
["10000","11000","12000","13000","14000","bad","16000","17000","18000","19000","20000","","22000","
23000","24000","25000","26000","27000"]
# Tạo luồng văn bản rỗng để ghi CSV đầu vào.
source = StringIO()
# DictWriter ghi từng dict theo thứ tự fieldnames; lineterminator='\n' cố định dấu xuống dòng.
writer = csv.DictWriter(source, fieldnames=["sku", "price_vnd"], lineterminator="\n")
# Ghi dòng đầu tên cột sku,price_vnd.
writer.writeheader()
# Đánh số giá bắt đầu từ 1 để tạo SKU P01..P18.
for i, price in enumerate(raw_prices, 1):
    # writerow() ghi một dictionary thành một dòng CSV.
    writer.writerow({"sku": f"P{i:02d}", "price_vnd": price})
# seek(0) đưa con trỏ về đầu trước khi đọc lại, nếu không sẽ không đọc được hàng.
source.seek(0)
cleaned, errors = [], []
# DictReader duyệt các hàng theo tên khóa, cột price_vnd vẫn ở dạng chuỗi.
for row in csv.DictReader(source):
    # isdigit() chỉ nhận chuỗi toàn chữ số, loại bad/rỗng; int() đổi chuỗi hợp lệ thành số.
    if row["price_vnd"].isdigit():
        # append() giữ dict đã làm sạch, bảo toàn mã SKU.
        cleaned.append({"sku": row["sku"], "price_vnd": int(row["price_vnd"])})
    else:
        # append() lưu mã có giá lỗi để báo cáo.
        errors.append(row["sku"])
# Luồng CSV đầu ra riêng với nguồn, tránh chồng ghi dữ liệu thô.
output = StringIO()
# Ghi cột đúng thứ tự như CSV nguồn; dấu dòng \n.
writer = csv.DictWriter(output, fieldnames=["sku", "price_vnd"], lineterminator="\n")
# Ghi dòng đầu tên cột sku,price_vnd.
writer.writeheader()
# writerows() ghi toàn bộ hàng hợp lệ; khác writerow() ghi một hàng.
writer.writerows(cleaned)
# Đặt lại con trỏ để kiểm thử đọc bản CSV vừa xuất.
output.seek(0)
# Đọc lại file giả đầu ra để đối chiếu số dòng.
written = list(csv.DictReader(output))
# len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
if len(raw_prices) != 18 or len(cleaned) + len(errors) != 18:
    # ValueError báo dữ liệu không hợp lệ và dừng khi kiểm tra thất bại.
    raise ValueError("Cần đối chiếu đủ 18 dòng CSV.")
# len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
print("Dòng đầu vào:", len(raw_prices))
# len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
```

```
print("Dòng CSV sạch:", len(written))
# join() ghép các mã bằng dâú phâỷ và khoảng trăńg.
print("Cân sửả:", ", ".join(errors))
total_price = 0
for row in cleaned:
    # Giá trong cleaned đã là sô´int; += cộng đôàng của môĩ SKU.
    total_price += row["price_vnd"]
print("Tôỏng đơn giá (đôàng):", total_price)
```

Kết quả khi chạy:

```
Dòng đâù vào: 18
Dòng CSV sạch: 16
Cân sửả: P06, P12
Tôỏng đơn giá (đôàng): 297000
```

Đây là **tổng các đơn giá** của mẫu để kiểm tra chuyển kiểu, không phải doanh thu: doanh thu cần số lượng bán tương ứng.

Tình huống 5 — Cấp 5: Pipeline đọc 20 giao dịch, phát hiện giá sai và ID trùng

Từ vựng trong mục: CSV pipeline — quy trình CSV; **Exception** — ngoại lệ; **Atomic report** — báo cáo một lần ghi.

Tình huống thực tế: Phòng kế toán phải xử lý **20 dòng giao dịch CSV**. Mỗi dòng có mã đơn và tiền thanh toán tính bằng **đồng**: dòng thứ 8 ghi `bad`, dòng thứ 17 để trống số tiền, dòng thứ 13 dùng lại mã O05 của dòng thứ 5. Một dòng trùng mã có thể khiến kế toán cộng tiền hai lần. Nhân viên cần tạo file báo cáo chỉ có mã không trùng và tiền hợp lệ, đồng thời ghi rõ **số dòng và nguyên nhân lỗi**, rồi kiểm tra tổng tiền của những dòng đã xuất.

Vì sao chọn cách này? Pipeline xử lý CSV nghĩa là đi qua nhiều bước có thứ tự: ghi dữ liệu đầu vào → đọc từng dòng → kiểm tra → ghi file kết quả → đọc lại để đối chiếu. Chương trình dùng `seen` (set mã đã gặp) để từ chối mã xuất hiện lần thứ hai, trước khi cộng tiền; với dòng `bad` hoặc trống, `isdigit()` xác định tiền không hợp lệ. `enumerate(..., 1)` đánh số các **dòng dữ liệu** bắt đầu từ 1 để ghi lỗi `D08:giá`, `D13:trùng`... chính xác. `with open(..., encoding='utf-8', newline='')` giúp file được đóng sau khi dùng và đọc/ghi CSV đúng cách; `TemporaryDirectory` dọn các tệp mô phỏng. Đầu ra được đọc lại để xác nhận chỉ những dòng hợp lệ mới được cộng.

Quy trình:

1. Tạo file CSV với đủ 20 dòng và header trong thư mục tạm.
2. Đọc từng hàng: kiểm tra mã trùng, giá đúng kiểu; ghi lỗi theo vị trí.
3. Ghi file sạch bằng `DictWriter`, đọc lại kiểm tra số dòng và tổng doanh thu.

No Note — Không chú thích

```
import csv
from pathlib import Path
from tempfile import TemporaryDirectory
rows = [{"order_id":"O01","amount_vnd":"20000"},{"order_id":"O02","amount_vnd":"30000"},
{"order_id":"O03","amount_vnd":"40000"},{"order_id":"O04","amount_vnd":"50000"},
{"order_id":"O05","amount_vnd":"60000"},{"order_id":"O06","amount_vnd":"20000"},
{"order_id":"O07","amount_vnd":"30000"},{"order_id":"O08","amount_vnd":"bad"},
{"order_id":"O09","amount_vnd":"50000"},{"order_id":"O10","amount_vnd":"60000"},
{"order_id":"O11","amount_vnd":"20000"},{"order_id":"O12","amount_vnd":"30000"},
{"order_id":"O05","amount_vnd":"40000"},{"order_id":"O14","amount_vnd":"50000"},
```

```
{"order_id":"O15","amount_vnd":"60000"},{"order_id":"O16","amount_vnd":"20000"},
{"order_id":"O17","amount_vnd":""},{"order_id":"O18","amount_vnd":"40000"},
{"order_id":"O19","amount_vnd":"50000"},{"order_id":"O20","amount_vnd":"60000"}]
with TemporaryDirectory() as folder:
    incoming = Path(folder) / "incoming.csv"
    cleaned = Path(folder) / "validated.csv"
    with incoming.open("w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["order_id", "amount_vnd"])
        writer.writeheader()
        writer.writerows(rows)
    seen, valid, errors = set(), [], []
    with incoming.open("r", encoding="utf-8", newline="") as f:
        for line_number, row in enumerate(csv.DictReader(f), 1):
            key = row["order_id"]
            if key in seen:
                errors.append(f"D{line_number:02d}:trùng")
            elif not row["amount_vnd"].isdigit():
                errors.append(f"D{line_number:02d}:giá")
            else:
                seen.add(key)
                valid.append(row)
    with cleaned.open("w", encoding="utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["order_id", "amount_vnd"])
        writer.writeheader()
        writer.writerows(valid)
    with cleaned.open("r", encoding="utf-8", newline="") as f:
        exported = list(csv.DictReader(f))
    if len(rows) != 20 or len(exported) + len(errors) != 20:
        raise ValueError("Cần đối chiếu đủ 20 dòng hóa đơn.")
    print("Đầu vào:", len(rows))
    print("Đã xuất:", len(exported))
    print("Lỗi:", ", ".join(errors))
    total_amount = 0
    for row in exported:
        total_amount += int(row["amount_vnd"])
    print("Tổng tiền (đồng):", total_amount)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tra 20 hóa đơn CSV thật trong thư mục tạm, loại mã trùng và giá lỗi.
# Dữ liệu: 20 dict hóa đơn (order_id, amount_vnd chuỗi); có một mã lặp O05, giá bad và rỗng.
# csv hỗ trợ đọc/ghi file bảng có cột tên.
import csv
# Path cung cấp đường dẫn file để kết hợp với thư mục tạm.
from pathlib import Path
# TemporaryDirectory bảo đảm không để lại incoming.csv/validated.csv sau with.
from tempfile import TemporaryDirectory
rows = [{"order_id":"O01","amount_vnd":"20000"},{"order_id":"O02","amount_vnd":"30000"},
{"order_id":"O03","amount_vnd":"40000"},{"order_id":"O04","amount_vnd":"50000"},
{"order_id":"O05","amount_vnd":"60000"},{"order_id":"O06","amount_vnd":"20000"},
{"order_id":"O07","amount_vnd":"30000"},{"order_id":"O08","amount_vnd":"bad"},
{"order_id":"O09","amount_vnd":"50000"},{"order_id":"O10","amount_vnd":"60000"},
{"order_id":"O11","amount_vnd":"20000"},{"order_id":"O12","amount_vnd":"30000"},
{"order_id":"O05","amount_vnd":"40000"},{"order_id":"O14","amount_vnd":"50000"},
{"order_id":"O15","amount_vnd":"60000"},{"order_id":"O16","amount_vnd":"20000"},
{"order_id":"O17","amount_vnd":""},{"order_id":"O18","amount_vnd":"40000"},
{"order_id":"O19","amount_vnd":"50000"},{"order_id":"O20","amount_vnd":"60000"}]
# with tạo thư mục và tự xóa cả hai file khi ra khỏi khối.
with TemporaryDirectory() as folder:
    # Phép / ghép thư mục và tên file .csv.
    incoming = Path(folder) / "incoming.csv"
    # Phép / ghép thư mục và tên file .csv.
    cleaned = Path(folder) / "validated.csv"
    # open('w') ghi mới, encoding='utf-8' giữ Unicode; newline='' tránh dòng trống khi module csv
tự quản xuống dòng.
```

```
    with incoming.open("w", encoding="utf-8", newline="") as f:
        # DictWriter găń khóa dict với thứ tự cột order_id,amount_vnd.
        writer = csv.DictWriter(f, fieldnames=["order_id", "amount_vnd"])
        # Ghi tiêu đề cột trước dữ liệu.
        writer.writeheader()
        # writerows() ghi nhiều hàng dictionary qua CSV.
        writer.writerows(rows)
    # set() lưu mã đã xuất hiện, valid lưu dict hợp lệ, errors lưu nhãn lỗi.
    seen, valid, errors = set(), [], []
    # open('r') đọc lại UTF-8; newline='' để thư viện csv phân tích đúng kết thúc dòng.
    with incoming.open("r", encoding="utf-8", newline="") as f:
        # DictReader trả dict theo cột; enumerate(...,1) đánh số dòng dữ liệu bắt đầu 1 (không gồm
header).
        for line_number, row in enumerate(csv.DictReader(f), 1):
            key = row["order_id"]
            # Toán tử in kiểm tra mã đã gặp; phát hiện dòng trùng.
            if key in seen:
                # append() lưu thông tin dòng hỏng; :02d in số dòng thành hai chữ số.
                errors.append(f"D{line_number:02d}:trùng")
            # isdigit() yêu cầu tất cả ký tự là chữ số; loại bad và chuỗi rỗng, chưa nhận số âm/
thập phân.
            elif not row["amount_vnd"].isdigit():
                # append() lưu thông tin dòng hỏng; :02d in số dòng thành hai chữ số.
                errors.append(f"D{line_number:02d}:giá")
            else:
                # add() ghi mã của dòng hợp lệ vào set để lần sau bắt trùng.
                seen.add(key)
                # append() ghi hàng hợp lệ, giữ thứ tự gốc.
                valid.append(row)
    # Mở CSV đích để ghi các hàng đã kiểm tra.
    with cleaned.open("w", encoding="utf-8", newline="") as f:
        # DictWriter găń khóa dict với thứ tự cột order_id,amount_vnd.
        writer = csv.DictWriter(f, fieldnames=["order_id", "amount_vnd"])
        # Ghi tiêu đề cột trước dữ liệu.
        writer.writeheader()
        # writerows() ghi nhiều hàng dictionary qua CSV.
        writer.writerows(valid)
    # Đọc file kết quả để xác minh xuất thành công.
    with cleaned.open("r", encoding="utf-8", newline="") as f:
        # list() gom các hàng CSV đã xuất để so đếm.
        exported = list(csv.DictReader(f))
    # len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
    if len(rows) != 20 or len(exported) + len(errors) != 20:
        # ValueError báo dữ liệu không hợp lệ và dừng khi kiểm tra thất bại.
        raise ValueError("Cần đối chiếu đủ 20 dòng hóa đơn.")
    # len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
    print("Đầu vào:", len(rows))
    # len() đếm bản ghi để đối chiếu số lượng kỳ vọng.
    print("Đã xuất:", len(exported))
    # join() ghép các mã bằng dấu phẩy và khoảng trắng.
    print("Lỗi:", ", ".join(errors))
    total_amount = 0
    for row in exported:
        # Giá CSV vốn là chuỗi; int() chuyển sang số trước khi cộng tổng đồng.
        total_amount += int(row["amount_vnd"])
    print("Tổng tiền (đồng):", total_amount)
```

Kết quả khi chạy:

```
Đầu vào: 20
Đã xuất: 17
Lỗi: D08:giá, D13:trùng, D17:giá
Tổng tiền (đồng): 690000
```

Mỗi mã chỉ được ghi một lần trong báo cáo; hàng lỗi được lưu để xử lý lại, không biến mất âm thầm. Ví dụ dùng file tạm để an toàn và không thao tác trên hồ sơ thật.

6. Sai lầm thường gặp

- Khi đọc CSV bằng `DictReader`, dữ liệu **mặc định là chuỗi**; muốn cộng tiền cần chuyển kiểu số hợp lệ.
- Không nhầm **hàng tiêu đề CSV** với hàng dữ liệu đầu tiên. `rows[0]` là sản phẩm đầu trong ví dụ, không phải dòng `name,price`.
- Dùng `encoding='utf-8'` với dữ liệu tiếng Việt để tránh lỗi hiển thị do mã hóa.
- Nên dùng `with` khi làm việc với file để đóng tài nguyên ngay cả khi có ngoại lệ.
- Chỉ bắt lỗi dự kiến như `ValueError` hay `FileNotFoundError`; `except:` chung chung có thể che lỗi lập trình.
- `print(text)` sẽ tự thêm ký tự xuống dòng; nếu `text` đã kết thúc bằng `\n`, màn hình có thể thêm một dòng trống.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does File I/O mean? | File I/O có nghĩa gì?

A. Reading data from or writing data to files | Đọc dữ liệu từ hoặc ghi dữ liệu vào tệp

B. Drawing a plot | Vẽ một biểu đồ

C. Sorting a list in memory | Sắp xếp List trong bộ nhớ

D. Training a model | Huấn luyện mô hình

Đáp án: A

Giải thích: Nhập/xuất tệp là đọc dữ liệu từ tệp đã lưu hoặc ghi dữ liệu vào tệp. Ví dụ đọc CSV cung cấp các hàng đầu vào, còn lưu báo cáo tạo đầu ra tệp; chỉ sắp xếp trong bộ nhớ chưa phải thao tác I/O tệp.

Q2. What is the purpose of a file path? | Đường dẫn file dùng để làm gì?

A. Compute a matrix rank | Tính hạng ma trận

B. Sort rows automatically | Tự sắp xếp các hàng

C. Store a Boolean value | Lưu giá trị Boolean

D. Identify a file location | Xác định vị trí một tệp

Đáp án: D

Giải thích: Đường dẫn chỉ vị trí tệp hay thư mục, ví dụ `data/scores.csv`. Đường dẫn tương đối được hiểu theo thư mục làm việc, còn đường dẫn tuyệt đối xác định vị trí tính từ gốc hệ thống tệp.

Q3. What does `Path.read_text()` do? | `Path.read_text()` làm gì?

A. Convert the path into a function | Chuyển đường dẫn thành hàm

B. Sort CSV by price | Sắp CSV theo giá

C. Read text content from a file | Đọc nội dung văn bản từ tệp

D. Create a pandas index | Tạo chỉ số Pandas

Đáp án: C

Giải thích: `Path.read_text(encoding='utf-8')` đọc tệp văn bản và trả nội dung dưới dạng chuỗi Python. Nó không tự phân tách cột CSV hay biến đường dẫn thành hàm; giải mã có thể lỗi nếu chọn sai encoding.

Q4. What does `Path.write_text()` do? | `Path.write_text()` dùng để làm gì?

A. Read and sort all CSV rows | Đọc và sắp mọi hàng CSV

B. Only check whether a file exists | Chỉ kiểm tra tệp tồn tại

C. Count class methods | Đếm phương thức lớp

D. Write text to a file | Ghi văn bản vào tệp

Đáp án: D

Giải thích: `Path.write_text(text, encoding='utf-8')` ghi chuỗi vào tệp và thông thường thay nội dung trước đó. Muốn nối thêm, cần chế độ mở tệp phù hợp như `'a'`; `write_text` không phải thao tác chỉ kiểm tra tệp tồn tại.

Q5. Why specify `encoding='utf-8'` for text? | Vì sao dùng `encoding='utf-8'` với văn bản?

A. To encode and decode Unicode text consistently | Để mã hóa và giải mã chữ Unicode nhất quán

B. To guarantee a file is secret | Để bảo đảm tệp bí mật

C. To make all values integers | Để mọi giá trị thành số nguyên

D. To create three CSV columns | Để tạo ba cột CSV

Đáp án: A

Giải thích: UTF-8 quy định cách mã hóa ký tự Unicode thành byte khi ghi và giải mã byte thành ký tự khi đọc. Chỉ rõ UTF-8 nhất quán giúp giữ đúng chữ `Tiếng Việt`; nó không mã hóa bảo mật hoặc bảo đảm tệp bí mật.

Q6. How does a CSV file normally organize data? | CSV thông thường tổ chức dữ liệu ra sao?

A. Rows of text with separated fields | Các dòng văn bản chứa trường dữ liệu được phân cách

B. Binary images with pixels | Ảnh nhị phân gồm điểm ảnh

C. A database server connection | Kết nối máy chủ cơ sở dữ liệu

D. Python classes only | Chỉ chứa lớp Python

Đáp án: A

Giải thích: CSV lưu dữ liệu dạng bảng bằng các bản ghi có trường phân cách, thường có hàng đầu ghi tên cột. Mỗi hàng dữ liệu là một bản ghi, mỗi cột là một trường; văn bản trong dấu nháy có thể chứa dấu phẩy mà không tạo cột mới.

Q7. What does `csv.DictReader` use as dictionary keys? | `csv.DictReader` dùng gì làm khóa Dictionary?

A. CSV column names | Tên cột trong CSV

B. Names of Python packages | Tên gói Python

C. The current file size | Kích thước tệp hiện tại

D. Always numbers 0, 1, 2 | Luôn là số 0, 1, 2

Đáp án: A

Giải thích: Mặc định `csv.DictReader` lấy tên trường ở hàng CSV đầu tiên làm khóa từ điển. Với tiêu đề `name,age`, các hàng sau có khóa `'name'` và `'age'`; truyền `fieldnames=` tường minh sẽ thay cách lấy tiêu đề mặc định.

Q8. For a well-formed CSV row with all fields present, what type does `csv.DictReader` return for field values by default? | Với hàng CSV đủ các trường đúng cấu trúc, mặc định `csv.DictReader` trả giá trị ô thuộc kiểu gì?

A. String | Chuỗi

B. Float | Số thực

C. Integer | Số nguyên

D. Boolean | Giá trị logic

Đáp án: A

Giải thích: Với hàng CSV đúng cấu trúc, `csv.DictReader` trả dữ liệu ô dưới dạng chuỗi, nên ô `12` thành `'12'` chứ không phải số nguyên `12`. Cột bị thiếu có thể tạo `None` theo mặc định; muốn tính toán số vẫn phải đổi kiểu rõ ràng.

Q9. What does `csv.DictWriter.writeheader()` write? | `csv.DictWriter.writeheader()` ghi gì?

A. A Python class definition | Định nghĩa lớp Python

B. The column-name header row | Hàng tiêu đề chứa tên các cột

C. An exception message | Thông báo ngoại lệ

D. The final record twice | Nhân đôi bản ghi cuối

Đáp án: B

Giải thích: `csv.DictWriter.writeheader()` ghi một hàng chứa các tên trường đã cấu hình theo thứ tự. Nó không ghi toàn bộ bản ghi dữ liệu; các bản ghi cần dùng riêng `writerow()` hoặc `writerows()`.

Q10. Why prefer the CSV module over simply splitting every line by comma? | Vì sao nên dùng module CSV thay vì luôn tách chuỗi theo dấu phẩy?

A. It encrypts each file | Nó mã hóa bảo mật mọi tệp

B. It converts every field to a float | Nó đổi mọi trường thành số thực

C. It removes all empty rows automatically | Nó tự bỏ mọi dòng rỗng

D. It handles quoted fields and delimiters correctly | Nó xử lý đúng trường được đặt trong dấu nháy và dấu phân cách

Đáp án: D

Giải thích: Bộ đọc CSV hiểu trường có dấu nháy và quy tắc escape nên dấu phẩy trong ``"An, Binh"vẫn thuộc một trường. Tách máy móc bằngline.split(',')` sẽ chia sai giá trị có dấu nháy thành hai trường.

Q11. What is an exception in Python? | Ngoại lệ trong Python là gì?

A. A CSV row header | Hàng tiêu đề CSV

B. A guaranteed successful result | Một kết quả chắc chắn thành công

C. An error or abnormal condition raised during execution | Lỗi hoặc tình huống bất thường phát sinh khi chạy

D. A list slicing operator | Toán tử cắt List

Đáp án: C

Giải thích: Ngoại lệ báo tình huống bất thường khi chương trình chạy, như đổi kiểu thất bại hoặc thiếu tệp. Mã có thể bắt ngoại lệ dự kiến để xử lý; ngoại lệ không được bắt thường làm gián đoạn thao tác hiện tại.

Q12. What does `int('abc')` raise? | `int('abc')` phát sinh lỗi gì?

A. No error; returns zero | Không lỗi và trả về 0

B. `ValueError` | `ValueError`

C. `FileNotFoundError` | `FileNotFoundError`

D. `ZeroDivisionError` | `ZeroDivisionError`

Đáp án: B

Giải thích: `int('abc')` phát sinh `ValueError` vì các ký tự không biểu diễn số nguyên thập phân hợp lệ. Hàm `int` có tồn tại và nhận chuỗi, nên đây không phải lỗi thiếu tệp hoặc chia cho không.

Q13. Which exceptions does `except ValueError:` handle? | `except ValueError:` xử lý những ngoại lệ nào?

A. Every possible error | Mọi lỗi có thể xảy ra

B. `ValueError` exceptions, including invalid conversions | Ngoại lệ `ValueError`, kể cả lỗi đổi kiểu không hợp lệ

C. Only successful conversions | Chỉ khi chuyển kiểu thành công

D. All file permission failures | Mọi lỗi quyền truy cập tệp

Đáp án: B

Giải thích: `except ValueError:` bắt ngoại lệ kiểu `ValueError` và các kiểu con, trong đó có lỗi đổi số như `int('abc')`. Nó không bắt `FileNotFoundError` không liên quan hay mọi lỗi khác; vì vậy cần chọn nhánh bắt phù hợp với rủi ro của mã.

Q14. Which exception commonly indicates a missing file? | Ngoại lệ nào thường báo tệp không tồn tại?

A. `TypeError` always | Luôn `TypeError`

B. `ValueError` only | Chỉ `ValueError`

C. `KeyError` only | Chỉ `KeyError`

D. `FileNotFoundError` | `FileNotFoundError`

Đáp án: D

Giải thích: Mở một đường dẫn không tồn tại để đọc thường gây `FileNotFoundError`. Lỗi này khác `PermissionError`, khi tệp có thể tồn tại nhưng quyền truy cập bị từ chối; hai trường hợp cần cách xử lý khác nhau.

Q15. What does a `with` block provide for a file? | Khối `with` hỗ trợ việc gì khi dùng file?

A. Automatic numeric conversion | Tự chuyển toàn bộ số

B. Automatic sorting | Tự sắp xếp

C. Automatic resource cleanup on exit | Tự dọn dẹp và đóng tài nguyên khi ra khỏi khối

D. Automatic internet download | Tự tải từ Internet

Đáp án: C

Giải thích: Trình quản lý ngữ cảnh tệp như `with open(path) as f:` đóng tệp khi thoát khỏi khối, kể cả khi có ngoại lệ. Nó quản lý dọn dẹp tài nguyên, không tự sắp xếp dữ liệu hoặc đổi các ô thành số.

Q16. What is `StringIO` useful for? | `StringIO` hữu ích cho việc gì?

A. Reading only images | Chỉ đọc ảnh

B. Operating a camera sensor | Điều khiển cảm biến camera

C. File-like text operations in memory | Đọc ghi văn bản như file ngay trong bộ nhớ

D. Turning a set into a tuple | Biến Set thành Tuple

Đáp án: C

Giải thích: `io.StringIO` cung cấp luồng văn bản trong bộ nhớ với thao tác tương tự tệp như `read()` và `write()`. Nó hữu ích khi thử bộ đọc CSV với chuỗi mẫu nhỏ mà không phải tạo tệp vật lý.

Q17. What is the difference between `write` mode and append mode? | Chế độ ghi `w` và nối thêm `a` khác nhau thế nào?

A. Both only read existing content | Cả hai chỉ đọc nội dung

B. `w` replaces contents; `a` adds to the end | `w` thay nội dung; `a` nối thêm cuối tệp

C. `a` always deletes files | `a` luôn xóa tệp

D. `w` only opens network sockets | `w` chỉ mở socket mạng

Đáp án: B

Giải thích: Mở tệp có sẵn bằng chế độ `'w'` sẽ xóa nội dung cũ trước khi ghi, còn `'a'` nối dữ liệu mới vào cuối. Ví dụ tệp đang chứa `A`, ghi `B` bằng `'w'` được `B`, còn bằng `'a'` được `AB`.

Q18. Why is a bare `except:` often a poor choice? | Vì sao thường không nên dùng `except:` không chỉ rõ lỗi?

A. It prevents file creation | Nó ngăn tạo file

B. It never catches any error | Nó không bắt được lỗi nào

C. It can hide unexpected programming errors | Nó có thể che các lỗi lập trình không lường trước

D. It converts errors into CSV | Nó chuyển lỗi thành CSV

Đáp án: C

Giải thích: `except:` không ghi loại sẽ bắt phạm vi ngoại lệ quá rộng, kể cả các tín hiệu ngắt thường nên được chuyển tiếp. Nên dùng `except ValueError:` hoặc loại lỗi cụ thể để những bug không dự kiến vẫn lộ ra.

Q19. What does `print(text, end='')` change? | `print(text, end='')` thay đổi điều gì?

A. It closes every open file | Nó đóng mọi file đang mở

B. It removes existing newlines from text | Nó xóa mọi dấu xuống dòng có sẵn trong text

C. It converts text to numbers | Nó chuyển chữ thành số

D. It avoids adding the default newline | Nó không tự thêm dấu xuống dòng mặc định

Đáp án: D

Giải thích: Tham số `end` quy định chuỗi mà `print()` thêm vào sau các đối số; mặc định là xuống dòng `\n`. Với `end=''`, `print('A', end='')` không thêm dòng mới, nhưng cũng không xóa ký tự xuống dòng vốn có trong chuỗi.

Q20. Which distinction between `csv.reader` and `csv.DictReader` is correct? | `csv.reader` khác `csv.DictReader` ở điểm nào?

A. reader always validates all values | reader luôn kiểm tra mọi giá trị

B. reader yields field lists; DictReader uses column keys | reader trả danh sách trường; DictReader dùng khóa theo tên cột

C. Both always return float arrays | Cả hai luôn trả mảng số thực

D. DictReader writes files but cannot read | DictReader chỉ ghi mà không đọc

Đáp án: B

Giải thích: `csv.reader` trả mỗi bản ghi thành List chuỗi trường, ví dụ `['An', '9']`. `csv.DictReader` gắn trường với tên cột như `{'name': 'An', 'score': '9'}`, giúp tra theo tên thay vì chỉ số vị trí.

8. Nối sang bài tiếp theo

`Dictionary → File/CSV → DataFrame → Dataset AI.`

PY-13 — NumPy ndarray, shape, dtype và axis

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: ndarray — mảng NumPy; **Shape** — kích thước mảng.

`List → NumPy Array → Matrix → PCA`

Học xong dùng để làm gì? Biểu diễn dữ liệu số trong mảng có shape và dtype, tính tổng/trung bình theo chiều dữ liệu.

Kết nối kiến thức tiếp theo: PY-14 chọn hàng, cột hoặc phần tử thỏa ngưỡng trong mảng.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
ndarray	mảng NumPy	Một mảng đồng nhất về kiểu phần tử
Shape	kích thước mảng	Số phần tử trên từng chiều
dtype	kiểu phần tử	Kiểu dữ liệu của giá trị trong mảng
Axis	trục tính toán	Chiều dọc theo phép tổng hợp

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`np.array(data,dtype=float)`	Tạo mảng số thực, dtype=float cho phép lưu phần thập phân
`X.shape/X.ndim/X.size`	Xem hàng/cột, số chiều, số phần tử
`X.mean(axis=0)`	Tính trung bình của từng CỘT từ các hàng
`np.zeros()/np.ones()`	Tạo dữ liệu số được khởi tạo rõ hình dạng

4. Khái niệm liên quan

NumPy ndarray là công cụ lưu số liệu có cấu trúc. Một bảng có 3 học viên × 2 môn sẽ xuyên suốt phần khái niệm: từ tạo bảng, xem kích thước, hiểu kiểu phần tử cho tới tính toán theo cột. Nhờ giữ nguyên ý nghĩa hàng và cột, kết quả thống kê sẽ không bị hiểu nhầm.

4.1. ndarray — Mảng số cho nhiều học viên

Từ vựng trong mục: ndarray — mảng NumPy; **Element** — phần tử.

NumPy là thư viện hỗ trợ tính toán trên nhiều số cùng lúc. Kiểu dữ liệu chính là `ndarray` — một **mảng nhiều chiều**. Ví dụ trung tâm có ba học viên An, Bình, Lan (theo thứ tự ba hàng) và hai môn Toán, Python (theo thứ tự hai cột). Các điểm `[8,9]`, `[6,7]`, `[9,8]` nằm cùng một bảng. **Mỗi hàng** là một người, **mỗi cột** là một môn, **mỗi ô** là điểm của người đó ở môn đó trên thang 10.

No Note — Không chú thích

```
import numpy as np

scores = np.array([
    [8, 9],
    [6, 7],
    [9, 8],
])
print(scores)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo mảng NumPy 3 học viên × 2 môn để xem dữ liệu điểm dạng bảng.
# Dữ liệu: Mỗi hàng là một học viên, hai cột là điểm môn 1 và môn 2 (thang 10).
# NumPy là thư viện mảng số; as np tạo tên ngắn cho lời gọi np.array.
import numpy as np

# np.array() tạo ndarray 2 chiều từ list lồng; mỗi list con là một hàng học viên.
scores = np.array([
    [8, 9],
    [6, 7],
    [9, 8],
])
# print() hiển thị mảng hai chiều thành các hàng và cột.
print(scores)
```

Kết quả khi chạy:

```
[[8 9]
 [6 7]
 [9 8]]
```

Hàng đầu là điểm của An, hàng thứ hai của Bình, hàng thứ ba của Lan; hai số trên một hàng lần lượt thuộc hai môn.

4.2. Shape — Kích thước của mảng

Từ vựng trong mục: Shape — kích thước; **Row/column** — hàng/cột.

Thuộc tính `shape` cho biết **số phần tử ở từng chiều** của mảng. Với bảng điểm ba học viên, hai môn, `shape == (3, 2)`: con số 3 là **số hàng/học viên**, con số 2 là **số cột/môn**. Thuộc tính `size` là số ô dữ liệu (3 × 2 = 6), còn `ndim` là số chiều (2). Không nhầm mảng `(3,2)` với ba môn của hai học viên.

No Note — Không chú thích

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]])
print(scores.shape, scores.size)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt shape là kích thước theo chiều và size là tổng số ô của mảng.
# Dữ liệu: Mảng 3 hàng học viên × 2 cột điểm; có tổng cộng 6 giá trị.
# NumPy quản lý số liệu dạng mảng nhiều chiều.
import numpy as np

# np.array() ghép các hàng điểm thành ndarray.
scores = np.array([[8, 9], [6, 7], [9, 8]])
# shape là tuple (số hàng, số cột), ở đây (3,2); size là tổng ô 6.
print(scores.shape, scores.size)
```

Kết quả khi chạy:

```
(3, 2) 6
```

Bảng có 3 người × 2 môn = 6 giá trị điểm; đây là mảng hai chiều.

4.3. dtype — Kiểu của các phần tử

Từ vựng trong mục: dtype — kiểu dữ liệu phần tử; **Float** — số thực.

Một `ndarray` thông thường lưu các phần tử bằng một kiểu dữ liệu chung (`dtype`). Với điểm có phần thập phân như 8,5, ta dùng `dtype=float` khi tạo mảng để giữ phần lẻ và thuận tiện tính trung bình. NumPy trên môi trường thông dụng dùng `float64` cho lựa chọn `float`. Ngược lại, nếu ép điểm thập phân thành số nguyên, phần lẻ có thể mất.

No Note — Không chú thích

```
import numpy as np

scores = np.array([8.5, 7.0], dtype=float)
print(scores.dtype, scores.size)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng dtype=float tạo số thực và đọc dtype/size của mảng.
# Dữ liệu: 2 điểm số thực của hai học viên, dtype là kiểu các ô.
# np là tên tắt của NumPy.
import numpy as np

# dtype=float ép các ô sang số thực để có thể lưu số thập phân.
scores = np.array([8.5, 7.0], dtype=float)
# dtype là kiểu ô (float64 thông thường), size là số phần tử, không phải byte.
print(scores.dtype, scores.size)
```

Kết quả khi chạy:

```
float64 2
```

Mảng có hai điểm dạng số thực; `8.5` được giữ chứ không bị cắt thành `8`.

4.4. Axis — Tính theo hàng hoặc cột

Từ vựng trong mục: Axis — trục tính toán; **Mean** — trung bình.

Khi tổng hợp dữ liệu hai chiều, **`axis=0` gom các hàng để cho ra một kết quả trên mỗi cột**; **`axis=1` gom các cột để cho ra một kết quả trên mỗi hàng**. Trong bảng ba học viên × hai môn, `scores.mean(axis=0)` tính điểm trung bình **từng môn**: môn Toán `(8+6+9)/3 ≈ 7,67`, môn Python `(9+7+8)/3 = 8`. Nếu đổi sang `axis=1`, sẽ ra ba điểm trung bình tương ứng ba học viên.

No Note — Không chú thích

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]], dtype=float)
print(scores.mean(axis=0))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính điểm trung bình theo từng môn bằng mean(axis=0).
# Dữ liệu: Mảng 3 học viên × 2 môn; axis=0 gom theo cột môn, không phải theo từng học viên.
# NumPy cung cấp np.array() và hàm trung bình ndarray.mean().
import numpy as np

# dtype=float giữ các điểm là số thực; ba hàng học viên, hai cột môn.
scores = np.array([[8, 9], [6, 7], [9, 8]], dtype=float)
# axis=0 gộp theo hàng để tạo một trung bình cho MỖI CỘT môn; khác axis=1 tính cho từng hàng học viên.
print(scores.mean(axis=0))
```

Kết quả khi chạy:

```
[7.66666667 8.        ]
```

Mảng đầu ra có hai số vì bảng có hai **cột/môn**; kết quả lần lượt là trung bình Toán và Python.

5. Năm tình huống NumPy Array — tăng từ shape đến dữ liệu nhiều cột

Tình huống 1 — Cấp 1: Tạo bảng điểm 10 học viên có hai cột

Từ vựng trong mục: ndarray — mảng NumPy; **Shape** — hình dạng; **dtype** — kiểu phần tử.

Tình huống thực tế: Giáo viên lưu kết quả của **10 học viên H01–H10** trong bảng điểm có **hai môn Toán và Python**, mỗi môn trên thang **10 điểm**. Mỗi hàng là một học viên, hai ô trên hàng là điểm của đúng người đó. Giáo viên muốn kiểm tra bảng có bao nhiêu học viên, bao nhiêu môn, tổng số ô điểm và dữ liệu được lưu dưới dạng số nào; không được hiểu 20 ô là 20 học viên.

Vì sao chọn cách này? NumPy `ndarray` là một mảng dữ liệu giúp lưu và tính toán nhiều con số cùng cấu trúc. `np.array(rows, dtype=float)` chuyển danh sách điểm thành bảng số thực, nên có thể chứa cả điểm có phần lẻ khi cần. `shape` cho biết **hình dạng bảng**: `(10, 2)` nghĩa là 10 hàng học viên và 2 cột môn học. `size` đếm **tất cả ô**, ở đây là 20, còn `dtype.kind == 'f'` kiểm tra mảng dùng kiểu số thực. Phải phân biệt số hàng với số ô, vì một học viên có tới hai điểm.

Quy trình:

1. Ghi 10 dòng điểm theo đúng thứ tự hai môn.
2. Tạo ndarray kiểu số thực với NumPy.

3. Đọc shape, size và kiểu phần tử.

No Note — Không chú thích

```
import numpy as np
rows = [[8,9],[7,8],[9,10],[6,7],[8,7],[5,8],[9,9],[7,6],[10,8],[8,9]]
scores = np.array(rows, dtype=float)
if scores.shape != (10, 2):
    raise ValueError("Bảng phải có đúng 10 học viên và 2 môn.")
print("Sô´ học viên:", scores.shape[0])
print("Sô´ môn:", scores.shape[1])
print("Sô´ ô:", scores.size)
print("Kiểu sô´ thực:", scores.dtype.kind == "f")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm kích thước và kiểu dữ liệu bảng 10 học viên học hai môn.
# Dữ liệu: 10 hàng học viên, hai cột điểm các môn; kiểu float lưu cả giá trị thập phân.
# np.array từ thư viện NumPy lưu bảng sô´ theo hàng/cột.
import numpy as np
# 10 list con là 10 học viên, môĩ list có 2 điểm môn 1/2.
rows = [[8,9],[7,8],[9,10],[6,7],[8,7],[5,8],[9,9],[7,6],[10,8],[8,9]]
# Chuyển list lồng thành ndarray sô´ thực; dtype=float ép kiểu để tính tiếp.
scores = np.array(rows, dtype=float)
# shape (10,2) yêu câù đúng mười học viên và hai môn.
if scores.shape != (10, 2):
    # raise ValueError thông báo đâù vào sai và dừng để không xử lý kết quả thiếú hàng/cột.
    raise ValueError("Bảng phải có đúng 10 học viên và 2 môn.")
# shape[0] là sô´ hàng (học viên), shape[1] sô´ cột (môn).
print("Sô´ học viên:", scores.shape[0])
# int() đổi sô´ NumPy sang sô´ nguyên Python để in hoặc dùng làm chỉ sô´.
print("Sô´ môn:", scores.shape[1])
# size = 10×2 ô điểm toàn mảng.
print("Sô´ ô:", scores.size)
# dtype.kind == 'f' kiểm kiểu thuộc nhóm floating-point chứ không phải một tên float cụ thể.
print("Kiểu sô´ thực:", scores.dtype.kind == "f")
```

Kết quả khi chạy:

```
Sô´ học viên: 10
Sô´ môn: 2
Sô´ ô: 20
Kiểu sô´ thực: True
```

Một hàng đại diện cho một học viên; `dtype=float` cho phép mở rộng với điểm có phần thập phân dù dữ liệu ban đầu chỉ ghi số nguyên.

Tình huống 2 — Cấp 2: Trung bình doanh thu ba loại hàng trong 12 ngày

Từ vựng trong mục: Axis — trục tổng hợp; **Mean** — trung bình.

Tình huống thực tế: Siêu thị ghi doanh thu **12 ngày liên tiếp** theo ba nhóm **thực phẩm, sách, đồ dùng**. Một hàng tương ứng một ngày, ba cột là doanh thu từng nhóm, đơn vị **nghìn đồng/ngày**. Quản lý muốn biết trung bình của **mỗi nhóm qua cả 12 ngày** để so sánh kết quả bán, chứ không hỏi trung bình ba nhóm trong từng ngày.

Vì sao chọn cách này? Trục (`axis`) cho biết NumPy sẽ gộp dữ liệu theo hướng nào. Bảng có 12 hàng ngày và 3 cột mặt hàng; `mean(axis=0)` lấy trung bình **dọc xuống mỗi cột**, vì vậy trả ba số, mỗi số thuộc một

nhóm hàng. Nếu dùng `axis=1`, NumPy lại lấy trung bình **ngang trên mỗi hàng**, trả 12 số, mỗi số thuộc một ngày; đó không phải điều quản lý yêu cầu. Mỗi số trung bình vẫn có đơn vị nghìn đồng/ngày, nên chỉ được so các cột có cùng cách đo.

Quy trình:

1. Tạo bảng 12 hàng × 3 cột đúng đơn vị.
2. Chọn `axis=0` để tổng hợp theo loại hàng.
3. In ba giá trị theo nhãn đã định và xác nhận shape đầu ra.

No Note — Không chú thích

```
import numpy as np
sales = np.array([[10,20,30],[11,21,31],[12,22,32],[13,23,33],[14,24,34],[15,25,35],[16,26,36],
[17,27,37],[18,28,38],[19,29,39],[20,30,40],[21,31,41]], dtype=float)
average = sales.mean(axis=0)
if sales.shape != (12, 3) or average.shape != (3,):
    raise ValueError("Bảng bán hàng phải có 12 ngày, 3 nhóm.")
print("Ngày:", sales.shape[0])
print("Thực phẩm:", f"{average[0]:.1f}")
print("Sách:", f"{average[1]:.1f}")
print("Đồ dùng:", f"{average[2]:.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính doanh thu trung bình mỗi nhóm theo 12 ngày bán hàng.
# Dữ liệu: 12 hàng là 12 ngày, ba cột là doanh thu Thực phẩm/Sách/Đồ dùng (đơn vị minh họa).
# NumPy lưu và tính trung bình bảng doanh thu sô.
import numpy as np
# 12 hàng ngày, 3 cột nhóm hàng; dtype=float cho phép trung bình có phần lẻ.
sales = np.array([[10,20,30],[11,21,31],[12,22,32],[13,23,33],[14,24,34],[15,25,35],[16,26,36],
[17,27,37],[18,28,38],[19,29,39],[20,30,40],[21,31,41]], dtype=float)
# mean(axis=0) tính trung bình theo cột, kết quả gồm 3 giá trị theo thứ tự các nhóm hàng.
average = sales.mean(axis=0)
# shape (3,) là vector 1 chiều gồm ba trung bình, dấu phẩy không tạo cột thứ hai.
if sales.shape != (12, 3) or average.shape != (3,):
    # raise ValueError thông báo đầu vào sai và dừng để không xử lý kết quả thiếu hàng/cột.
    raise ValueError("Bảng bán hàng phải có 12 ngày, 3 nhóm.")
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
print("Ngày:", sales.shape[0])
# [0] lấy trung bình nhóm Thực phẩm, [1] Sách, [2] Đồ dùng; :.1f in một số lẻ.
print("Thực phẩm:", f"{average[0]:.1f}")
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
print("Sách:", f"{average[1]:.1f}")
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
print("Đồ dùng:", f"{average[2]:.1f}")
```

Kết quả khi chạy:

```
Ngày: 12
Thực phẩm: 15.5
Sách: 25.5
Đồ dùng: 35.5
```

Mỗi kết quả là mức trung bình ngày của **một cột** (nghìn đồng/ngày), không phải tổng doanh thu mười hai ngày.

Tình huống 3 — Cấp 3: Tổng doanh thu hai ca cho 15 cửa hàng

Từ vựng trong mục: Axis=1 — tổng theo hàng; **argmax** — vị trí lớn nhất; **ndim** — số chiều.

Tình huống thực tế: Chuỗi cửa hàng quản lý doanh thu của **15 chi nhánh S01–S15**. Mỗi chi nhánh có hai giá trị: doanh thu **ca sáng** và **ca chiều**, đều tính bằng **nghìn đồng**. Giám sát muốn cộng hai ca của từng chi nhánh và xác định mã chi nhánh có tổng doanh thu lớn nhất, không cộng lẫn doanh thu của các chi nhánh khác nhau.

Vì sao chọn cách này? Trong mảng 15 hàng × 2 cột, **mỗi hàng là một cửa hàng**, hai cột là hai ca. `sum(axis=1)` cộng **ngang hai cột của từng hàng**, tạo đúng 15 tổng doanh thu. `argmax()` tìm **vị trí** của tổng lớn nhất, không trả sẵn mã chi nhánh; Python đánh số vị trí từ 0 nên phải cộng 1 rồi định dạng thành S01, S02... Nếu dùng `axis=0`, kết quả chỉ có hai tổng cho ca sáng và chiều trên tất cả cửa hàng, không thể chỉ ra cửa hàng dẫn đầu.

Quy trình:

1. Tạo ndarray `(15,2)`; hàng là cửa hàng, cột là ca.
2. Gọi `sum(axis=1)` để ra 15 tổng riêng.
3. Dùng `argmax()` và chuyển chỉ số bắt đầu từ 0 thành mã cửa hàng.

No Note — Không chú thích

```
import numpy as np
sales = np.array([[50,30],[52,31],[54,32],[56,33],[58,34],[60,35],[62,36],[64,37],[66,38],[68,39],
[70,40],[72,41],[74,42],[76,43],[78,44]], dtype=int)
per_store = sales.sum(axis=1)
peak = int(per_store.argmax())
if sales.shape != (15, 2):
    raise ValueError("Bảng phải có 15 cửa hàng và 2 nhóm doanh thu.")
print("Cửa hàng:", sales.shape[0])
print("Tổng S01 (nghìn đồng):", int(per_store[0]))
print("Cửa hàng cao nhất:", f"S{peak+1:02d}")
print("Doanh thu cao nhất:", int(per_store[peak]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm cửa hàng có tổng doanh thu cao nhất trong 15 cửa hàng.
# Dữ liệu: 15 hàng cửa hàng S01..S15, hai cột doanh thu nhóm 1 và nhóm 2 (nghìn đồng).
# NumPy cung cấp sum(axis) và argmax() cho bảng dữ liệu.
import numpy as np
# 15 hàng cửa hàng × 2 cột nhóm doanh thu; dtype=int lưu số nguyên nghìn đồng.
sales = np.array([[50,30],[52,31],[54,32],[56,33],[58,34],[60,35],[62,36],[64,37],[66,38],[68,39],
[70,40],[72,41],[74,42],[76,43],[78,44]], dtype=int)
# axis=1 gộp các CỘT trong từng hàng để tính tổng mỗi cửa hàng; khác axis=0 cộng mỗi nhóm trên mọi
cửa hàng.
per_store = sales.sum(axis=1)
# argmax() trả CHỈ SỐ phần tử tổng lớn nhất (bắt đầu 0), không phải giá trị doanh thu.
# int() đổi chỉ số NumPy thành số nguyên Python để dùng làm chỉ số.
peak = int(per_store.argmax())
if sales.shape != (15, 2):
    # raise ValueError thông báo đầu vào sai và dừng để không xử lý kết quả thiếu hàng/cột.
    raise ValueError("Bảng phải có 15 cửa hàng và 2 nhóm doanh thu.")
# shape[0] đếm 15 hàng cửa hàng.
print("Cửa hàng:", sales.shape[0])
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
print("Tổng S01 (nghìn đồng):", int(per_store[0]))
# Vì chỉ số bắt đầu 0 nên peak+1 là số cửa hàng; :02d thêm 0 trước mã nhỏ hơn 10.
print("Cửa hàng cao nhất:", f"S{peak+1:02d}")
```

```
# Lấy doanh thu lớn nhất tại vị trí tìm được; int() đưa về số nguyên để in.
print("Doanh thu cao nhất:", int(per_store[peak]))
```

Kết quả khi chạy:

```
Cửa hàng: 15
Tổng S01 (nghìn đồng): 80
Cửa hàng cao nhất: S15
Doanh thu cao nhất: 122
```

Mảng tổng có 15 phần tử, bằng số cửa hàng; giá trị trong mỗi phần tử là tổng **hai ca cùng một cửa hàng**.

Tình huống 4 — Cấp 4: Kiểm tra shape/dtype của 18 số đo có dữ liệu thiếu

Từ vựng trong mục: NaN — giá trị số thiếu; **np.isnan** — kiểm tra NaN; **np.nanmean** — trung bình bỏ qua NaN.

Tình huống thực tế: Trạm đo nhận **18 lượt đo**, mỗi lượt có nhiệt độ tính bằng **°C** và độ ẩm tính bằng **%**. Ở lượt thứ 6 và 15, thiết bị vẫn gửi nhiệt độ nhưng không gửi độ ẩm, nên cột độ ẩm ghi `None`. Nhân viên muốn biết có bao nhiêu lần thiếu độ ẩm, nhiệt độ có còn đủ ở cả 18 lượt không và độ ẩm trung bình chỉ tính từ các lượt có dữ liệu.

Vì sao chọn cách này? `None` là dấu hiệu **không có số đo**, khác với độ ẩm bằng 0%. Khi làm bảng số thực NumPy, chương trình chuyển chỗ thiếu sang `np.nan` (**giá trị đánh dấu số bị thiếu**) để vẫn giữ đúng 18 hàng và hai cột. `np.isnan()` tìm vị trí bị thiếu trong riêng cột độ ẩm, còn `np.isfinite()` đếm các số nhiệt độ thực sự có. `np.nanmean()` tính trung bình nhưng **bỏ qua NaN**, vì thay NaN bằng 0 hoặc bỏ cả hàng sẽ làm sai số đo còn hợp lệ ở cột kia.

Quy trình:

1. Tạo 18 cặp đo với `np.nan` ở đúng ô độ ẩm thiếu.
2. Kiểm tra shape `(18,2)` và mặt nạ NaN trên cột thứ hai.
3. Tính trung bình chỉ các giá trị độ ẩm không phải NaN.

No Note — Không chú thích

```
import numpy as np
raw = [[24,50],[24.25,51],[24.5,52],[24.75,53],[25,54],[25.25,None],[25.5,56],[25.75,57],[26,58],
[26.25,59],[26.5,60],[26.75,61],[27,62],[27.25,63],[27.5,None],[27.75,65],[28,66],[28.25,67]]
rows = []
for temperature, humidity in raw:
    if humidity is None:
        cleaned_humidity = np.nan
    else:
        cleaned_humidity = humidity
    rows.append((temperature, cleaned_humidity))
measurements = np.array(rows, dtype=float)
missing = np.isnan(measurements[:, 1])
if measurements.shape != (18, 2):
    raise ValueError("Cần đủ 18 lượt đo, mỗi lượt hai giá trị.")
print("Lượt đo:", measurements.shape[0])
print("Thiếu độ ẩm:", int(missing.sum()))
print("Nhiệt độ vẫn đủ:", int(np.isfinite(measurements[:, 0]).sum()))
print(f"Độ ẩm trung bình: {np.nanmean(measurements[:, 1]):.2f} %")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuyển hai độ ẩm thiếu thành np.nan, vẫn tính trung bình hợp lệ từ 18 lượt.
# Dữ liệu: 18 hàng lần đo; cột 0 nhiệt độ °C, cột 1 độ ẩm %; None xuất hiện ở độ ẩm.
# NumPy có NaN và hàm thống kê bỏ giá trị thiếu.
import numpy as np
# 18 hàng lần đo, cột nhiệt độ °C và độ ẩm %, hai None ở độ ẩm.
raw = [[24,50],[24.25,51],[24.5,52],[24.75,53],[25,54],[25.25,None],[25.5,56],[25.75,57],[26,58],
[26.25,59],[26.5,60],[26.75,61],[27,62],[27.25,63],[27.5,None],[27.75,65],[28,66],[28.25,67]]
rows = []
for temperature, humidity in raw:
    # is None xác định độ ẩm thiếu, không được tùy tiện đổi thành 0%.
    if humidity is None:
        # np.nan là marker số thực 'Not a Number' biểu thị thiếu, cho phép ndarray có dtype=float.
        cleaned_humidity = np.nan
    else:
        cleaned_humidity = humidity
    # append() lưu mỗi cặp nhiệt độ và độ ẩm đã làm sạch theo hàng.
    rows.append((temperature, cleaned_humidity))
# Ép mảng (18,2) sang float để chứa np.nan.
measurements = np.array(rows, dtype=float)
# [:,1] lấy cột độ ẩm của MỌI hàng; np.isnan() tạo mask True ở các ô NaN.
missing = np.isnan(measurements[:, 1])
# shape (18,2) kiểm đủ hàng và cột; shape[0] là 18 lần đo.
if measurements.shape != (18, 2):
    # raise ValueError thông báo đầu vào sai và dừng để không xử lý kết quả thiếu hàng/cột.
    raise ValueError("Cần đủ 18 lượt đo, mỗi lượt hai giá trị.")
# shape (18,2) kiểm đủ hàng và cột; shape[0] là 18 lần đo.
print("Lượt đo:", measurements.shape[0])
# sum() trên mask Boolean đếm số True, tức số lượt thiếu độ ẩm.
print("Thiếu độ ẩm:", int(missing.sum()))
# [:,0] lấy cột nhiệt độ; isfinite() nhận số hữu hạn, loại NaN/±inf; sum() đếm lượt hợp lệ.
print("Nhiệt độ vẫn đủ:", int(np.isfinite(measurements[:, 0]).sum()))
# nanmean() lấy trung bình độ ẩm trên các giá trị KHÔNG NaN, không để hai lượt thiếu kéo kết quả về
0.
print(f"Độ ẩm trung bình: {np.nanmean(measurements[:, 1]):.2f} %")
```

Kết quả khi chạy:

```
Lượt đo: 18
Thiếu độ ẩm: 2
Nhiệt độ vẫn đủ: 18
Độ ẩm trung bình: 58.38 %
```

Ở đây `np.nan` được dùng để **biểu diễn số đo thiếu**, không phải độ ẩm bằng 0. Trường hợp cột toàn NaN cần quy tắc riêng trước khi tính trung bình.

Tình huống 5 — Cấp 5: Chuẩn hóa báo cáo 20 ngày: số đo, trục và thiếu dữ liệu

Từ vựng trong mục: Shape validation — kiểm hình dạng; **dtype** — kiểu mảng; **Axis=0** — tổng hợp từng cột.

Tình huống thực tế: Nhóm nghiên cứu có **20 ngày dữ liệu mô phỏng**. Mỗi ngày ghi số giờ ngủ (**giờ**), thời gian vận động (**phút**) và nhịp tim lúc nghỉ (**nhịp/phút**). Hai ngày không có số đo nhịp tim (`None`), nhưng hai chỉ số còn lại vẫn đầy đủ. Nhóm muốn kiểm tra bảng gồm 20 hàng và 3 cột, thống kê trung bình riêng cho từng cột và báo số ngày thiếu nhịp tim, chưa dùng bảng để chẩn đoán sức khỏe.

Vì sao chọn cách này? Ba đại lượng **giờ ngủ**, **phút vận động**, **nhịp tim/phút** đo những thứ khác nhau, vì vậy mỗi thứ phải ở một cột riêng với đơn vị riêng. Mảng `(20, 3)` giữ 20 ngày và ba chỉ số, `dtype=float`

cho phép lưu số thập phân và `np.nan` cho những ngày chưa đo nhịp tim. `np.nanmean(data, axis=0)` tính **ba trung bình riêng theo cột**, bỏ qua các ô NaN của cột tương ứng nhưng không xóa thông tin còn có ở hai cột khác. Trước khi dùng dữ liệu cho bước học sau, cần biết hai ô nhịp tim đang thiếu; kết quả là thống kê minh họa, không phải nhận xét y khoa.

Quy trình:

1. Lưu 20 bộ ba giá trị, chuyển ô nhịp tim thiếu thành NaN.
2. Kiểm tra shape, các đơn vị/kiểu số và số ô NaN ở cột tim.
3. Tổng hợp theo `axis=0` và in ba trung bình có nhãn/đơn vị.

No Note — Không chú thích

```
import numpy as np
raw = [[6,40,60],[7,43,62],[8,46,64],[9,49,None],[10,52,68],[6,55,70],[7,58,72],[8,41,74],
[9,44,76],[10,47,78],[6,50,80],[7,53,82],[8,56,84],[9,59,61],[10,42,63],[6,45,65],[7,48,67],
[8,51,None],[9,54,71],[10,57,73]]
records = []
for sleep, movement, heart in raw:
    if heart is None:
        cleaned_heart = np.nan
    else:
        cleaned_heart = heart
    records.append([sleep, movement, cleaned_heart])
data = np.array(records, dtype=float)
if data.shape != (20, 3) or data.dtype.kind != "f":
    raise ValueError("Cần đủ 20 ngày, ba cột dữ liệu dạng số thực.")
means = np.nanmean(data, axis=0)
print("Số ngày:", data.shape[0])
print("Số cột:", data.shape[1])
print("Thiếu nhịp tim:", int(np.isnan(data[:, 2]).sum()))
print(f"Ngủ (giờ): {means[0]:.2f}")
print(f"Vận động (phút): {means[1]:.2f}")
print(f"Nhịp tim (nhịp/phút): {means[2]:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tổng hợp 20 ngày ngủ/vận động/nhịp tim, bỏ NaN theo từng cột.
# Dữ liệu: 20 hàng ngày, cột 0 ngủ giờ, cột 1 vận động phút, cột 2 nhịp tim nhịp/phút; hai None ở
cột 2.
# NumPy cho phép mảng số thực chứa NaN và thống kê theo cột.
import numpy as np
# 20 ngày, mỗi hàng gồm (giờ ngủ, phút vận động, nhịp tim nhịp/phút).
raw = [[6,40,60],[7,43,62],[8,46,64],[9,49,None],[10,52,68],[6,55,70],[7,58,72],[8,41,74],
[9,44,76],[10,47,78],[6,50,80],[7,53,82],[8,56,84],[9,59,61],[10,42,63],[6,45,65],[7,48,67],
[8,51,None],[9,54,71],[10,57,73]]
records = []
for sleep, movement, heart in raw:
    # None là nhịp tim không được ghi nhận, không được hiểu là 0 nhịp/phút.
    if heart is None:
        # np.nan thay dữ liệu thiếu để tạo mảng float nhưng vẫn nhận biết được sự thiếu.
        cleaned_heart = np.nan
    else:
        cleaned_heart = heart
    # append() thêm một hàng 3 đặc trưng có đơn vị riêng.
    records.append([sleep, movement, cleaned_heart])
# dtype=float tạo ndarray (20,3) để tính mean và lưu nan.
data = np.array(records, dtype=float)
# shape (20,3) kiểm 20 ngày × 3 cột; dtype.kind 'f' xác minh nhóm số thực.
if data.shape != (20, 3) or data.dtype.kind != "f":
    # raise ValueError thông báo đầu vào sai và dừng để không xử lý kết quả thiếu hàng/cột.
```

```
    raise ValueError("Cần đủ 20 ngày, ba cột dữ liệu dạng số thực.")
# nanmean(...,axis=0) lấy một trung bình cho mỗi cột, tự bỏ NaN từng cột thay vì bỏ cả ngày.
means = np.nanmean(data, axis=0)
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
print("Số ngày:", data.shape[0])
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
print("Số cột:", data.shape[1])
# [:,2] chọn toàn bộ cột nhịp tim (cột thứ 3); isnan() đánh dấu thiếu, sum() đếm lượt.
print("Thiếu nhịp tim:", int(np.isnan(data[:, 2]).sum()))
# means[0] là giờ ngủ, [1] là phút vận động, [2] là nhịp tim/phút; :.2f in hai số thập phân.
print(f"Ngủ (giờ): {means[0]:.2f}")
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
# :.2f là định dạng chỉ để hiển thị hai chữ số sau dấu thập phân.
print(f"Vận động (phút): {means[1]:.2f}")
# int() đổi số NumPy sang số nguyên Python để in hoặc dùng làm chỉ số.
# :.2f là định dạng chỉ để hiển thị hai chữ số sau dấu thập phân.
print(f"Nhịp tim (nhịp/phút): {means[2]:.2f}")
```

Kết quả khi chạy:

```
Số ngày: 20
Số cột: 3
Thiếu nhịp tim: 2
Ngủ (giờ): 8.00
Vận động (phút): 49.50
Nhịp tim (nhịp/phút): 70.56
```

Mỗi cột có **đơn vị và ý nghĩa riêng**; không cộng ba cột để thành một chỉ số duy nhất. Đây là dữ liệu minh họa, không dùng để chẩn đoán.

6. Sai lầm thường gặp

- `shape=(3,2)` là **3 hàng và 2 cột**, không phải ngược lại; phải ghi rõ mỗi hàng/cột đại diện điều gì.
- `axis=0` tổng hợp theo từng cột trong mảng hai chiều; `axis=1` tổng hợp theo từng hàng. Chọn nhầm trục khiến ý nghĩa kết quả khác đi.
- `dtype=float` giữ phần thập phân, còn kiểu số nguyên có thể cắt phần lẻ nếu ép dữ liệu không phù hợp.
- Kết quả trung bình là số thực; các chữ số cuối có thể phụ thuộc biểu diễn dấu phẩy động, nên giải thích bằng giá trị gần đúng khi cần.
- `np.zeros()` chỉ tạo giá trị khởi tạo 0; không được diễn giải là dự báo thực tế đã được mô hình tính.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a NumPy `ndarray`? | `ndarray` của NumPy là gì?

A. A class inheritance rule | Quy tắc kế thừa lớp

B. A Python exception | Một ngoại lệ Python

C. A single text file | Một tệp văn bản đơn

D. An array for data with a common dtype | Mảng dữ liệu có kiểu phần tử chung

Đáp án: D

Giải thích: `ndarray` lưu phần tử theo một hay nhiều chiều có chỉ số và một `dtype` chung. Ví dụ `np.array([1, 2, 3])` là mảng số một chiều, không phải tệp hay ngoại lệ Python.

Q2. What does `shape=(3, 2)` mean for a 2-D array? | `shape=(3, 2)` nghĩa là gì với mảng hai chiều?

A. Only three total elements | Chỉ có ba phần tử

B. Three rows and two columns | Ba hàng và hai cột

C. Two rows and three columns | Hai hàng và ba cột

D. Three dimensions and two values | Ba chiều và hai giá trị

Đáp án: B

Giải thích: Với mảng hai chiều, `shape` ghi `(số_hàng, số_cột)`. Vì vậy `(3, 2)` là ba quan sát ở các hàng và hai đặc trưng ở các cột, tổng cộng sáu ô dữ liệu.

Q3. What is the size of an array with shape `(3, 2)`? | Mảng có shape `(3, 2)` chứa bao nhiêu phần tử?

A. 5 | 5

B. 6 | 6

C. 3 | 3

D. 2 | 2

Đáp án: B

Giải thích: `size` của ndarray đếm toàn bộ phần tử và bằng tích các kích thước trong `shape`. Với `(3, 2)`, tính `3 × 2 = 6`; cộng hai kích thước lại sẽ cho số phần tử sai.

Q4. What does `ndim` report? | Thuộc tính `ndim` cho biết gì?

A. The data type of an element | Kiểu phần tử

B. The number of rows only | Chỉ số hàng

C. The number of array dimensions | Số chiều của mảng

D. The maximum array value | Giá trị lớn nhất

Đáp án: C

Giải thích: `ndim` cho số trục/chiều, không phải số giá trị được lưu. Ma trận có shape `(3, 2)` có `ndim == 2` dù `size == 6`.

Q5. What does `dtype` describe? | `dtype` mô tả gì?

A. The file's extension | Phần mở rộng của file

B. The data type of array elements | Kiểu dữ liệu của các phần tử

C. The mean of all entries | Trung bình các phần tử

D. The array's row count | Số hàng

Đáp án: B

Giải thích: `dtype` mô tả kiểu và cách biểu diễn chung của các phần tử mảng, như `int64` hoặc `float64`. Nó khác `shape`, vốn cho biết số phần tử trên từng trục.

Q6. Why specify `dtype=float` when storing decimal scores? | Vì sao dùng `dtype=float` khi lưu điểm có phần lẻ?

A. To preserve fractional values | Để giữ giá trị có phần thập phân

B. To force integer truncation | Để ép cắt về số nguyên

C. To delete missing records | Để xóa bản ghi thiếu

D. To change rows into columns | Để đổi hàng thành cột

Đáp án: A

Giải thích: `dtype=float` yêu cầu NumPy lưu giá trị dưới dạng dấu phẩy động để điểm như `8.5` giữ được phần lẻ. Nếu đổi sang kiểu nguyên, phần lẻ có thể bị bỏ trong quá trình chuyển kiểu.

Q7. For a 2-D array, what does `mean(axis=0)` return? | Với mảng hai chiều, `mean(axis=0)` trả về gì?

A. Only the first cell | Chỉ ô đầu tiên

B. One mean per column | Một giá trị trung bình cho mỗi cột

C. The original array unchanged | Mảng gốc không đổi

D. One mean per row | Một giá trị trung bình cho mỗi hàng

Đáp án: B

Giải thích: Với mảng 2D, `axis=0` gộp theo chiều hàng và để lại một kết quả cho từng cột. Với `[[1, 3], [5, 7]]`, `mean(axis=0)` được `[3., 5.]`, không phải các trung bình theo hàng.

Q8. For a 2-D array, what does `sum(axis=1)` produce? | Với mảng hai chiều, `sum(axis=1)` tạo gì?

A. A list of column names | List tên cột

B. The shape tuple only | Chỉ tuple shape

C. One sum per column | Một tổng cho mỗi cột

D. One sum per row | Một tổng cho mỗi hàng

Đáp án: D

Giải thích: `sum(axis=1)` cộng các cột trong từng hàng. Với `[[1, 3], [5, 7]]`, tổng theo hàng là `4` và `12`, tạo kết quả một chiều gồm hai giá trị.

Q9. Which statement correctly distinguishes `shape` and `size`? | Đâu là điểm khác nhau giữa `shape` và `size`?

A. shape lists dimensions; size counts elements | shape ghi kích thước từng chiều; size đếm phần tử

B. size gives dtype; shape gives values | size cho kiểu dữ liệu; shape cho giá trị

C. They are always identical numbers | Chúng luôn là cùng một số

D. Both change elements automatically | Cả hai tự đổi phần tử

Đáp án: A

Giải thích: `shape` là tuple chỉ độ dài từng chiều, còn `size` là một số đếm toàn bộ phần tử. Mảng có shape `(3, 2)` gồm hai chiều và `size == 6`; hai thuộc tính này không tự sắp xếp lại dữ liệu.

Q10. What does `np.array([1, 2, 3])` create? | `np.array([1, 2, 3])` tạo ra gì?

A. A one-dimensional NumPy array | Một mảng NumPy một chiều

B. A class named Array | Lớp tên Array

C. A CSV file on disk | File CSV trên đĩa

D. A two-column table | Bảng hai cột

Đáp án: A

Giải thích: `np.array([1, 2, 3])` tạo ndarray có shape `(3,)`, nghĩa là một chiều chứa ba phần tử. Đây không phải bảng hai cột, vì bảng đó cần cấu trúc hai chiều.

Q11. What does `np.zeros((2, 3))` create? | `np.zeros((2, 3))` tạo gì?

A. A 2×3 table of missing strings | Bảng 2×3 gồm chuỗi thiếu

B. A 3×2 array of ones | Mảng 3×2 toàn số 1

C. A 2×3 array initialized to zeros | Mảng 2×3 được khởi tạo toàn số 0

D. An empty array with no elements | Mảng rỗng không có phần tử

Đáp án: C

Giải thích: `np.zeros((2, 3))` tạo hai hàng và ba cột, mọi ô khởi tạo bằng không. Mặc định các ô thuộc kiểu số thực, không phải chuỗi thiếu dữ liệu hay mảng không có phần tử.

Q12. How is `np.ones((2, 2))` different from `np.zeros((2, 2))`? | `np.ones((2, 2))` khác `np.zeros((2, 2))` thế nào?

A. It has four dimensions | Nó có bốn chiều

B. It cannot be used in arithmetic | Nó không thể tính toán

C. It always creates a CSV file | Nó luôn tạo file CSV

D. It initializes entries to one instead of zero | Nó khởi tạo giá trị 1 thay cho 0

Đáp án: D

Giải thích: `np.ones((2, 2))` và `np.zeros((2, 2))` cùng shape và có bốn ô số. Hàm thứ nhất điền `1`, hàm thứ hai điền `0`; không hàm nào tạo bốn chiều.

Q13. How do NumPy array and Python List differ in numeric operations? | Mảng NumPy khác List Python về phép toán số như thế nào?

A. A NumPy array stores only strings | Mảng NumPy chỉ lưu chuỗi

B. A List always multiplies elements element-wise | List luôn nhân từng phần tử

C. A List cannot hold numbers | List không chứa được số

D. NumPy enables element-wise numerical operations | NumPy hỗ trợ phép toán số theo từng phần tử

Đáp án: D

Giải thích: Với ndarray số, `a * 2` nhân từng phần tử lên hai: `[1, 2]` thành `[2, 4]`. Ngược lại, `[1, 2] * 2` của List Python lặp dãy thành `[1, 2, 1, 2]`.

Q14. For an array with rows as learners and columns as subjects, what does axis=0 average? | Nếu hàng là học viên và cột là môn, `axis=0` tính trung bình theo gì?

A. Only the last subject | Chỉ môn cuối

B. Each learner across subjects | Từng học viên qua các môn

C. Each subject across learners | Từng môn qua các học viên

D. Only the first learner | Chỉ học viên đầu

Đáp án: C

Giải thích: Nếu mỗi hàng là một học viên và mỗi cột là một môn, `mean(axis=0)` tính trung bình qua các học viên cho từng môn riêng. Có hai môn thì được hai trung bình; `axis=1` lại tính trung bình các môn của từng học viên.

Q15. How many dimensions does a two-dimensional score table have? | Bảng điểm hai chiều có `ndim` bằng bao nhiêu?

A. The total number of scores | Tổng số điểm

B. 3 | 3

C. 2 | 2

D. 1 | 1

Đáp án: C

Giải thích: Bảng điểm sắp theo hàng và cột có hai trục, dù có bao nhiêu hàng. Vì vậy `ndim` bằng `2`; tổng số ô điểm cần đọc từ `size`.

Q16. Which code accesses array shape without computing new values? | Cú pháp nào lấy shape mà không tính thêm dữ liệu?

A. `X.sort()` | `X.sort()`

B. `X.shape` | `X.shape`

C. `X.delete()` | `X.delete()`

D. `X.write()` | `X.write()`

Đáp án: B

Giải thích: `X.shape` đọc thuộc tính ndarray, trả tuple kích thước các chiều mà không sửa giá trị. Với bảng ba hàng hai cột, kết quả là `(3, 2)`; `X.sort()` thì thay đổi thứ tự phần tử.

Q17. Which code counts all array entries? | Cú pháp nào đếm tổng số phần tử mảng?

A. `X.shape[0]` in every case | Luôn `X.shape[0]`

B. `X.dtype` | `X.dtype`

C. `X.size` | `X.size`

D. `X.mean()` | `X.mean()`

Đáp án: C

Giải thích: `X.size` đếm phần tử qua toàn bộ các chiều. Với mảng `(3, 2)`, kết quả là 6, còn `X.shape[0]` chỉ trả số hàng là 3.

Q18. For finite floating-point values within the target integer range, what does NumPy `.astype(int)` do to their fractional parts? | Với giá trị dấu phẩy động hữu hạn nằm trong phạm vi kiểu số nguyên đích, NumPy `.astype(int)` xử lý phần lẻ thế nào?

A. It discards fractional parts toward zero | Bỏ phần lẻ theo hướng về số 0

B. It rounds every value to the nearest integer | Làm tròn mọi giá trị về số nguyên gần nhất

C. It preserves fractional parts unchanged | Giữ nguyên toàn bộ phần lẻ

D. It forces the shape to become `(1, 1)` | Buộc shape chuyển thành `(1, 1)`

Đáp án: A

Giải thích: Với số thực hữu hạn thuộc phạm vi kiểu nguyên đích, `.astype(int)` bỏ phần lẻ theo hướng về không chứ không làm tròn gần nhất. Vì vậy `[2.9, -2.9]` đổi thành số nguyên `[2, -2]` và vẫn giữ shape một chiều ban đầu.

Q19. What is the output shape of `X.mean(axis=0)` for `X.shape == (3,2)`? | Nếu `X.shape == (3,2)`, `X.mean(axis=0)` có shape nào?

A. `(2,)` | `(2,)`

B. `(3,)` | `(3,)`

C. `(3,2)` | `(3,2)`

D. `(1,3)` | `(1,3)`

Đáp án: A

Giải thích: Ma trận `(3, 2)` gồm ba hàng và hai cột. `mean(axis=0)` gộp ba hàng, để lại một trung bình cho mỗi cột nên shape mặc định là `(2,)`, không phải `(3,)`.

Q20. Why label the meaning of each row and column? | Vì sao phải nêu rõ ý nghĩa từng hàng, cột?

A. To convert a matrix into text | Để chuyển ma trận thành chữ

B. To change dtype automatically | Để tự đổi dtype

C. To avoid all numeric calculations | Để tránh mọi phép tính

D. To interpret aggregated results correctly | Để hiểu đúng kết quả tổng hợp

Đáp án: D

Giải thích: Nhãn hàng và cột cho biết giá trị tổng hợp đang đo cái gì chứ không chỉ cách tính. Nếu hàng là học viên và cột là môn, `mean(axis=0)` trả điểm trung bình từng môn; hiểu ngược cấu trúc sẽ trả lời câu hỏi khác.

8. Nối sang bài tiếp theo

`List → NumPy Array → Matrix → PCA`.

PY-14 — NumPy indexing, slicing, boolean và fancy indexing

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Index — chỉ số; **Slice** — cắt lát.

`NumPy ndarray → Chọn hàng/cột → DataFrame/PCA`

Học xong dùng để làm gì? Lọc quan sát, chọn cột đặc trưng, hiểu lúc nào thao tác sửa dữ liệu gốc qua view.

Kết nối kiến thức tiếp theo: PY-15 thực hiện phép tổng hợp và broadcasting trên dữ liệu được chọn.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Index	chỉ số	Địa chỉ phần tử của mảng
Slice	cắt lát	Lấy một đoạn theo start:stop
Boolean mask	mặt nạ logic	Mảng True/False để lọc phần tử
Fancy indexing	chỉ số nâng cao	Chọn nhiều vị trí theo danh sách chỉ số

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`X[row,col]`	Chọn một ô theo hàng/cột, đếm từ 0
`X[:,j] / X[i,:]`	Lấy cả cột j hoặc hàng i
`X[condition]`	Giữ các phần tử thỏa điều kiện logic
`X[:,[0,2]]`	Chọn các cột không liền nhau

4. Khái niệm liên quan

Trong bảng điểm NumPy, **hàng là học viên**, **cột là môn học**, **ô là điểm một học viên ở một môn**. Bài này đi từ chọn một ô bằng chỉ số đến chọn nhiều ô/hàng/cột bằng slice, mặt nạ Boolean và fancy indexing. Mỗi cách trả lời một câu hỏi khác nhau khi làm việc với dữ liệu.

4.1. Indexing — Đọc một vị trí

Từ vựng trong mục: Index — chỉ số; **Row/column** — hàng/cột.

Mảng hai chiều giống một bảng có hàng và cột. Giả sử `scores` có **hai hàng học viên** An, Bình và **hai cột môn** Toán, Python, lần lượt là `[[8,9],[6,7]]`. **Chỉ số bắt đầu từ 0**; `scores[1,0]` lấy hàng thứ hai (Bình), cột đầu (Toán), cho điểm 6. Dấu phẩy trong `[row, col]` phân biệt vị trí hàng và cột.

No Note — Không chú thích

```
import numpy as np

scores = np.array([[8, 9], [6, 7]])
print(scores[1, 0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc điểm ở hàng thứ hai, cột thứ nhất của mảng hai chiều.
# Dữ liệu: 2 hàng học viên, 2 cột điểm theo hai môn; chỉ số NumPy đếm từ 0.
# NumPy dùng np.array() để tạo mảng 2 chiều.
import numpy as np

# Hai list con thành hai hàng, mỗi hàng có hai cột.
scores = np.array([[8, 9], [6, 7]])
# [1,0] chọn hàng chỉ số 1 (học viên thứ hai), cột 0 (môn thứ nhất), kết quả 6.
print(scores[1, 0])
```

Kết quả khi chạy:

```
6
```

Chỉ số hàng 1 và cột 0 lấy điểm Toán của học viên thứ hai, không phải điểm học viên đầu.

4.2. Slicing — Cắt một đoạn liên tiếp

Từ vựng trong mục: Slice — lát cắt; **Start/stop** — đầu/cuối.

Slicing chọn một đoạn theo chỉ số, cú pháp `[start:stop]` lấy từ start **đến trước stop**. Với điểm `[5,6,7,8]`, `scores[1:3]` lấy vị trí 1 và 2, nghĩa là hai điểm 6, 7. Trên bảng hai chiều, `scores[:,0]` lấy tất cả hàng nhưng chỉ cột đầu; dấu `:` đứng riêng nghĩa là lấy toàn bộ chiều đó. Lát cắt NumPy cơ bản có thể là **view** dùng chung bộ nhớ với mảng gốc.

No Note — Không chú thích

```
import numpy as np

scores = np.array([5, 6, 7, 8])
print(scores[1:3])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cắt lấy hai phần tử liên tiếp bằng chỉ số bắt đầu/kết thúc.
# Dữ liệu: Một vector 4 số 5,6,7,8; [1:3] lấy các chỉ số 1,2.
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np

# Tạo vector một chiều từ bốn phần tử.
scores = np.array([5, 6, 7, 8])
```

```
# Slice [bắt đầu:kết thúc] lấy chỉ số 1 và 2, KHÔNG lấy 3; trả mảng [6,7].
print(scores[1:3])
```

Kết quả khi chạy:

```
[6 7]
```

Giá trị 8 ở vị trí 3 không được lấy vì điểm stop bị loại khỏi lát cắt.

4.3. Boolean mask — Lọc theo điều kiện

Từ vựng trong mục: Boolean mask — mặt nạ logic; **Condition** — điều kiện.

Nếu các điểm là `[4,7,9]` và yêu cầu giữ điểm từ 7 trở lên, phép `scores >= 7` tạo mặt nạ `[False, True, True]`. Dùng `scores[mask]` chỉ lấy phần tử ứng với `True`. Với bảng hai chiều, `X[X[:,0] >= 8]` giữ **các hàng có điểm ở cột đầu đạt 8**, chứ không chỉ lấy các ô điểm cao.

No Note — Không chú thích

```
import numpy as np

scores = np.array([4, 7, 9])
mask = scores >= 7
print(scores[mask])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo mask Boolean chọn tất cả điểm từ 7 trở lên.
# Dữ liệu: 3 điểm số 4,7,9; mask so theo từng phần tử.
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np

# Mảng 3 điểm cho phép so sánh vector hóa.
scores = np.array([4, 7, 9])
# So từng ô với 7 tạo mask Boolean [False,True,True].
mask = scores >= 7
# Boolean indexing lấy chỉ các ô mask True, trả mảng mới [7,9].
print(scores[mask])
```

Kết quả khi chạy:

```
[7 9]
```

Hai giá trị 7 và 9 vượt hoặc bằng ngưỡng, còn 4 bị loại.

4.4. Fancy indexing — Chọn vị trí rời rạc

Từ vựng trong mục: Fancy indexing — chỉ số nâng cao; **Index array** — mảng chỉ số.

Khác với **Slice** lấy đoạn liên tiếp, **Fancy indexing** cho phép chỉ định nhiều vị trí không liền nhau. Với `[3,5,7]`, `scores[[0,2]]` lấy phần tử đầu và cuối, bỏ phần tử giữa. Trên bảng hai chiều, `X[:, [0,2]]` lấy cột thứ nhất và thứ ba. Fancy indexing bằng mảng chỉ số thường tạo **bản sao** thay vì view của dữ liệu gốc.

No Note — Không chú thích

```
import numpy as np

scores = np.array([3, 5, 7])
print(scores[[0, 2]])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn hai chỉ số không liền nhau bằng fancy indexing.
# Dữ liệu: 3 điểm 3,5,7; chỉ số được chọn là 0 và 2.
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np

# Mảng đầu gồm ba phần tử ở chỉ số 0,1,2.
scores = np.array([3, 5, 7])
# Fancy indexing dùng list chỉ số rời rạc [0,2], trả [3,7] thành một mảng mới.
print(scores[[0, 2]])
```

Kết quả khi chạy:

```
[3 7]
```

Chương trình lấy vị trí 0 và 2 mà không phải lấy cả đoạn 0–2.

5. Năm tình huống chỉ số NumPy — từ ô đơn đến lọc nhiều điều kiện

Tình huống 1 — Cấp 1: Lấy đúng một ô điểm từ bảng 10 học viên

Từ vựng trong mục: Index — chỉ số; **Row** — hàng; **Column** — cột.

Tình huống thực tế: Giáo viên có bảng điểm **10 học viên H01–H10**, mỗi hàng ghi điểm **Toán trước, Python sau** trên thang **10 điểm**. Họ muốn tra riêng điểm Python của học viên **H04** để so với phiếu giấy; các hàng và cột đã có thứ tự nên không cần duyệt mọi điểm để tìm một ô.

Vì sao chọn cách này? NumPy lưu bảng thành mảng **hai chiều**: chỉ số đầu là **hàng học viên**, chỉ số sau là **cột môn**. Python đếm chỉ số từ **0**, nên học viên thứ tư H04 nằm ở hàng `3`, còn môn Python là cột thứ hai nên có chỉ số `1`. Lệnh `scores[3, 1]` lấy đúng **một ô điểm**, không lấy toàn bộ hàng. Nếu dùng `scores[4, 1]`, chương trình sẽ đọc nhầm H05, một lỗi hay gặp khi người mới áp dụng số thứ tự thông thường cho chỉ số Python.

Quy trình:

1. Tạo mảng `(10,2)` đúng thứ tự mã người và môn.
2. Dùng một cặp chỉ số hàng–cột để lấy H04/Python.
3. In hình dạng dữ liệu và giá trị đã chọn.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[8,9],[7,8],[6,7],[9,9],[10,7],[7,6],[8,8],[5,7],[9,10],[8,6]])
if scores.shape != (10, 2):
    raise ValueError("Bảng cần đúng 10 học viên, 2 môn.")
print("Học viên:", scores.shape[0])
print("Python của H04:", int(scores[3, 1]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc điểm Python của H04 trong bảng 10 học viên × 2 môn.
# Dữ liệu: 10 hàng học viên H01..H10, cột 0 môn A, cột 1 điểm Python (thang 10).
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np
# Mảng 10 hàng × 2 cột, mỗi hàng là điểm của học viên tương ứng.
scores = np.array([[8,9],[7,8],[6,7],[9,9],[10,7],[7,6],[8,8],[5,7],[9,10],[8,6]])
# shape là (số hàng,số cột); shape (10,2) kiểm đầu vào.
if scores.shape != (10, 2):
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Bảng cần đúng 10 học viên, 2 môn.")
# shape là (số hàng,số cột); shape (10,2) kiểm đầu vào.
# shape[0] trả 10 học viên, khác shape[1] là hai môn.
print("Học viên:", scores.shape[0])
# [3,1] đọc hàng H04 (vì index bắt đầu 0), cột thứ hai Python; int() đổi thành số nguyên Python.
print("Python của H04:", int(scores[3, 1]))
```

Kết quả khi chạy:

```
Học viên: 10
Python của H04: 9
```

Giá trị `scores[3,1]` là 9 điểm trên thang 10; đây là một ô, không phải nguyên hàng học viên.

Tình huống 2 — Cấp 2: Cắt bốn ngày đầu và một cột trong 12 ngày

Từ vựng trong mục: Slice — lát cắt; **Step** — bước nhảy; **Array view** — vùng nhìn mảng.

Tình huống thực tế: Siêu thị lưu doanh thu trong **12 ngày**, mỗi ngày có ba cột **thực phẩm, sách, đồ dùng**, đơn vị **nghìn đồng**. Quản lý cần cộng doanh thu thực phẩm của **bốn ngày đầu** để xem kết quả đầu kỳ, đồng thời tra riêng doanh thu đồ dùng của **ngày thứ 12**. Hai yêu cầu này chỉ liên quan tới các phần nhất định của bảng.

Vì sao chọn cách này? Indexing là chọn một ô cụ thể; **slicing** là chọn một đoạn liên tiếp. `sales[:4, 0]` lấy các hàng từ đầu **đến trước hàng chỉ số 4**, tức bốn ngày đầu, và chỉ chọn cột thực phẩm có chỉ số 0. `sales[-1, 2]` đọc **hàng cuối** (ngày 12) và cột đồ dùng có chỉ số 2. Sau đó cộng các giá trị trong lát cắt thực phẩm; không cộng cả ba cột vì quản lý chỉ hỏi một nhóm. Số âm `-1` trong chỉ số nghĩa là vị trí cuối, không phải doanh thu âm.

Quy trình:

1. Tạo mảng `(12,3)`, mỗi hàng là một ngày.
2. Lấy cột thực phẩm của ngày 1–4 bằng `data[:4, 0]`.
3. Cộng lát cắt và lấy ô đồ dùng ngày cuối.

No Note — Không chú thích

```
import numpy as np
sales = np.array([[100,50,25],[110,55,27],[120,60,29],[130,65,31],[140,70,33],[150,75,35],
[160,80,37],[170,85,39],[180,90,41],[190,95,43],[200,100,45],[210,105,47]], dtype=int)
first_four_food = sales[:4, 0]
if sales.shape != (12, 3) or first_four_food.shape != (4,):
    raise ValueError("Cần đủ 12 ngày và 4 doanh thu thực phẩm đầu.")
print("Ngày:", sales.shape[0])
```

```
print("Thực phẩm 4 ngày (nghìn đồng):", int(first_four_food.sum()))
print("Đồ dùng ngày 12:", int(sales[-1, 2]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cắt doanh thu thực phẩm bốn ngày đầu và đồ dùng ngày 12.
# Dữ liệu: 12 hàng ngày × 3 cột doanh thu thực phẩm, sách và đồ dùng (nghìn đồng).
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np
# 12 hàng ngày và 3 cột doanh thu; dtype=int giữ số nguyên nghìn đồng.
sales = np.array([[100,50,25],[110,55,27],[120,60,29],[130,65,31],[140,70,33],[150,75,35],
[160,80,37],[170,85,39],[180,90,41],[190,95,43],[200,100,45],[210,105,47]], dtype=int)
# [:4,0] lấy hàng 0..3 (bốn ngày đầu), chỉ cột 0 thực phẩm; view theo slice.
first_four_food = sales[:4, 0]
# shape (12,3); first_four_food.shape (4,) là vector bốn giá trị.
if sales.shape != (12, 3) or first_four_food.shape != (4,):
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Cần đủ 12 ngày và 4 doanh thu thực phẩm đầu.")
# shape (12,3); first_four_food.shape (4,) là vector bốn giá trị.
print("Ngày:", sales.shape[0])
# sum() cộng bốn giá trị thực phẩm đầu (nghìn đồng).
print("Thực phẩm 4 ngày (nghìn đồng):", int(first_four_food.sum()))
# [-1,2] lấy hàng cuối (ngày 12), cột thứ ba đồ dùng; int() in số nguyên.
print("Đồ dùng ngày 12:", int(sales[-1, 2]))
```

Kết quả khi chạy:

```
Ngày: 12
Thực phẩm 4 ngày (nghìn đồng): 460
Đồ dùng ngày 12: 47
```

Vị trí cuối của lát cắt là **giới hạn không bao gồm**, nên `[:4]` gồm bốn ngày đầu chứ không phải năm ngày.

Tình huống 3 — Cấp 3: Lọc 15 học viên thỏa hai ngưỡng bằng mặt nạ

Từ vựng trong mục: Boolean mask — mặt nạ đúng/sai; **AND** — kết hợp điều kiện; **Filtering** — lọc.

Tình huống thực tế: Khóa học có **15 học viên H01–H15** với điểm ba môn **Toán, Văn, Python**, mỗi môn thang **10 điểm**. Lớp thực hành nâng cao chỉ nhận người có **Toán từ 8 điểm** và **Python từ 8 điểm**, cả hai phải đúng. Giáo vụ cần biết có bao nhiêu học viên đủ điều kiện và mã của họ để gửi lịch học.

Vì sao chọn cách này? Mặt nạ Boolean là một dãy giá trị `True`/`False`, tương ứng từng học viên có đạt điều kiện hay không. NumPy so toàn bộ cột Toán với mốc 8, rồi so cột Python với mốc 8; dấu `&` ghép hai kết quả **theo từng hàng**, nên chỉ người vượt cả hai ngưỡng mới có `True`. Đặt ngoặc quanh mỗi phép so sánh để Python hiểu đúng thứ tự thực hiện. `np.flatnonzero(mask)` lấy vị trí các hàng được chọn; cộng 1 khi chuyển vị trí thành mã H01… vì chỉ số bắt đầu từ 0. Không dùng `and` thông thường với hai mảng nhiều phần tử vì nó không thực hiện phép kiểm tra từng hàng như mong muốn.

Quy trình:

1. Tạo bảng `(15,3)`, xác định cột Toán 0 và Python 2.
2. Tạo mặt nạ `(X[:,0]>=8) & (X[:,2]>=8)`.
3. Dùng `np.flatnonzero` lấy chỉ số hàng và tạo mã học viên.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[5,7,4],[6,9,5],[7,7,6],[8,9,7],[9,7,8],[10,9,9],[5,7,10],[6,9,4],[7,7,5],
[8,9,6],[9,7,7],[10,9,8],[5,7,9],[6,9,10],[7,7,4]], dtype=int)
mask = (scores[:, 0] >= 8) & (scores[:, 2] >= 8)
selected = np.flatnonzero(mask)
ids = []
for i in selected:
    ids.append(f"H{i+1:02d}")
if scores.shape != (15, 3):
    raise ValueError("Bảng cần đúng 15 học viên và 3 điểm.")
print("Học viên:", scores.shape[0])
print("Đủ điều kiện:", len(ids))
if ids:
    labels = ", ".join(ids)
else:
    labels = "Không có"
print("Mã:", labels)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc 15 học viên đạt đồng thời ngưỡng điểm môn 1 và môn 3.
# Dữ liệu: 15 hàng học viên, cột 0/1/2 ứng với ba môn; yêu cầu môn đầu và môn thứ ba đều >=8.
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np
# 15 hàng học viên, 3 cột điểm; dtype=int dùng số nguyên.
scores = np.array([[5,7,4],[6,9,5],[7,7,6],[8,9,7],[9,7,8],[10,9,9],[5,7,10],[6,9,4],[7,7,5],
[8,9,6],[9,7,7],[10,9,8],[5,7,9],[6,9,10],[7,7,4]], dtype=int)
# [:,0] là toàn bộ 15 giá trị cột môn thứ nhất, [:,2] là cột thứ ba.
# & nối TỪNG phần tử Boolean của hai điều kiện, ngoặc bao mỗi so sánh; khác từ khóa and cho scalar.
mask = (scores[:, 0] >= 8) & (scores[:, 2] >= 8)
# flatnonzero() trả chỉ số 0-based của những vị trí mask True, không trả điểm.
selected = np.flatnonzero(mask)
ids = []
# Duyệt vị trí 0-based, cộng 1 rồi f-string :02d tạo mã H01..H15.
for i in selected:
    ids.append(f"H{i+1:02d}")
# shape (15,3) xác minh đúng số học viên/môn.
if scores.shape != (15, 3):
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Bảng cần đúng 15 học viên và 3 điểm.")
# shape (15,3) xác minh đúng số học viên/môn.
print("Học viên:", scores.shape[0])
# len() đếm mã đạt cả hai điều kiện.
print("Đủ điều kiện:", len(ids))
# List rỗng có giá trị False; nếu có mã mới ghép chuỗi.
if ids:
    # join() ghép các mã bằng dấu phẩy và khoảng trắng.
    labels = ", ".join(ids)
else:
    labels = "Không có"
# int() chuyển NumPy scalar về số nguyên Python để in.
print("Mã:", labels)
```

Kết quả khi chạy:

```
Học viên: 15
Đủ điều kiện: 3
Mã: H05, H06, H12
```

Mặt nạ đánh dấu từng **hàng học viên**, không xóa cột khác và không bỏ qua điều kiện nào.

Tình huống 4 — Cấp 4: Chọn bốn dòng theo mã từ 18 đơn và giữ bản gốc

Từ vựng trong mục: Fancy indexing — chọn nhiều vị trí; **Copy** — bản sao; **Repeated index** — chỉ số lặp.

Tình huống thực tế: Đội kiểm toán nhận bảng gốc gồm **18 đơn O01–O18**. Mỗi hàng có đơn giá (**nghìn đồng/món**), số lượng (**món**) và mã kênh bán. Kiểm toán viên được yêu cầu xem bốn dòng theo **thứ tự O18, O05, O12, O05**; mã O05 xuất hiện **hai lần trong yêu cầu tra cứu** chứ không phải vì bảng gốc có hai đơn O05. Cần tính tổng tiền của đúng bốn dòng đã yêu cầu và thử sửa một ô trong bảng kết quả mà không làm đổi dữ liệu gốc.

Vì sao chọn cách này? Fancy indexing nghĩa là chọn các hàng bằng một **danh sách vị trí bất kỳ**, không nhất thiết liên tiếp hay tăng dần. `[17, 4, 11, 4]` tương ứng O18, O05, O12, O05: hàng O05 được đưa vào **hai lần trong bảng truy vấn**, vì đúng như yêu cầu kiểm toán. Tổng được tính trên bốn dòng truy vấn, nên tiền O05 cũng được tính hai lần trong **tổng truy vấn**; đây không phải doanh thu thật của 18 đơn nguồn. Với cách chọn này NumPy tạo một **mảng mới**, vì vậy đặt giá ở dòng đầu của mảng kết quả về 0 không làm mất giá O18 trong bảng gốc.

Quy trình:

1. Tạo mảng `(18,3)` với ba cột được ghi rõ.
2. Chọn các vị trí 17, 4, 11, 4 bằng fancy indexing.
3. Tính tổng tiền bốn dòng truy vấn, sửa một ô trong bản chọn rồi đối chiếu mảng gốc.

No Note — Không chú thích

```
import numpy as np
orders = np.array([[100,1,0],[120,2,1],[140,3,2],[160,4,0],[180,5,1],[200,1,2],[220,2,0],[240,3,1],
[260,4,2],[280,5,0],[300,1,1],[320,2,2],[340,3,0],[360,4,1],[380,5,2],[400,1,0],[420,2,1],
[440,3,2]], dtype=int)
indices = [17, 4, 11, 4]
selected = orders[indices]
query_total = int((selected[:, 0] * selected[:, 1]).sum())
original_price = int(orders[17, 0])
selected[0, 0] = 0
if orders.shape != (18, 3) or selected.shape != (4, 3):
    raise ValueError("Bảng gốc phải có 18 đơn và truy vấn có 4 dòng.")
print("Đơn nguồn:", orders.shape[0])
print("Dòng truy vấn:", selected.shape[0])
print("Tổng dòng truy vấn (nghìn đồng):", query_total)
print("Giá O18 trong nguồn:", int(orders[17, 0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn các dòng hóa đơn rời rạc, bao gồm dòng trùng, mà không sửa mảng gốc.
# Dữ liệu: 18 đơn, cột 0 giá (nghìn đồng), cột 1 số món, cột 2 mã loại; chỉ số [17,4,11,4] truy vấn
O18, O05, O12, O05.
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np
# 18 hàng đơn × 3 cột (giá, số lượng, loại); dtype=int để tính tiền.
orders = np.array([[100,1,0],[120,2,1],[140,3,2],[160,4,0],[180,5,1],[200,1,2],[220,2,0],[240,3,1],
[260,4,2],[280,5,0],[300,1,1],[320,2,2],[340,3,0],[360,4,1],[380,5,2],[400,1,0],[420,2,1],
[440,3,2]], dtype=int)
# Danh sách [17,4,11,4] chọn bốn hàng, trong đó O05 được chọn hai lần.
indices = [17, 4, 11, 4]
# Fancy indexing tạo BẢN SAO mảng, giữ cả hai lần chỉ số 4; không phải slice/view.
selected = orders[indices]
# [:,0] lấy giá, [:,1] số lượng của bốn dòng đã chọn; nhân từng hàng rồi sum() tiền nghìn đồng.
query_total = int((selected[:, 0] * selected[:, 1]).sum())
```

```
# Đọc giá O18 từ mảng nguồn trước khi thay đổi bản sao.
# Đối chiếu giá gốc O18 không thay đổi; int() đổi NumPy scalar về int Python.
original_price = int(orders[17, 0])
# Sửa ô đầu trong BẢN SAO selected, không sửa orders do fancy indexing.
selected[0, 0] = 0
# shape (18,3), selected.shape (4,3) kiểm đúng dữ liệu truy vấn.
if orders.shape != (18, 3) or selected.shape != (4, 3):
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Bảng gốc phải có 18 đơn và truy vấn có 4 dòng.")
# shape (18,3), selected.shape (4,3) kiểm đúng dữ liệu truy vấn.
print("Đơn nguồn:", orders.shape[0])
# int() chuyển NumPy scalar về số nguyên Python để in.
print("Dòng truy vấn:", selected.shape[0])
# int() chuyển NumPy scalar về số nguyên Python để in.
print("Tổng dòng truy vấn (nghìn đồng):", query_total)
# Đọc giá O18 từ mảng nguồn trước khi thay đổi bản sao.
# Đối chiếu giá gốc O18 không thay đổi; int() đổi NumPy scalar về int Python.
print("Giá O18 trong nguồn:", int(orders[17, 0]))
```

Kết quả khi chạy:

```
Đơn nguồn: 18
Dòng truy vấn: 4
Tổng dòng truy vấn (nghìn đồng): 3760
Giá O18 trong nguồn: 440
```

Sửa bản chọn không đổi mảng gốc, vì fancy indexing tạo copy. Hai dòng truy vấn O05 là cố tình lặp để giải thích hành vi chọn mảng; **không** dùng tổng đó làm báo cáo doanh thu.

Tình huống 5 — Cấp 5: Kết hợp Boolean mask, NaN và copy cho 20 cảm biến

Từ vựng trong mục: View — vùng nhìn; **Boolean indexing** — chọn theo điều kiện; **NaN** — giá trị thiếu.

Tình huống thực tế: Trạm theo dõi **20 cảm biến S01–S20**, mỗi cảm biến có nhiệt độ **°C** và độ ẩm **%**. Hai cảm biến chưa gửi được nhiệt độ nên được đánh dấu là thiếu dữ liệu. Kỹ thuật viên cần chọn những thiết bị có nhiệt độ **hợp lệ từ 28 °C trở lên** và độ ẩm **từ 55% trở lên**, in số lượng và mã của chúng. Sau đó họ muốn đánh dấu một giá trị trong bảng đã lọc để kiểm tra rằng bảng số đo gốc không bị sửa.

Vì sao chọn cách này? `np.isfinite()` giúp phát hiện nhiệt độ có phải **một số hữu hạn hợp lệ** không; chỗ thiếu được chuyển thành `np.nan` và bị loại trước khi xét ngưỡng. Chương trình tạo ba mặt nạ: nhiệt độ hợp lệ, nhiệt độ ít nhất 28 °C, độ ẩm ít nhất 55%; dùng `&` để chỉ chọn cảm biến thỏa **cả ba**. `np.flatnonzero()` trả vị trí thiết bị được chọn, từ đó tạo mã S01… Boolean indexing tạo **mảng kết quả độc lập** trong trường hợp này, nên thay giá ở mảng đã lọc không sửa bảng nguồn. Không được coi NaN là 0 °C vì sẽ che mất hai thiết bị chưa gửi số đo.

Quy trình:

1. Tạo mảng `(20,2)` kiểu float, chuyển nhiệt độ thiếu sang NaN.
2. Lọc theo ba điều kiện hợp lệ, nhiệt độ và độ ẩm.
3. Lấy mã từ chỉ số hàng, thay thử một giá trị trên mảng được chọn và xác nhận nguồn không đổi.

No Note — Không chú thích

```
import numpy as np
raw_temp = [24,25,26,27,28,None,30,31,24,25,26,27,28,29,None,31,24,25,26,27]
humidity = [50,51,52,53,54,55,56,57,58,59,50,51,52,53,54,55,56,57,58,59]
rows = []
```

```
for temp, h in zip(raw_temp, humidity):
    if temp is None:
        clean_temp = np.nan
    else:
        clean_temp = temp
    rows.append([clean_temp, h])
data = np.array(rows, dtype=float)
mask = np.isfinite(data[:, 0]) & (data[:, 0] >= 28) & (data[:, 1] >= 55)
selected = data[mask]
selected_ids = []
for i in np.flatnonzero(mask):
    selected_ids.append(f"S{i+1:02d}")
if len(selected):
    selected[0, 0] = 0
if data.shape != (20, 2):
    raise ValueError("Cần 20 cảm biến với hai trường đo.")
if int(np.isnan(data[:, 0]).sum()) != 2:
    raise ValueError("Phải ghi nhận đúng hai nhiệt độ bị thiếu.")
if np.any(data[mask, 0] == 0):
    raise ValueError("Truy vấn không được thay đổi dữ liệu cảm biến gốc.")
print("Cảm biến:", data.shape[0])
print("Thiếu nhiệt độ:", int(np.isnan(data[:, 0]).sum()))
print("Cần kiểm tra:", len(selected_ids))
if selected_ids:
    labels = ", ".join(selected_ids)
else:
    labels = "Không có"
print("Mã:", labels)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc 20 cảm biến theo nhiệt độ và độ ẩm, kiểm bản sao không sửa nguồn.
# Dữ liệu: 20 cảm biến, cột 0 nhiệt độ °C với 2 None, cột 1 độ ẩm %; cần nhiệt độ >=28, độ ẩm >=55.
# import numpy as np: đặt tên ngắn np để dùng các thao tác ndarray.
import numpy as np
# 20 nhiệt độ °C, None biểu thị hai giá trị thiếu, không phải 0 độ.
raw_temp = [24,25,26,27,28,None,30,31,24,25,26,27,28,29,None,31,24,25,26,27]
# 20 độ ẩm % cùng vị trí với raw_temp.
humidity = [50,51,52,53,54,55,56,57,58,59,50,51,52,53,54,55,56,57,58,59]
rows = []
# zip() ghép nhiệt độ và độ ẩm theo chỉ số để mỗi hàng là một cảm biến.
for temp, h in zip(raw_temp, humidity):
    # is None bắt nhiệt độ chưa đo và gắn marker NaN.
    if temp is None:
        # np.nan là giá trị thiếu kiểu số thực, giữ được trong NumPy dtype=float.
        clean_temp = np.nan
    else:
        clean_temp = temp
    # append() ghi hàng (nhiệt độ đã xử lý, độ ẩm).
    rows.append([clean_temp, h])
# dtype=float cho phép lưu cả số và np.nan trong mảng 20×2.
data = np.array(rows, dtype=float)
# [:,0] lấy cột nhiệt độ mọi hàng; isfinite() trả True với số hữu hạn, False với NaN/inf.
# & nối Boolean từng hàng, ngoặc đảm bảo so sánh được tính riêng; nhiệt độ từ 28 °C.
# Cột 1 là độ ẩm, lấy từ 55% trở lên; cả ba mask phải True.
mask = np.isfinite(data[:, 0]) & (data[:, 0] >= 28) & (data[:, 1] >= 55)
# Boolean indexing trả BẢN SAO các hàng đạt điều kiện, không phải view.
selected = data[mask]
selected_ids = []
# flatnonzero() cho các vị trí 0-based của hàng được chọn.
for i in np.flatnonzero(mask):
    # f-string :02d tạo mã S01..S20 từ i+1, append() lưu mã.
    selected_ids.append(f"S{i+1:02d}")
# len() là số hàng được chọn; chỉ sửa selected khi có ít nhất một hàng.
if len(selected):
```

```
    # Thử thay dữ liệu trong mảng lọc; vì boolean indexing là copy, dữ liệu gốc data vẫn giữ
nguyên.
    selected[0, 0] = 0
# shape (20,2) đối chiếu số cảm biến và số trường đo.
if data.shape != (20, 2):
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Cần 20 cảm biến với hai trường đo.")
# isnan() đánh dấu đúng hai NaN nhiệt độ, sum() trên mask đếm chúng.
if int(np.isnan(data[:, 0]).sum()) != 2:
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Phải ghi nhận đúng hai nhiệt độ bị thiếu.")
# any() kiểm có bất kỳ nhiệt độ nguồn của các hàng đã chọn bị đổi thành 0 hay không.
if np.any(data[mask, 0] == 0):
    # ValueError thông báo dữ liệu hoặc phép kiểm không đạt, dừng việc tính tiếp.
    raise ValueError("Truy vấn không được thay đổi dữ liệu cảm biến gốc.")
# shape (20,2) đối chiếu số cảm biến và số trường đo.
print("Cảm biến:", data.shape[0])
# isnan() đánh dấu đúng hai NaN nhiệt độ, sum() trên mask đếm chúng.
print("Thiếu nhiệt độ:", int(np.isnan(data[:, 0]).sum()))
# int() chuyển NumPy scalar về số nguyên Python để in.
print("Cần kiểm tra:", len(selected_ids))
if selected_ids:
    labels = ", ".join(selected_ids)
else:
    labels = "Không có"
# int() chuyển NumPy scalar về số nguyên Python để in.
print("Mã:", labels)
```

Kết quả khi chạy:

```
Cảm biến: 20
Thiếu nhiệt độ: 2
Cần kiểm tra: 3
Mã: S07, S08, S16
```

Điều kiện chọn được áp theo **từng hàng có cả hai cột**. Đối với dữ liệu thiếu, chương trình không suy đoán 0 °C và không tự sửa mảng gốc.

6. Sai lầm thường gặp

- Chỉ số bắt đầu từ **0**. Mảng có ba hàng thì hàng thứ ba là chỉ số 2, không phải 3.
- `a[1:3]` lấy phần tử 1, 2 và **không** lấy 3. Lát cắt NumPy cơ bản thường là view: sửa view có thể sửa mảng gốc.
- Mặt nạ Boolean chọn phần tử/hàng thỏa điều kiện, còn Fancy indexing chọn vị trí theo danh sách chỉ số và thường tạo bản sao.
- Trong mảng 2-D, `X[:, 0]` là cột đầu, `X[0, :]` là hàng đầu: không đảo ý nghĩa hàng/cột.
- Không coi mã lỗi cảm biến `-1` hay giá trị tạm `0` là số đo hợp lệ khi phân tích dữ liệu thật.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. Where does Python/NumPy indexing begin? | Chỉ số Python/NumPy bắt đầu từ đâu?

A. -2 | -2

B. 0 | 0

C. The number of dimensions | Số chiều của mảng

D. 1 | 1

Đáp án: B

Giải thích: Chỉ số bắt đầu từ không: `a[0]` là phần tử đầu, còn `a[1]` là phần tử thứ hai. Chỉ số âm đếm ngược từ cuối.

Q2. For a 2-D array, what does `X[1, 0]` select? | Trong mảng hai chiều, `X[1, 0]` chọn gì?

A. Every row, first column | Mọi hàng, cột đầu

B. Row 2, column 1 | Hàng thứ hai, cột đầu tiên

C. All columns of the first row | Mọi cột của hàng đầu

D. Row 1, column 2 | Hàng đầu, cột thứ hai

Đáp án: B

Giải thích: Mảng 2D được truy cập theo `X[hàng, cột]`, với chỉ số bắt đầu từ không. Vì thế `X[1, 0]` là hàng hai, cột một.

Q3. What does `a[-1]` mean for a nonempty 1-D array? | `a[-1]` nghĩa là gì với mảng một chiều không rỗng?

A. An invalid index in all cases | Chỉ số luôn không hợp lệ

B. The last element | Phần tử cuối

C. All negative values | Mọi giá trị âm

D. The first element | Phần tử đầu

Đáp án: B

Giải thích: Với mảng không rỗng, chỉ số `-1` lấy phần tử cuối; ở `[4, 6, 8]` đó là `8`. Ngược lại, chỉ số `0` lấy phần tử đầu.

Q4. Which endpoint does a slice `a[1:4]` exclude? | Slice `a[1:4]` không lấy vị trí nào?

A. Index 3 | Chỉ số 3

B. Index 1 | Chỉ số 1

C. Index 4 | Chỉ số 4

D. Index 2 | Chỉ số 2

Đáp án: C

Giải thích: Chỉ số dừng của slice không được lấy: `a[1:4]` chọn vị trí 1, 2, 3 nếu có. Vị trí 4 không thuộc lát cắt.

Q5. What does `X[:, 0]` select? | `X[:, 0]` lấy gì?

A. Only the top-left value always | Luôn chỉ ô góc trên trái

B. All columns in the first row | Mọi cột ở hàng đầu

C. Only rows with value zero | Chỉ hàng có giá trị 0

D. All rows in the first column | Mọi hàng ở cột đầu

Đáp án: D

Giải thích: Trong `X[:, 0]`, dấu `:` chọn mọi hàng và `0` chọn cột đầu. Kết quả là dữ liệu theo cột, khác `X[0, :]` lấy một hàng.

Q6. What does `X[0, :]` select? | `X[0, :]` lấy gì?

A. All columns of the first row | Mọi cột của hàng đầu

B. The last row | Hàng cuối

C. All rows of the first column | Mọi hàng của cột đầu

D. The diagonal | Đường chéo

Đáp án: A

Giải thích: `X[0, :]` cố định hàng số không và giữ mọi cột. Với `[[1,2],[3,4]]`, kết quả là hàng đầu `[1,2]`, không phải cột đầu.

Q7. What does a Boolean mask contain? | Mặt nạ Boolean chứa gì?

A. Only positive integers | Chỉ số nguyên dương

B. Random floating-point numbers | Số thực ngẫu nhiên

C. Only column names | Chỉ tên cột

D. True/False values indicating which entries to keep | Giá trị True/False cho biết phần tử nào được giữ

Đáp án: D

Giải thích: Mặt nạ Boolean chứa các vị trí `True` và `False`, thường sinh từ phép so sánh. Với `[4, 8]`, điều kiện `a >= 7` chỉ chọn phần tử thứ hai.

Q8. What is selected by `a[a >= 7]`? | `a[a >= 7]` chọn gì?

A. All entries smaller than 7 | Mọi phần tử nhỏ hơn 7

B. Only the element at index 7 | Chỉ phần tử chỉ số 7

C. Elements at least 7 | Các phần tử từ 7 trở lên

D. The first seven entries | Bảy phần tử đầu

Đáp án: C

Giải thích: Biểu thức so giá trị với ngưỡng 7 rồi giữ phần tử thỏa điều kiện. Với `[4, 7, 9]`, kết quả là `[7, 9]`, không phải phần tử có chỉ số 7.

Q9. What does `a[[0, 2]]` illustrate? | `a[[0, 2]]` minh họa điều gì?

A. A logical AND operation | Phép AND logic

B. Slicing one continuous range | Cắt một đoạn liên tục

C. Fancy indexing with selected positions | Fancy indexing chọn các vị trí cụ thể

D. A matrix transpose | Chuyển vị ma trận

Đáp án: C

Giải thích: `a[[0, 2]]` dùng mảng chỉ số nguyên để chọn các vị trí cụ thể. Nó có thể lấy phần tử đầu và thứ ba không liền nhau, khác với lát cắt liên tục.

Q10. What is the difference between slicing and fancy indexing? | Slicing khác fancy indexing ở điểm nào?

A. Both always select entire matrices | Cả hai luôn lấy cả ma trận

B. Slicing requires Boolean values only | Slicing chỉ dùng Boolean

C. Slices describe ranges; fancy indexing specifies index arrays | Slice chỉ đoạn, fancy indexing chỉ mảng vị trí

D. Fancy indexing never uses integers | Fancy indexing không dùng số nguyên

Đáp án: C

Giải thích: Slice xác định dãy chỉ số có quy luật như `a[1:6:2]`; fancy indexing nguyên chỉ vị trí tùy ý như `a[[5, 1, 5]]`. Fancy indexing có thể lặp hoặc đổi thứ tự vị trí.

Q11. In NumPy, basic slicing generally returns what? | Basic slicing của NumPy thường trả về gì?

A. A view of the original data | View chia sẻ dữ liệu với mảng gốc

B. A guaranteed independent copy | Luôn là bản sao độc lập

C. A plain Python set | Một Set Python

D. A new CSV file | File CSV mới

Đáp án: A

Giải thích: Basic slicing thường trả view chia sẻ bộ nhớ nền. Sửa `part = a[1:3]` có thể ảnh hưởng giá trị gốc tương ứng; hãy dùng `.copy()` khi cần dữ liệu độc lập.

Q12. Does advanced integer-array indexing generally return a view? | Indexing bằng mảng số nguyên nâng cao thường tạo view không?

A. Yes, it always shares memory | Có, luôn chia sẻ bộ nhớ

B. No, it generally returns a copy | Không, thường trả bản sao

C. It converts the source to strings | Nó chuyển nguồn thành chuỗi

D. It only returns None | Nó chỉ trả None

Đáp án: B

Giải thích: Indexing nâng cao bằng số nguyên thường sao chép dữ liệu được chọn thay vì tạo view. Sửa `b = a[[0,2]]` thường không đổi `a`; gán trực tiếp `a[[0,2]] = ...` là thao tác khác.

Q13. How do you keep source data unchanged before assignment? | Làm sao giữ mảng gốc không đổi trước khi gán?

A. Make an explicit `copy()` | Tạo bản sao bằng `copy()`

B. Use `del X` first | Xóa X trước

C. Always modify a slice directly | Luôn sửa trực tiếp slice

D. Replace every value with zero | Thay mọi giá trị bằng 0

Đáp án: A

Giải thích: Lệnh `b = a.copy()` tạo bộ nhớ riêng trước khi sửa. Sau đó gán `b[0] = 9` không đổi `a[0]`, khác với sửa view dùng chung dữ liệu.

Q14. For shape (3,2), what does `X[:, 1]` return? | Nếu shape là (3,2), `X[:, 1]` trả về gì?

A. Three values from the second column | Ba giá trị ở cột thứ hai

B. Two values from the first row | Hai giá trị của hàng đầu

C. An array with five values | Mảng năm giá trị

D. One value in the center | Một giá trị giữa bảng

Đáp án: A

Giải thích: Mảng `(3,2)` có ba hàng hai cột. `X[:, 1]` lấy cột thứ hai qua cả ba hàng, cho kết quả có shape `(3,)`.

Q15. What does `X[:, [0, 2]]` mean when X has three columns? | `X[:, [0, 2]]` nghĩa là gì nếu X có ba cột?

A. First two rows, all columns | Hai hàng đầu, mọi cột

B. The sum of columns zero and two | Tổng hai cột 0 và 2

C. All rows and the first and third columns | Mọi hàng, cột thứ nhất và thứ ba

D. Only the second column | Chỉ cột thứ hai

Đáp án: C

Giải thích: Với ba cột, chỉ số `[0,2]` xác định cột thứ nhất và thứ ba. Dấu hai chấm giữ mọi hàng; biểu thức chọn hai cột chứ không cộng giá trị của chúng.

Q16. What is the effect of `a[:3]`? | `a[:3]` có tác dụng gì?

A. Take up to the first three entries | Lấy tối đa ba phần tử đầu

B. Take index 3 alone | Chỉ lấy phần tử số 3

C. Sort entries descending | Sắp phần tử giảm dần

D. Remove the first three entries | Xóa ba phần tử đầu

Đáp án: A

Giải thích: Bỏ start nghĩa là mặc định từ không, còn chỉ số dừng 3 không được lấy. Vì vậy `a[:3]` lấy tối đa ba phần tử đầu ngay cả khi mảng ngắn hơn; nó không xóa chúng.

Q17. Which operation filters entire rows based on the first column? | Lệnh nào lọc cả hàng theo điều kiện ở cột đầu?

A. `X.shape` | `X.shape`

B. `X[:, 0]` alone | Chỉ `X[:, 0]`

C. `X[0, 0]` | `X[0, 0]`

D. `X[X[:, 0] >= 8]` | `X[X[:, 0] >= 8]`

Đáp án: D

Giải thích: `X[:,0] >= 8` tạo một Boolean cho mỗi hàng bằng cách xét giá trị cột đầu. Dùng mask này để lấy từ `X` sẽ giữ nguyên các hàng thỏa điều kiện, không chỉ phần tử cột đầu.

Q18. What happens when a scalar index is beyond array bounds? | Khi chỉ số phần tử vượt giới hạn mảng thì sao?

A. It silently creates a new row | Nó âm thầm tạo hàng mới

B. NumPy raises IndexError | NumPy báo IndexError

C. It converts the array to a dictionary | Nó chuyển mảng thành Dictionary

D. It always returns zero | Nó luôn trả 0

Đáp án: B

Giải thích: Chỉ số đơn vượt phạm vi một trục mảng gây `IndexError`. Với mảng dài ba phần tử, chỉ số 3 không hợp lệ vì vị trí dương cuối là 2; truy cập không tự mở rộng mảng.

Q19. How does `X[mask]` differ from `X[[0,2]]`? | `X[mask]` khác `X[[0,2]]` ra sao?

A. Both necessarily select the same rows | Cả hai luôn lấy cùng hàng

B. A mask automatically sorts values | Mask tự sắp xếp giá trị

C. Indices always compare values to a threshold | Chỉ số luôn so giá trị với ngưỡng

D. A mask selects by condition; indices select by position | Mask chọn theo điều kiện; chỉ số chọn theo vị trí

Đáp án: D

Giải thích: Mask Boolean đánh dấu vị trí vượt qua phép kiểm tra, còn mảng chỉ số nguyên nêu thẳng các vị trí. Mask dựa trên điều kiện và `X[[0,2]]` không nhất thiết chọn cùng hàng.

Q20. Why state what rows and columns represent? | Vì sao cần nêu rõ ý nghĩa hàng và cột?

A. To make all arrays immutable | Để mọi mảng bất biến

B. To force arrays into 1-D | Để buộc mảng thành một chiều

C. To avoid using dtype | Để tránh dtype

D. To select the intended data without swapping dimensions | Để chọn đúng dữ liệu và không đảo chiều

Đáp án: D

Giải thích: Ý nghĩa từng trục quyết định lát cắt nào trả lời đúng vấn đề. Nếu hàng là học viên và cột là môn, `X[:,0]` lấy điểm môn đầu của mọi học viên, còn `X[0,:]` lấy điểm các môn của học viên đầu.

8. Nối bài tiếp theo

NumPy ndarray → Chọn hàng/cột → DataFrame/PCA.

PY-15 — NumPy tổng hợp, axis và broadcasting

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Aggregate — phép tổng hợp; **Axis** — trục tính.

`Index/Slice → Aggregate/Broadcast → Vectorization → AI`

Học xong dùng để làm gì? Tính trung bình, tổng và biến đổi hàng loạt các cột số có shape tương thích.

Kết nối kiến thức tiếp theo: PY-16 đổi shape, chuyển vị và nhân ma trận để xử lý dữ liệu dạng khối.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Aggregate	phép tổng hợp	Gom nhiều giá trị thành chỉ số thống kê
Axis	trục tính	Chiều dọc theo phép gom giá trị
Broadcasting	quảng bá mảng	Mở rộng kích thước tương thích khi tính
Vectorization	vector hóa	Tính cả mảng không cần viết vòng for từng phần tử

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`np.sum(X,axis=0)`	Tổng theo từng cột, axis=1 tổng theo hàng
`np.mean()/np.max()/np.min()`	Tổng hợp trung bình, cực đại và cực tiểu
`X + scalar / X + vector`	Broadcast nếu shape tương thích
`np.sqrt()/np.exp()`	Biến đổi từng phần tử, chú ý miền giá trị

4. Khái niệm liên quan

Sau khi biết tạo và chọn ô trong `ndarray`, ta cần **tổng hợp và biến đổi nhiều giá trị cùng lúc**. Phần này tiếp nối bảng doanh thu và điểm số: tổng hợp để rút ra thống kê, chọn `axis` để biết tổng hợp theo hàng hay cột, rồi dùng broadcasting và vectorization để xử lý đồng loạt mà không viết từng phép tính.

4.1. Aggregation — Tổng hợp từ nhiều giá trị

Từ vựng trong mục: Aggregation — tổng hợp; **Sum/mean** — tổng/trung bình.

Khi quản lý doanh thu, ta thường không cần xem từng giao dịch mà muốn biết **tổng**, **trung bình** hoặc **cao nhất**. Đó là **phép tổng hợp (aggregation)**: kết hợp nhiều giá trị thành một giá trị thống kê. Với doanh thu ba ngày `[10,20,30]` triệu đồng, `sum()` cho 60 triệu đồng; `mean()` cho 20 triệu đồng/ngày; `max()` cho 30 triệu đồng. Không nhầm tổng với trung bình.

No Note — Không chú thích

```
import numpy as np

sales = np.array([10, 20, 30])
print(sales.sum(), sales.mean(), sales.max())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So sánh sum(), mean(), max() trên cùng vector doanh thu.
# Dữ liệu: Vector ba sô´doanh thu 10,20,30 (đơn vị minh họa).
# import numpy as np đểsửdụng mảng và phép toán NumPy.
import numpy as np

# np.array() tạo ndarray từ list đểgọi các phương thức sô´học vector hóa.
sales = np.array([10, 20, 30])
# sum() cộng toàn bộ sô´trong vector.
# mean() lấy trung bình cộng, có thểlà sô´thực.
# max() lấy phần tửlớn nhất.
print(sales.sum(), sales.mean(), sales.max())
```

Kết quả khi chạy:

```
60 20.0 30
```

Tổng 60 triệu đồng, trung bình 20 triệu đồng/ngày và ngày cao nhất 30 triệu đồng.

4.2. Axis — Tổng hợp theo hàng hoặc cột

Từ vựng trong mục: Axis — trục tính; **Row/column** — hàng/cột.

Trong bảng doanh thu có **hai hàng cửa hàng** A, B và **ba cột ngày**, `sales.sum(axis=1)` tính tổng ba ngày trên từng **hàng cửa hàng**; còn `sales.sum(axis=0)` tính tổng hai cửa hàng theo từng **cột ngày**. `axis=0` gom **qua các hàng**, `axis=1` gom **qua các cột**. Vì vậy trước khi chọn axis, phải biết mỗi chiều đang mô tả gì.

No Note — Không chú thích

```
import numpy as np

sales = np.array([[10, 20, 30], [8, 12, 16]])
print(sales.sum(axis=1))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính tổng của từng hàng mảng bằng sum(axis=1).
# Dữ liệu: Hai hàng quan sát, ba cột giá trị; môĩ hàng gom thành một tổng riêng.
# import numpy as np đểsửdụng mảng và phép toán NumPy.
import numpy as np

# Mảng (2,3) có 2 hàng và 3 cột.
```

```
sales = np.array([[10, 20, 30], [8, 12, 16]])
# axis=1 gộp các CỘT trong từng hàng, tạo tổng cho mỗi hàng; axis=0 sẽ cộng theo từng cột.
print(sales.sum(axis=1))
```

Kết quả khi chạy:

```
[60 36]
```

Kết quả có hai phần tử vì mảng có hai hàng: cửa hàng A tổng 60 và B tổng 36 triệu đồng.

4.3. Broadcasting — Áp dụng phép toán theo chiều tương thích

Từ vựng trong mục: Broadcasting — quảng bá mảng; **Shape** — kích thước.

Broadcasting cho phép NumPy thực hiện phép toán giữa các đối tượng khác kích thước **nếu các chiều tương thích**. Ví dụ một trường cộng 1 điểm chuyên cần cho từng điểm trong bảng hai học viên × ba môn: phép `scores + 1` tự áp dụng 1 vào mọi ô, không cần vòng `for`. Khi một chiều có kích thước 1 hoặc bằng chiều tương ứng, NumPy thường mở rộng được; các chiều không tương thích sẽ báo `ValueError`.

No Note — Không chú thích

```
import numpy as np

scores = np.array([[7, 8, 9], [6, 7, 8]])
print(scores + 1)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Broadcast một số 1 lên từng ô của bảng điểm 2×3.
# Dữ liệu: Hai hàng học viên × ba cột môn; cộng 1 cho mỗi ô điểm.
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np

# Tạo bảng điểm NumPy 2 hàng × 3 cột.
scores = np.array([[7, 8, 9], [6, 7, 8]])
# Broadcasting tự áp cùng scalar 1 vào TẤT CẢ ô mà không cần for; mảng gốc scores không bị sửa.
print(scores + 1)
```

Kết quả khi chạy:

```
[[ 8  9 10]
 [ 7  8  9]]
```

Số 1 được cộng vào cả sáu ô, nhưng shape vẫn là 2 hàng × 3 cột.

4.4. Vectorization — Tính trực tiếp trên mảng

Từ vựng trong mục: Vectorization — vector hóa; **Element-wise** — theo từng phần tử.

Vectorization là viết phép tính trực tiếp trên toàn mảng thay vì tự lặp từng ô bằng Python. Khi ba kênh cảm biến có giá trị `[10,20,30]` và hệ số hiệu chỉnh `[1.0,0.5,2.0]`, phép `signals * factors` nhân các phần tử **cùng vị trí**: 10×1, 20×0,5 và 30×2. Đây là phép **nhân từng phần tử**, khác phép nhân ma trận `@`.

No Note — Không chú thích

```
import numpy as np

signals = np.array([10, 20, 30])
factors = np.array([1.0, 0.5, 2.0])
print(signals * factors)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhân từng phần tử của hai vector có cùng độ dài.
# Dữ liệu: signals có 3 giá trị, factors có 3 hệ số tương ứng từng vị trí.
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np

# Mảng gồm ba tín hiệu số với thứ tự cố định.
signals = np.array([10, 20, 30])
# Mỗi hệ số nhân tương ứng cùng vị trí với tín hiệu; 1 giữ nguyên, 0.5 giảm nửa, 2 tăng gấp đôi.
factors = np.array([1.0, 0.5, 2.0])
# Hai vector cùng chiều dài nhân THEO TỪNG VỊ TRÍ (10×1,20×0.5,30×2), không phải tích vô hướng.
print(signals * factors)
```

Kết quả khi chạy:

```
[10. 10. 60.]
```

Ba số là tín hiệu sau khi áp dụng hệ số riêng cho từng kênh; NumPy không cộng các kênh lại.

5. Năm tình huống Aggregate/Broadcast — tăng từ tổng một chiều đến báo cáo nhiều cột

Tình huống 1 — Cấp 1: Tổng hợp doanh thu 10 ngày bằng sum/min/max

Từ vựng trong mục: Aggregate — tổng hợp; **Sum** — tổng; **Mean** — trung bình.

Tình huống thực tế: Chủ cửa hàng xem bảng doanh thu **10 ngày liên tiếp**, mỗi số là tổng tiền của một ngày, đơn vị **nghìn đồng/ngày**. Chẳng hạn ngày đầu thu 120 nghìn đồng, ngày thứ hai 145 nghìn đồng. Người quản lý cần biết tổng doanh thu của cả 10 ngày, trung bình một ngày thu bao nhiêu và ngày có doanh thu thấp nhất, cao nhất để nhìn sơ bộ sự thay đổi.

Vì sao chọn cách này? NumPy có các hàm **tổng hợp** để lấy thông tin từ cả một mảng mà không cần viết vòng lặp cộng tay. `sum()` cộng 10 khoản doanh thu để ra **tổng 10 ngày**; `mean()` cho biết số tiền trung bình **trong một ngày**; `min()` và `max()` tìm mức thấp nhất và cao nhất đã ghi. Đây là bốn câu hỏi khác nhau: doanh thu cả kỳ có đơn vị nghìn đồng, còn trung bình có thể hiểu là nghìn đồng/ngày. Không được coi mức cao nhất là tổng doanh thu hoặc nhầm 10 phần tử với 10 nhóm hàng.

Quy trình:

1. Tạo mảng 10 số doanh thu theo ngày.
2. Tính tổng, trung bình, nhỏ nhất và lớn nhất bằng phép tổng hợp.
3. In số quan sát và số liệu theo đơn vị đã nêu.

No Note — Không chú thích

```
import numpy as np
sales = np.array([120,145,130,160,180,140,155,175,190,165], dtype=float)
if sales.shape != (10,):
    raise ValueError("Cần đủ 10 ngày bán hàng.")
print("Ngày:", sales.size)
print("Tổng (nghìn đồng):", int(sales.sum()))
print(f"Trung bình/ngày: {sales.mean():.1f}")
print("Thấp nhất:", int(sales.min()))
print("Cao nhất:", int(sales.max()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tổng hợp min, max, tổng và trung bình doanh số của 10 ngày.
# Dữ liệu: 10 doanh số ngày (nghìn đồng); mảng float cho phép tính trung bình có số lẻ.
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np
# dtype=float tạo vector 10 ngày ở dạng số thực.
sales = np.array([120,145,130,160,180,140,155,175,190,165], dtype=float)
# shape (10,) là mảng một chiều 10 phần tử, khác (10,1) hai chiều.
if sales.shape != (10,):
    # raise ValueError báo bảng dữ liệu sai shape, ngừng xử lý nếu kiểm tra thất bại.
    raise ValueError("Cần đủ 10 ngày bán hàng.")
# size đếm 10 ngày, không phải kích thước byte.
print("Ngày:", sales.size)
# sum() cộng toàn bộ doanh số (nghìn đồng); int() đổi scalar NumPy sang int Python.
print("Tổng (nghìn đồng):", int(sales.sum()))
# mean() tính doanh số trung bình một ngày; :.1f định dạng một số lẻ.
print(f"Trung bình/ngày: {sales.mean():.1f}")
# min() lấy mức bán thấp nhất của 10 ngày.
print("Thấp nhất:", int(sales.min()))
# max() lấy mức cao nhất trong 10 ngày.
print("Cao nhất:", int(sales.max()))
```

Kết quả khi chạy:

```
Ngày: 10
Tổng (nghìn đồng): 1560
Trung bình/ngày: 156.0
Thấp nhất: 120
Cao nhất: 190
```

Đây là tổng hợp dãy 1 chiều: chỉ có mười số doanh thu, chưa có hàng–cột để chọn `axis`.

Tình huống 2 — Cấp 2: Phân biệt trung bình theo môn và theo học viên

Từ vựng trong mục: Axis 0 — theo cột; **Axis 1** — theo hàng; **Argmax** — chỉ số cực đại.

Tình huống thực tế: Một lớp có **12 học viên H01–H12**, mỗi người được chấm **Toán, Văn, Python** theo thang **10 điểm**, lưu mỗi học viên trên một hàng. Giáo viên muốn biết điểm trung bình của **từng môn trên toàn lớp**, đồng thời tìm **học viên có trung bình ba môn cao nhất**. Đây là hai cách nhìn khác nhau trên cùng một bảng điểm.

Vì sao chọn cách này? Trục `axis` cho biết NumPy gộp theo hướng nào. Trong bảng 12 hàng học viên và 3 cột môn, `mean(axis=0)` gộp **theo từng cột** để cho ba điểm trung bình Toán, Văn, Python. Ngược lại, `mean(axis=1)` gộp **ba môn của mỗi hàng**, nên trả 12 trung bình học viên. `argmax()` tìm **vị trí** của trung bình học viên cao nhất, sau đó cộng 1 để ghép mã H01… Đừng dùng trung bình theo môn để chọn người học giỏi nhất: đó là số của cả lớp, không phải điểm một học viên.

Quy trình:

1. Tạo bảng `(12,3)` và xác định rõ nghĩa hàng/cột.
2. Tính trung bình cột bằng `axis=0`, tính trung bình hàng bằng `axis=1`.
3. Tìm học viên có trung bình cao nhất và in kết quả có nhãn.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[6,5,7],[7,6,9],[8,7,7],[9,8,9],[6,9,7],[7,5,9],[8,6,7],[9,7,9],[6,8,7],[7,9,9],
[8,5,7],[9,6,9]], dtype=float)
by_subject = scores.mean(axis=0)
by_student = scores.mean(axis=1)
best = int(by_student.argmax())
if scores.shape != (12, 3):
    raise ValueError("Bảng phải có 12 học viên và 3 môn.")
print("Học viên:", scores.shape[0])
print(f"Toán: {by_subject[0]:.2f}")
print(f"Văn: {by_subject[1]:.2f}")
print(f"Python: {by_subject[2]:.2f}")
print("Điểm TB cao nhất:", f"H{best+1:02d}", f"{by_student[best]:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính điểm trung bình theo môn/theo học viên và tìm học viên cao nhất.
# Dữ liệu: 12 hàng học viên H01..H12, 3 cột Toán/Văn/Python, điểm trên thang 10.
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np
# 12 hàng học viên × 3 cột môn, dtype=float hỗ trợ trung bình thập phân.
scores = np.array([[6,5,7],[7,6,9],[8,7,7],[9,8,9],[6,9,7],[7,5,9],[8,6,7],[9,7,9],[6,8,7],[7,9,9],
[8,5,7],[9,6,9]], dtype=float)
# axis=0 gom qua các HÀNG để lấy một trung bình cho mỗi CỘT môn.
by_subject = scores.mean(axis=0)
# axis=1 gom qua các CỘT, cho trung bình của mỗi HÀNG học viên.
by_student = scores.mean(axis=1)
# argmax() trả chỉ số 0-based học viên có trung bình cao nhất, không trả giá trị điểm.
best = int(by_student.argmax())
# shape (12,3) đối chiếu 12 người và 3 môn.
if scores.shape != (12, 3):
    # raise ValueError báo bảng dữ liệu sai shape, ngừng xử lý nếu kiểm tra thất bại.
    raise ValueError("Bảng phải có 12 học viên và 3 môn.")
# shape (12,3) đối chiếu 12 người và 3 môn.
print("Học viên:", scores.shape[0])
# [0] là trung bình Toán, [1] Văn, [2] Python; :.2f in hai số thập phân.
print(f"Toán: {by_subject[0]:.2f}")
# int() đổi NumPy scalar sang số nguyên Python khi in.
print(f"Văn: {by_subject[1]:.2f}")
# int() đổi NumPy scalar sang số nguyên Python khi in.
print(f"Python: {by_subject[2]:.2f}")
# best là chỉ số 0-based nên +1 thành số thứ tự người, :02d thêm 0 để tạo mã H01..H12.
# Lấy đúng điểm trung bình cao nhất tại chỉ số argmax tìm được.
print("Điểm TB cao nhất:", f"H{best+1:02d}", f"{by_student[best]:.2f}")
```

Kết quả khi chạy:

```
Học viên: 12
Toán: 7.50
Văn: 6.75
Python: 8.00
Điểm TB cao nhất: H04 8.67
```

Cả hai phép trung bình đều hợp lệ về toán nhưng **khác đối tượng**; khi chọn sai `axis`, báo cáo sẽ trả sai số lượng kết quả.

Tình huống 3 — Cấp 3: Broadcast điểm cộng cho 15 học viên nhưng không vượt 10

Từ vựng trong mục: Broadcasting — cộng vector theo cột; **Clip** — giới hạn; **Vectorization** — tính hàng loạt.

Tình huống thực tế: Trường có bảng điểm **15 học viên**, mỗi hàng chứa hai cột **Toán và Python** trên thang **10 điểm**. Theo quy định minh họa, mọi học viên được cộng **1 điểm Toán** và **0,5 điểm Python** vì chuyên cần, nhưng điểm một môn không được vượt quá 10. Giáo viên cần tạo bảng điểm sau cộng, xem kết quả của học viên đầu tiên và đếm số người đạt 10 ở ít nhất một môn, trong khi giữ bảng điểm gốc để đối chiếu.

Vì sao chọn cách này? Broadcasting là cơ chế NumPy tự áp dụng một dãy số ngắn lên nhiều hàng có cùng cấu trúc. Dãy cộng điểm `[1.0, 0.5]` có một phần cho cột Toán và một phần cho cột Python; NumPy cộng đúng hai mức ấy cho **mỗi trong 15 học viên**, không lấy mức 1 để cộng cho cả hai môn. `np.minimum(..., 10)` giới hạn mọi điểm cuối cùng ở mức 10, vì người đã có 9,5 điểm Python không được tăng thành 10,0 cộng thêm rồi vượt thang. `np.any(adjusted == 10, axis=1)` kiểm tra **trên từng học viên** xem có ít nhất một môn đạt 10. Bảng gốc không bị sửa vì kết quả được tính thành mảng mới.

Quy trình:

1. Tạo mảng 15 × 2 kiểu float và vector cộng theo môn.
2. Broadcast phép cộng cho mọi hàng rồi giới hạn bằng `np.minimum`.
3. Đếm số hàng có ít nhất một điểm đạt 10 và kiểm tra mảng gốc không bị sửa.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[7,5],[8,6],[9,7],[10,8],[7,9],[8,10],[9,5],[10,6],[7,7],[8,8],[9,9],[10,10],
[7,5],[8,6],[9,7]], dtype=float)
bonus = np.array([1.0, 0.5])
adjusted = np.minimum(scores + bonus, 10)
if scores.shape != (15, 2) or adjusted.shape != (15, 2):
    raise ValueError("Cần đúng 15 học viên với hai môn trước và sau cộng điểm.")
print("Học viên:", scores.shape[0])
print("Điểm H01 sau cộng:", f"{adjusted[0,0]:.1f}", f"{adjusted[0,1]:.1f}")
print("Đạt 10 ít nhất một môn:", int(np.any(adjusted == 10, axis=1).sum()))
print("Gốc H01:", f"{scores[0,0]:.1f}", f"{scores[0,1]:.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Broadcast điểm thưởng hai môn cho 15 học viên, giới hạn tối đa 10.
# Dữ liệu: 15 hàng học viên và 2 môn; bonus = [1.0,0.5] là điểm cộng cho hai cột.
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np
# 15 hàng học viên × hai cột điểm; dtype=float giữ số lẻ.
scores = np.array([[7,5],[8,6],[9,7],[10,8],[7,9],[8,10],[9,5],[10,6],[7,7],[8,8],[9,9],[10,10],
[7,5],[8,6],[9,7]], dtype=float)
# Vector hai điểm cộng, 1.0 cho cột 0 và 0.5 cho cột 1.
bonus = np.array([1.0, 0.5])
# Broadcasting vector có 2 ô qua 15 hàng: mỗi môn được cộng riêng một lượng.
# np.minimum(...,10) chọn giá trị nhỏ hơn theo từng ô, không cho điểm mới quá 10; không sửa scores
gốc.
adjusted = np.minimum(scores + bonus, 10)
# Kiểm shape (15,2) sau broadcast và cắt ngưỡng.
```

```
if scores.shape != (15, 2) or adjusted.shape != (15, 2):
    # raise ValueError báo bảng dữ liệu sai shape, ngừng xử lý nếu kiểm tra thất bại.
    raise ValueError("Cần đúng 15 học viên với hai môn trước và sau cộng điểm.")
# int() đổi NumPy scalar sang số nguyên Python khi in.
print("Học viên:", scores.shape[0])
# [0,0] là điểm môn đầu của H01, [0,1] là môn thứ hai; :.1f định dạng.
print("Điểm H01 sau cộng:", f"{adjusted[0,0]:.1f}", f"{adjusted[0,1]:.1f}")
# == 10 tạo mask từng ô; np.any(...,axis=1) trả True nếu MỘT TRONG hai môn của cùng học viên đạt
10.
# sum() trên mask Boolean đếm số học viên có ít nhất một môn chạm 10.
print("Đạt 10 ít nhất một môn:", int(np.any(adjusted == 10, axis=1).sum()))
# Đọc điểm H01 từ bảng GỐC để xác minh phép broadcast không làm thay đổi điểm ban đầu.
print("Gốc H01:", f"{scores[0,0]:.1f}", f"{scores[0,1]:.1f}")
```

Kết quả khi chạy:

```
Học viên: 15
Điểm H01 sau cộng: 8.0 5.5
Đạt 10 ít nhất một môn: 8
Gốc H01: 7.0 5.0
```

Đây là tính vector hóa theo cột: nhờ shape tương thích, cùng một vector hai mức cộng được áp dụng cho 15 học viên mà không phải viết 30 phép tính.

Tình huống 4 — Cấp 4: Hiệu chỉnh 18 cảm biến ba kênh bằng broadcasting

Từ vựng trong mục: Element-wise — theo ô; **Calibration** — hiệu chỉnh; **nanmean** — trung bình bỏ ô thiếu.

Tình huống thực tế: Trạm môi trường thu **18 lượt đo**; mỗi lượt ghi nhiệt độ (**°C**), độ ẩm (**%**) và áp suất (**kPa**). Hai lượt thiếu riêng số áp suất, còn nhiệt độ và độ ẩm vẫn có. Biên bản hiệu chuẩn trong bài yêu cầu với mọi lượt: cộng **0,5 °C** vào nhiệt độ, trừ **2 điểm phần trăm** khỏi độ ẩm và cộng **1 kPa** vào áp suất. Nhân viên muốn tính trung bình sau hiệu chỉnh của từng đại lượng mà không biến hai ô thiếu thành số.

Vì sao chọn cách này? Ba cột đo có **đơn vị và ý nghĩa khác nhau**, nên hiệu chỉnh phải áp dụng đúng từng cột. Dãy `[0.5, -2.0, 1.0]` có ba mức sửa, tương ứng tăng 0,5 °C, giảm 2 điểm phần trăm độ ẩm và tăng 1 kPa. Nhờ **broadcasting**, NumPy dùng lại dãy ba mức ấy trên đủ 18 hàng; không phải cộng cùng một số vào tất cả cột. Ô áp suất không có kết quả được lưu là `np.nan`, nên sau khi cộng vẫn là NaN chứ không biến thành áp suất 1 kPa. `np.nanmean(axis=0)` bỏ qua riêng các ô thiếu và trả ba trung bình riêng, không cộng nhiệt độ với độ ẩm hoặc áp suất.

Quy trình:

1. Tạo mảng `(18,3)` số thực, chuyển ô áp suất thiếu thành NaN.
2. Tạo vector ba độ lệch và broadcast phép cộng theo cột.
3. Tính ba trung bình sau hiệu chỉnh, đếm chính xác hai ô thiếu.

No Note — Không chú thích

```
import numpy as np
raw = [[20,50,100],[22,53,105],[24,56,110],[26,59,115],[28,62,None],[20,65,105],[22,50,110],
[24,53,115],[26,56,100],[28,59,105],[20,62,110],[22,65,115],[24,50,100],[26,53,105],[28,56,None],
[20,59,115],[22,62,100],[24,65,105]]
rows = []
for temp, humidity, pressure in raw:
    if pressure is None:
        clean_pressure = np.nan
```

```
    else:
        clean_pressure = pressure
    rows.append([temp, humidity, clean_pressure])
measurements = np.array(rows, dtype=float)
calibration = np.array([0.5, -2.0, 1.0])
corrected = measurements + calibration
means = np.nanmean(corrected, axis=0)
if corrected.shape != (18, 3):
    raise ValueError("Cần đủ 18 lượt đo và 3 thông số́.")
print("Số́ mẫu:", corrected.shape[0])
print("Thiếú áp suất:", int(np.isnan(corrected[:, 2]).sum()))
print(f"Nhiệt độ (°C): {means[0]:.2f}")
print(f"Độ ẩm (%): {means[1]:.2f}")
print(f"Áp suất (kPa): {means[2]:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Hiệu chỉnh 18 lần đo ba cảm biến theo vector bù, tính trung bình bỏ NaN.
# Dữ liệu: 18 hàng đo (nhiệt °C, độ ẩm %, áp suất kPa); cột áp suất có 2 None; calibration = [+0.5
°C,-2 %, +1 kPa].
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np
# 18 hàng (nhiệt °C, ẩm %, áp suất kPa); None chỉ 2 áp suất thiếú.
raw = [[20,50,100],[22,53,105],[24,56,110],[26,59,115],[28,62,None],[20,65,105],[22,50,110],
[24,53,115],[26,56,100],[28,59,105],[20,62,110],[22,65,115],[24,50,100],[26,53,105],[28,56,None],
[20,59,115],[22,62,100],[24,65,105]]
rows = []
for temp, humidity, pressure in raw:
    # is None phát hiện số́ đo áp suất bị thiếú.
    if pressure is None:
        # np.nan là marker số́ thực cho dữ liệu thiếú, KHÔNG mặc định thành 0 kPa.
        clean_pressure = np.nan
    else:
        clean_pressure = pressure
    # append() thêm 1 hàng ba trường theo thứ tự của bảng.
    rows.append([temp, humidity, clean_pressure])
# dtype=float cho phép ndarray chứa cả số́ thực và np.nan.
measurements = np.array(rows, dtype=float)
# Vector hiệu chỉnh có ba ô khớp ba cột: +0.5 °C, -2 điểm phần trăm ẩm, +1 kPa.
calibration = np.array([0.5, -2.0, 1.0])
# Broadcasting cộng vector hiệu chỉnh (shape (3,)) vào từng hàng (18,3).
corrected = measurements + calibration
# nanmean bỏ NaN trong mỗi cột riêng trước khi tính trung bình; axis=0 trả 3 trung bình theo cột.
means = np.nanmean(corrected, axis=0)
# shape (18,3) xác minh số́ hàng/cột còn nguyên.
if corrected.shape != (18, 3):
    # raise ValueError báo bảng dữ liệu sai shape, ngừng xử lý nếú kiểm tra thất bại.
    raise ValueError("Cần đủ 18 lượt đo và 3 thông số́.")
# shape (18,3) xác minh số́ hàng/cột còn nguyên.
print("Số́ mẫu:", corrected.shape[0])
# [:,2] chọn cột áp suất của mọi hàng; isnan() tạo mask thiếú, sum() đếm 2 giá trị.
print("Thiếú áp suất:", int(np.isnan(corrected[:, 2]).sum()))
# means[0] nhiệt độ °C, [1] độ ẩm %, [2] áp suất kPa; :.2f in hai chữ số́ lẻ.
print(f"Nhiệt độ (°C): {means[0]:.2f}")
# int() đổi NumPy scalar sang số́ nguyên Python khi in.
print(f"Độ ẩm (%): {means[1]:.2f}")
# int() đổi NumPy scalar sang số́ nguyên Python khi in.
print(f"Áp suất (kPa): {means[2]:.2f}")
```

Kết quả khi chạy:

```
Số́ mẫu: 18
Thiếú áp suất: 2
```

```
Nhiệt độ (°C): 24.17
Độ ẩm (%): 55.50
Áp suất (kPa): 108.19
```

Mỗi độ lệch có **đơn vị riêng** và tác động đúng cột. Khi áp suất thiếu, kết quả vẫn thiếu sau cộng vector — đúng hơn là giả định có số đo.

Tình huống 5 — Cấp 5: Phát hành báo cáo 20 cửa hàng với giá/ưu đãi theo cột

Từ vựng trong mục: Broadcasting — mở rộng theo cột; **Aggregation** — tổng hợp; **Axis** — trục tính.

Tình huống thực tế: Chuỗi bán lẻ gồm **20 cửa hàng C01–C20**, mỗi cửa hàng bán ba nhóm sản phẩm **A, B, C**. Bảng ghi **số món đã bán** của từng nhóm theo hàng cửa hàng. Trong dữ liệu giả lập, một món nhóm A giá **10 nghìn đồng**, B giá **20 nghìn đồng**, C giá **30 nghìn đồng**; khách lần lượt trả 100%, 90%, 80% giá niêm yết. Quản lý muốn biết doanh thu sau giảm giá của **từng cửa hàng**, tổng của cả hệ thống và mã cửa hàng có doanh thu lớn nhất.

Vì sao chọn cách này? Bảng `quantities` có **20 hàng cửa hàng và ba cột số món** A, B, C. Dãy giá `[10, 20, 30]` và dãy tỷ lệ thanh toán `[1.0, 0.9, 0.8]` được **broadcast** vào từng hàng: số món của nhóm nào được nhân với giá và tỷ lệ của **đúng nhóm đó**. Ví dụ một món B giá 20 nghìn đồng chỉ thu 90%, tức **18 nghìn đồng**, không dùng tỷ lệ 80% của nhóm C. Sau khi có tiền từng nhóm, `sum(axis=1)` cộng ba cột thành **20 tổng theo cửa hàng**; tiếp tục cộng các tổng này để ra toàn hệ thống. `argmax()` chỉ vị trí cửa hàng có doanh thu lớn nhất; vì vị trí bắt đầu từ 0, phải đổi thành mã C01… đúng với bảng gốc.

Quy trình:

1. Tạo mảng 20 × 3 số lượng theo thứ tự nhóm A/B/C.
2. Nhân với hai vector giá và tỷ lệ có thứ tự cột thống nhất.
3. Cộng `axis=1` để có 20 doanh thu; chọn cửa hàng cao nhất, cộng tổng toàn chuỗi và kiểm tra kích thước.

No Note — Không chú thích

```
import numpy as np
quantities = np.array([[1,2,1],[2,3,3],[3,4,5],[4,2,2],[1,3,4],[2,4,1],[3,2,3],[4,3,5],[1,4,2],
[2,2,4],[3,3,1],[4,4,3],[1,2,5],[2,3,2],[3,4,4],[4,2,1],[1,3,3],[2,4,5],[3,2,2],[4,3,4]],
dtype=float)
prices = np.array([10, 20, 30], dtype=float)
pay_rates = np.array([1.0, 0.9, 0.8])
amounts = quantities * prices * pay_rates
per_shop = amounts.sum(axis=1)
peak = int(per_shop.argmax())
if amounts.shape != (20, 3) or per_shop.shape != (20,):
    raise ValueError("Cần đủ 20 cửa hàng, ba mặt hàng và doanh thu từng cửa hàng.")
print("Cửa hàng:", quantities.shape[0])
print(f"Tổng 20 cửa hàng (nghìn đồng): {per_shop.sum():.1f}")
print("Doanh thu lớn nhất:", f"C{peak+1:02d}", f"{per_shop[peak]:.1f}")
print(f"C01 (nghìn đồng): {per_shop[0]:.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Broadcast giá và hệ số trả tiền theo ba mặt hàng để tính doanh thu 20 cửa hàng.
# Dữ liệu: 20 hàng cửa hàng, 3 cột số món; prices nghìn đồng/món và pay_rates 1.0/0.9/0.8 cho từng
mặt hàng.
# import numpy as np để sử dụng mảng và phép toán NumPy.
import numpy as np
```

```
# 20 hàng cửa hàng × 3 cột số´món từng loại, dtype=float giúp nhân hệ số´.
quantities = np.array([[1,2,1],[2,3,3],[3,4,5],[4,2,2],[1,3,4],[2,4,1],[3,2,3],[4,3,5],[1,4,2],
[2,2,4],[3,3,1],[4,4,3],[1,2,5],[2,3,2],[3,4,4],[4,2,1],[1,3,3],[2,4,5],[3,2,2],[4,3,4]],
dtype=float)
# Vector 3 đơn giá nghìn đồng/món, tương ứng từng cột của quantities.
prices = np.array([10, 20, 30], dtype=float)  # nghìn đồng/món
# Hệ số´tiền nhận thực tế´1.0/0.9/0.8, tương ứng 100%/90%/80% theo loại hàng.
pay_rates = np.array([1.0, 0.9, 0.8])
# Broadcast lần lượt hai vector (3,) qua 20 hàng: số´món × giá × tỷ lệ nhận, ra doanh thu nghìn
đồng theo cột.
amounts = quantities * prices * pay_rates
# axis=1 cộng ba mặt hàng theo từng cửa hàng, trả vector 20 tổng doanh thu.
per_shop = amounts.sum(axis=1)
# argmax() trả chỉ số´(bắt đầu 0) của cửa hàng có doanh thu lớn nhất.
peak = int(per_shop.argmax())
# Kiểm shape (20,3) và per_shop (20,) sau cộng theo hàng.
if amounts.shape != (20, 3) or per_shop.shape != (20,):
    # raise ValueError báo bảng dữ liệu sai shape, ngừng xử lý nếu kiểm tra thất bại.
    raise ValueError("Cần đủ 20 cửa hàng, ba mặt hàng và doanh thu từng cửa hàng.")
# int() đổi NumPy scalar sang số´nguyên Python khi in.
print("Cửa hàng:", quantities.shape[0])
# sum() cộng doanh thu của cả 20 cửa hàng, đơn vị nghìn đồng.
print(f"Tổng 20 cửa hàng (nghìn đồng): {per_shop.sum():.1f}")
# peak+1 chuyển index 0-based thành số´cửa hàng; :02d tạo mã hai chữ số´.
# Lấy doanh thu của cửa hàng có chỉ số´cao nhất theo argmax.
print("Doanh thu lớn nhất:", f"C{peak+1:02d}", f"{per_shop[peak]:.1f}")
# [0] là doanh thu cửa hàng đầu C01, dùng đối chiếu.
print(f"C01 (nghìn đồng): {per_shop[0]:.1f}")
```

Kết quả khi chạy:

```
Cửa hàng: 20
Tổng 20 cửa hàng (nghìn đồng): 3002.0
Doanh thu lớn nhất: C03 222.0
C01 (nghìn đồng): 70.0
```

Đây là tổng hợp cao nhất: làm việc theo hàng và cột có đơn vị, áp giá/ưu đãi bằng broadcasting, rồi tổng hợp trên đúng trục. Dữ liệu minh họa không đại diện doanh thu thật.

6. Sai lầm thường gặp

- Với mảng hai chiều, `axis=0` tổng hợp **theo cột** (gom các hàng), còn `axis=1` tổng hợp **theo hàng** (gom các cột). Không đoán axis chỉ từ tên hàm.
- `sum`, `mean`, `max` có ý nghĩa khác nhau: tổng, trung bình và cực đại, không thể thay thế tùy tiện.
- Broadcasting chỉ hoạt động khi **shape tương thích**. Hai mảng có chiều không tương thích có thể phát sinh `ValueError`.
- Dấu `*` của NumPy nhân **từng phần tử**, không phải phép nhân ma trận `@`.
- Khi cộng điểm, phải áp dụng giới hạn tối đa nếu quy định yêu cầu, ví dụ `np.minimum(scores + 1, 10)`.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is aggregation in NumPy? | Phép tổng hợp trong NumPy là gì?

A. Reading every file in a folder | Đọc mọi file trong thư mục

B. Converting all arrays to strings | Đổi mọi mảng thành chuỗi

C. Creating a new class by inheritance | Tạo lớp con nhờ kế thừa

D. Combining multiple values into summary statistics | Kết hợp nhiều giá trị thành thống kê tổng hợp

Đáp án: D

Giải thích: Tổng hợp gộp nhiều giá trị số thành thống kê như `sum`, `mean` hoặc `max`. Với `[2, 4, 6]`, trung bình `4` tóm tắt ba giá trị thay vì giữ từng ô.

Q2. How does `sum()` differ from `mean()`? | `sum()` khác `mean()` ra sao?

A. Both always return the same result | Cả hai luôn bằng nhau

B. sum totals values; mean computes their average | sum cộng tổng; mean tính trung bình

C. sum sorts; mean deletes values | sum sắp xếp; mean xóa giá trị

D. mean returns the largest number | mean trả số lớn nhất

Đáp án: B

Giải thích: `sum` cộng tất cả giá trị được chọn, còn `mean` lấy tổng chia cho số giá trị. Với `[2, 4, 6]`, tổng bằng `12` nhưng trung bình bằng `4`; hai phép không thường bằng nhau.

Q3. What does `max()` return? | `max()` trả về gì?

A. The arithmetic average | Trung bình cộng

B. The sum of every negative value | Tổng các số âm

C. The largest value overall or along a specified axis | Giá trị lớn nhất toàn mảng hoặc theo trục được chỉ định

D. The number of columns only | Chỉ số cột

Đáp án: C

Giải thích: `max` chọn giá trị lớn nhất của mảng hoặc riêng theo trục được chỉ định. Với `[[1, 7], [5, 3]]`, cực đại toàn mảng là `7`, khác `mean` dùng để tính trung bình.

Q4. In a 2-D array, what does `sum(axis=0)` return? | Trong mảng hai chiều, `sum(axis=0)` trả về gì?

A. One sum per row | Một tổng cho mỗi hàng

B. One sum per column | Một tổng cho mỗi cột

C. Only the first row | Chỉ hàng đầu

D. A boolean mask | Mặt nạ Boolean

Đáp án: B

Giải thích: Với mảng 2D, `axis=0` gộp theo chiều hàng và để lại mỗi cột một kết quả. Với `[[1, 2], [3, 4]]`, tổng cột là `[4, 6]`, không phải tổng hàng `[3, 7]`.

Q5. In a 2-D array, what does `sum(axis=1)` return? | Trong mảng hai chiều, `sum(axis=1)` trả về gì?

A. The data type of values | Kiểu dữ liệu

B. One sum per column | Một tổng cho mỗi cột

C. Only a scalar index | Chỉ một chỉ số

D. One sum per row | Một tổng cho mỗi hàng

Đáp án: D

Giải thích: `axis=1` gộp các cột trong từng hàng. Với `[[1, 2], [3, 4]]`, `sum(axis=1)` cho tổng hàng `[3, 7]`; `axis=0` lại cho tổng theo cột.

Q6. What does broadcasting allow? | Broadcasting cho phép làm gì?

A. Automatically converting all arrays to CSV | Tự chuyển mọi mảng thành CSV

B. Operating on arrays with compatible shapes | Thực hiện phép toán trên các mảng có shape tương thích

C. Replacing a function with a class | Thay hàm bằng lớp

D. Using any shapes without restrictions | Dùng mọi shape không giới hạn

Đáp án: B

Giải thích: Broadcasting ghép các chiều có kích thước tương thích để tính theo phần tử mà không phải viết vòng lặp Python. Ví dụ ma trận `(2,3)` cộng được vector `(3,)` trên các hàng, nhưng vector `(2,)` không khớp chiều cuối bằng 3.

Q7. Why does `a + 1` work on a numeric ndarray? | Vì sao `a + 1` hoạt động với ndarray số?

A. The array is converted to a string | Mảng bị chuyển thành chuỗi

B. Only the first element is changed | Chỉ phần tử đầu thay đổi

C. The scalar broadcasts to every array element | Số 1 được broadcast tới từng phần tử

D. It performs matrix multiplication | Nó nhân ma trận

Đáp án: C

Giải thích: Số đơn không có trục mảng gây xung đột nên NumPy broadcast nó tới mọi phần tử. Với `a=[2,4]`, `a + 1` cho `[3,5]`, không phải nhân ma trận hay chỉ sửa phần tử đầu.

Q8. Which pair of trailing dimensions is broadcast-compatible? | Hai chiều cuối nào tương thích broadcasting?

A. Only both zero | Chỉ khi cùng bằng 0

B. Equal sizes or one size equal to 1 | Bằng nhau hoặc một chiều có kích thước 1

C. Always 2 and 3 | Luôn là 2 và 3

D. Any unrelated positive sizes | Hai số dương bất kỳ

Đáp án: B

Giải thích: NumPy đối chiếu các chiều từ phải sang trái: mỗi cặp phải bằng nhau hoặc một bên có kích thước 1. Vì vậy chiều 3 và 1 tương thích, còn 3 và 2 trên cùng trục thường không tương thích.

Q9. What happens for incompatible broadcast shapes? | Điều gì xảy ra khi hai shape không tương thích broadcasting?

A. A ValueError is generally raised | Thường phát sinh ValueError

B. NumPy always returns None | NumPy luôn trả None

C. NumPy silently discards extra rows | NumPy tự bỏ hàng thừa

D. A CSV file is created | Một file CSV được tạo

Đáp án: A

Giải thích: Shape không tương thích không thể ghép cho phép toán theo phần tử, nên NumPy thường gây `ValueError`. Ví dụ cộng `(2,3)` với `(2,)` lỗi vì hai chiều cuối 3 và 2 không khớp.

Q10. What is vectorization? | Vectorization là gì?

A. Removing all calculations | Loại bỏ mọi phép tính

B. Applying numerical operations to arrays directly | Áp dụng phép toán số trực tiếp lên mảng

C. Turning every number into a set | Biến mọi số thành Set

D. Manually printing each item | Tự in từng phần tử

Đáp án: B

Giải thích: Vector hóa áp dụng phép toán trực tiếp trên mảng NumPy thay vì viết vòng lặp Python qua từng phần tử. Ví dụ `a * 2` nhân mọi ô số, còn tổng hợp như `a.sum()` gộp chúng thành một thống kê.

Q11. What does NumPy `a * b` do for equal-shaped numeric arrays? | NumPy `a * b` làm gì với hai mảng số cùng shape?

A. Multiply corresponding elements | Nhân các phần tử cùng vị trí

B. Concatenate the arrays | Nối hai mảng

C. Always compute matrix multiplication | Luôn nhân ma trận

D. Compare the shapes only | Chỉ so sánh shape

Đáp án: A

Giải thích: Với mảng số NumPy, `*` nhân các ô tương ứng và có thể dùng broadcasting. Vì vậy hai mảng `[2,3]` và `[4,5]` nhân theo phần tử cho `[8,15]`; phép nhân ma trận dùng `@`.

Q12. Which operator is used for matrix multiplication? | Toán tử nào dùng để nhân ma trận?

A. `and` | `and`

B. `%` | `%`

C. `@` | `@`

D. `*` alone | Chỉ `*`

Đáp án: C

Giải thích: Toán tử `@` thực hiện nhân ma trận theo quy tắc đại số tuyến tính, kết hợp hàng với cột. Khác `*`, nó không chỉ nhân các phần tử ở cùng vị trí.

Q13. For finite numeric array values, what does `np.minimum(a, 10)` do? | Với mảng chứa các giá trị số hữu hạn, `np.minimum(a, 10)` làm gì?

A. Sum all values and multiply by 10 | Cộng tổng rồi nhân 10

B. Select the largest value only | Chỉ chọn số lớn nhất

C. Remove every element below 10 | Xóa mọi phần tử dưới 10

D. Cap each value at 10 | Giới hạn mỗi giá trị tối đa 10

Đáp án: D

Giải thích: Với giá trị số hữu hạn, `np.minimum(a, 10)` so từng ô với 10 rồi lấy số nhỏ hơn. Vì vậy `[8,12]` thành `[8,10]`; nó không xóa phần tử nhỏ hơn 10.

Q14. How does `np.min()` differ from `np.minimum()`? | `np.min()` khác `np.minimum()` thế nào?

A. min performs sorting descending | min sắp xếp giảm dần

B. Both always return the same shape | Cả hai luôn trả cùng shape

C. min reduces values; minimum compares values element-wise | min tổng hợp lấy cực tiểu; minimum so từng phần tử

D. minimum reads files; min writes files | minimum đọc file, min ghi file

Đáp án: C

Giải thích: `np.min(a)` mặc định gộp nhiều giá trị thành một cực tiểu, hoặc lấy cực tiểu theo trục được chọn. `np.minimum(a,b)` so từng ô tương ứng có thể dùng broadcasting và thường trả mảng kết quả.

Q15. For shape `(2,3)`, what is the shape of `sum(axis=1)`? | Với shape `(2,3)`, `sum(axis=1)` có shape nào?

A. `(2,3)` | `(2,3)`

B. `(1,3)` | `(1,3)`

C. `(3,)` | `(3,)`

D. `(2,)` | `(2,)`

Đáp án: D

Giải thích: Ma trận shape `(2,3)` có hai hàng, mỗi hàng gồm ba số. `sum(axis=1)` cộng ba cột trong từng hàng, để lại hai tổng với shape `(2,)`.

Q16. For shape `(2,3)`, what is the shape of `mean(axis=0)`? | Với shape `(2,3)`, `mean(axis=0)` có shape nào?

A. `(3,2)` | `(3,2)`

B. `(2,3)` | `(2,3)`

C. `(2,)` | `(2,)`

D. `(3,)` | `(3,)`

Đáp án: D

Giải thích: Với mảng `(2,3)`, `mean(axis=0)` lấy trung bình qua hai hàng cho từng cột. Còn lại ba trung bình cột, nên shape kết quả mặc định là `(3,)`, không phải `(2,)`.

Q17. Which expression adds a distinct offset to each column of a `(2,3)` matrix? | Biểu thức nào cộng một giá trị riêng cho mỗi cột của ma trận (2,3)?

A. `X + np.array([1,2,3])` | `X + np.array([1,2,3])`

B. `X.shape + 1` | `X.shape + 1`

C. `X @ np.array([1,2,3])` | `X @ np.array([1,2,3])`

D. `X + np.array([1,2])` | `X + np.array([1,2])`

Đáp án: A

Giải thích: Vector bù dài ba khớp chiều cuối gồm ba cột của ma trận `(2,3)`. Broadcasting cộng `[1,2,3]` vào cả hai hàng, mỗi cột một mức riêng; vector dài hai không khớp chiều ba.

Q18. Which statement about broadcasting is correct? | Phát biểu nào đúng về broadcasting?

A. It may apply a smaller operand without physically copying it | Nó có thể áp toán hạng nhỏ mà không cần sao chép vật lý

B. It permanently changes the array shape in every case | Nó luôn thay shape vĩnh viễn

C. It always duplicates the entire array in memory first | Nó luôn sao chép toàn mảng trước

D. It requires writing explicit for loops | Nó bắt buộc viết vòng for

Đáp án: A

Giải thích: Broadcasting có thể tái sử dụng toán hạng nhỏ trên các chiều tương thích mà không cần nhân bản vật lý toàn bộ dữ liệu trước. Nó khác việc phải viết vòng lặp hoặc đổi shape của đầu vào vĩnh viễn.

Q19. How do aggregation and vectorization differ? | Aggregation khác vectorization thế nào?

A. Aggregation summarizes values; vectorization operates over arrays | Aggregation tổng hợp giá trị; vectorization tính trực tiếp trên mảng

B. Both mean sorting only | Cả hai chỉ là sắp xếp

C. Aggregation never uses NumPy | Aggregation không bao giờ dùng NumPy

D. Vectorization must reduce to one number | Vectorization luôn phải trả một số

Đáp án: A

Giải thích: Tổng hợp gộp các giá trị thành thống kê, thường giảm một trục, còn phép toán vector hóa tính trực tiếp trên nhiều ô. Ví dụ `a.mean()` tóm tắt dữ liệu, còn `a * 2` biến đổi từng phần tử.

Q20. Why should data meaning be known before choosing `axis`? | Vì sao phải hiểu ý nghĩa dữ liệu trước khi chọn `axis`?

A. It avoids specifying array shape | Nó giúp khỏi nêu shape

B. It ensures every value is positive | Nó bảo đảm mọi giá trị dương

C. It determines whether rows or columns answer the question | Nó quyết định cần tổng hợp theo hàng hay cột

D. It changes Python syntax rules | Nó đổi quy tắc cú pháp Python

Đáp án: C

Giải thích: Chọn trục đúng phụ thuộc ý nghĩa hàng và cột. Nếu hàng là học viên, cột là môn, `mean(axis=0)` tính trung bình từng môn qua học viên, còn `mean(axis=1)` tính trung bình các môn của mỗi học viên.

8. Nối bài tiếp theo

`Index/Slice → Aggregate/Broadcast → Vectorization → AI`.

PY-16 — NumPy reshape, transpose, ma trận và lưu file

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Reshape — đổi hình dạng; **Transpose** — chuyển vị.

`ndarray → Reshape/Transpose → Matrix → Model Input`

Học xong dùng để làm gì? Chuyển mảng thành ma trận, nhân ma trận và lưu dữ liệu NPY để dùng lại mà không mất shape/dtype.

Kết nối kiến thức tiếp theo: PY-17 đưa dữ liệu vào DataFrame có nhãn hàng và tên cột.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Reshape	đổi hình dạng	Đổi shape nhưng giữ số phần tử
Transpose	chuyển vị	Đổi trục hàng và cột
Matrix multiply	nhân ma trận	Tạo kết quả bằng phép nhân tuyến tính
Serialization	lưu dữ liệu	Ghi mảng vào tệp để đọc lại

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`reshape(rows,cols)`	Số phần tử trước/sau phải bằng nhau
`X.T / np.transpose(X)`	Đổi trục từ hàng thành cột
`A @ B / np.dot(A,B)`	Nhân tuyến tính yêu cầu kích thước tương thích
`np.save()/np.load()`	Lưu và đọc định dạng NPY, chỉ tải từ nguồn tin cậy

4. Khái niệm liên quan

Từ một dãy nhiệt độ hoặc bảng điểm, ta cần biết **bố trí lại dữ liệu**, **đổi chiều quan sát**, **tính điểm tổng hợp** và **lưu để sử dụng lại**. Bốn khái niệm này nối nhau: `reshape` bố trí hàng/cột, `.T` chuyển vị, `@` kết hợp các chiều đúng quy tắc, còn `np.save`/`np.load` lưu và phục hồi mảng.

4.1. Reshape — Đổi hình dạng mảng

Từ vựng trong mục: Reshape — đổi hình dạng; **Shape** — kích thước.

Reshape đổi cách sắp xếp một dãy phần tử thành các hàng/cột mới **mà không đổi số phần tử**. Ví dụ máy đo có 6 nhiệt độ theo thứ tự thời gian `[20,21,22,23,24,25]` °C. Nếu mỗi hàng đại diện một ngày và mỗi cột là một giờ đo, `reshape(2,3)` tạo bảng **2 ngày × 3 giờ**. Ngày đầu chứa ba số đầu, ngày hai chứa ba số sau. Không thể reshape 6 số thành `(4,2)` vì cần 8 số.

No Note — Không chú thích

```
import numpy as np

temperatures = np.array([20, 21, 22, 23, 24, 25])
print(temperatures.reshape(2, 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đổi 6 lượt đo liên tiếp thành bảng 2 hàng và 3 cột mà không mất dữ liệu.
# Dữ liệu: 6 nhiệt độ °C, sau reshape mỗi hàng có ba lượt liên tiếp.
# NumPy xử lý mảng số nhiều chiều; np là tên viết tắt của thư viện.
import numpy as np

# np.array() tạo ndarray một chiều từ 6 số đo.
temperatures = np.array([20, 21, 22, 23, 24, 25])
# reshape(2,3) đổi cách tổ chức thành 2 hàng × 3 cột, giữ nguyên giá trị và thứ tự đọc; không tự tính thêm dữ liệu.
print(temperatures.reshape(2, 3))
```

Kết quả khi chạy:

```
[[20 21 22]
 [23 24 25]]
```

Mỗi hàng là một ngày, mỗi cột là một giờ đo theo thứ tự. Shape đổi từ `(6,)` thành `(2,3)`, còn thứ tự giá trị không đổi.

4.2. Transpose — Hoán đổi ý nghĩa hàng và cột

Từ vựng trong mục: Transpose — chuyển vị; **Row/column** — hàng/cột.

Transpose (`.T`) tráo hai trục của bảng hai chiều. Ví dụ bảng `scores` có **2 hàng học viên** An, Bình và **3 cột môn** Toán, Văn, Python: `[8,7,9]` và `[6,5,8]`. Sau `.T`, bảng thành **3 hàng môn** và **2 cột học viên**. Các điểm giữ nguyên nhưng cách trình bày đổi, giúp thống kê theo môn dễ nhìn hơn. Khác `reshape`, chuyển vị đổi vị trí của các ô theo trục.

No Note — Không chú thích

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.T)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuyển vị ma trận để biến các cột môn thành các hàng.
# Dữ liệu: 2 học viên (hàng), 3 môn (cột), điểm theo thang 10.
# NumPy hỗ trợ mảng hai chiều và chuyển vị ma trận.
import numpy as np
```

```
# Tạo bảng điểm 2 hàng × 3 cột, mỗi hàng là một học viên.
scores = np.array([[8, 7, 9], [6, 5, 8]])
# .T là thuộc tính chuyển vị, hoán đổi trục hàng/cột thành 3 hàng môn × 2 cột học viên; thường là
view nên không mặc định sao chép.
print(scores.T)
```

Kết quả khi chạy:

```
[[8 6]
 [7 5]
 [9 8]]
```

Hàng đầu chứa điểm Toán của An và Bình, hàng hai là Văn, hàng ba là Python. Shape chuyển từ `(2,3)` thành `(3,2)`.

4.3. Matrix multiplication — Nhân ma trận

Từ vựng trong mục: Matrix multiplication — nhân ma trận; **Dot product** — tích vô hướng.

Toán tử `@` thực hiện **phép nhân ma trận**, không phải nhân từng ô như `*`. Với một học viên có điểm `[8,7,9]` cho ba môn và trọng số `[0.2,0.3,0.5]`, `scores @ weights` tính `8×0.2 + 7×0.3 + 9×0.5 = 8.2`. Với bảng nhiều học viên, các hàng là người và các cột là môn; ma trận trọng số phải có số hàng bằng số cột của bảng điểm, ví dụ `(2,3) @ (3,1) → (2,1)`.

No Note — Không chú thích

```
import numpy as np

scores = np.array([8, 7, 9])
weights = np.array([0.2, 0.3, 0.5])
print(scores @ weights)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính điểm tổng hợp theo trọng số´bằng phép nhân ma trận @.
# Dữ liệu: Ba điểm [8,7,9] nhân ba tỷ trọng [20%,30%,50%], tổng trọng số´là 100%.
# NumPy cung cấp phép nhân ma trận cho ndarray.
import numpy as np

# Vector 3 điểm có cùng thứ tự môn với weights.
scores = np.array([8, 7, 9])
# Vector trọng số´0.2/0.3/0.5 biểu thị mức đóng góp từng môn, không phải số´học viên.
weights = np.array([0.2, 0.3, 0.5])
# @ nhân từng điểm với trọng số´tương ứng rồi cộng thành một điểm tổng; khác * nhân từng ô và trả
vector.
print(scores @ weights)
```

Kết quả khi chạy:

```
8.2
```

Điểm có trọng số là 8,2 trên thang 10. `@` nhân các thành phần tương ứng rồi cộng thành một điểm tổng kết.

4.4. Serialization — Lưu và đọc lại mảng

Từ vựng trong mục: Serialization — tuần tự hóa; **NPY** — định dạng lưu mảng NumPy.

Tuần tự hóa (serialization) là lưu dữ liệu trong bộ nhớ thành định dạng có thể đọc lại. NumPy dùng `np.save` ghi mảng thành tệp `.npy`, chứa cả **giá trị, shape và dtype**; `np.load` phục hồi dữ liệu. Ví dụ lưu ba điểm `[3,5,7]` vào thư mục tạm rồi đọc lại để kiểm tra. `allow_pickle=False` phù hợp khi chỉ cần dữ liệu số, tránh đọc đối tượng Python được pickle hóa từ nguồn không tin cậy. `tolist()` chỉ đổi mảng thành List, **không lưu file**.

No Note — Không chú thích

```
import numpy as np
from pathlib import Path
from tempfile import TemporaryDirectory

scores = np.array([3, 5, 7])
with TemporaryDirectory() as folder:
    file_path = Path(folder) / 'scores.npy'
    np.save(file_path, scores)
    restored = np.load(file_path, allow_pickle=False)
    print(restored)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghi mảng NumPy ra tệp .npy trong thư mục tạm và đọc lại an toàn.
# Dữ liệu: Ba điểm số´[3,5,7], tệp .npy bảo toàn dữ liệu và kiểu NumPy.
# NumPy hỗ˜trợ định dạng lưu mảng nhị phân .npy bằng save/load.
import numpy as np
# Path biểu diễn đường dẫn để ghép file bằng toán tử /.
from pathlib import Path
# TemporaryDirectory tạo thư mục tạm và tự dọn file khi khối with kết thúc.
from tempfile import TemporaryDirectory

# Mảng gốc ba số´dùng làm dữ liệu thử lưu/đọc.
scores = np.array([3, 5, 7])
# with quản lý vòng đời thư mục, dọn sạch dù không cần lệnh xóa thủ công.
with TemporaryDirectory() as folder:
    # Path(folder) / 'scores.npy' ghép tên file vào thư mục tạm.
    file_path = Path(folder) / 'scores.npy'
    # np.save() lưu mảng trong định dạng .npy, giữ thông tin shape/dtype.
    np.save(file_path, scores)
    # np.load() đọc .npy thành ndarray; allow_pickle=False chặn dữ liệu pickle khi không cần object
Python, an toàn hơn với tệp không tin cậy.
    restored = np.load(file_path, allow_pickle=False)
    # In mảng tải lại để đối chiếu trực quan với [3,5,7].
    print(restored)
```

Kết quả khi chạy:

```
[3 5 7]
```

Mảng được ghi ra tệp `.npy` rồi đọc lại đủ ba phần tử. Thư mục tạm tự được dọn sau khi thoát khỏi `with`.

5. Năm tình huống NumPy — từ reshape đến lưu ma trận kiểm kê

Tình huống 1 — Cấp 1: Đổi 10 số đo thành bảng 5 ngày × 2 ca

Từ vựng trong mục: Reshape — đổi hình dạng; **Shape** — kích thước.

Tình huống thực tế: Trạm quan trắc có **10 lần đo nhiệt độ**, tính bằng **°C**, theo đúng thứ tự: ca sáng rồi ca chiều của ngày 1, tiếp đến ca sáng rồi ca chiều của ngày 2, cho tới hết **5 ngày**. Người trực nhận được một dãy 10 số, chưa có hàng hoặc cột. Họ muốn đổi thành bảng **5 hàng ngày × 2 cột ca** để xem ngay hai kết quả của ngày thứ ba, mà không làm thay đổi hay mất một số đo nào.

Vì sao chọn cách này? `reshape(5, 2)` chỉ **đổi cách nhìn** dãy 10 số thành 5 hàng và 2 cột; nó không đo lại, tính thêm hoặc tự sắp xếp ngày. Số 2 là hai ca của một ngày, số 5 là số ngày: mỗi hai giá trị liên tiếp được ghép vào cùng hàng vì **dữ liệu ban đầu đã được ghi theo thứ tự sáng–chiều**. Khi đọc ngày thứ ba, phải chọn hàng có chỉ số 2 vì Python bắt đầu đếm từ 0. Nếu dãy đầu vào bị đảo thứ tự ca thì `reshape` không tự phát hiện; cần xác nhận thứ tự trước khi đổi hình dạng.

Quy trình:

1. Lưu 10 nhiệt độ theo trình tự ngày–ca.
2. Kiểm tra số phần tử đúng bằng 5 × 2.
3. Reshape và tra hàng ngày thứ ba bằng chỉ số 2.

No Note — Không chú thích

```
import numpy as np
readings = np.array([26,27,25,28,29,24,26,30,27,25])
daily = readings.reshape(5, 2)
if readings.size != 10 or daily.size != 10:
    raise ValueError("Phải giữ đủ 10 lượt đo sau khi đổi dạng.")
print("Lượt đo:", readings.size)
print("Shape:", daily.shape)
print("Ngày 3 (sáng, chiều):", daily[2].tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Săp 10 lượt đo thành 5 ngày, môi ngày hai lần sáng/chiều.
# Dữ liệu: 10 nhiệt độ °C, cứ hai giá trị liên tiếp tương ứng một ngày.
# NumPy tạo mảng và đổi shape của sô´đo.
import numpy as np
# 10 giá trị là 10 lượt liên tục theo thứ tự ngày.
readings = np.array([26,27,25,28,29,24,26,30,27,25])
# reshape(5,2) tạo 5 hàng ngày × 2 cột sáng/chiều, không đổi thứ tự hoặc tổng sô´ô.
daily = readings.reshape(5, 2)
# size là tổng 10 phần tư; daily.size vẫn 10 sau đổi dạng.
if readings.size != 10 or daily.size != 10:
    # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
    raise ValueError("Phải giữ đủ 10 lượt đo sau khi đổi dạng.")
# size là tổng 10 phần tư; daily.size vẫn 10 sau đổi dạng.
print("Lượt đo:", readings.size)
# shape trả tuple (5,2) chỉ rõ sô´ngày và sô´lần môĩ ngày.
print("Shape:", daily.shape)
# [2] lấy hàng ngày thứ ba (vì bắt đầu 0); tolist() đổi ndarray của hai lượt vê`list Python dê˜đọc.
print("Ngày 3 (sáng, chiều):", daily[2].tolist())
```

Kết quả khi chạy:

```
Lượt đo: 10
Shape: (5, 2)
Ngày 3 (sáng, chiều): [29, 24]
```

Ngày thứ ba lấy đúng lượt 5–6 trong dãy ban đầu; `reshape` không làm thay đổi giá trị nhiệt độ.

Tình huống 2 — Cấp 2: Chuyển bảng 12 học viên × 3 môn sang bảng theo môn

Từ vựng trong mục: Transpose — chuyển vị; **Axis** — trục; **Column** — cột.

Tình huống thực tế: Phòng đào tạo nhận bảng điểm của **12 học viên H01–H12**. Mỗi hàng là một người, ba cột lần lượt là **Toán, Văn, Python**, đều theo thang **10 điểm**. Giáo viên muốn trình bày lại theo từng môn: mỗi hàng là một môn, 12 cột là 12 học viên; từ đó tính tổng điểm của từng môn trên cả lớp. Bảng mới vẫn phải chứa đủ **36 ô điểm** của bảng ban đầu.

Vì sao chọn cách này? Chuyển vị (`.T`) nghĩa là đổi **hàng thành cột và cột thành hàng**: bảng 12 học viên × 3 môn trở thành bảng 3 môn × 12 học viên, nhưng từng điểm vẫn ứng với đúng người và môn. Trên bảng mới, `sum(axis=1)` cộng **theo chiều ngang từng hàng**, tức cộng đủ 12 điểm của một môn. Nếu tiếp tục dùng `axis=0` như bảng ban đầu, kết quả trả theo học viên chứ không phải môn. Đây là cách thay góc nhìn để giải một câu hỏi khác, không phải cộng các môn vào làm một điểm.

Quy trình:

1. Tạo bảng điểm 12 × 3 và kiểm tra thứ tự cột.
2. Chuyển vị bằng `.T` và xác nhận shape 3 × 12.
3. Cộng từng hàng đã chuyển vị để lấy tổng mỗi môn.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[6,7,5],[7,9,8],[8,7,5],[9,9,8],[10,7,5],[6,9,8],[7,7,5],[8,9,8],[9,7,5],
[10,9,8],[6,7,5],[7,9,8]], dtype=int)
by_subject = scores.T
if scores.shape != (12, 3) or by_subject.shape != (3, 12):
    raise ValueError("Chuyển vị phải đổi bảng 12 học viên, 3 môn thành 3 hàng, 12 cột.")
print("Học viên:", scores.shape[0])
print("Sau chuyển vị:", by_subject.shape)
print("Tổng ba môn:", by_subject.sum(axis=1).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuyển bảng điểm 12 học viên × 3 môn thành bảng mỗi môn một hàng và tính tổng môn.
# Dữ liệu: 12 hàng học viên, 3 cột môn, điểm trên thang 10.
# NumPy hỗ trợ chuyển vị và cộng theo trục.
import numpy as np
# dtype=int giữ điểm nguyên; bảng gốc shape (12,3).
scores = np.array([[6,7,5],[7,9,8],[8,7,5],[9,9,8],[10,7,5],[6,9,8],[7,7,5],[8,9,8],[9,7,5],
[10,9,8],[6,7,5],[7,9,8]], dtype=int)
# Thuộc tính .T chuyển bảng thành (3,12): ba hàng môn, mỗi hàng 12 điểm.
by_subject = scores.T
# shape trả kích thước (12,3); sau chuyển vị là (3,12).
if scores.shape != (12, 3) or by_subject.shape != (3, 12):
    # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
    raise ValueError("Chuyển vị phải đổi bảng 12 học viên, 3 môn thành 3 hàng, 12 cột.")
# shape trả kích thước (12,3); sau chuyển vị là (3,12).
print("Học viên:", scores.shape[0])
print("Sau chuyển vị:", by_subject.shape)
```

```
# axis=1 cộng toàn bộ 12 cột học viên trong TỪNG hàng môn; tolist() đổi ba tổng sang list Python để in.
print("Tổng ba môn:", by_subject.sum(axis=1).tolist())
```

Kết quả khi chạy:

```
Học viên: 12
Sau chuyển vị: (3, 12)
Tổng ba môn: [93, 96, 78]
```

Tổng trả theo thứ tự Toán, Văn, Python; thay trục mà không thay dữ liệu là mục đích chính của `transpose`.

Tình huống 3 — Cấp 3: Nhân ma trận điểm 15 học viên với vector trọng số

Từ vựng trong mục: Matrix multiplication — nhân ma trận; **Vector** — vector; **Weights** — trọng số.

Tình huống thực tế: Một khóa học có **15 học viên**, mỗi người có ba điểm đánh giá: **lý thuyết, thực hành, dự án**, đều theo thang 10. Quy định minh họa tính điểm cuối với mức đóng góp tương ứng **20%, 30%, 50%**; dự án chiếm nhiều nhất. Giáo vụ cần tính đúng **một điểm tổng kết cho mỗi học viên** và đếm bao nhiêu người đạt **từ 8 điểm**, không lấy điểm của người này ghép với người khác.

Vì sao chọn cách này? Trọng số là phần đóng góp của từng loại điểm vào điểm cuối; ở đây lý thuyết góp 20%, thực hành 30% và dự án 50%, cộng lại thành 100%. Mỗi hàng của mảng là ba điểm của **cùng một học viên**. Phép nhân ma trận `@` áp cùng ba mức đóng góp lên từng hàng và trả về **15 điểm cuối**, mỗi người một điểm. Chẳng hạn người có điểm cao ở dự án sẽ được phần đóng góp lớn hơn tương ứng chính sách. Nếu gộp theo cột thay vì theo hàng, kết quả sẽ là điểm chung từng môn cho cả lớp chứ không phải điểm tổng kết từng người.

Quy trình:

1. Tạo ma trận có đúng 15 hàng và ba cột đánh giá.
2. Lưu vector trọng số theo cùng thứ tự, kiểm tra tổng trọng số bằng 1.
3. Nhân `@`, đếm số điểm tổng kết đạt 8 và in kết quả đầu.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[6,5,7],[7,7,8],[8,9,9],[9,6,10],[10,8,7],[6,5,8],[7,7,9],[8,9,10],[9,6,7],
[10,8,8],[6,5,9],[7,7,10],[8,9,7],[9,6,8],[10,8,9]], dtype=float)
weights = np.array([0.2, 0.3, 0.5])
final = scores @ weights
if scores.shape != (15, 3) or final.shape != (15,):
    raise ValueError("Cần đủ 15 bộ điểm ba môn và 15 điểm tổng hợp.")
if not np.isclose(weights.sum(), 1.0):
    raise ValueError("Tổng trọng số ba môn phải bằng 1.")
print("Học viên:", len(final))
print(f"Điểm H01: {final[0]:.2f}")
print("Đạt từ 8:", int((final >= 8).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng phép @ tính điểm trung bình có trọng số cho 15 học viên và kiểm tra tổng trọng số.
# Dữ liệu: 15 hàng học viên × 3 môn; trọng số giả định 0.2,0.3,0.5 cho ba môn tương ứng.
# NumPy thực hiện phép nhân ma trận nhanh trên toàn bộ bảng điểm.
import numpy as np
# dtype=float lưu bảng điểm dưới dạng số thực 15×3.
```

```
scores = np.array([[6,5,7],[7,7,8],[8,9,9],[9,6,10],[10,8,7],[6,5,8],[7,7,9],[8,9,10],[9,6,7],
[10,8,8],[6,5,9],[7,7,10],[8,9,7],[9,6,8],[10,8,9]], dtype=float)
# Ba tỷ lệ là 20%,30%,50%; tổng bằng 1 tức 100% sức ảnh hưởng.
weights = np.array([0.2, 0.3, 0.5])
# @ nhân môĩ hàng 3 điểm với 3 trọng sô´và cộng; trả vector 15 điểm tổng hợp.
final = scores @ weights
# shape (15,) nghĩa 15 kết quả một chiều, một điểm cho một học viên.
if scores.shape != (15, 3) or final.shape != (15,):
    # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
    raise ValueError("Cần đủ 15 bộ điểm ba môn và 15 điểm tổng hợp.")
# weights.sum() cộng 3 trọng sô; isclose() so với 1.0 có dung sai sô´thực, đáng tin hơn == tuyệt
đôí.
if not np.isclose(weights.sum(), 1.0):
    # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
    raise ValueError("Tổng trọng sô´ba môn phải bằng 1.")
print("Học viên:", len(final))
# [0] lấy kết quả H01; :.2f in hai chữ sô´thập phân.
print(f"Điểm H01: {final[0]:.2f}")
# So từng điểm với ngưỡng 8 tạo mask Boolean; sum() đếm sô´học viên đạt (True được tính là 1).
print("Đạt từ 8:", int((final >= 8).sum()))
```

Kết quả khi chạy:

```
Học viên: 15
Điểm H01: 6.20
Đạt từ 8: 7
```

Đây là nhân ma trận thực sự: mười lăm điểm đầu ra tương ứng mười lăm học viên, không phải phép nhân 15 × 3 tạo ra 45 kết quả.

Tình huống 4 — Cấp 4: Tái cấu trúc 18 số đo và lưu bản snapshot độc lập

Từ vựng trong mục: Reshape — đổi hình dạng; **Copy** — bản sao; **Transpose** — chuyển vị.

Tình huống thực tế: Nhà máy lưu **18 lượt ghi điện năng tiêu thụ**, mỗi lượt tính bằng **kWh**. Máy ghi lần lượt ba ca của ngày 1, ba ca của ngày 2... cho đến đủ **6 ngày**. Bộ phận vận hành cần xem mức cao nhất mỗi ngày, còn kiểm toán muốn một bảng khác xếp **ca thành hàng, ngày thành cột** để tiện đối chiếu. Họ sẽ thử sửa một ô trên bảng kiểm toán, nhưng bản dữ liệu ban đầu phải được giữ nguyên.

Vì sao chọn cách này? `reshape(6, 3)` nhóm cứ ba lượt đo liên tiếp thành **một ngày**, rồi có thể tìm mức cao nhất trong ba ca của từng ngày. **Chuyển vị** `.T` đổi góc nhìn thành **3 hàng ca × 6 cột ngày**. Tuy nhiên mảng chuyển vị có thể chỉ là một **view**, tức cách nhìn khác của cùng vùng dữ liệu, nên sửa nó có thể làm dữ liệu gốc đổi theo. `.copy()` tạo **bản độc lập** để kiểm toán thử hiệu chỉnh mà không ghi đè sổ đo ban đầu. Đây là lý do chương trình dùng `.T.copy()` thay vì chỉ `.T`.

Quy trình:

1. Tổ chức 18 số thành 6 ngày × 3 ca.
2. Tính giá trị lớn nhất theo hàng và chuyển vị để xem theo ca.
3. Sao chép bản chuyển vị, sửa thử một ô và kiểm chứng dữ liệu nguồn không thay đổi.

No Note — Không chú thích

```
import numpy as np
values = np.array([16,19,22,25,28,18,21,24,27,17,20,23,26,16,19,22,25,28], dtype=int)
by_day = values.reshape(6, 3)
for_audit = by_day.T.copy()
original = int(by_day[0, 0])
```

```
for_audit[0, 0] = -1
if by_day.shape != (6, 3) or for_audit.shape != (3, 6):
    raise ValueError("Bảng dữ liệu phải có 6 ngày, 3 lượt và bản đối soát đã chuyển vị.")
if int(by_day[0, 0]) != original:
    raise ValueError("Chỉnh sửa bản đối soát không được thay đổi bản dữ liệu gốc.")
print("Lượt đo:", values.size)
print("Đối soát:", for_audit.shape)
print("Đỉnh 6 ngày:", by_day.max(axis=1).tolist())
print("Nguồn đầu:", int(by_day[0, 0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo bản đối soát chuyển vị độc lập từ 18 lượt đo để sửa bản đối soát không làm đổi
nguồn.
# Dữ liệu: 18 giá trị được chia thành 6 ngày × 3 lượt; bản đối soát có 3 hàng lượt × 6 ngày.
# NumPy cho reshape(), chuyển vị .T và copy().
import numpy as np
# dtype=int lưu các số đo nguyên ban đầu.
values = np.array([16,19,22,25,28,18,21,24,27,17,20,23,26,16,19,22,25,28], dtype=int)
# reshape(6,3) nhóm 18 giá trị thành sáu hàng ngày, mỗi hàng ba lượt.
by_day = values.reshape(6, 3)
# .T đổi thành 3 hàng lượt × 6 cột ngày; .copy() tạo bộ nhớ độc lập, tránh thay đổi bản gốc qua
view chuyển vị.
for_audit = by_day.T.copy()
# [0,0] lấy lượt thứ nhất ngày đầu; int() đổi scalar NumPy thành int Python để lưu kiểm chứng.
# Đối chiếu nguồn vẫn bằng giá trị original trước khi sửa bản copy.
original = int(by_day[0, 0])
# Thay số trong bản đối soát COPY để thử tính độc lập, không thay giá trị nguồn.
for_audit[0, 0] = -1
# shape (6,3) và (3,6) xác minh cấu trúc đã chuyển vị.
if by_day.shape != (6, 3) or for_audit.shape != (3, 6):
    # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
    raise ValueError("Bảng dữ liệu phải có 6 ngày, 3 lượt và bản đối soát đã chuyển vị.")
# [0,0] lấy lượt thứ nhất ngày đầu; int() đổi scalar NumPy thành int Python để lưu kiểm chứng.
# Đối chiếu nguồn vẫn bằng giá trị original trước khi sửa bản copy.
if int(by_day[0, 0]) != original:
    # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
    raise ValueError("Chỉnh sửa bản đối soát không được thay đổi bản dữ liệu gốc.")
print("Lượt đo:", values.size)
print("Đối soát:", for_audit.shape)
# max(axis=1) tìm đỉnh qua ba lượt CỦA MỖI NGÀY; tolist() chuyển sáu giá trị thành list Python.
print("Đỉnh 6 ngày:", by_day.max(axis=1).tolist())
# [0,0] lấy lượt thứ nhất ngày đầu; int() đổi scalar NumPy thành int Python để lưu kiểm chứng.
# Đối chiếu nguồn vẫn bằng giá trị original trước khi sửa bản copy.
print("Nguồn đầu:", int(by_day[0, 0]))
```

Kết quả khi chạy:

```
Lượt đo: 18
Đối soát: (3, 6)
Đỉnh 6 ngày: [22, 28, 27, 23, 26, 28]
Nguồn đầu: 16
```

Dữ liệu theo ca là kWh mỗi lượt, không phải công suất tức thời. Bản chỉnh sửa `for_audit` độc lập với bảng gốc vì đã copy.

Tình huống 5 — Cấp 5: Lưu và phục hồi mảng 20 kiểm kê theo hình dạng 5 × 4

Từ vựng trong mục: np.save — lưu mảng NPY; **np.load** — đọc mảng NPY; **Validation** — kiểm dữ liệu.

Tình huống thực tế: Kho kiểm kê **20 ngăn hàng**, sắp theo **5 kệ**, mỗi kệ **4 ngăn**. Mỗi số trong danh sách là lượng sản phẩm ở một ngăn; riêng ngăn thứ 14 ghi **-1** để báo rằng nhân viên kiểm kê nhập lỗi, **không phải tồn kho âm**. Người quản lý cần sắp 20 số vào bảng 5 × 4, đếm số ngăn có lượng hợp lệ, lưu bảng vào một tệp **NPY** tạm rồi đọc lại để kiểm tra tất cả số vẫn đúng và bảng không bị mất dạng.

Vì sao chọn cách này? NPY là định dạng lưu mảng riêng của NumPy; khác văn bản thông thường ở chỗ nó giữ cả **các giá trị, kiểu dữ liệu và hình dạng** bảng. `reshape(5, 4)` ghép đúng 20 ngăn thành năm hàng kệ và bốn cột ngăn, nhưng không tự nhận ra `-1` là số lượng sai: chương trình cần kiểm tra riêng và chỉ đếm ngăn hợp lệ. `np.save()` ghi bảng vào tệp; `np.load(..., allow_pickle=False)` đọc lại mà không bật chức năng đọc đối tượng Python không cần thiết. Dùng thư mục tạm để học viên có thể thử lưu và đọc mà không để tệp mẫu tồn tại lâu; sau khi đọc, so bảng với bản gốc để xác nhận không sai số.

Quy trình:

1. Tạo mảng 20 số theo vị trí, reshape thành 5 × 4.
2. Lưu `.npy` trong thư mục tạm, đọc lại với `allow_pickle=False`.
3. Xác nhận shape/kiểu và từng phần tử trùng bản gốc; chỉ tổng hợp 19 ô không âm.

No Note — Không chú thích

```
from pathlib import Path
from tempfile import TemporaryDirectory
import numpy as np
values = np.array([10,17,24,11,18,25,12,19,26,13,20,27,14,-1,28,15,22,29,16,23], dtype=int)
stocks = values.reshape(5, 4)
with TemporaryDirectory() as folder:
    path = Path(folder) / "stock.npy"
    np.save(path, stocks)
    loaded = np.load(path, allow_pickle=False)
    if loaded.shape != (5, 4):
        raise ValueError("Tệp đã lưu phải chứa 5 hàng và 4 cột.")
    if not np.array_equal(loaded, stocks):
        raise ValueError("Dữ liệu đọc lại phải khớp dữ liệu đã lưu.")
    valid = loaded[loaded >= 0]
    print("Vị trí:", loaded.size)
    print("Shape:", loaded.shape)
    print("Ô hợp lệ:", valid.size)
    print("Tổng món hợp lệ:", int(valid.sum()))
    print("Giữ nguyên dữ liệu:", bool(np.array_equal(loaded, stocks)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lưu bảng kho 5×4 ra .npy và kiểm dữ liệu tải lại, chỉ tổng hợp ô tồn không âm.
# Dữ liệu: 20 ô tồn kho (5 hàng × 4 cột), -1 là ô dữ liệu sai cần loại khỏi tổng món.
# Path cho phép ghép vị trí lưu file bằng toán tử /.
from pathlib import Path
# TemporaryDirectory() tự tạo và dọn thư mục thử nghiệm.
from tempfile import TemporaryDirectory
# NumPy tạo, lưu, đọc và so sánh mảng số.
import numpy as np
# 20 giá trị, dtype=int lưu tồn kho dạng số nguyên, -1 là giá trị lỗi.
values = np.array([10,17,24,11,18,25,12,19,26,13,20,27,14,-1,28,15,22,29,16,23], dtype=int)
# reshape(5,4) sắp 20 ô thành 5 hàng và 4 cột, không đổi giá trị.
stocks = values.reshape(5, 4)
# with đảm bảo stock.npy không còn sau khi kết thúc thử nghiệm.
with TemporaryDirectory() as folder:
    # Ghép thư mục tạm với tên file stock.npy.
    path = Path(folder) / "stock.npy"
    # np.save() ghi mảng với shape và dtype vào file nhị phân .npy.
```

```
    np.save(path, stocks)
    # np.load() đọc mảng lại; allow_pickle=False tránh giải mã Python pickle không cần thiết.
    loaded = np.load(path, allow_pickle=False)
    if loaded.shape != (5, 4):
        # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
        raise ValueError("Tệp đã lưu phải chứa 5 hàng và 4 cột.")
    # array_equal() so shape và từng giá trị hai mảng sau ghi/đọc; khác so sánh == từng ô.
    if not np.array_equal(loaded, stocks):
        # ValueError báo dữ liệu hoặc kích thước sai và dừng thay vì cho kết luận sai.
        raise ValueError("Dữ liệu đọc lại phải khớp dữ liệu đã lưu.")
    # loaded >= 0 tạo mặt nạ True cho ô không âm; lập chỉ mục Boolean giữ giá trị hợp lệ, loại -1.
    valid = loaded[loaded >= 0]
    print("Vị trí:", loaded.size)
    print("Shape:", loaded.shape)
    # size trả số ́ô được giữ; valid.sum() cộng số ́món hợp lệ.
    print("Ô hợp lệ:", valid.size)
    print("Tổng món hợp lệ:", int(valid.sum()))
    # array_equal() so shape và từng giá trị hai mảng sau ghi/đọc; khác so sánh == từng ô.
    # bool() chuyển kết quả kiểm toàn mảng sang bool Python để hiển thị rõ True/False.
    print("Giữ nguyên dữ liệu:", bool(np.array_equal(loaded, stocks)))
```

Kết quả khi chạy:

```
Vị trí: 20
Shape: (5, 4)
Ô hợp lệ: 19
Tổng món hợp lệ: 369
Giữ nguyên dữ liệu: True
```

Ngăn -1 được giữ lại trong file để kiểm toán, nhưng không đưa vào số lượng món hợp lệ. Chỉ đọc file NPY đáng tin cậy, không tải file không rõ nguồn.

6. Sai lầm thường gặp

- `reshape` chỉ thành công khi **tích các kích thước mới bằng số phần tử ban đầu**. Sáu giá trị tạo được bảng 2×3, không phải 4×2.
- `reshape` và `.T` không tương đương: reshape nhóm lại các phần tử theo thứ tự, transpose tráo vị trí theo trục.
- `A * B` nhân từng ô khi broadcast được; `A @ B` nhân ma trận theo chiều tương thích. Với ma trận 2×3, bên phải cần có 3 hàng.
- Phải xác định rõ hàng là học viên/ngày, cột là môn/giờ đo trước khi đọc kết quả.
- `np.save` tạo file thực; `array.tolist()` chỉ chuyển sang List, không ghi file. Không bật `allow_pickle=True` với dữ liệu không đáng tin.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does `reshape()` change? | `reshape()` thay đổi điều gì?

A. The array's shape without changing its total element count | Hình dạng mảng mà không đổi số phần tử

B. Writes an array to an NPY file | Ghi mảng vào tệp NPY

C. Swaps rows and columns | Hoán đổi hàng và cột

D. Matrix multiplication | Phép nhân ma trận

Đáp án: A

Giải thích: reshape bố trí lại cùng các phần tử theo kích thước mới. Với sáu phần tử, đổi `(2,3)` thành `(3,2)` hợp lệ, nhưng đổi thành `(4,2)` thì không.

Q2. What does `.T` do to a 2-D array? | `.T` làm gì với mảng 2-D?

A. The array's shape without changing its total element count | Hình dạng mảng mà không đổi số phần tử

B. Writes an array to an NPY file | Ghi mảng vào tệp NPY

C. Matrix multiplication | Phép nhân ma trận

D. Swaps rows and columns | Hoán đổi hàng và cột

Đáp án: D

Giải thích: Chuyển vị tráo hai trục của ma trận. Với `[[1,2,3],[4,5,6]]`, chuyển vị tạo ba hàng lần lượt là `[1,4]`, `[2,5]`, `[3,6]`.

Q3. What does `A @ B` mean for compatible matrices? | `A @ B` nghĩa là gì với các ma trận tương thích?

A. Swaps rows and columns | Hoán đổi hàng và cột

B. Matrix multiplication | Phép nhân ma trận

C. The array's shape without changing its total element count | Hình dạng mảng mà không đổi số phần tử

D. Writes an array to an NPY file | Ghi mảng vào tệp NPY

Đáp án: B

Giải thích: Toán tử @ tuân theo quy tắc nhân ma trận chứ không nhân riêng từng ô. Nếu `A` có shape `(2,3)` và `B` là `(3,4)`, thì `A @ B` có shape `(2,4)`; dấu `*` lại nhân từng phần tử.

Q4. What does `np.save()` do? | `np.save()` làm gì?

A. Swaps rows and columns | Hoán đổi hàng và cột

B. Matrix multiplication | Phép nhân ma trận

C. Writes an array to an NPY file | Ghi mảng vào tệp NPY

D. The array's shape without changing its total element count | Hình dạng mảng mà không đổi số phần tử

Đáp án: C

Giải thích: NumPy lưu giá trị, shape và dtype ở định dạng NPY. Khác CSV văn bản, tệp NPY giữ thông tin riêng của NumPy để khôi phục đúng mảng.

Q5. Which shape specifically has two rows and three columns? | Shape nào có đúng hai hàng và ba cột?

A. `(3, 2)` | `(3, 2)`

B. `(2, 3)` | `(2, 3)`

C. `(6,)` | `(6,)`

D. `(2, 4)` | `(2, 4)`

Đáp án: B

Giải thích: Chiều thứ nhất là số hàng và chiều thứ hai là số cột: (2,3). Shape `(2,3)` có tổng sáu ô; `(3,2)` cũng sáu ô nhưng số hàng và cột đảo vị trí.

Q6. What is the transpose shape of a `(2, 3)` matrix? | Chuyển vị ma trận `(2, 3)` có shape nào?

A. `(2, 3)` | `(2, 3)`

B. `(2, 4)` | `(2, 4)`

C. `(3, 2)` | `(3, 2)`

D. `(6,)` | `(6,)`

Đáp án: C

Giải thích: Chuyển vị đảo kích thước hàng và cột. Ma trận `(2,3)` trở thành `(3,2)` sau `.T` vì mỗi cột cũ trở thành một hàng.

Q7. What is the shape of `(2, 3) @ (3, 4)`? | Phép `(2, 3) @ (3, 4)` cho shape gì?

A. `(3, 2)` | `(3, 2)`

B. `(2, 3)` | `(2, 3)`

C. `(2, 4)` | `(2, 4)`

D. `(6,)` | `(6,)`

Đáp án: C

Giải thích: Hai chiều trong cùng bằng 3, còn hai chiều ngoài là 2 và 4. Chiều chung bằng 3 được kết hợp khi nhân, còn hai chiều ngoài 2 và 4 được giữ trong kết quả.

Q8. What shape has a one-dimensional array of six values? | Mảng một chiều có sáu giá trị có shape nào?

A. `(3, 2)` | `(3, 2)`

B. `(2, 4)` | `(2, 4)`

C. `(2, 3)` | `(2, 3)`

D. `(6,)` | `(6,)`

Đáp án: D

Giải thích: Mảng phẳng sáu phần tử có một chiều độ dài sáu. Dấu phẩy trong `(6,)` biểu diễn shape một chiều; `(1,6)` mới có hai chiều.

Q9. How does `reshape` differ from transpose? | reshape khác transpose ở đâu?

A. Reshape regroups positions; transpose exchanges axes | Reshape nhóm lại vị trí; transpose đổi trục

B. Converts the array into Python lists or scalars | Chuyển mảng thành List hoặc số Python

C. * multiplies element-wise; @ performs matrix multiplication | * nhân từng phần tử; @ nhân ma trận

D. Reads the saved array | Đọc lại mảng đã lưu

Đáp án: A

Giải thích: reshape và transpose có thể cho cùng shape nhưng giá trị ở vị trí khác. Với `[[1,2,3],[4,5,6]]`, `reshape(3,2)` tạo các hàng `[1,2]`, `[3,4]`, `[5,6]`, còn `.T` cho `[1,4]`, `[2,5]`, `[3,6]`.

Q10. How does * differ from @ for numeric ndarrays? | * khác @ thế nào với ndarray số?

A. Reads the saved array | Đọc lại mảng đã lưu

B. * multiplies element-wise; @ performs matrix multiplication | * nhân từng phần tử; @ nhân ma trận

C. Reshape regroups positions; transpose exchanges axes | Reshape nhóm lại vị trí; transpose đổi trục

D. Converts the array into Python lists or scalars | Chuyển mảng thành List hoặc số Python

Đáp án: B

Giải thích: Dấu sao nhân theo phần tử, @ tuân quy tắc nhân ma trận. Với hai ma trận 2×2 toàn số một, `A * B` vẫn toàn số một, còn `A @ B` toàn số hai do cộng các tích theo hàng–cột.

Q11. What does `np.load()` normally do for an NPY file? | `np.load()` làm gì với tệp NPY?

A. Reads the saved array | Đọc lại mảng đã lưu

B. Reshape regroups positions; transpose exchanges axes | Reshape nhóm lại vị trí; transpose đổi trục

C. * multiplies element-wise; @ performs matrix multiplication | * nhân từng phần tử; @ nhân ma trận

D. Converts the array into Python lists or scalars | Chuyển mảng thành List hoặc số Python

Đáp án: A

Giải thích: load phục hồi mảng NumPy đã ghi. Với tệp NPY đáng tin cậy, `np.load(path)` khôi phục mảng; tránh bật pickle cho dữ liệu đối tượng chưa tin cậy.

Q12. What does `array.tolist()` do? | `array.tolist()` làm gì?

A. Converts the array into Python lists or scalars | Chuyển mảng thành List hoặc số Python

B. Reshape regroups positions; transpose exchanges axes | Reshape nhóm lại vị trí; transpose đổi trục

C. * multiplies element-wise; @ performs matrix multiplication | * nhân từng phần tử; @ nhân ma trận

D. Reads the saved array | Đọc lại mảng đã lưu

Đáp án: A

Giải thích: tolist chuyển biểu diễn dữ liệu trong bộ nhớ chứ không ghi tệp. Mảng hai chiều chuyển thành List lồng nhau, còn mảng không chiều chuyển thành giá trị vô hướng Python.

Q13. What happens if you reshape six elements to `(4, 2)`? | Điều gì xảy ra nếu reshape sáu phần tử thành `(4, 2)`?

A. `allow_pickle=False` | `allow_pickle=False`

B. Left columns and right rows | Số cột bên trái và số hàng bên phải

C. A ValueError due to incompatible element counts | Phát sinh ValueError vì số phần tử không khớp

D. Shape and dtype | Shape và dtype

Đáp án: C

Giải thích: Bảng 4×2 cần tám giá trị, không phải sáu. Shape `(4,2)` cần `4 × 2 = 8` ô, nên NumPy không thể phân bố đúng sáu phần tử hiện có vào shape này.

Q14. What must match to multiply two matrices with `@`? | Muốn nhân hai ma trận bằng `@`, chiều nào phải khớp?

A. `allow_pickle=False` | `allow_pickle=False`

B. Left columns and right rows | Số cột bên trái và số hàng bên phải

C. A ValueError due to incompatible element counts | Phát sinh ValueError vì số phần tử không khớp

D. Shape and dtype | Shape và dtype

Đáp án: B

Giải thích: Nhân ma trận yêu cầu hai chiều trong bằng nhau. Nếu `A` là `(m,n)` và `B` là `(n,p)`, phép nhân hợp lệ và trả shape `(m,p)`.

Q15. Which option avoids loading pickled objects from an NPY file? | Tùy chọn nào tránh đọc đối tượng pickle trong file NPY?

A. A ValueError due to incompatible element counts | Phát sinh ValueError vì số phần tử không khớp

B. Left columns and right rows | Số cột bên trái và số hàng bên phải

C. Shape and dtype | Shape và dtype

D. `allow_pickle=False` | `allow_pickle=False`

Đáp án: D

Giải thích: Tắt pickle an toàn hơn khi đọc dữ liệu số chưa tin cậy. Với `np.load(path, allow_pickle=False)`, mảng object cần pickle sẽ không được đọc, giúp giảm rủi ro chạy mã từ tệp không tin cậy.

Q16. What does NPY preserve in addition to array values? | NPY lưu thêm gì ngoài các giá trị?

A. `allow_pickle=False` | `allow_pickle=False`

B. A ValueError due to incompatible element counts | Phát sinh ValueError vì số phần tử không khớp

C. Shape and dtype | Shape và dtype

D. Left columns and right rows | Số cột bên trái và số hàng bên phải

Đáp án: C

Giải thích: NPY lưu thông tin để khôi phục cấu trúc và kiểu phần tử. Khác các dòng văn bản thường, shape và dtype được lưu để mảng số `(2,3)` khôi phục đúng cấu trúc.

Q17. What is a 1-D dot product of two equal-length numeric vectors? | Tích vô hướng của hai vector một chiều bằng nhau độ dài là gì?

A. The sum of pairwise products | Tổng các tích của từng cặp phần tử

B. A 3-by-3 identity matrix | Ma trận đơn vị 3×3

C. `dtype` | `dtype`

D. `shape` | `shape`

Đáp án: A

Giải thích: Tích vô hướng nhân các vị trí tương ứng rồi cộng kết quả. Với `[1,2]` và `[3,4]`, tích vô hướng bằng `1×3 + 2×4 = 11`, là một số đơn chứ không phải List.

Q18. What does `np.eye(3)` create? | `np.eye(3)` tạo gì?

A. `shape` | `shape`

B. The sum of pairwise products | Tổng các tích của từng cặp phần tử

C. `dtype` | `dtype`

D. A 3-by-3 identity matrix | Ma trận đơn vị 3×3

Đáp án: D

Giải thích: Ma trận đơn vị có 1 ở đường chéo và 0 ở vị trí khác. Đường chéo chính là `[1,1,1]`, còn sáu ô ngoài chéo đều bằng không; nó không phải ma trận toàn số một.

Q19. Which property reports the array dimensions? | Thuộc tính nào cho biết kích thước các chiều?

A. A 3-by-3 identity matrix | Ma trận đơn vị 3×3

B. The sum of pairwise products | Tổng các tích của từng cặp phần tử

C. `dtype` | `dtype`

D. `shape` | `shape`

Đáp án: D

Giải thích: shape liệt kê độ dài của từng trục mảng. Với mảng 2D ba hàng hai cột, `X.shape` là `(3,2)` chứ không phải kiểu dữ liệu.

Q20. Which property reports element data type? | Thuộc tính nào cho biết kiểu phần tử?

A. `shape` | `shape`

B. `dtype` | `dtype`

C. A 3-by-3 identity matrix | Ma trận đơn vị 3×3

D. The sum of pairwise products | Tổng các tích của từng cặp phần tử

Đáp án: B

Giải thích: dtype ghi kiểu số hoặc kiểu dữ liệu khác của phần tử. Ví dụ mảng số nguyên có dtype nguyên, mảng số thập phân có dtype thực; `shape` mới cho kích thước các chiều.

8. Nối bài tiếp theo

ndarray → Reshape/Transpose → Matrix → Model Input.

PY-17 — Pandas Series, DataFrame và đọc CSV

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Series — chuỗi Pandas; **DataFrame** — bảng dữ liệu.

`NumPy ndarray → Pandas DataFrame → Làm sạch → EDA/AI`

Học xong dùng để làm gì? Đọc CSV thành bảng, kiểm tra kiểu và tên cột, tính kết quả theo từng bản ghi.

Kết nối kiến thức tiếp theo: PY-18 làm sạch giá trị thiếu, trùng và sai định dạng trước khi phân tích.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Series	chuỗi Pandas	Dữ liệu một chiều có nhãn chỉ mục
DataFrame	bảng dữ liệu	Bảng có hàng và cột đặt tên
Column	cột	Một feature mang ý nghĩa xác định
CSV	tệp giá trị phân cách	Định dạng văn bản cho bảng có header

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`pd.Series(values)`	Tạo dãy có chỉ mục
`pd.DataFrame(data)`	Tạo bảng từ dictionary/list record
`pd.read_csv(path)`	Đọc CSV có tiêu đề, quy định đường dẫn/encoding
`df.head()/df.info()/df.describe()`	Xem nhanh bản ghi, schema, thống kê

4. Khái niệm liên quan

Pandas giúp quản lý dữ liệu dạng bảng trước khi phân tích và huấn luyện AI. Ta bắt đầu từ một dãy có nhãn (**Series**), mở rộng thành bảng nhiều thuộc tính (**DataFrame**), chọn và tính trên từng cột, rồi đọc dữ liệu thật từ **CSV**. Trong mọi ví dụ, phải nêu rõ một hàng đại diện cho ai/cái gì và một cột là thuộc tính nào.

4.1. Series — Dữ liệu một chiều có nhãn

Từ vựng trong mục: Series — dãy có nhãn; **Index** — nhãn hàng.

Series là dãy một chiều của Pandas, trong đó mỗi giá trị gắn với một **index** (nhãn). Ví dụ theo dõi điểm hai học viên An, Bình: nhãn `An`, `Binh` xác định ai sở hữu giá trị 8 hay 6. Khác List thuần túy thường truy cập

bằng vị trí, Series còn truy cập theo **nhãn** bằng `.loc`. Khi không tự đặt nhãn, Pandas thường tạo chỉ số 0, 1, 2...

No Note — Không chú thích

```
import pandas as pd

scores = pd.Series([8, 6], index=['An', 'Binh'])
print(scores.loc['Binh'])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo Series có nhãn học viên rồi đọc điểm theo nhãn Binh bằng .loc.
# Dữ liệu: Hai điểm của học viên An và Binh trên thang 10; index là mã nhãn, không phải cột.
# Pandas quản lý Series và DataFrame; pd là bí danh chuẩn.
import pandas as pd

# Series là dãy giá trị 1 chiều có nhãn; index=['An','Binh'] gắn tên cho từng điểm.
scores = pd.Series([8, 6], index=['An', 'Binh'])
# .loc['Binh'] tra theo NHÃN Binh, không phải vị trí số thứ tự.
print(scores.loc['Binh'])
```

Kết quả khi chạy:

```
6
```

Nhãn `Binh` trỏ tới giá trị điểm 6; nhãn là chỉ số hàng, không phải nội dung một cột riêng.

4.2. DataFrame — Bảng hàng và cột

Từ vựng trong mục: DataFrame — bảng dữ liệu; **Row/column** — hàng/cột.

DataFrame là bảng hai chiều. Cửa hàng có hai sản phẩm: Pen giá 10.000 đồng và Book giá 35.000 đồng. **Mỗi hàng** là một sản phẩm, **mỗi cột** là một thuộc tính (`Product` tên hàng; `Price` giá đơn vị đồng). Chỉ số 0, 1 bên trái là **index**, không phải một cột dữ liệu mặc định. Mỗi cột tương tự một Series có chung nhãn hàng.

No Note — Không chú thích

```
import pandas as pd

products = pd.DataFrame({
    'Product': ['Pen', 'Book'],
    'Price': [10000, 35000],
})
print(products)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo DataFrame hai hàng sản phẩm với hai cột tên và giá.
# Dữ liệu: Hai sản phẩm Pen/Book có giá tương ứng 10.000/35.000 đồng.
# pd.DataFrame tạo bảng dữ liệu có hàng và cột.
import pandas as pd

# Dictionary gồm các cột Product và Price, hai list cùng chiều dài tạo hai hàng.
products = pd.DataFrame({
```

```
    'Product': ['Pen', 'Book'],
    'Price': [10000, 35000],
})
# In DataFrame với tiêu đề cột và chỉ số hàng tự sinh 0,1.
print(products)
```

Kết quả khi chạy:

```
  Product  Price
0     Pen  10000
1    Book  35000
```

Bảng có 2 hàng sản phẩm × 2 cột thuộc tính; số 0 và 1 là nhãn index mặc định.

4.3. Column và dtype — Đọc và tính trên thuộc tính

Từ vựng trong mục: Column — cột; **dtype** — kiểu dữ liệu cột.

Cột là tập các giá trị của **một thuộc tính** trên nhiều hàng. Với đơn giá `[10,20]` nghìn đồng và số lượng `[2,3]` cái, ta tạo hai cột `price` và `qty` rồi thêm `revenue = price * qty`. Pandas thực hiện phép nhân **theo hàng có cùng index**. Đơn giá nhân số lượng tạo doanh thu 20 và 60 nghìn đồng. `df['price']` chọn một cột, còn `df[['price','qty']]` chọn bảng hai cột.

No Note — Không chú thích

```
import pandas as pd

sales = pd.DataFrame({'price': [10, 20], 'qty': [2, 3]})
sales['revenue'] = sales['price'] * sales['qty']
print(sales['revenue'].tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhân hai cột giá và số lượng theo từng hàng để tạo cột doanh thu.
# Dữ liệu: Hai hàng bán hàng; price là đơn giá, qty là số món, revenue là tiền hàng.
# Pandas cho phép tính trực tiếp trên các cột Series.
import pandas as pd

# Tạo bảng gồm hai cột price và qty, mỗi vị trí tương ứng một giao dịch.
sales = pd.DataFrame({'price': [10, 20], 'qty': [2, 3]})
# Nhân THEO TỪNG HÀNG giữa hai cột Series, gán kết quả vào cột mới revenue.
sales['revenue'] = sales['price'] * sales['qty']
# .tolist() chuyển Series doanh thu thành list Python [20,60] để in.
print(sales['revenue'].tolist())
```

Kết quả khi chạy:

```
[20, 60]
```

Hai số là doanh thu của hàng thứ nhất và thứ hai. Nhân cột có ý nghĩa khi các giá trị đã căn đúng theo cùng hàng.

4.4. read_csv — Đọc bảng từ văn bản CSV

Từ vựng trong mục: CSV — bảng văn bản phân cách; **read_csv** — đọc CSV.

CSV lưu dữ liệu theo các dòng văn bản, thường dòng đầu chứa tên cột. `pd.read_csv()` chuyển nội dung thành DataFrame, có thể từ đường dẫn file hoặc bộ đệm `StringIO`. Ví dụ hai đơn hàng O1, O2 với `amount` 100 và 200 đồng; **mỗi hàng là một đơn**, cột `id` là mã đơn, cột `amount` là số tiền. Pandas thường tự nhận dạng cột số, nhưng dữ liệu phức tạp có thể cần chỉ định kiểu hoặc làm sạch trước khi tính.

No Note — Không chú thích

```
import pandas as pd
from io import StringIO

csv_text = 'id,amount\nO1,100\nO2,200\n'
orders = pd.read_csv(StringIO(csv_text))
print(orders['amount'].sum())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc CSV từ chuoĩ trong RAM bằng pandas và cộng cột amount.
# Dữ liệu: CSV gôm tiêu đề`id,amount và hai đơn O1=100, O2=200; amount biểủ diễn sô´tiền.
# pd.read_csv() biến CSV thành bảng DataFrame.
import pandas as pd
# StringIO cho phép đọc chuoĩ như một file CSV trong RAM.
from io import StringIO

# \n là xuống dòng, dòng đầu tên cột id,amount; các dòng sau là đơn.
csv_text = 'id,amount\nO1,100\nO2,200\n'
# StringIO bọc chuoĩ thành luồng, read_csv() phân cột và suy ra kiểủ; không ghi file vật lý.
orders = pd.read_csv(StringIO(csv_text))
# ['amount'] chọn cột tiền; sum() cộng các hàng, trả 300.
print(orders['amount'].sum())
```

Kết quả khi chạy:

```
300
```

Tổng số tiền hai đơn là 300 đồng. `read_csv` đọc bảng từ văn bản chứ không cần nhập lại thủ công từng dòng.

5. Năm tình huống Pandas — từ Series đến bảng có nhãn để phân tích

Tình huống 1 — Cấp 1: Series có nhãn của 10 học viên

Từ vựng trong mục: Series — dãy có nhãn; **Index** — nhãn hàng; **loc** — tra theo nhãn.

Tình huống thực tế: Phòng giáo vụ có **10 học viên H01 đến H10**, mỗi người được ghi một tuổi tính bằng **năm**, ví dụ H01 18 tuổi. Giáo vụ muốn tra chính xác tuổi của **H07** bằng mã học viên, dù sau này bảng có thể được sắp xếp lại, và tính tuổi trung bình của cả 10 người; không được nhầm mã H07 với hàng có chỉ số 7.

Vì sao chọn cách này? Series của Pandas có thể hiểu là **một cột số liệu kèm nhãn cho từng dòng**. `pd.Series(ages, index=ids)` gắn các tuổi với H01, H02... một cách rõ ràng. `series.loc['H07']` tìm theo **nhãn H07**, không phải vị trí số 7; nếu danh sách sắp xếp lại, mã vẫn xác định đúng người. `series.mean()` tính trung bình 10 tuổi hợp lệ và kết quả vẫn có đơn vị năm. Chỉ dùng `ages[6]` có thể đúng với thứ tự ban đầu nhưng dễ nhầm khi thứ tự dòng thay đổi.

Quy trình:

1. Tạo 10 mã đúng thứ tự tuổi đã ghi.
2. Tạo Series gắn index và đọc H07 bằng `.loc`.
3. Kiểm tra số bản ghi và in trung bình một chữ số thập phân.

No Note — Không chú thích

```
import pandas as pd
ages = [18,19,20,21,19,22,23,20,21,24]
ids = []
for i in range(1, 11):
    ids.append(f"H{i:02d}")
series = pd.Series(ages, index=ids)
if len(series) != 10:
    raise ValueError("Cần đủ 10 tuổi học viên.")
print("Học viên:", len(series))
print("Tuổi H07:", int(series.loc["H07"]))
print(f"Tuổi TB: {series.mean():.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo Series tuổi gắn nhãn H01..H10, đọc H07 và tính trung bình.
# Dữ liệu: 10 tuổi học viên tính theo năm; mã H01 đến H10 ghép cùng vị trí.
# Pandas dùng Series cho dãy một chiều có nhãn.
import pandas as pd
# 10 số nguyên là tuổi theo thứ tự học viên.
ages = [18,19,20,21,19,22,23,20,21,24]
ids = []
# range(1,11) tạo 1..10 và không lấy 11.
for i in range(1, 11):
    # f-string :02d thêm số 0 ở đầu mã, ví dụ H01.
    ids.append(f"H{i:02d}")
# index=ids gán mã người học làm nhãn cho từng giá trị tuổi.
series = pd.Series(ages, index=ids)
# len() đếm 10 mục trong Series, không phải số cột.
if len(series) != 10:
    # ValueError thông báo dữ liệu thiếu/sai và dừng, tránh tiếp tục kết luận từ bảng không đúng.
    raise ValueError("Cần đủ 10 tuổi học viên.")
# len() đếm 10 mục trong Series, không phải số cột.
print("Học viên:", len(series))
# .loc tra NHÃN H07, int() đổi scalar số về int Python để in.
print("Tuổi H07:", int(series.loc["H07"]))
# .mean() cộng tuổi rồi chia số học viên hợp lệ; :.1f chỉ định hiển thị 1 chữ số thập phân.
print(f"Tuổi TB: {series.mean():.1f}")
```

Kết quả khi chạy:

```
Học viên: 10
Tuổi H07: 23
Tuổi TB: 20.7
```

Nhãn H07 là mã học viên, không phải vị trí 7. Tuổi trung bình là thống kê của 10 người, không dùng để suy ra độ tuổi của một người cụ thể.

Tình huống 2 — Cấp 2: DataFrame của 12 dòng giỏ hàng với các cột có đơn vị

Từ vựng trong mục: DataFrame — bảng dữ liệu; **Column** — cột; **Calculated field** — cột tính.

Tình huống thực tế: Quầy sách có **12 dòng mặt hàng P01 đến P12** trong giỏ mua hàng. Mỗi dòng ghi **giá một món** tính bằng **nghìn đồng/món** và **số lượng món** mua. Ví dụ một món giá 10 nghìn đồng, mua hai món thì thành tiền 20 nghìn đồng. Thu ngân cần thêm cột thành tiền cho mỗi dòng và tính tổng phải trả của cả giỏ; không được cộng giá từng món mà quên số lượng.

Vì sao chọn cách này? DataFrame là bảng có **hàng và cột đặt tên**: mỗi hàng là một mặt hàng, các cột chứa mã `sku`, đơn giá và số lượng. Pandas tính `df['price_thousand'] * df['quantity']` **theo từng hàng**, tạo cột tiền tương ứng với đúng món và số lượng, không lấy giá P01 nhân nhầm với lượng P02. Sau đó `sum()` cộng **cột thành tiền**, không cộng cột đơn giá, để ra số tiền cả giỏ. Vì giá dùng nghìn đồng/món còn số lượng dùng món, cột thành tiền là **nghìn đồng**.

Quy trình:

1. Ghép ba danh sách mã hàng, giá, số lượng đồng hàng.
2. Tạo DataFrame và thêm `amount`.
3. Đếm bản ghi, kiểm tra thành tiền mặt hàng đầu và tổng toàn giỏ.

No Note — Không chú thích

```
import pandas as pd
prices = [10,15,20,12,30,18,14,25,11,17,35,22]
quantities = [2,1,4,3,2,5,1,2,4,2,1,3]
sku_ids = []
for i in range(1, 13):
    sku_ids.append(f"P{i:02d}")
df = pd.DataFrame({"sku": sku_ids,
                   "price_thousand": prices, "quantity": quantities})
df["amount_thousand"] = df["price_thousand"] * df["quantity"]
if df.shape != (12, 4):
    raise ValueError("Bảng mặt hàng phải có 12 dòng và 4 cột.")
print("Mặt hàng:", len(df))
print("P01 thành tiền:", int(df.loc[0, "amount_thousand"]))
print("Tổng (nghìn đồng):", int(df["amount_thousand"].sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo DataFrame 12 mặt hàng và tính thành tiền từng dòng bằng phép nhân cột.
# Dữ liệu: 12 SKU P01..P12; price_thousand là nghìn đồng/món và quantity là số món.
# DataFrame là bảng Pandas, phép nhân giữa hai Series căn theo index hàng.
import pandas as pd
# 12 đơn giá tính bằng nghìn đồng mỗi món.
prices = [10,15,20,12,30,18,14,25,11,17,35,22]
# 12 số lượng món, cùng vị trí với list giá.
quantities = [2,1,4,3,2,5,1,2,4,2,1,3]
sku_ids = []
# range(1,13) sinh 12 thứ tự, không lấy 13.
for i in range(1, 13):
    # append() thêm mã P01..P12; f-string :02d tạo hai chữ số.
    sku_ids.append(f"P{i:02d}")
# Các list sku/price_thousand/quantity phải khớp độ dài để ghép thành 12 hàng.
df = pd.DataFrame({"sku": sku_ids,
                   "price_thousand": prices, "quantity": quantities})
# Tạo cột tiền theo từng hàng: giá nghìn đồng/món × số món.
df["amount_thousand"] = df["price_thousand"] * df["quantity"]
# shape (12,4) nghĩa 12 hàng sản phẩm và bốn cột, gồm cột tiền mới.
if df.shape != (12, 4):
    # ValueError thông báo dữ liệu thiếu/sai và dừng, tránh tiếp tục kết luận từ bảng không đúng.
    raise ValueError("Bảng mặt hàng phải có 12 dòng và 4 cột.")
print("Mặt hàng:", len(df))
# .loc[0, 'amount_thousand'] tra hàng có nhãn index 0 và cột tên amount_thousand; int() để in.
```

```
print("P01 thành tiền:", int(df.loc[0, "amount_thousand"]))
# sum() cộng tiền của 12 hàng, đơn vị nghìn đồng.
print("Tổng (nghìn đồng):", int(df["amount_thousand"].sum()))
```

Kết quả khi chạy:

```
Mặt hàng: 12
P01 thành tiền: 20
Tổng (nghìn đồng): 544
```

Một hàng tương ứng một mặt hàng của giỏ; đơn giá tính theo nghìn đồng, số lượng theo món, thành tiền theo nghìn đồng.

Tình huống 3 — Cấp 3: Đọc 15 đơn CSV và đối chiếu schema

Từ vựng trong mục: read_csv — đọc CSV; **Schema** — cấu trúc dữ liệu; **dtype** — kiểu cột.

Tình huống thực tế: Cửa hàng tạo **15 đơn O01 đến O15** trong một bảng **CSV giả lập**. Mỗi dòng có mã đơn `id` (chẳng hạn O01), kênh nhận đơn `web`, `store` hoặc `phone`, và tiền đơn tính bằng **đồng**. Người quản lý muốn đọc bảng, xem đủ 15 đơn và các tên cột, đếm đơn qua web và tổng doanh thu; mã đơn phải được giữ dưới dạng chữ, không bị đổi thành số.

Vì sao chọn cách này? CSV là dữ liệu dạng văn bản với các cột được đặt tên và ngăn cách, ví dụ `id,channel,amount_vnd`. `StringIO` giả làm một tệp trong bộ nhớ, để học viên chạy ví dụ không cần tải file ngoài. `pd.read_csv(..., dtype={'id': 'string'})` yêu cầu giữ mã đơn như **chuỗi định danh**, kể cả khi một hệ thống khác có mã bắt đầu bằng số 0; không nên coi mã là giá trị để cộng. Cột `amount_vnd` được đọc thành số để cộng tiền, còn điều kiện bằng `web` cho phép đếm kênh bán. Đếm dòng dữ liệu, không tính tiêu đề CSV như một đơn.

Quy trình:

1. Tạo CSV với một dòng tiêu đề và 15 đơn hàng.
2. Đọc qua pandas, chỉ định kiểu chuỗi cho mã đơn.
3. Kiểm tra schema, tổng đơn và số đơn từ kênh web.

No Note — Không chú thích

```
import pandas as pd
from io import StringIO
source_rows = [{"id":"O01","channel":"web","amount":10000},
{"id":"O02","channel":"store","amount":15000},{"id":"O03","channel":"phone","amount":20000},
{"id":"O04","channel":"web","amount":25000},{"id":"O05","channel":"store","amount":30000},
{"id":"O06","channel":"phone","amount":10000},{"id":"O07","channel":"web","amount":15000},
{"id":"O08","channel":"store","amount":20000},{"id":"O09","channel":"phone","amount":25000},
{"id":"O10","channel":"web","amount":30000},{"id":"O11","channel":"store","amount":10000},
{"id":"O12","channel":"phone","amount":15000},{"id":"O13","channel":"web","amount":20000},
{"id":"O14","channel":"store","amount":25000},{"id":"O15","channel":"phone","amount":30000}]
csv_lines = []
for row in source_rows:
    csv_lines.append(f"{row['id']},{row['channel']},{row['amount']}")
csv_text = "id,channel,amount_vnd\n" + "\n".join(csv_lines)
df = pd.read_csv(StringIO(csv_text), dtype={"id": "string"})
if len(df) != 15:
    raise ValueError("Cần đọc đúng 15 đơn từ CSV.")
print("Đơn:", len(df))
print("Cột:", ", ".join(df.columns))
```

```
print("Kênh web:", int((df["channel"] == "web").sum()))
print("Tổng (đồng):", int(df["amount_vnd"].sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Biến 15 đơn hàng thành CSV trong RAM, đọc bằng dtype chuỗi và đếm kênh web.
# Dữ liệu: 15 bản ghi gồm id đơn, channel web/store/phone, amount_vnd (đồng).
# Pandas đọc dữ liệu CSV và thao tác cột theo tên.
import pandas as pd
# StringIO biểu diễn CSV bằng chuỗi trong bộ nhớ, không phải file bên ngoài.
from io import StringIO
# 15 dict đơn; id là mã, channel là kênh, amount số tiền đồng.
source_rows = [{"id":"O01","channel":"web","amount":10000},
{"id":"O02","channel":"store","amount":15000},{"id":"O03","channel":"phone","amount":20000},
{"id":"O04","channel":"web","amount":25000},{"id":"O05","channel":"store","amount":30000},
{"id":"O06","channel":"phone","amount":10000},{"id":"O07","channel":"web","amount":15000},
{"id":"O08","channel":"store","amount":20000},{"id":"O09","channel":"phone","amount":25000},
{"id":"O10","channel":"web","amount":30000},{"id":"O11","channel":"store","amount":10000},
{"id":"O12","channel":"phone","amount":15000},{"id":"O13","channel":"web","amount":20000},
{"id":"O14","channel":"store","amount":25000},{"id":"O15","channel":"phone","amount":30000}]
# List sẽ lưu các dòng CSV từ 15 dict.
csv_lines = []
for row in source_rows:
    # append() thêm dòng theo thứ tự cột; f-string ghép các trường với dấu phẩy.
    csv_lines.append(f"{row['id']},{row['channel']},{row['amount']}")
# join() nối các dòng bằng xuống dòng; thêm header 'id,channel,amount_vnd'.
csv_text = "id,channel,amount_vnd\n" + "\n".join(csv_lines)
# read_csv() suy đoán các kiểu còn lại; dtype={'id':'string'} ép mã đơn về kiểu chuỗi để tránh mất
số 0 nếu mã là số.
df = pd.read_csv(StringIO(csv_text), dtype={"id": "string"})
# len() là số hàng đơn, kỳ vọng 15.
if len(df) != 15:
    # ValueError thông báo dữ liệu thiếu/sai và dừng, tránh tiếp tục kết luận từ bảng không đúng.
    raise ValueError("Cần đọc đúng 15 đơn từ CSV.")
# len() là số hàng đơn, kỳ vọng 15.
print("Đơn:", len(df))
# df.columns là danh sách nhãn cột; join() ghép các tên bằng dấu phẩy.
print("Cột:", ", ".join(df.columns))
# So từng hàng channel == 'web' thành Series bool; sum() đếm số True.
print("Kênh web:", int((df["channel"] == "web").sum()))
# sum() cộng số tiền trong cột đồng; int() đổi scalar sang số nguyên khi in.
print("Tổng (đồng):", int(df["amount_vnd"].sum()))
```

Kết quả khi chạy:

```
Đơn: 15
Cột: id, channel, amount_vnd
Kênh web: 5
Tổng (đồng): 300000
```

Mã `001` được giữ nguyên ký tự; phần cộng chỉ áp dụng cho doanh số (đồng), không áp dụng cho mã khách hay tên kênh.

Tình huống 4 — Cấp 4: Kiểm tra 18 hồ sơ đầu vào bằng info và to_numeric

Từ vựng trong mục: info — mô tả cột; **to_numeric** — đổi sang số; **Missing value** — giá trị thiếu.

Tình huống thực tế: Một biểu mẫu có **18 hồ sơ khách hàng**; mỗi dòng ghi tuổi (**năm**) và mức tiền đã chi (**đồng**). Hai dòng mức chi chưa có số hợp lệ: một để trống và một nhập `N/A`; tuổi của những người đó vẫn

có. Nhân viên cần biết cột tuổi và cột mức chi đã là kiểu số hay chưa, có bao nhiêu mức chi bị thiếu và tổng tiền chỉ tính từ các dòng có số.

Vì sao chọn cách này? `DataFrame` ban đầu có thể chứa **chữ và số trộn lẫn**, dù cột ấy vốn được dùng để tính tiền. `pd.to_numeric(errors='coerce')` thử chuyển mức chi sang số; các chuỗi `''` và `N/A` không phải số hợp lệ sẽ thành **NaN**, dấu hiệu thiếu dữ liệu, thay vì làm báo cáo dừng. `isna()` đếm các ô cần bổ sung; `dtypes/is_numeric_dtype` kiểm tra kiểu dữ liệu của cột sau khi chuyển. Tổng `sum()` chỉ cộng các giá trị có số, không tự coi hai khoản chưa khai bằng 0 đồng, vì chúng vẫn phải được báo là thiếu.

Quy trình:

1. Tạo bảng 18 dòng từ danh sách tuổi và mức chi dạng chữ.
2. Đổi kiểu cột mức chi có kiểm soát; kiểm tra kiểu dữ liệu.
3. Đếm số thiếu và tổng hợp những số hợp lệ.

No Note — Không chú thích

```
import pandas as pd
ages = [21,22,20,19,25,23,24,20,21,26,22,23,24,19,20,21,25,22]
raw_spent =
["100000","110000","120000","130000","140000","","160000","170000","180000","190000","200000","2100
00","220000","N/A","240000","250000","260000","270000"]
df = pd.DataFrame({"age_years": ages, "spent_vnd": raw_spent})
df["spent_vnd"] = pd.to_numeric(df["spent_vnd"], errors="coerce")
if len(df) != 18:
    raise ValueError("Cần đủ 18 hồ`sơ với tuổi và mức chi.")
print("Hồ`sơ:", len(df))
print("Tuổi là số:", bool(pd.api.types.is_numeric_dtype(df["age_years"])))
print("Mức chi là số:", bool(pd.api.types.is_numeric_dtype(df["spent_vnd"])))
print("Mức chi thiếú:", int(df["spent_vnd"].isna().sum()))
print("Tổng hợp lệ (đồng):", int(df["spent_vnd"].sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuẩn hóa 18 mức chi dạng chuôĩ sang số, nhận giá trị trống/N/A và kiểm tra kiểu cột.
# Dữ liệu: 18 hàng tuổi năm và spent_vnd chuôĩ đồng, có chuôĩ rỗng và 'N/A' là dữ liệu thiếú.
# Pandas hô˜trợ chuyển chuôĩ sang số, nhận diện giá trị thiếú.
import pandas as pd
# 18 sốtuổi (năm) tương ứng với từng mức chi cùng chỉ số.
ages = [21,22,20,19,25,23,24,20,21,26,22,23,24,19,20,21,25,22]
# 18 chuôĩ mức chi VND, trong đó '' và 'N/A' không là số.
raw_spent =
["100000","110000","120000","130000","140000","","160000","170000","180000","190000","200000","2100
00","220000","N/A","240000","250000","260000","270000"]
# Tạo DataFrame hai cột age_years và spent_vnd từ hai list cùng vị trí.
df = pd.DataFrame({"age_years": ages, "spent_vnd": raw_spent})
# to_numeric() cốchuyển chuôĩ sang số; errors='coerce' biến dữ liệu không chuyển được thành NaN
thay vì báo lôĩ.
df["spent_vnd"] = pd.to_numeric(df["spent_vnd"], errors="coerce")
if len(df) != 18:
    # ValueError thông báo dữ liệu thiếú/sai và dừng, tránh tiếp tục kết luận từ bảng không đúng.
    raise ValueError("Cần đủ 18 hồ`sơ với tuổi và mức chi.")
print("Hồ`sơ:", len(df))
# is_numeric_dtype() trả bool cho biết dtype của cột là kiểu số, không kiểm từng giá trị riêng lẻ.
print("Tuổi là số:", bool(pd.api.types.is_numeric_dtype(df["age_years"])))
# Kiểm cột chi tiêu đã là kiểu sốsau to_numeric, dùng bool() đôi kết quả sang bool Python.
print("Mức chi là số:", bool(pd.api.types.is_numeric_dtype(df["spent_vnd"])))
# isna() đánh dấú ô thiếú/NaN, sum() đếm sốô True.
print("Mức chi thiếú:", int(df["spent_vnd"].isna().sum()))
```

```
# sum() cộng các giá trị số hợp lệ, bỏ qua NaN theo mặc định; int() in tổng đồng.
print("Tổng hợp lệ (đồng):", int(df["spent_vnd"].sum()))
```

Kết quả khi chạy:

```
Hồ sơ: 18
Tuổi là số: True
Mức chi là số: True
Mức chi thiếu: 2
Tổng hợp lệ (đồng): 2950000
```

Pandas tự bỏ NaN khi tính `sum`, nhưng báo cáo vẫn phải hiển thị rõ **hai hồ sơ chưa đủ dữ liệu**. Tổng này là tổng mức chi của mẫu, không phải số tiền có được từ một khách.

Tình huống 5 — Cấp 5: Tạo bảng đặc trưng 20 học viên có nhãn kết quả

Từ vựng trong mục: Feature — đặc trưng; **Target** — nhãn dự đoán; **DataFrame schema** — cấu trúc bảng.

Tình huống thực tế: Một khóa học có **20 học viên H01 đến H20**. Mỗi dòng ghi mã, tuổi (**năm**), số giờ tự học trong tuần (**giờ/tuần**) và nhãn kết quả: `1` là đã hoàn thành, `0` là chưa hoàn thành. Hai người chưa được xác nhận kết quả nên nhãn để `None`. Trước khi đưa dữ liệu sang bước phân tích, giáo viên muốn tách 18 dòng đã có nhãn, đếm hai dòng chờ và số người đã hoàn thành; không được suy đoán nhãn từ tuổi hay giờ học.

Vì sao chọn cách này? Đặc trưng là thông tin đầu vào như tuổi và giờ tự học; **nhãn** là kết quả đã biết để đối chiếu khi học máy, ở đây là 0 hoặc 1. DataFrame giữ từng trường trong cột có tên để không nhầm nhãn với giờ học. `notna()` tạo điều kiện chọn đúng những hàng **đã có nhãn**; `loc` lấy 18 hàng đó thành bảng làm việc, còn bản gốc vẫn giữ cả hai hàng chờ. Kiểm tra `isin([0, 1])` giúp tránh vô tình cho nhãn 2 hoặc chữ lạ vào bộ dữ liệu. Đây mới là **chuẩn bị và kiểm tra bảng**, chưa huấn luyện hay đánh giá AI; không được tự gán nhãn chưa biết.

Quy trình:

1. Tạo 20 bản ghi với mã, tuổi, số giờ học và nhãn hoàn thành.
2. Kiểm tra khóa mã duy nhất, nhãn chỉ 0/1 hoặc thiếu.
3. Dùng `.loc` lấy các dòng có nhãn và in số cần bổ sung.

No Note — Không chú thích

```
import pandas as pd
student_ids = []
for i in range(1, 21):
    student_ids.append(f"H{i:02d}")
records = pd.DataFrame({
    "student_id": student_ids,
    "age_years": [18,19,20,21,22,23,24,25,26,18,19,20,21,22,23,24,25,26,18,19],
    "study_hours": [5,7,9,11,5,7,9,11,5,7,9,11,5,7,9,11,5,7,9,11],
    "completed": [1,1,0,0,1,1,None,0,1,1,0,0,1,1,0,0,1,1,None,0],
})
labelled = records.loc[records["completed"].notna()].copy()
if len(records) != 20 or not records["student_id"].is_unique:
    raise ValueError("Cần đủ 20 học viên có mã không trùng.")
if not labelled["completed"].isin([0, 1]).all():
    raise ValueError("Nhãn hoàn thành chỉ được bằng 0 hoặc 1.")
print("Học viên:", len(records))
```

```
print("Có nhãn:", len(labelled))
print("Chờ nhãn:", int(records["completed"].isna().sum()))
print("Hoàn thành:", int((labelled["completed"] == 1).sum()))
print("Cột đặc trưng:", ", ".join(["age_years", "study_hours"]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc 20 hồ`sơ học viên có nhãn hoàn thành, kiểm không trùng ID và đếm nhãn 1.
# Dữ liệu: 20 hàng: student_id, age_years (năm), study_hours (giờ), completed=1/0/None, None là
chưa có nhãn.
# Pandas hỗ˜trợ bảng có cột đặc trưng và cột nhãn.
import pandas as pd
student_ids = []
# range(1,21) tạo mã 1..20, cận 21 không được lấy.
for i in range(1, 21):
    # append() thêm mã học viên dạng H01..H20; :02d tạo hai chữ số́.
    student_ids.append(f"H{i:02d}")
# Mỗi list cùng độ dài tạo một cột, mỗi hàng là một học viên.
records = pd.DataFrame({
    "student_id": student_ids,
    "age_years": [18,19,20,21,22,23,24,25,26,18,19,20,21,22,23,24,25,26,18,19],
    "study_hours": [5,7,9,11,5,7,9,11,5,7,9,11,5,7,9,11,5,7,9,11],
    "completed": [1,1,0,0,1,1,None,0,1,1,0,0,1,1,0,0,1,1,None,0],
})
# .notna() tạo mask True cho hàng có nhãn hoàn thành 0/1, False với None/NaN.
# .loc[mask] lọc đúng hàng có nhãn; .copy() tạo bản sao độc lập để̉ có thể̉ xử̉ lý riêng.
labelled = records.loc[records["completed"].notna()].copy()
# .is_unique trả True khi tất cả mã học viên đều khác nhau, không cần tạo set thủ công.
if len(records) != 20 or not records["student_id"].is_unique:
    # ValueError thông báo dữ liệu thiếu/sai và dừng, tránh tiếp tục kết luận từ bảng không đúng.
    raise ValueError("Cần đủ 20 học viên có mã không trùng.")
# .isin([0,1]) tạo mask nhãn hợp lệ; .all() yêu cầu tất cả hàng đã chọn đều đúng 0 hoặc 1.
if not labelled["completed"].isin([0, 1]).all():
    # ValueError thông báo dữ liệu thiếu/sai và dừng, tránh tiếp tục kết luận từ bảng không đúng.
    raise ValueError("Nhãn hoàn thành chỉ được bằng 0 hoặc 1.")
print("Học viên:", len(records))
print("Có nhãn:", len(labelled))
# .isna() nhận nhãn thiếu, .sum() đếm số́ hồ`sơ chờ bổ̉ sung.
print("Chờ nhãn:", int(records["completed"].isna().sum()))
# So từng hàng với 1 rồi .sum() đếm số́ học viên hoàn thành.
print("Hoàn thành:", int((labelled["completed"] == 1).sum()))
# join() ghép hai tên cột đặc trưng; completed là cột nhãn, không phải đầu vào dự đoán.
print("Cột đặc trưng:", ", ".join(["age_years", "study_hours"]))
```

Kết quả khi chạy:

```
Học viên: 20
Có nhãn: 18
Chờ nhãn: 2
Hoàn thành: 10
Cột đặc trưng: age_years, study_hours
```

Không sử dụng các dòng thiếu nhãn để báo tỷ lệ hoàn thành. Bảng đặc trưng hai cột và nhãn là các khái niệm khác nhau; không suy luận kết quả học tập từ dữ liệu giả lập.

6. Sai lầm thường gặp

- **Series** một chiều, **DataFrame** hai chiều. `df['x']` chọn Series; `df[['x']]` giữ dạng DataFrame một cột.

- Phải hiểu **index** là nhãn hàng, không nhất thiết là cột dữ liệu. Pandas căn dữ liệu theo index khi kết hợp các Series.
- Đơn giá, số lượng và doanh thu có **đơn vị khác nhau**; nhân `price × qty` mới có ý nghĩa là thành tiền.
- CSV vốn là văn bản; cần xem lại `dtype` vì dữ liệu bẩn có thể khiến cột số bị đọc thành chuỗi.
- Không đưa mã định danh hoặc nhãn `city` vào mô hình dự đoán chỉ vì chúng có mặt trong bảng; phải chọn đặc trưng theo mục tiêu.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a Pandas Series? | Pandas Series là gì?

A. Parse tabular CSV data into a DataFrame | Đọc dữ liệu CSV thành DataFrame

B. One field across multiple records | Một thuộc tính qua nhiều bản ghi

C. A two-dimensional labeled table | Bảng dữ liệu hai chiều có nhãn

D. A one-dimensional labeled data structure | Cấu trúc dữ liệu một chiều có nhãn

Đáp án: D

Giải thích: Series liên kết từng giá trị với một nhãn index. Ví dụ `pd.Series([5,6], index=['a','b'])` gắn nhãn `'a'`, `'b'` cho các giá trị, khác với List không có hệ nhãn riêng.

Q2. What is a Pandas DataFrame? | Pandas DataFrame là gì?

A. A one-dimensional labeled data structure | Cấu trúc dữ liệu một chiều có nhãn

B. A two-dimensional labeled table | Bảng dữ liệu hai chiều có nhãn

C. Parse tabular CSV data into a DataFrame | Đọc dữ liệu CSV thành DataFrame

D. One field across multiple records | Một thuộc tính qua nhiều bản ghi

Đáp án: B

Giải thích: DataFrame tổ chức bản ghi thành hàng và trường thành cột. Bảng gồm ba quan sát và hai trường có shape `(3,2)`; Series lại là cấu trúc một chiều có nhãn.

Q3. What does a column represent? | Một cột trong bảng thể hiện gì?

A. A one-dimensional labeled data structure | Cấu trúc dữ liệu một chiều có nhãn

B. One field across multiple records | Một thuộc tính qua nhiều bản ghi

C. A two-dimensional labeled table | Bảng dữ liệu hai chiều có nhãn

D. Parse tabular CSV data into a DataFrame | Đọc dữ liệu CSV thành DataFrame

Đáp án: B

Giải thích: Cột lưu cùng một thuộc tính ở các hàng. Nếu các hàng là đơn hàng, cột `price` ghi giá của từng đơn và cần xác định rõ đơn vị tiền tệ.

Q4. What is the purpose of `pd.read_csv()`? | `pd.read_csv()` dùng để làm gì?

A. A one-dimensional labeled data structure | Cấu trúc dữ liệu một chiều có nhãn

B. One field across multiple records | Một thuộc tính qua nhiều bản ghi

C. A two-dimensional labeled table | Bảng dữ liệu hai chiều có nhãn

D. Parse tabular CSV data into a DataFrame | Đọc dữ liệu CSV thành DataFrame

Đáp án: D

Giải thích: read_csv đọc các trường được phân cách cùng hàng tên cột. Ví dụ hàng tiêu đề CSV `name,score` tạo tên cột tương ứng, nhưng các ô số sai định dạng vẫn có thể cần làm sạch sau khi đọc.

Q5. What does `pd.Series([1,2])` return? | `pd.Series([1,2])` trả về gì?

A. A Series containing column x | Series chứa cột x

B. A Series with default index 0,1 | Series có index mặc định 0,1

C. A DataFrame containing one column | DataFrame có một cột

D. A tuple with row and column counts | Tuple gồm số hàng và số cột

Đáp án: B

Giải thích: Không chỉ định nhãn thì Pandas tự gán chỉ số nguyên. Không truyền index, hai giá trị nhận nhãn `0` và `1`; kết quả là Series chứ không phải DataFrame một cột.

Q6. What does `df[['x']]` return? | `df[['x']]` trả về gì?

A. A DataFrame containing one column | DataFrame có một cột

B. A tuple with row and column counts | Tuple gồm số hàng và số cột

C. A Series containing column x | Series chứa cột x

D. A Series with default index 0,1 | Series có index mặc định 0,1

Đáp án: A

Giải thích: Hai lớp ngoặc vuông chọn danh sách cột và giữ dạng bảng. Nếu `df` có năm hàng, `df[['x']]` giữ kết quả hai chiều với shape `(5,1)`, khác `df['x']`.

Q7. When DataFrame column labels are unique, what does `df['x']` return? | Khi tên cột DataFrame không trùng nhau, `df['x']` trả về gì?

A. A tuple with row and column counts | Tuple gồm số hàng và số cột

B. A Series containing column x | Series chứa cột x

C. A DataFrame containing one column | DataFrame có một cột

D. A Series with default index 0,1 | Series có index mặc định 0,1

Đáp án: B

Giải thích: Tra cứu một cột bằng ngoặc đơn trả Series. Với tên cột không trùng, `df['x']` là Series một chiều; hai lớp ngoặc vuông thì trả DataFrame.

Q8. What does `df.shape` return? | `df.shape` trả về gì?

A. A Series with default index 0,1 | Series có index mặc định 0,1

B. A DataFrame containing one column | DataFrame có một cột

C. A Series containing column x | Series chứa cột x

D. A tuple with row and column counts | Tuple gồm số hàng và số cột

Đáp án: D

Giải thích: Shape cho biết hai kích thước của DataFrame. Với DataFrame bốn bản ghi và ba trường, `df.shape` là `(4,3)`, không phải số ô không thiếu.

Q9. What does `df.head(3)` show? | `df.head(3)` hiển thị gì?

A. Columns, non-null counts and data types | Các cột, số ô không thiếu và kiểu dữ liệu

B. Data types of columns | Kiểu dữ liệu của các cột

C. The first three rows | Ba hàng đầu

D. Descriptive statistics such as mean and quartiles | Thống kê mô tả như trung bình và các tứ phân vị

Đáp án: C

Giải thích: head lấy số hàng đầu theo đối số. Nếu DataFrame chỉ có hai hàng, `head(3)` trả hai hàng đó chứ không tạo thêm hàng giả.

Q10. What does `df.dtypes` describe? | `df.dtypes` mô tả gì?

A. Descriptive statistics such as mean and quartiles | Thống kê mô tả như trung bình và các tứ phân vị

B. The first three rows | Ba hàng đầu

C. Data types of columns | Kiểu dữ liệu của các cột

D. Columns, non-null counts and data types | Các cột, số ô không thiếu và kiểu dữ liệu

Đáp án: C

Giải thích: Mỗi cột có thể mang dtype riêng. Cột `price` có thể có dtype thực, còn `name` mang kiểu văn bản; `dtypes` không tính thống kê mô tả.

Q11. What does `df.info()` primarily summarize? | `df.info()` chủ yếu tóm tắt gì?

A. The first three rows | Ba hàng đầu

B. Data types of columns | Kiểu dữ liệu của các cột

C. Descriptive statistics such as mean and quartiles | Thống kê mô tả như trung bình và các tứ phân vị

D. Columns, non-null counts and data types | Các cột, số ô không thiếu và kiểu dữ liệu

Đáp án: D

Giải thích: info cho biết cấu trúc và mức độ đầy đủ của bảng. Nó hiển thị tên cột, số giá trị không thiếu, dtype và bộ nhớ; khác thao tác xem trước hàng dữ liệu bằng `head()`.

Q12. What does `df.describe()` commonly provide for numeric columns? | `df.describe()` thường cung cấp gì cho cột số?

A. Descriptive statistics such as mean and quartiles | Thống kê mô tả như trung bình và các tứ phân vị

B. The first three rows | Ba hàng đầu

C. Columns, non-null counts and data types | Các cột, số ô không thiếu và kiểu dữ liệu

D. Data types of columns | Kiểu dữ liệu của các cột

Đáp án: A

Giải thích: describe tính những thống kê cơ bản cho cột số. Với số `[2,4,6]`, trung bình bằng 4 và các tứ phân vị cho biết mức phân tán; `head()` chỉ hiển thị hàng dữ liệu gốc.

Q13. What is an index in a DataFrame? | Index của DataFrame là gì?

A. Column labels | Các tên cột

B. One observation or record | Một quan sát hoặc bản ghi

C. Labels identifying rows | Các nhãn xác định hàng

D. Index labels | Nhãn index

Đáp án: C

Giải thích: Index tách với các cột thông thường trừ khi được chuyển đổi. Index có thể là nhãn như mã học viên và tách khỏi các cột dữ liệu thường, trừ khi được đưa trở lại thành cột.

Q14. How are Series values aligned during arithmetic? | Pandas căn các giá trị Series khi tính toán bằng gì?

A. Labels identifying rows | Các nhãn xác định hàng

B. One observation or record | Một quan sát hoặc bản ghi

C. Index labels | Nhãn index

D. Column labels | Các tên cột

Đáp án: C

Giải thích: Pandas ghép giá trị theo các nhãn index tương ứng. Nếu hai Series sắp nhãn theo thứ tự khác nhau, Pandas ghép giá trị cùng nhãn trước khi cộng chứ không cộng máy móc theo vị trí.

Q15. What does `df.columns` contain? | `df.columns` chứa gì?

A. One observation or record | Một quan sát hoặc bản ghi

B. Index labels | Nhãn index

C. Column labels | Các tên cột

D. Labels identifying rows | Các nhãn xác định hàng

Đáp án: C

Giải thích: Thuộc tính columns liệt kê tên trường. Với các cột `name` và `age`, `list(df.columns)` trả danh sách tên cột, không phải nhãn hàng.

Q16. What is a DataFrame row usually intended to represent? | Mỗi hàng DataFrame thường nên biểu diễn gì?

A. One observation or record | Một quan sát hoặc bản ghi

B. Column labels | Các tên cột

C. Index labels | Nhãn index

D. Labels identifying rows | Các nhãn xác định hàng

Đáp án: A

Giải thích: Ý nghĩa hàng nhất quán giúp hiểu đúng các phép tổng hợp. Một hàng có thể là một học viên, giao dịch hoặc lần đo, nhưng cần định nghĩa nhất quán để hiểu đúng thống kê.

Q17. What does `df['price'] * df['qty']` calculate? | `df['price'] * df['qty']` tính gì?

A. An element-wise product aligned by index | Tích từng phần tử được căn theo index

B. To interpret the data and calculations correctly | Để hiểu đúng dữ liệu và phép tính

C. A numeric column may be parsed as text | Cột số có thể bị đọc thành chữ

D. Two named columns | Hai cột được nêu tên

Đáp án: A

Giải thích: Giá và lượng tương ứng được nhân ở từng bản ghi. Nếu cùng một hàng có giá 10 đô la và số lượng 3 món, tích là 30 đô la, không phải tổng của nhiều hàng.

Q18. What does `df[['a','b']]` select? | `df[['a','b']]` chọn gì?

A. A numeric column may be parsed as text | Cột số có thể bị đọc thành chữ

B. An element-wise product aligned by index | Tích từng phần tử được căn theo index

C. To interpret the data and calculations correctly | Để hiểu đúng dữ liệu và phép tính

D. Two named columns | Hai cột được nêu tên

Đáp án: D

Giải thích: Danh sách tên cho phép lấy nhiều cột. Kết quả vẫn là DataFrame chứa `a` và `b` theo thứ tự đã nêu, còn `df['a']` thường trả Series.

Q19. What is a frequent risk when loading CSV numeric data? | Nguy cơ thường gặp khi đọc số từ CSV là gì?

A. A numeric column may be parsed as text | Cột số có thể bị đọc thành chữ

B. Two named columns | Hai cột được nêu tên

C. An element-wise product aligned by index | Tích từng phần tử được căn theo index

D. To interpret the data and calculations correctly | Để hiểu đúng dữ liệu và phép tính

Đáp án: A

Giải thích: Giá trị không hợp lệ có thể khiến Pandas không nhận kiểu số. Ô như `'unknown'` nằm trong cột số có thể khiến kiểu bị suy ra thành văn bản/object, nên phải kiểm tra và đổi kiểu trước khi tính.

Q20. Why define what each row and column means? | Vì sao cần nêu ý nghĩa từng hàng và cột?

A. A numeric column may be parsed as text | Cột số có thể bị đọc thành chữ

B. To interpret the data and calculations correctly | Để hiểu đúng dữ liệu và phép tính

C. An element-wise product aligned by index | Tích từng phần tử được căn theo index

D. Two named columns | Hai cột được nêu tên

Đáp án: B

Giải thích: Ý nghĩa hàng và cột quyết định cách diễn giải kết quả. Nếu hàng là giao dịch và cột gồm giá cùng số lượng, nêu rõ đơn vị giúp tránh nhân nhầm trường và hiểu sai doanh thu.

8. Nối bài tiếp theo

`NumPy ndarray → Pandas DataFrame → Làm sạch → EDA/AI.`

PY-18 — Pandas: missing values, duplicates và kiểu dữ liệu

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Missing value — giá trị thiếu; **Duplicate** — bản ghi trùng.

`DataFrame → Data Cleaning → EDA → ML Pipeline`

Học xong dùng để làm gì? Phát hiện và xử lý dữ liệu thiếu, trùng; tránh dùng thông tin test khi điền khuyết.

Kết nối kiến thức tiếp theo: PY-19 ghép những bảng dữ liệu sạch và tổng hợp theo nhóm.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Missing value	giá trị thiếu	Ô không chứa số liệu hợp lệ
Duplicate	bản ghi trùng	Hai hàng dữ liệu giống nhau
Imputation	điền khuyết	Thay ô trống bằng giá trị phù hợp
Data validation	kiểm tra dữ liệu	Xác nhận dữ liệu đúng kiểu và giới hạn

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`df.isna().sum()`	Đếm số ô khuyết theo cột
`df.fillna()/dropna()`	Điền hoặc bỏ dữ liệu thiếu; cân nhắc mất mẫu
`df.duplicated()/drop_duplicates()`	Phát hiện và loại bỏ hàng trùng
`pd.to_numeric(errors='coerce')`	Chuyển chuỗi lỗi thành NaN để xử lý có kiểm soát

4. Khái niệm liên quan

Data cleaning (làm sạch dữ liệu) là quá trình tìm và xử lý các giá trị thiếu, dòng lặp và giá trị sai kiểu trước khi phân tích. Với dữ liệu khách hàng và đơn hàng, ta đi từ **phát hiện** thiếu/trùng → **quyết định** xử lý → **kiểm tra** kết quả; không xóa hoặc điền giá trị chỉ vì hàm có sẵn.

4.1. Missing values — Phát hiện dữ liệu thiếu

Từ vựng trong mục: Missing value — giá trị thiếu; **NaN** — giá trị số thiếu.

Giá trị thiếu là ô chưa có dữ liệu hợp lệ, chẳng hạn ba khách hàng có tuổi `[20, None, 25]` và khách thứ hai chưa khai báo. Trong DataFrame, **mỗi hàng là một khách**, cột `age` là tuổi (năm). Pandas thường biểu diễn số thiếu là `NaN`; `isna()` tạo giá trị Boolean đánh dấu ô thiếu, rồi `sum()` đếm số trường hợp `True`. Thiếu khác với tuổi 0: không được tùy tiện coi khách chưa nhập tuổi là trẻ sơ sinh.

No Note — Không chú thích

```
import pandas as pd

customers = pd.DataFrame({'age': [20, None, 25]})
print(customers['age'].isna().sum())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm tuổi còn thiếu trong ba hồ sơ khách hàng.
# Dữ liệu: Ba hàng khách hàng, cột age tuổi (năm), None nghĩa chưa biết.
# Pandas DataFrame quản lý dữ liệu dạng bảng và đánh dấu ô trống.
import pandas as pd

# Tạo một cột age chứa tuổi 20, None và 25; None thường được Pandas coi là giá trị thiếu NaN.
customers = pd.DataFrame({'age': [20, None, 25]})
# .isna() tạo mask True tại tuổi thiếu, .sum() đếm số True (một hồ sơ), không phải cộng tuổi.
print(customers['age'].isna().sum())
```

Kết quả khi chạy:

```
1
```

Có một khách chưa có tuổi. Kết quả là **số ô thiếu**, không phải giá trị tuổi của khách.

4.2. Duplicates — Nhận diện dòng trùng

Từ vựng trong mục: Duplicate — bản ghi trùng; **Row** — hàng.

Dữ liệu có thể chứa một giao dịch được ghi hai lần. Ví dụ bảng gồm các dòng `(O1, 10)`, `(O1, 10)`, `(O2, 20)`: **mỗi hàng là một giao dịch**, cột `id` là mã đơn, `amount` là tiền. `duplicated()` đánh dấu dòng lặp giống hệt **sau dòng đầu tiên**; `drop_duplicates()` giữ một bản. Tuy nhiên hai giao dịch cùng mã chưa chắc trùng hoàn toàn nếu số tiền hoặc chi tiết khác nhau — phải xác định quy tắc nhận diện trùng trước khi xóa.

No Note — Không chú thích

```
import pandas as pd

orders = pd.DataFrame({'id': ['O1', 'O1', 'O2'], 'amount': [10, 10, 20]})
print(orders.duplicated().tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phát hiện hàng đơn đặt giống hệt hàng trước trong bảng gồm ba bản ghi.
# Dữ liệu: Ba hàng đơn (id,amount); cặp (O1,10) xuất hiện hai lần, (O2,20) một lần.
# Pandas cung cấp phép kiểm hàng trùng trong DataFrame.
import pandas as pd

# Tạo bảng id và amount; mỗi list cùng vị trí tạo một hàng.
```

```
orders = pd.DataFrame({'id': ['O1', 'O1', 'O2'], 'amount': [10, 10, 20]})
# .duplicated() đánh dấu bản sao thứ hai trở đi của HÀNG GIỐNG NHAU trên mọi cột; tolist() đổi mask
thành list bool để in.
print(orders.duplicated().tolist())
```

Kết quả khi chạy:

```
[False, True, False]
```

Dòng thứ hai lặp đúng mọi cột của dòng đầu, nên chỉ dòng ấy được đánh dấu trùng.

4.3. Imputation — Điền dữ liệu thiếu hợp lý

Từ vựng trong mục: Imputation — điền khuyết; **Median** — trung vị.

Điền khuyết thay giá trị thiếu bằng một giá trị được lựa chọn theo ý nghĩa dữ liệu. Giả sử ba mức chi tiêu là `[100, None, 200]` nghìn đồng. Trung vị của hai số có thật là `(100+200)/2 = 150`; `fillna(median)` điền 150 vào ô thiếu. Không có một cách điền đúng cho mọi bài toán: với mã đơn thiếu nên xác minh nguồn, không tự thay bằng số trung vị. Nếu có tập train/test, thống kê dùng để điền phải học từ **train** để tránh rò rỉ dữ liệu.

No Note — Không chú thích

```
import pandas as pd

spent = pd.Series([100, None, 200], dtype=float)
median = spent.median()
print(spent.fillna(median).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thay một giá trị chi tiêu thiếu bằng trung vị của các số có sẵn.
# Dữ liệu: Ba mức chi 100, None và 200 (đơn vị minh họa); thiếu không phải bằng 0.
# Series hỗ trợ thống kê và điền dữ liệu bị thiếu.
import pandas as pd

# dtype=float cho phép Series lưu NaN bên cạnh số thực.
spent = pd.Series([100, None, 200], dtype=float)
# .median() lấy trung vị của 100 và 200, mặc định bỏ qua NaN, kết quả 150.
median = spent.median()
# .fillna(median) thay mỗi NaN bằng 150 trong Series kết quả; .tolist() xuất list, không sửa spent
gốc theo mặc định.
print(spent.fillna(median).tolist())
```

Kết quả khi chạy:

```
[100.0, 150.0, 200.0]
```

Ô thứ hai được điền 150 nghìn đồng; hai số đo hợp lệ 100 và 200 được giữ nguyên.

4.4. Data validation — Kiểm tra giá trị và kiểu

Từ vựng trong mục: Validation — kiểm tra tính hợp lệ; **Coerce** — chuyển giá trị sai thành thiếu.

Kiểm tra dữ liệu bảo đảm kiểu và phạm vi phù hợp trước khi tính. Một bảng giá nhận ba chuỗi `'100'`, `'bad'`, `'250'`. `pd.to_numeric(errors='coerce')` chuyển chuỗi số thành số thực, còn chữ `bad` không đổi được thành `NaN`. Sau đó mới quyết định bỏ hàng, điền khuyết hay yêu cầu sửa ở nguồn. Nếu dùng `errors='raise'`, trường hợp không hợp lệ sẽ phát sinh lỗi để chương trình xử lý.

No Note — Không chú thích

```
import pandas as pd

raw = pd.Series(['100', 'bad', '250'])
prices = pd.to_numeric(raw, errors='coerce')
print(prices.isna().sum())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuyển chuỗi giá thành số, đánh dấu 'bad' là thiếu để đếm.
# Dữ liệu: Ba chuỗi đầu vào '100','bad','250', bad không thể chuyển thành số.
# Pandas cung cấp to_numeric() để chuẩn hóa dữ liệu số dạng chuỗi.
import pandas as pd

# Mỗi phần tử là chuỗi từ dữ liệu gốc, chưa phải số nguyên.
raw = pd.Series(['100', 'bad', '250'])
# to_numeric() cố chuyển sang số; errors='coerce' đặt NaN khi gặp 'bad', không dừng toàn bộ.
prices = pd.to_numeric(raw, errors='coerce')
# isna() xác định vị trí NaN mới tạo; sum() đếm số giá không hợp lệ (1).
print(prices.isna().sum())
```

Kết quả khi chạy:

```
1
```

Có đúng một giá trị giá không hợp lệ. Chuyển thành NaN giúp phát hiện, không có nghĩa đã sửa được giá trị thật.

5. Năm tình huống làm sạch dữ liệu — từ phát hiện đến tránh leakage

Tình huống 1 — Cấp 1: Phát hiện tuổi thiếu trong 10 hồ sơ

Từ vựng trong mục: Missing value — giá trị thiếu; **isna** — kiểm tra ô trống.

Tình huống thực tế: Trung tâm tuyển sinh có **10 hồ sơ H01–H10**. Mỗi hàng gồm mã học viên và tuổi tính bằng **năm**; hai người H03, H08 chưa khai tuổi nên ô dữ liệu là `None`. Nhân viên muốn đếm chính xác hai hồ sơ cần bổ sung và liệt kê mã của họ, trong khi giữ nguyên tuổi đã có của tám người còn lại. Người chưa khai tuổi không có nghĩa là 0 tuổi.

Vì sao chọn cách này? `isna()` tạo kết quả **đúng/sai theo từng dòng**: chỗ nào chưa có tuổi được đánh `True`. Cộng các giá trị `True` cho biết số hồ sơ thiếu, còn `df.loc[missing, 'id']` lấy **mã đúng của những hồ sơ đó**, không phải mã tùy ý theo thứ tự. Bài này chỉ phát hiện và báo thiếu; chưa có cơ sở để biết tuổi thật nên **không xóa hồ sơ và không tự điền 0**. Làm vậy giúp người tiếp nhận biết chính xác phải hỏi lại ai.

Quy trình:

1. Tạo DataFrame 10 mã học viên và tuổi.

2. Dùng `isna()` trên cột tuổi để xác định mã thiếu.
3. In tổng số hồ sơ, số thiếu và mã cần bổ sung.

No Note — Không chú thích

```
import pandas as pd
ids = []
for i in range(1, 11):
    ids.append(f"H{i:02d}")
df = pd.DataFrame({"id": ids,
                   "age": [25,28,None,31,22,29,35,None,26,32]})
missing = df["age"].isna()
if len(df) != 10:
    raise ValueError("Cần đủ 10 hồ sơ tuổi.")
print("Hồ sơ:", len(df))
print("Thiếu tuổi:", int(missing.sum()))
print("Cần bổ sung:", ", ".join(df.loc[missing, "id"]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm mã của những học viên chưa có tuổi trong bảng 10 hồ sơ.
# Dữ liệu: 10 dòng H01..H10, cột age tuổi năm có hai None.
# Pandas dùng mask Boolean để lọc hàng thiếu.
import pandas as pd
ids = []
# range(1,11) tạo số thứ tự H01..H10, không lấy 11.
for i in range(1, 11):
    # append() lưu mã; f-string :02d thêm 0 đầu mã nhỏ hơn 10.
    ids.append(f"H{i:02d}")
# Ghép list mã và tuổi cùng chỉ số thành từng hồ sơ.
df = pd.DataFrame({"id": ids,
                   "age": [25,28,None,31,22,29,35,None,26,32]})
# .isna() tạo Series bool True ở dòng chưa có tuổi, khác tuổi 0.
missing = df["age"].isna()
# len() đếm tổng 10 hồ sơ, cả hồ sơ thiếu tuổi.
if len(df) != 10:
    # ValueError thông báo dữ liệu sai và dừng xử lý nếu không thể đổi chiếu số lượng hoặc chất
lượng nguồn.
    raise ValueError("Cần đủ 10 hồ sơ tuổi.")
# len() đếm tổng 10 hồ sơ, cả hồ sơ thiếu tuổi.
print("Hồ sơ:", len(df))
print("Thiếu tuổi:", int(missing.sum()))
# .loc[mask,'id'] lấy mã những hàng mask True; join() nối mã bằng dấu phẩy để thông báo.
print("Cần bổ sung:", ", ".join(df.loc[missing, "id"]))
```

Kết quả khi chạy:

```
Hồ sơ: 10
Thiếu tuổi: 2
Cần bổ sung: H03, H08
```

Hai học viên còn thiếu tuổi vẫn tồn tại trong báo cáo; số 0 không được dùng làm tuổi điền vào.

Tình huống 2 — Cấp 2: Điền trung vị cho hai trong 12 mức chi

Từ vựng trong mục: Imputation — điền khuyết; **Median** — trung vị; **Copy** — bản sao.

Tình huống thực tế: Cửa hàng khảo sát mức chi tiêu của **12 khách hàng**; mỗi con số tính bằng **nghìn đồng/khách**. Khách thứ tư và thứ chín chưa khai mức chi (`None`). Để thử một bước phân tích, nhóm muốn

điền vào hai chỗ trống bằng **trung vị** tính từ mười người có số liệu, nhưng vẫn giữ lại bảng gốc để phân biệt thông tin thật với số được điền tạm.

Vì sao chọn cách này? Trung vị là số nằm ở **giữa dãy đã sắp xếp**; với mười số hợp lệ, nó nằm giữa hai giá trị giữa của dãy. Trung vị thường ít bị ảnh hưởng bởi một khoản chi quá lớn hơn trung bình, nên bài chọn nó làm **giá trị thử điền**. `median()` bỏ qua `NaN` khi tính trên cột mức chi; `copy()` tạo bảng làm việc riêng; `fillna(median)` chỉ thay các vị trí thiếu trong bảng ấy. Đây là **số ước lượng**, không phải khoản tiền khách đã thật sự chi; bảng `raw` phải giữ hai vị trí trống để còn đối chiếu.

Quy trình:

1. Tạo 12 khoản chi và đếm số thiếu trước sửa.
2. Tính trung vị từ dữ liệu có thật; sao chép bảng rồi `fillna`.
3. So sánh số thiếu của bản gốc và bản thử, in giá trị đã điền.

No Note — Không chú thích

```
import pandas as pd
raw = pd.DataFrame({"spent_thousand": [100,120,140,None,160,180,200,220,None,240,260,280]})
median = raw["spent_thousand"].median()
working = raw.copy()
working["spent_thousand"] = working["spent_thousand"].fillna(median)
if len(raw) != 12:
    raise ValueError("Cần đủ 12 khách hàng.")
print("Khách:", len(raw))
print("Thiếu ban đầu:", int(raw["spent_thousand"].isna().sum()))
print(f"Trung vị (nghìn đồng): {median:.1f}")
print("Thiếu sau điền:", int(working["spent_thousand"].isna().sum()))
print("Bản gốc vẫn thiếu:", int(raw["spent_thousand"].isna().sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Điền trung vị cho 12 mức chi thiếu trên bản sao, bảo vệ dữ liệu gốc.
# Dữ liệu: 12 giá trị chi tiêu tính nghìn đồng, trong đó hai None là chưa có dữ liệu.
# Pandas Series có median(), fillna(), isna(); DataFrame có copy().
import pandas as pd
# Tạo một bảng cột spent_thousand; đơn vị nghìn đồng mỗi khách.
raw = pd.DataFrame({"spent_thousand": [100,120,140,None,160,180,200,220,None,240,260,280]})
# .median() tính số nằm giữa dãy giá hợp lệ đã sắp xếp, bỏ qua NaN; không lấy trung bình cộng.
median = raw["spent_thousand"].median()
# .copy() tạo bản sao DataFrame độc lập để làm sạch, vẫn giữ raw nguyên gốc.
working = raw.copy()
# .fillna(median) thay từng NaN trong cột bản sao bằng trung vị; mặc định trả Series mới.
working["spent_thousand"] = working["spent_thousand"].fillna(median)
if len(raw) != 12:
    # ValueError thông báo dữ liệu sai và dừng xử lý nếu không thể đối chiếu số lượng hoặc chất
lượng nguồn.
    raise ValueError("Cần đủ 12 khách hàng.")
print("Khách:", len(raw))
# isna() xác định thiếu trong bản GỐC, sum() đếm số ô; vẫn thiếu sau khi working đã được sửa.
print("Thiếu ban đầu:", int(raw["spent_thousand"].isna().sum()))
# f-string :.1f hiển thị trung vị tới một chữ số thập phân, đơn vị nghìn đồng.
print(f"Trung vị (nghìn đồng): {median:.1f}")
# Kiểm cột bản sao sau điền, phải còn 0 ô thiếu.
print("Thiếu sau điền:", int(working["spent_thousand"].isna().sum()))
# isna() xác định thiếu trong bản GỐC, sum() đếm số ô; vẫn thiếu sau khi working đã được sửa.
print("Bản gốc vẫn thiếu:", int(raw["spent_thousand"].isna().sum()))
```

Kết quả khi chạy:

```
Khách: 12
Thiếu ban đầu: 2
Trung vị (nghìn đồng): 190.0
Thiếu sau điền: 0
Bản gốc vẫn thiếu: 2
```

Trung vị là một lựa chọn minh họa; quyết định điền hay bỏ dữ liệu phải xét mục đích phân tích và ý nghĩa giá trị thiếu.

Tình huống 3 — Cấp 3: Loại đúng hàng trùng hoàn toàn trong 15 giao dịch

Từ vựng trong mục: Duplicate — bản ghi trùng; **drop_duplicates** — bỏ hàng trùng.

Tình huống thực tế: Nhân viên nhập sổ bán hàng gồm **15 dòng giao dịch**, mỗi dòng có mã đơn O01, O02... và tiền hàng (**nghìn đồng**). Ba dòng được nhập **lặp lại y hệt** một dòng trước đó: giống cả mã đơn lẫn số tiền. Quản lý cần biết có bao nhiêu dòng trùng và còn bao nhiêu bản ghi sau khi bỏ trùng, nhưng không được xóa hai đơn có mã khác nhau chỉ vì tiền mua bằng nhau.

Vì sao chọn cách này? Dòng trùng hoàn toàn nghĩa là mọi cột được so đều giống nhau, chẳng hạn cùng `001` và cùng số tiền 100 nghìn đồng. `df.duplicated()` đánh dấu các lần xuất hiện lặp sau dòng đầu; cộng các dấu này cho biết có bao nhiêu dòng dư. `drop_duplicates()` giữ một bản của từng dòng giống hệt, rồi `.copy()` tạo bảng để tiếp tục dùng. Không thể chỉ so **số tiền** vì nhiều khách có thể mua cùng giá; cũng không nên khử trùng các giao dịch thật chỉ dựa vào suy đoán, vì có thể làm báo doanh thu thấp.

Quy trình:

1. Tạo bảng với 15 cặp mã–tiền.
2. Đếm hàng trùng hoàn toàn bằng `duplicated()`.
3. Khử trùng trên bản làm việc, đối chiếu số dòng còn lại và tổng tiền.

No Note — Không chú thích

```
import pandas as pd
rows = [("O01",100),("O02",150),("O03",100),("O01",100),("O04",200),("O05",180),("O02",150),
("O06",130),("O07",220),("O08",170),("O09",160),("O09",160),("O10",190),("O11",110),("O12",250)]
df = pd.DataFrame(rows, columns=["order_id", "amount_thousand"])
repeat_count = int(df.duplicated().sum())
cleaned = df.drop_duplicates().copy()
if len(df) != 15 or len(cleaned) + repeat_count != 15:
    raise ValueError("Cần đôí chiếú đủ 15 bản ghi, bao gồm dòng trùng.")
print("Dòng gôć:", len(df))
print("Dòng trùng:", repeat_count)
print("Dòng duy nhât:", len(cleaned))
print("Mã O01 còn:", int((cleaned["order_id"] == "O01").sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Loại hàng đơn đặt trùng hoàn toàn trong 15 bản ghi và giữ hàng đâù tiên.
# Dữ liệu: 15 cặp (order_id,amount_thousand), có những dòng trùng cả mã và sô´tiền nghìn đôòng.
# DataFrame hô˜trợ xác minh và loại hàng trùng.
import pandas as pd
# Môĩ tuple là (mã đơn, sô´tiền nghìn đôòng), 15 bản ghi đâù vào.
rows = [("O01",100),("O02",150),("O03",100),("O01",100),("O04",200),("O05",180),("O02",150),
("O06",130),("O07",220),("O08",170),("O09",160),("O09",160),("O10",190),("O11",110),("O12",250)]
# columns đặt hai tên khóa order_id và amount_thousand cho từng cột.
df = pd.DataFrame(rows, columns=["order_id", "amount_thousand"])
# .duplicated() mặc định so MỌI cột và đánh dâú lần xuất hiện sau của bản ghi giôńg; sum() đêḿ
```

```
True.
repeat_count = int(df.duplicated().sum())
# .drop_duplicates() giữ lần xuất hiện đầu của mỗi hàng duy nhất; copy() tạo bảng độc lập.
cleaned = df.drop_duplicates().copy()
# Đối chiếu số dòng duy nhất cộng số dòng trùng đúng 15 bản ghi.
if len(df) != 15 or len(cleaned) + repeat_count != 15:
    # ValueError thông báo dữ liệu sai và dừng xử lý nếu không thể đối chiếu số lượng hoặc chất
lượng nguồn.
    raise ValueError("Cần đối chiếu đủ 15 bản ghi, bao gồm dòng trùng.")
print("Dòng gốc:", len(df))
print("Dòng trùng:", repeat_count)
print("Dòng duy nhất:", len(cleaned))
# So từng mã với O01, sum() đếm số hàng True để kiểm O01 còn đúng một.
print("Mã O01 còn:", int((cleaned["order_id"] == "O01").sum()))
```

Kết quả khi chạy:

```
Dòng gốc: 15
Dòng trùng: 3
Dòng duy nhất: 12
Mã O01 còn: 1
```

Chỉ khử ba hàng xác định là bị nhập lặp; nếu có số hóa đơn hợp lệ khác, không được xóa chỉ vì giá trị thanh toán trùng nhau.

Tình huống 4 — Cấp 4: Chuyển 18 giá tiền dạng chữ và rà soát giá sai

Từ vựng trong mục: to_numeric — chuyển sang số; **errors=coerce** — biến lỗi thành NaN; **Validation** — kiểm dữ liệu.

Tình huống thực tế: Người phụ trách giá có **18 loại hàng P01–P18**, mỗi dòng ghi một đơn giá dạng **chuỗi** và đơn vị cần tính là **đồng/món**. Ba ô không hợp lệ: P04 ghi `bad`, P11 trống, P17 ghi `N/A`. Họ cần đổi các giá còn lại thành số, biết có đúng **15 giá có thể tính**, cộng tổng các đơn giá hợp lệ và đưa ra danh sách ba mã chưa có giá để nhập lại.

Vì sao chọn cách này? `pd.to_numeric(..., errors='coerce')` cố đổi chuỗi giá sang số; nếu gặp `bad`, `N/A` hoặc chuỗi rỗng, nó ghi **NaN** (chưa có số hợp lệ) thay vì ném lỗi khiến cả bảng không xử lý được. `isna()` tìm ba chỗ cần sửa, `notna()` hoặc dấu đảo ~ xác định **15 giá hợp lệ**. Chỉ cộng những giá đã chuyển thành số; nếu coi NaN là 0 đồng thì báo cáo sẽ che đi hàng chưa nhập giá. Giá đang tính bằng **đồng/món**, nên tổng ở đây là tổng **15 đơn giá**, không phải doanh thu bán hàng vì chưa có số món bán.

Quy trình:

1. Tạo 18 chuỗi giá có mã P01…P18.
2. Chuyển kiểu theo cột bằng `to_numeric(errors='coerce')`.
3. Đếm dòng hợp lệ/lỗi, in mã lỗi và tổng giá các dòng có số.

No Note — Không chú thích

```
import pandas as pd
skus = []
for i in range(1, 19):
    skus.append(f"P{i:02d}")
df = pd.DataFrame({"sku": skus,
                   "raw_price_vnd":
["50000","55000","60000","bad","70000","75000","80000","85000","90000","95000","","105000","110000"
,"115000","120000","125000","N/A","135000"]})
```

```
df["price_vnd"] = pd.to_numeric(df["raw_price_vnd"], errors="coerce")
bad = df["price_vnd"].isna()
if len(df) != 18:
    raise ValueError("Cần đủ 18 mã sản phẩm.")
print("SKU:", len(df))
print("Giá hợp lệ:", int((~bad).sum()))
print("Cần sửa:", ", ".join(df.loc[bad, "sku"]))
print("Tổng đơn giá (đồng):", int(df["price_vnd"].sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuyển 18 chuỗi đơn giá sang số và liệt kê SKU bị lỗi định dạng.
# Dữ liệu: 18 mã P01..P18, đơn giá dạng chuỗi VND; 'bad', '' và 'N/A' là lỗi/thiếu.
# Pandas cho to_numeric() và mặt nạ giá không hợp lệ.
import pandas as pd
skus = []
# range(1,19) tạo 18 chỉ số cho P01..P18.
for i in range(1, 19):
    # append() thêm mã SKU, :02d định dạng hai chữ số.
    skus.append(f"P{i:02d}")
# DataFrame có sku và raw_price_vnd dạng chuỗi gốc.
df = pd.DataFrame({"sku": skus,
                   "raw_price_vnd":
["50000","55000","60000","bad","70000","75000","80000","85000","90000","95000","","105000","110000"
,"115000","120000","125000","N/A","135000"]})
# to_numeric() chuyển số trong chuỗi thành kiểu số; 'coerce' chuyển bad/rỗng/N/A thành NaN.
df["price_vnd"] = pd.to_numeric(df["raw_price_vnd"], errors="coerce")
# .isna() đánh dấu giá không đổi được; mask bad=True ở dòng lỗi.
bad = df["price_vnd"].isna()
if len(df) != 18:
    # ValueError thông báo dữ liệu sai và dừng xử lý nếu không thể đối chiếu số lượng hoặc chất
lượng nguồn.
    raise ValueError("Cần đủ 18 mã sản phẩm.")
print("SKU:", len(df))
# ~ đảo mặt nạ Boolean, sum() đếm các giá hợp lệ.
print("Giá hợp lệ:", int((~bad).sum()))
# .loc[bad,'sku'] lấy mã SKU lỗi theo nhãn cột.
print("Cần sửa:", ", ".join(df.loc[bad, "sku"]))
# .sum() bỏ qua NaN mặc định để cộng các đơn giá hợp lệ; đơn vị đồng.
print("Tổng đơn giá (đồng):", int(df["price_vnd"].sum()))
```

Kết quả khi chạy:

```
SKU: 18
Giá hợp lệ: 15
Cần sửa: P04, P11, P17
Tổng đơn giá (đồng): 1370000
```

Tổng đơn giá không phải doanh thu. Nếu cần doanh thu phải ghép thêm số lượng bán cho từng SKU.

Tình huống 5 — Cấp 5: Tách 20 hồ sơ train/test trước khi tính trung vị điền khuyết

Từ vựng trong mục: Data leakage — rò rỉ dữ liệu; **Train/test split** — chia tập; **Imputation** — điền khuyết.

Tình huống thực tế: Nhóm phân tích muốn chuẩn bị **20 hồ sơ học viên giả lập** H01–H20 cho một thử nghiệm học máy. Mỗi hồ sơ ghi tuổi (**năm**) và mức chi (**nghìn đồng**); hai mức chi của nhóm huấn luyện còn thiếu. Theo kế hoạch, **15 người đầu thuộc train**, **5 người sau thuộc test**; tập test chứa một mức chi

rất cao là **999 nghìn đồng**. Nhóm cần lấy trung vị từ train để điền chỗ thiếu khi làm việc, không dùng bất kỳ mức chi nào của test để quyết định cách xử lý train.

Vì sao chọn cách này? Train là nhóm dữ liệu để chương trình học quy tắc hoặc tính thông số xử lý; **test** là nhóm giữ riêng để kiểm tra trên dữ liệu chưa dùng khi chuẩn bị train. `iloc[:15]` chọn 15 người đầu, còn `iloc[15:]` chọn 5 người cuối; `.copy()` giữ hai bảng làm việc riêng. Trung vị phải được tính **chỉ trên train**, rồi dùng cùng giá trị đó điền vị trí thiếu của train và, nếu có, test. Nếu tính từ cả 20 người, mức 999 nghìn đồng của test có thể ảnh hưởng lựa chọn xử lý trước khi đánh giá: đó là **rò rỉ dữ liệu (data leakage)**. Bản `raw` vẫn giữ những chỗ thiếu thật để phân biệt dữ liệu gốc với dữ liệu đã điền.

Quy trình:

1. Tạo DataFrame 20 người; tách 15 dòng đầu/5 dòng cuối.
2. Tính median từ `train` trên số hợp lệ, không dùng `test`.
3. Dùng một median cố định điền vào bản train/test, đối chiếu số dòng và giá trị của train.

No Note — Không chú thích

```
import pandas as pd
ids = []
for i in range(1, 21):
    ids.append(f"H{i:02d}")
raw = pd.DataFrame({"id": ids,
                    "age_years": [20,21,22,23,24,25,26,27,20,21,22,23,24,25,26,27,20,21,22,23],
                    "spent_thousand":
[100,113,None,139,152,165,178,191,204,None,110,123,136,149,162,175,999,201,214,107]})
train = raw.iloc[:15].copy()
test = raw.iloc[15:].copy()
train_median = train["spent_thousand"].median()
train["spent_thousand"] = train["spent_thousand"].fillna(train_median)
test["spent_thousand"] = test["spent_thousand"].fillna(train_median)
if len(raw) != 20 or len(train) + len(test) != 20:
    raise ValueError("Tổng bản ghi Train và Test phải đúng 20 hồ sơ.")
if raw["spent_thousand"].isna().sum() != 2:
    raise ValueError("Dữ liệu gốc phải có đúng hai giá trị bị thiếu.")
print("Hồ sơ:", len(raw))
print("Train:", len(train))
print("Test:", len(test))
print("Thiếu train trước:", 2)
print(f"Trung vị học từ train: {train_median:.1f}")
print("Thiếu sau điền:", int(train["spent_thousand"].isna().sum() +
test["spent_thousand"].isna().sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Học trung vị chi tiêu từ 15 hồ sơ train và dùng cùng trung vị để điền train/test, tránh
rò rỉ dữ liệu.
# Dữ liệu: 20 hồ sơ id H01..H20 có tuổi năm và chi tiêu nghìn đồng; hai None, điểm ngoại lệ 999 ở
test.
# Pandas hỗ trợ cắt hàng theo vị trí, median, fillna, copy.
import pandas as pd
ids = []
# Tạo mã 1..20, f-string :02d tạo hai chữ số.
for i in range(1, 21):
    ids.append(f"H{i:02d}")
# Mỗi hàng là một học viên có id, tuổi năm, mức chi nghìn đồng.
raw = pd.DataFrame({"id": ids,
                    "age_years": [20,21,22,23,24,25,26,27,20,21,22,23,24,25,26,27,20,21,22,23],
                    "spent_thousand":
[100,113,None,139,152,165,178,191,204,None,110,123,136,149,162,175,999,201,214,107]})
```

```
# .iloc[:15] lấy 15 hàng vị trí 0..14 làm train; copy() tạo dữ liệu riêng, tránh ghi ngược vào raw.
train = raw.iloc[:15].copy()
# .iloc[15:] lấy năm hàng vị trí 15..19 làm test; không lấy test khi học cách điền.
test = raw.iloc[15:].copy()
# Chỉ tính trung vị từ TRAIN (bỏ NaN) để không rò rỉ thống kê của test vào huấn luyện.
train_median = train["spent_thousand"].median()
# Điền NaN train bằng trung vị đã học từ train.
train["spent_thousand"] = train["spent_thousand"].fillna(train_median)
# Điền NaN test bằng CÙNG trung vị train, không tính lại trên test (tránh data leakage).
test["spent_thousand"] = test["spent_thousand"].fillna(train_median)
# 15+5=20 hàng, đảm bảo chia không mất bản ghi.
if len(raw) != 20 or len(train) + len(test) != 20:
    # ValueError thông báo dữ liệu sai và dừng xử lý nếu không thể đối chiếu số lượng hoặc chất
lượng nguồn.
    raise ValueError("Tổng bản ghi Train và Test phải đúng 20 hồ sơ.")
# .isna().sum() đếm hai giá trị thiếu trong raw để kiểm dữ liệu gốc chưa bị chỉnh.
if raw["spent_thousand"].isna().sum() != 2:
    # ValueError thông báo dữ liệu sai và dừng xử lý nếu không thể đối chiếu số lượng hoặc chất
lượng nguồn.
    raise ValueError("Dữ liệu gốc phải có đúng hai giá trị bị thiếu.")
print("Hồ sơ:", len(raw))
print("Train:", len(train))
print("Test:", len(test))
print("Thiếu train trước:", 2)
# Định dạng :.1f in trung vị có một chữ số sau dấu thập phân, đơn vị nghìn đồng.
print(f"Trung vị học từ train: {train_median:.1f}")
# Tổng số NaN hai tập sau điền phải bằng 0; không có thống kê test trong fillna.
print("Thiếu sau điền:", int(train["spent_thousand"].isna().sum() +
test["spent_thousand"].isna().sum()))
```

Kết quả khi chạy:

```
Hồ sơ: 20
Train: 15
Test: 5
Thiếu train trước: 2
Trung vị học từ train: 149.0
Thiếu sau điền: 0
```

Trung vị của train được tái sử dụng khi xử lý test; bản dữ liệu thô vẫn giữ dấu vết dữ liệu thiếu. Không biến chỉ số từ test thành tham số chọn trong lúc chuẩn bị train.

6. Sai lầm thường gặp

- **Thiếu dữ liệu** không đồng nghĩa với 0; điền 0 vào tuổi, giá hay lượng hàng thiếu có thể làm sai thống kê.
- `duplicated()` mặc định xét toàn bộ các cột; xóa trùng phải hiểu **khóa định danh** và ý nghĩa từng bản ghi.
- `drop_duplicates()` thường giữ **index cũ**; cần `reset_index(drop=True)` nếu muốn đánh lại nhãn hàng.
- `to_numeric(errors='coerce')` đổi dữ liệu sai thành NaN, **không tự đoán** giá trị thật.
- Với Machine Learning, chỉ tính trung vị/trung bình để điền từ **tập train**, sau đó áp dụng cho test để tránh data leakage.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a missing value? | Giá trị thiếu là gì?

A. Replacing missing entries with chosen estimates | Thay ô thiếu bằng giá trị ước lượng đã chọn

B. Checking types and rules before analysis | Kiểm tra kiểu và quy tắc trước khi phân tích

C. An absent or unknown data entry | Ô dữ liệu chưa có giá trị xác định

D. A record repeated under a defined matching rule | Bản ghi lặp theo tiêu chí đối chiếu xác định

Đáp án: C

Giải thích: Thiếu nghĩa là chưa có giá trị quan sát hợp lệ ở ô đó. Trong Pandas, `NaN` và `None` có thể biểu thị ô thiếu; giá trị thiếu khác một số 0 đã được đo thật.

Q2. What is a duplicate record? | Bản ghi trùng là gì?

A. A record repeated under a defined matching rule | Bản ghi lặp theo tiêu chí đối chiếu xác định

B. Checking types and rules before analysis | Kiểm tra kiểu và quy tắc trước khi phân tích

C. An absent or unknown data entry | Ô dữ liệu chưa có giá trị xác định

D. Replacing missing entries with chosen estimates | Thay ô thiếu bằng giá trị ước lượng đã chọn

Đáp án: A

Giải thích: Nhận diện trùng phụ thuộc các trường dùng làm tiêu chí. Hai hàng cùng mã khách có thể là hai đơn riêng, nên phải xác định tiêu chí trùng trước khi xóa dữ liệu.

Q3. What is imputation? | Điền khuyết là gì?

A. An absent or unknown data entry | Ô dữ liệu chưa có giá trị xác định

B. Checking types and rules before analysis | Kiểm tra kiểu và quy tắc trước khi phân tích

C. Replacing missing entries with chosen estimates | Thay ô thiếu bằng giá trị ước lượng đã chọn

D. A record repeated under a defined matching rule | Bản ghi lặp theo tiêu chí đối chiếu xác định

Đáp án: C

Giải thích: Điền khuyết dùng giá trị lựa chọn chứ không coi đó là dữ liệu đã đo thật. Điền tuổi thiếu bằng trung vị của tập train là phép ước lượng, không chứng minh tuổi thật bằng trung vị đó.

Q4. What is data validation? | Kiểm tra dữ liệu hợp lệ là gì?

A. A record repeated under a defined matching rule | Bản ghi lặp theo tiêu chí đối chiếu xác định

B. An absent or unknown data entry | Ô dữ liệu chưa có giá trị xác định

C. Checking types and rules before analysis | Kiểm tra kiểu và quy tắc trước khi phân tích

D. Replacing missing entries with chosen estimates | Thay ô thiếu bằng giá trị ước lượng đã chọn

Đáp án: C

Giải thích: Kiểm tra hợp lệ phát hiện sai kiểu và giá trị vượt phạm vi. Với trường tuổi, bước kiểm tra có thể yêu cầu giá trị số không âm và loại chuỗi không hợp lệ.

Q5. What does `isna()` identify? | `isna()` phát hiện gì?

A. Missing entries | Các ô thiếu

B. Turns invalid numeric text into NaN | Chuyển chữ không đổi được sang số thành NaN

C. Repeated rows after the first occurrence | Các hàng lặp sau lần xuất hiện đầu

D. Replaces missing entries with a value | Thay ô thiếu bằng giá trị cho trước

Đáp án: A

Giải thích: isna trả mặt nạ Boolean đánh dấu giá trị thiếu. Với `[2, NaN]`, `isna()` đánh dấu `[False, True]`; hàm xác định chỗ thiếu mà không xóa dữ liệu.

Q6. What does `duplicated()` mark by default? | `duplicated()` mặc định đánh dấu gì?

A. Turns invalid numeric text into NaN | Chuyển chữ không đổi được sang số thành NaN

B. Repeated rows after the first occurrence | Các hàng lặp sau lần xuất hiện đầu

C. Replaces missing entries with a value | Thay ô thiếu bằng giá trị cho trước

D. Missing entries | Các ô thiếu

Đáp án: B

Giải thích: duplicated xét toàn hàng nếu không chỉ định subset. Với hai hàng giống nhau, hàng đầu thường nhận `False`, hàng sau nhận `True`; `subset=` thay đổi các cột dùng để xét trùng.

Q7. What does `fillna(value)` do? | `fillna(value)` làm gì?

A. Turns invalid numeric text into NaN | Chuyển chữ không đổi được sang số thành NaN

B. Missing entries | Các ô thiếu

C. Repeated rows after the first occurrence | Các hàng lặp sau lần xuất hiện đầu

D. Replaces missing entries with a value | Thay ô thiếu bằng giá trị cho trước

Đáp án: D

Giải thích: fillna thay ô thiếu mà giữ các giá trị hợp lệ. Với điểm thiếu, `df.fillna(0)` mặc định trả DataFrame đã điền mà không đổi `df` nếu chưa gán kết quả hoặc yêu cầu sửa tại chỗ.

Q8. What does `pd.to_numeric(..., errors='coerce')` do with invalid text? | `to_numeric(..., errors='coerce')` xử lý chuỗi không phải số thế nào?

A. Replaces missing entries with a value | Thay ô thiếu bằng giá trị cho trước

B. Turns invalid numeric text into NaN | Chuyển chữ không đổi được sang số thành NaN

C. Repeated rows after the first occurrence | Các hàng lặp sau lần xuất hiện đầu

D. Missing entries | Các ô thiếu

Đáp án: B

Giải thích: Cơ chế coerce đánh dấu lỗi chuyển kiểu là thiếu chứ không đoán số. Khi đổi kiểu số, chuỗi `'bad'` trở thành dấu thiếu, còn chuỗi hợp lệ như `'12'` được chuyển thành số 12.

Q9. What does `dropna()` normally remove? | `dropna()` mặc định xóa gì?

A. Rows containing missing data | Các hàng chứa dữ liệu thiếu

B. The middle value after sorting, or mean of the two middle values | Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

C. Raise an exception on invalid input | Báo ngoại lệ khi đầu vào không hợp lệ

D. The first occurrence of each distinct row | Lần xuất hiện đầu của từng hàng khác nhau

Đáp án: A

Giải thích: DataFrame.dropna mặc định bỏ hàng có ít nhất một ô thiếu. Với DataFrame có ô thiếu, `dropna()` mặc định bỏ hàng chứa ít nhất một ô thiếu; nó không tự điền các ô đó.

Q10. What does `drop_duplicates()` normally keep? | `drop_duplicates()` thường giữ gì?

A. The middle value after sorting, or mean of the two middle values | Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

B. The first occurrence of each distinct row | Lần xuất hiện đầu của từng hàng khác nhau

C. Rows containing missing data | Các hàng chứa dữ liệu thiếu

D. Raise an exception on invalid input | Báo ngoại lệ khi đầu vào không hợp lệ

Đáp án: B

Giải thích: Mặc định các lần lặp sau của cùng hàng sẽ bị bỏ. Với các hàng A, A, B, mặc định giữ A đầu và B, không giữ A cuối trừ khi đặt `keep='last'`.

Q11. What is the median? | Trung vị là gì?

A. Raise an exception on invalid input | Báo ngoại lệ khi đầu vào không hợp lệ

B. The first occurrence of each distinct row | Lần xuất hiện đầu của từng hàng khác nhau

C. Rows containing missing data | Các hàng chứa dữ liệu thiếu

D. The middle value after sorting, or mean of the two middle values | Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

Đáp án: D

Giải thích: Trung vị ít nhạy với giá trị cực đoan hơn trung bình cộng. Với dãy đã sắp `[1,2,100]`, trung vị là 2 còn trung bình lớn hơn 34; nếu số phần tử chẵn thì lấy trung bình hai số ở giữa.

Q12. What is the meaning of `errors='raise'` in numeric conversion? | `errors='raise'` khi đổi sang số có nghĩa gì?

A. Rows containing missing data | Các hàng chứa dữ liệu thiếu

B. The middle value after sorting, or mean of the two middle values | Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

C. The first occurrence of each distinct row | Lần xuất hiện đầu của từng hàng khác nhau

D. Raise an exception on invalid input | Báo ngoại lệ khi đầu vào không hợp lệ

Đáp án: D

Giải thích: Raise làm lỗi lộ rõ thay vì chuyển âm thầm thành thiếu. Dùng `pd.to_numeric(['bad'], errors='raise')` báo lỗi, còn `errors='coerce'` trả dấu giá trị số bị thiếu.

Q13. What is data leakage in train/test preprocessing? | Data leakage khi chuẩn bị train/test là gì?

A. Zero may have a different real-world meaning | Số 0 có thể mang ý nghĩa thực tế khác

B. To audit and reverse cleaning decisions | Để đối chiếu và phục hồi quyết định xử lý

C. Records with shared IDs may still represent distinct events | Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

D. Using information from test data while fitting preprocessing | Dùng thông tin của test khi học tham số tiền xử lý

Đáp án: D

Giải thích: Thống kê tiền xử lý phải học chỉ từ tập train. Tính trung vị điền khuyết trên cả train và test làm thông tin của tập đánh giá lọt vào giai đoạn chuẩn bị mô hình.

Q14. Why is replacing every missing value with zero risky? | Vì sao thay mọi ô thiếu bằng 0 là rủi ro?

A. To audit and reverse cleaning decisions | Để đối chiếu và phục hồi quyết định xử lý

B. Using information from test data while fitting preprocessing | Dùng thông tin của test khi học tham số tiền xử lý

C. Zero may have a different real-world meaning | Số 0 có thể mang ý nghĩa thực tế khác

D. Records with shared IDs may still represent distinct events | Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

Đáp án: C

Giải thích: Tuổi chưa nhập không tự đồng nghĩa tuổi bằng 0. Số lượng hàng thiếu nghĩa là chưa biết, còn số lượng 0 nói rõ không có món nào; thay hai trường hợp cho nhau sẽ đổi ý nghĩa.

Q15. Why check key columns before removing duplicates? | Vì sao phải xem cột khóa trước khi xóa trùng?

A. Records with shared IDs may still represent distinct events | Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

B. Using information from test data while fitting preprocessing | Dùng thông tin của test khi học tham số tiền xử lý

C. Zero may have a different real-world meaning | Số 0 có thể mang ý nghĩa thực tế khác

D. To audit and reverse cleaning decisions | Để đối chiếu và phục hồi quyết định xử lý

Đáp án: A

Giải thích: Quy tắc nghiệp vụ xác định hai dòng có thực sự trùng không. Hai giao dịch của cùng khách hàng có thể là hai sự kiện hợp lệ khác nhau dù trùng một mã định danh.

Q16. Why preserve the original raw data while cleaning? | Vì sao nên giữ dữ liệu gốc khi làm sạch?

A. Zero may have a different real-world meaning | Số 0 có thể mang ý nghĩa thực tế khác

B. To audit and reverse cleaning decisions | Để đối chiếu và phục hồi quyết định xử lý

C. Records with shared IDs may still represent distinct events | Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

D. Using information from test data while fitting preprocessing | Dùng thông tin của test khi học tham số tiền xử lý

Đáp án: B

Giải thích: Bản gốc giúp kiểm tra từng phép biến đổi. Giữ bản nguồn không đổi giúp so sánh trước/sau và xem xét lại các quyết định làm sạch chưa đúng.

Q17. What does a Boolean missing-value mask contain? | Mặt nạ dữ liệu thiếu chứa gì?

A. True and False values | Các giá trị True và False

B. Original retained-row labels remain | Nhãn index của hàng được giữ vẫn còn

C. Choose a justified missing-data treatment | Chọn cách xử lý dữ liệu thiếu có cơ sở

D. Rebuilds sequential indices without retaining old index as a column | Đánh lại index tuần tự, không giữ index cũ thành cột

Đáp án: A

Giải thích: Mỗi vị trí cho biết ô dữ liệu có thiếu hay không. Một ô mask là `True` nếu đầu vào tương ứng bị thiếu và `False` nếu không; mask không phải danh sách giá trị thay thế.

Q18. What happens to index labels after `drop_duplicates()` by default? | Sau `drop_duplicates()`, index mặc định thay đổi thế nào?

A. True and False values | Các giá trị True và False

B. Choose a justified missing-data treatment | Chọn cách xử lý dữ liệu thiếu có cơ sở

C. Rebuilds sequential indices without retaining old index as a column | Đánh lại index tuần tự, không giữ index cũ thành cột

D. Original retained-row labels remain | Nhãn index của hàng được giữ vẫn còn

Đáp án: D

Giải thích: Xóa hàng trùng không tự đánh lại index cho hàng còn lại. Xóa các hàng mang nhãn 1 và 3 không tự đánh lại nhãn còn lại; phải reset index riêng nếu muốn nhãn liên tiếp.

Q19. What does `reset_index(drop=True)` do? | `reset_index(drop=True)` làm gì?

A. Original retained-row labels remain | Nhãn index của hàng được giữ vẫn còn

B. Choose a justified missing-data treatment | Chọn cách xử lý dữ liệu thiếu có cơ sở

C. Rebuilds sequential indices without retaining old index as a column | Đánh lại index tuần tự, không giữ index cũ thành cột

D. True and False values | Các giá trị True và False

Đáp án: C

Giải thích: Đánh lại nhãn hàng giúp bảng đã xử lý dễ đọc hơn. Sau khi bỏ hàng, `reset_index(drop=True)` tạo nhãn `0,1,2,...` mà không đưa index cũ thành cột mới.

Q20. What should happen after coercing invalid numeric text to NaN? | Sau khi chuyển chuỗi số sai thành NaN cần làm gì?

A. Rebuilds sequential indices without retaining old index as a column | Đánh lại index tuần tự, không giữ index cũ thành cột

B. Choose a justified missing-data treatment | Chọn cách xử lý dữ liệu thiếu có cơ sở

C. Original retained-row labels remain | Nhãn index của hàng được giữ vẫn còn

D. True and False values | Các giá trị True và False

Đáp án: B

Giải thích: Coerce phát hiện lỗi, không khôi phục được giá trị thực chưa biết. Sau khi coerce, hãy đếm số ô bị thiếu rồi quyết định loại bỏ, từ chối hay điền khuyết theo yêu cầu bài toán.

8. Nối bài tiếp theo

`DataFrame → Data Cleaning → EDA → ML Pipeline`.

PY-19 — Pandas: lọc, groupby, merge và sắp xếp

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: loc — chọn theo nhãn; **iloc** — chọn theo vị trí.

`DataFrame → lọc → nhóm → ghép bảng → AI`

Học xong dùng để làm gì? Gộp doanh thu theo thành phố và ghép thông tin khách hàng với đơn hàng theo khóa.

Kết nối kiến thức tiếp theo: PY-20 dùng logic và nguyên lý đếm để diễn đạt điều kiện và số trường hợp.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
loc	chọn theo nhãn	Chọn hàng/cột theo tên
iloc	chọn theo vị trí	Chọn hàng/cột theo số nguyên tính từ 0
groupby	nhóm dữ liệu	Gom các hàng cùng giá trị để tổng hợp
merge	ghép bảng	Nối bảng qua khóa chung

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`df.loc[label]`	Chọn theo nhãn dòng/cột; slicing theo nhãn có thể gồm điểm cuối
`df.iloc[position]`	Chọn theo vị trí; stop của slice không bao gồm
`df.groupby(key).agg()`	Gộp nhóm rồi tính tổng/trung bình/số dòng
`pd.merge(left,right,on=,how=)`	Ghép theo khóa, cách inner/left/right/outer khác nhau

4. Khái niệm liên quan

Phân tích bảng với Pandas thường gồm ba thao tác nối tiếp: **lọc** để giữ dòng đúng điều kiện, **nhóm** để tính thống kê theo loại, rồi **ghép** để bổ sung thông tin từ bảng khác; muốn chọn hàng chính xác cần phân biệt `.loc` theo nhãn và `.iloc` theo vị trí. Ví dụ hàng sản phẩm, giao dịch và khách hàng sẽ cho thấy vì sao cách tổ chức bảng quyết định phép xử lý.

4.1. Filtering, loc — Lọc hàng theo điều kiện hoặc nhãn

Từ vựng trong mục: Filter — lọc; **loc** — chọn theo nhãn.

Lọc dữ liệu là giữ các hàng thỏa một điều kiện. Ví dụ bảng có ba mặt hàng A, B, C, giá lần lượt 300, 900, 1200 (nghìn đồng). **Mỗi hàng** là một sản phẩm, **cột price** là giá. `df['price'] >= 800` tạo một Series Boolean; `df.loc[mask, ['name','price']]` chỉ giữ các hàng có giá từ 800 và hai cột muốn xem. `.loc` cũng chọn trực tiếp theo **nhãn index**, khác chọn vị trí số.

No Note — Không chú thích

```
import pandas as pd

products = pd.DataFrame({'name': ['A', 'B', 'C'], 'price': [300, 900, 1200]})
selected = products.loc[products['price'] >= 800, 'name']
print(selected.tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn sản phẩm có giá từ 800 trở lên bằng .loc với điều kiện Boolean.
# Dữ liệu: Ba hàng sản phẩm A/B/C và giá 300/900/1200 (đơn vị minh họa).
# Pandas hỗ trợ lọc hàng DataFrame theo điều kiện.
import pandas as pd

# Tạo bảng name và price theo từng hàng sản phẩm.
products = pd.DataFrame({'name': ['A', 'B', 'C'], 'price': [300, 900, 1200]})
# So từng giá với 800, tạo Series Boolean True/False.
# .loc[điều_kiện_hàng, tên_cột] chọn tên ở hàng True; ở đây B và C.
selected = products.loc[products['price'] >= 800, 'name']
# .tolist() chuyển Series tên đã lọc thành list Python để in.
print(selected.tolist())
```

Kết quả khi chạy:

```
['B', 'C']
```

Sản phẩm B, C thỏa mức giá; A giá 300 thấp hơn 800 nên bị loại. Kết quả chuyển sang List để tập trung vào tên.

4.2. iloc — Chọn theo vị trí hàng và cột

Từ vựng trong mục: iloc — chọn theo vị trí; **Index** — chỉ số.

`.iloc` chọn **vị trí số nguyên**, tính từ 0, bất kể nhãn index của bảng. Với ba khách A, B, C theo thứ tự hàng, `df.iloc[1]` là khách ở **hàng thứ hai**. Cú pháp `df.iloc[row, col]` lấy một ô, ví dụ `[1, 0]` là cột đầu của hàng hai. Trái lại `.loc['B']` chỉ hoạt động khi **nhãn index thực sự là B**, không phải chỉ vì cột `id` có chữ B.

No Note — Không chú thích

```
import pandas as pd

people = pd.DataFrame({'id': ['A', 'B', 'C']})
print(people.iloc[1, 0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng .iloc đọc ô theo vị trí hàng 1, cột 0 của DataFrame.
# Dữ liệu: Ba mã A, B, C; thứ tự hàng bắt đầu từ vị trí 0.
# Pandas DataFrame có thể tra theo nhãn .loc hoặc vị trí .iloc.
import pandas as pd

# Tạo bảng một cột id và ba hàng A,B,C.
people = pd.DataFrame({'id': ['A', 'B', 'C']})
# .iloc[1,0] đọc hàng thứ hai và cột đầu tiên theo VỊ TRÍ, trả B; khác .loc tra nhãn.
print(people.iloc[1, 0])
```

Kết quả khi chạy:

```
B
```

Chỉ số 1 lấy hàng thứ hai và chỉ số cột 0 lấy trường `id`, nên in B.

4.3. GroupBy — Tổng hợp trong từng nhóm

Từ vựng trong mục: GroupBy — nhóm theo khóa; **Aggregation** — phép tổng hợp.

`groupby` gom những hàng có **cùng giá trị ở một cột khóa**, rồi tính thống kê riêng cho từng nhóm. Giả sử ba giao dịch có `city = A,A,B` và `sale = 10,20,8` nghìn đồng. **Mỗi hàng** là một giao dịch; **cột city** là vùng, **sale** là tiền. `groupby('city')['sale'].sum()` cộng doanh thu của A thành 30 và B thành 8. Không nhầm GroupBy với lọc: lọc loại hàng, còn groupby kết hợp các hàng thành thống kê nhóm.

No Note — Không chú thích

```
import pandas as pd

sales = pd.DataFrame({'city': ['A', 'A', 'B'], 'sale': [10, 20, 8]})
print(sales.groupby('city')['sale'].sum().to_dict())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Gộp các đơn theo city rồi cộng cột sale riêng từng thành phố.
# Dữ liệu: Ba hàng: city A có hai sale 10,20; B một sale 8.
# DataFrame.groupby() tập hợp hàng cùng giá trị của khóa nhóm.
import pandas as pd

# Tạo bảng 3 giao dịch với cột city và sale.
sales = pd.DataFrame({'city': ['A', 'A', 'B'], 'sale': [10, 20, 8]})
# groupby('city') tạo các nhóm hàng cùng tên city, không phải nối bảng.
# Chọn cột sale trong từng nhóm rồi sum() cộng từng nhóm A=30, B=8.
# .to_dict() đổi Series kết quả thành dictionary {city: tổng}, tiện xem bằng print().
print(sales.groupby('city')['sale'].sum().to_dict())
```

Kết quả khi chạy:

```
{'A': 30, 'B': 8}
```

Vùng A có hai giao dịch nên tổng 30 nghìn đồng; vùng B chỉ có một giao dịch 8 nghìn đồng.

4.4. Merge — Ghép bảng theo khóa

Từ vựng trong mục: Merge — ghép bảng; **Join key** — khóa ghép.

Khi thông tin nằm trong hai bảng, `merge` dùng cột **khóa chung** để ghép các hàng liên quan. Bảng `orders` có đơn của khách `id=1` và `id=2`, cột `sale` là giá trị đơn; bảng `customers` có tên An, Lan theo cùng id. `merge(on='id', how='left')` giữ mọi dòng bên trái và bổ sung tên nếu tìm thấy. Nếu bảng bên phải có nhiều dòng trùng khóa, một dòng đơn hàng có thể bị nhân bản; dùng `validate='many_to_one'` để kiểm tra mỗi id bên phải là duy nhất.

No Note — Không chú thích

```
import pandas as pd

orders = pd.DataFrame({'id': [1, 2], 'sale': [10, 20]})
customers = pd.DataFrame({'id': [1, 2], 'name': ['An', 'Lan']})
merged = orders.merge(customers, on='id', how='left', validate='many_to_one')
print(merged['name'].tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghép bảng đơn với bảng khách theo id, giữ tất cả đơn và xác minh mỗi id chỉ có một
khách.
# Dữ liệu: Hai đơn id 1,2 và hai khách id 1,2, trường sale của đơn và name của khách.
# Pandas merge() là phép nối bảng theo khóa chung.
import pandas as pd

# Tạo hai bảng có cột id trùng tên; sale là giá đơn, name là tên khách.
orders = pd.DataFrame({'id': [1, 2], 'sale': [10, 20]})
# Tạo hai bảng có cột id trùng tên; sale là giá đơn, name là tên khách.
customers = pd.DataFrame({'id': [1, 2], 'name': ['An', 'Lan']})
# on='id' chọn khóa ghép; how='left' giữ mọi đơn dù thiếú khách; validate='many_to_one' yêu câù
khóa id của BẢNG KHÁCH không trùng (nhiều đơn được phép cùng khách).
merged = orders.merge(customers, on='id', how='left', validate='many_to_one')
# Sau ghép, ['name'] lấý tên khách theo thứ tự đơn; tolist() đổi sang list Python.
print(merged['name'].tolist())
```

Kết quả khi chạy:

```
['An', 'Lan']
```

Hai tên khách được ghép đúng theo id, không phải dựa vào vị trí hàng ngẫu nhiên.

5. Năm tình huống Pandas — từ lọc hàng đến ghép và tổng hợp nhiều bảng

Tình huống 1 — Cấp 1: Lọc 10 sản phẩm bằng loc theo ngưỡng

Từ vựng trong mục: Filter — lọc; **loc** — chọn hàng theo nhãn.

Tình huống thực tế: Cửa hàng có danh mục **10 sản phẩm P01–P10**, mỗi sản phẩm ghi một đơn giá tính bằng **nghìn đồng/món**. Chủ cửa hàng muốn chọn những món có giá **từ 800 nghìn đồng trở lên**, rồi in đúng **mã và giá** của chúng để nhân viên đối chiếu bảng hàng. Mã P01 không phải là vị trí hàng và cũng không phải số tiền.

Vì sao chọn cách này? `DataFrame` là bảng có cột được đặt tên, giúp phân biệt rõ **mã sản phẩm** và **giá tiền**. Chương trình kiểm tra từng giá có `>= 800` hay không để tạo một **mặt nạ Boolean** (dãy đúng/sai). `df.loc[mask, ['id', 'price_thousand']]` chỉ lấy những hàng đúng điều kiện, đồng thời chỉ giữ hai cột

cần xem. `loc` chọn dựa trên nhãn hoặc điều kiện, không phải so sánh mã P01 với con số 800. Cách lọc này còn giữ nguyên dữ liệu gốc; đổi thứ tự bảng không làm ngưỡng giá thay đổi.

Quy trình:

1. Tạo bảng mã/giá của 10 sản phẩm.
2. Lọc bằng `price_thousand >= 800` và chọn cột mã.
3. In số sản phẩm phù hợp và danh sách mã.

No Note — Không chú thích

```
import pandas as pd
prices = [300,900,1200,400,850,2000,500,700,1500,650]
ids = []
for i in range(1, 11):
    ids.append(f"P{i:02d}")
df = pd.DataFrame({"id": ids, "price_thousand": prices})
selected = df.loc[df["price_thousand"] >= 800, ["id", "price_thousand"]]
if len(df) != 10:
    raise ValueError("Cần đủ 10 sản phẩm trong bảng giá.")
print("Sản phẩm:", len(df))
print("Giá cao:", len(selected))
print("Mã:", ", ".join(selected["id"]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc 10 sản phẩm có giá từ 800 nghìn đồng bằng .loc và in mã.
# Dữ liệu: 10 hàng sản phẩm P01..P10, price_thousand là nghìn đồng/món.
# Pandas dùng Series Boolean để chọn đúng hàng quan tâm.
import pandas as pd
prices = [300,900,1200,400,850,2000,500,700,1500,650]
ids = []
# range(1,11) sinh 10 số thứ tự mã sản phẩm, không lấy 11.
for i in range(1, 11):
    # append() giữ mã P01..P10, f-string :02d thêm 0 đầu.
    ids.append(f"P{i:02d}")
# Ghép id và price_thousand thành 10 hàng theo thứ tự tương ứng.
df = pd.DataFrame({"id": ids, "price_thousand": prices})
# So ngưỡng 800 với từng giá để tạo mask Boolean.
# loc[mask, [cột...]] lọc hàng đạt giá và chỉ lấy hai cột id/price_thousand.
selected = df.loc[df["price_thousand"] >= 800, ["id", "price_thousand"]]
if len(df) != 10:
    # ValueError báo sớm bản ghi/khóa không hợp lệ rồi dừng, tránh công bố kết quả ghép sai.
    raise ValueError("Cần đủ 10 sản phẩm trong bảng giá.")
print("Sản phẩm:", len(df))
# len() trả số sản phẩm đạt ngưỡng sau lọc.
print("Giá cao:", len(selected))
# join() ghép danh sách mã từ Series id bằng dấu phẩy và khoảng trắng.
print("Mã:", ", ".join(selected["id"]))
```

Kết quả khi chạy:

```
Sản phẩm: 10
Giá cao: 5
Mã: P02, P03, P05, P06, P09
```

Ngưỡng **bao gồm** 800 do dùng >=. Mã hàng được giữ riêng để biết chính xác những sản phẩm nào cần rà soát.

Tình huống 2 — Cấp 2: Phân biệt vị trí iloc với mã giao dịch ở 12 phiếu

Từ vựng trong mục: iloc — chọn theo vị trí; **Slice** — lát cắt; **Index label** — nhãn hàng.

Tình huống thực tế: Kho nhận **12 phiếu xuất K01–K12**, theo đúng thứ tự ghi nhận; mỗi phiếu có số món đã xuất. Trưởng kho yêu cầu xem **phiếu thứ 4 đến phiếu thứ 7**, tức bốn phiếu liên tiếp, và cộng tổng số món của bốn phiếu này. Nhân viên cần ghi lại mã phiếu đầu và cuối trong đoạn đã chọn để không chọn lệch một hàng.

Vì sao chọn cách này? `iloc` chọn theo **vị trí hàng tính từ 0**, khác `loc` chọn theo nhãn hàng hoặc điều kiện. Phiếu thứ tư có vị trí 3, nên `df.iloc[3:7]` lấy các vị trí **3, 4, 5, 6**, tương ứng K04 đến K07; số 7 ở cuối lát cắt **không được lấy**. Sau đó cộng cột `quantity` của đúng bốn dòng này để ra số món xuất. Nếu dùng `[4:7]` vì thấy chữ 'phiếu thứ 4', chương trình sẽ bỏ K04 và chỉ còn ba phiếu khác với yêu cầu.

Quy trình:

1. Ghi 12 phiếu theo thứ tự phát sinh.
2. Cắt các vị trí 3–6 bằng `iloc[3:7]`.
3. In mã đầu/cuối, số dòng và tổng món của lát cắt.

No Note — Không chú thích

```
import pandas as pd
quantities = [1,2,3,4,1,2,3,4,1,2,3,4]
ids = []
for i in range(1, 13):
    ids.append(f"K{i:02d}")
df = pd.DataFrame({"id": ids,
                   "quantity": quantities})
portion = df.iloc[3:7]
if len(df) != 12 or len(portion) != 4:
    raise ValueError("Cần 12 phiếu và đúng 4 phiếu được chọn.")
print("Phiếu:", len(df))
print("Đã chọn:", len(portion))
print("Mã đầu/cuối:", portion["id"].iloc[0], portion["id"].iloc[-1])
print("Món xuất:", int(portion["quantity"].sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng .iloc chọn đúng bốn phiếu liên tiếp trong 12 phiếu và tính số món.
# Dữ liệu: 12 dòng K01..K12, quantity là số món cần xuất trên mỗi phiếu.
# Pandas .iloc chọn hàng theo vị trí số nguyên.
import pandas as pd
quantities = [1,2,3,4,1,2,3,4,1,2,3,4]
ids = []
# range(1,13) tạo mã thứ tự 1..12, không lấy 13.
for i in range(1, 13):
    # f-string :02d tạo mã K01..K12 và append() giữ theo thứ tự.
    ids.append(f"K{i:02d}")
# Ghép hai cột id và quantity với 12 hàng.
df = pd.DataFrame({"id": ids,
                   "quantity": quantities})
# iloc[3:7] lấy chỉ số 3,4,5,6 (K04..K07); không lấy vị trí 7.
portion = df.iloc[3:7]
if len(df) != 12 or len(portion) != 4:
    # ValueError báo số bản ghi/khóa không hợp lệ rồi dừng, tránh công bố kết quả ghép sai.
    raise ValueError("Cần 12 phiếu và đúng 4 phiếu được chọn.")
print("Phiếu:", len(df))
print("Đã chọn:", len(portion))
```

```
# Lấy mã đầu ở vị trí 0 của phần đã cắt; iloc[-1] là mã cuối.
print("Mã đầu/cuối:", portion["id"].iloc[0], portion["id"].iloc[-1])
# sum() cộng số´món của đúng bốn phiếu đã chọn, int() hiển thị số´nguyên.
print("Món xuất:", int(portion["quantity"].sum()))
```

Kết quả khi chạy:

```
Phiếu: 12
Đã chọn: 4
Mã đầu/cuối: K04 K07
Món xuất: 10
```

Lát cắt vẫn là bốn dòng thật từ bảng 12 phiếu; không tạo thêm hay loại ngẫu nhiên dữ liệu.

Tình huống 3 — Cấp 3: Nhóm 15 giao dịch theo vùng và sắp doanh số

Từ vựng trong mục: groupby — nhóm theo khóa; **agg** — tổng hợp; **sort_values** — sắp xếp.

Tình huống thực tế: Một đại lý có **15 giao dịch** bán hàng; mỗi dòng gồm tên vùng **Bắc, Nam hoặc Trung** và tiền giao dịch tính bằng **triệu đồng**. Chủ đại lý cần biết từng vùng có **bao nhiêu giao dịch**, tổng doanh thu từng vùng, rồi trình bày vùng có tổng tiền cao hơn trước. Đếm 15 dòng chung không trả lời được vùng nào đang hoạt động mạnh nhất.

Vì sao chọn cách này? Groupby nghĩa là gom các hàng có chung một thông tin, ở đây là tên vùng. `groupby('area')` tách 15 giao dịch thành ba nhóm Bắc, Nam, Trung; `agg` yêu cầu hai phép thống kê cho mỗi nhóm: `size` đếm **số giao dịch**, `sum` cộng **tiền giao dịch**. Hai kết quả không thay thế nhau: vùng ít giao dịch có thể vẫn có doanh thu lớn. `sort_values` sắp theo tổng doanh thu **giảm dần** và tên vùng để có thứ tự nhất quán khi hòa tiền. Không cộng tất cả 15 giao dịch rồi gán cùng một tổng cho mỗi vùng.

Quy trình:

1. Tạo 15 dòng với đơn vị doanh thu rõ ràng.
2. Nhóm theo vùng, đếm số giao dịch và cộng doanh thu.
3. Sắp theo doanh thu giảm rồi theo tên vùng để khi hòa điểm vẫn ổn định.

No Note — Không chú thích

```
import pandas as pd
rows = [{"area":"Băć","amount":10},{"area":"Nam","amount":15},{"area":"Trung","amount":20},
{"area":"Băć","amount":25},{"area":"Nam","amount":30},{"area":"Trung","amount":10},
{"area":"Băć","amount":15},{"area":"Nam","amount":20},{"area":"Trung","amount":25},
{"area":"Băć","amount":30},{"area":"Nam","amount":10},{"area":"Trung","amount":15},
{"area":"Băć","amount":20},{"area":"Nam","amount":25},{"area":"Trung","amount":30}]
df = pd.DataFrame(rows)
grouped = df.groupby("area", as_index=False).agg(
    orders=("amount", "size"), total_million=("amount", "sum"))
grouped = grouped.sort_values(["total_million", "area"], ascending=[False, True])
if len(df) != 15 or grouped["orders"].sum() != 15:
    raise ValueError("Cần tổng hợp đủ 15 phiếú theo vùng.")
print("Giao dịch:", len(df))
print("Sô´vùng:", len(grouped))
for record in grouped.to_dict("records"):
    print(f"{record['area']}: {record['orders']} phiếú, {record['total_million']} triệu")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: GroupBy 15 giao dịch theo ba vùng, đếm phiếu và cộng tiền rồi xếp hạng.
# Dữ liệu: 15 giao dịch; mỗi hàng area Bắc/Nam/Trung, amount tính triệu đồng.
# Pandas groupby() và agg() tổng hợp theo nhóm.
import pandas as pd
# 15 dict, mỗi dict là một phiếu theo vùng với số tiền triệu đồng.
rows = [{"area":"Bắc","amount":10},{"area":"Nam","amount":15},{"area":"Trung","amount":20},
{"area":"Bắc","amount":25},{"area":"Nam","amount":30},{"area":"Trung","amount":10},
{"area":"Bắc","amount":15},{"area":"Nam","amount":20},{"area":"Trung","amount":25},
{"area":"Bắc","amount":30},{"area":"Nam","amount":10},{"area":"Trung","amount":15},
{"area":"Bắc","amount":20},{"area":"Nam","amount":25},{"area":"Trung","amount":30}]
# Tạo bảng với hai cột area và amount.
df = pd.DataFrame(rows)
# groupby('area') gom dòng cùng vùng; as_index=False giữ area thành CỘT thay vì đưa vào index.
grouped = df.groupby("area", as_index=False).agg(
    # agg() dạng đặt tên: size đếm SỐ HÀNG trong vùng, bao gồm cả giá trị thiếu nếu có.
    # agg() sum cộng tiền mỗi vùng, tạo cột total_million (triệu đồng).
    orders=("amount", "size"), total_million=("amount", "sum"))
# sort_values theo tổng giảm dần (False), nếu hòa xếp tên vùng tăng dần (True), cho kết quả ổn
định.
grouped = grouped.sort_values(["total_million", "area"], ascending=[False, True])
# sum() cộng số phiếu của các nhóm; phải bằng 15 giao dịch đầu vào.
if len(df) != 15 or grouped["orders"].sum() != 15:
    # ValueError báo sớm bản ghi/khóa không hợp lệ rồi dừng, tránh công bố kết quả ghép sai.
    raise ValueError("Cần tổng hợp đủ 15 phiếu theo vùng.")
print("Giao dịch:", len(df))
print("Số vùng:", len(grouped))
# to_dict('records') biến DataFrame thành list dict, mỗi dict chứa một vùng để in trường area/
orders/total_million.
for record in grouped.to_dict("records"):
    print(f"{record['area']}: {record['orders']} phiếu, {record['total_million']} triệu")
```

Kết quả khi chạy:

```
Giao dịch: 15
Số vùng: 3
Bắc: 5 phiếu, 100 triệu
Nam: 5 phiếu, 100 triệu
Trung: 5 phiếu, 100 triệu
```

Từng giá trị là **doanh thu tổng** trong một vùng, không phải doanh thu trung bình mỗi đơn; muốn trung bình phải yêu cầu phép tính khác.

Tình huống 4 — Cấp 4: Ghép 18 đơn với 12 khách mà không đánh mất đơn lạ

Từ vựng trong mục: merge — ghép bảng; **Left join** — giữ bảng trái; **Unmatched key** — khóa không khớp.

Tình huống thực tế: Ứng dụng có bảng **18 đơn hàng O01–O18** và bảng thông tin **12 khách C01–C12**. Mỗi đơn ghi mã khách và tiền hàng (**nghìn đồng**); bảng khách ghi **thành phố** của mỗi người. Riêng đơn **O17** tham chiếu mã khách **C99**, chưa có trong danh mục. Nhân viên cần bổ sung thành phố cho những đơn tìm được khách, nhưng vẫn phải giữ đủ **18 đơn** và báo chính xác mã đơn không khớp để sửa.

Vì sao chọn cách này? Ghép bảng (merge) là tìm dòng thuộc bảng khách bằng **mã khách** trên từng đơn hàng, rồi bổ sung thông tin như thành phố. Một khách có thể đặt **nhiều đơn**, còn mã khách trong danh mục chỉ được xuất hiện **một lần**: `validate='many_to_one'` kiểm tra đúng kiểu quan hệ này để tránh nhân đôi đơn khi bảng khách bị trùng mã. `how='left'` giữ **tất cả 18 đơn** của bảng bên trái, kể cả O17 không tìm thấy C99; `indicator=True` đánh dấu dòng đã ghép được hay còn thiếu, nhờ đó in ra O17 để sửa. Nếu dùng `inner` merge, O17 có thể biến mất khỏi báo cáo mà không có cảnh báo.

Quy trình:

1. Tạo bảng 18 đơn và danh mục 12 khách không trùng mã.
2. Left-merge theo khóa `customer_id`; kiểm tra quan hệ nhiều–một.
3. Đếm số dòng không khớp và xác nhận không mất đơn hàng.

No Note — Không chú thích

```
import pandas as pd
orders = pd.DataFrame([{"id":"O01","customer_id":"C01","amount":100},
{"id":"O02","customer_id":"C02","amount":125},{"id":"O03","customer_id":"C03","amount":150},
{"id":"O04","customer_id":"C04","amount":175},{"id":"O05","customer_id":"C05","amount":200},
{"id":"O06","customer_id":"C06","amount":100},{"id":"O07","customer_id":"C07","amount":125},
{"id":"O08","customer_id":"C08","amount":150},{"id":"O09","customer_id":"C09","amount":175},
{"id":"O10","customer_id":"C10","amount":200},{"id":"O11","customer_id":"C11","amount":100},
{"id":"O12","customer_id":"C12","amount":125},{"id":"O13","customer_id":"C01","amount":150},
{"id":"O14","customer_id":"C02","amount":175},{"id":"O15","customer_id":"C03","amount":200},
{"id":"O16","customer_id":"C04","amount":100},{"id":"O17","customer_id":"C99","amount":125},
{"id":"O18","customer_id":"C06","amount":150}])
customers = pd.DataFrame([{"customer_id":"C01","city":"Hà Nội"},{"customer_id":"C02","city":"Đà
Nẵng"},{"customer_id":"C03","city":"TP.HCM"},{"customer_id":"C04","city":"Hà Nội"},
{"customer_id":"C05","city":"Đà Nẵng"},{"customer_id":"C06","city":"TP.HCM"},
{"customer_id":"C07","city":"Hà Nội"},{"customer_id":"C08","city":"Đà Nẵng"},
{"customer_id":"C09","city":"TP.HCM"},{"customer_id":"C10","city":"Hà Nội"},
{"customer_id":"C11","city":"Đà Nẵng"},{"customer_id":"C12","city":"TP.HCM"}])
merged = orders.merge(customers, on="customer_id", how="left",
                      validate="many_to_one", indicator=True)
unmatched = merged.loc[merged["_merge"] == "left_only", "id"].tolist()
if len(orders) != 18 or len(merged) != 18 or not customers["customer_id"].is_unique:
    raise ValueError("Cần đủ 18 đơn sau ghép và mã khách hàng duy nhất.")
print("Đơn:", len(orders))
print("Khách danh mục:", len(customers))
print("Đơn sau ghép:", len(merged))
print("Chưa tìm thấy khách:", ", ".join(unmatched))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghép 18 đơn với danh mục 12 khách, giữ đơn thiếu khách và phát hiện mã C99.
# Dữ liệu: 18 đơn có id/customer_id/amount và danh mục 12 khách customer_id/city; C99 không tồn tại
ở bảng khách.
# Pandas merge() kết hợp hai bảng theo khóa chung.
import pandas as pd
# Bảng trái gồm 18 đơn; customer_id trỏ vào mã khách, mỗi khách có thể nhiều đơn.
orders = pd.DataFrame([{"id":"O01","customer_id":"C01","amount":100},
{"id":"O02","customer_id":"C02","amount":125},{"id":"O03","customer_id":"C03","amount":150},
{"id":"O04","customer_id":"C04","amount":175},{"id":"O05","customer_id":"C05","amount":200},
{"id":"O06","customer_id":"C06","amount":100},{"id":"O07","customer_id":"C07","amount":125},
{"id":"O08","customer_id":"C08","amount":150},{"id":"O09","customer_id":"C09","amount":175},
{"id":"O10","customer_id":"C10","amount":200},{"id":"O11","customer_id":"C11","amount":100},
{"id":"O12","customer_id":"C12","amount":125},{"id":"O13","customer_id":"C01","amount":150},
{"id":"O14","customer_id":"C02","amount":175},{"id":"O15","customer_id":"C03","amount":200},
{"id":"O16","customer_id":"C04","amount":100},{"id":"O17","customer_id":"C99","amount":125},
{"id":"O18","customer_id":"C06","amount":150}])
# Bảng phải có 12 mã khách duy nhất và thành phố tương ứng.
customers = pd.DataFrame([{"customer_id":"C01","city":"Hà Nội"},{"customer_id":"C02","city":"Đà
Nẵng"},{"customer_id":"C03","city":"TP.HCM"},{"customer_id":"C04","city":"Hà Nội"},
{"customer_id":"C05","city":"Đà Nẵng"},{"customer_id":"C06","city":"TP.HCM"},
{"customer_id":"C07","city":"Hà Nội"},{"customer_id":"C08","city":"Đà Nẵng"},
{"customer_id":"C09","city":"TP.HCM"},{"customer_id":"C10","city":"Hà Nội"},
{"customer_id":"C11","city":"Đà Nẵng"},{"customer_id":"C12","city":"TP.HCM"}])
# on='customer_id' chọn khóa; how='left' giữ mọi đơn, kể cả khách C99 chưa có danh mục.
merged = orders.merge(customers, on="customer_id", how="left",
                      # validate='many_to_one' buộc bảng khách có mã duy nhất; indicator=True tạo
```

```
cột _merge nêu nguồn khớp.
                     validate="many_to_one", indicator=True)
# _merge='left_only' nghĩa đơn ở bảng trái không tìm được khách bên phải.
# .loc[mask,'id'] lấy mã đơn không khớp; tolist() chuyển Series sang list để báo cáo.
unmatched = merged.loc[merged["_merge"] == "left_only", "id"].tolist()
# .is_unique xác minh không có trùng khóa khách, tránh một đơn bị nhân bản khi merge.
# Kiểm số dòng sau ghép vẫn là 18 (left join với khóa phía phải duy nhất).
if len(orders) != 18 or len(merged) != 18 or not customers["customer_id"].is_unique:
    # ValueError báo số bản ghi/khóa không hợp lệ rồi dừng, tránh công bố kết quả ghép sai.
    raise ValueError("Cần đủ 18 đơn sau ghép và mã khách hàng duy nhất.")
print("Đơn:", len(orders))
print("Khách danh mục:", len(customers))
# Kiểm số dòng sau ghép vẫn là 18 (left join với khóa phía phải duy nhất).
print("Đơn sau ghép:", len(merged))
print("Chưa tìm thấy khách:", ", ".join(unmatched))
```

Kết quả khi chạy:

```
Đơn: 18
Khách danh mục: 12
Đơn sau ghép: 18
Chưa tìm thấy khách: O17
```

O17 vẫn ở bảng kết quả với thành phố thiếu, nhờ đó lỗi khóa còn nhìn thấy để sửa. Không gán thành phố giả cho C99.

Tình huống 5 — Cấp 5: Ghép 20 dòng bán hàng với giá và phân nhóm doanh thu

Từ vựng trong mục: Merge validation — kiểm quan hệ khóa; **Groupby** — nhóm; **Data quality** — chất lượng dữ liệu.

Tình huống thực tế: Kế toán nhận **20 dòng bán hàng O01–O20**, mỗi dòng ghi **SKU** (mã loại hàng) và **số món bán**. Một bảng giá riêng chỉ có năm SKU **P01–P05**, đơn giá tương ứng tính bằng **nghìn đồng/món**. Dòng O19 ghi **P99** nên chưa tìm được đơn giá. Cần ghép bảng để tính tiền cho các dòng có giá, giữ riêng mã hóa đơn cần bổ sung giá, tổng hợp doanh thu theo SKU và xác định SKU doanh thu cao nhất; không tự tính P99 bằng 0 đồng.

Vì sao chọn cách này? SKU là mã nhận diện loại sản phẩm, dùng để tìm **đơn giá** tương ứng trong bảng giá. Chương trình ghép 20 dòng bán với năm giá theo SKU bằng `how='left'`, tức vẫn giữ dòng O19 dù P99 chưa có giá. `validate='many_to_one'` bảo đảm mỗi SKU chỉ có **một giá** trong danh mục để không tạo thêm dòng bán giả. `isna()` xác định các dòng thiếu giá; chỉ lấy những dòng `notna()` rồi nhân **số món × giá một món** để có doanh thu từng dòng. `groupby('sku').sum()` cộng các dòng của cùng một sản phẩm và `sort_values` xếp SKU doanh thu cao nhất trước. O19 được báo cần sửa, không được đổi thành doanh thu 0 vì số tiền thực tế chưa biết.

Quy trình:

1. Tạo 20 dòng bán và bảng giá 5 SKU duy nhất.
2. Left-merge theo `sku`, tách các dòng có giá thiếu để báo lỗi.
3. Tính doanh thu theo dòng `price * qty`, groupby theo SKU, sắp top; xác nhận 19 + 1 = 20.

No Note — Không chú thích

```
import pandas as pd
sales = pd.DataFrame([{"invoice_id":"O01","sku":"P01","qty":1},
{"invoice_id":"O02","sku":"P02","qty":2},{"invoice_id":"O03","sku":"P03","qty":3},
```

```
{"invoice_id":"O04","sku":"P04","qty":4},{"invoice_id":"O05","sku":"P05","qty":1},
{"invoice_id":"O06","sku":"P01","qty":2},{"invoice_id":"O07","sku":"P02","qty":3},
{"invoice_id":"O08","sku":"P03","qty":4},{"invoice_id":"O09","sku":"P04","qty":1},
{"invoice_id":"O10","sku":"P05","qty":2},{"invoice_id":"O11","sku":"P01","qty":3},
{"invoice_id":"O12","sku":"P02","qty":4},{"invoice_id":"O13","sku":"P03","qty":1},
{"invoice_id":"O14","sku":"P04","qty":2},{"invoice_id":"O15","sku":"P05","qty":3},
{"invoice_id":"O16","sku":"P01","qty":4},{"invoice_id":"O17","sku":"P02","qty":1},
{"invoice_id":"O18","sku":"P03","qty":2},{"invoice_id":"O19","sku":"P99","qty":3},
{"invoice_id":"O20","sku":"P05","qty":4}])
prices = pd.DataFrame([["P01",10],["P02",20],["P03",30],["P04",15],["P05",25]], columns=["sku",
"price_thousand"])
merged = sales.merge(prices, on="sku", how="left", validate="many_to_one")
missing = merged.loc[merged["price_thousand"].isna(), "invoice_id"].tolist()
valid = merged.loc[merged["price_thousand"].notna()].copy()
valid["revenue_thousand"] = valid["qty"] * valid["price_thousand"]
by_sku = valid.groupby("sku", as_index=False)["revenue_thousand"].sum()
ranking = by_sku.sort_values(["revenue_thousand", "sku"], ascending=[False, True])
if len(sales) != 20 or len(merged) != 20 or len(valid) + len(missing) != 20:
    raise ValueError("Cần đối chiếu đủ 20 dòng bán, gồm cả SKU chưa có giá.")
print("Dòng bán:", len(sales))
print("Có giá:", len(valid))
print("Chưa có giá:", ", ".join(missing))
print("Doanh thu (nghìn đồng):", int(valid["revenue_thousand"].sum()))
print("SKU cao nhất:", ranking.iloc[0]["sku"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghép 20 lượt bán với bảng giá theo SKU, bỏ dòng thiếu giá và xếp doanh thu theo mặt
hàng.
# Dữ liệu: 20 lượt bán (invoice_id, sku, qty món), bảng giá 5 SKU (nghìn đồng/món); P99 chưa có
giá.
# Pandas xử lý merge, lọc NaN và groupby để tính doanh thu.
import pandas as pd
# 20 hàng: mỗi hóa đơn một SKU và số món bán.
sales = pd.DataFrame([{"invoice_id":"O01","sku":"P01","qty":1},
{"invoice_id":"O02","sku":"P02","qty":2},{"invoice_id":"O03","sku":"P03","qty":3},
{"invoice_id":"O04","sku":"P04","qty":4},{"invoice_id":"O05","sku":"P05","qty":1},
{"invoice_id":"O06","sku":"P01","qty":2},{"invoice_id":"O07","sku":"P02","qty":3},
{"invoice_id":"O08","sku":"P03","qty":4},{"invoice_id":"O09","sku":"P04","qty":1},
{"invoice_id":"O10","sku":"P05","qty":2},{"invoice_id":"O11","sku":"P01","qty":3},
{"invoice_id":"O12","sku":"P02","qty":4},{"invoice_id":"O13","sku":"P03","qty":1},
{"invoice_id":"O14","sku":"P04","qty":2},{"invoice_id":"O15","sku":"P05","qty":3},
{"invoice_id":"O16","sku":"P01","qty":4},{"invoice_id":"O17","sku":"P02","qty":1},
{"invoice_id":"O18","sku":"P03","qty":2},{"invoice_id":"O19","sku":"P99","qty":3},
{"invoice_id":"O20","sku":"P05","qty":4}])
# Danh mục có một mức giá nghìn đồng/món cho mỗi SKU P01..P05.
prices = pd.DataFrame([["P01",10],["P02",20],["P03",30],["P04",15],["P05",25]], columns=["sku",
"price_thousand"])
# merge theo SKU; left giữ tất cả 20 giao dịch; many_to_one bảo đảm mỗi SKU có tối đa một hàng giá,
không nhân doanh thu.
merged = sales.merge(prices, on="sku", how="left", validate="many_to_one")
# isna() tạo mask nơi SKU không có giá sau khi nối (ví dụ P99).
# .loc[mask,'invoice_id'] ghi hóa đơn thiếu giá, không ngầm coi là giá 0.
missing = merged.loc[merged["price_thousand"].isna(), "invoice_id"].tolist()
# notna() nhận dòng có giá; .copy() tạo bảng hợp lệ độc lập.
valid = merged.loc[merged["price_thousand"].notna()].copy()
# Tạo cột doanh thu nghìn đồng = số món × giá nghìn đồng/món, theo từng hàng.
valid["revenue_thousand"] = valid["qty"] * valid["price_thousand"]
# groupby('sku') gom dòng bán cùng mặt hàng, as_index=False giữ SKU thành cột; sum() cộng doanh thu
từng SKU.
by_sku = valid.groupby("sku", as_index=False)["revenue_thousand"].sum()
# Xếp doanh thu giảm dần, nếu bằng tiền thì tên SKU tăng dần để giải quyết hòa.
ranking = by_sku.sort_values(["revenue_thousand", "sku"], ascending=[False, True])
# Đối chiếu 20 dòng = có giá + chưa có giá, không đánh mất giao dịch.
if len(sales) != 20 or len(merged) != 20 or len(valid) + len(missing) != 20:
```

```
    # ValueError báo sô´bản ghi/khóa không hợp lệ rôi dừng, tránh công bô´kêt quả ghép sai.
    raise ValueError("Cần đối chiếu đủ 20 dòng bán, gồm cả SKU chưa có giá.")
print("Dòng bán:", len(sales))
print("Có giá:", len(valid))
print("Chưa có giá:", ", ".join(missing))
# sum() tổng doanh thu hợp lệ theo nghìn đồng, không cộng hàng chưa có giá.
print("Doanh thu (nghìn đồng):", int(valid["revenue_thousand"].sum()))
# iloc[0] là hàng đầu của bảng xếp hạng, ['sku'] trả mã SKU có doanh thu cao nhất.
print("SKU cao nhất:", ranking.iloc[0]["sku"])
```

Kết quả khi chạy:

```
Dòng bán: 20
Có giá: 19
Chưa có giá: O19
Doanh thu (nghìn đồng): 955
SKU cao nhất: P03
```

Doanh thu tổng hợp ở đây chỉ gồm 19 dòng có giá; O19 cần bổ sung SKU thật trước khi có thể hoàn thiện báo cáo.

6. Sai lầm thường gặp

- `.loc` dùng **nhãn**, `.iloc` dùng **vị trí số**; không nhầm nhãn cột `id` với index của DataFrame.
- Với lọc Boolean, phải dùng điều kiện trên đúng cột; `df.loc[mask, columns]` khác `.iloc`.
- `groupby('city').sum()` chỉ có ý nghĩa khi hiểu **mỗi hàng là giao dịch**, cột `city` là khóa phân nhóm và `sale` là lượng cần tổng hợp.
- `merge` dùng khóa; nếu bảng phụ có **khóa trùng**, số hàng sau ghép có thể tăng. Có thể dùng `validate='many_to_one'` khi bảng phải phải duy nhất.
- `sort_values()` mặc định tăng dần; để lấy nhóm doanh thu cao nhất cần `ascending=False` trước `head()`.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does a Boolean filter do? | Lọc Boolean làm gì?

A. Integer positions | Vị trí số nguyên

B. Groups records sharing a key | Gom bản ghi có chung khóa

C. Keeps rows matching a condition | Giữ hàng thỏa điều kiện

D. Index labels | Nhãn index

Đáp án: C

Giải thích: Mặt nạ Boolean xác định hàng cần giữ. Với `df[df['score'] >= 8]`, chỉ những bản ghi đạt ngưỡng điểm được giữ lại; các hàng khác bị lọc, không phải được gộp thống kê.

Q2. What does `loc` use to select rows? | `loc` chọn hàng bằng gì?

A. Keeps rows matching a condition | Giữ hàng thỏa điều kiện

B. Integer positions | Vị trí số nguyên

C. Groups records sharing a key | Gom bản ghi có chung khóa

D. Index labels | Nhãn index

Đáp án: D

Giải thích: loc chọn theo nhãn hàng. Nếu DataFrame có nhãn hàng `'A'`, `df.loc['A']` chọn theo nhãn đó dù hàng không nằm ở vị trí đầu.

Q3. What does `iloc` use to select rows? | `iloc` chọn hàng bằng gì?

A. Integer positions | Vị trí số nguyên

B. Keeps rows matching a condition | Giữ hàng thỏa điều kiện

C. Groups records sharing a key | Gom bản ghi có chung khóa

D. Index labels | Nhãn index

Đáp án: A

Giải thích: iloc chọn theo vị trí đếm từ 0. `df.iloc[0]` chọn hàng đầu theo vị trí; hàng có nhãn `0` không nhất thiết nằm đầu sau khi sắp xếp dữ liệu.

Q4. What does `groupby` do? | `groupby` làm gì?

A. Index labels | Nhãn index

B. Keeps rows matching a condition | Giữ hàng thỏa điều kiện

C. Groups records sharing a key | Gom bản ghi có chung khóa

D. Integer positions | Vị trí số nguyên

Đáp án: C

Giải thích: Phân nhóm gom các hàng có chung một trường. Với các thành phố `HN`, `HN`, `HCM`, groupby theo thành phố tạo hai nhóm để tổng hợp mà không tự loại các thành phố còn lại.

Q5. What is the join key in `merge`? | Khóa ghép trong `merge` là gì?

A. A sale total per city | Tổng doanh thu theo từng vùng

B. Descending order | Thứ tự giảm dần

C. A shared field used to match rows | Trường chung để ghép các hàng

D. All rows from the left table | Mọi hàng bảng trái

Đáp án: C

Giải thích: Các hàng được ghép khi khóa phù hợp. Nếu hai bảng đều có `customer_id`, các hàng có mã khớp có thể ghép với nhau; vẫn cần xem kiểu dữ liệu và khóa trùng.

Q6. What does `how='left'` retain? | `how='left'` giữ gì?

A. Descending order | Thứ tự giảm dần

B. A shared field used to match rows | Trường chung để ghép các hàng

C. A sale total per city | Tổng doanh thu theo từng vùng

D. All rows from the left table | Mọi hàng bảng trái

Đáp án: D

Giải thích: Ghép trái không bỏ hàng bên trái chưa khớp. Left join giữ mọi bản ghi bảng trái kể cả khi bảng phải không khớp và điền giá trị thiếu phía phải; nhiều hàng phải cùng khớp có thể làm tăng số hàng kết quả.

Q7. What does `groupby('city')['sale'].sum()` return? | `groupby('city')['sale'].sum()` trả gì?

A. A shared field used to match rows | Trường chung để ghép các hàng

B. A sale total per city | Tổng doanh thu theo từng vùng

C. Descending order | Thứ tự giảm dần

D. All rows from the left table | Mọi hàng bảng trái

Đáp án: B

Giải thích: sum cộng giá trị sale theo từng vùng. Nếu `HN` có doanh thu 10, 15 và `HCM` có 8, tổng từng thành phố lần lượt là 25 và 8, không chỉ một tổng toàn bảng.

Q8. What does `ascending=False` mean in sorting? | `ascending=False` khi sắp xếp nghĩa là gì?

A. All rows from the left table | Mọi hàng bảng trái

B. A shared field used to match rows | Trường chung để ghép các hàng

C. Descending order | Thứ tự giảm dần

D. A sale total per city | Tổng doanh thu theo từng vùng

Đáp án: C

Giải thích: False chuyển sang thứ tự giảm dần. Sắp điểm `[3,1,2]` với `ascending=False` cho `[3,2,1]`, không giữ thứ tự gốc.

Q9. What distinguishes `loc` from `iloc`? | `loc` khác `iloc` ở đâu?

A. Retains selected rows versus grouping by key | Giữ hàng được chọn so với gom theo khóa

B. First two rows versus last two | Hai hàng đầu so với hai hàng cuối

C. Labels versus positions | Nhãn so với vị trí

D. Matching rows only versus all left rows | Chỉ hàng khớp so với mọi hàng trái

Đáp án: C

Giải thích: Nhãn 7 không nhất thiết nằm ở vị trí 7. Nhãn `'7'` có thể nằm tại vị trí đầu, nên `loc['7']` và `iloc[7]` có thể chọn các hàng khác nhau hoặc gây lỗi.

Q10. How does filtering differ from grouping? | Lọc khác phân nhóm như thế nào?

A. Matching rows only versus all left rows | Chỉ hàng khớp so với mọi hàng trái

B. Labels versus positions | Nhãn so với vị trí

C. First two rows versus last two | Hai hàng đầu so với hai hàng cuối

D. Retains selected rows versus grouping by key | Giữ hàng được chọn so với gom theo khóa

Đáp án: D

Giải thích: Lọc bỏ hàng không khớp; nhóm tổ chức bản ghi. Lọc chỉ giữ bản ghi thỏa điều kiện, còn phân nhóm tổ chức bản ghi theo khóa để có thể tính tổng từng nhóm bằng `sum()`.

Q11. How does `head(2)` differ from `tail(2)`? | `head(2)` khác `tail(2)` thế nào?

A. Retains selected rows versus grouping by key | Giữ hàng được chọn so với gom theo khóa

B. First two rows versus last two | Hai hàng đầu so với hai hàng cuối

C. Labels versus positions | Nhãn so với vị trí

D. Matching rows only versus all left rows | Chỉ hàng khớp so với mọi hàng trái

Đáp án: B

Giải thích: Hai hàm lấy dữ liệu ở hai đầu bảng. Với năm hàng, `head(2)` lấy hai hàng đầu, còn `tail(2)` lấy hàng thứ tư và thứ năm theo thứ tự hiện tại.

Q12. How does an inner join differ from a left join? | Inner join khác left join thế nào?

A. First two rows versus last two | Hai hàng đầu so với hai hàng cuối

B. Matching rows only versus all left rows | Chỉ hàng khớp so với mọi hàng trái

C. Retains selected rows versus grouping by key | Giữ hàng được chọn so với gom theo khóa

D. Labels versus positions | Nhãn so với vị trí

Đáp án: B

Giải thích: Inner join bỏ hàng trái không ghép được. Nếu một khách trong bảng trái không có hàng ghép bên phải, inner join bỏ khách đó, còn left join giữ lại với các trường phải bị thiếu.

Q13. What risk comes from duplicate right-side join keys? | Khóa ghép bảng phải bị trùng gây nguy cơ gì?

A. Unexpected row multiplication | Nhân số hàng ngoài ý muốn

B. Ascending order | Thứ tự tăng dần

C. Unique right-side join keys | Khóa ghép bảng phải duy nhất

D. Rebuilds sequential row labels | Đánh lại nhãn hàng liên tiếp

Đáp án: A

Giải thích: Mỗi hàng phải khớp thêm có thể nhân dữ liệu trái. Một khóa trái khớp hai hàng phải có thể sinh hai hàng ghép, làm tổng bị phóng đại nếu không dự kiến khóa trùng.

Q14. What does `validate='many_to_one'` check? | `validate='many_to_one'` kiểm tra gì?

A. Rebuilds sequential row labels | Đánh lại nhãn hàng liên tiếp

B. Unique right-side join keys | Khóa ghép bảng phải duy nhất

C. Unexpected row multiplication | Nhân số hàng ngoài ý muốn

D. Ascending order | Thứ tự tăng dần

Đáp án: B

Giải thích: Nhiều hàng trái được phép ghép một hàng phải. Kiểm tra này cho phép nhiều hàng trái có cùng khóa nhưng yêu cầu phía phải tối đa một hàng cho mỗi khóa, nếu không Pandas báo lỗi kiểm chứng merge.

Q15. What does `reset_index(drop=True)` change? | `reset_index(drop=True)` thay đổi gì?

A. Ascending order | Thứ tự tăng dần

B. Unique right-side join keys | Khóa ghép bảng phải duy nhất

C. Unexpected row multiplication | Nhân số hàng ngoài ý muốn

D. Rebuilds sequential row labels | Đánh lại nhãn hàng liên tiếp

Đáp án: D

Giải thích: Đánh lại index loại bỏ nhãn hàng cũ. Sau khi lọc còn nhãn `[2,5]`, `reset_index(drop=True)` đánh nhãn lại `[0,1]` mà không giữ nhãn cũ thành cột.

Q16. What does `ascending=True` request? | `ascending=True` yêu cầu gì?

A. Ascending order | Thứ tự tăng dần

B. Unique right-side join keys | Khóa ghép bảng phải duy nhất

C. Unexpected row multiplication | Nhân số hàng ngoài ý muốn

D. Rebuilds sequential row labels | Đánh lại nhãn hàng liên tiếp

Đáp án: A

Giải thích: Tăng dần là hướng sắp xếp mặc định. Khi sắp tăng dần, các số 1, 2, 3 theo đúng thứ tự đó; `ascending=False` chuyển sang giảm dần.

Q17. What does `df.iloc[0]` select? | `df.iloc[0]` chọn gì?

A. The first row by position | Hàng đầu theo vị trí

B. Keys must have compatible representations | Khóa cần biểu diễn tương thích

C. Rows within each group | Số hàng trong từng nhóm

D. Matching rows in named columns a and b | Hàng phù hợp ở hai cột a và b

Đáp án: A

Giải thích: Vị trí đầu tiên mang số 0. `iloc[0]` chọn vị trí không bất kể nhãn hàng là gì; thao tác thường trả một Series đại diện cho hàng đầu.

Q18. What does `df.loc[mask, ['a','b']]` select? | `df.loc[mask, ['a','b']]` chọn gì?

A. Rows within each group | Số hàng trong từng nhóm

B. The first row by position | Hàng đầu theo vị trí

C. Keys must have compatible representations | Khóa cần biểu diễn tương thích

D. Matching rows in named columns a and b | Hàng phù hợp ở hai cột a và b

Đáp án: D

Giải thích: Mask lọc hàng, danh sách tên lọc cột. Mask Boolean chọn các hàng đạt điều kiện, còn `['a','b']` giới hạn kết quả ở đúng hai cột có tên đó.

Q19. What does `groupby('type').size()` count? | `groupby('type').size()` đếm gì?

A. Rows within each group | Số hàng trong từng nhóm

B. Matching rows in named columns a and b | Hàng phù hợp ở hai cột a và b

C. The first row by position | Hàng đầu theo vị trí

D. Keys must have compatible representations | Khóa cần biểu diễn tương thích

Đáp án: A

Giải thích: size đếm số hàng ở mỗi nhóm. Khác `count()` trên một cột có giá trị thiếu, `size()` của nhóm vẫn đếm hàng ngay cả khi ô ở cột khác bị thiếu.

Q20. Why check types of columns used as merge keys? | Vì sao cần kiểm tra kiểu cột khóa khi merge?

A. Rows within each group | Số hàng trong từng nhóm

B. Keys must have compatible representations | Khóa cần biểu diễn tương thích

C. Matching rows in named columns a and b | Hàng phù hợp ở hai cột a và b

D. The first row by position | Hàng đầu theo vị trí

Đáp án: B

Giải thích: Kiểu khóa khác nhau hoặc sai có thể cản trở ghép đúng. Khóa chuỗi `'001'` không có biểu diễn giống số `1`, nên cần chuẩn hóa kiểu và định dạng trước khi ghép dữ liệu.

8. Nối sang bài tiếp

`DataFrame → lọc → nhóm → ghép bảng → AI.`

PY-20 — Logic mệnh đề, bảng chân trị và quy tắc đếm

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Proposition — mệnh đề; **Truth table** — bảng chân trị.

`Boolean → logic mệnh đề → tổ hợp → xác suất`

Học xong dùng để làm gì? Lập bảng đúng/sai của quy tắc và đếm tổ hợp lựa chọn, ví dụ điều kiện xét duyệt.

Kết nối kiến thức tiếp theo: PY-21 dùng tập hợp và bao hàm–loại trừ để xử lý nhóm bị chồng lấn.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Proposition	mệnh đề	Câu có giá trị True hoặc False
Truth table	bảng chân trị	Liệt kê kết quả ứng với mọi tổ hợp logic
Implication	kéo theo	Quan hệ P⇒Q tương đương (not P) or Q
Counting	nguyên lý đếm	Đếm số khả năng khi ghép lựa chọn

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`and/or/not`	Ghép điều kiện, chọn ít nhất một hoặc phủ định
`(not p) or q`	Biểu diễn phép kéo theo P⇒Q
`itertools.product(A,B)`	Tạo tích Descartes của hai tập tùy chọn
`math.prod(counts)`	Nhân số lựa chọn từng bước độc lập

4. Khái niệm liên quan

Logic mệnh đề giúp chương trình quyết định đúng/sai theo quy tắc rõ ràng; **bảng chân trị** giúp kiểm tra tất cả trường hợp của nhiều điều kiện; **phép kéo theo** biểu diễn quy định "nếu... thì...". Khi phải đếm các lựa chọn có thể xảy ra, **nguyên lý cộng và nhân** cho kết quả mà không cần thử bằng tay từng cách.

4.1. Proposition — Phát biểu đúng hoặc sai

Từ vựng trong mục: Proposition — mệnh đề; **Boolean** — đúng/sai.

Mệnh đề là một phát biểu có giá trị đúng hoặc sai xác định. Chẳng hạn `3 > 1` là đúng, còn `3 < 1` là sai; trong Python, hai kết quả được biểu diễn bằng `True` và `False`. Không phải mọi câu đều là mệnh đề: câu hỏi "Bạn khỏe không?" chưa mang một giá trị đúng/sai để tính toán. Ta dùng mệnh đề khi viết điều kiện kiểm tra truy cập hoặc quy tắc vận hành.

No Note — Không chú thích

```
print(3 > 1)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm một phát biểu logic bằng phép so sánh lớn hơn.
# Dữ liệu: Phát biểu 3 lớn hơn 1 là đúng (True), không phải phép gán.
# Toán tử> so sánh 3 với 1 và trả giá trị Boolean True; print() hiển thị kết quả.
print(3 > 1)
```

Kết quả khi chạy:

```
True
```

Mệnh đề `3 > 1` đúng vì số 3 lớn hơn 1; `print` hiển thị Boolean `True`.

4.2. Truth table — Bảng chân trị

Từ vựng trong mục: Truth table — bảng chân trị; **AND/OR/NOT** — và/hoặc/phủ định.

Bảng chân trị liệt kê mọi kết hợp đúng/sai có thể có của các điều kiện. Với hai biến p, q, mỗi biến có hai trạng thái nên có `2×2 = 4` cặp. `p and q` chỉ đúng khi cả hai đúng; `p or q` đúng nếu **ít nhất một** đúng; `not p` đảo đúng thành sai và ngược lại. Bảng giúp kiểm tra quy tắc ngay cả ở trường hợp khó nhận ra bằng trực giác.

No Note — Không chú thích

```
for p in [True, False]:
    for q in [True, False]:
        print(p, q, p and q, p or q)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Liệt kê bảng chân trị đủ bốn cặp p/q và kết quả AND, OR.
# Dữ liệu: Hai mệnh đề`p và q, mỗi mệnh đề`có hai giá trị True/False.
# Vòng ngoài lần lượt đặt p=True rồi p=False.
for p in [True, False]:
    # Vòng trong thử q=True và q=False cho MỖI p, tạo 2×2=4 cặp.
    for q in [True, False]:
        # and chỉ True khi CẢ p và q đều đúng; or True nếu ÍT NHẤT một trong hai đúng.
        print(p, q, p and q, p or q)
```

Kết quả khi chạy:

```
True True True True
True False False True
```

```
False True False True
False False False False
```

Hai cột đầu là đầu vào p, q; cột thứ ba là AND; cột cuối là OR. Bốn dòng bao phủ mọi trường hợp.

4.3. Implication — Nếu p thì q

Từ vựng trong mục: Implication — kéo theo; **Premise/conclusion** — tiền đề/kết luận.

Mệnh đề kéo theo `p ⇒ q` diễn đạt "nếu p thì q", trong đó p là tiền đề và q là kết luận. Nó **chỉ sai** khi p đúng mà q sai. Ví dụ quy định "nếu đơn được phê duyệt (p), đơn phải có chữ ký (q)": phê duyệt `True` nhưng có chữ ký `False` là vi phạm. Trong Python ta viết `(not p) or q`. Lưu ý đây là cách kiểm tra logic, không tự chứng minh quan hệ nguyên nhân.

No Note — Không chú thích

```
approved = True
signed = False
print((not approved) or signed)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Biểu diễn quy tắc nếu được duyệt thì phải ký bằng (not p) or q.
# Dữ liệu: p là approved (đã duyệt), q là signed (đã ký); p=True, q=False vi phạm điều kiện.
# Giả định hồ sơ đã được duyệt (p đúng).
approved = True
# Giả định hồ sơ chưa ký (q sai).
signed = False
# Mệnh đề kéo theo p→q tương đương (not p) or q: chỉ False khi p đúng mà q sai.
print((not approved) or signed)
```

Kết quả khi chạy:

```
False
```

Đã phê duyệt nhưng chưa ký nên quy tắc kéo theo không được thỏa mãn.

4.4. Counting — Nguyên lý cộng và nhân

Từ vựng trong mục: Counting — quy tắc đếm; **Product** — tích Descartes.

Nguyên lý nhân: nếu có 3 cách chọn màu và mỗi màu đều có 4 cỡ, số cặp (màu, cỡ) là `3×4 = 12`. **Nguyên lý cộng** áp dụng cho lựa chọn thuộc **một trong các nhóm không trùng nhau**: có 3 mẫu áo hoặc 4 mẫu quần để chọn **một món**, tổng cộng 7 cách. Nếu các nhóm có phần giao, phải trừ phần bị đếm lặp; bài tiếp theo sẽ học kỹ phép bao hàm–loại trừ. `itertools.product` liệt kê từng cặp thay vì chỉ đếm.

No Note — Không chú thích

```
colors = 3
sizes = 4
print(colors * sizes)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Áp dụng nguyên lý nhân: sô´kêt hợp màu săć và kích cỡ sản phâm̉.
# Dữ liệu: Có 3 màu và 4 cỡ độc lập; môĩ màu kêt hợp được với từng cỡ.
# Có ba lựa chọn màu, không phải ba sản phâm̉ đã sản xuât.
colors = 3
# Có bôń lựa chọn kích cỡ cho môĩ màu.
sizes = 4
# Nguyên lý nhân cho 3×4=12 cặp câú hình, giả sửmọi cặp đêù hợp lệ.
print(colors * sizes)
```

Kết quả khi chạy:

```
12
```

Có 12 cách chọn một màu và một cỡ khi mọi màu đều có đủ bốn cỡ.

5. Năm tình huống logic — từ hai mệnh đề đến kiểm thử quy tắc phức hợp

Tình huống 1 — Cấp 1: Kiểm tra hai điều kiện truy cập của 10 tài khoản

Từ vựng trong mục: Proposition — mệnh đề; **AND** — và; **Boolean** — đúng/sai.

Tình huống thực tế: Cổng tài liệu học tập có **10 tài khoản U01–U10**. Với mỗi tài khoản, hệ thống ghi hai thông tin **đã đăng nhập** và **đã xác minh**, mỗi thông tin là `True` (đúng) hoặc `False` (chưa đúng). Quy tắc giả lập: chỉ người **đã đăng nhập và đã xác minh** mới được truy cập. Nhân viên cần đếm số tài khoản được phép và bị từ chối; một tài khoản chỉ đạt một trong hai yêu cầu thì vẫn phải bị từ chối.

Vì sao chọn cách này? Boolean là kiểu thông tin chỉ có hai trạng thái: đúng (`True`) hoặc sai (`False`). `and` có nghĩa là **cả hai điều kiện đều phải đúng**: đã đăng nhập nhưng chưa xác minh vẫn trả `False`. Chương trình duyệt đồng thời hai danh sách, kiểm từng cặp của **cùng tài khoản**, rồi lưu một kết quả cho mỗi người. `sum(allowed)` đếm số `True` (trong phép cộng, `True` được tính như 1), còn số bị từ chối là số tài khoản trừ số được phép. Không dùng `or`, vì `or` chỉ đòi **một** điều kiện, sẽ mở quyền quá sớm.

Quy trình:

1. Lưu hai trạng thái theo cùng thứ tự 10 tài khoản.
2. Dùng `logged_in and verified` trên từng tài khoản.
3. Báo số được phép và số từ chối, đối chiếu tổng là 10.

No Note — Không chú thích

```
logged_in = [True,True,False,True,False,True,True,False,True,True]
verified = [True,False,True,True,False,False,True,True,True,False]
allowed = []
for logged, verified_status in zip(logged_in, verified):
    allowed.append(logged and verified_status)
if len(allowed) != 10:
    raise ValueError("Cân kiêm̉ tra đủ 10 tài khoản.")
print("Tài khoản:", len(allowed))
print("Cho phép:", sum(allowed))
print("Từ chôí:", len(allowed) - sum(allowed))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm quyền 10 tài khoản: chỉ tài khoản vừa đăng nhập vừa xác minh mới được phép.
# Dữ liệu: 10 trạng thái logged_in True/False và 10 verified True/False cùng vị trí tài khoản.
# List 10 cờ đăng nhập theo thứ tự tài khoản.
logged_in = [True,True,False,True,False,True,True,False,True,True]
# List 10 cờ xác minh khớp từng vị trí với logged_in.
verified = [True,False,True,True,False,False,True,True,True,False]
# List rỗng lưu kết quả cho phép True/False của từng tài khoản.
allowed = []
# zip() ghép hai cờ ở CÙNG vị trí thành một tài khoản, không tạo mọi tổ hợp.
for logged, verified_status in zip(logged_in, verified):
    # and chỉ True nếu đăng nhập và xác minh đều True; append() giữ đúng một kết quả mỗi tài khoản.
    allowed.append(logged and verified_status)
# len() đếm đủ 10 tài khoản đã kiểm.
if len(allowed) != 10:
    # ValueError dùng khi số lượng hoặc quy tắc logic không nhất quán với dữ liệu đầu vào.
    raise ValueError("Cần kiểm tra đủ 10 tài khoản.")
# len() đếm đủ 10 tài khoản đã kiểm.
print("Tài khoản:", len(allowed))
# Trong Python True đếm như 1 và False như 0; sum() đếm tài khoản được phép.
print("Cho phép:", sum(allowed))
# len() đếm đủ 10 tài khoản đã kiểm.
# Trong Python True đếm như 1 và False như 0; sum() đếm tài khoản được phép.
# Tổng trừ được phép ra số tài khoản bị từ chối.
print("Từ chối:", len(allowed) - sum(allowed))
```

Kết quả khi chạy:

```
Tài khoản: 10
Cho phép: 4
Từ chối: 6
```

Tài khoản chỉ đăng nhập nhưng chưa xác minh bị từ chối, vì một điều kiện đúng chưa đủ thỏa phép `and`.

Tình huống 2 — Cấp 2: Kiểm thử đầy đủ bảng chân trị trên 12 lượt kiểm tra

Từ vựng trong mục: Truth table — bảng chân trị; **OR** — hoặc; **NOT** — phủ định.

Tình huống thực tế: Đội kiểm thử phần mềm muốn xem quy tắc truy cập hoạt động ra sao với **hai trạng thái đúng/sai**: `p` là đã đăng nhập, `q` là đã xác minh. Hai trạng thái này tạo **4 cách kết hợp**, từ cả hai đúng đến cả hai sai. Nhân viên chạy lại cả bốn cách trong **3 ngày**, tổng cộng **12 lượt kiểm tra**, và muốn xác nhận không thiếu trường hợp nào, đồng thời đếm `p and q` và `p or q` đúng bao nhiêu lượt.

Vì sao chọn cách này? Bảng chân trị là cách liệt kê **tất cả các trường hợp đúng/sai có thể có** để kiểm tra một quy tắc, thay vì chỉ thử trường hợp thuận lợi. `itertools.product([True, False], repeat=2)` sinh bốn cặp `p, q` có thể có. Vòng lặp bên ngoài lặp các cặp ấy trong 3 ngày thành 12 lượt; **lượt thử** là số lần chạy, còn **tổ hợp khác nhau** vẫn chỉ có bốn. Với `and`, chỉ cặp **cả hai đúng** mới cho kết quả đúng; với `or`, mọi cặp có **ít nhất một trạng thái đúng** đều đúng. Dùng set các cặp kiểm tra đã bao phủ đủ bốn trường hợp, không nhầm ba lần thử lặp là ba trường hợp mới.

Quy trình:

1. Sinh bốn tổ hợp p/q từ `product` và gắn với ba ngày kiểm thử.
2. Kiểm tra mỗi tổ hợp xuất hiện ba lần.
3. Đếm số lần hai phép AND/OR nhận True và báo độ phủ bảng chân trị.

No Note — Không chú thích

```
from itertools import product
truth_table = list(product([True, False], repeat=2))
checks = []
for day in range(1, 4):
    for p, q in truth_table:
        checks.append((day, p, q))
pairs = set()
for day, p, q in checks:
    pairs.add((p, q))
if len(checks) != 12 or len(pairs) != 4:
    raise ValueError("Cần có 12 lượt kiểm tra với đủ bốn tổ hợp logic.")
print("Lượt kiểm tra:", len(checks))
print("Tổ hợp logic:", len(truth_table))
and_count = 0
or_count = 0
for day, p, q in checks:
    if p and q:
        and_count += 1
    if p or q:
        or_count += 1
print("AND đúng:", and_count)
print("OR đúng:", or_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng itertools.product sinh bảng chân trị bốn cặp, lặp ba ngày tạo 12 lượt kiểm.
# Dữ liệu: p,q là hai mệnh đề Boolean; đủ 4 tổ hợp mỗi ngày × 3 ngày.
# product() của itertools tạo tích Descartes: mọi tổ hợp từ các nhóm lựa chọn.
from itertools import product
# repeat=2 dùng cùng tập hai giá trị cho p và q, sinh 4 cặp; list() lưu lại.
truth_table = list(product([True, False], repeat=2))
checks = []
# range(1,4) sinh ngày 1,2,3, không lấy 4.
for day in range(1, 4):
    # Tách mỗi tuple (p,q) của bảng chân trị cho từng ngày.
    for p, q in truth_table:
        # append() thêm bộ (ngày,p,q) vào list, tổng 12 bộ.
        checks.append((day, p, q))
# set() tập rỗng chỉ lưu cặp logic không trùng, bỏ thông tin ngày.
pairs = set()
for day, p, q in checks:
    # add() tự loại cặp trùng giữa ba ngày để còn bốn tổ hợp duy nhất.
    pairs.add((p, q))
# Kiểm đúng 12 lượt và đủ bốn cặp logic trước khi kết luận.
if len(checks) != 12 or len(pairs) != 4:
    # ValueError dùng khi số lượng hoặc quy tắc logic không nhất quán với dữ liệu đầu vào.
    raise ValueError("Cần có 12 lượt kiểm tra với đủ bốn tổ hợp logic.")
print("Lượt kiểm tra:", len(checks))
print("Tổ hợp logic:", len(truth_table))
and_count = 0
or_count = 0
for day, p, q in checks:
    # AND chỉ đúng khi cả p và q cùng True; += 1 đếm lượt thỏa.
    if p and q:
        and_count += 1
    # OR đúng khi ít nhất một mệnh đề đúng; += 1 đếm lượt thỏa.
    if p or q:
        or_count += 1
print("AND đúng:", and_count)
print("OR đúng:", or_count)
```

Kết quả khi chạy:

```
Lượt kiểm tra: 12
Tổ hợp logic: 4
AND đúng: 3
OR đúng: 9
```

Bảng chân trị chỉ có bốn tổ hợp **khác nhau**; 12 là số lượt chạy có lặp theo ngày để kiểm tra tính nhất quán.

Tình huống 3 — Cấp 3: Kiểm phép kéo theo trên 15 hồ sơ phê duyệt

Từ vựng trong mục: Implication — kéo theo; **Premise** — tiền đề; **Counterexample** — phản ví dụ.

Tình huống thực tế: Một tổ chức kiểm tra **15 hồ sơ phê duyệt R01–R15**. Mỗi dòng ghi hai trạng thái đúng/sai: hồ sơ **đã được phê duyệt** (p) và **đã có chứng chỉ** (q). Nội quy giả lập nói: **nếu đã được phê duyệt thì phải có chứng chỉ**. Nhân viên cần tìm đúng những hồ sơ được phê duyệt nhưng chưa có chứng chỉ, không gắn nhãn vi phạm cho một hồ sơ chưa phê duyệt chỉ vì người đó cũng chưa có chứng chỉ.

Vì sao chọn cách này? Kéo theo là cách diễn đạt quy định kiểu '**nếu p thì q**'. Quy định này **chỉ bị vi phạm** khi điều kiện đầu đã xảy ra (`p=True`: được phê duyệt) nhưng yêu cầu sau **không được đáp ứng** (`q=False`: thiếu chứng chỉ). Biểu thức `(not p) or q` diễn tả chính xác điều đó: `not` đảo đúng thành sai và ngược lại; `or` chỉ cần một vế đúng. Người chưa được phê duyệt chưa phải đối tượng bắt buộc của quy định này nên không tự động vi phạm, dù chưa có chứng chỉ. Nếu thay bằng `p and q`, chương trình sẽ đánh dấu sai cả nhóm chưa phê duyệt.

Quy trình:

1. Lưu 15 cặp Boolean đúng theo ID.
2. Tính điều kiện tuân thủ bằng `(not p) or q`.
3. Liệt kê phản ví dụ và số hồ sơ tuân thủ.

No Note — Không chú thích

```
approved = [True,False,True,True,False,True,False,True,True,False,True,False,True,True,True]
certified = [True,False,False,True,False,True,True,False,True,True,True,False,True,False,True]
compliant = []
for approved_status, certificate_status in zip(approved, certified):
    compliant.append((not approved_status) or certificate_status)
violations = []
for i, ok in enumerate(compliant, 1):
    if not ok:
        violations.append(f"R{i:02d}")
if len(compliant) != 15:
    raise ValueError("Cần kiểm tra đủ 15 hồ sơ.")
print("Hồ sơ:", len(compliant))
print("Tuân thủ:", sum(compliant))
print("Vi phạm:", ", ".join(violations))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm 15 hồ sơ tuân thủ mệnh đề 'nếu duyệt thì phải có chứng nhận'.
# Dữ liệu: 15 cờ approved/certified theo cùng vị trí, quy tắc p→q chỉ sai khi p=True và q=False.
# 15 cờ hồ sơ đã duyệt; True nghĩa điều kiện tiền đề p đúng.
approved = [True,False,True,True,False,True,False,True,True,False,True,False,True,True,True]
# 15 cờ có chứng nhận tương ứng từng hồ sơ; True nghĩa hệ quả q đúng.
certified = [True,False,False,True,False,True,True,False,True,True,True,False,True,False,True]
```

```
compliant = []
# zip() ghép cờ duyệt và chứng nhận của chính một hồ sơ theo vị trí.
for approved_status, certificate_status in zip(approved, certified):
    # Quy tắc p→q: khi không duyệt thì không vi phạm; nếu đã duyệt phải có chứng nhận.
    # append() lưu Boolean tuân thủ của mỗi hồ sơ theo thứ tự.
    compliant.append((not approved_status) or certificate_status)
violations = []
# enumerate(...,1) đánh số hồ sơ R01..R15 (không dùng chỉ số 0 làm mã).
for i, ok in enumerate(compliant, 1):
    # not tìm những kết quả tuân thủ bằng False.
    if not ok:
        # append() lưu mã vi phạm; f-string :02d tạo số thứ tự hai chữ số.
        violations.append(f"R{i:02d}")
if len(compliant) != 15:
    # ValueError dừng khi số lượng hoặc quy tắc logic không nhất quán với dữ liệu đầu vào.
    raise ValueError("Cần kiểm tra đủ 15 hồ sơ.")
print("Hồ sơ:", len(compliant))
# sum() đếm số True (tuân thủ) trong list Boolean.
print("Tuân thủ:", sum(compliant))
# join() ghép mã vi phạm bằng dấu phẩy và khoảng trắng.
print("Vi phạm:", ", ".join(violations))
```

Kết quả khi chạy:

```
Hồ sơ: 15
Tuân thủ: 12
Vi phạm: R03, R08, R14
```

Việc chưa phê duyệt khiến tiền đề sai; với một phép kéo theo, trường hợp đó không làm hỏng chính sách đang được kiểm tra.

Tình huống 4 — Cấp 4: Đếm 18 cấu hình sản phẩm và lọc ràng buộc

Từ vựng trong mục: Cartesian product — tích Descartes; **Counting rule** — quy tắc nhân; **NOT** — phủ định.

Tình huống thực tế: Nhà bán hàng thiết kế áo với **3 màu** (Đỏ, Xanh, Trắng), **3 cỡ** (S, M, L) và **2 chất liệu** (Cotton, Poly). Một cấu hình áo là một lựa chọn từ **mỗi nhóm**, ví dụ Đỏ–M–Cotton. Theo quy định minh họa, áo **màu Đỏ không được làm bằng Poly** ở bất kỳ cỡ nào. Người quản lý muốn liệt kê đủ **18 cấu hình ban đầu**, rồi đếm bao nhiêu cấu hình hợp lệ và bao nhiêu cấu hình phải loại.

Vì sao chọn cách này? Tổ hợp sản phẩm là cách chọn **một màu, một cỡ và một chất liệu** cùng lúc. `itertools.product` sinh lần lượt mọi bộ ba mà không bỏ sót: ba màu, ba cỡ và hai chất liệu tạo 18 cấu hình. Sau đó kiểm riêng từng cấu hình: chỉ loại khi **đồng thời** màu là Đỏ **và** chất liệu là Poly; cỡ S, M hay L không làm thay đổi quy tắc này. Dùng `not (...)` nghĩa là **giữ lại những bộ không rơi vào trường hợp bị cấm**. Không xóa tất cả áo Đỏ hoặc tất cả chất liệu Poly, vì nhiều cách phối khác vẫn hợp lệ.

Quy trình:

1. Tạo 18 tuple từ màu, cỡ, chất liệu.
2. Lọc những tuple có đồng thời Đỏ và Poly.
3. Đối chiếu số hợp lệ + số loại = 18 và in kết quả.

No Note — Không chú thích

```
from itertools import product
colors = ["Đỏ", "Xanh", "Trắng"]
```

```
sizes = ["S", "M", "L"]
materials = ["Cotton", "Poly"]
options = list(product(colors, sizes, materials))
valid = []
for item in options:
    if not (item[0] == "Đỏ" and item[2] == "Poly"):
        valid.append(item)
if len(options) != 18 or len(colors) * len(sizes) * len(materials) != 18:
    raise ValueError("Cần liệt kê đúng 18 tổ hợp sản phẩm.")
print("Cấu hình:", len(options))
print("Hợp lệ:", len(valid))
print("Loại:", len(options) - len(valid))
print("Ví dụ đầu:", " - ".join(valid[0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Sinh 18 cấu hình màu/cỡ/chất liệu, loại mọi cấu hình Đỏ + Poly.
# Dữ liệu: 3 màu Đỏ/Xanh/Trắng × 3 cỡ S/M/L × 2 chất liệu Cotton/Poly = 18 cấu hình.
# itertools.product sinh tất cả tổ hợp giữa nhiều danh sách độc lập.
from itertools import product
# Ba màu là ba lựa chọn của thành phần thứ nhất.
colors = ["Đỏ", "Xanh", "Trắng"]
# Ba cỡ là ba lựa chọn thành phần thứ hai.
sizes = ["S", "M", "L"]
# Hai chất liệu tạo chiều lựa chọn thứ ba.
materials = ["Cotton", "Poly"]
# product() tạo 3×3×2=18 tuple (màu,cỡ,chất liệu), list() giữ toàn bộ.
options = list(product(colors, sizes, materials))
valid = []
# Duyệt từng tuple cấu hình.
for item in options:
    # [0] là màu, [2] chất liệu; and nhận tổ hợp Đỏ cùng Poly, not đảo để chỉ giữ tổ hợp KHÔNG bị
cấm.
    if not (item[0] == "Đỏ" and item[2] == "Poly"):
        # append() lưu tuple hợp lệ; không sửa list gốc options.
        valid.append(item)
# Nguyên lý nhân xác minh phải có 18 tổ hợp trước khi lọc.
if len(options) != 18 or len(colors) * len(sizes) * len(materials) != 18:
    # ValueError dừng khi số lượng hoặc quy tắc logic không nhất quán với dữ liệu đầu vào.
    raise ValueError("Cần liệt kê đúng 18 tổ hợp sản phẩm.")
print("Cấu hình:", len(options))
print("Hợp lệ:", len(valid))
print("Loại:", len(options) - len(valid))
# valid[0] là tuple cấu hình hợp lệ đầu; join() nối ba trường thành chuỗi dễ đọc.
print("Ví dụ đầu:", " - ".join(valid[0]))
```

Kết quả khi chạy:

```
Cấu hình: 18
Hợp lệ: 15
Loại: 3
Ví dụ đầu: Đỏ - S - Cotton
```

Tích Descartes liệt kê **toàn bộ cấu hình**; trong bài này ràng buộc giảm số cấu hình từ 18 còn 15, chứ không tạo thêm cấu hình mới.

Tình huống 5 — Cấp 5: Thử 20 yêu cầu truy cập với quy tắc nhiều điều kiện và ngoại lệ

Từ vựng trong mục: Compound logic — logic phối hợp; **Precedence** — độ ưu tiên; **Invariant** — điều kiện bất biến.

Tình huống thực tế: Cổng tài liệu nhận **20 yêu cầu truy cập giả lập**. Mỗi yêu cầu ghi tuổi (**năm**), có được người giám hộ đồng ý hay không, tài khoản đã xác minh hay chưa và tài khoản có đang bị khóa hay không. Quy tắc minh họa: người **từ 18 tuổi hoặc đã có sự đồng ý** được xét tiếp; sau đó bắt buộc phải **đã xác minh** và **không bị khóa**. Người quản lý cần đếm số yêu cầu được phép, bị từ chối và số tài khoản bị khóa, đồng thời xác nhận mọi yêu cầu đã khóa đều không thể được cho phép.

Vì sao chọn cách này? Quy tắc gồm **ba phần**, cần đặt ngoặc để người học nhìn thấy rõ: (1) **tuổi từ 18 hoặc có đồng ý** — chỉ cần một điều kiện; (2) **đã xác minh** — bắt buộc; (3) **không bị khóa** — bắt buộc. `or` kết nối hai cách đáp ứng phần (1), còn `and` kết nối kết quả ấy với hai phần bắt buộc. `not blocked` đổi cờ bị khóa thành điều kiện **không bị khóa**; vì thế yêu cầu đã khóa luôn không đủ điều kiện, kể cả người lớn đã xác minh. Chương trình kiểm tra lại từng yêu cầu bị khóa và **báo lỗi nếu có trường hợp được phép**, giúp phát hiện sai quy tắc. Đây chỉ là chính sách **mô phỏng** của bài tập, không mô tả luật hay điều kiện truy cập thật của dịch vụ.

Quy trình:

1. Tạo 20 bản ghi đầy đủ bốn thuộc tính.
2. Dùng một biểu thức Boolean có ngoặc cho mọi bản ghi.
3. Đếm kết quả, kiểm tra tổng 20 và khẳng định mọi yêu cầu bị khóa đều bị từ chối.

No Note — Không chú thích

```
requests = [{"age":16,"consent":False,"verified":True,"blocked":False},
{"age":17,"consent":True,"verified":True,"blocked":False},
{"age":18,"consent":True,"verified":False,"blocked":False},
{"age":19,"consent":True,"verified":True,"blocked":False},
{"age":20,"consent":True,"verified":True,"blocked":False},
{"age":21,"consent":False,"verified":True,"blocked":False},
{"age":22,"consent":True,"verified":False,"blocked":False},
{"age":23,"consent":True,"verified":True,"blocked":True},
{"age":16,"consent":True,"verified":True,"blocked":False},
{"age":17,"consent":True,"verified":True,"blocked":False},
{"age":18,"consent":False,"verified":False,"blocked":False},
{"age":19,"consent":True,"verified":True,"blocked":False},
{"age":20,"consent":True,"verified":True,"blocked":False},
{"age":21,"consent":True,"verified":True,"blocked":False},
{"age":22,"consent":True,"verified":False,"blocked":False},
{"age":23,"consent":False,"verified":True,"blocked":False},
{"age":16,"consent":True,"verified":True,"blocked":False},
{"age":17,"consent":True,"verified":True,"blocked":False},
{"age":18,"consent":True,"verified":False,"blocked":True},
{"age":19,"consent":True,"verified":True,"blocked":False}]
allowed = []
for item in requests:
    age_or_consent_ok = item["age"] >= 18 or item["consent"]
    can_access = age_or_consent_ok and item["verified"] and not item["blocked"]
    allowed.append(can_access)
if len(requests) != 20 or len(allowed) != 20:
    raise ValueError("Cần kiểm tra đủ 20 yêu cầu truy cập.")
for i, item in enumerate(requests):
    if item["blocked"] and allowed[i]:
        raise ValueError("Yêu cầu bị khóa không được phép truy cập.")
print("Yêu cầu:", len(requests))
print("Cho phép:", sum(allowed))
print("Từ chối:", len(allowed) - sum(allowed))
blocked_count = 0
for item in requests:
    if item["blocked"]:
```

```
        blocked_count += 1
print("Yêu câù bị khóa:", blocked_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm 20 yêu câù truy cập theo tuổi hoặc đồng ý, đồng thời đã xác minh và không bị khóa.
# Dữ liệu: 20 dict gồm age năm, consent/verified/blocked là cờ Boolean; quy tắc chỉ mang tính ví
dụ, không phải chính sách pháp luật.
# Mỗi dict là một yêu câù: tuổi (năm), cờ đồng ý, cờ xác minh và cờ bị khóa.
requests = [{"age":16,"consent":False,"verified":True,"blocked":False},
{"age":17,"consent":True,"verified":True,"blocked":False},
{"age":18,"consent":True,"verified":False,"blocked":False},
{"age":19,"consent":True,"verified":True,"blocked":False},
{"age":20,"consent":True,"verified":True,"blocked":False},
{"age":21,"consent":False,"verified":True,"blocked":False},
{"age":22,"consent":True,"verified":False,"blocked":False},
{"age":23,"consent":True,"verified":True,"blocked":True},
{"age":16,"consent":True,"verified":True,"blocked":False},
{"age":17,"consent":True,"verified":True,"blocked":False},
{"age":18,"consent":False,"verified":False,"blocked":False},
{"age":19,"consent":True,"verified":True,"blocked":False},
{"age":20,"consent":True,"verified":True,"blocked":False},
{"age":21,"consent":True,"verified":True,"blocked":False},
{"age":22,"consent":True,"verified":False,"blocked":False},
{"age":23,"consent":False,"verified":True,"blocked":False},
{"age":16,"consent":True,"verified":True,"blocked":False},
{"age":17,"consent":True,"verified":True,"blocked":False},
{"age":18,"consent":True,"verified":False,"blocked":True},
{"age":19,"consent":True,"verified":True,"blocked":False}]
# List kết quả giữ một giá trị cho phép True/False trên mỗi yêu câù.
allowed = []
# Duyệt từng dict, lấy trường qua item['tên_trường'].
for item in requests:
    # or chỉ đòi từ 18 tuổi HOẶC đã có đồng ý; đây là giả định bài học, không phải quy định truy
cập thực tế.
    age_or_consent_ok = item["age"] >= 18 or item["consent"]
    # and yêu câù đủ ba điều kiện: tuổi/đồng ý đạt, đã xác minh, không bị khóa; not đảo cờ blocked.
    can_access = age_or_consent_ok and item["verified"] and not item["blocked"]
    # append() lưu kết quả của đúng một yêu câù sau kiểm.
    allowed.append(can_access)
# Kiểm đủ 20 yêu câù và 20 kết quả tương ứng để không bỏ bản ghi.
if len(requests) != 20 or len(allowed) != 20:
    # ValueError dừng khi số lượng hoặc quy tắc logic không nhất quán với dữ liệu đầu vào.
    raise ValueError("Cần kiểm tra đủ 20 yêu câù truy cập.")
# enumerate() không có start mặc định đánh số từ 0 để cùng index với allowed[i].
for i, item in enumerate(requests):
    # Phát hiện lỗi logic nếu bất kỳ yêu câù đã khóa nào lại được cho phép.
    if item["blocked"] and allowed[i]:
        # ValueError dừng khi số lượng hoặc quy tắc logic không nhất quán với dữ liệu đầu vào.
        raise ValueError("Yêu câù bị khóa không được phép truy cập.")
print("Yêu câù:", len(requests))
# sum() trên Boolean đếm số True (cho phép); số từ chối = tổng - True.
print("Cho phép:", sum(allowed))
# sum() trên Boolean đếm số True (cho phép); số từ chối = tổng - True.
print("Từ chối:", len(allowed) - sum(allowed))
blocked_count = 0
# Duyệt từng dict, lấy trường qua item['tên_trường'].
for item in requests:
    if item["blocked"]:
        # += 1 đếm riêng các yêu câù có cờ bị khóa, không đồng nhất với mọi lý do từ chối.
        blocked_count += 1
print("Yêu câù bị khóa:", blocked_count)
```

Kết quả khi chạy:

```
Yêu cầu: 20
Cho phép: 13
Từ chối: 7
Yêu cầu bị khóa: 2
```

Từng yêu cầu là một hàng gồm nhiều mệnh đề, không phải một phần tử đơn. Quy tắc là ví dụ học thuật; hệ thống thực phải dựa trên chính sách xác thực và bảo vệ dữ liệu được phê duyệt.

6. Sai lầm thường gặp

- Không nhầm `and` (cả hai đúng) với `or` (ít nhất một đúng); `not` đảo giá trị Boolean.
- Phép kéo theo `p ⇒ q` chỉ sai khi `p=True`, `q=False`; tiền đề sai không tự chứng minh kết luận đúng.
- Bảng chân trị hai biến có bốn tổ hợp; nếu thiếu một tổ hợp, kiểm tra logic có thể bỏ sót lỗi.
- Nguyên lý **nhân** đếm lựa chọn theo nhiều bước kết hợp được với nhau; nguyên lý **cộng** đếm các phương án thay thế không trùng. Nếu trùng phải điều chỉnh số đếm.
- Set không bảo đảm thứ tự in, còn kết quả `itertools.product` gồm các tuple theo thứ tự nguồn đã đưa vào.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a proposition? | Mệnh đề là gì?

A. A table of all input truth-value combinations | Bảng các tổ hợp giá trị đúng/sai đầu vào

B. The logical conditional 'if p then q' | Điều kiện logic 'nếu p thì q'

C. A statement that is either true or false | Phát biểu có thể xác định đúng hoặc sai

D. Multiply choice counts across sequential stages when each earlier choice permits the stated number of later choices | Nhân số cách chọn qua các bước khi mỗi lựa chọn trước cho phép đúng số cách ở bước sau

Đáp án: C

Giải thích: Mệnh đề có một giá trị đúng/sai xác định. Ví dụ `2 + 2 = 4` là mệnh đề đúng, còn câu hỏi như `Mấy giờ rồi?` không tự khẳng định một giá trị đúng hay sai.

Q2. What is a truth table? | Bảng chân trị là gì?

A. Multiply the numbers of sequential independent choices | Nhân số lựa chọn qua các bước kết hợp đầy đủ

B. A statement that is either true or false | Phát biểu có thể xác định đúng hoặc sai

C. A table of all input truth-value combinations | Bảng các tổ hợp giá trị đúng/sai đầu vào

D. The logical conditional 'if p then q' | Điều kiện logic 'nếu p thì q'

Đáp án: C

Giải thích: Bảng chân trị liệt kê các khả năng Boolean. Với hai mệnh đề p và q, bốn hàng là (T,T), (T,F), (F,T), (F,F); bảng xét mọi cách gán giá trị.

Q3. What is logical implication $p \Rightarrow q$? | Phép kéo theo $p \Rightarrow q$ là gì?

A. The logical conditional 'if p then q' | Điều kiện logic 'nếu p thì q'

B. A statement that is either true or false | Phát biểu có thể xác định đúng hoặc sai

C. Multiply the numbers of sequential independent choices | Nhân số lựa chọn qua các bước kết hợp đầy đủ

D. A table of all input truth-value combinations | Bảng các tổ hợp giá trị đúng/sai đầu vào

Đáp án: A

Giải thích: Kéo theo chỉ sai khi tiền đề đúng, kết luận sai. Phép kéo theo đòi khi tiền đề p đúng thì kết luận q không được sai; khi p sai, kéo theo vật chất vẫn đúng bất kể q.

Q4. What is the multiplication counting principle? | Nguyên lý nhân trong đếm là gì?

A. A statement that is either true or false | Phát biểu có thể xác định đúng hoặc sai

B. A table of all input truth-value combinations | Bảng các tổ hợp giá trị đúng/sai đầu vào

C. The logical conditional 'if p then q' | Điều kiện logic 'nếu p thì q'

D. Multiply stage-choice counts when each earlier choice permits the same stated number of valid continuations | Nhân số cách chọn mỗi bước khi mọi lựa chọn trước đều cho phép cùng số cách đi tiếp hợp lệ

Đáp án: D

Giải thích: Nếu bước đầu có m cách và mỗi cách có đúng n lựa chọn nối tiếp thì có m×n kết quả hoàn chỉnh. Quy tắc đếm theo từng bước này không đòi các biến cố độc lập về xác suất. Nếu có hai cách chọn áo và mỗi áo đều ghép được với ba quần, tổng cộng $2 \times 3 = 6$ bộ; không cần giả thiết độc lập xác suất.

Q5. When is $p \text{ and } q$ true? | Khi nào $p \text{ and } q$ đúng?

A. Reverses the truth value of p | Đảo giá trị đúng/sai của p

B. When p is true and q is false | Khi p đúng và q sai

C. When at least one of p or q is true | Khi có ít nhất một trong p hoặc q đúng

D. When both p and q are true | Khi p và q đều đúng

Đáp án: D

Giải thích: AND cần cả hai điều kiện đúng. Nếu p đúng nhưng q sai thì $p \text{ and } q$ sai; một điều kiện đúng chưa đủ cho phép AND.

Q6. When is $p \text{ or } q$ true? | Khi nào $p \text{ or } q$ đúng?

A. Reverses the truth value of p | Đảo giá trị đúng/sai của p

B. When p is true and q is false | Khi p đúng và q sai

C. When at least one of p or q is true | Khi có ít nhất một trong p hoặc q đúng

D. When both p and q are true | Khi p và q đều đúng

Đáp án: C

Giải thích: OR bao hàm chấp nhận một hoặc cả hai điều kiện đúng. Đây là OR bao hàm: p và q có thể cùng đúng, khác XOR chỉ đúng khi đúng một vế.

Q7. What does `not p` do? | `not p` làm gì?

A. When p is true and q is false | Khi p đúng và q sai

B. Reverses the truth value of p | Đảo giá trị đúng/sai của p

C. When both p and q are true | Khi p và q đều đúng

D. When at least one of p or q is true | Khi có ít nhất một trong p hoặc q đúng

Đáp án: B

Giải thích: NOT phủ định giá trị Boolean. Nếu p=True thì `not p=False`; nếu p=False thì phủ định trở thành True.

Q8. When is `p ⇒ q` false? | Khi nào `p ⇒ q` sai?

A. When both p and q are true | Khi p và q đều đúng

B. When at least one of p or q is true | Khi có ít nhất một trong p hoặc q đúng

C. When p is true and q is false | Khi p đúng và q sai

D. Reverses the truth value of p | Đảo giá trị đúng/sai của p

Đáp án: C

Giải thích: Tiền đề đúng nhưng kết luận sai làm phép kéo theo sai. Với p=True và q=False, `(not p) or q` thành `(False or False)`, nên phép kéo theo sai.

Q9. How many rows does a truth table with two variables have? | Bảng chân trị hai biến có mấy hàng?

A. 8 | 8

B. 16 | 16

C. 2 | 2

D. 4 | 4

Đáp án: D

Giải thích: Hai biến có hai trạng thái mỗi biến, tạo 2^2 trường hợp. Mỗi biến Boolean có hai giá trị, tạo $2 \times 2 = 4$ tổ hợp chứ không chỉ hai tổ hợp.

Q10. How many rows does a truth table with three variables have? | Bảng chân trị ba biến có mấy hàng?

A. 2 | 2

B. 4 | 4

C. 16 | 16

D. 8 | 8

Đáp án: D

Giải thích: Ba lựa chọn nhị phân tạo 2^3 tổ hợp. Ba biến Boolean tạo `2 × 2 × 2 = 8` cách gán đầu vào cho bảng chân trị.

Q11. How many rows does a truth table with one variable have? | Bảng chân trị một biến có mấy hàng?

A. 4 | 4

B. 8 | 8

C. 2 | 2

D. 16 | 16

Đáp án: C

Giải thích: Một biến Boolean có True và False. Hai hàng ứng với biến duy nhất nhận True và False; bảng có `2^1 = 2` hàng.

Q12. How many rows does a truth table with four variables have? | Bảng chân trị bốn biến có mấy hàng?

A. 16 | 16

B. 4 | 4

C. 2 | 2

D. 8 | 8

Đáp án: A

Giải thích: Bốn biến Boolean tạo 2^4 khả năng. Mỗi biến Boolean thêm vào nhân đôi số hàng, nên bốn biến cần `2^4 = 16` tổ hợp.

Q13. Which expression is equivalent to implication `p ⇒ q`? | Biểu thức nào tương đương `p ⇒ q`?

A. `False` | `False`

B. `(not p) or q` | `(not p) or q`

C. `(True, False)` | `(True, False)`

D. `True` | `True`

Đáp án: B

Giải thích: Biểu thức chỉ sai khi p=True, q=False. Cả `p ⇒ q` và `(not p) or q` chỉ sai khi p=True, q=False nên bảng chân trị của chúng trùng nhau.

Q14. Which input pair makes `p ⇒ q` false? | Cặp p, q nào làm `p ⇒ q` sai?

A. `(True, False)` | `(True, False)`

B. `(not p) or q` | `(not p) or q`

C. `True` | `True`

D. `False` | `False`

Đáp án: A

Giải thích: Tiền đề đúng và kết luận sai là trường hợp duy nhất làm phép kéo theo sai. Thứ tự cặp là `(p, q)`, nên `(True, False)` có tiền đề đúng và kết luận sai như yêu cầu.

Q15. What is `True and False`? | `True and False` bằng gì?

A. `True` | `True`

B. `False` | `False`

C. `(not p) or q` | `(not p) or q`

D. `(True, False)` | `(True, False)`

Đáp án: B

Giải thích: AND sai nếu có một vế sai. Kết quả là Boolean `False`, không phải số nguyên 0; vế thứ hai sai trong khi AND cần cả hai đúng.

Q16. What is `False or True`? | `False or True` bằng gì?

A. `True` | `True`

B. `(True, False)` | `(True, False)`

C. `False` | `False`

D. `(not p) or q` | `(not p) or q`

Đáp án: A

Giải thích: OR đúng khi có một vế đúng. Vì có một vế đúng, OR bao hàm trả Boolean `True`; `False and True` thì lại sai.

Q17. When does the addition counting principle apply directly? | Khi nào có thể áp dụng trực tiếp nguyên lý cộng?

A. All ordered pairs taking one element from each input | Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

B. When alternatives are disjoint | Khi các phương án thay thế không trùng nhau

C. When some outcomes belong to more than one counted group | Khi một kết quả nằm trong nhiều nhóm được đếm

D. When each step has the stated choices for every earlier selection | Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

Đáp án: B

Giải thích: Các nhóm không trùng có thể cộng mà không đếm lặp. Nếu năm kết quả chỉ thuộc nhóm A và ba kết quả chỉ thuộc nhóm B, tổng có `5 + 3 = 8` phương án không đếm trùng.

Q18. When can choices be counted by multiplying options? | Khi nào có thể đếm bằng phép nhân?

A. When each step has the stated choices for every earlier selection | Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

B. When alternatives are disjoint | Khi các phương án thay thế không trùng nhau

C. All ordered pairs taking one element from each input | Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

D. When some outcomes belong to more than one counted group | Khi một kết quả nằm trong nhiều nhóm được đếm

Đáp án: A

Giải thích: Nguyên lý nhân yêu cầu số nhánh lựa chọn phù hợp ở từng bước. Nếu mỗi lựa chọn bước đầu đều có đúng ba cách đi tiếp thì nhân số cách mỗi bước; khi số nhánh thay đổi phải cộng số cách theo từng nhánh cẩn thận.

Q19. What does `itertools.product(A, B)` generate? | `itertools.product(A, B)` tạo gì?

A. When each step has the stated choices for every earlier selection | Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

B. All ordered pairs taking one element from each input | Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

C. When alternatives are disjoint | Khi các phương án thay thế không trùng nhau

D. When some outcomes belong to more than one counted group | Khi một kết quả nằm trong nhiều nhóm được đếm

Đáp án: B

Giải thích: product liệt kê tích Descartes. Với `A=[1,2]` và `B=['x','y']`, product tạo `(1,'x')`, `(1,'y')`, `(2,'x')`, `(2,'y')`, không chỉ ghép vị trí tương ứng.

Q20. When should overlapping alternatives be adjusted during counting? | Khi nào cần điều chỉnh phần giao khi đếm?

A. All ordered pairs taking one element from each input | Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

B. When each step has the stated choices for every earlier selection | Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

C. When alternatives are disjoint | Khi các phương án thay thế không trùng nhau

D. When some outcomes belong to more than one counted group | Khi một kết quả nằm trong nhiều nhóm được đếm

Đáp án: D

Giải thích: Nhóm có giao nhau không được đếm trùng. Với hai nhóm có giao, công thức `|A ∪ B| = |A| + |B| − |A ∩ B|` trừ phần được tính hai lần.

8. Nối sang bài tiếp

Boolean → logic mệnh đề → tổ hợp → xác suất.

PY-21 — Tập hợp, bao hàm–loại trừ và phân hoạch

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Union — phép hợp; **Intersection** — phép giao.

`Set → hợp/giao/hiệu → bao hàm–loại trừ → train/test`

Học xong dùng để làm gì? Đếm số người thuộc nhiều nhóm mà không tính trùng, phân hoạch đối tượng thành các tập không giao nhau.

Kết nối kiến thức tiếp theo: PY-22 đếm số cách chọn nhóm và xếp thứ tự từ tập phần tử.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Union	phép hợp	Lấy tất cả phần tử thuộc ít nhất một tập
Intersection	phép giao	Lấy phần tử có trong cả hai tập
Inclusion–exclusion	bao hàm–loại trừ	Cộng từng tập rồi trừ phần giao bị đếm hai lần
Partition	phân hoạch	Chia thành các nhóm rời nhau phủ đủ tập gốc

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`a | b, a & b`	Hợp và giao các tập hợp
`a - b, a ^ b`	Hiệu và hiệu đối xứng
`len(a | b)`	Đếm phần tử khác nhau
`a.isdisjoint(b)`	Kiểm tra hai nhóm có trùng phần tử không

4. Khái niệm liên quan

Tập hợp giúp lưu các đối tượng **khác nhau** rồi so sánh thành viên giữa các nhóm. Ta dùng hai lớp học Python và AI làm ví dụ xuyên suốt: phép hợp tìm tất cả học viên, phép giao tìm người học cả hai, quy tắc bao hàm–loại trừ giúp đếm không trùng, còn phân hoạch giúp chia bộ dữ liệu thành các phần rời nhau.

4.1. Set và phép hợp — Ai tham gia ít nhất một nhóm?

Từ vựng trong mục: Set — tập hợp; **Union** — phép hợp.

Set không giữ phần tử trùng. Lớp Python gồm SV1, SV2; lớp AI gồm SV2, SV3. Phép **hợp** `python | ai` chứa tất cả người có mặt trong **ít nhất một** lớp. SV2 nằm ở cả hai nhưng chỉ xuất hiện một lần. Khi cần đầu ra ổn định, dùng `sorted()` vì Set không bảo đảm thứ tự hiển thị.

No Note — Không chú thích

```
python = {'SV1', 'SV2'}
ai = {'SV2', 'SV3'}
print(sorted(python | ai))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm danh sách sinh viên thuộc ít nhất một trong hai nhóm Python/AI bằng phép hợp tập
hợp.
# Dữ liệu: Mã SV1, SV2 thuộc Python; SV2, SV3 thuộc AI; SV2 có mặt ở cả hai nhưng chỉ đếm một lần.
# Tập set lưu mã không trùng; dấu ngoặc nhọn tạo tập hợp sinh viên Python.
python = {'SV1', 'SV2'}
# Tập AI chứa SV2 chung với Python và SV3 riêng.
ai = {'SV2', 'SV3'}
# | là hợp hai set, lấy người thuộc ít nhất một nhóm và bỏ SV2 trùng; sorted() xếp mã để in ổn định
vì set không có thứ tự bảo đảm.
print(sorted(python | ai))
```

Kết quả khi chạy:

```
['SV1', 'SV2', 'SV3']
```

Ba mã học viên khác nhau thuộc ít nhất một lớp; `sorted` tạo List để hiển thị theo thứ tự rõ ràng.

4.2. Intersection và Difference — Phần chung và phần riêng

Từ vựng trong mục: Intersection — phép giao; **Difference** — hiệu tập hợp.

Giao `a & b` chỉ lấy phần tử thuộc cả hai nhóm; với lớp Python {SV1,SV2} và AI {SV2,SV3}, phần chung là SV2. **Hiệu** `a - b` chỉ lấy phần tử thuộc a nhưng không thuộc b, tức SV1 trong ví dụ. Hiệu khác hiệu đối xứng `a ^ b`, vốn chứa các phần tử chỉ có ở đúng một nhóm.

No Note — Không chú thích

```
a = {'SV1', 'SV2'}
b = {'SV2', 'SV3'}
print(sorted(a & b), sorted(a - b))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách mã học viên thuộc cả hai nhóm và chỉ thuộc nhóm thứ nhất bằng phép giao/hiệu.
# Dữ liệu: a={SV1,SV2}, b={SV2,SV3}; SV2 là phần chung, SV1 chỉ ở a.
# set không lưu phần tử trùng; a là nhóm đầu.
a = {'SV1', 'SV2'}
# b là nhóm sau, có một thành viên chung với a.
b = {'SV2', 'SV3'}
# & lấy giao: mã xuất hiện trong CẢ HAI set; - lấy hiệu a-b: chỉ mã ở a; sorted() giữ thứ tự xuất
ổn định.
print(sorted(a & b), sorted(a - b))
```

Kết quả khi chạy:

```
['SV2'] ['SV1']
```

SV2 học cả hai lớp, còn SV1 chỉ học Python. Hai phép toán trả lời hai câu hỏi khác nhau.

4.3. Inclusion–exclusion — Đếm không lặp người

Từ vựng trong mục: Inclusion–exclusion — bao hàm–loại trừ; **Cardinality** — số phần tử.

Nếu cộng số học viên hai lớp trực tiếp, người học cả hai bị đếm **hai lần**. Ví dụ lớp Python có 2 người, lớp AI có 2 người, một người chung: số người khác nhau bằng `2 + 2 - 1 = 3`. Công thức `|A ∪ B| = |A| + |B| - |A ∩ B|` trừ phần bị đếm lặp. Với hơn hai tập, cần tính thêm các giao nhiều tập.

No Note — Không chú thích

```
a = {'SV1', 'SV2'}
b = {'SV2', 'SV3'}
print(len(a) + len(b) - len(a & b))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm học viên duy nhất của hai nhóm bằng nguyên lý cộng trừ phần giao.
# Dữ liệu: Hai tập hai người, có SV2 lặp trong cả hai.
# Mỗi set đếm một người tối đa một lần.
a = {'SV1', 'SV2'}
# SV2 ở cả hai nên nếu cộng hai số lượng sẽ bị đếm hai lần.
b = {'SV2', 'SV3'}
# len() đếm từng set; trừ len(a & b) vì người thuộc cả hai đã bị cộng hai lần; kết quả là số người
duy nhất 3.
print(len(a) + len(b) - len(a & b))
```

Kết quả khi chạy:

```
3
```

Kết quả đếm ba học viên duy nhất, không coi SV2 là hai người khác nhau.

4.4. Partition — Chia đủ dữ liệu thành phần rời

Từ vựng trong mục: Partition — phân hoạch; **Disjoint** — không giao nhau.

Phân hoạch tập dữ liệu gốc thành các nhóm **không rỗng**, **không giao nhau**, và **hợp lại bằng đúng tập gốc**. Ví dụ toàn bộ mã `{1,2,3,4,5}`, train `{1,2,3}`, test `{4,5}`. `isdisjoint` chỉ kiểm tra không trùng; để chứng minh phân hoạch còn phải kiểm tra hợp của chúng có phủ đủ dữ liệu gốc. Với Machine Learning, không trùng ID **chưa chắc** loại bỏ mọi hình thức rò rỉ dữ liệu.

No Note — Không chú thích

```
all_ids = {1, 2, 3, 4, 5}
train = {1, 2, 3}
test = {4, 5}
print(bool(train) and bool(test) and train.isdisjoint(test) and train | test == all_ids)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xác minh Train/Test chia đủ năm mã nguồn, mỗi nhóm có dữ liệu và không chồng lấn.
# Dữ liệu: all_ids là năm mẫu nguồn; train có ba mẫu, test hai mẫu.
# Tập tất cả mã là tập nguồn cần được chia không sót.
all_ids = {1, 2, 3, 4, 5}
# Tập mẫu dùng để học mô hình.
train = {1, 2, 3}
# Tập mẫu dành kiểm tra, phải không trùng train.
test = {4, 5}
# Tập mẫu dành kiểm tra, phải không trùng train.
# bool(set) là False khi set rỗng; hai bool đảm bảo cả train và test không rỗng.
# isdisjoint(test) trả True nếu giao hai tập RỖNG, tránh dùng lại mẫu train trong test.
# | hợp hai tập; == so với nguồn để xác nhận bao phủ đủ và không có mã lạ.
print(bool(train) and bool(test) and train.isdisjoint(test) and train | test == all_ids)
```

Kết quả khi chạy:

```
True
```

Hai tập đều có dữ liệu, không trùng và phủ đủ toàn bộ năm mã, nên tạo một phân hoạch hợp lệ.

5. Năm tình huống Set — từ đếm duy nhất đến kiểm tra phân hoạch dữ liệu

Tình huống 1 — Cấp 1: Đếm học viên duy nhất từ 10 lượt đăng ký

Từ vựng trong mục: Set — tập hợp; **Union** — phép hợp.

Tình huống thực tế: Trung tâm ngoại ngữ nhận **10 lượt đăng ký môn học**. Mỗi lượt là một cặp gồm **mã học viên** (SV01 đến SV08) và môn **Python hoặc AI**. SV01 và SV05 đều đăng ký cả hai môn nên một người có thể tạo **hai lượt đăng ký**. Giáo vụ cần biết có bao nhiêu **người khác nhau** đã đăng ký ít nhất một môn, đồng thời liệt kê mã những người học cả hai, tránh nhầm số dòng đăng ký với số học viên.

Vì sao chọn cách này? Set (tập hợp) là kiểu dữ liệu chỉ giữ **mỗi mã duy nhất một lần**, dù mã ấy xuất hiện ở nhiều lượt đăng ký. Chương trình duyệt 10 dòng rồi thêm mã vào `python_students` hoặc `ai_students` theo môn. Phép **hợp** `|` cho tất cả mã có ở ít nhất một tập, còn phép **giao** `&` giữ những mã có ở **cả hai tập**. `sorted()` sắp mã trước khi in vì set không đảm bảo một thứ tự hiển thị cố định. Không dùng `len(registrations)` để đếm người, vì SV01 và SV05 mỗi người được ghi hai lần.

Quy trình:

1. Ghi 10 bản ghi, mỗi bản ghi là một lượt đăng ký.
2. Tạo tập mã học viên Python và AI.
3. Lấy hợp/giao, đối chiếu số người và số lượt.

No Note — Không chú thích

```
registrations = [["SV01","Python"],["SV02","AI"],["SV03","Python"],["SV01","AI"],["SV04","AI"],
["SV05","Python"],["SV06","AI"],["SV05","AI"],["SV07","Python"],["SV08","AI"]]
python_students = set()
ai_students = set()
for sid, subject in registrations:
    if subject == "Python":
```

```
        python_students.add(sid)
    if subject == "AI":
        ai_students.add(sid)
if len(registrations) != 10:
    raise ValueError("Cần đủ 10 lượt đăng ký.")
print("Lượt đăng ký:", len(registrations))
print("Học viên duy nhất:", len(python_students | ai_students))
print("Học cả hai:", ", ".join(sorted(python_students & ai_students)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Từ 10 lượt đăng ký Python/AI, đếm học viên duy nhất và người học cả hai.
# Dữ liệu: 10 cặp (mã SV, tên môn); một người có thể xuất hiện hai lượt ở hai môn.
# Mỗi list con là một lượt (student_id, subject), KHÔNG phải mỗi hàng một học viên duy nhất.
registrations = [["SV01","Python"],["SV02","AI"],["SV03","Python"],["SV01","AI"],["SV04","AI"],
["SV05","Python"],["SV06","AI"],["SV05","AI"],["SV07","Python"],["SV08","AI"]]
# set() tạo tập rỗng, dùng để tự bỏ những mã lặp khi thu thập đăng ký.
python_students = set()
# set() tạo tập rỗng, dùng để tự bỏ những mã lặp khi thu thập đăng ký.
ai_students = set()
# Duyệt từng lượt đăng ký và tách mã học viên/môn của lượt đó.
for sid, subject in registrations:
    if subject == "Python":
        # add() thêm mã vào tập Python; nếu lặp cùng mã cũng chỉ lưu một lần.
        python_students.add(sid)
    if subject == "AI":
        # add() thêm mã vào tập AI, giữ một bản sao mỗi người.
        ai_students.add(sid)
# len() xác nhận có đúng 10 LƯỢT đăng ký, không phải 10 người.
if len(registrations) != 10:
    # ValueError dùng khi dữ liệu hoặc điều kiện đối chiếu không hợp lệ.
    raise ValueError("Cần đủ 10 lượt đăng ký.")
print("Lượt đăng ký:", len(registrations))
# | là hợp: người tham gia ít nhất một môn, bỏ trùng hai môn.
print("Học viên duy nhất:", len(python_students | ai_students))
# & là giao: người học cả hai; sorted() sắp mã vì thứ tự set không cố định, join() ghép bằng ', '.
print("Học cả hai:", ", ".join(sorted(python_students & ai_students)))
```

Kết quả khi chạy:

```
Lượt đăng ký: 10
Học viên duy nhất: 8
Học cả hai: SV01, SV05
```

SV01 đăng ký hai lớp vẫn là **một người**. Đây là bài tập phép hợp/giao với dữ liệu theo hàng, không đếm trùng người.

Tình huống 2 — Cấp 2: Tách người chỉ học một môn qua 12 lượt

Từ vựng trong mục: Difference — hiệu tập hợp; **Symmetric difference** — hiệu đối xứng.

Tình huống thực tế: Quầy tư vấn nhận **12 lượt đăng ký học** từ các học viên H01 đến H10. Mỗi dòng chứa **mã người đăng ký** và một trong hai môn Python hoặc AI. H01 và H07 đăng ký cả hai môn; người chỉ có một môn cần được liên hệ để giới thiệu môn còn lại. Nhân viên muốn biết ai **chỉ học Python**, ai **chỉ học AI**, và có tất cả bao nhiêu người chỉ đăng ký đúng một môn.

Vì sao chọn cách này? Để tìm người **chỉ học Python**, lấy tập Python **trừ** tập AI bằng `pyt - ai`; chiều ngược lại `ai - pyt` cho người chỉ học AI. Đây là **phép hiệu tập hợp**, giữ mã có ở nhóm đầu nhưng không có nhóm thứ hai. Phép **hiệu đối xứng** ^ gom cả hai nhóm chỉ học một môn, đồng thời loại người học cả

hai. H01 và H07 không được đưa vào danh sách cần mời học môn còn lại vì họ đã đăng ký đủ hai môn. Cần chuẩn hóa theo **mã học viên**, không so hai lượt theo số thứ tự đăng ký.

Quy trình:

1. Chuyển lượt đăng ký thành tập mã theo từng môn.
2. Tính hiệu từng chiều để biết học viên cần tư vấn môn nào.
3. Tính hiệu đối xứng, đối chiếu tổng hai nhóm.

No Note — Không chú thích

```
registrations = [["H01","AI"],["H02","Python"],["H03","AI"],["H04","Python"],["H05","AI"],
["H01","Python"],["H06","Python"],["H07","AI"],["H08","Python"],["H09","AI"],["H07","Python"],
["H10","Python"]]
pyt = set()
ai = set()
for sid, sub in registrations:
    if sub == "Python":
        pyt.add(sid)
    if sub == "AI":
        ai.add(sid)
only_python = sorted(pyt - ai)
only_ai = sorted(ai - pyt)
only_one = pyt ^ ai
if len(registrations) != 12:
    raise ValueError("Cần đủ 12 lượt đăng ký.")
print("Lượt:", len(registrations))
print("Chỉ Python:", ", ".join(only_python))
print("Chỉ AI:", ", ".join(only_ai))
print("Chỉ một môn:", len(only_one))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách học viên chỉ học Python, chỉ học AI và học đúng một môn trong 12 lượt.
# Dữ liệu: 12 lượt môn Python/AI có người đăng ký hai lần; H01 và H07 học hai môn.
# 12 cặp mã Hxx và môn; một học viên có thể được ghi nhiều lần nếu học hai môn.
registrations = [["H01","AI"],["H02","Python"],["H03","AI"],["H04","Python"],["H05","AI"],
["H01","Python"],["H06","Python"],["H07","AI"],["H08","Python"],["H09","AI"],["H07","Python"],
["H10","Python"]]
# set() khởi tạo tập mã Python không trùng.
pyt = set()
# set() khởi tạo tập mã AI không trùng.
ai = set()
# Duyệt từng cặp và dùng môn để chọn tập cần cập nhật.
for sid, sub in registrations:
    if sub == "Python":
        # add() ghi mã Python duy nhất, không làm tăng tập nếu đã có.
        pyt.add(sid)
    if sub == "AI":
        # add() ghi mã AI duy nhất.
        ai.add(sid)
# pyt - ai lấy người CHỈ Python; sorted() xếp mã để in dễ đối chiếu.
only_python = sorted(pyt - ai)
# ai - pyt lấy người CHỈ AI, không có trong Python.
only_ai = sorted(ai - pyt)
# ^ là hiệu đối xứng: thuộc ĐÚNG MỘT trong hai tập, không thuộc giao; khác | là hợp.
only_one = pyt ^ ai
# len() kiểm đủ số LƯỢT nguồn, không đếm số người.
if len(registrations) != 12:
    # ValueError dùng khi dữ liệu hoặc điều kiện đối chiếu không hợp lệ.
    raise ValueError("Cần đủ 12 lượt đăng ký.")
print("Lượt:", len(registrations))
```

```
# join() ghép các mã chỉ Python bằng dấu phẩy + khoảng trắng.
print("Chỉ Python:", ", ".join(only_python))
# join() ghép các mã chỉ AI thành chuỗi báo cáo.
print("Chỉ AI:", ", ".join(only_ai))
# len() đếm số người chỉ học một trong hai môn.
print("Chỉ một môn:", len(only_one))
```

Kết quả khi chạy:

```
Lượt: 12
Chỉ Python: H02, H04, H06, H08, H10
Chỉ AI: H03, H05, H09
Chỉ một môn: 8
```

Một người có thể thuộc hai Set cùng lúc, nên không được dùng tổng kích thước hai Set để suy ra số người cần tư vấn một môn.

Tình huống 3 — Cấp 3: Đối chiếu 15 lượt truy cập bằng bao hàm–loại trừ

Từ vựng trong mục: Inclusion–exclusion — bao hàm–loại trừ; **Intersection** — phần chung.

Tình huống thực tế: Ứng dụng dịch vụ ghi **15 lượt truy cập** của khách K01 đến K12. Mỗi dòng cho biết mã khách và **kênh truy cập** là `app` (ứng dụng) hoặc `web` (trang web). K01, K03 và K12 đã dùng cả hai kênh, nên đếm số khách trên từng kênh rồi cộng thẳng sẽ bị lặp. Bộ phận kinh doanh cần biết số **khách duy nhất** đã dùng ít nhất một kênh, số khách dùng cả hai và xác nhận phép đếm không bị tính trùng.

Vì sao chọn cách này? Mỗi kênh được lưu thành một set mã khách để việc khách ghé nhiều lần không làm số người tăng lên. Khách K01 có trong **cả `app` và `web`**, vì vậy phần giao `app & web` nhận biết người bị đếm hai lần nếu cộng số khách của hai kênh. Quy tắc **bao hàm–loại trừ** diễn giải bằng lời là: cộng người của hai kênh, rồi **trừ một lần số người thuộc cả hai**. Chương trình cũng kiểm tra kết quả bằng `len(app | web)`, tức số mã khác nhau xuất hiện trên ít nhất một kênh. Đây là cách đếm **khách duy nhất**, không phải số lượt truy cập.

Quy trình:

1. Tạo hai Set khách hàng dựa trên 15 lượt ghi.
2. Đếm khách riêng từng kênh và khách dùng cả hai.
3. Tính số khách duy nhất bằng quy tắc bao hàm–loại trừ, đối chiếu phép hợp.

No Note — Không chú thích

```
visits = [["K01","app"],["K02","web"],["K01","web"],["K03","app"],["K04","web"],["K05","app"],
["K06","web"],["K07","app"],["K03","web"],["K08","app"],["K09","web"],["K10","app"],["K11","web"],
["K12","app"],["K12","web"]]
app = set()
web = set()
for sid, channel in visits:
    if channel == "app":
        app.add(sid)
    if channel == "web":
        web.add(sid)
via_count = len(app) + len(web) - len(app & web)
if len(visits) != 15 or via_count != len(app | web):
    raise ValueError("Cần đối chiếu đủ 15 lượt và số khách duy nhất.")
print("Lượt:", len(visits))
print("Dùng app:", len(app))
print("Dùng web:", len(web))
```

```
print("Cả hai:", len(app & web))
print("Khách duy nhất:", via_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm khách duy nhất ở 15 lượt ghé app/web bằng nguyên lý cộng trừ phần chung.
# Dữ liệu: 15 cặp (mã khách, kênh); khách ghé hai kênh phải được đếm đúng một lần.
# 15 lượt tiếp xúc, mỗi hàng gồm customer_id và channel app/web; không có nghĩa 15 khách.
visits = [["K01","app"],["K02","web"],["K01","web"],["K03","app"],["K04","web"],["K05","app"],
["K06","web"],["K07","app"],["K03","web"],["K08","app"],["K09","web"],["K10","app"],["K11","web"],
["K12","app"],["K12","web"]]
# Set chứa mỗi khách app một lần.
app = set()
# Set chứa mỗi khách web một lần.
web = set()
# Duyệt từng lượt và phân loại theo kênh.
for sid, channel in visits:
    if channel == "app":
        # Thêm mã khách vào nhóm app, tự loại lượt lặp.
        app.add(sid)
    if channel == "web":
        # Thêm mã khách vào nhóm web, tự loại lượt lặp.
        web.add(sid)
# Cộng lượng người của hai kênh rồi trừ giao (&) để không đếm hai lần người dùng cả hai.
# & là giao app và web, số người ghé cả hai.
via_count = len(app) + len(web) - len(app & web)
# | là hợp tạo tập tất cả khách, dùng đối chiếu với cách cộng-trừ.
# Kiểm đủ 15 lượt nguồn; or báo sai nếu số khách hợp không khớp.
if len(visits) != 15 or via_count != len(app | web):
    # ValueError dùng khi dữ liệu hoặc điều kiện đối chiếu không hợp lệ.
    raise ValueError("Cần đối chiếu đủ 15 lượt và số khách duy nhất.")
print("Lượt:", len(visits))
print("Dùng app:", len(app))
print("Dùng web:", len(web))
# & là giao app và web, số người ghé cả hai.
print("Cả hai:", len(app & web))
print("Khách duy nhất:", via_count)
```

Kết quả khi chạy:

```
Lượt: 15
Dùng app: 7
Dùng web: 8
Cả hai: 3
Khách duy nhất: 12
```

Số khách duy nhất khác số lượt truy cập, vì mỗi người có thể truy cập hơn một lần hoặc qua hai kênh.

Tình huống 4 — Cấp 4: Rà soát phân hoạch 18 mẫu train/test bị lặp ID

Từ vựng trong mục: Partition — phân hoạch; **Disjoint** — không giao nhau; **Coverage** — bao phủ.

Tình huống thực tế: Nhóm học máy có **18 mẫu dữ liệu D01 đến D18**, mỗi mã biểu thị một mẫu cần phân vào tập **train** (dùng để học) hoặc **test** (dùng để đánh giá). Trong danh sách được chuẩn bị sẵn, **D12 xuất hiện ở cả hai tập** và **D18 chưa được đưa vào tập nào**. Trước khi huấn luyện, nhóm cần báo rõ mã trùng, mã bị bỏ sót và xác nhận cách chia hiện tại có hợp lệ hay không, chứ không tự động sửa bằng cách xóa một mã.

Vì sao chọn cách này? Một cách chia train/test đúng phải thỏa **hai điều kiện cùng lúc**: không có mẫu nằm ở hai tập và không bỏ quên mẫu nào trong danh sách nguồn. `train & test` tìm mã **bị trùng**, còn `source - (train | test)` tìm mã **chưa thuộc nhóm nào**. `isdisjoint()` hỏi hai tập có hoàn toàn **không giao nhau** không; kết hợp thêm việc hợp hai tập phải bằng nguồn mới kiểm tra đủ. Chỉ thấy hai tập không trùng **chưa đủ**, vì vẫn có thể bỏ quên D18. Khi D12 nằm ở cả train lẫn test, kết quả đánh giá mô hình còn có nguy cơ bị lạc quan do gặp lại mẫu đã học.

Quy trình:

1. Tạo tập nguồn 18 ID; lưu train/test theo quyết định hiện tại.
2. Tìm ID trùng bằng phép giao và ID thiếu bằng hiệu nguồn với hợp.
3. In các mã cần sửa và kiểm tra phân hoạch đạt/không đạt.

No Note — Không chú thích

```
all_ids =
["D01","D02","D03","D04","D05","D06","D07","D08","D09","D10","D11","D12","D13","D14","D15","D16","D
17","D18"]
train = set(["D01","D02","D03","D04","D05","D06","D07","D08","D09","D10","D11","D12"])
test = set(["D12","D13","D14","D15","D16","D17"])
source = set(all_ids)
overlap = sorted(train & test)
missing = sorted(source - (train | test))
valid_partition = train.isdisjoint(test) and (train | test) == source
if len(source) != 18:
    raise ValueError("Cần đủ 18 mẫũ nguồn trước khi chia Train/Test.")
print("Mẫũ nguồn:", len(source))
print("Trùng:", ", ".join(overlap))
print("Chưa phân nhóm:", ", ".join(missing))
print("Phân hoạch đúng:", valid_partition)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm lỗĩ chia Train/Test trong 18 mẫũ bằng giao nhau, hợp và phần thiếú.
# Dữ liệu: 18 mã D01..D18, D12 xuất hiện ở cả train/test; D18 chưa được phân nhóm.
# List 18 mã nguồn; dùng set bên dưới để đối chiếú như tập hợp.
all_ids =
["D01","D02","D03","D04","D05","D06","D07","D08","D09","D10","D11","D12","D13","D14","D15","D16","D
17","D18"]
# set() biến list 12 mã thành tập train không trùng nội bộ.
train = set(["D01","D02","D03","D04","D05","D06","D07","D08","D09","D10","D11","D12"])
# set() biến list mã test thành tập; D12 bị trùng với train.
test = set(["D12","D13","D14","D15","D16","D17"])
# set(all_ids) lấý tập 18 mã nguồn không trùng.
source = set(all_ids)
# & là giao train/test: những mẫũ BỊ LỘ DỮ LIỆU giữa hai tập; sorted() xếp mã.
overlap = sorted(train & test)
# | hợp mã đã phân nhóm, - trừ khỏi nguồn để tìm mẫũ chưa được giao vào nhóm nào.
missing = sorted(source - (train | test))
# isdisjoint() True chỉ khi train và test không có chung một mẫũ.
# Hợp phải bằng nguồn để không sót mẫũ hay phát sinh mã lạ; and đòi cả 2 phép kiểm.
# Cờ True chỉ khi tập không giao nhau VÀ hợp bằng toàn nguồn.
valid_partition = train.isdisjoint(test) and (train | test) == source
# Kiểm đủ 18 mã khác nhau, không đơn thuần số dòng list.
if len(source) != 18:
    # ValueError dừng khi dữ liệu hoặc điều kiện đối chiếú không hợp lệ.
    raise ValueError("Cần đủ 18 mẫũ nguồn trước khi chia Train/Test.")
print("Mẫũ nguồn:", len(source))
print("Trùng:", ", ".join(overlap))
print("Chưa phân nhóm:", ", ".join(missing))
```

```
# Cờ True chỉ khi tập không giao nhau VÀ hợp bằng toàn nguồn.
print("Phân hoạch đúng:", valid_partition)
```

Kết quả khi chạy:

```
Mẫu nguồn: 18
Trùng: D12
Chưa phân nhóm: D18
Phân hoạch đúng: False
```

Chỉ khi hai điều kiện cùng đúng mới sử dụng train/test: một mẫu trùng có thể làm sai đánh giá, và mẫu bỏ sót làm phân chia chưa đầy đủ.

Tình huống 5 — Cấp 5: Kiểm kê 20 hồ sơ đa vùng, cảnh báo ID xuất hiện chéo

Từ vựng trong mục: Disjoint sets — tập rời nhau; **Union** — phép hợp; **Data integrity** — toàn vẹn dữ liệu.

Tình huống thực tế: Một chương trình học trực tuyến ghi **20 dòng hoạt động** từ người học ở ba vùng Bắc, Nam và Trung. Mỗi dòng có mã học viên, vùng, kênh truy cập và điểm hoạt động (**điểm**); một người có thể có nhiều dòng. **S04** được ghi ở Bắc và Nam, **S07** ở Bắc và Trung. Nhân viên muốn biết tổng **người khác nhau**, có bao nhiêu dòng dư do mã xuất hiện lại và danh sách học viên có mặt ở nhiều vùng, nhưng không tự đoán vùng đúng để xóa dữ liệu.

Vì sao chọn cách này? Nhóm người mỗi vùng được biểu diễn bằng một set mã, vì **một người có nhiều dòng trong cùng vùng** cũng chỉ nên được tính là một thành viên vùng đó. `set().union(*regions.values())` hợp ba nhóm để tìm toàn bộ **mã khác nhau**. Sau đó chương trình duyệt các mã này và đếm một mã nằm trong bao nhiêu set vùng; số lớn hơn một thì ghi vào danh sách **đa vùng**. Lấy số dòng gốc trừ số người duy nhất cho biết có bao nhiêu lượt ghi vượt số mã khác nhau, nhưng **không chứng minh dòng nào sai**. Việc xóa một vùng của S04 hay S07 phải dựa vào hồ sơ đối chiếu thật, không dựa vào phép tính tập hợp.

Quy trình:

1. Ghi 20 dòng đầy đủ ID/vùng/kênh/điểm và tạo Set cho từng vùng.
2. Lấy hợp ba vùng để đếm ID duy nhất, kiểm tra giao từng cặp.
3. Liệt kê ID đa vùng và số dòng dư do trùng ID; giữ lại dữ liệu thô để sửa.

No Note — Không chú thích

```
records = [{"id":"S01","region":"Bắć","channel":"app","score":10},
{"id":"S02","region":"Nam","channel":"web","score":11},
{"id":"S03","region":"Trung","channel":"app","score":12},
{"id":"S04","region":"Bắć","channel":"web","score":13},
{"id":"S05","region":"Nam","channel":"app","score":14},
{"id":"S06","region":"Trung","channel":"web","score":15},
{"id":"S07","region":"Bắć","channel":"app","score":16},
{"id":"S08","region":"Nam","channel":"web","score":17},
{"id":"S09","region":"Trung","channel":"app","score":18},
{"id":"S10","region":"Bắć","channel":"web","score":19},
{"id":"S11","region":"Nam","channel":"app","score":20},
{"id":"S12","region":"Trung","channel":"web","score":21},
{"id":"S13","region":"Bắć","channel":"app","score":22},
{"id":"S14","region":"Nam","channel":"web","score":23},
{"id":"S15","region":"Trung","channel":"app","score":24},
{"id":"S16","region":"Bắć","channel":"web","score":25},
```

```
{"id":"S04","region":"Nam","channel":"app","score":26},
{"id":"S07","region":"Trung","channel":"web","score":27},
{"id":"S19","region":"Bắć","channel":"app","score":28},
{"id":"S20","region":"Nam","channel":"web","score":29}]
regions = {}
for name in ("Bắć", "Nam", "Trung"):
    regions[name] = set()
for row in records:
    regions[row["region"]].add(row["id"])
all_people = set().union(*regions.values())
multiple_regions = []
for sid in all_people:
    membership_count = 0
    for members in regions.values():
        if sid in members:
            membership_count += 1
    if membership_count > 1:
        multiple_regions.append(sid)
multiple_regions.sort()
if len(records) != 20:
    raise ValueError("Cần kiểm tra đủ 20 lượt ghi nhận theo vùng.")
print("Dòng nhận:", len(records))
print("Người duy nhất:", len(all_people))
print("Dòng dư do trùng ID:", len(records) - len(all_people))
print("Đa vùng:", ", ".join(multiple_regions))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Gộp 20 lượt ghi nhận ba vùng, tìm mã người duy nhất và người xuất hiện ở nhiều vùng.
# Dữ liệu: 20 dictionary (id,region,channel,score), ID S04/S07 có thể xuất hiện ở nhiều vùng; một
lượt là một hàng.
# 20 bản ghi; region là vùng, channel kênh, score là số điểm giả lập; khóa id có thể trùng giữa các
dòng.
records = [{"id":"S01","region":"Bắć","channel":"app","score":10},
{"id":"S02","region":"Nam","channel":"web","score":11},
{"id":"S03","region":"Trung","channel":"app","score":12},
{"id":"S04","region":"Bắć","channel":"web","score":13},
{"id":"S05","region":"Nam","channel":"app","score":14},
{"id":"S06","region":"Trung","channel":"web","score":15},
{"id":"S07","region":"Bắć","channel":"app","score":16},
{"id":"S08","region":"Nam","channel":"web","score":17},
{"id":"S09","region":"Trung","channel":"app","score":18},
{"id":"S10","region":"Bắć","channel":"web","score":19},
{"id":"S11","region":"Nam","channel":"app","score":20},
{"id":"S12","region":"Trung","channel":"web","score":21},
{"id":"S13","region":"Bắć","channel":"app","score":22},
{"id":"S14","region":"Nam","channel":"web","score":23},
{"id":"S15","region":"Trung","channel":"app","score":24},
{"id":"S16","region":"Bắć","channel":"web","score":25},
{"id":"S04","region":"Nam","channel":"app","score":26},
{"id":"S07","region":"Trung","channel":"web","score":27},
{"id":"S19","region":"Bắć","channel":"app","score":28},
{"id":"S20","region":"Nam","channel":"web","score":29}]
# Dictionary ánh xạ tên vùng tới tập mã học viên riêng.
regions = {}
# Duyệt ba nhãn Bắć/Nam/Trung để tạo set rỗng cho mỗi vùng.
for name in ("Bắć", "Nam", "Trung"):
    # Tạo tập mã chưa có phần tử; set không lặp cùng ID trong vùng.
    regions[name] = set()
# Mỗi vòng xử lý đúng một dictionary nguồn.
for row in records:
    # Tra set theo khóa region rồi add() mã id; tự loại bản ghi trùng trong một vùng.
    regions[row["region"]].add(row["id"])
# regions.values() lấy ba set; toán tử* trải chúng thành đối số cho union(), hợp mọi người không
trùng giữa vùng.
```

```
all_people = set().union(*regions.values())
multiple_regions = []
# Duyệt từng người duy nhất để xem họ xuất hiện ở bao nhiêu vùng.
for sid in all_people:
    membership_count = 0
    # Duyệt tập người của ba vùng.
    for members in regions.values():
        # in kiểm tra mã sid nằm trong set vùng hiện tại.
        if sid in members:
            # Đếm số VÙNG chứa một mã, không phải số lượt đăng ký.
            membership_count += 1
    # Người thuộc hơn một vùng được ghi riêng để kiểm tra chồng chéo.
    if membership_count > 1:
        # append() lưu mã người đa vùng.
        multiple_regions.append(sid)
# .sort() xếp list ngay tại chỗ để báo cáo có thứ tự cố định.
multiple_regions.sort()
# Kiểm đủ 20 lượt ghi nhận gốc.
if len(records) != 20:
    # ValueError dừng khi dữ liệu hoặc điều kiện đối chiếu không hợp lệ.
    raise ValueError("Cần kiểm tra đủ 20 lượt ghi nhận theo vùng.")
print("Dòng nhận:", len(records))
print("Người duy nhất:", len(all_people))
# Hiệu số dòng so với người duy nhất cho biết số lượt dư do lặp ID trên toàn dữ liệu.
print("Dòng dư do trùng ID:", len(records) - len(all_people))
# join() nối các mã đa vùng thành một chuỗi có dấu phẩy.
print("Đa vùng:", ", ".join(multiple_regions))
```

Kết quả khi chạy:

```
Dòng nhận: 20
Người duy nhất: 18
Dòng dư do trùng ID: 2
Đa vùng: S04, S07
```

Người học phải phân biệt **dòng**, **ID duy nhất**, **ID trùng xuyên vùng**; đây là các câu hỏi khác nhau và cần kiểm tra cả ba trước khi báo cáo.

6. Sai lầm thường gặp

- Set không giữ thứ tự; dùng `sorted()` khi muốn kết quả mẫu có thứ tự ổn định.
- Phép hợp `|` khác phép giao `&`: hợp lấy ít nhất một nhóm, giao lấy cả hai nhóm.
- Hiệu `A - B` khác hiệu đối xứng `A ^ B`: một chiều so với chỉ xuất hiện ở đúng một trong hai tập.
- Bao hàm–loại trừ tránh đếm trùng. Nếu có ba tập trở lên, công thức cần cộng/trừ thêm các giao nhiều tập.
- `isdisjoint()` chỉ chứng minh hai tập không trùng, chưa chứng minh chúng phủ toàn bộ tập gốc.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does a union contain? | Phép hợp chứa gì?

A. Elements in at least one set | Phần tử thuộc ít nhất một tập

B. Elements common to both sets | Phần tử có ở cả hai tập

C. Elements in A but not in B | Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set | Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: A

Giải thích: Hợp chứa các phần tử khác nhau của hai tập.

Q2. What does an intersection contain? | Phép giao chứa gì?

A. Elements in at least one set | Phần tử thuộc ít nhất một tập

B. Elements common to both sets | Phần tử có ở cả hai tập

C. Elements in A but not in B | Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set | Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: B

Giải thích: Giao chỉ giữ phần tử chung của hai tập.

Q3. What is set difference A−B? | Hiệu tập hợp A−B là gì?

A. Elements in at least one set | Phần tử thuộc ít nhất một tập

B. Elements common to both sets | Phần tử có ở cả hai tập

C. Elements in A but not in B | Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set | Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: C

Giải thích: Hiệu A−B loại phần tử của B ra khỏi A.

Q4. What defines a partition of a set? | Điều gì xác định một phân hoạch?

A. Elements in at least one set | Phần tử thuộc ít nhất một tập

B. Elements common to both sets | Phần tử có ở cả hai tập

C. Elements in A but not in B | Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set | Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: D

Giải thích: Mọi phần không giao nhau và hợp bằng tập gốc.

Q5. Which operator computes union for sets? | Toán tử nào tính phép hợp Set?

A. `a | b` | `a | b`

B. `a & b` | `a & b`

C. `a ^ b` | `a ^ b`

D. `a.isdisjoint(b)` | `a.isdisjoint(b)`

Đáp án: A

Giải thích: Dấu | biểu diễn phép hợp tập hợp.

Q6. Which operator computes intersection? | Toán tử nào tính phép giao?

A. `a | b` | `a | b`

B. `a & b` | `a & b`

C. `a ^ b` | `a ^ b`

D. `a.isdisjoint(b)` | `a.isdisjoint(b)`

Đáp án: B

Giải thích: Dấu & giữ các phần tử chung.

Q7. Which operator computes symmetric difference? | Toán tử nào tính hiệu đối xứng?

A. `a | b` | `a | b`

B. `a & b` | `a & b`

C. `a ^ b` | `a ^ b`

D. `a.isdisjoint(b)` | `a.isdisjoint(b)`

Đáp án: C

Giải thích: Hiệu đối xứng giữ phần tử chỉ ở một bên.

Q8. Which method checks that two sets do not overlap? | Phương thức nào kiểm tra hai Set không trùng nhau?

A. `a | b` | `a | b`

B. `a & b` | `a & b`

C. `a ^ b` | `a ^ b`

D. `a.isdisjoint(b)` | `a.isdisjoint(b)`

Đáp án: D

Giải thích: isdisjoint đúng khi phần giao rỗng.

Q9. Why does inclusion–exclusion subtract an intersection? | Vì sao bao hàm–loại trừ phải trừ phần giao?

A. To correct double counting | Để sửa việc đếm trùng

B. Distinct elements in either set | Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members | Khi hai tập không có phần tử chung

D. An ordered List of its elements | List chứa các phần tử được sắp thứ tự

Đáp án: A

Giải thích: Phần tử giao đã được tính trong cả hai số đếm.

Q10. What does `len(a | b)` count? | `len(a | b)` đếm gì?

A. To correct double counting | Để sửa việc đếm trùng

B. Distinct elements in either set | Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members | Khi hai tập không có phần tử chung

D. An ordered List of its elements | List chứa các phần tử được sắp thứ tự

Đáp án: B

Giải thích: Hợp tự loại phần trùng của các nhóm.

Q11. When is `a.isdisjoint(b)` True? | Khi nào `a.isdisjoint(b)` bằng True?

A. To correct double counting | Để sửa việc đếm trùng

B. Distinct elements in either set | Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members | Khi hai tập không có phần tử chung

D. An ordered List of its elements | List chứa các phần tử được sắp thứ tự

Đáp án: C

Giải thích: Hai tập rời có phần giao rỗng.

Q12. What does `sorted(my_set)` produce? | `sorted(my_set)` tạo gì?

A. To correct double counting | Để sửa việc đếm trùng

B. Distinct elements in either set | Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members | Khi hai tập không có phần tử chung

D. An ordered List of its elements | List chứa các phần tử được sắp thứ tự

Đáp án: D

Giải thích: sorted tạo List và không sửa Set gốc.

Q13. How many distinct values are in `{1,1,2}`? | `{1,1,2}` có mấy phần tử khác nhau?

A. 2 | 2

B. 4 | 4

C. Every part is nonempty and the union covers the original set | Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared | Không có phần tử chung

Đáp án: A

Giải thích: Giá trị trùng chỉ xuất hiện một lần trong Set.

Q14. If |A|=3, |B|=2, |A∩B|=1, what is |A∪B|? | Nếu |A|=3, |B|=2, |A∩B|=1 thì |A∪B| bằng bao nhiêu?

A. 2 | 2

B. 4 | 4

C. Every part is nonempty and the union covers the original set | Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared | Không có phần tử chung

Đáp án: B

Giải thích: Dùng công thức 3+2−1=4.

Q15. Which two conditions besides disjointness are needed to verify a nonempty partition? | Ngoài rời nhau, cần kiểm tra gì để xác nhận các phần không rỗng là phân hoạch?

A. 2 | 2

B. 4 | 4

C. Every part is nonempty and the union covers the original set | Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared | Không có phần tử chung

Đáp án: C

Giải thích: Phân hoạch phải phủ đủ phần tử gốc và không có phần rỗng.

Q16. What does the empty intersection mean? | Phần giao rỗng nghĩa là gì?

A. 2 | 2

B. 4 | 4

C. Every part is nonempty and the union covers the original set | Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared | Không có phần tử chung

Đáp án: D

Giải thích: Giao rỗng biểu thị hai tập rời nhau.

Q17. What does A∪A equal? | A∪A bằng gì?

A. A | A

B. Empty set | Tập rỗng

C. No fixed order is promised | Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits | Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: A

Giải thích: Hợp một tập với chính nó không thêm phần tử.

Q18. What does A∩∅ equal? | A∩∅ bằng gì?

A. A | A

B. Empty set | Tập rỗng

C. No fixed order is promised | Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits | Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: B

Giải thích: Tập rỗng không có phần tử nào chung với A nên phép giao rỗng.

Q19. Is set iteration order guaranteed? | Thứ tự duyệt Set có được đảm bảo không?

A. A | A

B. Empty set | Tập rỗng

C. No fixed order is promised | Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits | Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: C

Giải thích: Set Python không phải dãy có thứ tự.

Q20. Why can two data splits with disjoint IDs still leak information? | Vì sao hai tập dữ liệu không trùng ID vẫn có thể rò rỉ thông tin?

A. A | A

B. A itself | Chính tập A

C. No fixed order is promised | Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits | Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: D

Giải thích: ID không trùng chưa loại được mọi nguồn rò rỉ.

8. Nối sang bài tiếp

`Set → hợp/giao/hiệu → bao hàm–loại trừ → train/test.`

PY-22 — Hoán vị, tổ hợp và tam giác Pascal

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Permutation — hoán vị; **Combination** — tổ hợp.

`Nguyên lý đếm → hoán vị/tổ hợp → phân phối nhị thức`

Học xong dùng để làm gì? Tính số cách chọn người, chia vai hoặc sắp lịch, phân biệt có và không xét thứ tự.

Kết nối kiến thức tiếp theo: PY-23 mô hình hóa quan hệ và độ tương đồng giữa đối tượng bằng đồ thị/ Jaccard.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Permutation	hoán vị	Chọn có phân biệt thứ tự
Combination	tổ hợp	Chọn nhóm không xét thứ tự
Factorial	giai thừa	Tích các số từ 1 đến n
Binomial coefficient	hệ số nhị thức	Số cách chọn k từ n

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`math.factorial(n)`	Tính n!, n không âm
`math.perm(n,k)`	Đếm cách chọn có thứ tự k từ n
`math.comb(n,k)`	Đếm cách chọn không thứ tự
`itertools.combinations/permutations`	Liệt kê kết quả khi n nhỏ

4. Khái niệm liên quan

Khi chọn người hoặc sắp xếp công việc, câu hỏi đầu tiên là **đổi thứ tự có tạo ra kết quả khác không?** Nếu có, ta đếm các cách sắp xếp bằng **hoán vị**; nếu không, ta đếm **tổ hợp**. **Giai thừa** là phép nhân nền tảng để tính những số cách đó, còn **hệ số nhị thức** tạo nên các hàng của tam giác Pascal.

4.1. Permutation — Chọn và phân vai có thứ tự

Từ vựng trong mục: Permutation — chỉnh hợp/hoán vị có thứ tự; **Order** — thứ tự.

Có năm ứng viên và hai vị trí **khác nhau** là trưởng nhóm, phó nhóm. Chọn trưởng có 5 cách; sau đó chọn phó còn 4 cách, nên có `5 × 4 = 20` phương án. Đảo vai trò hai người tạo phương án khác. `math.perm(n,k)` đếm cách lấy **k người khác nhau từ n người** và phân thứ tự, với `0 ≤ k ≤ n`. Nếu dùng hết n người, kết quả là `n!`.

No Note — Không chú thích

```
import math

print(math.perm(5, 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm cách chọn hai người vào hai vai CÓ phân biệt thứ tự trong năm ứng viên.
# Dữ liệu: 5 ứng viên khác nhau, hai vị trí vai trò riêng; đổi chỗ hai người thành cách khác.
# math là thư viện chuẩn, cung cấp phép đếm hoán vị perm().
import math

# perm(n,k) đếm số cách chọn K người có THỨ TỰ từ N người khác nhau; 5×4=20, không liệt kê danh sách.
print(math.perm(5, 2))
```

Kết quả khi chạy:

```
20
```

Có 20 cách phân vai trưởng/phó; đổi thứ tự hai người được tính là cách khác.

4.2. Combination — Chọn nhóm không phân vai

Từ vựng trong mục: Combination — tổ hợp; **Order-independent** — không xét thứ tự.

Nếu năm ứng viên chỉ cần chọn **hai người cùng một đội**, cặp An–Bình và Bình–An là **một** đội, không phải hai. Công thức `C(n,k) = n! / (k!(n-k)!)`; với 5 chọn 2 có 10 nhóm. `math.comb(n,k)` tính trực tiếp số nhóm mà không cần liệt kê từng cặp. Trong thực tế cần xác nhận có được chọn một người nhiều lần hay không; ở đây mỗi ứng viên chỉ chọn tối đa một lần.

No Note — Không chú thích

```
import math

print(math.comb(5, 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm nhóm hai người KHÔNG phân vai từ năm ứng viên.
# Dữ liệu: 5 người, nhóm hai người không có vai thứ nhất/thứ hai.
# math.comb() dùng để đếm tổ hợp không phân biệt thứ tự.
import math

# comb(5,2) đếm số cách chọn hai người không thứ tự: hai hoán vị (An,Lan)/(Lan,An) cùng một nhóm, tổng 10.
print(math.comb(5, 2))
```

Kết quả khi chạy:

```
10
```

Số nhóm bằng nửa số cách phân vai hai người vì hoán đổi hai vai không tạo đội mới.

4.3. Factorial — Sắp xếp tất cả phần tử

Từ vựng trong mục: Factorial — giai thừa; **Arrangement** — thứ tự sắp xếp.

Với bốn nhiệm vụ **khác nhau**, có 4 cách chọn việc đầu, 3 cách chọn việc thứ hai, rồi 2 và 1. Tổng số cách sắp xếp là `4! = 4×3×2×1 = 24`. Đây là **giai thừa**. Quy ước `0! = 1` vì có đúng một cách sắp xếp dãy rỗng. `math.factorial` áp dụng cho số nguyên không âm.

No Note — Không chú thích

```
import math

print(math.factorial(4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm số´cách săp thứ tự bốn đối tượng khác nhau bằng giai thừa.
# Dữ liệu: 4 vật khác nhau, muốn xếp đủ vào bốn vị trí.
# math.factorial() hỗ˜trợ phép đếm thứ tự.
import math

# factorial(4) tính 4×3×2×1=24 cách xếp cả bốn vật, không giống comb chọn không thứ tự.
print(math.factorial(4))
```

Kết quả khi chạy:

```
24
```

Bốn nhiệm vụ khác nhau có 24 thứ tự thực hiện nếu mỗi nhiệm vụ xuất hiện đúng một lần.

4.4. Binomial coefficient và tam giác Pascal

Từ vựng trong mục: Binomial coefficient — hệ số nhị thức; **Pascal triangle** — tam giác Pascal.

Hệ số nhị thức `C(n,k)` là số cách chọn k phần tử không xét thứ tự, đồng thời là hệ số của `a^(n-k)b^k` trong khai triển `(a+b)^n`. Các giá trị `C(n,0)` đến `C(n,n)` tạo thành **hàng n của tam giác Pascal** (đếm từ hàng 0). Với n = 4, hàng có năm hệ số `1,4,6,4,1`; hai cạnh bằng 1 và mỗi ô trong bằng tổng hai ô ngay phía trên.

No Note — Không chú thích

```
import math

counts = []
for k in range(5):
    counts.append(math.comb(4, k))
print(counts)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo hàng thứ tư của tam giác Pascal bằng sô´tô̂hợp chọn k trong bôń.
# Dữ liệu: k nhận các giá trị 0..4, môĩ phần tửbằng sô´cách chọn k đôí tượng từ 4.
# math.comb(n,k) đêḿ cách chọn k phần tửkhông xét thứ tự.
import math

# List lưu năm hệ sô´tô̂hợp theo k tăng dần.
counts = []
# range(5) cho các sô´0..4; không lấý 5.
for k in range(5):
    # comb(4,k) lấý hệ sô´tương ứng hàng n=4 của tam giác Pascal, lần lượt 1,4,6,4,1.
    # append() thêm hệ sô´vào cuôí list đểgiữ thứ tự k.
    counts.append(math.comb(4, k))
# In năm hệ sô´đểquan sát hàng Pascal.
print(counts)
```

Kết quả khi chạy:

```
[1, 4, 6, 4, 1]
```

Hàng thứ tư tính từ 0 có năm hệ số, ứng với k từ 0 đến 4.

5. Năm tình huống hoán vị và tổ hợp — từ phân vai đến chọn nhóm có ràng buộc

Tình huống 1 — Cấp 1: Phân vai trưởng và phó từ 10 ứng viên

Từ vựng trong mục: Permutation — chọn có thứ tự; **Role** — vai trò.

Tình huống thực tế: Câu lạc bộ có **10 ứng viên U01–U10** đã đủ điều kiện. Cần chọn **một trưởng nhóm** và **một phó nhóm** từ hai người khác nhau. Nếu U01 làm trưởng và U02 làm phó thì đó là phương án khác với U02 làm trưởng, U01 làm phó. Người phụ trách muốn biết tổng số cách phân vai và xem một phương án đầu tiên từ chương trình.

Vì sao chọn cách này? Chỉnh hợp phù hợp khi **vị trí, vai trò hoặc thứ tự có ý nghĩa**. Trưởng và phó không giống nhau, nên đổi hai người tạo phương án mới. `math.perm(len(candidates), 2)` đếm cách điền lần lượt hai vai bằng hai người khác nhau; `itertools.permutations(candidates, 2)` có thể liệt kê các cặp để kiểm tra. Vòng `for` lấy cặp đầu và `break` vì bài chỉ cần **một ví dụ**, không phải in toàn bộ các phương án. Nếu dùng tổ hợp không xét thứ tự thì sẽ gộp nhầm hai cách giao chức danh khác nhau.

Quy trình:

1. Lưu đúng 10 ID ứng viên.
2. Đếm phương án có thứ tự bằng `math.perm`.
3. Liệt kê kết quả đầu tiên để xác minh vai trò khác nhau.

No Note — Không chú thích

```
import math
from itertools import permutations
candidates = ["U01","U02","U03","U04","U05","U06","U07","U08","U09","U10"]
first = None
for pair in permutations(candidates, 2):
    first = pair
    break
```

```
if first is None:
    raise ValueError("Cần ít nhất hai ứng viên để phân vai.")
if len(candidates) != 10:
    raise ValueError("Cần đúng 10 ứng viên.")
print("Ứng viên:", len(candidates))
print("Cách phân vai:", math.perm(len(candidates), 2))
print("Phân vai đầu:", first[0], first[1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Liệt kê cách phân hai vai từ 10 ứng viên và lấy phương án đầu tiên làm ví dụ.
# Dữ liệu: 10 ứng viên U01..U10; hai vai không giống nhau nên hoán đổi người là lịch phân vai khác.
# math.perm() tính SỐ hoán vị mà không dựng mọi hoán vị.
import math
# permutations() sinh từng tuple có thứ tự gồm r ứng viên, không lặp người trong một tuple.
from itertools import permutations
# List mười mã ứng viên, được duyệt theo thứ tự liệt kê.
candidates = ["U01","U02","U03","U04","U05","U06","U07","U08","U09","U10"]
# None đánh dấu chưa chọn được bất kỳ bộ hai vai.
first = None
# Sinh từng cặp có phân biệt vai thứ nhất/thứ hai; iterator không tạo sẵn mọi cặp thành list.
for pair in permutations(candidates, 2):
    # Giữ cặp đầu tiên do iterator trả về.
    first = pair
    # break dừng vòng sớm vì chỉ cần minh họa một phương án, không duyệt đủ.
    break
# is None phát hiện không có cặp khả thi (ít hơn hai ứng viên).
if first is None:
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Cần ít nhất hai ứng viên để phân vai.")
# len() xác nhận dữ liệu có đúng mười ứng viên.
if len(candidates) != 10:
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Cần đúng 10 ứng viên.")
print("Ứng viên:", len(candidates))
# Đếm tất cả cách gán hai vai có thứ tự từ mười người: 10×9=90.
print("Cách phân vai:", math.perm(len(candidates), 2))
# [0] là người vai đầu, [1] người vai sau, Python đánh chỉ số từ 0.
print("Phân vai đầu:", first[0], first[1])
```

Kết quả khi chạy:

```
Ứng viên: 10
Cách phân vai: 90
Phân vai đầu: U01 U02
```

Người U01 làm trưởng và U02 làm phó khác cách đảo vai; không cho phép một ứng viên bị chọn hai lần.

Tình huống 2 — Cấp 2: Chọn nhóm 3 người từ 12 tình nguyện viên

Từ vựng trong mục: Combination — chọn không xét thứ tự; **Distinct** — không trùng người.

Tình huống thực tế: Ban tổ chức sự kiện có **12 tình nguyện viên S01–S12**. Mỗi đội trực chỉ cần **3 người ngang vai**, không có vai trò trưởng hay phó; một người không được xuất hiện hai lần trong một đội. Đội gồm S01, S02, S03 vẫn là cùng một đội khi đọc tên theo thứ tự S03, S02, S01. Người quản lý muốn đếm các đội khác nhau và liệt kê một đội đầu tiên.

Vì sao chọn cách này? Tổ hợp dùng khi chỉ quan tâm **ai được chọn**, không quan tâm tên nào được viết trước. Đội S01–S02–S03 và S03–S01–S02 là một đội, vì cả ba người ngang vai. `math.comb(12, 3)` tính số đội

khác nhau; `itertools.combinations(volunteers, 3)` liệt kê mỗi bộ ba đúng một lần để chương trình có thể đối chiếu kết quả. Không dùng chỉnh hợp, vì chỉnh hợp tính riêng các thứ tự của cùng ba người nên số đội sẽ lớn hơn thực tế.

Quy trình:

1. Lưu 12 mã thành viên.
2. Dùng tổ hợp chọn ba mã, không xét thứ tự.
3. So số tổ với danh sách liệt kê để xác nhận.

No Note — Không chú thích

```
import math
from itertools import combinations
volunteers = ["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10","S11","S12"]
groups = list(combinations(volunteers, 3))
if len(volunteers) != 12:
    raise ValueError("Cần đủ 12 tình nguyện viên.")
if len(groups) != math.comb(12, 3):
    raise ValueError("Số nhóm ba người phải khớp số tổ hợp lý thuyết.")
print("Tình nguyện viên:", len(volunteers))
print("Số nhóm:", len(groups))
print("Nhóm đầu:", ", ".join(groups[0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm và liệt kê nhóm ba tình nguyện viên từ 12 người, không xét thứ tự trong nhóm.
# Dữ liệu: 12 mã S01..S12, chọn ba người không phân vai; [S01,S02,S03] là cùng nhóm dù đổi thứ tự.
# math.comb() dùng để đối chiếu lý thuyết số nhóm.
import math
# combinations() tạo từng nhóm r phần tử KHÔNG xét thứ tự và không lặp người.
from itertools import combinations
# List 12 mã người có thể được chọn.
volunteers = ["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10","S11","S12"]
# Tạo list tất cả bộ ba không phân vai; số lượng có thể lớn nếu n tăng, ở đây chỉ 220 nhóm.
groups = list(combinations(volunteers, 3))
# Kiểm đủ 12 người nguồn trước khi kết luận.
if len(volunteers) != 12:
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Cần đủ 12 tình nguyện viên.")
# comb(12,3) tính 220 nhóm, để đối chiếu với số nhóm được tạo thực tế.
if len(groups) != math.comb(12, 3):
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Số nhóm ba người phải khớp số tổ hợp lý thuyết.")
print("Tình nguyện viên:", len(volunteers))
print("Số nhóm:", len(groups))
# [0] là tuple nhóm đầu tiên; ', '.join(...) ghép ba mã thành chuỗi.
print("Nhóm đầu:", ", ".join(groups[0]))
```

Kết quả khi chạy:

```
Tình nguyện viên: 12
Số nhóm: 220
Nhóm đầu: S01, S02, S03
```

Bài này khác cấp 1 ở chỗ đổi vị trí ba người **không** tạo nhóm mới; nhận biết thứ tự quan trọng hay không quyết định phép đếm.

Tình huống 3 — Cấp 3: Lọc 15 ứng viên trước khi chọn hai vai kiểm thử

Từ vựng trong mục: Eligibility — đủ điều kiện; **Permutation** — phân vai; **Combination** — chọn nhóm.

Tình huống thực tế: Đội kiểm thử phần mềm có **15 hồ sơ nhân sự T01–T15**. Mỗi hồ sơ ghi kỹ năng Python, AI hoặc SQL và cờ `available` cho biết người đó có sẵn sàng hay không. Dự án cần **hai người có vai trò khác nhau**, một người viết kiểm thử và một người duyệt; chỉ người sẵn sàng và có kỹ năng **Python hoặc AI** được chọn. Nhân sự muốn đếm người đủ chuẩn, số đội hai người không phân vai và số cách giao hai vai thực tế.

Vì sao chọn cách này? Đầu tiên phải **lọc người đủ điều kiện** theo `available` và kỹ năng; số 15 là tổng hồ sơ, **không phải tổng người được chọn**. Sau lọc, nếu chỉ hỏi chọn hai đồng đội thì dùng `math.comb` vì hai người ngang vai. Nếu hỏi ai viết và ai duyệt thì dùng `math.perm` vì đổi vai tạo cách giao nhiệm vụ khác. Như vậy cùng một danh sách hợp lệ có thể trả hai con số khác nhau vì **câu hỏi kinh doanh khác nhau**, chứ không phải hai thuật toán mâu thuẫn.

Quy trình:

1. Kiểm tra từng hồ sơ trong 15 dòng để lọc kỹ năng và cờ sẵn sàng.
2. Đếm số cặp không phân vai bằng `comb`.
3. Đếm số phương án gán hai vai bằng `perm`, xác nhận mỗi cặp có hai cách phân vai.

No Note — Không chú thích

```
import math
people = [{"id":"T01","skill":"Python","available":True},
{"id":"T02","skill":"AI","available":True},{"id":"T03","skill":"SQL","available":True},
{"id":"T04","skill":"Python","available":True},{"id":"T05","skill":"AI","available":False},
{"id":"T06","skill":"SQL","available":True},{"id":"T07","skill":"Python","available":True},
{"id":"T08","skill":"AI","available":True},{"id":"T09","skill":"SQL","available":True},
{"id":"T10","skill":"Python","available":False},{"id":"T11","skill":"AI","available":True},
{"id":"T12","skill":"SQL","available":True},{"id":"T13","skill":"Python","available":True},
{"id":"T14","skill":"AI","available":True},{"id":"T15","skill":"SQL","available":False}]
eligible = []
for person in people:
    if person["available"] and person["skill"] in ("Python", "AI"):
        eligible.append(person["id"])
if len(people) != 15:
    raise ValueError("Cần kiểm tra 15 hồ sơ nhân sự.")
print("Hồ sơ:", len(people))
print("Đủ điều kiện:", len(eligible))
print("Chọn đội hai người:", math.comb(len(eligible), 2))
print("Phân hai vai:", math.perm(len(eligible), 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc 15 nhân sự đủ sẵn sàng và kỹ năng Python/AI, so sô´đội hai người với phân hai vai.
# Dữ liệu: 15 hồ sơ id, skill, available; chỉ người available=True và môn Python/AI được xét.
# math.comb() đếm nhóm không phân vai, math.perm() đếm phân vai có thứ tự.
import math
# 15 dict, môĩ hàng một nhân sự với mã, kỹ năng và trạng thái sẵn sàng Boolean.
people = [{"id":"T01","skill":"Python","available":True},
{"id":"T02","skill":"AI","available":True},{"id":"T03","skill":"SQL","available":True},
{"id":"T04","skill":"Python","available":True},{"id":"T05","skill":"AI","available":False},
{"id":"T06","skill":"SQL","available":True},{"id":"T07","skill":"Python","available":True},
{"id":"T08","skill":"AI","available":True},{"id":"T09","skill":"SQL","available":True},
{"id":"T10","skill":"Python","available":False},{"id":"T11","skill":"AI","available":True},
{"id":"T12","skill":"SQL","available":True},{"id":"T13","skill":"Python","available":True},
{"id":"T14","skill":"AI","available":True},{"id":"T15","skill":"SQL","available":False}]
```

```
# Chứa mã những nhân sự thỏa cả hai tiêu chí.
eligible = []
# Duyệt từng dict hồ sơ để kiểm tiêu chí.
for person in people:
    # and yêu cầu người vừa sẵn sàng vừa có kỹ năng thuộc tuple hai môn; in kiểm tra một trong các
giá trị, KHÔNG phải chuỗi con.
    if person["available"] and person["skill"] in ("Python", "AI"):
        # append() giữ mã đúng những người được chọn.
        eligible.append(person["id"])
# Kiểm tổng 15 hồ sơ nguồn, kể cả người không đủ điều kiện.
if len(people) != 15:
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Cần kiểm tra 15 hồ sơ nhân sự.")
print("Hồ sơ:", len(people))
print("Đủ điều kiện:", len(eligible))
# comb() đếm cách chọn đội 2 người không vai: A+B giống B+A.
print("Chọn đội hai người:", math.comb(len(eligible), 2))
# perm() đếm cách phân hai vai có thứ tự: A giữ vai 1 khác B giữ vai 1.
print("Phân hai vai:", math.perm(len(eligible), 2))
```

Kết quả khi chạy:

```
Hồ sơ: 15
Đủ điều kiện: 8
Chọn đội hai người: 28
Phân hai vai: 56
```

Khác hai cấp đầu, phải lọc dữ liệu trước khi đếm. Không được chọn người không sẵn sàng hoặc thiếu kỹ năng phù hợp.

Tình huống 4 — Cấp 4: Lập lịch ba lượt xử lý từ 18 ca lỗi có mức khẩn cấp

Từ vựng trong mục: Ordered selection — chọn có thứ tự; **Factorial** — giai thừa; **Constraint** — ràng buộc.

Tình huống thực tế: Đội trực vận hành có **18 phiếu sự cố C01–C18**; mỗi phiếu ghi mức `blocker` (ngăn hệ thống hoạt động), `high` (nghiêm trọng) hoặc `normal` (thông thường), cùng cờ `ready` về mức đầy đủ thông tin. Ca trực cần chọn **ba sự cố khác nhau xử lý theo thứ tự**. Chỉ các phiếu `blocker` hoặc `high` có `ready=True` được dùng; chẳng hạn C12 chưa sẵn sàng và cũng thuộc loại thông thường. Điều phối viên cần số lịch ba vị trí hợp lệ và một lịch đầu để đối chiếu.

Vì sao chọn cách này? Lịch xử lý gồm ba vị trí đầu–giữa–cuối, nên thứ tự C01 rồi C04 rồi C07 khác C07 rồi C04 rồi C01, dù chọn cùng ba sự cố. Chương trình duyệt 18 hồ sơ, giữ các dòng vừa **đủ thông tin (`ready`)** vừa ở mức `blocker` hoặc `high`; không mặc định mọi phiếu là hợp lệ. `permutations(eligible, 3)` liệt kê các lịch gồm ba sự cố **không lặp**; `math.perm` đếm để kiểm tra. Cách này mới chỉ liệt kê **phương án thứ tự** theo tiêu chí đã cho, chưa tính thời lượng xử lý hay ưu tiên `blocker` đứng trước `high` vì ví dụ không đặt thêm quy định ấy.

Quy trình:

1. Duyệt 18 hồ sơ, chỉ giữ ca khẩn cấp có cờ sẵn sàng.
2. Đếm các lịch có thứ tự gồm ba ca không lặp.
3. Liệt kê lịch đầu, kiểm số lịch với `math.perm`.

No Note — Không chú thích

```
import math
from itertools import permutations
incidents = [{"id":"C01","severity":"blocker","ready":True},
{"id":"C02","severity":"normal","ready":True},{"id":"C03","severity":"normal","ready":True},
{"id":"C04","severity":"high","ready":True},{"id":"C05","severity":"normal","ready":True},
{"id":"C06","severity":"normal","ready":True},{"id":"C07","severity":"blocker","ready":True},
{"id":"C08","severity":"normal","ready":True},{"id":"C09","severity":"normal","ready":True},
{"id":"C10","severity":"high","ready":True},{"id":"C11","severity":"normal","ready":True},
{"id":"C12","severity":"normal","ready":False},{"id":"C13","severity":"blocker","ready":True},
{"id":"C14","severity":"normal","ready":True},{"id":"C15","severity":"normal","ready":True},
{"id":"C16","severity":"high","ready":True},{"id":"C17","severity":"normal","ready":True},
{"id":"C18","severity":"normal","ready":True}]
eligible = []
for row in incidents:
    if row["ready"] and row["severity"] in ("blocker", "high"):
        eligible.append(row["id"])
schedules = list(permutations(eligible, 3))
if len(incidents) != 18:
    raise ValueError("Cần đủ 18 sự cố đã ghi nhận.")
if len(schedules) != math.perm(len(eligible), 3):
    raise ValueError("Số lịch ba ca phải khớp số cách xếp có thứ tự.")
print("Sự cố:", len(incidents))
print("Đủ tiêu chí:", len(eligible))
print("Lịch 3 ca:", len(schedules))
print("Lịch đầu:", ", ".join(schedules[0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lọc sự cố nghiêm trọng đã sẵn sàng trong 18 bản ghi rồi xếp ba vị trí xử lý có thứ tự.
# Dữ liệu: 18 dict sự cố id/severity/ready; chỉ blocker hoặc high và ready=True được lập lịch.
# math.perm() đếm lịch có thứ tự theo số ứng viên sự cố đủ điều kiện.
import math
# permutations(iterable,r) tạo các tuple r phần tử có thứ tự và không lặp ID trong một lịch.
from itertools import permutations
# 18 sự cố, severity có blocker/high/normal, ready là cờ đã sẵn sàng.
incidents = [{"id":"C01","severity":"blocker","ready":True},
{"id":"C02","severity":"normal","ready":True},{"id":"C03","severity":"normal","ready":True},
{"id":"C04","severity":"high","ready":True},{"id":"C05","severity":"normal","ready":True},
{"id":"C06","severity":"normal","ready":True},{"id":"C07","severity":"blocker","ready":True},
{"id":"C08","severity":"normal","ready":True},{"id":"C09","severity":"normal","ready":True},
{"id":"C10","severity":"high","ready":True},{"id":"C11","severity":"normal","ready":True},
{"id":"C12","severity":"normal","ready":False},{"id":"C13","severity":"blocker","ready":True},
{"id":"C14","severity":"normal","ready":True},{"id":"C15","severity":"normal","ready":True},
{"id":"C16","severity":"high","ready":True},{"id":"C17","severity":"normal","ready":True},
{"id":"C18","severity":"normal","ready":True}]
# List sự cố thỏa mức nghiêm trọng và sẵn sàng.
eligible = []
for row in incidents:
    # and yêu cầu cả ready=True và mức thuộc hai nhãn cần ưu tiên.
    if row["ready"] and row["severity"] in ("blocker", "high"):
        # append() chỉ thêm mã sự cố hợp lệ.
        eligible.append(row["id"])
# Tạo tất cả lịch có ba vị trí thứ tự (ca 1,2,3); đổi vị trí tạo lịch khác; list() hiện thực toàn
bộ iterator.
schedules = list(permutations(eligible, 3))
# Đối chiếu nguồn phải đủ 18 sự cố.
if len(incidents) != 18:
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Cần đủ 18 sự cố đã ghi nhận.")
# perm() đếm số lịch lý thuyết để so với len(schedules) đã liệt kê.
if len(schedules) != math.perm(len(eligible), 3):
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Số lịch ba ca phải khớp số cách xếp có thứ tự.")
print("Sự cố:", len(incidents))
```

```
print("Đủ tiêu chí:", len(eligible))
print("Lịch 3 ca:", len(schedules))
# [0] lấy lịch ba ca đầu tiên, join() ghép ba mã bằng dấu phẩy.
print("Lịch đầu:", ", ".join(schedules[0]))
```

Kết quả khi chạy:

```
Sự cố: 18
Đủ tiêu chí: 6
Lịch 3 ca: 120
Lịch đầu: C01, C04, C07
```

Ba vị trí là ba lượt thực hiện khác nhau nên dùng hoán vị; nhân viên không được đưa ca chưa có dữ liệu vào danh sách xử lý.

Tình huống 5 — Cấp 5: Chọn ban phản biện 3 nhóm từ 20 người có ràng buộc

Từ vựng trong mục: Constrained combination — tổ hợp có ràng buộc; **Cross-check** — kiểm tra chéo.

Tình huống thực tế: Ban chuyên môn có **20 hồ sơ U01–U20**, mỗi người thuộc một nhóm **A, B hoặc C** và có cờ `ready`. U06 và U15 chưa sẵn sàng. Ban cần chọn **đúng ba người ngang vai**, gồm **một người nhóm A, một người nhóm B và một người nhóm C**; không được chọn hai người cùng nhóm hay đếm ba người đó nhiều lần do đổi thứ tự. Người điều phối muốn biết số ứng viên đủ điều kiện từng nhóm và tổng số ban hợp lệ.

Vì sao chọn cách này? Trước tiên bỏ người `ready=False` vì chưa thể tham gia. Sau đó, **một ban hợp lệ** phải có tập tên nhóm đúng bằng `{A, B, C}`: điều này bắt buộc đủ ba nhóm, không có hai người cùng nhóm. `combinations(ready, 3)` liệt kê các nhóm ba người **không xét thứ tự** rồi kiểm tra điều kiện; chương trình còn đếm số người sẵn sàng của từng nhóm. Nếu chọn độc lập đúng **một người từ mỗi nhóm**, số cách là tích số lựa chọn của A, B và C, dùng để đối chiếu kết quả liệt kê. Chỉ dùng tổ hợp của tất cả người sẵn sàng sẽ tính cả ban có hai người nhóm A, nên sai yêu cầu.

Quy trình:

1. Tách 20 hồ sơ theo nhóm A/B/C và trạng thái sẵn sàng.
2. Liệt kê mọi tổ ba người **không thứ tự** và chỉ giữ đội đủ ba nhóm.
3. So số ban hợp lệ với số lựa chọn từ mỗi nhóm, xác minh không chứa ứng viên chưa sẵn sàng.

No Note — Không chú thích

```
from itertools import combinations
people = [{"id":"U01","team":"A","ready":True},{"id":"U02","team":"B","ready":True},
{"id":"U03","team":"C","ready":True},{"id":"U04","team":"A","ready":True},
{"id":"U05","team":"B","ready":True},{"id":"U06","team":"C","ready":False},
{"id":"U07","team":"A","ready":True},{"id":"U08","team":"B","ready":True},
{"id":"U09","team":"C","ready":True},{"id":"U10","team":"A","ready":True},
{"id":"U11","team":"B","ready":True},{"id":"U12","team":"C","ready":True},
{"id":"U13","team":"A","ready":True},{"id":"U14","team":"B","ready":True},
{"id":"U15","team":"C","ready":False},{"id":"U16","team":"A","ready":True},
{"id":"U17","team":"B","ready":True},{"id":"U18","team":"C","ready":True},
{"id":"U19","team":"A","ready":True},{"id":"U20","team":"B","ready":True}]
ready = []
for person in people:
    if person["ready"]:
        ready.append(person)
valid = []
for team in combinations(ready, 3):
```

```
    team_names = set()
    for person in team:
        team_names.add(person["team"])
    if team_names == {"A", "B", "C"}:
        valid.append(team)
counts = {"A": 0, "B": 0, "C": 0}
for person in ready:
    counts[person["team"]] += 1
if len(people) != 20:
    raise ValueError("Cần đủ 20 hồ sơ ban đầu.")
if len(valid) != counts["A"] * counts["B"] * counts["C"]:
    raise ValueError("Số ban ba nhóm phải bằng số cách chọn một người mỗi nhóm.")
print("Hồ sơ:", len(people))
print("Sẵn sàng:", len(ready))
print("Người theo nhóm:", counts["A"], counts["B"], counts["C"])
print("Ban ba nhóm:", len(valid))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lập ban ba người từ 20 hồ sơ, mỗi người ở một nhóm A/B/C và đều sẵn sàng.
# Dữ liệu: 20 dict (id,team,ready), hai người không sẵn sàng; ban hợp lệ có đúng một người nhóm A,
B, C.
# combinations() liệt kê bộ ba KHÔNG phân thứ tự từ những người được chọn.
from itertools import combinations
# 20 hồ sơ; team A/B/C và ready True/False của từng người.
people = [{"id":"U01","team":"A","ready":True},{"id":"U02","team":"B","ready":True},
{"id":"U03","team":"C","ready":True},{"id":"U04","team":"A","ready":True},
{"id":"U05","team":"B","ready":True},{"id":"U06","team":"C","ready":False},
{"id":"U07","team":"A","ready":True},{"id":"U08","team":"B","ready":True},
{"id":"U09","team":"C","ready":True},{"id":"U10","team":"A","ready":True},
{"id":"U11","team":"B","ready":True},{"id":"U12","team":"C","ready":True},
{"id":"U13","team":"A","ready":True},{"id":"U14","team":"B","ready":True},
{"id":"U15","team":"C","ready":False},{"id":"U16","team":"A","ready":True},
{"id":"U17","team":"B","ready":True},{"id":"U18","team":"C","ready":True},
{"id":"U19","team":"A","ready":True},{"id":"U20","team":"B","ready":True}]
# Chỉ lưu người sẵn sàng trước khi tổ hợp.
ready = []
for person in people:
    # Cờ False khiến hồ sơ bị bỏ khỏi list ready.
    if person["ready"]:
        # append() giữ cả dict để đọc thuộc tính team khi ghép nhóm.
        ready.append(person)
valid = []
# Duyệt từng bộ ba không phân vai; iterator sinh lần lượt, không cần giữ tất cả bộ ba trước.
for team in combinations(ready, 3):
    # set() dùng để kiểm ba loại nhóm không trùng.
    team_names = set()
    for person in team:
        # add() đưa tên nhóm vào tập, nhóm lặp sẽ không tăng số phần tử.
        team_names.add(person["team"])
    # == với tập ba nhãn đảm bảo mỗi nhóm góp đúng một người khi tuple chỉ có ba người.
    if team_names == {"A", "B", "C"}:
        # append() lưu bộ ba đáp ứng đúng ba nhóm.
        valid.append(team)
# Dictionary đếm số người sẵn sàng từng nhóm A/B/C.
counts = {"A": 0, "B": 0, "C": 0}
for person in ready:
    # Tra số người theo khóa nhóm rồi tăng 1.
    counts[person["team"]] += 1
# Kiểm đủ 20 hồ sơ nguồn.
if len(people) != 20:
    # raise ValueError báo dữ liệu không đạt yêu cầu và dừng trước khi in kết quả đếm sai.
    raise ValueError("Cần đủ 20 hồ sơ ban đầu.")
# Nguyên lý nhân: chọn độc lập một người từ mỗi nhóm, số ban bằng lượng A × lượng B × lượng C;
không phải hoán vị ba vai.
```

```
if len(valid) != counts["A"] * counts["B"] * counts["C"]:
    # raise ValueError báo dữ liệu không đạt yêu câù và dừng trước khi in kêt quả đêm sai.
    raise ValueError("Sô´ban ba nhóm phải bằng sô´cách chọn một người môĩ nhóm.")
print("Hô`sơ:", len(people))
print("Săñ sàng:", len(ready))
print("Người theo nhóm:", counts["A"], counts["B"], counts["C"])
print("Ban ba nhóm:", len(valid))
```

Kết quả khi chạy:

```
Hô`sơ: 20
Săñ sàng: 18
Người theo nhóm: 7 7 4
Ban ba nhóm: 196
```

Ở cấp này, không thể chỉ gọi `comb(20,3)`: hai điều kiện **sẵn sàng** và **mỗi nhóm đúng một người** quyết định số đội thực sự khả thi.

6. Sai lầm thường gặp

- Phân biệt bài toán **có thứ tự** (`perm`) với **không xét thứ tự** (`comb`).
- `itertools.permutations(items,2)` chỉ tạo các cặp có thứ tự, không phải sắp xếp toàn bộ ba phần tử; không truyền k để lấy toàn bộ.
- Cả `math.perm` và `math.comb` giả định các phần tử phân biệt và không chọn trùng một phần tử.
- `math.factorial` cần số nguyên không âm; `0! = 1`.
- `itertools` để **liệt kê** các phương án; `math` giúp **đếm** khi số phương án quá lớn không nên liệt kê.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. When should permutations be used? | Khi nào dùng hoán vị/chỉnh hợp?

A. When order matters | Khi thứ tự quan trọng

B. When order does not matter | Khi thứ tự không quan trọng

C. The product 1×2×...×n | Tích 1×2×...×n

D. The number of unordered k-subsets from n distinct items | Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: A

Giải thích: Đổi thứ tự người được chọn tạo cách sắp khác.

Q2. When should combinations be used? | Khi nào dùng tổ hợp?

A. When order matters | Khi thứ tự quan trọng

B. When order does not matter | Khi thứ tự không quan trọng

C. The product 1×2×...×n | Tích 1×2×...×n

D. The number of unordered k-subsets from n distinct items | Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: B

Giải thích: Nhóm không đổi khi thành viên đổi chỗ.

Q3. What is factorial n!? | Giai thừa n! là gì?

A. When order matters | Khi thứ tự quan trọng

B. When order does not matter | Khi thứ tự không quan trọng

C. The product 1×2×…×n | Tích 1×2×…×n

D. The number of unordered k-subsets from n distinct items | Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: C

Giải thích: Giai thừa đếm cách sắp xếp n phần tử phân biệt.

Q4. What is a binomial coefficient C(n,k)? | Hệ số nhị thức C(n,k) là gì?

A. When order matters | Khi thứ tự quan trọng

B. When order does not matter | Khi thứ tự không quan trọng

C. The product 1×2×…×n | Tích 1×2×…×n

D. The number of unordered k-subsets from n distinct items | Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: D

Giải thích: C(n,k) còn là hệ số trong khai triển (a+b)^n.

Q5. Which function counts ordered selections? | Hàm nào đếm cách chọn có thứ tự?

A. `math.perm(n, k)` | `math.perm(n, k)`

B. `math.comb(n, k)` | `math.comb(n, k)`

C. `math.factorial(n)` | `math.factorial(n)`

D. `itertools.permutations(items)` | `itertools.permutations(items)`

Đáp án: A

Giải thích: perm đếm cách chọn có thứ tự không lặp.

Q6. Which function counts unordered selections? | Hàm nào đếm cách chọn không thứ tự?

A. `math.perm(n, k)` | `math.perm(n, k)`

B. `math.comb(n, k)` | `math.comb(n, k)`

C. `math.factorial(n)` | `math.factorial(n)`

D. `itertools.permutations(items)` | `itertools.permutations(items)`

Đáp án: B

Giải thích: comb đếm các nhóm khác nhau, không xét sắp xếp.

Q7. Which function calculates n!? | Hàm nào tính n!?

A. `math.perm(n, k)` | `math.perm(n, k)`

B. `math.comb(n, k)` | `math.comb(n, k)`

C. `math.factorial(n)` | `math.factorial(n)`

D. `itertools.permutations(items)` | `itertools.permutations(items)`

Đáp án: C

Giải thích: factorial nhân các số nguyên từ một tới n.

Q8. Which function lists all ordered arrangements? | Hàm nào liệt kê các cách sắp xếp có thứ tự?

A. `math.perm(n, k)` | `math.perm(n, k)`

B. `math.comb(n, k)` | `math.comb(n, k)`

C. `math.factorial(n)` | `math.factorial(n)`

D. `itertools.permutations(items)` | `itertools.permutations(items)`

Đáp án: D

Giải thích: Không truyền r, permutations liệt kê toàn bộ thứ tự.

Q9. What is the value of 0!? | 0! bằng bao nhiêu?

A. 1 | 1

B. 6 | 6

C. 10 pairs | 10 cặp

D. 12 pairs | 12 cặp

Đáp án: A

Giải thích: Dãy rỗng có một cách sắp xếp.

Q10. How many full orderings of three distinct items exist? | Ba phần tử phân biệt có bao nhiêu thứ tự đầy đủ?

A. 1 | 1

B. 6 | 6

C. 10 pairs | 10 cặp

D. 12 pairs | 12 cặp

Đáp án: B

Giải thích: 3! = 3×2×1 = 6.

Q11. How many unordered pairs can be chosen from five distinct items? | Chọn cặp không thứ tự từ năm phần tử có bao nhiêu cách?

A. 1 | 1

B. 6 | 6

C. 10 pairs | 10 cặp

D. 12 pairs | 12 cặp

Đáp án: C

Giải thích: Chọn hai từ năm không xét thứ tự có C(5,2)=10 cặp.

Q12. How many ordered pairs can be formed from four distinct items without repetition? | Từ bốn phần tử phân biệt có bao nhiêu cặp có thứ tự không lặp?

A. 1 | 1

B. 6 | 6

C. 10 pairs | 10 cặp

D. 12 pairs | 12 cặp

Đáp án: D

Giải thích: 4×3 = 12.

Q13. What identity relates combinations and permutations? | Công thức nào liên hệ tổ hợp và chỉnh hợp?

A. P(n,k)=C(n,k)×k! | P(n,k)=C(n,k)×k!

B. 1 | 1

C. n | n

D. Each inner entry sums two entries directly above | Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: A

Giải thích: Mỗi nhóm k người có k! cách sắp.

Q14. What is C(n,0)? | C(n,0) bằng gì?

A. P(n,k)=C(n,k)×k! | P(n,k)=C(n,k)×k!

B. 1 | 1

C. n | n

D. Each inner entry sums two entries directly above | Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: B

Giải thích: Có một cách chọn tập rỗng.

Q15. What is C(n,n−1)? | C(n,n−1) bằng gì?

A. P(n,k)=C(n,k)×k! | P(n,k)=C(n,k)×k!

B. 1 | 1

C. n | n

D. Each inner entry sums two entries directly above | Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: C

Giải thích: Chọn n−1 từ n tương đương chọn một phần tử bỏ lại nên có n cách.

Q16. How are entries in Pascal's triangle obtained? | Các ô trong tam giác Pascal được tạo ra sao?

A. P(n,k)=C(n,k)×k! | P(n,k)=C(n,k)×k!

B. 1 | 1

C. n | n

D. Each inner entry sums two entries directly above | Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: D

Giải thích: Hệ thức Pascal cộng hai hệ số ở hàng trước.

Q17. What does `itertools.combinations(items,2)` list? | `itertools.combinations(items,2)` liệt kê gì?

A. Unordered two-item groups | Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements | Cặp hai phần tử có thứ tự

C. It returns zero | Hàm trả về 0

D. The number grows factorially | Số cách tăng theo giai thừa

Đáp án: A

Giải thích: Hàm tạo mỗi cặp khác nhau đúng một lần.

Q18. What does `itertools.permutations(items,2)` list? | `itertools.permutations(items,2)` liệt kê gì?

A. Unordered two-item groups | Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements | Cặp hai phần tử có thứ tự

C. It returns zero | Hàm trả về 0

D. The number grows factorially | Số cách tăng theo giai thừa

Đáp án: B

Giải thích: Đổi thứ tự hai phần tử tạo kết quả khác.

Q19. What happens if k>n in `math.comb(n,k)` for nonnegative integers? | Nếu k>n trong math.comb với số nguyên không âm thì sao?

A. Unordered two-item groups | Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements | Cặp hai phần tử có thứ tự

C. It returns zero | Hàm trả về 0

D. The number grows factorially | Số cách tăng theo giai thừa

Đáp án: C

Giải thích: Không thể chọn k phần tử nếu tổng phần tử ít hơn k.

Q20. Why avoid materializing all permutations for large n? | Vì sao tránh liệt kê toàn bộ hoán vị khi n lớn?

A. Unordered two-item groups | Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements | Cặp hai phần tử có thứ tự

C. It returns zero | Hàm trả về 0

D. The number grows factorially | Số cách tăng theo giai thừa

Đáp án: D

Giải thích: n! tăng nhanh khiến không thể lưu hoặc in hết.

8. Nối sang bài tiếp

`Nguyên lý đếm → hoán vị/tổ hợp → phân phối nhị thức.`

PY-23 — Đồ thị, danh sách kề và Jaccard similarity

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Graph — đồ thị **Vertex** — đỉnh.

`Set → Graph → Similarity → Gợi ý bạn học`

Học xong dùng để làm gì? Dùng cạnh và nút mô tả liên kết; so nhãn/sở thích bằng hệ số tương đồng Jaccard.

Kết nối kiến thức tiếp theo: PY-24 chuyển đặc trưng đối tượng thành vector có thể đo độ dài và hướng.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Graph	đồ thị	Các đỉnh và cạnh mô tả quan hệ giữa đối tượng
Vertex	đỉnh	Một đối tượng được biểu diễn trong đồ thị
Edge	cạnh	Liên hệ hoặc đường nối giữa hai đỉnh
Jaccard	độ tương đồng	Tỷ lệ số phần tử giao trên hợp giữa hai tập

3. Hàm và cấu trúc dữ liệu liên quan

Từ vựng trong mục: Function — hàm; **Adjacency list** — danh sách kề.

Hàm / tham số	Dùng làm gì
`dict[str,set[str]]`	Danh sách kề đơn giản: key là đỉnh, value là các đỉnh nối
`networkx.Graph()/DiGraph()`	Đồ thị vô hướng/có hướng, cần cài networkx
`nx.shortest_path(G,a,b)`	Tìm đường nối ngắn nhất nếu có kết nối
`len(a & b) / len(a | b)`	Tương đồng Jaccard khi hợp khác rỗng

4. Khái niệm liên quan

Đồ thị (graph) mô tả mạng quan hệ: **đỉnh (vertex)** là đối tượng, **cạnh (edge)** là mối nối giữa đối tượng. Trong mạng học viên, mỗi người là một đỉnh và một cạnh biểu thị quan hệ bạn bè. Ta biểu diễn mạng bằng **danh sách kề** rồi liên hệ với **Jaccard**, thước đo phần sở thích chung so với toàn bộ sở thích của hai người.

4.1. Graph và adjacency list — Mạng và danh sách kề

Từ vựng trong mục: Graph — đồ thị; **Adjacency list** — danh sách kề.

Một **đồ thị** gồm tập **đỉnh** và tập **cạnh** nối chúng. Để biểu diễn mạng bạn bè An–Bình–Lan mà không cần vẽ, dùng Dictionary: **khóa** là tên người, **Set giá trị** là những người kết nối trực tiếp với họ. Nếu quan hệ hai chiều, cả hai đầu phải ghi tên nhau. Ví dụ An kết bạn với Bình và Lan, còn Bình/Lan đều kết nối ngược lại An.

No Note — Không chú thích

```
graph = {
    'An': {'Binh', 'Lan'},
    'Binh': {'An'},
    'Lan': {'An'},
}
print(sorted(graph['An']))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Biểu diễn quan hệ bạn bè bằng dictionary danh sách kề và đọc bạn của An.
# Dữ liệu: Mỗi khóa là một người, giá trị là set những người nối với họ bằng cạnh bạn bè.
# Dictionary dạng {đỉnh: set(đỉnh_kề)} là danh sách kề; An nối Binh và Lan.
graph = {
    'An': {'Binh', 'Lan'},
    'Binh': {'An'},
    'Lan': {'An'},
}
# Tra danh sách kề của An; sorted() chuyển set thành list mã có thứ tự ổn định để in.
print(sorted(graph['An']))
```

Kết quả khi chạy:

```
['Binh', 'Lan']
```

An có hai kết nối trực tiếp là Bình và Lan. Dictionary giúp tra nhanh những người kề với một đỉnh.

4.2. Vertex — Đỉnh đại diện một đối tượng

Từ vựng trong mục: Vertex — đỉnh; **Node** — nút.

Đỉnh là một đối tượng trong mạng: một học viên, trạm xe hoặc trang web. Tên đỉnh chỉ là định danh, không nói lên nó đã kết nối với ai. Trong ví dụ có hai học viên An và Bình, ta dùng Set chứa hai tên làm tập đỉnh; `len(vertices)` đếm đối tượng khác nhau, không phải số liên kết.

No Note — Không chú thích

```
vertices = {'An', 'Binh'}
print(len(vertices))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm đỉnh là những đối tượng độc lập trong đồ thị.
# Dữ liệu: Hai đỉnh An và Binh, tập set loại tên trùng nếu có.
# Set chứa đỉnh duy nhất, không có cạnh hay hướng trong ví dụ nhỏ này.
```

```
vertices = {'An', 'Binh'}
# len() đếm số đỉnh khác nhau, không đếm số quan hệ.
print(len(vertices))
```

Kết quả khi chạy:

```
2
```

Mạng có hai đỉnh khác nhau; chưa suy ra có cạnh nào giữa họ.

4.3. Edge — Cạnh có hướng và vô hướng

Từ vựng trong mục: Edge — cạnh; **Directed** — có hướng.

Cạnh biểu diễn mối quan hệ giữa hai đỉnh. Với bạn bè hai chiều, cạnh An–Bình là **vô hướng** và chỉ cần tính một lần; với hành động A gửi thông báo tới B, cạnh A→B **có hướng**, không tự suy ra B→A. Danh sách kề có hướng `{'A':{'B'},'B':set()}` diễn tả A gửi B, nhưng B không gửi lại A.

No Note — Không chú thích

```
graph = {'A': {'B'}, 'B': set()}
print('B' in graph['A'], 'A' in graph['B'])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt cạnh có hướng A→B với cạnh ngược B→A không tồn tại.
# Dữ liệu: graph['A'] có B, graph['B'] là set rỗng; kết nối chỉ đi một chiều.
# Dictionary danh sách kề cho đồ thị có hướng; set() tạo danh sách kề rỗng của B.
graph = {'A': {'B'}, 'B': set()}
# in kiểm có cung A→B hay không (True); kiểm ngược A trong graph['B'] là False nên KHÔNG mặc định
vô hướng.
print('B' in graph['A'], 'A' in graph['B'])
```

Kết quả khi chạy:

```
True False
```

Quan hệ A→B tồn tại; chiều ngược lại không tồn tại. Cách lưu cạnh phải phù hợp loại đồ thị.

4.4. Jaccard similarity — Độ giống nhau của hai tập

Từ vựng trong mục: Jaccard — tương đồng Jaccard; **Union/intersection** — hợp/giao.

Jaccard đo mức trùng nhau của hai tập: `J(A,B) = |A ∩ B| / |A ∪ B|`, trong khoảng **0–1** khi hợp không rỗng. Người A biết `{Python, AI}`, người B biết `{Python, SQL}`: giao có 1 kỹ năng `Python`, hợp có 3 kỹ năng, nên độ tương đồng là `1/3 ≈ 0.333`. Không dùng số lượng phần tử trùng trực tiếp làm tỷ lệ. Với cả hai tập rỗng phải định nghĩa riêng quy ước để tránh chia 0.

No Note — Không chú thích

```
a = {'Python', 'AI'}
b = {'Python', 'SQL'}
print(round(len(a & b) / len(a | b), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính độ tương đồng Jaccard giữa hai bộ kỹ năng bằng giao chia hợp.
# Dữ liệu: a={Python,AI}, b={Python,SQL}; có một kỹ năng chung và ba kỹ năng khác nhau trong hợp.
# Set kỹ năng A, mỗi kỹ năng chỉ xuất hiện một lần.
a = {'Python', 'AI'}
# Set kỹ năng B, giao với a ở Python.
b = {'Python', 'SQL'}
# & lấy tập chung (1), | lấy tất cả kỹ năng không trùng (3), chia để có Jaccard 1/3; round(...,3)
chỉ làm tròn khi in.
print(round(len(a & b) / len(a | b), 3))
```

Kết quả khi chạy:

```
0.333
```

Hai hồ sơ giống nhau khoảng 33,3% theo Jaccard vì chỉ chung một trên ba kỹ năng khác nhau.

5. Năm tình huống Graph/Jaccard — từ bạn trực tiếp đến đường đi và xếp hạng

Tình huống 1 — Cấp 1: Xây danh sách kề của 10 người từ 12 kết nối

Từ vựng trong mục: Graph — đồ thị; **Adjacency list** — danh sách kề; **Degree** — bậc đỉnh.

Tình huống thực tế: Cộng đồng học tập gồm **10 học viên A đến J**. Danh sách có **12 mối quan hệ trực tiếp**; mỗi dòng lưu hai mã, ví dụ A–B có nghĩa A và B có thể liên hệ với nhau **theo cả hai chiều**. Nhân viên cần biết A liên hệ trực tiếp được với những ai, A có bao nhiêu bạn trực tiếp và toàn mạng có bao nhiêu mối quan hệ, không tính A–B và B–A như hai quan hệ riêng.

Vì sao chọn cách này? Đồ thị là cách biểu diễn người bằng **đỉnh** và quan hệ trực tiếp bằng **cạnh**. Vì liên hệ trong bài là **hai chiều**, khi có A–B, chương trình thêm B vào set bạn của A **và** thêm A vào set bạn của B. Dictionary `graph` ánh xạ mỗi mã với tập bạn trực tiếp; đây là **danh sách kề**, không phải bảng điểm. Set không giữ một người hai lần, còn `sorted()` giúp in mã đúng thứ tự. Khi cộng số bạn của mọi người, mỗi quan hệ được tính ở **hai đầu**, nên tổng số lượt nối gấp đôi số cạnh thật; bước kiểm tra này phát hiện việc quên ghi một chiều.

Quy trình:

1. Tạo 10 đỉnh, mỗi đỉnh có Set hàng xóm rỗng.
2. Duyệt 12 cạnh và thêm hai chiều.
3. Đếm hàng xóm A và kiểm tra tổng bậc bằng hai lần số cạnh.

No Note — Không chú thích

```
nodes = list("ABCDEFGHIJ")
edges = [["A","B"],["A","C"],["A","D"],["B","E"],["C","E"],["D","F"],["E","F"],["F","G"],["G","H"],
["H","I"],["I","J"],["C","J"]]
graph = {}
for node in nodes:
    graph[node] = set()
for left, right in edges:
    graph[left].add(right)
    graph[right].add(left)
```

```
if len(nodes) != 10 or len(edges) != 12:
    raise ValueError("Cần đúng 10 người và 12 quan hệ.")
if sum(map(len, graph.values())) != 2 * len(edges):
    raise ValueError("Đồ thị vô hướng phải ghi hai đầu mỗi cạnh.")
print("Người:", len(nodes))
print("Quan hệ:", len(edges))
print("Bạn của A:", ", ".join(sorted(graph["A"])))
print("Số bạn A:", len(graph["A"]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo đồ thị vô hướng từ 10 người và 12 cạnh, đếm bậc của A và kiểm hai chiều.
# Dữ liệu: 10 đỉnh A..J, 12 cặp quan hệ [người đầu, người cuối]; cạnh bạn bè không có hướng.
# list() tách chuỗi thành 10 tên đỉnh A..J.
nodes = list("ABCDEFGHIJ")
# Mỗi list con là một cạnh vô hướng giữa hai người; số cạnh thực là 12.
edges = [["A","B"],["A","C"],["A","D"],["B","E"],["C","E"],["D","F"],["E","F"],["F","G"],["G","H"],
["H","I"],["I","J"],["C","J"]]
# Dictionary lưu mỗi đỉnh và set hàng xóm của nó.
graph = {}
for node in nodes:
    # Tạo set kề rỗng trước khi thêm quan hệ, tránh KeyError.
    graph[node] = set()
# Tách hai đầu của mỗi cạnh.
for left, right in edges:
    # add() thêm đầu phải vào danh sách kề của đầu trái.
    graph[left].add(right)
    # Đồ thị VÔ HƯỚNG cần thêm chiều ngược lại; mỗi cạnh đóng góp hai lần vào tổng bậc.
    graph[right].add(left)
# Kiểm 10 đỉnh và 12 cạnh trước khi in.
if len(nodes) != 10 or len(edges) != 12:
    # ValueError báo không đạt điều kiện sở bản ghi, cạnh hoặc tính nhất quán của đồ thị.
    raise ValueError("Cần đúng 10 người và 12 quan hệ.")
# graph.values() là mọi set kề; map(len, ...) tính số hàng xóm từng đỉnh; sum() cộng tổng bậc, phải
bằng 2×số cạnh.
if sum(map(len, graph.values())) != 2 * len(edges):
    # ValueError báo không đạt điều kiện sở bản ghi, cạnh hoặc tính nhất quán của đồ thị.
    raise ValueError("Đồ thị vô hướng phải ghi hai đầu mỗi cạnh.")
print("Người:", len(nodes))
print("Quan hệ:", len(edges))
# Set kề A không bảo đảm thứ tự; sorted() xếp chữ cái, join() ghép tên để in.
print("Bạn của A:", ", ".join(sorted(graph["A"])))
# len() trả bậc của đỉnh A, tức số hàng xóm trực tiếp.
print("Số bạn A:", len(graph["A"]))
```

Kết quả khi chạy:

```
Người: 10
Quan hệ: 12
Bạn của A: B, C, D
Số bạn A: 3
```

Một dòng cạnh là một quan hệ giữa hai học viên; khi danh sách kề ghi hai chiều, không được hiểu rằng số quan hệ đã tăng gấp đôi.

Tình huống 2 — Cấp 2: So Jaccard từ 12 hồ sơ kỹ năng có nhãn

Từ vựng trong mục: Jaccard similarity — tương đồng tập hợp; **Intersection** — phần giao.

Tình huống thực tế: Ứng dụng ghép nhóm có **12 hồ sơ kỹ năng H01–H12**; mỗi hồ sơ ghi những kỹ năng người đó khai, như Python, AI, SQL, Math, Data và Cloud. H01 khai `Python` hai lần nhưng đó vẫn là **một kỹ năng**. Người quản lý chọn H01 làm hồ sơ mẫu, muốn biết mức giống nhau về kỹ năng giữa H01 và H02, đồng thời tìm học viên có bộ kỹ năng **giống H01 nhất** trong 11 người còn lại; nếu bằng điểm phải quyết định thứ tự ổn định.

Vì sao chọn cách này? Jaccard đo mức độ giống nhau giữa **hai tập**: xem chúng chia sẻ bao nhiêu kỹ năng so với tất cả kỹ năng khác nhau của cả hai người. `set()` loại `Python` bị lặp trong hồ sơ H01 trước khi so. Phép **giao** `&` cho kỹ năng chung, phép **hợp** `|` cho các kỹ năng thuộc ít nhất một người; ví dụ chia sẻ một kỹ năng trong tổng ba kỹ năng khác nhau nghĩa là giống một phần ba. Nếu hai tập đều rỗng, chương trình trả 0 thay vì chia cho 0. Danh sách ứng viên được sắp theo điểm từ cao xuống; nếu đồng điểm, mã học viên nhỏ hơn được xếp trước, tránh kết quả thay đổi do thứ tự dữ liệu.

Quy trình:

1. Tạo 12 dòng hồ sơ, mỗi hồ sơ là tập kỹ năng.
2. Viết hàm Jaccard dùng độ lớn phần giao và hợp.
3. Tính điểm H01–H02 và tìm người gần nhất với H01.

No Note — Không chú thích

```
profiles = [{"id":"H01","skills":["Python","AI","Python"]},{"id":"H02","skills":["AI","SQL"]},
{"id":"H03","skills":["SQL","Math"]},{"id":"H04","skills":["Math","Data"]},{"id":"H05","skills":
["Data","Cloud","Python"]},{"id":"H06","skills":["Cloud","Python"]},{"id":"H07","skills":
["Python","AI"]},{"id":"H08","skills":["AI","SQL"]},{"id":"H09","skills":["SQL","Math","Python"]},
{"id":"H10","skills":["Math","Data"]},{"id":"H11","skills":["Data","Cloud"]},{"id":"H12","skills":
["Cloud","Python"]}]
def jaccard(first, second):
    a, b = set(first), set(second)
    union = a | b
    if union:
        return len(a & b) / len(union)
    return 0.0
reference = profiles[0]["skills"]
score = jaccard(reference, profiles[1]["skills"])
ranked = []
for person in profiles[1:]:
    ranked.append((person["id"], jaccard(reference, person["skills"])))
def sort_by_score(row):
    return (-row[1], row[0])
ranked.sort(key=sort_by_score)
if len(profiles) != 12:
    raise ValueError("Cần đúng 12 hồ sơ kỹ năng.")
print("Hồ sơ:", len(profiles))
print(f"Jaccard H01-H02: {score:.2f}")
print("Gần H01 nhất:", ranked[0][0], f"{ranked[0][1]:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính Jaccard kỹ năng từ 12 hồ sơ và xếp người gần hồ sơ H01 nhất.
# Dữ liệu: 12 dict id/skills, một số kỹ năng bị lặp trong list; H01 là hồ sơ tham chiếu.
# 12 hồ sơ kỹ năng, mỗi row chứa id và list skills; Python có thể lặp trong một hồ sơ.
profiles = [{"id":"H01","skills":["Python","AI","Python"]},{"id":"H02","skills":["AI","SQL"]},
{"id":"H03","skills":["SQL","Math"]},{"id":"H04","skills":["Math","Data"]},{"id":"H05","skills":
["Data","Cloud","Python"]},{"id":"H06","skills":["Cloud","Python"]},{"id":"H07","skills":
["Python","AI"]},{"id":"H08","skills":["AI","SQL"]},{"id":"H09","skills":["SQL","Math","Python"]},
{"id":"H10","skills":["Math","Data"]},{"id":"H11","skills":["Data","Cloud"]},{"id":"H12","skills":
["Cloud","Python"]}]
# Hàm trả mức giống nhau từ 0 đến 1 dựa trên tập kỹ năng, không dựa số lần lặp.
```

```
def jaccard(first, second):
    # set() loại kỹ năng trùng nội bộ trước khi so sánh.
    a, b = set(first), set(second)
    # | hợp hai tập, chứa mọi kỹ năng có ở ít nhất một người, không trùng.
    union = a | b
    # Set rỗng được xem là False; kiểm trước chia để tránh chia cho 0 khi cả hai người không có kỹ
năng.
    if union:
        # & lấy giao kỹ năng chung; tỷ lệ số kỹ năng chung / số kỹ năng khác nhau trong hợp là
Jaccard.
        return len(a & b) / len(union)
    # Hai tập cùng rỗng được quy ước độ giống 0 trong ví dụ này (cần nêu rõ quy ước).
    return 0.0
# [0] lấy H01, ['skills'] lấy danh sách kỹ năng của H01.
reference = profiles[0]["skills"]
# [1] là H02; so H01 với H02 để minh họa điểm.
score = jaccard(reference, profiles[1]["skills"])
ranked = []
# [1:] bỏ hồ sơ H01 để không tự so H01 với chính mình.
for person in profiles[1:]:
    # append() lưu tuple (mã người, điểm Jaccard so với H01).
    ranked.append((person["id"], jaccard(reference, person["skills"])))
# Hàm key nhận một tuple (mã, điểm), quy định cách sắp xếp.
def sort_by_score(row):
    # row[1] là điểm; dấu âm đưa điểm lớn lên trước khi sort tăng; row[0] là mã phá hòa theo ABC.
    return (-row[1], row[0])
# sort(key=...) sắp trực tiếp list ranked theo khóa vừa định nghĩa.
ranked.sort(key=sort_by_score)
if len(profiles) != 12:
    # ValueError báo không đạt điều kiện số bản ghi, cạnh hoặc tính nhất quán của đồ thị.
    raise ValueError("Cần đúng 12 hồ sơ kỹ năng.")
print("Hồ sơ:", len(profiles))
print(f"Jaccard H01-H02: {score:.2f}")
# [0] lấy người đầu bảng xếp hạng, [0] tiếp lấy mã và [1] lấy điểm; :.2f in hai chữ số thập phân.
print("Gần H01 nhất:", ranked[0][0], f"{ranked[0][1]:.2f}")
```

Kết quả khi chạy:

```
Hồ sơ: 12
Jaccard H01-H02: 0.33
Gần H01 nhất: H07 1.00
```

Độ tương đồng 1 là hai tập kỹ năng giống hệt nhau; đây chỉ là độ tương đồng, **không** tự động chứng minh hai người hợp làm việc.

Tình huống 3 — Cấp 3: Phân hai cụm kết nối từ 15 cạnh của mạng học viên

Từ vựng trong mục: Connected component — thành phần liên thông; **Traversal** — duyệt đồ thị.

Tình huống thực tế: Một câu lạc bộ có **15 học viên A đến O** và **15 mối quan hệ hai chiều** được ghi nhận. Các mối liên hệ tạo hai nhóm tách biệt: một nhóm từ A đến G và một nhóm từ H đến O. Điều phối viên muốn biết có bao nhiêu **cụm bạn bè nối được với nhau qua một hoặc nhiều người trung gian**, mỗi cụm lớn bao nhiêu người và liệu A có thể liên hệ nối tiếp tới O hay không.

Vì sao chọn cách này? Thành phần liên thông là một nhóm đỉnh mà từ một người có thể lần theo các liên hệ hai chiều để đến bất kỳ người nào khác trong nhóm, kể cả phải qua bạn trung gian. **DFS/BFS** là hai cách đi thăm lần lượt các người có thể tới; dùng set `visited` đánh dấu người đã gặp để không đi lòng vòng hay đếm lại. Khi hết người có thể thăm, ta hoàn thành một cụm rồi bắt đầu từ người chưa thăm của

cụm khác. A và O nằm ở hai cụm riêng trong dữ liệu này nên không có đường nối; chỉ đếm số quan hệ chưa đủ để biết ai liên hệ được với ai.

Quy trình:

1. Dựng danh sách kề từ 15 cạnh; mỗi quan hệ ghi cả hai chiều.
2. Duyệt những đỉnh chưa thăm để thu từng cụm kết nối.
3. Sắp các kích thước cụm và kiểm tra A/O có cùng cụm không.

No Note — Không chú thích

```
edges = [["A","B"],["B","C"],["C","D"],["A","D"],["D","E"],["E","F"],["F","G"],["H","I"],["I","J"],
["J","K"],["K","L"],["H","L"],["L","M"],["M","N"],["N","O"]]
vertex_set = set()
for edge in edges:
    for name in edge:
        vertex_set.add(name)
vertices = sorted(vertex_set)
graph = {}
for vertex in vertices:
    graph[vertex] = set()
for a, b in edges:
    graph[a].add(b)
    graph[b].add(a)
seen, components = set(), []
for start in vertices:
    if start in seen:
        continue
    stack, members = [start], set()
    while stack:
        node = stack.pop()
        if node in seen:
            continue
        seen.add(node)
        members.add(node)
        stack.extend(graph[node] - seen)
    components.append(members)
if len(edges) != 15 or len(vertices) != 15:
    raise ValueError("Đồ thị cần 15 cạnh và 15 đỉnh.")
print("Cạnh:", len(edges))
print("Cụm:", len(components))
sizes = []
for group in components:
    sizes.append(len(group))
sizes.sort(reverse=True)
labels = []
for size in sizes:
    labels.append(str(size))
connected = False
for group in components:
    if {"A", "O"} <= group:
        connected = True
        break
print("Kích thước cụm:", ", ".join(labels))
print("A kết nối O:", connected)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm các thành phần liên thông trong đồ thị 15 cạnh bằng ngăn xếp DFS.
# Dữ liệu: 15 cạnh vô hướng nối 15 đỉnh A..O thành nhiều cụm; cần biết A có cùng cụm với O hay
không.
# 15 list đôi là các cạnh vô hướng; mỗi đầu nối vào set hàng xóm của nhau.
```

```
edges = [["A","B"],["B","C"],["C","D"],["A","D"],["D","E"],["E","F"],["F","G"],["H","I"],["I","J"],
["J","K"],["K","L"],["H","L"],["L","M"],["M","N"],["N","O"]]
# Set thu các tên đỉnh xuất hiện trong cạnh, bỏ trùng.
vertex_set = set()
# Duyệt mỗi cặp cạnh để lấy hai đỉnh.
for edge in edges:
    for name in edge:
        # add() đưa đỉnh vào set duy nhất.
        vertex_set.add(name)
# sorted() tạo list đỉnh theo thứ tự cố định để duyệt thành phần.
vertices = sorted(vertex_set)
graph = {}
for vertex in vertices:
    # Tạo set kề rỗng cho từng đỉnh.
    graph[vertex] = set()
for a, b in edges:
    # Thêm hai chiều a→b và b→a để biểu diễn cạnh vô hướng.
    graph[a].add(b)
    graph[b].add(a)
# seen là tập đỉnh đã duyệt toàn đồ thị, components lưu tập đỉnh từng cụm.
seen, components = set(), []
for start in vertices:
    # Bỏ đỉnh đã thuộc cụm tìm trước, tránh ghi cùng cụm hai lần.
    if start in seen:
        continue
    # stack là ngăn xếp LIFO để duyệt DFS; members lưu đỉnh trong cụm hiện tại.
    stack, members = [start], set()
    # Còn đỉnh chờ xử lý thì tiếp tục đi sâu.
    while stack:
        # pop() không chỉ số lấy và xóa phần tử cuối stack, mô phỏng DFS.
        node = stack.pop()
        # Bỏ qua đỉnh đã xử lý để tránh vòng lặp vô hạn trong chu trình.
        if node in seen:
            continue
        # Đánh dấu đỉnh đã duyệt, dùng chung cho cả các cụm.
        seen.add(node)
        # Ghi nhận node thuộc cụm hiện tại.
        members.add(node)
        # graph[node]-seen lấy hàng xóm chưa duyệt; extend() nối nhiều đỉnh vào stack. Set không có
thứ tự, nhưng số cụm không đổi.
        stack.extend(graph[node] - seen)
    # Hoàn tất một cụm, đưa set thành viên vào danh sách các cụm.
    components.append(members)
# Kiểm đúng số cạnh và đỉnh nguồn.
if len(edges) != 15 or len(vertices) != 15:
    # ValueError báo không đạt điều kiện số bản ghi, cạnh hoặc tính nhất quán của đồ thị.
    raise ValueError("Đồ thị cần 15 cạnh và 15 đỉnh.")
print("Cạnh:", len(edges))
print("Cụm:", len(components))
sizes = []
for group in components:
    # len(group) đếm số đỉnh mỗi cụm, append() ghi kích cỡ.
    sizes.append(len(group))
# sort(reverse=True) xếp kích cỡ cụm giảm dần tại chỗ.
sizes.sort(reverse=True)
labels = []
for size in sizes:
    # str() đổi số thành chuỗi vì join() chỉ nối chuỗi.
    labels.append(str(size))
connected = False
for group in components:
    # <= giữa set nghĩa tập con: True nếu CẢ A và O đều thuộc cùng group.
    if {"A", "O"} <= group:
        connected = True
        # break dừng khi tìm được cụm có đủ hai đỉnh cần kiểm.
        break
```

```
# join() ghép chuỗi kích thước cụm có dấu phẩy.
print("Kích thước cụm:", ", ".join(labels))
print("A kết nối O:", connected)
```

Kết quả khi chạy:

```
Cạnh: 15
Cụm: 2
Kích thước cụm: 8, 7
A kết nối O: False
```

Hai người có thể không có cạnh trực tiếp nhưng vẫn cùng cụm qua trung gian. A và O nằm ở hai cụm khác nhau nên không có đường nối qua dữ liệu hiện tại.

Tình huống 4 — Cấp 4: Xếp hạng 17 đối tác từ 18 hồ sơ, xử lý kỹ năng trống

Từ vựng trong mục: Ranking — xếp hạng; **Empty set** — tập rỗng; **Tie-break** — phân xử hòa điểm.

Tình huống thực tế: Hệ thống gợi ý bạn học có **18 hồ sơ U01–U18**, mỗi người nhập danh sách kỹ năng. U01 là người cần tìm bạn phù hợp; một số kỹ năng trong hồ sơ được ghi lặp, còn U14 **chưa khai kỹ năng nào**. Người quản lý cần gợi ý **ba người giống U01 nhất** theo kỹ năng, đồng thời báo còn bao nhiêu hồ sơ thiếu thông tin. Hồ sơ trống không được hiểu là người đó thực sự không có khả năng.

Vì sao chọn cách này? Hàm so sánh Jaccard trong bài chuyển mỗi danh sách kỹ năng sang set, để kỹ năng bị nhập lặp chỉ được tính một lần. Sau đó nó xem **kỹ năng chung** chiếm bao nhiêu trong toàn bộ kỹ năng khác nhau của hai người; hai tập không có phần tử thì trả 0 để tránh lỗi chia cho 0. U14 có danh sách rỗng nên bài **báo là thiếu dữ liệu**, không kết luận người này không có kỹ năng. Chương trình tính điểm U01 với từng người còn lại, sắp theo **điểm giảm dần**, nếu hòa thì theo **mã tăng dần**, rồi lấy ba mã đầu; đây là gợi ý dựa trên khai báo kỹ năng, chưa phải đánh giá năng lực thực tế.

Quy trình:

1. Chuẩn hóa kỹ năng về Set trên 18 hồ sơ.
2. Tính tương đồng U01 với 17 người còn lại, kiểm tra tập rỗng.
3. Sắp xếp ổn định, in ba ứng viên đầu và số hồ sơ thiếu kỹ năng.

No Note — Không chú thích

```
people = [{"id":"U01","skills":["Python","SQL","Python"]},{"id":"U02","skills":["AI","Math"]},
{"id":"U03","skills":["SQL","Data"]},{"id":"U04","skills":["Math","Cloud","Python"]},
{"id":"U05","skills":["Data","Vision"]},{"id":"U06","skills":["Cloud","NLP"]},{"id":"U07","skills":
["Vision","Python","Python"]},{"id":"U08","skills":["NLP","AI"]},{"id":"U09","skills":
["Python","SQL"]},{"id":"U10","skills":["AI","Math","Python"]},{"id":"U11","skills":
["SQL","Data"]},{"id":"U12","skills":["Math","Cloud"]},{"id":"U13","skills":
["Data","Vision","Python"]},{"id":"U14","skills":[]},{"id":"U15","skills":["Vision","Python"]},
{"id":"U16","skills":["NLP","AI","Python"]},{"id":"U17","skills":["Python","SQL"]},
{"id":"U18","skills":["AI","Math"]}]
def similarity(a, b):
    a, b = set(a), set(b)
    total = a | b
    if total:
        return len(a & b) / len(total)
    return 0.0
base = people[0]["skills"]
ranked = []
for person in people[1:]:
    ranked.append((person["id"], similarity(base, person["skills"])))
```

```
def ranking_key(item):
    return (-item[1], item[0])
ranked.sort(key=ranking_key)
if len(people) != 18:
    raise ValueError("Cần đúng 18 hồ sơ kỹ năng.")
print("Hồ sơ:", len(people))
missing_skills = 0
for person in people:
    if not person["skills"]:
        missing_skills += 1
print("Thiếu kỹ năng:", missing_skills)
for uid, score in ranked[:3]:
    print(f"{uid}: {score:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xếp hạng người có kỹ năng giống U01 từ 18 hồ sơ, xử lý list kỹ năng thiếu.
# Dữ liệu: 18 dict id/skills; người U14 có list rỗng, có hồ sơ lặp kỹ năng Python.
# 18 hồ sơ gồm id và list skills, kỹ năng lặp không được tăng độ tương đồng.
people = [{"id":"U01","skills":["Python","SQL","Python"]},{"id":"U02","skills":["AI","Math"]},
{"id":"U03","skills":["SQL","Data"]},{"id":"U04","skills":["Math","Cloud","Python"]},
{"id":"U05","skills":["Data","Vision"]},{"id":"U06","skills":["Cloud","NLP"]},{"id":"U07","skills":
["Vision","Python","Python"]},{"id":"U08","skills":["NLP","AI"]},{"id":"U09","skills":
["Python","SQL"]},{"id":"U10","skills":["AI","Math","Python"]},{"id":"U11","skills":
["SQL","Data"]},{"id":"U12","skills":["Math","Cloud"]},{"id":"U13","skills":
["Data","Vision","Python"]},{"id":"U14","skills":[]},{"id":"U15","skills":["Vision","Python"]},
{"id":"U16","skills":["NLP","AI","Python"]},{"id":"U17","skills":["Python","SQL"]},
{"id":"U18","skills":["AI","Math"]}]
# Hàm tự định nghĩa so Jaccard của hai bộ kỹ năng.
def similarity(a, b):
    # set() bỏ kỹ năng trùng, bảo đảm so tập các loại kỹ năng.
    a, b = set(a), set(b)
    # | là hợp hai tập, đếm mọi kỹ năng có ở ít nhất một người.
    total = a | b
    # Không chia cho 0 khi hai tập cùng rỗng; set rỗng được xem là False.
    if total:
        # & lấy tập chung; chia số loại chung cho số loại trong hợp, trả tỷ lệ 0..1.
        return len(a & b) / len(total)
    # Quy ước Jaccard=0 nếu hợp rỗng trong tài liệu này.
    return 0.0
# [0] là kỹ năng của U01, dùng làm tham chiếu.
base = people[0]["skills"]
ranked = []
# [1:] bỏ U01 để chỉ xếp những người còn lại.
for person in people[1:]:
    # append() thêm tuple (mã người, điểm gần U01).
    ranked.append((person["id"], similarity(base, person["skills"])))
# Hàm key trả tiêu chí xếp dựa vào (mã, điểm).
def ranking_key(item):
    # Dấu âm của điểm để xếp cao trước; tên mã phá hòa tăng dần.
    return (-item[1], item[0])
# sort() xếp ngay list theo hàm key; không thay đổi people.
ranked.sort(key=ranking_key)
if len(people) != 18:
    # ValueError báo không đạt điều kiện số bản ghi, cạnh hoặc tính nhất quán của đồ thị.
    raise ValueError("Cần đúng 18 hồ sơ kỹ năng.")
print("Hồ sơ:", len(people))
missing_skills = 0
for person in people:
    # not list đúng khi list rỗng, đếm hồ sơ chưa khai kỹ năng.
    if not person["skills"]:
        # Đếm người không có kỹ năng ghi nhận, khác với người không trùng kỹ năng với U01.
        missing_skills += 1
print("Thiếu kỹ năng:", missing_skills)
# [:3] lấy ba người đầu, không gồm vị trí 3; :.2f định dạng điểm hai số sau dấu thập phân.
```

```
for uid, score in ranked[:3]:
    print(f"{uid}: {score:.2f}")
```

Kết quả khi chạy:

```
Hồ sơ: 18
Thiếu kỹ năng: 1
U09: 1.00
U17: 1.00
U03: 0.33
```

Đây là **xếp hạng theo điểm tập hợp**, không phải kết luận con người nào phù hợp nhất về khả năng học tập thực tế. Hồ sơ trống cần bổ sung dữ liệu.

Tình huống 5 — Cấp 5: Tìm đường ngắn nhất và so hàng xóm trong mạng có hướng 20 cạnh

Từ vựng trong mục: Directed graph — đồ thị có hướng; **BFS** — duyệt theo chiều rộng; **Jaccard** — tương đồng tập kề.

Tình huống thực tế: Mạng chuyển tiếp thông báo có **20 đường liên kết một chiều** giữa các trạm A đến S. Mỗi dòng `(trạm gửi, trạm nhận)` nghĩa là gửi được theo chiều đó, không mặc nhiên gửi ngược lại; riêng trạm R chỉ có đường tới S. Kỹ thuật viên cần tìm đường từ **A đến K** có ít chặng nhất, kiểm tra **R có tới được K** không, và so mức trùng nhau giữa hai tập trạm nhận trực tiếp của **A và B**.

Vì sao chọn cách này? Đây là **đồ thị có hướng**: cạnh A→B chỉ cho phép chuyển từ A sang B, không tự suy ra đường B→A. **BFS** (tìm kiếm theo chiều rộng) thử các trạm ở khoảng cách **một chặng**, rồi hai chặng, rồi ba chặng; trong mạng mà mỗi chặng tính như nhau, lần đầu tới K cho một đường **ít cạnh nhất**. `deque` giữ các điểm cần xét theo thứ tự đã vào hàng, `visited` tránh thử lại trạm và đường đã thăm. Trạm R chỉ dẫn tới S, không nối tới K nên không có đường như yêu cầu. **Jaccard** trên tập hàng xóm của A/B là câu hỏi khác: mức giống nhau của các trạm nhận **ngay trong một bước**, không phản ánh việc A và B có cùng đường nhiều chặng hay không.

Quy trình:

1. Tạo danh sách kề **một chiều** từ 20 cạnh và xác nhận trạm nguồn/đích.
2. Dùng BFS có kiểm soát đỉnh đã thăm để tìm đường A→K và R→K.
3. Tính Jaccard tập trạm nhận trực tiếp A/B, báo đường và hai kết quả.

No Note — Không chú thích

```
from collections import deque
edges = [["A","B"],["A","C"],["B","D"],["C","D"],["D","E"],["E","F"],["B","G"],["G","H"],["H","I"],
["I","J"],["F","J"],["J","K"],["C","L"],["L","M"],["M","N"],["N","O"],["O","K"],["K","P"],
["P","Q"],["R","S"]]
graph = {}
for source, destination in edges:
    graph.setdefault(source, set()).add(destination)
def shortest(start, end):
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == end:
            return path
        for neighbor in sorted(graph.get(node, set())):
```

```
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None
path = shortest("A", "K")
a, b = graph.get("A", set()), graph.get("B", set())
union = a | b
if union:
    score = len(a & b) / len(union)
else:
    score = 0.0
if len(edges) != 20:
    raise ValueError("Cần đủ 20 cạnh có hướng.")
print("Cạnh có hướng:", len(edges))
print("Đường A-K:", " -> ".join(path))
print("Số cạnh đường:", len(path) - 1)
print("R tới K:", shortest("R", "K") is not None)
print(f"Jaccard kề A/B: {score:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng BFS tìm đường đi ngắn nhất theo số cạnh có hướng giữa A và K, so Jaccard danh sách
kề A/B.
# Dữ liệu: 20 cung [nguồn,đích] có hướng, một cung R→S tách rời; A và K có thể nối qua nhiều bước.
# deque là hàng đợi hai đầu, popleft() lấy đầu hiệu quả cho duyệt BFS theo từng lớp.
from collections import deque
# Mỗi bộ nguồn-đích là một CUNG CÓ HƯỚNG, không mặc định thêm chiều ngược.
edges = [["A","B"],["A","C"],["B","D"],["C","D"],["D","E"],["E","F"],["B","G"],["G","H"],["H","I"],
["I","J"],["F","J"],["J","K"],["C","L"],["L","M"],["M","N"],["N","O"],["O","K"],["K","P"],
["P","Q"],["R","S"]]
# Dictionary ánh xạ đỉnh nguồn sang set các đích đi được từ đó.
graph = {}
for source, destination in edges:
    # setdefault(k,set()) lấy set kề hiện có hoặc tạo rỗng nếu thiếu; add() thêm cung
source→destination duy nhất.
    graph.setdefault(source, set()).add(destination)
# Hàm BFS trả list các đỉnh trên đường ngắn nhất theo số CẠNH; trả None nếu không tới được.
def shortest(start, end):
    # Hàng đợi chứa tuple (đỉnh đang xét, danh sách đường đi tới đỉnh); ban đầu đường chỉ có start.
    queue = deque([(start, [start])])
    # Set các đỉnh đã đưa vào hàng đợi, tránh thăm lại hoặc vòng lặp chu trình.
    visited = {start}
    # BFS xử lý cho đến khi hàng đợi hết hoặc tìm thấy đích.
    while queue:
        # popleft() lấy phần tử ĐẦU hàng đợi FIFO, giúp BFS duyệt đường ít cạnh trước.
        node, path = queue.popleft()
        # Đến đích thì trả ngay đường được lấy ra; do BFS FIFO nên đây là đường ngắn nhất theo
cạnh.
        if node == end:
            return path
        # get(node,set()) lấy hàng xóm có hướng hoặc tập rỗng nếu đỉnh chưa có khóa; sorted() cố
định thứ tự đi khi nhiều đường đồng độ dài.
        for neighbor in sorted(graph.get(node, set())):
            # Không mở rộng lại đỉnh đã vào queue, tránh lặp.
            if neighbor not in visited:
                # Ghi nhận đỉnh ngay khi thêm vào queue.
                visited.add(neighbor)
                # append() thêm vào CUỐI queue; path+[neighbor] tạo list đường mới, không sửa path
cũ.
                queue.append((neighbor, path + [neighbor]))
    # Trả None khi đã vét hết các đỉnh có thể đi mà không thấy end.
    return None
# Tìm đường A→K theo đúng chiều cung, không phải đường vô hướng.
path = shortest("A", "K")
# Nếu đỉnh A/B không có hàng xóm thì get trả set rỗng để phép toán tập không lỗi.
```

```
a, b = graph.get("A", set()), graph.get("B", set())
# | hợp hai set hàng xóm của A và B, dùng làm mẫu số Jaccard.
union = a | b
if union:
    # & giao hàng xóm chung; chia cho số hàng xóm khác nhau trong hợp, chỉ khi union không rỗng.
    score = len(a & b) / len(union)
else:
    score = 0.0
# Xác nhận đủ 20 cung nguồn.
if len(edges) != 20:
    # ValueError báo không đạt điều kiện số bản ghi, cạnh hoặc tính nhất quán của đồ thị.
    raise ValueError("Cần đủ 20 cạnh có hướng.")
print("Cạnh có hướng:", len(edges))
# join() ghép các đỉnh dọc đường tìm được, mũi tên thể hiện chiều di chuyển.
print("Đường A-K:", " -> ".join(path))
# Một đường có n đỉnh chứa n−1 cạnh, nên trừ 1 khi báo độ dài.
print("Số cạnh đường:", len(path) - 1)
# is not None kiểm có đường R→K; False vì thành phần R→S không dẫn tới K.
print("R tới K:", shortest("R", "K") is not None)
print(f"Jaccard kề A/B: {score:.2f}")
```

Kết quả khi chạy:

```
Cạnh có hướng: 20
Đường A-K: A -> B -> D -> E -> F -> J -> K
Số cạnh đường: 6
R tới K: False
Jaccard kề A/B: 0.00
```

Đường A→K có thể đi qua trạm trung gian, nhưng R→S không thể tự quay ngược để tới K. Độ giống nhau của tập kề không thay thế thuật toán tìm đường.

6. Sai lầm thường gặp

- **Đỉnh** là đối tượng, **cạnh** là quan hệ; không nhầm số đỉnh với số cạnh.
- Đồ thị có hướng không mặc nhiên có cạnh ngược; đồ thị vô hướng cần diễn giải kết nối hai chiều.
- Set không bảo đảm thứ tự in; dùng `sorted()` khi liệt kê tên trong tài liệu.
- Jaccard bằng `|A∩B| / |A∪B|`; nếu cả hai tập rỗng thì phải xử lý trước khi chia.
- `networkx` là thư viện ngoài cần cài đặt; `shortest_path` có thể báo lỗi nếu hai đỉnh không nối với nhau.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a graph? | Đồ thị là gì?

A. A collection of vertices and edges | Tập các đỉnh và cạnh

B. An entity or node in the network | Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices | Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors | Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: A

Giải thích: Đồ thị mô tả quan hệ giữa các đối tượng.

Q2. What is a vertex? | Đỉnh trong đồ thị là gì?

A. A collection of vertices and edges | Tập các đỉnh và cạnh

B. An entity or node in the network | Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices | Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors | Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: B

Giải thích: Mỗi đỉnh đại diện một đối tượng.

Q3. What is an edge? | Cạnh của đồ thị là gì?

A. A collection of vertices and edges | Tập các đỉnh và cạnh

B. An entity or node in the network | Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices | Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors | Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: C

Giải thích: Cạnh biểu diễn kết nối giữa các đỉnh.

Q4. What is an adjacency list? | Danh sách kề là gì?

A. A collection of vertices and edges | Tập các đỉnh và cạnh

B. An entity or node in the network | Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices | Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors | Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: D

Giải thích: Danh sách kề lưu các kết nối của từng đỉnh.

Q5. What does a directed edge A→B assert? | Cạnh có hướng A→B cho biết gì?

A. A connection from A to B | Kết nối đi từ A đến B

B. A connection without direction | Kết nối không phân biệt chiều

C. The neighbors stored for An | Những đỉnh kề được lưu cho An

D. A sequence of connected vertices | Chuỗi các đỉnh nối tiếp nhau

Đáp án: A

Giải thích: Chiều kết nối quan trọng; B→A không tự tồn tại.

Q6. What does an undirected edge mean? | Cạnh vô hướng có ý nghĩa gì?

A. A connection from A to B | Kết nối đi từ A đến B

B. A connection without direction | Kết nối không phân biệt chiều

C. The neighbors stored for An | Những đỉnh kề được lưu cho An

D. A sequence of connected vertices | Chuỗi các đỉnh nối tiếp nhau

Đáp án: B

Giải thích: Quan hệ vô hướng đi được theo cả hai chiều.

Q7. What does `graph['An']` return in an adjacency dictionary? | `graph['An']` trong Dictionary kề trả gì?

A. A connection from A to B | Kết nối đi từ A đến B

B. A connection without direction | Kết nối không phân biệt chiều

C. The neighbors stored for An | Những đỉnh kề được lưu cho An

D. A sequence of connected vertices | Chuỗi các đỉnh nối tiếp nhau

Đáp án: C

Giải thích: Giá trị của khóa đỉnh chứa các nút lân cận.

Q8. What does a path in a graph describe? | Đường đi trong đồ thị biểu diễn gì?

A. A connection from A to B | Kết nối đi từ A đến B

B. A connection without direction | Kết nối không phân biệt chiều

C. The neighbors stored for An | Những đỉnh kề được lưu cho An

D. A sequence of connected vertices | Chuỗi các đỉnh nối tiếp nhau

Đáp án: D

Giải thích: Hai đỉnh liên tiếp trên đường đi phải có cạnh nối.

Q9. What is Jaccard similarity for sets A and B? | Jaccard giữa hai Set A, B bằng gì?

A. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

B. 1 | 1

C. 0 | 0

D. A zero union denominator | Mẫu số hợp bằng 0

Đáp án: A

Giải thích: Jaccard so phần chung với toàn bộ phần tử khác nhau.

Q10. What is Jaccard for two identical nonempty sets? | Jaccard của hai Set giống nhau và không rỗng là gì?

A. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

B. 1 | 1

C. 0 | 0

D. A zero union denominator | Mẫu số hợp bằng 0

Đáp án: B

Giải thích: Phép giao bằng phép hợp.

Q11. What is Jaccard for two disjoint nonempty sets? | Jaccard của hai Set không giao nhau, không rỗng là gì?

A. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

B. 1 | 1

C. 0 | 0

D. A zero union denominator | Mẫu số hợp bằng 0

Đáp án: C

Giải thích: Phép giao không có phần tử nào.

Q12. What must be handled before computing Jaccard on two empty sets? | Cần xử lý gì trước khi tính Jaccard cho hai Set rỗng?

A. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

B. 1 | 1

C. 0 | 0

D. A zero union denominator | Mẫu số hợp bằng 0

Đáp án: D

Giải thích: Nếu không xử lý sẽ chia cho 0.

Q13. What does `nx.Graph()` create? | `nx.Graph()` tạo gì?

A. An undirected NetworkX graph | Đồ thị vô hướng NetworkX

B. A directed NetworkX graph | Đồ thị có hướng NetworkX

C. A path with fewest edges | Đường đi có ít cạnh nhất

D. Install and import the package | Cài và import thư viện

Đáp án: A

Giải thích: Graph tạo cạnh vô hướng theo mặc định.

Q14. What does `nx.DiGraph()` create? | `nx.DiGraph()` tạo gì?

A. An undirected NetworkX graph | Đồ thị vô hướng NetworkX

B. A directed NetworkX graph | Đồ thị có hướng NetworkX

C. A path with fewest edges | Đường đi có ít cạnh nhất

D. Install and import the package | Cài và import thư viện

Đáp án: B

Giải thích: DiGraph phân biệt chiều cạnh.

Q15. What does `nx.shortest_path()` find on an unweighted graph? | `nx.shortest_path()` tìm gì trên đồ thị không trọng số?

A. An undirected NetworkX graph | Đồ thị vô hướng NetworkX

B. A directed NetworkX graph | Đồ thị có hướng NetworkX

C. A path with fewest edges | Đường đi có ít cạnh nhất

D. Install and import the package | Cài và import thư viện

Đáp án: C

Giải thích: Đường ngắn nhất không trọng số có ít cạnh nhất.

Q16. What is required to use NetworkX in Python? | Để dùng NetworkX trong Python cần gì?

A. An undirected NetworkX graph | Đồ thị vô hướng NetworkX

B. A directed NetworkX graph | Đồ thị có hướng NetworkX

C. A path with fewest edges | Đường đi có ít cạnh nhất

D. Install and import the package | Cài và import thư viện

Đáp án: D

Giải thích: NetworkX không thuộc thư viện chuẩn Python.

Q17. How can neighbor display order be made consistent? | Làm sao in danh sách các đỉnh kề theo thứ tự ổn định?

A. Call `sorted()` on neighbor sets | Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned | Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously | Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges | Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: A

Giải thích: Set không bảo đảm thứ tự khi in.

Q18. What happens if no path exists between two vertices? | Nếu không có đường nối hai đỉnh thì sao?

A. Call `sorted()` on neighbor sets | Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned | Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously | Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges | Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: B

Giải thích: Thuật toán tìm đường cần xử lý hai đỉnh không liên thông.

Q19. Why should node names be unique? | Vì sao tên đỉnh nên duy nhất?

A. Call `sorted()` on neighbor sets | Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned | Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously | Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges | Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: C

Giải thích: Mã đỉnh trùng khiến các đối tượng bị nhập làm một.

Q20. What distinguishes Jaccard from graph shortest path? | Jaccard khác đường đi ngắn nhất ở đâu?

A. Call `sorted()` on neighbor sets | Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned | Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously | Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges | Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: D

Giải thích: Hai công cụ trả lời hai loại câu hỏi khác nhau.

8. Nối sang bài kế tiếp

`Set → Graph → Similarity → Gợi ý bạn học.`

PY-24 — Scalar, vector, norm, đơn vị và dot product

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Scalar — đại lượng vô hướng; **Vector** — véc-tơ.

`NumPy → Scalar/Vector → Matrix → PCA`

Học xong dùng để làm gì? Biểu diễn học viên hay sản phẩm dưới dạng vector đặc trưng số và thực hiện phép cộng/trừ vector.

Kết nối kiến thức tiếp theo: PY-25 xếp nhiều vector thành ma trận để tính cùng lúc nhiều quan sát.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Scalar	đại lượng vô hướng	Một số đơn lẻ có thể đo độ lớn
Vector	véc-tơ	Dãy số mô tả nhiều thành phần của một đối tượng
Norm	độ lớn véc-tơ	Độ dài theo chuẩn đã chọn
Dot product	tích vô hướng	Nhân từng cặp tọa độ tương ứng rồi cộng

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.array([...],dtype=float)`	Tạo véc-tơ kiểu thực khi phép tính tạo phần thập phân
`np.linalg.norm(v)`	Độ lớn Euclidean của véc-tơ
`v/np.linalg.norm(v)`	Chuẩn hóa về độ dài 1, phải tránh norm bằng 0
`a @ b / np.dot(a,b)`	Tích vô hướng nếu hai vector một chiều cùng độ dài

4. Khái niệm liên quan

Bài học nối bốn ý: **scalar** là số đơn lẻ, **vector** chứa tọa độ, **norm** đo độ dài, **dot product** kết hợp các thành phần. Robot dịch chuyển `(3,4)` mét có vector hai chiều nhưng quãng đường thẳng chỉ là một số bằng 5 mét.

4.1. Scalar — Số đơn lẻ và đơn vị

Scalar là một giá trị số như vận tốc 2,5 mét/giây. Python lưu số, còn ý nghĩa và đơn vị phải do người lập trình nêu. Scalar khác vector vì không chứa nhiều thành phần tọa độ.

No Note — Không chú thích

```
speed = 2.5
print(speed)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt một giá trị vô hướng với đại lượng có nhiều thành phần.
# Dữ liệu: speed=2.5 là một giá trị tốc độ giả lập; chưa biểu diễn hướng hay các trục tọa độ.
# 2.5 là scalar (số đơn) mô tả độ lớn; chưa phải vector hướng trong không gian.
speed = 2.5
# print() hiển thị số đang lưu để phân biệt scalar với mảng tọa độ.
print(speed)
```

Kết quả khi chạy:

```
2.5
```

Số 2.5 đại diện vận tốc m/s, không phải tọa độ.

4.2. Vector — Các tọa độ có thứ tự

Vector là dãy tọa độ theo một hệ trục. Robot đi 3 m theo x và 4 m theo y nên dùng vector `[3,4]`. Phần tử đầu là x, phần tử sau là y. Đổi `[3,4]` thành `[4,3]` sẽ đổi ý nghĩa hai trục.

No Note — Không chú thích

```
import numpy as np

movement = np.array([3, 4])
print(movement)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Biểu diễn độ dịch chuyển hai chiều bằng vector các tọa độ x,y có thứ tự.
# Dữ liệu: Vector [3,4] gồm hai thành phần tọa độ x và y, không phải hai mẫu dữ liệu độc lập.
# NumPy tạo vector từ list và thực hiện các phép toán theo phần tử.
import numpy as np

# np.array() tạo ndarray một chiều shape (2,); vị trí 0 là x=3, vị trí 1 là y=4.
movement = np.array([3, 4])
# In hai tọa độ theo thứ tự, không tự tính độ dài.
print(movement)
```

Kết quả khi chạy:

```
[3 4]
```

Hai phần tử lần lượt là độ dịch chuyển theo x và y (mét).

4.3. Norm — Độ dài vector

Chuẩn Euclid là độ dài `sqrt(x²+y²)`. Với vector `(3,4)`, độ dài bằng `sqrt(9+16)=5` m. Vector đơn vị giữ hướng và có norm bằng 1; muốn chuẩn hóa phải chia các thành phần cho norm, trừ trường hợp norm bằng 0.

No Note — Không chú thích

```
import numpy as np

movement = np.array([3.0, 4.0])
print(np.linalg.norm(movement))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính độ dài Euclid của vector chuyển động (3,4) bằng NumPy.
# Dữ liệu: Hai thành phần x=3, y=4 theo cùng đơn vị khoảng cách.
# NumPy hỗ trợ phép tính độ dài vector trong np.linalg.
import numpy as np

# dtype suy ra số thực vì tọa độ có dấu thập phân; mỗi ô là một trục.
movement = np.array([3.0, 4.0])
# linalg.norm(vector) mặc định tính độ dài Euclid: căn tổng bình phương các thành phần, ở đây 5 đơn vị.
print(np.linalg.norm(movement))
```

Kết quả khi chạy:

```
5.0
```

Robot cách gốc tọa độ 5 mét theo đường thẳng.

4.4. Dot product — Nhân và cộng cùng vị trí

Tích vô hướng của hai vector cùng số chiều bằng tổng các tích từng thành phần: `(1,2)·(3,4)=1×3+2×4=11`. Dùng để tính điểm có trọng số hoặc kiểm tra hai vector khác không vuông góc (khi tích vô hướng bằng 0).

No Note — Không chú thích

```
import numpy as np

a = np.array([1, 2])
b = np.array([3, 4])
print(a @ b)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính tích vô hướng của hai vector hai chiều bằng toán tử @.
# Dữ liệu: a=[1,2], b=[3,4]; các thành phần cùng trục được nhân rồi cộng.
# NumPy định nghĩa @ cho nhân ma trận/tích vô hướng tùy kích thước.
import numpy as np

# Vector a có hai tọa độ theo thứ tự x,y.
a = np.array([1, 2])
# Vector b phải có cùng số thành phần với a để tính tích vô hướng.
b = np.array([3, 4])
```

```
# Với hai vector 1 chiều, @ trả scalar 1×3 + 2×4 = 11; khác * tạo vector nhân từng vị trí.
print(a @ b)
```

Kết quả khi chạy:

```
11
```

Tích vô hướng trả một scalar bằng 11.

5. Năm tình huống véc-tơ — từ khoảng cách đến kiểm tra sai số đa trục

Tình huống 1 — Cấp 1: Cộng 10 bước di chuyển và đo độ lệch cuối

Từ vựng trong mục: Vector — véc-tơ có thứ tự; **Scalar** — số đơn lẻ; **Norm** — độ dài.

Tình huống thực tế: Một robot di chuyển trên sàn theo **10 lệnh**; mỗi lệnh chứa hai số: đi theo chiều ngang **x** và chiều dọc **y**, đều tính bằng **mét**. Số âm nghĩa là đi ngược hướng dương của trục, ví dụ `(-2, 0)` là lùi 2 m trên trục x. Sau cả 10 lệnh, người điều khiển muốn biết robot đang lệch điểm xuất phát bao nhiêu mét theo từng trục và **khoảng cách đường thẳng từ điểm xuất phát**, không phải tổng quãng đường đã di chuyển.

Vì sao chọn cách này? Véc-tơ là cặp số ghi **mức dịch chuyển và hướng** theo từng trục, nên số âm là hợp lệ. `steps.sum(axis=0)` cộng **riêng các bước x** và **riêng các bước y** để ra véc-tơ vị trí cuối so với gốc: những bước đi ngược nhau có thể triệt tiêu. `np.linalg.norm(ending)` tính độ dài **đường thẳng từ gốc đến vị trí cuối** dựa trên cả hai thành phần; nó khác việc cộng độ dài từng bước. Robot có thể đi một vòng rồi về gần gốc: đường đã đi dài nhưng khoảng cách cuối tới gốc nhỏ.

Quy trình:

1. Tạo 10 véc-tơ 2 chiều với đơn vị mét.
2. Cộng theo trục x/y bằng `sum(axis=0)`.
3. Tính norm của véc-tơ tổng và in đơn vị.

No Note — Không chú thích

```
import numpy as np
steps = np.array([[3,4],[0,2],[-2,0],[1,-1],[2,2],[-3,1],[0,-4],[4,0],[-1,3],[2,-2]], dtype=float)
ending = steps.sum(axis=0)
if steps.shape != (10, 2):
    raise ValueError("Cần đủ 10 bước chuyển động hai chiều.")
print("Bước:", len(steps))
print("Lệch x (m):", int(ending[0]))
print("Lệch y (m):", int(ending[1]))
print(f"Cách gốc (m): {np.linalg.norm(ending):.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cộng 10 bước chuyển động hai chiều thành độ lệch x/y và khoảng cách tới gốc.
# Dữ liệu: 10 hàng bước [dx,dy], đơn vị mét; bước âm là đi ngược chiều trục.
# NumPy cho phép cộng vector từng cột theo nhiều hàng.
import numpy as np
# 10 cặp dx/dy, dtype=float tạo bảng hai cột số´thực.
steps = np.array([[3,4],[0,2],[-2,0],[1,-1],[2,2],[-3,1],[0,-4],[4,0],[-1,3],[2,-2]], dtype=float)
# sum(axis=0) cộng THEO HÀNG, trả hai tổng cột dx/dy, là vị trí cuối so với gốc.
```

```
ending = steps.sum(axis=0)
# shape (10,2) kiểm 10 bước và 2 tọa độ môĩ bước.
if steps.shape != (10, 2):
    # raise ValueError() chặn kết luận nêú sô´bản ghi hoặc shape không đạt yêu câù.
    raise ValueError("Cần đủ 10 bước chuyển động hai chiêù.")
# len() đếm sô´hàng bước, ở đây 10, không phải tổng ô.
print("Bước:", len(steps))
# [0] là độ lệch x, [1] độ lệch y; int() đưa giá trị NumPy vê`int đe̓in.
print("Lệch x (m):", int(ending[0]))
print("Lệch y (m):", int(ending[1]))
# norm() lấý khoảng cách thăng từ gôć đến tọa độ cuôí; :.2f chỉ định in hai chữ sô´sau dâú thập
phân, đơn vị mét.
print(f"Cách gôć (m): {np.linalg.norm(ending):.2f}")
```

Kết quả khi chạy:

```
Bước: 10
Lệch x (m): 6
Lệch y (m): 5
Cách gôć (m): 7.81
```

Véc-tơ tổng có thể nhỏ hơn tổng quãng đường đi, vì các bước di chuyển ngược chiều triệt tiêu nhau.

Tình huống 2 — Cấp 2: Chuẩn hóa 12 hướng robot nhưng không chia cho 0

Từ vựng trong mục: Unit vector — véc-tơ đơn vị; **Zero vector** — véc-tơ không; **Normalization** — chuẩn hóa.

Tình huống thực tế: Bộ điều khiển nhận **12 véc-tơ chỉ hướng**, mỗi véc-tơ là hai số `(x, y)` chưa cần gắn đơn vị. Mười véc-tơ có ít nhất một thành phần khác 0, nhưng có hai cặp `(0, 0)` không chỉ ra được hướng nào. Nhóm kỹ thuật muốn đổi **mười hướng hợp lệ** thành véc-tơ dài 1 để dễ so sánh và ghi riêng hai yêu cầu chưa xác định, không tự bịa một hướng cho chúng.

Vì sao chọn cách này? Chuẩn hóa véc-tơ ở đây nghĩa là giữ nguyên **hướng chỉ**, nhưng đưa độ dài mỗi véc-tơ hợp lệ về **1**; nhờ vậy hai lệnh cùng hướng có thể được so mà không bị ảnh hưởng bởi số lớn nhỏ ban đầu. `np.linalg.norm(..., axis=1)` đo độ dài từng hàng; điều kiện `lengths > 0` chỉ chọn các hàng có hướng xác định. NumPy chỉ chia các hàng ấy cho độ dài tương ứng; các hàng `(0,0)` không được chia vì sẽ chia cho 0. Chương trình đếm riêng hai hàng này và để chúng ở dạng 0 trong mảng kết quả, nhưng **không coi đó là hướng chuẩn hợp lệ**.

Quy trình:

1. Tạo mảng `(12,2)` và tính norm riêng từng hàng.
2. Chỉ chia các hàng có norm dương; giữ véc-tơ không cho kiểm tra.
3. Đếm số hướng hợp lệ, in véc-tơ đầu sau chuẩn hóa.

No Note — Không chú thích

```
import numpy as np
vectors = np.array([[3,4],[0,0],[5,0],[-3,4],[0,5],[6,8],[1,1],[-4,0],[2,-2],[0,-2],[4,3],[0,0]],
dtype=float)
lengths = np.linalg.norm(vectors, axis=1)
valid = lengths > 0
normalized = np.zeros_like(vectors)
normalized[valid] = vectors[valid] / lengths[valid, None]
if len(vectors) != 12:
    raise ValueError("Cần đủ 12 véc-tơ cần chuẩn hóa.")
```

```
print("Yêu câù:", len(vectors))
print("Hướng hợp lệ:", int(valid.sum()))
print("Hướng chưa xác định:", int((~valid).sum()))
values = []
for x in normalized[0]:
    values.append(f"{x:.2f}")
print("Véc-tơ đâù:", ", ".join(values))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuẩn hóa 12 vector vê`hướng dài một đơn vị, tránh chia cho 0 ở vector rôñg.
# Dữ liệu: 12 hàng vector hai trục, gôm hai vector [0,0] không có hướng xác định.
# NumPy áp dụng phép toán vector hóa trên toàn bộ mảng 12×2.
import numpy as np
# dtype=float đê’phép chia ra thành phâǹ hướng thực; hai cột x/y.
vectors = np.array([[3,4],[0,0],[5,0],[-3,4],[0,5],[6,8],[1,1],[-4,0],[2,-2],[0,-2],[4,3],[0,0]],
dtype=float)
# norm(...,axis=1) tính độ dài riêng cho TỪNG HÀNG vector, trả 12 độ dài; axis=0 sẽ gom theo cột.
lengths = np.linalg.norm(vectors, axis=1)
# Tạo mặt nạ True tại những vector có độ dài khác 0; vector [0,0] không chia được.
valid = lengths > 0
# zeros_like() tạo mảng 12×2 toàn 0 cùng shape/dtype làm vùng chứa kêt quả.
normalized = np.zeros_like(vectors)
# vectors[valid] chọn những hàng có hướng; lengths[valid,None] đôỉ vector độ dài thành cột (n,1) đê’
broadcast chia cả x,y theo độ dài từng hàng.
# Chỉ cập nhật hàng hợp lệ; hàng vector 0 vâñ giữ [0,0] làm giá trị thay thê´không có hướng.
normalized[valid] = vectors[valid] / lengths[valid, None]
if len(vectors) != 12:
    # raise ValueError() chặn kêt luận nêú sô´bản ghi hoặc shape không đạt yêu câù.
    raise ValueError("Cân đủ 12 véc-tơ cân chuâñ hóa.")
print("Yêu câù:", len(vectors))
print("Hướng hợp lệ:", int(valid.sum()))
# ~ đảo mặt nạ Boolean, .sum() đêḿ hàng có độ dài 0; valid.sum() đêḿ hàng có hướng.
print("Hướng chưa xác định:", int((~valid).sum()))
values = []
# [0] lâý hai thành phâǹ hướng đã chuâñ hóa của vector đâù tiên.
for x in normalized[0]:
    # f-string :.2f hiên thị môĩ thành phâǹ với hai chữ sô´thập phân.
    values.append(f"{x:.2f}")
# join() nôí hai tọa độ dạng chuôĩ bằng ', ' thành một dòng báo cáo.
print("Véc-tơ đâù:", ", ".join(values))
```

Kết quả khi chạy:

```
Yêu câù: 12
Hướng hợp lệ: 10
Hướng chưa xác định: 2
Véc-tơ đâù: 0.60, 0.80
```

Véc-tơ không vẫn được bảo toàn trên bản dữ liệu xử lý nhưng **không** được coi là hướng di chuyển có độ dài 1.

Tình huống 3 — Cấp 3: Dot product cho 15 điểm tổng kết theo ba tiêu chí

Từ vựng trong mục: Dot product — tích vô hướng; **Weight** — trọng số; **Threshold** — ngưỡng.

Tình huống thực tế: Giáo viên lưu bảng **15 học viên H01–H15**, mỗi học viên có ba điểm **lý thuyết, thực hành, dự án** trên thang 10. Quy tắc chấm minh họa cho các phần này lần lượt **20%, 30%, 50%** và yêu cầu điểm cuối đạt **ít nhất 7,5**. Giáo vụ cần tính một điểm tổng kết cho mỗi người, xem điểm của H01 và H15, rồi đếm bao nhiêu người đạt, không ghép điểm giữa các học viên.

Vì sao chọn cách này? Tích vô hướng là cách kết hợp hai dãy số cùng thứ tự bằng việc nhân các phần tương ứng rồi cộng. Mỗi hàng là ba điểm **của đúng một học viên**, còn dãy trọng số có thứ tự **lý thuyết, thực hành, dự án**. `scores @ weights` áp một quy tắc tính điểm như nhau cho cả 15 hàng, tạo **15 điểm cuối**; dự án góp 50% nên có ảnh hưởng lớn nhất. Nếu đổi thứ tự các trọng số mà không đổi thứ tự cột điểm, kết quả sẽ sai. Điều kiện `>= 7.5` tính cả học viên đạt **đúng 7,5**, không chỉ người cao hơn ngưỡng.

Quy trình:

1. Tạo ma trận 15 × 3 và véc-tơ ba trọng số.
2. Nhân `scores @ weights` để có 15 kết quả.
3. Đếm người đạt ngưỡng 7,5; in điểm đầu và cuối để đối chiếu.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[5,6,7],[6,8,10],[7,10,9],[8,7,8],[9,9,7],[5,6,10],[6,8,9],[7,10,8],[8,7,7],
[9,9,10],[5,6,9],[6,8,8],[7,10,7],[8,7,10],[9,9,9]], dtype=float)
weights = np.array([0.2, 0.3, 0.5])
final_scores = scores @ weights
if scores.shape != (15, 3) or final_scores.shape != (15,):
    raise ValueError("Cần đủ 15 học viên với 3 môn và 15 điểm cuối.")
print("Học viên:", len(final_scores))
print(f"H01: {final_scores[0]:.2f}")
print(f"H15: {final_scores[-1]:.2f}")
print("Đạt 7.5:", int((final_scores >= 7.5).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính điểm tổng hợp có trọng số cho 15 học viên bằng tích vector theo mỗi hàng.
# Dữ liệu: 15 hàng học viên × 3 môn; trọng số giả định 20%, 30%, 50% tương ứng theo ba cột, tổng
100%.
# NumPy thực hiện phép nhân bảng điểm với vector trọng số qua @.
import numpy as np
# dtype=float lưu bảng 15×3, một người mỗi hàng, một môn mỗi cột.
scores = np.array([[5,6,7],[6,8,10],[7,10,9],[8,7,8],[9,9,7],[5,6,10],[6,8,9],[7,10,8],[8,7,7],
[9,9,10],[5,6,9],[6,8,8],[7,10,7],[8,7,10],[9,9,9]], dtype=float)
# Vector [0.2,0.3,0.5] là mức đóng góp TỪNG MÔN; không phải tân suất học viên.
weights = np.array([0.2, 0.3, 0.5])
# @ nhân từng hàng ba điểm với ba trọng số rồi cộng, trả 15 điểm tổng hợp một chiều.
final_scores = scores @ weights
# shape (15,3) và final_scores (15,) xác nhận một kết quả cho mỗi học viên.
if scores.shape != (15, 3) or final_scores.shape != (15,):
    # raise ValueError() chặn kết luận nếu số bản ghi hoặc shape không đạt yêu cầu.
    raise ValueError("Cần đủ 15 học viên với 3 môn và 15 điểm cuối.")
print("Học viên:", len(final_scores))
# [0] là H01, [-1] H15; :.2f hiển thị điểm hai chữ số lẻ.
print(f"H01: {final_scores[0]:.2f}")
# [-1] lấy điểm học viên cuối mà không phải ghi chỉ số 14.
print(f"H15: {final_scores[-1]:.2f}")
# >= 7.5 tạo mask Boolean theo từng người; sum() đếm số True đạt ngưỡng.
print("Đạt 7.5:", int((final_scores >= 7.5).sum()))
```

Kết quả khi chạy:

```
Học viên: 15
H01: 6.30
H15: 9.00
Đạt 7.5: 12
```

Trọng số phản ánh quy định giả lập của lớp; dot product tạo một số theo mỗi học viên, không phải một bảng 15 × 3 sau khi tính.

Tình huống 4 — Cấp 4: Phân 18 hướng vuông góc và ngược chiều với hướng chuẩn

Từ vựng trong mục: Orthogonality — vuông góc; **Dot product** — tích vô hướng; **Projection** — chiếu theo trục.

Tình huống thực tế: Robot nhận **18 lệnh di chuyển hai chiều** `(x, y)`; mỗi số cho biết mức điều khiển dọc một trục. Kỹ thuật viên lấy hướng chuẩn **sang phải** là véc-tơ `(1, 0)` và muốn chia các lệnh thành ba nhóm: có thành phần **cùng chiều** với hướng chuẩn, **ngược chiều**, hoặc **vuông góc**. Một lệnh vừa đi sang phải vừa đi lên vẫn có phần cùng chiều; không nên chỉ nhìn toàn bộ mũi tên mà kết luận theo cảm giác.

Vì sao chọn cách này? Tích vô hướng còn cho biết một hướng điều khiển có phần **cùng hay ngược** với hướng chuẩn. Với hướng chuẩn `(1,0)`, kết quả dương khi lệnh có phần đi sang phải; âm khi có phần đi sang trái; bằng 0 khi lệnh **vuông góc** với hướng sang phải, ví dụ chỉ đi lên hoặc xuống. `directions @ reference` tính cùng cách cho đủ 18 lệnh, sau đó chương trình đếm ba nhóm theo dấu kết quả. Vì dữ liệu mẫu là các số nguyên nhỏ và hướng chuẩn đơn giản, so đúng bằng 0 phù hợp ở đây; với dữ liệu đo số thực có sai số, có thể cần ngưỡng gần 0.

Quy trình:

1. Tạo mảng 18 hàng × 2 cột và véc-tơ chuẩn.
2. Tính 18 tích vô hướng `directions @ reference`.
3. Đếm trường hợp âm, 0 và dương; xác minh tổng bằng 18.

No Note — Không chú thích

```
import numpy as np
directions = np.array([[-1,0],[0,1],[1,0],[0,-1],[1,1],[-1,1],[-1,0],[0,1],[1,0],[0,-1],[1,1],
[-1,1],[-1,0],[0,1],[1,0],[0,-1],[1,1],[-1,1]], dtype=float)
reference = np.array([1.0, 0.0])
projections = directions @ reference
if directions.shape != (18, 2):
    raise ValueError("Cần đủ 18 lệnh chuyển động hai chiều.")
print("Lệnh:", len(directions))
print("Ngược chiều:", int((projections < 0).sum()))
print("Vuông góc:", int((projections == 0).sum()))
print("Cùng chiều:", int((projections > 0).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng tích vô hướng để phân loại 18 hướng so với trục +x: ngược, vuông góc, cùng hướng.
# Dữ liệu: 18 lệnh (dx,dy), trục tham chiếu [1,0] hướng +x; các giá trị chưa nhất thiết là vector
đơn vị.
# NumPy hỗ trợ tích vô hướng theo từng hàng của ma trận hướng.
import numpy as np
# 18 hàng là các hướng di chuyển 2 chiều x,y, dtype=float.
directions = np.array([[-1,0],[0,1],[1,0],[0,-1],[1,1],[-1,1],[-1,0],[0,1],[1,0],[0,-1],[1,1],
[-1,1],[-1,0],[0,1],[1,0],[0,-1],[1,1],[-1,1]], dtype=float)
# Vector tham chiếu nằm trên trục x dương, chiều dài 1; cùng thứ tự thành phần x,y.
reference = np.array([1.0, 0.0])
# @ lấy tích vô hướng với tham chiếu cho từng hàng, trả 18 scalar; vì reference=[1,0], chính là
thành phần x của hướng.
projections = directions @ reference
# shape (18,2) kiểm đủ lệnh và hai tọa độ.
if directions.shape != (18, 2):
```

```
    # raise ValueError() chặn kết luận nếu số´bản ghi hoặc shape không đạt yêu câù.
    raise ValueError("Cần đủ 18 lệnh chuyển động hai chiều.")
print("Lệnh:", len(directions))
# Tích âm nghĩa ngược chiều x dương; sum() đếm số´mask True.
print("Ngược chiều:", int((projections < 0).sum()))
# Tích bằng 0 nghĩa vuông góc với trục x (đối với vector khác 0), đếm lượt bằng 0.
print("Vuông góc:", int((projections == 0).sum()))
# Tích dương nghĩa có thành phần cùng chiều x dương, không nhất thiết vector song song trục.
print("Cùng chiều:", int((projections > 0).sum()))
```

Kết quả khi chạy:

```
Lệnh: 18
Ngược chiều: 6
Vuông góc: 6
Cùng chiều: 6
```

Phân loại bằng tích vô hướng là về **thành phần hướng so với véc-tơ chuẩn**, không suy ra robot đã thực sự di chuyển bao xa.

Tình huống 5 — Cấp 5: Xếp hạng sai số 20 cảm biến ba trục bằng độ dài véc-tơ

Từ vựng trong mục: Residual vector — véc-tơ sai lệch; **Norm** — độ lớn sai số; **Ranking** — xếp hạng.

Tình huống thực tế: Nhà máy có **20 lượt đo vị trí cảm biến ba chiều**, mỗi lượt ghi vị trí quan sát và vị trí kỳ vọng theo ba trục **X, Y, Z**, tất cả tính bằng **mm**. Có lượt lệch nhiều trên một trục, có lượt lệch nhỏ theo cả ba trục. Nhân viên muốn biết **độ lớn sai lệch** của mỗi lượt, đếm những lượt lệch **lớn hơn 2 mm**, xác định lượt sai nhất và mức lệch trung bình; đây chỉ là kiểm tra số liệu hình học, chưa kết luận thiết bị bị hỏng.

Vì sao chọn cách này? Sai lệch véc-tơ là chênh lệch giữa vị trí cảm biến quan sát và vị trí dự kiến theo cả ba trục. `observed - expected` cho một bộ ba sai lệch của từng lượt; `np.linalg.norm(..., axis=1)` chuyển mỗi bộ ba thành **một độ lớn luôn không âm**, đơn vị vẫn là mm. Điều này tránh việc lệch +2 mm trên trục X và -2 mm trên trục Y cộng lại thành 0 rồi bị hiểu nhầm là đo đúng. Sau đó so độ lớn với **ngưỡng 2 mm**, dùng `argmax()` tìm **vị trí lượt có sai số lớn nhất**, và `mean()` tính mức sai lệch trung bình. Ngưỡng 2 mm là quy tắc bài tập, chưa phải chuẩn chất lượng công nghiệp.

Quy trình:

1. Lưu 20 cặp véc-tơ 3D theo đúng thứ tự trục.
2. Tính sai lệch từng hàng, sau đó norm của 20 véc-tơ.
3. Đếm vượt ngưỡng 2 mm, tìm mã lượt lớn nhất và báo trung bình.

No Note — Không chú thích

```
import numpy as np
observed = np.array([[10,20,30],[11,21,31],[12,22,32],[13,23,30],[14,20,31],[10,21,32],[11,22,30],
[12,23,31],[13,20,32],[14,21,30],[10,22,31],[11,23,32],[12,20,30],[13,21,31],[14,22,32],[10,23,30],
[11,20,31],[12,21,32],[13,22,30],[14,23,31]], dtype=float)
expected = np.array([[11,19,33],[12,20,31],[13,21,32],[14,22,30],[15,19,31],[11,20,32],[12,21,30],
[13,22,34],[14,19,32],[15,20,30],[11,21,31],[12,22,32],[13,19,30],[14,20,31],[15,21,35],[11,22,30],
[12,19,31],[13,20,32],[14,21,30],[15,22,31]], dtype=float)
residual = observed - expected
errors = np.linalg.norm(residual, axis=1)
worst = int(errors.argmax())
if observed.shape != (20, 3) or expected.shape != (20, 3):
    raise ValueError("Cần 20 lượt đo, mỗi lượt có ba tọa độ nguồn và kỳ vọng.")
```

```
print("Lượt đo:", len(errors))
print("Vượt 2 mm:", int((errors > 2).sum()))
print("Lượt sai nhất:", f"M{worst+1:02d}")
print(f"Sai số lớn nhất (mm): {errors[worst]:.2f}")
print(f"Sai số TB (mm): {errors.mean():.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính vector sai lệch 3D và độ dài sai số của 20 lượt đo, tìm lượt sai nhất.
# Dữ liệu: 20 hàng quan sát và 20 hàng kỳ vọng theo 3 trục tọa độ cùng thứ tự, đơn vị mm.
# NumPy cho phép trừ bảng theo từng ô và tính độ dài sai lệch vector.
import numpy as np
# 20 hàng đo thực tế, mỗi hàng [x,y,z] mm, dtype=float.
observed = np.array([[10,20,30],[11,21,31],[12,22,32],[13,23,30],[14,20,31],[10,21,32],[11,22,30],
[12,23,31],[13,20,32],[14,21,30],[10,22,31],[11,23,32],[12,20,30],[13,21,31],[14,22,32],[10,23,30],
[11,20,31],[12,21,32],[13,22,30],[14,23,31]], dtype=float)
# 20 hàng tọa độ kỳ vọng cùng thứ tự với observed, cũng đơn vị mm.
expected = np.array([[11,19,33],[12,20,31],[13,21,32],[14,22,30],[15,19,31],[11,20,32],[12,21,30],
[13,22,34],[14,19,32],[15,20,30],[11,21,31],[12,22,32],[13,19,30],[14,20,31],[15,21,35],[11,22,30],
[12,19,31],[13,20,32],[14,21,30],[15,22,31]], dtype=float)
# Trừ THEO TỪNG Ô cùng hàng/cột để lấy vector sai số tọa độ của mỗi lượt.
residual = observed - expected
# norm(...,axis=1) tính độ dài Euclid vector sai số ba trục cho từng hàng, trả 20 khoảng cách mm.
errors = np.linalg.norm(residual, axis=1)
# argmax() trả CHỈ SỐ 0-based có sai số lớn nhất, không trả trị số sai số; int() chuyển thành int
Python.
worst = int(errors.argmax())
# shape (20,3) của cả hai bảng bảo đảm so đúng số lượt và số trục.
if observed.shape != (20, 3) or expected.shape != (20, 3):
    # raise ValueError() chặn kết luận nếu số bản ghi hoặc shape không đạt yêu cầu.
    raise ValueError("Cần 20 lượt đo, mỗi lượt có ba tọa độ nguồn và kỳ vọng.")
print("Lượt đo:", len(errors))
# So từng sai số với ngưỡng 2 mm, sum() trên Boolean đếm lượt vượt ngưỡng.
print("Vượt 2 mm:", int((errors > 2).sum()))
# Chỉ số argmax bắt đầu 0 nên +1 lấy số lượt, :02d tạo mã M01..M20.
print("Lượt sai nhất:", f"M{worst+1:02d}")
# Lấy sai số đúng lượt lớn nhất, :.2f hiển thị mm hai chữ số lẻ.
print(f"Sai số lớn nhất (mm): {errors[worst]:.2f}")
# mean() lấy độ dài sai số trung bình của 20 lượt, không phải trung bình từng tọa độ.
print(f"Sai số TB (mm): {errors.mean():.2f}")
```

Kết quả khi chạy:

```
Lượt đo: 20
Vượt 2 mm: 3
Lượt sai nhất: M01
Sai số lớn nhất (mm): 3.32
Sai số TB (mm): 1.70
```

Mỗi hàng là một lượt đo 3 trục có cùng đơn vị mm. Mức 2 mm là **ngưỡng ví dụ**, phải do chuyên gia thiết bị quyết định khi ứng dụng thật.

6. Sai lầm thường gặp

- Scalar là số đơn; vector là dãy các thành phần có thứ tự.
- Khi tính khoảng cách, các thành phần phải cùng hệ trục và đơn vị phù hợp.
- Không chuẩn hóa vector không vì norm bằng 0.
- Dấu `*` NumPy nhân từng phần tử; `@` với hai vector 1-D tính tích vô hướng.

• Khi kiểm tra vuông góc với dữ liệu float, nên so với một ngưỡng sai số nhỏ.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a scalar? | Scalar là gì?

A. One numeric value | Một giá trị số

B. Sum of pairwise component products | Tổng tích các cặp thành phần tương ứng

C. An ordered list of components | Dãy thành phần có thứ tự

D. Square root of the sum of squared components | Căn của tổng bình phương các thành phần

Đáp án: A

Giải thích: Scalar không gồm nhiều thành phần tọa độ.

Q2. What is a vector? | Vector là gì?

A. Square root of the sum of squared components | Căn của tổng bình phương các thành phần

B. One numeric value | Một giá trị số

C. Sum of pairwise component products | Tổng tích các cặp thành phần tương ứng

D. An ordered list of components | Dãy thành phần có thứ tự

Đáp án: D

Giải thích: Vị trí từng thành phần biểu thị tọa độ hoặc đặc trưng.

Q3. What is Euclidean norm? | Norm Euclid là gì?

A. An ordered list of components | Dãy thành phần có thứ tự

B. Sum of pairwise component products | Tổng tích các cặp thành phần tương ứng

C. Square root of the sum of squared components | Căn của tổng bình phương các thành phần

D. One numeric value | Một giá trị số

Đáp án: C

Giải thích: Norm L2 đo độ dài theo Euclid.

Q4. What is the dot product? | Tích vô hướng là gì?

A. Sum of pairwise component products | Tổng tích các cặp thành phần tương ứng

B. An ordered list of components | Dãy thành phần có thứ tự

C. One numeric value | Một giá trị số

D. Square root of the sum of squared components | Căn của tổng bình phương các thành phần

Đáp án: A

Giải thích: Tích vô hướng cho một số từ hai vector cùng số thành phần.

Q5. Given u = [1, 4] and v = [3, 2], what is u · v? | Cho u = [1, 4] và v = [3, 2], tích vô hướng u · v bằng bao nhiêu?

A. 9 | 9

B. 11 | 11

C. 14 | 14

D. 8 | 8

Đáp án: B

Giải thích: Tính các tích cùng vị trí rồi cộng: 1 × 3 + 4 × 2 = 11. Không cộng riêng từng vector trước khi nhân.

Q6. What is the length of vector (6, 8)? | Độ dài vector (6, 8) là bao nhiêu?

A. 10 | 10

B. 8 | 8

C. 14 | 14

D. 100 | 100

Đáp án: A

Giải thích: Độ dài Euclid là căn bậc hai của $6^2 + 8^2 = 100$, nên bằng 10.

Q7. If u = (2, −1), what is −2u? | Nếu u = (2, −1), vector −2u bằng bao nhiêu?

A. (−4, 2) | (−4, 2)

B. (−2, 1) | (−2, 1)

C. (4, −2) | (4, −2)

D. (−4, −2) | (−4, −2)

Đáp án: A

Giải thích: Nhân từng tọa độ của u với −2: −2 × 2 = −4 và −2 × (−1) = 2.

Q8. Given a = (2, 1) and b = (−1, 2), what is their dot product? | Cho a = (2, 1) và b = (−1, 2), tích vô hướng bằng bao nhiêu?

A. 1 | 1

B. 0 | 0

C. 4 | 4

D. 3 | 3

Đáp án: B

Giải thích: 2 × (−1) + 1 × 2 = 0. Hai vector khác không có tích vô hướng bằng 0 thì vuông góc.

Q9. What is the norm of vector (3,4)? | Norm của vector (3,4) bằng bao nhiêu?

A. 0 | 0

B. 11 | 11

C. 5 | 5

D. (0.6,0.8) | (0.6,0.8)

Đáp án: C

Giải thích: Độ dài Pythagoras là căn của 9+16.

Q10. What is the unit vector in the direction (3,4)? | Vector đơn vị theo hướng (3,4) là gì?

A. 5 | 5

B. (0.6,0.8) | (0.6,0.8)

C. 11 | 11

D. 0 | 0

Đáp án: B

Giải thích: Chia mỗi thành phần cho độ dài năm.

Q11. What is the dot product (1,2)·(3,4)? | Tích vô hướng (1,2)·(3,4) bằng bao nhiêu?

A. 5 | 5

B. 11 | 11

C. (0.6,0.8) | (0.6,0.8)

D. 0 | 0

Đáp án: B

Giải thích: Tính 1×3 + 2×4.

Q12. What is the dot product of (1,0) and (0,1)? | Tích vô hướng của (1,0) và (0,1) là bao nhiêu?

A. (0.6,0.8) | (0.6,0.8)

B. 0 | 0

C. 5 | 5

D. 11 | 11

Đáp án: B

Giải thích: Cả hai tích thành phần đều bằng không.

Q13. What is the dot product of two nonzero perpendicular vectors? | Tích vô hướng của hai vector khác không vuông góc là gì?

A. That its norm is nonzero | Norm khác không

B. Zero | Bằng không

C. They must have equal lengths | Chúng phải có cùng số phần tử

D. The result can be physically misleading | Kết quả có thể sai ý nghĩa vật lý

Đáp án: B

Giải thích: Hai hướng trực giao có tích vô hướng bằng không.

Q14. What must be checked before normalizing a vector? | Cần kiểm tra gì trước khi chuẩn hóa vector?

A. They must have equal lengths | Chúng phải có cùng số phần tử

B. Zero | Bằng không

C. The result can be physically misleading | Kết quả có thể sai ý nghĩa vật lý

D. That its norm is nonzero | Norm khác không

Đáp án: D

Giải thích: Chia cho norm bằng không không xác định.

Q15. What happens if coordinate units are mixed in a distance calculation? | Nếu đơn vị tọa độ không thống nhất khi tính khoảng cách thì sao?

A. They must have equal lengths | Chúng phải có cùng số phần tử

B. That its norm is nonzero | Norm khác không

C. The result can be physically misleading | Kết quả có thể sai ý nghĩa vật lý

D. Zero | Bằng không

Đáp án: C

Giải thích: Tính norm có ý nghĩa khi đơn vị phù hợp.

Q16. What must be true to compute a 1-D dot product? | Muốn tính tích vô hướng của hai vector 1-D cần điều kiện gì?

A. That its norm is nonzero | Norm khác không

B. They must have equal lengths | Chúng phải có cùng số phần tử

C. Zero | Bằng không

D. The result can be physically misleading | Kết quả có thể sai ý nghĩa vật lý

Đáp án: B

Giải thích: Từng thành phần được ghép theo vị trí tương ứng.

Q17. What does multiplying a vector by a scalar do? | Nhân vector với scalar làm gì?

A. To multiply each weight by its intended feature | Để trọng số nhân đúng đặc trưng tương ứng

B. Subtract corresponding components | Trừ các thành phần cùng vị trí

C. Scales every component | Nhân từng thành phần với scalar

D. Euclidean distance between a and b | Khoảng cách Euclid giữa a và b

Đáp án: C

Giải thích: Nhân scalar làm đổi độ lớn từng thành phần.

Q18. How is vector subtraction performed? | Phép trừ vector được thực hiện thế nào?

A. Scales every component | Nhân từng thành phần với scalar

B. Euclidean distance between a and b | Khoảng cách Euclid giữa a và b

C. To multiply each weight by its intended feature | Để trọng số nhân đúng đặc trưng tương ứng

D. Subtract corresponding components | Trừ các thành phần cùng vị trí

Đáp án: D

Giải thích: Phép trừ vector diễn ra theo từng thành phần.

Q19. What does `np.linalg.norm(a-b)` measure? | `np.linalg.norm(a-b)` đo điều gì?

A. To multiply each weight by its intended feature | Để trọng số nhân đúng đặc trưng tương ứng

B. Subtract corresponding components | Trừ các thành phần cùng vị trí

C. Scales every component | Nhân từng thành phần với scalar

D. Euclidean distance between a and b | Khoảng cách Euclid giữa a và b

Đáp án: D

Giải thích: Norm của độ dịch chuyển cho khoảng cách.

Q20. Why are feature weights paired with values in the same order? | Vì sao trọng số cần cùng thứ tự với các giá trị?

A. Euclidean distance between a and b | Khoảng cách Euclid giữa a và b

B. To multiply each weight by its intended feature | Để trọng số nhân đúng đặc trưng tương ứng

C. Scales every component | Nhân từng thành phần với scalar

D. Subtract corresponding components | Trừ các thành phần cùng vị trí

Đáp án: B

Giải thích: Sai thứ tự tạo ra tích vô hướng khác.

8. Nối sang bài kế tiếp

`NumPy → Scalar/Vector → Matrix → PCA`.

PY-25 — Ma trận, chuyển vị, nhân ma trận và định thức

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Matrix — ma trận; **Transpose** — chuyển vị.

`Vector → Matrix → Hệ tuyến tính → Eigenvalue/PCA`

Học xong dùng để làm gì? Tổ chức bảng đặc trưng thành ma trận và nhân ma trận để biến đổi dữ liệu nhiều chiều.

Kết nối kiến thức tiếp theo: PY-26 giải các ràng buộc dưới dạng hệ tuyến tính và kiểm tra rank.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Matrix	ma trận	Bảng hai chiều chứa giá trị theo hàng và cột
Transpose	chuyển vị	Đổi vai trò hàng và cột của ma trận
Determinant	định thức	Số của ma trận vuông dùng xét tính khả nghịch
Inverse	ma trận nghịch đảo	Ma trận B làm AB=BA=I khi tồn tại

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.eye(n)`	Ma trận đơn vị vuông, nhân vào không đổi vector
`A.T / np.transpose(A)`	Chuyển vị ma trận
`A @ B`	Nhân ma trận khi số cột A bằng số hàng B
`np.linalg.det()/inv()`	Định thức và nghịch đảo khi khả nghịch; giải hệ nên ưu tiên solve

4. Khái niệm liên quan

Ma trận (matrix) là bảng có hàng và cột được xác định rõ theo dữ liệu. Từ bảng điểm hai học viên và ba môn, bài học giải thích chuyển vị để đổi cách xem, nhân ma trận để tổng hợp trọng số, rồi định thức và nghịch đảo khi làm việc với ma trận vuông.

4.1. Matrix — Bảng số có hàng và cột

Ma trận là bảng số hai chiều. Với điểm học viên An `[8,7,9]` và Bình `[6,5,8]` ở ba môn Toán, Văn, Python, bảng có **hai hàng học viên** và **ba cột môn**. Giá trị `[1,0]` nghĩa là điểm Toán của Bình bằng 6. `shape` cho kích thước, theo thứ tự (số hàng, số cột).

No Note — Không chú thích

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.shape)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Quan sát shape của ma trận điểm có hai hàng học viên và ba cột môn.
# Dữ liệu: 2 học viên, 3 môn; mỗi hàng [điểm môn1, môn2, môn3].
# NumPy biểu diễn ma trận bằng ndarray hai chiều.
import numpy as np

# Hai list con tạo hai hàng, mỗi hàng có ba phần tử (cột).
scores = np.array([[8, 7, 9], [6, 5, 8]])
# shape trả tuple (số hàng, số cột), ở đây (2,3); khác size là tổng 6 ô.
print(scores.shape)
```

Kết quả khi chạy:

```
(2, 3)
```

Bảng có hai học viên và ba môn học.

4.2. Transpose — Hoán đổi hai trục

Chuyển vị `A.T` biến hàng thành cột và cột thành hàng. Với bảng điểm hai học viên × ba môn, ma trận sau chuyển vị có ba hàng môn và hai cột học viên. Ví dụ hàng đầu là điểm Toán của An và Bình, `[8,6]`; không có điểm nào bị thay đổi.

No Note — Không chú thích

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.T)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng thuộc tính .T chuyển vị ma trận hai hàng ba cột.
# Dữ liệu: Ma trận gốc 2 học viên × 3 môn; sau chuyển vị 3 môn × 2 học viên.
# NumPy cho thuộc tính .T để đổi thứ tự trục.
import numpy as np

# Mỗi hàng là điểm của một học viên, mỗi cột là một môn.
scores = np.array([[8, 7, 9], [6, 5, 8]])
# .T đổi hàng thành cột và ngược lại; ma trận có shape (3,2). Với ndarray thông thường là view,
```

```
không tự sao chép dữ liệu.
print(scores.T)
```

Kết quả khi chạy:

```
[[8 6]
 [7 5]
 [9 8]]
```

Kết quả gồm ba hàng môn × hai cột học viên.

4.3. Matrix multiplication — Nhân ma trận

Nhân `X @ W` yêu cầu số cột của X bằng số hàng của W. Bảng điểm X có shape `(2,3)`, trọng số ba môn W có shape `(3,1)`, tích có shape `(2,1)` cho điểm tổng của từng học viên. Dòng An là `8×0.2 + 7×0.3 + 9×0.5 = 8.2`. `@` khác phép nhân từng phần tử `*`.

No Note — Không chú thích

```
import numpy as np

X = np.array([[8, 7, 9], [6, 5, 8]])
W = np.array([[0.2], [0.3], [0.5]])
print(X @ W)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhân ma trận điểm X với vector trọng sô´cột W đểcho hai điểm tông hợp.
# Dữ liệu: X có shape (2,3), W có shape (3,1) gôm̀ ba hệ sô´0.2/0.3/0.5.
# np.array() tạo ma trận và toán tử@ nhân ma trận.
import numpy as np

# X có hai hàng học viên, ba cột môn.
X = np.array([[8, 7, 9], [6, 5, 8]])
# W là cột 3×1, môĩ hệ sô´là trọng sô´môn, tông bằng 1.
W = np.array([[0.2], [0.3], [0.5]])
# @ nhân ma trận theo quy tăć hàng × cột: (2,3) @ (3,1) → (2,1), môĩ kêt quả là tông điêm̉ có trọng
sô; KHÁC dâú * nhân từng ô.
print(X @ W)
```

Kết quả khi chạy:

```
[[8.2]
 [6.7]]
```

Hai hàng là hai học viên; một cột là điểm tổng kết của mỗi người.

4.4. Determinant và inverse — Khả nghịch

Định thức `det(A)` là số xác định cho **ma trận vuông**. Với A=`[[2,1],[1,2]]`, định thức bằng `2×2 - 1×1 = 3`, khác 0, nên tồn tại ma trận nghịch đảo A^{-1} thỏa `A @ A⁻¹ = I`. Nếu định thức bằng 0 thì không có nghịch đảo. Khi giải hệ phương trình, ưu tiên `np.linalg.solve` hơn việc tính nghịch đảo trực tiếp.

No Note — Không chú thích

```
import numpy as np

A = np.array([[2.0, 1.0], [1.0, 2.0]])
print(round(float(np.linalg.det(A)), 6))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính định thức để nhận biết ma trận vuông hai chiều có khả nghịch.
# Dữ liệu: Ma trận A=[[2,1],[1,2]], định thức là 3 khác 0 nên tồn tại nghịch đảo.
# NumPy linalg có hàm det() tính định thức ma trận vuông.
import numpy as np

# dtype suy ra float, A là ma trận 2 hàng × 2 cột.
A = np.array([[2.0, 1.0], [1.0, 2.0]])
# det(A) trả định thức, ở đây 2×2−1×1=3; khác 0 nghĩa A có nghịch đảo trong toán học.
# float() đưa NumPy scalar về số Python; round(...,6) làm tròn khi IN để giảm sai số hiển thị dấu
phẩy động.
print(round(float(np.linalg.det(A)), 6))
```

Kết quả khi chạy:

```
3.0
```

Định thức 3 khác 0, nên ma trận vuông này khả nghịch.

5. Năm tình huống ma trận — từ chuyển vị đến kiểm tra hệ suy biến

Tình huống 1 — Cấp 1: Lập ma trận bán hàng từ 10 lượt ghi

Từ vựng trong mục: Matrix — ma trận; **Transpose** — chuyển vị; **Shape** — số hàng/cột.

Tình huống thực tế: Một chuỗi có **5 cửa hàng S01–S05**, mỗi cửa hàng ghi doanh thu của **hai ngày liên tiếp**, tính bằng **triệu đồng/ngày**. Phần mềm xuất **10 con số theo thứ tự**: S01 ngày 1, S01 ngày 2, rồi tới S02… Nhân viên muốn xếp thành bảng **5 hàng cửa hàng × 2 cột ngày**, đọc tổng hai ngày của S03, sau đó chuyển bảng sang cách xem theo **ngày** để đối chiếu, không làm mất số liệu nào.

Vì sao chọn cách này? Ma trận là bảng số theo hàng và cột có ý nghĩa rõ ràng. `reshape(5, 2)` chia dãy 10 số thành **mỗi hàng hai ngày của cùng cửa hàng** theo đúng thứ tự nguồn; không đo hay sắp xếp thêm. Vì Python đánh số hàng từ 0, `sales[2]` là cửa hàng thứ ba S03; cộng hai ô của hàng ấy để ra tổng **triệu đồng** trong hai ngày. `.T` là **chuyển vị**: đổi thành hai hàng ngày × năm cột cửa hàng để có góc nhìn ngược lại. Nếu đầu vào không theo đúng thứ tự S01 ngày 1 rồi ngày 2 thì reshape không tự biết để sửa.

Quy trình:

1. Lưu 10 số liệu theo thứ tự cửa hàng–ngày.
2. Reshape thành ma trận `(5,2)`, tổng hai ngày của S03.
3. Chuyển vị sang `(2,5)` và đối chiếu tổng số phần tử.

No Note — Không chú thích

```
import numpy as np
raw = np.array([10,12,9,14,11,13,15,16,12,18], dtype=float)
sales = raw.reshape(5, 2)
by_day = sales.T
```

```
if raw.size != 10 or sales.size != 10 or by_day.size != 10:
    raise ValueError("Cần giữ đủ 10 số´liệu khi đổi dạng và chuyển vị.")
print("Lượt ghi:", raw.size)
print("Bảng cửa hàng:", sales.shape)
print("Bảng theo ngày:", by_day.shape)
print("S03 (triệu đồng):", int(sales[2].sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đổi mười lượt doanh thu thành ma trận 5 cửa hàng × 2 ngày rồi chuyển vị.
# Dữ liệu: 10 doanh số´triệu đồng, mỗi hai số´liên tiếp thuộc một cửa hàng ở hai ngày.
# NumPy tạo ndarray và chuyển hình dạng bảng.
import numpy as np
# 10 giá trị doanh số´float theo thứ tự hai ngày mỗi cửa hàng.
raw = np.array([10,12,9,14,11,13,15,16,12,18], dtype=float)
# reshape(5,2) tạo 5 hàng cửa hàng × 2 cột ngày, không đổi thứ tự số´liệu.
sales = raw.reshape(5, 2)
# .T chuyển vị từ (5,2) thành (2,5): mỗi hàng là một ngày của năm cửa hàng.
by_day = sales.T
# size đếm tổng 10 ô; reshape và chuyển vị phải vẫn có 10 ô.
if raw.size != 10 or sales.size != 10 or by_day.size != 10:
    # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
    raise ValueError("Cần giữ đủ 10 số´liệu khi đổi dạng và chuyển vị.")
# size đếm tổng 10 ô; reshape và chuyển vị phải vẫn có 10 ô.
print("Lượt ghi:", raw.size)
# shape (5,2) còn by_day (2,5); phân biệt vị trí của cửa hàng/ngày.
print("Bảng cửa hàng:", sales.shape)
print("Bảng theo ngày:", by_day.shape)
# [2] lấy hàng cửa hàng S03 (index từ 0), sum() cộng doanh thu của hai ngày; int() in triệu đồng.
print("S03 (triệu đồng):", int(sales[2].sum()))
```

Kết quả khi chạy:

```
Lượt ghi: 10
Bảng cửa hàng: (5, 2)
Bảng theo ngày: (2, 5)
S03 (triệu đồng): 24
```

Đầu ra tổng của S03 là doanh thu **hai ngày** theo đơn vị triệu đồng, không phải trung bình ngày.

Tình huống 2 — Cấp 2: Nhân ma trận 12 học viên × 3 tiêu chí với 2 thang đánh giá

Từ vựng trong mục: Matrix multiplication — nhân ma trận; **Weights** — trọng số; **Dimension** — kích thước.

Tình huống thực tế: Trung tâm có **12 học viên**, mỗi người được chấm ba phần **lý thuyết, thực hành, dự án** (thang 10). Để đánh giá toàn diện, trường dùng **hai bộ trọng số giả lập**: điểm **chuyên môn** lấy 50%, 20%, 30%, còn điểm **hợp tác** lấy 10%, 30%, 60% tương ứng ba phần. Giáo viên muốn tính **cả hai điểm cho từng người** và đếm bao nhiêu người có điểm hợp tác từ 8 trở lên.

Vì sao chọn cách này? Ma trận trọng số giúp áp **hai cách chấm điểm** lên cùng ba cột điểm một lượt. Mỗi hàng của bảng điểm là **một học viên**; mỗi cột của bảng trọng số là **một bộ quy tắc đánh giá**. Phép `scores @ weights` ghép đúng lý thuyết–thực hành–dự án theo từng bộ, nên trả về **12 hàng học viên × 2 cột đánh giá**; một cột chuyên môn, một cột hợp tác. Các phần trăm trong **mỗi bộ** cộng thành 100%, nên điểm vẫn trên thang tương ứng. `ratings[:, 1]` lấy cột hợp tác để đếm người đạt từ 8; không chọn nhầm cột chuyên môn hoặc nhân từng vị trí rời rạc.

Quy trình:

1. Ghi rõ thứ tự ba tiêu chí và hai nhóm trọng số.
2. Kiểm tra kích thước 12×3 và 3×2; dùng `@`.
3. In hai điểm của người đầu tiên và đếm ngưỡng hợp tác.

No Note — Không chú thích

```
import numpy as np
scores = np.array([[5,6,7],[6,8,10],[7,10,9],[8,7,8],[9,9,7],[5,6,10],[6,8,9],[7,10,8],[8,7,7],
[9,9,10],[5,6,9],[6,8,8]], dtype=float)
weights = np.array([[0.5, 0.1], [0.2, 0.3], [0.3, 0.6]])
ratings = scores @ weights
if scores.shape != (12, 3) or weights.shape != (3, 2):
    raise ValueError("Bảng điểm 12x3 phải ghép đúng ma trận trọng số 3x2.")
if ratings.shape != (12, 2):
    raise ValueError("Kết quả phải có hai đánh giá cho mỗi trong 12 học viên.")
print("Học viên:", ratings.shape[0])
print("Cột đánh giá:", ratings.shape[1])
print(f"H01 chuyên môn: {ratings[0,0]:.2f}")
print(f"H01 hợp tác: {ratings[0,1]:.2f}")
print("Hợp tác >=8:", int((ratings[:,1] >= 8).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhân bảng điểm 12 học viên × 3 môn với ma trận trọng số 3×2 để tính hai đánh giá.
# Dữ liệu: Mỗi hàng điểm 3 môn; weights là ba hệ số cho hai tiêu chí chuyên môn/hợp tác, mỗi cột
trọng số tổng bằng 1.
# NumPy thực hiện nhân ma trận với @ cho toàn bộ học viên.
import numpy as np
# Mảng float shape (12,3), mỗi hàng một học viên, ba cột điểm môn.
scores = np.array([[5,6,7],[6,8,10],[7,10,9],[8,7,8],[9,9,7],[5,6,10],[6,8,9],[7,10,8],[8,7,7],
[9,9,10],[5,6,9],[6,8,8]], dtype=float)
# Ma trận (3,2) gồm trọng số theo môn ở ba hàng và hai tiêu chí ở hai cột.
weights = np.array([[0.5, 0.1], [0.2, 0.3], [0.3, 0.6]])
# @ nhân (12,3)×(3,2)→(12,2): mỗi học viên nhận hai điểm tổng hợp, không dùng * theo ô.
ratings = scores @ weights
# Kiểm hai kích thước trong phép nhân khớp ở trục trong (3), tránh sai số học viên/môn.
if scores.shape != (12, 3) or weights.shape != (3, 2):
    # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
    raise ValueError("Bảng điểm 12x3 phải ghép đúng ma trận trọng số 3x2.")
# shape (12,2) chứng minh có đúng hai tiêu chí trên mỗi người.
if ratings.shape != (12, 2):
    # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
    raise ValueError("Kết quả phải có hai đánh giá cho mỗi trong 12 học viên.")
# shape (12,2) chứng minh có đúng hai tiêu chí trên mỗi người.
print("Học viên:", ratings.shape[0])
# shape (12,2) chứng minh có đúng hai tiêu chí trên mỗi người.
print("Cột đánh giá:", ratings.shape[1])
# [0,0] lấy đánh giá chuyên môn của H01, [0,1] lấy hợp tác; :.2f in hai số thập phân.
print(f"H01 chuyên môn: {ratings[0,0]:.2f}")
print(f"H01 hợp tác: {ratings[0,1]:.2f}")
# [:,1] lấy cột hợp tác của mọi học viên; >=8 tạo mask Bool; .sum() đếm số học viên đạt.
print("Hợp tác >=8:", int((ratings[:,1] >= 8).sum()))
```

Kết quả khi chạy:

```
Học viên: 12
Cột đánh giá: 2
H01 chuyên môn: 5.80
```

```
H01 hợp tác: 6.50
Hợp tác >=8: 6
```

Đầu ra có hai cột **với ý nghĩa khác nhau**; tỷ lệ đạt là yêu cầu giả lập, không phải thang chấm chính thức.

Tình huống 3 — Cấp 3: Đổi hệ trục 15 tọa độ bằng ma trận quay khả nghịch

Từ vựng trong mục: Transformation — phép biến đổi; **Determinant** — định thức; **Inverse** — ma trận nghịch đảo.

Tình huống thực tế: Robot đã ghi **15 điểm trên sàn**, mỗi điểm là cặp tọa độ `(x, y)` tính bằng **mét** so với gốc. Hệ thống muốn đổi cách biểu diễn các điểm bằng phép **quay góc 90°** quanh gốc theo quy ước véc-tơ hàng của bài, nhưng không được làm điểm tự nhiên gần gốc hơn hay xa gốc hơn. Kỹ thuật viên cần đọc tọa độ trước/sau của điểm đầu, xác nhận phép quay có thể đảo ngược và bảo toàn khoảng cách tới gốc cho cả 15 điểm.

Vì sao chọn cách này? Ma trận quay mô tả cách biến đổi tọa độ nhưng vẫn giữ hình dạng khi chỉ quay quanh gốc. Chương trình nhân từng cặp `(x,y)` với ma trận quay để nhận tọa độ mới; ví dụ điểm đầu `(1,2)` có tọa độ mới khác nhưng **độ dài từ gốc** vẫn bằng cũ. **Định thức** là một con số cho biết phép biến đổi có thể khôi phục ngược hay không: định thức khác 0 nghĩa là có thể đảo ngược, còn bằng 0 sẽ làm mất thông tin về một hướng. `np.linalg.norm()` đo khoảng cách tới gốc trước/sau, `np.allclose()` so hai dãy có **gần bằng nhau trong sai số máy tính** không. Chỉ kiểm định thức khác 0 chưa đủ để kết luận là phép quay vì nhiều phép co giãn cũng đảo ngược được; vì vậy cần kiểm tra khoảng cách nữa.

Quy trình:

1. Tạo ma trận 15×2 chứa các điểm có đơn vị mét.
2. Nhân với ma trận xoay 2×2, kiểm tra định thức khác 0.
3. Dùng `np.allclose` kiểm độ dài và in điểm đầu trước/sau.

No Note — Không chú thích

```
import numpy as np
points = np.array([[1,2],[2,2],[3,2],[4,2],[5,2],[1,3],[2,3],[3,3],[4,3],[5,3],[1,4],[2,4],[3,4],
[4,4],[5,4]], dtype=float)
rotate = np.array([[0.0, -1.0], [1.0, 0.0]])
converted = points @ rotate
if points.shape != (15, 2) or converted.shape != (15, 2):
    raise ValueError("Phải có đúng 15 điểm gốc và điểm đã đổi tọa độ.")
if not np.allclose(np.linalg.norm(points, axis=1), np.linalg.norm(converted, axis=1)):
    raise ValueError("Phép quay phải bảo toàn chiều dài của từng điểm.")
print("Điểm:", len(points))
print(f"Định thức: {np.linalg.det(rotate):.1f}")
original_labels = []
converted_labels = []
for value in points[0]:
    original_labels.append(f"{value:.1f}")
for value in converted[0]:
    converted_labels.append(f"{value:.1f}")
print("Điểm đầu (gốc):", ", ".join(original_labels))
print("Điểm đầu (mới):", ", ".join(converted_labels))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng ma trận 2×2 biến đổi tọa độ 15 điểm và kiểm giữ nguyên độ dài.
# Dữ liệu: 15 điểm (x,y) không gian 2D; rotate đổi hướng các điểm quanh gốc với góc vuông.
```

```
# NumPy lưu các tọa độ ở bảng (15,2) và nhân ma trận.
import numpy as np
# 15 hàng điểm, cột 0 là x và cột 1 là y, dtype=float.
points = np.array([[1,2],[2,2],[3,2],[4,2],[5,2],[1,3],[2,3],[3,3],[4,3],[5,3],[1,4],[2,4],[3,4],
[4,4],[5,4]], dtype=float)
# Ma trận [[0,-1],[1,0]] tác động lên vector HÀNG bằng @: (x,y)→(y,-x), quay 90° theo chiêù kim
đôìng hô`theo quy ước x/y thông thường.
rotate = np.array([[0.0, -1.0], [1.0, 0.0]])
# @ áp cùng ma trận quay lên từng hàng điểm, trả ma trận (15,2), không sưả points gôć.
converted = points @ rotate
if points.shape != (15, 2) or converted.shape != (15, 2):
    # raise ValueError() dừng nêú kích thước, khả nghịch hoặc kêt quả đôí chiêú không hợp lệ.
    raise ValueError("Phải có đúng 15 điểm gôć và điểm đã đôỉ tọa độ.")
# norm(...,axis=1) tính độ dài từ gôć từng điểm; làm tương tự trên converted.
# allclose() so từng độ dài với dung sai sô´thực, phù hợp tính ma trận floating-point; không dùng
== tuyệt đôí.
if not np.allclose(np.linalg.norm(points, axis=1), np.linalg.norm(converted, axis=1)):
    # raise ValueError() dừng nêú kích thước, khả nghịch hoặc kêt quả đôí chiêú không hợp lệ.
    raise ValueError("Phép quay phải bảo toàn chiêù dài của từng điểm.")
print("Điểm:", len(points))
# det() tính định thức ma trận quay, bằng +1 cho phép quay giữ diện tích/hướng định thức.
print(f"Định thức: {np.linalg.det(rotate):.1f}")
original_labels = []
converted_labels = []
# [0] lấy tọa độ điểm đâù trong dữ liệu gôć.
for value in points[0]:
    # f-string :.1f định dạng tọa độ có một chữ sô´thập phân.
    original_labels.append(f"{value:.1f}")
# [0] lấy tọa độ mới của cùng điểm sau biến đôỉ.
for value in converted[0]:
    # f-string :.1f định dạng tọa độ có một chữ sô´thập phân.
    converted_labels.append(f"{value:.1f}")
# join() nôí hai tọa độ thành một chuôĩ; không thay tọa độ sô´học trong mảng.
print("Điểm đâù (gôć):", ", ".join(original_labels))
print("Điểm đâù (mới):", ", ".join(converted_labels))
```

Kết quả khi chạy:

```
Điểm: 15
Định thức: 1.0
Điểm đâù (gôć): 1.0, 2.0
Điểm đâù (mới): 2.0, -1.0
```

Độ dài giữ nguyên nhưng tọa độ trục mới khác, thể hiện tính chất của phép quay. `np.linalg.det` là **một số** dùng kiểm tra khả nghịch, không phải tọa độ.

Tình huống 4 — Cấp 4: Giải hệ cảm biến 2 kênh cho 18 lượt đo

Từ vựng trong mục: Linear system — hệ tuyến tính; **np.linalg.solve** — giải hệ; **Residual** — sai số đối chiếu.

Tình huống thực tế: Máy đo theo dõi **18 lượt tín hiệu**. Mỗi lượt có hai **nguồn tín hiệu gốc** chưa biết là x và y, cùng hai giá trị kênh A và B đọc được. Máy kết hợp chúng theo một quy tắc cố định: **A nhận hai phần x và một phần y, B nhận một phần x và ba phần y**. Nhân viên muốn khôi phục **hai nguồn gốc của từng lượt**, rồi đưa kết quả trở lại quy tắc pha trộn để kiểm tra có tạo đúng hai giá trị đã đọc hay không.

Vì sao chọn cách này? Đây là **hệ phương trình tuyến tính**: mỗi kênh đo là một cách trộn hai nguồn chưa biết theo các hệ số đã cho. Ma trận `mix` giữ quy tắc pha trộn cố định; `measured` chứa 18 cặp kết quả đo. `np.linalg.solve()` tìm hai giá trị gốc cho từng cặp khi ma trận có đủ thông tin để giải duy nhất, thay

vì phải thử ngược nhiều khả năng hay tự tính nghịch đảo. `.T` đưa dữ liệu vào đúng hướng hàng–cột mà hàm yêu cầu rồi đưa kết quả về **mỗi lượt một hàng**. Cuối cùng nhân lại các nguồn đã khôi phục với ma trận để tái tạo kênh A/B, dùng `np.allclose()` kiểm xem gần bằng số đo ban đầu hay không. Nếu không đối chiếu bước cuối, lỗi đảo cột hoặc sai hệ số có thể bị bỏ sót.

Quy trình:

1. Lưu ma trận hệ số 2×2 và 18 cặp số đo theo kênh.
2. Dùng `solve` trên ma trận kênh đã chuyển vị để giải 18 lượt cùng lúc.
3. Ghép nghiệm trở lại số đo, xác nhận sai số số học nhỏ và in nghiệm đầu.

No Note — Không chú thích

```
import numpy as np
mix = np.array([[2.0, 1.0], [1.0, 3.0]])
measured = np.array([[4,7],[6,8],[8,9],[10,10],[12,11],[14,12],[5,10],[7,11],[9,12],[11,13],
[13,14],[15,15],[6,13],[8,14],[10,15],[12,16],[14,17],[16,18]], dtype=float)
restored = np.linalg.solve(mix, measured.T).T
reconstructed = restored @ mix.T
if measured.shape != (18, 2) or restored.shape != (18, 2):
    raise ValueError("Cần đủ 18 lượt đo và hai thông số´khôi phục.")
if not np.allclose(reconstructed, measured):
    raise ValueError("Tái tạo số´đo phải khớp dữ liệu đầu vào.")
print("Lượt đo:", len(measured))
print(f"Định thức: {np.linalg.det(mix):.1f}")
first_values = []
for value in restored[0]:
    first_values.append(f"{value:.1f}")
print("Nguồn đầu:", ", ".join(first_values))
print("Khôi phục đúng:", bool(np.allclose(reconstructed, measured)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Khôi phục 18 vector nguồn từ các phép đo bị trộn bởi ma trận và đối chiếu tái tạo.
# Dữ liệu: 18 lượt đo mỗi lượt hai giá trị; mix 2×2 là ma trận hệ số´giả lập biết trước.
# NumPy linalg.solve() giải hệ phương trình tuyến tính với ma trận hệ số´.
import numpy as np
# Ma trận trộn (2,2) biết trước; không phải dữ liệu đo.
mix = np.array([[2.0, 1.0], [1.0, 3.0]])
# 18 hàng đo hỗn hợp, mỗi hàng gồm hai thông số´đã bị trộn.
measured = np.array([[4,7],[6,8],[8,9],[10,10],[12,11],[14,12],[5,10],[7,11],[9,12],[11,13],
[13,14],[15,15],[6,13],[8,14],[10,15],[12,16],[14,17],[16,18]], dtype=float)
# measured.T đổi (18,2)→(2,18) để từng lượt là cột về´phải; solve(mix, ...) tìm nguồn không cần tự
tính nghịch đảo; .T cuối đổi lại thành 18 hàng.
restored = np.linalg.solve(mix, measured.T).T
# Vì restored được lưu theo hàng, nhân @ mix.T tái tạo phép trộn đúng quy ước vector cột ban đầu.
reconstructed = restored @ mix.T
# Đảm bảo bảng đo và nguồn khôi phục đều (18,2).
if measured.shape != (18, 2) or restored.shape != (18, 2):
    # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
    raise ValueError("Cần đủ 18 lượt đo và hai thông số´khôi phục.")
# allclose() đối chiếu từng ô số´thực với dung sai, tránh sai số´làm tròn của solve.
if not np.allclose(reconstructed, measured):
    # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
    raise ValueError("Tái tạo số´đo phải khớp dữ liệu đầu vào.")
print("Lượt đo:", len(measured))
# det(mix) khác 0 cho biết ma trận trộn có thể giải duy nhất; :.1f chỉ làm tròn hiển thị.
print(f"Định thức: {np.linalg.det(mix):.1f}")
first_values = []
# [0] lấy hai thành phần của nguồn khôi phục từ lượt đầu.
for value in restored[0]:
```

```
    # f-string :.1f in giá trị khôi phục một chữ sô´sau dâú thập phân.
    first_values.append(f"{value:.1f}")
print("Nguôn đâù:", ", ".join(first_values))
# allclose() đôí chiếú từng ô sô´thực với dung sai, tránh sai sô´làm tròn của solve.
# bool() đôỉ kêt quả kiêm NumPy thành Boolean Python dê˜đọc.
print("Khôi phục đúng:", bool(np.allclose(reconstructed, measured)))
```

Kết quả khi chạy:

```
Lượt đo: 18
Định thức: 5.0
Nguôn đâù: 1.0, 2.0
Khôi phục đúng: True
```

Mỗi hàng là một bài toán hai ẩn, cùng chung ma trận hệ số. Ví dụ đo lý tưởng không có nhiễu; đo thật còn cần đánh giá sai số và độ nhạy.

Tình huống 5 — Cấp 5: Kiểm tra 20 ma trận hiệu chuẩn và không giải hệ suy biến

Từ vựng trong mục: Singular matrix — ma trận suy biến; **Determinant** — định thức; **Numerical stability** — ổn định số.

Tình huống thực tế: Nhà máy ghi **20 lượt hiệu chuẩn M01–M20**. Mỗi lượt có bốn **hệ số pha trộn** xếp thành bảng 2 × 2 và hai giá trị tín hiệu đo được. Ở ba lượt **M06, M12 và M18**, hai hàng hệ số phụ thuộc nhau, khiến kết quả không đủ thông tin để biết duy nhất hai tín hiệu nguồn. Kỹ thuật viên cần tách các lượt **không giải duy nhất được**, chỉ giải những lượt đủ điều kiện, kiểm lại phép pha trộn và báo số lượt thành công, thất bại mà không thay kết quả chưa giải được bằng 0.

Vì sao chọn cách này? Ma trận suy biến là ma trận không đủ thông tin để tìm **một nghiệm duy nhất**; trong bài, hai hàng hệ số ở M06, M12, M18 phụ thuộc nhau nên không thể phân biệt riêng hai nguồn x và y chỉ từ hai kênh. **Định thức** gần 0 là dấu hiệu cần loại khỏi bước giải bằng `np.linalg.solve()`. Chương trình duyệt từng lượt, ghi mã suy biến vào danh sách `singular`, dùng `continue` bỏ riêng lượt ấy, rồi chỉ giải các lượt hợp lệ. Sau giải, lấy hệ số nhân lại nghiệm và dùng `np.allclose()` xác nhận tái tạo đúng số đo. `solved` đếm lượt đã giải, `singular` đếm lượt cần xử lý khác; không gán 0 cho lượt không giải được vì 0 là một giá trị đo cụ thể, không phải dấu hiệu thiếu nghiệm.

Quy trình:

1. Tạo 20 bản ghi với ID, ma trận 2×2 và vector quan sát.
2. Kiểm tra định thức theo ngưỡng nhỏ đã nêu, đưa trường hợp suy biến vào nhóm cần xem.
3. Chỉ gọi `np.linalg.solve()` trên trường hợp đạt, ghép kết quả lại để xác nhận và đối chiếu số lượt.

No Note — Không chú thích

```
import numpy as np
records = [{"id":"M01","coeff":[[2,1],[1,2]],"measurement":[4,5],"original":[1,2]},
{"id":"M02","coeff":[[3,1],[1,3]],"measurement":[9,11],"original":[2,3]},{"id":"M03","coeff":
[[4,1],[1,4]],"measurement":[16,19],"original":[3,4]},{"id":"M04","coeff":[[2,1],
[1,5]],"measurement":[13,29],"original":[4,5]},{"id":"M05","coeff":[[3,1],[1,2]],"measurement":
[17,9],"original":[5,2]},{"id":"M06","coeff":[[2,1],[4,2]],"measurement":[5,10],"original":[1,3]},
{"id":"M07","coeff":[[2,1],[1,4]],"measurement":[8,18],"original":[2,4]},{"id":"M08","coeff":
[[3,1],[1,5]],"measurement":[14,28],"original":[3,5]},{"id":"M09","coeff":[[4,1],
[1,2]],"measurement":[18,8],"original":[4,2]},{"id":"M10","coeff":[[2,1],[1,3]],"measurement":
[13,14],"original":[5,3]},{"id":"M11","coeff":[[3,1],[1,4]],"measurement":[7,17],"original":[1,4]},
{"id":"M12","coeff":[[2,1],[4,2]],"measurement":[9,18],"original":[2,5]},{"id":"M13","coeff":
[[2,1],[1,2]],"measurement":[8,7],"original":[3,2]},{"id":"M14","coeff":[[3,1],
```

```
[1,3]],"measurement":[15,13],"original":[4,3]},{"id":"M15","coeff":[[4,1],[1,4]],"measurement":
[24,21],"original":[5,4]},{"id":"M16","coeff":[[2,1],[1,5]],"measurement":[7,26],"original":[1,5]},
{"id":"M17","coeff":[[3,1],[1,2]],"measurement":[8,6],"original":[2,2]},{"id":"M18","coeff":[[2,1],
[4,2]],"measurement":[9,18],"original":[3,3]},{"id":"M19","coeff":[[2,1],[1,4]],"measurement":
[12,20],"original":[4,4]},{"id":"M20","coeff":[[3,1],[1,5]],"measurement":[20,30],"original":
[5,5]}]
solved, singular = [], []
for record in records:
    matrix = np.array(record["coeff"], dtype=float)
    measured = np.array(record["measurement"], dtype=float)
    if abs(np.linalg.det(matrix)) < 1e-9:
        singular.append(record["id"])
        continue
    values = np.linalg.solve(matrix, measured)
    if not np.allclose(matrix @ values, measured):
        raise ValueError("Giải hệ phải tái tạo đúng số´đo hiệu chuẩn.")
    solved.append(record["id"])
if len(records) != 20 or len(solved) + len(singular) != 20:
    raise ValueError("Phải kiểm tra đủ 20 lượt hiệu chuẩn, kể cả ma trận suy biến.")
print("Lượt hiệu chuẩn:", len(records))
print("Giải được:", len(solved))
print("Không thể giải:", ", ".join(singular))
print("Đối chiếu đủ:", len(solved) + len(singular) == len(records))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm 20 phép hiệu chuẩn 2×2, loại ma trận gần suy biến rồi giải và tái kiểm số´đo.
# Dữ liệu: 20 dict id, coeff ma trận hệ số´2×2, measurement hai số´đo, original tham chiếú; vài ma
trận không có nghiệm duy nhất.
# NumPy hỗ˜trợ det, solve, allclose cho ma trận số´thực.
import numpy as np
# 20 bản ghi mỗi lượt có id, hệ số´coeff, measurement đầu ra và original tham chiếú (original không
được sử dụng trong phép giải ở ví dụ).
records = [{"id":"M01","coeff":[[2,1],[1,2]],"measurement":[4,5],"original":[1,2]},
{"id":"M02","coeff":[[3,1],[1,3]],"measurement":[9,11],"original":[2,3]},{"id":"M03","coeff":
[[4,1],[1,4]],"measurement":[16,19],"original":[3,4]},{"id":"M04","coeff":[[2,1],
[1,5]],"measurement":[13,29],"original":[4,5]},{"id":"M05","coeff":[[3,1],[1,2]],"measurement":
[17,9],"original":[5,2]},{"id":"M06","coeff":[[2,1],[4,2]],"measurement":[5,10],"original":[1,3]},
{"id":"M07","coeff":[[2,1],[1,4]],"measurement":[8,18],"original":[2,4]},{"id":"M08","coeff":
[[3,1],[1,5]],"measurement":[14,28],"original":[3,5]},{"id":"M09","coeff":[[4,1],
[1,2]],"measurement":[18,8],"original":[4,2]},{"id":"M10","coeff":[[2,1],[1,3]],"measurement":
[13,14],"original":[5,3]},{"id":"M11","coeff":[[3,1],[1,4]],"measurement":[7,17],"original":[1,4]},
{"id":"M12","coeff":[[2,1],[4,2]],"measurement":[9,18],"original":[2,5]},{"id":"M13","coeff":
[[2,1],[1,2]],"measurement":[8,7],"original":[3,2]},{"id":"M14","coeff":[[3,1],
[1,3]],"measurement":[15,13],"original":[4,3]},{"id":"M15","coeff":[[4,1],[1,4]],"measurement":
[24,21],"original":[5,4]},{"id":"M16","coeff":[[2,1],[1,5]],"measurement":[7,26],"original":[1,5]},
{"id":"M17","coeff":[[3,1],[1,2]],"measurement":[8,6],"original":[2,2]},{"id":"M18","coeff":[[2,1],
[4,2]],"measurement":[9,18],"original":[3,3]},{"id":"M19","coeff":[[2,1],[1,4]],"measurement":
[12,20],"original":[4,4]},{"id":"M20","coeff":[[3,1],[1,5]],"measurement":[20,30],"original":
[5,5]}]
# Hai list lưu mã lượt giải được và mã ma trận suy biến; giúp tổng 20 lượt không mất bản ghi.
solved, singular = [], []
# Duyệt từng lượt hiệu chuẩn độc lập, không giả định mọi ma trận đều khả nghịch.
for record in records:
    # Lấy hệ số´2×2 từ dict, đổi sang ndarray float để dùng hàm đại số´.
    matrix = np.array(record["coeff"], dtype=float)
    # Vector hai giá trị đo là vế´phải của hệ cần giải.
    measured = np.array(record["measurement"], dtype=float)
    # det() định thức; abs() lấy trị tuyệt đối, <1e-9 là ngưỡng gần 0 do người soạn chọn để tránh
giải ma trận suy biến/gần suy biến; không phải quy tắc ổn định cho mọi thang số´.
    if abs(np.linalg.det(matrix)) < 1e-9:
        # append() ghi mã ma trận không thể tin cậy để giải; không xóa bản ghi.
        singular.append(record["id"])
        # continue bỏ bước solve của chính lượt này để không phát sinh lỗi chia theo ma trận suy
biến.
```

```
        continue
    # solve(A,b) tìm vector nguồn x sao cho A@x=b mà không cần tính nghịch đảo tường minh.
    values = np.linalg.solve(matrix, measured)
    # @ tái tạo số đo từ nguồn đã giải; allclose() so với đo gốc có dung sai floating-point.
    if not np.allclose(matrix @ values, measured):
        # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
        raise ValueError("Giải hệ phải tái tạo đúng số đo hiệu chuẩn.")
    # Chỉ đưa mã vào danh sách solved nếu qua kiểm tra tái tạo.
    solved.append(record["id"])
# Đối chiếu mọi bản ghi hoặc giải được hoặc bị đánh dấu suy biến, không bỏ sót.
if len(records) != 20 or len(solved) + len(singular) != 20:
    # raise ValueError() dừng nếu kích thước, khả nghịch hoặc kết quả đối chiếu không hợp lệ.
    raise ValueError("Phải kiểm tra đủ 20 lượt hiệu chuẩn, kể cả ma trận suy biến.")
print("Lượt hiệu chuẩn:", len(records))
print("Giải được:", len(solved))
# join() ghép mã không giải được thành chuỗi để trả lại.
print("Không thể giải:", ", ".join(singular))
print("Đối chiếu đủ:", len(solved) + len(singular) == len(records))
```

Kết quả khi chạy:

```
Lượt hiệu chuẩn: 20
Giải được: 17
Không thể giải: M06, M12, M18
Đối chiếu đủ: True
```

Ngưỡng định thức trong ví dụ dùng để phát hiện ma trận suy biến rõ ràng, chưa phải quy tắc số học tổng quát cho mọi ma trận rất lớn hoặc có thang đo chênh lệch; những hệ gần suy biến cần kiểm tra điều kiện số kỹ hơn.

6. Sai lầm thường gặp

- `shape=(2,3)` nghĩa là hai hàng và ba cột, không phải ba hàng hai cột.
- `.T` hoán đổi trục; phép nhân `@` yêu cầu số cột bên trái bằng số hàng bên phải.
- Định thức chỉ tính trên ma trận vuông; ma trận có định thức 0 không có nghịch đảo.
- Dữ liệu số thực có sai số làm tròn; dùng `np.allclose` khi kiểm tra gần đúng với ma trận đơn vị.
- Khi giải hệ tuyến tính, thường ưu tiên `np.linalg.solve` thay vì tính nghịch đảo trực tiếp.

7. Trắc nghiệm

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a matrix? | Ma trận là gì?

A. Rows of the left matrix with columns of the right | Hàng ma trận trái với cột ma trận phải

B. A two-dimensional array of values | Bảng dữ liệu hai chiều

C. A scalar associated with a square matrix | Một số gắn với ma trận vuông

D. A matrix with rows and columns exchanged | Ma trận đổi vai trò hàng và cột

Đáp án: B

Giải thích: Ma trận sắp phần tử theo hàng và cột.

Q2. What is the transpose of a matrix? | Chuyển vị của ma trận là gì?

A. A matrix with rows and columns exchanged | Ma trận đổi vai trò hàng và cột

B. A two-dimensional array of values | Bảng dữ liệu hai chiều

C. Rows of the left matrix with columns of the right | Hàng ma trận trái với cột ma trận phải

D. A scalar associated with a square matrix | Một số gắn với ma trận vuông

Đáp án: A

Giải thích: Chuyển vị hoán đổi các trục ma trận.

Q3. What does matrix multiplication combine? | Phép nhân ma trận kết hợp gì?

A. Rows of the left matrix with columns of the right | Hàng ma trận trái với cột ma trận phải

B. A two-dimensional array of values | Bảng dữ liệu hai chiều

C. A matrix with rows and columns exchanged | Ma trận đổi vai trò hàng và cột

D. A scalar associated with a square matrix | Một số gắn với ma trận vuông

Đáp án: A

Giải thích: Mỗi ô kết quả là tích vô hướng một hàng với một cột.

Q4. What is a determinant? | Định thức là gì?

A. Rows of the left matrix with columns of the right | Hàng ma trận trái với cột ma trận phải

B. A two-dimensional array of values | Bảng dữ liệu hai chiều

C. A scalar associated with a square matrix | Một số gắn với ma trận vuông

D. A matrix with rows and columns exchanged | Ma trận đổi vai trò hàng và cột

Đáp án: C

Giải thích: Định thức hỗ trợ kiểm tra tính khả nghịch.

Q5. Let A = [[2, 1], [0, 3]] and B = [[1, 4], [5, 2]]. What is element (row 1, column 2) of AB? | Cho A và B như trên. Phần tử hàng 1, cột 2 của AB là bao nhiêu?

A. 8 | 8

B. 12 | 12

C. 6 | 6

D. 10 | 10

Đáp án: D

Giải thích: Nhân hàng 1 của A với cột 2 của B: 2 × 4 + 1 × 2 = 10.

Q6. What is the determinant of [[3, 1], [2, 1]]? | Định thức của ma trận [[3, 1], [2, 1]] bằng bao nhiêu?

A. 2 | 2

B. 1 | 1

C. 5 | 5

D. 0 | 0

Đáp án: B

Giải thích: Với ma trận 2 × 2, lấy tích đường chéo chính trừ đường chéo phụ: 3 × 1 − 1 × 2 = 1.

Q7. A square matrix has determinant 0. Which statement is correct? | Một ma trận vuông có định thức bằng 0. Phát biểu nào đúng?

A. It must be diagonal | Nó bắt buộc là ma trận đường chéo

B. All entries are zero | Mọi phần tử đều bằng 0

C. It has no inverse | Nó không có ma trận nghịch đảo

D. It is the identity matrix | Nó là ma trận đơn vị

Đáp án: C

Giải thích: Định thức 0 nghĩa là ma trận suy biến và không khả nghịch; các kết luận khác không nhất thiết đúng.

Q8. If A has shape (2, 3) and B has shape (3, 4), what is the shape of AB? | Nếu A có kích thước (2, 3), B có kích thước (3, 4), AB có kích thước nào?

A. (4, 2) | (4, 2)

B. (2, 4) | (2, 4)

C. (2, 3) | (2, 3)

D. (3, 3) | (3, 3)

Đáp án: B

Giải thích: Hai chiều ở giữa (3 và 3) phải khớp; ma trận kết quả lấy số hàng của A và số cột của B: (2, 4).

Q9. Which operator performs matrix multiplication? | Toán tử nào thực hiện nhân ma trận?

A. `@` | `@`

B. `np.linalg.inv(A)` | `np.linalg.inv(A)`

C. `.T` | `.T`

D. `np.linalg.det(A)` | `np.linalg.det(A)`

Đáp án: A

Giải thích: @ thực hiện đúng quy tắc nhân ma trận.

Q10. Which attribute returns the transpose of a 2-D ndarray? | Thuộc tính nào cho chuyển vị mảng 2-D?

A. `@` | `@`

B. `np.linalg.det(A)` | `np.linalg.det(A)`

C. `.T` | `.T`

D. `np.linalg.inv(A)` | `np.linalg.inv(A)`

Đáp án: C

Giải thích: T là thuộc tính chuyển vị tiện dụng.

Q11. Which function computes a square matrix determinant? | Hàm nào tính định thức ma trận vuông?

A. `@` | `@`

B. `.T` | `.T`

C. `np.linalg.det(A)` | `np.linalg.det(A)`

D. `np.linalg.inv(A)` | `np.linalg.inv(A)`

Đáp án: C

Giải thích: det trả định thức dạng số.

Q12. Which function computes a nonsingular matrix inverse? | Hàm nào tính nghịch đảo ma trận khả nghịch?

A. `np.linalg.inv(A)` | `np.linalg.inv(A)`

B. `.T` | `.T`

C. `np.linalg.det(A)` | `np.linalg.det(A)`

D. `@` | `@`

Đáp án: A

Giải thích: inv tính nghịch đảo khi ma trận khả nghịch.

Q13. When is a square matrix invertible? | Khi nào ma trận vuông khả nghịch?

A. `np.allclose(A, B)` | `np.allclose(A, B)`

B. Identity matrix | Ma trận đơn vị

C. Its determinant is nonzero | Định thức khác không

D. 3 | 3

Đáp án: C

Giải thích: Định thức khác 0 tương đương khả nghịch.

Q14. What is determinant of [[2,1],[1,2]]? | Định thức [[2,1],[1,2]] bằng gì?

A. Identity matrix | Ma trận đơn vị

B. 3 | 3

C. `np.allclose(A, B)` | `np.allclose(A, B)`

D. Its determinant is nonzero | Định thức khác không

Đáp án: B

Giải thích: Định thức 2×2 bằng 2×2−1×1.

Q15. What does `A @ inv(A)` approximate? | `A @ inv(A)` gần bằng gì?

A. Its determinant is nonzero | Định thức khác không

B. 3 | 3

C. Identity matrix | Ma trận đơn vị

D. `np.allclose(A, B)` | `np.allclose(A, B)`

Đáp án: C

Giải thích: Nhân ma trận với nghịch đảo bằng I trong tính toán chính xác.

Q16. How do you test approximate equality of float arrays? | Kiểm tra hai mảng float gần bằng nhau bằng gì?

A. Identity matrix | Ma trận đơn vị

B. `np.allclose(A, B)` | `np.allclose(A, B)`

C. Its determinant is nonzero | Định thức khác không

D. 3 | 3

Đáp án: B

Giải thích: allclose chấp nhận sai số dấu phẩy động nhỏ.

Q17. Which dimensions must match for A @ B? | Nhân A @ B cần các chiều nào bằng nhau?

A. Columns of A and rows of B | Số cột A và số hàng B

B. Element-wise multiplication | Nhân theo từng phần tử

C. `np.linalg.solve(A, b)` | `np.linalg.solve(A, b)`

D. A linear algebra error | Lỗi đại số tuyến tính

Đáp án: A

Giải thích: Nhân ma trận yêu cầu hai chiều trong trùng.

Q18. What does `A * B` do for compatible NumPy arrays? | `A * B` trên mảng NumPy tương thích làm gì?

A. A linear algebra error | Lỗi đại số tuyến tính

B. `np.linalg.solve(A, b)` | `np.linalg.solve(A, b)`

C. Columns of A and rows of B | Số cột A và số hàng B

D. Element-wise multiplication | Nhân theo từng phần tử

Đáp án: D

Giải thích: Dấu sao nhân phần tử tương ứng có broadcasting.

Q19. What is preferred to solve Ax=b numerically? | Để giải hệ Ax=b bằng NumPy nên ưu tiên gì?

A. Element-wise multiplication | Nhân theo từng phần tử

B. Columns of A and rows of B | Số cột A và số hàng B

C. A linear algebra error | Lỗi đại số tuyến tính

D. `np.linalg.solve(A, b)` | `np.linalg.solve(A, b)`

Đáp án: D

Giải thích: Giải trực tiếp thường tốt hơn tính nghịch đảo rồi nhân.

Q20. What happens to a singular matrix when using `np.linalg.inv`? | Dùng `np.linalg.inv` với ma trận suy biến thường gây gì?

A. Columns of A and rows of B | Số cột A và số hàng B

B. A linear algebra error | Lỗi đại số tuyến tính

C. Element-wise multiplication | Nhân theo từng phần tử

D. `np.linalg.solve(A, b)` | `np.linalg.solve(A, b)`

Đáp án: B

Giải thích: Ma trận suy biến không có nghịch đảo.

8. Nối sang bài kế tiếp

`Vector → Matrix → Hệ tuyến tính → Eigenvalue/PCA`.

PY-26 — Hệ phương trình tuyến tính, rank và nghiệm

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Linear system — hệ tuyến tính; **Rank** — hạng ma trận.

`Ma trận → Rank/Solve → Regression → AI`

Học xong dùng để làm gì? Kiểm tra hệ ràng buộc có đủ thông tin để giải hay có phương trình trùng, thiếu hoặc mâu thuẫn.

Kết nối kiến thức tiếp theo: PY-27 xác định hướng đặc biệt của phép biến đổi ma trận qua trị riêng/ vector riêng.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Linear system	hệ tuyến tính	Nhiều phương trình chứa cùng các ẩn số
Rank	hạng ma trận	Số hàng/cột độc lập tuyến tính
Coefficient matrix	ma trận hệ số	Các hệ số của ẩn trong Ax=b
Residual	sai số dư	Độ sai khác giữa dự đoán Ax và b

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.linalg.solve(A,b)`	Giải Ax=b khi ma trận A vuông không suy biến
`np.linalg.matrix_rank(A)`	Kiểm tra số chiều độc lập tuyến tính
`np.column_stack((A,b))`	Ghép ma trận hệ số và vế phải thành augmented matrix
`np.linalg.lstsq(A,b,rcond=None)`	Tìm nghiệm bình phương tối thiểu khi quá nhiều phương trình

4. Khái niệm liên quan

Nội dung chính: một hệ tuyến tính liên kết nhiều phương trình qua cùng các ẩn. Ta xác định đúng **hàng = phương trình**, **cột = ẩn**, **vế phải = giá trị đã biết**, sau đó kiểm tra tính độc lập và độ khớp của nghiệm.

4.1. Linear system — Một hệ phương trình chung các ẩn

Từ vựng trong mục: Linear system — hệ tuyến tính; **Equation** — phương trình.

Hệ tuyến tính là **nhiều phương trình cùng chứa các ẩn cần tìm**. Cửa hàng bán loại A giá x và loại B giá y (đơn vị nghìn đồng): mua một A và một B hết 5; mua hai A và một B hết 8. Ta có x+y=5 và 2x+y=8. Để viết Ax=b, mỗi **hàng của A là một phương trình**, mỗi **cột là hệ số của một ẩn**, b chứa tổng tiền từng hóa đơn. Giải hệ tìm được x=3, y=2.

No Note — Không chú thích

```
import numpy as np

A = np.array([[1, 1], [2, 1]], dtype=float)
b = np.array([5, 8], dtype=float)
print(np.linalg.solve(A, b))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Giải hệ hai phương trình tuyến tính để tìm hai ẩn từ bảng hệ số và vế phải.
# Dữ liệu: A có hai hàng phương trình và hai cột ẩn; b=[5,8] là kết quả từng phương trình.
# NumPy cung cấp ndarray và np.linalg.solve() để giải hệ tuyến tính.
import numpy as np

# A là ma trận hệ số 2×2; dtype=float ép hệ số sang số thực cho phép giải có phần lẻ.
A = np.array([[1, 1], [2, 1]], dtype=float)
# b là vector vế phải dài 2, mỗi ô ứng với một hàng của A.
b = np.array([5, 8], dtype=float)
# solve(A,b) tìm vector x thỏa A@x=b bằng thuật toán giải hệ, không phải nhân nghịch đảo thủ công;
yêu cầu A vuông không suy biến.
print(np.linalg.solve(A, b))
```

Kết quả khi chạy:

```
[3. 2.]
```

Loại A giá 3 nghìn và loại B giá 2 nghìn đồng; nghiệm đồng thời thỏa hai hóa đơn.

4.2. Rank — Số thông tin độc lập

Từ vựng trong mục: Rank — hạng; **Linear dependence** — phụ thuộc tuyến tính.

Rank cho biết có bao nhiêu hàng/cột độc lập tuyến tính trong một ma trận. Ví dụ hai hàng [1,2] và [2,4]: hàng sau chỉ bằng hai lần hàng đầu nên không đưa thêm thông tin độc lập. Rank bằng 1 dù bảng có hai hàng và hai cột. Với một hệ 2 ẩn, nếu rank(A)=2 và A vuông thì nghiệm duy nhất khi hệ có vế phải b phù hợp (ở đây ma trận khả nghịch); nếu rank thấp cần kiểm tra cả ma trận mở rộng để biết vô nghiệm hay vô số nghiệm.

No Note — Không chú thích

```
import numpy as np

A = np.array([[1, 2], [2, 4]])
print(np.linalg.matrix_rank(A))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đo hạng ma trận để biết có bao nhiêu hàng/cột mang thông tin độc lập.
# Dữ liệu: Ma trận 2×2, hàng thứ hai gấp đôi hàng thứ nhất, nên chỉ có một hướng thông tin độc lập.
```

```
# NumPy linalg có phương thức xác định hạng ma trận.
import numpy as np

# Hai hàng [1,2] và [2,4] phụ thuộc tuyến tính vì một hàng là hai lần hàng kia.
A = np.array([[1, 2], [2, 4]])
# matrix_rank() dùng phân rã số học để đếm số hướng độc lập của ma trận; trả 1, KHÔNG phải số hàng
hay số cột.
print(np.linalg.matrix_rank(A))
```

Kết quả khi chạy:

```
1
```

Hai hàng phụ thuộc nhau nên số chiều thông tin độc lập chỉ là một.

4.3. Coefficient matrix — Bảng hệ số và vế phải

Từ vựng trong mục: Coefficient matrix — ma trận hệ số; **Augmented matrix** — ma trận mở rộng.

Trong `Ax=b`, **A chỉ chứa hệ số của các ẩn**, còn b chứa **giá trị đích của từng phương trình**. Ví dụ hệ `x+y=5`, `2x+y=8` có A hai hàng × hai cột, b dài hai phần tử. Ghép b thành một cột cuối bằng `np.column_stack((A,b))` cho ma trận mở rộng 2×3. Cột cuối này **không phải một ẩn mới**. Khi giải, số hàng A phải bằng số phần tử b.

No Note — Không chú thích

```
import numpy as np

A = np.array([[1, 1], [2, 1]])
b = np.array([5, 8])
print(np.column_stack((A, b)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Ghép ma trận hệ số A với cột vế phải b thành ma trận mở rộng.
# Dữ liệu: A có 2 phương trình × 2 ẩn; b=[5,8] là cột kết quả tương ứng.
# NumPy column_stack ghép các cột theo chiều ngang.
import numpy as np

# A chứa hai cột hệ số ứng với hai ẩn cho từng phương trình.
A = np.array([[1, 1], [2, 1]])
# b là vector vế phải có đúng hai giá trị theo hai hàng A.
b = np.array([5, 8])
# column_stack((A,b)) ghép b thành CỘT thứ ba, trả ma trận mở rộng 2×3; các hàng phải cùng số
lượng.
print(np.column_stack((A, b)))
```

Kết quả khi chạy:

```
[[1 1 5]
 [2 1 8]]
```

Hai cột đầu là hệ số của x, y; cột cuối là tổng tiền hóa đơn.

4.4. Residual — Sai lệch và nghiệm bình phương tối thiểu

Từ vựng trong mục: Residual — sai số dư; **Least squares** — bình phương tối thiểu.

Khi thử nghiệm x cho hệ `Ax=b`, **sai số dư** `r=Ax-b` đo mỗi phương trình lệch bao nhiêu so với vế phải. Nếu `r=0` thì nghiệm thỏa chính xác; dữ liệu đo thực tế thường có nhiễu nên không tìm được nghiệm thỏa mọi hàng. Khi số phương trình nhiều hơn số ẩn, `np.linalg.lstsq` tìm x làm **tổng bình phương các sai số dư nhỏ nhất**. Ví dụ ba số đo có x=1,2,3 và giá trị y=2,4,6.1; fit đường `y=m*x+c` không khớp hoàn toàn và cho gần `m=2.05`, `c=-0.067`.

No Note — Không chú thích

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([2, 4, 6.1])
A = np.column_stack((x, np.ones_like(x)))
coeff = np.linalg.lstsq(A, y, rcond=None)[0]
print(np.round(coeff, 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm đường thẳng gần ba điểm nhất bằng nghiệm bình phương tối thiểu.
# Dữ liệu: Ba điểm (x,y) = (1,2), (2,4), (3,6.1); muốn ước lượng độ dốc và hệ số chặn.
# NumPy hỗ trợ tạo ma trận thiết kế và hồi quy bình phương tối thiểu.
import numpy as np

# x là ba giá trị đầu vào (tọa độ ngang), mỗi giá trị thuộc cùng lượt với y.
x = np.array([1, 2, 3])
# y chứa ba kết quả thực đo, có nhiễu 6.1 khiến ba phương trình không khớp tuyệt đối.
y = np.array([2, 4, 6.1])
# ones_like(x) tạo vector ba số 1 cùng shape, đại diện hệ số chặn của đường thẳng.
# Ghép cột x và cột 1 thành A shape (3,2), phù hợp mô hình y≈slope*x+intercept.
A = np.column_stack((x, np.ones_like(x)))
# lstsq giải nghiệm làm TỔNG BÌNH PHƯƠNG sai lệch nhỏ nhất khi dữ liệu nhiễu; rcond=None dùng
ngưỡng mặc định NumPy để quyết định hạng.
# [0] lấy vector HỆ SỐ đầu tiên của tuple lstsq trả về, không phải điểm dữ liệu đầu tiên.
coeff = np.linalg.lstsq(A, y, rcond=None)[0]
# np.round(...,3) chỉ làm tròn hệ số khi hiển thị ba chữ số thập phân, không làm thay đổi fit ban
đầu.
print(np.round(coeff, 3))
```

Kết quả khi chạy:

```
[ 2.05  -0.067]
```

Hệ số góc xấp xỉ 2,05 và hệ số chặn xấp xỉ −0,067; đây là nghiệm xấp xỉ do số đo có nhiễu.

5. Năm tình huống hệ tuyến tính — từ giải đúng đến xấp xỉ với dữ liệu nhiễu

Tình huống 1 — Cấp 1: Giải giá hai mặt hàng từ 10 hóa đơn

Từ vựng trong mục: Linear system — hệ tuyến tính; **solve** — giải nghiệm; **Residual** — sai lệch.

Tình huống thực tế: Một cửa hàng ghi **10 hóa đơn** bán hai loại hàng A và B. Trên mỗi hóa đơn có **số món A**, **số món B** và **tổng tiền**, đơn vị **nghìn đồng**; ví dụ hóa đơn đầu có mỗi loại một món, tổng 5 nghìn đồng, hóa đơn thứ hai có hai món A và một món B, tổng 8 nghìn đồng. Giá một món của mỗi loại chưa biết nhưng giữ nguyên trong cả 10 hóa đơn. Thu ngân muốn tìm giá từng loại rồi kiểm lại **tất cả hóa đơn** để bảo đảm giá ấy giải thích đúng số tiền.

Vì sao chọn cách này? Hệ phương trình ở đây là hai hóa đơn chứa hai mức giá chưa biết; với ví dụ đầu tiên, cộng giá một món A và một món B được 5 nghìn đồng, còn hóa đơn thứ hai có thêm một món A nên tổng là 8 nghìn đồng. Hai cách phối số lượng khác nhau cho đủ thông tin tìm riêng từng giá. `np.linalg.solve()` giải **hai phương trình đầu** để tìm hai giá, nhưng một nghiệm khớp hai dòng chưa chắc đúng cả sổ. Vì vậy `quantities @ prices` tính lại tiền của **cả 10 hóa đơn** rồi lấy chênh lệch với tiền thực; sai số rất nhỏ được bỏ qua bằng ngưỡng tính toán. Không nên kết luận mọi dòng đúng chỉ vì giải được hai dòng mẫu.

Quy trình:

1. Tạo bảng 10 hàng số lượng và 10 tổng tiền cùng thứ tự.
2. Chọn hai hàng độc lập đầu để giải hệ 2 ẩn.
3. Dự tính tổng tiền cả 10 hàng rồi đếm sai lệch lớn hơn ngưỡng nhỏ.

No Note — Không chú thích

```
import numpy as np
quantities = np.array([[1,1],[2,1],[3,1],[4,1],[1,2],[2,2],[3,2],[4,2],[1,3],[2,3]], dtype=float)
totals = np.array([5,8,11,14,7,10,13,16,9,12], dtype=float)
prices = np.linalg.solve(quantities[:2], totals[:2])
residual = quantities @ prices - totals
if quantities.shape != (10, 2):
    raise ValueError("Cần đủ 10 hóa đơn và hai số lượng.")
print("Hóa đơn:", len(totals))
print("Giá A (nghìn đồng):", f"{prices[0]:.1f}")
print("Giá B (nghìn đồng):", f"{prices[1]:.1f}")
print("Hóa đơn sai:", int((np.abs(residual) > 1e-8).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Giải hai giá sản phẩm từ 10 hóa đơn và dùng phần dư kiểm toàn bộ hóa đơn.
# Dữ liệu: 10 hàng hóa đơn, hai cột số lượng món A/B; totals là tổng tiền nghìn đồng từng hóa đơn.
# NumPy giải hệ hai ẩn và tính phần dư cho nhiều hóa đơn.
import numpy as np
# Ma trận (10,2); mỗi hàng là số lượng sản phẩm A, B, dtype=float phục vụ giải.
quantities = np.array([[1,1],[2,1],[3,1],[4,1],[1,2],[2,2],[3,2],[4,2],[1,3],[2,3]], dtype=float)
# Vector 10 tổng tiền (nghìn đồng) cùng thứ tự hàng quantities.
totals = np.array([5,8,11,14,7,10,13,16,9,12], dtype=float)
# [:2] lấy HAI hóa đơn đầu (vị trí 0 và 1), làm hệ 2×2 đủ để tìm hai giá.
# [:2] lấy vế phải tương ứng với hai hóa đơn đã chọn.
# solve() tìm hai đơn giá nghìn đồng sao cho hai hóa đơn đầu khớp chính xác; cần ma trận con khả
nghịch.
prices = np.linalg.solve(quantities[:2], totals[:2])
# @ nhân mỗi hàng số lượng với vector giá, trả tổng tiền dự đoán từng hóa đơn.
# Trừ totals để có phần dư dự đoán trừ thực tế từng hóa đơn (nghìn đồng).
residual = quantities @ prices - totals
# shape (10,2) bảo đảm đủ 10 hóa đơn và 2 mặt hàng.
if quantities.shape != (10, 2):
    # raise ValueError() dừng nếu không đủ hàng/cột hoặc điều kiện dữ liệu kiểm thử sai.
    raise ValueError("Cần đủ 10 hóa đơn và hai số lượng.")
print("Hóa đơn:", len(totals))
# [0] là giá A, [1] giá B; :.1f chỉ định in một chữ số thập phân.
print("Giá A (nghìn đồng):", f"{prices[0]:.1f}")
```

```
print("Giá B (nghìn đồng):", f"{prices[1]:.1f}")
# abs() lấy độ lớn phần dư; ngưỡng 1e-8 nghìn đồng loại sai số́ số́ thực cực nhỏ, .sum() đếm hóa đơn
vượt ngưỡng.
print("Hóa đơn sai:", int((np.abs(residual) > 1e-8).sum()))
```

Kết quả khi chạy:

```
Hóa đơn: 10
Giá A (nghìn đồng): 3.0
Giá B (nghìn đồng): 2.0
Hóa đơn sai: 0
```

Đơn giá là **nghìn đồng/món**, còn tổng tiền là nghìn đồng; nghiệm chỉ có ý nghĩa vì hai phương trình đầu độc lập.

Tình huống 2 — Cấp 2: Nhận biết hai cột phụ thuộc trong 12 lượt vận hành

Từ vựng trong mục: Rank — hạng ma trận; **Linear dependence** — phụ thuộc tuyến tính.

Tình huống thực tế: Trạm máy theo dõi **12 lượt vận hành**, mỗi lượt ghi số **giờ chạy máy x** và một cột báo cáo được tạo bằng **hai lần số giờ x**. Người quản lý định dùng hai cột để dự đoán, nhưng lo rằng cột thứ hai không bổ sung thông tin mới. Họ cần biết bảng có thực sự **hai nguồn số liệu độc lập** hay chỉ một nguồn được ghi lại theo cách khác.

Vì sao chọn cách này? Hạng ma trận (rank) là số hướng thông tin **độc lập về mặt tuyến tính**. Hãy hình dung biết cột giờ chạy là 5 giờ thì cột kia chắc chắn là 10; nó không kể thêm một điều gì chưa biết. `np.column_stack()` ghép hai cột thành bảng 12 hàng; `np.linalg.matrix_rank(X)` kiểm tra số cột thực sự độc lập. Kết quả hạng **1** nghĩa là hai cột chỉ tạo **một hướng thông tin**. Đây là phụ thuộc tuyến tính giữa các cột, không có nghĩa 12 lượt vận hành chỉ còn một lượt hoặc số giờ chạy là sai.

Quy trình:

1. Lưu 12 cặp số giờ, cột hai bằng hai lần cột một.
2. Gọi `matrix_rank` để kiểm số chiều độc lập.
3. Kiểm tra hai cột phụ thuộc và in kích thước/hạng.

No Note — Không chú thích

```
import numpy as np
hours = np.array([1,2,3,4,5,6,7,8,9,10,11,12], dtype=float)
X = np.column_stack((hours, 2 * hours))
if X.shape != (12, 2):
    raise ValueError("Cần đủ 12 lượt vận hành với hai cột.")
print("Lượt vận hành:", X.shape[0])
print("Cột:", X.shape[1])
print("Hạng:", int(np.linalg.matrix_rank(X)))
print("Cột 2 = 2 × cột 1:", bool(np.allclose(X[:, 1], 2 * X[:, 0])))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phát hiện hai cột dữ liệu có quan hệ phụ thuộc bằng matrix_rank.
# Dữ liệu: 12 lượt vận hành theo giờ; cột thứ nhất giờ, cột thứ hai luôn đúng gấp đôi số́ giờ.
# NumPy tạo mảng và xác định số́ cột độc lập.
import numpy as np
# Vector 12 số́ giờ, dtype=float để̉ dùng tính toán số́ học.
hours = np.array([1,2,3,4,5,6,7,8,9,10,11,12], dtype=float)
```

```
# column_stack ghép hai cột, cột thứ hai bằng 2×cột đầu nên không có thông tin độc lập bổ sung.
X = np.column_stack((hours, 2 * hours))
# shape (12,2) nghĩa 12 lượt quan sát và 2 cột dữ liệu.
if X.shape != (12, 2):
    # raise ValueError() dừng nếu không đủ hàng/cột hoặc điều kiện dữ liệu kiểm thử sai.
    raise ValueError("Cần đủ 12 lượt vận hành với hai cột.")
# shape (12,2) nghĩa 12 lượt quan sát và 2 cột dữ liệu.
# shape[0] đếm lượt/hàng, shape[1] đếm cột biến đo.
print("Lượt vận hành:", X.shape[0])
# shape (12,2) nghĩa 12 lượt quan sát và 2 cột dữ liệu.
print("Cột:", X.shape[1])
# matrix_rank() đếm số hướng độc lập, chỉ bằng 1 khi hai cột tỷ lệ cố định.
print("Hạng:", int(np.linalg.matrix_rank(X)))
# [:,1] lấy tất cả hàng của cột thứ hai, [:,0] lấy cột đầu.
# allclose() so từng giá trị với dung sai số thực, an toàn hơn == khi phép toán float.
# bool() biểu diễn kết quả kiểm NumPy dưới dạng True/False Python.
print("Cột 2 = 2 × cột 1:", bool(np.allclose(X[:, 1], 2 * X[:, 0])))
```

Kết quả khi chạy:

```
Lượt vận hành: 12
Cột: 2
Hạng: 1
Cột 2 = 2 × cột 1: True
```

Hai cột không phải hai đặc trưng độc lập; rank bằng 1 dù ma trận có 12 hàng và 2 cột.

Tình huống 3 — Cấp 3: Phục hồi 15 cặp tín hiệu từ hai kênh

Từ vựng trong mục: Coefficient matrix — ma trận hệ số; **Augmented data** — dữ liệu mở rộng; **solve** — giải hệ.

Tình huống thực tế: Một máy đo có **15 lượt ghi tín hiệu**; mỗi lượt cho hai số đo kênh A và B theo một quy tắc pha trộn cố định. A chứa **hai phần nguồn x và một phần nguồn y**; B chứa **một phần x và ba phần y**. Hai nguồn x, y chưa biết và có cùng đơn vị tín hiệu như số đo. Kỹ thuật viên cần khôi phục hai nguồn **cho từng lượt**, rồi thử pha lại để chắc chắn khớp cả 15 cặp số đã nhận.

Vì sao chọn cách này? Ma trận hệ số `A` ghi cách hai nguồn góp vào mỗi kênh. Ma trận này **giữ nguyên cho cả 15 lần đo** và có đủ thông tin để phân biệt nguồn x với y. `np.linalg.solve(A, measured.T)` giải nhiều cặp số đo trong **một lần gọi**; `.T` đổi từ dạng mỗi lượt một hàng sang cách hàm giải nhận các vế phải ở từng cột, rồi chuyển kết quả về mỗi lượt một hàng. Sau giải, `restored @ A.T` thử **pha ngược lại** hai nguồn vừa tìm; `np.allclose()` xác nhận số tái tạo gần bằng số đo gốc. Nếu chỉ đọc nghiệm mà không tái tạo, sai thứ tự kênh hoặc nhầm hệ số có thể không được phát hiện.

Quy trình:

1. Tạo ma trận hệ số và 15 cặp kênh được đo.
2. Chuyển vị số đo, đưa vào `np.linalg.solve` rồi chuyển vị kết quả.
3. Tái tạo hai kênh và kiểm tra mọi lượt.

No Note — Không chú thích

```
import numpy as np
A = np.array([[2., 1.], [1., 3.]])
measured = np.array([[4,7],[6,8],[8,9],[10,10],[12,11],[5,10],[7,11],[9,12],[11,13],[13,14],[6,13],
[8,14],[10,15],[12,16],[14,17]], dtype=float)
restored = np.linalg.solve(A, measured.T).T
```

```
if measured.shape != (15, 2) or restored.shape != (15, 2):
    raise ValueError("Cần đúng 15 lượt đo hai tín hiệu.")
print("Lượt đo:", len(restored))
print("Hạng hệ số:", int(np.linalg.matrix_rank(A)))
first_values = []
for value in restored[0]:
    first_values.append(f"{value:.1f}")
print("Tín hiệu gốc đầu:", ", ".join(first_values))
print("Tái tạo đúng:", bool(np.allclose(restored @ A.T, measured)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Khôi phục 15 cặp tín hiệu gốc từ hỗn hợp đo bằng hệ tuyến tính hai chiều.
# Dữ liệu: 15 hàng số đo hỗn hợp, hai cột cảm biến, A là ma trận trộn 2×2 biết trước.
# NumPy giải hệ nhiều vế phải đồng thời bằng linalg.solve().
import numpy as np
# Ma trận hệ số trộn tín hiệu 2×2, float; hạng đủ 2 thì có nghiệm duy nhất.
A = np.array([[2., 1.], [1., 3.]])
# 15 hàng × 2 số đo thực tế, mỗi hàng là một lượt đo.
measured = np.array([[4,7],[6,8],[8,9],[10,10],[12,11],[5,10],[7,11],[9,12],[11,13],[13,14],[6,13],
[8,14],[10,15],[12,16],[14,17]], dtype=float)
# Chuyển vị từ (15,2) thành (2,15) để mỗi lượt trở thành một cột vế phải.
# solve(A,B) giải 15 hệ cùng A; .T cuối đưa đáp số về 15 hàng × 2 cột gốc.
restored = np.linalg.solve(A, measured.T).T
# Đối chiếu cả nguồn và số liệu khôi phục có shape (15,2).
if measured.shape != (15, 2) or restored.shape != (15, 2):
    # raise ValueError() dừng nếu không đủ hàng/cột hoặc điều kiện dữ liệu kiểm thử sai.
    raise ValueError("Cần đúng 15 lượt đo hai tín hiệu.")
print("Lượt đo:", len(restored))
# matrix_rank(A) xác minh hệ số có 2 hướng độc lập, không phải 15 lượt đo.
print("Hạng hệ số:", int(np.linalg.matrix_rank(A)))
first_values = []
# [0] lấy hai tín hiệu khôi phục của lượt đo thứ nhất.
for value in restored[0]:
    # f-string :.1f hiển thị một chữ số thập phân cho từng tín hiệu.
    first_values.append(f"{value:.1f}")
# join() nối các giá trị đã định dạng bằng dấu phẩy và khoảng trắng.
print("Tín hiệu gốc đầu:", ", ".join(first_values))
# Vì mỗi tín hiệu khôi phục là vector HÀNG, @ A.T tái tạo đúng hỗn hợp; .T của A phù hợp quy ước
solve theo cột.
# allclose() so mảng tái tạo với dữ liệu đo trong dung sai số thực; bool() để in Boolean Python.
print("Tái tạo đúng:", bool(np.allclose(restored @ A.T, measured)))
```

Kết quả khi chạy:

```
Lượt đo: 15
Hạng hệ số: 2
Tín hiệu gốc đầu: 1.0, 2.0
Tái tạo đúng: True
```

Một cặp giá trị quan sát phải ứng với **cùng lượt**; không ghép kênh A của lượt này với B của lượt khác.

Tình huống 4 — Cấp 4: Phát hiện 18 phương trình không nhất quán bằng hạng mở rộng

Từ vựng trong mục: Augmented matrix — ma trận mở rộng; **Inconsistent system** — hệ vô nghiệm; **Residual** — sai số dư.

Tình huống thực tế: Phòng hiệu chuẩn ghi **18 lượt dữ liệu**, mỗi lượt có thời gian máy hoạt động **x (giờ)** và mức tiêu thụ **y (đơn vị mô phỏng)**. Nhóm giả định tất cả số đo tuân theo **một đường thẳng chung**,

tức tăng thêm một lượng cố định theo giờ. Tuy nhiên, ở lượt thứ 10, mức tiêu thụ bị ghi cao hơn dự kiến **7 đơn vị**. Kỹ thuật viên muốn biết liệu có thể tìm **một đường thẳng đi qua chính xác cả 18 điểm** không, trước khi chuyển sang cách tìm đường gần đúng.

Vì sao chọn cách này? Hạng của hệ số cho biết hai thành phần đường thẳng (mức tăng và mức nền) có độc lập không; **hạng ma trận mở rộng** còn tính cả cột số đo thực tế y. Nếu thêm cột y làm hạng tăng, nghĩa là các số đo **không thể đồng thời khớp hoàn toàn** cùng một quy tắc tuyến tính. Trong ví dụ, lượt 10 bị lệch 7 đơn vị nên hệ 18 dòng bị mâu thuẫn. `np.linalg.matrix_rank()` kiểm tra hai hạng trước khi tìm nghiệm. `np.linalg.solve()` không phù hợp cho bảng **18 hàng × 2 cột** vì đó không phải ma trận vuông 2×2; bài này chỉ kiểm tra **có thể khớp chính xác hay không**, chưa tìm đường gần đúng.

Quy trình:

1. Tạo bảng hệ số 18 hàng, hai cột x và cột 1; y là vế phải.
2. Ghép ma trận mở rộng bằng `column_stack`, tính hai hạng.
3. Báo kết luận có hay không nghiệm khớp mọi lượt.

No Note — Không chú thích

```
import numpy as np
hours = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18], dtype=float)
y = np.array([7,9,11,13,15,17,19,21,23,32,27,29,31,33,35,37,39,41], dtype=float)
A = np.column_stack((hours, np.ones_like(hours)))
augmented = np.column_stack((A, y))
rank_a = int(np.linalg.matrix_rank(A))
rank_aug = int(np.linalg.matrix_rank(augmented))
if A.shape != (18, 2) or augmented.shape != (18, 3):
    raise ValueError("Hệ có 18 phương trình, hai hệ số´và một vê´phải.")
print("Số´phương trình:", len(y))
print("Hạng hệ số:", rank_a)
print("Hạng mở rộng:", rank_aug)
print("Khớp tất cả:", rank_a == rank_aug)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So hạng hệ số´và ma trận mở rộng để phát hiện 18 phương trình không khớp một đường
thẳng duy nhất.
# Dữ liệu: 18 cặp (giờ,giá trị đo), đâu ra tại giờ 10 lệch khỏi xu thê´chung.
# NumPy cho phép dựng ma trận mở rộng và so hạng tuyến tính.
import numpy as np
# 18 giá trị giờ làm biến đầu vào, dtype=float.
hours = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18], dtype=float)
# 18 số´đo cùng vị trí với giờ, có một điểm lệch khiến hệ thiếu nghiệm chính xác.
y = np.array([7,9,11,13,15,17,19,21,23,32,27,29,31,33,35,37,39,41], dtype=float)
# Tạo 18 số´1 cho hệ số´chặn (intercept) trong mô hình y=slope*hours+intercept.
# A=(giờ,cột1) có shape (18,2), hai cột tương ứng hai hệ số´chưa biết.
A = np.column_stack((hours, np.ones_like(hours)))
# Thêm y thành cột thứ ba cho ma trận mở rộng shape (18,3).
augmented = np.column_stack((A, y))
# Hạng A đếm số´cột phương trình độc lập.
rank_a = int(np.linalg.matrix_rank(A))
# Hạng mở rộng đếm thêm ràng buộc của vê´phải; lớn hơn hạng A nghĩa không có nghiệm khớp TẤT CẢ
phương trình.
rank_aug = int(np.linalg.matrix_rank(augmented))
# Đối chiếu shape 18×2 và 18×3 trước báo cáo.
if A.shape != (18, 2) or augmented.shape != (18, 3):
    # raise ValueError() dừng nếu không đủ hàng/cột hoặc điều kiện dữ liệu kiểm thử sai.
    raise ValueError("Hệ có 18 phương trình, hai hệ số´và một vê´phải.")
print("Số´phương trình:", len(y))
print("Hạng hệ số:", rank_a)
```

```
print("Hạng mở rộng:", rank_aug)
# So hạng là điều kiện tồn tại nghiệm chính xác; False ở dữ liệu có sai lệch.
print("Khớp tất cả:", rank_a == rank_aug)
```

Kết quả khi chạy:

```
Số phương trình: 18
Hạng hệ số: 2
Hạng mở rộng: 3
Khớp tất cả: False
```

Điều này không chứng minh toàn bộ dữ liệu vô dụng; nó chỉ chứng minh không thể có **nghiệm chính xác chung** cho tất cả phương trình.

Tình huống 5 — Cấp 5: Ước lượng tuyến tính từ 20 số đo nhiễu và kiểm tra sai số dư

Từ vựng trong mục: Least squares — bình phương tối thiểu; **Residual** — sai số dư; **Rank** — hạng.

Tình huống thực tế: Nhà máy đo **20 cặp số liệu** giờ chạy máy (**giờ**) và năng lượng dùng (**kWh**). Phần lớn dữ liệu tăng gần **4 kWh cho mỗi giờ thêm**, từ mức nền khoảng **10 kWh**, nhưng lượt 5 và 17 có sai lệch giả lập. Người phân tích cần tìm một đường thẳng **gần phù hợp nhất** với toàn bộ 20 lượt, báo mức tăng theo giờ, mức nền, sai số trung bình bình phương **MSE** và số lượt dự báo sai quá **5 kWh**.

Vì sao chọn cách này? Bình phương tối thiểu được dùng khi có nhiều số đo hơn số thông số cần tìm và các điểm **không nằm chính xác trên cùng một đường**. `np.linalg.lstsq()` chọn mức tăng theo giờ (**kWh/giờ**) và mức nền (**kWh**) sao cho tổng các **sai lệch đã bình phương** là nhỏ nhất; các sai lệch lớn bị phạt mạnh hơn. Sau đó chương trình dự đoán lại 20 lượt, tính **MSE** bằng trung bình bình phương sai lệch (đơn vị **kWh²**) và đếm lượt lệch quá 5 kWh. Đây là đường khớp **gần đúng**, không phải cách sửa số đo nhiễu thành sự thật; nếu cần giải thích sự cố ở lượt 5, 17 phải kiểm tra dữ liệu nguồn.

Quy trình:

1. Tạo A từ 20 cặp x/y với một cột x và một cột 1.
2. Fit bằng `lstsq` khi A có hạng 2.
3. Tính phần dư, MSE và số lượt có sai lệch lớn.

No Note — Không chú thích

```
import numpy as np
hours = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20], dtype=float)
energy = np.array([14,18,22,26,42,34,38,42,46,50,54,58,62,66,70,74,70,82,86,90], dtype=float)
A = np.column_stack((hours, np.ones_like(hours)))
slope, intercept = np.linalg.lstsq(A, energy, rcond=None)[0]
residual = A @ np.array([slope, intercept]) - energy
if A.shape != (20, 2) or np.linalg.matrix_rank(A) != 2:
    raise ValueError("Cần đủ 20 lượt đo và hai cột độc lập.")
print("Lượt đo:", len(energy))
print(f"Độ dốc (kWh/giờ): {slope:.2f}")
print(f"Nền (kWh): {intercept:.2f}")
print(f"MSE (kWh²): {np.mean(residual ** 2):.2f}")
print("Lệch > 5 kWh:", int((np.abs(residual) > 5).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Fit đường tuyến tính cho 20 lượt năng lượng bằng least squares và đếm số đo lệch quá 5
kWh.
```

```
# Dữ liệu: 20 cặp (giờ hoạt động,điện tiêu thụ kWh); một số lượt có nhiễu, ước lượng độ dốc kWh/giờ và nền kWh.
# NumPy cung cấp matrix_rank(), lstsq() và thống kê phần dư.
import numpy as np
# 20 giờ vận hành, dtype=float làm cột đặc trưng.
hours = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20], dtype=float)
# 20 số điện kWh theo thứ tự giờ, có vài giá trị nhiễu so đường thẳng.
energy = np.array([14,18,22,26,42,34,38,42,46,50,54,58,62,66,70,74,70,82,86,90], dtype=float)
# Vector 20 số 1 tạo cột hệ số chặn (điện nền).
# Ghép giờ và hằng số 1 thành A shape (20,2) cho y≈slope*hours+intercept.
A = np.column_stack((hours, np.ones_like(hours)))
# lstsq tìm độ dốc và hệ số nền giảm tổng bình phương sai lệch; rcond=None dùng ngưỡng hạng mặc định; [0] lấy vector nghiệm.
# Giải cấu trúc hai giá trị: slope kWh mỗi giờ, intercept kWh nền khi giờ bằng 0.
slope, intercept = np.linalg.lstsq(A, energy, rcond=None)[0]
# @ tính lượng điện dự đoán từ hệ số fit cho từng giờ, một giá trị mỗi hàng.
# Trừ số thực đo để ra residual (kWh) theo từng lượt.
residual = A @ np.array([slope, intercept]) - energy
# shape (20,2) kiểm đúng số mẫu và hai hệ số.
# Đòi hai cột độc lập để xác định slope/intercept, tránh fit không xác định.
if A.shape != (20, 2) or np.linalg.matrix_rank(A) != 2:
    # raise ValueError() dừng nếu không đủ hàng/cột hoặc điều kiện dữ liệu kiểm thử sai.
    raise ValueError("Cần đủ 20 lượt đo và hai cột độc lập.")
print("Lượt đo:", len(energy))
print(f"Độ dốc (kWh/giờ): {slope:.2f}")
print(f"Nền (kWh): {intercept:.2f}")
# **2 bình phương mỗi phần dư kWh; np.mean() tính MSE có đơn vị kWh², nhấn mạnh phạt sai số lớn.
print(f"MSE (kWh²): {np.mean(residual ** 2):.2f}")
# abs() lấy độ lệch không dấu, >5 tạo mask, sum() đếm lượt lệch hơn 5 kWh.
print("Lệch > 5 kWh:", int((np.abs(residual) > 5).sum()))
```

Kết quả khi chạy:

```
Lượt đo: 20
Độ dốc (kWh/giờ): 3.82
Nền (kWh): 12.06
MSE (kWh²): 9.31
Lệch > 5 kWh: 2
```

MSE bị ảnh hưởng bởi hai điểm nhiễu; không nên lấy đường fit này làm quy luật tiêu thụ chắc chắn của một thiết bị thật.

6. Sai lầm thường gặp

- Một hàng của A là một phương trình; một cột là hệ số của một ẩn. Không được đảo hai chiều khi đọc dữ liệu.
- b là giá trị đã biết ở vế phải, không phải cột hệ số của một ẩn mới.
- Hệ phụ thuộc hàng có thể vô nghiệm hoặc vô số nghiệm; xét cả rank(A) và rank([A, b]), không kết luận chỉ từ rank(A).
- `solve` dành cho ma trận vuông khả nghịch; `lstsq` giải gần đúng được cả hệ có nhiều số đo.
- Residual có dấu; bình phương tối thiểu tối ưu tổng **bình phương** dư chứ không tối ưu tổng dư có dấu.
- Chỉ dùng `np.allclose` để kiểm nghiệm trong sai số số học, không coi đó là bằng nhau tuyệt đối.

7. Trắc nghiệm

Các câu hỏi chỉ kiểm tra khái niệm và cách áp dụng kiến thức, không yêu cầu nhớ tình huống thực hành.

Q1. What is a linear system? | Hệ tuyến tính là gì?

A. The coefficients multiplying the unknowns | Các hệ số nhân với từng ẩn

B. The number of linearly independent rows or columns | Số hàng hoặc cột độc lập tuyến tính

C. `A @ x - b` | `A @ x - b`

D. Several linear equations sharing unknowns | Nhiều phương trình tuyến tính chung các ẩn

Đáp án: D

Giải thích: Các phương trình cùng chứa các ẩn cần giải.

Q2. What does the rank of a matrix measure? | Rank của ma trận đo gì?

A. The number of linearly independent rows or columns | Số hàng hoặc cột độc lập tuyến tính

B. `A @ x - b` | `A @ x - b`

C. Several linear equations sharing unknowns | Nhiều phương trình tuyến tính chung các ẩn

D. The coefficients multiplying the unknowns | Các hệ số nhân với từng ẩn

Đáp án: A

Giải thích: Rank đếm hướng độc lập, không chỉ số hàng.

Q3. What does a coefficient matrix contain? | Ma trận hệ số chứa gì?

A. The number of linearly independent rows or columns | Số hàng hoặc cột độc lập tuyến tính

B. Several linear equations sharing unknowns | Nhiều phương trình tuyến tính chung các ẩn

C. The coefficients multiplying the unknowns | Các hệ số nhân với từng ẩn

D. `A @ x - b` | `A @ x - b`

Đáp án: C

Giải thích: Ma trận chứa hệ số số học của từng ẩn.

Q4. What is the residual vector for Ax=b? | Vector sai số dư của Ax=b là gì?

A. The number of linearly independent rows or columns | Số hàng hoặc cột độc lập tuyến tính

B. `A @ x - b` | `A @ x - b`

C. The coefficients multiplying the unknowns | Các hệ số nhân với từng ẩn

D. Several linear equations sharing unknowns | Nhiều phương trình tuyến tính chung các ẩn

Đáp án: B

Giải thích: Lấy Ax trừ b cho độ lệch từng phương trình.

Q5. Solve x + y = 7 and x − y = 3. What are x and y? | Giải hệ x + y = 7 và x − y = 3. Tìm x và y.

A. x = 2, y = 5 | x = 2, y = 5

B. x = 7, y = 0 | x = 7, y = 0

C. x = 5, y = 2 | x = 5, y = 2

D. x = 4, y = 3 | x = 4, y = 3

Đáp án: C

Giải thích: Cộng hai phương trình: 2x = 10 nên x = 5; thay vào x + y = 7 được y = 2.

Q6. What can be concluded about x + y = 3 and x + y = 5? | Hệ x + y = 3 và x + y = 5 có kết luận gì?

A. A unique solution | Có nghiệm duy nhất

B. Exactly two solutions | Có đúng hai nghiệm

C. The solution is (3, 5) | Nghiệm là (3, 5)

D. No solution | Vô nghiệm

Đáp án: D

Giải thích: Cùng một tổng x + y không thể đồng thời bằng 3 và 5, nên hai phương trình mâu thuẫn.

Q7. What is the determinant of [[1, 2], [2, 4]]? | Định thức của [[1, 2], [2, 4]] là bao nhiêu?

A. 0 | 0

B. 8 | 8

C. 4 | 4

D. 2 | 2

Đáp án: A

Giải thích: 1 × 4 − 2 × 2 = 0; hàng thứ hai gấp đôi hàng thứ nhất, nên ma trận thiếu hạng.

Q8. Does (x, y) = (2, 1) solve x + 2y = 4 and 3x + y = 7? | Cặp (x, y) = (2, 1) có giải đúng hệ x + 2y = 4 và 3x + y = 7 không?

A. No, both equations fail | Không, cả hai đều sai

B. No, the second equation fails | Không, phương trình sau sai

C. No, the first equation fails | Không, phương trình đầu sai

D. Yes, both equations hold | Có, cả hai đều đúng

Đáp án: D

Giải thích: Thế x = 2, y = 1: 2 + 2 × 1 = 4 và 3 × 2 + 1 = 7, nên đây là một nghiệm.

Q9. For Ax=b with A shape (3,2), what do the rows represent? | Với A shape (3,2), các hàng biểu diễn gì?

A. The right-hand-side values | Các giá trị vế phải

B. Coefficients of two unknowns | Hệ số của hai ẩn

C. It cannot have a unique solution for every b | Không thể có nghiệm duy nhất với mọi b

D. Three equations or observations | Ba phương trình hoặc quan sát

Đáp án: D

Giải thích: Ba hàng tương ứng ba quan sát hoặc phương trình.

Q10. For Ax=b with A shape (3,2), what do the columns represent? | Với A shape (3,2), các cột biểu diễn gì?

A. Three equations or observations | Ba phương trình hoặc quan sát

B. The right-hand-side values | Các giá trị vế phải

C. It cannot have a unique solution for every b | Không thể có nghiệm duy nhất với mọi b

D. Coefficients of two unknowns | Hệ số của hai ẩn

Đáp án: D

Giải thích: Hai cột là hệ số của hai ẩn ở từng phương trình.

Q11. What does the final column of an augmented matrix represent? | Cột cuối của ma trận mở rộng biểu diễn gì?

A. It cannot have a unique solution for every b | Không thể có nghiệm duy nhất với mọi b

B. Coefficients of two unknowns | Hệ số của hai ẩn

C. Three equations or observations | Ba phương trình hoặc quan sát

D. The right-hand-side values | Các giá trị vế phải

Đáp án: D

Giải thích: Ma trận mở rộng ghi các vế phải ở cột cuối.

Q12. What does a rank-deficient square matrix imply? | Ma trận vuông thiếu hạng gợi ý điều gì?

A. It cannot have a unique solution for every b | Không thể có nghiệm duy nhất với mọi b

B. Coefficients of two unknowns | Hệ số của hai ẩn

C. Three equations or observations | Ba phương trình hoặc quan sát

D. The right-hand-side values | Các giá trị vế phải

Đáp án: A

Giải thích: Thiếu hạng đầy đủ khiến không bảo đảm nghiệm duy nhất.

Q13. If the residual is exactly zero, what is true? | Nếu sai số dư bằng 0 thì điều gì đúng?

A. Ranks of A and its augmented matrix | Hạng của A và ma trận mở rộng

B. Whether a candidate solution approximately satisfies the system | Nghiệm thử có thỏa hệ gần đúng hay không

C. The sum of squared residuals | Tổng bình phương sai số dư

D. Every equation is satisfied | Mọi phương trình đều được thỏa

Đáp án: D

Giải thích: Sai số dư bằng 0 nghĩa là Ax=b.

Q14. What does least squares minimize? | Bình phương tối thiểu làm nhỏ nhất gì?

A. Ranks of A and its augmented matrix | Hạng của A và ma trận mở rộng

B. Every equation is satisfied | Mọi phương trình đều được thỏa

C. The sum of squared residuals | Tổng bình phương sai số dư

D. Whether a candidate solution approximately satisfies the system | Nghiệm thử có thỏa hệ gần đúng hay không

Đáp án: C

Giải thích: Bình phương sai lệch làm lỗi lớn bị phạt mạnh.

Q15. What does `np.allclose(A@x,b)` check? | `np.allclose(A@x,b)` kiểm tra gì?

A. Ranks of A and its augmented matrix | Hạng của A và ma trận mở rộng

B. Whether a candidate solution approximately satisfies the system | Nghiệm thử có thỏa hệ gần đúng hay không

C. The sum of squared residuals | Tổng bình phương sai số dư

D. Every equation is satisfied | Mọi phương trình đều được thỏa

Đáp án: B

Giải thích: allclose so kết quả trong ngưỡng sai số số học.

Q16. What should be compared to diagnose inconsistency of a system? | Để xét hệ vô nghiệm cần so sánh gì?

A. Whether a candidate solution approximately satisfies the system | Nghiệm thử có thỏa hệ gần đúng hay không

B. The sum of squared residuals | Tổng bình phương sai số dư

C. Every equation is satisfied | Mọi phương trình đều được thỏa

D. Ranks of A and its augmented matrix | Hạng của A và ma trận mở rộng

Đáp án: D

Giải thích: Hạng A khác hạng [A, b] cho biết hệ vô nghiệm.

Q17. Why isn't the right-hand side b another unknown? | Vì sao b không phải một ẩn mới?

A. It contains known target values | Nó chứa giá trị mục tiêu đã biết

B. They add no new independent direction | Chúng không bổ sung hướng độc lập mới

C. When observations outnumber unknowns or do not fit exactly | Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

D. Prediction is lower than actual | Dự báo thấp hơn thực tế

Đáp án: A

Giải thích: b là dữ liệu đã biết, x mới là biến chưa biết.

Q18. What happens when rows of A are multiples of each other? | Nếu các hàng A là bội nhau thì sao?

A. Prediction is lower than actual | Dự báo thấp hơn thực tế

B. They add no new independent direction | Chúng không bổ sung hướng độc lập mới

C. It contains known target values | Nó chứa giá trị mục tiêu đã biết

D. When observations outnumber unknowns or do not fit exactly | Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

Đáp án: B

Giải thích: Hàng tỷ lệ chỉ lặp thông tin tuyến tính.

Q19. When is `lstsq` useful? | Khi nào `lstsq` hữu ích?

A. Prediction is lower than actual | Dự báo thấp hơn thực tế

B. When observations outnumber unknowns or do not fit exactly | Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

C. It contains known target values | Nó chứa giá trị mục tiêu đã biết

D. They add no new independent direction | Chúng không bổ sung hướng độc lập mới

Đáp án: B

Giải thích: Bình phương tối thiểu tìm cách khớp tốt nhất khi hệ không thỏa hết.

Q20. What does a negative residual mean under r=prediction−actual? | Sai số dư âm theo r=dự báo−thực tế nghĩa là gì?

A. When observations outnumber unknowns or do not fit exactly | Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

B. Prediction is lower than actual | Dự báo thấp hơn thực tế

C. They add no new independent direction | Chúng không bổ sung hướng độc lập mới

D. It contains known target values | Nó chứa giá trị mục tiêu đã biết

Đáp án: B

Giải thích: Dự báo trừ thực tế là số âm khi dự báo thiếu.

8. Nối sang bài kế tiếp

`Ma trận → Rank/Solve → Regression → AI.`

PY-27 — Eigenvalue, Eigenvector và hướng biến thiên

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Eigenvector — vector riêng; **Eigenvalue** — giá trị riêng.

`Hệ tuyến tính → Eigenvalues/Vectors → Covariance → PCA`

Học xong dùng để làm gì? Tìm các hướng giữ nguyên phương khi ma trận tác động, làm nền để hiểu PCA và biến đổi dữ liệu.

Kết nối kiến thức tiếp theo: PY-28 mô tả quan hệ giữa vector đầu vào và dự đoán bằng hàm toán học.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Eigenvector	vector riêng	Hướng mà phép biến đổi tuyến tính chỉ co/dãn
Eigenvalue	giá trị riêng	Hệ số co/dãn theo một vector riêng
Covariance	hiệp phương sai	Cách hai đặc trưng biến thiên cùng nhau
Principal direction	hướng chính	Hướng chứa nhiều phương sai nhất trong PCA

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.linalg.eigh(C)`	Trả eigenvalues/eigenvectors cho ma trận đối xứng
`np.linalg.eigvalsh(C)`	Chỉ trả eigenvalues của ma trận đối xứng
`np.cov(X,rowvar=False)`	Cột là feature, hàng là mẫu khi tính covariance
`np.argsort(values)[::-1]`	Sắp eigenvalue giảm dần để chọn hướng lớn

4. Khái niệm liên quan

Nội dung chính: ma trận biến đổi có thể giữ nguyên phương của một số vector riêng. Khi ma trận là hiệp phương sai, trị riêng đo phương sai theo hướng đó, tạo cầu nối sang PCA.

4.1. Eigenvector — Hướng không bị xoay sau biến đổi

Từ vựng trong mục: Eigenvector — vector riêng; **Transformation** — phép biến đổi.

Một **eigenvector** là vector khác không mà khi nhân với ma trận vuông A, nó chỉ đổi **độ lớn hoặc đảo chiều**, không rời khỏi đường phương ban đầu: `A @ v = λ*v`. Ví dụ ma trận `diag(2,3)` nhân vector `[0,1]` cho `[0,3]`, vẫn nằm trên trục tung. Điều kiện là vector khác không và phù hợp số chiều của A; vector riêng không phải bất kỳ vector nào.

No Note — Không chú thích

```
import numpy as np

A = np.diag([2.0, 3.0])
v = np.array([0.0, 1.0])
print(A @ v)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa vector riêng giữ nguyên hướng khi bị ma trận giãn theo trục.
# Dữ liệu: A=diag(2,3) giãn x gấp 2, y gấp 3; v=[0,1] trùng trục y.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np

# np.diag() đặt hai hệ số trên đường chéo, các ô khác bằng 0.
A = np.diag([2.0, 3.0])
# v là vector hướng y với hai thành phần x=0,y=1.
v = np.array([0.0, 1.0])
# @ nhân ma trận với vector: ra [0,3], giữ phương y và giãn gấp 3; đó là hướng riêng.
print(A @ v)
```

Kết quả khi chạy:

```
[0. 3.]
```

Đầu ra vẫn có hoành độ 0; phương trục tung được giữ, độ dài theo chiều đó nhân ba.

4.2. Eigenvalue — Hệ số biến đổi trên hướng riêng

Từ vựng trong mục: Eigenvalue — trị riêng; **Scale factor** — hệ số co giãn.

Trị riêng λ cho biết A phóng đại, thu nhỏ hoặc đảo chiều **vector riêng** bao nhiêu lần. Với `diag(2,3)`, hướng trục x có λ=2 và trục y có λ=3. Đó là tính chất của ma trận A; trị riêng **không** tự động mang nghĩa phương sai trừ khi A đang là **ma trận hiệp phương sai**. `np.linalg.eigvalsh` lấy các trị riêng của ma trận đối xứng theo thứ tự tăng dần.

No Note — Không chú thích

```
import numpy as np

A = np.diag([2.0, 3.0])
print(np.linalg.eigvalsh(A))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc các trị riêng của ma trận đường chéo đối xứng bằng eigvalsh().
# Dữ liệu: A là ma trận 2×2 đối xứng, hệ số giãn theo hai trục bằng 2 và 3.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np

# diag() tạo ma trận đường chéo thực đối xứng.
A = np.diag([2.0, 3.0])
# eigvalsh() trả TRỊ RIÊNG tăng dần của ma trận đối xứng, KHÔNG trả vector riêng; phù hợp vì A đối xứng.
print(np.linalg.eigvalsh(A))
```

Kết quả khi chạy:

```
[2. 3.]
```

Hai trị riêng là 2 và 3, tương ứng mức co giãn của hai trục tọa độ.

4.3. Covariance — Hai đặc trưng cùng thay đổi ra sao?

Từ vựng trong mục: Covariance — hiệp phương sai; **Feature** — đặc trưng.

Hiệp phương sai đo xu hướng hai đặc trưng tăng/giảm cùng nhau. Dữ liệu có ba **hàng mẫu** `(1,2)`, `(2,4)`, `(3,6)` và hai **cột đặc trưng** x, y, trong đó y=2x. `np.cov(X,rowvar=False)` lấy **cột là biến**, **hàng là quan sát**, trả ma trận 2×2. Các ô đường chéo là phương sai từng biến, hai ô còn lại là hiệp phương sai. Không nhầm covariance với tương quan chuẩn hóa từ −1 đến 1.

No Note — Không chú thích

```
import numpy as np

X = np.array([[1, 2], [2, 4], [3, 6]])
print(np.cov(X, rowvar=False))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính covariance của hai cột số liệu có quan hệ y=2x.
# Dữ liệu: Ba hàng quan sát, cột y gấp đôi x; covariance mô tả biến thiên cùng nhau.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np

# Mỗi hàng là một mẫu, mỗi cột là một biến x/y.
X = np.array([[1, 2], [2, 4], [3, 6]])
# np.cov() tính hiệp phương sai mẫu (mẫu số n−1); rowvar=False nghĩa biến theo CỘT, không theo hàng; kết quả ma trận 2×2.
print(np.cov(X, rowvar=False))
```

Kết quả khi chạy:

```
[[1. 2.]
 [2. 4.]]
```

Biến y có phương sai gấp bốn x và hiệp phương sai dương 2, vì y tăng theo x.

4.4. Principal direction — Hướng biến thiên chính trong PCA

Từ vựng trong mục: Principal direction — hướng chính; **PCA** — phân tích thành phần chính.

Với **ma trận hiệp phương sai đối xứng**, vector riêng của **trị riêng lớn nhất** biểu diễn hướng có phương sai lớn nhất. Đây là nguyên lý của PCA. Ví dụ hiệp phương sai `diag(2,3)` có phương sai theo x là 2 và theo y là 3, nên hướng chính là trục y. **Dấu của eigenvector không cố định**: `[0,1]` và `[0,-1]` đều mô tả cùng một trục. Lấy `abs` khi chỉ muốn minh họa độ lớn thành phần.

No Note — Không chú thích

```
import numpy as np

C = np.diag([2.0, 3.0])
values, vectors = np.linalg.eigh(C)
print(np.abs(vectors[:, -1]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn vector riêng ứng với trị riêng lớn nhất để hiểu hướng PCA.
# Dữ liệu: C=diag(2,3) là covariance giả định; chiều y biến thiên nhiều nhất.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np

# Ma trận đường chéo có phương sai lớn nhất nằm ở hướng y.
C = np.diag([2.0, 3.0])
# eigh() trả tuple (values,vectors) với values tăng dần; mỗi CỘT vectors là một vector riêng tương ứng.
values, vectors = np.linalg.eigh(C)
# [:, -1] lấy cột vector riêng CUỐI ứng với trị riêng lớn nhất.
# abs() bỏ khác biệt dấu +/- vì hai vector đối hướng biểu diễn cùng trục riêng.
print(np.abs(vectors[:, -1]))
```

Kết quả khi chạy:

```
[0. 1.]
```

Hướng có phương sai lớn nhất nằm trên trục y; đổi dấu vector riêng không đổi hướng trục.

5. Năm tình huống trị riêng — từ nhận biết trục đến tái tạo bằng PCA

Tình huống 1 — Cấp 1: Tìm chiều biến thiên trong 10 phép đo hai trục

Từ vựng trong mục: Covariance — hiệp phương sai; **Eigenvalue** — trị riêng; **Axis** — trục.

Tình huống thực tế: Kỹ thuật viên nhận **10 vị trí cảm biến** trên mặt phẳng, mỗi dòng gồm tọa độ **x và y (mm)**. Trong dữ liệu thử nghiệm, **x thay đổi từ 1 đến 10 mm**, còn **y luôn là 20 mm**. Người trực muốn biết dữ liệu biến thiên theo hướng nào, không nhầm số đo y bằng 20 với việc y có biến động lớn; đây là bài quan sát độ thay đổi chứ không phải tìm vị trí xa gốc nhất.

Vì sao chọn cách này? Hiệp phương sai (covariance) mô tả hai đại lượng **thay đổi riêng và cùng nhau** ra sao quanh mức trung bình; `np.cov(X, rowvar=False)` xem **mỗi cột** x/y là một đại lượng, mỗi hàng là một lượt đo. **Trị riêng (eigenvalue)** của bảng hiệp phương sai cho biết mức biến thiên trên các **hướng đặc biệt**. Vì mọi y đều bằng 20, y có **độ thay đổi bằng 0**, dù giá trị của nó lớn hơn x; một trị riêng sẽ bằng

0. `eigvalsh` dùng cho ma trận đối xứng và trả trị riêng mà không phải tính hướng, vì câu hỏi đầu chỉ cần mức biến thiên.

Quy trình:

1. Tạo 10 hàng × 2 cột và tính covariance với cột là đặc trưng.
2. Dùng `eigvalsh` đọc hai trị riêng đã sắp tăng.
3. Chọn trục có phương sai lớn nhất và diễn giải.

No Note — Không chú thích

```
import numpy as np
X = np.array([[1,20],[2,20],[3,20],[4,20],[5,20],[6,20],[7,20],[8,20],[9,20],[10,20]], dtype=float)
C = np.cov(X, rowvar=False)
values = np.linalg.eigvalsh(C)
if X.shape != (10, 2):
    raise ValueError("Cần 10 lượt đo với hai đại lượng.")
print("Lượt đo:", len(X))
print("Trị riêng nhỏ:", f"{values[0]:.2f}")
print("Trị riêng lớn:", f"{values[-1]:.2f}")
print("Y không thay đổi:", bool(np.all(X[:, 1] == 20)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm một cột không đổi khiến một trị riêng bằng 0 trong 10 mẫu.
# Dữ liệu: 10 hàng, x tăng 1..10 nhưng y luôn 20; y không mang thông tin biến thiên.
# NumPy (np) cung cấp mảng số´và các hàm đại số´tuyến tính/covariance.
import numpy as np
# 10 điểm có y=20 ở mọi hàng nên độ biến thiên y bằng 0.
X = np.array([[1,20],[2,20],[3,20],[4,20],[5,20],[6,20],[7,20],[8,20],[9,20],[10,20]], dtype=float)
# rowvar=False dùng các cột làm biến; covariance ước lượng theo mẫu, chia cho n−1.
C = np.cov(X, rowvar=False)
# eigvalsh trả hai trị riêng xếp tăng; trị riêng gần 0 ở trục không đổi.
values = np.linalg.eigvalsh(C)
if X.shape != (10, 2):
    # ValueError dừng nếu số´hàng/cột hoặc kết quả PCA không đúng yêu cầu.
    raise ValueError("Cần 10 lượt đo với hai đại lượng.")
print("Lượt đo:", len(X))
# [0] là trị nhỏ nhất, [-1] lớn nhất, :.2f định dạng hai chữ số´lẻ.
print("Trị riêng nhỏ:", f"{values[0]:.2f}")
print("Trị riêng lớn:", f"{values[-1]:.2f}")
# [:,1] lấy toàn bộ cột y; ==20 tạo mask bool; np.all() yêu cầu tất cả True; bool() chuyển về`
Boolean Python.
print("Y không thay đổi:", bool(np.all(X[:, 1] == 20)))
```

Kết quả khi chạy:

```
Lượt đo: 10
Trị riêng nhỏ: 0.00
Trị riêng lớn: 9.17
Y không thay đổi: True
```

Ở cấp này, phương sai y bằng 0 không có nghĩa tọa độ y bằng 0; cảm biến luôn ghi **20 mm**.

Tình huống 2 — Cấp 2: Nhận biết chiều dư thừa của 12 bản ghi phụ thuộc

Từ vựng trong mục: Linear dependence — phụ thuộc tuyến tính; **Eigenvalue** — trị riêng.

Tình huống thực tế: Một máy ghi **12 mẫu**, mỗi mẫu gồm chỉ số **x** và chỉ số thứ hai **luôn bằng hai lần x**. Đây là hai cột số giả lập không có đơn vị; nhìn bảng thấy có hai cột, nhưng cột thứ hai có thể dự đoán hoàn toàn từ cột thứ nhất. Người phân tích muốn biết có mấy **hướng biến thiên độc lập** trước khi dùng hai cột làm đầu vào cho mô hình.

Vì sao chọn cách này? Khi cột thứ hai **luôn gấp đôi** cột thứ nhất, chúng không phải hai nguồn biến thiên khác nhau. Ma trận hiệp phương sai được tính sau khi xét sự thay đổi quanh trung bình của 12 mẫu; nó sẽ có **một trị riêng lớn hơn 0** và **một trị riêng gần 0**. `np.linalg.eigvalsh()` phù hợp vì hiệp phương sai là ma trận đối xứng; dùng ngưỡng `1e-9` khi đếm trị riêng dương để tránh sai số máy tính được hiểu là thông tin mới. Bớt một cột ở đây không làm mất thông tin tuyến tính, nhưng kết luận ấy phụ thuộc đúng quan hệ trong dữ liệu mẫu.

Quy trình:

1. Ghi 12 cặp x/2x và tính hiệp phương sai.
2. Tính các trị riêng qua `eigvalsh` để tránh số phức do sai số thuật toán.
3. Đếm số trị riêng dương theo ngưỡng nhỏ.

No Note — Không chú thích

```
import numpy as np
X = np.array([[1,2],[2,4],[3,6],[4,8],[5,10],[6,12],[7,14],[8,16],[9,18],[10,20],[11,22],[12,24]],
dtype=float)
C = np.cov(X, rowvar=False)
eigenvalues = np.linalg.eigvalsh(C)
if X.shape != (12, 2):
    raise ValueError("Cần đúng 12 mẫu hai chiều.")
print("Mẫu:", len(X))
labels = []
for value in eigenvalues:
    labels.append(f"{value:.2f}")
print("Trị riêng:", ", ".join(labels))
print("Chiều có biến thiên:", int((eigenvalues > 1e-9).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm hướng biến thiên độc lập khi 12 điểm đều nằm trên đường thẳng.
# Dữ liệu: 12 điểm hai chiều y=2x; thực chất chỉ một hướng độc lập.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np
# 12 hàng (x,y) với y luôn gấp đôi x: hai cột không độc lập.
X = np.array([[1,2],[2,4],[3,6],[4,8],[5,10],[6,12],[7,14],[8,16],[9,18],[10,20],[11,22],[12,24]],
dtype=float)
# rowvar=False nghĩa mỗi cột x/y là một biến, các hàng là lượt đo.
C = np.cov(X, rowvar=False)
# Trị riêng từ ma trận đối xứng C xếp tăng, một hướng gần 0 vì dữ liệu trên đường.
eigenvalues = np.linalg.eigvalsh(C)
if X.shape != (12, 2):
    # ValueError dừng nếu số hàng/cột hoặc kết quả PCA không đúng yêu cầu.
    raise ValueError("Cần đúng 12 mẫu hai chiều.")
print("Mẫu:", len(X))
labels = []
for value in eigenvalues:
    # f-string :.2f hiển thị từng trị riêng với hai chữ số sau dấu thập phân.
    labels.append(f"{value:.2f}")
print("Trị riêng:", ", ".join(labels))
# Ngưỡng 1e-9 bỏ sai số float gần 0; > tạo mask, sum() đếm chiều biến thiên đáng kể.
print("Chiều có biến thiên:", int((eigenvalues > 1e-9).sum()))
```

Kết quả khi chạy:

```
Mẫu: 12
Trị riêng: 0.00, 65.00
Chiều có biến thiên: 1
```

Trị riêng 0 không nói rằng một cột toàn số 0, mà nói hai cột không độc lập với nhau.

Tình huống 3 — Cấp 3: Đối chiếu vector riêng trên 15 tọa độ dạng lưới

Từ vựng trong mục: Eigenvector — véc-tơ riêng; **Eigen-equation** — quan hệ riêng; **eigh** — tìm trị và véc-tơ riêng.

Tình huống thực tế: Robot khảo sát **15 vị trí trên sàn** theo hình lưới **5 cột × 3 hàng**, mỗi vị trí có tọa độ **(x, y) tính bằng mét**. Nhóm kỹ thuật muốn biết hai **hướng biến thiên chính** của các điểm và kiểm tra bằng phép tính rằng mỗi hướng được thuật toán xác định đúng, thay vì chỉ ước lượng bằng mắt trên hình vẽ.

Vì sao chọn cách này? Véc-tơ riêng (eigenvector) là một **hướng** mà khi nhân với ma trận, hướng ấy **không bị xoay sang hướng khác**, chỉ thay đổi độ lớn theo **trị riêng** tương ứng. `np.linalg.eigh(C)` cho cả các mức biến thiên và hai hướng riêng của ma trận hiệp phương sai **đối xứng**. Chương trình dùng `C @ vectors` so với `vectors * values`, rồi `np.allclose()` để kiểm sai khác số thực nhỏ do máy tính. Dấu của véc-tơ riêng có thể đảo (+ thành -) nhưng vẫn chỉ cùng một trục, nên không ép dấu theo hình vẽ. Đây là phép **kiểm chứng kết quả**, không phải robot phải di chuyển theo hướng ấy.

Quy trình:

1. Tạo 15 mẫu lưới và tính covariance 2×2.
2. Nhận trị riêng và cột véc-tơ riêng bằng `eigh`.
3. Kiểm chứng từng cột véc-tơ qua quan hệ riêng.

No Note — Không chú thích

```
import numpy as np
X = np.array([[0,0],[1,0],[2,0],[3,0],[4,0],[0,1],[1,1],[2,1],[3,1],[4,1],[0,2],[1,2],[2,2],[3,2],
[4,2]], dtype=float)
C = np.cov(X, rowvar=False)
values, vectors = np.linalg.eigh(C)
verified = np.allclose(C @ vectors, vectors * values)
if X.shape != (15, 2):
    raise ValueError("Cần đúng 15 điểm hai chiều.")
print("Điểm:", len(X))
labels = []
for value in values:
    labels.append(f"{value:.3f}")
print("Trị riêng:", ", ".join(labels))
print("Kiểm chứng:", bool(verified))
print("Số hướng:", vectors.shape[1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm phương trình C@v=λv cho các vector riêng của 15 điểm hai chiều.
# Dữ liệu: 15 điểm tạo lưới 5×3, mỗi hàng một điểm x,y.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np
# 15 hàng điểm trên lưới hai chiều, dtype=float phục vụ tính covariance.
```

```
X = np.array([[0,0],[1,0],[2,0],[3,0],[4,0],[0,1],[1,1],[2,1],[3,1],[4,1],[0,2],[1,2],[2,2],[3,2],
[4,2]], dtype=float)
# cov 2×2 dùng hai cột làm hai biến, chia n−1 theo mặc định.
C = np.cov(X, rowvar=False)
# eigh() trả trị riêng và vector riêng (vector được xếp theo CỘT).
values, vectors = np.linalg.eigh(C)
# @ nhân C đồng thời với mỗi cột vector riêng.
# Broadcast trị riêng tương ứng theo cột, tạo λv cho mỗi vector.
# allclose() kiểm C@v ≈ λv trong dung sai floating-point, không dùng == tuyệt đối.
verified = np.allclose(C @ vectors, vectors * values)
if X.shape != (15, 2):
    # ValueError dừng nếu sô´hàng/cột hoặc kết quả PCA không đúng yêu câù.
    raise ValueError("Cần đúng 15 điểm hai chiều.")
print("Điểm:", len(X))
labels = []
for value in values:
    # Định dạng :.3f in ba sô´thập phân.
    labels.append(f"{value:.3f}")
print("Trị riêng:", ", ".join(labels))
print("Kiểm chứng:", bool(verified))
# shape[1] đếm sô´hướng/ sô´cột vector riêng.
print("Sô´hướng:", vectors.shape[1])
```

Kết quả khi chạy:

```
Điểm: 15
Trị riêng: 0.714, 2.143
Kiểm chứng: True
Sô´hướng: 2
```

Chỉ khi nhân ma trận hiệp phương sai với **đúng véc-tơ riêng** mới thu được cùng phương đã nhân hệ số.

Tình huống 4 — Cấp 4: Xếp hai hướng biến thiên từ 18 điểm và tỷ lệ thông tin

Từ vựng trong mục: Variance ratio — tỷ lệ phương sai; **Principal direction** — hướng chính.

Tình huống thực tế: Đội khảo sát thu **18 vị trí** trên lưới **6 cột × 3 hàng**, mỗi vị trí có hai tọa độ x và y cùng đơn vị. Quan sát sơ bộ cho thấy các điểm trải dài theo chiều ngang nhiều hơn chiều dọc. Nhóm muốn xác nhận **hướng có biến thiên lớn hơn** và nó chiếm **bao nhiêu phần trăm tổng biến thiên** của cả hai hướng để chọn cách biểu diễn dữ liệu.

Vì sao chọn cách này? Để biết dữ liệu trải dài theo **hướng nào**, chương trình tính hiệp phương sai của 18 điểm rồi lấy **trị riêng lớn nhất**: hướng tương ứng có phương sai lớn nhất. `argsort(...)[::-1]` đưa trị riêng lớn nhất lên trước và dùng véc-tơ riêng của nó; so độ lớn hai thành phần để nhận biết hướng gần trục x hay y. **Tỷ lệ phương sai giải thích** cho biết phần biến thiên trên hướng chính so với **tổng biến thiên của hai hướng**, không phải phần trăm điểm đo đúng hay độ chính xác dự báo. Vì lưới rộng theo x, kết quả ưu tiên x là phù hợp với dữ liệu mô phỏng.

Quy trình:

1. Tính covariance của 18 điểm quan sát.
2. Sắp hai trị riêng giảm dần và lấy tỷ trọng trị lớn trong tổng.
3. Kiểm tra hướng chính bằng giá trị tuyệt đối của véc-tơ riêng, tránh phụ thuộc dấu.

No Note — Không chú thích

```
import numpy as np
points = np.array([[0,0],[1,0],[2,0],[3,0],[4,0],[5,0],[0,1],[1,1],[2,1],[3,1],[4,1],[5,1],[0,2],
```

```
[1,2],[2,2],[3,2],[4,2],[5,2]], dtype=float)
C = np.cov(points, rowvar=False)
values, vectors = np.linalg.eigh(C)
order = np.argsort(values)[::-1]
principal = np.abs(vectors[:, order[0]])
if points.shape != (18, 2):
    raise ValueError("Cần đúng 18 điểm hai chiều.")
print("Điểm:", len(points))
if principal[0] > principal[1]:
    principal_axis = "x"
else:
    principal_axis = "y"
print("Hướng chính:", principal_axis)
print(f"Tỷ lệ phương sai lớn nhất: {100*values[order[0]]/values.sum():.2f}%")
print("Số chiều:", len(values))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn hướng phương sai chính trên lưới 18 điểm và tính tỷ lệ giữ được.
# Dữ liệu: 18 hàng điểm tạo lưới 6×3; phương sai x lớn hơn y.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np
# 18 mẫu điểm x/y, dtype=float.
points = np.array([[0,0],[1,0],[2,0],[3,0],[4,0],[5,0],[0,1],[1,1],[2,1],[3,1],[4,1],[5,1],[0,2],
[1,2],[2,2],[3,2],[4,2],[5,2]], dtype=float)
# rowvar=False tính covariance giữa CỘT x,y của các hàng quan sát.
C = np.cov(points, rowvar=False)
# eigh trả trị riêng tăng dần và vector riêng ở các cột.
values, vectors = np.linalg.eigh(C)
# argsort trả CHỈ SỐ tăng dần theo trị riêng; [::-1] đảo thứ tự thành giảm dần.
order = np.argsort(values)[::-1]
# [:,order[0]] chọn cột vector riêng cho trị lớn nhất.
# abs() bỏ dấu vector riêng vốn có thể đảo chiều ±.
principal = np.abs(vectors[:, order[0]])
if points.shape != (18, 2):
    # ValueError dừng nếu số hàng/cột hoặc kết quả PCA không đúng yêu cầu.
    raise ValueError("Cần đúng 18 điểm hai chiều.")
print("Điểm:", len(points))
# So độ lớn x với y để chọn tên trục gần vector chính nhất.
if principal[0] > principal[1]:
    principal_axis = "x"
else:
    principal_axis = "y"
print("Hướng chính:", principal_axis)
# Chia trị riêng lớn nhất cho tổng trị riêng, nhân 100 để ra % phương sai giữ bởi một hướng.
print(f"Tỷ lệ phương sai lớn nhất: {100*values[order[0]]/values.sum():.2f}%")
# len() đếm hai chiều/trị riêng ban đầu.
print("Số chiều:", len(values))
```

Kết quả khi chạy:

```
Điểm: 18
Hướng chính: x
Tỷ lệ phương sai lớn nhất: 81.40%
Số chiều: 2
```

Trị riêng chỉ có ý nghĩa **phương sai** vì ma trận ở đây là covariance; với ma trận biến đổi khác cần giải thích theo ngữ cảnh khác.

Tình huống 5 — Cấp 5: PCA từ 20 hồ sơ có bốn đặc trưng nhưng chỉ hai chiều độc lập

Từ vựng trong mục: PCA — thành phần chính; **Rank** — hạng; **Reconstruction** — tái tạo.

Tình huống thực tế: Một bài thực hành AI tạo **20 hồ sơ mô phỏng**, mỗi hồ sơ ghi **4 chỉ số không đơn vị**: x, 2x, y, 3y. Hai chỉ số là bản phóng to của chỉ số khác, nên thực chất chỉ có **hai hướng thông tin độc lập**. Học viên muốn nén bảng 20 hàng × 4 cột thành **20 hàng × 2 tọa độ mới**, rồi thử dựng lại bốn cột để xem có làm mất thông tin tuyến tính đáng kể không.

Vì sao chọn cách này? PCA (phân tích thành phần chính) tìm những hướng mà dữ liệu thay đổi nhiều nhất, để mô tả bảng bằng **ít tọa độ hơn**. Trước tiên `centered` trừ **trung bình từng cột** để chuyển về biến thiên quanh tâm; sau đó lấy trị riêng/véc-tơ riêng của hiệp phương sai, sắp theo mức biến thiên giảm dần và giữ **hai hướng lớn nhất**. Phép `projected = centered @ basis` chuyển 20 hồ sơ thành hai tọa độ mới; nhân ngược và cộng lại trung bình để `recreated` thử dựng lại bốn cột. Vì 2x và 3y không đưa thông tin độc lập mới, hai hướng đủ để dựng lại gần chính xác trong bài này. Với dữ liệu thật có nhiễu và nhiều hướng, giảm còn hai chiều **có thể mất thông tin**; không tự kết luận PCA luôn giữ trọn dữ liệu.

Quy trình:

1. Căn giữa bảng 20×4, tính covariance và `eigh`.
2. Chọn hai eigenvector ứng với hai trị riêng lớn nhất rồi chiếu.
3. Tái tạo bảng, kiểm tra sai số và báo tỷ lệ phương sai của hướng đầu.

No Note — Không chú thích

```
import numpy as np
X = np.array([[0,0,0,0],[1,2,0,0],[2,4,0,0],[3,6,0,0],[4,8,0,0],[0,0,1,3],[1,2,1,3],[2,4,1,3],
[3,6,1,3],[4,8,1,3],[0,0,2,6],[1,2,2,6],[2,4,2,6],[3,6,2,6],[4,8,2,6],[0,0,3,9],[1,2,3,9],
[2,4,3,9],[3,6,3,9],[4,8,3,9]], dtype=float)
centered = X - X.mean(axis=0)
C = np.cov(X, rowvar=False)
values, vectors = np.linalg.eigh(C)
order = np.argsort(values)[::-1]
basis = vectors[:, order[:2]]
projected = centered @ basis
recreated = projected @ basis.T + X.mean(axis=0)
if X.shape != (20, 4) or projected.shape != (20, 2):
    raise ValueError("Cần 20 mẫu bốn đặc trưng và hai chiều sau PCA.")
print("Hồ sơ:", len(X))
print("Cột gốc:", X.shape[1])
print("Cột sau PCA:", projected.shape[1])
print("Trị riêng dương:", int((values > 1e-9).sum()))
print(f"Hướng đầu giữ: {100*values[order[0]]/values.sum():.2f}%")
print("Tái tạo đúng:", bool(np.allclose(recreated, X)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chiếu 20 hồ sơ bốn đặc trưng phụ thuộc tuyến tính xuống hai chiều rồi tái tạo.
# Dữ liệu: 20 hàng × 4 cột, cột 1=2×cột 0 và cột 3=3×cột 2; chỉ có hai hướng độc lập.
# NumPy (np) cung cấp mảng số và các hàm đại số tuyến tính/covariance.
import numpy as np
# 20 hồ sơ có 4 cột nhưng chỉ 2 cột mang biến thiên độc lập; dtype=float.
X = np.array([[0,0,0,0],[1,2,0,0],[2,4,0,0],[3,6,0,0],[4,8,0,0],[0,0,1,3],[1,2,1,3],[2,4,1,3],
[3,6,1,3],[4,8,1,3],[0,0,2,6],[1,2,2,6],[2,4,2,6],[3,6,2,6],[4,8,2,6],[0,0,3,9],[1,2,3,9],
[2,4,3,9],[3,6,3,9],[4,8,3,9]], dtype=float)
# mean(axis=0) tính trung bình TỪNG CỘT trên mọi hàng.
# Broadcast trừ vector bốn trung bình cho mỗi hàng, đưa từng biến về trung bình 0.
centered = X - X.mean(axis=0)
```

```
# rowvar=False tính ma trận covariance 4×4 theo CỘT (np.cov tự trừ trung bình và chia n−1).
C = np.cov(X, rowvar=False)
# eigh trả 4 trị riêng tăng dần và vector riêng ở các cột.
values, vectors = np.linalg.eigh(C)
# argsort trả chỉ số́ sắp tăng, [::-1] đảo để̉ hướng phương sai lớn lên trước.
order = np.argsort(values)[::-1]
# order[:2] chọn hai trị lớn nhất; vectors[:,...] lấý hai vector riêng làm basis 4×2.
basis = vectors[:, order[:2]]
# @ chiếú từng hàng bốn chiều lên hai hướng mới, trả bảng 20×2.
projected = centered @ basis
# mean(axis=0) tính trung bình TỪNG CỘT trên mọi hàng.
# @ với basis.T (2×4) khôi phục bốn tọa độ đã căn giữa.
# Cộng lại bốn trung bình từng cột để̉ quay về thang dữ liệu ban đầu.
recreated = projected @ basis.T + X.mean(axis=0)
if X.shape != (20, 4) or projected.shape != (20, 2):
    # ValueError dừng nêú số́ hàng/cột hoặc kết quả PCA không đúng yêu câù.
    raise ValueError("Cần 20 mẫu bốn đặc trưng và hai chiều sau PCA.")
print("Hồ sơ:", len(X))
print("Cột gốć:", X.shape[1])
print("Cột sau PCA:", projected.shape[1])
# Mask trị riêng dương đáng kể̉, dùng đếm số́ hướng biến thiên.
print("Trị riêng dương:", int((values > 1e-9).sum()))
# Tỷ lệ phương sai hướng đầu = trị riêng lớn nhất / tổng trị riêng ×100 (%).
print(f"Hướng đầu giữ: {100*values[order[0]]/values.sum():.2f}%")
# allclose đối chiếú tái tạo với dữ liệu gốc có dung sai; giữ hai hướng đủ vì dữ liệu chỉ có hai
hướng độc lập.
print("Tái tạo đúng:", bool(np.allclose(recreated, X)))
```

Kết quả khi chạy:

```
Hồ sơ: 20
Cột gốć: 4
Cột sau PCA: 2
Trị riêng dương: 2
Hướng đầu giữ: 55.56%
Tái tạo đúng: True
```

Véc-tơ riêng đổi dấu vẫn cho không gian PCA như nhau. Không nên so chính xác từng hệ số véc-tơ theo dấu nếu mục tiêu chỉ là **hướng**.

6. Sai lầm thường gặp

- Eigenvector là **hướng**, eigenvalue là **hệ số** nhân theo hướng ấy; không hoán đổi hai khái niệm.
- Chỉ diễn giải eigenvalue như phương sai khi A là ma trận hiệp phương sai.
- Với covariance, các hàng là mẫu và cột là đặc trưng khi `rowvar=False`; đảo trục làm thay đổi kết quả.
- Eigenvector và vector đối dấu biểu diễn cùng trục; không kiểm tra dấu từng phần tử cứng nhắc.
- `np.linalg.eigh` thích hợp cho ma trận đối xứng. PCA cần xem thang đo và đơn vị từng đặc trưng.
- scikit-learn là thư viện ngoài; cần cài trước khi chạy ví dụ PCA.

7. Trắc nghiệm

Chỉ kiểm tra kiến thức bài học; không kiểm tra ghi nhớ tình huống đánh số.

Q1. What defines an eigenvector v of A? | Điều kiện định nghĩa vector riêng v của A là gì?

A. `A @ v = λ*v` with v nonzero | `A @ v = λ*v` với v khác không

B. A covariance eigenvector with high eigenvalue | Vector riêng hiệp phương sai có trị riêng lớn

C. How two variables vary together | Hai biến cùng biến thiên ra sao

D. Scaling along a corresponding eigenvector | Hệ số co giãn theo vector riêng tương ứng

Đáp án: A

Giải thích: Vector khác không giữ phương đến hệ số nhân.

Q2. What does an eigenvalue represent? | Trị riêng biểu thị gì?

A. `A @ v = λ*v` with v nonzero | `A @ v = λ*v` với v khác không

B. Scaling along a corresponding eigenvector | Hệ số co giãn theo vector riêng tương ứng

C. How two variables vary together | Hai biến cùng biến thiên ra sao

D. A covariance eigenvector with high eigenvalue | Vector riêng hiệp phương sai có trị riêng lớn

Đáp án: B

Giải thích: Trị riêng là hệ số λ nhân vector riêng.

Q3. What does covariance describe? | Hiệp phương sai mô tả gì?

A. How two variables vary together | Hai biến cùng biến thiên ra sao

B. `A @ v = λ*v` with v nonzero | `A @ v = λ*v` với v khác không

C. Scaling along a corresponding eigenvector | Hệ số co giãn theo vector riêng tương ứng

D. A covariance eigenvector with high eigenvalue | Vector riêng hiệp phương sai có trị riêng lớn

Đáp án: A

Giải thích: Hiệp phương sai đo sự biến thiên cùng nhau.

Q4. What is a principal direction in PCA? | Hướng chính trong PCA là gì?

A. A covariance eigenvector with high eigenvalue | Vector riêng hiệp phương sai có trị riêng lớn

B. Scaling along a corresponding eigenvector | Hệ số co giãn theo vector riêng tương ứng

C. `A @ v = λ*v` with v nonzero | `A @ v = λ*v` với v khác không

D. How two variables vary together | Hai biến cùng biến thiên ra sao

Đáp án: A

Giải thích: Vector riêng hiệp phương sai nổi bật mang phương sai lớn nhất.

Q5. For A = [[3, 0], [0, 1]], what eigenvalue belongs to v = [1, 0]? | Với A = [[3, 0], [0, 1]], vector riêng v = [1, 0] ứng với trị riêng nào?

A. 1 | 1

B. 4 | 4

C. 0 | 0

D. 3 | 3

Đáp án: D

Giải thích: Nhân A với v được [3, 0] = 3v; do đó trị riêng tương ứng bằng 3.

Q6. A PCA model explains 70% variance with PC1 and 20% with PC2. How much is retained by both? | PCA giữ 70% phương sai ở PC1 và 20% ở PC2. Giữ cả hai thì được bao nhiêu?

A. 50% | 50%

B. 100% | 100%

C. 70% | 70%

D. 90% | 90%

Đáp án: D

Giải thích: Tỷ lệ phương sai giải thích được cộng khi chọn các thành phần chính: 70% + 20% = 90%.

Q7. In a 2-feature dataset, variance along x is 9 and along y is 1, with zero covariance. Which PCA direction comes first? | Dữ liệu hai đặc trưng có phương sai theo x là 9, theo y là 1, hiệp phương sai bằng 0. PCA chọn hướng đầu tiên nào?

A. The y-axis | Trục y

B. No direction exists | Không có hướng nào

C. The x-axis | Trục x

D. An equal mix of x and y | Pha đều x và y

Đáp án: C

Giải thích: Khi hiệp phương sai bằng 0, trục có phương sai lớn nhất (trục x, bằng 9) là hướng thành phần chính đầu tiên.

Q8. If the first principal component retains 80% of total variance, what percentage is not retained by that component? | Nếu thành phần chính đầu tiên giữ 80% tổng phương sai, bao nhiêu phần trăm không được giữ bởi riêng thành phần đó?

A. 20% | 20%

B. 10% | 10%

C. 0% | 0%

D. 80% | 80%

Đáp án: A

Giải thích: Phần còn lại là 100% − 80% = 20%, nằm ở các hướng trực giao khác.

Q9. What is the shape of a covariance matrix for three features? | Ma trận covariance cho ba đặc trưng có shape gì?

A. Columns of the second returned array | Các cột của mảng trả về thứ hai

B. v and −v describe the same eigendirection | v và −v biểu diễn cùng trục riêng

C. No variance along that direction | Không biến thiên theo hướng đó

D. `(3,3) | (3,3)`

Đáp án: D

Giải thích: Mỗi đặc trưng ghép với từng đặc trưng.

Q10. Which object contains eigenvectors returned by `eigh`? | Trong kết quả của `eigh`, vector riêng nằm đâu?

A. v and −v describe the same eigendirection | v và −v biểu diễn cùng trục riêng

B. `(3,3) | (3,3)`

C. No variance along that direction | Không biến thiên theo hướng đó

D. Columns of the second returned array | Các cột của mảng trả về thứ hai

Đáp án: D

Giải thích: eigh lưu mỗi vector riêng ở một cột.

Q11. Why can an eigenvector's sign change? | Vì sao dấu vector riêng có thể đảo?

A. v and −v describe the same eigendirection | v và −v biểu diễn cùng trục riêng

B. No variance along that direction | Không biến thiên theo hướng đó

C. `(3,3) | (3,3)`

D. Columns of the second returned array | Các cột của mảng trả về thứ hai

Đáp án: A

Giải thích: Đảo dấu vector riêng vẫn thỏa phương trình.

Q12. What does a zero covariance eigenvalue suggest? | Trị riêng covariance bằng 0 gợi ý gì?

A. Columns of the second returned array | Các cột của mảng trả về thứ hai

B. `(3,3) | (3,3)`

C. v and −v describe the same eigendirection | v và −v biểu diễn cùng trục riêng

D. No variance along that direction | Không biến thiên theo hướng đó

Đáp án: D

Giải thích: Trị riêng 0 nghĩa là không có phương sai theo trục đó.

Q13. How does PCA choose its first component? | PCA chọn thành phần đầu ra sao?

A. By largest explained variance | Theo phương sai giải thích lớn nhất

B. Variance of one feature | Phương sai của một đặc trưng

C. Two | Hai

D. `A @ v` approximately equals `λ*v` | `A @ v` xấp xỉ `λ*v`

Đáp án: A

Giải thích: Thành phần đầu tương ứng phương sai cao nhất.

Q14. What does a covariance diagonal entry represent? | Ô đường chéo covariance biểu thị gì?

A. `A @ v` approximately equals `λ*v` | `A @ v` xấp xỉ `λ*v`

B. By largest explained variance | Theo phương sai giải thích lớn nhất

C. Variance of one feature | Phương sai của một đặc trưng

D. Two | Hai

Đáp án: C

Giải thích: Hiệp phương sai của biến với chính nó là phương sai.

Q15. How many eigenvalues does a 2×2 symmetric matrix have counting multiplicity? | Ma trận đối xứng 2×2 có bao nhiêu trị riêng tính cả bội?

A. `A @ v` approximately equals `λ*v` | `A @ v` xấp xỉ `λ*v`

B. Two | Hai

C. By largest explained variance | Theo phương sai giải thích lớn nhất

D. Variance of one feature | Phương sai của một đặc trưng

Đáp án: B

Giải thích: Ma trận 2×2 có hai trị riêng tính bội.

Q16. Which relation validates an eigenpair? | Quan hệ nào dùng kiểm tra cặp riêng?

A. `A @ v` approximately equals `λ*v` | `A @ v` xấp xỉ `λ*v`

B. Variance of one feature | Phương sai của một đặc trưng

C. Two | Hai

D. By largest explained variance | Theo phương sai giải thích lớn nhất

Đáp án: A

Giải thích: Cặp riêng thỏa Av=λv theo sai số số học.

Q17. Which data layout corresponds to `rowvar=False`? | `rowvar=False` tương ứng bố trí nào?

A. Only when the matrix is an appropriate covariance matrix | Chỉ khi ma trận là covariance phù hợp

B. One principal-component feature per sample | Một đặc trưng thành phần chính cho mỗi mẫu

C. Rows are samples and columns are features | Hàng là mẫu và cột là đặc trưng

D. Feature scales and units | Thang đo và đơn vị các đặc trưng

Đáp án: C

Giải thích: Quan sát nằm theo hàng khi rowvar=False.

Q18. Does an eigenvalue always mean variance? | Trị riêng có luôn mang nghĩa phương sai không?

A. Feature scales and units | Thang đo và đơn vị các đặc trưng

B. Only when the matrix is an appropriate covariance matrix | Chỉ khi ma trận là covariance phù hợp

C. Rows are samples and columns are features | Hàng là mẫu và cột là đặc trưng

D. One principal-component feature per sample | Một đặc trưng thành phần chính cho mỗi mẫu

Đáp án: B

Giải thích: Diễn giải phương sai cần ma trận hiệp phương sai.

Q19. What does `PCA(n_components=1)` preserve? | `PCA(n_components=1)` giữ lại gì?

A. Feature scales and units | Thang đo và đơn vị các đặc trưng

B. Rows are samples and columns are features | Hàng là mẫu và cột là đặc trưng

C. One principal-component feature per sample | Một đặc trưng thành phần chính cho mỗi mẫu

D. Only when the matrix is an appropriate covariance matrix | Chỉ khi ma trận là covariance phù hợp

Đáp án: C

Giải thích: Dữ liệu sau chiếu còn một cột, vẫn giữ số hàng.

Q20. What should be considered before comparing PCA variances across features? | Cần lưu ý gì trước khi so phương sai PCA giữa đặc trưng?

A. Feature scales and units | Thang đo và đơn vị các đặc trưng

B. One principal-component feature per sample | Một đặc trưng thành phần chính cho mỗi mẫu

C. Rows are samples and columns are features | Hàng là mẫu và cột là đặc trưng

D. Only when the matrix is an appropriate covariance matrix | Chỉ khi ma trận là covariance phù hợp

Đáp án: A

Giải thích: Đặc trưng có thang lớn có thể lấn át PCA nếu chưa chuẩn hóa.

8. Nối sang bài kế tiếp

`Hệ tuyến tính → Eigenvalues/Vectors → Covariance → PCA`.

PY-28 — Hàm tuyến tính, bậc hai, đa thức và hàm mũ

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Linear function — hàm tuyến tính; **Quadratic** — hàm bậc hai.

`Hàm Python → Hàm toán theo dữ liệu → Loss → Gradient`

Học xong dùng để làm gì? Mô tả dự đoán bằng hàm f(x), nhận biết đầu vào, đầu ra và ảnh hưởng của tham số.

Kết nối kiến thức tiếp theo: PY-29 bổ sung hàm kích hoạt để biểu diễn quan hệ phi tuyến.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Linear function	hàm tuyến tính	Đường thẳng có tốc độ thay đổi hằng số
Quadratic	hàm bậc hai	Hàm chứa x^2, đồ thị là parabol khi hệ số bậc hai khác 0
Polynomial	hàm đa thức	Tổng các lũy thừa nguyên không âm của x
Exponential	hàm mũ	Biến nằm trong số mũ, thường mô tả tăng/giảm theo tỷ lệ

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`y=m*x+b`	Quan hệ tuyến tính, m là độ dốc, b là hệ số chặn
`y=a*x**2+b*x+c`	Quan hệ bậc hai, dùng khi đường cong
`np.polyval(coefficients,x)`	Đánh giá đa thức theo hệ số cao xuống thấp
`np.exp(x)/base**x`	Hàm mũ; cẩn thận overflow và miền thực

4. Khái niệm liên quan

Nội dung chính: đầu vào x được chuyển thành đầu ra y theo một quy tắc. Hàm bậc nhất tạo đường thẳng với độ dốc không đổi; hàm bậc hai và đa thức tạo đường cong; hàm mũ nhân theo tỷ lệ sau mỗi kỳ. Mỗi ví dụ nói rõ biến, hệ số và đơn vị.

4.1. Linear function — Mỗi bước tăng một mức cố định

Từ vựng trong mục: Linear function — hàm bậc nhất; **Slope/intercept** — hệ số góc/hệ số chặn.

Trong bài, **hàm tuyến tính dạng affine** `y=m*x+b` mô tả đầu ra thay đổi đều theo đầu vào. Dịch vụ thu 10 nghìn đồng ban đầu và 5 nghìn mỗi km: x là số km, m=5 nghìn/km, b=10 nghìn. Đi 3 km hết 25 nghìn đồng. Đồ thị là đường thẳng; m nói **mỗi km thêm bao nhiêu**, b nói **giá ở x=0**. Trong toán tuyến tính thuần túy, `mx+b` với b khác 0 gọi chính xác là hàm affine.

No Note — Không chú thích

```
def delivery_fee(km):
    return 5 * km + 10

print(delivery_fee(3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng hàm bậc nhất để tính phí giao hàng tăng đều theo quãng đường.
# Dữ liệu: km là số kilômét, phí có đơn vị minh họa nghìn đồng, phí nền 10 và thêm 5 mỗi km.
# def định nghĩa hàm nhận quãng đường km; return trả phí để nơi gọi sử dụng.
def delivery_fee(km):
    # 5 là phí mỗi km, +10 là phần cố định; mỗi km tăng thêm đúng 5 nên đây là quan hệ tuyến tính.
    return 5 * km + 10

# Gọi hàm với quãng đường 3 km, print() hiển thị 25 nghìn đồng.
print(delivery_fee(3))
```

Kết quả khi chạy:

```
25
```

Phí là 25 nghìn đồng, gồm phí gốc 10 và 3×5 nghìn theo khoảng cách.

4.2. Quadratic — Mức thay đổi không cố định

Từ vựng trong mục: Quadratic — bậc hai; **Coefficient** — hệ số.

Hàm bậc hai $y=ax^2+bx+c$ (a khác 0) có đồ thị **parabol**, khác đường thẳng vì tốc độ thay đổi không cố định. Ví dụ chi phí mẫu $2x^2 + 3x + 10$ với x là mức tải (đơn vị tải đã quy định), 2 có đơn vị chi phí/tải², 3 là chi phí/tải và 10 là chi phí cố định. Khi x=4, giá trị bằng 54. Không hiểu hệ số bình phương là chi phí thực nếu mô hình mới chỉ là giả định.

No Note — Không chú thích

```
def cost(load):
    return 2 * load ** 2 + 3 * load + 10

print(cost(4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa chi phí bậc hai tăng nhanh hơn khi tải ngày càng lớn.
# Dữ liệu: load là mức tải không âm theo đơn vị giả lập, cost là điểm chi phí giả lập.
# Định nghĩa hàm nhận mức tải, trả giá trị chi phí tương ứng.
```

```
def cost(load):
    # **2 bình phương mức tải tạo độ cong: tăng thêm 1 tải về sau tốn nhiều hơn lúc đầu.
    # Gồm chi phí bậc hai, thành phần tăng đều và chi phí nền 10.
    return 2 * load ** 2 + 3 * load + 10

# Gọi mức tải 4; print() cho chi phí của đúng đầu vào này.
print(cost(4))
```

Kết quả khi chạy:

```
54
```

Với tải 4, mô hình cho chi phí 54 đơn vị tiền; đây là kết quả theo hàm đã đặt.

4.3. Polynomial — Nhiều lũy thừa của một biến

Từ vựng trong mục: Polynomial — đa thức; **Degree** — bậc đa thức.

Đa thức cộng các hạng `a_n*x^n` theo lũy thừa nguyên không âm. Tuyến tính là bậc 1, bậc hai là bậc 2, và còn có đa thức bậc cao hơn. `np.polyval([1,0,2], x)` biểu diễn $x^2 + 0*x + 2$; **hệ số đặt từ bậc cao xuống bậc thấp**, rất dễ viết ngược. Với ba đầu vào 0,1,2, hàm cho kết quả 2,3,6.

No Note — Không chú thích

```
import numpy as np

x = np.array([0, 1, 2])
print(np.polyval([1, 0, 2], x))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng np.polyval() tính một đa thức bậc hai trên ba đầu vào cùng lúc.
# Dữ liệu: x là vector [0,1,2], hệ số [1,0,2] biểu diễn x² + 0*x +2 theo thứ tự bậc cao xuống thấp.
# NumPy hỗ trợ tính đa thức vector hóa trên nhiều x.
import numpy as np

# Mảng x chứa ba điểm cần tính chung.
x = np.array([0, 1, 2])
# polyval(hệ_số,x) tính đa thức; hệ số theo LŨY THỪA GIẢM DẦN (x², x, hằng), trả mảng kết quả cho từng x.
print(np.polyval([1, 0, 2], x))
```

Kết quả khi chạy:

```
[2 3 6]
```

Hệ số `[1,0,2]` nghĩa là x^2+2, không phải $1+2x^2$.

4.4. Exponential — Nhân tỷ lệ lặp qua mỗi kỳ

Từ vựng trong mục: Exponential — hàm mũ; **Growth rate** — tỷ lệ tăng.

Hàm mũ có biến ở **số mũ**, ví dụ `y=y0*(1+r)^t`. Với một chỉ số ban đầu 1.000 đơn vị và tỷ lệ tăng 5% mỗi kỳ, sau 3 kỳ là $1000 \times 1.05^3 = 1157.625$. Làm tròn hai chữ số được 1157.63. Hàm mũ khác tuyến tính vì mỗi

kỳ tăng 5% của **giá trị mới**, không phải luôn tăng 50 đơn vị. Mô hình này chỉ mô phỏng một giả định tăng trưởng đều.

No Note — Không chú thích

```
initial = 1000
rate = 0.05
periods = 3
print(round(initial * (1 + rate) ** periods, 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa tăng trưởng lũy kế khi mỗi kỳ cộng cùng một tỷ lệ trên tổng mới.
# Dữ liệu: Giá trị đầu 1000 đơn vị, rate=0.05 là 5%/kỳ, periods=3 kỳ.
# Số dư ban đầu trước khi nhân theo từng kỳ.
initial = 1000
# 0.05 tương ứng 5% mỗi kỳ, không phải tăng 5 đơn vị.
rate = 0.05
# Ba lần áp dụng tỷ lệ tăng trên giá trị vừa có được.
periods = 3
# 1+rate là hệ số tăng 1.05 mỗi kỳ; **3 nhân lặp hệ số ba lần (lãi kép).
# round(...,2) làm tròn kết quả HIỂN THỊ hai chữ số thập phân, không ảnh hưởng đầu vào.
print(round(initial * (1 + rate) ** periods, 2))
```

Kết quả khi chạy:

```
1157.63
```

Sau ba kỳ tăng trưởng lũy kế, chỉ số khoảng 1157,63 đơn vị, không phải giá trị dự báo chắc chắn.

5. Năm tình huống hàm số — từ mô hình tuyến tính đến so khớp đường cong

Tình huống 1 — Cấp 1: Tính phí giao hàng cho 10 chuyến bằng hàm bậc nhất

Từ vựng trong mục: Affine function — hàm bậc nhất có hệ số chặn; **Slope** — độ dốc.

Tình huống thực tế: Đơn vị giao hàng có **10 chuyến**, mỗi chuyến có quãng đường lần lượt từ **1 đến 10 km**. Theo bảng phí **giả lập**, mỗi chuyến thu **10 nghìn đồng phí khởi đầu** và **5 nghìn đồng cho mỗi km**, không có phụ phí khác. Thu ngân muốn biết phí chuyến dài 1 km và tổng tiền của cả 10 chuyến mà không phải viết riêng một phép tính cho từng chuyến.

Vì sao chọn cách này? Hàm số mô tả một quy tắc biến đầu vào thành đầu ra: `delivery_fee(km)` nhận **số km** rồi trả **số tiền nghìn đồng**. Quy tắc này **tuyến tính**, vì mỗi thêm 1 km làm phí tăng đúng 5 nghìn đồng, còn khoản 10 nghìn là phí nền ngay cả trước khi tính quãng đường. Dùng `def` viết quy tắc **một lần**, vòng `for` gọi lại cho mười quãng đường rồi cộng kết quả. Chuyến 1 km phải gồm cả phí nền và phí theo km; nếu chỉ nhân số km với 5 thì sẽ bỏ quên phí cố định.

Quy trình:

1. Tạo list 10 số km và hàm phí đơn giản.
2. Áp dụng hàm cho từng chuyến, giữ đúng đơn vị.
3. Báo số chuyến, phí đầu và tổng cộng.

No Note — Không chú thích

```
distances_km = [1,2,3,4,5,6,7,8,9,10]
def delivery_fee(km):
    return 10 + 5 * km
fees = []
for km in distances_km:
    fees.append(delivery_fee(km))
if len(fees) != 10:
    raise ValueError("Cần tính phí đủ 10 chuyến.")
print("Chuyến:", len(fees))
print("Phí chuyến đầu (nghìn đồng):", fees[0])
print("Tổng phí (nghìn đồng):", sum(fees))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính giá giao hàng cho 10 chuyến theo km bằng hàm phí tuyến tính.
# Dữ liệu: 10 chuyến có quãng đường 1..10 km, giá nền 10 và giá theo km 5 nghìn đồng.
# List khoảng cách 10 chuyến, mỗi số là km của một đơn.
distances_km = [1,2,3,4,5,6,7,8,9,10]
# Hàm trả phí nghìn đồng cho quãng đường km; không tự in kết quả.
def delivery_fee(km):
    # Tiền nền 10 nghìn đồng và 5 nghìn đồng cho mỗi km.
    return 10 + 5 * km
# List kết quả phí mỗi chuyến, theo cùng thứ tự nguồn.
fees = []
# Duyệt 10 khoảng cách, gọi một lần mỗi chuyến.
for km in distances_km:
    # append() nhận giá trị return của hàm và lưu vào list.
    fees.append(delivery_fee(km))
# len() kiểm đã tính đủ 10 lượt, ValueError báo thiếu chuyến.
if len(fees) != 10:
    # ValueError báo số lượng hoặc shape không đủ yêu cầu, tránh dùng kết quả chưa kiểm chứng.
    raise ValueError("Cần tính phí đủ 10 chuyến.")
print("Chuyến:", len(fees))
# [0] là tiền chuyến đầu tiên, không phải tổng mọi chuyến.
print("Phí chuyến đầu (nghìn đồng):", fees[0])
# sum() cộng phí của tất cả 10 chuyến, đơn vị nghìn đồng.
print("Tổng phí (nghìn đồng):", sum(fees))
```

Kết quả khi chạy:

```
Chuyến: 10
Phí chuyến đầu (nghìn đồng): 15
Tổng phí (nghìn đồng): 375
```

Hệ số 5 có đơn vị **nghìn đồng/km**, không phải 5 km; chính sách này chỉ là dữ liệu minh họa.

Tình huống 2 — Cấp 2: Nhận biết chi phí bậc hai từ 12 mức tải

Từ vựng trong mục: Quadratic — hàm bậc hai; **Marginal increase** — mức tăng giữa hai bước.

Tình huống thực tế: Xưởng thử máy có **12 mức tải** được đánh số từ 1 đến 12; mỗi mức tạo một chi phí **nghìn đồng** theo quy tắc minh họa. Khi tải tăng cao, chi phí tăng nhanh hơn chứ không tăng đều cùng một số tiền. Người vận hành muốn so phần chi phí tăng **từ tải 1 lên 2** với **từ tải 11 lên 12** để hiểu vì sao không thể dùng một mức tăng cố định cho mọi mức tải.

Vì sao chọn cách này? Hàm bậc hai có phần phụ thuộc vào **bình phương mức tải**, nên khi tải càng cao thì cùng một bước tăng tải có thể làm chi phí tăng **nhiều hơn trước**. Hàm `expense(load)` trong bài còn có

phần tăng đều theo tải và phí nền; ba phần cùng tạo ra giá dự kiến. Vòng lặp tính đủ 12 mức, rồi chương trình lấy **chi phí tải 2 trừ tải 1** và **chi phí tải 12 trừ tải 11** để so hai lần tăng. Đây là mô hình giả lập để hiểu hình dạng hàm, không phải chi phí tiêu thụ của một máy thật.

Quy trình:

1. Tạo 12 mức tải và áp dụng hàm chi phí.
2. Lấy hiệu chi phí giữa các mức tải kề nhau.
3. In chi phí ở hai đầu và hai mức tăng để giải thích độ cong.

No Note — Không chú thích

```
loads = [1,2,3,4,5,6,7,8,9,10,11,12]
def expense(load):
    return 2 * load ** 2 + 3 * load + 10
costs = []
for load in loads:
    costs.append(expense(load))
if len(costs) != 12:
    raise ValueError("Cần tính chi phí đủ 12 mức tải.")
print("Mức tải:", len(loads))
print("Tải 1:", costs[0])
print("Tải 12:", costs[-1])
print("Tăng 1→2:", costs[1] - costs[0])
print("Tăng 11→12:", costs[-1] - costs[-2])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cho thấy hàm bậc hai làm chi phí tăng theo mức tải nhanh dần trên 12 mức.
# Dữ liệu: 12 mức tải nguyên 1..12; output là chi phí số giả lập, không phải hóa đơn thực tế.
# List 12 mức tải ngày càng tăng để so mức tăng chi phí.
loads = [1,2,3,4,5,6,7,8,9,10,11,12]
# Hàm tự định nghĩa nhận một mức tải và trả chi phí tại mức đó.
def expense(load):
    # Bình phương load làm phần chi phí tăng theo độ lớn của tải, khác hàm bậc nhất tăng cố định.
    # Gồm phần bậc hai, phần tuyến tính và mức nền 10.
    return 2 * load ** 2 + 3 * load + 10
costs = []
for load in loads:
    # append() giữ chi phí từng tải theo thứ tự loads.
    costs.append(expense(load))
if len(costs) != 12:
    # ValueError báo số lượng hoặc shape không đủ yêu cầu, tránh dùng kết quả chưa kiểm chứng.
    raise ValueError("Cần tính chi phí đủ 12 mức tải.")
print("Mức tải:", len(loads))
# [0] phí mức tải 1; [-1] phí tải 12 (chỉ số âm lấy phần tử cuối).
print("Tải 1:", costs[0])
# [-1] là phí cuối, [-2] phí áp chót; hiệu hai ô cho mức tăng tải 11→12.
print("Tải 12:", costs[-1])
# [0] phí mức tải 1; [-1] phí tải 12 (chỉ số âm lấy phần tử cuối).
# Chênh lệch tải 1→2 để đối chiếu với mức 11→12: không cố định vì hàm bậc hai.
print("Tăng 1→2:", costs[1] - costs[0])
# [-1] là phí cuối, [-2] phí áp chót; hiệu hai ô cho mức tăng tải 11→12.
print("Tăng 11→12:", costs[-1] - costs[-2])
```

Kết quả khi chạy:

```
Mức tải: 12
Tải 1: 15
Tải 12: 334
```

```
Tăng 1→2: 9
Tăng 11→12: 49
```

Chi phí trong bài là số **giả lập**, không phải quy luật tiêu hao được đo của một máy thực tế.

Tình huống 3 — Cấp 3: Đánh giá đa thức trên 15 vị trí cảm biến

Từ vựng trong mục: Polynomial — đa thức; **polyval** — đánh giá đa thức; **Coefficient order** — thứ tự hệ số.

Tình huống thực tế: Thiết bị hiệu chỉnh thử **15 vị trí từ -7 đến 7**, mỗi vị trí được gán chỉ số đầu ra **không đơn vị** theo một quy tắc **đa thức bậc hai** đã cho. Kỹ thuật viên cần tạo bảng kết quả của đủ 15 vị trí, tra giá trị tại **x = 0** và tìm **vị trí có đầu ra nhỏ nhất** để đối chiếu với đường cong dự kiến. Các vị trí âm chỉ biểu thị phía bên kia gốc, không phải dữ liệu lỗi.

Vì sao chọn cách này? Đa thức là cách ghép các thành phần như một số cố định, phần tăng theo x và phần tăng theo bình phương x. `np.polyval([1, 2, 3], positions)` dùng **hệ số theo thứ tự từ bậc cao xuống thấp**, tính cùng quy tắc cho cả 15 vị trí; nếu đảo thứ tự hệ số thì sẽ thành đường cong khác. `np.argmin(output)` tìm **vị trí trong mảng** có đầu ra nhỏ nhất, không trả luôn giá trị x; chương trình dùng vị trí ấy để lấy cả **giá trị nhỏ nhất** và **tọa độ của nó**. Cần phân biệt vị trí thứ mấy trong danh sách với tọa độ x có thể âm.

Quy trình:

1. Tạo 15 giá trị x và ma trận hệ số `[1,2,3]`.
2. Áp `np.polyval` lên cả vector vị trí.
3. Tra đầu ra tại x=0, xác định vị trí cho giá trị nhỏ nhất.

No Note — Không chú thích

```
import numpy as np
positions = np.array([-7,-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6,7], dtype=float)
output = np.polyval([1, 2, 3], positions)
minimum = int(np.argmin(output))
if len(output) != 15:
    raise ValueError("Cần đúng 15 vị trí tính hàm.")
print("Vị trí:", len(positions))
print("Tại x=0:", int(output[positions == 0][0]))
print("Đầu ra nhỏ nhất:", int(output[minimum]))
print("Vị trí nhỏ nhất:", int(positions[minimum]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Khảo sát 15 vị trí x từ -7 đến 7 để tìm giá trị nhỏ nhất của đa thức.
# Dữ liệu: x có đơn vị vị trí giả lập, output của đa thức x²+2x+3 là thang điểm chi phí giả lập.
# NumPy polyVal và argmin cho phép xử lý một vector vị trí.
import numpy as np
# 15 x từ -7..7, dtype=float tạo mảng số thực.
positions = np.array([-7,-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6,7], dtype=float)
# polyval() tính x²+2x+3 theo hệ số bậc cao→thấp tại từng phần tử positions.
output = np.polyval([1, 2, 3], positions)
# argmin() trả CHỈ SỐ 0-based của giá trị nhỏ nhất, không trả chính giá trị; int() đổi scalar sang
số nguyên.
minimum = int(np.argmin(output))
if len(output) != 15:
    # ValueError báo số lượng hoặc shape không đủ yêu cầu, tránh dùng kết quả chưa kiểm chứng.
    raise ValueError("Cần đúng 15 vị trí tính hàm.")
```

```
print("Vị trí:", len(positions))
# positions==0 tạo mask; output[mask] lọc giá trị tại x=0; [0] chọn phần tử đầu trong kết quả lọc.
print("Tại x=0:", int(output[positions == 0][0]))
# Trả giá trị đầu ra ở chỉ số minimum tìm thấy.
print("Đầu ra nhỏ nhất:", int(output[minimum]))
# Dùng cùng chỉ số minimum truy ngược vị trí x ứng với giá trị nhỏ nhất.
print("Vị trí nhỏ nhất:", int(positions[minimum]))
```

Kết quả khi chạy:

```
Vị trí: 15
Tại x=0: 3
Đầu ra nhỏ nhất: 2
Vị trí nhỏ nhất: -1
```

Giá trị nhỏ nhất xảy ra tại x=-1, không phải nhất thiết tại x=0; thứ tự hệ số ảnh hưởng trực tiếp hình dạng đường cong.

Tình huống 4 — Cấp 4: Mô phỏng 18 kỳ tăng trưởng lũy kế bằng hàm mũ

Từ vựng trong mục: Exponential — hàm mũ; **Compounding** — tăng lũy kế; **Growth rate** — tỷ lệ tăng.

Tình huống thực tế: Bộ phận kinh doanh so **hai giả định tăng trưởng** của một chỉ số bắt đầu ở **1.000 điểm** vào kỳ 0. Với giả định A, chỉ số tăng **3% trên giá trị đang có sau mỗi kỳ**; với B, mỗi kỳ chỉ cộng **30 điểm cố định**. Báo cáo gồm **18 kỳ, từ 0 đến 17**. Quản lý muốn xem hai chỉ số ở kỳ cuối và khoản chênh lệch, vì dù kỳ đầu tăng như nhau, về sau hai quy tắc có thể cho kết quả khác.

Vì sao chọn cách này? Tăng trưởng lũy kế có nghĩa mức tăng kỳ sau được tính từ **giá trị đã tăng của kỳ trước**; ví dụ 3% trên 1.030 điểm sẽ lớn hơn 3% trên 1.000 điểm. Đó là cách hoạt động của **hàm mũ** trong `compound`. Còn `linear` chỉ cộng 30 điểm mỗi kỳ, nên có **mức tăng tuyệt đối không đổi**. `np.arange(18)` tạo đúng các kỳ **0–17**, rồi cả hai quy tắc dùng cùng danh sách mốc để so công bằng. Nếu tưởng tăng 3% là cộng thêm 30 điểm trong **mọi kỳ**, người học sẽ nhầm tăng theo tỷ lệ với tăng cố định.

Quy trình:

1. Tạo vector 18 kỳ từ 0 tới 17.
2. Tính chuỗi tăng lũy kế bằng phép lũy thừa và chuỗi tuyến tính tăng 30 điểm/kỳ.
3. In kỳ cuối của hai mô hình và chênh lệch.

No Note — Không chú thích

```
import numpy as np
periods = np.arange(18)
compound = 1000 * (1.03 ** periods)
linear = 1000 + 30 * periods
if compound.size != 18 or linear.size != 18:
    raise ValueError("Cần đủ 18 kỳ của cả hai cách tính.")
print("Kỳ:", len(periods))
print(f"Lũy kế cuối: {compound[-1]:.2f}")
print(f"Tuyến tính cuối: {linear[-1]:.2f}")
print(f"Chênh lệch: {compound[-1]-linear[-1]:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So giá trị tích lũy tuyến tính và tăng lũy kế 3% qua 18 kỳ.
# Dữ liệu: 18 kỳ được đánh số từ 0 đến 17, ban đầu 1000 đơn vị, lũy kế tăng 3%/kỳ.
```

```
# NumPy arange tạo chuỗi kỳ và lũy thừa theo từng phần tử.
import numpy as np
# arange(18) sinh vector 18 số 0..17, không chứa 18; kỳ 0 là thời điểm ban đầu.
periods = np.arange(18)
# Broadcast phép lũy thừa: mỗi kỳ t dùng hệ số 1.03^t, lãi kép trên số dư mới.
# Nhân giá trị gốc 1000 để có lũy kế từng kỳ.
compound = 1000 * (1.03 ** periods)
# Tăng tuyến tính đúng 30 đơn vị mỗi kỳ, KHÔNG tái đầu tư lãi tăng.
linear = 1000 + 30 * periods
# size đếm đủ 18 phần tử của mỗi vector, không phải giá trị cuối.
if compound.size != 18 or linear.size != 18:
    # ValueError báo số lượng hoặc shape không đủ yêu cầu, tránh dùng kết quả chưa kiểm chứng.
    raise ValueError("Cần đủ 18 kỳ của cả hai cách tính.")
print("Kỳ:", len(periods))
# [-1] là kỳ cuối số 17; :.2f làm tròn hiển thị hai chữ số thập phân.
print(f"Lũy kế cuối: {compound[-1]:.2f}")
# Đọc mức tuyến tính tại cùng kỳ cuối để so công bằng.
print(f"Tuyến tính cuối: {linear[-1]:.2f}")
# [-1] là kỳ cuối số 17; :.2f làm tròn hiển thị hai chữ số thập phân.
# Đọc mức tuyến tính tại cùng kỳ cuối để so công bằng.
# Chênh lệch ở kỳ cuối biểu thị ảnh hưởng của cộng dồn tỷ lệ so với tăng cố định.
print(f"Chênh lệch: {compound[-1]-linear[-1]:.2f}")
```

Kết quả khi chạy:

```
Kỳ: 18
Lũy kế cuối: 1652.85
Tuyến tính cuối: 1510.00
Chênh lệch: 142.85
```

Không suy ra giá trị tăng trưởng chắc chắn từ mô hình giả lập; nếu giả định 3% mỗi kỳ sai thì đường cong không còn phù hợp.

Tình huống 5 — Cấp 5: Đánh giá hai mô hình trên 20 quan sát cong

Từ vựng trong mục: Model fit — độ khớp mô hình; **MSE** — sai số bình phương; **Extrapolation** — ngoại suy.

Tình huống thực tế: Nhóm phân tích dựng **20 mức tải từ 0 đến 19**, mỗi mức có một chi phí mô phỏng **nghìn đồng** được tạo theo đường cong bậc hai. Họ thử hai cách dự báo trên **chính 20 điểm này**: mô hình **tuyến tính** chỉ tăng theo một mức cố định và mô hình **bậc hai** có thể uốn cong theo tải. Nhóm muốn tính mức sai lệch bình phương trung bình của từng mô hình và xem mô hình nào khớp **bộ dữ liệu minh họa** hơn, chưa khẳng định dự báo dữ liệu mới sẽ tốt hơn.

Vì sao chọn cách này? Mô hình là một quy tắc tạo giá trị dự báo từ mức tải. **MSE** (sai số bình phương trung bình) đo trung bình các sai lệch đã bình phương giữa **giá trị quan sát và dự báo**; sai lệch lớn bị tính nặng hơn. Với bộ dữ liệu này, chi phí được tạo **đúng từ một quy tắc bậc hai**, nên mô hình bậc hai khớp gần như hoàn toàn, trong khi đường thẳng bỏ sót phần cong. `np.polyval()` dùng các hệ số bậc hai, còn `np.mean()` tổng hợp các sai lệch bình phương trên đủ 20 dòng. Kết quả chỉ chứng minh độ khớp **trên dữ liệu đã dựng**, chưa cho biết mô hình có khái quát tốt trên dữ liệu mới hay không.

Quy trình:

1. Tạo 20 đầu vào và 20 chi phí quan sát có đơn vị.
2. Tính dự báo bằng hai hàm khác nhau.
3. Đối chiếu MSE, chọn mô hình có sai số nhỏ nhất và báo quy mô dữ liệu.

No Note — Không chú thích

```
import numpy as np
loads = np.arange(20, dtype=float)
observed = 2 * loads**2 + 3 * loads + 10
linear = 3 * loads + 10
quadratic = np.polyval([2, 3, 10], loads)
mse_linear = np.mean((observed - linear) ** 2)
mse_quadratic = np.mean((observed - quadratic) ** 2)
if observed.shape != (20,) or linear.shape != (20,) or quadratic.shape != (20,):
    raise ValueError("Cần 20 quan sát để đối chiếu hai mô hình.")
print("Quan sát:", len(loads))
print(f"MSE tuyến tính: {mse_linear:.2f}")
print(f"MSE bậc hai: {mse_quadratic:.2f}")
if mse_quadratic < mse_linear:
    best_model = "bậc hai"
else:
    best_model = "tuyến tính"
print("Khớp hơn:", best_model)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So mô hình tuyến tính và bậc hai khi dữ liệu thật được sinh theo quan hệ bậc hai.
# Dữ liệu: 20 mức tải từ 0..19; observed có bậc hai, linear chỉ có thành phần bậc nhất, quadratic
mô phỏng đúng.
# NumPy hỗ trợ arange(), polyval(), mean() để so mô hình.
import numpy as np
# Tạo 20 đầu vào 0..19 ở dạng float, mỗi giá trị một lượt tải.
loads = np.arange(20, dtype=float)
# Giá trị observed tuân đa thức bậc hai; **2 lấy bình phương từng phần tử.
# Mô hình linear bỏ qua ảnh hưởng bình phương nên sai nhiều ở tải lớn.
observed = 2 * loads**2 + 3 * loads + 10
# Mô hình linear bỏ qua ảnh hưởng bình phương nên sai nhiều ở tải lớn.
linear = 3 * loads + 10
# polyval áp bộ hệ số 2*x²+3*x+10 đúng thứ tự bậc cao→thấp.
quadratic = np.polyval([2, 3, 10], loads)
# Trừ từng dự đoán khỏi quan sát, bình phương để phạt lệch lớn, giữ đơn vị đầu ra bình phương.
# np.mean() lấy trung bình sai số bình phương của 20 lượt, chính là MSE.
mse_linear = np.mean((observed - linear) ** 2)
# MSE của mô hình đa thức so với observed, gần 0 vì công thức trùng nhau.
mse_quadratic = np.mean((observed - quadratic) ** 2)
# shape (20,) kiểm mỗi vector đầu ra có đúng 20 giá trị, không mất lượt.
if observed.shape != (20,) or linear.shape != (20,) or quadratic.shape != (20,):
    # ValueError báo số lượng hoặc shape không đủ yêu cầu, tránh dùng kết quả chưa kiểm chứng.
    raise ValueError("Cần 20 quan sát để đối chiếu hai mô hình.")
print("Quan sát:", len(loads))
print(f"MSE tuyến tính: {mse_linear:.2f}")
print(f"MSE bậc hai: {mse_quadratic:.2f}")
# So MSE nhỏ hơn để chọn mô hình khớp dữ liệu đã giả lập tốt hơn; không có nghĩa luôn tốt nhất khi
gặp dữ liệu mới.
if mse_quadratic < mse_linear:
    best_model = "bậc hai"
else:
    best_model = "tuyến tính"
print("Khớp hơn:", best_model)
```

Kết quả khi chạy:

```
Quan sát: 20
MSE tuyến tính: 112533.20
```

```
MSE bậc hai: 0.00
Khớp hơn: bậc hai
```

MSE ở đây có đơn vị **(nghìn đồng)²**, khác đơn vị chi phí. Mô hình B khớp tuyệt đối vì dữ liệu được **tạo ra bằng chính dạng bậc hai**.

6. Sai lầm thường gặp

- Hàm affine `mx+b` có hệ số chặn; không phải mọi biểu thức có chữ x đều là hàm tuyến tính theo nghĩa toán học chặt chẽ.
- Không nhầm bậc hai với hàm mũ: bậc hai có x ở cơ số rồi bình phương, hàm mũ có x ở số mũ.
- `np.polyval` đọc hệ số từ **bậc cao xuống thấp**.
- Độ dốc và hệ số chặn cần đơn vị tương thích với x và y.
- Tăng trưởng mũ giả định tỷ lệ ổn định, không bảo đảm dự báo đúng trong thực tế.
- Đồ thị đẹp hoặc khớp vài điểm chưa chứng minh mô hình dự báo tốt trên dữ liệu mới.

7. Trắc nghiệm

Câu hỏi chỉ về nội dung lý thuyết, cú pháp và cách phân biệt hàm.

Q1. Which form is a first-degree affine function? | Dạng nào là hàm affine bậc nhất?

A. `y=a*x**2+b*x+c` with a≠0 | `y=a*x**2+b*x+c` với a≠0

B. A variable appears in the exponent | Biến xuất hiện ở số mũ

C. `y=m*x+b` | `y=m*x+b`

D. Highest nonzero power of x | Lũy thừa cao nhất có hệ số khác 0

Đáp án: C

Giải thích: Dạng affine gồm hệ số góc và hệ số chặn.

Q2. Which form contains a nonzero quadratic term? | Dạng nào có hạng bậc hai khác 0?

A. `y=m*x+b` | `y=m*x+b`

B. A variable appears in the exponent | Biến xuất hiện ở số mũ

C. Highest nonzero power of x | Lũy thừa cao nhất có hệ số khác 0

D. `y=a*x**2+b*x+c` with a≠0 | `y=a*x**2+b*x+c` với a≠0

Đáp án: D

Giải thích: Hàm bậc hai có hạng x bình phương.

Q3. What does polynomial degree measure? | Bậc đa thức cho biết gì?

A. A variable appears in the exponent | Biến xuất hiện ở số mũ

B. `y=m*x+b` | `y=m*x+b`

C. `y=a*x**2+b*x+c` with a≠0 | `y=a*x**2+b*x+c` với a≠0

D. Highest nonzero power of x | Lũy thừa cao nhất có hệ số khác 0

Đáp án: D

Giải thích: Bậc là số mũ cao nhất có hệ số khác 0.

Q4. What characterizes an exponential function? | Đặc điểm của hàm mũ là gì?

A. `y=a*x**2+b*x+c` with $a \neq 0$ | `y=a*x**2+b*x+c` với $a \neq 0$

B. Highest nonzero power of x | Lũy thừa cao nhất có hệ số khác 0

C. `y=m*x+b` | `y=m*x+b`

D. A variable appears in the exponent | Biến xuất hiện ở số mũ

Đáp án: D

Giải thích: Hàm mũ có số mũ thay đổi.

Q5. Given f(x) = 2x + 3, what is f(4)? | Cho f(x) = 2x + 3, f(4) bằng bao nhiêu?

A. 14 | 14

B. 8 | 8

C. 7 | 7

D. 11 | 11

Đáp án: D

Giải thích: Thay x = 4: f(4) = 2 × 4 + 3 = 11.

Q6. At which x is $f(x) = (x - 3)^2$ equal to zero? | Hàm $f(x) = (x - 3)^2$ bằng 0 tại giá trị x nào?

A. 6 | 6

B. 3 | 3

C. −3 | −3

D. 0 | 0

Đáp án: B

Giải thích: Bình phương bằng 0 khi biểu thức bên trong bằng 0: x − 3 = 0, nên x = 3.

Q7. Which best describes exponential growth compared with linear growth? | Đâu là điểm khác nhau điển hình giữa tăng trưởng hàm mũ và tuyến tính?

A. Exponential growth always stops | Tăng trưởng mũ luôn dừng

B. Both add the same amount at each step | Cả hai cùng cộng số lượng cố định mỗi bước

C. Linear growth always decreases | Tuyến tính luôn giảm

D. Exponential growth multiplies by a fixed factor in equal steps | Tăng trưởng mũ nhân theo một tỷ lệ cố định qua các bước đều

Đáp án: D

Giải thích: Hàm tuyến tính có mức tăng tuyệt đối cố định; hàm mũ tăng theo tỷ lệ nhân cố định khi đầu vào tăng đều.

Q8. What is f(2) if $f(x) = (x - 2)^2$? | Nếu $f(x) = (x - 2)^2$, f(2) bằng bao nhiêu?

A. 2 | 2

B. 0 | 0

C. 8 | 8

D. 4 | 4

Đáp án: B

Giải thích: Thay x = 2: $(2 - 2)^2 = 0$.

Q9. Which NumPy function evaluates a polynomial? | Hàm NumPy nào tính đa thức?

A. Highest degree first | Bậc cao nhất trước

B. `np.linspace(start,stop,num)` | `np.linspace(start,stop,num)`

C. `np.exp(x)` | `np.exp(x)`

D. `np.polyval(coefficients,x)` | `np.polyval(coefficients,x)`

Đáp án: D

Giải thích: polyval tính đa thức theo các hệ số có thứ tự.

Q10. In what order does `np.polyval` take coefficients? | `np.polyval` nhận hệ số theo thứ tự gì?

A. `np.linspace(start,stop,num)` | `np.linspace(start,stop,num)`

B. `np.polyval(coefficients,x)` | `np.polyval(coefficients,x)`

C. `np.exp(x)` | `np.exp(x)`

D. Highest degree first | Bậc cao nhất trước

Đáp án: D

Giải thích: Hệ số bậc cao nhất đứng đầu.

Q11. Which function computes e raised to x? | Hàm nào tính e mũ x?

A. Highest degree first | Bậc cao nhất trước

B. `np.linspace(start,stop,num)` | `np.linspace(start,stop,num)`

C. `np.polyval(coefficients,x)` | `np.polyval(coefficients,x)`

D. `np.exp(x)` | `np.exp(x)`

Đáp án: D

Giải thích: exp tính hàm mũ cơ số e.

Q12. Which function makes evenly spaced samples? | Hàm nào tạo các điểm cách đều?

A. Highest degree first | Bậc cao nhất trước

B. `np.polyval(coefficients,x)` | `np.polyval(coefficients,x)`

C. `np.exp(x)` | `np.exp(x)`

D. `np.linspace(start,stop,num)` | `np.linspace(start,stop,num)`

Đáp án: D

Giải thích: linspace tạo số lượng điểm cách đều yêu cầu.

Q13. What is the value of y=2x+5 at x=0? | Giá trị y=2x+5 khi x=0 là gì?

A. 16 | 16

B. 4 | 4

C. 6 | 6

D. 5 | 5

Đáp án: D

Giải thích: Khi x=0, ta có 2×0+5=5.

Q14. What is the value of x^2+2 at x=2? | Giá trị x^2+2 khi x=2 là gì?

A. 6 | 6

B. 4 | 4

C. 16 | 16

D. 5 | 5

Đáp án: A

Giải thích: Bình phương 2 rồi cộng 2 bằng 6.

Q15. What is 2 raised to the fourth power? | 2 mũ 4 bằng gì?

A. 6 | 6

B. 5 | 5

C. 4 | 4

D. 16 | 16

Đáp án: D

Giải thích: 2 nhân bốn lần bằng 16.

Q16. How many intervals are between five evenly spaced points? | Năm điểm cách đều tạo bao nhiêu khoảng?

A. 5 | 5

B. 16 | 16

C. 6 | 6

D. 4 | 4

Đáp án: D

Giải thích: n điểm tạo n−1 khoảng.

Q17. What does `np.exp(0)` equal? | `np.exp(0)` bằng bao nhiêu?

A. A constant additive increment | Mỗi bước cộng cùng một mức

B. Accuracy on unseen real-world data | Độ chính xác trên dữ liệu thực chưa gặp

C. 1 | 1

D. Its slope changes with x | Độ dốc thay đổi theo x

Đáp án: C

Giải thích: Cơ số khác 0 mũ 0 bằng 1.

Q18. How does a quadratic differ from a first-degree affine function? | Hàm bậc hai khác hàm affine bậc nhất ở đâu?

A. A constant additive increment | Mỗi bước cộng cùng một mức

B. 1 | 1

C. Accuracy on unseen real-world data | Độ chính xác trên dữ liệu thực chưa gặp

D. Its slope changes with x | Độ dốc thay đổi theo x

Đáp án: D

Giải thích: Mức tăng của hàm bậc hai thay đổi theo x.

Q19. Which constraint gives a constant-per-step growth model? | Mô hình tăng đều mỗi bước cần điều kiện gì?

A. A constant additive increment | Mỗi bước cộng cùng một mức

B. 1 | 1

C. Accuracy on unseen real-world data | Độ chính xác trên dữ liệu thực chưa gặp

D. Its slope changes with x | Độ dốc thay đổi theo x

Đáp án: A

Giải thích: Cộng hằng số tạo mức tăng cố định.

Q20. What does a fitted function alone not prove? | Một hàm được khớp dữ liệu chưa chứng minh được gì?

A. A constant additive increment | Mỗi bước cộng cùng một mức

B. 1 | 1

C. Its slope changes with x | Độ dốc thay đổi theo x

D. Accuracy on unseen real-world data | Độ chính xác trên dữ liệu thực chưa gặp

Đáp án: D

Giải thích: Khả năng dự đoán cần được kiểm chứng ngoài các ví dụ khớp.

8. Bài tiếp theo

`Hàm Python → Hàm toán theo dữ liệu → Loss → Gradient.`

PY-29 — Activation: ReLU, sigmoid, tanh, softmax

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: ReLU — hàm chỉnh lưu; **Sigmoid** — hàm sigmoid.

`Hàm số → Activation → Neural Network → Classification`

Học xong dùng để làm gì? Hiểu ReLU/sigmoid biến đổi tín hiệu và giúp mô hình vượt giới hạn của lớp tuyến tính thuần túy.

Kết nối kiến thức tiếp theo: PY-30 dùng loss để đo dự đoán sai thực tế bao nhiêu.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
ReLU	hàm chỉnh lưu	Giữ số dương, số âm thành 0
Sigmoid	hàm sigmoid	Ánh xạ số thực về khoảng (0,1)
Tanh	hàm tanh	Ánh xạ số thực về (-1,1)
Softmax	hàm softmax	Biến logits thành phân phối xác suất tổng bằng 1

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`np.maximum(0,x)`	ReLU, tạo phi tuyến cho mạng
`1/(1+np.exp(-x))`	Sigmoid, cẩn thận số quá lớn
`np.tanh(x)`	Kích hoạt có đầu ra âm/dương
`np.exp(z-max(z))/sum(... )`	Softmax ổn định hơn tính exp(z) trực tiếp

4. Khái niệm liên quan

Nội dung chính: một mạng nơ-ron áp hàm kích hoạt lên điểm thô để tạo quan hệ phi tuyến. ReLU chặn phần âm, sigmoid nén về (0,1), tanh giữ dấu trong (−1,1), còn softmax ràng buộc nhiều lớp thành phân phối có tổng 1.

4.1. ReLU — Giữ phần dương, chặn phần âm

Từ vựng trong mục: ReLU — Rectified Linear Unit; **Activation** — kích hoạt.

Activation function (hàm kích hoạt) biến đổi đầu ra của một nơ-ron trước khi chuyển sang bước kế tiếp. **ReLU** tính `max(0,x)`: đầu vào âm được chuyển về 0, đầu vào dương được giữ. Ví dụ các tín hiệu `[-2,0,3]` thành `[0,0,3]`. Đây là hàm gấp khúc tại 0, giúp mạng có quan hệ phi tuyến; song nơ-ron có thể ngừng cập nhật nếu liên tục nhận vùng âm.

No Note — Không chú thích

```
import numpy as np

signals = np.array([-2, 0, 3])
print(np.maximum(0, signals).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Áp dụng ReLU để giữ tín hiệu không âm và thay mọi tín hiệu âm bằng 0.
# Dữ liệu: Vector ba tín hiệu đầu vào -2, 0, 3, mỗi ô là một nơ-ron giả lập.
# NumPy giúp áp hàm kích hoạt trực tiếp trên mọi phần tử mảng.
import numpy as np

# Mảng đầu vào một chiều gồm tín hiệu âm, 0 và dương.
signals = np.array([-2, 0, 3])
# maximum(0,signals) chọn số lớn hơn giữa 0 và từng tín hiệu, tức ReLU(x)=max(0,x); đầu ra [0,0,3].
# tolist() đổi ndarray về list Python để hiển thị dễ đọc.
print(np.maximum(0, signals).tolist())
```

Kết quả khi chạy:

```
[0, 0, 3]
```

ReLU không giữ phần âm; đầu vào dương bằng 3 đi qua không đổi.

4.2. Sigmoid — Ánh xạ một điểm thô sang (0,1)

Từ vựng trong mục: Sigmoid — hàm chữ S; **Logit** — điểm thô.

Sigmoid là `1/(1+exp(-x))`, đổi một **logit** (điểm thô có thể âm hoặc dương) thành số trong khoảng **(0,1)**. Ở x=0 đầu ra đúng 0.5; x âm dưới 0.5 và x dương trên 0.5. Trong phân loại nhị phân, thường dùng sigmoid cho lớp dương, nhưng bản thân giá trị đầu ra **không bảo đảm là xác suất đã hiệu chuẩn**. Khi x quá lớn về độ lớn, cần triển khai số học ổn định hơn công thức trực tiếp.

No Note — Không chú thích

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
scores = 1 / (1 + np.exp(-x))
print(np.round(scores, 3).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng Sigmoid biến ba logit âm, không, dương thành giá trị nằm giữa 0 và 1.
# Dữ liệu: Logit x=-1,0,1 là điểm thô không phải xác suất; đầu ra sigmoid có dạng xác suất.
# NumPy np.exp() tính hàm mũ tự nhiên trên mảng.
import numpy as np

# Ba điểm đầu vào thực, cả âm và dương.
```

```
x = np.array([-1.0, 0.0, 1.0])
# exp() tính e^(-x) theo từng ô; dấu âm đổi dấu đầu vào trước phép mũ.
# Sigmoid có đường cong chữ S: đầu ra gần 0 với x âm lớn, bằng 0.5 khi x=0, gần 1 với x dương lớn.
scores = 1 / (1 + np.exp(-x))
# round(...,3) làm tròn ba chữ số sau dấu thập phân để in, không thay scores gốc.
# tolist() chuyển ndarray của ba kết quả thành list Python.
print(np.round(scores, 3).tolist())
```

Kết quả khi chạy:

```
[0.269, 0.5, 0.731]
```

Điểm âm được ánh xạ dưới 0.5, điểm 0 thành 0.5 và điểm dương trên 0.5.

4.3. Tanh — Giữ dấu âm và dương quanh zero

Từ vựng trong mục: Tanh — tang hyperbolic; **Zero-centered** — cân quanh 0.

Tanh cũng cong chữ S nhưng đầu ra nằm trong **(−1,1)** thay vì (0,1). Đầu vào âm cho đầu ra âm, đầu vào 0 cho 0, đầu vào dương cho dương. Vì cân quanh 0, tanh có thể thích hợp khi cần giữ dấu tín hiệu. Với x=−1,0,1 kết quả khoảng −0.762, 0, 0.762. Khi |x| rất lớn, tanh bão hòa gần ±1 nên độ dốc nhỏ.

No Note — Không chú thích

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
print(np.round(np.tanh(x), 3).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Áp tanh để nén tín hiệu về khoảng -1 đến 1 và giữ dấu.
# Dữ liệu: Ba đầu vào -1,0,1, điểm dương/âm vẫn được phân biệt.
# NumPy có hàm tanh vector hóa.
import numpy as np

# Vector ba logit có giá trị âm, bằng 0, dương.
x = np.array([-1.0, 0.0, 1.0])
# tanh(x) có giá trị từ -1 đến 1, đi qua 0, giữ dấu đầu vào; khác sigmoid nằm 0..1.
# np.round(...,3) làm tròn kết quả hiển thị đến ba chữ số thập phân.
# tolist() đổi mảng số thành list Python để print().
print(np.round(np.tanh(x), 3).tolist())
```

Kết quả khi chạy:

```
[-0.762, 0.0, 0.762]
```

Hàm giữ dấu âm/dương và bằng 0 tại đầu vào 0.

4.4. Softmax — So sánh nhiều lớp cùng lúc

Từ vựng trong mục: Softmax — chuẩn hóa điểm nhiều lớp; **Class logit** — điểm thô theo lớp.

Khi mô hình phân loại **một nhãn trong ba lớp loại trừ nhau**, nó có ba logit (điểm thô) `z=[2,1,0.1]`. Softmax dùng `exp(z_i)/sum(exp(z_j))` để mỗi lớp nhận giá trị trong (0,1) và **tổng các lớp bằng 1**. Trừ

`max(z)` trước khi tính exp giúp giảm nguy cơ tràn số mà không đổi kết quả. Khác sigmoid áp riêng từng giá trị, softmax ràng buộc các lớp cạnh tranh nhau.

No Note — Không chú thích

```
import numpy as np

z = np.array([2.0, 1.0, 0.1])
e = np.exp(z - z.max())
p = e / e.sum()
print(np.round(p, 3).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng Softmax chuyển ba điểm lớp thô thành tỷ lệ dương tổng bằng 1.
# Dữ liệu: Ba logit của ba lớp [2,1,0.1], không phải xác suất ban đầu.
# NumPy có exp(), max(), sum() để tính softmax ổn định.
import numpy as np

# Mỗi phần tử là điểm thô của một lớp; lớp 0 đang lớn nhất.
z = np.array([2.0, 1.0, 0.1])
# z.max() lấy logit lớn nhất; trừ nó khỏi tất cả ô giúp tránh exp của số quá lớn (chống overflow),
không đổi tỷ lệ softmax.
# np.exp() lấy e mũ cho từng logit đã dịch, biến điểm thành trọng số dương.
e = np.exp(z - z.max())
# e.sum() cộng trọng số ba lớp, chia từng ô để ra xác suất chuẩn hóa tổng đúng 1.
p = e / e.sum()
# round(...,3) chỉ làm tròn để in ba số thập phân, không làm lại phép chuẩn hóa.
# tolist() chuyển ndarray thành list Python.
print(np.round(p, 3).tolist())
```

Kết quả khi chạy:

```
[0.659, 0.242, 0.099]
```

Lớp đầu có điểm chuẩn hóa cao nhất; tổng ba điểm trước làm tròn là 1.

5. Năm tình huống Activation — từ ReLU đến phân loại 20 mẫu

Tình huống 1 — Cấp 1: Chặn tín hiệu âm của 10 nơ-ron bằng ReLU

Từ vựng trong mục: ReLU — hàm giữ phần dương; **Activation** — kích hoạt.

Tình huống thực tế: Trong một mô hình mạng nơ-ron **giả lập**, tầng xử lý nhận **10 tín hiệu thô** không có đơn vị. Có tín hiệu âm, bằng 0 và dương, ví dụ -5 và 5. Bài yêu cầu dùng quy tắc **ReLU** để chặn tín hiệu âm thành 0, giữ tín hiệu dương, rồi đếm bao nhiêu đầu ra bằng 0 và bao nhiêu đầu ra còn dương. Đây là phép thử tính toán, không phải tín hiệu điện của thiết bị thật.

Vì sao chọn cách này? Hàm kích hoạt biến tín hiệu của một tầng thành đầu ra theo quy tắc nhất định. **ReLU** giữ số dương như cũ, nhưng đổi mọi số âm thành **0**; nó **không xóa vị trí nơ-ron** hay rút ngắn mảng. `np.maximum(0, signals)` áp quy tắc ấy cho cả 10 tín hiệu trong một lần; sau đó so từng đầu ra với 0 để đếm. Cần phân biệt **tín hiệu đầu vào âm** và **đầu ra bằng 0**: đầu ra bằng 0 còn gồm cả tín hiệu đầu vào vốn bằng 0. Quy tắc này đơn giản và hay dùng ở các tầng giữa, không có nghĩa mọi số âm ngoài thực tế đều vô giá trị.

Quy trình:

1. Lưu 10 giá trị gồm âm, 0 và dương.
2. Áp ReLU cho toàn bộ mảng.
3. Đếm phần tử có đầu ra bằng 0, đối chiếu đầu ra đầu/cuối.

No Note — Không chú thích

```
import numpy as np
signals = np.array([-5,-3,-1,0,1,2,4,-2,3,5], dtype=float)
activated = np.maximum(0, signals)
if activated.shape != (10,) or signals.shape != (10,):
    raise ValueError("Cần đúng 10 tín hiệu và 10 giá trị ReLU.")
print("Nơ-ron:", len(activated))
print("Đầu ra bằng 0:", int((activated == 0).sum()))
print("Đầu ra dương:", int((activated > 0).sum()))
print("Mẫu đầu/cuối:", int(activated[0]), int(activated[-1]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm đầu ra ReLU trên 10 tín hiệu và đếm số´bị chặn ở 0.
# Dữ liệu: 10 tín hiệu từ âm tới dương, dtype=float, mỗi tín hiệu ứng với một nơ-ron giả lập.
# NumPy maximum và các mask Boolean chạy trên vector.
import numpy as np
# 10 giá trị thực từ -5 đến 5 theo thứ tự, có âm/0/dương.
signals = np.array([-5,-3,-1,0,1,2,4,-2,3,5], dtype=float)
# ReLU chọn max(0,x) TỪNG Ô, thay x âm bằng 0, giữ nguyên x dương.
activated = np.maximum(0, signals)
# shape (10,) là một vector 10 đầu ra; kiểm cả tín hiệu và đầu ra đều đủ.
if activated.shape != (10,) or signals.shape != (10,):
    # ValueError dừng nếu shape, số´mẫu hoặc giới hạn đầu ra không thỏa kỳ vọng.
    raise ValueError("Cần đúng 10 tín hiệu và 10 giá trị ReLU.")
# len() đếm 10 tín hiệu đã kích hoạt.
print("Nơ-ron:", len(activated))
# == 0 tạo mask Boolean ở ô bị ReLU tắt hoặc vốn 0; .sum() đếm số´True.
print("Đầu ra bằng 0:", int((activated == 0).sum()))
# Mask >0 đếm số´ô giữ đầu ra dương; int() in số´nguyên Python.
print("Đầu ra dương:", int((activated > 0).sum()))
# [0] là mẫu đầu, [-1] mẫu cuối; int() đổi scalar số´đẻ'in.
print("Mẫu đầu/cuối:", int(activated[0]), int(activated[-1]))
```

Kết quả khi chạy:

```
Nơ-ron: 10
Đầu ra bằng 0: 5
Đầu ra dương: 5
Mẫu đầu/cuối: 0 5
```

Đầu ra ReLU bằng 0 có thể bắt nguồn từ **đầu vào âm hoặc bằng 0**, hai trường hợp cần phân biệt khi phân tích.

Tình huống 2 — Cấp 2: Ánh xạ 12 logit sang điểm sigmoid

Từ vựng trong mục: Sigmoid — hàm chữ S; **Logit** — điểm thô; **Threshold** — ngưỡng.

Tình huống thực tế: Mô hình phân loại **hai lựa chọn** cho **12 mẫu thử** tạo ra 12 **logit**, tức điểm thô chưa nằm trong khoảng 0 đến 1. Trong số đó có logit âm, dương và bằng 0. Người phân tích cần biến từng

điểm qua **sigmoid** để có số nằm giữa 0 và 1, đếm số mẫu có điểm **ít nhất 0,5** và kiểm tra logit bằng 0 cho kết quả đúng **0,5**.

Vì sao chọn cách này? Logit là điểm mô hình tạo ra trước khi quy đổi, có thể âm hoặc dương và **chưa phải xác suất**. **Sigmoid** đưa điểm đó về khoảng **giữa 0 và 1**; logit 0 nằm đúng giữa nên thành **0,5**, logit dương lớn hơn 0,5 và logit âm nhỏ hơn 0,5. `np.exp()` được dùng trong tính toán cho cả 12 mẫu; `>= 0.5` chỉ là **ngưỡng phân lớp minh họa**. Số sau sigmoid trông giống xác suất, nhưng **chưa thể kết luận mô hình dự báo xác suất chính xác** nếu chưa được kiểm tra hiệu chuẩn trên dữ liệu thực.

Quy trình:

1. Tạo 12 điểm thô gồm số âm và dương.
2. Tính vector sigmoid và đối chiếu đầu ra của logit 0.
3. Đếm số điểm đạt ngưỡng 0,5, xác nhận tất cả đầu ra nằm trong (0,1).

No Note — Không chú thích

```
import numpy as np
logits = np.array([-3,-2,-1.5,-1,0,0.2,0.8,1,2,3,4,-0.5], dtype=float)
probabilities = 1 / (1 + np.exp(-logits))
if logits.size != 12 or not np.all((probabilities > 0) & (probabilities < 1)):
    raise ValueError("Cần 12 mẫu Sigmoid, mỗi xác suất phải nằm giữa 0 và 1.")
print("Mẫu:", len(logits))
print("Từ 0.5:", int((probabilities >= 0.5).sum()))
print("Logit 0:", f"{probabilities[logits == 0][0]:.3f}")
print("Logit -3:", f"{probabilities[0]:.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đánh giá Sigmoid trên 12 logit và dùng ngưỡng 0.5 để đếm nhãn dương.
# Dữ liệu: 12 logit điểm thô từ -3 tới 4, trước kích hoạt chưa phải xác suất.
# NumPy vector hóa phép mũ và so sánh xác suất.
import numpy as np
# 12 điểm thô số thực; giá trị âm/0/dương.
logits = np.array([-3,-2,-1.5,-1,0,0.2,0.8,1,2,3,4,-0.5], dtype=float)
# exp() tính hàm mũ e^(-logit) từng ô, thành phần của Sigmoid.
# Sigmoid chuyển từng điểm thô sang 0..1; 0 map thành 0.5.
probabilities = 1 / (1 + np.exp(-logits))
# size là số ô đầu vào, kỳ vọng 12.
# Mỗi so sánh tạo mask, & là AND theo từng ô, np.all() buộc mọi đầu ra nằm TRONG khoảng (0,1),
không dùng 'and' với ndarray.
if logits.size != 12 or not np.all((probabilities > 0) & (probabilities < 1)):
    # ValueError dừng nếu shape, số mẫu hoặc giới hạn đầu ra không thỏa kỳ vọng.
    raise ValueError("Cần 12 mẫu Sigmoid, mỗi xác suất phải nằm giữa 0 và 1.")
print("Mẫu:", len(logits))
# So xác suất với ngưỡng quyết định 0.5 rồi sum() mask để đếm số thuộc lớp dương.
print("Từ 0.5:", int((probabilities >= 0.5).sum()))
# logits==0 tạo mask; probabilities[mask] lọc giá trị của logit 0, [0] lấy một phần tử trong mảng
lọc.
print("Logit 0:", f"{probabilities[logits == 0][0]:.3f}")
# [0] lấy giá trị logit -3 theo vị trí nguồn; :.3f in ba chữ số lẻ.
print("Logit -3:", f"{probabilities[0]:.3f}")
```

Kết quả khi chạy:

```
Mẫu: 12
Từ 0.5: 7
```

```
Logit 0: 0.500
Logit -3: 0.047
```

Ngưỡng 0,5 là lựa chọn minh họa; dùng ngưỡng thật cần xét chi phí sai lệch và dữ liệu kiểm chứng.

Tình huống 3 — Cấp 3: So dấu và hiện tượng bão hòa của tanh trên 15 tín hiệu

Từ vựng trong mục: Tanh — hàm hyperbolic; **Saturation** — bão hòa; **Zero-centered** — đối xứng qua 0.

Tình huống thực tế: Một tầng xử lý thử nghiệm nhận **15 tín hiệu từ -7 đến 7**, gồm các giá trị âm, 0 và dương. Nhóm muốn dùng **tanh** để đưa tín hiệu về khoảng gần **-1 đến 1**, rồi xem dấu của tín hiệu có được giữ không và có bao nhiêu đầu ra tiến sát hai đầu mút, cụ thể **độ lớn lớn hơn 0,99**. Các con số không có đơn vị và chỉ dùng để minh họa quy tắc kích hoạt.

Vì sao chọn cách này? Tanh cũng là một hàm kích hoạt nhưng khác sigmoid ở chỗ đầu ra có thể **âm, bằng 0 hoặc dương**. Nó giữ dấu của đầu vào và đưa các số có độ lớn rất lớn về **gần -1 hoặc gần +1**; đó gọi là **bão hòa**. `np.tanh(signals)` tính đủ 15 đầu ra; `np.abs()` lấy **độ lớn bất kể dấu** để đếm những đầu ra có độ lớn **vượt 0,99**. Khi gần bão hòa, tăng thêm đầu vào thường làm đầu ra đổi rất ít, điều cần chú ý khi huấn luyện mô hình; bài chỉ quan sát hiện tượng này, chưa huấn luyện mạng.

Quy trình:

1. Tạo 15 giá trị đầu vào đối xứng qua 0.
2. Dùng `np.tanh` và kiểm dấu bằng so sánh.
3. Đếm các trường hợp đầu ra có độ lớn trên 0,99.

No Note — Không chú thích

```
import numpy as np
signals = np.array([-7,-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6,7], dtype=float)
activated = np.tanh(signals)
if signals.size != 15:
    raise ValueError("Cần đủ 15 tín hiệu Tanh.")
print("Tín hiệu:", len(signals))
print("Tanh(0):", f"{activated[signals == 0][0]:.3f}")
print("Bão hòa |y| > 0.99:", int((np.abs(activated) > 0.99).sum()))
print("Giữ dấu:", bool(np.all(np.sign(activated) == np.sign(signals))))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm Tanh trên 15 tín hiệu, sô´tín hiệu bão hòa và khả năng giữ dấu.
# Dữ liệu: 15 logit -7..7, điểm không bằng 0 có dấu âm hoặc dương.
# NumPy np.tanh() kích hoạt theo từng ô, np.sign() kiểm dấu.
import numpy as np
# 15 tín hiệu, dtype=float; có giá trị đôí xứng âm/dương quanh 0.
signals = np.array([-7,-6,-5,-4,-3,-2,-1,0,1,2,3,4,5,6,7], dtype=float)
# tanh nén giá trị vào (-1,1), âm vẫn âm, dương vẫn dương, 0→0.
activated = np.tanh(signals)
# size đếm 15 phần tư ̉vector.
if signals.size != 15:
    # ValueError dừng nếu shape, sô´mẫu hoặc giới hạn đầu ra không thỏa kỳ vọng.
    raise ValueError("Cần đủ 15 tín hiệu Tanh.")
print("Tín hiệu:", len(signals))
# signals==0 là mask tìm logit 0; [0] chọn đầu ra tương ứng, :.3f in 3 sô´lẻ.
print("Tanh(0):", f"{activated[signals == 0][0]:.3f}")
# abs() lấy độ lớn đầu ra, ngưỡng 0.99 đánh dấu tín hiệu gần ±1 (bão hòa); .sum() đếm.
print("Bão hòa |y| > 0.99:", int((np.abs(activated) > 0.99).sum()))
# np.sign() trả -1,0,+1 theo dấu; == so từng ô hai vector đê ̉xác minh tanh giữ dấu.
```

```
# np.all() True chỉ khi mọi vị trí đều giữ dấú; bool() đổi vê`bool Python.
print("Giữ dấú:", bool(np.all(np.sign(activated) == np.sign(signals))))
```

Kết quả khi chạy:

```
Tín hiệu: 15
Tanh(0): 0.000
Bão hòa |y| > 0.99: 10
Giữ dấú: True
```

Không được coi đầu ra gần ±1 là tín hiệu ban đầu cũng chỉ có độ lớn 1; hàm đã nén đầu vào.

Tình huống 4 — Cấp 4: Softmax ổn định cho 18 mẫu phân loại ba lớp

Từ vựng trong mục: Softmax — chuẩn hóa theo hàng; **Numerical stability** — ổn định số; **Argmax** — lớp lớn nhất.

Tình huống thực tế: Một bộ phân loại có **18 mẫu**, mỗi mẫu cần chọn một trong **ba lớp A, B hoặc C**. Mô hình xuất ba **logit** không đơn vị cho từng mẫu; một dòng có điểm thô gần **1.000** nên nếu tính hàm mũ trực tiếp có nguy cơ ra số quá lớn. Nhóm muốn biến ba điểm mỗi mẫu thành ba **điểm softmax có tổng bằng 1**, đếm số mẫu chọn từng lớp và kiểm tra dòng logit rất lớn vẫn cho kết quả hữu hạn.

Vì sao chọn cách này? Softmax chuyển ba logit của **một mẫu** thành ba điểm dương có **tổng bằng 1**, hữu ích khi chọn **một lớp trong ba lớp loại trừ nhau**. Vì một logit tới gần 1.000, tính `np.exp()` trực tiếp có thể bị **tràn số** (quá lớn để máy lưu). Chương trình trừ **logit lớn nhất của mỗi hàng** trước rồi mới tính mũ; điều này không đổi tỷ lệ so sánh giữa ba lớp trong cùng mẫu mà giúp tính ổn định. `axis=1` chỉ việc xử lý **ba lớp ngang một hàng**, không gộp 18 mẫu vào chung một phân phối. `argmax(axis=1)` chọn lớp điểm cao nhất của từng mẫu; tổng mỗi hàng gần 1 chứ tổng tất cả 18 hàng không phải 1.

Quy trình:

1. Tạo mảng 18×3 có một hàng logit rất lớn.
2. Trừ max theo `axis=1`, lấy mũ rồi chia tổng hàng.
3. Kiểm tra tổng xác suất từng hàng, đếm nhãn dự đoán A/B/C.

No Note — Không chú thích

```
import numpy as np
logits = np.array([[3,1,0],[0,3,1],[1,0,3],[1000,999,998],[0,3,1],[1,0,3],[3,1,0],[0,3,1],[1,0,3],
[3,1,0],[0,3,1],[1,0,3],[3,1,0],[0,3,1],[1,0,3],[3,1,0],[0,3,1],[1,0,3]], dtype=float)
shifted = logits - logits.max(axis=1, keepdims=True)
exp_values = np.exp(shifted)
probs = exp_values / exp_values.sum(axis=1, keepdims=True)
predictions = probs.argmax(axis=1)
if logits.shape != (18, 3) or probs.shape != (18, 3):
    raise ValueError("Cần đủ 18 mẫũ, môĩ mẫũ có 3 lớp.")
print("Mẫũ:", len(logits))
print("Tổng hàng đèu là 1:", bool(np.allclose(probs.sum(axis=1), 1)))
counts = []
for k in range(3):
    counts.append(int((predictions == k).sum()))
print("Nhãn A/B/C:", *counts)
print("Mẫũ logit lớn hữu hạn:", bool(np.isfinite(probs[3]).all()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính Softmax ổn định cho 18 dòng ba lớp và thống kê lớp dự đoán.
# Dữ liệu: 18 hàng mẫu, 3 cột điểm logit lớp A/B/C, gồm một hàng logit [1000,999,998] rất lớn.
# NumPy tính exp, max, sum, argmax và kiểm tính hữu hạn.
import numpy as np
# Bảng shape (18,3): một hàng ứng với một mẫu, ba cột A/B/C là điểm thô.
logits = np.array([[3,1,0],[0,3,1],[1,0,3],[1000,999,998],[0,3,1],[1,0,3],[3,1,0],[0,3,1],[1,0,3],
[3,1,0],[0,3,1],[1,0,3],[3,1,0],[0,3,1],[1,0,3],[3,1,0],[0,3,1],[1,0,3]], dtype=float)
# axis=1 lấy điểm cao nhất CỦA TỪNG HÀNG, keepdims=True giữ shape (18,1) để broadcast an toàn về
18×3.
# Trừ max từng hàng trước khi exp để tránh overflow với logit 1000; Softmax không đổi khi mọi điểm
một hàng cùng trừ hằng số.
shifted = logits - logits.max(axis=1, keepdims=True)
# e^x cho từng logit đã dịch, ra trọng số dương hữu hạn.
exp_values = np.exp(shifted)
# sum(axis=1) cộng ba trọng số mỗi hàng, keepdims=True giữ cột (18,1) để chia từng ô đúng hàng.
# Broadcast chia mỗi hàng cho tổng của chính hàng để xác suất mỗi hàng cộng bằng 1.
probs = exp_values / exp_values.sum(axis=1, keepdims=True)
# argmax(axis=1) trả chỉ số cột lớp lớn nhất trên TỪNG HÀNG, mã 0/1/2; không trả xác suất.
predictions = probs.argmax(axis=1)
# Kiểm đúng 18 mẫu và ba lớp, cả logits/probs đều shape (18,3).
if logits.shape != (18, 3) or probs.shape != (18, 3):
    # ValueError dừng nếu shape, số mẫu hoặc giới hạn đầu ra không thỏa kỳ vọng.
    raise ValueError("Cần đủ 18 mẫu, mỗi mẫu có 3 lớp.")
print("Mẫu:", len(logits))
# sum(axis=1) lấy tổng ba lớp mỗi hàng, allclose() so gần 1 có dung sai số thực.
print("Tổng hàng đều là 1:", bool(np.allclose(probs.sum(axis=1), 1)))
counts = []
# range(3) tạo các mã lớp 0,1,2; không có 3.
for k in range(3):
    # Mask xác định hàng dự đoán là lớp k, sum() đếm số mẫu của lớp đó.
    counts.append(int((predictions == k).sum()))
# *counts giải nén ba phần tử list thành ba đối số print riêng, ứng với A/B/C.
print("Nhãn A/B/C:", *counts)
# [3] là hàng logit 1000 (index từ 0); isfinite() kiểm không NaN/inf, .all() đòi cả ba ô hợp lệ.
print("Mẫu logit lớn hữu hạn:", bool(np.isfinite(probs[3]).all()))
```

Kết quả khi chạy:

```
Mẫu: 18
Tổng hàng đều là 1: True
Nhãn A/B/C: 6 6 6
Mẫu logit lớn hữu hạn: True
```

Softmax xử lý **ba lớp cạnh tranh trong mỗi mẫu**, không dùng một sigmoid riêng rẽ rồi giả định tổng bằng 1.

Tình huống 5 — Cấp 5: Ghép ReLU và sigmoid để phân loại 20 mẫu, kiểm cả nhãn thật

Từ vựng trong mục: Activation pipeline — chuỗi kích hoạt; **ReLU** — chặn âm; **Sigmoid** — điểm nhị phân.

Tình huống thực tế: Một mô hình thử có **20 mẫu**, mỗi mẫu gồm hai tín hiệu đầu vào **x và y** cùng nhãn đúng **0 hoặc 1**. Dữ liệu được tạo theo quy tắc nhãn là 1 khi **tổng x và y lớn hơn 0**. Mô hình lại dùng ReLU chặn riêng từng tín hiệu âm, cộng các tín hiệu đã chặn, bớt 1 rồi dùng sigmoid quyết định theo ngưỡng **0,5**. Nhóm muốn biết dự đoán đúng và sai bao nhiêu lần để thấy biến đổi đầu vào có thể làm mất thông tin cần phân lớp.

Vì sao chọn cách này? ReLU và sigmoid làm **hai việc khác nhau** trong mô hình. ReLU chặn phần âm của từng tín hiệu đầu vào, khiến một số thông tin về **độ lớn âm** biến mất; sigmoid chỉ đổi điểm cuối thành số

từ 0 đến 1 để áp ngưỡng dự đoán. Nhãn đúng trong bài dựa trên **tổng x và y gốc**, vì vậy các mẫu có số âm mạnh có thể bị mô hình ReLU hiểu khác: không thể lấy điểm sigmoid lớn rồi cho rằng dự đoán đúng. Chương trình phải so `predicted` với **20 nhãn tham chiếu** để đếm đúng/sai. Đây là ví dụ cho thấy chọn phép biến đổi đầu vào ảnh hưởng thông tin mô hình giữ lại, không khẳng định ReLU nói chung là không tốt.

Quy trình:

1. Tạo 20 hàng hai tín hiệu cùng 20 nhãn được định nghĩa rõ.
2. Áp ReLU, tính logit tổng và sigmoid, chuyển điểm thành nhãn theo ngưỡng 0,5.
3. Đếm số dự đoán đúng/sai và kiểm tổng bằng 20.

No Note — Không chú thích

```
import numpy as np
X = np.array([[-2,-1],[-1,-1],[0,-1],[1,-1],[2,-1],[-2,0],[-1,0],[0,0],[1,0],[2,0],[-2,1],[-1,1],
[0,1],[1,1],[2,1],[-2,2],[-1,2],[0,2],[1,2],[2,2]], dtype=float)
labels = np.array([0,0,0,0,1,0,0,0,1,1,0,0,1,1,1,0,1,1,1,1], dtype=int)
hidden = np.maximum(0, X)
logits = hidden.sum(axis=1) - 1
scores = 1 / (1 + np.exp(-logits))
predicted = (scores >= 0.5).astype(int)
correct = int((predicted == labels).sum())
if X.shape != (20, 2) or labels.shape != (20,) or predicted.shape != (20,):
    raise ValueError("Cần đủ 20 mẫu, hai đặc trưng và 20 nhãn đối chiếu.")
print("Mẫu:", len(labels))
print("Dự đoán đúng:", correct)
print("Dự đoán sai:", len(labels) - correct)
print("Có nhãn dương:", int(labels.sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng ReLU hai đặc trưng, tổng tín hiệu và Sigmoid để dự đoán 20 nhãn 0/1.
# Dữ liệu: 20 hàng X có 2 đặc trưng số; labels là 20 nhãn thật 0/1 cùng thứ tự hàng.
# NumPy hỗ trợ mạng kích hoạt đơn giản hoàn toàn bằng phép toán mảng.
import numpy as np
# X shape (20,2), mỗi hàng là một mẫu, mỗi cột là một đặc trưng có thể âm hoặc dương.
X = np.array([[-2,-1],[-1,-1],[0,-1],[1,-1],[2,-1],[-2,0],[-1,0],[0,0],[1,0],[2,0],[-2,1],[-1,1],
[0,1],[1,1],[2,1],[-2,2],[-1,2],[0,2],[1,2],[2,2]], dtype=float)
# 20 nhãn thật 0/1, dtype=int dùng làm đáp án đối chiếu.
labels = np.array([0,0,0,0,1,0,0,0,1,1,0,0,1,1,1,0,1,1,1,1], dtype=int)
# ReLU TỪNG Ô: âm thành 0, dương giữ nguyên; hidden vẫn shape (20,2).
hidden = np.maximum(0, X)
# axis=1 cộng hai đặc trưng của TỪNG MẪU thành một số; trừ 1 là bias/ngưỡng giả lập trước sigmoid.
logits = hidden.sum(axis=1) - 1
# exp() tính e mũ âm logit để Sigmoid hoạt động từng mẫu.
# Sigmoid biến 20 logit thành 20 xác suất lớp dương từ 0 đến 1.
scores = 1 / (1 + np.exp(-logits))
# Ngưỡng 0.5 tạo mask Boolean: True dự đoán 1, False dự đoán 0.
# astype(int) đổi mask True/False sang mảng nhãn 1/0 để so với labels.
predicted = (scores >= 0.5).astype(int)
# So sánh từng vị trí nhãn dự đoán và nhãn thật (không tự căn lại thứ tự hàng).
# sum() mask đếm số nhãn đúng; int() chuyển sang số nguyên Python.
correct = int((predicted == labels).sum())
# shape (20,2) của X và (20,) của labels/predicted kiểm đủ mẫu.
if X.shape != (20, 2) or labels.shape != (20,) or predicted.shape != (20,):
    # ValueError dừng nếu shape, số mẫu hoặc giới hạn đầu ra không thỏa kỳ vọng.
    raise ValueError("Cần đủ 20 mẫu, hai đặc trưng và 20 nhãn đối chiếu.")
print("Mẫu:", len(labels))
print("Dự đoán đúng:", correct)
print("Dự đoán sai:", len(labels) - correct)
```

```
# Nhãn chỉ 0/1 nên sum() đếm số nhãn dương thật; không phải số dự đoán dương.
print("Có nhãn dương:", int(labels.sum()))
```

Kết quả khi chạy:

```
Mẫu: 20
Dự đoán đúng: 16
Dự đoán sai: 4
Có nhãn dương: 10
```

Đây là pipeline với trọng số cố định **không được huấn luyện**; số mẫu đúng chỉ kiểm tra hành vi trên dữ liệu giả lập.

6. Sai lầm thường gặp

- ReLU không chặn **giá trị dương lớn**; nó chỉ chặn phần âm.
- Sigmoid nén mỗi giá trị về (0,1), nhưng các giá trị này không tự trở thành xác suất đã hiệu chuẩn.
- Tanh nằm trong $(-1,1)$, còn sigmoid trong (0,1); không thay thế khi cần giữ tín hiệu âm.
- Softmax áp **cùng lúc** lên các logit của nhiều lớp; không áp độc lập từng logit.
- Khi tính softmax nên trừ logit lớn nhất trước `exp` để giảm tràn số.
- Softmax phù hợp các lớp loại trừ nhau; nhiều nhãn có thể đồng thời đúng thường dùng nhiều đầu ra sigmoid thay vì ép tổng một.

7. Trắc nghiệm

Chỉ hỏi kiến thức, phân biệt hàm và cách đọc đầu ra; không hỏi tình huống đánh số.

Q1. What is a ReLU activation? | Hàm kích hoạt ReLU là gì?

A. `1/(1+exp(-x))` | `1/(1+exp(-x))`

B. A smooth activation mapping into $(-1,1)$ | Hàm trơn đưa giá trị vào $(-1,1)$

C. Normalization of several class logits to sum one | Chuẩn hóa nhiều logit lớp có tổng bằng một

D. `max(0,x)` | `max(0,x)`

Đáp án: D

Giải thích: ReLU chặn giá trị âm về không.

Q2. What is sigmoid? | Sigmoid là gì?

A. `max(0,x)` | `max(0,x)`

B. Normalization of several class logits to sum one | Chuẩn hóa nhiều logit lớp có tổng bằng một

C. A smooth activation mapping into $(-1,1)$ | Hàm trơn đưa giá trị vào $(-1,1)$

D. `1/(1+exp(-x))` | `1/(1+exp(-x))`

Đáp án: D

Giải thích: Sigmoid là phép biến đổi logistic hình chữ S.

Q3. What is tanh? | Tanh là gì?

A. `max(0,x)` | `max(0,x)`

B. `1/(1+exp(-x))` | `1/(1+exp(-x))`

C. Normalization of several class logits to sum one | Chuẩn hóa nhiều logit lớp có tổng bằng một

D. A smooth activation mapping into (−1,1) | Hàm trơn đưa giá trị vào (−1,1)

Đáp án: D

Giải thích: Tanh cân quanh 0 và giới hạn giữa −1 và 1.

Q4. What is softmax? | Softmax là gì?

A. `1/(1+exp(-x))` | `1/(1+exp(-x))`

B. `max(0,x)` | `max(0,x)`

C. A smooth activation mapping into (−1,1) | Hàm trơn đưa giá trị vào (−1,1)

D. Normalization of several class logits to sum one | Chuẩn hóa nhiều logit lớp có tổng bằng một

Đáp án: D

Giải thích: Softmax ràng buộc điểm lớp thành một phân phối chuẩn hóa.

Q5. What is ReLU(−3) if ReLU(x) = max(0, x)? | ReLU(−3) bằng bao nhiêu nếu ReLU(x) = max(0, x)?

A. 3 | 3

B. −3 | −3

C. 0 | 0

D. 1 | 1

Đáp án: C

Giải thích: ReLU chọn giá trị lớn hơn giữa 0 và x. Với −3, giá trị lớn hơn là 0.

Q6. What is the sigmoid output at input x = 0? | Hàm sigmoid nhận x = 0 thì kết quả là bao nhiêu?

A. 0 | 0

B. 1 | 1

C. −1 | −1

D. 0.5 | 0.5

Đáp án: D

Giải thích: Sigmoid(0) = 1/(1 + e^0) = 1/(1 + 1) = 0.5.

Q7. A softmax layer outputs probabilities for three classes. What must their sum be (up to rounding)? | Softmax trả về xác suất cho ba lớp. Tổng các xác suất phải gần bằng bao nhiêu?

A. 0.5 | 0.5

B. 3 | 3

C. 1 | 1

D. 0 | 0

Đáp án: C

Giải thích: Softmax chuẩn hóa điểm số của tất cả các lớp để các xác suất không âm và cộng lại bằng 1.

Q8. Which statement about sigmoid is correct for any finite input? | Phát biểu nào đúng về đầu ra sigmoid với mọi đầu vào hữu hạn?

A. The output always exceeds 1 | Kết quả luôn lớn hơn 1

B. The output lies strictly between 0 and 1 | Kết quả nằm giữa 0 và 1, không chạm hai đầu

C. The output can be negative | Kết quả có thể âm

D. The output is always exactly 0 or 1 | Kết quả luôn chính xác 0 hoặc 1

Đáp án: B

Giải thích: Với đầu vào hữu hạn, mẫu số 1 + e^(−x) lớn hơn 1, nên đầu ra dương nhưng nhỏ hơn 1.

Q9. Which NumPy function applies ReLU? | Hàm NumPy nào thực hiện ReLU?

A. `np.maximum(0,x)` | `np.maximum(0,x)`

B. The largest logit | Logit lớn nhất

C. Tanh | Tanh

D. `np.tanh(x)` | `np.tanh(x)`

Đáp án: A

Giải thích: maximum chặn từng phần tử.

Q10. Which NumPy function directly computes tanh? | Hàm NumPy nào tính tanh trực tiếp?

A. `np.tanh(x)` | `np.tanh(x)`

B. Tanh | Tanh

C. The largest logit | Logit lớn nhất

D. `np.maximum(0,x)` | `np.maximum(0,x)`

Đáp án: A

Giải thích: NumPy có hàm tanh trực tiếp.

Q11. Which quantity should be subtracted from logits before stable softmax exponentiation? | Nên trừ gì khỏi logit trước khi tính exp softmax ổn định?

A. `np.tanh(x)` | `np.tanh(x)`

B. `np.maximum(0,x)` | `np.maximum(0,x)`

C. The largest logit | Logit lớn nhất

D. Tanh | Tanh

Đáp án: C

Giải thích: Trừ cực đại giữ các số exp không vượt một.

Q12. Which activation outputs exactly 0 at input 0? | Hàm nào trả đúng 0 với đầu vào 0?

A. The largest logit | Logit lớn nhất

B. `np.tanh(x)` | `np.tanh(x)`

C. `np.maximum(0,x)` | `np.maximum(0,x)`

D. Tanh | Tanh

Đáp án: D

Giải thích: Tanh(0)=0, còn sigmoid(0)=0.5.

Q13. What is sigmoid(0)? | sigmoid(0) bằng gì?

A. A negative output | Giá trị đầu ra âm

B. It reduces overflow risk without changing the result | Giảm nguy cơ tràn số mà không đổi kết quả

C. 0.5 | 0.5

D. 0 | 0

Đáp án: C

Giải thích: Sigmoid tại zero bằng một nửa.

Q14. What is ReLU(−2)? | ReLU(−2) bằng gì?

A. A negative output | Giá trị đầu ra âm

B. It reduces overflow risk without changing the result | Giảm nguy cơ tràn số mà không đổi kết quả

C. 0.5 | 0.5

D. 0 | 0

Đáp án: D

Giải thích: ReLU chặn đầu vào âm.

Q15. What does tanh produce for negative input? | Tanh tạo đầu ra thế nào khi đầu vào âm?

A. A negative output | Giá trị đầu ra âm

B. 0.5 | 0.5

C. 0 | 0

D. It reduces overflow risk without changing the result | Giảm nguy cơ tràn số mà không đổi kết quả

Đáp án: A

Giải thích: Tanh giữ dấu của đầu vào hữu hạn khác 0.

Q16. Why does subtracting max(logits) help softmax? | Vì sao trừ cực đại logit giúp softmax?

A. 0.5 | 0.5

B. It reduces overflow risk without changing the result | Giảm nguy cơ tràn số mà không đổi kết quả

C. 0 | 0

D. A negative output | Giá trị đầu ra âm

Đáp án: B

Giải thích: Dịch đều các logit không đổi softmax nhưng giảm tràn số.

Q17. Which output structure suits mutually exclusive multi-class prediction? | Dạng đầu ra nào phù hợp phân loại nhiều lớp loại trừ nhau?

A. Their inputs remain in the nonpositive flat region | Đầu vào luôn ở vùng phẳng không dương

B. No, calibration needs checking | Không, cần đánh giá độ hiệu chuẩn

C. Softmax couples scores so they sum to one | Softmax ràng buộc điểm các lớp có tổng một

D. A softmax distribution over classes | Phân phối softmax trên các lớp

Đáp án: D

Giải thích: Softmax thường dùng cho đầu ra các lớp loại trừ nhau.

Q18. Why might ReLU neurons stop updating? | Vì sao một số nơ-ron ReLU có thể ngừng cập nhật?

A. A softmax distribution over classes | Phân phối softmax trên các lớp

B. No, calibration needs checking | Không, cần đánh giá độ hiệu chuẩn

C. Softmax couples scores so they sum to one | Softmax ràng buộc điểm các lớp có tổng một

D. Their inputs remain in the nonpositive flat region | Đầu vào luôn ở vùng phẳng không dương

Đáp án: D

Giải thích: Độ dốc ReLU bằng không ở đầu vào âm.

Q19. Do sigmoid outputs automatically prove calibrated probabilities? | Đầu ra sigmoid có mặc nhiên là xác suất đã hiệu chuẩn không?

A. Softmax couples scores so they sum to one | Softmax ràng buộc điểm các lớp có tổng một

B. No, calibration needs checking | Không, cần đánh giá độ hiệu chuẩn

C. Their inputs remain in the nonpositive flat region | Đầu vào luôn ở vùng phẳng không dương

D. A softmax distribution over classes | Phân phối softmax trên các lớp

Đáp án: B

Giải thích: Điểm số cần kiểm tra với kết quả thực tế.

Q20. Which difference distinguishes softmax from independent sigmoid outputs? | Điểm nào phân biệt softmax với sigmoid độc lập?

A. A softmax distribution over classes | Phân phối softmax trên các lớp

B. Their inputs remain in the nonpositive flat region | Đầu vào luôn ở vùng phẳng không dương

C. No, calibration needs checking | Không, cần đánh giá độ hiệu chuẩn

D. Softmax couples scores so they sum to one | Softmax ràng buộc điểm các lớp có tổng một

Đáp án: D

Giải thích: Các sigmoid độc lập không bắt buộc có tổng bằng một.

8. Bài tiếp theo

`Hàm sô´→ Activation → Neural Network → Classification.`

PY-30 — Loss: MSE, MAE, Huber, cross-entropy và KL

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: MSE — sai số bình phương trung bình; **MAE** — sai số tuyệt đối trung bình.

`Dự đoán → Sai số → Loss → Tối ưu hóa`

Học xong dùng để làm gì? Đo sai số dự đoán bằng MSE hoặc loss thích hợp và đặt mục tiêu cần tối ưu.

Kết nối kiến thức tiếp theo: PY-31 tính đạo hàm để biết loss thay đổi ra sao khi chỉnh tham số.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
MSE	sai số bình phương trung bình	Trung bình bình phương hiệu dự đoán và giá trị thật
MAE	sai số tuyệt đối trung bình	Trung bình trị tuyệt đối sai số
Cross-entropy	entropy chéo	Phạt dự đoán xác suất sai nhãn mạnh hơn
KL divergence	độ lệch KL	Đo sai khác một chiều giữa hai phân phối xác suất

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`np.mean((y-pred)**2)`	MSE nhạy với sai số lớn do bình phương
`np.mean(np.abs(y-pred))`	MAE ít khuếch đại outlier hơn MSE
`np.where(abs_err<=delta,... )`	Huber phối hợp bình phương và tuyến tính
`-np.sum(y*np.log(probs))`	Cross-entropy trên xác suất có clip để tránh log(0)

4. Khái niệm liên quan

Nội dung chính: loss đo mức sai giữa dự báo và mục tiêu. Bài học đi từ hai cách tính sai lệch trên số (**MSE**, **MAE**), tới cách kết hợp hai mức phạt (**Huber**), rồi sang loss trên **xác suất và phân phối** (**cross-entropy**, **KL**). Chọn loss phải dựa vào ý nghĩa dữ liệu và mức phạt cần thiết.

4.1. MSE — Sai số bình phương trung bình

Từ vựng trong mục: MSE — Mean Squared Error; **Residual** — sai lệch.

Loss function là hàm cho biết dự báo sai lệch đến mức nào so với mục tiêu, để ta đánh giá hoặc tối ưu mô hình. **MSE** lấy sai số `(dự báo−thực tế)`, bình phương từng sai số rồi **trung bình**. Với thực tế `[2,3]` và dự báo `[1,4]`, sai số là `[-1,+1]`, bình phương thành `[1,1]` và MSE bằng 1. Đơn vị MSE là **bình phương đơn vị đầu ra**, không phải cùng đơn vị như giá nhà.

No Note — Không chú thích

```
import numpy as np

y_true = np.array([2, 3])
y_pred = np.array([1, 4])
print(np.mean((y_true - y_pred) ** 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính MSE bằng cách bình phương sai lệch từng dự đoán rồi lấy trung bình.
# Dữ liệu: Hai giá trị thật [2,3] và dự đoán [1,4] theo cùng thứ tự, cùng đơn vị giả lập.
# NumPy thực hiện phép trừ, lũy thừa và trung bình trên mảng.
import numpy as np

# Vector giá trị thật của hai mẫu.
y_true = np.array([2, 3])
# Vector dự đoán tương ứng từng mẫu, không thay đổi thứ tự.
y_pred = np.array([1, 4])
# Trừ THEO TỪNG VỊ TRÍ tạo vector lỗi, dấu âm/dương không ảnh hưởng khi bình phương.
# **2 bình phương lỗi từng mẫu để lỗi lớn bị phạt nặng hơn; đơn vị thành đơn vị đầu ra bình phương.
# mean() tính trung bình của hai lỗi bình phương, đó là MSE; không phải tổng hoặc căn bậc hai.
print(np.mean((y_true - y_pred) ** 2))
```

Kết quả khi chạy:

```
1.0
```

MSE bằng 1 đơn vị bình phương. Sai số lớn được phạt mạnh hơn vì bình phương.

4.2. MAE — Sai số tuyệt đối trung bình

Từ vựng trong mục: MAE — Mean Absolute Error; **Absolute error** — sai số tuyệt đối.

MAE lấy trị tuyệt đối của từng sai số rồi chia cho số quan sát. Ví dụ thực tế `[10,20,30]` và dự báo `[9,19,50]` có sai lệch tuyệt đối `[1,1,20]`, nên MAE bằng `22/3≈7.333`. Khác MSE, MAE có **cùng đơn vị** với đầu ra và không bình phương để khuếch đại lỗi lớn. Nó không có nghĩa là loại bỏ ảnh hưởng của ngoại lệ hoàn toàn.

No Note — Không chú thích

```
import numpy as np

y_true = np.array([10, 20, 30])
y_pred = np.array([9, 19, 50])
print(round(float(np.mean(np.abs(y_true - y_pred))), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính MAE: lấy trị tuyệt đối từng lỗi rồi trung bình, ít khuếch đại lỗi lớn hơn MSE.
# Dữ liệu: Ba giá trị thật [10,20,30] và đoán [9,19,50], lỗi mẫu cuối lớn hơn nhiều.
```

```
# NumPy có abs() cho trị tuyệt đối và mean() cho trung bình.
import numpy as np

# Vector ba giá trị thật cùng thang với dự đoán.
y_true = np.array([10, 20, 30])
# Vector ba dự đoán cùng vị trí với y_true.
y_pred = np.array([9, 19, 50])
# Tính lỗi theo từng mẫu, chênh lệch dương/âm đều là sai số.
# np.abs() bỏ dấu âm của mỗi lỗi, nên lỗi không tự triệt tiêu khi cộng.
# mean() lấy trung bình độ lớn lỗi, kết quả là MAE cùng đơn vị với giá trị gốc.
# float() đổi NumPy scalar thành float Python; round(...,3) làm tròn kết quả hiển thị 3 chữ số lẻ.
print(round(float(np.mean(np.abs(y_true - y_pred))), 3))
```

Kết quả khi chạy:

```
7.333
```

Sai lệch trung bình tuyệt đối khoảng 7,333 đơn vị đo.

4.3. Huber — Trộn bình phương và tuyến tính theo ngưỡng

Từ vựng trong mục: Huber loss — tổn thất Huber; **Delta** — ngưỡng.

Huber dùng mức phạt giống **bình phương** khi $|e| \le \delta$ và chuyển sang **tuyến tính** khi lỗi vượt δ. Với δ=1: lỗi có độ lớn 1 mất 0.5, độ lớn 2 mất 1.5, độ lớn 5 mất 4.5. Trung bình ba lỗi `[-1,2,5]` là $6.5/3 \approx 2.167$. Huber là cách cân bằng giữa MSE nhạy với lỗi lớn và MAE tăng tuyến tính; δ quyết định điểm chuyển và mang cùng đơn vị với sai số.

No Note — Không chú thích

```
import numpy as np

e = np.array([-1.0, 2.0, 5.0])
delta = 1.0
loss = np.where(
    np.abs(e) <= delta,
    0.5 * e ** 2,
    delta * (np.abs(e) - 0.5 * delta),
)
print(round(float(loss.mean()), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa Huber loss: lỗi nhỏ bình phương, lỗi lớn tăng tuyến tính theo ngưỡng δ.
# Dữ liệu: Ba lỗi [-1,2,5] đơn vị giả lập; delta=1 quy định điểm chuyển chế độ.
# NumPy np.where() cho phép chọn công thức khác nhau cho từng lỗi trong mảng.
import numpy as np

# Ba số lỗi có dấu; np.abs(e) dùng độ lớn để phân biệt ngưỡng.
e = np.array([-1.0, 2.0, 5.0])
# delta (δ) là ngưỡng chuyển từ vùng bình phương sang tuyến tính, cùng đơn vị lỗi.
delta = 1.0
# where(điều_kiện,giá_trị_nếu_đúng,giá_trị_nếu_sai) trả phần tử phù hợp từng vị trí, không phải một
if duy nhất cho cả mảng.
loss = np.where(
    # abs() lấy độ lớn lỗi; <= delta kiểm lỗi nhỏ/đúng ngưỡng.
    np.abs(e) <= delta,
    # Với lỗi nhỏ, loss bằng nửa bình phương lỗi để hàm mượt gần 0.
    0.5 * e ** 2,
```

```
    # Với lỗi lớn, loss tăng tuyến tính theo độ lớn, bớt bị ngoại lệ kéo mạnh hơn MSE.
    delta * (np.abs(e) - 0.5 * delta),
)
# mean() lấy Huber trung bình ba lỗi; float() chuyển scalar, round(...,3) chỉ làm tròn để in.
print(round(float(loss.mean()), 3))
```

Kết quả khi chạy:

```
2.167
```

Tổn thất trung bình xấp xỉ 2,167 theo Huber δ=1; lỗi lớn không bị bình phương mãi.

4.4. Cross-entropy và KL — So nhãn hoặc phân phối

Từ vựng trong mục: Cross-entropy — entropy chéo; **KL divergence** — phân kỳ KL.

Cross-entropy đánh giá phân phối dự báo so với phân phối nhãn/đích. Với phân loại một nhãn đúng, dự báo xác suất cho nhãn đúng là 0.8 thì loss bằng $-\ln(0.8)\approx 0.2231$: xác suất cho nhãn đúng càng thấp, phạt càng lớn. **KL divergence** $D_{KL}(P||Q)=\Sigma\ P_i\ \ln(P_i/Q_i)$ so sự khác nhau giữa **hai phân phối** P và Q; với $P=[0.5,0.5]$, $Q=[0.6,0.4]$, KL≈0.0204. KL **không đối xứng** và không phải khoảng cách hình học. Các xác suất trong log cần dương; trong code thực tế có thể clip hoặc xử lý giá trị 0 đúng quy ước.

No Note — Không chú thích

```
import numpy as np

p_true = 0.8
P = np.array([0.5, 0.5])
Q = np.array([0.6, 0.4])
print(round(float(-np.log(p_true)), 4))
print(round(float(np.sum(P * np.log(P / Q))), 4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So negative log-likelihood cho nhãn đúng với độ lệch KL giữa hai phân phối.
# Dữ liệu: p_true=0.8 là xác suất gán cho nhãn thật; P=[0.5,0.5], Q=[0.6,0.4] là hai phân phối 2
lớp.
# NumPy log() là log tự nhiên, sum() tổng hợp phần tử.
import numpy as np

# Xác suất mô hình cấp cho đúng nhãn (không phải nhãn 0/1).
p_true = 0.8
# P là phân phối mục tiêu hai lớp, tổng xác suất bằng 1.
P = np.array([0.5, 0.5])
# Q là phân phối so sánh hai lớp, khác P nhưng cũng tổng bằng 1.
Q = np.array([0.6, 0.4])
# -log(p) là loss khi nhãn đúng có xác suất p; xác suất thấp hơn thì loss lớn hơn.
# float() đổi NumPy scalar thành float Python; round(...,4) giới hạn bốn chữ số thập phân khi in.
print(round(float(-np.log(p_true)), 4))
# P/Q lấy tỷ lệ từng lớp; log() là log tự nhiên; nhân P là đóng góp KL(P||Q) của từng lớp.
# np.sum() cộng đóng góp các lớp để ra KL(P||Q), đơn vị nat; thứ tự P/Q quan trọng và KL không đối
xứng.
# float() đổi NumPy scalar thành float Python; round(...,4) giới hạn bốn chữ số thập phân khi in.
print(round(float(np.sum(P * np.log(P / Q))), 4))
```

Kết quả khi chạy:

```
0.2231
0.0204
```

Dòng đầu là cross-entropy của nhãn đúng; dòng sau là KL từ phân phối P sang Q. Hai số mang ý nghĩa khác nhau.

5. Năm tình huống Loss — từ MSE đến entropy chéo và KL trên nhiều mẫu

Tình huống 1 — Cấp 1: Đánh giá dự báo 10 chuyến giao hàng bằng MSE

Từ vựng trong mục: MSE — trung bình bình phương sai số; **Residual** — sai lệch dự báo.

Tình huống thực tế: Đơn vị giao hàng lưu **10 chuyến**, mỗi chuyến có một thời gian giao **thực tế** và một thời gian do hệ thống **dự đoán**, cùng đơn vị **phút/chuyến**. Ví dụ chuyến đầu thực tế 20 phút nhưng dự đoán 21 phút. Người quản lý muốn biết mô hình dự báo lệch bao nhiêu xét chung cả 10 chuyến bằng **MSE**, và chuyến nào đó có độ lệch tuyệt đối lớn nhất bao nhiêu phút; chưa cần điều chỉnh mô hình.

Vì sao chọn cách này? Loss (hàm mất mát) là cách đo mức dự đoán **khác kết quả thật** để đánh giá mô hình. Trước tiên chương trình trừ thời gian thực tế khỏi dự đoán của **từng chuyến**; chênh lệch có thể âm hoặc dương. **MSE** lấy bình phương từng chênh lệch rồi tính trung bình nên các sai lệch lớn ảnh hưởng mạnh hơn; đơn vị kết quả là **phút bình phương (phút²)**, không phải phút giao hàng. `np.abs()` bỏ dấu âm khi tìm **sai số tuyệt đối lớn nhất**, giá trị này mới có đơn vị phút. Không thể nhìn MSE rồi nói trung bình mô hình sai từng ấy phút.

Quy trình:

1. Tạo hai dãy 10 thời gian theo cùng thứ tự chuyến.
2. Trừ hai dãy để nhận sai số, bình phương rồi lấy trung bình.
3. In MSE và sai lệch tuyệt đối lớn nhất để đối chiếu.

No Note — Không chú thích

```
import numpy as np
actual_minutes = np.array([20,23,25,28,30,32,34,35,37,40], dtype=float)
predicted_minutes = np.array([21,22,24,30,31,30,36,35,39,38], dtype=float)
errors = predicted_minutes - actual_minutes
if actual_minutes.shape != (10,) or predicted_minutes.shape != (10,):
    raise ValueError("Cần đủ 10 thời gian thực tế´và dự đoán.")
print("Chuyến:", len(actual_minutes))
print(f"MSE (phút²): {np.mean(errors ** 2):.2f}")
print("Sai sô´lớn nhất (phút):", int(np.max(np.abs(errors))))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính MSE thời gian giao hàng 10 chuyến và độ lệch lớn nhất tính bằng phút.
# Dữ liệu: 10 thời gian thực tế´và dự đoán (phút), môĩ ô cùng vị trí là một chuyến.
# NumPy hô˜trợ tính lôĩ trên hai vector thời gian.
import numpy as np
# 10 thời gian thực tế´tính bằng phút, dtype=float.
actual_minutes = np.array([20,23,25,28,30,32,34,35,37,40], dtype=float)
# 10 thời gian dự đoán phút theo cùng thứ tự chuyến.
predicted_minutes = np.array([21,22,24,30,31,30,36,35,39,38], dtype=float)
# Trừ THEO VỊ TRÍ để ̉ra lôĩ có dâú (phút) của từng chuyến.
```

```
errors = predicted_minutes - actual_minutes
# shape (10,) xác minh hai vector đều có 10 mẫu.
if actual_minutes.shape != (10,) or predicted_minutes.shape != (10,):
    # ValueError báo dữ liệu thiếu, sai kích thước hoặc vi phạm điều kiện hợp lệ.
    raise ValueError("Cần đủ 10 thời gian thực tế´và dự đoán.")
print("Chuyến:", len(actual_minutes))
# Bình phương lỗi làm đơn vị thành phút², phạt nặng chuyến dự đoán sai nhiều.
# mean() lấy lỗi bình phương trung bình trên 10 chuyến, chính là MSE phút²; :.2f in 2 số´thập phân.
print(f"MSE (phút²): {np.mean(errors ** 2):.2f}")
# abs() bỏ dấu trước khi chọn độ lệch lớn nhất, tránh bỏ sót lỗi âm lớn.
# max() lấy lỗi tuyệt đối lớn nhất (phút); int() chuyển scalar về`số´nguyên Python.
print("Sai số´lớn nhất (phút):", int(np.max(np.abs(errors))))
```

Kết quả khi chạy:

```
Chuyến: 10
MSE (phút²): 2.40
Sai số´lớn nhất (phút): 2
```

MSE tính trên **cùng 10 chuyến**; không được trộn sai thứ tự thực tế và dự báo khi đánh giá.

Tình huống 2 — Cấp 2: So MAE và MSE khi 12 số đo có một ngoại lệ lớn

Từ vựng trong mục: MAE — sai số tuyệt đối trung bình; **Outlier** — ngoại lệ; **Comparison** — so sánh.

Tình huống thực tế: Phòng kiểm định có **12 lần cân**, mỗi lần gồm khối lượng thực tế và dự đoán, tính bằng **gam (g)**. Mười một lần dự đoán lệch khoảng **2 g**, nhưng ở lần thứ chín hệ thống dự đoán 230 g thay vì 180 g, lệch **50 g**. Người kiểm định muốn so mức sai lệch khi tính bằng **MAE** với **MSE** để hiểu vì sao một lần đo bị lỗi lớn có thể ảnh hưởng mạnh tới đánh giá chung.

Vì sao chọn cách này? MAE là trung bình **độ lớn sai lệch**, không quan tâm dự báo cao hay thấp hơn thực tế; kết quả vẫn tính bằng **gam**. **MSE** bình phương sai lệch trước khi lấy trung bình nên trường hợp **50 g** đóng góp mạnh hơn nhiều so với mỗi lượt lệch 2 g; kết quả có đơn vị **g²**. Chương trình tính cả hai trên **cùng 12 lượt**, đồng thời đếm lượt sai hơn 10 g. Hai con số MAE và MSE **khác đơn vị**, nên không so số lớn nhỏ của chúng để chọn mô hình tốt hơn; phải hiểu mục đích phạt lỗi và dùng cùng thước đo khi so các mô hình.

Quy trình:

1. Tạo 12 cặp số đo–dự báo, có đúng một ngoại lệ 50 g.
2. Tính sai số tuyệt đối và bình phương trên từng hàng.
3. Báo MAE, MSE và số điểm có lỗi lớn hơn 10 g.

No Note — Không chú thích

```
import numpy as np
actual_g = np.array([100,110,120,130,140,150,160,170,180,190,200,210], dtype=float)
predicted_g = np.array([102,108,122,128,142,148,162,168,230,188,202,208], dtype=float)
e = predicted_g - actual_g
if len(e) != 12:
    raise ValueError("Cần đúng 12 lỗi cân đo.")
print("Lượt đo:", len(e))
print(f"MAE (g): {np.mean(np.abs(e)):.2f}")
print(f"MSE (g²): {np.mean(e ** 2):.2f}")
print("Lỗi > 10 g:", int((np.abs(e) > 10).sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So MAE và MSE của 12 lượt cân, đếm sai sô´cân vượt 10 g.
# Dữ liệu: 12 khôí lượng thực tê´và dự đoán tính gram; một dự đoán 230 g tạo lôĩ lớn bất thường.
# NumPy cho vector lôĩ, abs(), mean() và mask.
import numpy as np
# 12 khôí lượng thật (gram), dtype=float.
actual_g = np.array([100,110,120,130,140,150,160,170,180,190,200,210], dtype=float)
# 12 giá trị cân dự đoán (gram), theo đúng vị trí actual_g.
predicted_g = np.array([102,108,122,128,142,148,162,168,230,188,202,208], dtype=float)
# Trừ từng cặp đểlấý sai sô´có dâú theo gram.
e = predicted_g - actual_g
# len() đêḿ 12 lượt cân, tránh dùng thông kê trên dữ liệu thiếú.
if len(e) != 12:
    # ValueError báo dữ liệu thiếú, sai kích thước hoặc vi phạm điêù kiện hợp lệ.
    raise ValueError("Cần đúng 12 lôĩ cân đo.")
# len() đêḿ 12 lượt cân, tránh dùng thông kê trên dữ liệu thiếú.
print("Lượt đo:", len(e))
# abs() lâý độ lớn lôĩ mà không bị lôĩ âm và dương triệt tiêu.
# MAE là trung bình độ lớn lôĩ, giữ đơn vị g; :.2f định dạng sô´.
print(f"MAE (g): {np.mean(np.abs(e)):.2f}")
# MSE lâý trung bình bình phương lôĩ, đơn vị g²; nhạy hơn với lôĩ 230 g bất thường.
print(f"MSE (g²): {np.mean(e ** 2):.2f}")
# abs() lâý độ lớn lôĩ mà không bị lôĩ âm và dương triệt tiêu.
# Tạo mask những sai lệch lớn hơn 10 g, sum() Boolean đêḿ lượt vượt ngưỡng.
print("Lôĩ > 10 g:", int((np.abs(e) > 10).sum()))
```

Kết quả khi chạy:

```
Lượt đo: 12
MAE (g): 6.00
MSE (g²): 212.00
Lôĩ > 10 g: 1
```

MAE không loại bỏ sự cố ngoại lệ; nó chỉ **không bình phương** sự cố ấy như MSE.

Tình huống 3 — Cấp 3: Dùng Huber cho 15 sai lệch cảm biến có hai lỗi bất thường

Từ vựng trong mục: Huber — hàm mất mát lai; **Delta** — ngưỡng; **Robustness** — giảm độ nhạy với ngoại lệ.

Tình huống thực tế: Một trạm dự báo nhiệt độ ghi **15 sai lệch**, mỗi số là nhiệt độ dự đoán trừ giá trị chuẩn, đơn vị **°C**. Hầu hết sai số nhỏ nhưng có hai lượt lệch **+8 °C** và **-7 °C**. Đội kỹ thuật chọn **mức chuyển δ = 1 °C** trong phép đo Huber minh họa, muốn đếm số lần sai lệch vượt ngưỡng này và xem Huber trung bình khác MAE ra sao trên đúng 15 lượt.

Vì sao chọn cách này? Huber loss là một cách phạt sai lệch có hai vùng: khi sai lệch nhỏ thì phạt theo kiểu **bình phương** để nhạy với việc cải thiện gần đúng, còn khi sai lệch lớn thì chuyển sang tăng **gần tuyến tính** để ngoại lệ không chi phối mạnh như MSE. **δ = 1 °C** là ngưỡng chuyển giữa hai cách phạt do bài tập chọn, không phải ngưỡng an toàn của trạm đo. `np.abs(errors_c)` lấy độ lớn 15 sai số; `np.where(...)` chọn mức phạt tương ứng **cho từng lượt**, kể cả hai lượt +8 °C và -7 °C. Huber trung bình và MAE dùng cách phạt khác nhau, nên cần ghi tên phương pháp khi báo kết quả; không coi số Huber là nhiệt độ trung bình của cảm biến.

Quy trình:

1. Lưu 15 sai số có dấu và đặt ngưỡng Huber δ=1.
2. Áp từng nhánh theo trị tuyệt đối của sai số bằng `np.where`.
3. In Huber, MAE và số lượt có trị lỗi vượt δ.

No Note — Không chú thích

```
import numpy as np
errors_c = np.array([-1,1,-2,0,0.5,-0.5,1.5,-1.5,2,0,0.2,-0.2,8,-7,1], dtype=float)
delta = 1.0
abserr = np.abs(errors_c)
huber = np.where(abserr <= delta,
                 0.5 * errors_c**2,
                 delta * (abserr - 0.5 * delta))
if errors_c.size != 15:
    raise ValueError("Cần đúng 15 sai số́nhiệt độ.")
print("Số́đo:", len(errors_c))
print("Lỗi > δ:", int((abserr > delta).sum()))
print(f"Huber trung bình: {huber.mean():.3f}")
print(f"MAE (°C): {abserr.mean():.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So Huber loss với MAE của 15 sai số́nhiệt độ, ngưỡng chuyển chế́độ δ=1 °C.
# Dữ liệu: 15 sai số́nhiệt độ °C có dấú; có hai lỗi lớn +8 và -7 °C.
# NumPy np.where() tính loss theo điều kiện từng phần tử.
import numpy as np
# 15 sai số́nhiệt độ °C gồm các lỗi nhỏ và ngoại lệ lớn.
errors_c = np.array([-1,1,-2,0,0.5,-0.5,1.5,-1.5,2,0,0.2,-0.2,8,-7,1], dtype=float)
# Ngưỡng δ=1 °C quyết định khi nào chuyển từ mất mát bậc hai sang tuyến tính.
delta = 1.0
# abs() lấy độ lớn lỗi, bỏ dấú trước khi so δ.
abserr = np.abs(errors_c)
# where() chọn công thức bình phương khi |lỗi|<=δ, ngược lại chọn nhánh tuyến tính.
huber = np.where(abserr <= delta,
                 # Nhánh nhỏ = 0.5×lỗi², phạt mượt gần 0.
                 0.5 * errors_c**2,
                 # Nhánh lớn tăng theo |lỗi| thay vì bình phương, giảm ảnh hưởng của ngoại lệ ±8/7
°C.
                 delta * (abserr - 0.5 * delta))
# size đếm đúng 15 đầu vào.
if errors_c.size != 15:
    # ValueError báo dữ liệu thiếú, sai kích thước hoặc vi phạm điều kiện hợp lệ.
    raise ValueError("Cần đúng 15 sai số́nhiệt độ.")
print("Số́đo:", len(errors_c))
# So |lỗi| với δ, sum() trên mask đếm số́sai số́vượt ngưỡng.
print("Lỗi > δ:", int((abserr > delta).sum()))
# mean() lấy Huber trung bình; đơn vị giá trị loss theo cách định nghĩa của ngưỡng δ, không trực
tiếp là MAE.
print(f"Huber trung bình: {huber.mean():.3f}")
# Trung bình lỗi tuyệt đối là MAE có đơn vị °C, dùng so cách đánh giá khác Huber.
print(f"MAE (°C): {abserr.mean():.3f}")
```

Kết quả khi chạy:

```
Số́đo: 15
Lỗi > δ: 6
Huber trung bình: 1.386
MAE (°C): 1.760
```

Ví dụ chỉ minh họa lựa chọn loss; dữ liệu thực cần điều chỉnh δ dựa trên thang đo và mục tiêu học máy.

Tình huống 4 — Cấp 4: Cross-entropy của 18 nhãn trong bài toán ba lớp

Từ vựng trong mục: Cross-entropy — entropy chéo; **True-class probability** — xác suất lớp đúng; **Clipping** — chặn biên số.

Tình huống thực tế: Nhóm phân loại ảnh có **18 ảnh thử**, mỗi ảnh được gắn một **nhãn đúng** thuộc A, B hoặc C và một **điểm mô hình gán cho chính lớp đúng**, nằm trong khoảng 0 đến 1. Ở ảnh thứ sáu và mười lăm, điểm dành cho lớp đúng rất nhỏ (**0,05** và **0,01**). Người phân tích muốn tính **cross-entropy trung bình**, tìm ảnh bị phạt nặng nhất và tránh lỗi máy tính nếu một mô hình khác gán điểm bằng 0.

Vì sao chọn cách này? Cross-entropy trong bài đo mức mô hình **không tin vào lớp thật** đến đâu: điểm của lớp đúng càng nhỏ thì **mức phạt càng lớn**. Ví dụ ảnh có lớp thật là B mà mô hình chỉ gán điểm 0,01 cho B sẽ bị phạt nặng hơn ảnh có điểm 0,75 cho lớp đúng. Chương trình dùng `-np.log(...)` để đổi điểm của lớp đúng thành loss, rồi lấy trung bình **18 ảnh**; `argmax()` tìm **vị trí ảnh bị phạt lớn nhất**. `np.clip(..., 1e-12, 1)` giới hạn giá trị trước khi lấy log vì `log(0)` không xác định; đây là **bảo vệ tính toán**, không phải cách sửa một dự báo kém thành dự báo đúng. Nhãn 0/1/2 dùng để nhận diện lớp, không phải số tiền hay điểm số.

Quy trình:

1. Tạo 18 nhãn đúng và điểm của lớp đúng tương ứng.
2. Clip điểm trong khoảng an toàn rồi lấy `-log`.
3. Tìm mất mát trung bình và mã mẫu có loss lớn nhất.

No Note — Không chú thích

```
import numpy as np
labels = np.array([0,1,2,0,1,2,0,1,2,0,1,2,0,1,2,0,1,2], dtype=int)
true_class_scores =
np.array([0.55,0.6000000000000001,0.65,0.7000000000000001,0.75,0.05,0.6000000000000001,0.65,0.70000
00000000001,0.75,0.55,0.6000000000000001,0.65,0.7000000000000001,0.01,0.55,0.6000000000000001,0.65]
, dtype=float)
losses = -np.log(np.clip(true_class_scores, 1e-12, 1.0))
worst = int(losses.argmax())
if labels.shape != (18,) or true_class_scores.shape != (18,):
    raise ValueError("Cần 18 nhãn và 18 xác suất của nhãn đúng.")
if not np.all((labels >= 0) & (labels < 3)):
    raise ValueError("Nhãn chỉ được là 0, 1 hoặc 2.")
print("Mẫu:", len(labels))
print(f"Cross-entropy TB: {losses.mean():.3f}")
print("Loss lớn nhất:", f"M{worst+1:02d}")
print(f"Loss mẫu lớn nhất: {losses[worst]:.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính cross-entropy cho 18 dự đoán xác suất nhãn đúng và tìm mẫu gây mất mát lớn nhất.
# Dữ liệu: 18 nhãn thật 0/1/2, mỗi mẫu đi kèm xác suất riêng gán cho CHÍNH nhãn đúng, hai xác suất
rất thấp 0.05 và 0.01.
# NumPy log(), clip() và argmax() xử lý vector mất mát.
import numpy as np
# Vector nhãn 0/1/2 của 18 mẫu, dtype=int.
labels = np.array([0,1,2,0,1,2,0,1,2,0,1,2,0,1,2,0,1,2], dtype=int)
# 18 xác suất mô hình gán cho nhãn ĐÚNG tương ứng từng mẫu, chưa phải ma trận 3 lớp.
true_class_scores =
np.array([0.55,0.6000000000000001,0.65,0.7000000000000001,0.75,0.05,0.6000000000000001,0.65,0.70000
00000000001,0.75,0.55,0.6000000000000001,0.65,0.7000000000000001,0.01,0.55,0.6000000000000001,0.65]
, dtype=float)
# clip(min,max) chặn xác suất bên dưới 1e-12 để tránh log(0)=-inf và phía trên 1 để giữ miền xác
```

```
suất hợp lệ.
# -log(p_nhãn_đúng) tạo loss từng mẫu; xác suất thấp như 0.01 bị phạt lớn, log là log tự nhiên.
losses = -np.log(np.clip(true_class_scores, 1e-12, 1.0))
# argmax() trả CHỈ SỐ´0-based của mẫu có loss lớn nhất, không phải giá trị loss; int() đổi scalar.
worst = int(losses.argmax())
# shape (18,) của cả labels/scores bảo đảm cặp nhãn/xác suất không mất mẫu.
if labels.shape != (18,) or true_class_scores.shape != (18,):
    # ValueError báo dữ liệu thiếú, sai kích thước hoặc vi phạm điều kiện hợp lệ.
    raise ValueError("Cần 18 nhãn và 18 xác suất của nhãn đúng.")
# & so Boolean theo từng ô, chỉ cho nhãn 0,1,2; np.all() bắt mọi ô hợp lệ.
if not np.all((labels >= 0) & (labels < 3)):
    # ValueError báo dữ liệu thiếú, sai kích thước hoặc vi phạm điều kiện hợp lệ.
    raise ValueError("Nhãn chỉ được là 0, 1 hoặc 2.")
print("Mẫũ:", len(labels))
# mean() lấý mất mát cross-entropy trung bình trên 18 mẫũ; :.3f in ba sô´lẻ.
print(f"Cross-entropy TB: {losses.mean():.3f}")
# worst là vị trí bắt đầù 0, +1 đổi thành sô´thứ tự mẫũ, :02d thêm 0 đầù mã.
print("Loss lớn nhất:", f"M{worst+1:02d}")
# Tra loss mẫũ sai nhất bằng chỉ sô´argmax đã tìm.
print(f"Loss mẫũ lớn nhất: {losses[worst]:.3f}")
```

Kết quả khi chạy:

```
Mẫũ: 18
Cross-entropy TB: 0.823
Loss lớn nhất: M15
Loss mẫũ lớn nhất: 4.605
```

Điểm softmax chưa chắc là xác suất đã hiệu chuẩn. Vì thế đây là đánh giá loss trên dữ liệu minh họa, không phải chứng minh chất lượng mô hình.

Tình huống 5 — Cấp 5: Kiểm KL divergence của 20 phân phối dự báo và phân biệt chiều so sánh

Từ vựng trong mục: KL divergence — phân kỳ KL; **Probability distribution** — phân phối; **Asymmetry** — không đối xứng.

Tình huống thực tế: Đội kiểm thử có **20 bộ dự báo ba lớp A/B/C**. Mỗi bộ gồm ba số dương có tổng bằng 1, gọi là phân phối **Q**; nhóm lấy một phân phối chuẩn **P = [0,5; 0,3; 0,2]** để đối chiếu. Trong 20 bộ có các dòng Q trùng P và các dòng lệch dần. Họ muốn tính độ khác nhau theo **KL(P||Q)**, xem khi hai phân phối giống nhau thì kết quả ra sao, và kiểm tra vì sao đổi thứ tự thành **KL(Q||P)** có thể cho kết quả khác.

Vì sao chọn cách này? Phân phối xác suất ở đây là ba mức tỷ lệ cho ba nhóm, mỗi số dương và **cả ba cộng lại bằng 1**. **KL divergence** đo mức khác nhau của Q **khi dùng P làm phân phối tham chiếu**, nhưng cách đo này **có hướng**: so P với Q không nhất thiết bằng so Q với P. Chương trình kiểm tra mọi Q có đủ ba giá trị dương, tổng từng hàng gần bằng 1, rồi tính KL cho **20 hàng độc lập**, không gộp 20 hàng thành một phân phối. Khi Q đúng bằng P, KL bằng **0**; kết quả lý thuyết không âm, sai số âm cực nhỏ có thể do máy tính. Không gọi KL là **khoảng cách hình học đối xứng** hay tự hiểu KL là tỷ lệ dự báo chính xác.

Quy trình:

1. Tạo P một hàng chuẩn và 20 phân phối Q đều hợp lệ.
2. Tính log theo từng cột, cộng theo hàng để có 20 giá trị `KL(P||Q)`.
3. Đối chiếu dòng Q=P, trung bình và hai chiều trên dòng khác P.

No Note — Không chú thích

```
import numpy as np
P = np.array([0.5,0.3,0.2], dtype=float)
Q = np.array([[0.5,0.3,0.2],[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002],[0.5,0.3,0.2],
[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002],[0.5,0.3,0.2],
[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002],[0.5,0.3,0.2],
[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002]], dtype=float)
if Q.shape != (20, 3) or not np.all(Q > 0):
    raise ValueError("Cần 20 phân phối ba nhãn có xác suất dương.")
if not np.allclose(Q.sum(axis=1), 1.0):
    raise ValueError("Tổng xác suất của mỗi phân phối phải gần bằng 1.")
kl_pq = np.sum(P * np.log(P / Q), axis=1)
kl_qp = np.sum(Q * np.log(Q / P), axis=1)
print("Bộ phân phối:", len(Q))
print(f"KL đầu (P||Q): {kl_pq[0]:.4f}")
print(f"KL TB (P||Q): {kl_pq.mean():.4f}")
print("Hai chiều có thể khác:", bool(abs(kl_pq[1] - kl_qp[1]) > 1e-9))
print("Không âm:", bool(np.all(kl_pq >= -1e-12)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So KL(P||Q) và KL(Q||P) trên 20 phân phối ba nhãn, kiểm tổng xác suất và tính không âm.
# Dữ liệu: P là phân phối chuẩn ba lớp [0.5,0.3,0.2]; Q gồm 20 hàng ba xác suất dương tổng 1, đơn
vị KL là nat.
# NumPy hỗ trợ log, phép toán broadcast, sum(axis) và allclose kiểm xác suất.
import numpy as np
# Vector phân phối tham chiếu ba lớp, tổng đúng 1; dtype=float.
P = np.array([0.5,0.3,0.2], dtype=float)
# 20 hàng, mỗi hàng phân phối ba lớp (Q_i), đã dựng để tổng 1; không phải 20 nhãn đơn.
Q = np.array([[0.5,0.3,0.2],[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002],[0.5,0.3,0.2],
[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002],[0.5,0.3,0.2],
[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002],[0.5,0.3,0.2],
[0.525,0.2875,0.1875],[0.55,0.27499999999999997,0.17500000000000002],
[0.575,0.26249999999999996,0.1625],[0.6,0.25,0.15000000000000002]], dtype=float)
# shape (20,3) kiểm đủ 20 bộ, mỗi bộ ba lớp.
# Q>0 tạo mask cho từng xác suất; np.all() buộc mọi xác suất dương để log(P/Q) hữu hạn.
if Q.shape != (20, 3) or not np.all(Q > 0):
    # ValueError báo dữ liệu thiếu, sai kích thước hoặc vi phạm điều kiện hợp lệ.
    raise ValueError("Cần 20 phân phối ba nhãn có xác suất dương.")
# axis=1 cộng các CỘT lớp cho TỪNG HÀNG, trả 20 tổng xác suất.
# allclose() kiểm mỗi hàng gần 1 với dung sai float, không dùng == tuyệt đối.
if not np.allclose(Q.sum(axis=1), 1.0):
    # ValueError báo dữ liệu thiếu, sai kích thước hoặc vi phạm điều kiện hợp lệ.
    raise ValueError("Tổng xác suất của mỗi phân phối phải gần bằng 1.")
# Broadcast P shape (3,) qua 20 hàng Q (20,3), chia xác suất từng lớp.
# log là log tự nhiên của tỷ số từng lớp; giá trị thành phần có thể âm.
# Nhân từng lớp với P rồi sum(axis=1) cho 20 KL(P||Q), một giá trị mỗi hàng, đơn vị nat.
kl_pq = np.sum(P * np.log(P / Q), axis=1)
# Đảo P/Q để tính KL(Q||P); KL nói chung KHÔNG đối xứng.
kl_qp = np.sum(Q * np.log(Q / P), axis=1)
print("Bộ phân phối:", len(Q))
# [0] là KL cho phân phối Q đầu tiên trùng P nên gần 0; :.4f in bốn số thập phân.
print(f"KL đầu (P||Q): {kl_pq[0]:.4f}")
# mean() trung bình KL(P||Q) trên 20 phân phối.
print(f"KL TB (P||Q): {kl_pq.mean():.4f}")
# abs() so chênh hai chiều ở bộ thứ hai; 1e-9 là dung sai tránh nhiễu float khi kiểm khác nhau.
print("Hai chiều có thể khác:", bool(abs(kl_pq[1] - kl_qp[1]) > 1e-9))
```

```
# KL lý thuyết >=0; cho sai số âm nhỏ -1e-12 do tính float, np.all() kiểm mọi bộ.
print("Không âm:", bool(np.all(kl_pq >= -1e-12)))
```

Kết quả khi chạy:

```
Bộ phân phối: 20
KL đầu (P||Q): 0.0000
KL TB (P||Q): 0.0078
Hai chiều có thể khác: True
Không âm: True
```

KL không đối xứng, và một giá trị nhỏ không tự nói hai phân phối phù hợp mục tiêu vận hành thật. Khi Q có xác suất 0, cần một quy ước xử lý riêng để tránh log không xác định.

6. Sai lầm thường gặp

- **MSE** có đơn vị đầu ra bình phương, **MAE** có đơn vị gốc; không đọc MSE như mức lệch tiền trung bình.
- Huber cần ngưỡng δ phù hợp đơn vị sai số; ngưỡng này quyết định khi nào chuyển từ phạt bình phương sang tuyến tính.
- Cross-entropy với nhãn đúng cần xác suất **của lớp đúng**, không phải điểm lớn nhất bất kỳ.
- `log(0)` không xác định; phải xử lý xác suất bằng công thức ổn định hoặc clip phù hợp khi dùng NumPy đơn giản.
- KL **bất đối xứng**, không phải khoảng cách Euclid và chỉ có nghĩa với các phân phối hợp lệ.
- Loss huấn luyện thấp không thay thế việc đánh giá trên tập validation/test độc lập.

7. Trắc nghiệm

Chỉ hỏi kiến thức lý thuyết, so sánh loss và đọc công thức, không hỏi dữ liệu tình huống.

Q1. What is MSE? | MSE là gì?

A. Mean absolute prediction error | Trung bình trị tuyệt đối sai số

B. Mean squared prediction error | Trung bình sai số dự báo bình phương

C. Compare two probability distributions in a directed way | So hai phân phối xác suất có phân biệt chiều

D. Quadratic for small errors and linear for large | Bình phương lỗi nhỏ, tuyến tính lỗi lớn

Đáp án: B

Giải thích: MSE lấy trung bình bình phương sai lệch.

Q2. What is MAE? | MAE là gì?

A. Mean squared prediction error | Trung bình sai số dự báo bình phương

B. Quadratic for small errors and linear for large | Bình phương lỗi nhỏ, tuyến tính lỗi lớn

C. Compare two probability distributions in a directed way | So hai phân phối xác suất có phân biệt chiều

D. Mean absolute prediction error | Trung bình trị tuyệt đối sai số

Đáp án: D

Giải thích: MAE lấy trung bình độ lệch tuyệt đối.

Q3. What characterizes Huber loss? | Đặc trưng của Huber loss là gì?

A. Mean absolute prediction error | Trung bình trị tuyệt đối sai số

B. Mean squared prediction error | Trung bình sai số dự báo bình phương

C. Compare two probability distributions in a directed way | So hai phân phối xác suất có phân biệt chiều

D. Quadratic for small errors and linear for large | Bình phương lỗi nhỏ, tuyến tính lỗi lớn

Đáp án: D

Giải thích: Huber đổi tốc độ tăng mức phạt sau một ngưỡng.

Q4. What is KL divergence used for? | KL divergence dùng để làm gì?

A. Mean absolute prediction error | Trung bình trị tuyệt đối sai số

B. Compare two probability distributions in a directed way | So hai phân phối xác suất có phân biệt chiều

C. Mean squared prediction error | Trung bình sai số dự báo bình phương

D. Quadratic for small errors and linear for large | Bình phương lỗi nhỏ, tuyến tính lỗi lớn

Đáp án: B

Giải thích: KL dùng tỷ số logarit theo một chiều giữa các phân phối.

Q5. Actual values are [2, 4] and predictions are [3, 2]. What is MSE? | Giá trị thật [2, 4], dự đoán [3, 2]. MSE bằng bao nhiêu?

A. 1.5 | 1.5

B. 3 | 3

C. 5 | 5

D. 2.5 | 2.5

Đáp án: D

Giải thích: Sai số là 1 và −2; bình phương là 1 và 4. MSE = (1 + 4)/2 = 2.5.

Q6. Actual values are [2, 4] and predictions are [3, 2]. What is MAE? | Giá trị thật [2, 4], dự đoán [3, 2]. MAE bằng bao nhiêu?

A. 0.5 | 0.5

B. 2.5 | 2.5

C. 3 | 3

D. 1.5 | 1.5

Đáp án: D

Giải thích: Lấy trị tuyệt đối các sai số: $|3 - 2| = 1$ và $|2 - 4| = 2$; MAE = (1 + 2)/2 = 1.5.

Q7. For a correct class, which predicted probability gives the smaller cross-entropy loss? | Với lớp đúng, xác suất dự đoán nào tạo ra cross-entropy nhỏ hơn?

A. 0.2 | 0.2

B. 0.1 | 0.1

C. 0.9 | 0.9

D. 0.5 | 0.5

Đáp án: C

Giải thích: Loss của lớp đúng là $-\log(p)$. Khi p càng gần 1, loss càng nhỏ; 0.9 là lớn nhất trong bốn giá trị.

Q8. Which loss penalizes a very large error more strongly than MAE? | Hàm mất mát nào phạt sai số cực lớn mạnh hơn MAE?

A. Mean squared error | MSE

B. Median absolute deviation | Độ lệch tuyệt đối trung vị

C. Mean absolute error | MAE

D. Zero-one loss | Loss đúng/sai 0–1

Đáp án: A

Giải thích: MSE bình phương sai số, nên lỗi lớn tăng nhanh hơn so với MAE chỉ lấy trị tuyệt đối.

Q9. What does `-log(p_true)` represent for one correct class? | `-log(p_true)` biểu thị gì cho một nhãn đúng?

A. Cross-entropy loss for that sample | Cross-entropy của mẫu đó

B. The distributions are identical | Hai phân phối giống nhau

C. Nonnegative | Không âm

D. No, KL is asymmetric | Không, KL bất đối xứng

Đáp án: A

Giải thích: Với one-hot, chỉ còn hạng log của lớp đúng.

Q10. What is the sign of KL divergence for valid distributions? | KL divergence có dấu gì với phân phối hợp lệ?

A. The distributions are identical | Hai phân phối giống nhau

B. Cross-entropy loss for that sample | Cross-entropy của mẫu đó

C. Nonnegative | Không âm

D. No, KL is asymmetric | Không, KL bất đối xứng

Đáp án: C

Giải thích: KL không âm theo bất đẳng thức Gibbs.

Q11. Does swapping P and Q generally preserve KL? | Đổi chỗ P và Q có giữ nguyên KL không?

A. Nonnegative | Không âm

B. The distributions are identical | Hai phân phối giống nhau

C. No, KL is asymmetric | Không, KL bất đối xứng

D. Cross-entropy loss for that sample | Cross-entropy của mẫu đó

Đáp án: C

Giải thích: D(P||Q) và D(Q||P) có thể khác.

Q12. What does a zero KL divergence mean for valid finite distributions? | KL bằng 0 nghĩa là gì với các phân phối hữu hạn hợp lệ?

A. The distributions are identical | Hai phân phối giống nhau

B. Cross-entropy loss for that sample | Cross-entropy của mẫu đó

C. No, KL is asymmetric | Không, KL bất đối xứng

D. Nonnegative | Không âm

Đáp án: A

Giải thích: KL bằng 0 khi các phân phối trùng nhau trong điều kiện phù hợp.

Q13. What is the unit of MSE when predictions are in meters? | Dự báo theo mét thì MSE có đơn vị gì?

A. Meters | Mét

B. MSE | MSE

C. Huber loss | Huber loss

D. Square meters | Mét bình phương

Đáp án: D

Giải thích: Bình phương sai lệch mét cho đơn vị m^2.

Q14. What is the unit of MAE when predictions are in meters? | Dự báo theo mét thì MAE có đơn vị gì?

A. MSE | MSE

B. Huber loss | Huber loss

C. Square meters | Mét bình phương

D. Meters | Mét

Đáp án: D

Giải thích: Sai lệch tuyệt đối giữ đơn vị gốc.

Q15. Which loss increases quadratically with error magnitude? | Loss nào tăng theo bình phương độ lớn lỗi?

A. Meters | Mét

B. Huber loss | Huber loss

C. MSE | MSE

D. Square meters | Mét bình phương

Đáp án: C

Giải thích: MSE bình phương độ lớn lỗi.

Q16. Which regression loss has a threshold between quadratic and linear penalties? | Loss hồi quy nào có ngưỡng chuyển bình phương sang tuyến tính?

A. Huber loss | Huber loss

B. MSE | MSE

C. Meters | Mét

D. Square meters | Mét bình phương

Đáp án: A

Giải thích: Huber chuyển miền tại delta.

Q17. Why clip predicted probabilities before a naive log implementation? | Vì sao clip xác suất trước khi lấy log bằng công thức đơn giản?

A. To avoid log(0) and numerical infinities | Để tránh log(0) và vô cực số học

B. No, evaluate on held-out data | Không, cần đánh giá trên dữ liệu độc lập

C. Predicted probability assigned to the true class | Xác suất dự báo dành cho lớp đúng

D. MAE | MAE

Đáp án: A

Giải thích: Xác suất 0 gây kỳ dị log trong công thức số học đơn giản.

Q18. Which loss is easier to interpret in the original output units? | Loss nào dễ đọc theo đơn vị đầu ra ban đầu?

A. Predicted probability assigned to the true class | Xác suất dự báo dành cho lớp đúng

B. No, evaluate on held-out data | Không, cần đánh giá trên dữ liệu độc lập

C. To avoid log(0) and numerical infinities | Để tránh log(0) và vô cực số học

D. MAE | MAE

Đáp án: D

Giải thích: MAE báo sai số trung bình cùng đơn vị đo.

Q19. Does a lower training loss guarantee better unseen-data performance? | Training loss thấp có bảo đảm dự đoán tốt trên dữ liệu mới?

A. To avoid log(0) and numerical infinities | Để tránh log(0) và vô cực số học

B. MAE | MAE

C. Predicted probability assigned to the true class | Xác suất dự báo dành cho lớp đúng

D. No, evaluate on held-out data | Không, cần đánh giá trên dữ liệu độc lập

Đáp án: D

Giải thích: Khả năng tổng quát cần kiểm chứng bằng dữ liệu độc lập.

Q20. For one-hot labels, what information matters in cross-entropy? | Với nhãn one-hot, điều gì quyết định cross-entropy?

A. To avoid log(0) and numerical infinities | Để tránh log(0) và vô cực số học

B. Predicted probability assigned to the true class | Xác suất dự báo dành cho lớp đúng

C. MAE | MAE

D. No, evaluate on held-out data | Không, cần đánh giá trên dữ liệu độc lập

Đáp án: B

Giải thích: Âm log-likelihood dùng xác suất lớp đúng.

8. Bài tiếp theo

Dự đoán → Sai số → Loss → Tối ưu hóa.

PY-31 — Giới hạn, tính liên tục và đạo hàm

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Limit — giới hạn; **Continuity** — tính liên tục.

`Hàm số → Limit → Derivative → Gradient`

Học xong dùng để làm gì? Đo tốc độ thay đổi của hàm theo biến và suy ra chiều tăng/giảm của sai số.

Kết nối kiến thức tiếp theo: PY-32 dùng gradient và tốc độ học để cập nhật trọng số mô hình.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Limit	giới hạn	Giá trị hàm tiến gần khi đầu vào tiến đến điểm
Continuity	tính liên tục	Giới hạn và giá trị hàm phù hợp tại điểm xét
Derivative	đạo hàm	Tốc độ thay đổi cục bộ của hàm theo đầu vào
Tangent slope	độ dốc tiếp tuyến	Giá trị đạo hàm tại một điểm

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`sympy.symbols()/limit()`	Tạo biến ký hiệu và tính giới hạn kể cả điểm không xác định
`sympy.diff(f,x)`	Đạo hàm theo biến x
`expr.subs(x,value)`	Thay giá trị vào công thức symbolic
`np.gradient(y,x)`	Ước lượng đạo hàm từ dãy mẫu, phụ thuộc bước lấy mẫu

4. Khái niệm chính và liên quan

Mạch kiến thức: Giới hạn mô tả xu hướng gần một điểm; liên tục đòi hỏi xu hướng ấy bằng giá trị tại điểm; đạo hàm dùng giới hạn để đo độ thay đổi; độ dốc tiếp tuyến là đạo hàm sau khi thay một tọa độ cụ thể.

4.1. Limit — Giá trị hàm tiến đến

Từ vựng: Limit — giới hạn; **Approach** — tiến gần.

Giới hạn hỏi **f(x) tiến gần giá trị nào khi x tiến gần a**, không bắt buộc f(a) phải có giá trị. Ví dụ $(x^2-4)/(x-2)$ không xác định tại x=2 vì mẫu số bằng 0. Với x khác 2, rút gọn thành x+2; khi x→2, kết quả tiến đến **4**. `sympy.limit` tính giá trị tiến đến của biểu thức thay vì thay trực tiếp x=2 gây chia 0.

No Note — Không chú thích

```
import sympy as sp

x = sp.symbols('x')
f = (x ** 2 - 4) / (x - 2)
print(sp.limit(f, x, 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm giới hạn ở x=2 của biểu thức không xác định trực tiếp tại x=2.
# Dữ liệu: Hàm (x²-4)/(x-2); điểm x=2 làm mẫu số bằng 0 nhưng có giới hạn hữu hạn.
# SymPy tính toán biểu thức toán học bằng ký hiệu, sp là tên ngắn của thư viện.
import sympy as sp

# symbols('x') tạo biến KÝ HIỆU, khác một số x đã gán giá trị.
x = sp.symbols('x')
# Tại x=2 biểu thức 0/0 chưa xác định; với x khác 2 biểu thức rút gọn được thành x+2.
f = (x ** 2 - 4) / (x - 2)
# limit(biểu_thức, biến, điểm) xét giá trị tiến GẦN 2, không thay x=2 trực tiếp; kết quả 4.
print(sp.limit(f, x, 2))
```

Kết quả khi chạy:

```
4
```

Giới hạn bằng 4 dù biểu thức gốc không xác định ngay tại x=2.

4.2. Continuity — Hàm liên tục tại một điểm

Từ vựng: Continuity — tính liên tục; **Function value** — giá trị hàm.

Hàm liên tục tại x=a nếu **f(a) tồn tại**, giới hạn từ hai phía tồn tại và bằng f(a). Ví dụ quãng đường mô phỏng $s(t)=t^2$ là hàm liên tục; tại t=2 giây, $s(2)=4$ mét và khi t tiến gần 2, quãng đường cũng tiến đến 4 mét. Đây là tính chất của **mô hình hàm**, không tự suy ra cảm biến vật lý hoạt động liên tục.

No Note — Không chú thích

```
import sympy as sp

t = sp.symbols('t')
s = t ** 2
print(sp.limit(s, t, 2) == s.subs(t, 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tính liên tục bằng so giới hạn với giá trị hàm tại t=2.
# Dữ liệu: Hàm vị trí s=t² không đứt đoạn tại t=2.
# SymPy biểu diễn ký hiệu và tính giới hạn.
import sympy as sp

# Tạo biến ký hiệu t dùng cho hàm s.
```

```
t = sp.symbols('t')
# **2 nâng t lên bình phương, đây là hàm vị trí giả lập.
s = t ** 2
# Lấy giá trị hàm TIẾN ĐẾN khi t gần 2.
# subs(t,2) THẾ t=2 vào biểu thức để lấy đúng giá trị tại điểm; hai giá trị bằng nhau nghĩa liên
tục ở đây.
print(sp.limit(s, t, 2) == s.subs(t, 2))
```

Kết quả khi chạy:

```
True
```

Giới hạn bằng giá trị hàm ở t=2, nên mô hình liên tục tại điểm xét.

4.3. Derivative — Tốc độ thay đổi tức thời

Từ vựng: Derivative — đạo hàm; **Rate of change** — tốc độ thay đổi.

Đạo hàm `f'(x)` cho biết đầu ra thay đổi bao nhiêu **khi đầu vào thay đổi rất nhỏ**. Về công thức, đó là giới hạn `(f(x+h)−f(x))/h` khi h→0 nếu giới hạn tồn tại. Với $f(x)=x^2$, ta có $f'(x)=2x$; độ dốc không cố định: tại x=1 là 2, tại x=3 là 6. Nếu f là quãng đường (mét) theo thời gian (giây), đơn vị đạo hàm là mét/giây.

No Note — Không chú thích

```
import sympy as sp

x = sp.symbols('x')
print(sp.diff(x ** 2, x))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng đạo hàm ký hiệu tìm tốc độ thay đổi của x² theo x.
# Dữ liệu: Hàm f=x², cần biểu thức độ dốc thay đổi theo từng x.
# SymPy sp.diff() lấy đạo hàm của biểu thức theo biến ký hiệu.
import sympy as sp

# Khai báo x là biến tượng trưng thay vì một số cụ thể.
x = sp.symbols('x')
# diff(f,x) đạo hàm theo x; độ dốc của x² là 2x tại từng vị trí, print() hiển thị biểu thức.
print(sp.diff(x ** 2, x))
```

Kết quả khi chạy:

```
2*x
```

Đạo hàm là biểu thức 2x; tăng x thì độ dốc cục bộ tăng.

4.4. Tangent slope — Độ dốc tại một giá trị cụ thể

Từ vựng: Tangent — tiếp tuyến; **Slope** — độ dốc.

Nếu đạo hàm là **hàm mô tả độ dốc**, thì độ dốc tiếp tuyến tại x=a là **một số**, chính là `f'(a)`. Với đường $f(x)=x^2$, đạo hàm 2x, nên tại x=3 độ dốc bằng 6. Phân biệt `sp.diff` tạo biểu thức đạo hàm và `.subs(x,3)` thay tọa độ để lấy độ dốc tại đúng điểm đó.

No Note — Không chú thích

```
import sympy as sp

x = sp.symbols('x')
slope = sp.diff(x ** 2, x)
print(slope.subs(x, 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính độ dốc tức thời tại x=3 bằng đạo hàm rồi thế vào biểu thức.
# Dữ liệu: Hàm f=x², hỏi độ dốc tại x=3 (đơn vị đầu ra trên một đơn vị đầu vào).
# SymPy tách bước tính đạo hàm và bước thế giá trị.
import sympy as sp

# Biến x ký hiệu giữ công thức còn phụ thuộc x.
x = sp.symbols('x')
# diff() tạo biểu thức đạo hàm 2x, chưa phải độ dốc tại một điểm.
slope = sp.diff(x ** 2, x)
# subs(x,3) thay x=3 trong ĐẠO HÀM, kết quả độ dốc 6, không phải f(3)=9.
print(slope.subs(x, 3))
```

Kết quả khi chạy:

```
6
```

Đường cong có độ dốc tiếp tuyến bằng 6 tại x=3.

5. Năm tình huống Giới hạn/Đạo hàm — từ điểm khuyết đến sai phân không đều

Tình huống 1 — Cấp 1: Quan sát 10 giá trị tiến gần một điểm biểu thức không xác định

Từ vựng trong mục: Limit — giới hạn; **Removable discontinuity** — điểm khuyết có thể bổ sung.

Tình huống thực tế: Một phần mềm hiệu chuẩn nhận **10 giá trị tín hiệu x không có đơn vị**, gồm năm giá trị ngay dưới 2 và năm giá trị ngay trên 2. Công thức đổi tín hiệu thành chỉ số có phép chia cho phần chênh lệch giữa x và 2, nên nếu nhập đúng **x = 2** chương trình không chia được. Kỹ thuật viên muốn xem kết quả **tiến gần giá trị nào từ cả hai phía**, để hiểu điểm bị khuyết chứ không tự coi phép chia tại 2 là hợp lệ.

Vì sao chọn cách này? Giới hạn giúp trả lời giá trị của biểu thức đang **tiến tới đâu khi đầu vào tới gần một điểm**, kể cả khi không thể tính biểu thức đúng tại điểm đó. Chương trình thử năm số **nhỏ hơn 2** và năm số **lớn hơn 2**, nhưng cố ý **không có đúng 2** để tránh chia cho 0. Hai phía cùng tiến gần một chỉ số; `sympy.limit` dùng tính toán ký hiệu để xác định giá trị tiến tới một cách chính xác hơn việc đoán bằng vài số thập phân. Giá trị giới hạn **không tự làm công thức gốc tại x=2 có nghĩa**; nếu muốn dùng ở điểm khuyết, phải định nghĩa rõ quy tắc bổ sung.

Quy trình:

1. Tạo 10 số đầu vào, không có số nào bằng 2.
2. Tính đầu ra ở hai giá trị gần nhất, quan sát chúng xấp xỉ nhau.
3. Dùng SymPy xác định giới hạn tại 2 và đối chiếu.

No Note — Không chú thích

```
import sympy as sp
x = sp.symbols("x")
f = (x**2 - 4) / (x - 2)
inputs = [1.9,1.95,1.99,1.995,1.999,2.1,2.05,2.01,2.005,2.001]
results = []
for value in inputs:
    results.append(float(f.subs(x, value)))
limit_at_two = sp.limit(f, x, 2)
if len(inputs) != 10 or 2 in inputs:
    raise ValueError("Cần đúng 10 điểm tiến gần 2, không lấy đúng x=2.")
print("Lượt đo:", len(inputs))
print(f"Gần từ trái: {results[4]:.3f}")
print(f"Gần từ phải: {results[-1]:.3f}")
print("Giới hạn:", limit_at_two)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dò 10 giá trị hàm từ hai phía gần x=2 và so với giới hạn chính xác.
# Dữ liệu: 10 giá trị x từ 1.9 đến 2.001, không có x=2 vì mẫu số bằng 0; kết quả tiến gần 4.
# SymPy giúp tính công thức ở các điểm và lấy giới hạn chính xác.
import sympy as sp
# Tạo ký hiệu x cho biểu thức hữu tỉ.
x = sp.symbols("x")
# Ở đúng x=2 phép chia không xác định; ở gần 2 hàm tiến tới 4.
f = (x**2 - 4) / (x - 2)
# 10 điểm khảo sát từ trái (<2) và phải (>2), không phải 10 mẫu ngẫu nhiên.
inputs = [1.9,1.95,1.99,1.995,1.999,2.1,2.05,2.01,2.005,2.001]
# List rỗng để lưu giá trị f tại từng điểm theo thứ tự.
results = []
# Lặp qua từng giá trị x trong list.
for value in inputs:
    # subs() thế từng x vào biểu thức; float() đổi kết quả SymPy sang số thực Python để in.
    # append() giữ một đầu ra cho mỗi x.
    results.append(float(f.subs(x, value)))
# limit() lấy giới hạn ký hiệu tại 2, không đánh giá f(2).
limit_at_two = sp.limit(f, x, 2)
# in kiểm có vô tình đưa giá trị bị chia 0 vào list hay không.
if len(inputs) != 10 or 2 in inputs:
    # ValueError báo lỗi cấu trúc dữ liệu và ngừng xử lý, tránh kết quả thiếu/không phù hợp.
    raise ValueError("Cần đúng 10 điểm tiến gần 2, không lấy đúng x=2.")
print("Lượt đo:", len(inputs))
# [4] là điểm cuối nhóm bên trái x=1.999; [-1] là điểm cuối nhóm bên phải x=2.001, :.3f in ba số lẻ.
print(f"Gần từ trái: {results[4]:.3f}")
print(f"Gần từ phải: {results[-1]:.3f}")
print("Giới hạn:", limit_at_two)
```

Kết quả khi chạy:

```
Lượt đo: 10
Gần từ trái: 3.999
Gần từ phải: 4.001
Giới hạn: 4
```

Biểu thức gốc có điểm khuyết tại x=2. **Giới hạn bằng 4** không đồng nghĩa biểu thức gốc đã có giá trị tại điểm ấy.

Tình huống 2 — Cấp 2: Tính vận tốc tức thời ở 12 mốc thời gian

Từ vựng trong mục: Derivative — đạo hàm; **Rate of change** — tốc độ thay đổi.

Tình huống thực tế: Robot chạy theo quy tắc quãng đường **tăng theo bình phương thời gian**, với thời gian tính bằng **giây** và quãng đường tính bằng **mét**. Người vận hành có **12 mốc từ 0 đến 11 giây** và muốn biết **vận tốc tức thời** ở mỗi mốc, đặc biệt so mốc **3 giây và 11 giây**. Vận tốc là mức quãng đường đang tăng trong một giây tại thời điểm ấy, không phải tổng số mét robot đã đi.

Vì sao chọn cách này? Đạo hàm theo thời gian cho biết quãng đường **đang thay đổi nhanh đến mức nào** tại một thời điểm, vì vậy dùng để tìm **vận tốc tức thời**, có đơn vị **mét/giây**. `sympy.diff(position, t)` tìm quy tắc tính vận tốc một lần từ quy tắc quãng đường; `subs(t, second)` thế lần lượt **12 mốc thời gian** để lấy kết quả cụ thể. Ở mốc sau, quãng đường tăng nhanh hơn nên vận tốc lớn hơn. Nếu lấy quãng đường tại 11 giây rồi gọi là vận tốc 11 m/giây thì đã **nhầm tổng quãng đường với tốc độ thay đổi**.

Quy trình:

1. Định nghĩa quãng đường bằng SymPy và lấy đạo hàm theo t.
2. Thay 12 mốc thời gian vào biểu thức vận tốc.
3. In vận tốc ở hai mốc và kiểm số lượng đầu ra.

No Note — Không chú thích

```
import sympy as sp
t = sp.symbols("t")
position = t**2
velocity = sp.diff(position, t)
times = [0,1,2,3,4,5,6,7,8,9,10,11]
values = []
for second in times:
    values.append(velocity.subs(t, second))
if len(times) != 12 or len(values) != 12:
    raise ValueError("Cần đủ 12 mốc thời gian và 12 vận tốc.")
print("Mốc thời gian:", len(times))
print("Vận tốc ở 3 giây (m/giây):", values[3])
print("Vận tốc ở 11 giây (m/giây):", values[-1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính vận tốc từ hàm vị trí s=t² tại 12 mốc thời gian.
# Dữ liệu: Thời gian t=0..11 giây, vị trí s=t² tính mét, vận tốc đạo hàm có đơn vị m/giây.
# SymPy đạo hàm công thức vị trí để suy ra vận tốc tức thời.
import sympy as sp
# t là ký hiệu thời gian có thể thay các mốc sau.
t = sp.symbols("t")
# Mô hình vị trí theo thời gian, đơn vị mét giả định.
position = t**2
# diff(s,t) cho vận tốc tức thời 2t (m/s); chưa gán mốc thời gian.
velocity = sp.diff(position, t)
# 12 giây từ 0 đến 11, mỗi mốc một số đo tính toán.
times = [0,1,2,3,4,5,6,7,8,9,10,11]
# Danh sách vận tốc sẽ được lưu cùng thứ tự với times.
values = []
for second in times:
    # subs() thế giây hiện tại vào biểu thức vận tốc, trả giá trị SymPy.
    # append() giữ kết quả một lần cho mỗi mốc.
    values.append(velocity.subs(t, second))
# Kiểm đủ cả 12 thời điểm và 12 kết quả để tránh thiếu mẫu.
if len(times) != 12 or len(values) != 12:
```

```
    # ValueError báo lỗi cấu trúc dữ liệu và ngừng xử lý, tránh kết quả thiếu/không phù hợp.
    raise ValueError("Cần đủ 12 mốc thời gian và 12 vận tốc.")
print("Mốc thời gian:", len(times))
# [3] ứng với giây thứ 3 (t=3); [-1] là mốc cuối t=11.
print("Vận tốc ở 3 giây (m/giây):", values[3])
print("Vận tốc ở 11 giây (m/giây):", values[-1])
```

Kết quả khi chạy:

```
Mốc thời gian: 12
Vận tốc ở 3 giây (m/giây): 6
Vận tốc ở 11 giây (m/giây): 22
```

Tại t=3, **quãng đường là 9 m nhưng vận tốc là 6 m/giây**; hai đại lượng này có đơn vị khác nhau.

Tình huống 3 — Cấp 3: Đo mức tăng chi phí tại 15 mức tải bằng tiếp tuyến

Từ vựng trong mục: Tangent slope — độ dốc tiếp tuyến; **Marginal cost** — chi phí cận biên.

Tình huống thực tế: Một xưởng thử máy ở **15 mức tải từ 1 đến 15**, mỗi mức cách mức sau một đơn vị tải. Chi phí theo quy tắc mô phỏng **tăng nhanh hơn khi tải lớn**, tính bằng **nghìn đồng**. Quản đốc không chỉ hỏi tổng chi phí mà muốn biết **nếu tăng tải thêm một chút**, chi phí đang tăng nhanh cỡ nào tại tải **1** và tải **15**, để thấy vì sao phần tải cao tốn kém hơn.

Vì sao chọn cách này? Chi phí cận biên trong ví dụ là mức thay đổi của chi phí khi **tải tăng một lượng rất nhỏ** quanh một mức đang xét; đây là **độ dốc**, không phải tổng tiền hóa đơn. `sympy.diff(cost, q)` tìm quy tắc độ dốc của đường chi phí, sau đó chương trình thế vào **15 mức tải** để so mức đầu và cuối. Với đường cong bậc hai đang dùng, độ dốc lớn dần khi tải tăng. Cần phân biệt **đạo hàm tại một điểm** với phần chênh lệch tổng chi phí giữa hai mức tải cách xa nhau; chúng không phải cùng một câu hỏi.

Quy trình:

1. Lưu 15 mức tải và hàm chi phí tương ứng.
2. Lấy đạo hàm theo q, thay từng mức tải vào.
3. Báo độ dốc đầu và cuối, xác nhận dãy độ dốc tăng.

No Note — Không chú thích

```
import sympy as sp
q = sp.symbols("q")
cost = 2*q**2 + 3*q + 10
marginal = sp.diff(cost, q)
loads = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
slopes = []
for load in loads:
    slopes.append(int(marginal.subs(q, load)))
if len(slopes) != 15:
    raise ValueError("Cần tính đủ 15 mức tải.")
print("Mức tải:", len(slopes))
print("Mức tăng ở tải 1:", slopes[0])
print("Mức tăng ở tải 15:", slopes[-1])
slopes_increase = True
for a, b in zip(slopes, slopes[1:]):
    if a >= b:
        slopes_increase = False
        break
print("Độ dốc tăng:", slopes_increase)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm chi phí biên tại 15 mức tải bằng đạo hàm và kiểm độ dốc có tăng.
# Dữ liệu: Mô hình cost=2q²+3q+10, q là mức tải 1..15; đơn vị chi phí giả lập.
# SymPy đạo hàm hàm chi phí theo ký hiệu q.
import sympy as sp
# q là biến tải tượng trưng để tạo đạo hàm tổng quát.
q = sp.symbols("q")
# Mô hình phi tuyến: bình phương tải khiến chi phí biên tăng khi q lớn.
cost = 2*q**2 + 3*q + 10
# Đạo hàm theo q trả tốc độ tăng chi phí tại từng tải, gọi là marginal cost.
marginal = sp.diff(cost, q)
# 15 mức tải liên tục từ 1 đến 15.
loads = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
slopes = []
for load in loads:
    # Thế từng tải vào đạo hàm; int() đổi SymPy Integer về int Python.
    # append() lưu 15 chi phí biên theo thứ tự tải.
    slopes.append(int(marginal.subs(q, load)))
if len(slopes) != 15:
    # ValueError báo lỗi cấu trúc dữ liệu và ngừng xử lý, tránh kết quả thiếu/không phù hợp.
    raise ValueError("Cần tính đủ 15 mức tải.")
print("Mức tải:", len(slopes))
# [0] chi phí biên tại q=1, [-1] tại q=15.
print("Mức tăng ở tải 1:", slopes[0])
print("Mức tăng ở tải 15:", slopes[-1])
slopes_increase = True
# zip() ghép mỗi độ dốc với độ dốc KẾ TIẾP; slopes[1:] bỏ mục đầu, giúp kiểm tính tăng.
for a, b in zip(slopes, slopes[1:]):
    # Nếu một độ dốc không nhỏ hơn độ dốc sau thì không còn tăng nghiêm ngặt.
    if a >= b:
        slopes_increase = False
        # break thoát sớm khi tìm được trường hợp không tăng.
        break
print("Độ dốc tăng:", slopes_increase)
```

Kết quả khi chạy:

```
Mức tải: 15
Mức tăng ở tải 1: 7
Mức tăng ở tải 15: 63
Độ dốc tăng: True
```

Đơn vị độ dốc là **nghìn đồng trên một đơn vị tải**; không coi nó là tổng tiền của chuyến sản xuất.

Tình huống 4 — Cấp 4: Phát hiện điểm nhảy dù đã ghi 18 mốc số đo

Từ vựng trong mục: Continuity — tính liên tục; **One-sided limit** — giới hạn một phía.

Tình huống thực tế: Một ứng dụng tính **chỉ số phụ phí giả lập** theo số giờ t: trước **8 giờ** dùng một quy định, từ **8 giờ trở đi** đổi sang quy định khác. Hệ thống ghi **18 mốc 0–17 giờ**; ngay trước mốc 8, chỉ số tiến gần **13**, còn bắt đầu từ 8 nó bằng **15**. Người quản lý muốn biết có một **bước nhảy đột ngột** tại thời điểm đổi quy định hay không, chứ không chỉ nhìn bảng giờ nguyên.

Vì sao chọn cách này? Liên tục tại một thời điểm nghĩa là khi tiến tới điểm đó từ **bên trái và bên phải**, kết quả đều tiến về **chính giá trị tại điểm ấy**. `sp.limit` kiểm hai biểu thức riêng ở mốc **8 giờ**, còn `values[8]` cho giá trị chương trình thật sự áp dụng tại mốc đó. Bên trái tiến tới 13 nhưng bên phải và tại 8 là 15, nên **không liên tục**: có bước nhảy 2 đơn vị chỉ số phụ phí. Một bảng chỉ có các mốc giờ nguyên không thể thay thế việc kiểm tra sát hai phía, vì có thể bỏ qua hành vi ngay nơi đổi chính sách.

Quy trình:

1. Tạo 18 thời điểm và tính đầu ra theo đúng hai nhánh.
2. Dùng `sympy.limit` cho biểu thức nhánh trái/phải khi t tới 8.
3. So hai giới hạn với giá trị hàm tại 8 rồi báo kết luận.

No Note — Không chú thích

```
import sympy as sp
t = sp.symbols("t")
times = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17]
values = []
for hour in times:
    if hour < 8:
        value = hour + 5
    else:
        value = hour + 7
    values.append(value)
left = sp.limit(t + 5, t, 8)
right = sp.limit(t + 7, t, 8)
continuous = left == right == values[8]
if len(times) != 18:
    raise ValueError("Cần đủ 18 mốc giờ.")
print("Mốc giờ:", len(times))
print("Giới hạn trái:", left)
print("Giới hạn phải:", right)
print("Giá trị ở 8:", values[8])
print("Liên tục ở 8:", continuous)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xét tính liên tục tại giờ 8 của hàm được định nghĩa theo hai nhánh.
# Dữ liệu: 18 mốc giờ 0..17; trước giờ 8 là t+5, từ giờ 8 là t+7, nên có bước nhảy tại 8.
# SymPy tính giới hạn biểu thức của từng nhánh tại ranh giới.
import sympy as sp
# t là biến thời gian ký hiệu.
t = sp.symbols("t")
# 18 mốc giờ, giờ 8 là điểm chuyển từ nhánh trái sang phải.
times = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17]
values = []
for hour in times:
    # Nhánh áp dụng trước giờ 8: output=hour+5.
    if hour < 8:
        # Giá trị của nhánh trái, ứng với giờ nhỏ hơn 8.
        value = hour + 5
    else:
        # Giá trị tại và sau giờ 8, nhánh phải.
        value = hour + 7
    # Ghi đầu ra của từng giờ, [8] là giá trị thực tế´tại điểm chuyển.
    values.append(value)
# Tính giới hạn công thức NHÁNH TRÁI t+5 tại mốc 8; không ghép hai nhánh.
left = sp.limit(t + 5, t, 8)
# Tính giới hạn công thức NHÁNH PHẢI t+7 tại 8; hai nhánh cho giới hạn khác nhau.
right = sp.limit(t + 7, t, 8)
# Chuỗi == yêu cầu giới hạn trái, phải và giá trị tại điểm đều bằng nhau; False nghĩa gián đoạn.
# [8] lấy giá trị của mốc 8 vì list bắt đầu từ 0.
continuous = left == right == values[8]
if len(times) != 18:
    # ValueError báo lỗi cấu trúc dữ liệu và ngừng xử lý, tránh kết quả thiếu/không phù hợp.
    raise ValueError("Cần đủ 18 mốc giờ.")
print("Mốc giờ:", len(times))
print("Giới hạn trái:", left)
print("Giới hạn phải:", right)
```

```
# [8] lấy giá trị của mốc 8 vì list bắt đầu từ 0.
print("Giá trị ở 8:", values[8])
print("Liên tục ở 8:", continuous)
```

Kết quả khi chạy:

```
Mốc giờ: 18
Giới hạn trái: 13
Giới hạn phải: 15
Giá trị ở 8: 15
Liên tục ở 8: False
```

Điểm t=8 có giá trị thật theo **nhánh phải**; vẫn không liên tục vì khi đến từ bên trái, hàm tiến đến giá trị khác.

Tình huống 5 — Cấp 5: Đối chiếu đạo hàm giải tích với 20 điểm đo không đều

Từ vựng trong mục: Numerical gradient — đạo hàm số; **Sampling interval** — khoảng lấy mẫu; **Boundary effect** — sai số ở mép.

Tình huống thực tế: Robot ghi **20 cặp thời gian và quãng đường**, tính bằng **giây** và **mét**, nhưng khoảng cách giữa các lần lấy mẫu **không đều**: có lần sau 1 giây, lần sau 2 giây. Quãng đường được tạo theo một quy tắc cong tăng dần. Kỹ thuật viên muốn ước lượng **vận tốc** từ chính các số đo, so với vận tốc chính xác của quy tắc mô phỏng và xem vì sao lượt đầu/lượt cuối dễ sai khác hơn những lượt ở giữa.

Vì sao chọn cách này? Đạo hàm số là ước lượng tốc độ thay đổi bằng các **giá trị đo gần nhau**, hữu ích khi có bảng số thay vì chỉ một biểu thức dễ lấy đạo hàm. `np.gradient(s, t)` nhận **cả quãng đường và các thời điểm thực tế**, vì các lần đo không cách đều; bỏ mảng thời gian sẽ ngầm giả định khoảng cách giữa lượt đo giống nhau. Ở các điểm **giữa**, NumPy có số đo ở **cả phía trước và phía sau** để ước lượng, trong khi ở hai **mép** mặc định chỉ dùng chênh lệch một phía. Với đường cong bậc hai trong bài, các điểm giữa khớp đạo hàm chính xác nhưng hai mép có sai số: đó là giới hạn của cách ước lượng, không có nghĩa cảm biến biên bị hỏng.

Quy trình:

1. Tạo 20 thời điểm và quãng đường theo cùng thứ tự.
2. Tính vận tốc số bằng `np.gradient` có truyền thời gian, so với `2*t+2`.
3. Kiểm 18 điểm trong khớp và in sai số mép đầu/cuối.

No Note — Không chú thích

```
import numpy as np
t = np.array([0,1,3,4,6,7,9,10,12,13,15,16,18,19,21,22,24,25,27,28], dtype=float)
s = t**2 + 2*t
numerical = np.gradient(s, t)
analytical = 2*t + 2
if len(t) != 20 or not np.all(np.diff(t) > 0):
    raise ValueError("Cần 20 mốc thời gian tăng dần.")
print("Số lần đo:", len(t))
print("Giữa khớp:", bool(np.allclose(numerical[1:-1], analytical[1:-1])))
print(f"Sai số mép đầu (m/giây): {abs(numerical[0]-analytical[0]):.1f}")
print(f"Sai số mép cuối (m/giây): {abs(numerical[-1]-analytical[-1]):.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So đạo hàm sô´ np.gradient với đạo hàm giải tích tại 20 thời điểm không cách đêu.
# Dữ liệu: 20 thời điểm t tăng dần, vị trí s=t²+2t tính mét, vận tốc chuẩn 2t+2 m/giây.
# NumPy np.gradient() ước lượng đạo hàm khi chỉ có giá trị đo rời rạc.
import numpy as np
# 20 thời điểm tăng dần KHÔNG đêu, dtype=float đe tính khoảng cách chính xác.
t = np.array([0,1,3,4,6,7,9,10,12,13,15,16,18,19,21,22,24,25,27,28], dtype=float)
# Mảng s chứa vị trí tại môĩ thời điểm, với **2 bình phương từng phần tư.
s = t**2 + 2*t
# gradient(y,x) xấp xỉ ds/dt từ dữ liệu, dùng chính vector thời gian t đe xư lý bước lấy mâũ không
đêu; mép dùng sai phân một phía mặc định.
numerical = np.gradient(s, t)
# Đạo hàm giải tích thật của t²+2t tại từng t, làm môć đôí chiếú.
analytical = 2*t + 2
# diff(t) lấy độ chênh môć liên tiếp; >0 kiểm mọi thời điểm đi lên đe không có bước âm/trùng.
# np.all() yêu câù tất cả khoảng cách dương, không chỉ một khoảng.
if len(t) != 20 or not np.all(np.diff(t) > 0):
    # ValueError báo lôĩ câú trúc dữ liệu và ngừng xư lý, tránh kêt quả thiêú/không phù hợp.
    raise ValueError("Cần 20 môć thời gian tăng dần.")
print("Sô´ lần đo:", len(t))
# Slice [1:-1] lấy toàn bộ các điểm GIỮA, bỏ hai điểm biên; so với analytical bằng allclose() có
dung sai float.
# Dùng cùng slice bỏ hai biên ở giá trị chuẩn đe so từng môć.
# allclose() so hai vector gần nhau trong dung sai sô´ thực, không dùng == tuyệt đôí.
print("Giữa khớp:", bool(np.allclose(numerical[1:-1], analytical[1:-1])))
# [0] là đạo hàm ước lượng mép đâù; [-1] mép cuôí, độ lệch được lấy abs() đe không âm.
# So ước lượng tại đúng mép đâù với giá trị giải tích tương ứng.
print(f"Sai sô´ mép đâù (m/giây): {abs(numerical[0]-analytical[0]):.1f}")
# [-1] lấy ước lượng đạo hàm mép cuôí, có thê sai do chỉ dùng điểm một phía.
print(f"Sai sô´ mép cuôí (m/giây): {abs(numerical[-1]-analytical[-1]):.1f}")
```

Kết quả khi chạy:

```
Sô´ lần đo: 20
Giữa khớp: True
Sai sô´ mép đâù (m/giây): 1.0
Sai sô´ mép cuôí (m/giây): 1.0
```

Trong dữ liệu thực có nhiễu đo, điểm giữa cũng có thể lệch; ở đây khớp vì **hàm được tạo đúng theo quy luật bậc hai**.

6. Sai lầm thường gặp

- Giới hạn không nhất thiết bằng giá trị hàm tại điểm nếu hàm không liên tục hoặc không xác định ở đó.
- Muốn liên tục cần giới hạn trái, phải và giá trị hàm bằng nhau.
- Đạo hàm là tốc độ thay đổi tức thời; độ dốc trung bình trên một khoảng có thể khác.
- Với quãng đường theo giây, đạo hàm có đơn vị m/s; không diễn giải thành mét.
- `np.gradient` xấp xỉ từ mẫu; ở đầu và cuối dãy thường dùng sai phân một phía.
- SymPy là thư viện ngoài cần cài khi chạy các ví dụ ký hiệu.

7. Trắc nghiệm

Q1. What does a limit describe? | Giới hạn mô tả gì?

A. The two-sided limit exists and equals f(a) | Giới hạn hai phía tồn tại và bằng f(a)

B. Instantaneous rate of change | Tốc độ thay đổi tức thời

C. The derivative evaluated at a | Giá trị đạo hàm tại a

D. What f(x) approaches as x approaches a point | Giá trị f(x) tiến đến khi x tiến gần một điểm

Đáp án: D

Giải thích: Giới hạn xét đầu vào lân cận điểm.

Q2. What does continuity at x=a require? | Liên tục tại x=a cần gì?

A. Instantaneous rate of change | Tốc độ thay đổi tức thời

B. The two-sided limit exists and equals f(a) | Giới hạn hai phía tồn tại và bằng f(a)

C. The derivative evaluated at a | Giá trị đạo hàm tại a

D. What f(x) approaches as x approaches a point | Giá trị f(x) tiến đến khi x tiến gần một điểm

Đáp án: B

Giải thích: Liên tục đòi hỏi giới hạn bằng giá trị hàm.

Q3. What does a derivative describe? | Đạo hàm mô tả gì?

A. What f(x) approaches as x approaches a point | Giá trị f(x) tiến đến khi x tiến gần một điểm

B. Instantaneous rate of change | Tốc độ thay đổi tức thời

C. The two-sided limit exists and equals f(a) | Giới hạn hai phía tồn tại và bằng f(a)

D. The derivative evaluated at a | Giá trị đạo hàm tại a

Đáp án: B

Giải thích: Đạo hàm đo độ thay đổi tại điểm.

Q4. What is tangent slope at x=a? | Độ dốc tiếp tuyến tại x=a là gì?

A. Instantaneous rate of change | Tốc độ thay đổi tức thời

B. The derivative evaluated at a | Giá trị đạo hàm tại a

C. The two-sided limit exists and equals f(a) | Giới hạn hai phía tồn tại và bằng f(a)

D. What f(x) approaches as x approaches a point | Giá trị f(x) tiến đến khi x tiến gần một điểm

Đáp án: B

Giải thích: Độ dốc tiếp tuyến là f'(a).

Q5. What is the derivative of $f(x) = 4x^2 - 3x + 1$? | Đạo hàm của $f(x) = 4x^2 - 3x + 1$ là gì?

A. $4x - 3$ | $4x - 3$

B. $8x - 3$ | $8x - 3$

C. $8x + 1$ | $8x + 1$

D. $4x^2 - 3$ | $4x^2 - 3$

Đáp án: B

Giải thích: Đạo hàm từng hạng: $4x^2$ thành $8x$, $-3x$ thành -3, hằng số 1 thành 0.

Q6. If $f(x) = x^2$, what is the slope of the tangent at $x = 2$? | Nếu $f(x) = x^2$, độ dốc tiếp tuyến tại $x = 2$ bằng bao nhiêu?

A. 8 | 8

B. 4 | 4

C. 2 | 2

D. 0 | 0

Đáp án: B

Giải thích: Đạo hàm $f'(x) = 2x$, thay $x = 2$ được 4.

Q7. What is the derivative of $f(x) = 7$? | Đạo hàm của hàm hằng $f(x) = 7$ bằng bao nhiêu?

A. x | x

B. 7 | 7

C. 1 | 1

D. 0 | 0

Đáp án: D

Giải thích: Giá trị hàm hằng không thay đổi theo x nên tốc độ thay đổi, tức đạo hàm, bằng 0.

Q8. If a function is differentiable at $x = a$, what must also hold there? | Nếu một hàm có đạo hàm tại $x = a$ thì điều kiện nào chắc chắn đúng?

A. It has a global minimum | Hàm có cực tiểu toàn cục

B. It is continuous at $x = a$ | Hàm liên tục tại $x = a$

C. Its derivative is zero | Đạo hàm bằng 0

D. Its value is positive | Giá trị hàm dương

Đáp án: B

Giải thích: Có đạo hàm tại một điểm là điều kiện mạnh hơn liên tục tại điểm đó; chiều ngược lại không luôn đúng.

Q9. What is the derivative of x^2? | Đạo hàm của x^2 là gì?

A. 6 | 6

B. 4 | 4

C. Zero | Bằng 0

D. 2*x | 2*x

Đáp án: D

Giải thích: Quy tắc đạo hàm lũy thừa cho 2x.

Q10. What is the derivative of a constant? | Đạo hàm của một hằng số là gì?

A. 6 | 6

B. Zero | Bằng 0

C. 2*x | 2*x

D. 4 | 4

Đáp án: B

Giải thích: Hằng số không đổi theo biến.

Q11. What is the slope of x^2 at x=3? | Độ dốc của x^2 tại x=3 là gì?

A. 6 | 6

B. 2*x | 2*x

C. Zero | Bằng 0

D. 4 | 4

Đáp án: A

Giải thích: 2 nhân 3 bằng sáu.

Q12. What is the limit of $(x^2-4)/(x-2)$ at x=2? | Giới hạn của $(x^2-4)/(x-2)$ khi $x \to 2$ là gì?

A. 4 | 4

B. 2*x | 2*x

C. Zero | Bằng 0

D. 6 | 6

Đáp án: A

Giải thích: Rút gọn gần điểm còn x+2.

Q13. What may happen if the left and right limits differ? | Nếu giới hạn trái và phải khác nhau thì sao?

A. Distance per unit time | Khoảng cách trên mỗi đơn vị thời gian

B. The two-sided limit does not exist | Giới hạn hai phía không tồn tại

C. Sampling spacing and numerical scheme | Khoảng cách mẫu và cách lấy sai phân

D. Yes, if nearby values approach one number | Có, nếu giá trị lân cận tiến đến cùng một số

Đáp án: B

Giải thích: Giới hạn một phía khác nhau khiến giới hạn hai phía không có.

Q14. What does a derivative unit become for distance over time? | Đạo hàm quãng đường theo thời gian có đơn vị gì?

A. Sampling spacing and numerical scheme | Khoảng cách mẫu và cách lấy sai phân

B. The two-sided limit does not exist | Giới hạn hai phía không tồn tại

C. Yes, if nearby values approach one number | Có, nếu giá trị lân cận tiến đến cùng một số

D. Distance per unit time | Khoảng cách trên mỗi đơn vị thời gian

Đáp án: D

Giải thích: Quãng đường chia thời gian cho đơn vị vận tốc.

Q15. What determines finite-difference accuracy? | Độ chính xác sai phân hữu hạn phụ thuộc gì?

A. Sampling spacing and numerical scheme | Khoảng cách mẫu và cách lấy sai phân

B. The two-sided limit does not exist | Giới hạn hai phía không tồn tại

C. Distance per unit time | Khoảng cách trên mỗi đơn vị thời gian

D. Yes, if nearby values approach one number | Có, nếu giá trị lân cận tiến đến cùng một số

Đáp án: A

Giải thích: Khoảng lấy mẫu ảnh hưởng độ dốc ước lượng.

Q16. Can a function have a finite limit at an undefined point? | Hàm chưa xác định tại một điểm có thể có giới hạn hữu hạn không?

A. The two-sided limit does not exist | Giới hạn hai phía không tồn tại

B. Distance per unit time | Khoảng cách trên mỗi đơn vị thời gian

C. Yes, if nearby values approach one number | Có, nếu giá trị lân cận tiến đến cùng một số

D. Sampling spacing and numerical scheme | Khoảng cách mẫu và cách lấy sai phân

Đáp án: C

Giải thích: Giới hạn có thể có dù hàm không xác định ngay điểm đó.

Q17. How does a derivative differ from an average slope? | Đạo hàm khác độ dốc trung bình thế nào?

A. 2 | 2

B. It often uses one-sided differences | Nó thường dùng sai phân một phía

C. `sympy.diff` | `sympy.diff`

D. It describes a local infinitesimal change | Nó mô tả độ thay đổi cục bộ rất nhỏ

Đáp án: D

Giải thích: Độ dốc tức thời là đại lượng giới hạn cục bộ.

Q18. What is the derivative of 2x+3? | Đạo hàm của 2x+3 là gì?

A. It describes a local infinitesimal change | Nó mô tả độ thay đổi cục bộ rất nhỏ

B. `sympy.diff` | `sympy.diff`

C. It often uses one-sided differences | Nó thường dùng sai phân một phía

D. 2 | 2

Đáp án: D

Giải thích: Hệ số của x cho đạo hàm bằng 2.

Q19. Which tool calculates symbolic derivatives? | Công cụ nào tính đạo hàm ký hiệu?

A. It describes a local infinitesimal change | Nó mô tả độ thay đổi cục bộ rất nhỏ

B. It often uses one-sided differences | Nó thường dùng sai phân một phía

C. 2 | 2

D. `sympy.diff` | `sympy.diff`

Đáp án: D

Giải thích: SymPy hỗ trợ đạo hàm ký hiệu bằng diff.

Q20. Why is a numerical gradient at a boundary less accurate? | Vì sao gradient số ở biên có thể kém chính xác hơn?

A. `sympy.diff` | `sympy.diff`

B. It describes a local infinitesimal change | Nó mô tả độ thay đổi cục bộ rất nhỏ

C. 2 | 2

D. It often uses one-sided differences | Nó thường dùng sai phân một phía

Đáp án: D

Giải thích: Ước lượng ở đầu dãy thường dùng dữ liệu một phía.

8. Bài tiếp theo

`Hàm số → Limit → Derivative → Gradient.`

PY-32 — Đạo hàm riêng, gradient descent và learning rate

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Partial derivative — đạo hàm riêng; **Gradient** — vector gradient.

`Derivative → Gradient → Optimization → Training`

Học xong dùng để làm gì? Cập nhật tham số từng bước để giảm loss, phát hiện hiện tượng học quá nhanh gây dao động.

Kết nối kiến thức tiếp theo: PY-33 dùng Jacobian/Hessian để xem độ nhạy và độ cong trong bài toán nhiều biến.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Partial derivative	đạo hàm riêng	Đạo hàm theo một biến khi các biến khác giữ cố định
Gradient	vector gradient	Vector gồm các đạo hàm riêng của hàm nhiều biến
Learning rate	tốc độ học	Bước nhân gradient khi cập nhật tham số
Epoch	vòng huấn luyện	Một lượt sử dụng hết dữ liệu train để cập nhật mô hình

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`sympy.diff(f,x)`	Đạo hàm riêng của hàm f theo x
`grad=[df/dx,df/dy,...]`	Ghép các đạo hàm riêng thành vector
`w = w - lr * gradient`	Cập nhật đi ngược hướng tăng loss
`np.mean((pred-y)**2)`	Loss MSE mẫu cho tối ưu hồi quy

4. Khái niệm chính và liên quan

Mạch kiến thức: Đạo hàm riêng đo độ dốc theo từng tham số; gradient gom các độ dốc; gradient descent dùng hướng ngược gradient với learning rate thích hợp; epoch nói về số lượt đi qua tập train, không mặc định bằng số bước cập nhật.

4.1. Partial derivative — Độ dốc theo một tham số

Từ vựng: Partial derivative — đạo hàm riêng; **Fixed variable** — biến giữ cố định.

Một mô hình có hai tham số x,y và loss $f(x,y)=x^2+3xy+y^2$. **Đạo hàm riêng theo x** giả sử y giữ nguyên, được $2x+3y$, còn theo y là $3x+2y$. Ở điểm $(1,2)$, hai độ dốc lần lượt 8 và 7. Đạo hàm riêng khác đạo hàm hàm một biến vì ta phải chỉ rõ **biến đang thay đổi**.

No Note — Không chú thích

```
import sympy as sp

x, y = sp.symbols('x y')
f = x ** 2 + 3 * x * y + y ** 2
print(sp.diff(f, x))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm đạo hàm riêng của hàm có hai tham số, chỉ xét thay đổi theo x.
# Dữ liệu: f=x²+3xy+y²; x và y là hai biến độc lập, tính đạo hàm theo x khi y được giữ cố định.
# SymPy tính đạo hàm biểu thức có nhiều biến bằng ký hiệu.
import sympy as sp

# Tạo hai biến x, y tượng trưng; có thể đạo hàm theo một biến trong khi giữ biến kia.
x, y = sp.symbols('x y')
# Biểu thức có phần x², tương tác xy và y².
f = x ** 2 + 3 * x * y + y ** 2
# diff(f,x) tính ĐẠO HÀM RIÊNG theo x, xem y như hằng số; kết quả 2x+3y, không phải đạo hàm theo cả hai.
print(sp.diff(f, x))
```

Kết quả khi chạy:

```
2*x + 3*y
```

Đạo hàm riêng theo x bằng 2x+3y, coi y là hằng số.

4.2. Gradient — Tập hợp độ dốc của tất cả tham số

Từ vựng: Gradient — vector gradient; **Parameter** — tham số mô hình.

Nếu loss phụ thuộc nhiều tham số, **gradient** gom các đạo hàm riêng **theo đúng thứ tự tham số**: $\nabla f=(\partial f/\partial x,\ \partial f/\partial y)$. Với $f=x^2+y^2$ tại $(3,4)$, gradient $[6,8]$ là vector hai chiều. Gradient chỉ **hướng tăng nhanh nhất cục bộ** của f; đi theo **gradient âm** giúp thử giảm loss khi bước đủ nhỏ.

No Note — Không chú thích

```
import numpy as np

params = np.array([3.0, 4.0])
print((2 * params).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính gradient của tổng bình phương hai tham số qua phép toán vector hóa.
# Dữ liệu: Tham số [3,4], hàm mục tiêu ngầm f=w1²+w2², gradient là hai đạo hàm riêng.
```

```
# NumPy cho tính trực tiếp theo từng phần tử của vector.
import numpy as np

# Vector hai tham số w1=3, w2=4, dtype float.
params = np.array([3.0, 4.0])
# Với loss là tổng bình phương tham số, gradient là 2*w ở từng thành phần, không phải một độ dốc
scalar.
# tolist() chuyển ndarray gradient thành list Python [6,8] để hiển thị.
print((2 * params).tolist())
```

Kết quả khi chạy:

```
[6.0, 8.0]
```

Gradient tại (3,4) có hai thành phần: đạo hàm theo x bằng 6, theo y bằng 8.

4.3. Learning rate — Độ dài bước cập nhật

Từ vựng: Learning rate — tốc độ học; **Update** — cập nhật.

Gradient descent cập nhật tham số bằng `w_new = w − lr × gradient`. Với $L(w)=w^2$, ta có đạo hàm $2w$. Bắt đầu w=3 và lr=0.1, gradient=6 nên w mới bằng $3-0.1\times6=2.4$. **Learning rate** không phải một đạo hàm; nó quyết định bước đi. Quá lớn có thể dao động/phân kỳ, quá nhỏ có thể rất chậm.

No Note — Không chú thích

```
w = 3.0
learning_rate = 0.1
w = w - learning_rate * 2 * w
print(round(w, 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thực hiện một bước gradient descent với learning rate 0.1 trên hàm w².
# Dữ liệu: Trọng số w=3 ban đầu, độ dốc là 2w; learning_rate=0.1 là độ lớn bước đi theo hướng giảm.
# w là một tham số cần tối ưu, ban đầu bằng 3.
w = 3.0
# Tốc độ học 0.1 nhân với gradient để quyết định kích thước mỗi bước; quá lớn có thể vượt điểm tối
ưu.
learning_rate = 0.1
# Đạo hàm w² là 2w; lấy w trừ learning_rate×gradient để đi NGƯỢC chiều tăng loss.
w = w - learning_rate * 2 * w
# round(...,2) làm tròn trọng số mới để in, không phải số epoch hoặc loss.
print(round(w, 2))
```

Kết quả khi chạy:

```
2.4
```

Một bước đi ngược gradient đưa tham số từ 3 xuống 2.4, gần cực tiểu w=0 hơn.

4.4. Epoch — Một lượt qua dữ liệu train

Từ vựng: Epoch — một vòng qua tập train; **Step** — một bước cập nhật.

Epoch thường là một lượt xử lý toàn bộ dữ liệu train. Nó **không bằng một bước gradient** trong mọi trường hợp: khi huấn luyện minibatch, một epoch có nhiều batch và nhiều lần cập nhật. Để minh họa vòng lặp, có thể in số thứ tự epoch 1, 2, 3. Với **gradient descent một batch toàn bộ dữ liệu**, mỗi epoch thường tương ứng một lần cập nhật; cách đếm và kiểm tra loss cần nhất quán.

No Note — Không chú thích

```
for epoch in range(1, 4):
    print(epoch)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa epoch là một vòng huấn luyện qua dữ liệu, bằng ba lần đếm.
# Dữ liệu: Ba epoch đánh số´1..3; ví dụ chỉ in thứ tự, chưa thật sự fit mô hình.
# range(start,stop) sinh 1,2,3; cận cuối 4 bị loại.
# Mỗi vòng tượng trưng một epoch, tức một lượt học trọn bộ train trong ứng dụng thật.
for epoch in range(1, 4):
    # Ở đây chỉ hiển thị số´epoch, không có bước cập nhật mô hình hay dữ liệu train.
    print(epoch)
```

Kết quả khi chạy:

```
1
2
3
```

Có ba lượt epoch được đánh số 1 đến 3; đây mới là bộ đếm, không phải bản thân việc huấn luyện.

5. Năm tình huống Gradient Descent — từ một bước đến hồi quy hai tham số

Tình huống 1 — Cấp 1: Một bước gradient từ 10 điểm giá trị mục tiêu

Từ vựng trong mục: Gradient — hướng dốc; **Learning rate** — tốc độ học; **Loss** — mức sai.

Tình huống thực tế: Một công cụ dự báo nhận **10 điểm mục tiêu từ 2 đến 11**, mỗi điểm đại diện một lượt đo giả lập. Công cụ đang quá đơn giản: nó trả **cùng một số w cho tất cả 10 lượt**, ban đầu w = 0. Người học muốn thử **một lần điều chỉnh w** để dự báo đỡ lệch hơn, rồi so **MSE trước và sau**, chứ không thay đổi từng điểm đo.

Vì sao chọn cách này? Loss (mất mát) đo mức mô hình đoán lệch so với điểm mục tiêu; **MSE** lấy trung bình những sai lệch đã bình phương nên các lần sai nhiều bị phạt nặng hơn. **Gradient** nói cho ta biết khi thay đổi w rất ít thì loss có xu hướng tăng theo phía nào. Chương trình tính gradient từ **đủ 10 mục tiêu**, sau đó bước **ngược hướng gradient** để thử giảm sai số. **Learning rate 0,1** quyết định độ lớn bước sửa, không phải 10% số điểm đo; sau đúng một bước, tính lại MSE để kiểm chứng, không mặc định hễ cập nhật là tốt hơn.

Quy trình:

1. Tạo 10 số mục tiêu và đặt w=0.
2. Tính MSE, gradient rồi cập nhật với lr=0,1.
3. So MSE trước/sau và in giá trị w mới.

No Note — Không chú thích

```
import numpy as np
targets = np.array([2,3,4,5,6,7,8,9,10,11], dtype=float)
w = 0.0
before = np.mean((w - targets)**2)
gradient = 2 * np.mean(w - targets)
w = w - 0.1 * gradient
after = np.mean((w - targets)**2)
if len(targets) != 10:
    raise ValueError("Cần đúng 10 bản ghi mục tiêu.")
print("Bản ghi:", len(targets))
print(f"Trọng số sau bước: {w:.2f}")
print("Loss giảm:", bool(after < before))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Giảm MSE dự đoán một hằng w trên 10 mục tiêu bằng một bước gradient.
# Dữ liệu: 10 số mục tiêu 2..11, w khởi tạo 0; hàm loss là trung bình bình phương (w-target).
# NumPy vector hóa hiệu dự đoán và mục tiêu cho toàn bộ train.
import numpy as np
# Mảng 10 mục tiêu số thực, một giá trị trên mỗi bản ghi.
targets = np.array([2,3,4,5,6,7,8,9,10,11], dtype=float)
# Khởi tạo dự đoán hằng cho mọi mẫu bằng 0 trước học.
w = 0.0
# Trừ w cho từng target, **2 bình phương và mean() lấy MSE ban đầu.
before = np.mean((w - targets)**2)
# Gradient theo tham số hằng w của MSE bằng hai lần trung bình sai lệch; không dùng độ lệch tuyệt
đối.
gradient = 2 * np.mean(w - targets)
# Cập nhật w ngược chiều gradient với learning rate=0.1, chỉ MỘT bước.
w = w - 0.1 * gradient
# Trừ w cho từng target, **2 bình phương và mean() lấy MSE ban đầu.
# Tính lại MSE với w mới để so sau bước cập nhật.
after = np.mean((w - targets)**2)
# len() xác minh đủ 10 mẫu huấn luyện.
if len(targets) != 10:
    # raise ValueError dừng khi số mẫu hoặc số lần cập nhật không đúng điều kiện kiểm tra.
    raise ValueError("Cần đúng 10 bản ghi mục tiêu.")
print("Bản ghi:", len(targets))
print(f"Trọng số sau bước: {w:.2f}")
# So loss sau với trước; bool() đưa kết quả NumPy thành Boolean Python.
print("Loss giảm:", bool(after < before))
```

Kết quả khi chạy:

```
Bản ghi: 10
Trọng số sau bước: 1.30
Loss giảm: True
```

Một bước chưa đưa w tới giá trị tối ưu, nhưng cho thấy hướng cập nhật làm sai số trên mười điểm giảm.

Tình huống 2 — Cấp 2: Đạo hàm riêng hệ số từ 12 cặp x/y có chặn cố định

Từ vựng trong mục: Partial derivative — đạo hàm riêng; **Coefficient** — hệ số; **Full batch** — toàn bộ dữ liệu.

Tình huống thực tế: Trạm thử nghiệm có **12 cặp x và y**, x là mức đầu vào từ **1 đến 12**, y là chỉ số kết quả được tạo theo đường thẳng có **mức nền 1** và phần tăng phụ thuộc x. Mô hình đã biết mức nền nhưng

chưa biết **mức tăng cho một đơn vị x**, gọi là w, nên bắt đầu thử w = 0. Nhóm muốn dùng **cả 12 cặp** để điều chỉnh w một lần và kiểm tra xem mức sai lệch trung bình có giảm không.

Vì sao chọn cách này? Với mô hình đường thẳng này, mức nền **1 đã cố định**, chỉ còn phải học **độ dốc w**. Khi dự báo của mẫu x còn thấp hơn y, mẫu có x lớn cũng ảnh hưởng mạnh hơn tới việc phải chỉnh w; vì vậy gradient tính từ **sai lệch của từng hàng nhân với x**, rồi lấy trung bình 12 mẫu. `w -= 0.005 * gradient` nghĩa là chỉnh w **ngược hướng loss tăng**, với bước khá nhỏ vì x lên tới 12. Cập nhật bằng cách chỉ lấy sai số của một mẫu sẽ không đại diện cho đủ dữ liệu; ngược lại bước quá lớn có thể vượt qua mức phù hợp.

Quy trình:

1. Tạo 12 hàng, dự báo bằng `w*x+1`.
2. Tính đạo hàm loss theo w trên toàn bộ 12 hàng.
3. Cập nhật một bước, xác nhận MSE sau cập nhật nhỏ hơn trước.

No Note — Không chú thích

```
import numpy as np
x = np.array([1,2,3,4,5,6,7,8,9,10,11,12], dtype=float)
y = 2*x + 1
w = 0.0
before = np.mean((w*x + 1 - y)**2)
gradient = 2*np.mean((w*x + 1 - y)*x)
w -= 0.005*gradient
after = np.mean((w*x + 1 - y)**2)
if x.shape != (12,) or y.shape != (12,):
    raise ValueError("Cần 12 cặp x và y.")
print("Mẫu:", len(x))
print(f"Gradient ban đầu: {gradient:.2f}")
print(f"Hệ số´ sau bước: {w:.3f}")
print("Loss giảm:", bool(after < before))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Điều chỉnh hệ số´ dốc w của mô hình y≈w*x+1 bằng một bước gradient trên 12 mẫu.
# Dữ liệu: 12 đầu vào x=1..12, nhãn y=2x+1; chặn b=1 đã cố´ định, chỉ cần học w.
# NumPy hỗ˜ trợ lấy mean của lỗi theo 12 mẫu.
import numpy as np
# 12 đặc trưng x số´ thực tương ứng với từng nhãn y.
x = np.array([1,2,3,4,5,6,7,8,9,10,11,12], dtype=float)
# Tạo nhãn giả lập đúng từ đường thẳng độ dốc 2, chặn 1.
y = 2*x + 1
# w khởi tạo 0, trong khi hệ số´ thực là 2.
w = 0.0
# MSE của dự đoán w*x+1 so với y, dùng lỗi theo từng hàng.
before = np.mean((w*x + 1 - y)**2)
# Gradient MSE theo w: sai lệch nhân thêm x (vì dự đoán phụ thuộc w qua w*x), rồi mean×2.
gradient = 2*np.mean((w*x + 1 - y)*x)
# -= là gán w mới bằng w cũ trừ learning rate 0.005 nhân gradient; tốc độ nhỏ vì x có thể˙ lớn.
w -= 0.005*gradient
# MSE của dự đoán w*x+1 so với y, dùng lỗi theo từng hàng.
# Tính lại MSE sau cập nhật để˙ xem một bước có giảm lỗi không.
after = np.mean((w*x + 1 - y)**2)
# shape (12,) bảo đảm cả x và y có 12 mẫu khớp vị trí.
if x.shape != (12,) or y.shape != (12,):
    # raise ValueError dừng khi số´ mẫu hoặc số´ lần cập nhật không đúng điều kiện kiểm tra.
    raise ValueError("Cần 12 cặp x và y.")
print("Mẫu:", len(x))
# f-string :.2f định dạng gradient hai số´ thập phân.
```

```
print(f"Gradient ban đầu: {gradient:.2f}")
# f-string :.3f định dạng trọng sô´mới ba sô´thập phân.
print(f"Hệ sô´sau bước: {w:.3f}")
print("Loss giảm:", bool(after < before))
```

Kết quả khi chạy:

```
Mẫu: 12
Gradient ban đầu: -216.67
Hệ sô´sau bước: 1.083
Loss giảm: True
```

Gradient của w không phải hệ số w: gradient phụ thuộc **tất cả x và sai số dự báo**.

Tình huống 3 — Cấp 3: Theo dõi 6 bước tối ưu trên 15 mẫu chuẩn hóa

Từ vựng trong mục: Gradient descent — hạ dốc; **Iteration** — vòng cập nhật; **Loss history** — lịch sử sai số.

Tình huống thực tế: Mô hình được giao **15 mẫu đầu vào x đã đưa về khoảng 0 đến 1**, mỗi mẫu có mục tiêu y đúng bằng hai lần x. Ban đầu mô hình dự báo theo quy tắc nhân x với hệ số **w = 0**, nên kết quả thấp hơn mục tiêu. Người học sẽ điều chỉnh w theo **6 bước học**, ghi lại mức mất mát tại mỗi bước và kiểm tra các mức ghi được có giảm dần hay không, thay vì chỉ nhìn hệ số cuối.

Vì sao chọn cách này? Gradient descent (giảm dần theo gradient) là quá trình thử sửa tham số theo hướng làm sai số nhỏ đi, rồi đo lại nhiều lần. Chương trình tính sai lệch của **15 mẫu**, ghi MSE vào `history`, tính gradient và mới chỉnh w cho lượt kế. Như vậy `history` chứa **6 lần đo loss trước mỗi lần cập nhật**, không phải sáu giá trị đều đo sau bước chỉnh tương ứng. Dùng vòng `for` giúp lặp cùng quy trình; so các số trong `history` theo thứ tự để phát hiện loss không giảm. Việc chọn learning rate **0,2** chỉ phù hợp với ví dụ đầu vào đã chuẩn hóa này, không phải mức tốt cho mọi bài toán.

Quy trình:

1. Chuẩn bị đúng 15 cặp x/y.
2. Lặp sáu lần: đo loss, tính gradient toàn batch rồi cập nhật w.
3. Kiểm lịch sử loss giảm, in bước đầu/cuối và hệ số mới.

No Note — Không chú thích

```
import numpy as np
x =
np.array([0.06666666666666667,0.13333333333333333,0.2,0.26666666666666666,0.3333333333333333,0.4,0.
4666666666666667,0.5333333333333333,0.6,0.6666666666666666,0.7333333333333333,0.8,0.866666666666666
7,0.9333333333333333,1], dtype=float)
y = 2*x
w = 0.0
history = []
for step in range(6):
    error = w*x - y
    history.append(float(np.mean(error**2)))
    w -= 0.2*2*np.mean(error*x)
if len(x) != 15 or len(history) != 6:
    raise ValueError("Cần 15 mẫu và 6 bước kiểm tra Loss.")
print("Mẫu:", len(x))
print("Bước cập nhật:", len(history))
print(f"Loss đầu: {history[0]:.3f}")
print(f"Loss bước 6: {history[-1]:.3f}")
```

```
decreases = True
for before, after in zip(history, history[1:]):
    if before <= after:
        decreases = False
        break
print("Giảm từng bước:", decreases)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Theo dõi 6 giá trị MSE trong quá trình học trọng số́trên 15 điểm của y=2x.
# Dữ liệu: 15 x trong (0,1], nhãn y=2x; w từ 0, learning rate 0.2 và 6 lần cập nhật.
# NumPy thực hiện vector hóa tính dự đoán/lỗi cho cả 15 mẫu.
import numpy as np
# 15 đầu vào x từ 1/15 đến 1 (số́thực), giúp mức gradient vừa phải.
x =
np.array([0.0666666666666667,0.1333333333333333,0.2,0.2666666666666666,0.3333333333333333,0.4,0.
4666666666666667,0.5333333333333333,0.6,0.6666666666666666,0.7333333333333333,0.8,0.866666666666666
7,0.9333333333333333,1], dtype=float)
# Nhãn đích là đường thẳng hệ số́thực 2, không có chặn.
y = 2*x
# Trọng số́dự đoán ban đầu bằng 0.
w = 0.0
# List lưu loss TRƯỚC mỗi bước cập nhật, tương ứng 6 lần kiểm.
history = []
# range(6) có 6 lượt step=0..5, không lấy 6.
for step in range(6):
    # Dự đoán vector w*x trừ nhãn y để được sai số́từng mẫu.
    error = w*x - y
    # **2 bình phương lỗi rồi mean() tính MSE tại w hiện tại.
    # float() đổi NumPy scalar loss thành số́Python; append() lưu trước khi đổi w.
    history.append(float(np.mean(error**2)))
    # Gradient theo w là 2×mean(error*x); -= cập nhật ngược chiều với learning rate=0.2.
    w -= 0.2*2*np.mean(error*x)
# Kiểm đúng 15 mẫu và 6 bản ghi loss.
if len(x) != 15 or len(history) != 6:
    # raise ValueError dừng khi số́mẫu hoặc số́lần cập nhật không đúng điều kiện kiểm tra.
    raise ValueError("Cần 15 mẫu và 6 bước kiểm tra Loss.")
print("Mẫu:", len(x))
print("Bước cập nhật:", len(history))
# [0] là loss trước bước 1, [-1] loss ngay TRƯỚC bước 6 (không phải loss sau bước 6).
print(f"Loss đầu: {history[0]:.3f}")
print(f"Loss bước 6: {history[-1]:.3f}")
decreases = True
# Ghép từng loss với loss tiếṕ theo nhờ [1:] bỏ mục đầu để so mức giảm.
for before, after in zip(history, history[1:]):
    # Nếu loss mới không nhỏ hơn loss trước, đặt cờ False và break sớm.
    if before <= after:
        decreases = False
        break
print("Giảm từng bước:", decreases)
```

Kết quả khi chạy:

```
Mẫu: 15
Bước cập nhật: 6
Loss đầu: 1.470
Loss bước 6: 0.300
Giảm từng bước: True
```

Sáu **bước full-batch** ở đây cũng tương ứng sáu lượt xem toàn bộ 15 mẫu; nếu dùng minibatch, số bước và số epoch có thể khác.

Tình huống 4 — Cấp 4: So learning rate ổn định và quá lớn trên 18 mục tiêu

Từ vựng trong mục: Learning rate — độ dài bước; **Overshoot** — bước vượt quá; **Divergence** — phân kỳ.

Tình huống thực tế: Một hệ thống dự đoán **một số chung w** cho **18 điểm mục tiêu từ 10 đến 15** (mỗi giá trị xuất hiện ba lần). Nhóm muốn xem vì sao chọn mức học **0,1** có thể cải thiện sau **5 lần cập nhật**, còn mức học quá lớn **1,1** có thể khiến dự đoán liên tục nhảy qua mức phù hợp và càng sai. Cả hai phép thử phải bắt đầu cùng w = 0 và dùng đúng 18 điểm để so sánh công bằng.

Vì sao chọn cách này? Learning rate là độ lớn mỗi lần chỉnh tham số. Với loss dạng **chiếc bát hướng lên**, đáy bát là vị trí dự báo chung w khiến sai số nhỏ nhất. Learning rate **0,1** đưa w tiến dần về vùng đáy, còn **1,1** lớn quá làm nó nhảy từ phía này qua phía kia của đáy với biên độ tăng; sau 5 bước MSE có thể **cao hơn cả ban đầu**. Hàm `train(lr)` cho hai lần chạy cùng **18 mục tiêu** và cùng xuất phát w = 0, chỉ đổi learning rate để thấy rõ nguyên nhân. Không phải mọi learning rate lớn đều sai; vấn đề là **lớn so với độ cong của loss**.

Quy trình:

1. Tạo 18 giá trị mục tiêu và MSE cho dự báo hằng w.
2. Chạy hai bản sao 5 bước, chỉ khác learning rate.
3. Báo loss ban đầu, hai loss cuối và kiểm bản tốc độ học lớn bị tăng loss.

No Note — Không chú thích

```
import numpy as np
targets = np.array([10,11,12,13,14,15,10,11,12,13,14,15,10,11,12,13,14,15], dtype=float)
def train(lr):
    w = 0.0
    for step in range(5):
        gradient = 2*np.mean(w - targets)
        w -= lr*gradient
    return float(np.mean((w - targets)**2))
initial = float(np.mean(targets**2))
small = train(0.1)
large = train(1.1)
if len(targets) != 18:
    raise ValueError("Cần 18 giá trị mục tiêu khi so learning rate.")
print("Mẫu:", len(targets))
print(f"Loss đầu: {initial:.2f}")
print(f"Loss sau lr=0.1: {small:.2f}")
print(f"Loss sau lr=1.1: {large:.2f}")
print("lr lớn làm loss tăng:", bool(large > initial))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So learning rate nhỏ 0.1 với quá lớn 1.1 khi fit 18 mục tiêu bằng một tham số hằng.
# Dữ liệu: 18 mục tiêu số thực 10..15 lặp ba lần, mỗi thử nghiệm học 5 bước từ w=0.
# NumPy hỗ trợ MSE và gradient trung bình.
import numpy as np
# 18 mục tiêu cùng đơn vị giả lập, dùng chung cho hai tốc độ học.
targets = np.array([10,11,12,13,14,15,10,11,12,13,14,15,10,11,12,13,14,15], dtype=float)
# Định nghĩa hàm thử nghiệm nhận learning rate lr, trả loss sau 5 bước, mỗi lần khởi động w=0.
def train(lr):
    # Khởi tạo w lại trong hàm để so hai lr công bằng.
    w = 0.0
    # Năm lượt cập nhật liên tục, không tạo dữ liệu mới.
    for step in range(5):
```

```
        # Gradient của trung bình bình phương sai lệch theo hằng w; còn dấu cho biết chiều giảm.
        gradient = 2*np.mean(w - targets)
        # Learning rate nhân độ dốc rồi trừ; với lr=1.1, bước có thể vượt qua điểm tối ưu và dao
động tăng.
        w -= lr*gradient
    # Hàm train trả MSE cuối dưới dạng float Python.
    return float(np.mean((w - targets)**2))
# Vì w=0, MSE trước học bằng mean(target²); dùng làm mốc ban đầu.
initial = float(np.mean(targets**2))
# Chạy năm bước với lr nhỏ, thường giảm dần loss.
small = train(0.1)
# Chạy từ đầu với lr=1.1; lớn hơn ngưỡng ổn định cho bài toán hằng w, có thể làm loss tăng.
large = train(1.1)
if len(targets) != 18:
    # raise ValueError dừng khi số́ mẫu hoặc số́ lần cập nhật không đúng điều kiện kiểm tra.
    raise ValueError("Cần 18 giá trị mục tiêu khi so learning rate.")
print("Mẫu:", len(targets))
print(f"Loss đầu: {initial:.2f}")
print(f"Loss sau lr=0.1: {small:.2f}")
print(f"Loss sau lr=1.1: {large:.2f}")
# Cờ True báo loss lr lớn vượt loss trước học, không phải cứ lr lớn luôn xấu với mọi bài toán.
print("lr lớn làm loss tăng:", bool(large > initial))
```

Kết quả khi chạy:

```
Mẫu: 18
Loss đầu: 159.17
Loss sau lr=0.1: 19.69
Loss sau lr=1.1: 970.38
lr lớn làm loss tăng: True
```

Learning rate lớn không **luôn** gây phân kỳ, nhưng ở hàm và bước cập nhật cụ thể này, nó khiến quá trình không hội tụ.

Tình huống 5 — Cấp 5: Huấn luyện hai tham số hồi quy với 20 cặp đo qua 300 epoch

Từ vựng trong mục: Batch gradient — gradient toàn bộ; **Epoch** — lượt học; **MSE** — sai số bình phương.

Tình huống thực tế: Nhóm nghiên cứu có **20 cặp đầu vào–đầu ra**, x từ **0 đến 1,9** và y được tạo theo quy tắc đường thẳng có **độ dốc 2, mức nền 1**. Mô hình ban đầu không biết cả hai thông số **w** (độ dốc) và **b** (mức nền), đều khởi đầu 0. Nhóm muốn cập nhật hai thông số bằng **300 epoch**, mỗi epoch sử dụng cả **20 mẫu**, rồi xem w, b có gần giá trị tạo dữ liệu và MSE có thấp hơn ban đầu không.

Vì sao chọn cách này? Ở đây **w** quy định mức y tăng khi x tăng, còn **b** là y dự báo khi x = 0. Nếu chỉ học w mà cố định b = 0, mô hình sẽ không diễn tả đúng dữ liệu có mức nền 1. Trong mỗi **epoch**, chương trình dùng cả **20 dòng** để tính hai gradient riêng `dw`, `db`, sau đó sửa **cả hai tham số** bằng learning rate 0,1. **Full-batch** nghĩa là mỗi epoch xét toàn bộ mẫu, không phải lấy một mẫu bất kỳ. Chạy **300 epoch** chỉ là thiết lập cho bài minh họa đã biết dữ liệu; phải xem **w, b cuối và MSE** mới kết luận mô hình đã học gần quy tắc tạo dữ liệu, không lấy số epoch làm bằng chứng tự thân.

Quy trình:

1. Tạo 20 mẫu (x,y) và đặt w=b=0.
2. Trong mỗi epoch, tính lỗi 20 hàng, gradient cho w và b, cập nhật cùng learning rate 0,1.
3. So loss đầu/cuối, in hai tham số cuối và xác nhận MSE giảm.

No Note — Không chú thích

```
import numpy as np
x = np.array([0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1,1.1,1.2,1.3,1.4,1.5,1.6,1.7,1.8,1.9],
dtype=float)
y = 2*x + 1
w, b = 0.0, 0.0
initial = float(np.mean((w*x+b-y)**2))
for epoch in range(300):
    error = w*x+b-y
    dw = 2*np.mean(error*x)
    db = 2*np.mean(error)
    w -= 0.1*dw
    b -= 0.1*db
mse = float(np.mean((w*x+b-y)**2))
if x.size != 20:
    raise ValueError("Cần đủ 20 điểm train.")
print("Mẫu:", len(x))
print("Epoch:", 300)
print(f"Trọng số w: {w:.2f}")
print(f"Chặn b: {b:.2f}")
print(f"MSE cuối: {mse:.6f}")
print("MSE giảm:", mse < initial)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Huấn luyện hai tham số w,b qua 300 epoch để khớp đường y=2x+1 trên 20 mẫu.
# Dữ liệu: 20 đầu vào x=0..1.9, y=2x+1; tham số w,b đều khởi tạo 0 và learning rate 0.1.
# NumPy cho phép tính gradient trên toàn bộ 20 mẫu mỗi epoch (batch gradient descent).
import numpy as np
# 20 x thực từ 0 đến 1.9, mỗi giá trị gắn một y tương ứng.
x = np.array([0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1,1.1,1.2,1.3,1.4,1.5,1.6,1.7,1.8,1.9],
dtype=float)
# Sinh nhãn mục tiêu từ độ dốc 2 và chặn 1.
y = 2*x + 1
# Hai tham số ban đầu: w là độ dốc, b là hệ số chặn.
w, b = 0.0, 0.0
# Tính MSE đầu với w=b=0 trước khi học.
# Tính MSE cuối sau 300 cập nhật và đổi thành float Python.
initial = float(np.mean((w*x+b-y)**2))
# 300 epoch; mỗi vòng cập nhật dùng tất cả mẫu, không phải 300 bản ghi khác nhau.
for epoch in range(300):
    # Dự đoán hiện tại trừ nhãn thật tại từng điểm; error là vector 20 phần tử.
    error = w*x+b-y
    # dw là đạo hàm MSE theo w: nhân mỗi sai số với x, lấy trung bình rồi ×2.
    dw = 2*np.mean(error*x)
    # db là đạo hàm MSE theo b vì dự đoán phụ thuộc b với hệ số 1.
    db = 2*np.mean(error)
    # Trừ 0.1×gradient để đi theo hướng giảm loss, cập nhật độ dốc.
    w -= 0.1*dw
    # Cập nhật chặn b bằng learning rate 0.1 và gradient riêng db.
    b -= 0.1*db
# Tính MSE đầu với w=b=0 trước khi học.
# Tính MSE cuối sau 300 cập nhật và đổi thành float Python.
mse = float(np.mean((w*x+b-y)**2))
# size đếm đúng 20 mẫu train.
if x.size != 20:
    # raise ValueError dừng khi số mẫu hoặc số lần cập nhật không đúng điều kiện kiểm tra.
    raise ValueError("Cần đủ 20 điểm train.")
print("Mẫu:", len(x))
print("Epoch:", 300)
# f-string :.2f in độ dốc gần 2, có làm tròn hiển thị.
print(f"Trọng số w: {w:.2f}")
# In hệ số chặn gần 1 với hai chữ số thập phân.
print(f"Chặn b: {b:.2f}")
# In MSE cuối với sáu chữ số thập phân để thấy độ lỗi nhỏ.
```

```
print(f"MSE cuối: {mse:.6f}")
# So loss cuối với trước học, không đồng nghĩa đã kiểm mô hình trên tập test độc lập.
print("MSE giảm:", mse < initial)
```

Kết quả khi chạy:

```
Mẫu: 20
Epoch: 300
Trọng số w: 2.00
Chặn b: 1.00
MSE cuối: 0.000000
MSE giảm: True
```

Vì dữ liệu được tạo đúng theo đường thẳng, hồi quy có thể đạt MSE rất nhỏ. Với dữ liệu thực có nhiễu phải đánh giá trên tập chưa dùng để học.

6. Sai lầm thường gặp

- Đạo hàm riêng chỉ thay một tham số; gradient chứa nhiều đạo hàm riêng theo đúng thứ tự tham số.
- Gradient chỉ hướng **tăng nhanh nhất**, nên gradient descent thường trừ gradient.
- Learning rate quá lớn có thể phân kỳ; quá nhỏ khiến học chậm.
- Một epoch không luôn tương ứng một lần cập nhật vì minibatch có nhiều bước.
- Ghi loss giúp theo dõi tiến triển nhưng loss train giảm chưa bảo đảm kết quả validation tốt.
- Khi tính MSE theo dữ liệu, phải ghép đúng từng mẫu x với nhãn y.

7. Trắc nghiệm

Q1. What is a partial derivative? | Đạo hàm riêng là gì?

A. A vector of partial derivatives | Vector gồm các đạo hàm riêng

B. One pass through the training dataset | Một lượt đi qua tập dữ liệu train

C. A factor controlling an optimization update step | Hệ số điều khiển độ lớn bước cập nhật tối ưu

D. Change rate in one variable while others are held fixed | Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

Đáp án: D

Giải thích: Chỉ thay một biến và giữ biến khác cố định.

Q2. What is a gradient? | Gradient là gì?

A. Change rate in one variable while others are held fixed | Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

B. One pass through the training dataset | Một lượt đi qua tập dữ liệu train

C. A factor controlling an optimization update step | Hệ số điều khiển độ lớn bước cập nhật tối ưu

D. A vector of partial derivatives | Vector gồm các đạo hàm riêng

Đáp án: D

Giải thích: Các đạo hàm riêng hợp thành một vector.

Q3. What is a learning rate? | Learning rate là gì?

A. A factor controlling an optimization update step | Hệ số điều khiển độ lớn bước cập nhật tối ưu

B. Change rate in one variable while others are held fixed | Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

C. A vector of partial derivatives | Vector gồm các đạo hàm riêng

D. One pass through the training dataset | Một lượt đi qua tập dữ liệu train

Đáp án: A

Giải thích: Nó nhân mức độ cập nhật.

Q4. What is an epoch in training? | Epoch trong huấn luyện là gì?

A. One pass through the training dataset | Một lượt đi qua tập dữ liệu train

B. Change rate in one variable while others are held fixed | Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

C. A vector of partial derivatives | Vector gồm các đạo hàm riêng

D. A factor controlling an optimization update step | Hệ số điều khiển độ lớn bước cập nhật tối ưu

Đáp án: A

Giải thích: Một epoch xử lý hết dữ liệu train một lượt.

Q5. Given $L(w) = (w - 2)^2$, what is the gradient at $w = 5$? | Cho $L(w) = (w - 2)^2$, gradient tại $w = 5$ bằng bao nhiêu?

A. 4 | 4

B. 6 | 6

C. 2 | 2

D. 10 | 10

Đáp án: B

Giải thích: Đạo hàm $dL/dw = 2(w - 2)$. Thay $w = 5$: $2 \times (5 - 2) = 6$.

Q6. Gradient descent uses new_w = old_w − rate × gradient. If w = 4, rate = 0.25, gradient = 2, what is new_w? | Gradient Descent cập nhật w mới = w cũ − tốc độ học × gradient. Nếu w = 4, rate = 0.25, gradient = 2, w mới bằng bao nhiêu?

A. 3.5 | 3.5

B. 2 | 2

C. 4.5 | 4.5

D. 3 | 3

Đáp án: A

Giải thích: w mới = 4 − 0.25 × 2 = 3.5. Lấy w cũ trừ một bước theo gradient.

Q7. What may happen when learning rate is excessively large? | Điều gì có thể xảy ra khi tốc độ học quá lớn?

A. Training data always doubles | Dữ liệu huấn luyện luôn tăng gấp đôi

B. Loss may oscillate or diverge | Loss có thể dao động hoặc phân kỳ

C. Every gradient becomes zero | Mọi gradient đều bằng 0

D. The global optimum is guaranteed | Luôn tìm được cực tiểu toàn cục

Đáp án: B

Giải thích: Bước nhảy quá lớn có thể vượt qua cực tiểu, khiến hàm mất mát dao động hoặc tăng thay vì giảm.

Q8. A function has gradient zero at w = 0. Which conclusion is always justified? | Một hàm có gradient bằng 0 tại w = 0. Kết luận nào luôn đúng?

A. The loss is zero | Loss bằng 0

B. This is a global minimum | Đây là cực tiểu toàn cục

C. The function is constant | Hàm là hằng số

D. It is a stationary point, not necessarily a minimum | Đây là điểm dừng, chưa chắc là cực tiểu

Đáp án: D

Giải thích: Gradient bằng 0 chỉ cho biết tốc độ thay đổi cục bộ bằng 0. Điểm đó vẫn có thể là cực đại hoặc điểm yên ngựa.

Q9. What is $\partial(x^2+3xy)/\partial x$? | Đạo hàm riêng theo x của x^2+3xy là gì?

A. `2*x+3*y` | `2*x+3*y`

B. `[2*x,2*y]` | `[2*x,2*y]`

C. `2*w` | `2*w`

D. 2.4 | 2.4

Đáp án: A

Giải thích: Lấy đạo hàm x^2 và xy theo x.

Q10. What is the gradient of x^2+y^2? | Gradient của x^2+y^2 là gì?

A. 2.4 | 2.4

B. `2*x+3*y` | `2*x+3*y`

C. `2*w` | `2*w`

D. `[2*x,2*y]` | `[2*x,2*y]`

Đáp án: D

Giải thích: Mỗi đạo hàm bằng hai lần tọa độ.

Q11. For loss w^2, what is its derivative? | Đạo hàm loss w^2 là gì?

A. `[2*x,2*y]` | `[2*x,2*y]`

B. 2.4 | 2.4

C. `2*w` | `2*w`

D. `2*x+3*y` | `2*x+3*y`

Đáp án: C

Giải thích: Dùng quy tắc lũy thừa cho w^2.

Q12. If w=3, lr=0.1 and grad=6, what is next w? | Nếu w=3, lr=0.1 và gradient=6, w mới là gì?

A. `[2*x,2*y]` | `[2*x,2*y]`

B. 2.4 | 2.4

C. `2*w` | `2*w`

D. `2*x+3*y` | `2*x+3*y`

Đáp án: B

Giải thích: $3-0.1\times6=2.4$.

Q13. Why might a large learning rate fail? | Vì sao learning rate lớn dễ thất bại?

A. No, minibatches can cause multiple updates | Không, nhiều minibatch có thể tạo nhiều lần cập nhật

B. Updates can overshoot or diverge | Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

C. It can make convergence slow | Nó có thể làm hội tụ chậm

D. Locally reduce the objective | Giảm hàm mục tiêu về mặt cục bộ

Đáp án: B

Giải thích: Bước lớn có thể vượt qua cực tiểu liên tục.

Q14. Why might a very small learning rate be inefficient? | Vì sao learning rate quá nhỏ kém hiệu quả?

A. Locally reduce the objective | Giảm hàm mục tiêu về mặt cục bộ

B. It can make convergence slow | Nó có thể làm hội tụ chậm

C. Updates can overshoot or diverge | Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

D. No, minibatches can cause multiple updates | Không, nhiều minibatch có thể tạo nhiều lần cập nhật

Đáp án: B

Giải thích: Bước quá nhỏ khiến tối ưu chậm.

Q15. What does moving opposite the gradient aim to do? | Đi ngược gradient nhằm làm gì?

A. Updates can overshoot or diverge | Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

B. No, minibatches can cause multiple updates | Không, nhiều minibatch có thể tạo nhiều lần cập nhật

C. It can make convergence slow | Nó có thể làm hội tụ chậm

D. Locally reduce the objective | Giảm hàm mục tiêu về mặt cục bộ

Đáp án: D

Giải thích: Gradient âm là hướng giảm với bước đủ nhỏ.

Q16. Does one epoch always equal one update? | Một epoch luôn bằng một lần cập nhật không?

A. Locally reduce the objective | Giảm hàm mục tiêu về mặt cục bộ

B. It can make convergence slow | Nó có thể làm hội tụ chậm

C. Updates can overshoot or diverge | Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

D. No, minibatches can cause multiple updates | Không, nhiều minibatch có thể tạo nhiều lần cập nhật

Đáp án: D

Giải thích: Một epoch có thể gồm nhiều batch.

Q17. Why record loss history? | Vì sao lưu lịch sử loss?

A. Steepest increase | Hướng tăng nhanh nhất

B. No, generalization needs validation | Không, cần kiểm chứng trên dữ liệu mới

C. Each error must use its correct sample | Mỗi sai số cần đúng cặp quan sát

D. To inspect optimization progress | Để xem tiến trình tối ưu

Đáp án: D

Giải thích: Lịch sử cho thấy xu hướng giảm hoặc bất ổn.

Q18. Why pair features and targets in the same order? | Vì sao cần ghép x và y cùng thứ tự?

A. No, generalization needs validation | Không, cần kiểm chứng trên dữ liệu mới

B. Each error must use its correct sample | Mỗi sai số cần đúng cặp quan sát

C. To inspect optimization progress | Để xem tiến trình tối ưu

D. Steepest increase | Hướng tăng nhanh nhất

Đáp án: B

Giải thích: Hàng ghép sai làm sai số vô nghĩa.

Q19. Does falling training loss guarantee test accuracy? | Loss train giảm có đảm bảo độ chính xác test?

A. No, generalization needs validation | Không, cần kiểm chứng trên dữ liệu mới

B. To inspect optimization progress | Để xem tiến trình tối ưu

C. Steepest increase | Hướng tăng nhanh nhất

D. Each error must use its correct sample | Mỗi sai số cần đúng cặp quan sát

Đáp án: A

Giải thích: Cần dữ liệu độc lập để đo tổng quát hóa.

Q20. Which direction does a gradient point locally? | Gradient hướng theo phía nào về mặt cục bộ?

A. No, generalization needs validation | Không, cần kiểm chứng trên dữ liệu mới

B. Steepest increase | Hướng tăng nhanh nhất

C. To inspect optimization progress | Để xem tiến trình tối ưu

D. Each error must use its correct sample | Mỗi sai số cần đúng cặp quan sát

Đáp án: B

Giải thích: Gradient hướng tăng cục bộ nhanh nhất.

8. Bài tiếp theo

`Derivative → Gradient → Optimization → Training`.

PY-33 — Jacobian, Hessian và điểm cực trị

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Jacobian — ma trận Jacobian; **Hessian** — ma trận Hessian.

`Gradient → Jacobian → Hessian → Tối ưu mô hình`

Học xong dùng để làm gì? Phân tích nhiều đầu ra thay đổi theo nhiều đầu vào và độ cong của hàm mục tiêu.

Kết nối kiến thức tiếp theo: PY-34 mô hình hóa dữ liệu và sự kiện không chắc chắn bằng xác suất.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Jacobian	ma trận Jacobian	Bảng đạo hàm riêng bậc 1 của hàm nhiều đầu ra
Hessian	ma trận Hessian	Bảng đạo hàm riêng bậc hai của hàm vô hướng
Critical point	điểm tới hạn	Điểm có gradient bằng 0 hoặc không xác định
Curvature	độ cong	Cách độ dốc biến thiên quanh vị trí đang xét

3. Hàm Python và tham số quan trọng

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Cách sử dụng
`sympy.Matrix([...]).jacobian(vars)`	Đạo hàm riêng hàm vector theo biến
`sympy.hessian(f,vars)`	Đạo hàm bậc hai của hàm vô hướng
`sympy.solve(equations,vars)`	Tìm điểm thỏa hệ điều kiện
`np.linalg.eigvalsh(H)`	Xét dấu eigenvalue Hessian đối xứng tại điểm cực trị

4. Khái niệm chính và liên quan

Mạch kiến thức: Với hàm có nhiều đầu ra, Jacobian ghi độ nhạy từng đầu ra theo các biến đầu vào. Với hàm loss một đầu ra, Hessian ghi độ cong theo cặp tham số; kết hợp gradient bằng 0 và dấu trị riêng Hessian mới giúp phân biệt cực trị với yên ngựa.

4.1. Jacobian — Đầu ra thay đổi theo từng đầu vào

Từ vựng: Jacobian — ma trận Jacobian; **Partial derivative** — đạo hàm riêng.

Khi một hàm có **n đầu vào và m đầu ra**, Jacobian là bảng kích thước **m hàng đầu ra × n cột đầu vào**, tại mỗi ô là đạo hàm riêng của đầu ra hàng đó theo đầu vào cột đó. Với F1=x+y, F2=xy, các hàng lần lượt đại diện F1, F2; các cột theo x,y. Tại (x,y)=(3,2), Jacobian bằng [[1,1],[2,3]]. Đây là độ nhạy cục bộ, không phải ma trận số liệu gốc.

No Note — Không chú thích

```
import sympy as sp

x, y = sp.symbols('x y')
F = sp.Matrix([x + y, x * y])
J = F.jacobian([x, y])
print(J.subs({x: 3, y: 2}).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lập Jacobian để đo mức thay đổi của hai đầu ra theo hai đầu vào x,y.
# Dữ liệu: F=(x+y, x*y), điểm kiểm x=3,y=2; Jacobian có hai hàng đầu ra × hai cột đầu vào.
# SymPy xây hàm nhiều biến và đạo hàm riêng bằng ký hiệu.
import sympy as sp

# Tạo hai biến tượng trưng x,y; chưa thế bằng dữ liệu số.
x, y = sp.symbols('x y')
# Matrix tạo vector CỘT gồm hai biểu thức đầu ra: tổng và tích của x,y.
F = sp.Matrix([x + y, x * y])
# jacobian([x,y]) tạo bảng đạo hàm riêng, mỗi HÀNG cho một đầu ra, mỗi CỘT cho một biến; shape
(2,2).
J = F.jacobian([x, y])
# subs({x:3,y:2}) thế đồng thời hai tọa độ để có Jacobian số tại điểm đó.
# tolist() chuyển ma trận SymPy thành list hai hàng để hiển thị dễ đọc.
print(J.subs({x: 3, y: 2}).tolist())
```

Kết quả khi chạy:

```
[[1, 1], [2, 3]]
```

Hàng 1 là độ nhạy của F1 theo x,y; hàng 2 là độ nhạy của F2 theo x,y.

4.2. Hessian — Độ thay đổi của gradient

Từ vựng: Hessian — ma trận Hessian; **Second derivative** — đạo hàm bậc hai.

Hessian là ma trận các đạo hàm riêng **bậc hai** của một hàm **đầu ra vô hướng**. Hàm loss $L(x,y)=x^2+2y^2$ có Hessian [[2,0],[0,4]]. Hàng/cột đều tương ứng thứ tự **tham số x, y**; số 4 theo y lớn hơn 2 theo x nghĩa là độ cong theo y lớn hơn trong mô hình này. Khác Jacobian cho hàm nhiều đầu ra, Hessian giúp xem gradient đổi nhanh thế nào.

No Note — Không chú thích

```
import sympy as sp

x, y = sp.symbols('x y')
```

```
loss = x ** 2 + 2 * y ** 2
print(sp.hessian(loss, [x, y]).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính Hessian của hàm loss hai biến để biết độ cong ở từng hướng.
# Dữ liệu: Loss=x²+2y², độ cong theo y lớn gấp đôi độ cong x.
# SymPy sp.hessian() tính đạo hàm riêng cấp hai.
import sympy as sp

# x,y là hai tham số cần khảo sát độ cong.
x, y = sp.symbols('x y')
# Hàm loss parabol hai chiều, hệ số y lớn hơn làm bề mặt dốc hơn theo y.
loss = x ** 2 + 2 * y ** 2
# hessian(f,[x,y]) tạo ma trận các đạo hàm riêng bậc hai, gồm ∂²f/∂x², ∂²f/∂x∂y,...; ở đây [[2,0],
[0,4]].
# Chuyển SymPy Matrix thành list lồng để in hai hàng.
print(sp.hessian(loss, [x, y]).tolist())
```

Kết quả khi chạy:

```
[[2, 0], [0, 4]]
```

Hàm cong mạnh hơn theo y; hai đạo hàm chéo bằng 0.

4.3. Critical point — Điểm cần kiểm tra cực trị

Từ vựng: Critical point — điểm tới hạn; **Stationary point** — điểm dừng.

Điểm dừng của hàm khả vi có gradient bằng vector 0, tức không còn hướng tăng tuyến tính ngay tại điểm. Với $L(x,y)=(x-3)^2+(y-2)^2$, các đạo hàm $2(x-3)$, $2(y-2)$ đều bằng 0 khi $(3,2)$. Đây là **ứng viên cực trị**, chưa đủ để nói là cực tiểu: hàm hình yên ngựa cũng có thể có gradient 0. Khái niệm critical point rộng hơn còn có điểm không khả vi ở miền trong.

No Note — Không chú thích

```
import sympy as sp

x, y = sp.symbols('x y')
loss = (x - 3) ** 2 + (y - 2) ** 2
solution = sp.solve([sp.diff(loss, x), sp.diff(loss, y)], [x, y])
print(solution[x], solution[y])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Giải hệ gradient bằng 0 để tìm điểm dừng của hàm loss hai biến.
# Dữ liệu: Loss=(x-3)²+(y-2)², kỳ vọng điểm tối thiểu tại (3,2).
# SymPy tạo phương trình ký hiệu và giải đồng thời.
import sympy as sp

# Khai báo hai biến đầu vào x,y.
x, y = sp.symbols('x y')
# Hai bình phương không âm, bằng 0 tại x=3, y=2.
loss = (x - 3) ** 2 + (y - 2) ** 2
# diff(loss,x) lấy đạo hàm riêng theo x để xét điểm dừng.
# diff(loss,y) lấy đạo hàm riêng theo y.
# solve(danh_sách_phương_trình,[x,y]) tìm nghiệm đồng thời khi cả hai thành phần gradient bằng 0.
solution = sp.solve([sp.diff(loss, x), sp.diff(loss, y)], [x, y])
```

```
# solution là dictionary nghiệm theo ký hiệu; [x] là giá trị x, [y] là giá trị y.
print(solution[x], solution[y])
```

Kết quả khi chạy:

```
3 2
```

Gradient bằng 0 tại (3,2); cần xét độ cong để phân loại điểm này.

4.4. Curvature — Phân biệt cực tiểu, cực đại, yên ngựa

Từ vựng: Curvature — độ cong; **Eigenvalue** — trị riêng.

Tại một **điểm dừng** của hàm đủ trơn, dấu trị riêng Hessian giúp phân loại: **tất cả dương** → cực tiểu địa phương nghiêm ngặt; **tất cả âm** → cực đại địa phương nghiêm ngặt; **vừa dương vừa âm** → điểm yên ngựa. Ví dụ x^2-y^2 có Hessian `diag(2,–2)`, nên tại (0,0) đi theo x hàm tăng nhưng đi theo y hàm giảm. Nếu có trị riêng bằng 0, phép thử Hessian đơn giản **chưa kết luận**.

No Note — Không chú thích

```
import numpy as np

H = np.array([[2.0, 0.0], [0.0, -2.0]])
print(np.linalg.eigvalsh(H))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Quan sát dấu của trị riêng Hessian để nhận diện bề mặt có hướng lên và hướng xuống.
# Dữ liệu: Hessian H=diag(2,-2), một hướng cong lên và một hướng cong xuống: dạng yên ngựa.
# NumPy eigvalsh() lấy trị riêng của ma trận thực đối xứng.
import numpy as np

# Ma trận Hessian đối xứng 2×2 với một trị riêng dương và một âm.
H = np.array([[2.0, 0.0], [0.0, -2.0]])
# eigvalsh() trả trị riêng tăng dần [-2,2]; trái dấu nghĩa Hessian bất định, chỉ dạng yên ngựa tại
điểm dừng tương ứng.
print(np.linalg.eigvalsh(H))
```

Kết quả khi chạy:

```
[-2.  2.]
```

Một trị riêng âm, một dương nên Hessian bất định; (0,0) là điểm yên ngựa của x^2-y^2.

5. Năm tình huống Jacobian/Hessian — từ độ nhạy đến suy biến cục bộ

Tình huống 1 — Cấp 1: Jacobian của hai đầu ra trên 10 cặp tín hiệu

Từ vựng trong mục: Jacobian — ma trận độ nhạy; **Partial derivative** — đạo hàm riêng.

Tình huống thực tế: Trạm đo có **10 cặp đầu vào x và y không đơn vị**; mỗi hàng đại diện một lượt đo, chẳng hạn lượt đầu x = 2, y = 3. Phần mềm tạo **hai chỉ số đầu ra**: một chỉ số cộng x với y, chỉ số kia nhân x

với y. Kỹ thuật viên muốn biết khi **chỉ x hoặc chỉ y thay đổi rất ít**, hai kết quả nhạy đến đâu tại **lượt đầu**, để tránh nhầm đầu ra nào bị ảnh hưởng bởi biến nào.

Vì sao chọn cách này? Jacobian là một **bảng độ nhạy**: mỗi **hàng ứng với một chỉ số đầu ra**, mỗi **cột ứng với một biến đầu vào**. Một ô cho biết đầu ra ở hàng ấy thay đổi nhanh cỡ nào nếu tăng rất ít biến ở cột ấy, giữ biến kia cố định. `outputs.jacobian([x, y])` tạo bảng **2 × 2** từ hai quy tắc, rồi `subs` đưa x = 2 và y = 3 của lượt đầu vào để xem các độ nhạy tại đúng hoàn cảnh đó. Đây **không phải bảng 10 lượt đo**; 10 là số dữ liệu đã lấy, còn kích thước Jacobian được quyết định bởi số đầu ra và số đầu vào.

Quy trình:

1. Tạo 10 cặp x/y rồi định nghĩa hai đầu ra bằng SymPy.
2. Gọi `jacobian([x,y])` cho hàm vector.
3. Thay cặp đầu tiên vào Jacobian và in ma trận 2×2.

No Note — Không chú thích

```
import sympy as sp
x, y = sp.symbols("x y")
inputs = [[2,3],[3,3],[4,3],[5,3],[6,3],[2,4],[3,4],[4,4],[5,4],[6,4]]
outputs = sp.Matrix([x+y, x*y])
J = outputs.jacobian([x, y])
first = J.subs({x: inputs[0][0], y: inputs[0][1]})
if len(inputs) != 10 or J.shape != (2, 2):
    raise ValueError("Cần 10 cặp đầu vào và Jacobian 2×2.")
print("Cặp đầu vào:", len(inputs))
print("Jacobian cặp đầu:", first.tolist())
print("Shape Jacobian:", J.shape)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính Jacobian tại cặp đầu tiên trong 10 cặp (x,y) với hai đầu ra.
# Dữ liệu: 10 cặp điểm trên lưới x=2..6, y=3..4; đầu ra là x+y và x*y.
# SymPy xây biểu thức vector và Jacobian có thể thế tại nhiều điểm.
import sympy as sp
# Khai báo x,y là hai biến ký hiệu.
x, y = sp.symbols("x y")
# 10 list con, mỗi list là cặp [x,y], không phải 10 biến độc lập.
inputs = [[2,3],[3,3],[4,3],[5,3],[6,3],[2,4],[3,4],[4,4],[5,4],[6,4]]
# Matrix chứa hai hàm đầu ra theo thứ tự tổng, tích.
outputs = sp.Matrix([x+y, x*y])
# jacobian() cho ma trận 2 đầu ra × 2 đầu vào, hàng thể hiện đầu ra, cột thể hiện biến.
J = outputs.jacobian([x, y])
# [0] ngoài lấy cặp đầu tiên, [0] trong lấy x, [1] lấy y của cặp đó.
# subs() thế cả x và y của cặp đầu vào thứ nhất vào Jacobian ký hiệu.
first = J.subs({x: inputs[0][0], y: inputs[0][1]})
# shape là kích thước (2,2) của Jacobian, không phải 10 cặp điểm.
if len(inputs) != 10 or J.shape != (2, 2):
    # raise ValueError báo điều kiện số mẫu, shape hoặc phép đối chiếu thất bại.
    raise ValueError("Cần 10 cặp đầu vào và Jacobian 2×2.")
print("Cặp đầu vào:", len(inputs))
# tolist() đổi ma trận SymPy đã thế số thành list lồng hai hàng.
print("Jacobian cặp đầu:", first.tolist())
# shape là kích thước (2,2) của Jacobian, không phải 10 cặp điểm.
print("Shape Jacobian:", J.shape)
```

Kết quả khi chạy:

```
Cặp đâù vào: 10
Jacobian cặp đâù: [[1, 1], [3, 2]]
Shape Jacobian: (2, 2)
```

Hàng đầu của Jacobian nói về F1, hàng thứ hai về F2; số `3` ở hàng F2/cột x có nghĩa là F2 thay đổi khoảng 3 đơn vị khi x tăng 1 đơn vị rất nhỏ tại cặp đầu.

Tình huống 2 — Cấp 2: Hessian cho độ cong loss tại 12 cấu hình tham số

Từ vựng trong mục: Hessian — ma trận đạo hàm bậc hai; **Curvature** — độ cong.

Tình huống thực tế: Đội tối ưu thử **12 cặp thông số x, y không đơn vị** cho một hàm mất mát giả lập. Khi đi xa điểm tốt nhất, loss tăng theo cả hai hướng nhưng tăng **nhanh hơn theo y** so với x. Nhóm muốn xác nhận **độ cong** của bề mặt loss ở hai hướng, và xem độ cong ấy có đổi theo 12 cặp thông số hay giữ nguyên.

Vì sao chọn cách này? Gradient cho biết loss đổi nhanh theo từng thông số, còn **Hessian** cho biết chính các độ dốc ấy **thay đổi nhanh đến đâu**, tức **độ cong** của bề mặt. `sp.hessian(loss, [x, y])` tạo bảng độ cong theo cả hai hướng và sự tương tác giữa chúng. Với bề mặt bậc hai trong bài, Hessian **không phụ thuộc vị trí x, y**, nên kiểm cấu hình đầu và cuối sẽ giống nhau; thành phần theo y lớn hơn theo x, cho thấy cong mạnh hơn theo y. Đừng nhầm loss ở mỗi cấu hình khác nhau với việc **Hessian cũng phải khác**.

Quy trình:

1. Lưu 12 cặp thử và định nghĩa loss bằng SymPy.
2. Tính Hessian theo đúng thứ tự x, y.
3. Đánh giá Hessian tại điểm đầu/cuối và kiểm hai ma trận giống nhau.

No Note — Không chú thích

```
import sympy as sp
x, y = sp.symbols("x y")
points = [[-1,-1],[0,-1],[1,-1],[2,-1],[-1,0],[0,0],[1,0],[2,0],[-1,1],[0,1],[1,1],[2,1]]
loss = x**2 + 2*y**2
H = sp.hessian(loss, [x, y])
first = H.subs({x: points[0][0], y: points[0][1]})
last = H.subs({x: points[-1][0], y: points[-1][1]})
if len(points) != 12:
    raise ValueError("Cần đủ 12 cấú hình Hessian.")
print("Cấú hình:", len(points))
print("Hessian:", H.tolist())
print("Hessian không đổi:", first == last)
print("Cong hơn theo y:", H[1,1] > H[0,0])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xem Hessian không đổi theo 12 điểm vì hàm chi phí là đa thức bậc hai.
# Dữ liệu: 12 cấú hình (x,y), loss=x²+2y² nên Hessian luôn [[2,0],[0,4]].
# SymPy tính đạo hàm bậc hai chính xác trên biểu thức.
import sympy as sp
# Tạo hai biến ký hiệu.
x, y = sp.symbols("x y")
# 12 điểm là 12 cấú hình, môĩ list gồm hai tọa độ x/y.
points = [[-1,-1],[0,-1],[1,-1],[2,-1],[-1,0],[0,0],[1,0],[2,0],[-1,1],[0,1],[1,1],[2,1]]
# Bậc hai có độ cong không phụ thuộc vị trí: theo x hệ sô´cong 2, y hệ sô´cong 4.
loss = x**2 + 2*y**2
# hessian() tạo bảng các đạo hàm riêng cấp hai, theo thứ tự biến [x,y].
```

```
H = sp.hessian(loss, [x, y])
# Hai chỉ sô´chọn x/y của điểm ĐÂU; [-1] chọn điểm CUÔI.
# Thê´tọa độ đâu vào Hessian đểkiểm giá trị tại điểm đó.
first = H.subs({x: points[0][0], y: points[0][1]})
# [-1] chọn câú hình cuôí, [0]/[1] chọn x/y của câú hình.
last = H.subs({x: points[-1][0], y: points[-1][1]})
if len(points) != 12:
    # raise ValueError báo điêù kiện sô´mâũ, shape hoặc phép đôí chiêú thât bại.
    raise ValueError("Cân đủ 12 câú hình Hessian.")
print("Câú hình:", len(points))
# tolist() trình bày Hessian thành list 2×2.
print("Hessian:", H.tolist())
# So hai SymPy Matrix bằng == đểkiểm Hessian không đôỉ giữa hai câú hình.
print("Hessian không đôỉ:", first == last)
# [1,1] là đạo hàm riêng bậc hai theo y, [0,0] theo x; so xem bề`mặt cong hơn theo y.
print("Cong hơn theo y:", H[1,1] > H[0,0])
```

Kết quả khi chạy:

```
Câú hình: 12
Hessian: [[2, 0], [0, 4]]
Hessian không đôỉ: True
Cong hơn theo y: True
```

Độ cong là thông số của mô hình loss, không phải tọa độ x/y được đo từ 12 cấu hình.

Tình huống 3 — Cấp 3: Tìm cực tiểu của loss hai tham số từ 15 ứng viên

Từ vựng trong mục: Stationary point — điểm dừng; **Minimum** — cực tiểu; **Eigenvalues** — trị riêng.

Tình huống thực tế: Nhóm hiệu chỉnh có **15 phương án cài đặt máy** theo lưới gồm năm giá trị x và ba giá trị y. Điểm đánh giá (loss, không đơn vị) nhỏ nhất theo thiết kế ở **x = 3, y = 2**. Kỹ thuật viên muốn dùng quy tắc toán học tìm điểm tốt nhất, kiểm chứng đây là **đáy** chứ không phải đỉnh, rồi xem trong 15 phương án có đúng cấu hình tốt nhất ấy không.

Vì sao chọn cách này? Để tìm **điểm dừng**, chương trình dùng `sympy.diff` tính độ dốc theo x/y và `sympy.solve` tìm nơi cả hai độ dốc **đều bằng 0**. Nhưng điểm dừng **chưa chắc là nhỏ nhất**. `sp.hessian` lấy độ cong; `np.linalg.eigvalsh` kiểm các **trị riêng** của bảng Hessian: ở ví dụ này **tất cả đều dương**, nghĩa là bề mặt cong lên theo mọi hướng, nên điểm dừng là **cực tiểu nghiêm ngặt**. Vòng lặp đánh giá cả **15 cấu hình có sẵn**, dùng `argmin` tìm vị trí có loss nhỏ nhất và đổi vị trí đó sang mã T01…; đây là bước so nghiệm toán học với phương án thực tế.

Quy trình:

1. Dùng SymPy giải đồng thời hai thành phần gradient bằng 0.
2. Dùng Hessian và trị riêng để phân loại điểm dừng.
3. So loss của 15 ứng viên, in mã ứng viên tốt nhất và nghiệm chính xác.

No Note — Không chú thích

```
import sympy as sp
import numpy as np
x, y = sp.symbols("x y")
points = [[1,1],[2,1],[3,1],[4,1],[5,1],[1,2],[2,2],[3,2],[4,2],[5,2],[1,3],[2,3],[3,3],[4,3],
[5,3]]
loss = (x-3)**2 + 2*(y-2)**2
solution = sp.solve([sp.diff(loss,x), sp.diff(loss,y)], [x,y])
H = np.array(sp.hessian(loss,[x,y])).astype(float)
```

```
eigenvalues = np.linalg.eigvalsh(H)
losses = []
for px, py in points:
    losses.append((px-3)**2 + 2*(py-2)**2)
if len(points) != 15:
    raise ValueError("Cần đủ 15 ứng viên cần đánh giá.")
print("Ứng viên:", len(points))
print("Điểm dừng:", solution[x], solution[y])
print("Hessian dương:", bool(np.all(eigenvalues > 0)))
print("Ứng viên tốt nhất:", f"T{int(np.argmin(losses))+1:02d}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm điểm dừng tối thiểu trong 15 ứng viên và xác nhận Hessian dương xác định.
# Dữ liệu: 15 điểm ứng viên x=1..5, y=1..3; loss=(x-3)²+2(y-2)².
# SymPy giải gradient bằng 0 và tính Hessian ký hiệu.
import sympy as sp
# NumPy kiểm trị riêng của Hessian, argmin() chọn ứng viên.
import numpy as np
# x,y là hai tham số mục tiêu.
x, y = sp.symbols("x y")
# 15 list điểm ứng viên, mỗi hàng một cặp x,y.
points = [[1,1],[2,1],[3,1],[4,1],[5,1],[1,2],[2,2],[3,2],[4,2],[5,2],[1,3],[2,3],[3,3],[4,3],
[5,3]]
# Hàm không âm, bằng 0 ở (3,2); hệ số y khiến hướng y cong mạnh hơn.
loss = (x-3)**2 + 2*(y-2)**2
# Đạo hàm riêng theo x, kết hợp đạo hàm y thành hệ điều kiện điểm dừng.
# solve() giải đồng thời gradient_x=0 và gradient_y=0, cho nghiệm theo x,y.
solution = sp.solve([sp.diff(loss,x), sp.diff(loss,y)], [x,y])
# hessian() trả ma trận đạo hàm riêng bậc hai của loss theo [x,y].
# Đổi SymPy Hessian thành NumPy array để tính trị riêng bằng numpy.linalg.
# astype(float) ép giá trị ký hiệu sang float để eigvalsh() xử lý ma trận số.
H = np.array(sp.hessian(loss,[x,y])).astype(float)
# eigvalsh() trả trị riêng Hessian tăng dần; tất cả dương nghĩa cong lên ở mọi hướng.
eigenvalues = np.linalg.eigvalsh(H)
losses = []
# Duyệt từng cặp ứng viên, tách x/y.
for px, py in points:
    # append() lưu giá trị loss của mỗi ứng viên theo thứ tự points.
    losses.append((px-3)**2 + 2*(py-2)**2)
if len(points) != 15:
    # raise ValueError báo điều kiện số mãu, shape hoặc phép đối chiếu thất bại.
    raise ValueError("Cần đủ 15 ứng viên cần đánh giá.")
print("Ứng viên:", len(points))
# Lấy tọa độ điểm dừng đã giải từ dictionary SymPy.
print("Điểm dừng:", solution[x], solution[y])
# Mask trị riêng >0; np.all() yêu cầu TẤT CẢ dương, xác nhận Hessian dương xác định.
print("Hessian dương:", bool(np.all(eigenvalues > 0)))
# argmin() trả vị trí 0-based của loss nhỏ nhất trong 15 ứng viên, không trả chính trị loss.
# int() chuyển chỉ số, +1 đổi sang số thứ tự 1-based và :02d tạo mã hai chữ số.
print("Ứng viên tốt nhất:", f"T{int(np.argmin(losses))+1:02d}")
```

Kết quả khi chạy:

```
Ứng viên: 15
Điểm dừng: 3 2
Hessian dương: True
Ứng viên tốt nhất: T08
```

Tên 'tốt nhất' ở đây có nghĩa **loss nhỏ nhất trên đúng 15 phương án đã cho**; bài toán thực tế còn cần kiểm giới hạn vận hành của thiết bị.

Tình huống 4 — Cấp 4: Phân loại 18 mô hình có điểm dừng: min/max/yên ngựa

Từ vựng trong mục: Critical point — điểm dừng; **Positive definite** — xác định dương; **Saddle** — yên ngựa.

Tình huống thực tế: Đội nghiên cứu thử **18 phiên bản hàm loss giả lập** M01–M18. Mỗi phiên bản thuộc một trong ba kiểu bề mặt: **chiếc bát có đáy** (cực tiểu), **chiếc vòm có đỉnh** (cực đại), hoặc **mặt yên ngựa** (đi lên theo một hướng nhưng đi xuống theo hướng kia). Cả ba kiểu đều có một điểm mà độ dốc bằng 0, nên nhóm muốn phân loại thật sự từ **độ cong**, đếm từng loại thay vì gọi mọi điểm dừng là điểm tối ưu.

Vì sao chọn cách này? Độ dốc bằng 0 chỉ nói rằng **ngay tại điểm đó đường không nghiêng về một hướng cụ thể**, chứ không cho biết đó là đáy, đỉnh hay yên ngựa. **Hessian** cung cấp thông tin về độ cong; `eigvalsh` trả các trị riêng của bảng độ cong đối xứng. Trong ba hàm bậc hai minh họa, **toàn dương** nghĩa là đáy, **toàn âm** là đỉnh, còn **vừa dương vừa âm** là yên ngựa. Chương trình đọc lần lượt **18 mô hình** và tự phân loại từ kết quả tính, không chỉ tin nhãn `type` ghi trong dữ liệu. Quy tắc dấu này áp dụng rõ ràng cho các hàm ở bài; tình huống Hessian có trị riêng bằng 0 cần xem xét thêm, không vội kết luận.

Quy trình:

1. Lưu 18 mã/công thức và định nghĩa Hessian của ba công thức bằng SymPy.
2. Lấy các trị riêng Hessian thực đối xứng cho mỗi phiên bản.
3. Đếm ba loại, xác nhận tổng 18.

No Note — Không chú thích

```
import numpy as np
import sympy as sp
x,y = sp.symbols("x y")
models = [{"id":"M01","type":"minimum"},{"id":"M02","type":"maximum"},{"id":"M03","type":"saddle"},
{"id":"M04","type":"minimum"},{"id":"M05","type":"maximum"},{"id":"M06","type":"saddle"},
{"id":"M07","type":"minimum"},{"id":"M08","type":"maximum"},{"id":"M09","type":"saddle"},
{"id":"M10","type":"minimum"},{"id":"M11","type":"maximum"},{"id":"M12","type":"saddle"},
{"id":"M13","type":"minimum"},{"id":"M14","type":"maximum"},{"id":"M15","type":"saddle"},
{"id":"M16","type":"minimum"},{"id":"M17","type":"maximum"},{"id":"M18","type":"saddle"}]
functions = {"minimum": x*x+2*y*y,
             "maximum": -x*x-2*y*y,
             "saddle": x*x-y*y}
results = []
for row in models:
    H = np.array(sp.hessian(functions[row["type"]],[x,y]), dtype=float)
    eig = np.linalg.eigvalsh(H)
    if np.all(eig > 0):
        kind = "minimum"
    elif np.all(eig < 0):
        kind = "maximum"
    else:
        kind = "saddle"
    results.append(kind)
if len(models) != 18:
    raise ValueError("Cần đủ 18 mô hình cực trị.")
print("Mô hình:", len(models))
print("Cực tiểu:", results.count("minimum"))
print("Cực đại:", results.count("maximum"))
print("Yên ngựa:", results.count("saddle"))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân nhóm 18 hàm bậc hai theo trị riêng Hessian: cực tiểu, cực đại, yên ngựa.
# Dữ liệu: 18 mô hình lặp ba dạng loss: x²+2y², -x²-2y², x²-y², mỗi dạng sáu lượt.
# NumPy chuyển Hessian SymPy thành số và kiểm trị riêng.
import numpy as np
# SymPy tạo ba hàm bậc hai và tính Hessian.
import sympy as sp
# Hai biến x,y cho mỗi bề mặt.
x,y = sp.symbols("x y")
# 18 dictionary với id và type mô hình; type chỉ tên kiểm thử, phân loại vẫn dựa Hessian.
models = [{"id":"M01","type":"minimum"},{"id":"M02","type":"maximum"},{"id":"M03","type":"saddle"},
{"id":"M04","type":"minimum"},{"id":"M05","type":"maximum"},{"id":"M06","type":"saddle"},
{"id":"M07","type":"minimum"},{"id":"M08","type":"maximum"},{"id":"M09","type":"saddle"},
{"id":"M10","type":"minimum"},{"id":"M11","type":"maximum"},{"id":"M12","type":"saddle"},
{"id":"M13","type":"minimum"},{"id":"M14","type":"maximum"},{"id":"M15","type":"saddle"},
{"id":"M16","type":"minimum"},{"id":"M17","type":"maximum"},{"id":"M18","type":"saddle"}]
# Dictionary gắn type với ba công thức bậc hai tương ứng.
functions = {"minimum": x*x+2*y*y,
             "maximum": -x*x-2*y*y,
             "saddle": x*x-y*y}
results = []
# Duyệt từng mô hình và lấy đúng công thức qua khóa type.
for row in models:
    # sp.hessian() tính ma trận độ cong bậc hai của công thức đang xét.
    # Chuyển Hessian sang ndarray float cho eigvalsh.
    H = np.array(sp.hessian(functions[row["type"]],[x,y]), dtype=float)
    # eigvalsh() trả trị riêng của Hessian đối xứng; dấu trị riêng phân biệt các dạng cong.
    eig = np.linalg.eigvalsh(H)
    # Tất cả trị riêng dương nghĩa cực tiểu tại điểm dừng (Hessian dương xác định).
    if np.all(eig > 0):
        kind = "minimum"
    # Tất cả trị riêng âm nghĩa cực đại tại điểm dừng.
    elif np.all(eig < 0):
        kind = "maximum"
    else:
        # Với ba loại mô hình đã dựng, trường hợp còn lại có trị riêng trái dấu nên là yên ngựa;
quy tắc else này không đủ phân loại Hessian có trị riêng bằng 0 nói chung.
        kind = "saddle"
    # append() lưu nhãn kết quả để đếm thống kê.
    results.append(kind)
# Kiểm đủ 18 mô hình trước khi báo cáo.
if len(models) != 18:
    # raise ValueError báo điều kiện sớm mẫu, shape hoặc phép đối chiếu thất bại.
    raise ValueError("Cần đủ 18 mô hình cực trị.")
print("Mô hình:", len(models))
# count() đếm số chuỗi 'minimum'; tương tự 'maximum','saddle' trong list nhãn.
print("Cực tiểu:", results.count("minimum"))
print("Cực đại:", results.count("maximum"))
print("Yên ngựa:", results.count("saddle"))
```

Kết quả khi chạy:

```
Mô hình: 18
Cực tiểu: 6
Cực đại: 6
Yên ngựa: 6
```

Trị riêng bằng 0 sẽ khiến **phép thử Hessian cơ bản không đủ kết luận**; ba dạng hàm minh họa ở đây không có trị riêng bằng 0.

Tình huống 5 — Cấp 5: Kiểm tra Jacobian suy biến qua 20 trạng thái của hệ hai đầu ra

Từ vựng trong mục: Jacobian determinant — định thức Jacobian; **Sensitivity** — độ nhạy; **Singular** — suy biến.

Tình huống thực tế: Một hệ thống điều khiển có **20 cấu hình C01–C20**, mỗi cấu hình ghi hai tín hiệu đầu vào x, y và cho ra **hai chỉ số** theo các quy tắc phi tuyến. Kỹ thuật viên muốn biết tại cấu hình nào các đầu ra **không đủ nhạy theo hai hướng độc lập**, khiến cách suy ngược những thay đổi nhỏ của đầu vào từ đầu ra không còn được bảo đảm. Cần liệt kê mã các cấu hình ấy và đếm số cấu hình còn kiểm tra được tính khả nghịch cục bộ.

Vì sao chọn cách này? Jacobian đo mối quan hệ giữa **thay đổi nhỏ ở hai đầu vào** và **thay đổi ở hai đầu ra** tại một cấu hình. Nếu hai hàng hoặc cột của Jacobian không độc lập, một số thay đổi đầu vào không thể phân biệt chỉ từ cách các đầu ra thay đổi rất nhỏ ở điểm ấy. Với bảng **2 × 2**, **định thức gần 0** báo Jacobian suy biến, nên điều kiện đủ quen thuộc để **bảo đảm có hàm ngược cục bộ** không còn thỏa. Chương trình thế từng cặp x, y vào Jacobian, tính `np.linalg.det`, dùng ngưỡng `1e-9` để xử lý sai số rồi ghi đúng mã C01… bị suy biến. Điều đó **không tự chứng minh** mọi cách khôi phục đầu vào đều bất khả thi ở mọi vùng lân cận; chỉ không được áp dụng bảo đảm thông thường tại các điểm ấy.

Quy trình:

1. Dùng SymPy tính Jacobian của hai hàm đầu ra.
2. Thay từng cặp trong 20 hàng, tính định thức bằng số.
3. In mã suy biến, số cấu hình còn khả nghịch cục bộ và Jacobian mẫu.

No Note — Không chú thích

```
import numpy as np
import sympy as sp
x,y = sp.symbols("x y")
points = [[1,8],[2,2],[3,2],[4,2],[5,2],[1,3],[2,3],[3,36],[4,3],[5,3],[1,4],[2,4],[3,4],[4,4],
[5,80],[1,5],[2,5],[3,5],[4,5],[5,5]]
F = sp.Matrix([x*x+y, x*y+3*y])
J = F.jacobian([x,y])
singular = []
for i,(px,py) in enumerate(points,1):
    matrix = np.array(J.subs({x:px,y:py}),dtype=float)
    if abs(np.linalg.det(matrix)) < 1e-9:
        singular.append(f"C{i:02d}")
if len(points) != 20:
    raise ValueError("Cần đủ 20 cấu hình Jacobian.")
print("Cấu hình:", len(points))
print("Suy biến:", ", ".join(singular))
print("Còn định thức khác 0:", len(points)-len(singular))
print("Kích thước Jacobian:", J.shape)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm 20 Jacobian hai biến để tìm cấu hình gần suy biến theo định thức.
# Dữ liệu: 20 điểm (x,y), F=(x²+y,xy+3y); ngưỡng |det(J)|<1e-9 là tiêu chí gần suy biến.
# NumPy linalg.det() tính định thức của Jacobian số tại từng cấu hình.
import numpy as np
# SymPy lập ma trận Jacobian ký hiệu từ hàm vector F.
import sympy as sp
# Hai đầu vào ký hiệu cho hàm F.
x,y = sp.symbols("x y")
# 20 cấu hình [x,y], mỗi cặp một trường hợp kiểm đạo hàm.
points = [[1,8],[2,2],[3,2],[4,2],[5,2],[1,3],[2,3],[3,36],[4,3],[5,3],[1,4],[2,4],[3,4],[4,4],
```

```
[5,80],[1,5],[2,5],[3,5],[4,5],[5,5]]
# Hai đầu ra phi tuyến làm môĩ hàng Jacobian tương ứng một hàm.
F = sp.Matrix([x*x+y, x*y+3*y])
# jacobian() tạo ma trận đạo hàm riêng 2×2 theo thứ tự biến [x,y].
J = F.jacobian([x,y])
# List lưu mã cấú hình có định thức gần 0.
singular = []
# enumerate(...,1) cung cấp sô´thứ tự 1..20 cho mã C01..C20; cặp (px,py) là tọa độ.
for i,(px,py) in enumerate(points,1):
    # subs() thê´tọa độ cụ thể vào các đạo hàm ký hiệu.
    # Đổi Jacobian tại điểm thành ndarray sô´thực float trước khi lấy det().
    matrix = np.array(J.subs({x:px,y:py}),dtype=float)
    # det() lấy định thức Jacobian 2×2; bằng 0 là suy biến, không có nghịch đảo cục bộ.
    # abs() bỏ dấú định thức, so ngưỡng gần 0 để tránh sai sô´floating-point; ngưỡng này tùy thang
dữ liệu.
    if abs(np.linalg.det(matrix)) < 1e-9:
        # f-string :02d tạo mã cấú hình hai chữ sô´.
        # append() lưu mã của cấú hình gần suy biến.
        singular.append(f"C{i:02d}")
# Kiểm đủ 20 cấú hình đầu vào.
if len(points) != 20:
    # raise ValueError báo điều kiện sô´mấũ, shape hoặc phép đối chiếú thất bại.
    raise ValueError("Cần đủ 20 cấú hình Jacobian.")
print("Cấú hình:", len(points))
print("Suy biến:", ", ".join(singular))
# Sô´cấú hình không bị đánh dấú = tổng 20 trừ sô´gần suy biến.
print("Còn định thức khác 0:", len(points)-len(singular))
# shape (2,2) là sô´đầu ra × sô´đầu vào, không phải 20 điểm.
print("Kích thước Jacobian:", J.shape)
```

Kết quả khi chạy:

```
Cấú hình: 20
Suy biến: C01, C08, C15
Còn định thức khác 0: 17
Kích thước Jacobian: (2, 2)
```

Tập mã suy biến được suy từ **Jacobian tại từng điểm**, không phải từ việc x/y có giá trị lớn hay nhỏ.

6. Sai lầm thường gặp

- Jacobian có số hàng bằng số đầu ra và số cột bằng số đầu vào; không luôn là ma trận vuông.
- Hessian dùng cho hàm vô hướng đủ khả vi hai lần; hàng/cột là các tham số, không phải hai đầu ra.
- Gradient bằng 0 mới chỉ chỉ ra điểm dừng, chưa đủ nói là cực tiểu.
- Hessian có trị riêng vừa âm vừa dương ở điểm dừng chỉ ra yên ngựa; nếu có trị riêng 0, bài kiểm bậc hai đơn giản chưa đủ.
- Khi dùng `eigvalsh`, ma trận đầu vào phải đối xứng theo phép toán đang xét.
- SymPy là thư viện ngoài; các ví dụ ký hiệu cần cài đặt.

7. Trắc nghiệm

Q1. What does a Jacobian contain? | Jacobian chứa gì?

A. A differentiable point where the gradient is zero | Điểm khả vi có gradient bằng 0

B. Classifying local behavior near stationary points | Phân loại hành vi cục bộ gần điểm dừng

C. First partial derivatives of vector outputs | Các đạo hàm riêng bậc một của đầu ra vector

D. Second partial derivatives of a scalar output | Các đạo hàm riêng bậc hai của đầu ra vô hướng

Đáp án: C

Giải thích: Mỗi hàng chứa đạo hàm bậc một của một đầu ra.

Q2. What does a Hessian contain? | Hessian chứa gì?

A. Classifying local behavior near stationary points | Phân loại hành vi cục bộ gần điểm dừng

B. First partial derivatives of vector outputs | Các đạo hàm riêng bậc một của đầu ra vector

C. Second partial derivatives of a scalar output | Các đạo hàm riêng bậc hai của đầu ra vô hướng

D. A differentiable point where the gradient is zero | Điểm khả vi có gradient bằng 0

Đáp án: C

Giải thích: Hessian lấy đạo hàm tiếp của gradient.

Q3. What is a stationary point? | Điểm dừng là gì?

A. Classifying local behavior near stationary points | Phân loại hành vi cục bộ gần điểm dừng

B. First partial derivatives of vector outputs | Các đạo hàm riêng bậc một của đầu ra vector

C. Second partial derivatives of a scalar output | Các đạo hàm riêng bậc hai của đầu ra vô hướng

D. A differentiable point where the gradient is zero | Điểm khả vi có gradient bằng 0

Đáp án: D

Giải thích: Điểm dừng có các đạo hàm riêng bậc một bằng 0.

Q4. What is curvature information used for? | Thông tin độ cong dùng để làm gì?

A. Second partial derivatives of a scalar output | Các đạo hàm riêng bậc hai của đầu ra vô hướng

B. A differentiable point where the gradient is zero | Điểm khả vi có gradient bằng 0

C. Classifying local behavior near stationary points | Phân loại hành vi cục bộ gần điểm dừng

D. First partial derivatives of vector outputs | Các đạo hàm riêng bậc một của đầu ra vector

Đáp án: C

Giải thích: Độ cong giúp phân loại điểm dừng.

Q5. For $f(x, y) = x^2 + y^2$, what is its Hessian matrix? | Với $f(x, y) = x^2 + y^2$, ma trận Hessian bằng bao nhiêu?

A. [[1, 0], [0, 1]] | [[1, 0], [0, 1]]

B. [[0, 0], [0, 0]] | [[0, 0], [0, 0]]

C. [[2, 0], [0, 2]] | [[2, 0], [0, 2]]

D. [[0, 2], [2, 0]] | [[0, 2], [2, 0]]

Đáp án: C

Giải thích: Đạo hàm bậc hai theo x và y đều bằng 2, hai đạo hàm chéo bằng 0.

Q6. At which x is the derivative of $f(x) = (x - 3)^2$ zero? | Đạo hàm của $f(x) = (x - 3)^2$ bằng 0 tại x nào?

A. $x = 2$ | $x = 2$

B. $x = -3$ | $x = -3$

C. $x = 3$ | $x = 3$

D. $x = 0$ | $x = 0$

Đáp án: C

Giải thích: $f'(x) = 2(x - 3)$; đặt bằng 0 được $x = 3$.

Q7. At a stationary point, a positive-definite Hessian indicates what? | Tại một điểm dừng, Hessian xác định dương cho biết điều gì?

A. Always a global maximum | Luôn là cực đại toàn cục

B. A strict local minimum | Một cực tiểu địa phương nghiêm ngặt

C. A strict local maximum | Một cực đại địa phương nghiêm ngặt

D. A saddle point | Một điểm yên ngựa

Đáp án: B

Giải thích: Hessian xác định dương nghĩa là hàm cong lên theo mọi hướng ở điểm dừng; trong điều kiện trơn phù hợp, đó là cực tiểu địa phương nghiêm ngặt.

Q8. For $f(x, y) = x^2 - y^2$, how is the stationary point (0, 0) classified? | Với $f(x, y) = x^2 - y^2$, điểm dừng (0, 0) thuộc loại nào?

A. Neither stationary nor extremum | Không phải điểm dừng hay cực trị

B. Strict local minimum | Cực tiểu địa phương nghiêm ngặt

C. Saddle point | Điểm yên ngựa

D. Strict local maximum | Cực đại địa phương nghiêm ngặt

Đáp án: C

Giải thích: Theo trục x, f tăng khỏi 0; theo trục y, f giảm khỏi 0. Vì vậy (0, 0) là điểm yên ngựa.

Q9. What is Hessian of x^2+y^2? | Hessian của x^2+y^2 là gì?

A. `[[2,0],[0,-2]]` | `[[2,0],[0,-2]]`

B. `[[1,1],[y,x]]` | `[[1,1],[y,x]]`

C. `[[2,0],[0,2]]` | `[[2,0],[0,2]]`

D. `(3,2)` | `(3,2)`

Đáp án: C

Giải thích: Đạo hàm bậc hai của mỗi hạng bình phương bằng 2.

Q10. What is Hessian of x^2-y^2? | Hessian của x^2-y^2 là gì?

A. `[[2,0],[0,-2]]` | `[[2,0],[0,-2]]`

B. `(3,2)` | `(3,2)`

C. `[[1,1],[y,x]]` | `[[1,1],[y,x]]`

D. `[[2,0],[0,2]]` | `[[2,0],[0,2]]`

Đáp án: A

Giải thích: Độ cong theo y bằng âm 2.

Q11. What are stationary coordinates for $(x-3)^2+(y-2)^2$? | Tọa độ dừng của $(x-3)^2+(y-2)^2$ là gì?

A. `(3,2)` | `(3,2)`

B. `[[2,0],[0,2]]` | `[[2,0],[0,2]]`

C. `[[2,0],[0,-2]]` | `[[2,0],[0,-2]]`

D. `[[1,1],[y,x]]` | `[[1,1],[y,x]]`

Đáp án: A

Giải thích: Cả hai bình phương nhỏ nhất khi sai khác bằng 0.

Q12. What is Jacobian of outputs x+y and xy? | Jacobian của đầu ra x+y và xy là gì?

A. `[[2,0],[0,2]]` | `[[2,0],[0,2]]`

B. `[[2,0],[0,-2]]` | `[[2,0],[0,-2]]`

C. `(3,2)` | `(3,2)`

D. `[[1,1],[y,x]]` | `[[1,1],[y,x]]`

Đáp án: D

Giải thích: Lấy đạo hàm mỗi đầu ra theo x và y.

Q13. What does a positive-definite Hessian at a stationary point indicate? | Hessian xác định dương tại điểm dừng biểu thị gì?

A. A strict local minimum | Cực tiểu địa phương nghiêm ngặt

B. A saddle point | Điểm yên ngựa

C. The simple second-derivative test is inconclusive | Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

D. A strict local maximum | Cực đại địa phương nghiêm ngặt

Đáp án: A

Giải thích: Độ cong dương mọi hướng cho cực tiểu địa phương nghiêm ngặt.

Q14. What does a negative-definite Hessian at a stationary point indicate? | Hessian xác định âm tại điểm dừng biểu thị gì?

A. A strict local minimum | Cực tiểu địa phương nghiêm ngặt

B. A strict local maximum | Cực đại địa phương nghiêm ngặt

C. A saddle point | Điểm yên ngựa

D. The simple second-derivative test is inconclusive | Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

Đáp án: B

Giải thích: Độ cong âm mọi hướng cho cực đại địa phương nghiêm ngặt.

Q15. What do mixed-sign Hessian eigenvalues indicate at a stationary point? | Trị riêng Hessian trái dấu ở điểm dừng biểu thị gì?

A. A saddle point | Điểm yên ngựa

B. A strict local maximum | Cực đại địa phương nghiêm ngặt

C. A strict local minimum | Cực tiểu địa phương nghiêm ngặt

D. The simple second-derivative test is inconclusive | Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

Đáp án: A

Giải thích: Một số hướng tăng và hướng khác giảm.

Q16. What if a Hessian has a zero eigenvalue at a stationary point? | Nếu Hessian có trị riêng 0 tại điểm dừng thì sao?

A. A saddle point | Điểm yên ngựa

B. The simple second-derivative test is inconclusive | Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

C. A strict local maximum | Cực đại địa phương nghiêm ngặt

D. A strict local minimum | Cực tiểu địa phương nghiêm ngặt

Đáp án: B

Giải thích: Trị riêng 0 cần kiểm tra sâu hơn.

Q17. What is Hessian shape for a scalar function of three inputs? | Hessian hàm vô hướng ba biến có shape nào?

A. Sensitivities of one output to each input | Độ nhạy của một đầu ra theo từng đầu vào

B. How each output responds to one input | Các đầu ra đáp ứng với một đầu vào

C. `(3,3)` | `(3,3)`

D. It can also occur at a maximum or saddle | Nó còn xuất hiện ở cực đại hoặc yên ngựa

Đáp án: C

Giải thích: Các đạo hàm riêng bậc hai ghép từng biến với biến khác.

Q18. What does one Jacobian row represent? | Một hàng Jacobian biểu thị gì?

A. How each output responds to one input | Các đầu ra đáp ứng với một đầu vào

B. It can also occur at a maximum or saddle | Nó còn xuất hiện ở cực đại hoặc yên ngựa

C. `(3,3) | (3,3)`

D. Sensitivities of one output to each input | Độ nhạy của một đầu ra theo từng đầu vào

Đáp án: D

Giải thích: Hàng cố định đầu ra, cột đổi đầu vào.

Q19. What does one Jacobian column represent? | Một cột Jacobian biểu thị gì?

A. Sensitivities of one output to each input | Độ nhạy của một đầu ra theo từng đầu vào

B. It can also occur at a maximum or saddle | Nó còn xuất hiện ở cực đại hoặc yên ngựa

C. `(3,3) | (3,3)`

D. How each output responds to one input | Các đầu ra đáp ứng với một đầu vào

Đáp án: D

Giải thích: Cột cố định một đầu vào qua mọi đầu ra.

Q20. Why isn't zero gradient alone sufficient to prove a minimum? | Vì sao gradient 0 chưa đủ chứng minh cực tiểu?

A. Sensitivities of one output to each input | Độ nhạy của một đầu ra theo từng đầu vào

B. `(3,3) | (3,3)`

C. It can also occur at a maximum or saddle | Nó còn xuất hiện ở cực đại hoặc yên ngựa

D. How each output responds to one input | Các đầu ra đáp ứng với một đầu vào

Đáp án: C

Giải thích: Cực đại và yên ngựa cũng có thể có gradient 0.

8. Liên hệ bài tiếp theo

`Gradient → Jacobian → Hessian → Tối ưu mô hình.`

PY-34 — Xác suất, không gian mẫu và biến cố

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Experiment — phép thử; **Sample space** — không gian mẫu.

`Logic/Tập hợp → Xác suất → Bayes → Dự đoán bất định`

Học xong dùng để làm gì? Tính xác suất một sự kiện xảy ra và kiểm tra quy tắc của không gian mẫu.

Kết nối kiến thức tiếp theo: PY-35 tính xác suất khi đã có thông tin điều kiện và cập nhật bằng Bayes.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Experiment	phép thử	Thao tác có kết quả quan sát được
Sample space	không gian mẫu	Tập mọi kết quả khả dĩ
Event	biến cố	Nhóm kết quả được quan tâm
Probability	xác suất	Mức khả năng xảy ra một biến cố, từ 0 tới 1

3. Hàm Python và tham số quan trọng

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Cách sử dụng
`len(event)/len(space)`	Xác suất cổ điển khi kết quả đồng khả năng
`random.choice(values)`	Rút một phần tử ngẫu nhiên từ danh sách
`np.random.default_rng(seed)`	Tạo bộ sinh ngẫu nhiên tái lập
`np.mean(condition)`	Ước lượng tỷ lệ đúng trong mô phỏng

4. Khái niệm chính và liên quan

Mạch kiến thức: Từ một phép thử ta xác định không gian mẫu chứa mọi kết quả, chọn một tập con là biến cố, rồi mới tính xác suất dựa trên giả định đồng khả năng hoặc dựa vào phân phối đã biết. Mô phỏng cho tần suất quan sát, không thay thế xác suất lý thuyết.

4.1. Experiment — Phép thử và kết quả ngẫu nhiên

Từ vựng: Experiment — phép thử; **Outcome** — kết quả.

Phép thử ngẫu nhiên là thao tác có những kết quả khả dĩ biết trước, nhưng không biết chắc kết quả của **một lần thử**. Tung một đồng xu công bằng là ví dụ: kết quả có thể là ngửa hoặc sấp; mỗi lần chỉ xuất hiện một mặt. Để minh họa mà không in kết quả thất thường, tạo số ngẫu nhiên bằng `np.random.default_rng(seed)` giúp **tái lập cùng chuỗi giả ngẫu nhiên**, nhưng seed không biến thử nghiệm thành không ngẫu nhiên trong lý thuyết.

No Note — Không chú thích

```
import numpy as np

rng = np.random.default_rng(42)
print(rng.integers(0, 2, size=5).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Mô phỏng năm phép thử 0/1 với seed cố định để học cách tái lập thí nghiệm.
# Dữ liệu: Mỗi lượt nhận 0 hoặc 1; seed 42 để chạy lại dãy số giả ngẫu nhiên.
# NumPy cung cấp bộ sinh ngẫu nhiên và ndarray.
import numpy as np

# default_rng(42) tạo bộ sinh riêng với seed 42 để tái lập mô phỏng; không làm cho xác suất thực
thay đổi.
rng = np.random.default_rng(42)
# integers(low,high,size) sinh 5 số nguyên 0 hoặc 1; cận trên 2 KHÔNG được lấy.
# tolist() chuyển mảng kết quả sang list Python để hiển thị.
print(rng.integers(0, 2, size=5).tolist())
```

Kết quả khi chạy:

```
[0, 1, 1, 0, 0]
```

Năm kết quả mô phỏng 0/1 với seed 42 tái lập được trong phiên bản NumPy tương thích.

4.2. Sample space — Liệt kê toàn bộ kết quả có thể

Từ vựng: Sample space — không gian mẫu; **Outcome** — phần tử mẫu.

Không gian mẫu Ω chứa toàn bộ kết quả có thể của phép thử. Với xúc xắc sáu mặt chuẩn, Ω={1,2,3,4,5,6}; mỗi số chỉ là **một kết quả**, chưa phải xác suất. Đếm `len(Ω)=6` cho biết số trường hợp khả dĩ. Chỉ khi xúc xắc **công bằng** ta mới giả định cả sáu mặt đồng xác suất `1/6`. Set không bảo đảm thứ tự hiển thị nên in qua `sorted()`.

No Note — Không chú thích

```
space = set(range(1, 7))
print(sorted(space))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Liệt kê không gian mẫu sáu mặt của xúc xắc.
# Dữ liệu: Không gian mẫu là các mặt 1,2,3,4,5,6; giả định đồng khả năng.
# range(1,7) sinh từ 1 đến 6, không lấy số 7.
# set() lưu tập kết quả không trùng, đây là không gian mẫu.
space = set(range(1, 7))
```

```
# sorted() tạo danh sách tăng dần để in; set tự nó không bảo đảm thứ tự.
print(sorted(space))
```

Kết quả khi chạy:

```
[1, 2, 3, 4, 5, 6]
```

Không gian mẫu của một lần gieo có sáu mặt, được sắp thứ tự để dễ đọc.

4.3. Event — Một nhóm kết quả quan tâm

Từ vựng: Event — biến cố; **Subset** — tập con.

Biến cố là một tập con của không gian mẫu. Khi gieo xúc xắc, biến cố `E = ra số chẵn` gồm `{2,4,6}`. Nó không phải một kết quả duy nhất: trong một lần gieo, E xảy ra nếu số được gieo **thuộc** Set E. Gọi `sorted(E)` để hiển thị rõ ba trường hợp; phép kiểm tra `4 in E` cho True.

No Note — Không chú thích

```
even = {2, 4, 6}
print(sorted(even))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Biểu diễn tập biến cố các mặt chẵn của xúc xắc.
# Dữ liệu: Biến cố mặt chẵn gồm 2,4,6 trên tổng sáu mặt.
# Tập set chỉ chứa các mặt chẵn 2,4,6; là một biến cố con của không gian mẫu.
even = {2, 4, 6}
# sorted() sắp tập thành list ổn định để hiển thị.
print(sorted(even))
```

Kết quả khi chạy:

```
[2, 4, 6]
```

Biến cố ra số chẵn có ba kết quả thuận lợi trong sáu mặt xúc xắc.

4.4. Probability — Tỷ lệ khả năng dưới giả định

Từ vựng: Probability — xác suất; **Equally likely** — đồng khả năng.

Xác suất P(E) là số trong `[0,1]` đo khả năng của biến cố E. Khi các phần tử không gian mẫu **đồng khả năng**, `P(E)=số kết quả thuận lợi / số kết quả có thể`. Với xúc xắc công bằng, E là các mặt chẵn `{2,4,6}` nên `P(E)=3/6=0.5`. Nếu xúc xắc lệch, không thể dùng số mặt thuận lợi chia số mặt một cách máy móc: cần xác suất của từng mặt.

No Note — Không chú thích

```
space = set(range(1, 7))
even = {2, 4, 6}
print(len(even) / len(space))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính xác suất lý thuyết gieo mặt chẵn trong sáu mặt đồng khả năng.
# Dữ liệu: Ba mặt chẵn trên sáu mặt đồng khả năng; xác suất là 3/6.
# range(1,7) tạo sáu mặt; số 7 không nằm trong kết quả.
# set() tạo tập không gian mẫu gồm sáu kết quả.
space = set(range(1, 7))
# Set {2,4,6} là biến cố mặt chẵn với ba kết quả thuận lợi.
even = {2, 4, 6}
# len() đếm 3 mặt chẵn và 6 mặt toàn bộ; chỉ được coi là P=3/6 khi các mặt đồng khả năng.
print(len(even) / len(space))
```

Kết quả khi chạy:

```
0.5
```

Một lần gieo xúc xắc công bằng có xác suất 50% ra mặt chẵn.

5. Năm tình huống xác suất — từ vé đồng khả năng đến phân phối có trọng số

Tình huống 1 — Cấp 1: Không gian mẫu gồm 10 vé quay thưởng có xác suất bằng nhau

Từ vựng trong mục: Sample space — không gian mẫu; **Event** — biến cố; **Equally likely** — đồng khả năng.

Tình huống thực tế: Quầy sự kiện chuẩn bị **10 chiếc vé V01–V10**, mỗi vé được ghi màu **Xanh, Đỏ hoặc Vàng**. Có **4 vé Xanh**; mọi chiếc vé có cơ hội được rút như nhau, kể cả các vé cùng màu. Người tổ chức muốn biết cơ hội lấy **một vé Xanh** khi rút ngẫu nhiên một vé, và vì sao có ba tên màu **không có nghĩa** mỗi màu đều có cơ hội một phần ba.

Vì sao chọn cách này? Không gian mẫu trong bài gồm **10 chiếc vé riêng biệt**, không phải chỉ ba tên màu; mỗi vé là một khả năng được chọn. **Biến cố** là yêu cầu đang xét, ở đây **rút trúng vé Xanh**. Vì mọi chiếc vé đồng khả năng, chương trình đếm các dòng có màu Xanh bằng vòng `for`, rồi lấy **số vé Xanh so với cả 10 vé**. Bốn vé Xanh cho cơ hội **0,4**, không phải **1/3**, vì số vé của từng màu khác nhau. Phải xác định **đơn vị được rút** trước khi chọn mẫu số xác suất.

Quy trình:

1. Lưu màu trên mười vé theo thứ tự mã.
2. Xác định biến cố `vé Xanh`, đếm vé thuộc biến cố.
3. Chia số vé thuận lợi cho tổng số vé khi giả định mọi vé như nhau.

No Note — Không chú thích

```
colors = ["Xanh","Đỏ","Xanh","Vàng","Đỏ","Xanh","Đỏ","Vàng","Xanh","Đỏ"]
blue = []
for i, color in enumerate(colors, 1):
    if color == "Xanh":
        blue.append(i)
if len(colors) != 10:
    raise ValueError("Cần đúng 10 vé theo màu.")
print("Vé:", len(colors))
print("Vé xanh:", len(blue))
print(f"P(rút xanh): {len(blue)/len(colors):.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm vé màu xanh trong mười vé và tìm xác suất rút xanh.
# Dữ liệu: 10 vé có màu Xanh, Đỏ, Vàng; mỗi phần tử list là một vé, không phải một loại màu.
# Danh sách 10 vé, mỗi vị trí là một vé; màu trùng vẫn là hai vé riêng.
colors = ["Xanh","Đỏ","Xanh","Vàng","Đỏ","Xanh","Đỏ","Vàng","Xanh","Đỏ"]
# List rỗng để thu số thứ tự vé màu xanh.
blue = []
# enumerate(...,1) trả (số thứ tự từ 1,màu), khác chỉ số Python bắt đầu 0.
for i, color in enumerate(colors, 1):
    # Kiểm màu chính xác của từng vé, trả Boolean.
    if color == "Xanh":
        # append() lưu số thứ tự một vé xanh, không thêm màu duy nhất.
        blue.append(i)
# len() đếm tổng vé 10; len(blue) đếm số vé xanh.
if len(colors) != 10:
    # ValueError báo số lượt hoặc tính hợp lệ của phép thử không đạt điều kiện, tránh công bố xác
suất sai.
    raise ValueError("Cần đúng 10 vé theo màu.")
# len() đếm tổng vé 10; len(blue) đếm số vé xanh.
print("Vé:", len(colors))
print("Vé xanh:", len(blue))
# len() đếm tổng vé 10; len(blue) đếm số vé xanh.
# Tỷ lệ vé xanh/tổng vé là xác suất nếu rút ngẫu nhiên đồng đều một vé; :.2f chỉ làm tròn khi in.
print(f"P(rút xanh): {len(blue)/len(colors):.2f}")
```

Kết quả khi chạy:

```
Vé: 10
Vé xanh: 4
P(rút xanh): 0.40
```

Nếu quay thưởng thiên vị một vé, phép đếm tỷ lệ này không còn là xác suất đúng; giả định đồng khả năng là điều kiện không thể bỏ qua.

Tình huống 2 — Cấp 2: So tỷ lệ thực nghiệm từ 12 lần tung với lý thuyết

Từ vựng trong mục: Frequency — tần suất; **Probability** — xác suất; **Random experiment** — phép thử ngẫu nhiên.

Tình huống thực tế: Lớp học có **12 kết quả tung đồng xu giả lập** theo thứ tự ghi nhận; `N` là **ngửa**, `S` là **sấp**. Trong mẫu này số lần ngửa không nhất thiết bằng số lần sấp, dù ta **giả định đồng xu công bằng** khi nói về xác suất lý thuyết. Học viên muốn đếm số lần ngửa đã thấy và so **tỷ lệ quan sát từ 12 lượt** với **xác suất lý thuyết 0,5**, không kết luận đồng xu hỏng chỉ từ một mẫu nhỏ.

Vì sao chọn cách này? Xác suất lý thuyết là cơ hội do mô hình đồng xu công bằng giả định, mỗi mặt có cùng khả năng. **Tần suất quan sát** là tỷ lệ thực sự nhìn thấy trong **12 lượt đã ghi**; mẫu nhỏ có thể lệch 0,5 do may rủi. Chương trình đếm mỗi chữ `N` rồi chia cho đúng **12 lượt**, sau đó in riêng con số **0,500** của giả định lý thuyết. Không lấy tỷ lệ thực nghiệm bằng chứng đồng xu chắc chắn thiên lệch hay hoàn toàn công bằng; cần nhiều bằng chứng hơn để đánh giá điều đó.

Quy trình:

1. Lưu đúng 12 lượt kết quả.
2. Đếm ký hiệu N, lấy tỷ lệ chia 12.
3. In tỷ lệ quan sát và giả định lý thuyết để so sánh.

No Note — Không chú thích

```
observed = ["N","S","N","N","S","N","S","S","N","N","S","N"]
heads = 0
for item in observed:
    if item == "N":
        heads += 1
if len(observed) != 12:
    raise ValueError("Cần đủ 12 lượt quan sát tung đồng xu.")
print("Lượt tung:", len(observed))
print("Ngửa:", heads)
print(f"Tỷ lệ ngửa quan sát: {heads/len(observed):.3f}")
print("Xác suất lý thuyết: 0.500")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt tỷ lệ ngửa quan sát trong 12 lần tung và xác suất đồng xu công bằng.
# Dữ liệu: 12 kết quả N (ngửa) và S (sấp); tỷ lệ quan sát có thể lệch mức lý thuyết 0.5.
# 12 nhãn N/S là dữ liệu THỰC NGHIỆM, không phải số lần kỳ vọng lý thuyết.
observed = ["N","S","N","N","S","N","S","S","N","N","S","N"]
# Biến đếm lượt ngửa N, bắt đầu 0.
heads = 0
# Duyệt từng lần tung, không chỉ các nhãn khác nhau.
for item in observed:
    # So chính xác nhãn N để nhận lượt ngửa.
    if item == "N":
        # += 1 tăng số lần quan sát ngửa.
        heads += 1
# len() đếm 12 lần tung làm mẫu số của tỷ lệ thực nghiệm.
if len(observed) != 12:
    # ValueError báo số lượt hoặc tính hợp lệ của phép thử không đạt điều kiện, tránh công bố xác
suất sai.
    raise ValueError("Cần đủ 12 lượt quan sát tung đồng xu.")
# len() đếm 12 lần tung làm mẫu số của tỷ lệ thực nghiệm.
print("Lượt tung:", len(observed))
print("Ngửa:", heads)
# len() đếm 12 lần tung làm mẫu số của tỷ lệ thực nghiệm.
# Số lượt ngửa trên 12 lượt là tần suất quan sát; khác xác suất lý thuyết 0.5 của xu công bằng.
print(f"Tỷ lệ ngửa quan sát: {heads/len(observed):.3f}")
# Số 0.500 được in sẵn theo GIẢ ĐỊNH xu công bằng, không được suy ra từ 12 lượt.
print("Xác suất lý thuyết: 0.500")
```

Kết quả khi chạy:

```
Lượt tung: 12
Ngửa: 7
Tỷ lệ ngửa quan sát: 0.583
Xác suất lý thuyết: 0.500
```

Mẫu 12 lần chỉ minh họa **tần suất**, không thay thế toàn bộ xác suất lý thuyết của một đồng xu công bằng.

Tình huống 3 — Cấp 3: Phân biệt 15 kết quả gieo hai xúc xắc với 36 khả năng

Từ vựng trong mục: Outcome pair — cặp kết quả; **Theoretical probability** — xác suất lý thuyết.

Tình huống thực tế: Đội kiểm thử ghi **15 lượt tung hai xúc xắc**, mỗi lượt có hai mặt hiện lên từ 1 đến 6. Họ quan tâm tình huống **tổng hai mặt ít nhất là 9**. Cần tính một tỷ lệ dựa trên **15 lượt thực sự ghi được**, rồi so với xác suất lý thuyết khi xét **đủ 36 cặp mặt** có thể xuất hiện trong một lần tung hai xúc xắc công bằng.

Vì sao chọn cách này? Mỗi lượt tung **hai xúc xắc** tạo một **cặp có thứ tự**: ví dụ (3,6) và (6,3) là hai kết quả khác nhau của hai viên được phân biệt. `itertools.product(range(1, 7), repeat=2)` liệt kê **36 cặp khả dĩ** của **một lượt**, từ đó đếm bao nhiêu cặp có tổng ít nhất 9 để tính xác suất lý thuyết. Danh sách `observations` lại chỉ có **15 lượt đã quan sát**, nên phải đếm trường hợp đạt trong danh sách ấy và chia cho **15**, không chia cho 36. Hai mẫu số trả lời hai câu hỏi khác nhau; một mẫu nhỏ không buộc phải khớp chính xác lý thuyết.

Quy trình:

1. Lưu 15 cặp kết quả được quan sát và đếm tổng >=9.
2. Liệt kê 36 cặp khả dĩ khi hai xúc xắc độc lập/công bằng.
3. Báo tỷ lệ quan sát và xác suất lý thuyết có nhãn riêng.

No Note — Không chú thích

```
from itertools import product
observations = [[1,1],[2,5],[3,6],[4,5],[6,6],[1,4],[5,5],[2,2],[4,6],[3,3],[6,2],[5,4],[1,6],
[6,5],[2,4]]
space = list(product(range(1, 7), repeat=2))
observed_hits = 0
for a, b in observations:
    if a + b >= 9:
        observed_hits += 1
possible_hits = 0
for a, b in space:
    if a + b >= 9:
        possible_hits += 1
if len(observations) != 15 or len(space) != 36:
    raise ValueError("Cần 15 lượt quan sát và 36 khả năng khi tung hai xúc xắc.")
print("Lượt quan sát:", len(observations))
print("Quan sát tổng >=9:", observed_hits)
print(f"Tỷ lệ quan sát: {observed_hits/len(observations):.3f}")
print(f"Xác suất lý thuyết: {possible_hits/len(space):.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm biến cố tổng hai xúc xắc từ 9 trở lên trong 15 quan sát và 36 khả năng lý thuyết.
# Dữ liệu: 15 lượt gieo cặp xúc xắc; không gian lý thuyết gồm 6×6=36 cặp có thứ tự.
# product() sinh tất cả tổ hợp có thứ tự từ các nhóm lựa chọn.
from itertools import product
# 15 cặp xúc xắc được quan sát thực tế, không đại diện toàn bộ 36 khả năng.
observations = [[1,1],[2,5],[3,6],[4,5],[6,6],[1,4],[5,5],[2,2],[4,6],[3,3],[6,2],[5,4],[1,6],
[6,5],[2,4]]
# repeat=2 tạo hai vị trí xúc xắc từ 1..6, có 6×6=36 cặp có thứ tự; list() hiện thực iterator.
space = list(product(range(1, 7), repeat=2))
observed_hits = 0
# Tách hai mặt của từng lượt quan sát.
for a, b in observations:
    # Biến cố tổng của HAI mặt ít nhất bằng 9, so từng cặp.
    if a + b >= 9:
        # Tăng số lần biến cố xuất hiện trong dữ liệu 15 lượt.
        observed_hits += 1
possible_hits = 0
# Duyệt TOÀN BỘ 36 khả năng để đếm theo lý thuyết.
for a, b in space:
    # Biến cố tổng của HAI mặt ít nhất bằng 9, so từng cặp.
    if a + b >= 9:
        # Đếm số kết quả thuận lợi trong không gian 36 cặp.
        possible_hits += 1
# len() đếm số lượt quan sát thực nghiệm, ở đây 15.
# len() đếm số cặp lý thuyết, phải bằng 36.
```

```
if len(observations) != 15 or len(space) != 36:
    # ValueError báo sô´lượt hoặc tính hợp lệ của phép thư˙không đạt điêù kiện, tránh công bô´xác
suât sai.
    raise ValueError("Cân 15 lượt quan sát và 36 khả năng khi tung hai xúc xăć.")
# len() đêḿ sô´lượt quan sát thực nghiệm, ở đây 15.
print("Lượt quan sát:", len(observations))
print("Quan sát tổng >=9:", observed_hits)
# len() đêḿ sô´lượt quan sát thực nghiệm, ở đây 15.
# Tỷ lệ quan sát có thê˙lệch xác suât lý thuyêt vì chỉ có 15 lượt.
print(f"Tỷ lệ quan sát: {observed_hits/len(observations):.3f}")
# len() đêḿ sô´cặp lý thuyêt, phải bằng 36.
# Đêḿ kêt quả thuận lợi trên 36 cặp đôìng khả năng, cho xác suât lý thuyêt.
print(f"Xác suât lý thuyêt: {possible_hits/len(space):.3f}")
```

Kết quả khi chạy:

```
Lượt quan sát: 15
Quan sát tổng >=9: 7
Tỷ lệ quan sát: 0.467
Xác suât lý thuyêt: 0.278
```

Mười lăm lượt có thể có nhiều cặp lặp lại. Một phép thử có 36 cặp khả dĩ, còn bảng chỉ chứa 15 quan sát của những lần đã gieo.

Tình huống 4 — Cấp 4: Ước lượng biến cố hợp và giao từ 18 lượt tiếp cận

Từ vựng trong mục: Union — phép hợp; **Intersection** — phép giao; **Inclusion–exclusion** — bao hàm–loại trừ.

Tình huống thực tế: Chiến dịch marketing ghi **18 khách U01–U18**, mỗi người có hai trạng thái đúng/sai: đã dùng **ứng dụng** và đã nhận **email**. Một khách có thể dùng cả hai kênh. Khi chọn ngẫu nhiên một khách trong đúng 18 người của bảng, quản lý muốn biết cơ hội khách ấy thuộc **ít nhất một kênh**, cơ hội thuộc **cả hai kênh**, và kiểm tra rằng số người dùng hai kênh không bị cộng trùng.

Vì sao chọn cách này? Biến cố hợp là khách xuất hiện ở **ít nhất một** trong hai nhóm ứng dụng/email; **biến cố giao** là khách có ở **cả hai nhóm**. `or` kiểm hợp, `and` kiểm giao trên **từng người**, tránh nhầm giữa các khách. Để tính số người thuộc hợp, lấy **số người dùng app cộng số người nhận email**, rồi **trừ phần giao một lần**, vì người thuộc cả hai đã bị cộng hai lần. Chương trình cũng đếm trực tiếp bằng `or` để kiểm lại, sau đó chia theo **18 khách được chọn đồng khả năng trong bảng**. Đây là xác suất theo **mẫu 18 người**, không mặc nhiên suy ra toàn bộ khách hàng ngoài chiến dịch.

Quy trình:

1. Đếm số khách có từng cờ và số khách có cả hai.
2. Dùng `app or email` cho biến cố hợp, `app and email` cho giao.
3. So xác suất hợp trực tiếp với công thức hợp bằng hai tỷ lệ trừ giao.

No Note — Không chú thích

```
records = [{"id":"U01","app":False,"email":False},{"id":"U02","app":True,"email":True},
{"id":"U03","app":True,"email":True},{"id":"U04","app":False,"email":True},
{"id":"U05","app":True,"email":False},{"id":"U06","app":True,"email":True},
{"id":"U07","app":False,"email":True},{"id":"U08","app":True,"email":True},
{"id":"U09","app":True,"email":False},{"id":"U10","app":False,"email":True},
{"id":"U11","app":True,"email":True},{"id":"U12","app":True,"email":True},
{"id":"U13","app":False,"email":False},{"id":"U14","app":True,"email":True},
{"id":"U15","app":True,"email":True},{"id":"U16","app":False,"email":True},
```

```
{"id":"U17","app":True,"email":False},{"id":"U18","app":True,"email":True}]
app = 0
email = 0
both = 0
either = 0
for row in records:
    if row["app"]:
        app += 1
    if row["email"]:
        email += 1
    if row["app"] and row["email"]:
        both += 1
    if row["app"] or row["email"]:
        either += 1
if len(records) != 18 or app + email - both != either:
    raise ValueError("Cần 18 khách và phép đếm hợp tập hợp phải nhất quán.")
print("Khách:", len(records))
print("Cả hai kênh:", both)
print("Ít nhất một kênh:", either)
print(f"P(hợp): {either/len(records):.3f}")
print("Kiểm công thức:", either == app+email-both)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính xác suất khách dùng ít nhất một kênh app/email bằng nguyên lý cộng-trừ giao.
# Dữ liệu: 18 hồ sơ khách có hai cờ Boolean app/email; khách dùng hai kênh không được đếm hai lần.
# 18 dictionary, mỗi hồ sơ khách có hai cờ dùng app/email, có thể cùng True.
records = [{"id":"U01","app":False,"email":False},{"id":"U02","app":True,"email":True},
{"id":"U03","app":True,"email":True},{"id":"U04","app":False,"email":True},
{"id":"U05","app":True,"email":False},{"id":"U06","app":True,"email":True},
{"id":"U07","app":False,"email":True},{"id":"U08","app":True,"email":True},
{"id":"U09","app":True,"email":False},{"id":"U10","app":False,"email":True},
{"id":"U11","app":True,"email":True},{"id":"U12","app":True,"email":True},
{"id":"U13","app":False,"email":False},{"id":"U14","app":True,"email":True},
{"id":"U15","app":True,"email":True},{"id":"U16","app":False,"email":True},
{"id":"U17","app":True,"email":False},{"id":"U18","app":True,"email":True}]
# Đếm người dùng app theo hồ sơ, không phải số lượt truy cập.
app = 0
# Đếm người dùng email.
email = 0
# Đếm phần giao, khách dùng CẢ app và email.
both = 0
# Đếm phần hợp, khách dùng ít nhất một kênh.
either = 0
# Duyệt từng khách một lần.
for row in records:
    # Tra cờ Boolean của app; True nghĩa khách dùng app.
    if row["app"]:
        app += 1
    # Tra cờ Boolean của email; True nghĩa khách dùng email.
    if row["email"]:
        email += 1
    # Tra cờ Boolean của app; True nghĩa khách dùng app.
    # Tra cờ Boolean của email; True nghĩa khách dùng email.
    # and True khi cả hai cờ cùng đúng, chính là giao A∩B.
    if row["app"] and row["email"]:
        both += 1
    # Tra cờ Boolean của app; True nghĩa khách dùng app.
    # Tra cờ Boolean của email; True nghĩa khách dùng email.
    # or True khi có ít nhất một cờ đúng, chính là hợp A∪B.
    if row["app"] or row["email"]:
        either += 1
# Nguyên lý cộng-trừ giao: cộng hai nhóm rồi trừ khách đã bị đếm hai lần.
if len(records) != 18 or app + email - both != either:
    # ValueError báo số lượt hoặc tính hợp lệ của phép thử không đạt điều kiện, tránh công bố xác
```

```
suất sai.
    raise ValueError("Cần 18 khách và phép đếm hợp tập hợp phải nhất quán.")
print("Khách:", len(records))
print("Cả hai kênh:", both)
print("Ít nhất một kênh:", either)
# Số khách thuộc hợp chia tổng 18 khách cho xác suất thực nghiệm dùng ít nhất một kênh.
print(f"P(hợp): {either/len(records):.3f}")
print("Kiểm công thức:", either == app+email-both)
```

Kết quả khi chạy:

```
Khách: 18
Cả hai kênh: 9
Ít nhất một kênh: 16
P(hợp): 0.889
Kiểm công thức: True
```

Đây là xác suất **khi chọn đều một khách từ danh sách 18 người này**, không tự suy ra tỷ lệ của tất cả khách ngoài danh sách.

Tình huống 5 — Cấp 5: Hai mươi kết quả không đồng khả năng: xác suất có trọng số

Từ vựng trong mục: Weighted outcome — kết quả có trọng số; **Sampling distribution** — phân phối chọn; **Monte Carlo** — mô phỏng ngẫu nhiên.

Tình huống thực tế: Máy kiểm thử có **20 loại trạng thái S01–S20** nhưng **không xuất hiện đều như nhau**. Mỗi trạng thái được gán **trọng số 1 đến 20**: trọng số càng lớn thì cơ hội được rút càng cao; đây là **mức ưu tiên mô phỏng**, không phải số lần đo. Nhóm muốn biết xác suất rút một trạng thái trong **S16–S20**, rồi dùng **5.000 lần rút giả lập** với **seed 42** để xem tỷ lệ mô phỏng có gần xác suất lý thuyết không.

Vì sao chọn cách này? Vì **20 trạng thái có trọng số khác nhau**, không thể coi mỗi trạng thái có cơ hội như nhau. **Chuẩn hóa** là chia từng trọng số cho **tổng trọng số**, tạo một **phân phối xác suất** có tổng bằng 1. Chương trình cộng riêng phần xác suất của **năm trạng thái S16–S20** để có xác suất lý thuyết; cách lấy 5/20 sẽ sai vì bỏ qua trọng số. `np.random.default_rng(42)` dùng **seed 42** để lần chạy lại cùng code cho kết quả mô phỏng tái lập được; `rng.choice(..., size=5000, p=probs)` rút **5.000 lần có trọng số** nhằm nhìn tần suất dài hạn hơn 20 lượt, chứ **không phải có thêm 5.000 mẫu dữ liệu thật**. Tần suất gần lý thuyết là điều có thể kỳ vọng, **không bắt buộc trùng hoàn toàn** và không dùng 5.000 như một hằng số toán học.

Quy trình:

1. Tạo 20 trạng thái với trọng số 1–20, chuẩn hóa thành xác suất.
2. Cộng trọng số 5 trạng thái quan tâm rồi chia tổng.
3. Lấy 5.000 mẫu có seed 42, kiểm tần suất gần xác suất có trọng số.

No Note — Không chú thích

```
import numpy as np
weights = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20], dtype=float)
probs = weights / weights.sum()
theoretical = float(probs[15:].sum())
rng = np.random.default_rng(42)
sampled = rng.choice(np.arange(20), size=5000, p=probs)
frequency = float(np.mean(sampled >= 15))
if len(weights) != 20 or not np.isclose(probs.sum(), 1):
```

```
    raise ValueError("Cần 20 trọng số và tổng xác suất bằng 1.")
print("Trạng thái:", len(weights))
print(f"P(S16–S20): {theoretical:.3f}")
print("Đồng khả năng 5/20:", f"{5/20:.3f}")
print("Mô phỏng gần lý thuyết:", abs(frequency-theoretical) < 0.03)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy mẫu từ 20 trạng thái có xác suất không đều và so tần suất mô phỏng với lý thuyết.
# Dữ liệu: 20 trạng thái có trọng số 1..20; chỉ số 15..19 ứng với S16..S20, 5.000 lần rút thử.
# NumPy tạo phân phối trọng số và thực hiện mô phỏng rút ngẫu nhiên.
import numpy as np
# 20 trọng số 1..20 theo trạng thái S01..S20, dtype=float cho phép chia để chuẩn hóa.
weights = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20], dtype=float)
# weights.sum() cộng tất cả trọng số; chia từng trọng số cho tổng để được xác suất có tổng 1.
probs = weights / weights.sum()
# [15:] lấy chỉ số từ 15 đến cuối, tức năm trạng thái S16..S20; .sum() tính xác suất lý thuyết.
theoretical = float(probs[15:].sum())
# Generator với seed 42 giúp chạy lại thí nghiệm mô phỏng.
rng = np.random.default_rng(42)
# arange(20) tạo chỉ số 0..19, không chứa 20.
# choice(các_trạng_thái,size=5000,p=probs) chọn 5.000 lần có hoàn lại theo phân phối trọng số;
KHÔNG đồng khả năng.
sampled = rng.choice(np.arange(20), size=5000, p=probs)
# Tạo mask Boolean cho các trạng thái có chỉ số 15..19.
# mean() trên mask True/False là tỷ lệ lượt mô phỏng rơi vào S16..S20.
frequency = float(np.mean(sampled >= 15))
# isclose() kiểm tổng xác suất gần 1 trong dung sai số thực, tránh so == tuyệt đối.
if len(weights) != 20 or not np.isclose(probs.sum(), 1):
    # ValueError báo số lượt hoặc tính hợp lệ của phép thử không đạt điều kiện, tránh công bố xác
suất sai.
    raise ValueError("Cần 20 trọng số và tổng xác suất bằng 1.")
print("Trạng thái:", len(weights))
print(f"P(S16–S20): {theoretical:.3f}")
# Tỷ lệ 5/20 giả sử đồng khả năng, dùng so với phân phối thực tế có trọng số không đều.
print("Đồng khả năng 5/20:", f"{5/20:.3f}")
# abs() lấy độ lệch mô phỏng-lý thuyết; 0.03 là ngưỡng chấp nhận minh họa, KHÔNG bảo đảm mọi lần mô
phỏng đều đạt.
print("Mô phỏng gần lý thuyết:", abs(frequency-theoretical) < 0.03)
```

Kết quả khi chạy:

```
Trạng thái: 20
P(S16–S20): 0.429
Đồng khả năng 5/20: 0.250
Mô phỏng gần lý thuyết: True
```

Tần suất 5.000 lượt phụ thuộc seed và có dao động ngẫu nhiên; **20 bản ghi** ở đây là các trạng thái khả dĩ, không phải 20 lượt mẫu.

6. Sai lầm thường gặp

- Phép thử không giống biến cố: phép thử tạo kết quả, biến cố là nhóm kết quả được quan tâm.
- Công thức số kết quả thuận lợi chia tổng kết quả chỉ đúng với các kết quả **đồng khả năng**.
- Trong mô phỏng, tỷ lệ quan sát có thể khác xác suất thật ngay cả khi seed cố định.
- `np.random.default_rng(seed)` giúp tái lập trong môi trường tương thích, không đảm bảo mẫu luôn đúng tỷ lệ lý thuyết.
- Set không có thứ tự cố định, nên in bằng `sorted` nếu cần đối chiếu đầu ra.

- Khi số mẫu tăng, độ dao động của ước lượng thường giảm, không bắt buộc mỗi lần chạy đều gần hơn.

7. Trắc nghiệm

Q1. What is a random experiment? | Phép thử ngẫu nhiên là gì?

A. Set of all possible outcomes | Tập tất cả kết quả có thể

B. A subset of the sample space | Một tập con của không gian mẫu

C. A measure between zero and one of an event's chance | Thước đo từ 0 đến 1 cho khả năng biến cố

D. A repeatable procedure with uncertain individual outcome | Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

Đáp án: D

Giải thích: Phép thử có thể sinh kết quả khác nhau.

Q2. What is a sample space? | Không gian mẫu là gì?

A. A subset of the sample space | Một tập con của không gian mẫu

B. A measure between zero and one of an event's chance | Thước đo từ 0 đến 1 cho khả năng biến cố

C. A repeatable procedure with uncertain individual outcome | Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

D. Set of all possible outcomes | Tập tất cả kết quả có thể

Đáp án: D

Giải thích: Không gian mẫu chứa mọi kết quả khả dĩ.

Q3. What is an event? | Biến cố là gì?

A. Set of all possible outcomes | Tập tất cả kết quả có thể

B. A measure between zero and one of an event's chance | Thước đo từ 0 đến 1 cho khả năng biến cố

C. A repeatable procedure with uncertain individual outcome | Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

D. A subset of the sample space | Một tập con của không gian mẫu

Đáp án: D

Giải thích: Biến cố là một nhóm kết quả.

Q4. What is probability? | Xác suất là gì?

A. Set of all possible outcomes | Tập tất cả kết quả có thể

B. A repeatable procedure with uncertain individual outcome | Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

C. A subset of the sample space | Một tập con của không gian mẫu

D. A measure between zero and one of an event's chance | Thước đo từ 0 đến 1 cho khả năng biến cố

Đáp án: D

Giải thích: Xác suất trong đoạn 0 đến 1.

Q5. For a fair coin, what is the probability of heads on one toss? | Với đồng xu công bằng, xác suất được mặt ngửa trong một lần tung là bao nhiêu?

A. 0 | 0

B. 0.25 | 0.25

C. 1 | 1

D. 0.5 | 0.5

Đáp án: D

Giải thích: Đồng xu có hai mặt đồng khả năng; xác suất mặt ngửa là 1/2 = 0.5.

Q6. If P(A) = 0.35, what is P(not A)? | Nếu P(A) = 0.35, xác suất biến cố đối của A bằng bao nhiêu?

A. 1.35 | 1.35

B. 0.75 | 0.75

C. 0.35 | 0.35

D. 0.65 | 0.65

Đáp án: D

Giải thích: Tổng xác suất của A và biến cố đối là 1, nên P(không A) = 1 − 0.35 = 0.65.

Q7. A fair six-sided die is rolled once. What is P(result > 4)? | Tung một xúc xắc sáu mặt công bằng một lần, xác suất ra số lớn hơn 4 là bao nhiêu?

A. 1/2 | 1/2

B. 1/3 | 1/3

C. 0 | 0

D. 1/6 | 1/6

Đáp án: B

Giải thích: Hai mặt thuận lợi là 5 và 6 trong tổng 6 mặt đồng khả năng: 2/6 = 1/3.

Q8. Two fair coin tosses are independent. What is P(both heads)? | Hai lần tung đồng xu công bằng độc lập. Xác suất cả hai đều ra ngửa là bao nhiêu?

A. 0.75 | 0.75

B. 0.5 | 0.5

C. 1 | 1

D. 0.25 | 0.25

Đáp án: D

Giải thích: Nhân xác suất hai biến cố độc lập: 0.5 × 0.5 = 0.25.

Q9. What is P(even) for a fair six-sided die? | Xác suất mặt chẵn của xúc xắc công bằng sáu mặt?

A. 4 | 4

B. 36 | 36

C. 1/2 | 1/2

D. Zero | Bằng 0

Đáp án: C

Giải thích: Ba trong sáu mặt là số chẵn.

Q10. How many outcomes are possible with two ordered six-sided dice? | Hai xúc xắc sáu mặt theo cặp có thứ tự có bao nhiêu kết quả?

A. 4 | 4

B. 36 | 36

C. Zero | Bằng 0

D. 1/2 | 1/2

Đáp án: B

Giải thích: Hai xúc xắc có 6×6 cặp có thứ tự.

Q11. How many ordered pairs of dice sum to nine? | Bao nhiêu cặp có thứ tự của hai xúc xắc có tổng 9?

A. 36 | 36

B. 4 | 4

C. 1/2 | 1/2

D. Zero | Bằng 0

Đáp án: B

Giải thích: Bốn cặp là (3,6), (4,5), (5,4), (6,3).

Q12. What is the probability of the empty event? | Xác suất biến cố rỗng là gì?

A. 36 | 36

B. 1/2 | 1/2

C. Zero | Bằng 0

D. 4 | 4

Đáp án: C

Giải thích: Biến cố không thể có xác suất 0.

Q13. What is the probability of the entire sample space? | Xác suất của cả không gian mẫu là gì?

A. No, sampling variation occurs | Không, có dao động do lấy mẫu

B. `1-P(A)` | `1-P(A)`

C. One | Bằng 1

D. One die's result does not change probabilities of the other | Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

Đáp án: C

Giải thích: Biến cố chắc chắn có xác suất 1.

Q14. What is P(not A) in terms of P(A)? | P(không A) theo P(A) là gì?

A. One | Bằng 1

B. `1-P(A)` | `1-P(A)`

C. No, sampling variation occurs | Không, có dao động do lấy mẫu

D. One die's result does not change probabilities of the other | Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

Đáp án: B

Giải thích: Biến cố và biến cố đối có tổng xác suất 1.

Q15. Does a simulation always equal the true probability exactly? | Tỷ lệ mô phỏng có luôn bằng xác suất thật?

A. No, sampling variation occurs | Không, có dao động do lấy mẫu

B. One | Bằng 1

C. One die's result does not change probabilities of the other | Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

D. `1-P(A)` | `1-P(A)`

Đáp án: A

Giải thích: Mẫu hữu hạn dao động quanh xác suất nền.

Q16. What does independence of two dice mean? | Hai xúc xắc độc lập nghĩa là gì?

A. One die's result does not change probabilities of the other | Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

B. `1-P(A)` | `1-P(A)`

C. No, sampling variation occurs | Không, có dao động do lấy mẫu

D. One | Bằng 1

Đáp án: A

Giải thích: Độc lập nghĩa là một kết quả không đổi xác suất của biến cố còn lại.

Q17. Why is printing a Set directly fragile for sample output? | Vì sao in trực tiếp Set khó đối chiếu đầu ra mẫu?

A. Display order is not promised | Thứ tự hiển thị không được đảm bảo

B. That empirical frequency equals theoretical probability | Tần suất thực nghiệm đúng bằng xác suất lý thuyết

C. It tends to reduce sampling variability | Thường làm giảm dao động lấy mẫu

D. Its faces do not have equal probabilities | Các mặt không đồng xác suất

Đáp án: A

Giải thích: Set không bảo đảm thứ tự in.

Q18. What does a fixed RNG seed not guarantee? | Seed RNG cố định không bảo đảm điều gì?

A. Its faces do not have equal probabilities | Các mặt không đồng xác suất

B. It tends to reduce sampling variability | Thường làm giảm dao động lấy mẫu

C. Display order is not promised | Thứ tự hiển thị không được đảm bảo

D. That empirical frequency equals theoretical probability | Tần suất thực nghiệm đúng bằng xác suất lý thuyết

Đáp án: D

Giải thích: Seed giúp lặp lại mô phỏng nhưng không loại dao động.

Q19. How does larger sample size generally affect estimate variability? | Tăng số mẫu thường ảnh hưởng độ dao động ước lượng thế nào?

A. Display order is not promised | Thứ tự hiển thị không được đảm bảo

B. It tends to reduce sampling variability | Thường làm giảm dao động lấy mẫu

C. Its faces do not have equal probabilities | Các mặt không đồng xác suất

D. That empirical frequency equals theoretical probability | Tần suất thực nghiệm đúng bằng xác suất lý thuyết

Đáp án: B

Giải thích: Mẫu lớn thường ổn định tần suất tương đối hơn.

Q20. Why can't a biased die use simple favorable-count ratios? | Vì sao xúc xắc lệch không dùng phép đếm mặt đơn giản?

A. Its faces do not have equal probabilities | Các mặt không đồng xác suất

B. That empirical frequency equals theoretical probability | Tần suất thực nghiệm đúng bằng xác suất lý thuyết

C. It tends to reduce sampling variability | Thường làm giảm dao động lấy mẫu

D. Display order is not promised | Thứ tự hiển thị không được đảm bảo

Đáp án: A

Giải thích: Xác suất không đều cần cộng theo trọng số.

8. Liên hệ bài tiếp theo

`Logic/Tập hợp → Xác suất → Bayes → Dự đoán bất định.`

PY-35 — Xác suất có điều kiện, Bayes và xác suất toàn phần

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Conditional probability — xác suất có điều kiện; **Independence** — độc lập.

`Biến cố´→ Điều kiện → Bayes → Đánh giá mô hình AI`

Học xong dùng để làm gì? Cập nhật xác suất dựa trên kết quả xét nghiệm hoặc tín hiệu mới, phân biệt xác suất tiên nghiệm/hậu nghiệm.

Kết nối kiến thức tiếp theo: PY-36 mô tả mọi kết quả có thể xảy ra bằng phân phối xác suất.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Conditional probability	xác suất có điều kiện	Khả năng A xảy ra khi biết B đã xảy ra
Independence	độc lập	Hai biến cố không ảnh hưởng xác suất của nhau
Bayes theorem	định lý Bayes	Cập nhật xác suất giả thuyết khi có bằng chứng
Total probability	xác suất toàn phần	Cộng xác suất qua các nhóm loại trừ và đủ

3. Công thức và cách gọi bằng Python

Từ vựng: Event — biến cố; **Prior** — xác suất trước bằng chứng; **Posterior** — xác suất sau bằng chứng.

Công thức / cú pháp	Cách hiểu
`P(A given B) = P(A and B) / P(B)`	Xác suất có điều kiện khi `P(B)>0`
`P(A and B) = P(A given B) * P(B)`	Quy tắc nhân xác suất
`posterior = likelihood * prior / evidence`	Bayes với xác suất bằng chứng khác 0
`sum(group_share * group_rate)`	Xác suất toàn phần qua các nhóm rời và phủ đủ

4. Khái niệm chính và liên quan

Mạch kiến thức: Trước hết xác định nhóm điều kiện B để tính P(A khi B). Khi hai biến cố độc lập, biết B không đổi xác suất của A. Khi có bằng chứng mới, Bayes tính lại xác suất giả thuyết; xác suất toàn phần giúp tính tỷ lệ chung từ các nhóm có tỷ trọng khác nhau.

4.1. Conditional probability — Thu hẹp mẫu nền

Từ vựng: Conditional probability — xác suất có điều kiện; **Conditioning event** — biến cố điều kiện.

Xác suất có điều kiện `P(A given B)` chỉ xét những trường hợp **B đã xảy ra**, không còn xét toàn bộ quần thể ban đầu. Ví dụ trong 100 khách **đã xem quảng cáo** có 20 người mua. Khi đó P(mua **khi đã xem**) = 20/100 = 0.2, không phải tỷ lệ mua của tất cả khách. Công thức `P(A given B) = P(A∩B)/P(B)` chỉ dùng khi `P(B)>0`. Tỷ lệ này **không tự chứng minh** xem quảng cáo đã gây ra hành vi mua.

No Note — Không chú thích

```
seen = 100
bought = 20
print(bought / seen)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính tỷ lệ mua trong tập người đã xem bằng cách lấy số mua chia số đã xem.
# Dữ liệu: 100 người đã xem trang và có 20 người mua trong nhóm đã xem; P(mua|xem)=20/100.
# Mẫu nền gồm 100 lượt ĐÃ XEM, không phải toàn bộ khách hàng trên hệ thống.
seen = 100
# Có 20 lượt mua TRONG nhóm đã xem.
bought = 20
# P(mua|đã xem) bằng số mua trong nhóm đã xem chia số người đã xem; 0.2 = 20%.
print(bought / seen)
```

Kết quả khi chạy:

```
0.2
```

Trong nhóm 100 người đã xem quảng cáo, tỷ lệ mua quan sát được bằng 20%.

4.2. Independence — Không làm đổi xác suất của nhau

Từ vựng: Independence — độc lập; **Joint probability** — xác suất đồng thời.

Hai biến cố A và B **độc lập** nếu biết B xảy ra không đổi khả năng của A: `P(A given B)=P(A)` (khi P(B)>0). Tương đương `P(A∩B)=P(A)×P(B)`. Giả sử P(A)=0.5 và P(B)=0.4 thì nếu độc lập, P(cả A và B)=0.2. Trong dữ liệu thực, việc hai tỷ lệ gần nhau chưa tự chứng minh độc lập; cần biết cách lấy mẫu và đo sai số.

No Note — Không chú thích

```
p_a = 0.5
p_b = 0.4
print(round(p_a * p_b, 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhân xác suất hai sự kiện chỉ khi giả định chúng độc lập.
# Dữ liệu: Hai sự kiện giả định độc lập với P(A)=0.5 và P(B)=0.4.
# Xác suất riêng của sự kiện A là 0.5 (50%).
p_a = 0.5
# Xác suất riêng của sự kiện B là 0.4 (40%).
p_b = 0.4
```

```
# Chỉ nhân P(A)×P(B) để ra P(AnB) khi hai sự kiện ĐỘC LẬP; round(...,3) để in ba chữ số ́lẻ.
print(round(p_a * p_b, 3))
```

Kết quả khi chạy:

```
0.2
```

Nếu hai biến cố độc lập thì giao của chúng có xác suất 0.2.

4.3. Bayes theorem — Cập nhật từ bằng chứng

Từ vựng: Prior — tỷ lệ nền trước bằng chứng; **Likelihood** — khả năng bằng chứng khi giả thuyết đúng; **Posterior** — xác suất sau bằng chứng.

Định lý **Bayes** trả lời "khi đã thấy **tín hiệu dương**, xác suất thực sự có lỗi là bao nhiêu?" Ví dụ chỉ 1% sản phẩm lỗi; máy báo dương với 99% hàng lỗi (**độ nhạy**) và 5% hàng không lỗi (**dương giả**). Trên 10.000 sản phẩm giả định có 100 lỗi: khoảng 99 dương thật, nhưng 495 dương giả từ 9.900 hàng tốt. Tổng dương là 594, nên xác suất lỗi khi máy báo dương là `99/594 ≈ 0.167`, **không phải 99%**. Tỷ lệ nền quyết định nhiều đến kết luận.

No Note — Không chú thích

```
prior = 0.01
sensitivity = 0.99
false_positive = 0.05
positive = sensitivity * prior + false_positive * (1 - prior)
print(round(sensitivity * prior / positive, 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cập nhật xác suất sự kiện hiếm sau kết quả xét nghiệm dương bằng Bayes.
# Dữ liệu: Tỷ lệ nền 1%, xét nghiệm phát hiện 99% ca có bệnh, báo dương giả 5% ở người không bệnh;
dữ liệu chỉ để minh họa.
# prior là xác suất nền (1%) TRƯỚC khi biết kết quả xét nghiệm.
prior = 0.01
# Sensitivity = P(dương|thật sự có), ở đây 0.99.
sensitivity = 0.99
# False-positive rate = P(dương|thật sự không có), ở đây 0.05, khác tỷ lệ dự đoán dương sai tính
theo cảnh báo.
false_positive = 0.05
# Luật xác suất toàn phần: P(dương) gồm dương thật trong nhóm có bệnh và dương giả trong nhóm không
bệnh.
positive = sensitivity * prior + false_positive * (1 - prior)
# Bayes: P(có|dương)=P(dương|có)×P(có)/P(dương), không phải 0.99; round(...,3) chỉ làm tròn khi in.
print(round(sensitivity * prior / positive, 3))
```

Kết quả khi chạy:

```
0.167
```

Sau tín hiệu dương, xác suất lỗi khoảng 16,7% theo các giả định đã nêu.

4.4. Total probability — Cộng qua các nhóm đầy đủ

Từ vựng: Total probability — xác suất toàn phần; **Partition** — phân hoạch.

Nếu quần thể được chia thành các **nhóm rời nhau và phủ đủ**, xác suất chung là trung bình có trọng số của xác suất trong nhóm. Ví dụ 60% khách ở nhóm X, trong đó 80% ủng hộ, còn 40% ở nhóm Y với tỷ lệ ủng hộ 20%. Tỷ lệ chung là `0.6×0.8 + 0.4×0.2 = 0.56`. Không cộng trực tiếp 80% và 20%, vì hai nhóm có quy mô khác nhau. Tổng xác suất các nhóm phải bằng 1.

No Note — Không chú thích

```
shares = [0.6, 0.4]
support = [0.8, 0.2]
total = 0
for share, probability in zip(shares, support):
    total += share * probability
print(round(total, 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Cộng tỷ lệ xảy ra của hai nhóm có trọng số để tính xác suất toàn phần.
# Dữ liệu: Hai nhóm chiếm tỷ trọng 60%/40%, tỷ lệ hỗ trợ 80%/20% tương ứng.
# Hai phần tỷ trọng 0.6 và 0.4 của hai nhóm, tổng phải bằng 1.
shares = [0.6, 0.4]
# Hai xác suất hỗ trợ riêng trong từng nhóm, theo đúng thứ tự shares.
support = [0.8, 0.2]
# Bắt đầu tổng đóng góp xác suất của các nhóm bằng 0.
total = 0
# zip() ghép phần tỷ trọng và xác suất hỗ trợ của CÙNG nhóm theo vị trí.
for share, probability in zip(shares, support):
    # Mỗi lượt lấy tỷ trọng nhóm × xác suất trong nhóm, += cộng lại thành xác suất toàn phần.
    total += share * probability
# round(...,3) làm tròn kết quả cuối chỉ để trình bày.
print(round(total, 3))
```

Kết quả khi chạy:

```
0.56
```

Xác suất chung là 56%, được tính theo tỷ trọng mỗi nhóm.

5. Năm tình huống Xác suất có điều kiện — từ lọc nhóm đến Bayes theo dây chuyền

Tình huống 1 — Cấp 1: Tính tỷ lệ mua có điều kiện từ 10 khách

Từ vựng trong mục: Conditional probability — xác suất có điều kiện; **Condition** — điều kiện.

Tình huống thực tế: Quầy giới thiệu hàng ghi **10 khách K01–K10**, mỗi người có hai trạng thái: **đã xem giới thiệu** sản phẩm hay chưa và **đã mua** hay chưa. Một số người không xem vẫn mua; một số người đã xem nhưng không mua. Nhân viên muốn biết **trong những khách đã xem, bao nhiêu phần đã mua**, chứ không hỏi tỷ lệ mua trong cả 10 khách và cũng chưa kết luận xem giới thiệu là nguyên nhân họ mua.

Vì sao chọn cách này? Xác suất có điều kiện nghĩa là ta chỉ xét cơ hội xảy ra một việc **trong một nhóm đã biết trước điều kiện**. Ở đây điều kiện là `viewed=True`, nên chương trình lọc khách **đã xem** trước, rồi mới đếm `bought=True` **trong nhóm ấy**. Mẫu số là **số khách đã xem**, không phải cả 10 người. Khách chưa xem nhưng vẫn mua không thuộc phép tính này, dù vẫn nằm trong dữ liệu gốc. Đây là **tỷ lệ ghi nhận từ bảng mẫu**, không chứng minh việc xem giới thiệu **gây ra** hành vi mua.

Quy trình:

1. Lưu cờ xem/mua cho cả 10 khách.
2. Chỉ lấy hàng `viewed=True` để xác định mẫu nền.
3. Đếm người mua trong nhóm đã xem và tính tỷ lệ có điều kiện.

No Note — Không chú thích

```
records = [{"id":"K01","viewed":True,"bought":True},{"id":"K02","viewed":True,"bought":False},
{"id":"K03","viewed":True,"bought":True},{"id":"K04","viewed":False,"bought":False},
{"id":"K05","viewed":True,"bought":True},{"id":"K06","viewed":True,"bought":False},
{"id":"K07","viewed":True,"bought":False},{"id":"K08","viewed":False,"bought":True},
{"id":"K09","viewed":True,"bought":False},{"id":"K10","viewed":False,"bought":False}]
viewed = []
for row in records:
    if row["viewed"]:
        viewed.append(row)
bought_after_viewing = 0
for row in viewed:
    if row["bought"]:
        bought_after_viewing += 1
if len(records) != 10 or not viewed:
    raise ValueError("Cần 10 khách và ít nhất một khách đã xem.")
print("Khách:", len(records))
print("Đã xem:", len(viewed))
print("Mua trong nhóm xem:", bought_after_viewing)
print(f"P(mua|đã xem): {bought_after_viewing/len(viewed):.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm P(mua | đã xem) trong 10 hồ sơ thay vì lấy toàn bộ 10 người làm mẫu số.
# Dữ liệu: Mười khách có cờ viewed và bought; có khách mua mà chưa ghi nhận viewed, nên chỉ tính
trong nhóm viewed.
# 10 dictionary khách, mỗi bản ghi có hai cờ viewed/bought; có người mua nhưng không viewed.
records = [{"id":"K01","viewed":True,"bought":True},{"id":"K02","viewed":True,"bought":False},
{"id":"K03","viewed":True,"bought":True},{"id":"K04","viewed":False,"bought":False},
{"id":"K05","viewed":True,"bought":True},{"id":"K06","viewed":True,"bought":False},
{"id":"K07","viewed":True,"bought":False},{"id":"K08","viewed":False,"bought":True},
{"id":"K09","viewed":True,"bought":False},{"id":"K10","viewed":False,"bought":False}]
# Danh sách sẽ chỉ chứa hồ sơ đã xem, là MẪU ĐIỀU KIỆN.
viewed = []
# Duyệt một lượt toàn bộ 10 khách để lọc nhóm xem.
for row in records:
    # Tra cờ đã xem True/False từ dictionary, không dựa vào cờ mua.
    if row["viewed"]:
        # append() giữ toàn bộ dict của khách đã xem để xét mua trong nhóm này.
        viewed.append(row)
# Biến đếm số khách mua TRONG nhóm đã xem.
bought_after_viewing = 0
# Chỉ duyệt các khách thỏa viewed=True, không lấy 10 khách làm mẫu số.
for row in viewed:
    # Đọc cờ mua của khách thuộc nhóm đã xem.
    if row["bought"]:
        # +=1 tăng số khách vừa xem vừa mua.
        bought_after_viewing += 1
# Kiểm đủ 10 khách đầu vào.
# List rỗng được xem là False; not viewed bắt thiếu nhóm điều kiện, tránh chia 0.
if len(records) != 10 or not viewed:
    # ValueError báo thiếu mẫu hợp lệ hoặc nhóm điều kiện rỗng; không nên tiếp tục chia cho số mẫu
sai.
    raise ValueError("Cần 10 khách và ít nhất một khách đã xem.")
print("Khách:", len(records))
print("Đã xem:", len(viewed))
```

```
print("Mua trong nhóm xem:", bought_after_viewing)
# Lấy số´mua trong nhóm đã xem chia CHỈ số´khách đã xem; :.3f hiển thị xác suất ba chữ số´thập
phân.
print(f"P(mua|đã xem): {bought_after_viewing/len(viewed):.3f}")
```

Kết quả khi chạy:

```
Khách: 10
Đã xem: 7
Mua trong nhóm xem: 3
P(mua|đã xem): 0.429
```

Đây là tỷ lệ trên **10 khách mẫu giả lập**, không phải kết quả chứng minh quan hệ nhân quả của quảng cáo.

Tình huống 2 — Cấp 2: Kiểm tra độc lập trong bảng 12 lượt với bốn tổ hợp

Từ vựng trong mục: Independence — độc lập; **Joint probability** — xác suất đồng thời.

Tình huống thực tế: Đội kiểm thử tạo **12 bản ghi đúng/sai giả lập** cho hai hành vi: **A = dùng ứng dụng**, **B = đọc email**. Cả bốn trường hợp — có cả hai, chỉ A, chỉ B, không có cái nào — đều xuất hiện **đúng ba lần**. Nhóm muốn kiểm xem trong bảng cân bằng này, biết người đó có B có làm tỷ lệ A khác đi hay không, và so tỷ lệ có cả hai hành vi với các tỷ lệ riêng.

Vì sao chọn cách này? Độc lập trong xác suất nghĩa là biết biến cố B xảy ra **không làm thay đổi cơ hội xảy ra A** trong mô hình đang xét. Bài cố ý tạo **12 lượt** với đúng **ba lượt ở mỗi trong bốn tổ hợp**, nên có thể đếm riêng số lần A, B và **cả hai**. Chương trình lấy các tỷ lệ đó theo **cùng mẫu số 12**, rồi đối chiếu tỷ lệ cả hai với **tích hai tỷ lệ riêng**; chúng khớp trong bảng đã được thiết kế cân bằng. Điều đó xác minh thuộc tính của **dữ liệu giả lập này**, không chứng minh mọi hành vi dùng app và đọc email của khách hàng thật đều độc lập.

Quy trình:

1. Tạo 12 hàng với bốn tổ hợp lặp đều.
2. Tính P(A), P(B), P(A∩B) từ số dòng.
3. So giao với tích và in kết luận trên bảng này.

No Note — Không chú thích

```
records = [{"a":True,"b":True},{"a":True,"b":False},{"a":False,"b":True},{"a":False,"b":False},
{"a":True,"b":True},{"a":True,"b":False},{"a":False,"b":True},{"a":False,"b":False},
{"a":True,"b":True},{"a":True,"b":False},{"a":False,"b":True},{"a":False,"b":False}]
n = len(records)
a_count = 0
b_count = 0
both_count = 0
for row in records:
    if row["a"]:
        a_count += 1
    if row["b"]:
        b_count += 1
    if row["a"] and row["b"]:
        both_count += 1
p_a = a_count / n
p_b = b_count / n
p_both = both_count / n
if n != 12:
```

```
    raise ValueError("Cần đúng 12 lượt kiểm tra tính độc lập.")
print("Lượt kiểm:", n)
print(f"P(A): {p_a:.2f}")
print(f"P(B): {p_b:.2f}")
print(f"P(cả hai): {p_both:.2f}")
print("Khớp quy tắc độc lập:", abs(p_both-p_a*p_b) < 1e-12)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm tính độc lập giữa A và B bằng cách so P(A∩B) với P(A)×P(B) từ 12 lượt.
# Dữ liệu: 12 kết quả cờ A/B Boolean; mỗi hàng là một phép thử đồng thời hai biến cố.
# 12 lượt cùng hai cờ sự kiện A/B, cho phép đếm giao và từng biến cố.
records = [{"a":True,"b":True},{"a":True,"b":False},{"a":False,"b":True},{"a":False,"b":False},
{"a":True,"b":True},{"a":True,"b":False},{"a":False,"b":True},{"a":False,"b":False},
{"a":True,"b":True},{"a":True,"b":False},{"a":False,"b":True},{"a":False,"b":False}]
# len() đếm số phép thử chung, là mẫu số cho P(A), P(B), P(A∩B).
n = len(records)
# Bắt đầu đếm số lượt biến cố A đúng.
a_count = 0
# Bắt đầu đếm số lượt biến cố B đúng.
b_count = 0
# Bắt đầu đếm số lượt CẢ A và B đúng.
both_count = 0
# Duyệt từng lượt, đọc cả hai cờ cùng một bản ghi.
for row in records:
    # Truy cập cờ sự kiện A của lượt đang xét.
    if row["a"]:
        a_count += 1
    # Truy cập cờ sự kiện B của lượt đang xét.
    if row["b"]:
        b_count += 1
    # Truy cập cờ sự kiện A của lượt đang xét.
    # Truy cập cờ sự kiện B của lượt đang xét.
    # AND chỉ đúng khi A và B cùng xảy ra: biến cố giao A∩B.
    if row["a"] and row["b"]:
        both_count += 1
# Tần suất xuất hiện của A trên 12 lượt, dùng như xác suất thực nghiệm.
p_a = a_count / n
# Tần suất xuất hiện của B trên 12 lượt.
p_b = b_count / n
# Tần suất giao A∩B trên cùng 12 lượt.
p_both = both_count / n
if n != 12:
    # ValueError báo thiếu mẫu hợp lệ hoặc nhóm điều kiện rỗng; không nên tiếp tục chia cho số mẫu
sai.
    raise ValueError("Cần đúng 12 lượt kiểm tra tính độc lập.")
print("Lượt kiểm:", n)
print(f"P(A): {p_a:.2f}")
print(f"P(B): {p_b:.2f}")
print(f"P(cả hai): {p_both:.2f}")
# So P(A∩B) với tích P(A)P(B); abs() lấy độ lệch, 1e-12 là dung sai số thực, chỉ chứng tỏ bằng minh
họa khớp quy tắc, KHÔNG chứng minh độc lập trong quần thể.
print("Khớp quy tắc độc lập:", abs(p_both-p_a*p_b) < 1e-12)
```

Kết quả khi chạy:

```
Lượt kiểm: 12
P(A): 0.50
P(B): 0.50
P(cả hai): 0.25
Khớp quy tắc độc lập: True
```

Kết quả khớp công thức **trên bảng giả lập được xây dựng**; với dữ liệu khảo sát ngẫu nhiên cần kiểm định thống kê và kích thước mẫu phù hợp.

Tình huống 3 — Cấp 3: Áp Bayes cho 15 sản phẩm có nhãn lỗi và cảnh báo

Từ vựng trong mục: Prior — tỷ lệ nền; **Likelihood** — xác suất cảnh báo có điều kiện; **Posterior** — tỷ lệ sau cảnh báo.

Tình huống thực tế: Nhà máy kiểm tra **15 sản phẩm P01–P15** bằng một máy báo lỗi. Mỗi sản phẩm có **tình trạng lỗi thật** do kiểm tra đối chiếu và cờ **máy cảnh báo**. Có **3 sản phẩm lỗi thật**, nhưng máy có thể **bỏ sót lỗi** hoặc **báo nhầm hàng tốt**. Nhân viên muốn biết **nếu máy đã cảnh báo, xác suất sản phẩm thật sự lỗi là bao nhiêu**, rồi đối chiếu tính theo Bayes với phép đếm trực tiếp sản phẩm lỗi trong nhóm được cảnh báo.

Vì sao chọn cách này? Máy **cảnh báo** không đồng nghĩa sản phẩm **thực sự lỗi**, vì có thể báo nhầm hàng tốt; cũng có sản phẩm lỗi mà máy không cảnh báo. **Bayes** giúp đổi câu hỏi từ **'hàng lỗi thì máy báo đúng bao nhiêu?'** sang **'máy báo thì hàng thật sự lỗi bao nhiêu?'**. Đầu tiên tính **tỷ lệ lỗi nền** trong 15 sản phẩm, **độ nhạy** là tỷ lệ máy báo trong hàng lỗi và **tỷ lệ dương giả** là tỷ lệ máy báo trong hàng tốt. Các phần ấy được kết hợp để xét đúng **nhóm có cảnh báo**, rồi đối chiếu với cách đếm trực tiếp **hàng vừa cảnh báo vừa lỗi thật** chia cho **tất cả hàng có cảnh báo**. Mẫu 15 sản phẩm chỉ để học phương pháp; không dùng kết quả này thay độ chính xác của máy trên cả nhà máy.

Quy trình:

1. Tách 15 sản phẩm thành nhóm lỗi và tốt, đồng thời đếm cờ dương.
2. Tính prior, sensitivity, false-positive rate từ bảng.
3. Dùng Bayes để tính posterior, đối chiếu số sản phẩm lỗi trong nhóm dương.

No Note — Không chú thích

```
records = [{"id":"P01","defective":False,"positive":True},
{"id":"P02","defective":True,"positive":True},{"id":"P03","defective":False,"positive":False},
{"id":"P04","defective":False,"positive":True},{"id":"P05","defective":False,"positive":False},
{"id":"P06","defective":False,"positive":False},{"id":"P07","defective":True,"positive":True},
{"id":"P08","defective":False,"positive":False},{"id":"P09","defective":False,"positive":False},
{"id":"P10","defective":False,"positive":True},{"id":"P11","defective":False,"positive":False},
{"id":"P12","defective":True,"positive":False},{"id":"P13","defective":False,"positive":False},
{"id":"P14","defective":False,"positive":False},{"id":"P15","defective":False,"positive":False}]
n = len(records)
defects = []
good = []
positive = []
for row in records:
    if row["defective"]:
        defects.append(row)
    else:
        good.append(row)
    if row["positive"]:
        positive.append(row)
prior = len(defects)/n
true_positive_count = 0
for row in defects:
    if row["positive"]:
        true_positive_count += 1
false_positive_count = 0
for row in good:
    if row["positive"]:
        false_positive_count += 1
```

```
sensitivity = true_positive_count / len(defects)
false_positive = false_positive_count / len(good)
evidence = sensitivity*prior+false_positive*(1-prior)
posterior = sensitivity*prior/evidence
if n != 15 or not positive:
    raise ValueError("Cần 15 sản phẩm và ít nhất một cảnh báo.")
print("Sản phẩm:", n)
print("Lỗi thật:", len(defects))
print("Máy cảnh báo:", len(positive))
print(f"P(lỗi|cảnh báo): {posterior:.3f}")
defects_among_positive = 0
for row in positive:
    if row["defective"]:
        defects_among_positive += 1
print("Khớp kiểm đếm:", abs(posterior - defects_among_positive / len(positive)) < 1e-12)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính P(lỗi | cảnh báo) của 15 sản phẩm bằng Bayes, rồi đối chiếu kiểm đếm trực tiếp.
# Dữ liệu: 15 sản phẩm, hai cờ defective (lỗi thật) và positive (máy cảnh báo), gồm cả cảnh báo
nhầm.
# 15 sản phẩm, defective là lỗi thật, positive là máy cảnh báo; hai cờ có thể không trùng nhau.
records = [{"id":"P01","defective":False,"positive":True},
{"id":"P02","defective":True,"positive":True},{"id":"P03","defective":False,"positive":False},
{"id":"P04","defective":False,"positive":True},{"id":"P05","defective":False,"positive":False},
{"id":"P06","defective":False,"positive":False},{"id":"P07","defective":True,"positive":True},
{"id":"P08","defective":False,"positive":False},{"id":"P09","defective":False,"positive":False},
{"id":"P10","defective":False,"positive":True},{"id":"P11","defective":False,"positive":False},
{"id":"P12","defective":True,"positive":False},{"id":"P13","defective":False,"positive":False},
{"id":"P14","defective":False,"positive":False},{"id":"P15","defective":False,"positive":False}]
n = len(records)
# List chứa sản phẩm lỗi THỰC, là mẫu điều kiện để tính sensitivity.
defects = []
# List chứa sản phẩm không lỗi thực, là mẫu điều kiện tính tỷ lệ dương giả.
good = []
# List chứa mọi sản phẩm bị cảnh báo, gồm cả cảnh báo đúng và sai.
positive = []
# Đọc từng sản phẩm để phân nhóm.
for row in records:
    # True nghĩa sản phẩm có lỗi thật, không phụ thuộc kết luận máy.
    if row["defective"]:
        # append() lưu hàng vào tập lỗi thực.
        defects.append(row)
    else:
        # append() lưu hàng vào tập không lỗi.
        good.append(row)
    # True nghĩa máy cảnh báo dương; có thể là cảnh báo nhầm.
    if row["positive"]:
        # append() lưu tất cả hàng máy cảnh báo.
        positive.append(row)
# prior là tỷ lệ lỗi nền trong 15 sản phẩm, chưa xét cảnh báo.
prior = len(defects)/n
# Đếm dương tính THẬT trong nhóm defective.
true_positive_count = 0
# Chỉ duyệt các sản phẩm đã biết lỗi thật để tính sensitivity.
for row in defects:
    # True nghĩa máy cảnh báo dương; có thể là cảnh báo nhầm.
    if row["positive"]:
        # Tăng số sản phẩm lỗi thật được máy cảnh báo.
        true_positive_count += 1
# Đếm dương tính GIẢ trong nhóm không lỗi.
false_positive_count = 0
# Chỉ duyệt các sản phẩm thật sự tốt để tính false-positive rate.
for row in good:
    # True nghĩa máy cảnh báo dương; có thể là cảnh báo nhầm.
```

```
    if row["positive"]:
        # Tăng số sản phẩm không lỗi nhưng bị cảnh báo.
        false_positive_count += 1
# Sensitivity = P(cảnh báo|lỗi), mẫu số là các sản phẩm LỖI THẬT.
sensitivity = true_positive_count / len(defects)
# False-positive rate = P(cảnh báo|không lỗi), mẫu số là sản phẩm KHÔNG lỗi.
false_positive = false_positive_count / len(good)
# P(cảnh báo) theo xác suất toàn phần từ hai nhóm lỗi/không lỗi.
evidence = sensitivity*prior+false_positive*(1-prior)
# Bayes: P(lỗi|cảnh báo)=P(cảnh báo|lỗi)×P(lỗi)/P(cảnh báo).
posterior = sensitivity*prior/evidence
if n != 15 or not positive:
    # ValueError báo thiếu mẫu hợp lệ hoặc nhóm điều kiện rỗng; không nên tiếp tục chia cho số mẫu
sai.
    raise ValueError("Cần 15 sản phẩm và ít nhất một cảnh báo.")
print("Sản phẩm:", n)
print("Lỗi thật:", len(defects))
print("Máy cảnh báo:", len(positive))
print(f"P(lỗi|cảnh báo): {posterior:.3f}")
# Đếm số sản phẩm lỗi THẬT bên trong nhóm bị cảnh báo để kiểm độc lập kết quả Bayes.
defects_among_positive = 0
# Duyệt chính nhóm máy cảnh báo, không phải toàn bộ sản phẩm.
for row in positive:
    # True nghĩa sản phẩm có lỗi thật, không phụ thuộc kết luận máy.
    if row["defective"]:
        # Tăng số cảnh báo trúng lỗi thật.
        defects_among_positive += 1
# Tỷ lệ lỗi trực tiếp trong nhóm bị cảnh báo, phải gần posterior Bayes.
# abs() so Bayes với phép đếm trực tiếp, 1e-12 cho dung sai số thực.
print("Khớp kiểm đếm:", abs(posterior - defects_among_positive / len(positive)) < 1e-12)
```

Kết quả khi chạy:

```
Sản phẩm: 15
Lỗi thật: 3
Máy cảnh báo: 5
P(lỗi|cảnh báo): 0.400
Khớp kiểm đếm: True
```

Không được suy ra độ tin cậy của một máy kiểm hàng thật chỉ từ **15 sản phẩm ví dụ**; tỷ lệ nền khi triển khai thực tế có thể rất khác.

Tình huống 4 — Cấp 4: Xác suất toàn phần từ 18 khách ở ba nhóm có quy mô khác nhau

Từ vựng trong mục: Total probability — xác suất toàn phần; **Partition** — phân hoạch; **Weighted rate** — tỷ lệ có trọng số.

Tình huống thực tế: Khảo sát ghi **18 khách C01–C18**, chia vào đúng một trong ba nhóm địa bàn **A (9 người), B (6 người), C (3 người)**. Mỗi người có trạng thái **đã mua/chưa mua**. Quản lý muốn tính **tỷ lệ mua chung của 18 khách** bằng cách tổng hợp tỷ lệ từng nhóm, sau đó đối chiếu với việc đếm trực tiếp số người đã mua; không được lấy trung bình đều ba nhóm vì số người rất khác nhau.

Vì sao chọn cách này? Xác suất toàn phần giúp gộp các nhóm **không trùng nhau và bao phủ đủ dữ liệu** để biết tỷ lệ chung. Trong ví dụ, A có 9 khách, B có 6 và C có 3, nên mỗi nhóm phải đóng góp theo **số người thực sự thuộc nhóm đó**, không phải mỗi nhóm được tính ngang một phần ba. Chương trình đếm số người mua ở mỗi nhóm, tính tỷ lệ mua của nhóm, sau đó lấy **tỷ trọng nhóm trong 18 khách** để tổng

hợp. Kết quả được đối chiếu với số **người mua thật trong cả bảng chia 18**. Lấy trung bình đều ba tỷ lệ sẽ dễ làm nhóm C chỉ có ba người ảnh hưởng mạnh ngang nhóm A chín người.

Quy trình:

1. Đếm kích thước và số mua của từng nhóm từ 18 hàng.
2. Tính tỷ trọng nhóm và tỷ lệ mua có điều kiện trong từng nhóm.
3. Cộng tỷ lệ có trọng số; đối chiếu tỷ lệ mua trực tiếp trên 18 khách.

No Note — Không chú thích

```
records = [{"id":"C01","area":"A","bought":True},{"id":"C02","area":"A","bought":True},
{"id":"C03","area":"A","bought":True},{"id":"C04","area":"A","bought":True},
{"id":"C05","area":"A","bought":True},{"id":"C06","area":"A","bought":True},
{"id":"C07","area":"A","bought":False},{"id":"C08","area":"A","bought":False},
{"id":"C09","area":"A","bought":False},{"id":"C10","area":"B","bought":True},
{"id":"C11","area":"B","bought":True},{"id":"C12","area":"B","bought":False},
{"id":"C13","area":"B","bought":False},{"id":"C14","area":"B","bought":False},
{"id":"C15","area":"B","bought":False},{"id":"C16","area":"C","bought":True},
{"id":"C17","area":"C","bought":False},{"id":"C18","area":"C","bought":False}]
n = len(records)
areas = ("A", "B", "C")
overall = 0.0
for area in areas:
    subset = []
    bought_count = 0
    for row in records:
        if row["area"] == area:
            subset.append(row)
            if row["bought"]:
                bought_count += 1
    overall += (len(subset) / n) * (bought_count / len(subset))
all_bought_count = 0
for row in records:
    if row["bought"]:
        all_bought_count += 1
direct = all_bought_count / n
unique_ids = set()
for row in records:
    unique_ids.add(row["id"])
if len(records) != 18 or len(unique_ids) != 18:
    raise ValueError("Cần đủ 18 khách với mã duy nhất.")
print("Khách:", n)
group_counts = []
for group in areas:
    count = 0
    for row in records:
        if row["area"] == group:
            count += 1
    group_counts.append(count)
print("Nhóm A/B/C:", *group_counts)
print(f"Tỷ lệ mua toàn phần: {overall:.3f}")
print("Khớp đếm trực tiếp:", abs(overall-direct) < 1e-12)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính xác suất mua toàn phần trên 18 khách bằng cách cộng tỷ lệ có trọng số theo vùng.
# Dữ liệu: 18 khách chia các vùng A,B,C, mỗi vùng có tỷ lệ mua và quy mô khác nhau.
# 18 khách có id, area A/B/C và bought True/False; quy mô từng vùng khác nhau.
records = [{"id":"C01","area":"A","bought":True},{"id":"C02","area":"A","bought":True},
{"id":"C03","area":"A","bought":True},{"id":"C04","area":"A","bought":True},
{"id":"C05","area":"A","bought":True},{"id":"C06","area":"A","bought":True},
```

```
{"id":"C07","area":"A","bought":False},{"id":"C08","area":"A","bought":False},
{"id":"C09","area":"A","bought":False},{"id":"C10","area":"B","bought":True},
{"id":"C11","area":"B","bought":True},{"id":"C12","area":"B","bought":False},
{"id":"C13","area":"B","bought":False},{"id":"C14","area":"B","bought":False},
{"id":"C15","area":"B","bought":False},{"id":"C16","area":"C","bought":True},
{"id":"C17","area":"C","bought":False},{"id":"C18","area":"C","bought":False}]
# Tổng 18 khách là mẫu số của tỷ trọng vùng.
n = len(records)
# Tuple tên ba vùng dùng để duyệt đủ các nhóm loại trừ nhau.
areas = ("A", "B", "C")
# Biến tích lũy xác suất mua toàn phần, bắt đầu từ 0.
overall = 0.0
# Duyệt từng vùng A,B,C, tính tỷ lệ mua riêng trước khi gộp.
for area in areas:
    # Mẫu nền chỉ gồm những khách của vùng đang xét.
    subset = []
    # Đếm số khách mua trong vùng hiện tại.
    bought_count = 0
    for row in records:
        # So tên vùng trong từng dict, chọn đúng hồ sơ thuộc vùng.
        if row["area"] == area:
            # append() thêm khách vùng hiện tại, dùng len(subset) làm mẫu số cho tỷ lệ mua vùng.
            subset.append(row)
            # Cờ Boolean cho biết khách có mua hay không.
            if row["bought"]:
                # Đếm người mua trong nhóm vùng tương ứng.
                bought_count += 1
    # Xác suất toàn phần: tỷ trọng vùng × P(mua|vùng), cộng cả ba vùng, KHÔNG lấy trung bình các tỷ
lệ vùng ngang nhau.
    overall += (len(subset) / n) * (bought_count / len(subset))
# Đếm số khách mua trong vùng hiện tại.
# Đếm trực tiếp tất cả người mua để đối chiếu cách tính phân nhóm.
all_bought_count = 0
for row in records:
    # Cờ Boolean cho biết khách có mua hay không.
    if row["bought"]:
        # Đếm người mua trong nhóm vùng tương ứng.
        # Tăng số mua trên toàn bộ 18 hồ sơ.
        all_bought_count += 1
# direct là tỷ lệ mua từ toàn bộ khách, cần khớp overall đã gộp vùng.
direct = all_bought_count / n
# Set lưu ID duy nhất để phát hiện mã khách bị lặp.
unique_ids = set()
for row in records:
    # add() thêm ID, không nhân đôi khi ID đã tồn tại.
    unique_ids.add(row["id"])
# Kiểm 18 mã khách riêng biệt, không chỉ 18 hàng.
if len(records) != 18 or len(unique_ids) != 18:
    # ValueError báo thiếu mẫu hợp lệ hoặc nhóm điều kiện rỗng; không nên tiếp tục chia cho số mẫu
sai.
    raise ValueError("Cần đủ 18 khách với mã duy nhất.")
print("Khách:", n)
# List chứa số lượng khách từng nhóm, đúng thứ tự areas.
group_counts = []
# Duyệt các vùng để đếm mỗi nhóm.
for group in areas:
    count = 0
    for row in records:
        if row["area"] == group:
            count += 1
    # append() giữ số khách của vùng tương ứng.
    group_counts.append(count)
# Dấu * giải nén các số đếm thành ba đối số print riêng.
print("Nhóm A/B/C:", *group_counts)
print(f"Tỷ lệ mua toàn phần: {overall:.3f}")
```

```
# abs() kiểm chênh xác suất hai cách tính gần 0, tránh sai số dấu phẩy động.
print("Khớp đếm trực tiếp:", abs(overall-direct) < 1e-12)
```

Kết quả khi chạy:

```
Khách: 18
Nhóm A/B/C: 9 6 3
Tỷ lệ mua toàn phần: 0.500
Khớp đếm trực tiếp: True
```

Ví dụ này minh họa **trung bình có trọng số**; nếu chỉ cộng ba tỷ lệ chia 3, mỗi nhóm bị xem như có cùng quy mô, dễ dẫn đến kết luận sai.

Tình huống 5 — Cấp 5: Bayes hai dây chuyền: phân biệt tỷ lệ nền với độ chính xác cảnh báo

Từ vựng trong mục: Posterior — tỷ lệ lỗi sau tín hiệu; **Evidence** — xác suất tín hiệu; **Base-rate effect** — ảnh hưởng tỷ lệ nền.

Tình huống thực tế: Nhà máy thử **20 sản phẩm M01–M20**, gồm **12 từ dây chuyền A** và **8 từ dây chuyền B**. Mỗi sản phẩm có kết luận **lỗi thật** và dấu **máy cảnh báo dương**, nhưng cảnh báo có thể đúng hoặc sai. Nhóm chất lượng cần tính xác suất **lỗi thật khi đã có cảnh báo trên toàn nhà máy**, rồi tính riêng **trong nhóm cảnh báo ở dây chuyền B**, để xem vì sao thay đổi nhóm đang xét có thể làm kết quả khác nhau.

Vì sao chọn cách này? Xác suất hậu nghiệm (posterior) là cơ hội sản phẩm lỗi **sau khi đã biết máy báo cảnh báo**. Với toàn bộ 20 sản phẩm, Bayes dùng **tỷ lệ lỗi nền**, **tỷ lệ cảnh báo trên hàng lỗi** và **tỷ lệ cảnh báo nhầm trên hàng tốt** của **toàn nhà máy**, rồi đối chiếu với **số hàng lỗi thật trong nhóm cảnh báo**. Khi đổi sang **chỉ dây chuyền B**, mẫu số trở thành **những sản phẩm B có cảnh báo**, không còn là tất cả cảnh báo từ cả A và B. Vì B có tỷ lệ lỗi/cảnh báo khác, tỷ lệ lỗi sau cảnh báo có thể khác kết quả chung; đây không phải mâu thuẫn. Đừng lấy **độ nhạy** (cơ hội báo khi đã lỗi) thay cho **posterior** (cơ hội lỗi khi đã báo), và đừng dùng tỷ lệ của nhóm toàn nhà máy để phát biểu về B.

Quy trình:

1. Lưu 20 hàng đầy đủ mã, dây chuyền, cờ lỗi và cờ cảnh báo.
2. Tính prior, sensitivity, false positive và evidence trên cả 20 hàng; dùng Bayes.
3. Tính tỷ lệ lỗi trong nhóm B bị cảnh báo, báo hai kết quả và kiểm bằng đếm trực tiếp.

No Note — Không chú thích

```
records = [{"id":"M01","line":"A","defective":False,"positive":True},
{"id":"M02","line":"A","defective":True,"positive":True},
{"id":"M03","line":"A","defective":False,"positive":False},
{"id":"M04","line":"A","defective":False,"positive":False},
{"id":"M05","line":"A","defective":False,"positive":False},
{"id":"M06","line":"A","defective":False,"positive":True},
{"id":"M07","line":"A","defective":False,"positive":False},
{"id":"M08","line":"A","defective":False,"positive":False},
{"id":"M09","line":"A","defective":True,"positive":True},
{"id":"M10","line":"A","defective":False,"positive":False},
{"id":"M11","line":"A","defective":False,"positive":False},
{"id":"M12","line":"A","defective":False,"positive":False},
{"id":"M13","line":"B","defective":True,"positive":True},
{"id":"M14","line":"B","defective":True,"positive":True},
{"id":"M15","line":"B","defective":False,"positive":False},
```

```
{"id":"M16","line":"B","defective":False,"positive":False},
{"id":"M17","line":"B","defective":True,"positive":False},
{"id":"M18","line":"B","defective":False,"positive":True},
{"id":"M19","line":"B","defective":False,"positive":False},
{"id":"M20","line":"B","defective":False,"positive":False}]
n = len(records)
defects = []
good = []
positive = []
for row in records:
    if row["defective"]:
        defects.append(row)
    else:
        good.append(row)
    if row["positive"]:
        positive.append(row)
prior = len(defects)/n
true_positive_count = 0
for row in defects:
    if row["positive"]:
        true_positive_count += 1
false_positive_count = 0
for row in good:
    if row["positive"]:
        false_positive_count += 1
sensitivity = true_positive_count / len(defects)
false_positive = false_positive_count / len(good)
p_positive = sensitivity*prior + false_positive*(1-prior)
posterior = sensitivity*prior/p_positive
positive_b = []
for row in records:
    if row["line"] == "B" and row["positive"]:
        positive_b.append(row)
positive_b_defects = 0
for row in positive_b:
    if row["defective"]:
        positive_b_defects += 1
b_rate = positive_b_defects / len(positive_b)
if len(records) != 20 or not positive_b:
    raise ValueError("Cần đủ 20 sản phẩm và cảnh báo ở dây chuyền B.")
print("Sản phẩm:", n)
print("Lỗi thật:", len(defects))
print("Cảnh báo:", len(positive))
print(f"P(lỗi|cảnh báo) toàn nhà máy: {posterior:.3f}")
print(f"P(lỗi|cảnh báo, B): {b_rate:.3f}")
defects_among_positive = 0
for row in positive:
    if row["defective"]:
        defects_among_positive += 1
print("Bayes khớp đếm:", abs(posterior - defects_among_positive / len(positive)) < 1e-12)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính Bayes trên 20 sản phẩm và so xác suất lỗi khi cảnh báo trên toàn nhà máy với riêng dây chuyền B.
# Dữ liệu: 20 sản phẩm từ hai dây chuyền A/B; lỗi thật và cảnh báo không đồng nhất, cần điều kiện hóa theo nhóm.
# 20 sản phẩm từ line A/B; defective là lỗi thật, positive là máy báo động, hai cờ độc lập với nhãn line.
records = [{"id":"M01","line":"A","defective":False,"positive":True},
{"id":"M02","line":"A","defective":True,"positive":True},
{"id":"M03","line":"A","defective":False,"positive":False},
{"id":"M04","line":"A","defective":False,"positive":False},
{"id":"M05","line":"A","defective":False,"positive":False},
{"id":"M06","line":"A","defective":False,"positive":True},
```

```
{"id":"M07","line":"A","defective":False,"positive":False},
{"id":"M08","line":"A","defective":False,"positive":False},
{"id":"M09","line":"A","defective":True,"positive":True},
{"id":"M10","line":"A","defective":False,"positive":False},
{"id":"M11","line":"A","defective":False,"positive":False},
{"id":"M12","line":"A","defective":False,"positive":False},
{"id":"M13","line":"B","defective":True,"positive":True},
{"id":"M14","line":"B","defective":True,"positive":True},
{"id":"M15","line":"B","defective":False,"positive":False},
{"id":"M16","line":"B","defective":False,"positive":False},
{"id":"M17","line":"B","defective":True,"positive":False},
{"id":"M18","line":"B","defective":False,"positive":True},
{"id":"M19","line":"B","defective":False,"positive":False},
{"id":"M20","line":"B","defective":False,"positive":False}]
# Mẫu số của prior toàn nhà máy là 20 sản phẩm.
n = len(records)
# Nhóm sản phẩm lỗi thật của cả hai dây chuyền.
defects = []
# Nhóm sản phẩm không lỗi thật.
good = []
# Nhóm sản phẩm bị máy cảnh báo bất kể lỗi thật.
positive = []
# Duyệt từng sản phẩm của cả hai dây chuyền.
for row in records:
    # Cờ lỗi thật; khác cờ positive của máy.
    if row["defective"]:
        # Gom sản phẩm lỗi thật.
        defects.append(row)
    else:
        # Gom sản phẩm tốt.
        good.append(row)
    # Cờ máy cảnh báo, có thể đúng hoặc sai.
    if row["positive"]:
        # Gom mọi cảnh báo để kiểm Bayes.
        positive.append(row)
# Prior lỗi nền dựa trên tất cả 20 sản phẩm, chưa xét cảnh báo/line.
prior = len(defects)/n
# Đếm sản phẩm lỗi thật có máy cảnh báo.
true_positive_count = 0
# Chỉ đọc nhóm lỗi thật khi ước lượng sensitivity.
for row in defects:
    # Cờ máy cảnh báo, có thể đúng hoặc sai.
    if row["positive"]:
        # Tăng số cảnh báo đúng trong nhóm lỗi thật.
        true_positive_count += 1
# Đếm cảnh báo sai trong nhóm không lỗi.
false_positive_count = 0
# Duyệt nhóm không lỗi để tính false-positive rate.
for row in good:
    # Cờ máy cảnh báo, có thể đúng hoặc sai.
    if row["positive"]:
        # Tăng số sản phẩm tốt nhưng bị cảnh báo.
        false_positive_count += 1
# Sensitivity P(cảnh báo|lỗi), mẫu số các hàng defective True.
sensitivity = true_positive_count / len(defects)
# False-positive rate P(cảnh báo|không lỗi), mẫu số các hàng defective False.
false_positive = false_positive_count / len(good)
# P(cảnh báo) tính theo xác suất toàn phần trong cả hai nhóm lỗi/không lỗi.
p_positive = sensitivity*prior + false_positive*(1-prior)
# Bayes toàn nhà máy: P(lỗi|cảnh báo), chưa điều kiện hóa riêng theo dây chuyền.
posterior = sensitivity*prior/p_positive
# List chứa sản phẩm của CHỈ dây chuyền B và đang có cảnh báo.
positive_b = []
# Duyệt từng sản phẩm của cả hai dây chuyền.
for row in records:
    # Cờ máy cảnh báo, có thể đúng hoặc sai.
```

```
    # and yêu câù đồng thời line B và positive True; mâũ điêù kiện riêng khác toàn nhà máy.
    if row["line"] == "B" and row["positive"]:
        # append() lưu cảnh báo trong dây chuyêǹ B.
        positive_b.append(row)
# Đêḿ số cảnh báo B có lôĩ thật.
positive_b_defects = 0
# Chỉ xét những sản phâm̉ line B có positive True.
# Duyệt nhóm cảnh báo toàn nhà máy.
for row in positive_b:
    # Cờ lôĩ thật; khác cờ positive của máy.
    if row["defective"]:
        # Tăng số sản phâm̉ lôĩ thật trong nhóm cảnh báo B.
        positive_b_defects += 1
# P(lôĩ|cảnh báo, B): lấý số lôĩ thật TRONG cảnh báo B chia số cảnh báo B.
b_rate = positive_b_defects / len(positive_b)
# Nêú nhóm cảnh báo B rôńg, không thêt̉ính tỷ lệ có điêù kiện và phải báo lôĩ.
if len(records) != 20 or not positive_b:
    # ValueError báo thiếú mâũ hợp lệ hoặc nhóm điêù kiện rôńg; không nên tiếp tục chia cho số mâũ
sai.
    raise ValueError("Cân đủ 20 sản phâm̉ và cảnh báo ở dây chuyêǹ B.")
print("Sản phâm̉:", n)
print("Lôĩ thật:", len(defects))
print("Cảnh báo:", len(positive))
print(f"P(lôĩ|cảnh báo) toàn nhà máy: {posterior:.3f}")
print(f"P(lôĩ|cảnh báo, B): {b_rate:.3f}")
# Đêḿ lôĩ thật trong tât́ cả cảnh báo, dùng kiêḿ kêt́ quả Bayes toàn nhà máy.
defects_among_positive = 0
# Duyệt nhóm cảnh báo toàn nhà máy.
for row in positive:
    # Cờ lôĩ thật; khác cờ positive của máy.
    if row["defective"]:
        # Tăng số lôĩ thật có máy cảnh báo.
        defects_among_positive += 1
# Phép đêḿ trực tiêṕ tỷ lệ lôĩ trên nhóm cảnh báo toàn nhà máy, đôí chiêú posterior.
# So Bayes với tần suât́ đêḿ trực tiêṕ, dung sai 1e-12 giúp tránh sai số float.
print("Bayes khớp đêḿ:", abs(posterior - defects_among_positive / len(positive)) < 1e-12)
```

Kết quả khi chạy:

```
Sản phâm̉: 20
Lôĩ thật: 5
Cảnh báo: 7
P(lôĩ|cảnh báo) toàn nhà máy: 0.571
P(lôĩ|cảnh báo, B): 0.667
Bayes khớp đêḿ: True
```

Đây là tỷ lệ kinh nghiệm trên **20 sản phẩm giả lập**, không đủ cỡ mẫu để đánh giá máy trong sản xuất thật. Tập điều kiện thay đổi thì **posterior có thể thay đổi**.

6. Sai lầm thường gặp

- P(A khi B) và P(B khi A) có **hai mẫu nền khác nhau**, không được tráo hai xác suất.
- Độ nhạy của máy kiểm không phải xác suất sản phẩm thực sự lỗi sau cảnh báo; phải tính cả tỷ lệ nền và dương giả.
- Xác suất có điều kiện cần mẫu số P(B) lớn hơn 0.
- Các nhóm trong xác suất toàn phần phải rời nhau và phủ toàn bộ quần thể.
- Quan sát khách mua sau khi thấy quảng cáo không tự chứng minh quảng cáo gây ra mua hàng.
- Kiểm tra độc lập trên dữ liệu khảo sát cần xét sai số và cách lấy mẫu, không chỉ so hai số xấp xỉ.

7. Trắc nghiệm

Q1. What is conditional probability? | Xác suất có điều kiện là gì?

A. One event does not change the probability of the other | Biến cố này không đổi xác suất của biến cố kia

B. A posterior from evidence, likelihood and prior | Xác suất sau bằng chứng từ likelihood và prior

C. Probability of A given B has occurred | Xác suất A khi biết B đã xảy ra

D. Overall chance weighted over a partition | Xác suất chung theo trọng số một phân hoạch

Đáp án: C

Giải thích: Nó thu hẹp mẫu nền về nhóm B.

Q2. What is independence of events? | Độc lập giữa hai biến cố là gì?

A. One event does not change the probability of the other | Biến cố này không đổi xác suất của biến cố kia

B. A posterior from evidence, likelihood and prior | Xác suất sau bằng chứng từ likelihood và prior

C. Overall chance weighted over a partition | Xác suất chung theo trọng số một phân hoạch

D. Probability of A given B has occurred | Xác suất A khi biết B đã xảy ra

Đáp án: A

Giải thích: Độc lập nghĩa là biến cố này không đổi xác suất biến cố kia.

Q3. What does Bayes' theorem compute? | Định lý Bayes tính gì?

A. A posterior from evidence, likelihood and prior | Xác suất sau bằng chứng từ likelihood và prior

B. Probability of A given B has occurred | Xác suất A khi biết B đã xảy ra

C. Overall chance weighted over a partition | Xác suất chung theo trọng số một phân hoạch

D. One event does not change the probability of the other | Biến cố này không đổi xác suất của biến cố kia

Đáp án: A

Giải thích: Bayes cập nhật tỷ lệ nền từ bằng chứng.

Q4. What does total probability calculate? | Xác suất toàn phần tính gì?

A. A posterior from evidence, likelihood and prior | Xác suất sau bằng chứng từ likelihood và prior

B. One event does not change the probability of the other | Biến cố này không đổi xác suất của biến cố kia

C. Overall chance weighted over a partition | Xác suất chung theo trọng số một phân hoạch

D. Probability of A given B has occurred | Xác suất A khi biết B đã xảy ra

Đáp án: C

Giải thích: Xác suất toàn phần dùng tỷ trọng của từng nhóm.

Q5. Independent events A and B have P(A) = 0.3, P(B) = 0.6. What is P(A and B)? | Hai biến cố độc lập A và B có P(A) = 0.3, P(B) = 0.6. P(A giao B) bằng bao nhiêu?

A. 0.3 | 0.3

B. 0.9 | 0.9

C. 0.18 | 0.18

D. 0.6 | 0.6

Đáp án: C

Giải thích: Độc lập nghĩa là P(A giao B) = P(A) × P(B) = 0.3 × 0.6 = 0.18.

Q6. If P(A and B) = 0.12 and P(B) = 0.30, what is P(A|B)? | Nếu P(A giao B) = 0.12 và P(B) = 0.30, P(A|B) bằng bao nhiêu?

A. 0.4 | 0.4

B. 0.12 | 0.12

C. 2.5 | 2.5

D. 0.3 | 0.3

Đáp án: A

Giải thích: Xác suất có điều kiện P(A|B) = P(A giao B)/P(B) = 0.12/0.30 = 0.4.

Q7. Which statement about conditional probability is NOT always correct? | Phát biểu nào về xác suất có điều kiện KHÔNG luôn đúng?

A. P(A|B) uses B as its reference group | P(A|B) dùng B làm nhóm xét

B. P(A|B) needs P(B) > 0 | P(A|B) cần P(B) > 0

C. Independence implies P(A|B) = P(A) when P(B)>0 | Độc lập thì P(A|B) = P(A) nếu P(B)>0

D. P(A|B) always equals P(B|A) | P(A|B) luôn bằng P(B|A)

Đáp án: D

Giải thích: Đảo A và B làm đổi nhóm điều kiện và mẫu số; hai xác suất có điều kiện nói chung khác nhau.

Q8. A disease affects 10% of people. A test detects 90% of sick people and flags 10% of healthy people. What is P(sick | positive)? | Tỷ lệ mắc bệnh là 10%. Xét nghiệm báo dương ở 90% người bệnh và 10% người khỏe. Xác suất mắc bệnh khi đã có kết quả dương là bao nhiêu?

A. 0.09 | 0.09

B. 0.5 | 0.5

C. 0.1 | 0.1

D. 0.9 | 0.9

Đáp án: B

Giải thích: Trong 100 người: khoảng 10 người bệnh tạo 9 dương thật; 90 người khỏe tạo 9 dương giả. Vậy P(bệnh|dương) = 9/(9+9) = 0.5.

Q9. What does prior mean? | Prior có nghĩa gì?

A. Probability of positive signal when no defect exists | Xác suất báo dương khi thực tế không có lỗi

B. Probability before current evidence | Xác suất trước khi có bằng chứng hiện tại

C. Probability of evidence under a hypothesis | Xác suất bằng chứng khi giả thuyết đúng

D. Updated hypothesis probability after evidence | Xác suất giả thuyết được cập nhật sau bằng chứng

Đáp án: B

Giải thích: Prior là xác suất nền.

Q10. What does likelihood mean? | Likelihood biểu thị gì?

A. Updated hypothesis probability after evidence | Xác suất giả thuyết được cập nhật sau bằng chứng

B. Probability before current evidence | Xác suất trước khi có bằng chứng hiện tại

C. Probability of evidence under a hypothesis | Xác suất bằng chứng khi giả thuyết đúng

D. Probability of positive signal when no defect exists | Xác suất báo dương khi thực tế không có lỗi

Đáp án: C

Giải thích: Likelihood xét bằng chứng khi giả thuyết đúng.

Q11. What does posterior mean? | Posterior là gì?

A. Probability of evidence under a hypothesis | Xác suất bằng chứng khi giả thuyết đúng

B. Probability before current evidence | Xác suất trước khi có bằng chứng hiện tại

C. Probability of positive signal when no defect exists | Xác suất báo dương khi thực tế không có lỗi

D. Updated hypothesis probability after evidence | Xác suất giả thuyết được cập nhật sau bằng chứng

Đáp án: D

Giải thích: Posterior đã tính đến bằng chứng quan sát.

Q12. What is a false-positive rate? | Tỷ lệ dương giả là gì?

A. Probability of evidence under a hypothesis | Xác suất bằng chứng khi giả thuyết đúng

B. Updated hypothesis probability after evidence | Xác suất giả thuyết được cập nhật sau bằng chứng

C. Probability of positive signal when no defect exists | Xác suất báo dương khi thực tế không có lỗi

D. Probability before current evidence | Xác suất trước khi có bằng chứng hiện tại

Đáp án: C

Giải thích: Dương giả là cảnh báo dương ở trường hợp không lỗi.

Q13. What is P(A given B) if 20 of 100 B cases have A? | P(A khi B) là gì nếu 20 trên 100 ca B có A?

A. 0.2 | 0.2

B. 0.56 | 0.56

C. 0.3 | 0.3

D. No; base rate and false positives also matter | Không; còn tỷ lệ nền và dương giả

Đáp án: A

Giải thích: 20 chia 100 bằng 0.2.

Q14. What is the total with group shares 0.6,0.4 and rates 0.8,0.2? | Tỷ lệ chung từ nhóm 0.6/0.4 với xác suất 0.8/0.2 là gì?

A. 0.2 | 0.2

B. 0.3 | 0.3

C. 0.56 | 0.56

D. No; base rate and false positives also matter | Không; còn tỷ lệ nền và dương giả

Đáp án: C

Giải thích: 0.6×0.8+0.4×0.2 bằng 0.56.

Q15. If P(A)=0.5 and P(B)=0.6 and they are independent, what is P(A and B)? | Nếu A, B độc lập có xác suất 0.5 và 0.6 thì P(A và B) là gì?

A. 0.56 | 0.56

B. 0.2 | 0.2

C. 0.3 | 0.3

D. No; base rate and false positives also matter | Không; còn tỷ lệ nền và dương giả

Đáp án: C

Giải thích: Với biến cố độc lập, 0.5×0.6=0.3.

Q16. If an alert has 99% sensitivity, does that mean 99% of alerts are true defects? | Độ nhạy 99% có nghĩa 99% cảnh báo là lỗi thật không?

A. 0.3 | 0.3

B. 0.56 | 0.56

C. 0.2 | 0.2

D. No; base rate and false positives also matter | Không; còn tỷ lệ nền và dương giả

Đáp án: D

Giải thích: Độ nhạy và posterior xét hai nhóm mẫu nền khác nhau.

Q17. Why does Bayes need the base rate? | Vì sao Bayes cần tỷ lệ nền?

A. They condition on different reference groups | Chúng xét trên hai nhóm điều kiện khác nhau

B. That viewing ads caused purchases | Việc xem quảng cáo gây ra hành vi mua

C. The formula divides by P(B) | Vì công thức chia cho P(B)

D. Rare cases can yield many false alarms among positives | Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

Đáp án: D

Giải thích: Sự kiện hiếm có thể khiến phần lớn cảnh báo là dương giả.

Q18. Why must P(B)>0 for P(A given B)? | Vì sao cần P(B)>0 khi tính P(A khi B)?

A. They condition on different reference groups | Chúng xét trên hai nhóm điều kiện khác nhau

B. Rare cases can yield many false alarms among positives | Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

C. The formula divides by P(B) | Vì công thức chia cho P(B)

D. That viewing ads caused purchases | Việc xem quảng cáo gây ra hành vi mua

Đáp án: C

Giải thích: Chia cho 0 không xác định.

Q19. What does an observational conditional purchase rate not prove? | Tỷ lệ mua có điều kiện trong quan sát không chứng minh gì?

A. They condition on different reference groups | Chúng xét trên hai nhóm điều kiện khác nhau

B. That viewing ads caused purchases | Việc xem quảng cáo gây ra hành vi mua

C. Rare cases can yield many false alarms among positives | Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

D. The formula divides by P(B) | Vì công thức chia cho P(B)

Đáp án: B

Giải thích: Liên hệ quan sát không tự chứng minh nhân quả.

Q20. What distinguishes P(A given B) from P(B given A)? | P(A khi B) khác P(B khi A) ở đâu?

A. The formula divides by P(B) | Vì công thức chia cho P(B)

B. Rare cases can yield many false alarms among positives | Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

C. They condition on different reference groups | Chúng xét trên hai nhóm điều kiện khác nhau

D. That viewing ads caused purchases | Việc xem quảng cáo gây ra hành vi mua

Đáp án: C

Giải thích: Đảo điều kiện làm thay đổi mẫu số.

8. Liên hệ bài tiếp theo

`Biến cố → Điều kiện → Bayes → Đánh giá mô hình AI.`

PY-36 — Phân phối nhị thức và phân phối chuẩn

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Random variable — biến ngẫu nhiên; **Binomial** — phân phối nhị thức.

`Xác suất → Biến ngẫu nhiên → Phân phối → Thống kê`

Học xong dùng để làm gì? Mô tả xác suất của các mức số đếm hay kết quả đo và xem độ tập trung/phân tán của biến ngẫu nhiên.

Kết nối kiến thức tiếp theo: PY-37 tính trung bình và độ lệch chuẩn trực tiếp từ dữ liệu quan sát.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Random variable	biến ngẫu nhiên	Đại lượng có kết quả tùy phép thử
Binomial	phân phối nhị thức	Số thành công của n thử Bernoulli độc lập
Normal	phân phối chuẩn	Phân phối liên tục hình chuông với mean/std
PMF/PDF	hàm khối/mật độ	PMF cho biến rời rạc, PDF là mật độ cho biến liên tục

3. Các hàm Python cần nhớ

Hàm	Ý nghĩa
`binom.pmf(k, n, p)`	Xác suất **đúng** k thành công
`binom.cdf(k, n, p)`	Xác suất **không quá** k thành công
`norm.pdf(x, loc, scale)`	Mật độ chuẩn tại x, không phải xác suất tại điểm
`norm.cdf(x, loc, scale)`	Xác suất $X \leq x$
`rng.normal(loc, scale, size)`	Lấy số mẫu giả ngẫu nhiên theo phân phối chuẩn

4. Khái niệm chính và liên quan

Mạch kiến thức: Phép thử tạo kết quả; biến ngẫu nhiên biến kết quả thành số; phân phối cho khả năng từng giá trị. Nhị thức dành cho số lần thành công rời rạc, chuẩn cho biến liên tục; PMF, PDF và CDF trả lời những câu hỏi xác suất khác nhau.

4.1. Random variable — Chuyển kết quả thử thành con số

Từ vựng: Random variable — biến ngẫu nhiên; **Outcome** — kết quả thử.

Biến ngẫu nhiên X gán một giá trị số cho mỗi kết quả của phép thử. Ví dụ trong ba lần giao hàng, X là **số đơn đúng giờ** nên X có thể bằng 0, 1, 2 hoặc 3, không phải tên từng đơn. Một phép thử cụ thể chỉ tạo một giá trị X; **phân phối xác suất** mô tả khả năng của toàn bộ các giá trị đó. Mô phỏng sử dụng `seed` chỉ để có thể lặp lại kết quả.

No Note — Không chú thích

```
import numpy as np

rng = np.random.default_rng(42)
print(rng.binomial(n=3, p=0.8, size=5).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Mô phỏng năm nhóm, mỗi nhóm ba lần thử Bernoulli để đếm số thành công.
# Dữ liệu: n=3 phép thử độc lập/nhóm, xác suất thành công p=0.8, mô phỏng 5 nhóm bằng seed 42.
# NumPy cung cấp bộ sinh ngẫu nhiên và ndarray; np là tên viết tắt.
import numpy as np

# default_rng(seed=42) tạo bộ phát số giả ngẫu nhiên tái lập kết quả cho cùng phiên bản, không làm đổi phân phối xác suất.
rng = np.random.default_rng(42)
# binomial(n,p,size) tạo 5 số đếm thành công; mỗi số từ 0 đến 3 qua n=3 thử độc lập, p=0.8 cho mỗi thử.
# tolist() chuyển ndarray kết quả thành list Python để print dễ đọc.
print(rng.binomial(n=3, p=0.8, size=5).tolist())
```

Kết quả khi chạy:

```
[2, 3, 2, 2, 3]
```

Mỗi phần tử là số đơn đúng giờ trong một nhóm ba đơn; đây là năm lần thử giả lập.

4.2. Binomial — Đếm thành công từ các lần thử độc lập

Từ vựng: Binomial — nhị thức; **Bernoulli** — phép thử thành công/thất bại.

Phân phối nhị thức tính số lần **thành công trong n phép thử độc lập**, mỗi lần có cùng xác suất thành công p. Với 10 đơn hàng, xác suất giao đúng giờ từng đơn bằng 0,8, biến X đếm số đơn đúng giờ. `binom.pmf(8,10,0.8)` tính **đúng 8 đơn** đúng giờ; `binom.cdf(8,10,0.8)` lại tính **tối đa 8 đơn**. Không dùng nhị thức khi xác suất từng lần khác nhau hoặc các lần phụ thuộc mạnh mà không điều chỉnh mô hình.

No Note — Không chú thích

```
from scipy.stats import binom

print(round(float(binom.pmf(8, 10, 0.8)), 4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính xác suất đúng tám lần thành công trong mười phép thử nhị thức.
# Dữ liệu: 10 phép thử độc lập, mỗi thử có tỷ lệ thành công 0.8, quan tâm đúng k=8.
# binom là phân phối nhị thức trong SciPy, dùng để tính xác suất số lần thành công.
from scipy.stats import binom

# pmf(k,n,p) là Probability Mass Function: xác suất CHÍNH XÁC k=8 trong n=10 phép thử với p=0.8,
không phải xác suất từ 8 trở lên.
# float() chuyển scalar NumPy/SciPy thành float Python; round(...,4) làm tròn bốn chữ số sau dấu
thập phân khi in.
print(round(float(binom.pmf(8, 10, 0.8)), 4))
```

Kết quả khi chạy:

```
0.302
```

Xác suất đúng 8 trong 10 đơn đúng giờ khoảng 30,2% theo mô hình độc lập.

4.3. Normal — Phân phối liên tục quanh trung bình

Từ vựng: Normal — chuẩn; **Mean/standard deviation** — trung bình/độ lệch chuẩn.

Phân phối chuẩn có dạng chuông đối xứng quanh trung bình μ, với độ phân tán đo bằng σ (độ lệch chuẩn, σ>0). Ví dụ điểm cảm biến được **giả định** chuẩn với μ=100 và σ=15 (cùng đơn vị đo). Ngưỡng 115 cao hơn trung bình đúng một σ. `norm.cdf(115,loc=100,scale=15)` tính khả năng X **không vượt quá 115**, xấp xỉ 0,8413. Không nhầm giá trị CDF với mật độ tại điểm.

No Note — Không chú thích

```
from scipy.stats import norm

print(round(float(norm.cdf(115, loc=100, scale=15)), 4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng hàm phân phối tích lũy chuẩn tính khả năng số đo không quá 115.
# Dữ liệu: Mô hình chuẩn có trung bình 100 và độ lệch chuẩn 15 cùng đơn vị số đo; ngưỡng 115.
# norm biểu diễn phân phối chuẩn liên tục, không phải phân phối nhị thức số đếm.
from scipy.stats import norm

# cdf(x,loc,scale) trả P(X≤x); loc=100 là trung bình, scale=15 là độ lệch chuẩn dương, không phải
phương sai.
# float() đổi scalar thành số Python, round(...,4) chỉ rút gọn chữ số khi hiển thị.
print(round(float(norm.cdf(115, loc=100, scale=15)), 4))
```

Kết quả khi chạy:

```
0.8413
```

Khoảng 84,13% giá trị nằm từ phía âm vô hạn tới ngưỡng 115 theo giả định chuẩn.

4.4. PMF, PDF và CDF — Ba câu hỏi khác nhau

Từ vựng: PMF — khối xác suất; **PDF** — mật độ xác suất; **CDF** — xác suất tích lũy.

PMF cho biến rời rạc trả xác suất **bằng đúng** một giá trị, chẳng hạn P(X=1) khi X là số lần thành công. **PDF** cho biến liên tục trả *mật độ* tại x, **không phải P(X=x)** (với phân phối chuẩn, xác suất tại đúng một điểm là 0). **CDF** ở cả hai loại biến biểu diễn P(X≤x). Để tính xác suất trong một khoảng liên tục `(a,b]`, ta trừ CDF(b) −CDF(a).

No Note — Không chú thích

```
from scipy.stats import norm

probability = norm.cdf(1) - norm.cdf(-1)
print(round(float(probability), 4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính xác suất một biến chuẩn tắc nằm từ -1 đến +1 bằng hiệu hai CDF.
# Dữ liệu: Phân phối chuẩn mặc định N(0,1): mean=0, std=1, hai giới hạn -1 và 1.
# norm mặc định là chuẩn tắc loc=0, scale=1 nếu không truyền tham số.
from scipy.stats import norm

# CDF tại 1 tính P(X≤1); trừ CDF tại -1 để được P(-1<X≤1). Với biến liên tục, dấu < hay ≤ ở một
điểm không đổi xác suất.
probability = norm.cdf(1) - norm.cdf(-1)
# float() chuẩn hóa scalar về Python, round(...,4) làm tròn xác suất để in.
print(round(float(probability), 4))
```

Kết quả khi chạy:

```
0.6827
```

Theo chuẩn tắc N(0,1), khoảng −1 đến +1 chứa khoảng 68,27% xác suất.

5. Năm tình huống phân phối — từ PMF nhị thức đến mô phỏng nhiều lô

Tình huống 1 — Cấp 1: So 10 đơn đúng hạn với phân phối nhị thức

Từ vựng trong mục: Bernoulli — phép thử hai kết quả; **Binomial PMF** — xác suất đúng k thành công.

Tình huống thực tế: Đội giao hàng ghi **10 đơn giao**, mỗi đơn được đánh dấu `1` nếu đến đúng giờ và `0` nếu trễ; trong bảng có **8 đơn đúng giờ**. Để thử dự báo cho những lô tương tự, công ty giả định từng đơn có **cơ hội đúng giờ 80%**, các đơn không ảnh hưởng nhau. Nhân viên muốn biết theo **mô hình dự báo**, xác suất một lô 10 đơn có **đúng 8 đơn đúng giờ** là bao nhiêu; đây không phải tỷ lệ giao đúng giờ của cả công ty.

Vì sao chọn cách này? Phân phối nhị thức phù hợp khi có **số lần thử cố định** (10 đơn), **hai kết quả** (đúng/trễ), cùng cơ hội 0,8 và giả định các đơn **độc lập**. `binom.pmf(8, n=10, p=0.8)` trả **xác suất đúng tám lần thành công**, không phải xác suất có từ tám lần trở lên. Còn `sum(observed)` đếm tám đơn đúng giờ trong **lô đã xảy ra**; tỷ lệ 8/10 chỉ là kết quả quan sát, không xác nhận giả định 80% hoặc tính độc lập. Cần kiểm dữ liệu thực tế nhiều hơn trước khi dùng mô hình ngoài bài tập.

Quy trình:

1. Đếm mười cờ thực tế và xác định số thành công k.

2. Tính xác suất nhị thức cho đúng k trong mười đơn.
3. In số đơn và xác suất để tránh nhầm PMF với phần trăm quan sát.

No Note — Không chú thích

```
from scipy.stats import binom
observed = [1,1,0,1,1,1,0,1,1,1]
count = sum(observed)
prob = binom.pmf(count, n=10, p=0.8)
if len(observed) != 10:
    raise ValueError("Cần đúng 10 đơn giao hàng.")
print("Đơn:", len(observed))
print("Đúng hạn:", count)
print(f"P(đúng số´này): {prob:.4f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So số´đơn giao đúng hạn đã quan sát với xác suất nhị thức đúng số´đó.
# Dữ liệu: 10 đơn, 1 là đúng hạn và 0 là trễ, p giả định 0.8 cho mỗi đơn độc lập.
# binom từ SciPy cung cấp xác suất cho số´lượt thành công trên n phép thử.
from scipy.stats import binom
# List 10 cờ 1/0, một vị trí là một đơn; 1 đúng hạn, 0 trễ.
observed = [1,1,0,1,1,1,0,1,1,1]
# sum() cộng cờ 1/0 để đếm số´đơn đúng hạn, không tính tỷ lệ.
count = sum(observed)
# pmf(count,n,p) tính xác suất CHÍNH XÁC count lần đúng hạn nếu 10 đơn là độc lập với xác suất
p=0.8 mỗi đơn.
prob = binom.pmf(count, n=10, p=0.8)
# len() kiểm đủ 10 đơn trước khi tin kết quả mô hình.
if len(observed) != 10:
    # ValueError ngắt chạy khi thiếu dữ liệu, thay vì in tỷ lệ không hợp lệ.
    raise ValueError("Cần đúng 10 đơn giao hàng.")
# len() kiểm đủ 10 đơn trước khi tin kết quả mô hình.
print("Đơn:", len(observed))
print("Đúng hạn:", count)
# f-string :.4f hiển thị xác suất với bốn chữ số´thập phân.
print(f"P(đúng số´này): {prob:.4f}")
```

Kết quả khi chạy:

```
Đơn: 10
Đúng hạn: 8
P(đúng số´này): 0.3020
```

Xác suất 0,3020 là kết quả của **mô hình đã giả định**, không phải xác suất một đơn bất kỳ được giao đúng hạn.

Tình huống 2 — Cấp 2: Tính xác suất không quá hai lỗi trong 12 linh kiện

Từ vựng trong mục: Binomial CDF — xác suất tích lũy; **Threshold** — ngưỡng.

Tình huống thực tế: Xưởng lắp ráp kiểm **12 linh kiện**, trong đó **3 chiếc đã quan sát là lỗi**. Để đánh giá rủi ro cho một lô tương tự, đội chất lượng thử giả định mỗi linh kiện có **10% khả năng lỗi**, độc lập với linh kiện khác. Quản đốc muốn hỏi **một lô 12 chiếc có từ 0 tới 2 chiếc lỗi** thường gặp đến đâu theo giả định, rồi đặt cạnh số lỗi thực tế 3 chiếc của lô đã kiểm.

Vì sao chọn cách này? PMF cho biết khả năng **một số lỗi cụ thể**; còn **CDF** cộng cơ hội từ mức thấp nhất tới **một ngưỡng được hỏi**. Ở đây cụm **'tối đa hai lỗi'** bao gồm **0, 1 hoặc 2**, nên dùng `binom.cdf(2, n=12,`

`p=0.1)`, không dùng `binom.pmf(2, ...)` vốn chỉ hỏi **đúng hai**. Danh sách 12 linh kiện đã kiểm có **ba lỗi thực tế**, nhưng điều đó là kết quả đã ghi, còn CDF là dự báo cho lô giả lập theo giả định; không suy ra lô có ba lỗi là dữ liệu sai.

Quy trình:

1. Lưu 12 cờ lỗi và đếm lỗi quan sát.
2. Dùng `binom.cdf` cho biến cố số lỗi không vượt 2.
3. Phân biệt số lỗi thật với xác suất mô hình.

No Note — Không chú thích

```
from scipy.stats import binom
faults = [1,0,0,0,1,0,0,0,1,0,0,0]
probability = binom.cdf(2, n=len(faults), p=0.1)
if len(faults) != 12:
    raise ValueError("Cần đúng 12 linh kiện.")
print("Linh kiện:", len(faults))
print("Lỗi quan sát:", sum(faults))
print(f"P(không quá 2 lỗi): {probability:.4f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính xác suất có nhiều nhất hai linh kiện lỗi theo mô hình nhị thức.
# Dữ liệu: 12 linh kiện, 1 nghĩa lỗi và 0 đạt; mô hình giả định xác suất lỗi p=0.1 độc lập.
# Phân phối nhị thức của số linh kiện lỗi trong một lô n=12.
from scipy.stats import binom
# 12 cờ lỗi 1/0 là dữ liệu quan sát, riêng với giả định xác suất p.
faults = [1,0,0,0,1,0,0,0,1,0,0,0]
# cdf(2,n,p) trả P(K≤2) gồm 0,1,2 lỗi, KHÁC pmf(2) chỉ đúng 2 lỗi; n=len(faults)=12.
# len() là số linh kiện cần đối chiếu đủ 12; sum(faults) đếm lỗi quan sát.
probability = binom.cdf(2, n=len(faults), p=0.1)
# len() là số linh kiện cần đối chiếu đủ 12; sum(faults) đếm lỗi quan sát.
if len(faults) != 12:
    # ValueError cảnh báo thiếu hoặc dư linh kiện, không tiếp tục kết luận.
    raise ValueError("Cần đúng 12 linh kiện.")
# len() là số linh kiện cần đối chiếu đủ 12; sum(faults) đếm lỗi quan sát.
print("Linh kiện:", len(faults))
print("Lỗi quan sát:", sum(faults))
# Định dạng :.4f làm tròn khi in, không biến kết quả thành tần suất thực nghiệm.
print(f"P(không quá 2 lỗi): {probability:.4f}")
```

Kết quả khi chạy:

```
Linh kiện: 12
Lỗi quan sát: 3
P(không quá 2 lỗi): 0.8891
```

Một lô đã có ba lỗi không thay đổi cách tính xác suất của **lô tương lai dưới giả định**; muốn cập nhật xác suất lỗi p cần nhiều dữ liệu hơn.

Tình huống 3 — Cấp 3: Tỷ lệ quan sát dưới ngưỡng so với phân phối chuẩn của 15 số đo

Từ vựng trong mục: Normal CDF — xác suất tích lũy chuẩn; **Empirical rate** — tần suất quan sát.

Tình huống thực tế: Trạm nhiệt có **15 phép đo nhiệt độ thực tế**, tính bằng **°C**. Các số trải từ 90 đến 111 °C, trong đó một lần đo lớn hơn 110 °C. Nhóm kỹ thuật đang thử giả định **nhiệt độ theo phân phối**

chuẩn, trung bình 100 °C và độ lệch chuẩn 10 °C; họ muốn so **tỷ lệ 15 lượt không vượt 110 °C** với **xác suất lý thuyết theo giả định**, không nhận một mô hình là đúng chỉ vì kết quả khá gần.

Vì sao chọn cách này? Phân phối chuẩn là mô hình dạng **đường chuông** được mô tả bởi mức trung tâm và độ phân tán. `norm.cdf(110, loc=100, scale=10)` trả cơ hội **không vượt 110 °C** theo mô hình, tức bao gồm mọi nhiệt độ thấp hơn ngưỡng. `np.mean(celsius <= 110)` lại đếm tỷ lệ **trong 15 phép đo đã có**; hai số có thể khác vì mô hình là giả định, còn số đo là mẫu hữu hạn. `norm.pdf(110)` chỉ cho độ cao **mật độ tại 110**, không trả xác suất các lần đo nằm dưới 110.

Quy trình:

1. Nhập 15 giá trị °C và đếm mẫu ≤110.
2. Tính CDF tại 110 với μ=100, σ=10.
3. Báo tỷ lệ đo và xác suất giả định cùng nhãn riêng.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import norm
celsius = np.array([90,95,101,103,110,104,96,98,111,93,106,99,109,102,97], dtype=float)
empirical = np.mean(celsius <= 110)
theoretical = norm.cdf(110, loc=100, scale=10)
if len(celsius) != 15:
    raise ValueError("Cần đủ 15 lượt đo nhiệt độ.")
print("Lượt đo:", len(celsius))
print("Đo ≤110:", int((celsius <= 110).sum()))
print(f"Tỷ lệ đo: {empirical:.3f}")
print(f"CDF mô hình: {theoretical:.4f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So tỷ lệ nhiệt độ quan sát ≤110 với xác suất từ phân phối chuẩn giả định.
# Dữ liệu: 15 nhiệt độ °C đo thực tế; mô hình N(loc=100°C,scale=10°C), ngưỡng 110°C.
# NumPy lưu 15 giá trị số và tính tần suất bằng mặt nạ Boolean.
import numpy as np
# norm.cdf() tính xác suất dưới mô hình chuẩn liên tục.
from scipy.stats import norm
# np.array(...,dtype=float) tạo vector 15 nhiệt độ dạng float, giữ đơn vị °C.
# dtype=float ép số đo thành số thực để hỗ trợ thống kê.
celsius = np.array([90,95,101,103,110,104,96,98,111,93,106,99,109,102,97], dtype=float)
# So từng nhiệt độ với 110, trả mask True ở vị trí đạt ngưỡng.
# mean() trên mask True/False cho tỷ lệ thực nghiệm ≤110 trong 15 lần đo (True tính là 1).
empirical = np.mean(celsius <= 110)
# CDF tại 110 tính P(T≤110) theo mô hình N(100,10²); scale=10 là độ lệch chuẩn °C.
theoretical = norm.cdf(110, loc=100, scale=10)
# len() kiểm đủ 15 lượt, không phải số lượt đạt điều kiện.
if len(celsius) != 15:
    raise ValueError("Cần đủ 15 lượt đo nhiệt độ.")
# len() kiểm đủ 15 lượt, không phải số lượt đạt điều kiện.
print("Lượt đo:", len(celsius))
# So từng nhiệt độ với 110, trả mask True ở vị trí đạt ngưỡng.
# sum() trên mask đếm lượt đạt ngưỡng, khác mean() cho tỷ lệ.
print("Đo ≤110:", int((celsius <= 110).sum()))
# Tỷ lệ đo thực tế in ba số thập phân.
print(f"Tỷ lệ đo: {empirical:.3f}")
# Xác suất mô hình in bốn số thập phân, hai đại lượng có thể khác nhau.
print(f"CDF mô hình: {theoretical:.4f}")
```

Kết quả khi chạy:

```
Lượt đo: 15
Đo ≤110: 14
Tỷ lệ đo: 0.933
CDF mô hình: 0.8413
```

Việc 14/15 phép đo đạt ngưỡng không chứng minh dữ liệu thực sự phân phối chuẩn; μ và σ ở đây là giả định minh họa.

Tình huống 4 — Cấp 4: Phân biệt mật độ và xác suất khoảng trên 18 mức cảm biến

Từ vựng trong mục: PDF — mật độ; **CDF interval** — xác suất trong khoảng.

Tình huống thực tế: Bộ hiệu chuẩn nhận **18 mức tín hiệu**, mỗi mức là một lượt đo không có đơn vị vật lý riêng. Quy định giả lập chấp nhận **từ 90 đến 110**, tính cả hai đầu mốc. Kỹ thuật viên muốn biết **bao nhiêu lượt đã ghi nằm trong khoảng**, đồng thời nếu tín hiệu thật tuân theo **phân phối chuẩn trung bình 100, độ lệch chuẩn 10**, thì khả năng một lần đo ngẫu nhiên nằm trong khoảng đó là bao nhiêu. Họ cũng muốn xem độ cao đường mật độ tại 100 để hiểu nó khác xác suất một khoảng.

Vì sao chọn cách này? Với một đại lượng **liên tục**, không thể lấy độ cao **PDF (mật độ)** tại một điểm làm xác suất rơi **đúng vào điểm ấy**; xác suất của một giá trị chính xác riêng lẻ trong mô hình chuẩn bằng 0. Muốn hỏi **từ 90 đến 110**, chương trình lấy **CDF tại 110 trừ CDF tại 90**, tức phần xác suất nằm giữa hai mốc. Song song, nó đếm số trong **18 lượt thực tế**, bao gồm hai đầu mốc. `norm.pdf(100, 100, 10)` chỉ được in để người học thấy mật độ có vai trò khác; số ấy không được so trực tiếp với tỷ lệ các lượt nằm trong khoảng.

Quy trình:

1. Tạo 18 số đo có cùng đơn vị.
2. Đếm số đo trong khoảng 90–110 (kể cả hai biên).
3. Tính xác suất khoảng bằng CDF và mật độ tại 100 bằng PDF.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import norm
values = np.array([84,88,90,92,94,95,97,98,99,100,101,102,103,105,108,110,113,117], dtype=float)
observed = int(((values >= 90) & (values <= 110)).sum())
interval = norm.cdf(110, 100, 10) - norm.cdf(90, 100, 10)
density = norm.pdf(100, 100, 10)
if values.shape != (18,):
    raise ValueError("Cần đúng 18 giá trị đo.")
print("Lượt đo:", len(values))
print("Trong khoảng:", observed)
print(f"Xác suất mô hình: {interval:.4f}")
print(f"Mật độ tại 100: {density:.4f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt xác suất trong khoảng và mật độ xác suất tại một giá trị.
# Dữ liệu: 18 phép đo, mô hình chuẩn N(100,10²); xét khoảng 90..110 và mật độ tại 100 (đơn vị
nghịch số đo).
# NumPy tạo mảng số và lọc theo mặt nạ từng phần tử.
import numpy as np
# norm.cdf() cho xác suất tích lũy, norm.pdf() cho MẬT ĐỘ chứ không phải xác suất một điểm.
from scipy.stats import norm
# Mảng values gồm 18 lượt đo thực tế, dtype=float là kiểu số thực.
```

```
values = np.array([84,88,90,92,94,95,97,98,99,100,101,102,103,105,108,110,113,117], dtype=float)
# Hai mask môĩ ô dùng & (AND theo phần tư̛), có ngoặc riêng; chọn mọi giá trị thuộc đoạn 90..110.
# sum() trên mask Boolean đếm số´lượt True trong 18 lượt.
observed = int(((values >= 90) & (values <= 110)).sum())
# Hai cdf có tham số´vị trí loc=100, độ lệch chuẩn scale=10; hiệu trả xác suất khoảng [90,110].
interval = norm.cdf(110, 100, 10) - norm.cdf(90, 100, 10)
# pdf(x,loc,scale) trả MẬT ĐỘ tại x=100, đơn vị 1/(đơn vị đo), không phải P(X=100) (bằng 0 ở phân
phối liên tục).
density = norm.pdf(100, 100, 10)
# shape (18,) là vector 18 phần tư̛; đối chiếu kích thước trước báo cáo.
if values.shape != (18,):
    # Báo dữ liệu đo không có đủ 18 lượt.
    raise ValueError("Cần đúng 18 giá trị đo.")
print("Lượt đo:", len(values))
print("Trong khoảng:", observed)
# :.4f in xác suất khoảng với bốn chữ số´thập phân.
print(f"Xác suất mô hình: {interval:.4f}")
# :.4f in mật độ với bốn chữ số´thập phân, không được gọi là tỷ lệ lượt.
print(f"Mật độ tại 100: {density:.4f}")
```

Kết quả khi chạy:

```
Lượt đo: 18
Trong khoảng: 14
Xác suất mô hình: 0.6827
Mật độ tại 100: 0.0399
```

PDF có đơn vị **nghịch đảo đơn vị cảm biến**, trong khi xác suất khoảng không mang đơn vị và nằm trong [0,1].

Tình huống 5 — Cấp 5: Lặp 20 lô thử và đối chiếu phân phối số thành công

Từ vựng trong mục: Sampling distribution — phân phối kết quả thử; **PMF normalization** — tổng xác suất bằng một.

Tình huống thực tế: Nhóm vận hành tạo **20 lô giao hàng mô phỏng**, mỗi lô chứa **10 đơn**, chứ không phải 20 đơn riêng lẻ. Mỗi đơn được giả định đúng giờ với xác suất **70%**, độc lập, nên kết quả mỗi lô là **một số từ 0 đến 10**. Nhóm muốn kiểm trung bình dự kiến **7 đơn đúng hạn mỗi lô**, liệt kê đủ các khả năng 0–10 và dùng **seed 42** để có thể chạy lại đúng cùng 20 lô giả lập.

Vì sao chọn cách này? Một lô gồm **mười phép thử đúng/trễ**, còn `rng.binomial(n=10, p=0.7, size=20)` tạo **20 kết quả giả lập của 20 lô**, mỗi kết quả là số đơn đúng giờ trong lô. **Kỳ vọng** là mức trung bình lý thuyết nếu lặp đủ nhiều lô cùng giả định: ở đây **7 đơn/lô**, không buộc từng lô phải có đúng 7. `binom.pmf` được tính riêng cho **11 khả năng 0 đến 10** của **một lô**, vì thế cộng 11 xác suất phải gần **1**. **Seed 42** làm kết quả ngẫu nhiên của ví dụ tái lập được trên cùng môi trường; 20 lô là dữ liệu mô phỏng, không phải 20 lô giao hàng thật hay số lượt bắt buộc cho phân phối nhị thức.

Quy trình:

1. Mô phỏng 20 lô có 10 lần thử/lô bằng seed cố định.
2. Tính số trung bình quan sát và kỳ vọng 10×0,7.
3. Liệt kê xác suất PMF từ 0 đến 10 rồi đối chiếu tổng.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import binom
```

```
rng = np.random.default_rng(42)
counts = rng.binomial(n=10, p=0.7, size=20)
possible = np.arange(11)
probabilities = binom.pmf(possible, n=10, p=0.7)
if counts.shape != (20,) or not np.all((counts >= 0) & (counts <= 10)):
    raise ValueError("Cần 20 lô mô phỏng, mỗi lô có từ 0 đến 10 đơn đúng hạn.")
print("Lô mô phỏng:", len(counts))
print("Đơn mỗi lô:", 10)
print("Kỳ vọng mỗi lô:", f"{10*0.7:.1f}")
print("PMF có 11 giá trị:", len(probabilities))
print("Tổng PMF bằng 1:", bool(np.isclose(probabilities.sum(), 1)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Mô phỏng 20 lô 10 đơn và kiểm PMF nhị thức từ 0 tới 10 cộng thành 1.
# Dữ liệu: 20 lô, mỗi lô 10 đơn độc lập xác suất đúng hạn p=0.7; seed 42 giữ mô phỏng tái lập.
# NumPy tạo mô phỏng và vector các giá trị k=0..10.
import numpy as np
# binom.pmf() tính phân phối số lần thành công trong 10 phép thử.
from scipy.stats import binom
# Bộ sinh riêng seed 42 để tái hiện các số ngẫu nhiên khi chạy lại.
rng = np.random.default_rng(42)
# Sinh 20 số đếm, mỗi số là số đơn đúng hạn trong lô 10 đơn, giả định độc lập p=70%.
counts = rng.binomial(n=10, p=0.7, size=20)
# arange(11) sinh vector số nguyên 0..10 (cận 11 bị loại): tất cả 11 số lần thành công khả dĩ.
possible = np.arange(11)
# pmf() áp lên từng k=0..10, trả 11 xác suất đúng k đơn đúng hạn theo mô hình nhị thức.
probabilities = binom.pmf(possible, n=10, p=0.7)
# shape (20,) kiểm đúng 20 lô, không nhầm số đơn trong mỗi lô.
# & kết hợp hai mask giới hạn số thành công 0..10, np.all() yêu cầu MỌI lô đều hợp lệ.
if counts.shape != (20,) or not np.all((counts >= 0) & (counts <= 10)):
    raise ValueError("Cần 20 lô mô phỏng, mỗi lô có từ 0 đến 10 đơn đúng hạn.")
# len() trả số lô mô phỏng, không phải số đơn toàn bộ.
print("Lô mô phỏng:", len(counts))
print("Đơn mỗi lô:", 10)
# Kỳ vọng số đơn đúng hạn trong mỗi lô = n×p = 7, khác giá trị ngẫu nhiên thực tế của từng lô.
print("Kỳ vọng mỗi lô:", f"{10*0.7:.1f}")
# len() là số khả năng k=0..10, bằng 11.
print("PMF có 11 giá trị:", len(probabilities))
# sum() cộng xác suất của TẤT CẢ 11 giá trị k, isclose(...,1) kiểm gần 1 với dung sai float.
# bool() chuyển kết quả kiểm NumPy thành True/False Python để in.
print("Tổng PMF bằng 1:", bool(np.isclose(probabilities.sum(), 1)))
```

Kết quả khi chạy:

```
Lô mô phỏng: 20
Đơn mỗi lô: 10
Kỳ vọng mỗi lô: 7.0
PMF có 11 giá trị: 11
Tổng PMF bằng 1: True
```

20 là số **lô mô phỏng**, không phải n=20 trong từng phép thử nhị thức. Đối với các đơn phụ thuộc nhau, mô hình nhị thức này có thể không phù hợp.

6. Sai lầm thường gặp

- PMF tính xác suất đúng một giá trị **rời rạc**; PDF là mật độ cho biến liên tục, không phải P(X=x).
- CDF là P(X≤x), khác PMF tại k và PDF tại x.
- Mô hình nhị thức cần n lần thử độc lập với p giống nhau; thay điều kiện có thể làm kết quả sai.

- `scale` là độ lệch chuẩn dương, không phải phương sai.
- Seed cố định cho mô phỏng có thể tái lập trong môi trường tương thích; không bảo đảm thống kê mẫu bằng lý thuyết.
- Xác suất dưới mô hình không chứng minh quy trình thực tế đúng giả định.

7. Trắc nghiệm

Q1. What is a random variable? | Biến ngẫu nhiên là gì?

A. A continuous symmetric bell-shaped density | Mật độ liên tục dạng chuông đối xứng

B. A numerical quantity assigned to random outcomes | Đại lượng số gắn với kết quả ngẫu nhiên

C. Probability mass at a discrete value | Xác suất ở một giá trị rời rạc

D. Successes in independent same-p trials | Số thành công trong các lần thử độc lập cùng p

Đáp án: B

Giải thích: Nó biến kết quả phép thử thành số. Biến ngẫu nhiên không phải toàn bộ kết quả thử; nó gán một con số, chẳng hạn số đơn đúng giờ, cho mỗi kết quả có thể.

Q2. What does binomial X count? | Biến nhị thức X đếm gì?

A. A continuous symmetric bell-shaped density | Mật độ liên tục dạng chuông đối xứng

B. Successes in independent same-p trials | Số thành công trong các lần thử độc lập cùng p

C. Probability mass at a discrete value | Xác suất ở một giá trị rời rạc

D. A numerical quantity assigned to random outcomes | Đại lượng số gắn với kết quả ngẫu nhiên

Đáp án: B

Giải thích: Mỗi lần thử có hai kết quả thành công hoặc thất bại. Biến nhị thức X nhận từ 0 đến n; nó đếm số lần đạt, không đếm số loại kết quả của một phép thử.

Q3. What characterizes a normal distribution? | Phân phối chuẩn có đặc điểm gì?

A. A numerical quantity assigned to random outcomes | Đại lượng số gắn với kết quả ngẫu nhiên

B. Probability mass at a discrete value | Xác suất ở một giá trị rời rạc

C. Successes in independent same-p trials | Số thành công trong các lần thử độc lập cùng p

D. A continuous symmetric bell-shaped density | Mật độ liên tục dạng chuông đối xứng

Đáp án: D

Giải thích: Được xác định bởi trung bình và độ lệch chuẩn. Hai tham số có ý nghĩa khác nhau: mean xác định tâm và độ lệch chuẩn dương quyết định độ phân tán.

Q4. What is a PMF? | PMF là gì?

A. A continuous symmetric bell-shaped density | Mật độ liên tục dạng chuông đối xứng

B. Probability mass at a discrete value | Xác suất ở một giá trị rời rạc

C. A numerical quantity assigned to random outcomes | Đại lượng số gắn với kết quả ngẫu nhiên

D. Successes in independent same-p trials | Số thành công trong các lần thử độc lập cùng p

Đáp án: B

Giải thích: PMF tính xác suất đúng một kết quả rời rạc. Với X nhị thức, PMF tại k trả đúng k lần thành công; CDF tại k tính cả mọi số lần ít hơn k.

Q5. Two independent fair coins are tossed. What is P(exactly one head)? | Tung hai đồng xu công bằng độc lập. Xác suất có đúng một mặt ngửa là bao nhiêu?

A. 1 | 1

B. 0.25 | 0.25

C. 0.5 | 0.5

D. 0.75 | 0.75

Đáp án: C

Giải thích: Bốn kết quả đồng khả năng: NN, NS, SN, SS (N = ngửa, S = sấp). Hai kết quả có đúng một ngửa, nên 2/4 = 0.5.

Q6. In a standard normal distribution N(0,1), what is P(X ≤ 0)? | Trong phân phối chuẩn tắc N(0,1), P(X ≤ 0) bằng bao nhiêu?

A. 0 | 0

B. 1 | 1

C. 0.25 | 0.25

D. 0.5 | 0.5

Đáp án: D

Giải thích: Phân phối chuẩn tắc đối xứng quanh trung bình 0, nên một nửa diện tích nằm bên trái 0.

Q7. What happens to the spread of a normal distribution if its standard deviation increases while its mean stays fixed? | Nếu độ lệch chuẩn của phân phối chuẩn tăng còn trung bình không đổi, độ phân tán thay đổi thế nào?

A. It spreads out more | Phân phối trải rộng hơn

B. Its mean becomes zero | Trung bình chuyển thành 0

C. It becomes uniform | Chuyển thành phân phối đều

D. It becomes narrower | Phân phối hẹp lại

Đáp án: A

Giải thích: Độ lệch chuẩn đo mức phân tán; tăng độ lệch chuẩn làm đường cong rộng và thấp hơn, nhưng vẫn giữ tâm cũ.

Q8. For a normal distribution, how are mean and median related? | Trong phân phối chuẩn, trung bình và trung vị có quan hệ gì?

A. The mean is always larger | Trung bình luôn lớn hơn

B. The median is always zero | Trung vị luôn bằng 0

C. The mean is always negative | Trung bình luôn âm

D. They are equal | Hai giá trị bằng nhau

Đáp án: D

Giải thích: Phân phối chuẩn đối xứng quanh trung bình nên trung bình, trung vị và mode đều trùng nhau.

Q9. What condition is essential for a simple binomial model? | Điều kiện quan trọng của nhị thức đơn giản là gì?

A. Independent trials with constant success probability | Các lần thử độc lập cùng xác suất thành công

B. Center and spread | Trung tâm và độ phân tán

C. Zero | Bằng 0

D. Strictly positive standard deviation | Độ lệch chuẩn phải lớn hơn 0

Đáp án: A

Giải thích: Phụ thuộc giữa lần thử làm sai giả định mô hình. Nếu đơn giao cùng chịu một sự cố hoặc xác suất đúng giờ khác nhau, mô hình nhị thức đơn giản có thể không phù hợp.

Q10. What do mean and standard deviation determine for normal? | Mean và std xác định gì trong chuẩn?

A. Center and spread | Trung tâm và độ phân tán

B. Independent trials with constant success probability | Các lần thử độc lập cùng xác suất thành công

C. Zero | Bằng 0

D. Strictly positive standard deviation | Độ lệch chuẩn phải lớn hơn 0

Đáp án: A

Giải thích: Trung bình định vị và độ lệch chuẩn xác định độ rộng. Đổi độ lệch chuẩn làm chuông rộng/hẹp nhưng không nhất thiết thay tâm của phân phối.

Q11. What is P(X=x) for a continuous normal variable? | P(X=x) với biến chuẩn liên tục bằng bao nhiêu?

A. Center and spread | Trung tâm và độ phân tán

B. Zero | Bằng 0

C. Independent trials with constant success probability | Các lần thử độc lập cùng xác suất thành công

D. Strictly positive standard deviation | Độ lệch chuẩn phải lớn hơn 0

Đáp án: B

Giải thích: Một điểm đơn có xác suất bằng 0. Xác suất trong khoảng là diện tích dưới PDF; một điểm riêng lẻ không có diện tích.

Q12. Which quantity must be positive for norm.pdf scale? | scale của norm.pdf cần điều kiện nào?

A. Center and spread | Trung tâm và độ phân tán

B. Independent trials with constant success probability | Các lần thử độc lập cùng xác suất thành công

C. Zero | Bằng 0

D. Strictly positive standard deviation | Độ lệch chuẩn phải lớn hơn 0

Đáp án: D

Giải thích: Độ lệch chuẩn không dương không hợp lệ. Trong SciPy, `scale` là sigma chứ không phải sigma bình phương; truyền phương sai sẽ làm sai toàn bộ phân phối.

Q13. What does norm.cdf(0) equal for N(0,1)? | norm.cdf(0) với N(0,1) bằng gì?

A. No, sampling noise remains | Không, vẫn có dao động lấy mẫu

B. From 0 to 1 inclusive | Từ 0 đến 1 gồm hai đầu

C. CDF(b) minus CDF(a) | CDF(b) trừ CDF(a)

D. 0.5 | 0.5

Đáp án: D

Giải thích: Chuẩn tắc đối xứng quanh 0. Chuẩn tắc đối xứng quanh trung bình 0 nên mỗi phía chứa một nửa xác suất.

Q14. How do you calculate normal P(a<X≤b)? | Tính P(a<X≤b) cho chuẩn bằng gì?

A. 0.5 | 0.5

B. CDF(b) minus CDF(a) | CDF(b) trừ CDF(a)

C. From 0 to 1 inclusive | Từ 0 đến 1 gồm hai đầu

D. No, sampling noise remains | Không, vẫn có dao động lấy mẫu

Đáp án: B

Giải thích: Hiệu xác suất tích lũy cho xác suất đoạn. Ví dụ P(−1<X≤1) bằng `norm.cdf(1)-norm.cdf(-1)`, xấp xỉ 0,6827.

Q15. Does fixed RNG seed force sample mean to equal μ? | Seed cố định có buộc mean mẫu bằng μ?

A. No, sampling noise remains | Không, vẫn có dao động lấy mẫu

B. From 0 to 1 inclusive | Từ 0 đến 1 gồm hai đầu

C. CDF(b) minus CDF(a) | CDF(b) trừ CDF(a)

D. 0.5 | 0.5

Đáp án: A

Giải thích: Seed lặp được mẫu, không đảm bảo đúng moment lý thuyết. Mô phỏng cố định seed có thể lặp lại nhưng trung bình mẫu vẫn lệch so với kỳ vọng lý thuyết.

Q16. What is the range of probabilities? | Xác suất nằm trong đoạn nào?

A. No, sampling noise remains | Không, vẫn có dao động lấy mẫu

B. CDF(b) minus CDF(a) | CDF(b) trừ CDF(a)

C. From 0 to 1 inclusive | Từ 0 đến 1 gồm hai đầu

D. 0.5 | 0.5

Đáp án: C

Giải thích: Xác suất không âm và không vượt một. Hai đầu 0 và 1 tương ứng biến cố không thể và chắc chắn.

Q17. Can a continuous PDF exceed 1 at a point? | PDF liên tục có thể lớn hơn 1 tại một điểm không?

A. The distribution becomes wider | Phân phối trải rộng hơn

B. Yes; only its total integral must be one | Có; chỉ tích phân toàn miền phải bằng một

C. Cumulative probability up to x | Xác suất tích lũy tới x

D. Point probabilities behave differently | Xác suất tại một điểm có ý nghĩa khác nhau

Đáp án: B

Giải thích: Chiều cao mật độ không phải xác suất. Đường chuẩn có độ lệch chuẩn nhỏ có thể có đỉnh PDF cao hơn 1 nhưng tổng diện tích vẫn bằng 1.

Q18. What happens when standard deviation of normal increases? | Std phân phối chuẩn tăng sẽ thế nào?

A. Yes; only its total integral must be one | Có; chỉ tích phân toàn miền phải bằng một

B. Point probabilities behave differently | Xác suất tại một điểm có ý nghĩa khác nhau

C. The distribution becomes wider | Phân phối trải rộng hơn

D. Cumulative probability up to x | Xác suất tích lũy tới x

Đáp án: C

Giải thích: Cùng tổng diện tích được trải ra trên miền rộng hơn. Sigma lớn hơn không tự làm dịch mean; nó khiến số đo phân tán xa hơn quanh cùng tâm.

Q19. Why distinguish discrete and continuous variables? | Vì sao phân biệt biến rời rạc và liên tục?

A. The distribution becomes wider | Phân phối trải rộng hơn

B. Cumulative probability up to x | Xác suất tích lũy tới x

C. Point probabilities behave differently | Xác suất tại một điểm có ý nghĩa khác nhau

D. Yes; only its total integral must be one | Có; chỉ tích phân toàn miền phải bằng một

Đáp án: C

Giải thích: PMF là xác suất còn PDF là mật độ. Số đếm như X=3 có xác suất tại điểm dương, còn biến đo liên tục có xác suất bằng đúng một giá trị là 0.

Q20. What does norm.cdf(x,loc,scale) return? | norm.cdf(x,loc,scale) trả gì?

A. Cumulative probability up to x | Xác suất tích lũy tới x

B. Point probabilities behave differently | Xác suất tại một điểm có ý nghĩa khác nhau

C. Yes; only its total integral must be one | Có; chỉ tích phân toàn miền phải bằng một

D. The distribution becomes wider | Phân phối trải rộng hơn

Đáp án: A

Giải thích: Nó là P(X≤x) theo phân phối chuẩn. Không lấy chiều cao PDF nếu câu hỏi là P(X≤x); cần dùng `cdf` để tính xác suất tích lũy.

8. Liên hệ bài tiếp theo

`Xác suất → Biến ngẫu nhiên → Phân phối → Thống kê.`

PY-37 — Thống kê mô tả: mean, median, mode, variance và std

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Mean — trung bình; **Median** — trung vị.

`NumPy/Pandas → Thống kê mô tả → Outlier → ML`

Học xong dùng để làm gì? Tóm tắt bảng số bằng trung bình, trung vị, phương sai và độ lệch chuẩn để so sánh các nhóm.

Kết nối kiến thức tiếp theo: PY-38 dùng phân vị/IQR để tìm giá trị khác thường mà trung bình có thể che khuất.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Mean	trung bình	Tổng giá trị chia số quan sát
Median	trung vị	Giá trị giữa sau khi sắp xếp
Variance	phương sai	Mức phân tán bình phương quanh trung bình
Standard deviation	độ lệch chuẩn	Căn bậc hai của phương sai, cùng đơn vị dữ liệu

3. Hàm Python và tham số quan trọng

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Cách sử dụng
`np.mean()/np.median()`	Trung bình và trung vị, median bớt nhạy với giá trị cực lớn
`np.var(x,ddof=0/1)`	ddof=0 cho quần thể, ddof=1 cho mẫu
`np.std(x,ddof=0/1)`	Độ lệch chuẩn nhất quán quy ước ddof
`pd.Series.mode()/value_counts()`	Giá trị phổ biến và tần suất

4. Khái niệm chính và liên quan

Mạch kiến thức: Khi có nhiều số đo, mean và median trả lời câu hỏi giá trị ở giữa; variance và std đo mức phân tán. Mode trả lời câu hỏi nhãn nào xuất hiện nhiều nhất. Cần biết đơn vị và đang tính theo quần thể hay mẫu.

4.1. Mean — Điểm cân bằng của các quan sát

Từ vựng: Mean — trung bình; **Observation** — quan sát.

Mean cộng các giá trị rồi chia cho số quan sát. Ba đơn giao trong 2, 3, 4 ngày có trung bình `(2+3+4)/3=3` ngày. Mean sử dụng **mọi giá trị**, nên có thể bị kéo lệch nếu có một đơn rất chậm. Phải đảm bảo các thời gian dùng cùng đơn vị; mean của các mã sản phẩm không có ý nghĩa.

No Note — Không chú thích

```
import numpy as np

days = np.array([2, 3, 4])
print(float(np.mean(days)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính mean của ba sô´ngày đểhiểủ giá trị trung bình cộng.
# Dữ liệu: Ba sô´lượng ngày 2,3,4; môĩ phần tửlà một quan sát cùng đơn vị ngày.
# NumPy tính các thông kê cơ bản trên list và mảng sô´.
import numpy as np

# Tạo ndarray một chiêù ba lượt quan sát, không phải ma trận nhiêù cột.
days = np.array([2, 3, 4])
# mean() cộng các quan sát rôì chia sô´lượt (3), trả 3.0 ngày; khác median hay mode.
# float() đôỉ scalar NumPy vềsô´thực Python đểin.
print(float(np.mean(days)))
```

Kết quả khi chạy:

```
3.0
```

Thời gian trung bình ba đơn là 3 ngày, không có nghĩa đơn nào cũng giao đúng 3 ngày.

4.2. Median — Vị trí giữa sau khi xếp thứ tự

Từ vựng: Median — trung vị; **Outlier** — giá trị khác biệt lớn.

Median là số ở giữa khi dữ liệu được **sắp tăng dần**; nếu có số quan sát chẵn thì là trung bình hai số giữa. Với giá trị đơn hàng `[100,120,10000]` nghìn đồng, trung vị 120 nghìn, còn mean vượt 3.400 nghìn vì đơn 10.000 quá lớn. Median thường giúp mô tả giá trị trung tâm khi dữ liệu lệch, nhưng không thay thế việc kiểm tra liệu giá trị bất thường có hợp lệ.

No Note — Không chú thích

```
import numpy as np

amounts = [100, 120, 10000]
print(float(np.median(amounts)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm trung vị của ba sô´tiêǹ, tránh đểkhoản rât́ lớn làm lệch điêm̉ giữa.
# Dữ liệu: Ba hóa đơn 100,120,10000 nghìn đồng, hóa đơn cuôí lớn bât́ thường.
# NumPy hỗtrợ trung vị qua median().
```

```
import numpy as np

# List ba sô´tiền; khoản 10.000 rất lớn nhưng không làm dịch vị trí phần tư ̉giữa.
amounts = [100, 120, 10000]
# median() săp giá trị rồi lấy điểm ở GIỮA, ở đây 120; với sô´lượt chăn lấy trung bình hai ô giữa.
# float() ép kết quả vê`float Python cho print().
print(float(np.median(amounts)))
```

Kết quả khi chạy:

```
120.0
```

Trung vị 120 nghìn đồng ít bị ảnh hưởng bởi đơn 10.000 nghìn đồng.

4.3. Variance — Sai khác bình phương quanh mean

Từ vựng: Variance — phương sai; **ddof** — số bậc tự do trừ đi.

Phương sai đo độ phân tán bằng **trung bình bình phương sai lệch so với mean**. Điểm `[6,8,10]` có mean 8, các lệch `[-2,0,2]`, bình phương `[4,0,4]`. Phương sai **quần thể** `ddof=0` bằng `8/3≈2.667` điểm². Nếu dùng để **ước lượng từ mẫu**, phương sai mẫu với `ddof=1` là `8/2=4` điểm². Cần nêu rõ đang dùng mẫu hay toàn bộ quần thể.

No Note — Không chú thích

```
import numpy as np

scores = [6, 8, 10]
print(round(float(np.var(scores, ddof=0)), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính phương sai quân thê ̉của ba điểm và giải thích tham sô´ddof=0.
# Dữ liệu: Ba điểm 6,8,10 trên cùng thang điểm; coi ba lượt là toàn bộ quân thê ̉quan tâm.
# NumPy np.var() tính trung bình sai lệch bình phương với mean.
import numpy as np

# Ba quan sát cùng đơn vị điểm đê ̉tính mức phân tán.
scores = [6, 8, 10]
# var() tính phương sai; ddof=0 nghĩa chia cho N (quân thê ̃). Nêú ước lượng từ mâũ thường dùng
ddof=1 chia N−1; đơn vị là điểm².
# float() chuyên NumPy scalar sang float, round(...,3) làm tròn ba sô´thập phân khi in.
print(round(float(np.var(scores, ddof=0)), 3))
```

Kết quả khi chạy:

```
2.667
```

Phương sai quần thể 2,667 điểm bình phương; không phải số điểm trung bình.

4.4. Standard deviation — Độ phân tán cùng đơn vị dữ liệu

Từ vựng: Standard deviation — độ lệch chuẩn; **Mode** — yếu vị.

Standard deviation (std) là căn bậc hai của phương sai, có **cùng đơn vị** với số đo. Cảm biến nhiệt độ đo `[20,21,19,20]` °C: mean=20 °C, std quần thể bằng `sqrt(0.5)≈0.707` °C. Trong khi đó **mode** là giá trị xuất hiện nhiều nhất, ở đây 20 °C hai lần; đây là câu hỏi về tần suất, không phải độ phân tán.

No Note — Không chú thích

```
import numpy as np

celsius = [20, 21, 19, 20]
print(round(float(np.std(celsius, ddof=0)), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính độ lệch chuẩn quần thể để mô tả độ phân tán theo cùng đơn vị °C.
# Dữ liệu: Bốn nhiệt độ °C 20,21,19,20, coi là toàn bộ bốn quan sát cần tóm tắt.
# NumPy std() tính độ lệch chuẩn là căn bậc hai của phương sai.
import numpy as np

# Bốn lượt đo độ C, cùng đơn vị nên có thể tính độ phân tán.
celsius = [20, 21, 19, 20]
# std(ddof=0) lấy căn phương sai quần thể (chia N), kết quả cùng đơn vị °C; ddof=1 là ước lượng
mẫu.
# float() đổi scalar, round(...,3) định dạng ba chữ số thập phân.
print(round(float(np.std(celsius, ddof=0)), 3))
```

Kết quả khi chạy:

```
0.707
```

Độ lệch chuẩn 0,707 °C mô tả dao động nhiệt quanh mean 20 °C.

5. Năm tình huống thống kê mô tả — từ mean đến báo cáo dữ liệu thiếu

Tình huống 1 — Cấp 1: Trung bình thời gian xử lý của 10 yêu cầu

Từ vựng trong mục: Mean — trung bình; **Observation** — một lượt quan sát.

Tình huống thực tế: Nhóm hỗ trợ có **10 phiếu yêu cầu**, mỗi phiếu ghi thời gian xử lý thực tế tính bằng **phút**. Các lượt mất từ **2 đến 5 phút** và dữ liệu gồm đủ mười phiếu trong ca thử nghiệm. Quản lý muốn biết **tổng số phút** đã dùng, **trung bình mỗi phiếu** và phiếu ngắn nhất/dài nhất để có bức tranh sơ bộ về khối lượng công việc; không được tưởng tất cả phiếu đều bằng thời gian trung bình.

Vì sao chọn cách này? Thống kê mô tả là cách tóm tắt những giá trị đã quan sát, chưa phải dự báo tương lai. `sum()` cho **tổng thời gian 10 phiếu**, đơn vị **phút**, còn `mean()` chia đều tổng ấy trên **10 phiếu** để mô tả một phiếu trung bình. `min()` và `max()` cho hai mức biên, giúp thấy có phiếu nhanh hơn hoặc chậm hơn mức trung bình. Cả mười số đo có cùng đơn vị nên có thể cộng được; đừng hiểu mean là thời gian cố định bắt buộc cho mọi yêu cầu.

Quy trình:

1. Lưu 10 số phút xử lý theo thứ tự yêu cầu.
2. Tính tổng, trung bình bằng NumPy.

3. So khoảng giá trị ngắn nhất và dài nhất.

No Note — Không chú thích

```
import numpy as np
minutes = np.array([2,3,4,3,2,5,3,4,2,4], dtype=float)
if minutes.size != 10:
    raise ValueError("Cần đúng 10 yêu cầu.")
print("Yêu cầu:", minutes.size)
print("Tổng phút:", int(minutes.sum()))
print(f"Trung bình (phút): {minutes.mean():.1f}")
print("Nhỏ nhất/lớn nhất:", int(minutes.min()), int(minutes.max()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thống kê 10 thời gian xử lý: tổng phút, trung bình, nhỏ nhất và lớn nhất.
# Dữ liệu: 10 yêu cầu, mỗi số là số phút xử lý một yêu cầu.
# NumPy cung cấp mảng và thống kê sum/mean/min/max.
import numpy as np
# np.array(...,dtype=float) tạo vector 10 số phút, cho phép thống kê dạng số thực.
# dtype=float ép sang số thực, không làm thay đổi đơn vị phút.
minutes = np.array([2,3,4,3,2,5,3,4,2,4], dtype=float)
# size là SỐ PHẦN TỬ trong ndarray; với vector này đúng 10 yêu cầu.
if minutes.size != 10:
    # ValueError báo đầu vào không đủ 10 yêu cầu, không tiếp tục kết luận sai.
    raise ValueError("Cần đúng 10 yêu cầu.")
# size là SỐ PHẦN TỬ trong ndarray; với vector này đúng 10 yêu cầu.
print("Yêu cầu:", minutes.size)
# sum() cộng tổng số phút của mọi yêu cầu; int() đổi scalar sang số nguyên Python.
print("Tổng phút:", int(minutes.sum()))
# mean() tính thời gian trung bình mỗi yêu cầu; :.1f chỉ hiển thị một chữ số lẻ.
print(f"Trung bình (phút): {minutes.mean():.1f}")
# min() tìm yêu cầu nhanh nhất và max() tìm yêu cầu chậm nhất, không phải tổng.
# max() là thời gian chậm nhất, int() để in số nguyên.
print("Nhỏ nhất/lớn nhất:", int(minutes.min()), int(minutes.max()))
```

Kết quả khi chạy:

```
Yêu cầu: 10
Tổng phút: 32
Trung bình (phút): 3.2
Nhỏ nhất/lớn nhất: 2 5
```

Trung bình 3,2 phút là kết quả trên đúng mười yêu cầu đã ghi, không phải cam kết thời gian giải quyết từng yêu cầu.

Tình huống 2 — Cấp 2: So mean và median khi 12 hóa đơn có đơn rất lớn

Từ vựng trong mục: Median — trung vị; **Outlier** — giá trị khác biệt.

Tình huống thực tế: Cửa hàng ghi **12 hóa đơn**, mỗi dòng là tổng số tiền của một khách trong lần mua đó, tính bằng **nghìn đồng**. Mười một đơn trong khoảng **100–140 nghìn đồng**, nhưng một đơn hợp lệ lên tới **1.000 nghìn đồng**. Chủ cửa hàng muốn chọn con số mô tả mức chi tiêu thường gặp và xem vì sao **trung bình cộng** có thể khác **trung vị** rất nhiều mà không được xóa đơn lớn.

Vì sao chọn cách này? Mean là giá trị cân bằng từ **toàn bộ 12 khoản tiền**, nên một đơn 1.000 nghìn đồng kéo nó lên rõ rệt. **Median (trung vị)** là mức **nằm giữa danh sách đã sắp xếp**; với 12 giá trị, lấy mức giữa hai hóa đơn ở trung tâm, nên ít bị ảnh hưởng bởi một khoản lớn. `np.mean()` và `np.median()` trả hai

cách nhìn khác nhau, đều có đơn vị **nghìn đồng**. Đơn lớn chưa chắc là sai; nếu tự xóa chỉ để mean đẹp hơn thì báo cáo không còn trung thực. Dùng cả hai chỉ số và nêu mục tiêu khi chọn số đại diện.

Quy trình:

1. Ghi đủ 12 hóa đơn, giữ cả đơn lớn.
2. Tính mean và median, đối chiếu sự khác biệt.
3. In số lượng đơn và giá trị lớn nhất.

No Note — Không chú thích

```
import numpy as np
amounts = np.array([100,105,110,110,115,120,125,130,130,135,140,1000], dtype=float)
if len(amounts) != 12:
    raise ValueError("Cần đủ 12 hóa đơn.")
print("Hóa đơn:", len(amounts))
print(f"Mean (nghìn đồng): {amounts.mean():.2f}")
print(f"Median (nghìn đồng): {np.median(amounts):.2f}")
print("Đơn lớn nhất:", int(amounts.max()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So mean và median của 12 hóa đơn khi có một hóa đơn cực lớn.
# Dữ liệu: 12 hóa đơn nghìn đồng, trong đó 1000 là khoản lớn bất thường so 100..145.
# NumPy mean() và median() đo hai cách đại diện cho vị trí trung tâm.
import numpy as np
# Tạo vector 12 hóa đơn float, mỗi ô là một giá trị tiền nghìn đồng.
amounts = np.array([100,105,110,110,115,120,125,130,130,135,140,1000], dtype=float)
# len() đếm 12 hóa đơn, không phải tổng giá trị.
if len(amounts) != 12:
    # Dừng khi số hồ sơ không đúng kỳ vọng để tránh diễn giải sai.
    raise ValueError("Cần đủ 12 hóa đơn.")
# len() đếm 12 hóa đơn, không phải tổng giá trị.
print("Hóa đơn:", len(amounts))
# mean() cộng tiền chia 12, bị kéo lên bởi hóa đơn 1000; :.2f in hai chữ số lẻ.
print(f"Mean (nghìn đồng): {amounts.mean():.2f}")
# median() lấy điểm giữa sau sắp xếp, ít nhạy với một hóa đơn ngoại lệ.
print(f"Median (nghìn đồng): {np.median(amounts):.2f}")
# max() nhận giá trị lớn nhất của 12 hóa đơn; int() đổi sang số nguyên Python.
print("Đơn lớn nhất:", int(amounts.max()))
```

Kết quả khi chạy:

```
Hóa đơn: 12
Mean (nghìn đồng): 193.33
Median (nghìn đồng): 122.50
Đơn lớn nhất: 1000
```

Median là trung bình của hai giá trị giữa vì có số hóa đơn chẵn; không phải trung bình 12 hóa đơn.

Tình huống 3 — Cấp 3: Tìm mode của 15 lượt chọn sản phẩm

Từ vựng trong mục: Mode — giá trị phổ biến nhất; **Frequency** — số lần xuất hiện.

Tình huống thực tế: Quầy văn phòng phẩm ghi **15 lượt chọn nhóm hàng**, như Bút, Vở, Thước, Tẩy; mỗi lượt là **một lựa chọn** chứ không phải giá tiền. Nhân viên muốn biết nhóm nào được chọn nhiều nhất và

có bao nhiêu lượt của nhóm đó, để chuẩn bị hàng lên kệ. Họ không thể cộng chữ 'Bút' với chữ 'Vở' rồi tính trung bình số học.

Vì sao chọn cách này? Mode (mốt) là **giá trị hoặc nhãn xuất hiện nhiều nhất**. Vì đây là dữ liệu **phân loại** (tên nhóm hàng), `pd.Series(...).value_counts()` đếm số lượt theo từng nhãn; dòng có số đếm cao nhất cho biết nhóm được chọn nhiều nhất. **Mean** và phương sai đòi dữ liệu số có ý nghĩa tính toán, không phù hợp với chữ 'Bút' hay 'Tẩy'. Một nhãn được chọn nhiều nhất trong đúng **15 lượt mẫu** chưa đủ chứng minh nó luôn bán chạy nhất trong mọi ca.

Quy trình:

1. Ghi 15 nhãn sản phẩm theo thứ tự lượt.
2. Đếm tần suất bằng Series.
3. Sắp tần suất giảm và in nhãn nhiều nhất.

No Note — Không chú thích

```
import pandas as pd
categories =
["Bút","Vở","Bút","Thước","Bút","Tẩy","Vở","Bút","Thước","Vở","Bút","Vở","Tẩy","Bút","Thước"]
counts = pd.Series(categories).value_counts()
if len(categories) != 15:
    raise ValueError("Cần đủ 15 lượt chọn sản phẩm.")
print("Lượt chọn:", len(categories))
print("Nhãn khác nhau:", len(counts))
print("Mode:", counts.index[0])
print("Số lượt mode:", int(counts.iloc[0]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng mode để tìm loại mặt hàng được chọn phổ biến nhất trong 15 lượt.
# Dữ liệu: 15 lựa chọn Bút/Vở/Thước/Tẩy; mỗi lượt là một phiếu chọn, có nhãn lặp.
# Pandas Series chứa dãy nhãn và value_counts() đếm từng loại.
import pandas as pd
# List 15 tên sản phẩm, nhãn trùng có nghĩa nhiều lượt chọn.
categories =
["Bút","Vở","Bút","Thước","Bút","Tẩy","Vở","Bút","Thước","Vở","Bút","Vở","Tẩy","Bút","Thước"]
# Series() đổi list thành chuỗi Pandas; value_counts() đếm số lần mỗi nhãn, mặc định xếp theo tần
suất GIẢM DẦN.
counts = pd.Series(categories).value_counts()
# len() đếm 15 phiếu chọn; khác len(counts) là số nhãn KHÁC NHAU.
if len(categories) != 15:
    # Báo lỗi khi chưa đủ 15 phiếu chọn.
    raise ValueError("Cần đủ 15 lượt chọn sản phẩm.")
# len() đếm 15 phiếu chọn; khác len(counts) là số nhãn KHÁC NHAU.
print("Lượt chọn:", len(categories))
# Đếm loại sản phẩm đã xuất hiện, không phải tổng lượt.
print("Nhãn khác nhau:", len(counts))
# index là danh sách nhãn của Series đếm, [0] lấy nhãn tần suất cao nhất (mode), không phải giá trị
số.
print("Mode:", counts.index[0])
# iloc[0] lấy GIÁ TRỊ SỐ ĐẾM đầu tiên theo vị trí; int() chuyển sang số nguyên Python.
print("Số lượt mode:", int(counts.iloc[0]))
```

Kết quả khi chạy:

```
Lượt chọn: 15
Nhãn khác nhau: 4
```

```
Mode: Bút
Sô´lượt mode: 6
```

Mode cho biết nhãn thường thấy nhất **trong 15 lượt**, không kết luận sản phẩm ấy sẽ được chọn nhiều nhất trong mọi ca.

Tình huống 4 — Cấp 4: Phân biệt phương sai mẫu và quần thể trên 18 nhiệt độ

Từ vựng trong mục: Variance — phương sai; **Standard deviation** — độ lệch chuẩn; **ddof** — hiệu chỉnh mẫu.

Tình huống thực tế: Phòng máy có **18 lượt đo nhiệt độ**, tính bằng **°C**. Kỹ thuật viên muốn biết nhiệt độ **dao động quanh trung bình** nhiều hay ít. Họ muốn so hai cách tính: nếu 18 lượt là **toàn bộ dữ liệu của ca này** và nếu chúng chỉ là **mẫu để ước lượng các ca tương tự**, đồng thời đổi kết quả dao động về đơn vị °C để dễ đọc.

Vì sao chọn cách này? Phương sai mô tả mức các số đo phân tán quanh trung bình, bằng cách dùng **độ lệch đã bình phương**; vì nhiệt độ tính bằng °C nên phương sai có đơn vị **°C²**. **Độ lệch chuẩn** là căn của phương sai để trở về **°C**, dễ so với chênh lệch nhiệt độ trong thực tế hơn. `ddof=0` dùng mẫu số theo **18 lượt như toàn bộ nhóm đang xét**; `ddof=1` dùng **hiệu chỉnh khi 18 lượt là mẫu ước lượng phương sai quần thể**, nên với cùng dữ liệu thường cho phương sai lớn hơn một chút. Chọn `ddof` theo câu hỏi thống kê, không tùy chọn chỉ để ra con số mong muốn.

Quy trình:

1. Lưu 18 nhiệt độ và ghi rõ đơn vị °C.
2. Tính phương sai với hai quy ước ddof=0 và ddof=1.
3. Tính std mẫu và kiểm phương sai mẫu lớn hơn phương sai quần thể.

No Note — Không chú thích

```
import numpy as np
celsius = np.array([18,19,20,21,22,23,24,25,26,27,18,19,20,21,22,23,24,25], dtype=float)
pop_var = np.var(celsius, ddof=0)
sample_var = np.var(celsius, ddof=1)
if len(celsius) != 18:
    raise ValueError("Cần đủ 18 lượt đo nhiệt độ.")
print("Lượt đo:", len(celsius))
print(f"Phương sai quần thể (°C²): {pop_var:.3f}")
print(f"Phương sai mẫu (°C²): {sample_var:.3f}")
print(f"Std mẫu (°C): {np.std(celsius, ddof=1):.3f}")
print("Hiệu chỉnh làm tăng phương sai:", bool(sample_var > pop_var))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So phương sai quần thể và phương sai mẫu của 18 nhiệt độ, giải thích ddof.
# Dữ liệu: 18 số đo °C; ddof=0 chia N, ddof=1 chia N−1 vì ước lượng từ mẫu.
# NumPy cung cấp var() và std() với tùy chọn ddof (bậc tự do trừ đi).
import numpy as np
# Tạo vector 18 nhiệt độ kiểu float, đơn vị °C.
celsius = np.array([18,19,20,21,22,23,24,25,26,27,18,19,20,21,22,23,24,25], dtype=float)
# ddof=0 chia tổng bình phương sai lệch cho N=18, phù hợp phương sai quần thể quan sát.
pop_var = np.var(celsius, ddof=0)
# ddof=1 chia N−1=17, là phương sai mẫu điều chỉnh nên thường lớn hơn khi dữ liệu biến thiên.
sample_var = np.var(celsius, ddof=1)
# len() kiểm đủ 18 lượt đo.
if len(celsius) != 18:
```

```
    # Báo dữ liệu nhiệt độ thiếu/không đúng số´lượt.
    raise ValueError("Cần đủ 18 lượt đo nhiệt độ.")
# len() kiểm đủ 18 lượt đo.
print("Lượt đo:", len(celsius))
print(f"Phương sai quần thể (°C²): {pop_var:.3f}")
print(f"Phương sai mẫũ (°C²): {sample_var:.3f}")
# std() lấy căn của phương sai mẫũ ddof=1, trở lại đơn vị °C (khác phương sai °C²).
print(f"Std mẫũ (°C): {np.std(celsius, ddof=1):.3f}")
# So hai giá trị; hiệu chỉnh ddof=1 tăng ước lượng so ddof=0 với N>1 và phương sai dương.
# bool() đổi kết quả so sánh thành True/False Python khi in.
print("Hiệu chỉnh làm tăng phương sai:", bool(sample_var > pop_var))
```

Kết quả khi chạy:

```
Lượt đo: 18
Phương sai quần thể (°C²): 7.164
Phương sai mẫũ (°C²): 7.585
Std mẫũ (°C): 2.754
Hiệu chỉnh làm tăng phương sai: True
```

Khác biệt do **quy ước đang thống kê toàn bộ hay ước lượng từ mẫu**; bản thân 18 số đo không đổi.

Tình huống 5 — Cấp 5: Báo cáo mô tả 20 thời gian giao có một bản ghi thiếu

Từ vựng trong mục: Missingness — dữ liệu thiếu; **Describe** — bảng mô tả; **Median** — trung vị.

Tình huống thực tế: Đơn vị vận chuyển thu **20 phiếu thời gian giao**, đơn vị **phút**, nhưng **phiếu thứ 20** chưa được điền nên lưu `None`. Người quản lý cần giữ đủ **20 phiếu gốc**, đếm được **19 phiếu có thời gian** và **1 phiếu thiếu**, rồi tính trung bình, trung vị và độ lệch chuẩn từ đúng 19 phiếu có số. Phiếu thiếu không có nghĩa người giao mất 0 phút.

Vì sao chọn cách này? `None` hoặc **NaN** nghĩa là **chưa biết thời gian**, khác 0 phút là một số đo cụ thể. `pd.Series(raw, dtype='float64')` giữ được cả những giá trị có số và ô thiếu; `dropna()` tạo danh sách **19 giá trị hợp lệ** để tính, còn `isna()` đếm **một phiếu chưa ghi**. `mean()` tính thời gian bình quân, `median()` là mức giữa và `std(ddof=1)` cho độ lệch chuẩn mẫu, tất cả đều dựa trên dữ liệu có số. Không xóa phiếu thiếu khỏi báo cáo gốc hoặc thay bằng 0, vì như vậy có thể làm thời gian trung bình thấp đi và giấu mất việc cần bổ sung dữ liệu.

Quy trình:

1. Tạo 20 dòng và đổi giá trị thiếu thành NaN.
2. Đếm missing trước khi tính và lấy danh sách có số.
3. Tính mean, median, sample std có đơn vị.

No Note — Không chú thích

```
import pandas as pd
raw = [20,21,22,22,23,24,24,25,26,27,21,22,23,24,25,26,27,28,29,None]
data = pd.Series(raw, dtype="float64")
valid = data.dropna()
if len(data) != 20:
    raise ValueError("Cần đủ 20 phiếu ghi thời gian, kể cả lượt thiếu.")
print("Phiếu:", len(data))
print("Đủ thời gian:", len(valid))
print("Thiếu:", int(data.isna().sum()))
print(f"Mean (phút): {valid.mean():.2f}")
```

```
print(f"Median (phút): {valid.median():.2f}")
print(f"Std mẫu (phút): {valid.std(ddof=1):.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Làm sạch một thời gian thiếú rồi tính mean, median và độ lệch chuẩn mẫũ của 19 lượt hợp
lệ.
# Dữ liệu: 20 phiếú thời gian tính phút; một None là lượt thiếú, không phải thời gian 0.
# Pandas Series hỗ˜trợ missing values, dropna và thống kê có ddof.
import pandas as pd
# 20 lượt, có một None đại diện chưa ghi thời gian.
raw = [20,21,22,22,23,24,24,25,26,27,21,22,23,24,25,26,27,28,29,None]
# Series() tạo dãy sôʹ, dtype='float64' cho phép lưu NaN bên cạnh sôʹthực.
data = pd.Series(raw, dtype="float64")
# dropna() trả Series chỉ gôm̀ 19 lượt có sôʹ, mặc định KHÔNG sưả data gôć và không đôỉ None thành 0.
valid = data.dropna()
# len() vâñ bằng 20 phiếú gôm̀ cả lượt thiếú, khác len(valid)=19.
if len(data) != 20:
    # Dừng nêú sôʹphiếú gôć khác 20, không nhâm̀ sôʹphiếú với sôʹhợp lệ.
    raise ValueError("Cần đủ 20 phiếú ghi thời gian, kểʾcả lượt thiếú.")
# len() vâñ bằng 20 phiếú gôm̀ cả lượt thiếú, khác len(valid)=19.
print("Phiếú:", len(data))
print("Đủ thời gian:", len(valid))
# isna() đánh dâú ô thiếú True, sum() đêḿ sôʹphiếú thiếú; int() chuyêñ sôʹvêˋPython.
print("Thiếú:", int(data.isna().sum()))
# mean() lâý thời gian trung bình trong 19 lượt hợp lệ, đơn vị phút.
print(f"Mean (phút): {valid.mean():.2f}")
# median() tìm thời gian ở giữa dãy hợp lệ đã xêṕ.
print(f"Median (phút): {valid.median():.2f}")
# std(ddof=1) tính độ lệch chuẩn MẪU, chia N−1 trong phương sai, đơn vị phút; :.2f chỉ định dạng
khi in.
print(f"Std mẫu (phút): {valid.std(ddof=1):.2f}")
```

Kết quả khi chạy:

```
Phiếú: 20
Đủ thời gian: 19
Thiếú: 1
Mean (phút): 24.16
Median (phút): 24.00
Std mẫu (phút): 2.54
```

Trung bình mô tả 19 phiếu có số, **không đại diện** cho phiếu thiếu nếu lý do thiếu liên quan đến thời gian giao.

6. Sai lầm thường gặp

- Mean bị ảnh hưởng bởi ngoại lệ cực lớn; median thường bền hơn nhưng không phát hiện lỗi thay cho việc kiểm tra dữ liệu.
- `ddof=0` chia cho n; `ddof=1` chia cho n−1 và cần ít nhất hai quan sát để có ước lượng mẫu.
- Variance mang đơn vị bình phương; std mang đơn vị gốc.
- Mode/value_counts dành cho nhãn phân loại; không lấy trung bình của mã chữ.
- Đừng diễn giải std của quan sát thành sai số chuẩn của trung bình mẫu.
- Đổi dữ liệu về cùng đơn vị trước khi tính thống kê.

7. Trắc nghiệm

Q1. What does mean calculate? | Mean tính gì?

A. Squared dispersion around the mean | Độ phân tán bình phương quanh mean

B. Middle ranked value after sorting | Giá trị ở giữa sau khi sắp

C. Spread in original measurement units | Độ phân tán cùng đơn vị gốc

D. Sum divided by observation count | Tổng chia số quan sát

Đáp án: D

Giải thích: Mọi quan sát góp vào trung bình.

Q2. What is median? | Median là gì?

A. Spread in original measurement units | Độ phân tán cùng đơn vị gốc

B. Sum divided by observation count | Tổng chia số quan sát

C. Squared dispersion around the mean | Độ phân tán bình phương quanh mean

D. Middle ranked value after sorting | Giá trị ở giữa sau khi sắp

Đáp án: D

Giải thích: Trung vị dựa trên thứ tự, không dựa trên tổng độ lớn.

Q3. What does variance measure? | Variance đo gì?

A. Middle ranked value after sorting | Giá trị ở giữa sau khi sắp

B. Spread in original measurement units | Độ phân tán cùng đơn vị gốc

C. Sum divided by observation count | Tổng chia số quan sát

D. Squared dispersion around the mean | Độ phân tán bình phương quanh mean

Đáp án: D

Giải thích: Nó lấy trung bình bình phương độ lệch.

Q4. What does standard deviation measure? | Std đo gì?

A. Middle ranked value after sorting | Giá trị ở giữa sau khi sắp

B. Squared dispersion around the mean | Độ phân tán bình phương quanh mean

C. Sum divided by observation count | Tổng chia số quan sát

D. Spread in original measurement units | Độ phân tán cùng đơn vị gốc

Đáp án: D

Giải thích: Std là căn bậc hai phương sai.

Q5. What is the median of [1, 2, 3, 10, 10]? | Trung vị của [1, 2, 3, 10, 10] bằng bao nhiêu?

A. 3 | 3

B. 10 | 10

C. 5 | 5

D. 2 | 2

Đáp án: A

Giải thích: Dữ liệu đã tăng dần, có 5 số nên trung vị là số thứ ba: 3.

Q6. A score is 80, the mean is 60, and standard deviation is 10. What is the z-score? | Điểm số là 80, trung bình 60, độ lệch chuẩn 10. Điểm z là bao nhiêu?

A. 4 | 4

B. 2 | 2

C. 1 | 1

D. 20 | 20

Đáp án: B

Giải thích: z = (80 − 60)/10 = 2, nghĩa là cao hơn trung bình hai độ lệch chuẩn.

Q7. What is the mean of [2, 4, 9]? | Giá trị trung bình của [2, 4, 9] là bao nhiêu?

A. 5 | 5

B. 15 | 15

C. 6 | 6

D. 4 | 4

Đáp án: A

Giải thích: Cộng các giá trị 2 + 4 + 9 = 15 rồi chia cho 3 quan sát: mean = 5.

Q8. When is the population standard deviation equal to zero? | Khi nào độ lệch chuẩn của tổng thể bằng 0?

A. When the mean is zero | Khi trung bình bằng 0

B. When the sample is large | Khi mẫu lớn

C. When all values are positive | Khi tất cả số đều dương

D. When all observations are equal | Khi tất cả quan sát bằng nhau

Đáp án: D

Giải thích: Nếu tất cả giá trị giống nhau, không có độ lệch khỏi trung bình, nên phương sai và độ lệch chuẩn đều bằng 0.

Q9. What is mean of 2,3,4? | Mean của 2,3,4 là bao nhiêu?

A. 2/3 | 2/3

B. 4 | 4

C. 1 | 1

D. 3 | 3

Đáp án: D

Giải thích: Tổng 9 chia 3 bằng 3.

Q10. What is median of 2,4,100? | Median của 2,4,100 là bao nhiêu?

A. 1 | 1

B. 4 | 4

C. 2/3 | 2/3

D. 3 | 3

Đáp án: B

Giải thích: Bốn nằm giữa dãy đã sắp.

Q11. What is population variance of 1,2,3? | Phương sai quần thể của 1,2,3 là gì?

A. 3 | 3

B. 2/3 | 2/3

C. 4 | 4

D. 1 | 1

Đáp án: B

Giải thích: Bình phương lệch là 1,0,1 rồi chia ba.

Q12. What is sample variance of 1,2,3? | Phương sai mẫu của 1,2,3 là gì?

A. 4 | 4

B. 2/3 | 2/3

C. 1 | 1

D. 3 | 3

Đáp án: C

Giải thích: Tổng bình phương 2 chia $n-1=2$.

Q13. What unit does variance have? | Variance có đơn vị gì?

A. Original input units | Đơn vị gốc của dữ liệu

B. Median | Trung vị

C. Squared input units | Bình phương đơn vị dữ liệu

D. Mode | Yếu vị

Đáp án: C

Giải thích: Bình phương lệch làm bình phương đơn vị.

Q14. What unit does std have? | Std có đơn vị gì?

A. Median | Trung vị

B. Original input units | Đơn vị gốc của dữ liệu

C. Mode | Yếu vị

D. Squared input units | Bình phương đơn vị dữ liệu

Đáp án: B

Giải thích: Lấy căn đưa về đơn vị ban đầu.

Q15. Which statistic resists large outliers better? | Thống kê nào ít nhạy với ngoại lệ cực lớn hơn?

A. Squared input units | Bình phương đơn vị dữ liệu

B. Original input units | Đơn vị gốc của dữ liệu

C. Median | Trung vị

D. Mode | Yếu vị

Đáp án: C

Giải thích: Giá trị cực lớn ít tác động vị trí giữa.

Q16. Which statistic is useful for categorical labels? | Thống kê nào phù hợp nhãn phân loại?

A. Original input units | Đơn vị gốc của dữ liệu

B. Mode | Yếu vị

C. Squared input units | Bình phương đơn vị dữ liệu

D. Median | Trung vị

Đáp án: B

Giải thích: Lấy trung bình mã chữ không có ý nghĩa số học.

Q17. Does std equal standard error? | Std có giống sai số chuẩn không?

A. Summary statistics can lose meaning | Thống kê tổng hợp có thể mất ý nghĩa

B. No, std measures spread of individual values | Không, std đo độ phân tán quan sát

C. Different denominators give different results | Mẫu số khác nhau cho kết quả khác

D. Yes, large outliers can pull it up | Có, ngoại lệ lớn kéo mean lên

Đáp án: B

Giải thích: SE đo độ biến thiên của thống kê ước lượng.

Q18. Can mean lie above most observations? | Mean có thể lớn hơn phần lớn quan sát không?

A. No, std measures spread of individual values | Không, std đo độ phân tán quan sát

B. Yes, large outliers can pull it up | Có, ngoại lệ lớn kéo mean lên

C. Summary statistics can lose meaning | Thống kê tổng hợp có thể mất ý nghĩa

D. Different denominators give different results | Mẫu số khác nhau cho kết quả khác

Đáp án: B

Giải thích: Mean phụ thuộc tổng các giá trị.

Q19. Why state ddof when reporting variance? | Vì sao báo ddof khi trình bày phương sai?

A. Yes, large outliers can pull it up | Có, ngoại lệ lớn kéo mean lên

B. Different denominators give different results | Mẫu số khác nhau cho kết quả khác

C. Summary statistics can lose meaning | Thống kê tổng hợp có thể mất ý nghĩa

D. No, std measures spread of individual values | Không, std đo độ phân tán quan sát

Đáp án: B

Giải thích: Quy ước mẫu và quần thể khác nhau.

Q20. What happens if values use mismatched units? | Nếu số liệu dùng đơn vị không thống nhất thì sao?

A. Yes, large outliers can pull it up | Có, ngoại lệ lớn kéo mean lên

B. Different denominators give different results | Mẫu số khác nhau cho kết quả khác

C. No, std measures spread of individual values | Không, std đo độ phân tán quan sát

D. Summary statistics can lose meaning | Thống kê tổng hợp có thể mất ý nghĩa

Đáp án: D

Giải thích: Cần thống nhất đơn vị trước khi tính.

8. Liên hệ bài tiếp theo

`NumPy/Pandas → Thống kê mô tả → Outlier → ML.`

PY-38 — Quantile, IQR, outlier, skewness và kurtosis

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Quantile — phân vị; **IQR** — khoảng tứ phân vị.

`Thống kê mô tả → Quantile/IQR → Outlier → Kiểm tra dữ liệu ML`

Học xong dùng để làm gì? Dùng Q1/Q3/IQR phát hiện quan sát nằm xa phân bố, phân biệt ngoại lệ thật và nhập liệu sai.

Kết nối kiến thức tiếp theo: PY-39 xem kết quả thống kê thay đổi như thế nào khi lấy mẫu nhiều lần.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Quantile	phân vị	Giá trị chia phân phối theo tỷ lệ tích lũy
IQR	khoảng tứ phân vị	Khoảng cách Q3-Q1, mô tả 50% dữ liệu giữa
Skewness	độ lệch phân phối	Mức bất đối xứng của phân phối quanh tâm
Kurtosis	độ nhọn/đuôi phân phối	Chỉ số mô tả đặc điểm đuôi so với chuẩn theo quy ước

3. Thư viện và hàm Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cách viết	Khi nào dùng, ý nghĩa tham số
`np.quantile(x,[.25,.5,.75])`	Tìm các phân vị 25%, 50%, 75%
`Q1 - 1.5*IQR` và `Q3 + 1.5*IQR`	Ngưỡng dưới và trên để gắn cờ giá trị cần rà soát
`pd.Series.skew()`	Đánh giá bất đối xứng, nhạy với cỡ mẫu
`pd.Series.kurt()`	Excess kurtosis theo quy ước Pandas

4. Khái niệm chính và liên quan

Mạch kiến thức: Phân vị cho biết vị trí của một giá trị trong dãy đã sắp; IQR đo độ rộng phần giữa để sàng lọc ngoại lệ; skewness nói về bất đối xứng và kurtosis liên quan đặc điểm đuôi. Bất thường không đồng nghĩa sai dữ liệu.

4.1. Quantile — Mốc chia thứ tự dữ liệu

Từ vựng: Quantile — phân vị; **Q1/Q2/Q3** — tứ phân vị.

Phân vị q là mốc trên dãy số đã sắp, liên quan đến tỷ lệ giá trị ở phía dưới. **Q1** ứng với mức 25%, **Q2** là trung vị 50%, **Q3** ứng với 75%. Ví dụ năm hóa đơn 10,20,30,40,50 nghìn đồng: `np.quantile(x,0.75)` trả 40. Với bộ dữ liệu nhỏ, phương pháp **nội suy** giữa vị trí của các số làm cho kết quả phụ thuộc quy ước từng thư viện; không phải lúc nào phân vị cũng là số gốc trong dữ liệu.

No Note — Không chú thích

```
import numpy as np

amounts = [10, 20, 30, 40, 50]
print(float(np.quantile(amounts, 0.75)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy phân vị 75% để biết ngưỡng mà khoảng ba phần tư số tiền nằm dưới.
# Dữ liệu: Năm khoản tiền 10,20,30,40,50 cùng đơn vị, mốc q=0.75 là phân vị thứ ba.
# NumPy hỗ trợ tính phân vị qua quantile().
import numpy as np

# List 5 khoản tiền, không phải các giá trị xác suất.
amounts = [10, 20, 30, 40, 50]
# quantile(data,q) lấy giá trị tại phân vị q=0.75 (Q3), theo phương pháp nội suy tuyến tính mặc
định khi cần; 75% là vị trí dữ liệu, không phải nhân tổng với 0.75.
# float() đưa scalar NumPy về số Python để in.
print(float(np.quantile(amounts, 0.75)))
```

Kết quả khi chạy:

```
40.0
```

Mốc phân vị 75% bằng 40 nghìn đồng theo phương pháp nội suy mặc định NumPy.

4.2. IQR — Độ rộng nửa giữa bộ dữ liệu

Từ vựng: IQR — khoảng tứ phân vị; **Q1/Q3** — phân vị 25%/75%.

IQR = Q3−Q1 đo độ rộng của khoảng chứa phần giữa dữ liệu. Với `[1,2,3,4,5]`, Q1=2 và Q3=4, nên IQR=2 đơn vị. Một cách **gắn cờ** ngoại lệ là ngoài các ngưỡng `Q1−1.5×IQR` hoặc `Q3+1.5×IQR`. Đây chỉ là quy tắc rà soát, **không chứng minh dữ liệu sai**; chẳng hạn một hóa đơn lớn có thể hoàn toàn hợp lệ.

No Note — Không chú thích

```
import numpy as np

x = [1, 2, 3, 4, 5]
q1, q3 = np.quantile(x, [0.25, 0.75])
print(float(q3 - q1))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính IQR bằng khoảng cách giữa phân vị 25% và 75%.
# Dữ liệu: Năm giá trị đã sắp 1,2,3,4,5; IQR thể hiện độ rộng 50% số liệu ở giữa.
# NumPy quantile() cho phép lấy nhiều phân vị trong một lần gọi.
import numpy as np

# List 5 quan sát dạng số.
x = [1, 2, 3, 4, 5]
# Trả hai mốc Q1 (25%) và Q3 (75%) theo thứ tự yêu cầu; phép gán q1,q3 tách hai giá trị.
q1, q3 = np.quantile(x, [0.25, 0.75])
# IQR là khoảng rộng từ Q1 tới Q3, không phải max-min toàn bộ dữ liệu.
# float() chuyển scalar kết quả thành float Python.
print(float(q3 - q1))
```

Kết quả khi chạy:

```
2.0
```

Nửa giữa của dữ liệu trải rộng 2 đơn vị theo quy tắc phân vị đã chọn.

4.3. Skewness — Độ bất đối xứng của phân phối

Từ vựng: Skewness — độ lệch; **Right tail** — đuôi phải.

Skewness đo mức bất đối xứng. Một dãy như `[1,1,2,8]` có ba số nhỏ và một số lớn, khiến **đuôi phải kéo dài** và độ lệch dương. `pd.Series.skew()` trả hệ số skew đã được hiệu chỉnh theo công thức Pandas; tính trên mẫu ít quan sát rất kém ổn định. Không suy ra mọi giá trị lớn đều là lỗi từ skew dương.

No Note — Không chú thích

```
import pandas as pd

x = pd.Series([1, 1, 2, 8])
print(round(float(x.skew()), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng skewness đánh giá phân phối có đuôi kéo về phía giá trị lớn.
# Dữ liệu: Bốn số 1,1,2,8, giá trị 8 nằm xa về phía phải.
# Pandas Series có phương thức skew() để đo độ lệch dạng phân phối.
import pandas as pd

# Tạo Series gồm bốn giá trị, trong đó 8 là giá trị cao rõ rệt.
x = pd.Series([1, 1, 2, 8])
# skew() trả hệ số lệch đã điều chỉnh theo kích thước mẫu của pandas: dương thường nghĩa đuôi PHẢI
dài hơn, âm là đuôi trái; không phải xác suất.
# float() chuẩn hóa scalar về Python, round(...,3) làm tròn để hiển thị.
print(round(float(x.skew()), 3))
```

Kết quả khi chạy:

```
1.887
```

Độ lệch dương khoảng 1,887 gợi ý đuôi phải dài hơn đuôi trái.

4.4. Kurtosis — Đặc điểm đuôi theo quy ước

Từ vựng: Kurtosis — độ nhọn/hệ số liên quan đến đuôi; **Excess kurtosis** — kurtosis dư.

Kurtosis liên quan đến các moment bậc bốn và độ nặng của đuôi phân phối; không nên hiểu đơn giản chỉ là **độ cao của đỉnh**. Pandas `Series.kurt()` trả **excess kurtosis**, lấy phân phối chuẩn làm mốc 0. Với dãy `[2,3,3,4,4,5,5,6,7,20]`, số 20 rất lớn có thể làm kurtosis dư dương. Trên mẫu nhỏ, chỉ số này biến động mạnh.

No Note — Không chú thích

```
import pandas as pd

x = pd.Series([2, 3, 3, 4, 4, 5, 5, 6, 7, 20])
print(round(float(x.kurt()), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng excess kurtosis của Pandas để mô tả độ nặng đuôi phân phối.
# Dữ liệu: Mười giá trị có đa số từ 2 đến 7 và một số 20 lớn bất thường.
# Pandas kurt() dùng quy ước excess kurtosis (phân phối chuẩn có mốc khoảng 0).
import pandas as pd

# Series lưu mười quan sát số thực/số nguyên.
x = pd.Series([2, 3, 3, 4, 4, 5, 5, 6, 7, 20])
# kurt() trả excess kurtosis hiệu chỉnh theo mẫu; giá trị cao gợi ý đuôi nặng/ngoại lệ, KHÔNG trực tiếp đếm số ngoại lệ.
# round(...,3) giới hạn số thập phân để dễ đọc, không sửa giá trị dữ liệu.
print(round(float(x.kurt()), 3))
```

Kết quả khi chạy:

```
7.824
```

Excess kurtosis dương lớn cho thấy mẫu này có hành vi đuôi bất thường so với dạng chuẩn.

5. Năm tình huống Quantile/IQR — từ phân vị đến phát hiện ngoại lệ theo nhóm

Tình huống 1 — Cấp 1: Tìm các tứ phân vị của 10 hóa đơn

Từ vựng trong mục: Quantile — phân vị; **Interpolation** — nội suy.

Tình huống thực tế: Cửa hàng lập bảng **10 hóa đơn**, mỗi đơn có giá trị tính bằng **nghìn đồng**, từ 10 đến 28 theo thứ tự tăng dần. Chủ cửa hàng muốn tìm ba **mốc chia dữ liệu** là Q1, trung vị Q2 và Q3 để hiểu nhóm hóa đơn nằm ở vùng giá thấp, giữa và cao, thay vì chỉ xem đơn rẻ nhất và đắt nhất.

Vì sao chọn cách này? Phân vị là các mốc dùng để nhìn vị trí của giá trị trong **danh sách đã sắp thứ tự**. **Q1** đại diện vùng khoảng một phần tư thấp, **median/Q2** chia dữ liệu thành nửa thấp và nửa cao, **Q3** đại diện vùng khoảng ba phần tư. `np.quantile(amounts, [0.25, 0.5, 0.75])` lấy ba mốc này từ **10 khoản tiền có cùng đơn vị**. Vì vị trí phân vị có thể nằm **giữa hai hóa đơn thật**, NumPy có thể **nội suy** ra một mức không xuất hiện nguyên vẹn trong dữ liệu; đây là mốc mô tả, không phải một hóa đơn bị bỏ sót.

Quy trình:

1. Ghi đúng 10 giá trị cùng đơn vị.
2. Tính đồng thời các phân vị 25%, 50%, 75% bằng NumPy.
3. In ba mốc và số đơn để đọc đúng ý nghĩa.

No Note — Không chú thích

```
import numpy as np
amounts = np.array([10,12,14,16,18,20,22,24,26,28], dtype=float)
q1, median, q3 = np.quantile(amounts, [0.25, 0.5, 0.75])
if len(amounts) != 10:
    raise ValueError("Cần đủ 10 hóa đơn.")
print("Hóa đơn:", len(amounts))
print(f"Q1 (nghìn đồng): {q1:.1f}")
print(f"Median (nghìn đồng): {median:.1f}")
print(f"Q3 (nghìn đồng): {q3:.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Báo Q1, median và Q3 của 10 hóa đơn theo cùng một lệnh quantile.
# Dữ liệu: 10 khoản từ 10 tới 28 nghìn đồng, phân vị mô tả mốc theo thứ tự tiền tăng.
# NumPy quantile() hỗ trợ nhiều mốc phân vị trong một lần.
import numpy as np
# Vector 10 số tiền, dtype=float giữ khả năng nội suy ra số lẻ.
amounts = np.array([10,12,14,16,18,20,22,24,26,28], dtype=float)
# Trả Q1, median/Q2, Q3 tương ứng tỷ lệ 25%,50%,75%; NumPy mặc định nội suy tuyến tính khi vị trí nằm giữa hai mẫu.
q1, median, q3 = np.quantile(amounts, [0.25, 0.5, 0.75])
# len() kiểm đủ 10 hóa đơn, không phải giá trị phân vị.
if len(amounts) != 10:
    # ValueError cảnh báo thiếu hồ sơ nguồn trước khi kết luận.
    raise ValueError("Cần đủ 10 hóa đơn.")
# len() kiểm đủ 10 hóa đơn, không phải giá trị phân vị.
print("Hóa đơn:", len(amounts))
# :.1f hiển thị Q1 theo nghìn đồng với một số lẻ.
print(f"Q1 (nghìn đồng): {q1:.1f}")
# Median là trung vị sau sắp xếp, không phải trung bình cộng.
print(f"Median (nghìn đồng): {median:.1f}")
# Q3 là mốc phân vị 75%, không phải giá lớn nhất.
print(f"Q3 (nghìn đồng): {q3:.1f}")
```

Kết quả khi chạy:

```
Hóa đơn: 10
Q1 (nghìn đồng): 14.5
Median (nghìn đồng): 19.0
Q3 (nghìn đồng): 23.5
```

Mốc Q1=14,5 nghìn đồng không phải một đơn giá gốc; đó là kết quả nội suy theo quy tắc mặc định của NumPy.

Tình huống 2 — Cấp 2: Dùng IQR rà soát hóa đơn 500 trong 12 đơn

Từ vựng trong mục: IQR — khoảng tứ phân vị; **Fence** — ngưỡng gắn cờ; **Outlier** — điểm bất thường.

Tình huống thực tế: Một quầy bán hàng có **12 hóa đơn** tính bằng **nghìn đồng**; **11 hóa đơn từ 100 đến 145**, còn một hóa đơn giá **500**. Nhân viên muốn tìm xem hóa đơn nào **nằm xa vùng mua hàng thường**

gặp theo quy tắc IQR để đưa vào danh sách kiểm tra chứng từ. Họ không được xóa hóa đơn 500 vì chưa có bằng chứng đó là số tiền sai.

Vì sao chọn cách này? IQR là độ rộng của **vùng dữ liệu từ Q1 tới Q3**, tức phần nằm giữa của bảng đã sắp xếp. Chương trình lấy Q1/Q3, rồi tạo hai **hàng rào kiểm tra** ở phía thấp và cao, cách mỗi đầu thêm **1,5 lần IQR**. Số nào ngoài hàng rào được **gắn cờ ngoại lệ**; với 12 hóa đơn này, khoản 500 nghìn đồng quá cao so với nhóm còn lại nên đáng kiểm tra. `np.quantile()` và phép lọc chỉ tìm **điểm khác thường theo quy tắc thống kê**, không chứng minh khoản 500 là gian lận hay sai dữ liệu. 1,5 là ngưỡng quy ước thường dùng, không phải luật kinh doanh.

Quy trình:

1. Tìm Q1 và Q3 của đúng 12 đơn.
2. Tạo ngưỡng dưới/trên từ khoảng tứ phân vị.
3. Tìm số đơn ngoài khoảng và giá trị cần xem xét.

No Note — Không chú thích

```
import numpy as np
amounts = np.array([100,105,110,115,120,120,125,130,135,140,145,500], dtype=float)
q1, q3 = np.quantile(amounts, [0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5*iqr, q3 + 1.5*iqr
flagged = amounts[(amounts < low) | (amounts > high)]
if amounts.size != 12:
    raise ValueError("Cần đủ 12 hóa đơn để tính IQR.")
print("Hóa đơn:", amounts.size)
print(f"IQR (nghìn đồng): {iqr:.2f}")
print(f"Ngưỡng trên: {high:.1f}")
print("Đơn cần xem:", flagged.astype(int).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phát hiện hóa đơn cao bất thường bằng hàng rào 1.5×IQR mà không xóa dữ liệu.
# Dữ liệu: 12 hóa đơn nghìn đồng, một khoản 500 rất cao so dãy 100..145.
# NumPy tính phân vị và lọc mảng theo điều kiện.
import numpy as np
# Vector 12 khoản tiền float, đơn vị nghìn đồng.
amounts = np.array([100,105,110,115,120,120,125,130,135,140,145,500], dtype=float)
# Lấy Q1/Q3 từ 12 giá trị gốc, chưa loại bất cứ hóa đơn nào.
q1, q3 = np.quantile(amounts, [0.25, 0.75])
# IQR=Q3−Q1 là độ rộng khoảng giữa, làm thang đo cho hàng rào ngoại lệ.
iqr = q3 - q1
# Hàng rào dưới Q1−1.5 IQR; ngưỡng tham khảo thống kê chứ không tự kết luận dữ liệu sai.
# Hàng rào trên Q3+1.5 IQR; 1.5 là quy tắc Tukey thường dùng, cần xem xét bối cảnh.
low, high = q1 - 1.5*iqr, q3 + 1.5*iqr
# So từng ô với hai ngưỡng, | là OR theo phần tử; mask True nếu nằm ngoài bất kỳ hàng rào nào.
# Boolean indexing tạo mảng các giá trị cần kiểm tra mà không sửa amounts gốc.
flagged = amounts[(amounts < low) | (amounts > high)]
# size đếm 12 khoản trong vector.
if amounts.size != 12:
    # Dừng nếu không đủ 12 hóa đơn để so sánh phân vị.
    raise ValueError("Cần đủ 12 hóa đơn để tính IQR.")
# size đếm 12 khoản trong vector.
print("Hóa đơn:", amounts.size)
# Định dạng IQR hai chữ số lẻ theo nghìn đồng.
print(f"IQR (nghìn đồng): {iqr:.2f}")
print(f"Ngưỡng trên: {high:.1f}")
# astype(int) chuyển số tiền sang int để dễ in, tolist() đổi ndarray thành list Python; không xóa
```

```
giá trị khỏi nguồn.
print("Đơn cần xem:", flagged.astype(int).tolist())
```

Kết quả khi chạy:

```
Hóa đơn: 12
IQR (nghìn đồng): 22.50
Ngưỡng trên: 170.0
Đơn cần xem: [500]
```

Một hóa đơn thật có thể vượt ngưỡng IQR hợp lệ; cần đối chiếu hóa đơn nguồn trước khi quyết định xử lý.

Tình huống 3 — Cấp 3: So mean, median và độ lệch phải trên 15 thời gian

Từ vựng trong mục: Skewness — độ lệch bất đối xứng; **Right tail** — đuôi phải.

Tình huống thực tế: Trung tâm chăm sóc đo **15 lượt thời gian phục vụ**, đơn vị **phút**; 14 lượt kéo dài khoảng **10–16 phút**, có một lượt **90 phút**. Quản lý muốn xem mức thời gian thông thường, tính trung bình so với trung vị và hiểu bảng thời gian có **đuôi kéo dài về phía các lượt rất lâu** hay không. Phiếu 90 phút có thể là ca phức tạp thật, không nhất thiết lỗi nhập.

Vì sao chọn cách này? Skewness (độ lệch) mô tả hình dạng phân phối có **đuôi kéo dài về bên cao hay bên thấp**. Khi 14 lượt từ 10–16 phút nhưng một lượt 90 phút, **mean** bị kéo lên mạnh hơn **median**, và độ lệch thường **dương** do đuôi kéo về phía thời gian cao. `pd.Series.skew()` đo đặc điểm bất đối xứng từ chính **15 lượt**; kiểm `> 0` giúp giải thích hướng của đuôi. Không dùng kết quả này để tự xóa phiếu 90 phút: mẫu nhỏ và một trường hợp rất lâu có thể là sự kiện thực.

Quy trình:

1. Tạo 15 số phút phục vụ.
2. Tính mean, median và skewness bằng Pandas.
3. So dấu skew và hai thước đo trung tâm.

No Note — Không chú thích

```
import pandas as pd
minutes = pd.Series([10,10,11,11,12,12,12,13,14,14,15,15,15,16,90], dtype=float)
if len(minutes) != 15:
    raise ValueError("Cần đủ 15 lượt đo thời gian.")
print("Lượt:", len(minutes))
print(f"Mean (phút): {minutes.mean():.2f}")
print(f"Median (phút): {minutes.median():.2f}")
print(f"Skew: {minutes.skew():.3f}")
print("Lệch phải:", bool(minutes.skew() > 0))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So mean và median cùng hệ số lệch phải trong 15 thời gian có ngoại lệ 90 phút.
# Dữ liệu: 15 lần xử lý, đa số 10..16 phút, một lượt 90 phút kéo dài đuôi phải.
# Pandas Series tính mean, median và skew trên dãy số.
import pandas as pd
# Tạo Series 15 thời gian dtype=float, mỗi lượt tính phút.
minutes = pd.Series([10,10,11,11,12,12,12,13,14,14,15,15,15,16,90], dtype=float)
# len() bảo đảm đủ 15 thời gian trước khi đánh giá.
if len(minutes) != 15:
    # Báo lỗi số lượt không đúng thay vì so bảng thiếu.
```

```
    raise ValueError("Cần đủ 15 lượt đo thời gian.")
# len() bảo đảm đủ 15 thời gian trước khi đánh giá.
print("Lượt:", len(minutes))
# mean() là trung bình cộng, nhạy với thời gian 90 phút.
print(f"Mean (phút): {minutes.mean():.2f}")
# median() lấy điểm giữa sau sắp xếp, ít chịu tác động của lượt 90 phút.
print(f"Median (phút): {minutes.median():.2f}")
# skew() là độ lệch đã điều chỉnh trong pandas; dương cho thấy đuôi phân phối kéo sang phía giá trị lớn.
print(f"Skew: {minutes.skew():.3f}")
# skew() là độ lệch đã điều chỉnh trong pandas; dương cho thấy đuôi phân phối kéo sang phía giá trị lớn.
# bool() biểu diễn kết luận skew dương; không tự suy rằng mọi giá trị lớn đều là sai.
print("Lệch phải:", bool(minutes.skew() > 0))
```

Kết quả khi chạy:

```
Lượt: 15
Mean (phút): 18.00
Median (phút): 13.00
Skew: 3.811
Lệch phải: True
```

Skew dương chỉ mô tả phân bố dữ liệu đang có; không kết luận lượt 90 phút là sai số đo.

Tình huống 4 — Cấp 4: Phân biệt độ lệch phải và kurtosis của 18 mẫu

Từ vựng trong mục: Kurtosis — mô tả đuôi; **Excess kurtosis** — kurtosis dư; **Skewness** — bất đối xứng.

Tình huống thực tế: Phòng thí nghiệm ghi **18 lần đo mức tín hiệu**, cùng một đơn vị chỉ số; 17 lần nằm khoảng **20–36**, một lần lên tới **120**. Kỹ thuật viên muốn biết dữ liệu có **bất đối xứng về phía giá trị cao** không, số đo cực lớn làm hình dạng phân phối thay đổi ra sao và cờ cảnh báo IQR có chỉ ra 120 để kiểm tra lại thiết bị hay không.

Vì sao chọn cách này? Skew cho biết phân phối lệch trái/phải, còn **excess kurtosis** đo đặc điểm liên quan đến **mức nhạy với các độ lệch lớn/đuôi cực trị** so với chuẩn tham chiếu; đây không đơn thuần là độ nhọn ở đỉnh biểu đồ. `pd.Series.kurt()` trả excess kurtosis, với mốc tham chiếu của phân phối chuẩn là **0**. Một giá trị 120 so với 17 giá trị quanh 20–36 ảnh hưởng mạnh đến cả hai thống kê. Chương trình đồng thời dùng **hàng rào IQR phía trên** để liệt kê số đo cần xem; phát hiện điểm lạ là **đầu mối kiểm tra**, không phải bằng chứng cảm biến đã hỏng.

Quy trình:

1. Tạo 18 mức tín hiệu, giữ lại số đo 120 để rà soát.
2. Tính skew và kurt của Pandas theo quy ước thư viện.
3. Gắn cờ bằng IQR để nhận biết điểm cần xác minh.

No Note — Không chú thích

```
import numpy as np
import pandas as pd
signal = pd.Series([20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,120], dtype=float)
q1, q3 = np.quantile(signal, [0.25, 0.75])
outliers = signal[signal > q3 + 1.5*(q3-q1)]
if len(signal) != 18:
    raise ValueError("Cần đủ 18 số đo tín hiệu.")
print("Mẫu:", len(signal))
print(f"Skew: {signal.skew():.3f}")
```

```
print(f"Excess kurtosis: {signal.kurt():.3f}")
print("Giá trị cần xem:", outliers.astype(int).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng IQR đánh dấu tín hiệu lớn và xem skew cùng excess kurtosis của 18 mẫu.
# Dữ liệu: 18 tín hiệu, phần lớn 20..36 nhưng có một giá trị 120 cao vượt cụm.
# NumPy quantile() lấy Q1, Q3 và tạo ngưỡng theo IQR.
import numpy as np
# Pandas Series skew()/kurt() mô tả độ lệch và đuôi.
import pandas as pd
# 18 số đo kiểu float, không xóa điểm 120 khỏi nguồn.
signal = pd.Series([20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,120], dtype=float)
# Tính Q1/Q3 từ Series như vector số và dùng nội suy phân vị mặc định.
q1, q3 = np.quantile(signal, [0.25, 0.75])
# So từng giá trị với hàng rào TRÊN bằng Q3+1.5 IQR; chỉ lọc ngoại lệ cao, chưa kiểm hàng rào dưới.
# Lập chỉ mục bằng mask Boolean tạo Series con các điểm cần xem, nguồn signal vẫn giữ nguyên.
outliers = signal[signal > q3 + 1.5*(q3-q1)]
# len() xác nhận đủ 18 phép đo.
if len(signal) != 18:
    # Ngừng khi không đủ dữ liệu để diễn giải.
    raise ValueError("Cần đủ 18 số đo tín hiệu.")
# len() xác nhận đủ 18 phép đo.
print("Mẫu:", len(signal))
# skew() >0 thường mô tả đuôi phải dài, giá trị 120 có thể làm tăng hệ số.
print(f"Skew: {signal.skew():.3f}")
# kurt() theo Pandas là excess kurtosis; điểm 120 làm đuôi có thể nặng, không phải tỷ lệ ngoại lệ.
print(f"Excess kurtosis: {signal.kurt():.3f}")
# astype(int) đổi định dạng giá trị khi in, tolist() chuyển Series thành list Python; không xóa
khỏi dữ liệu gốc.
print("Giá trị cần xem:", outliers.astype(int).tolist())
```

Kết quả khi chạy:

```
Mẫu: 18
Skew: 3.900
Excess kurtosis: 15.980
Giá trị cần xem: [120]
```

Chỉ số độ lệch và kurtosis nhạy với giá trị rất lớn, vì vậy cần kết hợp hiểu nguồn số đo chứ không chỉ gắn nhãn bằng một con số.

Tình huống 5 — Cấp 5: Phát hiện ngoại lệ riêng hai nhóm trên 20 sản phẩm có ô thiếu

Từ vựng trong mục: Group-wise IQR — IQR theo nhóm; **Missing data** — dữ liệu thiếu; **Data integrity** — tính toàn vẹn.

Tình huống thực tế: Kho chứa **20 sản phẩm P01–P20** thuộc hai loại A và B, mỗi loại 10 sản phẩm. **Khối lượng** đo bằng **gam**: nhóm A chủ yếu **50–57 g**, nhóm B chủ yếu **150–158 g**. P09 của nhóm A chưa cân, P10 nặng **110 g** và P20 nặng **220 g**. Nhân viên muốn biết số mẫu thiếu và **mã khối lượng bất thường so với chính nhóm của nó**, không so sản phẩm A với giá trị bình thường của nhóm B.

Vì sao chọn cách này? Mỗi loại sản phẩm có **mức khối lượng thông thường khác nhau**, nên phải tính **Q1, Q3 và IQR riêng cho nhóm A và B** bằng `groupby('group').dropna()` bỏ riêng ô khối lượng **chưa cân** khi tính mốc, còn `isna()` vẫn giúp báo rõ số mẫu thiếu. Trong mỗi nhóm, quy tắc **1,5 lần IQR** gắn cờ những mã ở quá xa vùng giữa, như P10 hoặc P20; **mã sản phẩm** được giữ để kiểm lại hàng thật. Nếu gộp

A và B thành một bảng trước khi tính ngưỡng, hai cụm khối lượng 50 g và 150 g có thể làm ngưỡng chung trở nên vô ích, che các trường hợp bất thường trong từng loại.

Quy trình:

1. Tạo 20 hàng, ghi đơn vị g và tên nhóm.
2. Với từng nhóm, loại NaN chỉ khi tính Q1/Q3 và IQR.
3. Lưu ID ngoài ngưỡng của từng nhóm, báo tổng hàng, thiếu và danh sách cần xem.

No Note — Không chú thích

```
import numpy as np
import pandas as pd
mass = [50,51,52,53,54,55,56,57,None,110,150,151,152,153,154,155,156,157,158,220]
ids = []
for i in range(1, 21):
    ids.append(f"P{i:02d}")
df = pd.DataFrame({"id": ids,
                   "group": ["A"]*10 + ["B"]*10, "mass_g": mass})
flagged = []
for group, subset in df.groupby("group"):
    valid = subset["mass_g"].dropna()
    q1, q3 = np.quantile(valid, [0.25, 0.75])
    low, high = q1-1.5*(q3-q1), q3+1.5*(q3-q1)
    flagged.extend(subset.loc[(subset["mass_g"] < low) | (subset["mass_g"] > high), "id"])
if len(df) != 20:
    raise ValueError("Cần đủ 20 mẫu khối lượng, kể cả lượt thiếu.")
print("Sản phẩm:", len(df))
print("Thiếu khối lượng:", int(df["mass_g"].isna().sum()))
print("Ngoại lệ theo nhóm:", ", ".join(sorted(flagged)))
print("Hàng còn có khối lượng:", int(df["mass_g"].notna().sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phát hiện ngoại lệ riêng từng nhóm A/B trong 20 sản phẩm, không nhầm quy mô hai nhóm.
# Dữ liệu: 20 sản phẩm P01..P20; khối lượng gram, một None là thiếu; A/B mỗi nhóm 10 sản phẩm với
phân phối khác nhau.
# NumPy lấy phân vị cho dữ liệu số còn hợp lệ của từng nhóm.
import numpy as np
# Pandas DataFrame hỗ trợ phân nhóm, dropna, loc và cột thiếu.
import pandas as pd
# 20 khối lượng gram, None biểu thị chưa đo, không được thay bằng 0 trước tính phân vị.
mass = [50,51,52,53,54,55,56,57,None,110,150,151,152,153,154,155,156,157,158,220]
ids = []
# range(1,21) sinh thứ tự 1..20, không lấy 21.
for i in range(1, 21):
    # f-string :02d thêm số 0 đầu mã P01..P20.
    # append() giữ mã theo cùng vị trí mỗi khối lượng.
    ids.append(f"P{i:02d}")
# Tạo 20 hàng id/group/mass_g; list nhóm A 10 lần rồi B 10 lần, mỗi hàng một sản phẩm.
df = pd.DataFrame({"id": ids,
                   # List multiplication tạo 10 nhãn A rồi 10 B, không phải phép nhân xác suất.
                   "group": ["A"]*10 + ["B"]*10, "mass_g": mass})
flagged = []
# groupby('group') tách bảng thành hai nhóm độc lập A/B trước khi tìm hàng rào, tránh áp một ngưỡng
chung sai bối cảnh.
for group, subset in df.groupby("group"):
    # dropna() loại tạm ô None/NaN CHỈ khi tính phân vị của nhóm, không xóa hàng gốc khỏi df.
    valid = subset["mass_g"].dropna()
    # Tính Q1/Q3 trên số đo hợp lệ riêng từng nhóm; mốc có thể khác nhau giữa A và B.
    q1, q3 = np.quantile(valid, [0.25, 0.75])
    # Hàng rào dưới Q1−1.5 IQR, trên Q3+1.5 IQR của nhóm hiện tại.
```

```
    low, high = q1-1.5*(q3-q1), q3+1.5*(q3-q1)
    # Hai điều kiện nối bằng | Boolean từng ô; NaN không so đạt ngưỡng nên không bị gắn nhãn ngoại
lệ theo phép này.
    # loc[mask,'id'] trả các mã ngoại lệ trong nhóm hiện tại, không dùng chỉ số hàng làm mã.
    # extend() thêm từng mã ngoại lệ vào list chung, không lồng cả Series thành một phần tử.
    flagged.extend(subset.loc[(subset["mass_g"] < low) | (subset["mass_g"] > high), "id"])
# len() kiểm đủ 20 sản phẩm, gồm cả mẫu thiếu.
if len(df) != 20:
    # Dừng nếu số lượng hồ sơ nguồn không bằng 20.
    raise ValueError("Cần đủ 20 mẫu khối lượng, kể cả lượt thiếu.")
# len() kiểm đủ 20 sản phẩm, gồm cả mẫu thiếu.
print("Sản phẩm:", len(df))
# isna() nhận các khối lượng thiếu, sum() đếm True; không đồng nghĩa ngoại lệ.
print("Thiếu khối lượng:", int(df["mass_g"].isna().sum()))
# sorted() sắp mã được cảnh báo, join() ghép chuỗi để in ổn định.
print("Ngoại lệ theo nhóm:", ", ".join(sorted(flagged)))
# notna() đánh dấu mẫu có khối lượng, sum() đếm còn bao nhiêu mẫu đo được.
print("Hàng còn có khối lượng:", int(df["mass_g"].notna().sum()))
```

Kết quả khi chạy:

```
Sản phẩm: 20
Thiếu khối lượng: 1
Ngoại lệ theo nhóm: P10, P20
Hàng còn có khối lượng: 19
```

Đây là **hai điểm cần xem lại theo nhóm**, chưa phải hai sản phẩm bị hỏng. Mẫu P09 thiếu khối lượng cần xử lý riêng trước khi ra quyết định.

6. Sai lầm thường gặp

- Q1 và Q3 phụ thuộc phương pháp nội suy, nhất là với mẫu nhỏ; cần giữ nhất quán giữa các thư viện.
- IQR dùng ngưỡng **gợi ý** ngoại lệ, không chứng minh giá trị là lỗi hay phải xóa.
- Skew dương nói về đuôi phải; skew âm nói về đuôi trái, không đồng nghĩa mean luôn lớn/nhỏ trong mọi dữ liệu.
- Kurtosis liên quan moment bậc bốn/đuôi, không chỉ độ cao đỉnh; Pandas trả **excess kurtosis**.
- Skew và kurt mẫu nhỏ không ổn định; cần xem đồ thị và nguồn dữ liệu.
- Khi điền khuyết cho ML, tính median trên train, không lấy thông tin test.

7. Trắc nghiệm

Q1. What is a quantile? | Phân vị là gì?

A. Fourth-moment tail behavior relative to normal | Đặc trưng moment bậc bốn/đuôi so với chuẩn

B. The difference Q3−Q1 | Hiệu Q3−Q1

C. A value associated with a cumulative rank | Giá trị gắn với một mức thứ hạng tích lũy

D. Distribution asymmetry | Mức bất đối xứng phân phối

Đáp án: C

Giải thích: Phân vị chia dữ liệu sắp thứ tự theo tỷ lệ.

Q2. What is IQR? | IQR là gì?

A. The difference Q3−Q1 | Hiệu Q3−Q1

B. A value associated with a cumulative rank | Giá trị gắn với một mức thứ hạng tích lũy

C. Fourth-moment tail behavior relative to normal | Đặc trưng moment bậc bốn/đuôi so với chuẩn

D. Distribution asymmetry | Mức bất đối xứng phân phối

Đáp án: A

Giải thích: Nó đo độ rộng nửa giữa dữ liệu.

Q3. What does skewness measure? | Skewness đo gì?

A. Fourth-moment tail behavior relative to normal | Đặc trưng moment bậc bốn/đuôi so với chuẩn

B. Distribution asymmetry | Mức bất đối xứng phân phối

C. A value associated with a cumulative rank | Giá trị gắn với một mức thứ hạng tích lũy

D. The difference Q3−Q1 | Hiệu Q3−Q1

Đáp án: B

Giải thích: Dấu gợi ý phía đuôi dài.

Q4. What does excess kurtosis measure? | Excess kurtosis mô tả gì?

A. A value associated with a cumulative rank | Giá trị gắn với một mức thứ hạng tích lũy

B. Distribution asymmetry | Mức bất đối xứng phân phối

C. The difference Q3−Q1 | Hiệu Q3−Q1

D. Fourth-moment tail behavior relative to normal | Đặc trưng moment bậc bốn/đuôi so với chuẩn

Đáp án: D

Giải thích: Pandas dùng mốc 0 cho chuẩn.

Q5. If Q1 = 4 and Q3 = 10, what is the interquartile range (IQR)? | Nếu Q1 = 4, Q3 = 10, khoảng tứ phân vị IQR là bao nhiêu?

A. 6 | 6

B. 4 | 4

C. 2.5 | 2.5

D. 14 | 14

Đáp án: A

Giải thích: IQR = Q3 − Q1 = 10 − 4 = 6.

Q6. If Q1 = 4 and Q3 = 10, what is the upper outlier fence using 1.5 × IQR? | Nếu Q1 = 4 và Q3 = 10, ngưỡng ngoại lệ trên theo quy tắc 1.5 × IQR bằng bao nhiêu?

A. 19 | 19

B. 15 | 15

C. 21 | 21

D. 16 | 16

Đáp án: A

Giải thích: IQR = 6, nên ngưỡng trên = Q3 + 1.5 × IQR = 10 + 9 = 19.

Q7. A value is flagged as an outlier by the IQR rule. What can we conclude? | Một giá trị bị đánh dấu ngoại lệ theo quy tắc IQR. Kết luận nào hợp lý?

A. It must equal the median | Nó bằng trung vị

B. It must be deleted | Bắt buộc phải xóa

C. It needs checking, but is not necessarily wrong | Nên kiểm tra, nhưng chưa chắc là dữ liệu sai

D. It proves a sensor failed | Chứng minh cảm biến bị lỗi

Đáp án: C

Giải thích: Quy tắc IQR chỉ đánh dấu giá trị nằm xa vùng giữa dữ liệu. Ngoại lệ có thể là giá trị thật.

Q8. In a typically right-skewed distribution, which comparison is often observed? | Trong phân phối lệch phải điển hình, mối quan hệ nào thường gặp?

A. Mean = median always | Trung bình luôn bằng trung vị

B. Mean < median always | Trung bình luôn nhỏ hơn trung vị

C. Mean > median often | Trung bình thường lớn hơn trung vị

D. Standard deviation is zero | Độ lệch chuẩn bằng 0

Đáp án: C

Giải thích: Đuôi kéo dài về bên phải thường làm trung bình bị kéo lên mạnh hơn trung vị; đây không phải quy tắc bắt buộc cho mọi bộ dữ liệu.

Q9. What does positive skew suggest? | Skew dương gợi ý gì?

A. A longer left tail | Đuôi trái dài hơn

B. Excess kurtosis | Kurtosis dư

C. A skewness statistic | Hệ số độ lệch phân phối

D. A longer right tail | Đuôi phải dài hơn

Đáp án: D

Giải thích: Giá trị lớn kéo đuôi sang phải.

Q10. What does negative skew suggest? | Skew âm gợi ý gì?

A. Excess kurtosis | Kurtosis dư

B. A longer right tail | Đuôi phải dài hơn

C. A skewness statistic | Hệ số độ lệch phân phối

D. A longer left tail | Đuôi trái dài hơn

Đáp án: D

Giải thích: Giá trị nhỏ kéo đuôi về trái.

Q11. What does Series.kurt() return? | Series.kurt() trả gì?

A. A longer right tail | Đuôi phải dài hơn

B. A longer left tail | Đuôi trái dài hơn

C. A skewness statistic | Hệ số độ lệch phân phối

D. Excess kurtosis | Kurtosis dư

Đáp án: D

Giải thích: Pandas dùng quy ước kurtosis dư.

Q12. What does Series.skew() return? | Series.skew() trả gì?

A. A skewness statistic | Hệ số độ lệch phân phối

B. Excess kurtosis | Kurtosis dư

C. A longer left tail | Đuôi trái dài hơn

D. A longer right tail | Đuôi phải dài hơn

Đáp án: A

Giải thích: Nó ước lượng bất đối xứng từ mẫu.

Q13. What is Q3−Q1 called? | Q3−Q1 gọi là gì?

A. 5 | 5

B. No; legitimate extremes may be flagged | Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

C. Interquartile range | Khoảng tứ phân vị

D. 3 | 3

Đáp án: C

Giải thích: Hiệu này chính là IQR.

Q14. What is median of 1,3,5? | Median của 1,3,5 bằng gì?

A. 3 | 3

B. No; legitimate extremes may be flagged | Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

C. Interquartile range | Khoảng tứ phân vị

D. 5 | 5

Đáp án: A

Giải thích: Giá trị đứng giữa là ba.

Q15. What is IQR if Q1=2 and Q3=7? | IQR nếu Q1=2, Q3=7 bằng bao nhiêu?

A. No; legitimate extremes may be flagged | Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

B. 5 | 5

C. Interquartile range | Khoảng tứ phân vị

D. 3 | 3

Đáp án: B

Giải thích: Bảy trừ hai bằng năm.

Q16. Does an IQR outlier flag prove a measurement error? | Gắn cờ outlier theo IQR có chứng minh số đo sai không?

A. Interquartile range | Khoảng tứ phân vị

B. 5 | 5

C. No; legitimate extremes may be flagged | Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

D. 3 | 3

Đáp án: C

Giải thích: Ngưỡng chỉ là quy tắc sàng lọc.

Q17. Why can quantile values differ between software packages? | Vì sao phân vị có thể khác giữa các phần mềm?

A. Fit the imputation value on train only | Tính giá trị điền chỉ từ train

B. Median | Trung vị

C. Different interpolation conventions | Quy ước nội suy khác nhau

D. It concerns fourth moments and tails | Nó liên quan moment bậc bốn và đuôi

Đáp án: C

Giải thích: Mẫu nhỏ dễ cho thấy sự khác biệt quy ước.

Q18. Why should you avoid interpreting high kurtosis only as a tall peak? | Vì sao không nên hiểu kurtosis cao chỉ là đỉnh cao?

A. Fit the imputation value on train only | Tính giá trị điền chỉ từ train

B. Different interpolation conventions | Quy ước nội suy khác nhau

C. It concerns fourth moments and tails | Nó liên quan moment bậc bốn và đuôi

D. Median | Trung vị

Đáp án: C

Giải thích: Chỉ độ cao đỉnh không đủ diễn giải.

Q19. What statistic is often robust for imputation with extreme values? | Thống kê nào thường phù hợp điền thiếu khi có ngoại lệ lớn?

A. Different interpolation conventions | Quy ước nội suy khác nhau

B. It concerns fourth moments and tails | Nó liên quan moment bậc bốn và đuôi

C. Fit the imputation value on train only | Tính giá trị điền chỉ từ train

D. Median | Trung vị

Đáp án: D

Giải thích: Thứ tự khiến trung vị ít nhạy với cực trị.

Q20. What is a crucial train/test rule when imputing missing values? | Quy tắc train/test quan trọng khi điền khuyết là gì?

A. It concerns fourth moments and tails | Nó liên quan moment bậc bốn và đuôi

B. Different interpolation conventions | Quy ước nội suy khác nhau

C. Median | Trung vị

D. Fit the imputation value on train only | Tính giá trị điền chỉ từ train

Đáp án: D

Giải thích: Dữ liệu test không tham gia học tham số tiền xử lý.

8. Liên hệ bài kế tiếp

`Thống kê mô tả → Quantile/IQR → Outlier → Kiểm tra dữ liệu ML.`

PY-39 — Lấy mẫu, CLT và sai số chuẩn

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Population — tổng thể; **Sample** — mẫu.

`Thống kê → Sampling → Central Limit Theorem → Confidence Interval`

Học xong dùng để làm gì? Ước lượng đặc trưng tổng thể từ mẫu, nhận biết sai số lấy mẫu và quy luật của trung bình mẫu.

Kết nối kiến thức tiếp theo: PY-40 dùng sai số đó để xây khoảng tin cậy và kiểm định giả thuyết.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Population	tổng thể	Toàn bộ đối tượng cần nghiên cứu
Sample	mẫu	Nhóm con lấy từ tổng thể theo cách rõ ràng
CLT	định lý giới hạn trung tâm	Phân phối trung bình mẫu tiến gần chuẩn theo điều kiện thích hợp
Standard error	sai số chuẩn	Độ biến thiên của một thống kê giữa các mẫu

3. Thư viện và hàm Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cách viết	Khi nào dùng, ý nghĩa tham số
`df.sample(n=,random_state=)`	Lấy mẫu có thể lặp, chú ý phương pháp lấy mẫu
`rng.choice(values,size=,replace=)`	Lấy mẫu thay thế hoặc không thay thế
`np.mean(samples,axis=1)`	Tính trung bình từng lần lấy mẫu
`s / np.sqrt(n)`	Ước lượng SE của mean từ std mẫu s

4. Khái niệm chính và liên quan

Mạch kiến thức: Tổng thể là nhóm cần kết luận, mẫu là những quan sát thực sự lấy. Lặp lấy mẫu sinh ra nhiều trung bình mẫu; CLT giải thích xu hướng phân phối của chúng, còn sai số chuẩn đo mức dao động của thống kê giữa các mẫu.

4.1. Population — Toàn bộ nhóm cần nghiên cứu

Từ vựng: Population — tổng thể; **Unit** — đơn vị quan sát.

Tổng thể là toàn bộ các đối tượng mà ta muốn kết luận, như 100 hóa đơn của một cửa hàng trong một ngày. Mỗi hóa đơn là **một đơn vị quan sát**; các cột có thể là giá trị, thời gian, trạng thái giao. Nếu chỉ đo 5 hóa đơn, kết luận không tự mở rộng thành 100 hóa đơn mà không xem cách chọn mẫu. Trong ví dụ, tổng thể gồm đúng 100 mã 0–99, nên `len` bằng 100.

No Note — Không chú thích

```
population = list(range(100))
print(len(population))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo tổng thể gồm 100 mã số khác nhau và đếm kích thước.
# Dữ liệu: Mã 0..99 tượng trưng 100 thành viên của toàn bộ nhóm cần khảo sát.
# range(100) sinh số 0..99, KHÔNG lấy 100; list() biến dãy thành 100 mã thành viên tổng thể.
population = list(range(100))
# len() đếm kích thước tổng thể là 100, không phải kích thước mẫu.
print(len(population))
```

Kết quả khi chạy:

```
100
```

Tổng thể có 100 mã hóa đơn; số lượng tổng thể là N=100.

4.2. Sample — Nhóm con lấy theo cách xác định

Từ vựng: Sample — mẫu; **Sampling** — lấy mẫu.

Mẫu là một phần các đối tượng được chọn từ tổng thể. Ví dụ từ 5 mã hóa đơn `[1,2,3,4,5]`, chọn 3 mã **không hoàn lại** nghĩa là mỗi mã xuất hiện tối đa một lần trong mẫu. `rng.choice(...,replace=False)` cho chọn mẫu giả ngẫu nhiên, seed giúp tái lập. Chọn ngẫu nhiên hợp lý hỗ trợ giảm thiên lệch chọn mẫu, nhưng nếu dữ liệu nguồn đã thiếu nhóm quan trọng thì seed không thể khắc phục.

No Note — Không chú thích

```
import numpy as np

rng = np.random.default_rng(1)
sample = rng.choice([1, 2, 3, 4, 5], size=3, replace=False)
print(sample.tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy ngẫu nhiên ba phần tử khác nhau từ năm người, không hoàn lại.
# Dữ liệu: Tổng thể có năm mã 1..5; lấy đúng ba người không trùng, dùng seed 1 để tái lập.
# NumPy cung cấp bộ sinh ngẫu nhiên và lựa chọn mẫu.
import numpy as np

# seed=1 giúp chạy lại ra lựa chọn giả ngẫu nhiên giống nhau khi môi trường tương ứng, không bảo đảm mẫu đại diện.
```

```
rng = np.random.default_rng(1)
# choice(population,size,replace) chọn 3 mã từ 5; replace=False lấy KHÔNG hoàn lại nên không trùng
một người.
sample = rng.choice([1, 2, 3, 4, 5], size=3, replace=False)
# tolist() chuyển ndarray kết quả thành list Python, không làm thay đổi danh sách nguồn.
print(sample.tolist())
```

Kết quả khi chạy:

```
[3, 2, 4]
```

Mẫu có ba mã khác nhau vì lấy không hoàn lại; đây là một kết quả ngẫu nhiên có thể tái lập.

4.3. CLT — Phân phối trung bình của các mẫu

Từ vựng: CLT — định lý giới hạn trung tâm; **Sampling distribution** — phân phối lấy mẫu.

CLT nói rằng dưới các điều kiện phù hợp như quan sát độc lập, cùng phân phối, phương sai hữu hạn, khi **cỡ mẫu n lớn**, phân phối của **trung bình mẫu** sau chuẩn hóa thường tiến gần dạng chuẩn, ngay cả khi dữ liệu gốc không chuẩn. Nó **không** nói dữ liệu gốc trở thành chuẩn hay từng mẫu đều có mean giống nhau. Mô phỏng 200 nhóm, mỗi nhóm 40 giá trị từ phân phối mũ mean lý thuyết 2, cho trung bình các mean gần 2.

No Note — Không chú thích

```
import numpy as np

rng = np.random.default_rng(1)
groups = rng.exponential(scale=2, size=(200, 40))
means = groups.mean(axis=1)
print(round(float(means.mean()), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Mô phỏng phân phối của trung bình mẫu để minh họa định lý giới hạn trung tâm (CLT).
# Dữ liệu: 200 lần lấy mẫu độc lập, mỗi lần 40 thời gian từ phân phối mũ có mean=2; từng mẫu có 40
giá trị.
# NumPy cho phép tạo mảng mô phỏng nhiều lần lấy mẫu.
import numpy as np

# Bộ sinh seed 1 cho phép tái lập dữ liệu ngẫu nhiên mô phỏng.
rng = np.random.default_rng(1)
# exponential(scale=2) tạo biến mũ với TRUNG BÌNH lý thuyết 2 (không phải tham số rate=2);
size=(200,40) là 200 hàng mẫu × 40 quan sát.
groups = rng.exponential(scale=2, size=(200, 40))
# mean(axis=1) tính trung bình của 40 CỘT trong MỖI hàng, ra vector 200 giá trị trung bình mẫu.
means = groups.mean(axis=1)
# Lấy trung bình của 200 giá trị mean để so với trung bình tổng thể lý thuyết 2; round(...,3) làm
tròn khi in.
# float() đổi NumPy scalar thành float Python trước khi hiển thị.
print(round(float(means.mean()), 3))
```

Kết quả khi chạy:

```
2.011
```

200 trung bình mẫu xoay quanh giá trị kỳ vọng 2; một số mô phỏng không chứng minh CLT nhưng minh họa cách lấy mẫu lặp.

4.4. Standard error — Độ biến thiên của một ước lượng

Từ vựng: Standard error (SE) — sai số chuẩn; **Standard deviation (SD)** — độ lệch chuẩn.

SD mô tả độ phân tán của **quan sát riêng lẻ**, còn **SE của mean** mô tả mức dao động **giữa các trung bình mẫu** nếu lặp lấy mẫu cùng cách. Với n quan sát độc lập và ước lượng SD mẫu s, SE(mean) thường ước tính bằng `s/sqrt(n)` (trường hợp thích hợp). Năm điểm `[6,7,8,9,10]` có s≈1,581 điểm, nên SE≈0,707 điểm. Cỡ mẫu gấp 4 làm SE còn một nửa nếu độ phân tán giữ nguyên.

No Note — Không chú thích

```
import numpy as np

scores = np.array([6, 7, 8, 9, 10], dtype=float)
se = scores.std(ddof=1) / np.sqrt(len(scores))
print(round(float(se), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính standard error (SE) của mean từ độ lệch chuẩn mẫu của năm điểm.
# Dữ liệu: Năm điểm 6..10; SE ước tính độ biến thiên của TRUNG BÌNH mẫu qua nhiều lần lấy mẫu.
# NumPy cung cấp std(), sqrt() để ước lượng standard error.
import numpy as np

# Vector năm điểm dtype=float, mỗi ô một học viên.
scores = np.array([6, 7, 8, 9, 10], dtype=float)
# std(ddof=1) dùng mẫu số n−1 khi ước lượng độ lệch chuẩn mẫu, khác ddof=0 của quần thể.
# sqrt() lấy căn bậc hai của số người n=5; len() đếm số mẫu.
# SE=SD mẫu/chia căn n, biểu thị độ bất định của mean, KHÔNG phải SD của từng học viên.
se = scores.std(ddof=1) / np.sqrt(len(scores))
# float() chuyển scalar, round(...,3) định dạng ba chữ số thập phân khi in.
print(round(float(se), 3))
```

Kết quả khi chạy:

```
0.707
```

SE của trung bình mẫu xấp xỉ 0,707 điểm; nó không phải SD của từng điểm.

5. Năm tình huống lấy mẫu/CLT — từ mẫu không hoàn lại đến bootstrap

Tình huống 1 — Cấp 1: Chọn 10 hóa đơn không hoàn lại từ tổng thể 40

Từ vựng trong mục: Population — tổng thể; **Sample** — mẫu; **Without replacement** — không hoàn lại.

Tình huống thực tế: Kiểm toán viên có danh sách **40 hóa đơn của cả ca**, giá trị từ **100 đến 139 nghìn đồng**, mỗi số tương ứng một hóa đơn. Nhân viên chỉ đủ sức kiểm tra **10 hóa đơn**, nên chọn ngẫu nhiên **10 phiếu khác nhau**, không lấy một phiếu hai lần. Họ muốn xem **số tiền trung bình của 10 phiếu được**

chọn có gần **mức trung bình đã biết của cả 40 phiếu** không và hiểu vì sao có thể lệch dù cách lấy mẫu đúng.

Vì sao chọn cách này? Tổng thể là cả **40 hóa đơn** mà quản lý muốn mô tả, còn **mẫu** chỉ là **10 hóa đơn được chọn** để kiểm tra. `rng.choice(population, size=10, replace=False)` chọn **không hoàn lại**, bảo đảm không xem hai lần cùng một phiếu. `default_rng(31)` đặt **seed 31** để người khác chạy lại ví dụ có thể đối chiếu danh sách đã chọn. `sample.mean()` đo trung bình mẫu, còn `population.mean()` đo trung bình cả 40 khoản; hai số **không buộc bằng nhau** vì mẫu chỉ là một phần. Kết quả gần nhau giúp minh họa phương pháp, chưa bảo đảm mọi lần chọn 10 phiếu đều đại diện tốt.

Quy trình:

1. Tạo 40 giá trị hóa đơn làm tổng thể, chọn 10 không hoàn lại.
2. Kiểm tính duy nhất bằng Set và tính mean mẫu.
3. So mean mẫu với mean tổng thể để thấy sai lệch lấy mẫu.

No Note — Không chú thích

```
import numpy as np
population = np.arange(100, 140)
rng = np.random.default_rng(31)
sample = rng.choice(population, size=10, replace=False)
if len(sample) != 10 or len(set(sample)) != 10:
    raise ValueError("Cần chọn đủ 10 hóa đơn không trùng.")
print("Tổng thể:", len(population))
print("Hóa đơn chọn:", len(sample))
print(f"Mean mẫu (nghìn đồng): {sample.mean():.1f}")
print(f"Mean tổng thể (nghìn đồng): {population.mean():.1f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn ngẫu nhiên mười hóa đơn KHÔNG trùng từ tổng thể bốn mươi hóa đơn.
# Dữ liệu: Tổng thể hóa đơn có giá 100..139 nghìn đồng, chọn 10 không hoàn lại, seed 31.
# NumPy arange() tạo tổng thể số, choice() lấy mẫu.
import numpy as np
# arange(start,stop) tạo 40 giá từ 100 đến 139, không lấy 140.
population = np.arange(100, 140)
# seed 31 cố định mô phỏng cho phép đối chiếu.
rng = np.random.default_rng(31)
# Chọn 10 ô khác nhau KHÔNG hoàn lại, mỗi hóa đơn chỉ có thể xuất hiện một lần.
sample = rng.choice(population, size=10, replace=False)
# len() đếm 10 hóa đơn được lấy.
# set() loại các giá trị trùng, len() đối chiếu mẫu đúng 10 phần tử duy nhất.
if len(sample) != 10 or len(set(sample)) != 10:
    # Báo lỗi nếu thiếu mẫu hoặc mẫu lặp, không dùng số đo không hợp lệ.
    raise ValueError("Cần chọn đủ 10 hóa đơn không trùng.")
# Đếm số phần tử của tổng thể ban đầu (40), không phải số mẫu lấy.
print("Tổng thể:", len(population))
# len() đếm 10 hóa đơn được lấy.
print("Hóa đơn chọn:", len(sample))
# mean() lấy tiền trung bình của 10 hóa đơn mẫu; :.1f chỉ hiển thị một số lẻ.
print(f"Mean mẫu (nghìn đồng): {sample.mean():.1f}")
# Mean của 40 hóa đơn toàn bộ là mốc tham chiếu; khác mean mẫu do chọn ngẫu nhiên.
print(f"Mean tổng thể (nghìn đồng): {population.mean():.1f}")
```

Kết quả khi chạy:

```
Tổng thể: 40
Hóa đơn chọn: 10
```

```
Mean mẫu (nghìn đồng): 122.0
Mean tổng thể (nghìn đồng): 119.5
```

Chọn mẫu ngẫu nhiên không có nghĩa mean luôn bằng mean tổng thể; một lần rút 10 đơn chỉ cung cấp một ước lượng.

Tình huống 2 — Cấp 2: Chia mẫu 12 phiếu theo hai tầng kho

Từ vựng trong mục: Stratified sampling — lấy mẫu phân tầng; **Allocation** — phân bổ cỡ mẫu.

Tình huống thực tế: Kho có **60 phiếu kiểm hàng**, chia đều **30 phiếu nhóm A** và **30 phiếu nhóm B**. Nhân viên chỉ xem được **12 phiếu** nhưng cần bảo đảm nhóm nào cũng có đại diện, nên kế hoạch là rút **6 mã khác nhau ở A và 6 mã khác nhau ở B**. Họ muốn xác nhận danh sách 12 mã không có trùng lặp và không vô tình chọn cả 12 từ cùng một nhóm.

Vì sao chọn cách này? Lấy mẫu phân tầng nghĩa là chia tổng thể thành các nhóm có ý nghĩa **trước khi rút**, rồi rút trong từng nhóm thay vì rút cả 60 phiếu chung không kiểm soát. Hai tầng A/B đều có 30 phiếu nên kế hoạch **mỗi tầng 6 phiếu** vừa đảm bảo hiện diện, vừa giữ tỷ lệ chọn giống tỷ lệ tổng thể. `rng.choice(..., replace=False)` rút không trùng **trong mỗi tầng**; mã A và B cũng khác tiền tố, nên có thể kiểm tra danh sách chung không trùng. `default_rng(12)` giúp tái lập lần rút; phương pháp này không tự sửa dữ liệu ghi sai ở từng phiếu.

Quy trình:

1. Tạo 30 ID A và 30 ID B.
2. Chọn 6 từ mỗi nhóm với seed cố định và không hoàn lại.
3. Ghép 12 ID và xác minh đủ hai nhóm, không lặp.

No Note — Không chú thích

```
import numpy as np
rng = np.random.default_rng(12)
A = []
B = []
for i in range(1, 31):
    A.append(f"A{i:02d}")
    B.append(f"B{i:02d}")
selected_a = rng.choice(A, 6, replace=False).tolist()
selected_b = rng.choice(B, 6, replace=False).tolist()
selection = selected_a + selected_b
if len(selection) != 12 or len(set(selection)) != 12:
    raise ValueError("Cần đúng 12 phiếu chọn, không trùng ID.")
print("Tổng thể:", len(A)+len(B))
print("Phiếu lấy:", len(selection))
print("Nhóm A/B:", len(selected_a), len(selected_b))
print("Có trùng ID:", len(selection) != len(set(selection)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Lấy mẫu phân tầng: chọn sáu phiếu nhóm A và sáu phiếu nhóm B từ 60 phiếu.
# Dữ liệu: Hai nhóm A/B mỗi nhóm 30 phiếu, chọn 6 phiếu mỗi nhóm không hoàn lại; seed 12.
# NumPy Generator.choice() hỗ trợ lấy mẫu ở từng tầng.
import numpy as np
# Seed 12 làm lựa chọn mô phỏng tái lập được.
rng = np.random.default_rng(12)
# Danh sách rỗng chuẩn bị 30 mã phiếu A.
A = []
```

```
# Danh sách rỗng chuẩn bị 30 mã phiếu B.
B = []
# range(1,31) tạo số thứ tự 1..30, không lấy 31.
for i in range(1, 31):
    # f-string :02d tạo mã A01..A30 hai chữ số.
    # append() thêm một mã A vào list.
    A.append(f"A{i:02d}")
    # Tạo mã B01..B30 để không trùng định danh với nhóm A.
    # append() thêm một mã B vào list.
    B.append(f"B{i:02d}")
# Lấy 6 phiếu KHÔNG hoàn lại từ tầng A (30 phiếu), không lấy ngẫu nhiên 12 từ tổng thể một lần.
# tolist() đổi ndarray mã được chọn thành list Python.
selected_a = rng.choice(A, 6, replace=False).tolist()
# Lấy riêng 6 phiếu từ tầng B, bảo đảm thành phần mẫu của cả hai nhóm.
# tolist() đổi ndarray mã được chọn thành list Python.
selected_b = rng.choice(B, 6, replace=False).tolist()
# Nối hai list thành 12 phiếu, không phải phép cộng số.
selection = selected_a + selected_b
# set() bỏ mã trùng để kiểm 12 mã duy nhất.
if len(selection) != 12 or len(set(selection)) != 12:
    # Dừng nếu không chọn đủ số phiếu hoặc có mã trùng.
    raise ValueError("Cần đúng 12 phiếu chọn, không trùng ID.")
# Đếm kích thước tổng thể 30+30=60; không phải chỉ 12 mẫu.
print("Tổng thể:", len(A)+len(B))
print("Phiếu lấy:", len(selection))
print("Nhóm A/B:", len(selected_a), len(selected_b))
# set() bỏ mã trùng để kiểm 12 mã duy nhất.
print("Có trùng ID:", len(selection) != len(set(selection)))
```

Kết quả khi chạy:

```
Tổng thể: 60
Phiếu lấy: 12
Nhóm A/B: 6 6
Có trùng ID: False
```

Phân tầng đảm bảo đại diện từng khu vực **theo thiết kế**, nhưng không tự giải quyết dữ liệu nguồn sai hoặc khung lấy mẫu thiếu phiếu.

Tình huống 3 — Cấp 3: Mô phỏng 15 lần lấy mẫu để quan sát CLT

Từ vựng trong mục: Sampling distribution — phân phối trung bình mẫu; **CLT** — định lý giới hạn trung tâm.

Tình huống thực tế: Nhóm dịch vụ muốn hiểu **thời gian xử lý** tính bằng **phút** khi các lượt riêng lẻ thường rất ngắn nhưng đôi lúc kéo dài. Họ tạo **15 bộ mẫu giả lập độc lập**, mỗi bộ gồm **40 thời gian**, theo phân phối mũ có mức trung bình lý thuyết **2 phút**. Họ muốn tính **trung bình của từng bộ**, rồi so độ dao động của **15 số trung bình** với độ dao động của tất cả **600 thời gian đơn lẻ**, chứ không gọi mỗi lượt thời gian là một trung bình mẫu.

Vì sao chọn cách này? Định lý giới hạn trung tâm (CLT) nói về cách **phân phối các trung bình mẫu** có xu hướng gần hình chuông hơn khi mỗi mẫu đủ lớn và các điều kiện phù hợp, **không có nghĩa từng thời gian xử lý sẽ thành phân phối chuẩn**. `rng.exponential(scale=2, size=(15, 40))` tạo **15 hàng mẫu**, mỗi hàng **40 thời gian giả lập** có phân phối lệch phải và mức trung bình lý thuyết 2 phút. `mean(axis=1)` lấy một trung bình **cho từng hàng**, tạo đúng 15 trung bình. So độ lệch chuẩn của 15 trung bình với độ lệch chuẩn các thời gian đơn lẻ cho thấy trung bình thường dao động ít hơn. **Seed 12** giúp tái lập; 15 bộ mẫu chỉ là minh họa nhỏ, không đủ kiểm chứng hoàn toàn hình dạng phân phối.

Quy trình:

1. Tạo ma trận 15×40 theo cùng seed để tái lập.
2. Tính một mean cho mỗi trong 15 hàng.
3. So SD của 15 mean với SD của tất cả thời gian đơn lẻ và in mean chung.

No Note — Không chú thích

```
import numpy as np
rng = np.random.default_rng(12)
samples = rng.exponential(scale=2, size=(15, 40))
means = samples.mean(axis=1)
if samples.shape != (15, 40) or means.shape != (15,):
    raise ValueError("Cần 15 lần lấy mẫu, mỗi lần 40 quan sát.")
print("Lần lấy mẫu:", len(means))
print("Quan sát mỗi lần:", samples.shape[1])
print(f"Mean của các mean (phút): {means.mean():.3f}")
print("SD mean nhỏ hơn SD dữ liệu:", bool(means.std(ddof=1) < samples.ravel().std(ddof=1)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Quan sát CLT qua 15 lần lấy mẫu, mỗi lần 40 thời gian từ phân phối mũ.
# Dữ liệu: Phân phối mũ scale=2 phút, 15 hàng mẫu × 40 quan sát, seed 12.
# NumPy mô phỏng phân phối mũ và thống kê theo hàng.
import numpy as np
# Seed 12 giúp tái tạo cùng 15 lần mô phỏng.
rng = np.random.default_rng(12)
# scale=2 phút là mean của phân phối mũ; size=(15,40) có 15 lần lấy mẫu mỗi lần 40 điểm.
samples = rng.exponential(scale=2, size=(15, 40))
# axis=1 gom 40 CỘT của TỪNG HÀNG để có 15 trung bình mẫu.
means = samples.mean(axis=1)
# shape (15,40) kiểm đúng số lần lấy mẫu và kích cỡ từng mẫu.
# shape (15,) xác nhận một mean cho mỗi lần lấy mẫu.
if samples.shape != (15, 40) or means.shape != (15,):
    # Báo dữ liệu mô phỏng không đúng số hàng/cột.
    raise ValueError("Cần 15 lần lấy mẫu, mỗi lần 40 quan sát.")
# len() đếm 15 lần lấy mẫu chứ không phải 600 quan sát.
print("Lần lấy mẫu:", len(means))
# shape (15,40) kiểm đúng số lần lấy mẫu và kích cỡ từng mẫu.
# shape[1] lấy số cột quan sát trong mỗi mẫu, bằng 40.
print("Quan sát mỗi lần:", samples.shape[1])
# Trung bình của 15 trung bình mẫu để so với mean tổng thể lý thuyết 2 phút.
print(f"Mean của các mean (phút): {means.mean():.3f}")
# std(ddof=1) ước tính độ lệch chuẩn của 15 TRUNG BÌNH mẫu, không phải từng giá trị gốc.
# ravel() trải mảng (15,40) về vector 600 quan sát để tính độ phân tán dữ liệu từng lượt.
# So SD trung bình mẫu với SD của từng quan sát; CLT dự đoán mean mẫu ít phân tán hơn, đây chỉ là
mô phỏng giới hạn 15 lượt.
# bool() chuyển so sánh NumPy thành Boolean Python để in.
print("SD mean nhỏ hơn SD dữ liệu:", bool(means.std(ddof=1) < samples.ravel().std(ddof=1)))
```

Kết quả khi chạy:

```
Lần lấy mẫu: 15
Quan sát mỗi lần: 40
Mean của các mean (phút): 2.082
SD mean nhỏ hơn SD dữ liệu: True
```

15 bản ghi là 15 lần lấy mẫu, không phải 15 phép đo đơn lẻ; mỗi bản ghi chứa 40 phép đo bên trong.

Tình huống 4 — Cấp 4: Tính SE từ 18 điểm và tránh so hai mẫu không tương đương

Từ vựng trong mục: Standard error — sai số chuẩn; **Sample SD** — độ lệch chuẩn mẫu.

Tình huống thực tế: Giảng viên có **18 điểm thực hành** trên **thang 30 điểm**, một số tương ứng một học viên. Để hiểu độ chính xác của **trung bình điểm**, giáo viên thử tính **sai số chuẩn SE** từ **9 học viên đầu tiên**, rồi từ **cả 18 người**. Hai tập không có đúng cùng mức phân tán điểm, nên cần xem cả số người và độ trải rộng của điểm trước khi kết luận về sự khác nhau của SE.

Vì sao chọn cách này? SE (sai số chuẩn của trung bình) mô tả **mức trung bình mẫu có thể thay đổi giữa những lần chọn mẫu tương tự**, khác với **độ lệch chuẩn SD** mô tả các điểm **của từng học viên** trải rộng đến đâu. `std(ddof=1)` ước lượng SD từ các điểm trong mẫu; chương trình tính SE từ SD ấy và **số người** của từng nhóm. Khi số người tăng mà mức phân tán giữ tương đương, SE thường giảm; nhưng chín người đầu và 18 người trong ví dụ có **SD khác nhau**, nên không thể chỉ nhìn cỡ mẫu rồi suy ra chắc chắn mức SE nào. Đây là phép minh họa từ điểm thang 30, không có dữ liệu quần thể thật để xác nhận độ chính xác tuyệt đối.

Quy trình:

1. Tạo 18 điểm gốc, lấy thêm tập con 9 điểm đầu.
2. Tính std mẫu (ddof=1) và SE cho cả hai tập.
3. Báo cả hai SE kèm số người và giải thích giới hạn so sánh.

No Note — Không chú thích

```
import numpy as np
scores = np.array([8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25], dtype=float)
first9 = scores[:9]
se9 = first9.std(ddof=1)/np.sqrt(len(first9))
se18 = scores.std(ddof=1)/np.sqrt(len(scores))
if len(scores) != 18:
    raise ValueError("Cần đủ 18 điểm học viên.")
print("Tất cả học viên:", len(scores))
print("Nhóm con:", len(first9))
print(f"SE 9 người (điểm): {se9:.3f}")
print(f"SE 18 người (điểm): {se18:.3f}")
print("SE của hai nhóm khác nhau:", bool(not np.isclose(se9, se18)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So standard error ước lượng trên 9 và 18 điểm và nêu rõ đây là hai nhóm khác nhau.
# Dữ liệu: 18 điểm học viên 8..25; first9 lấy chín học viên đầu nên có phân phối hẹp hơn cả 18.
# NumPy cung cấp std(), sqrt(), isclose() cho hai ước lượng SE.
import numpy as np
# 18 số điểm float xếp tăng dần theo đầu vào, không phải mẫu ngẫu nhiên.
scores = np.array([8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25], dtype=float)
# Slice [:9] lấy chỉ số 0..8 (9 học viên ĐẦU), không lấy phần tử thứ 10.
first9 = scores[:9]
# std(ddof=1) lấy SD mẫu với mẫu số n−1, khác SD quần thể ddof=0.
# sqrt(n) là căn kích thước nhóm 9, dùng mẫu số của SE.
# SE chín người = SD mẫu của nhóm đầu / căn 9, đơn vị điểm.
se9 = first9.std(ddof=1)/np.sqrt(len(first9))
# std(ddof=1) lấy SD mẫu với mẫu số n−1, khác SD quần thể ddof=0.
# sqrt(n) cho nhóm 18, nhưng SD hai nhóm cũng có thể khác.
# SE mười tám người = SD mẫu của toàn bộ / căn 18, đơn vị điểm.
# len() xác minh 18 quan sát; len(first9) trả 9.
se18 = scores.std(ddof=1)/np.sqrt(len(scores))
# len() xác minh 18 quan sát; len(first9) trả 9.
```

```
if len(scores) != 18:
    # Dừng nếu bảng điểm không có đúng 18 người.
    raise ValueError("Cần đủ 18 điểm học viên.")
# len() xác minh 18 quan sát; len(first9) trả 9.
print("Tất cả học viên:", len(scores))
print("Nhóm con:", len(first9))
print(f"SE 9 người (điểm): {se9:.3f}")
print(f"SE 18 người (điểm): {se18:.3f}")
# isclose() so hai SE có dung sai float, not đảo kết quả; SE khác vì cả kích thước và phạm vi điểm
khác nhau.
# bool() ép kết quả so sánh sang bool Python dễ đọc.
print("SE của hai nhóm khác nhau:", bool(not np.isclose(se9, se18)))
```

Kết quả khi chạy:

```
Tất cả học viên: 18
Nhóm con: 9
SE 9 người (điểm): 0.913
SE 18 người (điểm): 1.258
SE của hai nhóm khác nhau: True
```

SE của 18 người **lớn hơn** trong dữ liệu cụ thể này vì độ phân tán cũng lớn hơn. Để chỉ quan sát tác động n, cần giữ độ phân tán như nhau.

Tình huống 5 — Cấp 5: Bootstrap từ 20 phiếu giao để đo dao động trung bình

Từ vựng trong mục: Bootstrap — lấy mẫu lặp có hoàn lại; **Percentile interval** — khoảng phân vị.

Tình huống thực tế: Đơn vị vận chuyển có **20 thời gian hoàn tất phiếu giao**, tính bằng **phút**, từ 30 đến 55. Người quản lý muốn hình dung **trung bình của 20 phiếu có thể dao động bao nhiêu** nếu khảo sát những bộ mẫu tương tự. Vì chưa có dữ liệu bổ sung, nhóm dựng **2.000 mẫu bootstrap giả lập**, mỗi mẫu gồm **20 lần chọn lại từ đúng 20 phiếu hiện có, cho phép chọn trùng**, rồi xem vùng trung bình thấp/cao của 2.000 mẫu.

Vì sao chọn cách này? Bootstrap là cách **rút lại từ mẫu đã có**, tạo nhiều bộ mẫu giả để xem một thống kê như trung bình có thể biến động. `replace=True` cho phép cùng một phiếu xuất hiện **nhiều lần trong một bộ 20 lượt**, đúng chủ đích bootstrap chứ không phải lỗi trùng hồ sơ; nếu không hoàn lại, mọi bộ đủ 20 phiếu sẽ giống hệt bộ ban đầu về trung bình. `rng.choice(..., size=(2000, 20))` tạo **2.000 bộ giả lập**, rồi `mean(axis=1)` cho 2.000 trung bình; `np.quantile` lấy **mốc 2,5% và 97,5%** để tạo khoảng percentile minh họa. **Seed 18** cho kết quả tái lập, còn số 2.000 là số lượt mô phỏng được chọn để ổn định ước lượng chứ không phải có 2.000 phiếu thật hoặc bảo đảm khoảng luôn bao phủ đúng 95%.

Quy trình:

1. Tạo 20 phiếu có thời gian hợp lệ.
2. Rút 2.000 hàng mẫu, mỗi hàng chứa 20 giá trị có hoàn lại.
3. Tính mean của 2.000 hàng và hai phân vị biên, kiểm mean gốc ở trong khoảng minh họa.

No Note — Không chú thích

```
import numpy as np
minutes = np.array([30,32,33,34,35,36,38,39,40,41,42,43,44,45,46,47,49,50,51,55], dtype=float)
rng = np.random.default_rng(18)
bootstrap = rng.choice(minutes, size=(2000, len(minutes)), replace=True)
means = bootstrap.mean(axis=1)
low, high = np.quantile(means, [0.025, 0.975])
```

```
if len(minutes) != 20 or bootstrap.shape != (2000, 20):
    raise ValueError("Cần 20 phiếu và 2000 lượt lấy mẫu bootstrap.")
print("Phiếu gốc:", len(minutes))
print("Mẫu bootstrap:", len(means))
print(f"Mean gốc (phút): {minutes.mean():.2f}")
print(f"Khoảng bootstrap (phút): {low:.2f} – {high:.2f}")
print("Mean gốc trong khoảng:", bool(low <= minutes.mean() <= high))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Bootstrap 2.000 lần từ 20 phiếu để ước lượng khoảng phân vị của trung bình.
# Dữ liệu: 20 thời gian phút, mỗi bootstrap lấy 20 lượt CÓ hoàn lại, seed 18, mức giữa 95% dùng
phân vị 2.5% và 97.5%.
# NumPy Generator.choice() lấy mẫu lặp và quantile() lấy phân vị bootstrap.
import numpy as np
# 20 số phút gốc dạng float, mỗi phần tử là một phiếu thời gian.
minutes = np.array([30,32,33,34,35,36,38,39,40,41,42,43,44,45,46,47,49,50,51,55], dtype=float)
# Seed 18 giúp tái lập 2.000 mẫu bootstrap.
rng = np.random.default_rng(18)
# choice(...,replace=True) lấy mẫu CÓ hoàn lại từ 20 phiếu; mỗi hàng 20 lượt, 2.000 hàng có thể lặp
cùng phiếu nhiều lần.
# len() đếm 20 phiếu gốc.
bootstrap = rng.choice(minutes, size=(2000, len(minutes)), replace=True)
# axis=1 tính mean của 20 CỘT trong mỗi hàng bootstrap, cho 2.000 trung bình mẫu.
means = bootstrap.mean(axis=1)
# Phân vị 2.5% và 97.5% của 2.000 mean tạo khoảng bootstrap percentile trung tâm 95%; không tự động
chứng minh 95% xác suất chứa tham số đã cố định.
low, high = np.quantile(means, [0.025, 0.975])
# shape (2000,20) kiểm 2.000 lần lấy mẫu, mỗi lần 20 ô.
# len() đếm 20 phiếu gốc.
if len(minutes) != 20 or bootstrap.shape != (2000, 20):
    # Báo dữ liệu/bootstrap sai kích thước trước khi diễn giải.
    raise ValueError("Cần 20 phiếu và 2000 lượt lấy mẫu bootstrap.")
# len() đếm 20 phiếu gốc.
print("Phiếu gốc:", len(minutes))
# len() đếm 2.000 giá trị mean từ bootstrap, không phải số quan sát gốc.
print("Mẫu bootstrap:", len(means))
# Mean 20 phiếu gốc dùng so vị trí với khoảng bootstrap.
print(f"Mean gốc (phút): {minutes.mean():.2f}")
print(f"Khoảng bootstrap (phút): {low:.2f} – {high:.2f}")
# Mean 20 phiếu gốc dùng so vị trí với khoảng bootstrap.
# Chuỗi so sánh kiểm mean gốc có nằm giữa hai mốc bootstrap hay không, kết quả Boolean, không phải
kiểm độ bao phủ dài hạn.
# bool() đổi kết quả so sánh về Python True/False để in.
print("Mean gốc trong khoảng:", bool(low <= minutes.mean() <= high))
```

Kết quả khi chạy:

```
Phiếu gốc: 20
Mẫu bootstrap: 2000
Mean gốc (phút): 41.50
Khoảng bootstrap (phút): 38.65 – 44.45
Mean gốc trong khoảng: True
```

Đây là **ước lượng bất định từ mẫu quan sát**, không phải khoảng chứa 95% thời gian giao của từng phiếu.

6. Sai lầm thường gặp

- CLT nói về phân phối của **trung bình mẫu** dưới các điều kiện thích hợp, không biến dữ liệu gốc thành chuẩn.
- SD mô tả số đo cá thể, SE mô tả mức biến thiên ước lượng giữa các mẫu.
- Công thức SE≈s/√n cần giả định lấy mẫu phù hợp; phụ thuộc, chọn mẫu không ngẫu nhiên và quần thể hữu hạn có thể cần điều chỉnh.
- Seed giúp tái lập phép mô phỏng, **không** sửa thiên lệch chọn mẫu hoặc bảo đảm mẫu đại diện.
- Cỡ mẫu lớn thường giảm SE, nhưng không sửa được sai lệch có hệ thống.
- Không kết luận mọi mẫu sẽ gần trung bình thật hay một ngưỡng n cố định đúng cho mọi phân phối.

7. Trắc nghiệm

Q1. What is a population? | Tổng thể là gì?

A. Sampling variability of an estimator | Độ biến thiên của thống kê ước lượng qua các mẫu

B. A selected subset of the population | Một nhóm con được chọn từ tổng thể

C. All units in the target group | Toàn bộ đơn vị của nhóm cần nghiên cứu

D. Approximate sampling distribution of sample means | Phân phối gần đúng của các trung bình mẫu

Đáp án: C

Giải thích: Kết luận dự kiến hướng đến tổng thể.

Q2. What is a sample? | Mẫu là gì?

A. Approximate sampling distribution of sample means | Phân phối gần đúng của các trung bình mẫu

B. A selected subset of the population | Một nhóm con được chọn từ tổng thể

C. Sampling variability of an estimator | Độ biến thiên của thống kê ước lượng qua các mẫu

D. All units in the target group | Toàn bộ đơn vị của nhóm cần nghiên cứu

Đáp án: B

Giải thích: Mẫu chứa một phần quan sát.

Q3. What does CLT describe? | CLT mô tả gì?

A. Approximate sampling distribution of sample means | Phân phối gần đúng của các trung bình mẫu

B. All units in the target group | Toàn bộ đơn vị của nhóm cần nghiên cứu

C. A selected subset of the population | Một nhóm con được chọn từ tổng thể

D. Sampling variability of an estimator | Độ biến thiên của thống kê ước lượng qua các mẫu

Đáp án: A

Giải thích: CLT nói về mean qua nhiều mẫu, không nói mọi giá trị gốc thành chuẩn.

Q4. What is standard error? | SE là gì?

A. Sampling variability of an estimator | Độ biến thiên của thống kê ước lượng qua các mẫu

B. A selected subset of the population | Một nhóm con được chọn từ tổng thể

C. All units in the target group | Toàn bộ đơn vị của nhóm cần nghiên cứu

D. Approximate sampling distribution of sample means | Phân phối gần đúng của các trung bình mẫu

Đáp án: A

Giải thích: SE mô tả độ bất định của thống kê.

Q5. A random sample contains [3, 5, 7]. What is its sample mean? | Một mẫu ngẫu nhiên có các giá trị [3, 5, 7]. Trung bình mẫu bằng bao nhiêu?

A. 7 | 7

B. 3 | 3

C. 5 | 5

D. 15 | 15

Đáp án: C

Giải thích: Trung bình mẫu = (3 + 5 + 7)/3 = 5.

Q6. If the sample size rises from 25 to 100 and the population standard deviation stays constant, how does the standard error of the mean change? | Nếu cỡ mẫu tăng từ 25 lên 100 và độ lệch chuẩn tổng thể không đổi, sai số chuẩn của trung bình thay đổi thế nào?

A. It doubles | Tăng gấp đôi

B. It stays unchanged | Không đổi

C. It halves | Giảm một nửa

D. It becomes zero | Bằng 0

Đáp án: C

Giải thích: SE = độ lệch chuẩn / căn(cỡ mẫu). Cỡ mẫu tăng 4 lần nên căn cỡ mẫu tăng 2 lần, vì vậy SE giảm một nửa.

Q7. Does setting a random seed guarantee a representative sample? | Đặt seed ngẫu nhiên có bảo đảm lấy được mẫu đại diện không?

A. Yes, for large populations | Có, nếu tổng thể lớn

B. Yes, if seed = 42 | Có, nếu seed = 42

C. No; it only helps reproduce the random draw | Không; seed chỉ giúp tái lập lượt lấy mẫu

D. Yes, always | Có, luôn luôn

Đáp án: C

Giải thích: Seed giúp lặp lại các số ngẫu nhiên, không khắc phục thiên lệch chọn mẫu hay bảo đảm đại diện.

Q8. What does the Central Limit Theorem mainly describe under suitable conditions and increasing sample size? | Định lý giới hạn trung tâm chủ yếu mô tả điều gì khi các điều kiện phù hợp và cỡ mẫu tăng?

A. Individual observations become equal | Các quan sát trở nên bằng nhau

B. The population mean becomes zero | Trung bình tổng thể trở về 0

C. All raw data become normal | Mọi dữ liệu gốc trở thành chuẩn

D. The distribution of sample means tends toward normal | Phân phối trung bình mẫu tiến gần phân phối chuẩn

Đáp án: D

Giải thích: CLT nói về phân phối của trung bình qua nhiều mẫu, không bắt buộc phân phối của từng quan sát chuyển thành chuẩn.

Q9. What does sample SD measure? | SD mẫu đo gì?

A. It halves | Giảm một nửa

B. s/sqrt(n) | s/sqrt(n)

C. Spread of individual values | Độ phân tán giá trị từng quan sát

D. Spread among hypothetical sample means | Độ phân tán giữa các mean mẫu giả định

Đáp án: C

Giải thích: SD khác bất định của mean mẫu.

Q10. What does the SE of a mean measure? | SE của mean đo gì?

A. s/sqrt(n) | s/sqrt(n)

B. It halves | Giảm một nửa

C. Spread among hypothetical sample means | Độ phân tán giữa các mean mẫu giả định

D. Spread of individual values | Độ phân tán giá trị từng quan sát

Đáp án: C

Giải thích: Lấy mẫu lặp gây biến thiên trung bình.

Q11. What is a common estimated SE formula? | Công thức ước lượng SE thường dùng?

A. s/sqrt(n) | s/sqrt(n)

B. Spread among hypothetical sample means | Độ phân tán giữa các mean mẫu giả định

C. Spread of individual values | Độ phân tán giá trị từng quan sát

D. It halves | Giảm một nửa

Đáp án: A

Giải thích: Công thức phù hợp trong các giả định lấy mẫu độc lập thường gặp.

Q12. How does SE change when n quadruples and sigma is fixed? | SE đổi sao khi n gấp bốn, sigma giữ nguyên?

A. s/sqrt(n) | s/sqrt(n)

B. It halves | Giảm một nửa

C. Spread among hypothetical sample means | Độ phân tán giữa các mean mẫu giả định

D. Spread of individual values | Độ phân tán giá trị từng quan sát

Đáp án: B

Giải thích: Căn bậc hai của bốn là hai.

Q13. Does CLT assert that raw data become normally distributed? | CLT có nói dữ liệu gốc thành phân phối chuẩn không?

A. It decreases for the sample mean under assumptions | Giảm đối với mean mẫu nếu giả định phù hợp

B. No, it concerns standardized sample means | Không, nói về mean mẫu được chuẩn hóa

C. No; sampling design matters | Không; thiết kế lấy mẫu mới quan trọng

D. A systematically unrepresentative sample | Mẫu không đại diện một cách có hệ thống

Đáp án: B

Giải thích: Tổng thể lệch vẫn có thể lệch.

Q14. Is a sample necessarily representative because a seed is fixed? | Có seed cố định thì mẫu chắc chắn đại diện không?

A. A systematically unrepresentative sample | Mẫu không đại diện một cách có hệ thống

B. No; sampling design matters | Không; thiết kế lấy mẫu mới quan trọng

C. No, it concerns standardized sample means | Không, nói về mean mẫu được chuẩn hóa

D. It decreases for the sample mean under assumptions | Giảm đối với mean mẫu nếu giả định phù hợp

Đáp án: B

Giải thích: Seed chỉ lặp cách chọn.

Q15. What often happens to sampling variability as n grows? | n tăng thì dao động lấy mẫu thường thế nào?

A. A systematically unrepresentative sample | Mẫu không đại diện một cách có hệ thống

B. It decreases for the sample mean under assumptions | Giảm đối với mean mẫu nếu giả định phù hợp

C. No; sampling design matters | Không; thiết kế lấy mẫu mới quan trọng

D. No, it concerns standardized sample means | Không, nói về mean mẫu được chuẩn hóa

Đáp án: B

Giải thích: SE thường tỷ lệ nghịch căn n.

Q16. What is the impact of selection bias? | Thiên lệch chọn mẫu gây gì?

A. It decreases for the sample mean under assumptions | Giảm đối với mean mẫu nếu giả định phù hợp

B. A systematically unrepresentative sample | Mẫu không đại diện một cách có hệ thống

C. No, it concerns standardized sample means | Không, nói về mean mẫu được chuẩn hóa

D. No; sampling design matters | Không; thiết kế lấy mẫu mới quan trọng

Đáp án: B

Giải thích: Tăng mẫu không tự sửa cách chọn sai.

Q17. Does n=40 guarantee a good normal approximation in every problem? | n=40 có bảo đảm xấp xỉ chuẩn tốt ở mọi bài toán không?

A. Clearly defined sampling assumptions | Giả định lấy mẫu được xác định rõ

B. No, underlying assumptions and tails matter | Không, còn phụ thuộc giả định và đuôi phân phối

C. Standard deviation | Độ lệch chuẩn

D. Uses n−1 in variance denominator | Dùng n−1 ở mẫu số phương sai

Đáp án: B

Giải thích: Không có ngưỡng cỡ mẫu đúng cho mọi trường hợp.

Q18. What does `ddof=1` change in SD? | `ddof=1` đổi gì trong SD?

A. No, underlying assumptions and tails matter | Không, còn phụ thuộc giả định và đuôi phân phối

B. Standard deviation | Độ lệch chuẩn

C. Uses n−1 in variance denominator | Dùng n−1 ở mẫu số phương sai

D. Clearly defined sampling assumptions | Giả định lấy mẫu được xác định rõ

Đáp án: C

Giải thích: Đây là quy ước cho ước lượng phương sai mẫu.

Q19. Which quantity stays in original units, SD or variance? | Đại lượng nào giữ đơn vị gốc, SD hay variance?

A. Uses n−1 in variance denominator | Dùng n−1 ở mẫu số phương sai

B. Clearly defined sampling assumptions | Giả định lấy mẫu được xác định rõ

C. Standard deviation | Độ lệch chuẩn

D. No, underlying assumptions and tails matter | Không, còn phụ thuộc giả định và đuôi phân phối

Đáp án: C

Giải thích: Phương sai có đơn vị bình phương.

Q20. What is a prerequisite for repeated-sampling inference? | Điều kiện quan trọng của suy luận từ lấy mẫu lặp?

A. No, underlying assumptions and tails matter | Không, còn phụ thuộc giả định và đuôi phân phối

B. Standard deviation | Độ lệch chuẩn

C. Clearly defined sampling assumptions | Giả định lấy mẫu được xác định rõ

D. Uses n−1 in variance denominator | Dùng n−1 ở mẫu số phương sai

Đáp án: C

Giải thích: Sự phụ thuộc và cách chọn mẫu ảnh hưởng SE.

8. Liên hệ bài kế tiếp

`Thống kê → Sampling → Central Limit Theorem → Confidence Interval.`

PY-40 — Kiểm định giả thuyết, p-value và khoảng tin cậy

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Null hypothesis — giả thuyết không; **p-value** — giá trị p.

`CLT/SE → Hypothesis Testing → Confidence Interval → A/B Test`

Học xong dùng để làm gì? Đánh giá bằng chứng mẫu bằng p-value và khoảng tin cậy mà không kết luận chắc chắn quá mức.

Kết nối kiến thức tiếp theo: PY-41 chia dữ liệu train/test và đo chất lượng mô hình trên tập chưa dùng để học.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Null hypothesis	giả thuyết không	Phát biểu gốc đem kiểm định, ký hiệu H0
p-value	giá trị p	Độ bất thường của dữ liệu nếu giả định H0 đúng
T-test	kiểm định t	Kiểm định trung bình khi phương sai tổng thể chưa biết
Confidence interval	khoảng tin cậy	Khoảng tính theo quy trình có mức bao phủ dài hạn xác định

3. Hàm và công thức áp dụng

Công cụ	Ý nghĩa
`ttest_1samp(values, popmean)`	Kiểm định t một mẫu so với trung bình giả thuyết
`sem(values)`	Sai số chuẩn của mean mẫu, mặc định theo SD mẫu
`t.interval(confidence, df, loc, scale)`	Khoảng t với mức tin cậy, bậc tự do, trung bình và SE
`pvalue < alpha`	Quy tắc bác bỏ H_0 nếu p nhỏ hơn mức ý nghĩa đã chọn

4. Khái niệm chính và liên quan

Mạch kiến thức: Đầu tiên đặt giả thuyết H_0 và H_1. Kiểm định t tạo thống kê kiểm định và p-value dưới giả định H_0; so p với ngưỡng α để quyết định có bác bỏ hay không. Khoảng tin cậy dùng thống kê mẫu và sai số chuẩn để biểu diễn mức bất định của ước lượng theo quy trình bao phủ dài hạn.

4.1. Null hypothesis — Giả thuyết gốc cần kiểm định

Từ vựng: Null hypothesis (H_0) — giả thuyết không; **Alternative (H_1)** — giả thuyết đối.

H_0 là phát biểu gốc để đối chiếu dữ liệu, thường là "trung bình μ=50 điểm", còn H_1 có thể là "μ khác 50 điểm". Không biết trước giả thuyết nào đúng; ta xem dữ liệu mẫu có **quá bất thường nếu H_0 đúng** hay không. H_0 là một phát biểu về quần thể, không phải câu hỏi liệu mean của bảy điểm đo có đúng bằng 50. Ví dụ in cặp giả thuyết rõ ràng trước khi tính.

No Note — Không chú thích

```
print('H0: population mean = 50')
print('H1: population mean != 50')
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Viết giả thuyết H0 và H1 của phép kiểm định trung bình tổng thể bằng 50.
# Dữ liệu: H0: mean quần thể=50; H1: mean khác 50 (kiểm định hai phía), chưa có dữ liệu hay p-
value.
# H0 là giả thuyết gốc về trung bình TỔNG THỂ, không phải mean mẫu đã quan sát.
print('H0: population mean = 50')
# H1 đặt trung bình khác 50 theo cả hai chiều, nên là kiểm định hai phía; print chỉ mô tả, chưa
chạy phép kiểm thống kê.
print('H1: population mean != 50')
```

Kết quả khi chạy:

```
H0: population mean = 50
H1: population mean != 50
```

Hai phát biểu về trung bình quần thể khác với giá trị trung bình của riêng một mẫu.

4.2. p-value — Độ bất thường dưới giả định H_0

Từ vựng: p-value — giá trị p; **Significance level (α)** — mức ý nghĩa.

p-value là xác suất (nếu H_0 đúng và mô hình kiểm định phù hợp) thấy một thống kê kiểm định **ít nhất cực đoan như đã quan sát**. Nó **không** phải xác suất H_0 đúng hoặc xác suất kết quả do ngẫu nhiên. Ví dụ bảy điểm `[48,52,55,49,51,47,53]`, kiểm định hai phía so μ=50 bằng t-test cho p≈0.5347. Với α=0.05, chưa đủ bằng chứng để bác bỏ H_0; không có nghĩa chứng minh μ=50.

No Note — Không chú thích

```
from scipy.stats import ttest_1samp

scores = [48, 52, 55, 49, 51, 47, 53]
p = ttest_1samp(scores, popmean=50).pvalue
print(round(float(p), 4))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính p-value của kiểm định t một mẫu so với trung bình giả thuyết 50.
# Dữ liệu: 7 điểm học viên, H0 trung bình tổng thể bằng 50; không mặc định p-value là xác suất H0
đúng.
# ttest_1samp() dùng kiểm định t một mẫu khi so mean mẫu với mean giả thuyết.
```

```
from scipy.stats import ttest_1samp

# Bảy sô´điểm, môĩ sô´một học viên; giả định dữ liệu đủ phù hợp với kiểm định t.
scores = [48, 52, 55, 49, 51, 47, 53]
# popmean=50 là trung bình theo H0; mặc định alternative='two-sided' kiểm hai phía, trả đôí tượng
kết quả có statistic và pvalue.
# pvalue là mức xác suất thấý dữ liệu/kiểm định cực đoan ít nhất như hiện có NÊU H0 đúng, không
phải P(H0 đúng|dữ liệu).
p = ttest_1samp(scores, popmean=50).pvalue
# float() chuyển scalar sang sô´Python, round(...,4) chỉ làm tròn p-value khi in.
print(round(float(p), 4))
```

Kết quả khi chạy:

```
0.5347
```

p≈0,5347 lớn hơn 0,05; chưa đủ bằng chứng bác bỏ H_0 với kiểm định hai phía.

4.3. T-test — So trung bình mẫu với mốc cho trước

Từ vựng: T-test — kiểm định t; **Standard error** — sai số chuẩn.

T-test một mẫu xét liệu trung bình của quần thể có thể khác một mốc μ_0 khi chưa biết phương sai quần thể. Thống kê t bằng $(\text{mean_mẫu} - \mu_0)/SE$, với $SE = s/\sqrt{n}$ và s là độ lệch chuẩn mẫu. Nếu mẫu [8,9,10,11,12] và μ_0=10, mean bằng mốc nên t=0 và p hai phía=1. Cần xét tính độc lập và mức phù hợp của phân phối/độ nhạy với ngoại lệ, nhất là mẫu nhỏ.

No Note — Không chú thích

```
from scipy.stats import ttest_1samp

values = [8, 9, 10, 11, 12]
result = ttest_1samp(values, popmean=10)
print(round(float(result.statistic), 3))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc thông kê t của kiểm định một mâũ với trung bình giả thuyết 10.
# Dữ liệu: Năm giá trị [8,9,10,11,12], H0 trung bình tổng thể=10.
# SciPy trả kết quả kiểm định gồm statistic (t) và pvalue.
from scipy.stats import ttest_1samp

# Năm quan sát quanh môć 10, mean mâũ cũng bằng 10.
values = [8, 9, 10, 11, 12]
# So mean mâũ với 10 qua t-statistic dùng độ lệch chuẩn mâũ và căn n; mặc định kiểm hai phía.
result = ttest_1samp(values, popmean=10)
# statistic là giá trị t đo chênh lệch đã chuẩn hóa, KHÔNG phải p-value.
# Chuyển giá trị t sang float Python rôi hiển thị ba chữ sô´thập phân.
print(round(float(result.statistic), 3))
```

Kết quả khi chạy:

```
0.0
```

Vì mean mẫu đúng bằng 10, thống kê kiểm định t=0.

4.4. Confidence interval — Khoảng tạo bởi quy trình suy luận

Từ vựng: Confidence interval (CI) — khoảng tin cậy; **Coverage** — tỷ lệ bao phủ.

Khoảng tin cậy 95% là khoảng được tạo bởi **một quy trình** sao cho trong các lần lấy mẫu lặp phù hợp, khoảng 95% các khoảng chứa tham số thật. Với một khoảng **đã tính**, μ thật là cố định; không diễn giải "xác suất 95% μ ở trong khoảng này" theo nghĩa tần suất. Ví dụ năm thời gian giao `[2,3,3,4,2]` ngày cho mean=2,8 và CI t 95% xấp xỉ `[1.761,3.839]` ngày. Mẫu nhỏ nên khoảng rộng và dựa vào giả định dữ liệu độc lập, phân phối phù hợp.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import t, sem

days = np.array([2.0, 3.0, 3.0, 4.0, 2.0])
ci = t.interval(0.95, df=len(days) - 1, loc=days.mean(), scale=sem(days))
print(np.round(ci, 3).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xây khoảng tin cậy 95% cho mean của năm ngày bằng phân phôí t.
# Dữ liệu: Năm thời lượng ngày [2,3,3,4,2], không biết độ lệch chuẩn tổng thể, dùng t với df=n−1.
# NumPy tạo ndarray và làm tròn hai môć khoảng.
import numpy as np
# t là phân phôí Student t, sem() ước lượng sai sô´chuẩn của trung bình mâũ.
from scipy.stats import t, sem

# Năm sô´ngày float, môĩ phần tư ̉một quan sát.
days = np.array([2.0, 3.0, 3.0, 4.0, 2.0])
# interval(confidence,df,loc,scale) tìm hai cận của khoảng 95%; df=n−1=4; loc=mean mâũ;
scale=sem(days) là sai sô´chuẩn SD_mâũ/√n, không phải SD dữ liệu.
ci = t.interval(0.95, df=len(days) - 1, loc=days.mean(), scale=sem(days))
# np.round(...,3) làm tròn hai cận đê ̉in; tolist() chuyển̉ tuple/ndarray kêt́ quả thành list Python;
khoảng là đặc tính quy trình lâý mâũ, không phải xác suất mean cô´định nằm trong một khoảng đã cho.
print(np.round(ci, 3).tolist())
```

Kết quả khi chạy:

```
[1.761, 3.839]
```

Khoảng ước lượng mean theo t từ khoảng 1,761 đến 3,839 ngày; đây là khoảng cho trung bình quần thể.

5. Năm tình huống suy luận thống kê — từ kiểm định một mẫu đến Welch hai nhóm

Tình huống 1 — Cấp 1: Kiểm định trung bình 10 bài thi so với mốc 50

Từ vựng trong mục: Null hypothesis — giả thuyết H_0; **p-value** — giá trị p; **Two-sided t-test** — kiểm định hai phía.

Tình huống thực tế: Giảng viên có **10 bài kiểm tra**, mỗi bài một điểm **trên thang 100**; điểm đều quanh 50. Nhà trường đưa **mốc tham khảo 50 điểm** và muốn xem số liệu có cho bằng chứng rằng **điểm trung**

bình của nhóm học viên tương tự khác 50 hay không. Nhóm dùng **kiểm định t hai phía** với ngưỡng quyết định **5%**, chứ không tìm cách chứng minh mốc 50 chắc chắn đúng.

Vì sao chọn cách này? Giả thuyết không H_0 là mốc đang được thử, ở đây trung bình quần thể bằng **50 điểm**; **H_1 hai phía** là trung bình **khác 50**, có thể cao hơn hoặc thấp hơn. `ttest_1samp` dùng **10 điểm mẫu**, mức trung bình và độ phân tán của chúng để tính **p-value**: mức độ bất thường của kết quả như quan sát **nếu H_0 đúng**. So p-value với **α = 0,05** để quyết định có **bác bỏ H_0** hay chưa; p lớn không chứng minh H_0 chắc chắn đúng, và p nhỏ **không phải xác suất H_0 sai**. Với mẫu nhỏ cần xem giả định các điểm đủ độc lập và phân phối phù hợp.

Quy trình:

1. Ghi 10 điểm có cùng thang đo.
2. Dùng `ttest_1samp` với mốc 50 và kiểm định hai phía.
3. So p với α=0,05 và phát biểu 'không bác bỏ' hoặc 'bác bỏ', không gọi là chứng minh H_0.

No Note — Không chú thích

```
from scipy.stats import ttest_1samp
scores = [48,49,50,51,52,48,49,50,51,52]
result = ttest_1samp(scores, popmean=50)
if len(scores) != 10:
    raise ValueError("Cần đúng 10 điểm học viên.")
print("Học viên:", len(scores))
print(f"Mean (điểm): {sum(scores)/len(scores):.2f}")
print(f"p-value: {result.pvalue:.4f}")
print("Bác bỏ H0 ở 0.05:", bool(result.pvalue < 0.05))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So p-value kiểm định t của 10 điểm với ngưỡng ý nghĩa alpha=0.05.
# Dữ liệu: 10 điểm học viên quanh 50, giả thuyết trung bình tổng thể=50, kiểm hai phía.
# Kiểm định t một mẫu của SciPy trả statistic và pvalue.
from scipy.stats import ttest_1samp
# 10 điểm học viên, mỗi số một quan sát.
scores = [48,49,50,51,52,48,49,50,51,52]
# popmean=50 là mốc H0; mặc định hai phía, yêu cầu diễn giải dựa giả định phép t-test.
result = ttest_1samp(scores, popmean=50)
# len() đếm 10 quan sát trước khi kiểm định.
if len(scores) != 10:
    # ValueError ngừng xử lý nếu số người khác 10.
    raise ValueError("Cần đúng 10 điểm học viên.")
# len() đếm 10 quan sát trước khi kiểm định.
print("Học viên:", len(scores))
# len() đếm 10 quan sát trước khi kiểm định.
# Tổng điểm chia số học viên tính mean MẪU, khác mean=50 của H0.
print(f"Mean (điểm): {sum(scores)/len(scores):.2f}")
# pvalue biểu thị mức bất thường dưới H0, không phải xác suất H0 sai hay đúng.
print(f"p-value: {result.pvalue:.4f}")
# pvalue biểu thị mức bất thường dưới H0, không phải xác suất H0 sai hay đúng.
# So p với ngưỡng alpha=5%; True nghĩa bác bỏ H0 theo quy tắc đặt trước, False nghĩa chưa đủ bằng
chứng bác bỏ (không chứng minh H0 đúng).
# bool() ép kết luận thành Boolean Python để print().
print("Bác bỏ H0 ở 0.05:", bool(result.pvalue < 0.05))
```

Kết quả khi chạy:

```
Học viên: 10
Mean (điểm): 50.00
```

```
p-value: 1.0000
Bác bỏ H0 ở 0.05: False
```

p-value lớn không chứng minh mean quần thể **chính xác** 50; chỉ là bộ dữ liệu này chưa cung cấp bằng chứng để bác bỏ.

Tình huống 2 — Cấp 2: Khoảng tin cậy t cho 12 thời gian giao hàng

Từ vựng trong mục: Confidence interval — khoảng tin cậy; **Standard error** — sai số chuẩn.

Tình huống thực tế: Đơn vị giao hàng ghi **12 thời gian giao**, tính bằng **ngày**, đa số từ 2,7 tới 3,3 ngày. Người quản lý muốn ước lượng **thời gian giao trung bình của nhiều chuyến tương tự** và mức bất định của con số ấy, nên yêu cầu **khoảng tin cậy 95%** cho **trung bình**, cùng số trung bình và sai số chuẩn. Họ không hỏi khoảng mà 95% **chuyến riêng lẻ** sẽ nằm trong đó.

Vì sao chọn cách này? Khoảng tin cậy (CI) ở đây nhằm ước lượng **trung bình quần thể**, không phải dự đoán thời gian của từng chuyến. Vì chỉ có **12 chuyến** và chưa biết độ lệch chuẩn quần thể, bài dùng **phân phối t** với **11 bậc tự do**, dựa trên độ phân tán mẫu. `sem(values)` tính **SE**, tức mức bất định của trung bình ước lượng, còn `t.interval(0.95, ...)` dựng khoảng hai phía quanh mean mẫu với mức 95%. Diễn giải theo **phương pháp lặp lại lấy mẫu**, khoảng được tạo bằng cách này có mục tiêu bao phủ trung bình thật trong 95% lần khi giả định đúng; không nói mỗi chuyến riêng lẻ có 95% khả năng nằm trong khoảng này.

Quy trình:

1. Tạo đủ 12 thời gian theo cùng đơn vị ngày.
2. Tính mean, SE bằng `sem`, khoảng t 95% với df=11.
3. In hai đầu khoảng và ý nghĩa cho trung bình.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import t, sem
values = np.array([2.8,3,2.9,3.1,3.2,2.7,3,2.8,3.1,2.9,3,3.3], dtype=float)
se = sem(values)
ci = t.interval(0.95, df=len(values)-1, loc=values.mean(), scale=se)
if len(values) != 12:
    raise ValueError("Cần 12 lượt đo thời gian giao.")
print("Chuyến:", len(values))
print(f"Mean (ngày): {values.mean():.3f}")
print(f"SE (ngày): {se:.3f}")
print(f"CI 95% (ngày): {ci[0]:.3f} – {ci[1]:.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính SE và CI 95% của thời gian giao trên 12 chuyến, dùng df=11.
# Dữ liệu: 12 số ngày giao (2.7..3.3), giả định quan sát độc lập và mean tổng thể chưa biết.
# NumPy array/mean() giữ thời gian kiểu số thực.
import numpy as np
# sem() cho SE của mean, t.interval() dùng Student t để dựng CI.
from scipy.stats import t, sem
# 12 ngày giao tính theo ngày, dtype=float để giữ số thập phân.
values = np.array([2.8,3,2.9,3.1,3.2,2.7,3,2.8,3.1,2.9,3,3.3], dtype=float)
# sem(values) mặc định ddof=1 (SD mẫu), rồi chia √n để ước lượng sai số chuẩn của mean.
se = sem(values)
# Khoảng hai phía mức tin cậy 95%; df=11; loc=mean ngày; scale=SE ngày, không phải độ lệch chuẩn
các chuyến.
# len() đếm 12 chuyến, df lấy n−1.
```

```
# Mean thời gian của 12 chuyến, đơn vị ngày.
ci = t.interval(0.95, df=len(values)-1, loc=values.mean(), scale=se)
# len() đếm 12 chuyến, df lấy n−1.
if len(values) != 12:
    # Dừng khi số chuyến không bằng 12.
    raise ValueError("Cần 12 lượt đo thời gian giao.")
# len() đếm 12 chuyến, df lấy n−1.
print("Chuyến:", len(values))
# Mean thời gian của 12 chuyến, đơn vị ngày.
print(f"Mean (ngày): {values.mean():.3f}")
# f-string :.3f hiển thị sai số chuẩn ba chữ số thập phân (ngày).
print(f"SE (ngày): {se:.3f}")
# [0] là cận dưới của tuple CI, [1] là cận trên; :.3f định dạng khi in.
print(f"CI 95% (ngày): {ci[0]:.3f} – {ci[1]:.3f}")
```

Kết quả khi chạy:

```
Chuyến: 12
Mean (ngày): 2.983
SE (ngày): 0.051
CI 95% (ngày): 2.872 – 3.094
```

Ý nghĩa 95% gắn với **quy trình lặp lấy mẫu tạo khoảng**, không diễn giải thành xác suất 95% cho tham số cố định nằm trong một khoảng đã có.

Tình huống 3 — Cấp 3: Kiểm độ nhạy của kiểm định t với điểm ngoại lệ trong 15 mẫu

Từ vựng trong mục: Outlier sensitivity — nhạy với ngoại lệ; **Robust review** — rà soát chắc chắn.

Tình huống thực tế: Trung tâm khảo thí có **15 điểm bài kiểm tra trên thang 100**. **14 điểm quanh 49–51**, nhưng một bài được **70 điểm**, có thể là thành tích thật hoặc cần kiểm tra cách nhập. Nhóm muốn kiểm định trung bình có khác **mốc 50** không bằng đủ 15 bài, rồi thử tính thêm kết quả trên **14 bài đầu** để biết con số 70 ảnh hưởng kết luận thế nào. Bản gốc 15 bài vẫn phải được giữ nguyên.

Vì sao chọn cách này? Kiểm tra độ nhạy hỏi rằng kết quả phân tích **thay đổi bao nhiêu nếu một quan sát đặc biệt được xem riêng**. `ttest_1samp(scores, 50)` tính kiểm định hai phía trên **đủ 15 bài**, còn `scores[:-1]` tạo dữ liệu làm việc **14 bài đầu** để thử một phép so sánh phụ. Điểm 70 ảnh hưởng **cả trung bình và độ phân tán**, nên p-value có thể thay đổi, nhưng điểm 70 chưa phải nhập sai. Phải **giữ báo cáo chính của 15 bài**, giải thích lý do phân tích phụ và không chỉ công bố p-value mình thích; nếu cố xóa điểm cao để đạt ngưỡng, kết luận có thể bị thiên lệch.

Quy trình:

1. Lưu 15 điểm, chạy t-test đầy đủ.
2. Làm bản so sánh chỉ 14 điểm còn lại, không sửa dữ liệu gốc.
3. In mean và p của hai cách tính, báo lại số mẫu.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import ttest_1samp
scores = np.array([49,50,50,51,49,50,51,50,49,50,51,50,50,49,70], dtype=float)
full = ttest_1samp(scores, 50)
without_last = ttest_1samp(scores[:-1], 50)
if len(scores) != 15:
    raise ValueError("Cần 15 học viên khi so tác động ngoại lệ.")
print("Học viên:", len(scores))
```

```
print(f"Mean đủ 15: {scores.mean():.3f}")
print(f"p đủ 15: {full.pvalue:.3f}")
print(f"p khi tạm xem 14: {without_last.pvalue:.3f}")
print("Vẫn giữ bản gốc:", len(scores) == 15)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So tác động một điểm ngoại lệ lên kiểm định t khi xét 15 điểm rồi tạm bỏ điểm cuối.
# Dữ liệu: 15 học viên, điểm cuối 70 cao bất thường; nhóm 14 là thư độ nhạy, không được xóa dữ liệu gốc.
# NumPy tạo mảng và tính mean mẫu.
import numpy as np
# SciPy tính t-test một mẫu so với mốc H0.
from scipy.stats import ttest_1samp
# 15 điểm, trong đó điểm cuối 70 là giá trị ngoại lệ có thể ảnh hưởng cả mean và SD.
scores = np.array([49,50,50,51,49,50,51,50,49,50,51,50,50,49,70], dtype=float)
# Dùng popmean=50 theo vị trí thứ hai của hàm; mặc định alternative='two-sided'.
full = ttest_1samp(scores, 50)
# Slice [:-1] bỏ CHỈ phần tử cuối trong một view tạm gồm 14 điểm, mảng scores gốc vẫn có 15 điểm.
without_last = ttest_1samp(scores[:-1], 50)
# len() kiểm bảng gốc vẫn 15 học viên để không nhầm bản thư 14 là dữ liệu mới.
if len(scores) != 15:
    # Báo lỗi nếu thiếu dữ liệu ngoại lệ/không đủ học viên.
    raise ValueError("Cần 15 học viên khi so tác động ngoại lệ.")
# len() kiểm bảng gốc vẫn 15 học viên để không nhầm bản thư 14 là dữ liệu mới.
print("Học viên:", len(scores))
# Mean của toàn bộ 15 điểm, có thể bị điểm 70 kéo lên.
print(f"Mean đủ 15: {scores.mean():.3f}")
# p-value hai phía của toàn bộ 15 điểm; việc ngoại lệ thay mean và phương sai có thể thay p không theo chiều đơn giản.
print(f"p đủ 15: {full.pvalue:.3f}")
# p-value của 14 điểm khi tạm loại giá trị cuối để minh họa độ nhạy; không tự động hợp thức hóa việc loại dữ liệu.
print(f"p khi tạm xem 14: {without_last.pvalue:.3f}")
# len() kiểm bảng gốc vẫn 15 học viên để không nhầm bản thư 14 là dữ liệu mới.
print("Vẫn giữ bản gốc:", len(scores) == 15)
```

Kết quả khi chạy:

```
Học viên: 15
Mean đủ 15: 51.267
p đủ 15: 0.364
p khi tạm xem 14: 0.720
Vẫn giữ bản gốc: True
```

Cùng bộ dữ liệu nhưng lựa chọn quan sát đem vào kiểm định có thể đổi p-value; việc loại mẫu phải có lý do dữ liệu minh bạch.

Tình huống 4 — Cấp 4: Một p-value nhưng hai quyết định alpha từ 18 kết quả

Từ vựng trong mục: Significance level — mức ý nghĩa α; **Decision rule** — quy tắc quyết định.

Tình huống thực tế: Phòng đào tạo có **18 điểm kiểm tra thang 100**, mỗi điểm thuộc một học viên. Giảng viên đang kiểm **trung bình điểm có khác mốc 50** hay không. Nhóm phải xem **cùng một p-value** với hai mức ý nghĩa **0,05 (5%)** và **0,01 (1%)**, để hiểu vì sao mức yêu cầu bằng chứng khắt khe hơn có thể đưa tới quyết định khác dù dữ liệu không đổi.

Vì sao chọn cách này? Mức ý nghĩa α là ngưỡng nhóm nghiên cứu **đặt trước khi xem kết quả kiểm định**, biểu thị mức chấp nhận nguy cơ **bác bỏ H_0 khi H_0 thật sự đúng** theo quy trình. **p-value** được tính

từ **18 điểm dữ liệu và cùng một kiểm định t hai phía**, nên **không đổi** khi chỉ đổi ngưỡng. Nếu p nằm giữa **0,01 và 0,05**, nó đủ nhỏ để bác bỏ ở 5% nhưng **chưa đủ nhỏ** để bác bỏ ở 1%. Hai quyết định không mâu thuẫn vì dùng **tiêu chuẩn bằng chứng khác nhau**; không chọn ngưỡng sau khi nhìn p-value để đạt kết luận thuận ý.

Quy trình:

1. Tạo 18 điểm và chạy kiểm định hai phía một mẫu.
2. Tính p một lần rồi so với cả hai α.
3. Báo quyết định ở từng ngưỡng và nhắc đây là bằng chứng, không phải xác suất H_0 đúng.

No Note — Không chú thích

```
from scipy.stats import ttest_1samp
scores = [48,50,51,52,53,51,50,49,52,53,51,50,52,49,51,52,50,53]
p = float(ttest_1samp(scores, popmean=50).pvalue)
if len(scores) != 18:
    raise ValueError("Cần 18 điểm học viên khi so mức ý nghĩa.")
print("Học viên:", len(scores))
print(f"p-value: {p:.4f}")
print("Bác bỏ H0 ở 0.05:", p < 0.05)
print("Bác bỏ H0 ở 0.01:", p < 0.01)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So cùng một p-value ở alpha=0.05 và alpha=0.01 để thấy ngưỡng quyết định khác nhau.
# Dữ liệu: 18 điểm học viên, H0 mean=50, alpha 5% và 1% do người nghiên cứu quyết định trước.
# ttest_1samp tính p-value hai phía so mean mẫu với mốc 50.
from scipy.stats import ttest_1samp
# 18 điểm cùng thang, mỗi số một học viên.
scores = [48,50,51,52,53,51,50,49,52,53,51,50,52,49,51,52,50,53]
# popmean=50 là giá trị H0, mặc định kiểm hai phía.
# Lấy pvalue từ kết quả t-test, không phải posterior của H0.
# float() đổi scalar p-value sang số Python.
p = float(ttest_1samp(scores, popmean=50).pvalue)
# len() xác minh 18 điểm trước khi kết luận.
if len(scores) != 18:
    # Dừng nếu thiếu số mẫu theo yêu cầu.
    raise ValueError("Cần 18 điểm học viên khi so mức ý nghĩa.")
# len() xác minh 18 điểm trước khi kết luận.
print("Học viên:", len(scores))
# p-value hiển thị 4 số thập phân, quyết định dùng giá trị p gốc không làm tròn.
print(f"p-value: {p:.4f}")
# Với alpha=0.05: bác bỏ H0 nếu p nhỏ hơn 0.05; ngưỡng cao hơn dễ bác bỏ hơn.
print("Bác bỏ H0 ở 0.05:", p < 0.05)
# Với alpha=0.01: cần bằng chứng mạnh hơn (p nhỏ hơn 0.01); dùng CÙNG p, không chạy kiểm định khác.
print("Bác bỏ H0 ở 0.01:", p < 0.01)
```

Kết quả khi chạy:

```
Học viên: 18
p-value: 0.0146
Bác bỏ H0 ở 0.05: True
Bác bỏ H0 ở 0.01: False
```

Chưa bác bỏ ở α=0,01 **không đồng nghĩa** chứng minh H_0 đúng; điều đó chỉ phản ánh ngưỡng bằng chứng khắt khe hơn.

Tình huống 5 — Cấp 5: So hai nhóm 10+10 bằng Welch t-test và CI chênh lệch

Từ vựng trong mục: Welch t-test — kiểm định hai mẫu không cần giả định phương sai bằng nhau; **Difference CI** — khoảng chênh lệch mean.

Tình huống thực tế: Trung tâm tổ chức kiểm tra kỹ năng cho **20 học viên**, chia **hai nhóm độc lập A và B, mỗi nhóm 10 người**, điểm trên thang **100**. Nhóm B có điểm nhìn chung cao hơn A. Người đánh giá muốn biết **trung bình B cao hơn A bao nhiêu điểm**, kiểm định liệu trung bình của hai nhóm quần thể có thể khác nhau hay không và lập **khoảng tin cậy 95% cho chênh lệch B trừ A**. Hai nhóm là những người khác nhau, không phải một người làm bài trước rồi làm lại.

Vì sao chọn cách này? Hai mẫu độc lập nghĩa là mỗi học viên thuộc **A hoặc B**, không có ghép cặp cùng người; vì vậy không dùng kiểm định cặp trước–sau. `ttest_ind(B, A, equal_var=False)` thực hiện **Welch t-test**, kiểm trung bình quần thể hai nhóm mà **không buộc phương sai bằng nhau**. Chương trình tính **chênh lệch trung bình B–A**, rồi **SE cho hiệu** từ độ phân tán và số người của cả hai nhóm, cùng **bậc tự do Welch** để dựng **CI 95% cho hiệu trung bình**. Khoảng không chứa **0** là bằng chứng chênh lệch theo mức tin cậy đang xét; nó **không phải khoảng chứa điểm của 95% học viên** và cũng không chứng minh nhóm B tốt hơn vì một nguyên nhân cụ thể.

Quy trình:

1. Tạo hai nhóm 10 điểm, kiểm không trộn thứ tự giữa nhóm.
2. Chạy `ttest_ind(..., equal_var=False)` và tính SE hiệu mean.
3. Tạo khoảng t Welch 95% cho B–A, báo chênh lệch, p và hai đầu khoảng.

No Note — Không chú thích

```
import numpy as np
from scipy.stats import ttest_ind, t
A = np.array([45,47,46,49,48,47,46,50,49,48], dtype=float)
B = np.array([50,52,51,54,53,52,51,55,54,53], dtype=float)
result = ttest_ind(B, A, equal_var=False)
va = A.var(ddof=1)/len(A)
vb = B.var(ddof=1)/len(B)
se = np.sqrt(va + vb)
df = (va+vb)**2/(va**2/(len(A)-1)+vb**2/(len(B)-1))
difference = B.mean()-A.mean()
ci = t.interval(0.95, df=df, loc=difference, scale=se)
if len(A) + len(B) != 20:
    raise ValueError("Cần đủ 20 học viên chia hai nhóm.")
print("Học viên:", len(A)+len(B))
print("Nhóm A/B:", len(A), len(B))
print(f"Chênh lệch B-A (điểm): {difference:.2f}")
print(f"p-value: {result.pvalue:.6f}")
print(f"CI 95% (điểm): {ci[0]:.3f} – {ci[1]:.3f}")
print("CI không chứa 0:", not (ci[0] <= 0 <= ci[1]))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng Welch t-test so mean hai nhóm 10 người và tính khoảng tin cậy cho chênh lệch B-A.
# Dữ liệu: Nhóm A/B mỗi nhóm 10 điểm; quan tâm chênh lệch trung bình điểm B trừ A, không giả định
hai nhóm có cùng phương sai.
# NumPy tính mean, var, sqrt và số mẫu cho hai nhóm.
import numpy as np
# ttest_ind() so hai nhóm độc lập, t.interval() xây CI từ trung bình, SE và bậc tự do.
from scipy.stats import ttest_ind, t
# 10 điểm nhóm A, dtype=float giữ số liệu số thực.
A = np.array([45,47,46,49,48,47,46,50,49,48], dtype=float)
```

```
# 10 điểm nhóm B, cùng thang với A; giả định hai nhóm độc lập.
B = np.array([50,52,51,54,53,52,51,55,54,53], dtype=float)
# Welch t-test: equal_var=False KHÔNG áp giả định phương sai hai nhóm bằng nhau; B trước A để chênh
theo chiều B−A; mặc định kiểm hai phía.
result = ttest_ind(B, A, equal_var=False)
# ddof=1 lấy phương sai mẫu (chia nA−1); chia tiếp số người nA tạo phần đóng góp vào phương sai của
hiệu mean.
va = A.var(ddof=1)/len(A)
# Phương sai mẫu B /nB là phần đóng góp riêng, không trộn hai nhóm bằng một phương sai chung.
vb = B.var(ddof=1)/len(B)
# sqrt(va+vb) là standard error của hiệu trung bình hai nhóm ĐỘC LẬP.
se = np.sqrt(va + vb)
# Công thức Welch–Satterthwaite cho bậc tự do hiệu dụng, có thể không nguyên; khác nA+nB−2 khi giả
phương sai bằng nhau.
df = (va+vb)**2/(va**2/(len(A)-1)+vb**2/(len(B)-1))
# difference tính điểm trung bình nhóm B trừ A, dấu dương nghĩa B cao hơn A.
difference = B.mean()-A.mean()
# CI hai phía 95% dùng Student t, df Welch, loc=ước lượng B−A, scale=SE của chênh lệch, đơn vị
điểm.
ci = t.interval(0.95, df=df, loc=difference, scale=se)
# Xác minh có 20 học viên ở hai nhóm, mỗi người chỉ thuộc một nhóm theo giả định.
if len(A) + len(B) != 20:
    # Báo thiếu số mẫu để tránh so hai nhóm không đúng kích thước.
    raise ValueError("Cần đủ 20 học viên chia hai nhóm.")
print("Học viên:", len(A)+len(B))
print("Nhóm A/B:", len(A), len(B))
print(f"Chênh lệch B-A (điểm): {difference:.2f}")
# p-value Welch kiểm H0 hai nhóm có mean bằng nhau, không phải xác suất H0 đúng.
print(f"p-value: {result.pvalue:.6f}")
# [0] và [1] là cận dưới/trên CI, in ba chữ số thập phân.
print(f"CI 95% (điểm): {ci[0]:.3f} – {ci[1]:.3f}")
# [0] và [1] là cận dưới/trên CI, in ba chữ số thập phân.
# Kiểm số 0 (không chênh lệch) nằm trong khoảng; not đảo để báo CI không chứa 0, tương ứng bác bỏ ở
mức 0.05 cho kiểm hai phía theo giả định chuẩn.
print("CI không chứa 0:", not (ci[0] <= 0 <= ci[1]))
```

Kết quả khi chạy:

```
Học viên: 20
Nhóm A/B: 10 10
Chênh lệch B-A (điểm): 5.00
p-value: 0.000001
CI 95% (điểm): 3.514 – 6.486
CI không chứa 0: True
```

Chênh lệch nhóm trong mẫu có thể do nhiều yếu tố khác ngoài cách dạy; kiểm định không tự chứng minh tác động nhân quả.

6. Sai lầm thường gặp

- p-value không phải xác suất H_0 đúng; nó là xác suất của kết quả cực đoan dưới giả định H_0.
- Không bác bỏ H_0 **không** đồng nghĩa chấp nhận hay chứng minh H_0 đúng.
- α=0.05 là ngưỡng kiểm định lựa chọn trước, không phải ngưỡng hiệu quả thực tế.
- CI 95% có **tỷ lệ bao phủ dài hạn 95%** khi lặp quy trình, không phải xác suất 95% tham số cố định trong một khoảng đã quan sát.
- SE khác SD: SE nói về bất định của mean, SD nói về số đo riêng lẻ.
- T-test trên mẫu nhỏ nhạy với ngoại lệ/phụ thuộc; cần xem giả định và thiết kế lấy mẫu.

7. Trắc nghiệm

Q1. What is H0? | H_0 là gì?

A. A null hypothesis subjected to statistical testing | Giả thuyết không được đem kiểm định

B. An interval from a procedure with 95% long-run coverage | Khoảng từ quy trình có mức bao phủ dài hạn 95%

C. Probability of equally or more extreme test results under H0 | Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

D. Test a population mean against a specified value | Kiểm định trung bình quần thể với một mốc

Đáp án: A

Giải thích: Đó là giả định gốc của kiểm định. Ví dụ H_0: μ=50 nói về mean quần thể; không đòi mean mẫu quan sát phải đúng 50.

Q2. What does p-value mean? | p-value nghĩa là gì?

A. Probability of equally or more extreme test results under H0 | Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

B. Test a population mean against a specified value | Kiểm định trung bình quần thể với một mốc

C. An interval from a procedure with 95% long-run coverage | Khoảng từ quy trình có mức bao phủ dài hạn 95%

D. A null hypothesis subjected to statistical testing | Giả thuyết không được đem kiểm định

Đáp án: A

Giải thích: Nó được tính với điều kiện mô hình H_0 đúng. p-value được tính từ thống kê kiểm định và phân phối dưới H_0, chứ không trực tiếp gán xác suất cho H_0.

Q3. What is a one-sample t-test for? | T-test một mẫu dùng để làm gì?

A. A null hypothesis subjected to statistical testing | Giả thuyết không được đem kiểm định

B. Test a population mean against a specified value | Kiểm định trung bình quần thể với một mốc

C. Probability of equally or more extreme test results under H0 | Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

D. An interval from a procedure with 95% long-run coverage | Khoảng từ quy trình có mức bao phủ dài hạn 95%

Đáp án: B

Giải thích: Nó dùng trung bình và ước lượng phương sai mẫu. Khác kiểm định hai mẫu, t-test một mẫu so trung bình một mẫu với mốc cố định.

Q4. What is a 95% confidence interval? | Khoảng tin cậy 95% là gì?

A. A null hypothesis subjected to statistical testing | Giả thuyết không được đem kiểm định

B. Test a population mean against a specified value | Kiểm định trung bình quần thể với một mốc

C. An interval from a procedure with 95% long-run coverage | Khoảng từ quy trình có mức bao phủ dài hạn 95%

D. Probability of equally or more extreme test results under H0 | Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

Đáp án: C

Giải thích: Bao phủ được hiểu qua những lần lấy mẫu lặp. Mức bao phủ 95% thuộc quy trình qua nhiều mẫu; một CI đã tính cần báo hai đầu mút và giả định đi kèm.

Q5. A hypothesis test has p-value = 0.03 and significance level α = 0.05. What is the usual decision? | Kiểm định có p-value = 0.03 và mức ý nghĩa α = 0.05. Thường đưa ra quyết định nào?

A. Set p-value to zero | Đặt p-value bằng 0

B. Accept H0 as proven | Chứng minh H0 đúng

C. Double the sample size automatically | Tự động tăng gấp đôi mẫu

D. Reject H0 | Bác bỏ H0

Đáp án: D

Giải thích: Vì 0.03 < 0.05, dữ liệu đủ bằng chứng để bác bỏ H0 ở mức ý nghĩa đã chọn; không chứng minh H1 chắc chắn đúng.

Q6. A hypothesis test has p-value = 0.15 and α = 0.05. Which statement is appropriate? | Kiểm định có p-value = 0.15 và α = 0.05. Phát biểu nào phù hợp?

A. Reject H0 | Bác bỏ H0

B. H0 is proven true | H0 đã được chứng minh đúng

C. Fail to reject H0 | Chưa đủ bằng chứng để bác bỏ H0

D. The sample mean must equal zero | Trung bình mẫu phải bằng 0

Đáp án: C

Giải thích: 0.15 ≥ 0.05 nên chưa đủ bằng chứng bác bỏ H0. Điều này không có nghĩa đã chứng minh H0 đúng.

Q7. A 95% two-sided confidence interval contains the null value. What normally happens in the matching two-sided test at α = 0.05? | Khoảng tin cậy 95% hai phía chứa giá trị giả thuyết H0. Trong kiểm định hai phía tương ứng với α = 0.05, thường kết luận gì?

A. H0 is definitely true | H0 chắc chắn đúng

B. Reject H0 | Bác bỏ H0

C. Fail to reject H0 | Chưa bác bỏ H0

D. The p-value is exactly 0 | p-value chính xác bằng 0

Đáp án: C

Giải thích: Với cùng mô hình và giả định, khi giá trị H0 nằm trong khoảng tin cậy 95% thì kiểm định hai phía tương ứng ở mức 5% không bác bỏ H0.

Q8. Which description of a p-value is correct? | Mô tả nào đúng về p-value?

A. Probability of observing results this extreme or more if H0 were true | Xác suất gặp kết quả cực đoan bằng hoặc hơn nếu H0 đúng

B. Probability H0 is true | Xác suất H0 đúng

C. Probability H1 is true | Xác suất H1 đúng

D. Probability the data were measured correctly | Xác suất dữ liệu được đo đúng

Đáp án: A

Giải thích: p-value xét mức độ bất thường của dữ liệu theo giả định H0 đúng, không phải xác suất H0 đúng.

Q9. How is a one-sample t statistic formed? | Thống kê t một mẫu có dạng gì?

A. n−1 | n−1

B. Lower and upper endpoints of a t interval | Hai đầu dưới/trên của khoảng t

C. (sample mean−mu0)/SE | (mean mẫu−mu0)/SE

D. Standard error of the mean | Sai số chuẩn của trung bình

Đáp án: C

Giải thích: Nó chuẩn hóa độ lệch của mean so với mốc. Chênh hai điểm chỉ có ý nghĩa kiểm định khi so với độ bất định lấy mẫu của mean.

Q10. What does `scipy.stats.sem` estimate? | `scipy.stats.sem` ước lượng gì?

A. n−1 | n−1

B. Lower and upper endpoints of a t interval | Hai đầu dưới/trên của khoảng t

C. (sample mean−mu0)/SE | (mean mẫu−mu0)/SE

D. Standard error of the mean | Sai số chuẩn của trung bình

Đáp án: D

Giải thích: Nó ước lượng độ dao động của mean mẫu. SE có xu hướng giảm khi n tăng dưới giả định phù hợp, còn SD của từng điểm không nhất thiết giảm.

Q11. What does t.interval return? | t.interval trả gì?

A. Lower and upper endpoints of a t interval | Hai đầu dưới/trên của khoảng t

B. (sample mean−mu0)/SE | (mean mẫu−mu0)/SE

C. Standard error of the mean | Sai số chuẩn của trung bình

D. n−1 | n−1

Đáp án: A

Giải thích: Khoảng phụ thuộc mức tin cậy, df, vị trí và độ rộng. Scale lớn làm khoảng rộng; loc là ước lượng trung tâm chứ không phải phần trăm tin cậy.

Q12. What degrees of freedom are usual for one-sample t inference? | Bậc tự do thông dụng của t một mẫu là gì?

A. n−1 | n−1

B. (sample mean−mu0)/SE | (mean mẫu−mu0)/SE

C. Lower and upper endpoints of a t interval | Hai đầu dưới/trên của khoảng t

D. Standard error of the mean | Sai số chuẩn của trung bình

Đáp án: A

Giải thích: Ước lượng phương sai mẫu làm mất một bậc tự do. Thống kê t ước lượng phương sai từ mẫu nên thường còn n−1 bậc tự do.

Q13. Is p-value equal to P(H0 is true)? | p-value có bằng P(H_0 đúng) không?

A. No; p is conditional on H0 | Không; p được tính khi giả định H_0 đúng

B. Failing to reject a false H0 | Không bác bỏ H_0 khi H_0 sai

C. It does not reject at the matching alpha | Không bác bỏ với alpha tương ứng

D. Rejecting a true null hypothesis | Bác bỏ H_0 khi H_0 thực sự đúng

Đáp án: A

Giải thích: p theo tần suất không phải posterior. Nhầm p với P(H_0|dữ liệu) là đảo chiều điều kiện; kiểm định tần suất xét mức cực đoan của dữ liệu khi H_0 đúng.

Q14. If a confidence interval covers a null value, what often happens in a matching two-sided test? | Nếu CI chứa mốc H_0 thì kiểm định hai phía tương ứng thường thế nào?

A. Rejecting a true null hypothesis | Bác bỏ H_0 khi H_0 thực sự đúng

B. No; p is conditional on H0 | Không; p được tính khi giả định H_0 đúng

C. Failing to reject a false H0 | Không bác bỏ H_0 khi H_0 sai

D. It does not reject at the matching alpha | Không bác bỏ với alpha tương ứng

Đáp án: D

Giải thích: Các quy trình tương ứng thường thống nhất dưới cùng giả định. Sự tương ứng cần kiểm định hai phía, mức tin cậy và giả định phù hợp; không đúng với mọi loại kiểm định và khoảng.

Q15. What is a type I error? | Sai lầm loại I là gì?

A. Rejecting a true null hypothesis | Bác bỏ H_0 khi H_0 thực sự đúng

B. No; p is conditional on H0 | Không; p được tính khi giả định H_0 đúng

C. Failing to reject a false H0 | Không bác bỏ H_0 khi H_0 sai

D. It does not reject at the matching alpha | Không bác bỏ với alpha tương ứng

Đáp án: A

Giải thích: Alpha kiểm soát lỗi loại I dưới giả định kiểm định. Sai lầm loại I là báo dương sai; alpha được chọn trước để kiểm soát xác suất lỗi này dưới giả định H_0.

Q16. What is a type II error? | Sai lầm loại II là gì?

A. It does not reject at the matching alpha | Không bác bỏ với alpha tương ứng

B. Rejecting a true null hypothesis | Bác bỏ H_0 khi H_0 thực sự đúng

C. No; p is conditional on H0 | Không; p được tính khi giả định H_0 đúng

D. Failing to reject a false H0 | Không bác bỏ H_0 khi H_0 sai

Đáp án: D

Giải thích: Nó khác lỗi bác bỏ H_0 đúng. Power bằng một trừ xác suất sai lầm loại II dưới giả thuyết đối cụ thể, liên quan đến thiết kế cỡ mẫu.

Q17. Does a 95% CI say the fixed parameter has 95% probability of lying in this realized interval? | CI 95% có nghĩa xác suất tham số cố định trong khoảng đã tính là 95%?

A. No; 95% describes repeated-interval coverage | Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. No; effect size and context still matter | Không; còn kích thước tác động và bối cảnh

C. Strong dependence or problematic extreme data in small samples | Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

D. It becomes wider | Trở nên rộng hơn

Đáp án: A

Giải thích: Trong diễn giải tần suất, tham số cố định và khoảng thay đổi theo mẫu. Phát biểu xác suất áp dụng cho quy trình tạo khoảng ngẫu nhiên trước lấy mẫu, không phải μ cố định sau khi quan sát một CI.

Q18. What can violate simple t-test assumptions? | Điều gì có thể vi phạm giả định t-test đơn giản?

A. No; 95% describes repeated-interval coverage | Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. No; effect size and context still matter | Không; còn kích thước tác động và bối cảnh

C. It becomes wider | Trở nên rộng hơn

D. Strong dependence or problematic extreme data in small samples | Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

Đáp án: D

Giải thích: Suy luận cần mô hình lấy mẫu phù hợp. Mẫu rất nhỏ, ngoại lệ nặng hoặc phụ thuộc có thể khiến p-value và CI báo ra thiếu tin cậy.

Q19. What happens to CI width if SE grows with other inputs fixed? | SE tăng khi yếu tố khác giữ nguyên thì CI rộng ra sao?

A. Strong dependence or problematic extreme data in small samples | Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

B. No; 95% describes repeated-interval coverage | Không; 95% mô tả tỷ lệ bao phủ khi lặp

C. No; effect size and context still matter | Không; còn kích thước tác động và bối cảnh

D. It becomes wider | Trở nên rộng hơn

Đáp án: D

Giải thích: Bất định cao làm khoảng rộng hơn. Trong khoảng t, SE tăng làm biên `t_critical × SE` tăng, khiến hai đầu mút cách mean xa hơn.

Q20. Is statistical significance identical to practical importance? | Ý nghĩa thống kê có giống giá trị thực tiễn không?

A. No; 95% describes repeated-interval coverage | Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. Strong dependence or problematic extreme data in small samples | Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

C. No; effect size and context still matter | Không; còn kích thước tác động và bối cảnh

D. It becomes wider | Trở nên rộng hơn

Đáp án: C

Giải thích: p nhỏ không chứng minh hiệu quả thực tế lớn. Ngoài p-value cần báo kích thước tác động và độ bất định; tác động rất nhỏ có thể có ý nghĩa thống kê nhưng không hữu ích thực tế.

8. Liên hệ bài kế tiếp

`CLT/SE → Hypothesis Testing → Confidence Interval → A/B Test.`

PY-41 — Phân hoạch dữ liệu, classification và clustering

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Train set — tập huấn luyện; **Test set** — tập kiểm tra.

`Set/NumPy → Train/Test → Preprocessing → Classification/Clustering`

Học xong dùng để làm gì? Tách tập dữ liệu đúng cách, huấn luyện mô hình và đánh giá kết quả trên dữ liệu độc lập.

Kết nối kiến thức tiếp theo: PY-42 chuẩn bị dữ liệu chữ thành token và đặc trưng phục vụ mô hình.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Train set	tập huấn luyện	Mẫu dùng để học tham số mô hình
Test set	tập kiểm tra	Mẫu giữ lại chỉ để đánh giá mô hình
Classification	phân loại	Dự đoán nhãn rời rạc từ feature
Clustering	phân cụm	Thử nhóm mẫu giống nhau mà không cần nhãn sẵn

3. Thư viện và hàm Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cách viết	Khi nào dùng, ý nghĩa tham số
`train_test_split(...,test_size=.25,stratify=y)`	Chia train/test có xét tỷ lệ nhãn
`Pipeline([('scale',...),('model',...)])`	Fit chỉ trên train để giảm leakage
`LogisticRegression(max_iter=1000)`	Mô hình phân loại có giám sát
`KMeans(n_clusters=2,random_state=42,n_init=10)`	Phân nhóm không giám sát, số nhóm là giả định

4. Khái niệm chính và liên quan

Mạch kiến thức: Để đánh giá mô hình đáng tin, trước hết chia dữ liệu thành train và test. Classification học cách đoán nhãn từ ví dụ đã có nhãn; clustering nhóm các điểm tương tự khi chưa có nhãn. Một test độc lập giúp tránh tự đánh giá trên dữ liệu đã học.

4.1. Train set — Tập để học tham số

Từ vựng: Train set.

Dữ liệu train chứa các hàng quan sát dùng để mô hình **học** tham số. Trong bảng Iris, mỗi hàng là một hoa, bốn cột là kích thước tính bằng cm và nhãn là loài. Nếu lấy 112 hoa để train thì các đặc trưng của chúng được đưa vào `fit`; không đưa test vào học scaler hoặc chỉnh siêu tham số.

No Note — Không chú thích

```
from sklearn.datasets import load_iris

data = load_iris()
print(data.data.shape, data.target.shape)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tải bộ Iris và quan sát kích thước đầu vào X và nhãn y để hiểu dữ liệu huấn luyện.
# Dữ liệu: Bộ Iris mẫu của scikit-learn chứa 150 hoa, bốn đặc trưng đo (cm) và một nhãn loài/hoa.
# load_iris() cung cấp bộ dữ liệu mẫu Iris đã gán nhãn; không tự tải dữ liệu riêng của học viên.
from sklearn.datasets import load_iris

# Hàm trả Bunch chứa data (đặc trưng) và target (nhãn lớp), có thể truy xuất bằng thuộc tính.
data = load_iris()
# data.data.shape là tuple (150 hàng mẫu,4 cột đặc trưng), khác data.target.shape (150,) là một
nhãn mỗi mẫu.
# target là vector nhãn nguyên theo mỗi hoa, không phải dữ liệu dùng làm feature.
print(data.data.shape, data.target.shape)
```

Kết quả khi chạy:

```
(150, 4) (150,)
```

150 hàng hoa, bốn cột đặc trưng và một nhãn cho mỗi hàng.

4.2. Test set — Tập chỉ dùng để đánh giá cuối

Từ vựng: Test set.

Test set là các mẫu giữ lại, không dùng để fit hay chọn cách xử lý. Một cách kiểm tra tối thiểu là **ID train không trùng ID test**. Điều đó chưa đủ loại rò rỉ qua các mẫu liên quan, đặc trưng sinh từ tương lai, hoặc bản ghi trùng nội dung nhưng khác ID.

No Note — Không chú thích

```
train_ids = {1, 2, 3}
test_ids = {4, 5}
print(train_ids.isdisjoint(test_ids))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm các tập Train và Test không dùng chung ID để tránh rò rỉ mẫu.
# Dữ liệu: Train gồm mã 1,2,3; Test gồm 4,5; mỗi mã đại diện một mẫu.
# Dấu {} chứa một set: mỗi mã chỉ xuất hiện một lần.
train_ids = {1, 2, 3}
# Dùng set riêng cho test để kiểm giao với train.
```

```
test_ids = {4, 5}
# isdisjoint() trả True khi hai set không có phần tử chung, quan trọng để kiểm tra không đánh giá trên chính
mẫu train.
print(train_ids.isdisjoint(test_ids))
```

Kết quả khi chạy:

```
True
```

Không có ID xuất hiện ở cả train và test; vẫn cần kiểm tra rò rỉ gián tiếp.

4.3. Classification — Dự đoán nhãn đã biết

Từ vựng: Classification.

Classification là học có giám sát: từ **X là các đặc trưng**, mô hình dự đoán **y là nhãn**. Iris có ba nhãn loài là 0,1,2. Một classifier học từ những hàng đã có nhãn rồi dự đoán loài của hoa mới; khác regression dự đoán giá trị liên tục và không thể thay độ chính xác bằng khoảng cách giữa nhãn số.

No Note — Không chú thích

```
from sklearn.datasets import load_iris

data = load_iris()
print(sorted(set(data.target.tolist())))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tìm những nhãn phân loại có sẵn trong Iris qua tập hợp các nhãn.
# Dữ liệu: Iris có 3 lớp nhãn mã 0,1,2 cho ba loài.
# load_iris() trả bộ dữ liệu Iris và nhãn lớp.
from sklearn.datasets import load_iris

# target lưu các mã lớp cho từng trong 150 hàng.
data = load_iris()
# tolist() đổi ndarray nhãn NumPy thành list Python.
# set() bỏ các nhãn lặp, chỉ còn mã lớp riêng biệt.
# sorted() xếp nhãn tăng dần để hiển thị ổn định, không đổi nhãn nguồn.
print(sorted(set(data.target.tolist())))
```

Kết quả khi chạy:

```
[0, 1, 2]
```

Ba số là mã phân loại ba loài, không phải các đại lượng đo theo cm.

4.4. Clustering — Nhóm mẫu khi chưa có nhãn

Từ vựng: Clustering.

Clustering là học không giám sát: tìm các nhóm điểm gần nhau trong không gian đặc trưng, **không dùng nhãn thật để huấn luyện**. Ví dụ hai cụm điểm quanh `[0,0]` và `[10,10]`; KMeans tìm hai tâm nhóm. Mã cụm `0/1` chỉ là định danh tùy cách khởi tạo, không đồng nghĩa nhãn lớp 0/1 của bài phân loại.

No Note — Không chú thích

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[0, 0], [0, 1], [10, 10], [10, 11]])
labels = KMeans(n_clusters=2, random_state=42, n_init=10).fit_predict(X)
print(len(set(labels.tolist())))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Thử KMeans gom bốn điểm thành hai cụm mà không sử dụng nhãn đúng.
# Dữ liệu: Bốn điểm tọa độ x/y hình thành hai nhóm xa nhau gần (0,0) và (10,10).
# NumPy tạo ma trận hai cột tọa độ cho thuật toán phân cụm.
import numpy as np
# KMeans là thuật toán phân cụm không giám sát, không yêu cầu nhãn y.
from sklearn.cluster import KMeans

# Bốn hàng điểm, mỗi hàng chứa hai tọa độ [x,y], shape (4,2).
X = np.array([[0, 0], [0, 1], [10, 10], [10, 11]])
# n_clusters=2 yêu cầu 2 tâm cụm; random_state=42 cố định ngẫu nhiên khởi tạo; n_init=10 thử 10
khởi tạo để giảm nguy cơ nghiệm kém.
# fit_predict() vừa học tâm cụm trên X vừa trả nhãn cụm 0/1 cho từng hàng, không phải nhãn loài
thật.
labels = KMeans(n_clusters=2, random_state=42, n_init=10).fit_predict(X)
# tolist() đổi nhãn cụm từ ndarray thành list, set() lấy các cụm duy nhất.
# len() đếm số mã cụm khác nhau mà thuật toán gán (2).
print(len(set(labels.tolist())))
```

Kết quả khi chạy:

```
2
```

KMeans tìm được hai cụm; mã số gắn cho từng cụm có thể đổi.

5. Năm tình huống chia dữ liệu, classification và clustering — từ 10 đến 20 hồ sơ

Tình huống 1 — Cấp 1: Giữ riêng 2 trong 10 học viên để đánh giá

Từ vựng trong mục: Train — tập huấn luyện; **Test** — tập đánh giá; **Disjoint** — không trùng.

Tình huống thực tế: Trung tâm đào tạo có **10 hồ sơ H01–H10**. Mỗi hồ sơ ghi mã học viên, số giờ luyện tập **(giờ/tuần)** và điểm kiểm tra **(thang 10)**. Nhóm muốn dùng **8 học viên đầu** để chuẩn bị mô hình và giữ riêng **2 người cuối** để đánh giá sau này. Trước khi học, cần kiểm tra không có mã nào vừa nằm trong train vừa nằm trong test và hai nhóm vẫn bao phủ đủ 10 người.

Vì sao chọn cách này? Train là dữ liệu được dùng khi học quy tắc; **test** được giữ lại để đo khả năng áp dụng cho người **chưa gặp lúc học**. `records[:8]` lấy 8 hồ sơ đầu còn `records[8:]` lấy hai hồ sơ cuối; đây là **chia theo thứ tự mã**, không phải lấy ngẫu nhiên. `set` các mã giúp kiểm `isdisjoint()` (hai nhóm không chung mã) và phép hợp giúp xác nhận đủ 10 người. Việc hai nhóm không trùng là điều kiện cần, nhưng nếu hồ sơ được sắp theo ngày hay nhãn thì cách cắt này vẫn có thể làm hai nhóm khác nhau thiếu đại diện.

Quy trình:

1. Tạo 10 bộ ID–giờ–điểm, mỗi hàng là một người.
2. Chia 8 dòng đầu train, 2 dòng cuối test.
3. Kiểm tra giao ID bằng Set và số người của cả hai nhóm.

No Note — Không chú thích

```
records = []
for i in range(1, 11):
    records.append({"id":f"H{i:02d}", "hours":i, "score":min(10, 4+i//2)})
train = records[:8]
test = records[8:]
train_ids = set()
for row in train:
    train_ids.add(row["id"])
test_ids = set()
for row in test:
    test_ids.add(row["id"])
if len(records) != 10:
    raise ValueError("Cần đủ 10 hồ sơ để chia Train/Test.")
print("Hồ sơ:", len(records))
print("Train/test:", len(train), len(test))
print("ID không giao nhau:", train_ids.isdisjoint(test_ids))
print("Phủ toàn bộ:", len(train_ids | test_ids) == 10)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo 10 hồ sơ, chia 8 Train/2 Test theo vị trí và kiểm không trùng hoặc thiếu ID.
# Dữ liệu: 10 học viên H01..H10 có giờ học và điểm, 8 đầu cho train và 2 cuối cho test, chỉ minh
họa cách phân nhóm.
# List rỗng lưu 10 dictionary hồ sơ, không phải tập nhãn.
records = []
# range(1,11) tạo chỉ số 1..10, không lấy 11.
for i in range(1, 11):
    # append() thêm dict học viên gồm id, hours và score theo một hàng.
    # f-string :02d định dạng mã H01..H10 có số 0 ở đầu.
    # // chia lấy phần nguyên để mô phỏng điểm tăng theo hai giờ; min(10,...) chặn điểm tối đa 10.
    records.append({"id":f"H{i:02d}", "hours":i, "score":min(10, 4+i//2)})
# Slice [:8] lấy vị trí 0..7 làm train; không thay dữ liệu records gốc.
train = records[:8]
# Slice [8:] lấy hai hàng cuối làm test; đây là phép chia theo thứ tự, KHÔNG phải phân ngẫu nhiên.
test = records[8:]
# Set rỗng tập hợp mã Train để kiểm giao.
train_ids = set()
for row in train:
    # add() chỉ ghi mã một lần, không nhân đôi một ID.
    train_ids.add(row["id"])
# Set rỗng mã Test để kiểm không chồng với Train.
test_ids = set()
for row in test:
    # Ghi ID của test vào set riêng.
    test_ids.add(row["id"])
# len() xác nhận đủ 10 hồ sơ.
if len(records) != 10:
    # Báo lỗi số hồ sơ không đúng, tránh công bố phân chia thiếu.
    raise ValueError("Cần đủ 10 hồ sơ để chia Train/Test.")
# len() xác nhận đủ 10 hồ sơ.
print("Hồ sơ:", len(records))
print("Train/test:", len(train), len(test))
# True nếu mọi ID Train khác ID Test, điều kiện cần chống dùng lại mẫu.
print("ID không giao nhau:", train_ids.isdisjoint(test_ids))
```

```
# | hợp hai set: tất cả mã thuộc Train hoặc Test; len()==10 xác minh đủ 10 ID.
print("Phủ toàn bộ:", len(train_ids | test_ids) == 10)
```

Kết quả khi chạy:

```
Hồ sơ: 10
Train/test: 8 2
ID không giao nhau: True
Phủ toàn bộ: True
```

ID rời nhau chỉ là điều kiện cần: nếu một người có hai ID khác nhau hoặc đặc trưng chứa thông tin tương lai, vẫn có thể xảy ra rò rỉ.

Tình huống 2 — Cấp 2: Chia phân tầng 12 hồ sơ cân bằng nhãn

Từ vựng trong mục: Stratify — giữ tỷ lệ nhãn; **Random state** — hạt giống ngẫu nhiên.

Tình huống thực tế: Đội tư vấn có **12 hồ sơ S01–S12**, mỗi người có điểm hoạt động và **nhãn 0** (chưa đăng ký) hoặc **1** (đã đăng ký). Có đúng **6 người ở mỗi nhãn**, nhưng nếu chia tùy tiện thì nhóm kiểm tra chỉ có một nhãn, khó đánh giá công bằng. Nhóm cần chia thành **8 hồ sơ train và 4 hồ sơ test**, giữ được cả nhãn 0 và 1 ở mỗi phần, đồng thời không dùng lại cùng khách.

Vì sao chọn cách này? Phân tầng (stratify) là chia mẫu sao cho **tỷ lệ hai nhãn** được duy trì trong những phần được tạo, khi số lượng cho phép. `train_test_split(..., test_size=4, stratify=y)` chia **12 khách** thành 8 người học và 4 người đánh giá; nhãn được dùng để **quyết định tỷ lệ chia**, không có nghĩa mô hình đã học từ nhãn test. `random_state=42` giữ kết quả chia có thể lặp lại. Chương trình đếm riêng số 0/1 của train và test và kiểm tập mã không giao nhau; mẫu 12 vẫn quá nhỏ để kết luận chắc chắn hiệu quả mô hình ngoài thực tế.

Quy trình:

1. Tạo 12 mã, đặc trưng điểm và nhãn nhị phân.
2. Chia bằng stratify với test_size=4, giữ ID tương ứng.
3. Đếm nhãn train/test, xác nhận ID không trùng và mỗi nhóm có hai nhãn.

No Note — Không chú thích

```
from sklearn.model_selection import train_test_split
records = [{"id":"S01","score":5,"label":0},{"id":"S02","score":6,"label":1},
{"id":"S03","score":7,"label":0},{"id":"S04","score":8,"label":1},{"id":"S05","score":9,"label":0},
{"id":"S06","score":10,"label":1},{"id":"S07","score":11,"label":0},
{"id":"S08","score":12,"label":1},{"id":"S09","score":13,"label":0},
{"id":"S10","score":14,"label":1},{"id":"S11","score":15,"label":0},
{"id":"S12","score":16,"label":1}]
ids = []
y = []
for row in records:
    ids.append(row["id"])
    y.append(row["label"])
train_ids, test_ids, y_train, y_test = train_test_split(
    ids, y, test_size=4, random_state=42, stratify=y)
if len(records) != 12:
    raise ValueError("Cần đủ 12 hồ sơ và nhãn.")
print("Hồ sơ:", len(records))
print("Train/test:", len(train_ids), len(test_ids))
print("Nhãn train 0/1:", y_train.count(0), y_train.count(1))
```

```
print("Nhãn test 0/1:", y_test.count(0), y_test.count(1))
print("ID không giao nhau:", set(train_ids).isdisjoint(test_ids))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dùng train_test_split phân tầng 12 hồ sơ để giữ nhãn 0/1 cân bằng khi chia Train/Test.
# Dữ liệu: 12 hồ sơ S01..S12 có nhãn 0/1 xen kẽ; Test chọn 4 mẫu, Train 8 mẫu.
# train_test_split() chia các list tương ứng theo CÙNG các chỉ số hàng.
from sklearn.model_selection import train_test_split
# 12 dict có id, score và label; nhãn y dùng để phân tầng, không tự huấn luyện mô hình ở đây.
records = [{"id":"S01","score":5,"label":0},{"id":"S02","score":6,"label":1},
{"id":"S03","score":7,"label":0},{"id":"S04","score":8,"label":1},{"id":"S05","score":9,"label":0},
{"id":"S06","score":10,"label":1},{"id":"S07","score":11,"label":0},
{"id":"S08","score":12,"label":1},{"id":"S09","score":13,"label":0},
{"id":"S10","score":14,"label":1},{"id":"S11","score":15,"label":0},
{"id":"S12","score":16,"label":1}]
# List mã học viên theo đúng vị trí của y.
ids = []
# List nhãn 0/1 phục vụ stratify.
y = []
for row in records:
    # append() giữ ID của hồ sơ theo thứ tự.
    ids.append(row["id"])
    # append() giữ nhãn tại cùng vị trí với ID.
    y.append(row["label"])
# Hàm trả bốn list theo THỨ TỰ ids_train, ids_test, y_train, y_test.
train_ids, test_ids, y_train, y_test = train_test_split(
    # test_size=4 yêu cầu đúng bốn hàng Test, không phải tỷ lệ 4%.
    # Cố định seed chia để chạy lại cùng phân hoạch.
    # stratify=y giữ gần tỷ lệ lớp 0/1 ở Train và Test, giảm nguy cơ thiếu lớp.
    ids, y, test_size=4, random_state=42, stratify=y)
# len() kiểm tổng 12 hồ sơ nguồn.
if len(records) != 12:
    # Báo đầu vào thiếu hồ sơ/nhãn.
    raise ValueError("Cần đủ 12 hồ sơ và nhãn.")
# len() kiểm tổng 12 hồ sơ nguồn.
print("Hồ sơ:", len(records))
print("Train/test:", len(train_ids), len(test_ids))
# count(0) đếm nhãn lớp 0 trong list Train, khác số mẫu toàn bộ.
print("Nhãn train 0/1:", y_train.count(0), y_train.count(1))
# Đếm lớp 0 trong Test; count(1) đếm lớp 1.
print("Nhãn test 0/1:", y_test.count(0), y_test.count(1))
# set() chuyển list thành tập, isdisjoint() kiểm ID Test không trùng Train.
print("ID không giao nhau:", set(train_ids).isdisjoint(test_ids))
```

Kết quả khi chạy:

```
Hồ sơ: 12
Train/test: 8 4
Nhãn train 0/1: 4 4
Nhãn test 0/1: 2 2
ID không giao nhau: True
```

Phân tầng dùng nhãn **chỉ để quyết định cách chia**, không có nghĩa mô hình được nhìn nhãn test khi fit.

Tình huống 3 — Cấp 3: Huấn luyện classifier trong Pipeline với 15 bông hoa

Từ vựng trong mục: Classification — phân loại có nhãn; **Pipeline** — chuỗi xử lý; **Leakage** — rò rỉ thông tin.

Tình huống thực tế: Nhóm học máy chuẩn bị **15 hồ sơ hoa F01–F15**, mỗi bông được ghi **chiều dài, chiều rộng cánh hoa (cm)** và nhãn thuộc **một trong ba loài 0, 1, 2**. Mỗi loài có 5 bông. Họ muốn học cách phân loại từ **9 mẫu train**, giữ **6 mẫu test** để thử dự đoán, và đặc biệt tránh dùng số đo của hoa test khi tính mức trung bình/độ phân tán cho bước chuẩn hóa.

Vì sao chọn cách này? StandardScaler chuẩn hóa số đo bằng các thông số như **trung bình** và **độ lệch chuẩn** học từ dữ liệu. Nếu tính các thông số ấy trên **cả 15 bông** trước khi chia, thông tin từ 6 hoa test đã đi vào quá trình chuẩn bị mô hình: đó là **rò rỉ dữ liệu (data leakage)**. `Pipeline` gói scaler và `LogisticRegression`: lệnh `fit` chỉ chạy trên 9 hoa train rồi áp **cùng phép chuẩn hóa đã học** lên 6 hoa test khi dự đoán. `stratify=y` giữ các loài có đại diện khi chia; `random_state=42` giúp chạy lại cùng một cách chọn. Điểm accuracy đo trên test là số dự đoán đúng trong **6 mẫu**, không đảm bảo tỷ lệ đúng của thế giới thật.

Quy trình:

1. Tạo 15 dòng có đủ ID, hai số đo cm, ba nhãn.
2. Tách train/test phân tầng 9/6 và ghép `StandardScaler` + `LogisticRegression`.
3. Fit chỉ trên train, kiểm scaler mean bằng train mean và in số dự đoán/độ chính xác test.

No Note — Không chú thích

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
records = [{"id":"F01","length_cm":1.5,"width_cm":0.5,"species":0},
{"id":"F02","length_cm":1.6,"width_cm":0.53,"species":0},
{"id":"F03","length_cm":1.7,"width_cm":0.56,"species":0},
{"id":"F04","length_cm":1.8,"width_cm":0.59,"species":0},
{"id":"F05","length_cm":1.9,"width_cm":0.62,"species":0},
{"id":"F06","length_cm":4.5,"width_cm":1.5,"species":1},
{"id":"F07","length_cm":4.6,"width_cm":1.53,"species":1},
{"id":"F08","length_cm":4.7,"width_cm":1.56,"species":1},
{"id":"F09","length_cm":4.8,"width_cm":1.59,"species":1},
{"id":"F10","length_cm":4.9,"width_cm":1.62,"species":1},
{"id":"F11","length_cm":7.5,"width_cm":2.5,"species":2},
{"id":"F12","length_cm":7.6,"width_cm":2.53,"species":2},
{"id":"F13","length_cm":7.7,"width_cm":2.56,"species":2},
{"id":"F14","length_cm":7.8,"width_cm":2.59,"species":2},
{"id":"F15","length_cm":7.9,"width_cm":2.62,"species":2}]
features = []
labels = []
for row in records:
    features.append([row["length_cm"], row["width_cm"]])
    labels.append(row["species"])
X = np.array(features)
y = np.array(labels)
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=6,
                                                 stratify=y,random_state=42)
model = Pipeline([("scale", StandardScaler()),
                  ("classifier", LogisticRegression(max_iter=1000))])
model.fit(X_train,y_train)
predicted = model.predict(X_test)
if len(records) != 15:
    raise ValueError("Cần đủ 15 mẫũ hoa có nhãn.")
print("Hoa:", len(records))
print("Train/test:", len(X_train), len(X_test))
print("Scaler chỉ fit train:", bool(np.allclose(model.named_steps["scale"].mean_,
X_train.mean(axis=0))))
```

```
print("Đã dự đoán:", len(predicted))
print("Accuracy hợp lệ:", bool(0 <= np.mean(predicted==y_test) <= 1))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chia 15 hoa theo ba loài và huấn luyện Pipeline scaler→LogisticRegression, tránh fit
scaler trên Test.
# Dữ liệu: 15 hoa F01..F15, mỗi hoa có chiều dài/rộng cm và nhãn species=0/1/2; Test 6 mẫu.
# NumPy chuyển đặc trưng/nhãn sang ndarray để train sklearn.
import numpy as np
# Chia X,y cùng chỉ số hàng trước huấn luyện.
from sklearn.model_selection import train_test_split
# Pipeline ghép tiền xử lý và mô hình theo thứ tự để fit chỉ trên Train.
from sklearn.pipeline import Pipeline
# StandardScaler học mean và độ lệch chuẩn từng cột Train, rồi chuẩn hóa đặc trưng.
from sklearn.preprocessing import StandardScaler
# LogisticRegression là mô hình phân loại, hỗ trợ nhiều lớp (không phải hồi quy giá trị liên tục).
from sklearn.linear_model import LogisticRegression
# 15 mẫu hoa, các cột length_cm/width_cm là INPUT, species là NHÃN cần đoán.
records = [{"id":"F01","length_cm":1.5,"width_cm":0.5,"species":0},
{"id":"F02","length_cm":1.6,"width_cm":0.53,"species":0},
{"id":"F03","length_cm":1.7,"width_cm":0.56,"species":0},
{"id":"F04","length_cm":1.8,"width_cm":0.59,"species":0},
{"id":"F05","length_cm":1.9,"width_cm":0.62,"species":0},
{"id":"F06","length_cm":4.5,"width_cm":1.5,"species":1},
{"id":"F07","length_cm":4.6,"width_cm":1.53,"species":1},
{"id":"F08","length_cm":4.7,"width_cm":1.56,"species":1},
{"id":"F09","length_cm":4.8,"width_cm":1.59,"species":1},
{"id":"F10","length_cm":4.9,"width_cm":1.62,"species":1},
{"id":"F11","length_cm":7.5,"width_cm":2.5,"species":2},
{"id":"F12","length_cm":7.6,"width_cm":2.53,"species":2},
{"id":"F13","length_cm":7.7,"width_cm":2.56,"species":2},
{"id":"F14","length_cm":7.8,"width_cm":2.59,"species":2},
{"id":"F15","length_cm":7.9,"width_cm":2.62,"species":2}]
# List lưu hai số đo (cm) của từng hoa.
features = []
# List nhãn loài y cùng thứ tự với features.
labels = []
for row in records:
    # append() ghép [length,width] thành một hàng X.
    features.append([row["length_cm"], row["width_cm"]])
    # append() giữ nhãn tương ứng của cùng hàng.
    labels.append(row["species"])
# Tạo X shape (15,2) gồm 15 hoa × 2 đặc trưng.
X = np.array(features)
# Tạo y shape (15,), một loài mỗi hoa.
y = np.array(labels)
# test_size=6 giữ 6 hoa chưa thấy để đánh giá; hai mảng X/y chia đồng bộ.
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=6,
                                                 # stratify=y giữ tỷ lệ ba loài, random_state=42
cố định kết quả chia.
                                                 stratify=y,random_state=42)
# Pipeline định nghĩa hai bước theo thứ tự scale trước classifier.
# Scaler mặc định chuẩn hóa z-score theo mean/std học từ Train, KHÔNG tự fit trên toàn X.
model = Pipeline([("scale", StandardScaler()),
                  # max_iter=1000 là giới hạn vòng tối ưu để dễ hội tụ trên dữ liệu nhỏ, không phải
số epoch mẫu hoặc số lớp.
                  ("classifier", LogisticRegression(max_iter=1000))])
# fit() học tham số scaler và bộ phân loại CHỈ từ X_train,y_train; test chưa tham gia fit.
model.fit(X_train,y_train)
# predict() tự transform X_test bằng scaler đã học, rồi dự đoán nhãn cho sáu hoa.
predicted = model.predict(X_test)
if len(records) != 15:
    # Ngừng nếu không đủ 15 hoa.
    raise ValueError("Cần đủ 15 mẫu hoa có nhãn.")
```

```
print("Hoa:", len(records))
print("Train/test:", len(X_train), len(X_test))
# named_steps['scale'] truy bước scaler đã fit; mean_ là trung bình TRAIN từng cột, khác mean của
toàn X.
# axis=0 lấy trung bình qua các hàng mẫu cho mỗi cột đặc trưng.
# allclose() đối chiếu hai mảng mean với dung sai số thực, bool() chuyển thành Boolean để in.
print("Scaler chỉ fit train:", bool(np.allclose(model.named_steps["scale"].mean_,
X_train.mean(axis=0))))
# Đếm nhãn dự đoán của Test (6), không phải tổng hoa.
print("Đã dự đoán:", len(predicted))
# So nhãn dự đoán với nhãn thật từng hàng; mean() Boolean tính accuracy trong [0,1] trên Test.
print("Accuracy hợp lệ:", bool(0 <= np.mean(predicted==y_test) <= 1))
```

Kết quả khi chạy:

```
Hoa: 15
Train/test: 9 6
Scaler chỉ fit train: True
Đã dự đoán: 6
Accuracy hợp lệ: True
```

Accuracy chỉ được tính từ **sáu hoa test**, không phải bằng chứng kết quả sẽ giữ nguyên ngoài dữ liệu mô phỏng.

Tình huống 4 — Cấp 4: KMeans tìm hai cụm trong 18 tọa độ không có nhãn

Từ vựng trong mục: Clustering — gom cụm không giám sát; **Centroid** — tâm cụm.

Tình huống thực tế: Kho robot ghi **18 vị trí vật thể trên sàn**, mỗi vị trí là hai tọa độ **x, y (mét)**. Có **9 vật thể gần (1,1)** và **9 vật thể gần (11,11)**, nhưng bảng không có nhãn 'vùng gần' hay 'vùng xa'. Người vận hành muốn chương trình tự tìm **hai cụm vị trí**, báo số vật thể từng cụm và vị trí tâm theo trục x, không tự gọi mã cụm 0 là bên trái.

Vì sao chọn cách này? KMeans là thuật toán **phân cụm không giám sát**: nó nhận tọa độ, tìm hai tâm và đưa mỗi điểm vào cụm gần tâm thích hợp, **không cần nhãn đúng**. Ở đây hai vùng trên sàn cách nhau khá xa nên yêu cầu **hai cụm** là giả định dễ quan sát; `n_clusters=2` không phải số cụm mà thuật toán tự xác nhận cho mọi dữ liệu. `random_state=42` và `n_init=10` giúp việc khởi tạo và lặp thử có thể tái lập tốt hơn. Số nhãn cụm `0` và `1` chỉ là **tên do thuật toán gán**, có thể đổi chỗ; vì vậy dùng `sorted` để so kích thước và sắp **tâm theo tọa độ x** thay vì gán cố định 0 là vùng trái.

Quy trình:

1. Tạo 18 điểm có đơn vị m, không tạo nhãn để fit.
2. Dùng KMeans với 2 nhóm và random_state cố định.
3. Sắp kích thước hai nhóm, sắp tâm theo tọa độ x để báo cáo ổn định.

No Note — Không chú thích

```
import numpy as np
from sklearn.cluster import KMeans
points = np.array([[0,0],[1,0],[2,0],[0,1],[1,1],[2,1],[0,2],[1,2],[2,2],[10,10],[11,10],[12,10],
[10,11],[11,11],[12,11],[10,12],[11,12],[12,12]],dtype=float)
model = KMeans(n_clusters=2, random_state=42, n_init=10).fit(points)
counts = sorted(np.bincount(model.labels_).tolist())
centers = model.cluster_centers_[np.argsort(model.cluster_centers_[:,0])]
if points.shape != (18,2):
    raise ValueError("Cần đúng 18 điểm để phân cụm.")
```

```
print("Vật thể:", len(points))
print("Số cụm:", len(counts))
print("Vật thể mỗi cụm:", counts)
center_positions = []
for value in centers[:, 0]:
    center_positions.append(round(float(value), 1))
print("Tâm theo trục x:", center_positions)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân cụm 18 điểm 2D với KMeans, đếm kích cỡ cụm và sắp tâm theo tọa độ x.
# Dữ liệu: 18 điểm gồm hai đám: gần tọa độ (1,1) và (11,11), không có nhãn thật.
# NumPy tạo mảng và xử lý nhãn/tâm KMeans.
import numpy as np
# KMeans học các cụm từ tọa độ mà KHÔNG cần nhãn đã biết.
from sklearn.cluster import KMeans
# 18 hàng điểm, 2 cột x,y; dtype=float cung cấp dữ liệu số cho thuật toán.
# Ép số thực cho X, không thêm một đặc trưng mới.
points = np.array([[0,0],[1,0],[2,0],[0,1],[1,1],[2,1],[0,2],[1,2],[2,2],[10,10],[11,10],[12,10],
[10,11],[11,11],[12,11],[10,12],[11,12],[12,12]],dtype=float)
# Yêu cầu 2 cụm, seed 42 cho khởi tạo tái lập, n_init=10 chạy 10 lần khởi tạo và chọn nghiệm tốt
theo inertia.
# fit() học tâm cụm từ toàn bộ points; không có Train/Test vì là demo phân cụm mô tả, không đánh
giá tổng quát hóa.
model = KMeans(n_clusters=2, random_state=42, n_init=10).fit(points)
# Thuộc tính labels_ chứa mã cụm cho từng điểm sau khi fit; mã 0/1 có thể hoán đổi giữa các lần
chạy.
# bincount() đếm số điểm gán vào mỗi mã cụm nguyên không âm.
# sorted() xếp kích cỡ cụm, không dựa vào số mã 0/1.
counts = sorted(np.bincount(model.labels_).tolist())
# cluster_centers_ là ma trận 2×2 tâm đã học, mỗi hàng là một tâm.
# [:,0] lấy tọa độ x của mọi tâm.
# argsort() trả chỉ số tâm theo x tăng dần, cho thứ tự báo cáo nhất quán.
centers = model.cluster_centers_[np.argsort(model.cluster_centers_[:,0])]
# shape (18,2) bảo đảm 18 vật thể có hai tọa độ.
if points.shape != (18,2):
    # Dừng nếu bảng không có đúng 18 điểm.
    raise ValueError("Cần đúng 18 điểm để phân cụm.")
print("Vật thể:", len(points))
print("Số cụm:", len(counts))
print("Vật thể mỗi cụm:", counts)
center_positions = []
# Duyệt tọa độ x của hai tâm đã sắp.
for value in centers[:, 0]:
    # float() đổi NumPy scalar, round(...,1) làm tròn tâm khi in.
    # append() ghi tâm x vào list báo cáo.
    center_positions.append(round(float(value), 1))
print("Tâm theo trục x:", center_positions)
```

Kết quả khi chạy:

```
Vật thể: 18
Số cụm: 2
Vật thể mỗi cụm: [9, 9]
Tâm theo trục x: [1.0, 11.0]
```

Người học không được gọi cụm số 0 là loài số 0 chỉ vì hai hệ thống cùng dùng số; nhãn cụm chỉ là định danh.

Tình huống 5 — Cấp 5: So dự đoán có giám sát và gom cụm trên 20 khách hàng

Từ vựng trong mục: Classification vs. clustering — phân loại so với gom cụm; **Adjusted Rand Index** — độ phù hợp cụm không lệ thuộc mã.

Tình huống thực tế: Đội phân tích có **20 hồ sơ khách H01–H20**, mỗi hồ sơ gồm hai chỉ số hành vi x/y **không đơn vị** và nhãn tham chiếu **0 hoặc 1** (mỗi nhãn 10 khách). Họ muốn so hai công việc: **phân loại** học nhãn từ **16 hồ sơ train** rồi kiểm trên **4 test**, còn **phân cụm** tìm hai nhóm chỉ từ các chỉ số x/y. Cần báo kết quả của mỗi cách đúng ý nghĩa, không nhầm tên cụm do máy tự đặt với nhãn tham chiếu.

Vì sao chọn cách này? Classification giải bài toán có **nhãn đúng đã biết**; mô hình `StandardScaler` + `LogisticRegression` học từ **16 hồ sơ** rồi tính **accuracy trên 4 hồ sơ test** chưa dùng học. **Clustering** dùng `KMeans` chỉ trên đặc trưng x/y của **20 hồ sơ**, không đưa nhãn tham chiếu vào `fit_predict`. Chỉ **sau khi tìm cụm**, `adjusted_rand_score` (ARI) dùng nhãn thật để **đối chiếu độ tương hợp hai cách chia**, đồng thời không bị ảnh hưởng bởi việc gọi cụm là 0 hay 1. Đây là hai nhiệm vụ và hai cách đánh giá khác nhau: accuracy không thay thế ARI, còn ARI cao không có nghĩa KMeans đã được dạy nhãn.

Quy trình:

1. Tạo 20 hàng có ID, x/y, nhãn; tách train/test phân tầng.
2. Fit Pipeline classifier trên 16 train và đánh giá 4 test.
3. Fit KMeans trên toàn bộ 20 đặc trưng **không có nhãn**, tính ARI chỉ để đối chiếu kết quả.

No Note — Không chú thích

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import adjusted_rand_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
records = [{"id":"H01","x":0,"y":1,"label":0},{"id":"H02","x":0.2,"y":1,"label":0},
{"id":"H03","x":0.4,"y":1,"label":0},{"id":"H04","x":0.6000000000000001,"y":1,"label":0},
{"id":"H05","x":0.8,"y":1,"label":0},{"id":"H06","x":0,"y":1.2,"label":0},
{"id":"H07","x":0.2,"y":1.2,"label":0},{"id":"H08","x":0.4,"y":1.2,"label":0},
{"id":"H09","x":0.6000000000000001,"y":1.2,"label":0},{"id":"H10","x":0.8,"y":1.2,"label":0},
{"id":"H11","x":8,"y":9,"label":1},{"id":"H12","x":8.2,"y":9,"label":1},
{"id":"H13","x":8.4,"y":9,"label":1},{"id":"H14","x":8.6,"y":9,"label":1},
{"id":"H15","x":8.8,"y":9,"label":1},{"id":"H16","x":8,"y":9.2,"label":1},
{"id":"H17","x":8.2,"y":9.2,"label":1},{"id":"H18","x":8.4,"y":9.2,"label":1},
{"id":"H19","x":8.6,"y":9.2,"label":1},{"id":"H20","x":8.8,"y":9.2,"label":1}]
features = []
labels = []
for row in records:
    features.append([row["x"], row["y"]])
    labels.append(row["label"])
X = np.array(features, dtype=float)
y = np.array(labels)
ids = np.arange(len(records))
train, test = train_test_split(ids, test_size=4, stratify=y, random_state=42)
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
clf.fit(X[train], y[train])
accuracy = float(clf.score(X[test], y[test]))
clusters = KMeans(n_clusters=2,random_state=42,n_init=10).fit_predict(X)
ari = adjusted_rand_score(y,clusters)
if len(records) != 20 or not set(train).isdisjoint(test):
    raise ValueError("Cần 20 hồ sơ và Train/Test không được trùng mẫu.")
print("Khách:", len(records))
print("Train/test:", len(train), len(test))
```

```
print("Accuracy test hợp lệ:", 0 <= accuracy <= 1)
print("Cụm:", len(set(clusters.tolist())))
print("ARI khi đối chiếu:", f"{ari:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So phân loại có giám sát và KMeans không giám sát trên 20 hồ sơ, tính Accuracy và ARI.
# Dữ liệu: 20 hồ sơ H01..H20 có tọa độ x,y và nhãn thật 0/1; Test có bốn hồ sơ, phân cụm dùng toàn
X để đối chiếu.
# NumPy tạo ma trận đặc trưng và vector chỉ số hàng.
import numpy as np
# KMeans phân cụm không dùng nhãn y.
from sklearn.cluster import KMeans
# LogisticRegression học phân loại có giám sát bằng X_train,y_train.
from sklearn.linear_model import LogisticRegression
# adjusted_rand_score(y_true,y_pred) so phân hoạch cụm với nhãn thật và điều chỉnh trùng khớp ngẫu
nhiên; ARI không phải Accuracy.
from sklearn.metrics import adjusted_rand_score
# Chia chỉ số hàng cùng với y để kiểm không chồng lấn mẫu.
from sklearn.model_selection import train_test_split
# make_pipeline() ghép scaler và classifier với tên bước tự sinh.
from sklearn.pipeline import make_pipeline
# StandardScaler fit mean/std trên Train trước LogisticRegression.
from sklearn.preprocessing import StandardScaler
# 20 mẫu gồm id, tọa độ x/y và label thật 0/1.
records = [{"id":"H01","x":0,"y":1,"label":0},{"id":"H02","x":0.2,"y":1,"label":0},
{"id":"H03","x":0.4,"y":1,"label":0},{"id":"H04","x":0.6000000000000001,"y":1,"label":0},
{"id":"H05","x":0.8,"y":1,"label":0},{"id":"H06","x":0,"y":1.2,"label":0},
{"id":"H07","x":0.2,"y":1.2,"label":0},{"id":"H08","x":0.4,"y":1.2,"label":0},
{"id":"H09","x":0.6000000000000001,"y":1.2,"label":0},{"id":"H10","x":0.8,"y":1.2,"label":0},
{"id":"H11","x":8,"y":9,"label":1},{"id":"H12","x":8.2,"y":9,"label":1},
{"id":"H13","x":8.4,"y":9,"label":1},{"id":"H14","x":8.6,"y":9,"label":1},
{"id":"H15","x":8.8,"y":9,"label":1},{"id":"H16","x":8,"y":9.2,"label":1},
{"id":"H17","x":8.2,"y":9.2,"label":1},{"id":"H18","x":8.4,"y":9.2,"label":1},
{"id":"H19","x":8.6,"y":9.2,"label":1},{"id":"H20","x":8.8,"y":9.2,"label":1}]
# List tạo X gồm hai tọa độ mỗi hàng.
features = []
# List tạo y nhãn thật cùng chỉ số hàng.
labels = []
for row in records:
    # append() thêm cặp [x,y] của một hồ sơ.
    features.append([row["x"], row["y"]])
    # append() thêm nhãn thật của cùng hồ sơ.
    labels.append(row["label"])
# X ndarray (20,2), dtype=float ép tọa độ sang số thực.
X = np.array(features, dtype=float)
# y ndarray (20,), nhãn phân loại 0/1.
y = np.array(labels)
# arange(20) tạo chỉ số 0..19 để chia mà không làm mất liên hệ với X,y.
ids = np.arange(len(records))
# Chia 4 chỉ số test, 16 train; stratify theo y giữ lớp cân bằng, seed 42 tái lập.
train, test = train_test_split(ids, test_size=4, stratify=y, random_state=42)
# Tạo pipeline tự đặt tên bước; scaler chỉ fit trên phần Train, max_iter=1000 giới hạn bước tối ưu.
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
# Boolean/fancy indexing X[train] và y[train] chọn cùng 16 hàng Train, fit scaler và classifier
không dùng Test.
clf.fit(X[train], y[train])
# score() của LogisticRegression trả Accuracy trên 4 hàng Test, không phải loss hay mức tin cậy.
# float() đổi kết quả score thành float Python.
accuracy = float(clf.score(X[test], y[test]))
# KMeans học hai tâm cụm, seed 42, thử 10 khởi tạo, không dùng nhãn y khi fit.
# fit_predict() vừa học cụm từ toàn X vừa trả 20 mã cụm, không huấn luyện classifier theo nhãn.
clusters = KMeans(n_clusters=2,random_state=42,n_init=10).fit_predict(X)
# ARI dùng nhãn thật CHỈ để đánh giá độ khớp phân cụm; gần 1 là khớp rất tốt, 0 gần ngẫu nhiên, có
thể âm.
```

```
ari = adjusted_rand_score(y,clusters)
# Kiểm không một chỉ số´hàng nào đồng thời có trong Train và Test.
if len(records) != 20 or not set(train).isdisjoint(test):
    # Chặn khi bảng thiếu mẫu hoặc Train/Test trùng nhau.
    raise ValueError("Cần 20 hồ`sơ và Train/Test không được trùng mẫu.")
print("Khách:", len(records))
print("Train/test:", len(train), len(test))
print("Accuracy test hợp lệ:", 0 <= accuracy <= 1)
# tolist() đổi nhãn sang list, set() lấy cụm duy nhất, len() đếm.
print("Cụm:", len(set(clusters.tolist())))
# f-string :.2f in ARI với hai số´thập phân.
print("ARI khi đối chiếu:", f"{ari:.2f}")
```

Kết quả khi chạy:

```
Khách: 20
Train/test: 16 4
Accuracy test hợp lệ: True
Cụm: 2
ARI khi đối chiếu: 1.00
```

Độ phù hợp cao ở ví dụ vì **hai cụm giả lập được tách xa nhau**. Kết quả gom cụm không tự chứng minh nhãn tham chiếu là bản chất của khách hàng.

6. Sai lầm thường gặp

- Dữ liệu test không được dùng để fit scaler hoặc chọn siêu tham số; cần tách trước khi tiền xử lý học tham số.
- ID không trùng vẫn có thể rò rỉ qua bản ghi liên quan hoặc giá trị tương lai.
- Classification học từ nhãn y; clustering nhóm X không có nhãn. Không gọi mã cụm là nhãn lớp thật.
- `stratify=y` giúp giữ tỷ lệ lớp nhưng không chứng minh tập test đại diện toàn bộ dân số.
- Accuracy test khác loss train và không phản ánh đầy đủ hiệu quả trên dữ liệu mất cân bằng.
- `scikit-learn` là thư viện ngoài; phiên bản khác nhau có thể thay đổi kết quả học tối ưu.

7. Trắc nghiệm

Q1. What is a train set? | Train set là gì?

A. Data used for fitting model parameters | Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation | Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels | Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity | Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: A

Giải thích: Mô hình học tham số từ mẫu train, không cập nhật bằng bản ghi test. Đơn vị học là hàng train với đặc trưng và nhãn, không phải hàng test để chấm điểm.

Q2. What is a test set? | Test set là gì?

A. Data used for fitting model parameters | Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation | Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels | Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity | Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: B

Giải thích: Test đo hiệu quả trên dữ liệu chưa học; chỉnh đi chỉnh lại theo test làm đánh giá thiên lệch. Điểm cốt lõi là test độc lập với quá trình học; điều chỉnh lặp dựa trên test sẽ làm mất tính độc lập ấy.

Q3. What is classification? | Classification là gì?

A. Data used for fitting model parameters | Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation | Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels | Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity | Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: C

Giải thích: Lúc train phải có nhãn; đầu ra dự đoán là loại/phân lớp. Khác phân cụm, bài phân loại có nhãn mục tiêu rõ ràng khi học.

Q4. What is clustering? | Clustering là gì?

A. Data used for fitting model parameters | Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation | Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels | Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity | Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: D

Giải thích: Phân cụm dựa vào X mà không fit nhãn lớp có sẵn. Khác classifier, KMeans không tự hiểu ý nghĩa đời thực của số hiệu cụm.

Q5. What does stratify=y help preserve? | stratify=y giúp giữ gì?

A. Approximate label proportions in train and test | Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only | Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction | Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers | Định danh cụm tùy ý

Đáp án: A

Giải thích: Nó chia các lớp theo tỷ lệ gần tương ứng khi số mẫu cho phép. Nó chỉ giữ tỷ lệ lớp xấp xỉ, không bảo đảm mọi nhóm nhỏ đều đại diện.

Q6. What does StandardScaler fit on? | StandardScaler phải fit trên dữ liệu nào?

A. Approximate label proportions in train and test | Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only | Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction | Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers | Định danh cụm tùy ý

Đáp án: B

Giải thích: Fit từ test làm rò rỉ phân phối đánh giá vào tiền xử lý. Được áp thống kê đã học từ train lên test, nhưng không được học lại thống kê từ test.

Q7. What does Pipeline combine? | Pipeline gộp gì?

A. Approximate label proportions in train and test | Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only | Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction | Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers | Định danh cụm tùy ý

Đáp án: C

Giải thích: Các phép đổi đã học được áp nhất quán khi dự đoán. Vì thế tiền xử lý phải nằm trong quy trình train, không fit trước trên toàn bộ dữ liệu.

Q8. What do KMeans cluster IDs mean? | Mã cụm KMeans có nghĩa gì?

A. Approximate label proportions in train and test | Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only | Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction | Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers | Định danh cụm tùy ý

Đáp án: D

Giải thích: Mã có thể đổi 0↔1 dù cách chia hình học không đổi. Hãy đánh giá theo thành viên nhóm hoặc ánh xạ có căn cứ, không mặc định mã 0 là một loài cụ thể.

Q9. What is a feature? | Feature là gì?

A. Input measurement used to predict | Số đo đầu vào dùng để dự đoán

B. The desired discrete output class | Lớp đầu ra muốn dự đoán

C. One predicted label per input row | Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels | Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: A

Giải thích: Đặc trưng nằm ở cột X, không phải nhãn đích. Mỗi cột cần có ý nghĩa và đơn vị đo rõ ràng trước khi đưa vào classifier.

Q10. What is a target label? | Target label là gì?

A. Input measurement used to predict | Số đo đầu vào dùng để dự đoán

B. The desired discrete output class | Lớp đầu ra muốn dự đoán

C. One predicted label per input row | Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels | Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: B

Giải thích: Classifier học ánh xạ từ X sang y. Nhãn là mục tiêu dự đoán, không phải đặc trưng được đưa sẵn khi dự đoán.

Q11. What does `model.predict(X_test)` return? | `model.predict(X_test)` trả gì?

A. Input measurement used to predict | Số đo đầu vào dùng để dự đoán

B. The desired discrete output class | Lớp đầu ra muốn dự đoán

C. One predicted label per input row | Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels | Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: C

Giải thích: Mỗi hàng test có một dự đoán lớp. Số nhãn dự đoán phải bằng số hàng đầu vào, nếu không sẽ thiếu kết quả cho một số mẫu.

Q12. What does `model.score(X_test,y_test)` mean for a standard classifier? | `model.score` cho classifier thông dụng đo gì?

A. Input measurement used to predict | Số đo đầu vào dùng để dự đoán

B. The desired discrete output class | Lớp đầu ra muốn dự đoán

C. One predicted label per input row | Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels | Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: D

Giải thích: Score so dự đoán với nhãn thật; không phải loss train. Nếu nhãn mất cân bằng, cần đọc thêm precision/recall theo lớp hoặc confusion matrix.

Q13. Which check detects overlapping IDs across train and test? | Phép kiểm tra nào phát hiện ID xuất hiện ở cả train và test?

A. `train_ids.isdisjoint(test_ids)` | `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak | Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X | Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations | Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: A

Giải thích: isdisjoint xác nhận không có cùng mã ở cả hai tập. Kiểm tra này bắt trùng mã đơn giản, không phát hiện phép đo liên quan mang mã khác nhau.

Q14. Does disjoint ID imply zero leakage? | Không trùng ID có chắc không rò rỉ?

A. `train_ids.isdisjoint(test_ids)` | `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak | Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X | Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations | Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: B

Giải thích: Rò rỉ có thể qua dữ liệu liên quan, phép đổi hoặc thông tin thời gian. Rò rỉ là thông tin đáng lẽ chưa biết lọt vào train, không chỉ là vấn đề trùng mã.

Q15. Does KMeans need y labels to fit? | KMeans cần nhãn y khi fit không?

A. `train_ids.isdisjoint(test_ids)` | `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak | Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X | Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations | Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: C

Giải thích: Phân cụm không giám sát và không tối ưu accuracy theo nhãn thật. KMeans chỉ cần tọa độ đặc trưng và số cụm muốn tìm.

Q16. Why not directly compare KMeans IDs to true classes for accuracy? | Vì sao không so trực tiếp mã KMeans với lớp thật để tính accuracy?

A. `train_ids.isdisjoint(test_ids)` | `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak | Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X | Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations | Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: D

Giải thích: Cùng nhóm có thể gắn mã đảo ngược mà chất lượng không đổi. Nếu đổi nhãn số của hai cụm, accuracy kiểu phân loại có thể đổi dù cách chia nhóm vẫn y hệt.

Q17. What shape does a 150-row four-feature table have? | Bảng 150 hàng, bốn đặc trưng có shape nào?

A. (150,4) | (150,4)

B. Allocate about a quarter to test | Dành khoảng một phần tư cho test

C. Held-out final test set | Tập test cuối cùng đã giữ lại

D. Predicted known class names | Tên lớp đã biết từ dữ liệu nhãn

Đáp án: A

Giải thích: Hàng là hoa, cột là bốn đặc trưng. Chiều thứ nhất đếm số hoa, chiều thứ hai đếm số đo trên mỗi hoa.

Q18. What does test_size=0.25 mean? | test_size=0.25 nghĩa là gì?

A. (150,4) | (150,4)

B. Allocate about a quarter to test | Dành khoảng một phần tư cho test

C. Held-out final test set | Tập test cuối cùng đã giữ lại

D. Predicted known class names | Tên lớp đã biết từ dữ liệu nhãn

Đáp án: B

Giải thích: Làm tròn quyết định số nguyên mẫu chính xác. Với số mẫu hữu hạn, thư viện phải làm tròn tỷ lệ yêu cầu thành số lượng nguyên.

Q19. Which dataset must be excluded from hyperparameter tuning? | Không được dùng tập nào để điều chỉnh siêu tham số?

A. (150,4) | (150,4)

B. Allocate about a quarter to test | Dành khoảng một phần tư cho test

C. Held-out final test set | Tập test cuối cùng đã giữ lại

D. Predicted known class names | Tên lớp đã biết từ dữ liệu nhãn

Đáp án: C

Giải thích: Chọn siêu tham số bằng validation/CV thuộc phần train. Hãy chọn siêu tham số bằng validation hoặc cross-validation trên phần train, sau đó đánh giá test cuối.

Q20. What can a classification model output that KMeans does not inherently provide? | Classifier có đầu ra gì mà KMeans không tự bảo đảm?

A. (150,4) | (150,4)

B. Allocate about a quarter to test | Dành khoảng một phần tư cho test

C. Held-out final test set | Tập test cuối cùng đã giữ lại

D. Predicted known class names | Tên lớp đã biết từ dữ liệu nhãn

Đáp án: D

Giải thích: KMeans tạo cụm chứ không tự gán ý nghĩa lớp thật. Mã lớp là nhãn đã xác định; mã cụm chỉ đánh dấu một nhóm vừa phát hiện.

8. Liên hệ bài kế tiếp

`Set/NumPy → Train/Test → Preprocessing → Classification/Clustering.`

PY-42 — Tiền xử lý văn bản, token và Jaccard trong NLP

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Token — đơn vị từ; **Vocabulary** — từ vựng.

`String → Set → Tokenization → Vector hóa văn bản`

Học xong dùng để làm gì? Làm sạch văn bản, tách token và tạo tập từ để so sánh hoặc phân loại nội dung.

Kết nối kiến thức tiếp theo: PY-43 quản lý các phiên bản mã/tiền xử lý bằng Git để thí nghiệm tái tạo được.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Token	đơn vị từ	Một phần văn bản sau khi tách theo quy tắc
Vocabulary	từ vựng	Tập token khác nhau trong dữ liệu
Normalization	chuẩn hóa văn bản	Đưa chữ về dạng phù hợp để so khớp
Jaccard	tương đồng Jaccard	Tỷ lệ giao trên hợp của hai tập token

3. Hàm và công thức quan trọng

Cú pháp	Cách dùng
`text.strip().lower()`	Bỏ khoảng trắng hai đầu rồi chuyển chữ thường
`text.split()`	Tách theo khoảng trắng; không đủ cho mọi ngôn ngữ
`set(tokens)`	Lấy token khác nhau, bỏ lượt lặp
`len(a & b) / len(a | b)`	Jaccard khi hợp khác rỗng; kiểm tra mẫu số trước khi chia

4. Khái niệm chính và liên quan

Mạch kiến thức: Một câu văn trước tiên được chuẩn hóa rồi tách thành token. Các token khác nhau tạo vocabulary; từ hai vocabulary, Jaccard đo mức trùng nhau theo mặt chữ. Cần hiểu sự khác nhau giữa tách từ đơn giản và hiểu ngữ nghĩa.

4.1. Token — Đơn vị nhỏ sau quy tắc tách

Từ vựng: Token.

Trong xử lý ngôn ngữ tự nhiên (NLP), **token** là đơn vị văn bản sau một bước tách: có thể là từ, dấu câu, hoặc phần nhỏ của từ. Với câu tiếng Anh đơn giản `AI helps Python`, `split()` theo khoảng trắng tạo ba token. Đây **không phải** bộ tách từ tiếng Việt chuẩn: một từ ghép tiếng Việt có thể gồm nhiều tiếng và cần tokenizer chuyên biệt.

No Note — Không chú thích

```
sentence = 'AI helps Python'
print(sentence.lower().split())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách một câu thành token chữ thường để chuẩn hóa từ ngữ trước khi so sánh.
# Dữ liệu: Câu 'AI helps Python' gồm ba từ cách nhau bởi dấu cách; chưa có dấu câu.
# Giữ câu gốc có chữ hoa để minh họa bước chuẩn hóa.
sentence = 'AI helps Python'
# lower() tạo chuỗi mới viết thường; split() tách theo khoảng trắng và trả list token, KHÔNG tự
loại dấu câu.
print(sentence.lower().split())
```

Kết quả khi chạy:

```
['ai', 'helps', 'python']
```

Ba token chữ thường sau tách theo khoảng trắng; dấu câu chưa được xử lý.

4.2. Vocabulary — Tập token duy nhất

Từ vựng: Vocabulary.

Vocabulary là tập các token **khác nhau** quan sát được trong kho văn bản. Hai câu `AI uses data` và `AI uses Python` có token chung `AI`, `uses`; dùng Set chỉ giữ mỗi từ một lần. Set giúp đếm từ duy nhất nhưng không giữ số lần xuất hiện, khác Bag-of-Words. In qua `sorted()` để không phụ thuộc thứ tự Set.

No Note — Không chú thích

```
sentences = ['AI uses data', 'AI uses Python']
tokens = ' '.join(sentences).lower().split()
print(sorted(set(tokens)))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo tập token duy nhất từ hai câu sau khi ghép và chuẩn hóa.
# Dữ liệu: Hai câu có từ AI và uses lặp lại; cần danh sách TỪ KHÁC NHAU, không phải tần suất.
# List gồm hai câu đầu vào; mỗi phần tử là một chuỗi.
sentences = ['AI uses data', 'AI uses Python']
# join() nối hai câu bằng dấu cách, tránh nối dính từ cuối câu trước với từ đầu câu sau.
# lower() chuẩn hóa chữ thường rồi split() tách token theo khoảng trắng.
tokens = ' '.join(sentences).lower().split()
# set() bỏ các token lặp; sorted() xếp tăng theo chữ để kết quả in ổn định.
print(sorted(set(tokens)))
```

Kết quả khi chạy:

```
['ai', 'data', 'python', 'uses']
```

Kho từ vựng có bốn token khác nhau; `ai` và `uses` không bị đếm hai lần.

4.3. Normalization — Chuẩn hóa trước khi so từ

Từ vựng: Normalization.

Normalization đưa văn bản về một quy ước nhất quán: bỏ khoảng trắng hai đầu, chuyển chữ về thường, đôi khi xử lý Unicode và dấu câu. Ví dụ `Machine LEARNING` chuẩn hóa thành `machine learning`. Nếu không chuẩn hóa, `AI` và `ai` có thể bị xem là hai token; tuy nhiên xóa dấu tiếng Việt tùy tiện có thể làm mất nghĩa.

No Note — Không chú thích

```
text = '  Machine LEARNING '
print(text.strip().lower())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Dọn dâú cách thừa hai đâù và viêt thường một cụm tìm kiêḿ.
# Dữ liệu: Chuôĩ gôć có khoảng trăńg thừa và chữ hoa 'Machine LEARNING'.
# Chuôĩ đâù vào vâń giữ nguyên dâú cách/chữ hoa khi chưa xử lý.
text = '  Machine LEARNING '
# strip() bỏ khoảng trăńg ở ĐÂÙ/CUÔI, không xóa khoảng trăńg giữa từ; lower() đôỉ sang chữ thường,
không sưả biêń text gôć.
print(text.strip().lower())
```

Kết quả khi chạy:

```
machine learning
```

Khoảng trắng ở đầu/cuối bị bỏ, chữ hoa chuyển thành chữ thường.

4.4. Jaccard — Tỷ lệ token chung so với tất cả token

Từ vựng: Jaccard similarity.

Jaccard đo độ giống của hai **tập token**: `J(A,B)=|A∩B|/|A∪B|` nếu hợp khác rỗng. Câu A `python for ai` có ba token; câu B `python and sql` cũng ba token. Giao chỉ có `python` (1 phần tử), hợp có năm từ, nên J=0.2. Jaccard Set không tính tần suất và không hiểu nghĩa tương đương của từ; với hai tập rỗng cần chọn quy ước riêng để tránh chia 0.

No Note — Không chú thích

```
a = set('python for ai'.split())
b = set('python and sql'.split())
print(round(len(a & b) / len(a | b), 2))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đo độ tương đôǹg Jaccard giữa hai câu qua tập các từ.
# Dữ liệu: Hai cụm có chung từ python; tập hợp hợp nhất có năm từ khác nhau.
# split() chia theo dâú cách, set() bỏ từ trùng để chỉ còn những từ duy nhất của câu A.
```

```
a = set('python for ai'.split())
# Tạo tập từ B theo cùng quy tắc để so công bằng.
b = set('python and sql'.split())
# & là giao các từ chung; | là hợp các từ không trùng; len() đếm số từ, chia tạo Jaccard từ 0 đến
1, round(...,2) chỉ làm tròn khi in.
print(round(len(a & b) / len(a | b), 2))
```

Kết quả khi chạy:

```
0.2
```

Hai tài liệu chung một trên năm token khác nhau; độ giống tập hợp 20%.

5. Năm tình huống tiền xử lý NLP — từ 10 từ khóa đến 20 phiếu hỗ trợ

Tình huống 1 — Cấp 1: Chuẩn hóa 10 từ khóa tìm kiếm khác kiểu viết

Từ vựng trong mục: Normalization — chuẩn hóa; **Vocabulary** — kho từ khác nhau.

Tình huống thực tế: Thư viện ghi **10 lượt nhập từ khóa tìm sách**; một người gõ `Python`, người khác gõ `PYTHON`, có người gõ `Máy học` và `máy học`. Bộ phận tìm kiếm muốn biết có bao nhiêu **cụm từ khác nhau theo cách viết đã chuẩn hóa**, không đếm nhiều lần cùng một nội dung chỉ vì chữ hoa hay khoảng trắng. Cần giữ nguyên **dấu tiếng Việt**, vì bỏ dấu có thể đổi nghĩa từ.

Vì sao chọn cách này? Chuẩn hóa văn bản là đưa các cách viết khác nhau về một dạng **nhất quán** trước khi so sánh. `strip()` bỏ khoảng trắng thừa ở **hai đầu** chuỗi, `lower()` đổi sang chữ thường nên `Python` và `PYTHON` trở thành cùng một từ khóa. `set(normalized)` chỉ giữ **mỗi cụm đã chuẩn hóa một lần**, còn `sorted()` sắp để in kết quả ổn định. Các bước này chỉ giải quyết **khác biệt cách viết**, không hiểu câu hỏi tìm kiếm hay tự nối hai từ đồng nghĩa; đặc biệt không tự bỏ dấu tiếng Việt.

Quy trình:

1. Lưu 10 cụm từ theo đúng đầu vào.
2. Chuẩn hóa khoảng trắng hai đầu và chữ hoa/thường.
3. In số lượt và danh sách cụm duy nhất theo thứ tự ổn định.

No Note — Không chú thích

```
queries = [" Python ","PYTHON","AI"," ai ","Data","DATA","NLP","nLp","Máy học","máy học"]
normalized = []
for text in queries:
    normalized.append(text.strip().lower())
unique = sorted(set(normalized))
if len(queries) != 10:
    raise ValueError("Cần đủ 10 lượt tìm kiếm.")
print("Lượt tìm:", len(queries))
print("Cụm khác nhau:", len(unique))
print("Cụm:", ", ".join(unique))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chuẩn hóa mười lượt tìm kiếm và đếm số cụm khác nhau sau khi bỏ chữ hoa/khoảng trắng.
# Dữ liệu: 10 chuỗi truy vấn có bản sao khác kiểu chữ và khoảng trắng; vẫn giữ tiếng Việt có dấu.
# Danh sách mười LƯỢT tìm kiếm; cùng ý định có thể xuất hiện nhiều lần.
queries = [" Python ","PYTHON","AI"," ai ","Data","DATA","NLP","nLp","Máy học","máy học"]
```

```
# List mới chứa chuôĩ chuẩn hóa theo cùng thứ tự, không ghi đè queries gôć.
normalized = []
# Duyệt một lượt cho môĩ truy vấń.
for text in queries:
    # strip() xóa khoảng trăńg hai đâù, lower() đôỉ chữ hoa sang chữ thường, không bỏ dâú tiếńg
Việt.
    # append() thêm kêt quả chuẩn hóa vào list mới.
    normalized.append(text.strip().lower())
# set() bỏ các truy vấń trùng sau chuẩn hóa; sorted() săṕ danh sách khác nhau cho dê˜đọc.
unique = sorted(set(normalized))
# len() kiêm̉ đủ mười lượt gôć trước khi kêt́ luận.
if len(queries) != 10:
    # Báo lôĩ khi dữ liệu thiêú hoặc sai sô´lượt, tránh đánh giá không đủ mâũ.
    raise ValueError("Cần đủ 10 lượt tìm kiêḿ.")
print("Lượt tìm:", len(queries))
# Đêḿ sô´cụm KHÁC NHAU, không phải tổng lượt.
print("Cụm khác nhau:", len(unique))
# join() nôí các cụm khác nhau bằng dâú phâỷ và khoảng trăńg.
print("Cụm:", ", ".join(unique))
```

Kết quả khi chạy:

```
Lượt tìm: 10
Cụm khác nhau: 5
Cụm: ai, data, máy học, nlp, python
```

Từ `máy học` vẫn gồm hai tiếng cách nhau bằng dấu cách. Nếu cần phân tích từ ghép tiếng Việt, phải có tokenizer phù hợp hơn.

Tình huống 2 — Cấp 2: Tách token của 12 lời nhắn có dấu câu

Từ vựng trong mục: Tokenization — tách token; **Regex** — biểu thức chính quy.

Tình huống thực tế: Đội hỗ trợ lưu **12 tin nhắn tiếng Anh ngắn**, chẳng hạn `AI helps Python.` và `Python helps teams!`. Mỗi hàng là **một tin nhắn**, có khoảng trắng và dấu câu. Nhân viên muốn tách thành từng từ **chữ thường**, biết tổng **lượt từ xuất hiện** và có bao nhiêu **từ khác nhau**, rồi kiểm tra từ `python` có xuất hiện không.

Vì sao chọn cách này? Token là đơn vị văn bản chương trình tách ra để xử lý, trong ví dụ này là **chuỗi chữ cái tiếng Anh**. `re.findall(r'[a-z]+', text.lower())` bỏ dấu chấm/chấm than rồi lấy từng nhóm chữ từ **12 lời nhắn**; danh sách `tokens` giữ **tất cả lượt xuất hiện**, còn `set(tokens)` tạo **vốn từ không lặp**. Từ `python` xuất hiện nhiều lần nhưng chỉ có một mục trong tập từ. Quy tắc `[a-z]` được chọn vì dữ liệu mẫu là **tiếng Anh ASCII**, không phải bộ tách từ chính xác cho tiếng Việt, emoji hay mọi ngôn ngữ.

Quy trình:

1. Tạo 12 tin nhắn không rỗng.
2. Chuẩn hóa chữ thường và tìm chuỗi chữ a–z bằng regex.
3. Đếm tổng token và vốn từ khác nhau để thấy một từ có thể lặp.

No Note — Không chú thích

```
import re
messages = ["AI helps Python.","Python helps teams!","Data drives AI.","AI uses data.","Python
cleans data.","Teams learn Python.","AI supports teams.","Data needs quality.","Learn AI with
Python.","Quality data matters.","Python helps AI.","Teams review data."]
tokens = []
for text in messages:
```

```
    for word in re.findall(r"[a-z]+", text.lower()):
        tokens.append(word)
vocabulary = set(tokens)
if len(messages) != 12:
    raise ValueError("Cần đủ 12 tin nhắn.")
print("Tin nhắn:", len(messages))
print("Lượt token:", len(tokens))
print("Token khác nhau:", len(vocabulary))
print("Có từ python:", "python" in vocabulary)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tách token chữ cái trong 12 tin nhắn và xây vốn từ duy nhất.
# Dữ liệu: 12 câu tiếng Anh có dấu chấm/chấm than; muốn đếm lượt token và số´từ khác nhau.
# re là thư viện chuẩn cho biểu thức chính quy (regex), dùng tách từ bỏ dấu câu.
import re
# List 12 tin nhắn; mỗi phần tư ̉là một câu với dấu câu có thể ̉khác nhau.
messages = ["AI helps Python.","Python helps teams!","Data drives AI.","AI uses data.","Python
cleans data.","Teams learn Python.","AI supports teams.","Data needs quality.","Learn AI with
Python.","Quality data matters.","Python helps AI.","Teams review data."]
# Lưu MỌI lượt token, kể ̉cả những từ lặp lại qua các câu.
tokens = []
# Duyệt từng tin nhắn riêng.
for text in messages:
    # lower() chuyển chữ thường; findall() tìm mọi CHUỖI LIÊN TIẾP chỉ gồm a..z, bỏ dấu câu/chữ số´
và không hỗ˜trợ nguyên vẹn từ tiếng Việt có dấu.
    for word in re.findall(r"[a-z]+", text.lower()):
        # append() giữ mỗi lần xuất hiện của từ, kể ̉cả khi trùng.
        tokens.append(word)
# set() biến các lượt token thành vốn từ duy nhất, bỏ lặp.
vocabulary = set(tokens)
# Kiểm đủ 12 tin nhắn nguồn.
if len(messages) != 12:
    # Dừng khi nguồn thiếu tin nhắn theo kịch bản.
    raise ValueError("Cần đủ 12 tin nhắn.")
print("Tin nhắn:", len(messages))
# Đếm tổng lượt xuất hiện của token (có lặp).
print("Lượt token:", len(tokens))
# Đếm số´token khác nhau (không lặp).
print("Token khác nhau:", len(vocabulary))
# in kiểm từ python có trong set hay không, trả Boolean.
print("Có từ python:", "python" in vocabulary)
```

Kết quả khi chạy:

```
Tin nhắn: 12
Lượt token: 37
Token khác nhau: 15
Có từ python: True
```

Tổng lượt token tính cả từ lặp, còn vocabulary chỉ tính từ duy nhất — hai đại lượng không thay thế nhau.

Tình huống 3 — Cấp 3: So Bag-of-Words và Set trên 15 tiêu đề khóa học

Từ vựng trong mục: Bag-of-Words — đếm lần xuất hiện; **Counter** — bảng tần suất.

Tình huống thực tế: Trang đào tạo có **15 tiêu đề khóa học**, mỗi tiêu đề gồm tên chủ đề `python`, `ai` hoặc `data` và từ `course`. Vì `course` được lặp lại trong mọi tiêu đề, nhân viên muốn tìm **từ phổ biến nhất và số lượt của từ ấy** để xây danh sách tra cứu. Họ cũng muốn biết **kho từ riêng biệt** gồm bao nhiêu từ; chỉ đếm từ khác nhau sẽ không cho biết mức độ phổ biến.

Vì sao chọn cách này? Tần suất trả lời 'một từ xuất hiện **bao nhiêu lần**', khác **vốn từ** chỉ trả lời 'có **những từ nào**'. Chương trình tách hai từ trong từng tiêu đề bằng `split()`, ghép tất cả thành một danh sách rồi dùng `Counter` cộng lượt xuất hiện của từng token. `most_common(1)` tìm từ nhiều lượt nhất: `course` xuất hiện ở **cả 15 tiêu đề**, còn các tên chủ đề xuất hiện ít hơn. Nếu đổi toàn bộ token sang set ngay lúc đầu, `course` sẽ chỉ còn một lần và **mất thông tin để xếp tần suất**; vì thế dùng Counter trước và nhìn số khóa để biết số từ khác nhau.

Quy trình:

1. Lưu 15 tiêu đề có hai token mỗi tiêu đề.
2. Tách từ, dùng Counter để đếm toàn bộ lượt.
3. In số từ khác nhau, tổng lượt token, tên và tần suất của từ phổ biến nhất.

No Note — Không chú thích

```
from collections import Counter
headings = ["python course","ai course","data course","python course","ai course","data
course","python course","ai course","data course","python course","ai course","data course","python
course","ai course","data course"]
tokens = []
for heading in headings:
    for token in heading.split():
        tokens.append(token)
counts = Counter(tokens)
most_common = counts.most_common(1)[0]
if len(headings) != 15:
    raise ValueError("Cần đủ 15 tiêu đề.")
print("Tiêu đề:", len(headings))
print("Lượt token:", len(tokens))
print("Từ khác nhau:", len(counts))
print("Từ phổ biến nhất:", most_common[0])
print("Số lượt:", most_common[1])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đếm tần suất từng từ trong 15 tiêu đề và tìm từ xuất hiện nhiều nhất.
# Dữ liệu: 15 tiêu đề xoay quanh python/ai/data và course; 'course' lặp ở mỗi tiêu đề.
# Counter là kiểu dictionary chuyên đếm số lần mỗi phần tử xuất hiện.
from collections import Counter
# 15 tiêu đề, không phải 15 từ riêng biệt.
headings = ["python course","ai course","data course","python course","ai course","data
course","python course","ai course","data course","python course","ai course","data course","python
course","ai course","data course"]
# List lưu mọi từ từ các tiêu đề, giữ lặp để tính tần suất.
tokens = []
# Duyệt từng tiêu đề.
for heading in headings:
    # split() tách token tại khoảng trắng, trả list các từ của tiêu đề.
    for token in heading.split():
        # append() lưu từng lượt token, giữ từ lặp.
        tokens.append(token)
# Counter() tạo ánh xạ từ→số lượt xuất hiện từ list có lặp.
counts = Counter(tokens)
# most_common(1) trả list một tuple (từ,số lượt) có tần suất cao nhất; [0] lấy chính tuple đó.
most_common = counts.most_common(1)[0]
# Kiểm đúng 15 tiêu đề nguồn.
if len(headings) != 15:
    # Dừng nếu thiếu tiêu đề.
    raise ValueError("Cần đủ 15 tiêu đề.")
print("Tiêu đề:", len(headings))
# Số lượt token, bao gồm trùng lặp.
```

```
print("Lượt token:", len(tokens))
# Số TỪ KHÁC NHAU được Counter lưu làm khóa.
print("Từ khác nhau:", len(counts))
# Chỉ số [0] lấy từ phổ biến nhất trong tuple; [1] là số lần xuất hiện.
print("Từ phổ biến nhất:", most_common[0])
# Đọc số lượt xuất hiện của từ được chọn.
print("Số lượt:", most_common[1])
```

Kết quả khi chạy:

```
Tiêu đề: 15
Lượt token: 30
Từ khác nhau: 4
Từ phổ biến nhất: course
Số lượt: 15
```

Từ `course` phổ biến nhất vì có mặt trong mọi tiêu đề; chưa chắc đây là từ tốt nhất để **phân biệt chủ đề** giữa các khóa học.

Tình huống 4 — Cấp 4: Xếp hạng 18 phiếu hỗ trợ theo Jaccard, có nội dung rỗng

Từ vựng trong mục: Jaccard — tỷ lệ token chung; **Ranking** — xếp hạng; **Empty-set guard** — bảo vệ mẫu rỗng.

Tình huống thực tế: Trung tâm nhận **18 phiếu hỗ trợ T01–T18**, mỗi phiếu là một mô tả ngắn **bằng tiếng Anh**, như `login password reset`; **T17 chưa có nội dung**. Khi có câu hỏi mới `login password`, nhân viên cần xếp hạng các phiếu theo **số từ chung so với toàn bộ từ khác nhau**, đề xuất **ba phiếu khớp chữ tốt nhất** và báo số phiếu thiếu nội dung, không suy diễn các phiếu cùng nguyên nhân sự cố.

Vì sao chọn cách này? Jaccard là mức giống nhau giữa **hai tập từ**: đếm từ chung và so với toàn bộ từ khác nhau của cả hai phía. Hàm `words()` đổi chuỗi thành set từ tiếng Anh chữ thường, tránh tính một từ lặp trong cùng phiếu thành nhiều lần. Hàm `similarity()` trả **0 khi hợp hai tập rỗng** để tránh chia cho 0; phiếu T17 trống vẫn được **đếm là thiếu nội dung**, không lặng lẽ xóa. Mỗi phiếu được gắn mã T01… và sắp **điểm tương đồng giảm dần, mã tăng dần khi hòa**, rồi lấy ba phiếu đầu. Giống chữ không chứng minh hai người bị cùng nguyên nhân lỗi, nên kết quả chỉ là gợi ý **rà soát**, không tự đóng phiếu.

Quy trình:

1. Chuẩn hóa token cho cả 18 phiếu; phiếu rỗng thành tập rỗng.
2. Tính Jaccard với hai token của truy vấn, sắp theo điểm giảm và ID tăng.
3. In ba phiếu gần nhất và số phiếu không có token.

No Note — Không chú thích

```
import re
texts = ["login password reset","login password issue","reset login account","payment invoice
request","refund payment card","shipment tracking delayed","delivery package status","account
password change","login device blocked","payment not received","invoice missing pdf","courier lost
shipment","delivery date change","reset password urgent","account verification failed","general
help needed","","password login help"]
def words(text):
    return set(re.findall(r"[a-z]+", text.lower()))
def similarity(a,b):
    common = a & b
    union = a | b
    if union:
        return len(common) / len(union)
```

```
    return 0.0
query = words("login password")
ranked = []
for i, text in enumerate(texts, 1):
    ranked.append((f"T{i:02d}", similarity(query, words(text))))
ranked.sort(key=lambda row:(-row[1],row[0]))
if len(texts) != 18:
    raise ValueError("Cần đủ 18 phiếu hỗ trợ.")
print("Phiếu:", len(texts))
missing_texts = 0
for text in texts:
    if not words(text):
        missing_texts += 1
print("Thiếu nội dung:", missing_texts)
for code, value in ranked[:3]:
    print(f"{code}: {value:.3f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tính Jaccard giữa từ khóa truy vấn và 18 phiếu hỗ trợ, xếp hạng ba phiếu liên quan
nhất.
# Dữ liệu: 18 nội dung phiếu, có một phiếu rỗng; truy vấn 'login password' được tách thành set
token.
# Regex re.findall() tìm các chuỗi chữ cái liên tiếp trong từng phiếu.
import re
# 18 phiếu, chuỗi rỗng thể hiện phiếu chưa ghi mô tả.
texts = ["login password reset","login password issue","reset login account","payment invoice
request","refund payment card","shipment tracking delayed","delivery package status","account
password change","login device blocked","payment not received","invoice missing pdf","courier lost
shipment","delivery date change","reset password urgent","account verification failed","general
help needed","","password login help"]
# Hàm nhận một câu và trả TẬP từ duy nhất sau chuẩn hóa.
def words(text):
    # lower() chuẩn hóa chữ; findall() chỉ tách từ a-z, bỏ dấu câu; set() loại từ lặp, không tách
chuẩn tiếng Việt có dấu.
    return set(re.findall(r"[a-z]+", text.lower()))
# Hàm nhận hai set token và trả điểm Jaccard.
def similarity(a,b):
    # & là giao các token chung giữa hai tập.
    common = a & b
    # | là hợp chứa mọi token có ở ít nhất một tập, không trùng.
    union = a | b
    # Kiểm tập hợp KHÔNG RỖNG trước khi chia, tránh ZeroDivisionError.
    if union:
        # Jaccard = số từ chung / số từ trong hợp; từ 0 đến 1.
        return len(common) / len(union)
    # Nếu cả hai tập đều rỗng, quy ước điểm bằng 0 thay vì chia 0.
    return 0.0
# Chuẩn hóa truy vấn thành set hai từ bằng cùng hàm words().
query = words("login password")
# List lưu tuple (mã phiếu, điểm tương đồng).
ranked = []
# enumerate(...,1) đánh số phiếu từ T01 đến T18 thay vì chỉ số Python 0.
for i, text in enumerate(texts, 1):
    # Định dạng số thứ tự hai chữ số để có mã T01..T18.
    # append() thêm một tuple kết quả cho mỗi phiếu.
    ranked.append((f"T{i:02d}", similarity(query, words(text))))
# sort(key=...) ưu tiên điểm row[1] GIẢM DẦN nhờ dấu âm, nếu hòa thì mã row[0] TĂNG DẦN để ổn định.
ranked.sort(key=lambda row:(-row[1],row[0]))
# Kiểm đủ 18 phiếu ban đầu.
if len(texts) != 18:
    # Dừng nếu sai số lượng phiếu.
    raise ValueError("Cần đủ 18 phiếu hỗ trợ.")
print("Phiếu:", len(texts))
# 18 phiếu, chuỗi rỗng thể hiện phiếu chưa ghi mô tả.
```

```
missing_texts = 0
for text in texts:
    # not set đúng khi không rút được token, phát hiện chuoĩ rôñg/không có từ a-z.
    if not words(text):
        # Đêḿ sô´phiêú thiêú token hữu ích.
        missing_texts += 1
print("Thiêú nội dung:", missing_texts)
# [:3] chọn ba phiêú đâù sau xêṕ hạng, không lâý phâǹ tư ̛ vị trí 3.
for code, value in ranked[:3]:
    # In mã và điêḿ Jaccard với ba chữ sô´thập phân.
    print(f"{code}: {value:.3f}")
```

Kết quả khi chạy:

```
Phiêú: 18
Thiêú nội dung: 1
T01: 0.667
T02: 0.667
T18: 0.667
```

Jaccard Set bỏ thông tin **thứ tự và tần suất**; phiếu điểm cao vẫn cần người xem ngữ cảnh để trả lời đúng.

Tình huống 5 — Cấp 5: Định tuyến 20 phiếu qua ba nhóm từ vựng và báo nhóm chưa rõ

Từ vựng trong mục: Rule-based classification — phân loại theo quy tắc; **Tie-break** — phân xử hòa; **Unknown** — chưa xác định.

Tình huống thực tế: Đội chăm sóc khách hàng có **20 phiếu P01–P20**, mỗi phiếu chứa một lời nhắn tiếng Anh hoặc chuỗi rỗng; **P20 chưa nhập nội dung**. Trung tâm có ba nhóm tiếp nhận: `account` xử lý đăng nhập/mật khẩu, `billing` xử lý thanh toán và `delivery` xử lý giao hàng. Cần định tuyến từng phiếu theo **từ khóa xuất hiện**, đếm phiếu mỗi nhóm và để `unknown` cho nội dung không khớp, không tự gán phiếu chưa hiểu cho một bộ phận.

Vì sao chọn cách này? Đây là **bộ định tuyến dựa trên từ khóa**, không phải mô hình hiểu ngôn ngữ tự nhiên. Mỗi nhóm có một **tập từ gợi ý**, còn `tokens(text)` lấy tập từ xuất hiện trong phiếu; chương trình tính **Jaccard** giữa phiếu và từng nhóm để xem mức trùng chữ. Nếu **tất cả điểm đều bằng 0**, `route` trả `unknown`, kể cả phiếu rỗng; nếu có điểm dương bằng nhau, tên nhóm được xếp theo **thứ tự chữ cái** để kết quả ổn định. `Counter` thống kê đủ **20 lượt định tuyến**, không biến P20 thành một phiếu hợp lệ đã biết nội dung. Các từ lạ hoặc cách diễn đạt khác có thể cần người kiểm tra lại; quy tắc này không đánh giá được ý định thực của khách.

Quy trình:

1. Lưu đủ 20 phiếu và ba nhóm từ khóa.
2. Chuẩn hóa token, tính Jaccard giữa phiếu với từng nhóm.
3. Chọn nhóm điểm dương cao nhất, còn 0 thì unknown; tổng hợp số phiếu của từng nhóm.

No Note — Không chú thích

```
import re
from collections import Counter
texts = ["login password reset urgent","invoice payment refund request","shipment delivery tracking
update","general question please","login password reset urgent","invoice payment refund
request","shipment delivery tracking update","general question please","login password reset
urgent","invoice payment refund request","shipment delivery tracking update","general question
please","login password reset urgent","invoice payment refund request","shipment delivery tracking
```

```
update","general question please","login password reset urgent","invoice payment refund
request","shipment delivery tracking update",""]
keywords = {"account": {"login","password","reset"},
            "billing": {"invoice","payment","refund"},
            "delivery": {"shipment","delivery","tracking"}}
def tokens(text):
    return set(re.findall(r"[a-z]+", text.lower()))
def route(text):
    words = tokens(text)
    scores = {}
    for name,vocabulary in keywords.items():
        union = words | vocabulary
        if union:
            scores[name] = len(words & vocabulary) / len(union)
        else:
            scores[name] = 0
    name = sorted(scores, key=lambda n:(-scores[n], n))[0]
    if scores[name] > 0:
        return name
    return "unknown"
counts = Counter()
for text in texts:
    category = route(text)
    counts[category] += 1
if len(texts) != 20:
    raise ValueError("Cần đủ 20 phiếu cần phân tuyến.")
print("Phiếu:", len(texts))
for name in ("account","billing","delivery","unknown"):
    print(f"{name}: {counts[name]}")
empty_count = 0
for text in texts:
    if not tokens(text):
        empty_count += 1
print("Phiếu rỗng:", empty_count)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tự động phân tuyến 20 phiếu vào account/billing/delivery bằng Jaccard từ khóa.
# Dữ liệu: 20 chuỗi phiếu, ba nhóm từ khóa; với điểm 0 cho mọi nhóm thì trả unknown.
# Dùng regex để tách token chỉ gồm ký tự a-z.
import re
# Counter đếm số phiếu được gán vào từng nhóm, mặc định khóa chưa có có số đếm 0.
from collections import Counter
# 20 phiếu; dòng rỗng sẽ có tập từ rỗng và thường được trả unknown.
texts = ["login password reset urgent","invoice payment refund request","shipment delivery tracking
update","general question please","login password reset urgent","invoice payment refund
request","shipment delivery tracking update","general question please","login password reset
urgent","invoice payment refund request","shipment delivery tracking update","general question
please","login password reset urgent","invoice payment refund request","shipment delivery tracking
update","general question please","login password reset urgent","invoice payment refund
request","shipment delivery tracking update",""]
# Dictionary ánh xạ tên chủ đề sang tập từ khóa duy nhất; mỗi chủ đề có vốn từ riêng.
keywords = {"account": {"login","password","reset"},
            "billing": {"invoice","payment","refund"},
            "delivery": {"shipment","delivery","tracking"}}
# Tách và chuẩn hóa chuỗi thành set từ không trùng.
def tokens(text):
    # Tìm từ chữ cái ASCII sau lower(); không tách nguyên vẹn tiếng Việt có dấu và bỏ dấu câu.
    return set(re.findall(r"[a-z]+", text.lower()))
# Hàm chọn nhãn chủ đề cho một phiếu theo mức giao từ khóa.
# Phân loại phiếu bằng hàm tính Jaccard vừa định nghĩa.
def route(text):
    # Chuẩn hóa nội dung phiếu thành set từ.
    words = tokens(text)
    # Dictionary điểm tương đồng theo từng tên chủ đề.
```

```
    scores = {}
    # items() duyệt cặp (tên nhóm, tập từ khóa) để chấm từng nhóm.
    for name,vocabulary in keywords.items():
        # | lấy hợp mọi từ của phiếu và bộ từ khóa nhóm.
        union = words | vocabulary
        # Tránh chia cho 0 khi cả phiếu và vocabulary rỗng.
        if union:
            # & là giao các từ trong phiếu trùng từ khóa chủ đề.
            # Điểm Jaccard bằng số từ chung chia tổng loại từ, không đếm lặp.
            scores[name] = len(words & vocabulary) / len(union)
        else:
            # Nếu hợp rỗng thì quy ước tương đồng 0.
            scores[name] = 0
    # Xếp nhãn theo điểm GIẢM (âm scores[n]), hòa điểm theo tên A-Z; [0] lấy tên tốt nhất.
    name = sorted(scores, key=lambda n:(-scores[n], n))[0]
    # Chỉ chấp nhận chủ đề khi có ít nhất một phần trùng, tránh tự gán phiếu không liên quan.
    if scores[name] > 0:
        return name
    # Không có từ khóa chung thì gắn unknown, không ép thuộc một nhóm bất kỳ.
    return "unknown"
# Khởi tạo bộ đếm số phiếu cho mỗi tuyến.
counts = Counter()
# Duyệt từng phiếu nguồn đúng một lần.
for text in texts:
    # Phân loại phiếu bằng hàm tính Jaccard vừa định nghĩa.
    category = route(text)
    # Tăng một lượt cho tuyến được chọn; Counter tự khởi tạo số đếm 0.
    counts[category] += 1
# Kiểm đúng hai mươi phiếu nguồn.
if len(texts) != 20:
    # Báo sai số phiếu để tránh báo cáo thiếu.
    raise ValueError("Cần đủ 20 phiếu cần phân tuyến.")
print("Phiếu:", len(texts))
for name in ("account","billing","delivery","unknown"):
    # Tra số phiếu từng tuyến, với Counter tuyến chưa gặp trả 0.
    print(f"{name}: {counts[name]}")
empty_count = 0
# Duyệt từng phiếu nguồn đúng một lần.
for text in texts:
    # Tập token rỗng đánh dấu phiếu rỗng hoặc không có từ phù hợp regex.
    if not tokens(text):
        # Đếm phiếu thiếu nội dung có thể tách token.
        empty_count += 1
print("Phiếu rỗng:", empty_count)
```

Kết quả khi chạy:

```
Phiếu: 20
account: 5
billing: 5
delivery: 5
unknown: 5
Phiếu rỗng: 1
```

Quy tắc này không được coi là AI đã **học** từ dữ liệu. Phiếu unknown là yêu cầu bổ sung nội dung hoặc người thật kiểm tra, không phải lỗi cần xóa.

6. Sai lầm thường gặp

- Tách theo khoảng trắng không luôn phân đoạn từ tiếng Việt chính xác; tokenizer phụ thuộc ngôn ngữ và mục tiêu.

- `lower()` không xóa dấu câu, `strip()` không xóa mọi khoảng trắng giữa câu.
- Set bỏ trùng nhưng đánh mất thứ tự và tần suất; Jaccard trên Set không hiểu từ đồng nghĩa.
- Mẫu số của Jaccard là kích thước **hợp**, không phải độ dài một tài liệu.
- Hai tập rỗng gây phép chia 0/0, cần nêu rõ quy ước ứng dụng.
- Không so sánh tài liệu khác cách chuẩn hóa, vì khác tiền xử lý sẽ tạo token khác nhau.

7. Trắc nghiệm

Q1. What is a token? | Token là gì?

A. A unit produced by a text-splitting rule | Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens | Tập các token khác nhau

C. Applying consistent text representations | Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

Đáp án: A

Giải thích: Token có thể là từ, mảnh từ hoặc dấu câu tùy bộ tách. Đơn vị cụ thể phụ thuộc quy tắc tokenizer; token không nhất thiết là một từ hoàn chỉnh.

Q2. What is a vocabulary? | Vocabulary là gì?

A. A unit produced by a text-splitting rule | Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens | Tập các token khác nhau

C. Applying consistent text representations | Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

Đáp án: B

Giải thích: Tập từ vựng bỏ trùng danh tính token, khác bảng tần suất. Nó trả lời có những token nào, chứ không trả lời mỗi token xuất hiện bao nhiêu lần.

Q3. What is text normalization? | Chuẩn hóa văn bản là gì?

A. A unit produced by a text-splitting rule | Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens | Tập các token khác nhau

C. Applying consistent text representations | Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

Đáp án: C

Giải thích: Chữ thường và cắt khoảng trắng giúp khớp dạng văn bản. Chuẩn hóa đổi cách biểu diễn; quy tắc phải nhất quán giữa các văn bản đem so.

Q4. What is set-based Jaccard similarity? | Jaccard trên Set là gì?

A. A unit produced by a text-splitting rule | Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens | Tập các token khác nhau

C. Applying consistent text representations | Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size | Số phần tử giao chia số phần tử hợp

Đáp án: D

Giải thích: Nó đo từ khác nhau chung so với tổng số từ khác nhau. Một từ chung trong năm từ khác nhau cho 1/5, không phải lấy độ dài một câu làm mẫu số.

Q5. What does `str.split()` do without arguments? | `str.split()` không đối số làm gì?

A. Splits on runs of whitespace | Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase | Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace | Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently | Để bỏ token trùng và hiển thị ổn định

Đáp án: A

Giải thích: Tách bằng khoảng trắng không hiểu mọi ranh giới từ. Đây là quy tắc tách chuỗi, không phải thuật toán phân đoạn từ tiếng Việt về mặt ngôn ngữ.

Q6. What does `lower()` do? | `lower()` làm gì?

A. Splits on runs of whitespace | Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase | Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace | Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently | Để bỏ token trùng và hiển thị ổn định

Đáp án: B

Giải thích: Giúp so biến thể chữ hoa/thường nhưng không bỏ dấu câu. `lower` chỉ đổi chữ hoa/thường, không tự xóa khoảng trắng hoặc tách token.

Q7. What does `strip()` do? | `strip()` làm gì?

A. Splits on runs of whitespace | Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase | Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace | Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently | Để bỏ token trùng và hiển thị ổn định

Đáp án: C

Giải thích: strip không xóa khoảng trắng giữa câu. Muốn xử lý khoảng trắng giữa câu cần `split` hoặc bước khác, không thể chỉ dùng `strip`.

Q8. Why use `sorted(set(tokens))` for examples? | Vì sao dùng `sorted(set(tokens))`?

A. Splits on runs of whitespace | Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase | Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace | Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently | Để bỏ token trùng và hiển thị ổn định

Đáp án: D

Giải thích: Set không bảo đảm thứ tự; sorted tạo List có thứ tự. Set loại phần tử trùng, còn `sorted` tạo List có thứ tự để in ổn định.

Q9. What does A & B produce for sets? | A & B trên Set tạo gì?

A. Common tokens | Các token chung

B. All distinct tokens from either set | Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences | Bỏ qua số lần lặp

D. 1.0 | 1.0

Đáp án: A

Giải thích: Giao chỉ giữ giá trị có ở cả hai tập. Với A={'ai','python'}, B={'ai'}, giao chỉ có 'ai', không chứa toàn bộ A.

Q10. What does A | B produce for sets? | A | B trên Set tạo gì?

A. Common tokens | Các token chung

B. All distinct tokens from either set | Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences | Bỏ qua số lần lặp

D. 1.0 | 1.0

Đáp án: B

Giải thích: Hợp loại token trùng ở cả hai bên. Hợp gồm từ riêng ở mỗi bên lẫn các từ chung giữa hai bên.

Q11. How does Set Jaccard treat token frequency? | Jaccard Set xử lý tần suất token ra sao?

A. Common tokens | Các token chung

B. All distinct tokens from either set | Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences | Bỏ qua số lần lặp

D. 1.0 | 1.0

Đáp án: C

Giải thích: Chuyển sang Set mất số lần và thứ tự từ. Vì vậy Jaccard trên Set không phân biệt từ xuất hiện một lần với mười lần.

Q12. What happens for identical nonempty token sets? | Hai tập token giống hệt và không rỗng cho Jaccard gì?

A. Common tokens | Các token chung

B. All distinct tokens from either set | Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences | Bỏ qua số lần lặp

D. 1.0 | 1.0

Đáp án: D

Giải thích: Giao bằng hợp nên tỷ số bằng một. Kết quả này áp dụng khi hợp không rỗng, đúng điều kiện đã nêu trong câu hỏi.

Q13. What happens for disjoint nonempty token sets? | Hai tập token rời không rỗng cho Jaccard gì?

A. 0.0 | 0.0

B. The formula has zero denominator; choose a convention | Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care | Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains | Không, dấu câu vẫn còn

Đáp án: A

Giải thích: Không có token chung nên tử số bằng không. Tử số bằng không nhưng mẫu số vẫn dương.

Q14. What happens when both sets are empty? | Nếu hai Set đều rỗng thì Jaccard sao?

A. 0.0 | 0.0

B. The formula has zero denominator; choose a convention | Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care | Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains | Không, dấu câu vẫn còn

Đáp án: B

Giải thích: Phép chia 0/0 không xác định; ứng dụng phải tự quy định. Ứng dụng có thể quy ước hai văn bản rỗng giống nhau 1, nhưng hệ khác có thể chọn 0 hoặc không xác định.

Q15. Does whitespace splitting correctly segment all Vietnamese words? | split theo khoảng trắng có tách đúng mọi từ tiếng Việt không?

A. 0.0 | 0.0

B. The formula has zero denominator; choose a convention | Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care | Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains | Không, dấu câu vẫn còn

Đáp án: C

Giải thích: Các tiếng ngăn bởi khoảng trắng chưa chắc là từ độc lập. Một từ tiếng Việt có thể gồm nhiều tiếng được ngăn bằng khoảng trắng.

Q16. Can lowercasing alone remove punctuation? | Chỉ lower() có xóa dấu câu không?

A. 0.0 | 0.0

B. The formula has zero denominator; choose a convention | Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care | Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains | Không, dấu câu vẫn còn

Đáp án: D

Giải thích: Đổi chữ thường và xử lý dấu câu là hai bước khác nhau. Dấu câu có thể còn dính vào token cho đến khi có bước làm sạch/tách riêng.

Q17. Does Jaccard understand semantic synonyms? | Jaccard theo Set có hiểu từ đồng nghĩa không?

A. No, it compares literal token identities | Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document | Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies | Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation | Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: A

Giải thích: Hai từ khác chữ dù đồng nghĩa vẫn có thể không chung token. Jaccard đo trùng mặt chữ; muốn so ý nghĩa cần biểu diễn ngôn ngữ phong phú hơn.

Q18. What does len(A|B) represent? | len(A|B) biểu thị gì?

A. No, it compares literal token identities | Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document | Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies | Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation | Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: B

Giải thích: Hợp là mẫu số của Jaccard trên Set. Khi hợp khác rỗng, đây là mẫu số; chỉ dùng len(A) có thể khiến điểm sai.

Q19. What is lost when building a vocabulary as a Set? | Tạo vocabulary bằng Set mất thông tin nào?

A. No, it compares literal token identities | Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document | Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies | Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation | Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: C

Giải thích: Set chỉ lưu sự có mặt, không lưu vị trí hay lượt xuất hiện. Khi cần đếm số lần hoặc giữ thứ tự, hãy dùng Counter hoặc List token.

Q20. When should a specialized tokenizer be considered? | Khi nào nên xem bộ tách từ chuyên biệt?

A. No, it compares literal token identities | Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document | Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies | Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation | Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: D

Giải thích: Tokenizer theo ngôn ngữ xử lý ranh giới vượt quá khoảng trắng. Chọn bộ tách phù hợp ngôn ngữ và dùng thống nhất lúc lập chỉ mục lẫn lúc truy vấn.

8. Liên hệ bài kế tiếp

`String → Set → Tokenization → Vector hóa văn bản.`

PY-43 — Git: quản lý phiên bản và hợp tác trên mã nguồn

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Repository — kho mã nguồn; **Commit** — bản ghi thay đổi.

`File/Project → git init/add/commit → branch → pull/push → Review`

Học xong dùng để làm gì? Lưu lịch sử thay đổi code bằng commit, tạo nhánh thử nghiệm và khôi phục phiên bản chạy tốt.

Kết nối kiến thức tiếp theo: PY-44 đóng gói chức năng thao tác dữ liệu thành CLI có kiểm tra đầu vào.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Repository	kho mã nguồn	Nơi lưu lịch sử phiên bản
Commit	bản ghi thay đổi	Điểm lưu thay đổi có thông điệp
Branch	nhánh	Hướng phát triển độc lập
Remote	kho từ xa	Đường dẫn đến repository cộng tác

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`git status`	Xem file được sửa và đã stage hay chưa
`git add <file> / git commit -m`	Chọn thay đổi và tạo lịch sử có thông điệp
`git switch -c <branch>`	Tách nhánh thử nghiệm để không sửa nhánh chính
`git pull --ff-only / git push`	Đồng bộ an toàn khi lịch sử cho phép, kiểm tra remote trước

4. Khái niệm chính và liên quan

Mạch kiến thức: Git theo dõi lịch sử của một repository. Ta chọn thay đổi để stage, ghi chúng vào commit, phát triển độc lập trên branch và dùng remote để đồng bộ với người khác. Bốn khái niệm này tạo thành một quy trình liền mạch.

4.1. Repository — Nơi lưu mã nguồn và lịch sử

Từ vựng trong mục: Repository.

Repository là thư mục dự án được Git theo dõi; bên trong `.git` lưu thông tin commit, nhánh và tham chiếu. Ví dụ bài tập có `lesson.py`. Sau khi chạy `git init`, Git chưa tự đưa mọi file vào commit; `git status` chỉ cho biết trạng thái so với commit gần nhất và khu vực stage. Lệnh dưới đây dành cho terminal trong thư mục dự án đã khởi tạo Git.

No Note — Không chú thích

```
git status
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nơi lưu mã nguồn và lịch sử.
git status
```

Kết quả có thể quan sát: Trạng thái hiển thị phụ thuộc file đang sửa và các commit hiện có; xem phần Changes not staged, Staged, Untracked.

4.2. Commit — Bản chụp các thay đổi đã stage

Từ vựng trong mục: Commit.

Commit ghi lại một trạng thái của những file đã **stage** vào lịch sử cùng tác giả, thời gian và thông điệp. Quy trình `git add lesson.py` đưa bản đang chọn vào staging area, sau đó `git commit -m ...` tạo commit. Nếu sửa file thêm **sau khi stage**, phần sửa mới chưa tự nằm trong commit. `git log -1 --oneline` xem commit gần nhất; mã hash phụ thuộc repository.

No Note — Không chú thích

```
git log -1 --oneline
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Bản chụp các thay đổi đã stage.
git log -1 --oneline
```

Kết quả có thể quan sát: Lệnh in mã commit rút gọn và thông điệp của bản gần nhất nếu dự án có commit; không có một mã cố định.

4.3. Branch — Nhánh để phát triển song song

Từ vựng trong mục: Branch.

Branch là một **tham chiếu đến commit**, giúp làm tính năng riêng mà không làm thay đổi ngay `main`. `git switch -c feature/score` tạo nhánh mới và chuyển sang nó từ commit hiện tại; code mới chỉ có trên nhánh đó cho tới khi được merge. Để tránh nhầm, kiểm tra `git branch --show-current` trước khi làm; tên nhánh này là ví dụ, không phải chính sách của mọi nhóm.

No Note — Không chú thích

```
git switch -c feature/score
git branch --show-current
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Nhánh để phát triển song song.
git switch -c feature/score
git branch --show-current
```

Kết quả có thể quan sát: Sau khi tạo thành công, lệnh sau in `feature/score`; lần chạy tiếp sẽ báo nhánh đã tồn tại.

4.4. Remote — Liên kết đến kho Git ở nơi khác

Từ vựng trong mục: Remote.

Remote là một tên tham chiếu đến địa chỉ repository khác, thường tên `origin` dùng cho kho trên GitHub. `git remote -v` liệt kê địa chỉ fetch/push đang cấu hình; nó không tự chứng minh đã đăng nhập hay có quyền push. `git fetch` tải tham chiếu từ remote, `git pull --ff-only` chỉ nhận thay đổi khi có thể đi thẳng không tạo merge commit, còn `git push` gửi commit nếu có quyền.

No Note — Không chú thích

```
git remote -v
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Liên kết đến kho Git ở nơi khác.
git remote -v
```

Kết quả có thể quan sát: Lệnh hiển thị tên và URL remote đã cấu hình; repository không có remote sẽ không in dòng nào.

5. Năm tình huống Git thực thi an toàn — từ 10 file đến merge 20 file

Tình huống 1 — Cấp 1: Kiểm trạng thái 10 script trước khi đưa vào Git

Từ vựng trong mục: Repository — kho phiên bản; **Untracked** — chưa được theo dõi; **Status** — trạng thái.

Tình huống thực tế: Một học viên vừa nhận **10 script Python**, từ `lesson01.py` đến `lesson10.py`, để bắt đầu quản lý phiên bản. Học viên muốn tạo một **repository Git thử nghiệm riêng** rồi xem cả mười file đã được Git nhận diện hay chưa, nhưng **chưa muốn đưa chúng vào lịch sử**. Mọi thao tác phải nằm trong thư mục tạm để không ảnh hưởng repository học tập đang sử dụng.

Vì sao chọn cách này? Repository là vùng dự án mà Git ghi nhận thay đổi; `git init` chỉ tạo thư mục quản lý **`.git`**, chưa tự thêm nội dung script vào lịch sử. `git status --porcelain` cho thông tin trạng thái ngắn, dễ xử lý bằng code; ký hiệu `??` nghĩa là file **chưa được Git theo dõi (untracked)**. `git diff --cached --name-only` xem phần **đã stage** để chuẩn bị commit; ở bước này danh sách đó còn rỗng. `TemporaryDirectory` cách ly tất cả thao tác trong **thư mục thử rồi tự dọn**, nên ví dụ không thay đổi repository thật của học viên.

Quy trình:

1. Tạo repository tạm và 10 file script có nội dung độc lập.
2. Chạy status, đếm dòng `??` và số file Git đã stage.
3. Báo cả hai số để phân biệt untracked và staged.

No Note — Không chú thích

```
import subprocess
import tempfile
from pathlib import Path

def git(root, *args):
    return subprocess.run(["git", *args], cwd=root, check=True,
                          text=True, capture_output=True).stdout.strip()

with tempfile.TemporaryDirectory() as temp:
    root = Path(temp)
    git(root, "init", "-q", "-b", "main")
    for i in range(1, 11):
        (root / f"lesson{i:02d}.py").write_text(f"print({i})\n", encoding="utf-8")
    lines = git(root, "status", "--porcelain").splitlines()
    untracked = 0
    for line in lines:
        if line.startswith("?? "):
            untracked += 1
    staged = git(root, "diff", "--cached", "--name-only").splitlines()
    print("Script:", 10)
    print("Untracked:", untracked)
    print("Đã stage:", len(staged))
    print("Đúng repository tạm:", (root / ".git").exists())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Khởi tạo một repository Git tạm với 10 script và phân biệt file untracked với staged.
# Dữ liệu: 10 file lesson01.py..lesson10.py, mọi thao tác chỉ xảy ra trong thư mục tạm tự xóa sau
khi chạy.
# subprocess cho phép chạy lệnh Git CLI từ Python, thay vì mô phỏng phản hồi Git.
import subprocess
# tempfile cấp thư mục tạm để ví dụ không đụng repository thật của người học.
import tempfile
# Path đại diện đường dẫn theo hệ điều hành và hỗ trợ thao tác file với toán tử /.
from pathlib import Path

# Định nghĩa hàm trợ giúp chạy Git; *args thu các phần đối số khác nhau thành tuple, root chỉ định
thư mục repository tạm.
def git(root, *args):
    # subprocess.run() chạy chương trình git theo list đối số; không dùng shell=True nên tránh diễn
giải chuỗi bởi shell.
    # cwd=root đặt thư mục làm việc của Git đúng repository tạm, không vô tình thao tác trong dự án
thật.
    # check=True phát sinh CalledProcessError nếu lệnh Git trả mã lỗi, tránh tiếp tục với kết quả
thất bại.
    return subprocess.run(["git", *args], cwd=root, check=True,
                          # text=True giải mã stdout thành chuỗi văn bản, không phải bytes.
                          # capture_output=True thu stdout/stderr để xử lý trong Python thay vì in
lấn lên terminal.
                          # stdout là đầu ra chuẩn của lệnh Git; strip() bỏ khoảng trắng/xuống dòng
hai đầu cho dễ so sánh.
                          text=True, capture_output=True).stdout.strip()

# TemporaryDirectory() tạo thư mục tạm và tự dọn toàn bộ repository khi ra khỏi khối with, kể cả
khi có lỗi.
```

```
with tempfile.TemporaryDirectory() as temp:
    # Đổi chuỗi đường dẫn thư mục tạm thành Path để ghép tên file.
    root = Path(temp)
    # git init -b main tạo repository mới với nhánh chính main; -q giảm thông báo, chỉ trong root
tạm.
    git(root, "init", "-q", "-b", "main")
    # range(1,11) tạo mười số 1..10; không bao gồm 11.
    for i in range(1, 11):
        # Toán tử / của Path ghép đường dẫn; :02d tạo tên lesson01..lesson10; write_text() tạo file
UTF-8 có một lệnh print tương ứng.
        (root / f"lesson{i:02d}.py").write_text(f"print({i})\n", encoding="utf-8")
    # status --porcelain cho trạng thái máy đọc được, tiền tố '?? ' nghĩa Git CHƯA theo dõi file.
    # splitlines() tách chuỗi stdout thành list từng dòng, không giữ ký tự xuống dòng.
    lines = git(root, "status", "--porcelain").splitlines()
    untracked = 0
    for line in lines:
        # startswith('?? ') chỉ đếm những file chưa được Git theo dõi, không đếm file đã stage.
        if line.startswith("?? "):
            # Tăng số file chưa theo dõi khi thấy một dòng trạng thái khớp.
            untracked += 1
    # splitlines() tách chuỗi stdout thành list từng dòng, không giữ ký tự xuống dòng.
    # diff --cached --name-only liệt kê tên file trong vùng STAGING (index) khác HEAD; trước git
add danh sách rỗng.
    staged = git(root, "diff", "--cached", "--name-only").splitlines()
    print("Script:", 10)
    print("Untracked:", untracked)
    # Đếm số file đã stage; bằng 0 khi chưa chạy git add.
    print("Đã stage:", len(staged))
    # exists() xác minh thư mục metadata .git được tạo bên trong root tạm, không động tới repo bài
học.
    print("Đúng repository tạm:", (root / ".git").exists())
```

Lệnh Git tương ứng: Chỉ thực hành trong repository thử, không chạy vào repository đang làm việc.

No Note — Không chú thích

```
git status --short
```

Code có chú thích — Giải thích từng bước

```
git status --short
```

Kết quả khi chạy:

```
Script: 10
Untracked: 10
Đã stage: 0
Đúng repository tạm: True
```

Thư mục tạm sẽ tự bị xóa sau ví dụ; script chỉ nghiên cứu Git **cục bộ**, không có lệnh đẩy lên GitHub.

Tình huống 2 — Cấp 2: Chỉ stage năm trong 12 file của bản cập nhật

Từ vựng trong mục: Staging area — vùng chuẩn bị; **git add** — chọn file; **Diff cached** — khác biệt đã stage.

Tình huống thực tế: Nhóm phát triển chuẩn bị **12 file module Python**, từ `module01.py` đến `module12.py`. Trong đợt review này, chỉ **5 file đầu** đã được kiểm thử và được phép đưa vào lần commit tiếp theo; **7 file**

còn lại vẫn đang viết. Trưởng nhóm muốn kiểm rõ năm file đã được **stage** và bảy file chưa được chọn, tránh đưa nhầm code chưa hoàn thành vào lịch sử.

Vì sao chọn cách này? Staging area là danh sách thay đổi **được chọn để đưa vào commit tiếp theo**, không phải mọi file đang nằm trong thư mục. `git add -- module01.py ... module05.py` chọn rõ **năm file đã duyệt**, khác `git add .` có thể đưa cả 12 file vào stage. `git diff --cached --name-only` cho năm file đã chọn; `git ls-files --others --exclude-standard` cho bảy file còn **untracked**. Đếm hai nhóm là cách kiểm nhanh trước khi commit: **stage chưa phải commit**, nên vẫn còn cơ hội sửa lựa chọn nếu thấy file sai.

Quy trình:

1. Tạo repo tạm và 12 file có dữ liệu riêng.
2. Stage đúng năm tên, không dùng ký tự chọn toàn bộ.
3. Đếm số staged/untracked và kiểm module06 chưa stage.

No Note — Không chú thích

```
import subprocess
import tempfile
from pathlib import Path

def git(root, *args):
    return subprocess.run(["git", *args], cwd=root, check=True,
                          text=True, capture_output=True).stdout.strip()

with tempfile.TemporaryDirectory() as temp:
    root = Path(temp)
    git(root, "init", "-q", "-b", "main")
    for i in range(1, 13):
        (root / f"module{i:02d}.py").write_text(f"VALUE = {i}\n", encoding="utf-8")
    chosen = []
    for i in range(1, 6):
        chosen.append(f"module{i:02d}.py")
    git(root, "add", "--", *chosen)
    staged = git(root, "diff", "--cached", "--name-only").splitlines()
    others = git(root, "ls-files", "--others", "--exclude-standard").splitlines()
    print("File:", 12)
    print("Đã stage:", len(staged))
    print("Chưa stage:", len(others))
    print("module06 chưa stage:", "module06.py" in others)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chọn stage năm file trong 12 module, giữ bảy file khác chưa theo dõi.
# Dữ liệu: 12 module01.py..module12.py trong repository tạm; chỉ module01..05 được thêm vào index.
# subprocess cho phép chạy lệnh Git CLI từ Python, thay vì mô phỏng phản hồi Git.
import subprocess
# tempfile cấp thư mục tạm để ví dụ không đụng repository thật của người học.
import tempfile
# Path đại diện đường dẫn theo hệ điều hành và hỗ trợ thao tác file với toán tử /.
from pathlib import Path

# Định nghĩa hàm trợ giúp chạy Git; *args thu các phần đối số khác nhau thành tuple, root chỉ định
thư mục repository tạm.
def git(root, *args):
    # subprocess.run() chạy chương trình git theo list đối số; không dùng shell=True nên tránh diễn
giải chuỗi bởi shell.
    # cwd=root đặt thư mục làm việc của Git đúng repository tạm, không vô tình thao tác trong dự án
thật.
```

```
    # check=True phát sinh CalledProcessError nêu lệnh Git trả mã lôĩ, tránh tiếp tục với kết quả
thất bại.
    return subprocess.run(["git", *args], cwd=root, check=True,
                          # text=True giải mã stdout thành chuôĩ văn bản, không phải bytes.
                          # capture_output=True thu stdout/stderr đểˇxưˇlý trong Python thay vì in
lấn lên terminal.
                          # stdout là đâu ra chuẩn của lệnh Git; strip() bỏ khoảng trăńg/xuống dòng
hai đâu cho dêˆso sánh.
                          text=True, capture_output=True).stdout.strip()

# TemporaryDirectory() tạo thư mục tạm và tự dọn toàn bộ repository khi ra khỏi khôí with, kểˇcả
khi có lôĩ.
with tempfile.TemporaryDirectory() as temp:
    # Đôỉ chuôĩ đường dâñ thư mục tạm thành Path đểˇghép tên file.
    root = Path(temp)
    # git init -b main tạo repository mới với nhánh chính main; -q giảm thông báo, chỉ trong root
tạm.
    git(root, "init", "-q", "-b", "main")
    # Sinh sôˊ1..12 đểˇtạo đủ 12 script, không chứa 13.
    for i in range(1, 13):
        # Path ghép root/tên file, :02d thêm sôˊ0; write_text() ghi UTF-8 và tạo file nêú chưa có.
        (root / f"module{i:02d}.py").write_text(f"VALUE = {i}\n", encoding="utf-8")
    # List tên năm file dự định stage; không stage toàn bộ thư mục.
    chosen = []
    # Tạo sôˊ1..5, đúng năm module được chọn.
    for i in range(1, 6):
        # append() đưa tên file module01..05 vào list.
        chosen.append(f"module{i:02d}.py")
    # git add chuyển file sang staging; '--' kết thúc tùy chọn đểˇtên file lạ không bị hiểủ là cờ;
*chosen giải nén năm tên thành năm đôí sôˊ.
    git(root, "add", "--", *chosen)
    # Liệt kê tên các file đã stage trong index, chứ chưa commit.
    staged = git(root, "diff", "--cached", "--name-only").splitlines()
    # ls-files --others liệt kê file chưa theo dõi; --exclude-standard loại file bị ignore
theo .gitignore/exclude.
    others = git(root, "ls-files", "--others", "--exclude-standard").splitlines()
    print("File:", 12)
    # Sôˊfile trong index phải là năm, khác len(others) là bảy chưa stage.
    print("Đã stage:", len(staged))
    print("Chưa stage:", len(others))
    # in kiểm module06 nằm trong các file chưa theo dõi, trả Boolean.
    print("module06 chưa stage:", "module06.py" in others)
```

Lệnh Git tương ứng: Chỉ thực hành trong repository thử, không chạy vào repository đang làm việc.

No Note — Không chú thích

```
git add -- module01.py module02.py module03.py module04.py module05.py
```

Code có chú thích — Giải thích từng bước

```
git add -- module01.py module02.py module03.py module04.py module05.py
```

Kết quả khi chạy:

```
File: 12
Đã stage: 5
Chưa stage: 7
module06 chưa stage: True
```

Stage là **lựa chọn nội dung cho commit tiếp theo**, không phải đã commit. Khi sửa file sau stage, cần stage lại phần sửa muốn ghi.

Tình huống 3 — Cấp 3: Tách nhánh thay đổi ba trong 15 script

Từ vựng trong mục: Branch — nhánh; **Commit** — bản ghi lịch sử; **Switch** — chuyển nhánh.

Tình huống thực tế: Một nhóm có **15 script** đã được ghi trong nhánh `main` với nội dung `VERSION = 1`. Một người muốn thử phiên bản mới cho **ba script đầu**, nhưng đồng đội vẫn cần dùng đúng phiên bản cũ trên `main`. Người đó sẽ tạo nhánh `feature`, sửa và commit riêng ba file rồi chuyển qua lại giữa hai nhánh để kiểm chứng sự khác biệt.

Vì sao chọn cách này? Branch (nhánh) là tên trỏ tới một chuỗi lịch sử commit, giúp thử thay đổi mà chưa đưa vào nhánh chung. `git switch -c feature` tạo và chuyển sang nhánh mới từ bản `main`; sửa ba script rồi `git add` và `git commit` lưu bản thay đổi **chỉ trên feature**. `git diff --name-only main feature` cho biết những file khác giữa hai đầu nhánh. Khi `switch` về `main`, `lesson01.py` vẫn là `VERSION = 1`; chuyển sang `feature` thì là `VERSION = 2`. Ví dụ này không cần remote, nên không phải kéo code hoặc tạo pull request.

Quy trình:

1. Tạo 15 script v1 và commit ban đầu với thông tin tác giả test.
2. Tạo nhánh `feature`, sửa ba script thành v2, commit.
3. Chuyển giữa hai nhánh, kiểm tra nội dung khác nhau và số file diff.

No Note — Không chú thích

```
import subprocess
import tempfile
from pathlib import Path

def git(root, *args):
    return subprocess.run(["git", *args], cwd=root, check=True,
                          text=True, capture_output=True).stdout.strip()

with tempfile.TemporaryDirectory() as temp:
    root = Path(temp)
    git(root, "init", "-q", "-b", "main")
    git(root, "config", "user.name", "Lesson Tester")
    git(root, "config", "user.email", "lesson@example.invalid")
    for i in range(1,16):
        (root / f"lesson{i:02d}.py").write_text("VERSION = 1\n", encoding="utf-8")
    git(root, "add", ".")
    git(root, "commit", "-q", "-m", "base")
    git(root, "switch", "-q", "-c", "feature")
    for i in range(1,4):
        (root / f"lesson{i:02d}.py").write_text("VERSION = 2\n", encoding="utf-8")
    git(root, "add", ".")
    git(root, "commit", "-q", "-m", "update three scripts")
    difference = git(root, "diff", "--name-only", "main", "feature").splitlines()
    git(root, "switch", "-q", "main")
    main_version = (root / "lesson01.py").read_text().strip()
    git(root, "switch", "-q", "feature")
    feature_version = (root / "lesson01.py").read_text().strip()
    print("File ban đầu:", 15)
    print("File thay đổi:", len(difference))
    print("Main:", main_version)
    print("Feature:", feature_version)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo nhánh feature, commit ba file thay đổi và chứng minh main chưa nhận sửa đổi.
# Dữ liệu: 15 script có VERSION=1 ban đầu; feature đổi ba script đầu thành VERSION=2, mỗi nhánh có
lịch sử riêng.
# subprocess cho phép chạy lệnh Git CLI từ Python, thay vì mô phỏng phản hồi Git.
import subprocess
# tempfile cấp thư mục tạm để ví dụ không đụng repository thật của người học.
import tempfile
# Path đại diện đường dẫn theo hệ điều hành và hỗ trợ thao tác file với toán tử /.
from pathlib import Path

# Định nghĩa hàm trợ giúp chạy Git; *args thu các phần đối số khác nhau thành tuple, root chỉ định
thư mục repository tạm.
def git(root, *args):
    # subprocess.run() chạy chương trình git theo list đối số; không dùng shell=True nên tránh diễn
giải chuỗi bởi shell.
    # cwd=root đặt thư mục làm việc của Git đúng repository tạm, không vô tình thao tác trong dự án
thật.
    # check=True phát sinh CalledProcessError nếu lệnh Git trả mã lỗi, tránh tiếp tục với kết quả
thất bại.
    return subprocess.run(["git", *args], cwd=root, check=True,
                          # text=True giải mã stdout thành chuỗi văn bản, không phải bytes.
                          # capture_output=True thu stdout/stderr để xử lý trong Python thay vì in
lấn lên terminal.
                          # stdout là đầu ra chuẩn của lệnh Git; strip() bỏ khoảng trắng/xuống dòng
hai đầu cho dễ so sánh.
                          text=True, capture_output=True).stdout.strip()

# TemporaryDirectory() tạo thư mục tạm và tự dọn toàn bộ repository khi ra khỏi khối with, kể cả
khi có lỗi.
with tempfile.TemporaryDirectory() as temp:
    # Đổi chuỗi đường dẫn thư mục tạm thành Path để ghép tên file.
    root = Path(temp)
    # git init -b main tạo repository mới với nhánh chính main; -q giảm thông báo, chỉ trong root
tạm.
    git(root, "init", "-q", "-b", "main")
    # git config cài tên tác giả commit CHỈ trong repository tạm, không đổi Git global.
    git(root, "config", "user.name", "Lesson Tester")
    # Đặt email minh họa .invalid cục bộ để Git cho phép tạo commit, không gửi email.
    git(root, "config", "user.email", "lesson@example.invalid")
    # Tạo 15 tên script với chỉ số 1..15.
    for i in range(1,16):
        # write_text() tạo nội dung bản gốc trên tất cả 15 file, encoding='utf-8' giữ mã Unicode.
        (root / f"lesson{i:02d}.py").write_text("VERSION = 1\n", encoding="utf-8")
    # git add . đưa file đang thay đổi trong root vào staging, trước khi commit.
    git(root, "add", ".")
    # commit tạo snapshot từ staging; -q chạy yên lặng, -m truyền thông điệp 'base'.
    git(root, "commit", "-q", "-m", "base")
    # switch -c feature tạo nhánh mới tại commit hiện tại và chuyển sang; -q giảm thông báo.
    git(root, "switch", "-q", "-c", "feature")
    # Chọn chỉ ba file đầu tiên để sửa trên nhánh feature.
    for i in range(1,4):
        # Ghi đè bản feature của ba file thành VERSION=2; không thay đổi ngay commit của main.
        (root / f"lesson{i:02d}.py").write_text("VERSION = 2\n", encoding="utf-8")
    # git add . đưa file đang thay đổi trong root vào staging, trước khi commit.
    git(root, "add", ".")
    # Ghi một commit sửa ba script trên feature.
    git(root, "commit", "-q", "-m", "update three scripts")
    # diff --name-only main feature liệt kê file có nội dung khác nhau giữa hai nhánh, không in
toàn bộ patch.
    difference = git(root, "diff", "--name-only", "main", "feature").splitlines()
    # Chuyển về `main: working tree phản ánh VERSION=1 từ commit main.
    git(root, "switch", "-q", "main")
    # read_text() đọc nội dung file hiện đang checkout; strip() bỏ dấu xuống dòng cuối, không chỉnh
```

```
file.
    main_version = (root / "lesson01.py").read_text().strip()
    # Chuyển lại feature: phiên bản working tree của lesson01 là VERSION=2.
    git(root, "switch", "-q", "feature")
    # read_text() đọc nội dung file hiện đang checkout; strip() bỏ dấú xuống dòng cuối, không chỉnh
file.
    feature_version = (root / "lesson01.py").read_text().strip()
    print("File ban đầu:", 15)
    # Sô´tên file khác nhau giữa hai nhánh, đúng ba.
    print("File thay đổi:", len(difference))
    print("Main:", main_version)
    print("Feature:", feature_version)
```

Lệnh Git tương ứng: Chỉ thực hành trong repository thử, không chạy vào repository đang làm việc.

No Note — Không chú thích

```
git switch -c feature
```

Code có chú thích — Giải thích từng bước

```
git switch -c feature
```

Kết quả khi chạy:

```
File ban đầu: 15
File thay đổi: 3
Main: VERSION = 1
Feature: VERSION = 2
```

Commit lưu ảnh chụp **file đã stage**. Quay về `main` không cần xóa thay đổi ở nhánh feature và cũng không tạo commit trên main.

Tình huống 4 — Cấp 4: Chia 18 file thành ba commit có thể kiểm tra

Từ vựng trong mục: Commit history — lịch sử; **Log** — nhật ký commit; **Tracked file** — file đang theo dõi.

Tình huống thực tế: Nhóm bảo trì nhận **18 script tool01.py–tool18.py** cần quản lý theo **ba giai đoạn**, mỗi giai đoạn gồm đúng 6 file. Quản lý muốn lịch sử Git có **ba commit có tên dễ hiểu**, mỗi commit đại diện một đợt làm việc, thay vì chỉ có một commit chứa toàn bộ 18 file. Sau cùng, cần xác nhận Git theo dõi đủ file và đọc lại tên các commit gần nhất.

Vì sao chọn cách này? Commit là mốc lưu lại một nhóm thay đổi đã được stage, kèm **thông điệp mô tả**. Chương trình chia 18 file thành ba đợt sáu file, dùng `git add` đúng tên file mỗi đợt rồi `git commit -m 'batch ...'`. `git rev-list --count HEAD` đếm ba commit đã tạo, `git ls-files` xác nhận 18 script đã được theo dõi. `git log -3 --format=%s` chỉ lấy **tiêu đề** ba commit gần nhất (mới trước, cũ sau), vì mã hash có thể đổi giữa các lần dựng thử nên không phù hợp để viết sẵn đáp án cố định. Chia lịch sử theo công việc giúp review hoặc quay lại từng giai đoạn rõ ràng.

Quy trình:

1. Tạo lần lượt ba nhóm, mỗi nhóm sáu file trong repository tạm.
2. Sau mỗi nhóm stage riêng các file và commit với thông điệp riêng.
3. Kiểm số file Git theo dõi, số commit và thứ tự ba thông điệp gần nhất.

No Note — Không chú thích

```
import subprocess
import tempfile
from pathlib import Path

def git(root, *args):
    return subprocess.run(["git", *args], cwd=root, check=True,
                          text=True, capture_output=True).stdout.strip()

with tempfile.TemporaryDirectory() as temp:
    root = Path(temp)
    git(root, "init", "-q", "-b", "main")
    git(root, "config", "user.name", "Lesson Tester")
    git(root, "config", "user.email", "lesson@example.invalid")
    for batch in range(1,4):
        filenames = []
        for i in range((batch-1)*6+1, batch*6+1):
            filename = f"tool{i:02d}.py"
            (root / filename).write_text(f"VALUE = {i}\n", encoding="utf-8")
            filenames.append(filename)
        git(root, "add", "--", *filenames)
        git(root, "commit", "-q", "-m", f"batch {batch}")
    tracked = git(root, "ls-files").splitlines()
    messages = git(root, "log", "-3", "--format=%s").splitlines()
    print("Script:", len(tracked))
    print("Commit:", git(root, "rev-list", "--count", "HEAD"))
    print("Lịch sử gần nhất:", ", ".join(messages))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo ba commit nhóm độc lập trên 18 file và đọc lịch sửGit gần nhất.
# Dữ liệu: Ba đợt, mỗi đợt tạo sáu file tool, tổng 18 file trên main của repository tạm.
# subprocess cho phép chạy lệnh Git CLI từ Python, thay vì mô phỏng phản hồi Git.
import subprocess
# tempfile cấp thư mục tạm để ví dụ không đụng repository thật của người học.
import tempfile
# Path đại diện đường dẫn theo hệ điều hành và hỗ trợ thao tác file với toán tử/.
from pathlib import Path

# Định nghĩa hàm trợ giúp chạy Git; *args thu các phần đối số khác nhau thành tuple, root chỉ định
thư mục repository tạm.
def git(root, *args):
    # subprocess.run() chạy chương trình git theo list đối số; không dùng shell=True nên tránh diễn
giải chuỗi bởi shell.
    # cwd=root đặt thư mục làm việc của Git đúng repository tạm, không vô tình thao tác trong dự án
thật.
    # check=True phát sinh CalledProcessError nếu lệnh Git trả mã lỗi, tránh tiếp tục với kết quả
thất bại.
    return subprocess.run(["git", *args], cwd=root, check=True,
                          # text=True giải mã stdout thành chuỗi văn bản, không phải bytes.
                          # capture_output=True thu stdout/stderr để xử lý trong Python thay vì in
lấn lên terminal.
                          # stdout là đầu ra chuẩn của lệnh Git; strip() bỏ khoảng trắng/xuống dòng
hai đầu cho dễ so sánh.
                          text=True, capture_output=True).stdout.strip()

# TemporaryDirectory() tạo thư mục tạm và tự dọn toàn bộ repository khi ra khỏi khối with, kể cả
khi có lỗi.
with tempfile.TemporaryDirectory() as temp:
    # Đổi chuỗi đường dẫn thư mục tạm thành Path để ghép tên file.
    root = Path(temp)
    # git init -b main tạo repository mới với nhánh chính main; -q giảm thông báo, chỉ trong root
tạm.
```

```
    git(root, "init", "-q", "-b", "main")
    git(root, "config", "user.name", "Lesson Tester")
    git(root, "config", "user.email", "lesson@example.invalid")
    # Ba vòng batch=1,2,3, đúng ba commit riêng biệt.
    for batch in range(1,4):
        # List riêng mỗĩ batch chứa sáu tên file sẽ stage.
        filenames = []
        # Tính dải chỉ sô´sáu file liên tiếp cho từng batch: 1..6, 7..12, 13..18.
        for i in range((batch-1)*6+1, batch*6+1):
            # f-string :02d đặt tên tool01.py..tool18.py theo thứ tự.
            filename = f"tool{i:02d}.py"
            # Ghi một script mới theo sô´i trong thư mục tạm, UTF-8.
            (root / filename).write_text(f"VALUE = {i}\n", encoding="utf-8")
            # Ghi tên file vừa tạo vào list của đúng batch hiện tại.
            filenames.append(filename)
        # Stage CHỈ sáu file của batch hiện tại; '--' ngăn hiểủ tên thành cờ, * giải nén list.
        git(root, "add", "--", *filenames)
        # Tạo một commit snapshot per batch, -q ẩn thông báo, -m găń tên batch.
        git(root, "commit", "-q", "-m", f"batch {batch}")
    # ls-files liệt kê tất cả file đang được Git theo dõi trong index, gồm̀ 18 file sau ba batch.
    tracked = git(root, "ls-files").splitlines()
    # log -3 lấý tôí đa ba commit gần nhất; --format=%s chỉ in CHỦ ĐỀ`commit, không in hash/body.
    messages = git(root, "log", "-3", "--format=%s").splitlines()
    print("Script:", len(tracked))
    # rev-list --count HEAD đếm sô´commit có thể̂đi tới từ HEAD, ở đây ba commit.
    print("Commit:", git(root, "rev-list", "--count", "HEAD"))
    # join() ghép các tiêu đề`commit theo thứ tự từ mới tới cũ.
    print("Lịch sử̛gần nhất:", ", ".join(messages))
```

Lệnh Git tương ứng: Chỉ thực hành trong repository thử, không chạy vào repository đang làm việc.

No Note — Không chú thích

```
git log -3 --format=%s
```

Code có chú thích — Giải thích từng bước

```
git log -3 --format=%s
```

Kết quả khi chạy:

```
Script: 18
Commit: 3
Lịch sử̛gần nhất: batch 3, batch 2, batch 1
```

Chỉ có ba commit vì mỗi commit chứa **sáu file**. Số file và số commit trả lời hai câu hỏi khác nhau.

Tình huống 5 — Cấp 5: Merge nhánh an toàn khi hai người sửa khác file trong 20 script

Từ vựng trong mục: Merge — hợp nhất lịch sử; **Merge commit** — commit có hai cha; **Clean working tree** — cây làm việc sạch.

Tình huống thực tế: Hai lập trình viên cùng làm trên **dự án thử nghiệm 20 file Python**. Từ phiên bản chung, người thứ nhất sửa **tool01 và tool02** trên nhánh `feature`; người thứ hai sửa **tool19 và tool20** trên `main`. Vì họ sửa **những file khác nhau**, nhóm muốn hợp nhất để cả bốn sửa đổi còn nguyên, thấy rõ **merge commit có hai nhánh cha**, và kiểm tra trạng thái làm việc sạch. Không kết nối remote hay đẩy code ra Internet.

Vì sao chọn cách này? Merge là đưa thay đổi từ hai lịch sử vào cùng một nhánh. Ví dụ tạo commit gốc 20 file, tách `feature` để sửa hai file đầu, rồi quay về `main` sửa hai file cuối, nên hai nhánh đã có **hai commit khác nhau**. `git merge --no-ff feature` yêu cầu Git tạo **merge commit riêng**, kể cả khi có thể gộp nhanh, để nhìn rõ dấu mốc hợp nhất. `git rev-list --parents -n 1 HEAD` cho thấy merge commit và **hai parent**; nội dung tool01/tool20 chứng minh hai phía đều được giữ, còn `status` trống nghĩa là thư mục làm việc sạch. Vì các file sửa không giao nhau, Git có thể tự gộp; nếu cùng sửa một dòng thì cần giải quyết **xung đột (conflict)**, không tự ghi đè theo cảm tính.

Quy trình:

1. Tạo 20 file và commit bản gốc.
2. Từ cùng commit gốc, tạo nhánh feature rồi sửa/commit hai file đầu; quay main sửa/commit hai file cuối.
3. Merge không fast-forward, xác minh nội dung bốn file, 20 file tracked và hai parent.

No Note — Không chú thích

```
import subprocess
import tempfile
from pathlib import Path

def git(root, *args):
    return subprocess.run(["git", *args], cwd=root, check=True,
                          text=True, capture_output=True).stdout.strip()

with tempfile.TemporaryDirectory() as temp:
    root = Path(temp)
    git(root, "init", "-q", "-b", "main")
    git(root, "config", "user.name", "Lesson Tester")
    git(root, "config", "user.email", "lesson@example.invalid")
    for i in range(1,21):
        (root / f"tool{i:02d}.py").write_text("VERSION = 1\n", encoding="utf-8")
    git(root, "add", ".")
    git(root, "commit", "-q", "-m", "base")
    git(root, "switch", "-q", "-c", "feature")
    for i in (1,2):
        (root / f"tool{i:02d}.py").write_text("VERSION = 2\n", encoding="utf-8")
    git(root, "add", ".")
    git(root, "commit", "-q", "-m", "feature edits")
    git(root, "switch", "-q", "main")
    for i in (19,20):
        (root / f"tool{i:02d}.py").write_text("VERSION = 3\n", encoding="utf-8")
    git(root, "add", ".")
    git(root, "commit", "-q", "-m", "main edits")
    git(root, "merge", "-q", "--no-ff", "feature", "-m", "merge feature")
    parents = git(root, "rev-list", "--parents", "-n", "1", "HEAD").split()
    print("File:", len(git(root, "ls-files").splitlines()))
    print("Commit trong lịch sử:", git(root, "rev-list", "--count", "HEAD"))
    print("Merge có hai cha:", len(parents) == 3)
    print("Sửa feature giữ lại:", (root / "tool01.py").read_text().strip() == "VERSION = 2")
    print("Sửa main giữ lại:", (root / "tool20.py").read_text().strip() == "VERSION = 3")
    print("Working tree sạch:", git(root, "status", "--porcelain") == "")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Merge nhánh feature chứa hai sửa đổi vào main cũng có hai sửa đổi KHÁC file, rồi kiểm
merge sạch.
# Dữ liệu: 20 file tool01..20; feature sửa 01/02 lên VERSION=2; main sửa 19/20 lên VERSION=3; hai
phía không sửa chung file.
# subprocess cho phép chạy lệnh Git CLI từ Python, thay vì mô phỏng phản hồi Git.
```

```
import subprocess
# tempfile cấp thư mục tạm để ví dụ không đụng repository thật của người học.
import tempfile
# Path đại diện đường dẫn theo hệ điều hành và hỗ trợ thao tác file với toán tử /.
from pathlib import Path

# Định nghĩa hàm trợ giúp chạy Git; *args thu các phần đối số khác nhau thành tuple, root chỉ định
thư mục repository tạm.
def git(root, *args):
    # subprocess.run() chạy chương trình git theo list đối số; không dùng shell=True nên tránh diễn
giải chuỗi bởi shell.
    # cwd=root đặt thư mục làm việc của Git đúng repository tạm, không vô tình thao tác trong dự án
thật.
    # check=True phát sinh CalledProcessError nếu lệnh Git trả mã lỗi, tránh tiếp tục với kết quả
thất bại.
    return subprocess.run(["git", *args], cwd=root, check=True,
                          # text=True giải mã stdout thành chuỗi văn bản, không phải bytes.
                          # capture_output=True thu stdout/stderr để xử lý trong Python thay vì in
lấn lên terminal.
                          # stdout là đầu ra chuẩn của lệnh Git; strip() bỏ khoảng trắng/xuống dòng
hai đầu cho dễ so sánh.
                          text=True, capture_output=True).stdout.strip()

# TemporaryDirectory() tạo thư mục tạm và tự dọn toàn bộ repository khi ra khỏi khối with, kể cả
khi có lỗi.
with tempfile.TemporaryDirectory() as temp:
    # Đổi chuỗi đường dẫn thư mục tạm thành Path để ghép tên file.
    root = Path(temp)
    # git init -b main tạo repository mới với nhánh chính main; -q giảm thông báo, chỉ trong root
tạm.
    git(root, "init", "-q", "-b", "main")
    git(root, "config", "user.name", "Lesson Tester")
    git(root, "config", "user.email", "lesson@example.invalid")
    # Sinh chỉ số 1..20 để tạo 20 script nguồn.
    for i in range(1,21):
        # Mỗi file ban đầu là VERSION=1, chung cho cả hai nhánh.
        (root / f"tool{i:02d}.py").write_text("VERSION = 1\n", encoding="utf-8")
    git(root, "add", ".")
    # Commit cơ sở lưu trạng thái 20 file giống nhau trước khi tách nhánh.
    git(root, "commit", "-q", "-m", "base")
    # Tạo/chuyển nhánh feature từ commit base; -q ẩn thông báo.
    git(root, "switch", "-q", "-c", "feature")
    # Tuple (1,2) chọn đúng hai file sửa trong feature.
    for i in (1,2):
        # Ghi VERSION=2 trong hai file đầu của feature.
        (root / f"tool{i:02d}.py").write_text("VERSION = 2\n", encoding="utf-8")
    git(root, "add", ".")
    # Commit những thay đổi feature đã stage.
    git(root, "commit", "-q", "-m", "feature edits")
    # Chuyển sang main trước khi sửa cặp file khác.
    git(root, "switch", "-q", "main")
    # Chỉ sửa tool19 và tool20 trên main để tránh đụng hai file của feature.
    for i in (19,20):
        # Ghi VERSION=3 vào hai file cuối của nhánh main.
        (root / f"tool{i:02d}.py").write_text("VERSION = 3\n", encoding="utf-8")
    git(root, "add", ".")
    # Tạo commit main độc lập với commit feature.
    git(root, "commit", "-q", "-m", "main edits")
    # merge feature vào main; --no-ff ép tạo MERGE COMMIT kể cả khi có thể fast-forward; -m đặt
tiêu đề; -q ẩn log.
    git(root, "merge", "-q", "--no-ff", "feature", "-m", "merge feature")
    # rev-list --parents -n 1 HEAD in commit mới nhất và các hash cha; merge hai nhánh có hai cha.
    parents = git(root, "rev-list", "--parents", "-n", "1", "HEAD").split()
    # ls-files liệt kê 20 file đang được theo dõi sau merge.
    print("File:", len(git(root, "ls-files").splitlines()))
    # Đếm mọi commit trong lịch sử có thể tới từ HEAD, bao gồm base, hai commit nhánh và merge.
```

```
    print("Commit trong lịch sử:", git(root, "rev-list", "--count", "HEAD"))
    # List gồm hash merge và HAI hash cha nên tổng ba phần tử; True chứng minh đây là merge commit.
    print("Merge có hai cha:", len(parents) == 3)
    # Kiểm script feature còn VERSION=2 trong main sau merge, strip() chỉ loại ký tự trắng khi đọc.
    print("Sửa feature giữ lại:", (root / "tool01.py").read_text().strip() == "VERSION = 2")
    # Kiểm thay đổi main VERSION=3 cũng còn sau merge.
    print("Sửa main giữ lại:", (root / "tool20.py").read_text().strip() == "VERSION = 3")
    # Trạng thái porcelain rỗng nghĩa không có sửa đổi chưa commit/untracked, working tree sạch.
    print("Working tree sạch:", git(root, "status", "--porcelain") == "")
```

Lệnh Git tương ứng: Chỉ thực hành trong repository thử, không chạy vào repository đang làm việc.

No Note — Không chú thích

```
git merge --no-ff feature
```

Code có chú thích — Giải thích từng bước

```
git merge --no-ff feature
```

Kết quả khi chạy:

```
File: 20
Commit trong lịch sử: 4
Merge có hai cha: True
Sửa feature giữ lại: True
Sửa main giữ lại: True
Working tree sạch: True
```

Toàn bộ thí nghiệm chạy trong **thư mục tạm có repo Git riêng**; lệnh không thao tác lên repository dự án của người học. Khi merge xảy ra conflict, phải dừng xử lý dựa trên bằng chứng thay vì tự bỏ thay đổi.

6. Sai lầm thường gặp

- `git add` chỉ stage, không tạo commit; `git commit` chỉ ghi nội dung đã stage.
- `git diff` xem phần chưa stage, `git diff --cached` xem phần đã stage.
- Tạo nhánh khác không có nghĩa đã đẩy nhánh lên remote; push cần có quyền truy cập.
- Đường dẫn remote có thể tồn tại dù chưa có quyền push hoặc chưa xác thực.
- `git pull --ff-only` từ chối trường hợp phải merge/rebase; không nên `reset --hard` chỉ để bỏ thông báo lệch lịch sử.
- Đầu ra các lệnh Git phụ thuộc repository và trạng thái; không được coi hash ví dụ là kết quả của dự án khác.

7. Trắc nghiệm

Q1. What does a Git repository contain? | Repository Git chứa gì?

A. Project content and version history metadata | Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content | Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit | Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL | Tên tham chiếu đến URL repository khác

Đáp án: A

Giải thích: Git lưu trạng thái và tham chiếu trong `.git`, không chỉ gom file vào thư mục. Nó là phạm vi quản lý lịch sử dự án, không phải một commit hay nhánh riêng lẻ.

Q2. What is a commit? | Commit là gì?

A. Project content and version history metadata | Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content | Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit | Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL | Tên tham chiếu đến URL repository khác

Đáp án: B

Giải thích: Commit lưu nội dung đã stage cùng mã định danh và thông điệp. Chỉnh file mà chưa stage không tự làm thay đổi nội dung của commit kế.

Q3. What is a branch? | Nhánh Git là gì?

A. Project content and version history metadata | Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content | Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit | Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL | Tên tham chiếu đến URL repository khác

Đáp án: C

Giải thích: Tạo nhánh không sao chép vật lý toàn bộ file sang thư mục khác. Nhánh dịch chuyển khi có commit mới, còn commit đã tạo là một bản ghi bất biến.

Q4. What is a remote? | Remote là gì?

A. Project content and version history metadata | Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content | Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit | Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL | Tên tham chiếu đến URL repository khác

Đáp án: D

Giải thích: Remote chỉ định địa chỉ đồng bộ; quyền truy cập được kiểm riêng. Đổi URL origin không tự xác thực người dùng hay tạo commit ở remote.

Q5. What does `git status` show? | `git status` hiển thị gì?

A. Untracked, staged and working changes | File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path | Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message | Lưu phần đã stage thành commit có thông điệp

D. Compact commit history | Lịch sử commit rút gọn

Đáp án: A

Giải thích: Lệnh chỉ báo trạng thái, không tạo commit. Hãy xem status trước add/commit để phát hiện file chưa theo dõi hoặc chưa stage.

Q6. What does `git add file.py` do? | `git add file.py` làm gì?

A. Untracked, staged and working changes | File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path | Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message | Lưu phần đã stage thành commit có thông điệp

D. Compact commit history | Lịch sử commit rút gọn

Đáp án: B

Giải thích: Stage chuẩn bị cho commit, không phải tải lên remote. Sau `git add`, xem `git diff --cached` để chắc chỉ những phần mong muốn được đưa vào stage.

Q7. What does `git commit -m ...` do? | `git commit -m ...` làm gì?

A. Untracked, staged and working changes | File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path | Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message | Lưu phần đã stage thành commit có thông điệp

D. Compact commit history | Lịch sử commit rút gọn

Đáp án: C

Giải thích: Sau commit vẫn có thể còn sửa đổi chưa stage. Sửa đổi không nằm trong index sẽ không thuộc commit cho tới khi stage lại.

Q8. What does `git log --oneline` show? | `git log --oneline` hiển thị gì?

A. Untracked, staged and working changes | File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path | Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message | Lưu phần đã stage thành commit có thông điệp

D. Compact commit history | Lịch sử commit rút gọn

Đáp án: D

Giải thích: Hiển thị hash và tiêu đề, không phải diff chi tiết. Muốn xem thay đổi từng dòng phải dùng diff hoặc show, không chỉ đọc tiêu đề log.

Q9. What does `git switch -c name` do? | `git switch -c name` làm gì?

A. Creates and checks out a new branch | Tạo và chuyển sang nhánh mới

B. Name of current branch | Tên nhánh hiện tại

C. Unstaged differences for that path | Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit | Khác biệt đã stage so với commit gần nhất

Đáp án: A

Giải thích: Nhánh bắt đầu từ commit hiện tại nếu không chỉ định mốc khác. Nếu tên đã tồn tại thì tạo nhánh mới sẽ báo lỗi; khi đó cần chuyển nhánh thay vì tạo tiếp.

Q10. What does `git branch --show-current` return? | `git branch --show-current` trả gì?

A. Creates and checks out a new branch | Tạo và chuyển sang nhánh mới

B. Name of current branch | Tên nhánh hiện tại

C. Unstaged differences for that path | Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit | Khác biệt đã stage so với commit gần nhất

Đáp án: B

Giải thích: Giúp tránh commit trên nhánh không mong muốn. Nếu đang ở detached HEAD, có thể không có tên nhánh hiện tại để in.

Q11. What does `git diff -- file` show? | `git diff -- file` hiển thị gì?

A. Creates and checks out a new branch | Tạo và chuyển sang nhánh mới

B. Name of current branch | Tên nhánh hiện tại

C. Unstaged differences for that path | Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit | Khác biệt đã stage so với commit gần nhất

Đáp án: C

Giải thích: diff thường so working tree với staging area. Với nội dung đã stage, `git diff` thường không hiện; phải xem diff cached.

Q12. What does `git diff --cached` show? | `git diff --cached` hiển thị gì?

A. Creates and checks out a new branch | Tạo và chuyển sang nhánh mới

B. Name of current branch | Tên nhánh hiện tại

C. Unstaged differences for that path | Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit | Khác biệt đã stage so với commit gần nhất

Đáp án: D

Giải thích: Giúp xem nội dung đã stage trước khi commit. Nó cho biết nội dung sẽ vào commit, khác phần mới chỉ đang sửa ngoài working tree.

Q13. What does `git fetch` do? | `git fetch` làm gì?

A. Downloads remote refs and objects without merging | Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted | Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration | Để tách công việc trước khi tích hợp

D. The index of content selected for next commit | Chỉ mục nội dung được chọn cho commit kế

Đáp án: A

Giải thích: Fetch cập nhật thông tin remote nhưng chưa nhập vào nhánh đang làm. Fetch có thể cập nhật nhánh theo dõi remote nhưng không nhập chúng vào nhánh local đang đứng.

Q14. What does `git push` do? | `git push` làm gì?

A. Downloads remote refs and objects without merging | Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted | Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration | Để tách công việc trước khi tích hợp

D. The index of content selected for next commit | Chỉ mục nội dung được chọn cho commit kế

Đáp án: B

Giải thích: Push không tự stage hoặc commit thay đổi chưa lưu. Các sửa đổi chưa commit phải được ghi vào lịch sử trước khi push chia sẻ bình thường.

Q15. Why use a feature branch? | Vì sao dùng nhánh tính năng?

A. Downloads remote refs and objects without merging | Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted | Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration | Để tách công việc trước khi tích hợp

D. The index of content selected for next commit | Chỉ mục nội dung được chọn cho commit kế

Đáp án: C

Giải thích: Tách nhánh cho phép xem thử trước khi ảnh hưởng main. Chỉ nên merge nhánh tính năng sau khi kiểm tra đúng phần thay đổi dự định.

Q16. What is staging area? | Staging area là gì?

A. Downloads remote refs and objects without merging | Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted | Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration | Để tách công việc trước khi tích hợp

D. The index of content selected for next commit | Chỉ mục nội dung được chọn cho commit kế

Đáp án: D

Giải thích: Nội dung đã stage có thể khác sửa đổi mới trong working tree. Index tách phần được ghi vào commit với những sửa đổi khác vẫn còn ở máy.

Q17. What does `git pull --ff-only` require? | `git pull --ff-only` đòi hỏi gì?

A. A fast-forward update without a merge commit | Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate | Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again | Không, cần stage lại phần sửa mới

D. Review status and staged diff | Xem status và diff đã stage

Đáp án: A

Giải thích: Nó từ chối lịch sử cần tích hợp kiểu không fast-forward. Lịch sử đã phân nhánh cần quyết định tích hợp riêng; ff-only không tự xóa commit.

Q18. Does setting origin grant push permissions? | Đặt origin có tự cấp quyền push không?

A. A fast-forward update without a merge commit | Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate | Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again | Không, cần stage lại phần sửa mới

D. Review status and staged diff | Xem status và diff đã stage

Đáp án: B

Giải thích: Có thể cấu hình URL dù không có quyền ghi. Quyền do máy chủ và thông tin đăng nhập quyết định, không phải do tên remote trên máy.

Q19. Can git commit include edits made after the last git add automatically? | git commit có tự chứa sửa đổi sau lần git add cuối không?

A. A fast-forward update without a merge commit | Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate | Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again | Không, cần stage lại phần sửa mới

D. Review status and staged diff | Xem status và diff đã stage

Đáp án: C

Giải thích: Commit lưu trạng thái index, có thể cũ hơn nội dung đang sửa. Stage lại sẽ chụp phiên bản mới; nếu không commit dùng bản đã stage cũ.

Q20. How should a developer check changes before committing? | Nên kiểm thay đổi trước commit bằng cách nào?

A. A fast-forward update without a merge commit | Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate | Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again | Không, cần stage lại phần sửa mới

D. Review status and staged diff | Xem status và diff đã stage

Đáp án: D

Giải thích: Kiểm tra giúp phát hiện file hoặc dòng thay đổi ngoài ý muốn. Nếu diff đã stage có bí mật hoặc file ngoài phạm vi, hãy bỏ stage hoặc sửa trước khi commit.

8. Nối sang bài tiếp

`File/Project → git init/add/commit → branch → pull/push → Review.`

PY-44 — Ứng dụng CLI: CRUD, kiểm tra dữ liệu và thống kê hồ sơ

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: CRUD — tạo/đọc/sửa/xóa; **Validation** — xác minh dữ liệu.

`Dictionary/Function → CRUD → CLI → File/CSV → Data App`

Học xong dùng để làm gì? Tạo công cụ thêm/đọc/sửa/xóa dữ liệu từ terminal, từ chối đầu vào sai và báo lỗi rõ ràng.

Kết nối kiến thức tiếp theo: PY-45 trực quan hóa dữ liệu đầu vào và kết quả để người dùng kiểm tra.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
CRUD	tạo/đọc/sửa/xóa	Bốn thao tác quản lý bản ghi
Validation	xác minh dữ liệu	Đảm bảo giá trị đúng giới hạn
Lookup	tra cứu	Tìm bản ghi qua khóa
Aggregate	tổng hợp	Tính chỉ số từ nhiều hồ sơ

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`dict.get(id)`	Đọc bản ghi và trả mặc định nếu thiếu
`dict[id] = record`	Thêm/sửa bản ghi khi đã kiểm tra khóa
`dict.pop(id,default)`	Xóa có xử lý mã chưa tồn tại
`sum()/max()/min()/sorted()`	Tính thống kê và sắp xếp hồ sơ

4. Khái niệm chính và liên quan

Mạch kiến thức: Một CLI nhận yêu cầu người dùng để tạo, đọc, sửa, xóa hồ sơ. Các thao tác CRUD cần Validation trước khi ghi, Lookup để tìm đúng mã và Aggregate để tạo báo cáo. Ví dụ xuyên suốt dùng mã sinh viên làm khóa và điểm trên thang 10 làm trường số.

4.1. CRUD — Bốn thao tác trên hồ sơ

Từ vựng trong mục: CRUD — Create/Read/Update/Delete.

Ứng dụng quản lý sinh viên cần tạo, đọc, sửa và xóa hồ sơ. Với Dictionary, **khóa là mã sinh viên duy nhất**, giá trị là Dictionary trường `name`, `score`. `Create` cần kiểm trùng mã; `Read` tra cứu; `Update` chỉ sửa mã tồn tại; `Delete` xử lý trường hợp mã không tìm thấy. Một CLI dùng lệnh nhập văn bản để thực hiện các thao tác này; đây mới là dữ liệu trong RAM, chưa được lưu bền xuống tệp.

No Note — Không chú thích

```
db = {}
db['S1'] = {'name': 'An', 'score': 8}
print(db['S1']['score'])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Minh họa tạo rồi đọc một hồ sơ trong database giả lập dạng dictionary.
# Dữ liệu: Hồ sơ S1 có name=An và score=8; dữ liệu chỉ tồn tại trong bộ nhớ Python.
# {} tạo dictionary rỗng làm database IN-MEMORY; không có file hay SQL.
db = {}
# CREATE: gán key 'S1' cho dictionary con chứa tên và điểm; cùng key gán lần nữa sẽ ghi đè.
db['S1'] = {'name': 'An', 'score': 8}
# READ: tra lần lượt mã S1 và trường score qua hai cặp dấu []; thiếu key sẽ gây KeyError.
print(db['S1']['score'])
```

Kết quả khi chạy:

```
8
```

Bản ghi S1 được tạo trong RAM; lấy `score` của S1 cho kết quả 8/10.

4.2. Validation — Chặn dữ liệu sai trước khi ghi

Từ vựng trong mục: Validation — kiểm tra hợp lệ.

Validation phải dựa trên **quy tắc dữ liệu**, ví dụ điểm là số nằm trong `[0,10]`, mã không rỗng, mã sinh viên chưa tồn tại khi thêm mới. Nếu chỉ thử `0 <= score <= 10`, một chuỗi nhập từ CLI phải được đổi sang số có xử lý lỗi trước; không so chuỗi với số. Kiểm tra ràng buộc **trước** khi ghi để tránh ghi sai rồi mới phát hiện.

No Note — Không chú thích

```
score = 8
print(isinstance(score, (int, float)) and 0 <= score <= 10)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm loại dữ liệu và giới hạn điểm sô´trước khi châ´p nhận.
# Dữ liệu: Điểm ví dụ 8 phải là int/float và nằm trong đoạn [0,10].
# Đâu vào một điểm sô´nguyên minh họa.
score = 8
# isinstance(x,(int,float)) True nêú x thuộc một trong hai kiểủ số; tuple trong ngoặc là các kiểủ được châ´p nhận.
# Phép so sánh chuôĩ đòi score từ 0 đêń 10, bao gôm hai đâù mút; and yêu câù cả kiểm kiểủ và khoảng đúng.
print(isinstance(score, (int, float)) and 0 <= score <= 10)
```

Kết quả khi chạy:

```
True
```

Điểm 8 là kiểu số và nằm trong thang điểm 0 đến 10.

4.3. Lookup — Tìm bản ghi theo mã

Từ vựng trong mục: Lookup — tra cứu theo khóa.

Với Dictionary `db`, `db.get('S1')` trả hồ sơ S1 nếu tồn tại, còn `db.get('S3')` trả `None` theo mặc định. Lệnh `db['S3']` sẽ ném `KeyError` khi thiếu mã. Trong CLI, việc không có hồ sơ cần được phản hồi rõ, không đánh đồng với một hồ sơ thật có điểm 0. Nếu cần thông báo thay vì `None`, truyền đối số mặc định cho `get`.

No Note — Không chú thích

```
db = {'S1': {'score': 8}}
print(db.get('S3', 'Not found'))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tra một mã hồ sơ không tồn tại mà không gây KeyError.
# Dữ liệu: Database chỉ có S1, nhưng cần đọc mã S3.
# Dictionary lưu khóa S1 có điểm 8.
db = {'S1': {'score': 8}}
# get(key,default) trả giá trị dự phòng 'Not found' nếu không có S3, thay vì ném KeyError như
db['S3'].
print(db.get('S3', 'Not found'))
```

Kết quả khi chạy:

```
Not found
```

Mã S3 không có trong Dictionary nên in thông báo thay vì gây KeyError.

4.4. Aggregate — Tóm tắt nhiều bản ghi

Từ vựng trong mục: Aggregate — tổng hợp.

Tổng hợp biến nhiều hồ sơ thành một con số hay báo cáo: ví dụ mean của ba điểm `[7,8,9]` bằng 8 và điểm cao nhất bằng 9. Trước khi tính cần lấy giá trị số từ trường `score`, không phải mã ID. Với danh sách rỗng, phép `sum(scores)/len(scores)` sẽ chia cho 0; chương trình phải xử lý trường hợp không có hồ sơ.

No Note — Không chú thích

```
scores = [7, 8, 9]
print(sum(scores) / len(scores), max(scores))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo báo cáo điểm trung bình và điểm cao nhất từ ba hồ sơ.
# Dữ liệu: Ba điểm 7,8,9 trên thang 0–10, mỗi số là một người.
# List điểm của ba người, không có thông tin ID.
scores = [7, 8, 9]
# sum() cộng tổng điểm, len() đếm số người, phép chia cho mean; cần list không rỗng để tránh chia
0.
```

```
# max() trả điểm cao nhất, khác với tổng/mean và sẽ lỗi trên list rỗng.
print(sum(scores) / len(scores), max(scores))
```

Kết quả khi chạy:

```
8.0 9
```

Điểm trung bình bằng 8/10, điểm cao nhất bằng 9/10 trên ba hồ sơ.

5. Năm tình huống CLI/CRUD — từ kiểm mã đến 20 lệnh có audit

Tình huống 1 — Cấp 1: Tạo hồ sơ từ 10 yêu cầu, từ chối ID trùng

Từ vựng trong mục: Create — tạo hồ sơ; **Unique key** — khóa duy nhất.

Tình huống thực tế: Phòng đào tạo nhận **10 yêu cầu tạo hồ sơ học viên**, mỗi yêu cầu ghi **mã S01…S09**, tên và điểm kiểm tra **trên thang 10**. Yêu cầu thứ 10 dùng lại **mã S03** với tên khác. Giáo vụ muốn tạo đúng **9 hồ sơ hợp lệ**, giữ nguyên điểm đã lưu của S03 và báo mã bị từ chối, không để người nhập sau vô tình ghi đè hồ sơ trước.

Vì sao chọn cách này? CRUD là bốn thao tác cơ bản với dữ liệu: **Create** (tạo), **Read** (đọc), **Update** (sửa), **Delete** (xóa). Bài đầu chỉ thực hiện **Create** trong `dictionary`: khóa là **mã học viên duy nhất**, giá trị là tên và điểm của người đó. Trước khi thêm, `if row['id'] in db` kiểm mã đã tồn tại hay chưa; nếu trùng thì ghi mã vào `rejected` và dùng `continue` bỏ riêng lượt này. Nếu cứ gán thẳng `db[id] = ...`, hồ sơ S03 ban đầu sẽ bị ghi đè không cảnh báo. Sau 10 yêu cầu, chương trình đối chiếu số hồ sơ và điểm S03, chứ không tự tính yêu cầu bị từ chối là một người mới.

Quy trình:

1. Tạo 10 yêu cầu có một mã lặp và điểm hợp lệ.
2. Kiểm mã tồn tại trước mỗi lệnh ADD rồi lưu hồ sơ.
3. Đếm kết quả, xác nhận điểm S03 gốc không bị thay.

No Note — Không chú thích

```
requests = [{"id":"S01","name":"Student01","score":5},{"id":"S02","name":"Student02","score":6},
{"id":"S03","name":"Student03","score":7},{"id":"S04","name":"Student04","score":8},
{"id":"S05","name":"Student05","score":9},{"id":"S06","name":"Student06","score":10},
{"id":"S07","name":"Student07","score":5},{"id":"S08","name":"Student08","score":6},
{"id":"S09","name":"Student09","score":7},{"id":"S03","name":"Student10","score":8}]
db = {}
rejected = []
for row in requests:
    if row["id"] in db:
        rejected.append(row["id"])
        continue
    db[row["id"]] = {"name":row["name"],"score":row["score"]}
if len(requests) != 10:
    raise ValueError("Cần đủ 10 yêu câù tạo hô`sơ.")
print("Yêu câù:", len(requests))
print("Đã tạo:", len(db))
print("Trùng mã:", ", ".join(rejected))
print("Điểm S03:", db["S03"]["score"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xử lý 10 yêu câu CREATE, từ chối mã S03 bị trùng mà không ghi đè hồ sơ đã có.
# Dữ liệu: 10 yêu câu thêm hồ sơ, ID S03 xuất hiện hai lần với điểm khác nhau; database chỉ giữ lần
tạo đầu.
# Mười dict yêu câu có id/name/score; số yêu câu khác số hồ sơ hợp lệ vì mã có thể trùng.
requests = [{"id":"S01","name":"Student01","score":5},{"id":"S02","name":"Student02","score":6},
{"id":"S03","name":"Student03","score":7},{"id":"S04","name":"Student04","score":8},
{"id":"S05","name":"Student05","score":9},{"id":"S06","name":"Student06","score":10},
{"id":"S07","name":"Student07","score":5},{"id":"S08","name":"Student08","score":6},
{"id":"S09","name":"Student09","score":7},{"id":"S03","name":"Student10","score":8}]
# Dictionary lưu hồ sơ theo ID duy nhất; tránh ghi đè yêu câu trước.
db = {}
# List mã bị từ chối để xuất báo cáo.
rejected = []
# Duyệt từng yêu câu thêm mới theo thứ tự xuất hiện.
for row in requests:
    # in kiểm ID đã là khóa trong db; nếu có thì bị trùng, không được tạo lại.
    if row["id"] in db:
        # append() ghi mã trùng vào list từ chối.
        rejected.append(row["id"])
        # continue bỏ phần còn lại của vòng hiện tại, bảo vệ hồ sơ gốc khỏi ghi đè.
        continue
    # CREATE hợp lệ: ghi dictionary con name/score theo ID chưa có.
    db[row["id"]] = {"name":row["name"],"score":row["score"]}
# Đòi đúng 10 yêu câu nguồn; đếm cả yêu câu bị từ chối.
if len(requests) != 10:
    # ValueError báo đầu vào không đúng số lượng, tránh công bố kết quả thiếu.
    raise ValueError("Cần đủ 10 yêu câu tạo hồ sơ.")
print("Yêu câu:", len(requests))
# len() đếm hồ sơ được tạo với mã duy nhất, không phải 10 yêu câu.
print("Đã tạo:", len(db))
# join() ghép mã trùng trong list bằng dấu phẩy và khoảng trắng.
print("Trùng mã:", ", ".join(rejected))
# Tra điểm S03 sau xử lý để xác nhận điểm lần tạo đầu vẫn còn, không bị cập nhật bởi yêu câu trùng.
print("Điểm S03:", db["S03"]["score"])
```

Kết quả khi chạy:

```
Yêu câu: 10
Đã tạo: 9
Trùng mã: S03
Điểm S03: 7
```

9 hồ sơ cuối là **9 mã khác nhau**; dòng nhập trùng phải được báo lại cho người dùng thay vì tự sửa hồ sơ cũ.

Tình huống 2 — Cấp 2: Tra cứu 12 mã và phân biệt ID vắng với điểm 0

Từ vựng trong mục: Read — đọc hồ sơ; **Lookup** — tra theo khóa; **Missing key** — mã không có.

Tình huống thực tế: Phòng học đang lưu **10 học viên S01–S10 trong bộ nhớ RAM**; mỗi người có mã và điểm, riêng **S01 có điểm 0 hợp lệ**. Nhân viên gửi **12 lượt yêu cầu đọc hồ sơ**, gồm cả 10 mã có thật và hai mã lạ **S99, S88**. Họ cần đếm bao nhiêu lượt tìm thấy và bao nhiêu lượt không thấy, nhưng không được coi học viên đạt 0 điểm là người chưa có hồ sơ.

Vì sao chọn cách này? Read là tra thông tin mà **không sửa** dữ liệu. `db.get(sid)` trả bản ghi cho mã có trong dictionary và trả `None` khi mã **không tồn tại**. Cần kiểm `record is None` chứ không kiểm giá trị điểm bằng `if not score`: trong Python số **0 được xem là False** khi dùng như điều kiện, nhưng ở đây **0 điểm vẫn là một điểm hợp lệ**. Chương trình duyệt đủ 12 yêu cầu rồi đưa mã vào nhóm tìm thấy hoặc nhóm

thiếu, bảo đảm đọc S01 không bị nhầm với S99/S88. Đây là dữ liệu trong RAM, chưa có file hay máy chủ cơ sở dữ liệu.

Quy trình:

1. Tạo 10 hồ sơ với mã khác nhau và 12 lượt tra ID.
2. Dùng `get` nhận hồ sơ hoặc None, đếm lượt tìm thấy/thất bại.
3. Kiểm riêng điểm 0 của S01 vẫn được trả về đúng.

No Note — Không chú thích

```
db = {}
for i in range(1, 11):
    db[f"S{i:02d}"] = {"score": i - 1}
requests = ["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10","S99","S88"]
found, missing = [], []
for sid in requests:
    record = db.get(sid)
    if record is None:
        missing.append(sid)
    else:
        found.append(sid)
if len(requests) != 12:
    raise ValueError("Cần đủ 12 yêu câu tra cứu.")
print("Lượt tra cứu:", len(requests))
print("Tìm thấy:", len(found))
print("Không thấy:", ", ".join(missing))
print("S01 có điểm 0:", db["S01"]["score"] == 0)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc 12 yêu câu GET trên cơ sở mười hô`sơ, phân biệt điểm 0 với không tìm thấy.
# Dữ liệu: Có hô`sơ S01..S10 (S01 có score=0) và hai yêu câu S99/S88 không tôn tại.
# Database in-memory lưu điểm theo mã hô`sơ.
db = {}
# Tạo 10 mã S01..S10, cận trên 11 không được lấy.
for i in range(1, 11):
    # f-string :02d định dạng mã hô`sơ có hai chữ sô´.
    # S01 có score=0, giá trị 0 vẫn là dữ liệu hợp lệ chứ không phải chưa tìm thấy.
    db[f"S{i:02d}"] = {"score": i - 1}
# List 12 mã cần tra; gôm hai mã không tôn tại.
requests = ["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10","S99","S88"]
# Hai list phân biệt mã tìm được và mã không có.
found, missing = [], []
# Duyệt từng yêu câu tra cứu.
for sid in requests:
    # get() trả dict hô`sơ nêú có, hoặc None khi không có khóa; không phát sinh KeyError.
    record = db.get(sid)
    # is None kiểm ĐÚNG dấú hiệu không tìm thấy, không kiểm `if not record` hay `if not score` vì
điểm 0 hợp lệ.
    if record is None:
        # Ghi mã không tôn tại đe^thông báo.
        missing.append(sid)
    else:
        # Ghi mã tìm được mà không sưả hô`sơ.
        found.append(sid)
# Kiểm có đủ 12 lượt GET.
if len(requests) != 12:
    # Dừng nêú sô´yêu câù nguôn sai.
    raise ValueError("Cần đủ 12 yêu câu tra cứu.")
print("Lượt tra cứu:", len(requests))
# Đếm các yêu câù tra tìm được.
```

```
print("Tìm thấy:", len(found))
# Nối các mã không tìm thấy thành dòng thông báo.
print("Không thấy:", ", ".join(missing))
# So điểm thực lưu với số 0 để xác nhận GET không nhầm điểm 0 thành missing.
print("S01 có điểm 0:", db["S01"]["score"] == 0)
```

Kết quả khi chạy:

```
Lượt tra cứu: 12
Tìm thấy: 10
Không thấy: S99, S88
S01 có điểm 0: True
```

Không tìm thấy mã là **vấn đề khóa**, khác với việc tìm thấy mã nhưng giá trị điểm bằng 0.

Tình huống 3 — Cấp 3: Kiểm dữ liệu trước khi sửa điểm qua 15 yêu cầu

Từ vựng trong mục: Update — sửa hồ sơ; **Validation** — kiểm dữ liệu; **Finite number** — số hữu hạn.

Tình huống thực tế: Trung tâm có **10 hồ sơ điểm S01–S10** và tiếp nhận **15 yêu cầu sửa điểm**, mỗi yêu cầu gồm mã và điểm nhập dưới dạng **chuỗi văn bản**. Mười yêu cầu đầu nhập điểm trong khoảng **0–10**; năm yêu cầu còn lại có điểm âm, điểm 11, chữ `bad`, chữ `nan` hoặc mã **S99** không tồn tại. Giáo vụ cần cập nhật đúng dữ liệu hợp lệ và ghi số yêu cầu bị từ chối mà không sinh thêm người học ngoài danh sách.

Vì sao chọn cách này? Update là thay giá trị của **mã đã tồn tại**, nên phải kiểm `sid in db` trước khi lưu. Vì CLI nhận đầu vào dạng chữ, `float(raw_score)` được gọi bên trong `try` để chuyển các chuỗi số thành điểm; `except ValueError` xử lý chuỗi như `bad` mà không làm dừng toàn bộ 15 lượt. Nhưng `float('nan')` vẫn chuyển được thành **số dấu phẩy động đặc biệt**, nên cần `math.isfinite` để loại **NaN hoặc vô cực**, cùng quy tắc điểm phải từ 0 đến 10. Khi một yêu cầu sai, chương trình tăng `rejected` và **giữ nguyên điểm cũ**. Chỉ 10 yêu cầu hợp lệ được ghi, không tự thêm S99.

Quy trình:

1. Tạo 10 mã ban đầu và 15 yêu cầu chứa điểm ở dạng chuỗi.
2. Kiểm mã có sẵn, thử đổi điểm, chặn số ngoài thang và NaN.
3. Đếm cập nhật/thất bại, đối chiếu số hồ sơ và điểm mới S03.

No Note — Không chú thích

```
import math
db = {}
for i in range(1, 11):
    db[f"S{i:02d}"] = {"score": 5.0}
updates = [["S01","6.5"],["S02","7.5"],["S03","8.5"],["S04","9.5"],["S05","6.5"],["S06","6.5"],
["S07","7.5"],["S08","8.5"],["S09","9.5"],["S10","6.5"],["S03","-1"],["S02","11"],["S04","bad"],
["S05","nan"],["S99","7.5"]]
accepted, rejected = 0, 0
for sid, raw_score in updates:
    if sid not in db:
        rejected += 1
        continue
    try:
        score = float(raw_score)
    except ValueError:
        rejected += 1
        continue
    if not math.isfinite(score) or not (0 <= score <= 10):
        rejected += 1
```

```
        continue
    db[sid]["score"] = score
    accepted += 1
if len(updates) != 15:
    raise ValueError("Cần đủ 15 yêu câu cập nhật.")
print("Yêu câu:", len(updates))
print("Cập nhật:", accepted)
print("Từ chôí:", rejected)
print("Hô`sơ còn:", len(db))
print("Điểm S03:", db["S03"]["score"])
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xử lý 15 yêu câu UPDATE, loại mã lạ, chuôĩ không đôỉ được sang số, NaN và điểm ngoài
0–10.
# Dữ liệu: Có 10 hô`sơ S01..S10; 15 yêu câu cập nhật điểm dạng chuôĩ có các trường hợp
-1,11,bad,nan,S99.
# Thư viện chuẩn math cho isfinite() đểloại NaN và sốvô cực.
import math
# Lưu mười hô`sơ đã tôn tại trước khi UPDATE.
db = {}
# Tạo 10 mã duy nhất S01..S10.
for i in range(1, 11):
    # Mã có hai chữ sốbằng f-string :02d.
    db[f"S{i:02d}"] = {"score": 5.0}
# 15 cặp (ID, chuôĩ điểm), gôm cả yêu câu hợp lệ và không hợp lệ.
updates = [["S01","6.5"],["S02","7.5"],["S03","8.5"],["S04","9.5"],["S05","6.5"],["S06","6.5"],
["S07","7.5"],["S08","8.5"],["S09","9.5"],["S10","6.5"],["S03","-1"],["S02","11"],["S04","bad"],
["S05","nan"],["S99","7.5"]]
# Hai bộ đêḿ phân biệt cập nhật thành công và từ chôí.
accepted, rejected = 0, 0
# Unpack một yêu câu thành ID và chuôĩ điểm thô.
for sid, raw_score in updates:
    # Không có hô`sơ đích thì không được tạo mới bằng UPDATE.
    if sid not in db:
        # Tăng sốyêu câu thât bại, kểcả sai mã hoặc giá trị.
        rejected += 1
        # Bỏ qua yêu câu hiện tại sau khi bị từ chôí, không chỉnh DB.
        continue
    # Thử chuyển chuôĩ thành số; cú pháp try/except chỉ bắt lôĩ mà nhánh except khai báo.
    try:
        # Chuyển chuôĩ như '6.5' sang float; 'bad' ném ValueError nhưng 'nan' tạo NaN hợp lệ vê`cú
pháp.
        score = float(raw_score)
    # Bắt lôĩ chuyển chuôĩ không hợp lệ đểtừ chôí thay vì dừng toàn chương trình.
    except ValueError:
        # Tăng sốyêu câu thât bại, kểcả sai mã hoặc giá trị.
        rejected += 1
        # Bỏ qua yêu câu hiện tại sau khi bị từ chôí, không chỉnh DB.
        continue
    # isfinite() kiểm sốhữu hạn, loại NaN/+inf/-inf vôn có thểvượt qua bước float().
    # Giới hạn điểm trong khoảng đóng [0,10]; and/ not kêt hợp cả kiểm hữu hạn và khoảng.
    if not math.isfinite(score) or not (0 <= score <= 10):
        # Tăng sốyêu câu thât bại, kểcả sai mã hoặc giá trị.
        rejected += 1
        # Bỏ qua yêu câu hiện tại sau khi bị từ chôí, không chỉnh DB.
        continue
    # UPDATE đúng mã/điểm: chỉ sưả trường score của hô`sơ hiện có.
    db[sid]["score"] = score
    # Tăng sốlần cập nhật thành công, không phải sốngười khác nhau.
    accepted += 1
# Kiểm đủ 15 yêu câu, không bỏ những yêu câu bị từ chôí.
if len(updates) != 15:
    # Dừng khi dữ liệu sốyêu câu không đúng.
    raise ValueError("Cần đủ 15 yêu câu cập nhật.")
```

```
print("Yêu câù:", len(updates))
print("Cập nhật:", accepted)
print("Từ chôí:", rejected)
# Sô´hô`sơ sau UPDATE vâñ là 10, vì UPDATE không tạo/xóa hô`sơ.
print("Hô`sơ còn:", len(db))
# Tra S03 sau các yêu câù đểkiêm̉ trạng thái điêm̉ cuôí đã được cập nhật hợp lệ.
print("Điêm̉ S03:", db["S03"]["score"])
```

Kết quả khi chạy:

```
Yêu câù: 15
Cập nhật: 10
Từ chôí: 5
Hô`sơ còn: 10
Điêm̉ S03: 8.5
```

NaN phải bị chặn **trước khi ghi**; nếu để NaN trong dữ liệu, tổng hợp điểm và so sánh ngưỡng về sau có thể bị sai.

Tình huống 4 — Cấp 4: Xóa an toàn từ 18 lệnh với mã lặp và mã lạ

Từ vựng trong mục: Delete — xóa; **Idempotence** — gọi lại không làm sai dữ liệu; **Audit** — đối soát.

Tình huống thực tế: Ứng dụng quản lý có **10 hồ sơ S01–S10** và nhận **18 lệnh xóa** theo thứ tự. Mười lệnh đầu gọi đúng mười mã, nhưng tám lệnh sau có mã **đã bị xóa** hoặc **chưa từng tồn tại**. Nhân viên cần biết có bao nhiêu hồ sơ thực sự đã mất, bao nhiêu lệnh không làm thay đổi dữ liệu và xác nhận cuối cùng không còn hồ sơ nào.

Vì sao chọn cách này? Delete là xóa một bản ghi khi **mã đang tồn tại**. `db.pop(sid, None)` vừa lấy vừa xóa khi thấy mã, còn nếu mã vắng thì trả `None` thay vì báo `KeyError` làm gián đoạn vòng lặp. Chương trình kiểm giá trị trả về để tách **xóa thành công** và **mã không còn trong DB**. Sau khi mười hồ sơ đầu đã xóa, gọi xóa lại S01 không thể được tính là thành công lần nữa, vì dữ liệu không thay đổi. Tổng số thành công và thất bại phải bằng **18 lệnh**, còn `len(db)` xác nhận danh sách cuối đã rỗng.

Quy trình:

1. Tạo 10 ID trong Dictionary và 18 mã được yêu cầu xóa.
2. Dùng `pop(..., None)`, đếm thành công khi trả hồ sơ thật.
3. Báo số lệnh lỗi/không thay đổi và số hồ sơ còn lại.

No Note — Không chú thích

```
db = {}
for i in range(1, 11):
    db[f"S{i:02d}"] = {"score": i}
requests =
["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10","S01","S02","S03","S99","S98","S97","S
05","S100"]
deleted, missing = 0, 0
for sid in requests:
    value = db.pop(sid, None)
    if value is None:
        missing += 1
    else:
        deleted += 1
if len(requests) != 18:
    raise ValueError("Cân đủ 18 yêu câù xóa.")
print("Lệnh xóa:", len(requests))
```

```
print("Xóa thật:", deleted)
print("Mã không còn:", missing)
print("Hồ sơ còn:", len(db))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Xử lý 18 yêu cầu DELETE có mã trùng và mã lạ, bảo đảm xóa một lần mỗi hồ sơ.
# Dữ liệu: Bắt đầu 10 hồ sơ S01..S10; 18 lượt DELETE gồm mã lặp lại, S99/S98/S97/S100 không tồn
tại.
# Database in-memory ban đầu rỗng.
db = {}
# Tạo 10 mã có số thứ tự 1..10.
for i in range(1, 11):
    # f-string :02d thêm số 0 ở đầu mã S01..S10.
    # Điểm ban đầu của một hồ sơ để có dữ liệu kiểm xóa.
    db[f"S{i:02d}"] = {"score": i}
# 18 lệnh xóa, gồm cả mã trùng và mã không tồn tại.
requests =
["S01","S02","S03","S04","S05","S06","S07","S08","S09","S10","S01","S02","S03","S99","S98","S97","S
05","S100"]
# Hai bộ đếm lệnh xóa thật và lệnh không tìm thấy hồ sơ.
deleted, missing = 0, 0
# Lần lượt xử lý từng yêu cầu, trạng thái db thay đổi sau mỗi xóa.
for sid in requests:
    # pop(key,default) vừa lấy VỪA XÓA hồ sơ nếu tồn tại; nếu thiếu khóa trả None thay vì KeyError.
    value = db.pop(sid, None)
    # Chỉ khi pop trả None mới tính lệnh không tìm thấy; xóa cùng ID lần hai không xóa thêm.
    if value is None:
        # Đếm yêu cầu xóa vào ID không còn tồn tại.
        missing += 1
    else:
        # Đếm số hồ sơ thực sự xóa, không phải tổng yêu cầu.
        deleted += 1
# Xác minh đủ 18 lệnh nguồn.
if len(requests) != 18:
    # Dừng nếu thiếu lệnh.
    raise ValueError("Cần đủ 18 yêu cầu xóa.")
print("Lệnh xóa:", len(requests))
print("Xóa thật:", deleted)
print("Mã không còn:", missing)
# Đếm hồ sơ còn lại trong database sau xóa.
print("Hồ sơ còn:", len(db))
```

Kết quả khi chạy:

```
Lệnh xóa: 18
Xóa thật: 10
Mã không còn: 8
Hồ sơ còn: 0
```

Xóa không tìm thấy được ghi nhận rõ trong **audit**; không được ngầm tạo lại hồ sơ trong quá trình xóa.

Tình huống 5 — Cấp 5: Chạy 20 câu lệnh CLI CRUD có kiểm tra và báo cáo cuối

Từ vựng trong mục: CLI command — lệnh văn bản; **CRUD** — tạo/đọc/sửa/xóa; **Audit trail** — lịch sử thao tác.

Tình huống thực tế: Trung tâm đào tạo xây **CLI (chương trình nhận lệnh văn bản)** để quản lý điểm học viên. Có **20 lệnh theo thứ tự**, gồm `ADD` (thêm), `GET` (xem), `UPDATE` (sửa), `DELETE` (xóa) và `REPORT` (báo cáo). Trong đó có lệnh hợp lệ, thêm trùng mã, điểm không phải số, điểm vượt 10, tìm mã chưa có và xóa lần

thứ hai. Nhóm muốn xử lý đủ 20 lệnh, đếm số thao tác thành công/thất bại và xem **số hồ sơ cùng điểm trung bình cuối** mà không làm hỏng cơ sở dữ liệu trong RAM.

Vì sao chọn cách này? CLI là giao diện để nhập lệnh bằng chữ thay vì bấm nút; ở ví dụ này **20 lệnh được chuẩn bị sẵn**, không yêu cầu người học gõ tương tác. `shlex.split` chia chữ thành các thành phần an toàn hơn cách chạy nội dung bằng `eval`; chương trình kiểm **tên lệnh và số đối số** trước khi làm. `parse_score` đổi điểm thành số và kiểm **hữu hạn, từ 0 đến 10**; thêm phải chưa có mã, sửa/xóa/tra phải có mã phù hợp. `audit` ghi các lượt thành công theo loại hoặc `REJECT` cho lệnh không đạt điều kiện; lệnh `REPORT` chỉ **đọc và tổng hợp**, không sửa hồ sơ. Vì các lệnh chạy **theo thứ tự**, xóa một mã ảnh hưởng lệnh tới sau, và điểm trung bình cuối chỉ được tính từ **những hồ sơ còn lại**.

Quy trình:

1. Lưu 20 dòng lệnh để chạy tự động, không cần người nhập bàn phím.
2. Phân tích từng lệnh theo tên và số đối số, kiểm khóa và giá trị điểm.
3. Đếm CREATE/UPDATE/DELETE/thất bại; tính số hồ sơ và mean cuối, xác nhận báo cáo không ghi dữ liệu.

No Note — Không chú thích

```
import math
import shlex
commands = ["ADD S01 7","ADD S02 8","ADD S03 9","ADD S04 6","ADD S05 5","ADD S06 10","ADD S07
7","ADD S08 8","ADD S09 9","ADD S10 6","ADD S01 9","UPDATE S03 9.5","UPDATE S05 bad","GET S03","GET
S99","DELETE S02","DELETE S02","ADD S11 11","UPDATE S08 8.5","REPORT"]
db = {}
audit = {"ADD":0,"UPDATE":0,"DELETE":0,"GET":0,"REPORT":0,"REJECT":0}
def parse_score(raw):
    try:
        value = float(raw)
    except ValueError:
        return None
    if math.isfinite(value) and 0 <= value <= 10:
        return value
    return None
for command in commands:
    parts = shlex.split(command)
    verb = parts[0]
    ok = False
    if verb == "ADD" and len(parts) == 3:
        sid, raw = parts[1:]
        score = parse_score(raw)
        if sid not in db and score is not None:
            db[sid] = score
            ok = True
    elif verb == "UPDATE" and len(parts) == 3:
        sid, raw = parts[1:]
        score = parse_score(raw)
        if sid in db and score is not None:
            db[sid] = score
            ok = True
    elif verb == "GET" and len(parts) == 2:
        ok = parts[1] in db
    elif verb == "DELETE" and len(parts) == 2:
        ok = db.pop(parts[1], None) is not None
    elif verb == "REPORT" and len(parts) == 1:
        ok = bool(db)
    if ok:
        audit[verb] += 1
    else:
        audit["REJECT"] += 1
```

```
if len(commands) != 20 or sum(audit.values()) != 20:
    raise ValueError("Cần đối chiếu đủ 20 lệnh CLI được xử lý.")
print("Lệnh:", len(commands))
print("Thêm/sửa/xóa:", audit["ADD"], audit["UPDATE"], audit["DELETE"])
print("Lệnh từ chối:", audit["REJECT"])
print("Hồ sơ cuối:", len(db))
print(f"Điểm TB cuối: {sum(db.values())/len(db):.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Mô phỏng CLI CRUD với 20 lệnh ADD/UPDATE/GET/DELETE/REPORT, kiểm dữ liệu và thống kê
audit.
# Dữ liệu: 20 chuỗi lệnh nhập sẵn; database là dictionary tạm, có lệnh trùng mã, điểm bad/11 và xóa
hai lần.
# math.isfinite() xác nhận điểm số thật sự là số hữu hạn.
import math
# shlex.split() phân tích chuỗi lệnh theo cú pháp dấu nháy/khoảng trắng giống shell, KHÔNG thực thi
shell.
import shlex
# 20 dòng lệnh mô phỏng đầu vào người dùng, không mở terminal tương tác thật.
commands = ["ADD S01 7","ADD S02 8","ADD S03 9","ADD S04 6","ADD S05 5","ADD S06 10","ADD S07
7","ADD S08 8","ADD S09 9","ADD S10 6","ADD S01 9","UPDATE S03 9.5","UPDATE S05 bad","GET S03","GET
S99","DELETE S02","DELETE S02","ADD S11 11","UPDATE S08 8.5","REPORT"]
# Dictionary lưu ID→điểm trong RAM, không có SQL hoặc file.
db = {}
# Dictionary bộ đếm kết quả từng loại lệnh thành công và REJECT; tổng đếm phải bằng 20.
audit = {"ADD":0,"UPDATE":0,"DELETE":0,"GET":0,"REPORT":0,"REJECT":0}
# Hàm nhận chuỗi điểm và trả float hợp lệ hoặc None để gọi xử lý từ chối.
# Kiểm kiểu, tính hữu hạn và phạm vi điểm trước khi ghi.
def parse_score(raw):
    # Bắt đầu đoạn có thể lỗi khi ép chuỗi sang số.
    try:
        # float() chuyển chuỗi điểm, có thể sinh ValueError nếu 'bad'; 'nan' sẽ chuyển được nên cần
isfinite().
        value = float(raw)
    # Bắt lỗi chuỗi không thể chuyển thành số.
    except ValueError:
        # None là dấu hiệu không hợp lệ; điểm 0 vẫn hợp lệ nên không dùng if not score.
        return None
    # isfinite() loại NaN, +inf/-inf; đồng thời yêu cầu điểm trong [0,10].
    # Chặn mọi điểm âm hoặc >10.
    if math.isfinite(value) and 0 <= value <= 10:
        # Trả float hợp lệ cho thao tác ADD/UPDATE.
        return value
    # None là dấu hiệu không hợp lệ; điểm 0 vẫn hợp lệ nên không dùng if not score.
    return None
# Duyệt từng lệnh đúng thứ tự, vì UPDATE/DELETE phụ thuộc trạng thái trước đó.
for command in commands:
    # Tách command thành list token, xử lý khoảng trắng và chuỗi có dấu nháy, không thực thi nội
dung.
    parts = shlex.split(command)
    # [0] lấy động từ lệnh; với lệnh rỗng sẽ IndexError, nhưng dữ liệu mẫu không có lệnh rỗng.
    verb = parts[0]
    # Mặc định lệnh bị từ chối cho tới khi một nhánh thỏa điều kiện.
    ok = False
    # Lệnh ADD cần đúng ba token (ADD ID điểm), tránh thiếu/dư tham số.
    if verb == "ADD" and len(parts) == 3:
        # Slice [1:] bỏ động từ, lấy hai tham số mã và điểm.
        sid, raw = parts[1:]
        # Kiểm kiểu, tính hữu hạn và phạm vi điểm trước khi ghi.
        score = parse_score(raw)
        # CREATE chỉ khi ID chưa có và điểm hợp lệ; phân biệt None với score=0.
        if sid not in db and score is not None:
            # Lưu điểm theo ID khi ADD/UPDATE hợp lệ; thao tác UPDATE kiểm tồn tại trước.
            db[sid] = score
```

```
            ok = True
    # UPDATE cần đúng ba token, không tự tạo hồ sơ mới.
    elif verb == "UPDATE" and len(parts) == 3:
        # Slice [1:] bỏ động từ, lấy hai tham số mã và điểm.
        sid, raw = parts[1:]
        # Kiểm kiểu, tính hữu hạn và phạm vi điểm trước khi ghi.
        score = parse_score(raw)
        # UPDATE chỉ cho phép sửa ID đã tồn tại với điểm được validate.
        if sid in db and score is not None:
            # Lưu điểm theo ID khi ADD/UPDATE hợp lệ; thao tác UPDATE kiểm tồn tại trước.
            db[sid] = score
            ok = True
    # GET chỉ nhận một tham số ID; trạng thái thành công khi ID có trong db.
    elif verb == "GET" and len(parts) == 2:
        # Tra ID trong dict không thay đổi hồ sơ.
        ok = parts[1] in db
    # DELETE chỉ nhận ID, không cần điểm.
    elif verb == "DELETE" and len(parts) == 2:
        # pop() xóa và trả giá trị hoặc None; xóa lần hai bị REJECT, không tạo KeyError.
        ok = db.pop(parts[1], None) is not None
    # REPORT không có tham số, chỉ hợp lệ nếu db có ít nhất một hồ sơ.
    elif verb == "REPORT" and len(parts) == 1:
        # bool(dictionary) là True khi database không rỗng.
        ok = bool(db)
    # Lệnh hợp lệ đóng góp đúng một bộ đếm theo verb.
    if ok:
        # Tăng đếm loại lệnh thành công; các verb hợp lệ đều có key trong audit.
        audit[verb] += 1
    else:
        # Mọi lệnh thất bại đi vào bộ đếm chung REJECT.
        audit["REJECT"] += 1
# Kiểm đủ 20 dòng lệnh.
# values() lấy số đếm từng loại; sum() phải bằng tổng lệnh để không bỏ/trùng thống kê.
if len(commands) != 20 or sum(audit.values()) != 20:
    # Dừng khi số yêu cầu và tổng audit không khớp.
    raise ValueError("Cần đúi chiếu đủ 20 lệnh CLI được xử lý.")
print("Lệnh:", len(commands))
print("Thêm/sửa/xóa:", audit["ADD"], audit["UPDATE"], audit["DELETE"])
print("Lệnh từ chối:", audit["REJECT"])
print("Hồ sơ cuối:", len(db))
# values() lấy điểm hồ sơ còn; sum()/len() tính điểm trung bình, cần db không rỗng (kịch bản bảo
đảm).
# f-string :.2f chỉ định hiển thị điểm trung bình với hai chữ số sau dấu thập phân.
print(f"Điểm TB cuối: {sum(db.values())/len(db):.2f}")
```

Kết quả khi chạy:

```
Lệnh: 20
Thêm/sửa/xóa: 10 2 1
Lệnh từ chối: 5
Hồ sơ cuối: 9
Điểm TB cuối: 7.56
```

Ứng dụng hiện chỉ lưu trạng thái **trong RAM**: khi tắt chương trình dữ liệu biến mất. Triển khai thật phải bổ sung lưu bền, phân quyền và kiểm thử xử lý lỗi.

6. Sai lầm thường gặp

- Thêm mã đã có bằng `dict[id]=...` sẽ ghi đè dữ liệu cũ nếu không kiểm trùng.
- Điểm từ CLI có thể là chuỗi; phải parse với xử lý ValueError trước khi so với số.
- `get()` khác `dict[key]` khi thiếu khóa; `pop` có default khác xóa trực tiếp.

- Update cần kiểm mã tồn tại và giới hạn điểm; xóa mã thiếu không được làm mất dữ liệu khác.
- Tính mean cần kiểm List không rỗng và chỉ tính trên trường số thực sự.
- Dictionary tồn tại trong RAM; chương trình thực tế cần lưu file/database và cân nhắc sao lưu/quyền truy cập.

7. Trắc nghiệm

Q1. What are the four CRUD operations? | Bốn thao tác CRUD là gì?

A. Create, Read, Update, Delete | Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing | Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record | Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records | Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: A

Giải thích: CRUD mô tả vòng đời bản ghi, không phải một lệnh cơ sở dữ liệu. Nếu coi thêm mới như ghi đè mà không có chính sách, dữ liệu sinh viên cũ có thể bị mất.

Q2. What is input validation? | Kiểm tra dữ liệu đầu vào là gì?

A. Create, Read, Update, Delete | Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing | Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record | Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records | Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: B

Giải thích: Validation chặn điểm sai hoặc mã trùng trước khi ghi. Validation kiểm cả kiểu dữ liệu và giới hạn nghiệp vụ, không chỉ kiểm code chạy được.

Q3. What is a lookup key? | Khóa tra cứu là gì?

A. Create, Read, Update, Delete | Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing | Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record | Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records | Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: C

Giải thích: Mã sinh viên dùng làm khóa duy nhất, còn điểm là giá trị bản ghi. Nên dùng mã sinh viên duy nhất và ổn định làm khóa, không dùng điểm có thể thay đổi.

Q4. What is aggregation? | Aggregate là gì?

A. Create, Read, Update, Delete | Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing | Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record | Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records | Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: D

Giải thích: Trung bình và cực đại tổng hợp trường số qua nhiều học viên. Với lớp rỗng, cần quy định kết quả 'không có dữ liệu' trước khi tính thống kê số.

Q5. What does `db.get('S1')` do? | `db.get('S1')` làm gì?

A. Returns S1 value or None if missing | Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value | Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent | Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list | Số điểm trong danh sách

Đáp án: A

Giải thích: get cho phép tra thiếu khóa mà không ném KeyError. `get` chỉ đọc; việc nhận giá trị mặc định không làm tạo hồ sơ còn thiếu.

Q6. What does `db['S1']=record` do? | `db['S1']=record` làm gì?

A. Returns S1 value or None if missing | Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value | Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent | Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list | Số điểm trong danh sách

Đáp án: B

Giải thích: Không kiểm khóa tồn tại, phép gán có thể ghi đè hồ sơ. Hàm thêm mới nên từ chối mã đã tồn tại, không chỉ dùng phép gán.

Q7. What does `db.pop('S2',None)` do? | `db.pop('S2',None)` làm gì?

A. Returns S1 value or None if missing | Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value | Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent | Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list | Số điểm trong danh sách

Đáp án: C

Giải thích: Giá trị mặc định tránh lỗi KeyError khi khóa vắng mặt. Nếu không có giá trị mặc định, pop khóa S2 còn thiếu sẽ ném KeyError.

Q8. What does `len(scores)` provide? | `len(scores)` cung cấp gì?

A. Returns S1 value or None if missing | Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value | Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent | Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list | Số điểm trong danh sách

Đáp án: D

Giải thích: Mean chia tổng cho số phần tử khi số lượng lớn hơn 0. Số lượng bằng 0 là ca biên, cần kiểm trước khi chia tổng.

Q9. What is a safe score validation for integer score 8? | Kiểm tra điểm 8 hợp lệ trên thang 0–10 thế nào?

A. `0 <= score <= 10` | `0 <= score <= 10`

B. Confirm the key does not exist | Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid | Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError | Lỗi KeyError

Đáp án: A

Giải thích: Cần kiểm cả cận dưới và cận trên. Nếu điểm đến từ chuỗi CLI bất kỳ, cần xác minh/đổi kiểu trước khi so sánh.

Q10. What is required before adding a unique student ID? | Trước khi thêm mã sinh viên duy nhất cần gì?

A. `0 <= score <= 10` | `0 <= score <= 10`

B. Confirm the key does not exist | Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid | Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError | Lỗi KeyError

Đáp án: B

Giải thích: Nhờ đó tránh ghi đè âm thầm hồ sơ khác. Kiểm khóa trước khi ghi; ghi đè rồi mới phát hiện có thể không khôi phục tự động được.

Q11. What is required before updating a student record? | Trước khi sửa hồ sơ cần gì?

A. `0 <= score <= 10` | `0 <= score <= 10`

B. Confirm the key does not exist | Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid | Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError | Lỗi KeyError

Đáp án: C

Giải thích: Sửa không nên vô tình tạo khóa mới. Nhờ đó vừa cập nhật đúng người vừa giữ điểm trong thang hợp lệ.

Q12. What happens with `db['missing']` for an absent key? | Truy cập `db['missing']` khi thiếu khóa sẽ sao?

A. `0 <= score <= 10` | `0 <= score <= 10`

B. Confirm the key does not exist | Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid | Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError | Lỗi KeyError

Đáp án: D

Giải thích: Truy cập trực tiếp gây lỗi khi khóa không tồn tại. Cần phân biệt khóa không tồn tại với học viên tồn tại nhưng điểm bằng 0.

Q13. What does `sum([7,8,9])/3` equal? | `sum([7,8,9])/3` bằng gì?

A. 8.0 | 8.0

B. 9 | 9

C. Check length before dividing | Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself | Không, tự nó không giữ qua lúc khởi động lại

Đáp án: A

Giải thích: Tổng bằng 24 chia ba bằng 8. Cả ba điểm có trọng số bằng nhau; kết quả không phải điểm cao nhất hay tổng điểm.

Q14. What does `max([7,8,9])` return? | `max([7,8,9])` trả gì?

A. 8.0 | 8.0

B. 9 | 9

C. Check length before dividing | Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself | Không, tự nó không giữ qua lúc khởi động lại

Đáp án: B

Giải thích: max chọn điểm lớn nhất. Phép này chọn điểm cao nhất, chưa tự xác định mã sinh viên nếu không giữ liên hệ bản ghi.

Q15. How should an empty score list be handled before mean? | Trước khi tính mean với List điểm rỗng cần gì?

A. 8.0 | 8.0

B. 9 | 9

C. Check length before dividing | Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself | Không, tự nó không giữ qua lúc khởi động lại

Đáp án: C

Giải thích: Nếu không kiểm, chia cho len=0 sẽ gây lỗi. Chương trình cần nêu cách xử lý lớp rỗng, không bịa ra một điểm trung bình.

Q16. What does a Python Dictionary persist automatically? | Dictionary Python tự lưu bền dữ liệu không?

A. 8.0 | 8.0

B. 9 | 9

C. Check length before dividing | Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself | Không, tự nó không giữ qua lúc khởi động lại

Đáp án: D

Giải thích: Hồ sơ trong RAM cần lưu ra file hoặc database để tồn tại lâu dài. Muốn dữ liệu còn sau khi mở lại, phải lưu bằng JSON, SQLite hoặc cơ chế lưu bền khác.

Q17. What is a duplicate ID risk in CRUD? | Nguy cơ mã trùng trong CRUD là gì?

A. Existing data may be overwritten | Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling | Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled | Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values | Các giá trị điểm dạng số

Đáp án: A

Giải thích: Kiểm tra khóa duy nhất tránh thay nhầm. Nên từ chối mã trùng hoặc dùng lệnh Update riêng nếu chủ ý sửa.

Q18. What should CLI do with a non-numeric score input? | CLI nên làm gì với điểm nhập chữ thay vì số?

A. Existing data may be overwritten | Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling | Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled | Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values | Các giá trị điểm dạng số

Đáp án: B

Giải thích: Chuỗi nhập từ terminal cần đổi số trước khi so ngưỡng. Hãy đổi kiểu an toàn, xử lý ValueError rồi mới kiểm tra thang điểm.

Q19. When is `dict.get` preferable to square indexing? | Khi nào dùng `dict.get` thay `dict[key]`?

A. Existing data may be overwritten | Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling | Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled | Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values | Các giá trị điểm dạng số

Đáp án: C

Giải thích: get trả mặc định thay cho ném KeyError. Khi chắc chắn mọi khóa đều có, truy cập [] phù hợp; nếu có thể thiếu thì `get` an toàn hơn.

Q20. Which fields should an average score computation use? | Tính điểm trung bình nên dùng trường nào?

A. Existing data may be overwritten | Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling | Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled | Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values | Các giá trị điểm dạng số

Đáp án: D

Giải thích: ID và tên để nhận diện, không đưa vào phép trung bình. Chỉ tổng hợp trường điểm dạng số, thống nhất thang đo và cách xử lý ô thiếu.

8. Nối sang bài tiếp

`Dictionary/Function → CRUD → CLI → File/CSV → Data App.`

PY-45 — Matplotlib: biểu đồ line, scatter và histogram

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Line chart — biểu đồ đường; **Scatter plot** — biểu đồ phân tán.

`NumPy/Pandas → Visualization → EDA → PCA/ML`

Học xong dùng để làm gì? Dùng biểu đồ thể hiện phân phối, xu hướng và so sánh, ghi nhãn trục/đơn vị để đọc đúng kết quả.

Kết nối kiến thức tiếp theo: PY-46 tích hợp dữ liệu, xử lý, mô hình và trực quan hóa trong một dự án nhỏ.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Line chart	biểu đồ đường	Biểu diễn thay đổi theo thứ tự thời gian
Scatter plot	biểu đồ phân tán	Mỗi cặp x,y tạo thành một điểm quan sát
Histogram	biểu đồ tần suất	Đếm số mẫu trong từng khoảng giá trị
Axis label	nhãn trục	Mô tả đại lượng và đơn vị trên trục

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`plt.plot(x,y)`	Đường thay đổi theo thời gian; x,y có cùng số phần tử
`plt.scatter(x,y)`	Vẽ điểm từng mẫu từ hai đại lượng
`plt.hist(values,bins=...)`	Gom quan sát vào các khoảng (bins)
`plt.xlabel()/ylabel()/savefig()`	Ghi nhãn kèm đơn vị và lưu hình

4. Khái niệm chính và liên quan

Mạch kiến thức: Khi đã có bảng số liệu, chọn biểu đồ phù hợp câu hỏi: line cho mốc có thứ tự, scatter cho cặp biến trên mỗi mẫu, histogram cho phân bố một biến số. Nhãn trục nối dữ liệu với ý nghĩa và đơn vị để người đọc không suy luận sai.

4.1. Line chart — Nối các mốc có thứ tự

Từ vựng trong mục: Line chart — biểu đồ đường.

Biểu đồ đường nối những điểm theo **thứ tự x**, thích hợp quan sát biến thiên doanh thu theo ngày. Ví dụ ba ngày 1,2,3 có doanh thu 10,15,12 nghìn đồng: trục x là ngày, trục y là nghìn đồng. Nối điểm cho thấy tăng rồi giảm; không có nghĩa tồn tại dữ liệu ở mọi thời điểm giữa hai ngày. `plt.plot(x,y)` yêu cầu hai dãy có cùng số phần tử.

No Note — Không chú thích

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 15, 12])
print(len(plt.gca().lines[0].get_xdata()))
plt.close()
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Vẽ đường nôi ba môć theo thứ tự, sau đó đọc lại dữ liệu trục x từ chính đôi tượng đường
vẽ.
# Dữ liệu: Ba môć x=[1,2,3] và y=[10,15,12]; không lưu ảnh ra file, chỉ kiêm sô´điêm.
# Nhập thư viện Matplotlib đểchọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chọn backend Agg vẽ ảnh raster KHÔNG cân cưả sổGUI, phù hợp Windows headless/CI; phải
đặt TRƯỚC import pyplot.
matplotlib.use('Agg')
# Nhập thư viện Matplotlib đểchọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồthị dùng tên ngắń plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt

# plt.plot(x,y) tạo Line2D nôi ba cặp sô´theo THỨ TỰ x/y; không tự săṕ hay nội suy thành dữ liệu
quan sát mới.
plt.plot([1, 2, 3], [10, 15, 12])
# plt.gca() lâý Axes hiện tại (Get Current Axes); .lines là list các Line2D đã vẽ; [0] chọn đường
đâù; .get_xdata() đọc ba tọa độ x gôć; len() đêḿ sô´tọa độ, không đêḿ sô´đoạn (2).
print(len(plt.gca().lines[0].get_xdata()))
# close() đóng figure hiện tại, giải phóng tài nguyên; không cân hiển thị cưả sổvới backend Agg.
plt.close()
```

Kết quả khi chạy:

```
3
```

Biểu đồ đã nhận ba điểm, rồi đóng Figure; số 3 là số mốc, không phải doanh thu.

4.2. Scatter — Mỗi hàng là một cặp quan sát

Từ vựng trong mục: Scatter plot — biểu đồ phân tán.

Biểu đồ scatter đặt **mỗi mẫu thành một điểm (x,y)**. Ví dụ ba sinh viên học 2,4,6 giờ/tuần, có điểm 5,7,9: mỗi vị trí x ghép đúng điểm y của cùng học viên. Scatter giúp thấy liên hệ, nhưng không tự chứng minh học nhiều gây điểm cao. `plt.scatter` tạo các điểm riêng không nối đường như line chart.

No Note — Không chú thích

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.scatter([2, 4, 6], [5, 7, 9])
print(len(plt.gca().collections[0].get_offsets()))
plt.close()
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Vẽ scatter từ ba cặp x/y và đếm chính xác các điểm đã vẽ bằng get_offsets().
# Dữ liệu: Ba điểm (2,5), (4,7), (6,9), mỗi cặp là một quan sát độc lập.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chọn backend Agg vẽ ảnh raster KHÔNG cần cửa sổ GUI, phù hợp Windows headless/CI; phải
đặt TRƯỚC import pyplot.
matplotlib.use('Agg')
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt

# scatter(x,y) vẽ điểm rời, không nối chúng thành đường như plot().
plt.scatter([2, 4, 6], [5, 7, 9])
# gca() lấy Axes hiện tại; collections[0] là PathCollection của scatter đầu tiên; get_offsets() trả
mảng tọa độ shape (3,2); len() đếm BA điểm (hàng).
print(len(plt.gca().collections[0].get_offsets()))
# Đóng figure để tránh tích lũy đối tượng vẽ khi chạy nhiều ví dụ liên tiếp.
plt.close()
```

Kết quả khi chạy:

```
3
```

Đã vẽ ba điểm đại diện ba sinh viên, giữ đúng cặp giờ học–điểm.

4.3. Histogram — Đếm số đo rơi vào các khoảng

Từ vựng trong mục: Histogram — biểu đồ tần suất; Bin — khoảng.

Histogram dùng **một dãy số** và chia trục giá trị thành các **bins**, rồi đếm số quan sát trong mỗi khoảng. Sáu điểm `[5,6,6,7,8,9]` chia bốn bins đều từ 5 đến 9 tạo tần suất `[1,2,1,2]`. Khác biểu đồ cột cho các loại (A,B,C), histogram phù hợp biến số liên tục hoặc có thứ tự giá trị và phụ thuộc độ rộng bins.

No Note — Không chú thích

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

counts, edges, bars = plt.hist([5, 6, 6, 7, 8, 9], bins=[5, 6, 7, 8, 10])
print(counts.astype(int).tolist())
plt.close()
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Vẽ histogram với biên khoảng tự chọn, kiểm đếm tần suất ở mỗi khoảng.
# Dữ liệu: Sáu số đo 5,6,6,7,8,9; các mốc bins 5/6/7/8/10 tạo bốn khoảng.
```

```
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chọn backend Agg vẽ ảnh raster KHÔNG cần cửa sổ GUI, phù hợp Windows headless/CI; phải đặt TRƯỚC import pyplot.
matplotlib.use('Agg')
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt

# hist(data,bins=...) phân số đo vào bốn khoảng [5,6),[6,7),[7,8),[8,10]; khoảng CUỐI bao gồm biên phải 10. Hàm trả (counts,edges,bars): số lượt mỗi khoảng, các ranh giới, các hình cột.
counts, edges, bars = plt.hist([5, 6, 6, 7, 8, 9], bins=[5, 6, 7, 8, 10])
# counts là ndarray float số lượt trong mỗi bin; astype(int) đổi sang số nguyên để in, tolist() đổi ndarray thành list Python; không biến thành dữ liệu gốc.
print(counts.astype(int).tolist())
# Đóng figure sau khi lấy counts, tránh giữ figure dư trong RAM.
plt.close()
```

Kết quả khi chạy:

```
[1, 2, 1, 2]
```

Bốn khoảng chứa lần lượt 1, 2, 1, 2 quan sát; khoảng cuối bao gồm cận phải.

4.4. Axis labels — Ghi rõ biến và đơn vị

Từ vựng trong mục: Axis label — nhãn trục; Figure — hình.

Nhãn trục cho biết đang đo gì: `Time (hours)` ở trục ngang và `Temperature (°C)` ở trục dọc. Không ghi đơn vị, người xem dễ nhầm giờ với phút hoặc độ C với độ F. `xlabel`/`ylabel` chỉ đổi chữ trên Figure, không tự đổi dữ liệu. Có thể lưu hình bằng `savefig` rồi đóng `close` để không giữ Figure trong bộ nhớ.

No Note — Không chú thích

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.plot([1, 2], [22, 24])
plt.xlabel('Time (hours)')
plt.ylabel('Temperature (C)')
print(plt.gca().get_xlabel())
plt.close()
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Gắn nhãn trục có đơn vị cho đường nhiệt độ theo thời gian, rồi đọc lại nhãn đã đặt.
# Dữ liệu: Hai thời điểm 1,2 giờ, nhiệt độ 22,24°C; trục x giờ và trục y độ C.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chọn backend Agg vẽ ảnh raster KHÔNG cần cửa sổ GUI, phù hợp Windows headless/CI; phải đặt TRƯỚC import pyplot.
matplotlib.use('Agg')
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt

# Vẽ hai cặp (thời gian giờ, nhiệt độ °C), nối hai mốc theo thứ tự.
plt.plot([1, 2], [22, 24])
# xlabel() đặt tên TRỤC x, ghi rõ đơn vị giờ để người đọc không nhầm với ngày.
```

```
plt.xlabel('Time (hours)')
# ylabel() đặt tên TRỤC y, đơn vị nhiệt độ độ C (chữ C trong nhãn).
plt.ylabel('Temperature (C)')
# gca() lấy Axes hiện tại, get_xlabel() đọc CHUỖI tên trục x để xác nhận đã gắn; không đọc dữ liệu đường.
print(plt.gca().get_xlabel())
# Đóng figure sau sử dụng để thu hồi bộ nhớ.
plt.close()
```

Kết quả khi chạy:

```
Time (hours)
```

Trục x được gắn đúng ý nghĩa thời gian; phải đặt cả y và đơn vị trước khi gửi biểu đồ.

5. Năm tình huống Matplotlib — từ đường 10 ngày đến hai đồ thị 20 cảm biến

Tình huống 1 — Cấp 1: Vẽ biểu đồ đường cho doanh thu của 10 ngày

Từ vựng trong mục: Line chart — đồ thị đường; **X-axis** — trục ngang; **Savefig** — lưu hình.

Tình huống thực tế: Cửa hàng lưu **doanh thu của 10 ngày liên tiếp**, ngày 1 đến ngày 10, mỗi số là **triệu đồng/ngày**. Người quản lý muốn biết doanh thu biến động ra sao theo đúng **thứ tự thời gian** và đọc doanh thu ngày cuối, thay vì chỉ nhìn một bảng 10 số. Cần tạo **biểu đồ đường**, ghi tên trục và lưu hình **PNG** để chèn vào báo cáo.

Vì sao chọn cách này? Biểu đồ đường (line chart) phù hợp khi trục ngang có **thứ tự thời gian**; nối các điểm liên tiếp giúp thấy ngày nào tăng hoặc giảm so với ngày trước. `ax.plot(days, revenues, marker='o')` ghép đúng **10 cặp ngày–doanh thu**, `ax.set` đặt tên trục **ngày** và **triệu đồng**, còn `savefig` ghi thành **tệp PNG**. `matplotlib.use('Agg')` chọn cách vẽ **không cần mở cửa sổ**, giúp chạy trong môi trường kiểm thử tự động; `plt.close(fig)` giải phóng hình sau khi lưu. Đường nối chỉ là cách hiển thị xu hướng giữa các ngày đã đo, không có nghĩa có dữ liệu thực tế ở mọi thời điểm nằm giữa.

Quy trình:

1. Tạo 10 giá trị doanh thu tương ứng 10 ngày.
2. Vẽ biểu đồ đường, tiêu đề và nhãn trục có đơn vị.
3. Lưu `sales_10_days.png`, kiểm 10 điểm trong đường và file tồn tại.

No Note — Không chú thích

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pathlib import Path

days = list(range(1,11))
revenues = [10,12,11,14,15,14,17,18,16,20]
fig, ax = plt.subplots()
ax.plot(days, revenues, marker="o")
ax.set(title="Sales over 10 days", xlabel="Day", ylabel="Revenue (million VND)")
fig.tight_layout()
path = Path("sales_10_days.png")
fig.savefig(path, dpi=130)
```

```
points = len(ax.lines[0].get_xdata())
plt.close(fig)
if len(revenues) != 10:
    raise ValueError("Cần đủ 10 ngày doanh thu.")
print("Ngày:", len(days))
print("Điểm đường:", points)
print("Doanh thu ngày 10 (triệu đồng):", revenues[-1])
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Vẽ biểu đồ đường doanh thu 10 ngày, lưu PNG và kiểm đủ 10 điểm dữ liệu.
# Dữ liệu: 10 ngày, doanh thu từng ngày đơn vị triệu VND; lưu ảnh sales_10_days.png vào thư mục làm
việc hiện tại.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chuyển sang backend Agg không cần cửa sổ màn hình, tạo ảnh raster để lưu file; gọi
TRƯỚC pyplot.
matplotlib.use("Agg")
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt
# Path cho thao tác tên file ảnh theo hệ điều hành và kiểm sự tồn tại/kích thước file.
from pathlib import Path

# range(1,11) tạo ngày 1..10, không lấy ngày 11; list() hiện thực dãy.
days = list(range(1,11))
# 10 giá trị doanh thu triệu đồng, khớp cùng vị trí với days.
revenues = [10,12,11,14,15,14,17,18,16,20]
# subplots() tạo (fig,ax): Figure là khung ảnh toàn bộ, Axes là vùng dữ liệu để vẽ.
fig, ax = plt.subplots()
# ax.plot() nối dữ liệu theo ngày; marker='o' thêm chấm tròn ngay các mốc quan sát, không thêm ngày
mới.
ax.plot(days, revenues, marker="o")
# ax.set() đặt cùng lúc tiêu đề và nhãn trục; xlabel Day, ylabel Revenue (million VND) nêu đơn vị.
ax.set(title="Sales over 10 days", xlabel="Day", ylabel="Revenue (million VND)")
# tight_layout() điều chỉnh khoảng cách để tiêu đề/nhãn trục ít bị cắt khi lưu PNG.
fig.tight_layout()
# Path() là đường dẫn TƯƠNG ĐỐI theo cwd, không phải đường dẫn tuyệt đối hoặc thư mục tạm.
path = Path("sales_10_days.png")
# savefig() ghi ảnh PNG tới path; dpi=130 là mật độ điểm ảnh mỗi inch khi raster hóa, không phải số
điểm dữ liệu.
fig.savefig(path, dpi=130)
# ax.lines chứa các Line2D; [0] là đường đầu, get_xdata() trả dãy ngày, len() đếm 10 tọa độ.
points = len(ax.lines[0].get_xdata())
# close(fig) đóng ĐÚNG Figure fig đã lưu để giải phóng tài nguyên dù file PNG vẫn còn trên đĩa.
plt.close(fig)
# Kiểm đủ 10 ngày doanh thu theo kịch bản; cũng nên giữ days khớp khi tạo dữ liệu khác.
if len(revenues) != 10:
    # Ném ValueError khi thiếu giá trị doanh thu, ngăn báo cáo sốliệu chưa đủ.
    raise ValueError("Cần đủ 10 ngày doanh thu.")
print("Ngày:", len(days))
print("Điểm đường:", points)
# [-1] lấy phần tửcuối list doanh thu (ngày 10), không phải sốngày.
print("Doanh thu ngày 10 (triệu đồng):", revenues[-1])
# is_file() kiểm đường dẫn là file; stat().st_size đo byte >0 xác nhận file ảnh có nội dung, không
chứng minh nội dung đẹp hay đúng ngữ nghĩa.
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Kết quả khi chạy:

```
Ngày: 10
Điểm đường: 10
```

```
Doanh thu ngày 10 (triệu đồng): 20
Đã lưu PNG: True
```

Chạy code sẽ tạo **sales_10_days.png** trong thư mục làm việc. Điểm giữa hai ngày trên đường nối không có nghĩa cửa hàng đã ghi dữ liệu cho mọi thời khắc giữa hai ngày.

Tình huống 2 — Cấp 2: Scatter 12 học viên để xem mối liên hệ giờ học–điểm

Từ vựng trong mục: Scatter plot — đồ thị phân tán; **Pair** — cặp quan sát.

Tình huống thực tế: Giáo viên ghi **12 cặp dữ liệu học viên**. Mỗi người có **số giờ tự học (giờ/tuần)** và **điểm kiểm tra trên thang 10**, ví dụ hai học viên có cùng 2 giờ nhưng đạt điểm khác nhau. Giáo viên muốn quan sát điểm có xu hướng tăng khi giờ học tăng không, mà vẫn giữ **một dấu chấm cho mỗi người**; bảng không có chuỗi thời gian để nối các học viên thành đường.

Vì sao chọn cách này? Scatter plot (biểu đồ phân tán) đặt **một dấu chấm cho từng học viên**, với trục x là **giờ/tuần** và trục y là **điểm trên thang 10**. `ax.scatter(hours, scores)` tạo đúng 12 vị trí; hai học viên có cùng giờ học nhưng khác điểm vẫn là **hai quan sát**, không nên nối chúng thành đường thời gian. Tên trục kèm đơn vị giúp tránh việc hiểu 4 giờ như 4 điểm. Nếu các chấm có xu hướng đi lên, đó chỉ là **mối liên hệ trong mẫu**, chưa chứng minh tự học nhiều là **nguyên nhân duy nhất** khiến điểm cao.

Quy trình:

1. Ghi 12 giờ học và 12 điểm theo đúng cặp học viên.
2. Vẽ scatter với tên trục/đơn vị, không nối điểm.
3. Lưu `study_vs_score.png`, kiểm scatter thực sự chứa 12 hàng.

No Note — Không chú thích

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pathlib import Path

hours = [1,2,2,3,3,4,4,5,5,6,7,8]
scores = [4,5,6,5,7,6,8,7,8,9,8,10]
fig, ax = plt.subplots()
points = ax.scatter(hours, scores)
ax.set(title="Study hours and score", xlabel="Study (hours/week)", ylabel="Score (0-10)")
ax.set_ylim(0, 11)
fig.tight_layout()
path = Path("study_vs_score.png")
fig.savefig(path, dpi=130)
count = len(points.get_offsets())
plt.close(fig)
if len(hours) != 12 or len(scores) != 12:
    raise ValueError("Cần đủ 12 học viên và 12 điểm.")
print("Học viên:", len(hours))
print("Điểm scatter:", count)
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Vẽ scatter giữa giờ học và điểm của 12 học viên rồi lưu file PNG.
# Dữ liệu: 12 cặp (giờ học/tuần, điểm 0–10), có người trùng giờ học nhưng điểm khác nhau.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chuyển sang backend Agg không cần cửa sổ màn hình, tạo ảnh raster để lưu file; gọi
```

```
TRƯỚC pyplot.
matplotlib.use("Agg")
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt
# Path() quản lý đường dẫn file ảnh và kiểm file đã ghi.
from pathlib import Path

# List 12 giờ học/tuần, mỗi vị trí là một học viên.
hours = [1,2,2,3,3,4,4,5,5,6,7,8]
# List 12 điểm 0–10 cùng vị trí với giờ học tương ứng.
scores = [4,5,6,5,7,6,8,7,8,9,8,10]
# Tạo Figure fig và vùng vẽ Axes ax để chỉnh nhãn/lưu.
fig, ax = plt.subplots()
# scatter() tạo PathCollection các chấm rời; `points` lưu đối tượng scatter, không phải list tọa độ số.
points = ax.scatter(hours, scores)
# Đặt tiêu đề và nhãn trục; x là giờ/tuần, y là điểm 0–10.
ax.set(title="Study hours and score", xlabel="Study (hours/week)", ylabel="Score (0-10)")
# set_ylim(ymin,ymax) khóa khoảng HIỂN THỊ trục y 0..11 để thấy cả điểm 10, không thay scores.
ax.set_ylim(0, 11)
# Chỉnh bố cục để tránh nhãn bị cắt ở mép PNG.
fig.tight_layout()
# Tên file tương đối so với thư mục làm việc hiện tại.
path = Path("study_vs_score.png")
# Lưu scatter thành PNG với dpi=130, không ảnh hưởng số học viên.
fig.savefig(path, dpi=130)
# PathCollection.get_offsets() trả tọa độ điểm shape (12,2); len() đếm 12 cặp thay vì đếm pixel ảnh.
count = len(points.get_offsets())
# Đóng figure để giải phóng đối tượng giao diện/đồ họa sau khi lưu.
plt.close(fig)
# or báo lỗi nếu MỘT trong hai list thiếu 12 phần tư; số học viên và điểm phải đồng bộ.
if len(hours) != 12 or len(scores) != 12:
    # Dừng trước khi báo cáo khi số lượt học không đủ.
    raise ValueError("Cần đủ 12 học viên và 12 điểm.")
print("Học viên:", len(hours))
print("Điểm scatter:", count)
# Kiểm có file PNG thật và số byte >0; không chỉ kiểm có tên file.
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Kết quả khi chạy:

```
Học viên: 12
Điểm scatter: 12
Đã lưu PNG: True
```

Trùng vị trí scatter có thể che lên nhau; số điểm đếm từ dữ liệu nguồn và scatter không được hiểu là số vị trí **nhìn thấy bằng mắt**.

Tình huống 3 — Cấp 3: Histogram 15 thời gian giao theo bốn khoảng phút

Từ vựng trong mục: Histogram — biểu đồ tần suất; **Bins** — các khoảng giá trị.

Tình huống thực tế: Đội giao nhận có **15 thời gian giao hàng**, tính bằng **phút/chuyến**, từ 4 đến 37 phút. Trưởng nhóm muốn biết có bao nhiêu chuyến thuộc từng **khoảng thời gian 0–10, 10–20, 20–30 và 30–40 phút** để hiểu phân bố nhanh/chậm. Họ không cần xem chuyến số mấy xảy ra trước; cần biểu đồ có tổng số lượt trong các cột vẫn bằng 15.

Vì sao chọn cách này? Histogram (biểu đồ tần suất theo khoảng) chia trục thời gian giao thành những **khoảng (bins)** rồi đếm **số chuyến** thuộc mỗi khoảng, không nối từng chuyến theo thứ tự.

`ax.hist(minutes, bins=[0,10,20,30,40])` tạo **bốn cột**; với các khoảng kề nhau của Matplotlib, mốc 10 đi vào khoảng **10–20**, còn khoảng cuối nhận cả mốc phải 40. Cần kiểm `counts.sum()` bằng **15**, vì nếu một số nằm ngoài toàn bộ các khoảng thì có thể không được đếm. Trục ngang vẫn là **phút**, còn độ cao cột là **số lượt**, không phải doanh thu hoặc tổng phút.

Quy trình:

1. Tạo 15 thời gian có số, định nghĩa ranh giới 0/10/20/30/40 phút.
2. Vẽ histogram, nhãn hai trục và lưu hình.
3. Kiểm tổng tần suất của bốn khoảng đúng 15 chuyến.

No Note — Không chú thích

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pathlib import Path

minutes = [4,7,8,9,10,11,14,16,18,20,21,24,27,31,37]
fig, ax = plt.subplots()
counts, edges, bars = ax.hist(minutes, bins=[0,10,20,30,40], edgecolor="black")
ax.set(title="Delivery duration", xlabel="Duration (minutes)", ylabel="Number of trips")
fig.tight_layout()
path = Path("delivery_histogram.png")
fig.savefig(path, dpi=130)
plt.close(fig)
if len(minutes) != 15 or int(counts.sum()) != 15:
    raise ValueError("Cần đủ 15 chuyến trong các cột tần suất.")
print("Chuyến:", len(minutes))
print("Số khoảng:", len(counts))
print("Tần suất:", counts.astype(int).tolist())
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Vẽ histogram 15 thời gian giao hàng trong bốn khoảng 0–10,10–20,20–30,30–40 phút.
# Dữ liệu: 15 chuyến giao phút, histogram đếm số chuyến theo khoảng, không biểu diễn trình tự thời
gian.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chuyển sang backend Agg không cần cửa sổ màn hình, tạo ảnh raster để lưu file; gọi
TRƯỚC pyplot.
matplotlib.use("Agg")
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt
# Dùng Path để đặt tên ảnh PNG và xác minh file được tạo.
from pathlib import Path

# List 15 độ dài chuyến giao hàng, đơn vị phút.
minutes = [4,7,8,9,10,11,14,16,18,20,21,24,27,31,37]
# Tạo Figure và Axes mới để không trộn biểu đồ với bài khác.
fig, ax = plt.subplots()
# hist() chia vào bốn khoảng [0,10),[10,20),[20,30),[30,40] (khoảng cuối gồm biên phải). Trả
counts=tần suất, edges=mốc biên, bars=hình cột; edgecolor='black' vẽ viền đen, không đổi số liệu.
counts, edges, bars = ax.hist(minutes, bins=[0,10,20,30,40], edgecolor="black")
# Đặt tên biểu đồ, nhãn x là phút, y là số chuyến (tần suất), không phải mật độ khi không đặt
density=True.
ax.set(title="Delivery duration", xlabel="Duration (minutes)", ylabel="Number of trips")
# Canh bố cục để nhãn/tiêu đề hiển thị đầy đủ.
fig.tight_layout()
```

```
# Tên PNG tương đối trong cwd hiện tại.
path = Path("delivery_histogram.png")
# Ghi histogram ra file PNG với dpi=130.
fig.savefig(path, dpi=130)
# Đóng figure sau lưu để không giữ ảnh dư trong bộ nhớ.
plt.close(fig)
# len() đếm 15 chuyến nguồn; counts.sum() cộng TẤT CẢ tần suất các bin, phải đủ 15 (các bin bao
trùm mọi số đo).
if len(minutes) != 15 or int(counts.sum()) != 15:
    # Dừng nếu có lượt ngoài bin hoặc thiếu bản ghi.
    raise ValueError("Cần đủ 15 chuyến trong các cột tần suất.")
print("Chuyến:", len(minutes))
# Có một phần tử counts cho mỗi KHOẢNG, tức 4 khoảng, khác len(edges)=5 mốc biên.
print("Số khoảng:", len(counts))
# counts là ndarray số thực, astype(int) chuyển đếm lượt thành số nguyên để in; tolist() chuyển
thành list Python.
print("Tần suất:", counts.astype(int).tolist())
# Kiểm ảnh PNG tồn tại và lớn hơn 0 byte.
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Kết quả khi chạy:

```
Chuyến: 15
Số khoảng: 4
Tần suất: [4, 5, 4, 2]
Đã lưu PNG: True
```

Histogram khác biểu đồ cột danh mục: trục ngang biểu diễn **khoảng số phút**, không phải tên từng khách.

Tình huống 4 — Cấp 4: Hai đường của 18 ngày: so sánh doanh thu hai chi nhánh

Từ vựng trong mục: Legend — chú giải; **Multiple series** — nhiều chuỗi; **Axis labels** — nhãn trục.

Tình huống thực tế: Doanh nghiệp theo dõi **18 ngày liên tiếp**, mỗi ngày ghi doanh thu của **chi nhánh A và chi nhánh B** trên cùng ngày, cùng đơn vị **triệu đồng/ngày**. Giám đốc muốn so xu hướng tăng giảm của **hai chi nhánh trên chung một trục thời gian**, xem A đạt bao nhiêu vào ngày 18, và lưu hình có **chú giải nhận diện hai đường** để người xem không nhầm.

Vì sao chọn cách này? Biểu đồ nhiều đường là cách so hai dãy có **cùng mốc thời gian và đơn vị đo**: doanh thu A và B trên đủ 18 ngày. Hai lệnh `ax.plot(days, branch_a, label='Branch A')` và tương tự cho B dùng **cùng mảng ngày**, tránh làm lệch hàng dữ liệu. `ax.legend()` hiện **chú giải** để biết đường nào thuộc chi nhánh nào; các kiểu dấu khác nhau cũng hỗ trợ phân biệt điểm. Chương trình kiểm cả **hai đường đều có 18 điểm**, in doanh thu A ngày cuối rồi `savefig` lưu PNG. Đồ thị cho biết xu hướng quan sát, không chứng minh nguyên nhân khiến doanh thu chi nhánh nào đó tăng.

Quy trình:

1. Tạo 18 cặp doanh thu A/B, dùng chung dãy ngày 1–18.
2. Vẽ hai đường có label riêng, thêm legend và tên trục.
3. Lưu `branches_18_days.png` và kiểm mỗi đường có đúng 18 điểm.

No Note — Không chú thích

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pathlib import Path
```

```
days = list(range(1,19))
branch_a = [10,12,14,13,15,17,16,18,20,19,21,23,22,24,26,25,27,29]
branch_b = [9,11,13,15,13,15,17,19,17,19,21,23,21,23,25,27,25,27]
fig, ax = plt.subplots()
ax.plot(days, branch_a, marker="o", label="Branch A")
ax.plot(days, branch_b, marker="s", label="Branch B")
ax.set(title="Branch revenue", xlabel="Day", ylabel="Revenue (million VND)")
ax.legend()
fig.tight_layout()
path = Path("branches_18_days.png")
fig.savefig(path, dpi=130)
count = []
for line in ax.lines:
    count.append(len(line.get_xdata()))
plt.close(fig)
if len(days) != 18 or len(branch_a) != 18 or len(branch_b) != 18:
    raise ValueError("Cần đủ 18 ngày và hai dãy doanh thu.")
print("Ngày:", len(days))
print("Đường:", len(count))
print("Điểm mỗi đường:", count)
print("A ngày 18:", branch_a[-1])
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So hai đường doanh thu trong 18 ngày, thêm legend và kiểm mỗi đường có đủ 18 điểm.
# Dữ liệu: 18 ngày, hai dãy doanh thu Branch A/B tính triệu VND; cả hai dùng cùng trục ngày.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chuyển sang backend Agg không cần cửa sổ màn hình, tạo ảnh raster để lưu file; gọi
TRƯỚC pyplot.
matplotlib.use("Agg")
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt
# Path biểu diễn file xuất đồ thị theo đường dẫn tương đối.
from pathlib import Path

# Tạo ngày từ 1 tới 18, không lấy 19.
days = list(range(1,19))
# 18 số doanh thu chi nhánh A, đơn vị triệu VND.
branch_a = [10,12,14,13,15,17,16,18,20,19,21,23,22,24,26,25,27,29]
# 18 số doanh thu chi nhánh B, theo đúng thứ tự ngày như A.
branch_b = [9,11,13,15,13,15,17,19,17,19,21,23,21,23,25,27,25,27]
# Tạo Figure và Axes riêng cho biểu đồ hai đường.
fig, ax = plt.subplots()
# Vẽ đường A, marker='o' dùng điểm tròn, label đặt tên đường để legend tra được.
ax.plot(days, branch_a, marker="o", label="Branch A")
# Vẽ đường B, marker='s' là ô vuông, hai đường dùng chung trục x nhưng phân biệt nhãn.
ax.plot(days, branch_b, marker="s", label="Branch B")
# Đặt tiêu đề và nhãn đơn vị trục doanh thu triệu VND.
ax.set(title="Branch revenue", xlabel="Day", ylabel="Revenue (million VND)")
# legend() hiển thị chú giải lấy từ label của mỗi Line2D, không tự tạo dữ liệu.
ax.legend()
# Điều chỉnh khoảng cách nhãn và vùng hình trước khi lưu.
fig.tight_layout()
# Tên file PNG của biểu đồ hai chi nhánh trong cwd.
path = Path("branches_18_days.png")
# Lưu file ảnh PNG ở độ phân giải dpi=130.
fig.savefig(path, dpi=130)
# List rỗng ghi số mốc dữ liệu trên mỗi đường, không lưu tổng pixel.
count = []
# ax.lines là danh sách hai đối tượng Line2D đã vẽ; lặp qua từng đường.
for line in ax.lines:
    # get_xdata() lấy dãy x của Line2D tương ứng; len() đếm 18 ngày trên đường đó.
```

```
    # append() thêm số điểm của mỗi Line2D vào list để đối chiếu.
    count.append(len(line.get_xdata()))
# Đóng figure sau khi đọc các đường và lưu file, tránh rò rỉ bộ nhớ đồ họa.
plt.close(fig)
# or yêu câù cả ba dãy ngày/A/B có đúng 18 phần tử.
if len(days) != 18 or len(branch_a) != 18 or len(branch_b) != 18:
    # Báo lỗi nếu bất kỳ dãy nào thiếu mốc, không tiếp tục báo cáo.
    raise ValueError("Cần đủ 18 ngày và hai dãy doanh thu.")
print("Ngày:", len(days))
print("Đường:", len(count))
print("Điểm mỗi đường:", count)
# [-1] là doanh thu ngày 18 của A, không phải doanh thu trung bình.
print("A ngày 18:", branch_a[-1])
# Xác minh file hình đã được ghi thực và không rỗng.
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Kết quả khi chạy:

```
Ngày: 18
Đường: 2
Điểm mỗi đường: [18, 18]
A ngày 18: 29
Đã lưu PNG: True
```

Đồ thị cho thấy **xu hướng mô tả**, không tự chứng minh nguyên nhân doanh thu giữa hai chi nhánh chênh lệch.

Tình huống 5 — Cấp 5: Vẽ hai biểu đồ cho 20 lượt cảm biến, tô điểm lệch chuẩn

Từ vựng trong mục: Dashboard — hai góc nhìn; **Residual** — sai lệch; **Highlight** — tô điểm cần xem.

Tình huống thực tế: Trạm hiệu chuẩn lưu **20 lượt đo tín hiệu**, mỗi lượt có mức đầu vào x và mức tín hiệu thực tế. Quy tắc tham chiếu giả lập là **tín hiệu kỳ vọng tăng hai đơn vị cho mỗi một mức x và bắt đầu từ 5 đơn vị**. Lượt **M18** cao hơn tham chiếu 30 đơn vị, còn các lượt khác khớp. Kỹ thuật viên muốn một hình gồm **biểu đồ điểm đo so với đường tham chiếu** và **histogram sai số**, đồng thời đánh dấu mã cần rà soát mà không tự xóa lượt khác thường.

Vì sao chọn cách này? Sai số dư (residual) là số đo **thực tế trừ mức dự kiến** ở **cùng một x**. Chương trình tạo **hai biểu đồ cạnh nhau**: bên trái `scatter` cho 20 điểm thực tế cùng đường tham chiếu, đồng thời tô dấu những điểm có **độ lớn sai số vượt 10 đơn vị**; bên phải `hist` gom sai số thành các khoảng để nhìn số lượt sai lệch nhỏ/lớn. Với dữ liệu này, **M18** có sai số +30 nên bị gắn cờ, nhưng chưa kết luận cảm biến hỏng: cần xem lại phép đo. `np.flatnonzero(flagged)` trả vị trí chỉ số **bắt đầu từ 0**, phải cộng 1 để ra mã M01…; `plt.subplots(1,2)` và `savefig` cho một PNG báo cáo, `Agg` tránh yêu cầu màn hình. Histogram bổ sung góc nhìn về **tần suất sai số**, không thay bảng đo hay xóa ngoại lệ.

Quy trình:

1. Tạo 20 cặp x/y và đường tham chiếu theo quy định hiệu chuẩn.
2. Tìm sai số, gắn cờ nếu trị tuyệt đối >10, vẽ scatter và đường chuẩn.
3. Vẽ histogram sai số ở ô thứ hai, gắn nhãn/trục, lưu hình và báo số điểm lệch.

No Note — Không chú thích

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from pathlib import Path

import numpy as np
x = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20], dtype=float)
observed = np.array([7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,71,43,45], dtype=float)
reference = 2*x + 5
residual = observed - reference
flagged = np.abs(residual) > 10
fig, (left,right) = plt.subplots(1,2,figsize=(10,4))
left.scatter(x,observed,label="Observed")
left.plot(x,reference,label="Reference")
left.scatter(x[flagged],observed[flagged],marker="x",s=90,label="Review")
left.set(xlabel="Input level",ylabel="Signal (units)",title="Measured vs reference")
left.legend()
right.hist(residual,bins=[-1,1,10,20,40],edgecolor="black")
right.set(xlabel="Residual (units)",ylabel="Count",title="Residual distribution")
fig.tight_layout()
path = Path("sensor_review_20.png")
fig.savefig(path,dpi=130)
plt.close(fig)
if len(x) != 20 or len(observed) != 20:
    raise ValueError("Cần đủ 20 mức đầu vào và tín hiệu đo.")
print("Lượt đo:", len(x))
print("Biểu đồ trong hình:", 2)
print("Điểm cần kiểm:", int(flagged.sum()))
flagged_ids = []
for i in np.flatnonzero(flagged):
    flagged_ids.append(f"M{i+1:02d}")
print("Mã cần kiểm:", ", ".join(flagged_ids))
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo hình hai biểu đồ: đo so chuẩn với điểm bất thường và histogram sai lệch trên 20
lượt.
# Dữ liệu: 20 cặp x/observed, chuẩn reference=2x+5; lượt M18 có độ lệch lớn, cảnh báo khi |
residual|>10.
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
import matplotlib
# use('Agg') chuyển sang backend Agg không cần cửa sổ màn hình, tạo ảnh raster để lưu file; gọi
TRƯỚC pyplot.
matplotlib.use("Agg")
# Nhập thư viện Matplotlib để chọn backend trước khi nhập pyplot.
# pyplot là giao diện vẽ đồ thị dùng tên ngắn plt; các lệnh plt thao tác trên figure/axes hiện tại.
import matplotlib.pyplot as plt
# Path đặt tên PNG và cung cấp is_file()/stat() để kiểm kết quả lưu.
from pathlib import Path

# NumPy giúp tính reference, residual và mask cảnh báo đồng thời trên 20 mẫu.
import numpy as np
# Tạo x mảng float từ 1 đến 20, mỗi phần tử là mức đầu vào tương ứng một lượt đo.
x = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20], dtype=float)
# 20 tín hiệu thực đo, cùng thứ tự với x; một giá trị 71 cao bất thường ở lượt 18.
observed = np.array([7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,71,43,45], dtype=float)
# NumPy broadcast phép nhân và cộng cho mọi x để dựng đường chuẩn y=2x+5.
reference = 2*x + 5
# Trừ THEO TỪNG VỊ TRÍ để tạo residual (đo trừ chuẩn), giữ đơn vị của tín hiệu.
residual = observed - reference
# abs() bỏ dấu sai số, >10 tạo mask True cho lượt đo cách đường chuẩn hơn 10 đơn vị.
flagged = np.abs(residual) > 10
# subplots(1,2) tạo MỘT Figure có hai Axes nằm ngang; figsize=(10,4) đặt kích thước hình theo inch,
không phải số quan sát.
fig, (left,right) = plt.subplots(1,2,figsize=(10,4))
# Vẽ 20 điểm đo trên Axes trái; label dùng cho legend.
left.scatter(x,observed,label="Observed")
```

```
# Vẽ đường tham chiếu y=2x+5 để so với các chấm quan sát.
left.plot(x,reference,label="Reference")
# Boolean indexing chỉ lấy các điểm có mask True; marker='x' là dấu nhân, s=90 là diện tích marker
tính pt², không phải giới hạn số điểm.
left.scatter(x[flagged],observed[flagged],marker="x",s=90,label="Review")
# Đặt nhãn và đơn vị biểu đồ trái: input và signal.
left.set(xlabel="Input level",ylabel="Signal (units)",title="Measured vs reference")
# Hiện chú giải Observed/Reference/Review theo label từng đối tượng.
left.legend()
# Histogram residual chia vào bốn khoảng từ -1 tới 40; bins là các BIÊN do người soạn chọn,
edgecolor vẽ viền đen; cần bảo đảm dữ liệu nằm trong phạm vi này.
right.hist(residual,bins=[-1,1,10,20,40],edgecolor="black")
# Nhãn biểu đồ phải là residual (đơn vị tín hiệu) và Count (số lượt trong từng khoảng).
right.set(xlabel="Residual (units)",ylabel="Count",title="Residual distribution")
# Căn cả hai Axes để nhãn không chồng khi lưu.
fig.tight_layout()
# Tên PNG tương đối so với working directory.
path = Path("sensor_review_20.png")
# Ghi Figure chứa cả hai đồ thị ra PNG với dpi=130.
fig.savefig(path,dpi=130)
# Đóng Figure sau ghi, thu hồi bộ nhớ; file PNG không bị xóa.
plt.close(fig)
# Kiểm đủ 20 mức đầu vào và 20 số đo theo cùng thứ tự.
if len(x) != 20 or len(observed) != 20:
    # Dừng nếu thiếu số đo để tránh đánh dấu sai.
    raise ValueError("Cần đủ 20 mức đầu vào và tín hiệu đo.")
print("Lượt đo:", len(x))
print("Biểu đồ trong hình:", 2)
# sum() trên mask Boolean đếm các giá trị True (lượt cảnh báo); int() đổi sang số nguyên Python.
print("Điểm cần kiểm:", int(flagged.sum()))
flagged_ids = []
# flatnonzero() trả chỉ số 0-based của các ô True trong mask phẳng, dùng tìm mã lượt cần kiểm.
for i in np.flatnonzero(flagged):
    # Chỉ số NumPy bắt đầu từ 0 nên +1 ra số lượt, :02d định dạng M01..M20.
    # append() lưu mã của lượt cảnh báo vào list.
    flagged_ids.append(f"M{i+1:02d}")
# join() ghép các mã cần kiểm bằng dấu phẩy/không gian.
print("Mã cần kiểm:", ", ".join(flagged_ids))
# Kiểm PNG đã tồn tại và có dữ liệu byte >0, không chỉ giả định savefig luôn thành công.
print("Đã lưu PNG:", path.is_file() and path.stat().st_size > 0)
```

Kết quả khi chạy:

```
Lượt đo: 20
Biểu đồ trong hình: 2
Điểm cần kiểm: 1
Mã cần kiểm: M18
Đã lưu PNG: True
```

Điểm M18 vẫn có mặt trên biểu đồ và dữ liệu đầu vào. Vượt ngưỡng minh họa chỉ là lý do kiểm tra nguồn đo, **không kết luận cảm biến hỏng**.

6. Sai lầm thường gặp

- Dùng line chart để nối dữ liệu không có thứ tự có thể tạo cảm giác xu hướng giả.
- Scatter phải ghép đúng x và y của **cùng hàng mẫu**; không suy ra nhân quả chỉ từ mối tương quan.
- Histogram phụ thuộc `bins`; ghi biên khoảng khi báo cáo.
- Biểu đồ thiếu đơn vị dễ khiến người xem nhầm thời gian, tiền hoặc nhiệt độ.
- Lưu PNG không thay thế việc lưu bảng dữ liệu gốc và script tạo hình.

- `matplotlib` là thư viện ngoài; với máy không có màn hình có thể dùng backend Agg và `close()` sau khi vẽ.

7. Trắc nghiệm

Q1. What is a line chart best suited for? | Biểu đồ đường phù hợp gì?

A. Showing changes across ordered positions | Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations | Từng quan sát theo cặp x-y

C. Counts of numeric values within bins | Số quan sát số học trong từng khoảng

D. Naming variables and their units | Đặt tên đại lượng và đơn vị đo

Đáp án: A

Giải thích: Nối các mốc cho xu hướng, không đảm bảo có số đo giữa các mốc. Nếu dữ liệu là các loại không có thứ tự, biểu đồ cột phù hợp hơn việc nối đường.

Q2. What does a scatter plot represent? | Scatter plot biểu diễn gì?

A. Showing changes across ordered positions | Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations | Từng quan sát theo cặp x-y

C. Counts of numeric values within bins | Số quan sát số học trong từng khoảng

D. Naming variables and their units | Đặt tên đại lượng và đơn vị đo

Đáp án: B

Giải thích: Mỗi điểm là x và y của cùng một bản ghi. Mảng x và y phải cùng thứ tự bản ghi, nếu không các điểm ghép sẽ sai.

Q3. What does a histogram represent? | Histogram biểu diễn gì?

A. Showing changes across ordered positions | Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations | Từng quan sát theo cặp x-y

C. Counts of numeric values within bins | Số quan sát số học trong từng khoảng

D. Naming variables and their units | Đặt tên đại lượng và đơn vị đo

Đáp án: C

Giải thích: Histogram chia khoảng giá trị số, không giống biểu đồ cột loại. Đổi biên bins sẽ đổi số đếm; khi so histogram cần nêu rõ khoảng chia.

Q4. What are axis labels for? | Nhãn trục để làm gì?

A. Showing changes across ordered positions | Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations | Từng quan sát theo cặp x-y

C. Counts of numeric values within bins | Số quan sát số học trong từng khoảng

D. Naming variables and their units | Đặt tên đại lượng và đơn vị đo

Đáp án: D

Giải thích: Nhãn tránh nhầm đơn vị thời gian, nhiệt độ hay tiền. Con số trên trục thiếu đơn vị khiến người xem không rõ ý nghĩa đo lường.

Q5. Which function draws a line? | Hàm nào vẽ đường?

A. `plt.plot(x,y)` | `plt.plot(x,y)`

B. `plt.scatter(x,y)` | `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` | `plt.hist(values,bins=...)`

D. `plt.savefig(path)` | `plt.savefig(path)`

Đáp án: A

Giải thích: plot nối điểm theo thứ tự đầu vào mặc định. Hàm vẽ đòi x,y tương ứng nhưng không tự kiểm dữ liệu có đúng ý nghĩa thực tế.

Q6. Which function draws separate x-y points? | Hàm nào vẽ các điểm x-y riêng?

A. `plt.plot(x,y)` | `plt.plot(x,y)`

B. `plt.scatter(x,y)` | `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` | `plt.hist(values,bins=...)`

D. `plt.savefig(path)` | `plt.savefig(path)`

Đáp án: B

Giải thích: scatter tạo dấu chấm riêng cho từng cặp quan sát. Đây là lựa chọn phù hợp khi các điểm là quan sát rời, không phải chuỗi thời gian cần nối.

Q7. Which function draws histograms? | Hàm nào vẽ histogram?

A. `plt.plot(x,y)` | `plt.plot(x,y)`

B. `plt.scatter(x,y)` | `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` | `plt.hist(values,bins=...)`

D. `plt.savefig(path)` | `plt.savefig(path)`

Đáp án: C

Giải thích: hist đếm trong từng khoảng rồi vẽ cột các khoảng số. `bins` xác định biên hoặc số khoảng; thay đổi tham số này làm thay đổi bản tóm tắt phân bố.

Q8. Which function saves a figure? | Hàm nào lưu hình?

A. `plt.plot(x,y)` | `plt.plot(x,y)`

B. `plt.scatter(x,y)` | `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` | `plt.hist(values,bins=...)`

D. `plt.savefig(path)` | `plt.savefig(path)`

Đáp án: D

Giải thích: savefig ghi hình đã vẽ ra tệp PNG. Phải lưu trước khi đóng Figure và kiểm đường dẫn cùng quyền ghi.

Q9. Why should x and y have matching observation counts? | Vì sao x và y phải có số quan sát tương ứng?

A. Each x needs its matching y | Mỗi x cần y tương ứng

B. Closes the current Figure | Đóng Figure hiện tại

C. Horizontal-axis label | Nhãn trục ngang

D. Vertical-axis label | Nhãn trục dọc

Đáp án: A

Giải thích: Không ghép được cặp khi thiếu phần tử tương ứng. Dãy lệch chiều có thể gây lỗi hoặc ghép sai đối tượng quan sát.

Q10. What does `plt.close()` do? | `plt.close()` làm gì?

A. Each x needs its matching y | Mỗi x cần y tương ứng

B. Closes the current Figure | Đóng Figure hiện tại

C. Horizontal-axis label | Nhãn trục ngang

D. Vertical-axis label | Nhãn trục dọc

Đáp án: B

Giải thích: Đóng giúp giải phóng tài nguyên và tránh mở nhiều hình. Nên đóng Figure sau khi xuất, nhất là trong vòng lặp tạo nhiều tệp.

Q11. What does `plt.xlabel()` set? | `plt.xlabel()` đặt gì?

A. Each x needs its matching y | Mỗi x cần y tương ứng

B. Closes the current Figure | Đóng Figure hiện tại

C. Horizontal-axis label | Nhãn trục ngang

D. Vertical-axis label | Nhãn trục dọc

Đáp án: C

Giải thích: Nó chỉ đổi chữ, không tự đổi dữ liệu hoặc đơn vị. Nhãn có ích nên ghi cả đại lượng và đơn vị như ngày hay giờ.

Q12. What does `plt.ylabel()` set? | `plt.ylabel()` đặt gì?

A. Each x needs its matching y | Mỗi x cần y tương ứng

B. Closes the current Figure | Đóng Figure hiện tại

C. Horizontal-axis label | Nhãn trục ngang

D. Vertical-axis label | Nhãn trục dọc

Đáp án: D

Giải thích: Nó mô tả y và nên kèm đơn vị. Trục nhiệt độ nên ghi rõ Celsius hoặc Fahrenheit, không chỉ ghi 'value'.

Q13. How many markers represent three scatter records? | Ba bản ghi scatter tạo mấy điểm?

A. Three | Ba

B. Yes, changing bins changes counts per interval | Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer | Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient | Không, chỉ liên hệ chưa đủ

Đáp án: A

Giải thích: Mỗi hàng tạo một điểm. Mỗi điểm tương ứng một sinh viên, không phải mỗi cột đặc trưng tạo một điểm.

Q14. Do histogram counts depend on bin boundaries? | Số đếm histogram có phụ thuộc biên bins không?

A. Three | Ba

B. Yes, changing bins changes counts per interval | Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer | Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient | Không, chỉ liên hệ chưa đủ

Đáp án: B

Giải thích: Cùng dữ liệu có thể được nhóm khác nhau. Chiều cao histogram thường là số lượng mẫu, không mặc nhiên là xác suất nếu chưa chuẩn hóa.

Q15. Is a histogram appropriate for names A, B, C as categories? | Histogram có thích hợp trực tiếp cho nhãn A,B,C không?

A. Three | Ba

B. Yes, changing bins changes counts per interval | Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer | Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient | Không, chỉ liên hệ chưa đủ

Đáp án: C

Giải thích: Histogram cần khoảng số thay vì tên loại. Cột phân loại so các nhóm tên; histogram dùng khoảng liên tục trên trục số.

Q16. Can scatter points establish causation? | Scatter có chứng minh quan hệ nhân quả không?

A. Three | Ba

B. Yes, changing bins changes counts per interval | Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer | Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient | Không, chỉ liên hệ chưa đủ

Đáp án: D

Giải thích: Biến khác hoặc cách lấy mẫu có thể tạo cùng xu hướng. Cần xem cách thiết kế nghiên cứu và yếu tố gây nhiễu trước khi nói một biến gây ra biến kia.

Q17. What does backend Agg help with? | Backend Agg giúp gì?

A. Rendering images without a GUI window | Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis | Thứ tự và đơn vị trục thời gian

C. Array of bin counts | Mảng số lượng theo mỗi bin

D. No, it is a rendered image | Không, nó là ảnh được kết xuất

Đáp án: A

Giải thích: Agg phù hợp chạy script hàng loạt hoặc lưu hình trên server. Agg đặc biệt hữu ích cho kiểm thử tự động không cần màn hình đồ họa.

Q18. What should be checked before interpreting a time plot? | Trước khi đọc biểu đồ thời gian cần kiểm gì?

A. Rendering images without a GUI window | Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis | Thứ tự và đơn vị trục thời gian

C. Array of bin counts | Mảng số lượng theo mỗi bin

D. No, it is a rendered image | Không, nó là ảnh được kết xuất

Đáp án: B

Giải thích: Sai thứ tự hoặc đơn vị khiến xu hướng bị hiểu sai. Nếu ngày bị xếp sai, đường nối có thể khiến người xem tưởng thời gian đi ngược.

Q19. What does `plt.hist` return first? | `plt.hist` trả gì ở vị trí đầu?

A. Rendering images without a GUI window | Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis | Thứ tự và đơn vị trục thời gian

C. Array of bin counts | Mảng số lượng theo mỗi bin

D. No, it is a rendered image | Không, nó là ảnh được kết xuất

Đáp án: C

Giải thích: Giá trị trả về đầu là tần suất, phần sau là biên và đối tượng vẽ. Mảng biên trả về giúp xác định giá trị nào đóng góp vào từng khoảng đếm.

Q20. Does a saved PNG retain an editable dataset automatically? | PNG lưu hình có tự chứa bảng dữ liệu có thể sửa không?

A. Rendering images without a GUI window | Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis | Thứ tự và đơn vị trục thời gian

C. Array of bin counts | Mảng số lượng theo mỗi bin

D. No, it is a rendered image | Không, nó là ảnh được kết xuất

Đáp án: D

Giải thích: Muốn tính lại và sửa biểu đồ cần lưu dữ liệu gốc riêng. Cần lưu script, nhãn và bảng số gốc để tái tạo lại biểu đồ khi cần.

8. Nối sang bài tiếp

`NumPy/Pandas → Visualization → EDA → PCA/ML.`

PY-46 — Mini project: dữ liệu → làm sạch → mô hình → đánh giá

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn.

`Python → NumPy/Pandas → Train/Test → Pipeline → Report`

Học xong dùng để làm gì? Kết hợp chuẩn bị dữ liệu, đặc trưng, huấn luyện mô hình, đánh giá và báo cáo thành một quy trình AI có thể chạy lại.

Kết nối kiến thức tiếp theo: Sau bài cuối, áp dụng toàn bộ chuỗi vào bài toán thực tế tự chọn: xác định dữ liệu, đặt tiêu chí đánh giá và lưu kết quả có thể tái tạo.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Feature	đặc trưng	Cột đầu vào cho mô hình
Label	nhãn	Giá trị mục tiêu cần dự đoán
Pipeline	chuỗi xử lý	Gom tiền xử lý và mô hình thành các bước
Evaluation	đánh giá	Đo kết quả trên dữ liệu chưa dùng để fit

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`pd.DataFrame()/dropna()`	Tạo bảng và đánh giá xử lý giá trị thiếu
`train_test_split(...,stratify=y)`	Chia dữ liệu theo nhãn, tránh leakage
`make_pipeline(StandardScaler(),LogisticRegression())`	Fit scaler/model trên train trong một chuỗi
`model.score(X_test,y_test)`	Đánh giá tập test, không suy từ điểm train

4. Khái niệm chính và liên quan

Mạch kiến thức: Một dự án AI bắt đầu từ việc hiểu mỗi hàng mẫu, cột đặc trưng và nhãn mục tiêu. Dữ liệu được làm sạch theo đúng quy tắc, chia train/test **trước khi học tham số xử lý**, huấn luyện bằng Pipeline rồi đánh giá trên nhãn test. Kết quả mô hình cần đọc cùng giới hạn, không chỉ nhìn một con số accuracy.

4.1. Feature và label — Đầu vào khác mục tiêu

Từ vựng trong mục: Feature — đặc trưng; Label — nhãn.

Trong mini-project phân loại hoa Iris, một **hàng** ứng với một bông hoa; bốn **cột feature** là chiều dài/rộng đài hoa và cánh hoa, đơn vị cm. **Label** (y) là mã loài 0,1,2 cần dự đoán. X có shape `(150,4)`, y có 150 nhãn. Không đưa cột chứa nhãn hoặc thông tin được biết sau kết quả vào X, vì đó là rò rỉ mục tiêu.

No Note — Không chú thích

```
from sklearn.datasets import load_iris

data = load_iris()
print(data.data.shape, data.target.shape)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Phân biệt đâu vào và nhãn đích qua dữ liệu Iris của scikit-learn.
# Dữ liệu: Iris có 150 hoa, 4 đặc trưng/hoa và một nhãn loài/hoa.
# sklearn.datasets cung cấp dữ liệu mẫu Iris cho ví dụ nhận dạng đâu vào/nhãn.
# load_iris() tải bộ dữ liệu mẫu Iris; không dùng dữ liệu của người học.
from sklearn.datasets import load_iris

# load_iris() tải bộ dữ liệu mẫu Iris; không dùng dữ liệu của người học.
# Đối tượng Bunch của sklearn giữ ma trận feature tại data.data và nhãn tại data.target.
data = load_iris()
# data.data.shape=(150,4): 150 hàng là hoa, bốn cột là số đo.
# data.target.shape=(150,): một mã loài cho mỗi hoa, không phải bốn nhãn.
print(data.data.shape, data.target.shape)
```

Kết quả khi chạy:

```
(150, 4) (150,)
```

Mỗi trong 150 hoa có bốn số đo dùng dự đoán một trong ba loài.

4.2. Cleaning — Xử lý ô khuyết đúng thứ tự

Từ vựng trong mục: Cleaning — làm sạch; Imputation — điền thiếu.

Trước khi huấn luyện cần kiểm kiểu dữ liệu, giá trị thiếu, lỗi nhập và đơn vị. Nếu thiếu số đo, có thể điền bằng **median của tập train**, nhưng **không** tính median từ cả train và test. Ví dụ ba chiều dài (cm) `[5,None,7]`, median từ hai giá trị có thực là 6; điền None bằng 6. Đây là giá trị ước lượng, không phải số đo gốc.

No Note — Không chú thích

```
import pandas as pd

values = pd.Series([5.0, None, 7.0])
print(values.fillna(values.median()).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Điền ô dữ liệu thiếu bằng trung vị của các giá trị còn lại.
# Dữ liệu: Series gồm ba giá trị 5, None, 7; None được hiểu là ô chưa ghi, không phải 0.
# Pandas quản lý bảng, dữ liệu thiếu và thao tác làm sạch.
import pandas as pd

# Series của Pandas giữ các vị trí có giá trị thiếu (NaN).
values = pd.Series([5.0, None, 7.0])
# median() lấy trung vị chỉ từ các ô có dữ liệu, ra 6; mặc định bỏ qua NaN.
# fillna(value) thay ô thiếu bằng trung vị, trả Series mới nếu không có inplace=True.
# tolist() đổi kết quả Series thành list Python để dễ đọc.
print(values.fillna(values.median()).tolist())
```

Kết quả khi chạy:

```
[5.0, 6.0, 7.0]
```

Giá trị thiếu được thay bằng trung vị 6 cm của hai số đã quan sát.

4.3. Pipeline — Gắn tiền xử lý với mô hình

Từ vựng trong mục: Pipeline — chuỗi xử lý; Fit — học tham số.

Pipeline ghép các bước `SimpleImputer` → `StandardScaler` → `LogisticRegression`. Khi `.fit(X_train,y_train)`, bước điền thiếu và chuẩn hóa **học tham số từ train**, mô hình sau đó học nhãn. Khi `.predict(X_test)`, các phép biến đổi dùng lại tham số train. Điều này hạn chế rò rỉ test do tính mean/median hay scale trên toàn bảng trước khi chia.

No Note — Không chú thích

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler(),
    LogisticRegression(max_iter=1000),
)
print(len(model.steps))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Tạo pipeline tiền xử lý và phân loại nhưng chưa huấn luyện mô hình.
# Dữ liệu: Ba bước imputer→scaler→LogisticRegression; chưa có X_train hoặc y_train.
# make_pipeline ghép các bước để transform Test bằng thống kê Train đã học.
from sklearn.pipeline import make_pipeline
# SimpleImputer thay dữ liệu thiếu theo thống kê chỉ học từ Train.
from sklearn.impute import SimpleImputer
# StandardScaler học mean/SD từng cột từ Train rồi chuẩn hóa.
from sklearn.preprocessing import StandardScaler
# LogisticRegression phân loại nhãn rời rạc, không dự báo giá trị liên tục.
from sklearn.linear_model import LogisticRegression

# make_pipeline() ghép các bước thành một quy trình; chỉ khi fit() mới học thống kê và tham số´từ
dữ liệu.
model = make_pipeline(
    # strategy='median' dùng trung vị theo TỪNG CỘT được học khi fit; không dùng mean.
```

```
    SimpleImputer(strategy='median'),
    # Scaler đưa đặc trưng về gần mean 0 và std 1 dựa trên dữ liệu train sau điền ô thiếu.
    StandardScaler(),
    # Bộ phân loại LogisticRegression tối ưu tối đa 1000 vòng lặp, không phải 1000 mẫu.
    LogisticRegression(max_iter=1000),
)
# steps là list ba cặp (tên,bước); len() đếm ba thành phần, không phải Accuracy.
print(len(model.steps))
```

Kết quả khi chạy:

```
3
```

Pipeline gồm ba bước nối tiếp: điền thiếu, chuẩn hóa, phân loại; chưa gọi fit nên chưa có mô hình đã học.

4.4. Evaluation — Đánh giá trên dữ liệu chưa fit

Từ vựng trong mục: Evaluation — đánh giá; Confusion matrix — ma trận nhầm lẫn.

Đánh giá phân loại phải so nhãn dự đoán với nhãn thật của **test giữ riêng**. **Accuracy** là số dự đoán đúng chia tổng số mẫu, nhưng có thể gây hiểu sai nếu một lớp rất phổ biến. **Confusion matrix** ghi số mẫu theo **hàng = nhãn thật**, **cột = nhãn dự đoán**; ô ngoài đường chéo cho biết những lỗi nhầm lớp cụ thể.

No Note — Không chú thích

```
from sklearn.metrics import confusion_matrix

actual = [0, 1, 1, 0]
predicted = [0, 1, 0, 0]
print(confusion_matrix(actual, predicted).tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Đọc bảng nhầm lẫn để hiểu mô hình dự đoán sai lớp nào.
# Dữ liệu: 4 nhãn thật và 4 nhãn đoán, cùng thứ tự mỗi học viên; có một lớp 1 bị đoán nhầm 0.
# confusion_matrix dùng để đếm các cặp (nhãn thật,nhãn dự đoán).
from sklearn.metrics import confusion_matrix

# Nhãn thật y_true của bốn hồ sơ, không phải xác suất.
actual = [0, 1, 1, 0]
# Nhãn dự đoán y_pred theo đúng thứ tự actual.
predicted = [0, 1, 0, 0]
# Hàng là nhãn THẬT, cột là nhãn DỰ ĐOÁN; phần tử hàng 1/cột 0 là false negative lớp 1.
# Chuyển ma trận NumPy 2×2 thành list lồng để in.
print(confusion_matrix(actual, predicted).tolist())
```

Kết quả khi chạy:

```
[[2, 0], [1, 1]]
```

Lớp thật 0 có hai mẫu dự đoán đúng; lớp thật 1 có một mẫu nhầm sang 0 và một mẫu đúng.

5. Năm tình huống mini-project AI — từ kiểm 10 hồ sơ đến báo cáo 20 dòng

Tình huống 1 — Cấp 1: Kiểm cấu trúc 10 hồ sơ trước khi làm dự án AI

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn; **Data validation** — kiểm dữ liệu.

Tình huống thực tế: Trung tâm đào tạo chuẩn bị **10 hồ sơ HV01–HV10** để thử làm dự án dự đoán kết quả học tập. Mỗi hàng ghi mã học viên, **số giờ tự học trong tuần (`hours`, giờ/tuần)**, **số bài luyện tập (`practice`, bài)** và kết quả `passed`: **0 là chưa đạt, 1 là đạt**. Có **6 người chưa đạt và 4 người đạt**. Trước khi đưa dữ liệu cho máy học, nhóm phải kiểm có đủ 10 người, mỗi người một mã, hai thông tin đầu vào đều có số, và **không đưa kết quả `passed` vào các cột dùng để dự đoán**. Câu hỏi ở bước này chỉ là dữ liệu đã sẵn sàng chưa, chưa phải mô hình dự đoán đúng bao nhiêu.

Vì sao chọn cách này? Feature (đặc trưng) là thông tin mô hình có thể nhìn để dự đoán; trong bài gồm **giờ học** và **số bài luyện tập**. **Label (nhãn)** là câu trả lời đúng mà sau này mô hình phải đoán, chính là `passed`. `pandas.DataFrame(records)` tạo bảng **mỗi hàng một người**; `X = df[['hours', 'practice']]` chọn đúng hai cột đầu vào, `y = df['passed']` giữ riêng câu trả lời. Sau đó kiểm số hàng, `X.shape` (hình dạng **10 hàng × 2 cột**), đếm hai nhãn, dùng `duplicated()` tìm mã trùng và `isna()` đếm ô thiếu. Mã HV01 là **định danh**, không phải một đại lượng đo mức học tập nên không đưa vào X; đưa `passed` vào X sẽ làm mô hình **nhìn thấy đáp án** và tạo kết quả đánh giá sai lệch. Bước này chỉ kiểm dữ liệu, chưa huấn luyện.

Quy trình:

1. Tạo đủ 10 hàng với ý nghĩa/đơn vị từng cột.
2. Lấy đúng hai feature số sang X, nhãn mục tiêu sang y.
3. Xác nhận số dòng khớp, ID duy nhất, không thiếu số đo và in phân bố nhãn.

No Note — Không chú thích

```
import pandas as pd
records = [{"id":"HV01","hours":1,"practice":2,"passed":0},
{"id":"HV02","hours":2,"practice":3,"passed":0},{"id":"HV03","hours":3,"practice":4,"passed":0},
{"id":"HV04","hours":4,"practice":5,"passed":0},{"id":"HV05","hours":5,"practice":2,"passed":0},
{"id":"HV06","hours":1,"practice":3,"passed":0},{"id":"HV07","hours":2,"practice":4,"passed":1},
{"id":"HV08","hours":3,"practice":5,"passed":1},{"id":"HV09","hours":4,"practice":2,"passed":1},
{"id":"HV10","hours":5,"practice":3,"passed":1}]
df = pd.DataFrame(records)
X = df[["hours", "practice"]]
y = df["passed"]
if len(df) != 10 or X.shape != (10, 2) or len(y) != 10:
    raise ValueError("Cần 10 hồ sơ, hai đặc trưng và 10 nhãn.")
print("Hồ sơ:", len(df))
print("Feature:", X.shape[1])
print("Nhãn 0/1:", int((y==0).sum()), int((y==1).sum()))
print("ID trùng:", int(df["id"].duplicated().sum()))
print("Ô feature thiếu:", int(X.isna().sum().sum()))
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Kiểm dữ liệu đầu vào của bài toán dự đoán đỗ: số hồ sơ, đặc trưng, nhãn, trùng mã và ô
khuyết.
# Dữ liệu: 10 học viên HV01..HV10; hours=giờ học, practice=số lần thực hành, passed=nhãn 0/1.
# Pandas quản lý bảng, dữ liệu thiếu và thao tác làm sạch.
import pandas as pd
# List 10 dictionary, mỗi dict là một học viên; hours/practice là đầu vào, passed là kết quả cần
```

```
đoán.
records = [{"id":"HV01","hours":1,"practice":2,"passed":0},
{"id":"HV02","hours":2,"practice":3,"passed":0},{"id":"HV03","hours":3,"practice":4,"passed":0},
{"id":"HV04","hours":4,"practice":5,"passed":0},{"id":"HV05","hours":5,"practice":2,"passed":0},
{"id":"HV06","hours":1,"practice":3,"passed":0},{"id":"HV07","hours":2,"practice":4,"passed":1},
{"id":"HV08","hours":3,"practice":5,"passed":1},{"id":"HV09","hours":4,"practice":2,"passed":1},
{"id":"HV10","hours":5,"practice":3,"passed":1}]
# DataFrame tạo bảng một hàng trên môĩ dictionary, cột từ các key.
df = pd.DataFrame(records)
# Hai cặp ngoặc vuông chọn DANH SÁCH hai cột thành DataFrame X shape (10,2), không dùng passed làm
feature.
X = df[["hours", "practice"]]
# Một cặp ngoặc chọn Series nhãn y 10 giá trị 0/1.
y = df["passed"]
# shape lưu (sô´hàng,sô´cột); kiểm đúng 10 học viên và 2 feature.
# Điều kiện bảo vệ kiểm cấu trúc sô´mẫũ và tránh Train/Test dùng chung trước khi báo cáo.
if len(df) != 10 or X.shape != (10, 2) or len(y) != 10:
    # raise ném lôĩ khi cấú trúc dữ liệu không hợp lệ; tránh in kết luận thiếú mẫũ.
    # raise ValueError dừng kịch bản khi dữ liệu không đúng sô´mẫũ hoặc điều kiện kiểm thử.
    raise ValueError("Cần 10 hô`sơ, hai đặc trưng và 10 nhãn.")
print("Hô`sơ:", len(df))
# shape[1] lấý SÔ´CỘT đặc trưng, khác shape[0] là sô´hàng.
print("Feature:", X.shape[1])
# So từng nhãn với 0/1 tạo mask Boolean; sum() đếm lượt True trong từng lớp.
print("Nhãn 0/1:", int((y==0).sum()), int((y==1).sum()))
# duplicated() đánh dấú ID lặp từ lần xuất hiện thứ hai; sum() đếm dòng trùng nhưng không xóa.
print("ID trùng:", int(df["id"].duplicated().sum()))
# isna() đánh dấú NaN từng ô; sum() thứ nhất tổng theo cột, thứ hai tổng tất cả cột.
print("Ô feature thiếú:", int(X.isna().sum().sum()))
```

Kết quả khi chạy:

```
Hô`sơ: 10
Feature: 2
Nhãn 0/1: 6 4
ID trùng: 0
Ô feature thiếú: 0
```

Đây là dữ liệu học tập **giả lập**, không phải hồ sơ thật. Đếm nhãn chưa cho biết mô hình dự đoán được bao nhiêu mẫu mới.

Tình huống 2 — Cấp 2: Tách 12 học viên có phân tầng và đặt điểm baseline

Từ vựng trong mục: Stratified split — chia giữ tỷ lệ lớp; **Baseline** — mốc so sánh; **Test set** — tập đánh giá.

Tình huống thực tế: Nhóm giáo viên có **12 hồ sơ HV01–HV12** với hai đặc trưng **giờ tự học (giờ/tuần)** và **số bài luyện tập (bài)**, cùng nhãn `passed` **0/1**. Sáu người thuộc lớp 0 và sáu người thuộc lớp 1. Trước khi thử mô hình học máy thực sự, họ muốn dành **8 hồ sơ để học (train)**, **4 hồ sơ chưa dùng để đánh giá (test)**, rồi xem một cách dự đoán cực đơn giản — **luôn đoán nhãn thường gặp nhất trong train** — đạt bao nhiêu trên bốn người test. Đây là **mức chuẩn ban đầu (baseline)** để biết mô hình phức tạp có cải thiện thật hay không.

Vì sao chọn cách này? Train/test split dành một phần người học làm **dữ liệu kiểm tra chưa được dùng trong huấn luyện**. `train_test_split` giữ đủ hai nhãn bằng `stratify=y`; `random_state=42` giúp tạo lại đúng cách chia 8/4 để đối chiếu. **DummyClassifier** kiểu `most_frequent` chỉ học xem **nhãn nào xuất hiện nhiều nhất trong tám người train**, rồi dùng nhãn ấy cho mọi dự đoán, không nhìn giờ học hay số bài để tìm quy luật. `accuracy` là **tỷ lệ nhãn dự đoán đúng trong bốn người test**; dữ liệu test cân bằng hai lớp

nên baseline đạt **0,50**, không phải kết quả mô hình đã hiểu việc học. Đặt baseline trước khi thử mô hình mới giúp có một mốc so sánh; mẫu bốn người quá nhỏ để khẳng định chất lượng trên toàn bộ học viên.

Quy trình:

1. Đọc 12 bộ X/y/ID có cùng thứ tự.
2. Chia 8 train và 4 test phân tầng, kiểm ID không giao nhau.
3. Fit baseline trên train, đánh giá accuracy trên test và in số mẫu của từng lớp test.

No Note — Không chú thích

```
import numpy as np
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
records = [{"id":"HV01","hours":1,"practice":2,"passed":0},
{"id":"HV02","hours":2,"practice":3,"passed":0},{"id":"HV03","hours":3,"practice":4,"passed":0},
{"id":"HV04","hours":1,"practice":2,"passed":0},{"id":"HV05","hours":2,"practice":3,"passed":0},
{"id":"HV06","hours":3,"practice":4,"passed":0},{"id":"HV07","hours":6,"practice":8,"passed":1},
{"id":"HV08","hours":7,"practice":9,"passed":1},{"id":"HV09","hours":8,"practice":10,"passed":1},
{"id":"HV10","hours":6,"practice":8,"passed":1},{"id":"HV11","hours":7,"practice":9,"passed":1},
{"id":"HV12","hours":8,"practice":10,"passed":1}]
features = []
labels = []
for row in records:
    features.append([row["hours"], row["practice"]])
    labels.append(row["passed"])
X = np.array(features, dtype=float)
y = np.array(labels)
train,test = train_test_split(np.arange(12),test_size=4,random_state=42,stratify=y)
model = DummyClassifier(strategy="most_frequent")
model.fit(X[train],y[train])
accuracy = model.score(X[test],y[test])
if len(records) != 12 or not set(train).isdisjoint(test):
    raise ValueError("Cần 12 hồ sơ và Train/Test không trùng mẫu.")
print("Hồ sơ:", len(records))
print("Train/test:", len(train), len(test))
print("Nhãn test 0/1:", int((y[test]==0).sum()), int((y[test]==1).sum()))
print(f"Accuracy baseline: {accuracy:.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chia Train/Test cân bằng nhãn và đo mô hình baseline luôn chọn lớp phổ biến.
# Dữ liệu: 12 học viên, hai feature hours/practice và nhãn 0/1, chọn 8 Train + 4 Test.
# NumPy hỗ trợ ndarray, NaN và chỉ số các hàng train/test.
import numpy as np
# DummyClassifier là baseline đoán lớp phổ biến, không học quan hệ feature.
from sklearn.dummy import DummyClassifier
# train_test_split chia đồng bộ hàng Train và Test trước khi fit.
from sklearn.model_selection import train_test_split
# Mỗi dictionary lưu id, giờ học, lượt luyện tập và nhãn đỗ; 12 mẫu tạo hai nhóm nhãn.
records = [{"id":"HV01","hours":1,"practice":2,"passed":0},
{"id":"HV02","hours":2,"practice":3,"passed":0},{"id":"HV03","hours":3,"practice":4,"passed":0},
{"id":"HV04","hours":1,"practice":2,"passed":0},{"id":"HV05","hours":2,"practice":3,"passed":0},
{"id":"HV06","hours":3,"practice":4,"passed":0},{"id":"HV07","hours":6,"practice":8,"passed":1},
{"id":"HV08","hours":7,"practice":9,"passed":1},{"id":"HV09","hours":8,"practice":10,"passed":1},
{"id":"HV10","hours":6,"practice":8,"passed":1},{"id":"HV11","hours":7,"practice":9,"passed":1},
{"id":"HV12","hours":8,"practice":10,"passed":1}]
# List sẽ chứa một cặp [hours,practice] cho mỗi học viên, theo đúng thứ tự nhãn.
features = []
# List nhãn passed cùng vị trí với features.
labels = []
for row in records:
```

```
    # append() đưa hai feature của hồ sơ hiện tại vào X.
    features.append([row["hours"], row["practice"]])
    # append() ghi nhãn tương ứng để X/y không lệch hàng.
    labels.append(row["passed"])
# Ép đặc trưng thành ma trận số thực shape (12,2), phù hợp sklearn.
X = np.array(features, dtype=float)
# Nhãn y thành vector 12 phần tử, một lớp cho mỗi người.
y = np.array(labels)
# Chia các chỉ số hàng thay vì chia X và y độc lập, giúp trích dữ liệu đồng bộ.
# arange(12) tạo các chỉ số 0..11, cận 12 không bao gồm.
# test_size=4 giữ đúng bốn mẫu CHƯA fit để kiểm tra, không có nghĩa 4%.
# Cố định trạng thái ngẫu nhiên để phép chia có thể lặp lại.
# Giữ tỷ lệ nhãn 0/1 gần giống nhau trên Train/Test.
train,test = train_test_split(np.arange(12),test_size=4,random_state=42,stratify=y)
# Baseline không học quy luật feature: luôn dự đoán nhãn đông nhất trong TRAIN.
model = DummyClassifier(strategy="most_frequent")
# fit() học baseline CHỈ từ các hàng Train; X[train] và y[train] dùng cùng index.
model.fit(X[train],y[train])
# score() trả Accuracy trên Test: tỷ lệ dự đoán đúng từ 4 hồ sơ chưa fit.
accuracy = model.score(X[test],y[test])
# Chuyển Train sang set rồi kiểm không có index trùng với Test, tránh rò rỉ mẫu.
# Điều kiện bảo vệ kiểm cấu trúc số mẫu và tránh Train/Test dùng chung trước khi báo cáo.
if len(records) != 12 or not set(train).isdisjoint(test):
    # raise ValueError dừng kịch bản khi dữ liệu không đúng số mẫu hoặc điều kiện kiểm thử
    raise ValueError("Cần 12 hồ sơ và Train/Test không trùng mẫu.")
print("Hồ sơ:", len(records))
# len() đếm tám mẫu train (len(test) đếm bốn mẫu test).
print("Train/test:", len(train), len(test))
# Boolean mask chỉ đếm nhãn lớp 0/1 trong Test, không phải toàn bộ 12 mẫu.
print("Nhãn test 0/1:", int((y[test]==0).sum()), int((y[test]==1).sum()))
# f-string :.2f chỉ làm tròn Accuracy khi IN, giá trị lưu vẫn là số thực gốc.
print(f"Accuracy baseline: {accuracy:.2f}")
```

Kết quả khi chạy:

```
Hồ sơ: 12
Train/test: 8 4
Nhãn test 0/1: 2 2
Accuracy baseline: 0.50
```

Baseline cố ý đơn giản; accuracy 0,50 ở đây là **hai trong bốn hồ sơ test**, không đủ để khẳng định hiệu quả trên người học mới.

Tình huống 3 — Cấp 3: Huấn luyện Pipeline trên 15 hoa có số đo bị khuyết

Từ vựng trong mục: Imputation — điền thiếu; **Pipeline** — chuỗi xử lý; **Leakage** — rò rỉ dữ liệu.

Tình huống thực tế: Nhóm nghiên cứu cây trồng có **15 bông hoa F01–F15**, mỗi bông ghi hai phép đo **chiều dài cánh hoa và chiều dài đài hoa (cm)**, cùng mã loài **0, 1 hoặc 2**; mỗi loài có năm bông. **F04 thiếu chiều dài cánh**, còn **F12 thiếu chiều dài đài** nên không thể đưa thẳng mọi ô vào mô hình. Nhóm cần giữ **6 bông test**, học từ **9 bông train**, dùng thông tin của train để điền số đo còn thiếu, rồi thử phân loại loài của sáu bông được giữ riêng. Mục tiêu còn là kiểm chắc chắn **sáu bông test không bị dùng khi tính cách điền thiếu**.

Vì sao chọn cách này? Dữ liệu thiếu có nghĩa là **chưa có phép đo**, không phải chiều dài 0 cm. **Median (trung vị)** là số nằm giữa những giá trị có số sau khi sắp thứ tự; `SimpleImputer(strategy='median')` dùng trung vị **tính riêng từng cột từ chín bông train** để điền ô thiếu khi cần. **StandardScaler** tiếp tục đưa các cột số đo về cách biểu diễn phù hợp cho mô hình, rồi **Logistic Regression** học cách phân biệt ba loài. `make_pipeline` xếp các bước theo đúng thứ tự **điền thiếu → chuẩn hóa → phân loại**: gọi `fit` chỉ với train

khiến thông số của cả ba bước được học **không có hoa test**. Nếu tính trung vị hoặc trung bình từ cả 15 bông trước khi chia sẽ có **rò rỉ dữ liệu (data leakage)**. Chương trình so trung vị bên trong pipeline với trung vị train, dự đoán sáu hoa test và tính accuracy; kết quả 1,00 trên mẫu nhỏ giả lập **không chứng minh mô hình luôn đúng**.

Quy trình:

1. Lưu 15 hoa và chuyển None trong cột số thành NaN; kiểm đúng hai ô thiếu.
2. Tách train/test phân tầng 9/6 trước khi `.fit()` tiền xử lý.
3. Huấn luyện Pipeline, so median đã học với `nanmedian(X_train)`, đo accuracy test và kiểm số dự đoán.

No Note — Không chú thích

```
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
records = [{"id":"F01","petal_cm":1.1,"sepal_cm":3,"species":0},
{"id":"F02","petal_cm":1.2000000000000002,"sepal_cm":3.03,"species":0},
{"id":"F03","petal_cm":1.3,"sepal_cm":3.06,"species":0},
{"id":"F04","petal_cm":None,"sepal_cm":3.09,"species":0},
{"id":"F05","petal_cm":1.5,"sepal_cm":3.12,"species":0},
{"id":"F06","petal_cm":4.1,"sepal_cm":4,"species":1},
{"id":"F07","petal_cm":4.199999999999999,"sepal_cm":4.03,"species":1},
{"id":"F08","petal_cm":4.3,"sepal_cm":4.06,"species":1},
{"id":"F09","petal_cm":4.3999999999999995,"sepal_cm":4.09,"species":1},
{"id":"F10","petal_cm":4.5,"sepal_cm":4.12,"species":1},
{"id":"F11","petal_cm":7.1,"sepal_cm":5,"species":2},
{"id":"F12","petal_cm":7.199999999999999,"sepal_cm":None,"species":2},
{"id":"F13","petal_cm":7.3,"sepal_cm":5.06,"species":2},
{"id":"F14","petal_cm":7.3999999999999995,"sepal_cm":5.09,"species":2},
{"id":"F15","petal_cm":7.5,"sepal_cm":5.12,"species":2}]
features = []
labels = []
for row in records:
    features.append([row["petal_cm"], row["sepal_cm"]])
    labels.append(row["species"])
X = np.array(features, dtype=float)
y = np.array(labels)
train,test = train_test_split(np.arange(len(records)),test_size=6,
                              random_state=42,stratify=y)
pipeline = make_pipeline(SimpleImputer(strategy="median"),
                         StandardScaler(),LogisticRegression(max_iter=1000))
pipeline.fit(X[train],y[train])
predicted = pipeline.predict(X[test])
imputer = pipeline.named_steps["simpleimputer"]
if X.shape != (15, 2) or len(predicted) != 6:
    raise ValueError("Cần đủ 15 mẫu hoa và 6 dự đoán test.")
print("Hoa:", len(records))
print("Ô thiếu ban đầu:", int(np.isnan(X).sum()))
print("Train/test:", len(train),len(test))
print("Median chỉ từ train:", bool(np.allclose(imputer.statistics_,np.nanmedian(X[train],axis=0))))
print(f"Accuracy test: {(predicted==y[test]).mean():.2f}")
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Huấn luyện phân loại 3 loài hoa với imputer→scaler→logistic chỉ fit trên Train.
# Dữ liệu: 15 hoa F01..F15, hai feature petal_cm/sepal_cm đo cm, vài None, ba loài 0/1/2; Test 6
hoa.
```

```
# NumPy hỗ˜trợ ndarray, NaN và chỉ số´các hàng train/test.
import numpy as np
# SimpleImputer thay dữ liệu thiếú theo thống kê chỉ học từ Train.
from sklearn.impute import SimpleImputer
# LogisticRegression phân loại nhãn rời rạc, không dự báo giá trị liên tục.
from sklearn.linear_model import LogisticRegression
# train_test_split chia đôìng bộ hàng Train và Test trước khi fit.
from sklearn.model_selection import train_test_split
# make_pipeline ghép các bước đểˆtransform Test bằng thống kê Train đã học.
from sklearn.pipeline import make_pipeline
# StandardScaler học mean/SD từng cột từ Train rồi chuẩn hóa.
from sklearn.preprocessing import StandardScaler
# List 15 hoa có None nghĩa số´đo thiếú, không phải 0 cm; nhãn species là mục tiêu dự đoán.
records = [{"id":"F01","petal_cm":1.1,"sepal_cm":3,"species":0},
{"id":"F02","petal_cm":1.2000000000000002,"sepal_cm":3.03,"species":0},
{"id":"F03","petal_cm":1.3,"sepal_cm":3.06,"species":0},
{"id":"F04","petal_cm":None,"sepal_cm":3.09,"species":0},
{"id":"F05","petal_cm":1.5,"sepal_cm":3.12,"species":0},
{"id":"F06","petal_cm":4.1,"sepal_cm":4,"species":1},
{"id":"F07","petal_cm":4.199999999999999,"sepal_cm":4.03,"species":1},
{"id":"F08","petal_cm":4.3,"sepal_cm":4.06,"species":1},
{"id":"F09","petal_cm":4.3999999999999995,"sepal_cm":4.09,"species":1},
{"id":"F10","petal_cm":4.5,"sepal_cm":4.12,"species":1},
{"id":"F11","petal_cm":7.1,"sepal_cm":5,"species":2},
{"id":"F12","petal_cm":7.199999999999999,"sepal_cm":None,"species":2},
{"id":"F13","petal_cm":7.3,"sepal_cm":5.06,"species":2},
{"id":"F14","petal_cm":7.3999999999999995,"sepal_cm":5.09,"species":2},
{"id":"F15","petal_cm":7.5,"sepal_cm":5.12,"species":2}]
# Tạo list hai đặc trưng (cánh hoa, đài hoa) theo đúng thứ tự môĩ hàng.
features = []
# Tạo list nhãn loài của cùng hàng feature.
labels = []
for row in records:
    # append() giữ cặp feature có thểˆcó None, imputer xưˆlý sau.
    features.append([row["petal_cm"], row["sepal_cm"]])
    # append() giữ nhãn species của hoa đang xét.
    labels.append(row["species"])
# Chuyển None thành NaN trong ndarray số´thực shape (15,2).
X = np.array(features, dtype=float)
# Mảng nhãn 15 giá trị nguyên (môĩ hoa một nhãn).
y = np.array(labels)
# Chia INDEX Train/Test trước khi fit đểˆtránh rò rỉ thống kê tiền xưˆlý.
# Sáu hoa dùng làm Test, chín hoa Train.
train,test = train_test_split(np.arange(len(records)),test_size=6,
                              # Seed cố´định phép chia; stratify=y giúp phân bố´cả ba loài vào hai
tập.
                              random_state=42,stratify=y)
# Imputer dùng trung vị môĩ cột đượ học CHỈ ở Train đểˆđiền NaN.
pipeline = make_pipeline(SimpleImputer(strategy="median"),
                         # Sau impute, chuẩn hóa theo Train rồi phân loại; max_iter=1000 giới hạn
bước tôí ưu.
                         StandardScaler(),LogisticRegression(max_iter=1000))
# fit() học ba bước của pipeline CHỈ từ Train, không nhìn trước Test.
pipeline.fit(X[train],y[train])
# predict() biến đổi Test bằng imputer/scaler đã fit và đoán nhãn.
predicted = pipeline.predict(X[test])
# named_steps chọn bước SimpleImputer theo tên tự sinh, đọc thống kê đã học.
imputer = pipeline.named_steps["simpleimputer"]
# Điều kiện bảo vệ kiểm cấú trúc số´mẫu và tránh Train/Test dùng chung trước khi báo cáo.
if X.shape != (15, 2) or len(predicted) != 6:
    # raise ValueError dừng kịch bản khi dữ liệu không đúng số´mẫu hoặc điều kiện kiểm thư.ˆ
    raise ValueError("Cần đủ 15 mẫu hoa và 6 dự đoán test.")
print("Hoa:", len(records))
# np.isnan(X) đánh dấú ô NaN trong dữ liệu gốć; sum() đếm số´ô bị thiếú.
print("Ô thiếú ban đầu:", int(np.isnan(X).sum()))
print("Train/test:", len(train),len(test))
```

```
# statistics_ là vector trung vị TỪ TRAIN cho từng cột feature sau fit.
# nanmedian bỏ qua NaN và tính trung vị TỪNG CỘT (axis=0) của TRAIN, không lấy từ toàn bộ X.
# allclose() đối chiếu hai mảng số´với dung sai floating-point, không dùng == tuyệt đối.
print("Median chỉ từ train:", bool(np.allclose(imputer.statistics_,np.nanmedian(X[train],axis=0))))
# Mask True khi dự đoán đúng từng mẫu Test, mean() là Accuracy 0..1.
print(f"Accuracy test: {(predicted==y[test]).mean():.2f}")
```

Kết quả khi chạy:

```
Hoa: 15
Ô thiếu ban đầu: 2
Train/test: 9 6
Median chỉ từ train: True
Accuracy test: 1.00
```

Accuracy phụ thuộc đúng **sáu hoa test giả lập**; cần dữ liệu ngoài bài để đánh giá thực tế. Mỗi số đo bị thiếu vẫn có đơn vị cm, không được tự đổi thiếu thành 0 cm.

Tình huống 4 — Cấp 4: Đánh giá 18 hồ sơ mất ô và lệch lớp bằng confusion matrix

Từ vựng trong mục: Class imbalance — lệch tỷ lệ lớp; **Confusion matrix** — ma trận nhầm lẫn; **Recall** — độ thu hồi.

Tình huống thực tế: Trung tâm đào tạo có **18 học viên C01–C18**. Mỗi dòng gồm **giờ tự học mỗi tuần**, **số hoạt động đã hoàn thành** và nhãn `completed`: **0 chưa hoàn thành**, **1 đã hoàn thành**. Có **12 người nhãn 0**, **6 người nhãn 1** và **ba ô dữ liệu đầu vào chưa có số**. Trung tâm giữ **12 hồ sơ train, 6 hồ sơ test**, rồi so mô hình Logistic Regression đã điền thiếu/chuẩn hóa với cách đoán lớp đông nhất. Điều quan trọng không chỉ là dự đoán đúng bao nhiêu người, mà còn là **trong hai người đã hoàn thành ở test, mô hình tìm được mấy người**.

Vì sao chọn cách này? Khi **lớp 0 nhiều gấp đôi lớp 1**, cách luôn đoán 'chưa hoàn thành' có thể đạt accuracy khoảng **hai phần ba** nhưng vẫn **bỏ sót mọi người thực sự hoàn thành**. Vì thế bài giữ riêng **6 người test** với **4 nhãn 0 và 2 nhãn 1**, kiểm cả **accuracy** (tỷ lệ dự đoán đúng) lẫn **recall lớp 1** (trong hai người thật sự hoàn thành, mô hình nhận ra bao nhiêu). `DummyClassifier` cung cấp baseline; pipeline `SimpleImputer → StandardScaler → LogisticRegression` học từ **12 người train** và xử lý **ba ô thiếu** mà không học thông số từ test. **Confusion matrix (ma trận nhầm lẫn)** đặt **nhãn thật theo hàng, dự đoán theo cột**, để đọc được số người đúng/sai của từng lớp, không chỉ một số accuracy tổng. Việc kết quả mẫu này tốt chưa nói lên mô hình có hiệu quả với học viên ngoài dữ liệu giả lập.

Quy trình:

1. Tạo 18 dòng với nhãn lệch và ba ô thiếu hợp lệ.
2. Chia test 6 theo nhãn; fit imputer/scaler/classifier chỉ trên 12 train, cùng tập test cho baseline.
3. Báo baseline accuracy, accuracy model, recall lớp 1 và ma trận nhầm lẫn test.

No Note — Không chú thích

```
import numpy as np
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix,recall_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
records = [{"id":"C01","hours":1,"activities":2,"completed":0},
{"id":"C02","hours":2,"activities":3,"completed":0},
{"id":"C03","hours":None,"activities":4,"completed":0},
{"id":"C04","hours":4,"activities":2,"completed":0},
{"id":"C05","hours":1,"activities":3,"completed":0},
{"id":"C06","hours":2,"activities":4,"completed":0},
{"id":"C07","hours":3,"activities":2,"completed":0},
{"id":"C08","hours":4,"activities":3,"completed":0},
{"id":"C09","hours":1,"activities":4,"completed":0},
{"id":"C10","hours":2,"activities":2,"completed":0},
{"id":"C11","hours":3,"activities":3,"completed":0},
{"id":"C12","hours":4,"activities":4,"completed":0},
{"id":"C13","hours":7,"activities":9,"completed":1},
{"id":"C14","hours":8,"activities":10,"completed":1},
{"id":"C15","hours":9,"activities":11,"completed":1},
{"id":"C16","hours":7,"activities":None,"completed":1},
{"id":"C17","hours":8,"activities":10,"completed":1},
{"id":"C18","hours":None,"activities":11,"completed":1}]
features = []
labels = []
for row in records:
    features.append([row["hours"], row["activities"]])
    labels.append(row["completed"])
X = np.array(features, dtype=float)
y = np.array(labels)
train,test = train_test_split(np.arange(len(records)),test_size=6,
                              random_state=42,stratify=y)
baseline = DummyClassifier(strategy="most_frequent").fit(X[train],y[train])
model = make_pipeline(SimpleImputer(strategy="median"),
                      StandardScaler(),LogisticRegression(max_iter=1000))
model.fit(X[train],y[train])
pred = model.predict(X[test])
cm = confusion_matrix(y[test],pred,labels=[0,1])
if len(records) != 18 or len(train) != 12 or len(test) != 6:
    raise ValueError("Cần đủ 18 hô`sơ chia 12 train và 6 test.")
print("Hô`sơ:", len(records))
print("Ô thiếu:", int(np.isnan(X).sum()))
print("Nhãn test 0/1:", int((y[test]==0).sum()),int((y[test]==1).sum()))
print(f"Accuracy baseline: {baseline.score(X[test],y[test]):.2f}")
print(f"Accuracy model: {(pred==y[test]).mean():.2f}")
print(f"Recall lớp 1: {recall_score(y[test],pred):.2f}")
print("Confusion (thật theo hàng):",cm.tolist())
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: So baseline với mô hình thực tế´bằng Accuracy, Recall lớp 1 và bảng nhầm lẫn.
# Dữ liệu: 18 học viên completed 0/1, 2 feature hours/activities có ô thiếu, Train=12/Test=6.
# NumPy hô˜trợ ndarray, NaN và chỉ số´các hàng train/test.
import numpy as np
# DummyClassifier là baseline đoán lớp phổ˙biến, không học quan hệ feature.
from sklearn.dummy import DummyClassifier
# SimpleImputer thay dữ liệu thiếu theo thống kê chỉ học từ Train.
from sklearn.impute import SimpleImputer
# LogisticRegression phân loại nhãn rời rạc, không dự báo giá trị liên tục.
from sklearn.linear_model import LogisticRegression
# confusion_matrix tạo bảng nhầm lẫn; recall_score đánh giá khả năng nhận lớp 1.
from sklearn.metrics import confusion_matrix,recall_score
# train_test_split chia đồng bộ hàng Train và Test trước khi fit.
from sklearn.model_selection import train_test_split
# make_pipeline ghép các bước để˙transform Test bằng thống kê Train đã học.
from sklearn.pipeline import make_pipeline
# StandardScaler học mean/SD từng cột từ Train rồi chuẩn hóa.
from sklearn.preprocessing import StandardScaler
# 18 dict, mỗi dict có giờ/hoạt động và nhãn completed; None nghĩa chưa ghi feature.
records = [{"id":"C01","hours":1,"activities":2,"completed":0},
```

```
{"id":"C02","hours":2,"activities":3,"completed":0},
{"id":"C03","hours":None,"activities":4,"completed":0},
{"id":"C04","hours":4,"activities":2,"completed":0},
{"id":"C05","hours":1,"activities":3,"completed":0},
{"id":"C06","hours":2,"activities":4,"completed":0},
{"id":"C07","hours":3,"activities":2,"completed":0},
{"id":"C08","hours":4,"activities":3,"completed":0},
{"id":"C09","hours":1,"activities":4,"completed":0},
{"id":"C10","hours":2,"activities":2,"completed":0},
{"id":"C11","hours":3,"activities":3,"completed":0},
{"id":"C12","hours":4,"activities":4,"completed":0},
{"id":"C13","hours":7,"activities":9,"completed":1},
{"id":"C14","hours":8,"activities":10,"completed":1},
{"id":"C15","hours":9,"activities":11,"completed":1},
{"id":"C16","hours":7,"activities":None,"completed":1},
{"id":"C17","hours":8,"activities":10,"completed":1},
{"id":"C18","hours":None,"activities":11,"completed":1}]
# Danh sách hai feature cho từng học viên.
features = []
# Danh sách nhãn 0/1 theo đúng thứ tự người.
labels = []
for row in records:
    # append() thêm cặp [hours,activities], có thể có None.
    features.append([row["hours"], row["activities"]])
    # append() thêm completed của cùng bản ghi.
    labels.append(row["completed"])
# X float shape (18,2), None trở thành NaN cho imputer.
X = np.array(features, dtype=float)
# y có 18 nhãn lớp.
y = np.array(labels)
# Chia bằng index chung cho X/y, không fit trên Test.
# Test có sáu học viên, Train có 12.
train,test = train_test_split(np.arange(len(records)),test_size=6,
                              # Cố định cách chia và giữ tỷ lệ nhãn trong hai tập.
                              random_state=42,stratify=y)
# Baseline chỉ đoán lớp phổ biến từ Train, bỏ qua feature.
baseline = DummyClassifier(strategy="most_frequent").fit(X[train],y[train])
# Imputer học median ở Train để điền thiếu trước khi scaler.
model = make_pipeline(SimpleImputer(strategy="median"),
                      # Scaler chuẩn hóa rồi logistic phân loại, max_iter giới hạn vòng tối ưu.
                      StandardScaler(),LogisticRegression(max_iter=1000))
# fit mô hình từ 12 hàng Train, không sử dụng nhãn Test.
model.fit(X[train],y[train])
# predict() trả sáu dự đoán Test.
pred = model.predict(X[test])
# labels=[0,1] đặt thứ tự lớp cố định; HÀNG là thật, CỘT là đoán, có cả TP/FP/FN/TN.
cm = confusion_matrix(y[test],pred,labels=[0,1])
# Điều kiện bảo vệ kiểm cấu trúc số mẫu và tránh Train/Test dùng chung trước khi báo cáo.
if len(records) != 18 or len(train) != 12 or len(test) != 6:
    # raise ValueError dừng kịch bản khi dữ liệu không đúng số mẫu hoặc điều kiện kiểm thử.
    raise ValueError("Cần đủ 18 hồ sơ chia 12 train và 6 test.")
print("Hồ sơ:", len(records))
# Mask NaN đếm số ô feature thiếu trên ma trận gốc.
print("Ô thiếu:", int(np.isnan(X).sum()))
# Dùng chỉ số Test và mask Boolean để đếm nhãn thật hai lớp.
print("Nhãn test 0/1:", int((y[test]==0).sum()),int((y[test]==1).sum()))
# Baseline Accuracy trên tập Test để có mốc so sánh.
print(f"Accuracy baseline: {baseline.score(X[test],y[test]):.2f}")
# Mask so từng nhãn đoán/thật, mean() cho Accuracy của mô hình.
print(f"Accuracy model: {(pred==y[test]).mean():.2f}")
# Recall mặc định lớp dương 1 = TP/(TP+FN), đo tỷ lệ người thật hoàn thành được phát hiện.
print(f"Recall lớp 1: {recall_score(y[test],pred):.2f}")
# tolist() đổi confusion matrix ndarray sang list hai hàng để in.
print("Confusion (thật theo hàng):",cm.tolist())
```

Kết quả khi chạy:

```
Hồ sơ: 18
Ô thiếu: 3
Nhãn test 0/1: 4 2
Accuracy baseline: 0.67
Accuracy model: 1.00
Recall lớp 1: 1.00
Confusion (thật theo hàng): [[4, 0], [0, 2]]
```

Những chỉ số này mô tả **sáu hồ sơ test** của bài. Không tự chọn mô hình theo test để rồi tuyên bố test vẫn là đánh giá cuối độc lập.

Tình huống 5 — Cấp 5: Mini-project 20 hồ sơ: kiểm trùng, điền thiếu, so ba mô hình và xuất báo cáo

Từ vựng trong mục: End-to-end pipeline — quy trình xuyên suốt; **PCA** — giảm chiều; **Model report** — báo cáo mô hình.

Tình huống thực tế: Trung tâm muốn hoàn thiện một báo cáo AI từ **20 dòng hồ sơ**, mỗi dòng có mã học viên, **giờ học (`hours`, giờ/tuần)**, **lượt truy cập (`visits`, lượt)**, **dự án hoàn thành (`projects`, dự án)** và nhãn `passed` **0/1**. Dòng cuối dùng lại **mã H15**, nên thực tế chỉ có **19 học viên duy nhất**; các ô `?`, trống và `N/A` làm **ba ô đầu vào không hợp lệ hoặc chưa có dữ liệu**. Nhóm cần phát hiện mã trùng, chuyển đầu vào về số, giữ **14 học viên train và 5 test**, rồi so **mức baseline**, **Logistic Regression dùng cả ba đặc trưng**, và **Logistic Regression sau khi giảm còn hai thành phần PCA** trên **cùng năm học viên test**. Cuối cùng cần xuất báo cáo **JSON** có số hồ sơ, độ đúng và ma trận nhầm lẫn, mà không dùng test để học cách xử lý dữ liệu.

Vì sao chọn cách này? Đây là một **quy trình AI đầu-cuối thu nhỏ** gồm **kiểm dữ liệu → chia train/test → xử lý thiếu → học mô hình → đánh giá → lưu báo cáo**. `drop_duplicates(subset='id', keep='first')` chỉ giữ **lần xuất hiện đầu của H15**, nên từ 20 dòng còn 19 mã duy nhất; cần đối chiếu hồ sơ thật trước khi tự quyết định xóa bản ghi nào trong dự án thực tế. `pd.to_numeric(errors='coerce')` đổi `?`, ô trống và `N/A` thành giá trị thiếu **NaN**, báo **ba ô** cần xử lý thay vì giả làm 0. Sau `train_test_split(..., stratify=y, random_state=42)`, cả baseline, mô hình dùng ba feature và mô hình PCA đều được huấn luyện trên **cùng 14 người train** và đánh giá trên **cùng 5 người test** để so công bằng. `SimpleImputer` và `StandardScaler` chỉ học từ train; **PCA** là cách chuyển ba đặc trưng thành **hai hướng biến thiên mới**, cũng phải fit trên train. **Accuracy** cho tỷ lệ đúng, **confusion matrix** cho biết từng lớp bị nhầm ra sao; `json.dumps` lưu các con số vào `model_report_20.json` để kiểm lại. Đây là thử nghiệm trên ít dữ liệu giả lập; không dùng kết quả test để điều chỉnh siêu tham số rồi vẫn gọi năm hồ sơ ấy là đánh giá độc lập.

Quy trình:

1. Tạo 20 bản ghi, báo một ID lặp; giữ bản ghi xuất hiện đầu tiên của mỗi ID.
2. Chuyển ba cột đo sang số, coi dữ liệu không đọc được là NaN; chia 19 hồ sơ còn lại thành train 14/test 5 có phân tầng.
3. Fit baseline, pipeline đầy đủ và pipeline PCA hai thành phần trên cùng train; dự đoán cùng test, xuất `model_report_20.json` chứa số dòng, missing, accuracy và confusion matrix.

No Note — Không chú thích

```
import json
from pathlib import Path
import numpy as np
```

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
raw = [{"id":"H01","hours":1,"visits":1,"projects":0,"passed":0},
{"id":"H02","hours":2,"visits":2,"projects":1,"passed":0},
{"id":"H03","hours":3,"visits":3,"projects":0,"passed":0},
{"id":"H04","hours":1,"visits":1,"projects":1,"passed":0},
{"id":"H05","hours":2,"visits":"?","projects":0,"passed":0},
{"id":"H06","hours":3,"visits":3,"projects":1,"passed":0},
{"id":"H07","hours":1,"visits":1,"projects":0,"passed":0},
{"id":"H08","hours":2,"visits":2,"projects":1,"passed":0},
{"id":"H09","hours":3,"visits":3,"projects":0,"passed":0},
{"id":"H10","hours":1,"visits":1,"projects":1,"passed":0},
{"id":"H11","hours":7,"visits":8,"projects":5,"passed":1},
{"id":"H12","hours":8,"visits":9,"projects":6,"passed":1},
{"id":"H13","hours":9,"visits":10,"projects":5,"passed":1},
{"id":"H14","hours":7,"visits":8,"projects":6,"passed":1},
{"id":"H15","hours":None,"visits":9,"projects":5,"passed":1},
{"id":"H16","hours":9,"visits":10,"projects":6,"passed":1},
{"id":"H17","hours":7,"visits":8,"projects":5,"passed":1},
{"id":"H18","hours":8,"visits":9,"projects":"N/A","passed":1},
{"id":"H19","hours":9,"visits":10,"projects":5,"passed":1},
{"id":"H15","hours":7,"visits":8,"projects":6,"passed":1}]
df = pd.DataFrame(raw)
duplicate_count = int(df["id"].duplicated().sum())
df = df.drop_duplicates(subset="id",keep="first").reset_index(drop=True)
feature_columns = ["hours","visits","projects"]
X = df[feature_columns].apply(pd.to_numeric,errors="coerce").to_numpy(dtype=float)
y = df["passed"].to_numpy(dtype=int)
missing_count = int(np.isnan(X).sum())
train,test = train_test_split(np.arange(len(df)),test_size=5,
                              random_state=42,stratify=y)
baseline = DummyClassifier(strategy="most_frequent").fit(X[train],y[train])
full = make_pipeline(SimpleImputer(strategy="median"),StandardScaler(),
                     LogisticRegression(max_iter=1000))
pca = make_pipeline(SimpleImputer(strategy="median"),StandardScaler(),
                    PCA(n_components=2),LogisticRegression(max_iter=1000))
full.fit(X[train],y[train])
pca.fit(X[train],y[train])
pred = full.predict(X[test])
report = {"raw_rows":len(raw),"unique_rows":len(df),"duplicates":duplicate_count,
          "missing_cells":missing_count,"train_rows":len(train),"test_rows":len(test),
          "accuracy_baseline":round(accuracy_score(y[test],baseline.predict(X[test])),2),
          "accuracy_full":round(accuracy_score(y[test],pred),2),
          "accuracy_pca":round(accuracy_score(y[test],pca.predict(X[test])),2),
          "confusion":confusion_matrix(y[test],pred,labels=[0,1]).tolist()}
path = Path("model_report_20.json")
path.write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding="utf-8")
if len(raw) != 20 or len(df) != 19 or not set(train).isdisjoint(test):
    raise ValueError("Cần 20 dòng đầu vào, 19 hồ sơ duy nhất và Train/Test không trùng.")
print("Dòng gốc/trùng/còn:",report["raw_rows"],report["duplicates"],report["unique_rows"])
print("Ô thiếu sau chuyển kiểu:",report["missing_cells"])
print("Train/test:",report["train_rows"],report["test_rows"])
print("Accuracy baseline/full/
PCA:",report["accuracy_baseline"],report["accuracy_full"],report["accuracy_pca"])
print("Confusion (thật theo hàng):",report["confusion"])
print("Đã lưu JSON:",path.is_file() and json.loads(path.read_text(encoding="utf-8"))==report)
```

Code có chú thích — Giải thích từng bước

```
# Mục đích: Chạy quy trình ML toàn diện: làm sạch 20 dòng, bỏ trùng, Train/Test, baseline, full,
PCA và lưu JSON.
# Dữ liệu: 20 hàng raw có ID H15 trùng thành 19 học viên; ba feature hours/visits/projects có '?',
'N/A', None; nhãn passed 0/1; Test=5.
# json xử lý chuyển đổi dictionary ↔ JSON văn bản (dumps/loads).
import json
# Path thao tác file báo cáo và đường dẫn tương đối theo hệ điều hành.
from pathlib import Path
# NumPy hỗ trợ ndarray, NaN và chỉ số các hàng train/test.
import numpy as np
# Pandas quản lý bảng, dữ liệu thiếu và thao tác làm sạch.
import pandas as pd
# PCA giảm chiều dữ liệu theo hướng có phương sai lớn, chỉ fit từ Train.
from sklearn.decomposition import PCA
# DummyClassifier là baseline đoán lớp phổ biến, không học quan hệ feature.
from sklearn.dummy import DummyClassifier
# SimpleImputer thay dữ liệu thiếu theo thống kê chỉ học từ Train.
from sklearn.impute import SimpleImputer
# LogisticRegression phân loại nhãn rời rạc, không dự báo giá trị liên tục.
from sklearn.linear_model import LogisticRegression
# accuracy_score đo tỷ lệ đúng, confusion_matrix đếm đúng/sai từng lớp.
from sklearn.metrics import accuracy_score,confusion_matrix
# train_test_split chia đồng bộ hàng Train và Test trước khi fit.
from sklearn.model_selection import train_test_split
# make_pipeline ghép các bước để transform Test bằng thống kê Train đã học.
from sklearn.pipeline import make_pipeline
# StandardScaler học mean/SD từng cột từ Train rồi chuẩn hóa.
from sklearn.preprocessing import StandardScaler
# 20 dictionary có id, giờ học, lượt truy cập, dự án và nhãn; ID H15 trùng, một số chuỗi không phải
số gây thiếu.
raw = [{"id":"H01","hours":1,"visits":1,"projects":0,"passed":0},
{"id":"H02","hours":2,"visits":2,"projects":1,"passed":0},
{"id":"H03","hours":3,"visits":3,"projects":0,"passed":0},
{"id":"H04","hours":1,"visits":1,"projects":1,"passed":0},
{"id":"H05","hours":2,"visits":"?","projects":0,"passed":0},
{"id":"H06","hours":3,"visits":3,"projects":1,"passed":0},
{"id":"H07","hours":1,"visits":1,"projects":0,"passed":0},
{"id":"H08","hours":2,"visits":2,"projects":1,"passed":0},
{"id":"H09","hours":3,"visits":3,"projects":0,"passed":0},
{"id":"H10","hours":1,"visits":1,"projects":1,"passed":0},
{"id":"H11","hours":7,"visits":8,"projects":5,"passed":1},
{"id":"H12","hours":8,"visits":9,"projects":6,"passed":1},
{"id":"H13","hours":9,"visits":10,"projects":5,"passed":1},
{"id":"H14","hours":7,"visits":8,"projects":6,"passed":1},
{"id":"H15","hours":None,"visits":9,"projects":5,"passed":1},
{"id":"H16","hours":9,"visits":10,"projects":6,"passed":1},
{"id":"H17","hours":7,"visits":8,"projects":5,"passed":1},
{"id":"H18","hours":8,"visits":9,"projects":"N/A","passed":1},
{"id":"H19","hours":9,"visits":10,"projects":5,"passed":1},
{"id":"H15","hours":7,"visits":8,"projects":6,"passed":1}]
# DataFrame tạo 20 hàng bảng thô, chưa làm sạch.
df = pd.DataFrame(raw)
# duplicated() True với lần xuất hiện ID lặp sau lần đầu; sum() đếm hàng trùng.
duplicate_count = int(df["id"].duplicated().sum())
# Bỏ ID lặp chỉ dựa cột id, giữ BẢN ĐẦU; không tự gộp số liệu bản sau.
# Đánh lại index bảng từ 0, drop=True không giữ index cũ thành cột.
df = df.drop_duplicates(subset="id",keep="first").reset_index(drop=True)
# Ba tên cột giờ, truy cập, dự án; không lấy ID hoặc passed làm feature.
feature_columns = ["hours","visits","projects"]
# to_numeric chuyển các cột sang số; errors='coerce' biến '?'/'N/A' thành NaN thay vì dừng lỗi.
# Chuyển bảng ba feature sang ndarray float shape (19,3) với NaN cho dữ liệu thiếu.
X = df[feature_columns].apply(pd.to_numeric,errors="coerce").to_numpy(dtype=float)
# Chuyển nhãn passed sang vector int 19 phần tử.
y = df["passed"].to_numpy(dtype=int)
# isnan đánh ô NaN, sum() đếm ô feature thiếu SAU đổi kiểu.
```

```
missing_count = int(np.isnan(X).sum())
# Chia các index học viên duy nhất thành Train/Test trước khi mọi phép fit.
# arange(n) sinh chỉ số´0..n−1 để dùng chung cho X/y.
# Dành 5 học viên duy nhất cho Test và 14 cho Train.
train,test = train_test_split(np.arange(len(df)),test_size=5,
                              # Cô´định phép chia và giữ tỷ lệ nhãn 0/1 gần tương đương.
                              random_state=42,stratify=y)
# Baseline dự đoán nhãn lớp đông nhất Train, không học feature.
baseline = DummyClassifier(strategy="most_frequent").fit(X[train],y[train])
# Full pipeline: impute median trên Train → scale trên Train → classifier.
full = make_pipeline(SimpleImputer(strategy="median"),StandardScaler(),
                     # Bộ phân loại logistic, max_iter giới hạn số´vòng tối ưu.
                     LogisticRegression(max_iter=1000))
# Full pipeline: impute median trên Train → scale trên Train → classifier.
pca = make_pipeline(SimpleImputer(strategy="median"),StandardScaler(),
                    # PCA giữ hai thành phần chính từ ba feature sau impute/scale; phải fit chỉ
trên TRAIN để tránh rò rỉ.
                    # Bộ phân loại logistic, max_iter giới hạn số´vòng tối ưu.
                    PCA(n_components=2),LogisticRegression(max_iter=1000))
# Fit full pipeline từ 14 hàng Train, không dùng Test.
full.fit(X[train],y[train])
# Fit cả PCA và bộ phân loại từ Train; tên biến pca ở đây là TOÀN PIPELINE.
pca.fit(X[train],y[train])
# Dự đoán nhãn của 5 mẫu Test với pipeline full.
pred = full.predict(X[test])
# Dictionary tổng hợp kích thước, chất lượng mô hình và confusion để lưu/so sánh.
report = {"raw_rows":len(raw),"unique_rows":len(df),"duplicates":duplicate_count,
          "missing_cells":missing_count,"train_rows":len(train),"test_rows":len(test),
          # Accuracy baseline: lấy nhãn thật Test so với nhãn baseline đoán, là tỷ lệ đúng.
          "accuracy_baseline":round(accuracy_score(y[test],baseline.predict(X[test])),2),
          # Accuracy full so nhãn y_test với nhãn pred của full pipeline.
          "accuracy_full":round(accuracy_score(y[test],pred),2),
          # Accuracy của pipeline PCA; không đánh giá PCA riêng lẻ mà classifier sau PCA.
          "accuracy_pca":round(accuracy_score(y[test],pca.predict(X[test])),2),
          # labels cô´định thứ tự lớp, hàng thật/cột dự đoán; tolist() đổi ndarray sang JSON-
compatible list.
          "confusion":confusion_matrix(y[test],pred,labels=[0,1]).tolist()}
# Tên báo cáo JSON tương đối với current working directory; file sẽ được ghi ra ổ đĩa.
path = Path("model_report_20.json")
# dumps() chuyển dict thành chuỗi JSON; ensure_ascii=False giữ dấu tiếng Việt, indent=2 thụt dòng
đẹp, không ảnh hưởng giá trị.
# write_text(...,encoding='utf-8') ghi chuỗi JSON tới file, có thể ghi đè file cùng tên trong cwd.
path.write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding="utf-8")
# Kiểm hai tập chỉ số´không có học viên trùng nhau, tránh leakage.
# Chỉ 19 ID duy nhất sau khi bỏ một hàng H15 trùng; 20 raw là dữ liệu nguồn.
# Điều kiện bảo vệ kiểm cấu trúc số´mẫu và tránh Train/Test dùng chung trước khi báo cáo.
if len(raw) != 20 or len(df) != 19 or not set(train).isdisjoint(test):
    # raise ValueError dừng kịch bản khi dữ liệu không đúng số´mẫu hoặc điều kiện kiểm thử.
    raise ValueError("Cần 20 dòng đầu vào, 19 hồ`sơ duy nhất và Train/Test không trùng.")
print("Dòng gốc/trùng/còn:",report["raw_rows"],report["duplicates"],report["unique_rows"])
print("Ô thiếu sau chuyển kiểu:",report["missing_cells"])
print("Train/test:",report["train_rows"],report["test_rows"])
print("Accuracy baseline/full/
PCA:",report["accuracy_baseline"],report["accuracy_full"],report["accuracy_pca"])
print("Confusion (thật theo hàng):",report["confusion"])
# read_text UTF-8 mở file vừa lưu; json.loads() parse về`dictionary để đối chiếu với report trong
RAM.
# Kiểm tồn tại file thật, không chỉ có tên path.
# So dict JSON đọc lại với dict đã tạo để kiểm file lưu/đọc chính xác.
print("Đã lưu JSON:",path.is_file() and json.loads(path.read_text(encoding="utf-8"))==report)
```

Kết quả khi chạy:

```
Dòng gốc/trùng/còn: 20 1 19
Ô thiếu sau chuyển kiểu: 3
```

```
Train/test: 14 5
Accuracy baseline/full/PCA: 0.6 1.0 1.0
Confusion (thật theo hàng): [[3, 0], [0, 2]]
Đã lưu JSON: True
```

Bản báo cáo chứa đúng dữ liệu **mô phỏng** và chỉ số từ năm test; không được coi kết quả này là chứng nhận độ tin cậy ngoài đời. Không thử nhiều phương án rồi chọn dựa trên chính năm test nếu vẫn muốn gọi đó là đánh giá cuối độc lập.

6. Sai lầm thường gặp

- X là cột đặc trưng, y là nhãn; không đưa nhãn hoặc kết quả biết sau sự kiện vào feature.
- Phải chia train/test trước khi học median, mean/std hoặc PCA để tránh rò rỉ.
- Không `fit_transform` PCA trên X_test; chỉ `transform` bằng PCA đã fit từ train.
- Accuracy cao vẫn có thể che lỗi lớp hiếm; đọc thêm confusion matrix theo hàng thật–cột dự đoán.
- Thay đổi split hoặc metric khi so hai phương pháp sẽ gây so sánh thiếu công bằng.
- Đánh giá trên test không phải huấn luyện; chỉnh siêu tham số dựa mãi vào test làm rò rỉ đánh giá.

7. Trắc nghiệm

Q1. What is a feature in supervised learning? | Feature trong học có giám sát là gì?

A. An input measurement used by the predictor | Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict | Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps | Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets | So dự đoán với nhãn thật giữ riêng

Đáp án: A

Giải thích: Trong Iris, bốn số đo cm nằm ở X, không phải nhãn y. Nếu đưa kết quả tương lai hoặc mã lớp vào X, đánh giá sẽ dễ giả tạo do rò rỉ.

Q2. What is a label? | Label là gì?

A. An input measurement used by the predictor | Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict | Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps | Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets | So dự đoán với nhãn thật giữ riêng

Đáp án: B

Giải thích: Classifier có giám sát học từ nhãn thật ghép với từng hàng train. Mỗi trong 150 hàng đặc trưng cần có nhãn tương ứng khi train.

Q3. What is a pipeline? | Pipeline là gì?

A. An input measurement used by the predictor | Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict | Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps | Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets | So dự đoán với nhãn thật giữ riêng

Đáp án: C

Giải thích: Nó fit bộ biến đổi trên train rồi tái sử dụng cho dữ liệu sau. Thứ tự rất quan trọng: điền thiếu và chuẩn hóa trước dự đoán, nhưng tham số chỉ học từ train.

Q4. What is evaluation? | Evaluation là gì?

A. An input measurement used by the predictor | Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict | Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps | Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets | So dự đoán với nhãn thật giữ riêng

Đáp án: D

Giải thích: Test cuối đánh giá dữ liệu chưa fit, không dùng lại mẫu học. Hãy dùng test giữ riêng không thay đổi; điểm train cao có thể chỉ là ghi nhớ dữ liệu.

Q5. What does an Iris feature row represent? | Một hàng feature Iris biểu diễn gì?

A. Four measurements for one flower | Bốn số đo của một bông hoa

B. The target class per sample | Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting | Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment | Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: A

Giải thích: Hàng là một mẫu hoa, cột là số đo như chiều dài cánh. Với shape (150,4), 150 là mẫu, bốn là biến đo theo cm.

Q6. What does y represent in classification? | y biểu diễn gì khi phân loại?

A. Four measurements for one flower | Bốn số đo của một bông hoa

B. The target class per sample | Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting | Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment | Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: B

Giải thích: Nhãn y phải ghép một-một với các hàng X. Nếu X và y sai thứ tự, thuật toán đúng vẫn học sai mối quan hệ dữ liệu–nhãn.

Q7. What is data leakage? | Rò rỉ dữ liệu là gì?

A. Four measurements for one flower | Bốn số đo của một bông hoa

B. The target class per sample | Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting | Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment | Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: C

Giải thích: Fit tiền xử lý trên test hoặc dùng kết quả tương lai có thể làm chỉ số ảo. Rò rỉ còn xảy ra nếu chọn đặc trưng từ toàn bộ dữ liệu trước khi chia.

Q8. Why inspect X.shape and y.shape first? | Vì sao kiểm X.shape và y.shape đầu tiên?

A. Four measurements for one flower | Bốn số đo của một bông hoa

B. The target class per sample | Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting | Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment | Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: D

Giải thích: Số hàng X lệch số nhãn y khiến việc fit sai. Hàng và nhãn lệch nhau phải được sửa trước khi fit hoặc tính điểm.

Q9. What does train_test_split do? | train_test_split làm gì?

A. Separates data into fitting and held-out portions | Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts | Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features | Các đặc trưng train

D. Training features only | Chỉ đặc trưng train

Đáp án: A

Giải thích: Hàm chia dữ liệu, không tự học mô hình hay đo accuracy. Phải giữ cả X và y train/test; chia dữ liệu không tự tương đương huấn luyện.

Q10. Why use stratify=y? | Vì sao dùng stratify=y?

A. Separates data into fitting and held-out portions | Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts | Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features | Các đặc trưng train

D. Training features only | Chỉ đặc trưng train

Đáp án: B

Giải thích: Giúp giảm lệch lớp khi mỗi lớp có đủ mẫu. Phân tầng hữu ích khi lớp ít mẫu có nguy cơ vắng khỏi tập test.

Q11. What should SimpleImputer learn from? | SimpleImputer nên học từ đâu?

A. Separates data into fitting and held-out portions | Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts | Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features | Các đặc trưng train

D. Training features only | Chỉ đặc trưng train

Đáp án: C

Giải thích: Median tính từ toàn bộ dữ liệu làm lộ thống kê test. Hàng test không được tác động đến giá trị điền học được, kể cả khi hàng ấy thiếu dữ liệu.

Q12. What should StandardScaler learn from? | StandardScaler nên fit từ đâu?

A. Separates data into fitting and held-out portions | Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts | Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features | Các đặc trưng train

D. Training features only | Chỉ đặc trưng train

Đáp án: D

Giải thích: Biến đổi test bằng mean/std train, không fit từ test. Mean và variance từ train phải được áp nhất quán lên mọi mẫu mới.

Q13. What does LogisticRegression produce in Iris? | LogisticRegression trên Iris dự đoán gì?

A. One of three discrete flower classes | Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions | Số dự đoán đúng chia tổng số dự đoán

C. Actual labels | Các nhãn thật

D. Predicted labels | Các nhãn dự đoán

Đáp án: A

Giải thích: Dù có chữ Regression, LogisticRegression dùng để phân loại. Mô hình có thể xuất xác suất lớp, nhưng nhiệm vụ đang xét chọn một nhãn rời rạc.

Q14. What does accuracy count? | Accuracy đếm gì?

A. One of three discrete flower classes | Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions | Số dự đoán đúng chia tổng số dự đoán

C. Actual labels | Các nhãn thật

D. Predicted labels | Các nhãn dự đoán

Đáp án: B

Giải thích: Accuracy hữu ích nhưng có thể che lỗi lớp hiếm. Với tập lệch lớp, accuracy cao có thể đến từ việc chỉ đoán lớp đông nhất.

Q15. What do confusion matrix rows mean? | Hàng của confusion matrix nghĩa là gì?

A. One of three discrete flower classes | Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions | Số dự đoán đúng chia tổng số dự đoán

C. Actual labels | Các nhãn thật

D. Predicted labels | Các nhãn dự đoán

Đáp án: C

Giải thích: Mỗi hàng đếm mẫu thuộc lớp thật tương ứng. Hàng đại diện nhãn thật; ô ngoài đường chéo là lỗi xuất phát từ lớp thật ở hàng đó.

Q16. What do confusion matrix columns mean? | Cột của confusion matrix nghĩa là gì?

A. One of three discrete flower classes | Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions | Số dự đoán đúng chia tổng số dự đoán

C. Actual labels | Các nhãn thật

D. Predicted labels | Các nhãn dự đoán

Đáp án: D

Giải thích: Ô ngoài đường chéo là trường hợp dự đoán sang lớp khác. Cột cho biết lớp dự đoán nào nhận các mẫu bị gán nhầm.

Q17. What does PCA reduce? | PCA giảm gì?

A. Feature dimensionality while retaining sample rows | Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes | Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric | Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed | Không, phải đánh giá dữ liệu giữ riêng

Đáp án: A

Giải thích: Thành phần PCA là trục đặc trưng mới, không phải nhãn. Trục PCA là tổ hợp đặc trưng ban đầu được học, không làm đổi số mẫu.

Q18. Why fit PCA on train only? | Vì sao chỉ fit PCA trên train?

A. Feature dimensionality while retaining sample rows | Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes | Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric | Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed | Không, phải đánh giá dữ liệu giữ riêng

Đáp án: B

Giải thích: Test chỉ được transform bằng PCA đã fit từ train. Nếu không, thông tin phân phối test đã lọt vào biểu diễn đặc trưng.

Q19. How should baseline and PCA models be compared? | So baseline và mô hình PCA thế nào?

A. Feature dimensionality while retaining sample rows | Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes | Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric | Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed | Không, phải đánh giá dữ liệu giữ riêng

Đáp án: C

Giải thích: Khác split hoặc metric không tách được ảnh hưởng của PCA. Train/test phải giống nhau giữa hai cách để chênh lệch metric phản ánh thay đổi mô hình.

Q20. Does a high train score prove model generalization? | Train score cao có chứng minh tổng quát hóa?

A. Feature dimensionality while retaining sample rows | Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes | Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric | Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed | Không, phải đánh giá dữ liệu giữ riêng

Đáp án: D

Giải thích: Mô hình có thể nhớ train nhưng dự đoán kém trên mẫu mới. Cần báo metric test và lỗi theo lớp trước khi đánh giá khả năng ứng dụng thực tế.

8. Nối sang bài tiếp

`Python → NumPy/Pandas → Train/Test → Pipeline → Report.`